



HỌC SÂU CHO THỊ GIÁC MÁY TÍNH

CÂY TRI THỨC COMPUTER VISION

Trình bày theo mạch

vấn đề → giải pháp → giả định → giá phải trả

“Độ khó không biến mất, nó chỉ bị đẩy đi chỗ khác.”

nguyên lý đọc hệ thống, Chương 3

Năm phần A · Nền tảng có giới hạn B · Tư duy hệ thống C · Thuật toán và mô hình

D · Kỹ thuật, tối ưu và triển khai E · Đánh giá, độ tin cậy và dự án

Bản gộp toàn cây — Ngày 9 tháng 9 năm 2026

Bản đồ cây

TÓM TẮT

Đây là bản đồ điều hướng của cả cây: nó nói cây gồm gì, vì sao được chia thành sáu phần theo thứ tự đó, và bốn nguyên lý sẽ gặp lại ở gần như mọi chương. Đọc xong bạn biết nên bắt đầu từ đâu, phần nào đáng đầu tư sâu, phần nào chỉ cần tra khi cần, và phần nào sẽ hết hạn trước tiên. Nếu chỉ nhớ một điều: cây được tổ chức theo *mạch vấn đề dẫn tới giải pháp* chứ không theo danh mục thuật ngữ, vì đó là khác biệt giữa cách người mới và cách chuyên gia phân loại bài toán.

Cây này được dựng từ một lộ trình tự soạn, với một mục tiêu hẹp và cụ thể: *đọc hiểu paper, hiểu vì sao tác giả làm như vậy, và gọi tên được trade-off của mỗi kỹ thuật*. Nó không nhằm biến người đọc thành người thuộc lòng danh mục thuật ngữ. Sự khác biệt giữa hai mục tiêu ấy quyết định toàn bộ cách trình bày phía sau, nên đáng nói rõ ngay từ đầu.

Vì sao trình bày theo mạch chứ không theo danh mục (Why a Problem-Driven Narrative)

Có một quan sát lặp lại trong nghiên cứu về chuyên gia: người mới phân loại bài toán theo *đặc điểm bề mặt* — tên module, tên layer, con số trong bảng — còn chuyên gia phân loại theo *nguyên lý sâu* — vấn đề nào đang được giải, giả định nào đang được đặt. Một danh sách gạch đầu dòng dạy rất tốt cách phân loại thứ nhất và rất tệ cách thứ hai. Do đó phần lớn cây này được viết theo mạch **vấn đề** → **giải pháp** → **giả định** → **giá phải trả** → **vấn đề mới sinh ra**, và các kỹ thuật cùng họ được đi qua *theo đúng thứ tự lịch sử chúng ra đời để gỡ nút thắt của nhau*.

Cách đọc kèm theo cũng khác. Mạch suy luận đã được viết sẵn ở đây chính là một *worked example* theo nghĩa của tâm lý học nhận thức: việc của người đọc không phải là tự khám phá lại, mà là *dự đoán bước tiếp theo trước khi đọc nó*. Nếu dự đoán đúng thì bạn đã nắm nguyên lý; nếu sai thì chỗ sai chính là chỗ đáng dừng lại.

Phân tầng theo chu kỳ bán rã của tri thức (Knowledge Half-Life Tiers)

Không phải thứ gì trong lĩnh vực này cũng đáng học sâu như nhau, vì chúng hết hạn với tốc độ rất khác nhau. Cây dùng bốn nhãn, và nhãn xuất hiện ngay ở tiêu đề mỗi chương.

Bảng 1. Bốn tầng bán rã, mức đầu tư tương ứng, và vị trí trong cây.

Tầng	Nội dung	Mức đầu tư	Nằm ở đâu
[T1]	Toán, vật lý, first principle	Chỉ học phần cần thiết, tới mức dùng được và giải thích được; không chứng minh định lý	Phần A
[T2]	Nguyên lý thiết kế hệ thống, giới hạn tính toán, trade-off	Trọng tâm chính — đây là thứ còn giá trị sau mười năm	Phần B, E
[T3]	Thuật toán và mô hình	Trọng tâm chính — học ý tưởng cốt lõi và lý do ra đời, không học thuộc danh sách	Phần C, F
[T4]	Thư viện, framework, công cụ	Học đủ để làm việc, không đầu tư sâu; tra khi cần	Phần D

Bảng 1 giải thích một điều thoát nhìn có vẻ mâu thuẫn: các bài tập tự viết backward pass và tự chạy gradient check, dù trông giống bài tập công cụ, thực ra thuộc tầng [T2]–[T3]. Chúng xây trực giác về đồ thị tính toán, về nơi gradient chết, về cái giá bộ nhớ của activation — những thứ không hết hạn. Ngược lại, một danh sách kiến trúc cụ thể sẽ cũ trong vài năm và nên được đọc như lịch sử tư duy, không như trạng thái hiện tại.

Vì sao chia thành sáu phần, và thứ tự phụ thuộc (The Six Parts)

Sáu phần không phải sáu chủ đề song song mà là sáu tầng câu hỏi nối tiếp nhau. Phần A trả lời *ảnh từ đâu ra và ngôn ngữ toán nào dùng để nói về nó*; nó ngắn có chủ ý vì nguyên tắc là học theo nhu cầu. Phần B trả lời một câu hỏi duy nhất và là câu hỏi bị bỏ qua nhiều nhất: *làm sao ra quyết định thiết kế đúng khi chưa có mô hình nào chạy?* Mọi chương trong đó là một dạng ràng buộc, và mọi kỹ thuật xuất hiện đều là câu trả lời cho một thất bại cụ thể trước đó. Chỉ sau khi có Phần B thì Phần C — các họ thuật toán và mô hình — mới đọc được như một chuỗi lựa chọn có lý do, thay vì một danh mục kiến trúc. Phần D là tầng công cụ và triển khai, cố ý đặt sau vì nó hết hạn nhanh nhất. Phần E đóng vòng phương pháp: đọc paper, thiết kế thí nghiệm, đánh giá độ tin cậy và làm dự án xuyên suốt. Phần F là phần áp dụng, và nó chỉ đọc được sau năm phần kia: nó hỏi một câu duy nhất, rằng có nên thay một khối trong hệ SLAM cổ điển bằng một mạng hay không, và nếu có thì khối nào.

Bảng 2. Sáu phần, câu hỏi mỗi phần trả lời, và mức độ ưu tiên.

	Phần	Câu hỏi nó trả lời	Mức
A	Nền tảng có giới hạn	Ảnh từ đâu ra, và cần đúng bao nhiêu toán để nói về nó?	nền, học vừa đủ
B	Tư duy hệ thống	Ra quyết định thiết kế đúng thể nào khi chưa có mô hình nào chạy?	cốt lõi
C	Thuật toán và mô hình	Các họ mô hình ra đời để gỡ nút thắt nào của nhau?	cốt lõi
D	Kỹ thuật, tối ưu, triển khai	Từ mô hình chạy được tới hệ thống chạy thật thì mất gì?	công cụ
E	Đánh giá, độ tin cậy, dự án	Làm sao biết mình không tự lừa mình?	cốt lõi
F	Học sâu cho visual SLAM	Có nên thay một khối cổ điển bằng mạng, và nếu có thì khối nào?	áp dụng

Bảng 2 cũng là thứ tự đọc đề xuất cho lần đầu, với một ngoại lệ: Phần F chỉ đáng đọc khi bạn đã có một hệ SLAM thật đang đau ở một chỗ cụ thể, và khi đó nên đọc chương 27 trước để lọc bẫy

chương còn lại. Từ lần thứ hai trở đi, cách đọc hiệu quả hơn là trộn các nhánh xa nhau và tự kiểm tra chéo, vì gần như mọi chương trong cây đều là một cách nhìn khác của cùng vài nguyên lý.

Vài nguyên lý xuyên suốt, gặp lại ở nhiều chương (Recurring Principles)

Bốn ý dưới đây xuất hiện lại ở những chỗ tưởng như không liên quan, và nhận ra chúng là dấu hiệu đã đọc đúng. Thứ nhất, **độ khó không biến mất, nó chỉ bị đẩy đi chỗ khác** — sang dữ liệu, sang hậu xử lý, sang thời gian huấn luyện, hay sang một giả định về đầu vào; câu hỏi luôn đáng hỏi là *họ đã đẩy nó đi đâu*. Thứ hai, trục đánh đổi trung tâm **giả định mạnh cộng ít dữ liệu** đổi lại **giả định yếu cộng nhiều dữ liệu**, giải thích gần như mọi bước tiến mười lăm năm qua và sẽ gặp lại ở CNN đổi lại ViT, ở hình học cổ điển đổi lại mô hình feed-forward. Thứ ba, **không thể không có giả định**; chỉ có việc chọn giả định nào, và chọn sai thì thêm dữ liệu không cứu được. Thứ tư, **hiểu một hệ thống nghĩa là biết nó hỏng ở đâu**, không phải biết nó chạy được gì — đây là lý do mỗi tài liệu trong cây đều có một mục bắt buộc về giới hạn và trường hợp hỏng.

Cách dùng cây (How to Use This Tree)

Mỗi thư mục có một tệp `00-map.tex` nói nhánh đó gồm gì và vì sao chia như vậy; đó là chỗ nên đọc trước khi vào nội dung. Mỗi tài liệu mở đầu bằng một dòng *Phụ thuộc* cho biết cần đọc gì trước, và kết bằng một mục *Kết nối* trỏ sang các nhánh liên quan cùng một câu giải thích quan hệ. Cuối mỗi tài liệu là ba tới bảy câu hỏi tự kiểm; theo lịch ôn của kho, hãy trả lời chúng *trước* khi mở lại bài, sau một ngày, một tuần, một tháng, ba tháng, và sửa ngay chỗ nào đọc lại thấy không hiểu.

Mỗi chương kết thúc bằng khối *Thực hành* gồm bốn mục: repo và tài nguyên, câu hỏi kiểm tra hiểu, một mini project làm trong vài giờ tới hai ngày, và với những chương đáng đầu tư thì thêm một project một tới ba tuần. Lời khuyên đi kèm là làm mini project của mọi chương, còn project thì chọn lọc — vì đọc mà không tái tạo lại thì chỉ là tiêu thụ.

CẠM BÃY

Phần *Repo* và *SOTA* trong các khối *Thực hành* chốt tại **tháng 9/2026** và là phần hết hạn nhanh nhất của cây: tên mô hình dẫn đầu sẽ cũ trong sáu tới mười hai tháng. Repo hạ tầng (COLMAP, timm, diffusers, ONNX Runtime, pycocotools) bền hơn nhiều so với tên mô hình. Đọc phần đó như *chỗ để bắt đầu tìm*, không phải *câu trả lời đúng*.

Câu hỏi còn mở (Open Questions)

Ba điều cần xử lý khi quay lại cây này. Thứ nhất, `refs.bib` được soạn từ trí nhớ về các công trình gốc; tên tác giả, năm và hội nghị đủ chính xác để tra cứu, nhưng phải đối chiếu bản gốc trước khi trích dẫn ra ngoài cây. Thứ hai, mọi con số định lượng trong cây (mAP, tốc độ, số tham số, độ lớn hiệu ứng) cần được kiểm lại tại nguồn trước khi dùng để ra quyết định thật. Thứ ba, phần *Repo* và *SOTA* cần được rà lại mỗi sáu tháng; lần rà gần nhất là lần soạn, tháng 9/2026.

MỤC LỤC

Bản đồ cây	i
A NỀN TẢNG CÓ GIỚI HẠN	1
1 Toán tối thiểu dùng lại suốt đời (The Minimal Mathematics You Reuse Forever)	3
1.1 Đại số tuyến tính ứng dụng (Applied Linear Algebra)	5
1.2 Vi phân nhiều biến và tối ưu hoá (Matrix Calculus and Optimization)	11
1.3 Xác suất, độ hợp lý và lý thuyết thông tin (Probability, Likelihood and Information Theory)	16
1.4 Hình học biến đổi và xử lý tín hiệu (Transformation Geometry and Signal Processing)	21
1.5 Thực hành	26
1.6 Kết nối	29
2 Vật lý tạo ảnh và mô hình camera (Image Formation Physics and Camera Models)	31
2.1 Ánh sáng, cảm biến và mô hình nhiễu (Light, Sensors and Noise Models)	34
2.2 ISP, RAW so với sRGB và không gian màu (Image Signal Processing and Color Spaces)	38
2.3 Mô hình camera và hình học nhiều view (Camera Models and Multi-View Geometry)	43
2.4 Cảm biến ngoài RGB và nguồn gốc domain gap (Beyond RGB: Sensors and the Origin of Domain Gap)	49
2.5 Thực hành	53
2.6 Kết nối	56
B TƯ DUY HỆ THỐNG	58
3 Giải phẫu một hệ thống thị giác (Anatomy of a Vision System)	60
3.1 Đặt vấn đề	61
3.2 Bảy câu hỏi để mổ xẻ (Seven Questions for Dissection)	61
3.3 Ba nguồn thông tin: paper, code, ablation	63
3.4 Độ khó không biến mất (Conservation of Difficulty)	64
3.5 Ba quy tắc quyết định	65
3.6 Thực hành	66
3.7 Giới hạn và trường hợp hỏng	67
3.8 Kết nối	70

4	Nền tảng học từ dữ liệu (Foundations of Learning from Data)	71
4.1	Đặt vấn đề	72
4.2	Từ ERM tới no free lunch (From ERM to No Free Lunch)	72
4.3	Vì sao khoảng cách pixel vô nghĩa (Why Pixel Distance Fails)	74
4.4	Bốn nguồn của inductive bias (Four Sources of Inductive Bias)	75
4.5	Chữ U cũ và hai lần đi xuống (Bias–Variance vs Double Descent)	76
4.6	Khi thêm dữ liệu là vô ích (Identifiability)	76
4.7	Ba đòn bẩy khi mô hình chưa đủ tốt (Three Levers)	77
4.8	Thực hành	77
4.9	Giới hạn và trường hợp hỏng	78
4.10	Kết nối	81
5	Thiết kế bài toán và định nghĩa dữ liệu (Problem Framing and Data Definition)	83
5.1	Đặt vấn đề	84
5.2	Tám quyết định theo thứ tự ràng buộc (Eight Decisions, Ordered by Reach)	84
5.3	Chọn dạng đầu ra (Choosing the Output Representation)	87
5.4	Quần thể và khung lấy mẫu (Population vs Sampling Frame)	87
5.5	Đường cong học theo kích thước dữ liệu (Data-Size Learning Curve)	88
5.6	Thực hành	89
5.7	Giới hạn và trường hợp hỏng	89
5.8	Kết nối	93
6	Chất lượng dữ liệu và nhãn (Data and Label Quality)	94
6.1	Đặt vấn đề	95
6.2	Gán nhãn là một quy trình đo (Annotation as Measurement)	96
6.3	Lỗi nhãn trong chính tập test chuẩn (Label Errors in Benchmarks)	97
6.4	Ngân sách hữu hạn: bốn đường thoát (Label-Efficient Learning)	98
6.5	Ba vấn đề âm thầm (Duplicates, Missingness, Documentation)	100
6.6	Giới hạn và trường hợp hỏng	101
6.7	Thực hành	101
6.8	Kết nối	104
7	Chia dữ liệu và giao thức đánh giá (Data Splitting and Evaluation Protocol)	106
7.1	Đặt vấn đề	108
7.2	Ba vai trò và các đường rò rỉ (Splits and Leakage)	108
7.3	Chọn metric khớp chi phí thật (Metric Selection)	111
7.4	Bốn ô của ma trận nhầm lẫn (Reading the Confusion Matrix)	112

7.5	Các tỉ lệ dẫn xuất và mẫu số của từng cái (Derived Rates and Denominators)	115
7.6	Bốn góc precision–recall (The Four Corners)	117
7.7	Gộp hai số làm một, và cái giá của việc gộp (F1, F-beta, MCC)	120
7.8	Đường cong, và cái đường cong giấu đi (PR and ROC Curves)	122
7.9	Xác suất có đáng tin không (Calibration)	124
7.10	Nhiều lớp và nhiều nhãn (Multi-class and Multi-label)	125
7.11	Metric của từng bài toán thị giác, và điều mỗi cái che giấu (Task-specific Metrics)	126
7.12	Bảng chẩn đoán ngược (Reading Numbers Backwards)	129
7.13	Mọi con số phải kèm một khoảng (Every Number Needs an Interval)	131
7.14	Con số tổng che nhóm bị bỏ rơi (Slice-based and Behavioral Evaluation)	131
7.15	Chênh lệch 0,3% có thật không? (Uncertainty in Reported Numbers)	132
7.16	Giới hạn và trường hợp hỏng	133
7.17	Thực hành	133
7.18	Kết nối	138
8	Phân phối, dịch chuyển và ngoài phân phối (Distribution Shift and OOD)	139
8.1	Đặt vấn đề	140
8.2	Ba loại dịch chuyển (Covariate, Label and Concept Shift)	141
8.3	Biết là đã trôi, chưa biết có hại (Shift Detection)	142
8.4	Thích nghi hay tổng quát hoá (Domain Adaptation vs Generalization)	142
8.5	Đo độ bền và quyền nói “tôi không biết” (Robustness and OOD Detection)	143
8.6	Đuôi dài và vòng phản hồi (Long-Tail and Feedback Loops)	144
8.7	Giới hạn và trường hợp hỏng	145
8.8	Thực hành	146
8.9	Kết nối	149
9	Thiết kế hàm mất mát (Loss Function Design)	150
9.1	Đặt vấn đề	151
9.2	Loss là một phát biểu xác suất (Loss as a Probabilistic Statement)	152
9.3	Phân loại: chọn loss là chọn hành vi (Classification: Choosing Behaviour)	153
9.4	Khi đầu ra có cấu trúc (Structured Outputs)	155
9.5	Ba nguồn giám sát ngoài nhãn lớp (Supervision Beyond Class Labels)	156
9.6	Cân bằng nhiều thành phần (Balancing Multi-Term Losses)	157
9.7	Thực hành	157
9.8	Giới hạn và trường hợp hỏng	158
9.9	Kết nối	161

10	Nguyên lý thiết kế kiến trúc (Architecture Design Principles)	163
10.1	Đặt vấn đề	164
10.2	Từ template matching tới convolution (From Template Matching to Convolution)	165
10.3	Bốn cách nhìn xa hơn, bốn cái giá (Widening the Receptive Field)	166
10.4	Residual và luồng gradient (Residual Connections and Gradient Flow)	167
10.5	Định tuyến cố định hay theo nội dung (Fixed vs Content-Based Routing)	168
10.6	Bốn thước đo chi phí (Four Cost Metrics)	169
10.7	Thực hành	170
10.8	Giới hạn và trường hợp hỏng	170
10.9	Kết nối	174
11	Giới hạn tính toán và hệ thống (Compute and Systems Limits)	175
11.1	Đặt vấn đề	176
11.2	Roofline và cường độ số học (Roofline and Arithmetic Intensity)	177
11.3	Cùng FLOPs, khác nhiều lần độ trễ (Equal FLOPs, Unequal Latency)	179
11.4	Bộ nhớ: kích hoạt so với tham số (Memory: Activations vs Parameters)	180
11.5	Độ trễ, thông lượng và ngân sách thời gian thực (Latency, Throughput, Real-Time Budgets)	181
11.6	Nút cổ chai không nằm ở GPU (Bottlenecks Outside the GPU)	182
11.7	Các con số cần thuộc lòng (Numbers Worth Memorizing)	182
11.8	Thực hành	182
11.9	Giới hạn và trường hợp hỏng	183
11.10	Kết nối	187
C	THUẬT TOÁN VÀ MÔ HÌNH	188
12	Cơ chế huấn luyện, từ vi mô tới vĩ mô (Training Mechanics, from Micro to Macro)	190
12.1	Vì sao chương này chia làm bốn mục	191
12.2	Cơ học của gradient (Mechanics of the Gradient)	193
12.3	Làm cho training chạy được (Making Training Work)	201
12.4	Tối ưu hoá và siêu tham số (Optimization and Hyperparameters)	207
12.5	Tổng quát hoá và chẩn đoán (Generalization and Diagnosis)	215
12.6	Thực hành	222
12.7	Kết nối	226
13	Các họ kiến trúc backbone (Backbone Architecture Families)	228
13.1	Vì sao chương này chia làm ba mạch	229

13.2	Dòng CNN: gỡ từng nút thắt (The CNN Lineage)	231
13.3	Từ chuỗi tới attention (From Sequences to Attention)	237
13.4	ViT và bài học về inductive bias (Vision Transformers and Inductive Bias)	243
13.5	Thực hành	249
13.6	Kết nối	253
14	Bài toán CV lõi: nhận dạng, phát hiện, phân vùng (Core Vision Tasks: Recognition, Detection, Segmentation)	255
14.1	Detection hai giai đoạn nuốt dần heuristic (Two-Stage Detection)	258
14.2	Một giai đoạn và cuộc đánh đổi tốc độ (One-Stage Detection and the Speed Trade-off)	264
14.3	Phân vùng ảnh và sự hợp nhất (Segmentation and Unification)	271
14.4	Các bài toán lân cận, mỗi cái một đánh đổi riêng (Neighbouring Tasks)	277
14.5	Thực hành	283
15	Thị giác chuỗi và không-thời gian (Sequential and Spatio-Temporal Vision)	288
15.1	Đầu ra độ dài thay đổi và sự ra đời của attention (Variable-Length Outputs and the Birth of Attention)	292
15.2	Biểu diễn video và cái giá của thời gian (Video Representation and the Cost of Time)	297
15.3	Flow, tracking và các ràng buộc riêng (Optical Flow, Tracking and Their Constraints)	302
15.4	Thực hành	309
16	Thị giác 3D và hình học (3D Vision and Geometry)	315
16.1	Bốn thế hệ, cùng một bài toán (Four Generations of 3D Reconstruction)	318
16.2	Chọn biểu diễn 3D, độ sâu và SLAM (3D Representations, Depth and SLAM)	324
16.3	NeRF, 3DGS và hợp nhất cảm biến (Neural Rendering and Sensor Fusion)	331
16.4	Thực hành	338
17	Mô hình sinh, style transfer và low-level vision (Generative Models and Low-Level Vision)	343
17.1	Bốn họ và cuộc đánh đổi ba chiều (Four Families of Generative Models)	346
17.2	Tối ưu trên pixel thay vì trên trọng số (Optimizing Pixels, Not Weights)	351
17.3	Diffusion và các bước làm cho nó khả thi (Diffusion Models)	358
17.4	Low-level vision và trade-off không thể tránh (Low-Level Vision)	363
17.5	Thực hành	369
18	Pretraining, transfer và mô hình nền tảng (Pretraining, Transfer and Foundation Models)	375
18.1	Nguồn giám sát ngày càng rẻ (Increasingly Cheap Supervision)	378
18.2	Kinh tế học của pretraining (The Economics of Pretraining)	385
18.3	Thực hành	393

D KỸ THUẬT, TỐI ƯU VÀ TRIỂN KHAI	397
19 Từ NumPy tự viết tới PyTorch thành thạo (From Hand-Written NumPy to Fluent PyTorch)	399
19.1 Giai đoạn 1: tự viết bằng NumPy (Building It Yourself in NumPy)	401
19.2 Giai đoạn 2: PyTorch thành thạo (Becoming Fluent in PyTorch)	407
19.3 Thực hành	413
20 Tối ưu và nén mô hình (Model Optimization and Compression)	417
20.1 Đặt vấn đề	418
20.2 Thứ tự thử nghiệm, và lý do của thứ tự đó	418
20.3 Mạch vấn đề → giải pháp	420
20.4 Thực hành	423
20.5 Giới hạn và trường hợp hỏng	424
20.6 Kết nối	427
21 Inference và triển khai (Inference and Deployment)	428
21.1 Đặt vấn đề	429
21.2 Mạch vấn đề → giải pháp, theo trình tự triển khai thật	429
21.3 Export và chọn runtime	430
21.4 Chỗ thời gian thực sự nằm	431
21.5 Batching động và ngân sách độ trễ	432
21.6 Mô hình sinh và VLM — cấu trúc chi phí khác hẳn	432
21.7 Vận hành và cách đo cho đúng	433
21.8 Thiết bị biên: nơi benchmark ngắn nói dối	433
21.9 Thay mô hình an toàn	433
21.10 Thực hành	433
21.11 Giới hạn và trường hợp hỏng	434
21.12 Kết nối	437
22 Tái lập và MLOps tối thiểu (Reproducibility and Minimal MLOps)	438
22.1 Đặt vấn đề	439
22.2 Ghim các đối số kiểm soát được	439
22.3 Các nguồn bất định — và vì sao chế độ tắt định không đủ	440
22.4 CI cho mô hình và giám sát sau triển khai	442
22.5 Thực hành	443
22.6 Giới hạn và trường hợp hỏng	443
22.7 Kết nối	446

E ĐÁNH GIÁ, ĐỘ TIN CẬY VÀ DỰ ÁN	448
23 Đọc paper và theo dõi một lĩnh vực (Reading Papers, Following a Field)	450
23.1 Đặt vấn đề	451
23.2 Ba lượt đọc và thứ tự bên trong một lượt (Three-Pass Reading)	451
23.3 Ghi chú và mục “chỗ tôi không tin” (Paper Notes)	453
23.4 Đóng góp thật và cách kể chuyện (Contribution vs. Narrative)	454
23.5 Nhận ra một so sánh không công bằng (Unfair Comparison)	454
23.6 Theo dõi một lĩnh vực mà không bị ngập (Field Tracking)	455
23.7 Giới hạn và trường hợp hỏng	456
23.8 Thực hành	456
23.9 Kết nối	459
24 Thiết kế thí nghiệm, ablation và phân tích lỗi (Experiment Design, Ablation, Error Analysis)	460
24.1 Đặt vấn đề	461
24.2 Nhóm A — thiết kế so sánh (Comparison Design)	462
24.3 Nhóm B — kỹ thuật ablation (Ablation Techniques)	463
24.4 Nhóm C — thống kê tối thiểu (Minimal Statistics)	464
24.5 Nhóm D — phân tích lỗi (Error Analysis)	466
24.6 Nhóm E — báo cáo và đọc ablation của người khác (Reporting)	467
24.7 Giới hạn và trường hợp hỏng	468
24.8 Thực hành	469
24.9 Kết nối	472
25 Trực quan hoá, độ tin cậy và vận hành (Visualization, Reliability, Operations)	473
25.1 Đặt vấn đề	474
25.2 Nhìn vào bên trong mô hình (Looking Inside the Model)	474
25.3 Bất định và hiệu chỉnh (Uncertainty and Calibration)	476
25.4 Vận hành (Operations)	478
25.5 Giới hạn và trường hợp hỏng	479
25.6 Thực hành	479
25.7 Kết nối	482
26 Lộ trình học theo mục tiêu (Learning Paths by Goal)	484
26.1 Đặt vấn đề	484
26.2 Ba chương nền, mọi lộ trình đều phải chạm	485
26.3 Sáu lộ trình	485
26.4 Chuỗi mini project chạy xuyên cây	487

26.5	Giới hạn và trường hợp hỏng	487
26.6	Kết nối	489

F HỌC SÂU CHO VISUAL SLAM 490

27	Đặt học sâu vào đâu trong một hệ SLAM (Where Learning Fits in a SLAM System)	492
27.1	Đặt vấn đề	493
27.2	Năm trụ: học sâu cắm được vào đâu (The Five Pillars)	494
27.3	Khung bốn cổng (The Four-Gate Decision Framework)	497
27.4	Bản đồ bốn nỗi đau (Mapping Four Pains to Method Families)	503
27.5	Ba mức can thiệp (Three Levels of Intervention)	506
27.6	Đọc một paper SLAM học sâu trong ba mươi phút (Reading a Deep SLAM Paper in 30 Minutes)	508
27.7	Giới hạn và trường hợp hỏng	510
27.8	Thực hành	511
27.9	Kết nối	514
28	Đặc trưng và ghép cặp học được (Learned Features and Matching)	516
28.1	Đặt vấn đề	517
28.2	Bốn chế độ hỏng của ORB, đọc bằng cơ chế (Four Failure Modes)	517
28.3	Bộ phát hiện và mô tả học được (Learned Detectors and Descriptors)	522
28.4	Ghép cặp học được và chi phí bậc hai (Learned Matching)	527
28.5	Bỏ hẳn bộ phát hiện (Detector-free Matching)	530
28.6	Đánh giá cho hệ thời gian thực (Evaluation for a Real-time System)	533
28.7	Giới hạn và trường hợp hỏng	539
28.8	Thực hành	539
28.9	Kết nối	543
29	Nhận dạng lại địa điểm và đóng vòng (Place Recognition and Loop Closure)	545
29.1	Đặt vấn đề	546
29.2	Bốn tầng và cách đo tầng nào đang giết vòng (Anatomy and Instrumentation)	547
29.3	Toàn ảnh học được: từ túi từ tới SALAD và AnyLoc (Learned Global Descriptors)	551
29.4	Định vị lại: hồi quy tư thế, hồi quy tọa độ cảnh và hloc (Relocalization)	557
29.5	Kiểm chứng: hình học so với học được (Verification)	560
29.6	Đo cho đúng: recall, precision và cái giá của một vòng sai (Measuring Loop Closure)	564
29.7	Giới hạn và trường hợp hỏng	569
29.8	Thực hành	570
29.9	Kết nối	573

30	Độ sâu, dòng quang và mô hình nền hình học (Depth, Flow and Geometric Foundation Models)	575
30.1	Đặt vấn đề	576
30.2	Độ sâu từ một ảnh và vấn đề tỉ lệ (Monocular Depth and the Scale Problem)	577
30.3	Dòng quang dày: từ FlowNet tới SEA-RAFT (Dense Optical Flow)	584
30.4	Mô hình nền hình học: DUS3R, MAST3R và VGGT (Geometric Foundation Models)	589
30.5	Cổng đánh giá: khi nào nó cứu, khi nào nó phá (The Evaluation Gate)	595
30.6	Giới hạn và trường hợp hỏng	599
30.7	Thực hành	600
30.8	Kết nối	604
31	Backend khả vi và hệ VO/SLAM đầu-cuối (Differentiable Backends and End-to-End Systems)	606
31.1	Đặt vấn đề	607
31.2	Đạo hàm xuyên qua một nghiệm tối ưu (Differentiating Through an Argmin)	609
31.3	Hạ tầng khả vi: LieTorch, Theseus và BA-Net (Differentiable Infrastructure)	614
31.4	DROID-SLAM và cuộc thừa hoá sau nó (DROID-SLAM and the Sparsification Line)	616
31.5	VO tự giám sát và giới hạn thật của nó (Self-Supervised VO)	621
31.6	Biểu diễn ẩn: nén hình học vào một mã ngắn (Latent Scene Representations)	623
31.7	Giới hạn và trường hợp hỏng	626
31.8	Thực hành	628
31.9	Kết nối	631
32	Bản đồ neural: NeRF và 3D Gaussian Splatting (Neural Map Representations)	633
32.1	Đặt vấn đề: bản đồ thừa đủ cho việc gì	634
32.2	Dòng NeRF: một MLP làm bản đồ, rồi mọi thứ sinh ra từ chỗ nó hỏng	636
32.3	Dòng 3D Gaussian Splatting: vì sao rasteri hoá đổi cuộc chơi	641
32.4	Hai trục đánh giá, và vì sao chúng bị trộn lẫn	645
32.5	Ngân sách trên Jetson Orin, và câu hỏi có đáng cho robot chưa	650
32.6	Giới hạn và trường hợp hỏng	654
32.7	Thực hành	654
32.8	Kết nối	657
33	Ngữ nghĩa, vật thể động và đồ thị cảnh (Semantics, Dynamics and Scene Graphs)	659
33.1	Đặt vấn đề	660
33.2	Công cụ nhận thức chạy được trong vòng lặp (Perception Front-Ends in the Loop)	661
33.3	Ba việc khác nhau khi loại vật thể động (Three Different Jobs)	666
33.4	Đồ thị cảnh 3D (3D Scene Graphs)	674

33.5	Ngữ nghĩa mở và ngôn ngữ (Open-Vocabulary Semantics and Language)	677
33.6	Giới hạn và trường hợp hỏng	678
33.7	Thực hành	679
33.8	Kết nối	683
34	IMU, bất định học được và đưa lên phần cứng (IMU, Learned Uncertainty and Deployment)	685
34.1	Đặt vấn đề	686
34.2	Trọng số chính là ma trận thông tin (Weights Are the Information Matrix)	687
34.3	Học nhiễu và bất định IMU (Learned Noise and Learned Uncertainty)	692
34.4	Ảnh mờ do chuyển động và chuyển động nhanh (Motion Blur and Fast Motion)	697
34.5	Đưa lên Jetson Orin: phần đặc thù SLAM (Deploying on Orin)	702
34.6	Lộ trình học cho riêng hướng này (A Learning Path for This Direction)	706
34.7	Giới hạn và trường hợp hỏng	707
34.8	Thực hành	708
34.9	Kết nối	712
	Bảng ánh xạ thuật ngữ	714
	Kết luận: cái gì còn lại sau khi mọi kiến trúc đã cũ	728
34.10	Bốn nguyên lý xuyên suốt	728
34.11	Cái gì bền và cái gì sẽ cũ	728
34.12	Ba thói quen đáng mang theo	729
34.13	Làm gì tiếp	729

NỀN TẢNG CÓ GIỚI HẠN



TÓM TẮT

Phần A dựng đúng lượng nền cần thiết cho bốn phần sau: ngôn ngữ toán tối thiểu, và chuỗi vật lý biến ánh sáng thành tensor mà mô hình nhìn thấy. Đọc xong bạn tự *derive* lại được đạo hàm khi cần, và quan trọng hơn, bạn nhận ra được khi một giả định về ánh đầu vào đã bị vi phạm. Nếu chỉ nhớ một điều: phần lớn khoảng cách giữa lúc huấn luyện và lúc triển khai sinh ra ở khâu tạo ảnh, không ở kiến trúc mạng.

Phần này tồn tại để phục vụ bốn phần sau, và nguyên tắc chi phối nó là *học theo nhu cầu*: mỗi mục chỉ đi tới mức dùng được và giải thích được, không chứng minh định lý. Đây là một lựa chọn có chủ ý chứ không phải sự lười biếng. Một người dừng lại học trọn vẹn giải tích ma trận trước khi chạm vào mô hình đầu tiên thường không bao giờ tới được mô hình đầu tiên; ngược lại, một người bỏ hẳn phần nền sẽ đọc paper như đọc bùa chú và không bao giờ đoán được một phương pháp sẽ hỏng ở đâu. Ranh giới đúng nằm ở chỗ: học đủ để *tự derive lại được khi cần* và để *nhận ra khi một giả định bị vi phạm*.

Hai chương của phần có vai trò rất khác nhau. Chương 1 là ngôn ngữ: đại số tuyến tính, vi phân, xác suất, lý thuyết thông tin, hình học biến đổi và xử lý tín hiệu — không phải toàn bộ các môn đó, mà đúng những mảnh quay lại nhiều lần nhất trong phần B và C. Chương 2 là thứ bị bỏ sót nhiều nhất và cũng là thứ phân biệt người làm computer vision với người chỉ dùng CNN: ảnh không rơi từ trên trời xuống thành một tensor, nó là kết quả của một chuỗi vật lý và xử lý dài, và phần lớn khoảng cách giữa lúc huấn luyện và lúc triển khai sinh ra ở chuỗi đó chứ không ở kiến trúc mạng.

Vì vậy thứ tự đọc đề xuất không phải là đọc hết phần này rồi mới sang phần B. Hãy đọc chương 2 kỹ ngay từ đầu, còn chương 1 thì đọc lướt một lượt để biết trong đó có gì, rồi quay lại từng mục khi phần sau thực sự cần đến — mỗi mục trong chương 1 đều ghi rõ nó quay lại ở chương nào.

Bảng 3. Bản đồ Phần A: các chương, nội dung, và mức đầu tư đề xuất.

Ch.	Thư mục	Nội dung	Mức
1	01-toan-toi-thieu	Ngôn ngữ toán tối thiểu: đại số tuyến tính, vi phân và tối ưu, xác suất và lý thuyết thông tin, hình học biến đổi và xử lý tín hiệu	tham khảo, tra khi cần
2	02-vat-ly-tao-anh	Ảnh sinh ra thế nào: ánh sáng và nhiễu, ISP và không gian màu, mô hình camera và hình học nhiều view, cảm biến ngoài RGB và nguồn gốc domain gap	cốt lõi

Chương 1 đọc lướt rồi quay lại từng mục khi phần B và C cần đến; chương 2 đọc kỹ ngay, vì phần lớn khoảng cách giữa huấn luyện và triển khai sinh ra ở đó.

Toán tối thiểu dùng lại suốt đời (The Minimal Mathematics You Reuse Forever)

PHỤ THUỘC không — đây là chương mở đầu của cây. Chương này chỉ nên đọc *theo nhu cầu*: quay lại đúng mục cần khi một chương ở Phần B, C hay D đòi hỏi, chứ không đọc một mạch từ đầu tới cuối.

TÓM TẮT

Chương này giữ lại đúng phần toán quay lại nhiều lần về sau, chia theo *vai trò* chứ không theo tên môn học. Mục 1 lo không gian và phổ. Mục 2 dựng bộ máy đưa mô hình từ khởi tạo tới nghiệm. Mục 3 nói nguồn gốc của hàm mất mát và độ tin cậy của một con số đo được. Mục 4 nói cấu trúc có sẵn trong dữ liệu ảnh. Đọc xong bạn chẩn đoán được nguyên nhân của một đường cong huấn luyện xấu, suy được loss từ giả định nhiễu thay vì tra bảng, và bắt được các lỗi im lặng ở khâu tiền xử lý. Nếu chỉ nhớ một điều: đây là chương [T1], nên dừng học cho đủ — hãy quay lại đúng mục mà một chương sau đang cần, vì kiến thức học không có nhu cầu đi kèm sẽ bay hết trong hai tuần.

KIẾN THỨC CẦN CÓ TRƯỚC

- **deep learning** — cách tiếp cận trong đó ta xếp chồng nhiều tầng biến đổi có tham số, rồi để dữ liệu tự quyết định các tham số đó thông qua tối ưu hoá, thay vì tự tay thiết kế đặc trưng.
- **mô hình, tham số, huấn luyện** — mô hình là một hàm có tham số nhận đầu vào và trả về dự đoán; tham số là các con số bên trong nó; huấn luyện là quá trình tìm bộ tham số làm dự đoán sát nhãn nhất.
- **hàm mất mát (loss function)** — một con số đo mức sai của dự đoán so với nhãn — đại lượng mà quá trình huấn luyện thực sự cực tiểu.
- **tensor** — mảng số nhiều chiều; một ảnh màu là tensor ba chiều (cao $\times$ rộng $\times$ kênh), một batch ảnh là tensor bốn chiều.
- **nhãn tầng bán rã [T1]–[T4]** — quy ước của cây này về tuổi thọ của tri thức: [T1] là toán và vật lý gần như không đổi; [T2] là nguyên lý thiết kế hệ thống; [T3] là thuật toán và mô hình; [T4] là thư viện và công cụ, cũ nhanh nhất.
- **SLAM và hiệu chỉnh chùm tia (bundle adjustment)** — SLAM là bài toán vừa dựng bản đồ vừa định vị trong chính bản đồ đó; hiệu chỉnh chùm tia là bước tinh chỉnh đồng thời mọi tư thế camera và điểm 3D bằng bình phương tối thiểu. Chương này dùng chúng làm điểm neo quen thuộc, không đòi hỏi bạn phải cài được chúng.

CÂU HỎI MỞ ĐẦU

- Q1. Nếu quy tắc chuỗi là thứ ai cũng học từ năm nhất, vì sao tính *gradient* cho một mạng 10^9 tham số lại từng là chuyện bất khả thi?
- Q2. Vì sao đổi đơn vị từ mét sang milimét trong một bài hiệu chỉnh chùm tia làm số điều kiện đổi ba bậc, trong khi bài toán không hề dễ hay khó đi?
- Q3. Hai mô hình chênh nhau một điểm phần trăm trên tập test 500 mẫu, mà khoảng tin cậy của mỗi mô hình rộng hơn năm điểm — vậy vì sao kết luận “A tốt hơn B” vẫn có thể đúng?
- Q4. Vì sao một mạng dự đoán phép quay bằng bốn số lại khó huấn luyện hơn bằng sáu số, dù bốn số đã thừa để mô tả ba bậc tự do?

Mọi lộ trình học deep learning đều bắt đầu bằng một chương toán, và phần lớn người học đều bỏ dở nó. Lý do không phải toán khó mà vì chương đó thường được viết như một giáo trình rút gọn: đầy đủ, tuyến tính, và không nói cho người đọc biết thứ nào sẽ quay lại vào lúc nào. Kết quả là người ta học đại số tuyến tính suốt sáu tuần rồi vẫn không biết vì sao huấn luyện của mình không hội tụ. Chương này cố ý làm ngược lại: chỉ giữ những gì *thật sự* quay lại nhiều lần, và mỗi mục phải trả lời được câu “thứ này xuất hiện lại ở đâu”.

Cách chia bốn mục theo vai trò, và giữa chúng có phụ thuộc thật chứ không phải thứ tự tùy tiện:

- **Mục 1 — đại số tuyến tính:** bộ từ vựng về không gian và phổ. Trả lời “bài toán này suy biến ở hướng nào, tôi mất bao nhiêu chữ số”.
- **Mục 2 — vi phân và tối ưu hoá:** bộ máy đưa mô hình từ khởi tạo tới nghiệm. *Phụ thuộc mục 1*, vì “hình dạng landscape” chính là phổ trị riêng của Hessian nói bằng lời khác.
- **Mục 3 — xác suất và lý thuyết thông tin:** trả lời hai câu mà hai mục trước không đụng tới — hàm mất mát từ đâu ra, và con số đo được đáng tin đến đâu. *Tiếp nhận câu hỏi mà mục 2 để ngỏ*.
- **Mục 4 — hình học và tín hiệu:** khác hẳn ba mục kia ở chỗ nó không nói về *cách học* mà về *cấu trúc có sẵn trong dữ liệu ảnh*. Đọc được đọc lập, và là cầu nối trực tiếp sang chương [A/02](#).

Thứ tự đọc đề nghị là $1 \rightarrow 2 \rightarrow 3$, còn mục 4 đọc được bất cứ lúc nào. Với người đã làm SLAM bằng C++, phần lớn mục 1 và nửa đầu mục 4 là ôn tập; bốn chỗ đáng dừng lại vì kinh nghiệm ước lượng trạng thái *không* chuyển sang miễn phí là: số điều kiện đọc theo nghĩa tốc độ huấn luyện, điều kiện dừng theo validation, toàn bộ mục 3, và phần lấy mẫu của mục 4.

Bản đồ chương — Toán tối thiểu dùng lại suốt đời

TÓM TẮT

Bản đồ này nói chương 1 gồm bốn mục nào, và vì sao chia theo vai trò chứ không theo tên môn học. Nó cũng nói mục nào phụ thuộc mục nào, và phần nào là cốt lõi so với phần nào chỉ cần biết là có. Đọc xong bạn quyết định được trong một phút nên đọc mục nào trước cho nhu cầu hiện tại thay vì đọc tuần tự cả chương. Nếu chỉ nhớ một điều: đây là chương [\[T1\]](#), nên nguyên tắc là học theo nhu cầu. Chuỗi phụ thuộc bắt buộc chỉ là $1 \rightarrow 2 \rightarrow 3$. Mục 4 vào được bất cứ lúc nào, và nên đọc ngay trước chương [A/02](#).

A. Chương này có gì

- **01-dai-so-tuyen-tinh** — không gian con, phép chiếu, phân rã trị riêng và SVD, số điều kiện, xấp xỉ hạng thấp.
- **02-vi-phan-va-toi-uu** — quy tắc chuỗi dạng ma trận, Jacobian, Hessian, hạ gradient, lỗi so với không lồi, điều kiện dừng, hình dạng landscape.
- **03-xac-suat-va-thong-tin** — sai số lấy mẫu và khoảng tin cậy, MLE và MAP như nguồn gốc chung của mọi hàm mất mát, entropy-cross-entropy-KL và sự bất đối xứng của KL.
- **04-hinh-hoc-va-tin-hieu** — tọa độ đồng nhất, $SO(3)$ và $SE(3)$ ở mức ba thao tác dùng được, convolution, Fourier, lấy mẫu và aliasing.

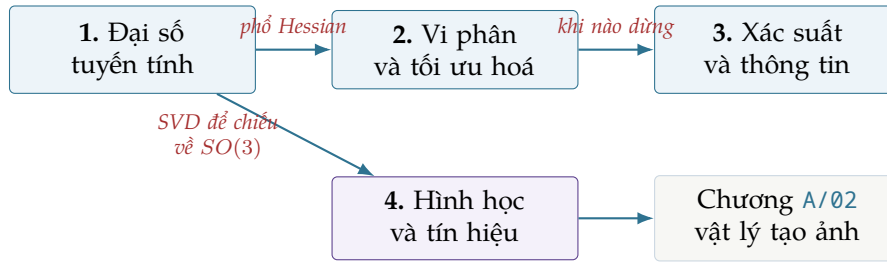
B. Vì sao chia như vậy

Cách chia thông thường theo tên môn học — “đại số tuyến tính, giải tích, xác suất” — không nói thứ nào quay lại vào lúc nào. Nó vì thế khuyến khích học tuyến tính cho đủ, rồi quên sạch. Bốn

mục ở đây được đặt theo *câu hỏi chúng trả lời*: mục 1 trả lời “bài toán này suy biến ở hướng nào và tôi mất bao nhiêu chữ số”; mục 2 trả lời “làm sao đi từ khởi tạo tới nghiệm, và cái gì làm nó chậm”; mục 3 trả lời “hàm mất mát từ đâu ra, và con số tôi vừa đo đáng tin đến đâu”; mục 4 khác hẳn ba mục kia vì nó không nói về cách học mà về *cấu trúc có sẵn trong dữ liệu ảnh trước khi mô hình nhìn thấy*.

C. Phụ thuộc và thứ tự đọc

Hình 1.1 vẽ quan hệ phụ thuộc thật giữa bốn mục.



Hình 1.1. Phụ thuộc giữa bốn mục của chương 1. Chuỗi ngang là chuỗi bắt buộc; mục 4 (tô khác màu) chỉ mượn một kết quả nhỏ của mục 1 nên vào được bất cứ lúc nào, và nó là cửa dẫn sang chương A/02.

D. Cốt lõi hay tham khảo

Toàn chương thuộc tầng [T1], tức học theo nhu cầu và không đầu tư sâu. Trong đó:

- **Cốt lõi — phải nắm chắc, sẽ quay lại nhiều lần:** số điều kiện và xấp xỉ hạng thấp (mục 1); quy tắc khớp shape, chi phí bộ nhớ của lan truyền ngược, quan hệ độ cong–learning rate (mục 2); *toàn bộ* mục 3; phần lấy mẫu và aliasing (mục 4).
- **Tham khảo — biết là có, tra khi cần:** chi tiết các phân rã ma trận, phân tích lỗi hình thức, phần Lie algebra vượt quá ba thao tác exp, log và cập nhật trên không gian tiếp tuyến.

E. Riêng với người đọc đã làm SLAM

Bốn chỗ mà kinh nghiệm ước lượng trạng thái *không* chuyển sang miễn phí, nên dừng lại kỹ: (i) số điều kiện đọc theo nghĩa tốc độ huấn luyện chứ không chỉ theo nghĩa ổn định số học; (ii) điều kiện dừng theo validation thay cho chuẩn gradient; (iii) toàn bộ mục 3, vì ước lượng trạng thái hiếm khi phải trả lời câu “hiệu số này có ý nghĩa thống kê không”; (iv) aliasing trong kiến trúc mạng, vốn không có đối ứng trong pipeline hình học cổ điển. Ngược lại, tọa độ đồng nhất, $SE(3)$ và bình phương tối thiểu thì đọc lướt được.

I 1.1 Đại số tuyến tính ứng dụng (Applied Linear Algebra)

PHỤ THUỘC không — đây là mục mở đầu của cây. Nếu bạn đã cài hiệu chỉnh chùm tia (*bundle adjustment*) thì phần không gian con là ôn lại; chỗ đáng đọc kỹ là *số điều kiện* và *hạng hiệu dụng*, hai chỗ mà ngôn ngữ SLAM và ngôn ngữ deep learning gọi cùng một hiện tượng bằng hai cái tên khác nhau.

TÓM TẮT

Mục này rút đại số tuyến tính xuống đúng bốn thứ quay lại suốt cây: không gian con và phép chiếu, hai phân rã eigen và SVD, số điều kiện, xấp xỉ hạng thấp. Đọc xong bạn phân biệt được “huấn luyện

chậm vì bài toán điều hoà tẻ” với “chậm vì learning rate sai”. Bạn đọc được phổ trị kỳ dị của một tầng đặc trưng. Và bạn biết vì sao một solver nghiêm túc không bao giờ lập $A^T A$. Toàn bộ được neo vào thứ bạn đã làm được: hiệu chỉnh chùm tia, ellipsoid hiệp phương sai, gauge freedom. Nếu chỉ nhớ một điều: bình phương tối thiểu là một phép chiếu vuông góc, còn số điều kiện là hệ số khuếch đại sai số của *bài toán* chứ không phải của thuật toán. Hệ quả: đối optimizer không cứu được một bài toán bị điều hoà tẻ.

KIẾN THỨC CẦN CÓ TRƯỚC

- **Quy ước ký hiệu** — vector luôn là vector *cột*; A^T là ma trận chuyển vị; ma trận $m \times n$ có m hàng và n cột; $\|\cdot\|$ không ghi chỉ số nghĩa là chuẩn Euclid (chuẩn 2).
- **mô hình (model)** — một hàm có tham số, nhận đầu vào và trả về dự đoán. “Huấn luyện” nghĩa là đi tìm bộ tham số làm dự đoán sát nhãn nhất.
- **trọng số (weight)** — chính là các tham số của mô hình. Trong một tầng tuyến tính, chúng được xếp thành ma trận W và phép tính là $Wx + b$.
- **gradient** — vector gồm các đạo hàm riêng của một hàm theo từng tham số; nó chỉ hướng làm hàm tăng nhanh nhất, nên thuật toán tối ưu đi *ngược* hướng đó.
- **ma trận Hessian** — ma trận đạo hàm bậc hai; nó đo *độ cong* của hàm — hướng nào dốc gắt, hướng nào phẳng.
- **trị riêng và vector riêng** — vector riêng là hướng mà ma trận chỉ kéo dãn chứ không xoay; trị riêng là hệ số kéo dãn theo hướng đó. Tập hợp các trị riêng gọi là *phổ* của ma trận.
- **hạng (rank)** — số chiều thật sự mà đầu ra của ma trận trải ra. Hạng thấp nghĩa là ma trận nén dữ liệu về một không gian con nhỏ hơn.
- **hiệu chỉnh chùm tia (bundle adjustment)** — trong SLAM và dựng lại 3D, bài toán tinh chỉnh đồng thời mọi tư thế camera và mọi điểm 3D bằng cách cực tiểu tổng bình phương sai số tái chiếu. Nó là ví dụ chuẩn của bình phương tối thiểu phi tuyến quy mô lớn.

Đọc nhanh. Nếu bốn ý trên đã rõ, đọc thẳng tiểu mục D (số điều kiện) và E (giới hạn).

A. Đặt vấn đề

Một kỹ sư SLAM giải hệ tuyến tính mỗi ngày mà không gọi tên nó là đại số tuyến tính: mỗi vòng *Gauss–Newton* của hiệu chỉnh chùm tia là một bài bình phương tối thiểu. Mỗi ma trận **hiệp phương sai** (*covariance*) của keyframe là một dạng toàn phương đối xứng. Và mỗi lần khôi phục pose từ ma trận cơ bản là một lần gọi **phân rã giá trị kỳ dị** (*singular value decomposition, SVD*). Khi bước sang deep learning, những đối tượng ấy quay lại nguyên vẹn nhưng dưới tên khác: “không gian đặc trưng” thay cho không gian cột, **rank collapse** thay cho ma trận suy biến, “*ill-conditioned loss*” thay cho số điều kiện lớn. Vấn đề không phải thiếu kiến thức mà thiếu ánh xạ giữa hai bộ từ vựng.

Học xong mục này thì làm được gì mà trước đó không làm được? Ba việc rất cụ thể: (i) nhìn một đường cong huấn luyện chậm và phân biệt được “bài toán bị điều hoà tẻ” với “learning rate sai” với “mô hình quá nhỏ”; (ii) đọc một biểu đồ phổ trị kỳ dị của tầng đặc trưng và nói được mô hình đang thu hẹp biểu diễn hay không; (iii) biết vì sao một thư viện giải bài bình phương tối thiểu nghiêm túc không bao giờ lập ma trận $A^T A$.

Mục này không dạy lại đại số tuyến tính. Nó chọn ra bốn ý sẽ quay lại suốt Phần B, C, D và cho mỗi ý một chỗ neo trong thứ bạn đã làm được bằng C++:

- **Bốn ý được giữ lại, và câu hỏi mà mỗi ý trả lời:**
 - **không gian con** (*subspace*) và **phép chiếu** (*projection*) — “nghiệm bình phương tối thiểu là cái gì, chứ không phải tính bằng công thức nào”.

- **phân rã trị riêng** (*eigendecomposition*) và SVD — “ma trận này kéo dãn không gian theo những trục nào, mạnh yếu ra sao”.
- **số điều kiện** (*condition number*) — “tôi sắp mất bao nhiêu chữ số, và hướng nào của bài toán gần như không bị ràng buộc”.
- **xấp xỉ hạng thấp** (*low-rank approximation*) — “giữ lại bao nhiêu là đủ, và bỏ đi cái gì”.

Nguyên tắc của cả Phần A áp dụng nguyên: chỉ tới mức dùng được và giải thích được, không chứng minh định lý. Ai muốn chứng minh thì đọc [1]; bản trình bày định hướng học máy ở [2].

TRỰC GIÁC

Bạn đã nhìn thấy SVD mỗi lần vẽ ellipsoid bất định của một keyframe mà không biết. Ellipsoid đó là hình ảnh của quả cầu đơn vị sau khi bị ma trận hiệp phương sai kéo dãn: hướng các trục chính là các **vector riêng** (*eigenvector*), độ dài các trục là căn của các **trị riêng** (*eigenvalue*). Mọi ma trận đều làm đúng ba động tác ấy với quả cầu đơn vị — xoay, kéo dãn theo vài trục ưu tiên, rồi xoay lần nữa — và SVD chỉ là biên bản ghi lại ba động tác. Khi ellipsoid dẹt gần thành đĩa, bài toán suy biến theo một hướng, và đó chính là lúc số điều kiện lớn.

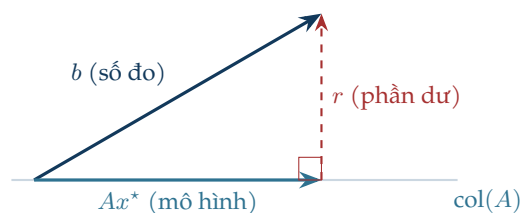
B. Không gian con và phép chiếu: bình phương tối thiểu không phải một công thức

Bình phương tối thiểu (*least squares*) thường được nhớ như công thức $x^* = (A^T A)^{-1} A^T b$. Nhớ như vậy thì không giải thích được vì sao nó hoạt động và hỏng khi nào. Cách nhớ dùng được lâu hơn là hình học: cột của A căng ra một không gian con của $\mathbb{R}^m$; vector đo b nói chung không nằm trong đó vì có nhiều; nghiệm bình phương tối thiểu là *hình chiếu vuông góc* của b xuống không gian cột, và phần dư $r = b - Ax^*$ vuông góc với mọi cột của A . Điều kiện vuông góc viết ra là

$$A^T r = 0 \iff A^T A x = A^T b.$$

Nói bằng lời: *phương trình chuẩn tắc không phải một mẹo đại số, nó chỉ là cách viết lại câu “phần dư phải vuông góc với mọi thứ mô hình giải thích được”*. Công thức là hệ quả, hình học là nguyên nhân.

Ví dụ nhỏ nhất tính tay được. Khớp một hằng số c vào ba số đo 1, 2, 6. Ở đây $A = (1, 1, 1)^T$, không gian cột là đường chéo. Chiều $b = (1, 2, 6)^T$ xuống đường ấy cho $c = (1 + 2 + 6)/3 = 3$, phần dư là $(-2, -1, 3)^T$, tổng bằng 0 — đúng là vuông góc với $(1, 1, 1)^T$. Trung bình cộng không phải một quy ước thống kê; nó là một phép chiếu. Hình 1.2 vẽ lại quan hệ đó ở dạng hai chiều.



Hình 1.2. Nghiệm bình phương tối thiểu là hình chiếu vuông góc của số đo b xuống không gian cột của A . Mọi biến thể — có trọng số, có ràng buộc, phi tuyến sau khi tuyến tính hoá — giữ nguyên bức tranh này và chỉ đổi cách đo góc vuông.

Bài toán phi tuyến trong SLAM không nằm ngoài bức tranh đó: Gauss–Newton tuyến tính hoá phần dư quanh điểm hiện tại, giải một bài chiếu, đi một bước, lặp lại. Ma trận $H = J^T J$ mà bạn quen gọi là “Hessian xấp xỉ” chính là $A^T A$ của bài chiếu ấy. Trong deep learning nó xuất hiện lại dưới tên **Gauss–Newton matrix**, và là bộ khung của mọi phương pháp bậc hai xấp xỉ. Đây là chỗ nối đầu tiên trong nhiều chỗ nối: cùng một ma trận, hai cộng đồng, hai cái tên.

I Mạch 1 — từ phép chiếu tới các phân rã

- 1 Vấn đề** giải $A^\top Ax = A^\top b$ trực tiếp mất chính xác một cách khó hiểu.

Giải pháp không lập $A^\top A$ mà phân rã A bằng **QR** hoặc SVD rồi giải trên dạng đã phân rã.

Hệ quả trị kỳ dị của $A^\top A$ là bình phương trị kỳ dị của A , nên số điều kiện bị bình phương và bạn mất khoảng một nửa số chữ số có nghĩa.
- 2 Vấn đề** nhưng hiệu chỉnh chùm tia vẫn lập $J^\top J$ mỗi vòng.

Giải pháp bài toán được điều hoà bằng **giảm chấn** (*damping*) kiểu Levenberg–Marquardt, và cấu trúc thưa làm **Cholesky** rẻ hơn QR nhiều bậc.

Giá phải trả bạn đánh đổi độ chính xác số học lấy tốc độ, và trả giá đúng lúc bài toán suy biến — ví dụ chuỗi keyframe gần thẳng hàng.
- 3 Vấn đề** cần biết “hướng nào của bài toán yếu”.

Giải pháp phân rã trị riêng của ma trận đối xứng H , hoặc SVD của J .

Hệ quả trị riêng nhỏ nhất chỉ ra hướng mà dữ liệu gần như không ràng buộc: trong SLAM đó là *gauge freedom*, trong deep learning đó là hướng phẳng của **bề mặt hàm mất mát** (*loss landscape*) mà **hạ gradient** (*gradient descent*) đi rất chậm.

C. Ba phân rã, chọn cái nào

Ba công cụ hay bị dùng lẫn. Bảng 1.1 đặt chúng cạnh nhau; cột cuối là cột đáng giá nhất.

Bảng 1.1. Ba phân rã ma trận dùng nhiều nhất, so trên cùng tiêu chí. $m \times n$ với $m \geq n$.

Phân rã	Cơ chế	Chi phí	Mạnh khi nào	Hỏng / không dùng được khi
Cholesky	$H = LL^\top$ trên ma trận đối xứng xác định dương	$\frac{1}{3}n^3$, rất rẻ khi thưa	Bài toán lớn, thưa, đủ điều hoà — lựa chọn mặc định trong BA	H suy biến hoặc gần suy biến; đã bình phương số điều kiện từ trước
QR	Trực giao hoá cột của A	$\approx 2mn^2$	Bình phương tối thiểu cần độ chính xác, A điều hoà kém	Ma trận rất lớn; không cho biết phổ trị kỳ dị
SVD	$A = U\Sigma V^\top$, mọi ma trận	$\approx O(mn^2)$, đắt nhất	Cần biết hạng, cần xấp xỉ hạng thấp, cần chiếu về $SO(3)$	Kích thước lớn (phải dùng phương pháp lặp hoặc phác thảo ngẫu nhiên)

Quan hệ giữa phân rã trị riêng và SVD gọn trong một câu: trị kỳ dị của A là căn bậc hai của trị riêng của $A^\top A$, và vector kỳ dị phải là vector riêng của $A^\top A$. Phân rã trị riêng chỉ đẹp khi ma trận đối xứng (hiệp phương sai, Hessian, ma trận Gram của đặc trưng); SVD tồn tại cho mọi ma trận nên nó là lựa chọn mặc định khi ma trận là một phép biến đổi giữa hai không gian khác nhau, chẳng hạn ma trận trọng số của một tầng tuyến tính.

Hệ quả thực dụng dùng lại nhiều nhất là **định lý xấp xỉ hạng thấp**: cắt bỏ các trị kỳ dị nhỏ cho ra ma trận hạng k gần A nhất. Ba chỗ trong cây này sống trên nó:

- **Nén và phân tích biểu diễn — phân tích thành phần chính** (*principal component analysis, PCA*) chỉ là chiếu dữ liệu đã trừ trung bình lên k vector kỳ dị phải đầu tiên.

- **Ép về cấu trúc hợp lệ** — ước lượng ma trận cơ bản rồi ép trị kỳ dị thứ ba về không; chiếu một ma trận nhiều về $SO(3)$ bằng cách đặt cả ba trị kỳ dị bằng một. Đây là xấp xỉ hạng thấp có ràng buộc, và nó quay lại ở A/01/04 khi mạng phải xuất ra một phép quay.
- **Tinh chỉnh rẻ** — *LoRA* [3] giả thiết phần cập nhật trọng số khi tinh chỉnh có hạng thấp, nên chỉ học hai ma trận gầy thay vì cả ma trận đầy.

↔ **trường hợp riêng của C** — LoRA áp dụng định lý xấp xỉ hạng thấp, nhưng cho ma trận *cập nhật trọng số* thay vì cho ma trận dữ liệu

D. Số điều kiện: đại lượng dự báo rắc rối

Số điều kiện $\kappa(A) = \sigma_{\max}/\sigma_{\min}$ trả lời đúng một câu hỏi: nếu đầu vào sai lệch tương đối ε , nghiệm có thể sai lệch tương đối tới $\kappa\varepsilon$. Nói bằng lời: κ là *hệ số khuếch đại sai số của bài toán, không phải của thuật toán*.

Ví dụ tính tay. Với $A = \text{diag}(1; 0,01)$ thì $\kappa = 100$; một nhiễu 1% đặt đúng vào thành phần thứ hai của b làm thành phần thứ hai của nghiệm sai 100%. Ma trận không hề suy biến, định thức khác không, mọi phép tính vẫn chạy — và kết quả vẫn vô nghĩa. Đó là lý do định thức gần như vô dụng làm chỉ báo, còn κ thì không.

Trong deep learning cùng đại lượng ấy chi phối tốc độ hội tụ, và gần như mọi “mẹo huấn luyện” quen thuộc là một cách hạ κ gián tiếp:

- **Chuẩn hoá đầu vào và chuẩn hoá theo batch (*batch normalization*)** [4] — kéo các hướng về cùng thang đo, tức trực tiếp giảm tỉ số trị riêng lớn nhất trên nhỏ nhất.
- *Adam* [5] — chia mỗi toạ độ cho một ước lượng độ lớn riêng, tương đương đổi thang đo từng chiều, nên nó “thấy” một bài toán điều hoà tốt hơn bài toán thật.
- **Khởi tạo cẩn thận** — giữ phổ trị kỳ dị của từng tầng không quá lệch ngay từ đầu, để κ không tích lũy qua độ sâu.

↔ **hệ quả của D** — chuẩn hoá, khởi tạo và optimizer thích nghi ở chương huấn luyện đều là cách hạ số điều kiện một cách gián tiếp

Một câu để mang theo: *phần lớn cái gọi là mẹo huấn luyện là kỹ thuật điều hoà số học đội lốt*. Mạch này quay lại đầy đủ ở D khi bàn optimizer.

E. Giới hạn và trường hợp hỏng

Bốn công cụ trên có giới hạn rõ, và biết giới hạn quan trọng hơn biết công thức.

- **Số điều kiện phụ thuộc đơn vị đo.** Trong một bài BA trộn tịnh tiến tính bằng mét với góc quay tính bằng radian, đổi mét sang milimét làm κ đổi ba bậc mà bài toán không dễ hay khó đi. So sánh κ giữa hai mô hình tham số hoá khác nhau là so sánh vô nghĩa.
- **Ma trận suy biến không phải lúc nào cũng là bệnh.** Hessian của BA không ràng buộc gauge suy biến đúng bảy chiều — đó là cấu trúc thật của bài toán. Hessian của mạng nơ-ron cũng suy biến vì nhiều đối xứng nội tại: hoán vị neuron trong một tầng, co giãn bù trừ giữa hai tầng liên tiếp qua *ReLU*. Cách xử lý đúng là cố định gauge hoặc chấp nhận, không phải thêm nhiễu cho đủ hạng.
- **PCA giả định cấu trúc tuyến tính và đo bằng phương sai.** Đặc trưng của mạng sâu nằm trên một đa tạp cong. Vì thế “95% phương sai trong k chiều” không có nghĩa thông tin hữu ích nằm ở đó. Hướng phương sai lớn thường là độ sáng toàn cục hoặc phong cách ảnh; hướng quyết định nhân có thể nằm ở phương sai nhỏ.

↔ **tiền đề E** — *linear probe* là bài bình phương tối thiểu trên không gian đặc trưng, nên chất lượng của nó bị chi phối bởi chính số điều kiện ở đây

Vì thể trực quan hoá đặc trưng hiện đại dùng *t-SNE* [6] hay *UMAP* [7]. Nhưng hai thứ đó đánh đổi tính trung thực về khoảng cách toàn cục — một trade-off mỗ ở E.

- **Chi phí.** SVD đầy đủ của ma trận $50\,000 \times 2048$ là chuyện của phương pháp lập hoặc phác thảo ngẫu nhiên, không phải một lời gọi thư viện vô tư.

CẠM BÃY

Đừng đọc “hạng thấp” như “ít thông tin”. Ma trận đặc trưng của một mô hình tự giám sát bị *rank collapse* là bệnh; ma trận cập nhật LoRA có hạng thấp là giả thiết thiết kế có chủ đích. Cùng một con số hạng, hai ý nghĩa trái ngược, và cái phân biệt chúng là *ma trận đó đang biểu diễn dữ liệu hay biểu diễn một phép hiệu chỉnh*. Trong thực hành nên đo **hạng hiệu dụng**, tức entropy của phổ trị kỳ dị đã chuẩn hoá. Đừng đếm số trị kỳ dị khác không: với số dấu phẩy động thì hầu như không trị kỳ dị nào đúng bằng không.

CẦN NHỚ

Bình phương tối thiểu là một phép chiếu vuông góc. Phân rã là danh sách trực kèm hệ số kéo dẫn. Số điều kiện nói bạn sắp mất bao nhiêu chữ số có nghĩa — và trong huấn luyện, nó nói bạn sắp chờ lâu cỡ nào. Ba câu đó phủ gần hết chỗ đại số tuyến tính cần cho cả cây này. [T1]

F. Kết nối

Mục này là tiền đề trực tiếp của [A/01/02-vi-phan-va-toi-uu](#): hình dạng landscape bàn ở đó chính là phổ trị riêng của Hessian nói ở đây. Nó cũng là tiền đề của [A/01/04-hinh-hoc-va-tin-hieu](#), nơi $SO(3)$ xuất hiện như một ràng buộc trực giao mà SVD dùng để ép về. Theo chiều ngang, C dùng lại xấp xỉ hạng thấp ở phần tinh chỉnh hiệu quả tham số. Còn E dùng lại phép chiếu khi bàn *linear probe*. Một linear probe *chính là* bài bình phương tối thiểu trên không gian đặc trưng, nên nó thừa hưởng đúng số điều kiện đã bàn ở trên. Mini project cuối chương (PCA và *whitening* trên CIFAR-10) là bài kiểm tra thực hành cho toàn mục.

▮ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao giải phương trình chuẩn tắc bằng cách lập $A^T A$ làm mất khoảng một nửa số chữ số có nghĩa, trong khi QR thì không?
2. Số điều kiện lớn của một ma trận trọng số nói gì và *không* nói gì về độ ổn định huấn luyện?
3. Hessian của hiệu chỉnh chùm tia suy biến bảy chiều. Vì sao đó không phải lỗi, và làm gì thì đúng?
4. Bạn đo được 95% phương sai của đặc trưng nằm trong 10 chiều đầu. Vì sao điều đó không cho phép kết luận có thể nén đặc trưng về 10 chiều mà không mất độ chính xác?
5. Cùng là “hạng thấp”, vì sao rank collapse trong học tự giám sát là bệnh còn LoRA lại là tính năng?
6. Bạn phải chọn giữa Cholesky, QR và SVD cho một bài bình phương tối thiểu 20 nghìn phương trình, 300 ẩn, ma trận thưa và điều hoà tốt. Chọn cái nào và vì sao?

▮ 1.2 Vi phân nhiều biến và tối ưu hoá (Matrix Calculus and Optimization)

PHỤ THUỘC A/01-toan-toi-thieu/01-dai-so-tuyen-tinh (phổ trị riêng, số điều kiện — toàn bộ phần “hình dạng landscape” dưới đây là cách đọc lại phổ ấy).

TÓM TẮT

Mục này giải thích cấu trúc của phép lấy đạo hàm trên đồ thị tính toán và hình dạng của bài toán tối ưu mà huấn luyện thực sự giải. Đọc xong bạn tự derive được backward của một tầng bất kỳ mà không nhầm chiều. Bạn biết vì sao **lan truyền ngược** (*backpropagation*) đặt về bộ nhớ chứ không đặt về phép tính. Và bạn chẩn đoán được “huấn luyện chậm” thuộc loại nào trong ba loại: điều hoà tệ, learning rate sai, hay tín hiệu giám sát yếu. Nếu chỉ nhớ một điều: *khớp shape trước, dấu và hệ số sau* — và trước khi đổ lỗi cho tính không lồi, hãy kiểm tra số điều kiện, vì phần lớn “bài toán khó tối ưu” hoá ra chỉ là bài toán bị điều hoà tệ.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình và tham số** — mô hình là một hàm có tham số nhận đầu vào và trả về dự đoán; tham số là các con số bên trong hàm đó mà quá trình huấn luyện được phép thay đổi.
- **hàm mất mát (loss function)** — một con số đo mức sai của dự đoán so với nhãn. Huấn luyện là quá trình đi tìm bộ tham số làm con số này nhỏ nhất.
- **gradient** — vector các đạo hàm riêng của hàm mất mát theo từng tham số; nó chỉ hướng làm loss tăng nhanh nhất, nên ta cập nhật theo hướng ngược lại.
- **learning rate** — độ dài bước mà thuật toán tối ưu đi theo hướng ngược gradient ở mỗi lần cập nhật. Quá lớn thì nghiệm bật ra ngoài và phân kỳ; quá nhỏ thì hội tụ chậm tới mức vô dụng.
- **mini-batch và epoch** — mini-batch là một nhóm nhỏ mẫu dùng để ước lượng gradient cho *một* lần cập nhật; một epoch là một lượt đi hết toàn bộ tập huấn luyện.
- **tập validation** — phần dữ liệu tách riêng, không dùng để cập nhật tham số, chỉ dùng để đo xem mô hình đang tốt lên hay xấu đi. Nó tồn tại vì loss trên tập huấn luyện luôn giảm kể cả khi mô hình đang học vẹt.
- **hàm lồi (convex)** — hàm mà đoạn thẳng nối hai điểm bất kỳ trên đồ thị luôn nằm trên đồ thị — hình dạng “lồng chảo”. Với hàm lồi, mọi cực tiểu địa phương đều là cực tiểu toàn cục; mạng nơ-ron thì không lồi.
- **Jacobian và Hessian** — Jacobian là ma trận đạo hàm bậc nhất của một hàm nhiều đầu vào nhiều đầu ra — nó là bản tuyến tính hoá tốt nhất tại một điểm. Hessian là ma trận đạo hàm bậc hai, đo độ cong.

A. Đặt vấn đề

Bạn đã viết **ma trận Jacobian** (*Jacobian*) bằng tay cho một hàm chi phí **sai số tái chiếu** (*reprojection error*) và đã từng mất một buổi chiều vì nhầm dấu hoặc nhầm thứ tự nhân. Trong deep learning, *autograd* làm hộ việc đó, nên câu hỏi đổi hình. Không còn là “đạo hàm bằng bao nhiêu”, mà là “vì sao **gradient** chết ở tầng thứ ba”, “vì sao đổi thứ tự hai phép toán làm bộ nhớ tăng gấp đôi”, và “vì sao **hàm mất mát** (*loss function*) giảm nhưng mô hình không học được gì”. Trả lời những câu đó cần hiểu *cấu trúc* của phép lấy đạo hàm, không cần kỹ năng đạo hàm tay.

Học xong mục này thì làm được gì mà trước đó không làm được? Ba việc: viết và kiểm chứng một **lượt ngược** (*backward pass*) tự cài trong mười lăm phút thay vì nửa ngày; đọc một đường cong huấn luyện và đặt tên đúng cho nguyên nhân trước khi thử ngẫu nhiên các siêu tham số; và hiểu vì sao mọi optimizer hiện đại đều là biến thể của cùng một ý tưởng, nên không cần học thuộc từng cái.

B. Bộ máy vi phân: khớp shape trước

TRỰC GIÁC

Quy tắc chuỗi ở dạng ma trận là một bài toán *lắp ống dẫn*. Mỗi tầng là một cái ống có đường kính vào và đường kính ra; Jacobian của tầng là tấm chuyển tiếp kích thước (đường kính ra) $\times$ (đường kính vào). Backprop là ghép các tấm chuyển tiếp từ cuối về đầu, và cách duy nhất để ghép sai là ghép hai tấm không khớp đường kính. Vì thế: khớp shape trước, lo dấu và hệ số sau — nếu shape khớp thì công thức chỉ còn sai được ở một hằng số hoặc một dấu trừ, và cả hai lộ ra ngay trong một lần **kiểm tra gradient** (*gradient check*).

Với hàm $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, Jacobian $J \in \mathbb{R}^{m \times n}$ là bản tuyến tính hoá tốt nhất tại một điểm:

$$f(x + \delta) \approx f(x) + J\delta.$$

Nói bằng lời: *trong một lân cận đủ nhỏ, mọi hàm trơn đều là một phép nhân ma trận* — và đó là toàn bộ lý do vi phân hữu ích. **Quy tắc chuỗi** (*chain rule*) nói Jacobian của hàm hợp là tích các Jacobian theo đúng thứ tự hợp, $J_{g \circ f} = J_g J_f$.

Toàn bộ lan truyền ngược [8] chỉ là một nhận xét về *thứ tự nhân*. Đầu ra cuối là một số vô hướng, tức $m = 1$ ở tầng hàm mất mát. Khi đó nhân tích ấy từ trái sang phải rẻ hơn từ phải sang trái rất nhiều. Lý do: mỗi bước ta nhân một vector hàng với một ma trận, thay vì nhân hai ma trận. Đó là lý do nó tên là *reverse-mode automatic differentiation*, tức là đi ngược từ đầu ra vô hướng về từng tham số; cần phân biệt *lan truyền ngược* là thuật toán còn **lượt ngược** (*backward pass*) là một lần chạy nó. Đây cũng là lý do nó tốn bộ nhớ: muốn nhân ngược thì phải giữ lại giá trị trung gian của lượt xuôi. Đánh đổi bộ nhớ lấy tính toán ấy quay lại nguyên vẹn ở D dưới cái tên **gradient checkpointing** [9].

↔ **ràng buộc bởi D** — bộ nhớ kích hoạt do lượt xuôi để lại mới là thứ chặn batch size trên một GPU, không phải số trọng số

Phiên bản đơn giản hoá của mọ khớp shape. Nếu $Y = XW$ với $X \in \mathbb{R}^{B \times d_{\text{in}}}$, $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$, thì gradient của một đại lượng phải có đúng shape của chính đại lượng đó. Vậy $\partial L / \partial W$ phải là $d_{\text{in}} \times d_{\text{out}}$; hai thứ có sẵn để ghép ra kích thước đó là $X^\top$ ($d_{\text{in}} \times B$) và $\partial L / \partial Y$ ($B \times d_{\text{out}}$); chỉ có một cách ghép hợp lệ, nên

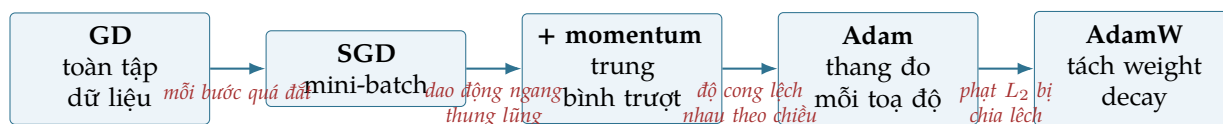
$$\frac{\partial L}{\partial W} = X^\top \frac{\partial L}{\partial Y}, \quad \frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} W^\top.$$

Không cần nhớ gì, chỉ cần đếm chiều. Quy tắc này giải quyết khoảng chín phần mười trường hợp thực tế; phần còn lại — chuẩn hoá, **softmax**, phép toán chia sẻ trọng số — thì kiểm bằng `torch.autograd.gradcheck` trên đầu vào kiểu `double`, và nếu sai thì gần như luôn sai ở một chỗ tổng-hợp-theo-*batch* bị bỏ quên.

Ma trận Hessian (*Hessian*) gần như không bao giờ được lập tường minh, vì kích thước của nó là bình phương số tham số. Nhưng *tích Hessian với một vector* tính được với chi phí cỡ một lượt ngược, và thế là đủ để đo độ cong theo một hướng. Vai trò của Hessian ở đây không phải để làm bước Newton mà để *đọc bài toán*: trị riêng lớn nhất chặn **learning rate** ổn định, phổ trị riêng cho biết landscape dẹt hay nhọn, trị riêng âm cho biết ta đang ở **điểm yên ngựa** (*saddle point*) chứ không phải cực tiểu.

C. Vì sao optimizer có hình dạng như hiện nay

Hình 1.3 đọc theo trục thời gian: mỗi bước sinh ra vì bước trước để lại một nút thắt cụ thể, không phải vì ai đó muốn thêm một siêu tham số.



Hình 1.3. Dòng tiến hoá của optimizer. Nhãn dưới mỗi mũi tên là *nút thắt* mà bước kế tiếp gỡ; đọc theo chiều này thì không cần học thuộc từng thuật toán, chỉ cần nhớ nó sinh ra để chữa cái gì.

Mạch 2 — cùng dòng tiến hoá, kèm giá phải trả

1 Vấn đề

hàm mất mát nhiều chiều, không có nghiệm đóng.

Giải pháp

đi ngược hướng gradient từng bước nhỏ.

Giả định

hàm khả vi và trơn đủ để tuyến tính hoá còn đúng trong lân cận cỡ learning rate.

Giá phải trả

phải chọn learning rate; chọn sai thì hoặc phân kỳ hoặc bò.

2 Vấn đề

tính gradient trên toàn tập dữ liệu quá đắt.

Giải pháp

ước lượng trên một mini-batch.

Hệ quả

mỗi bước rẽ đi hàng nghìn lần nên ta đi được nhiều bước hơn với cùng ngân sách.

Vấn đề mới

gradient thành biến ngẫu nhiên, với nhiễu tỉ lệ nghịch với căn của **kích thước batch** (*batch size*). Tăng batch gấp bốn chỉ giảm nhiễu một nửa. Quan hệ lợi ích giảm dần đó quyết định phần lớn cách chọn batch size [10].

3 Vấn đề

các hướng có độ cong rất khác nhau nên một learning rate chung không vừa cho hướng nào.

Giải pháp

co giãn từng tọa độ theo lịch sử độ lớn gradient, tức họ Adam [5].

Giả định

độ cong xấp xỉ tách rời được theo từng tọa độ.

Giá phải trả

mất đảm bảo hội tụ của SGD trong vài trường hợp, và sinh tương tác lạ với *weight decay* — chính chỗ đó đẻ ra AdamW [11].

4 Vấn đề

nhiều mini-batch làm quỹ đạo dao động ở cuối quá trình.

Giải pháp

giảm learning rate theo lịch trình.

Hệ quả

bước cuối đủ nhỏ để trung bình hoá nhiễu.

Vấn đề mới

lịch trình thành một siêu tham số nhạy, tương tác với cả batch size lẫn số epoch — nên chép kiến trúc mà không chép lịch trình thì không tái lập được kết quả.

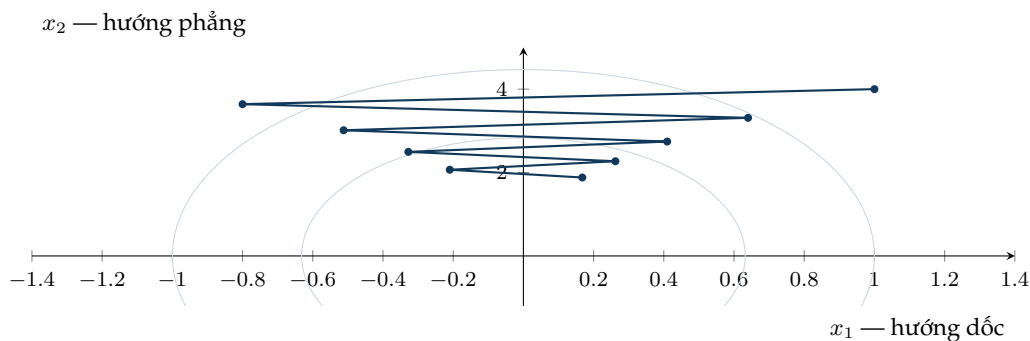
Bảng 1.2. Bốn optimizer thường gặp trên cùng tiêu chí. Cột cuối quan trọng nhất.

Thuật toán	Cơ chế	Mạnh khi nào	Hỏng khi nào
SGD	Bước theo gradient mini-batch	Bài toán điều hoà tốt sau chuẩn hoá; hay tổng quát hoá tốt hơn trên thị giác	Landscape lệch thang đo mạnh; cần rất nhiều công tune learning rate
SGD + momentum	Trung bình trượt gradient	Thung lũng hẹp dài — triệt tiêu dao động ngang	Momentum quá lớn gây vọt lố ở giai đoạn cuối
Adam	Chia mỗi toạ độ cho ước lượng độ lớn riêng	Gradient thưa hoặc lệch thang đo mạnh, ví dụ <i>transformer</i>	Đầu quá trình khi ước lượng phương sai chưa ổn (cần <i>warmup</i>); tổng quát hoá đôi khi kém hơn SGD
AdamW	Như Adam, tách weight decay khỏi bước thích nghi	Mặc định hợp lý cho hầu hết mô hình lớn hiện nay	Vẫn cần warmup và lịch trình; không cứu được bài toán mà tín hiệu giám sát sai

D. Lỗi hay không lỗi, và hình dạng thật của landscape

Câu nói phổ thông “deep learning khó vì hàm mất mát không lồi” đúng về chữ nhưng dẫn tới trực giác sai. Với bài toán **lồi** (*convex*), mọi cực tiểu địa phương là cực tiểu toàn cục và ta có chứng nhận dừng. Với bài toán không lồi cả hai thứ đó mất. Nhưng thực nghiệm trên mạng lớn cho thấy thủ phạm làm chậm không phải các cực tiểu địa phương tồi: trong không gian rất nhiều chiều, một điểm dừng ngẫu nhiên gần như chắc chắn là điểm yên ngựa, vì để là cực tiểu thì hàng triệu trị riêng phải dương *cùng lúc*. Cái làm chậm là vùng phẳng quanh yên ngựa và vùng có số điều kiện tệ — tức đúng hiện tượng ở mục trước.

Ví dụ nhỏ nhất tính tay được, và nó đủ giải thích cả hiện tượng. Với $L(x) = \frac{1}{2}(x_1^2 + \varepsilon x_2^2)$, hạ gradient bước η nhân thành phần thứ nhất với $(1 - \eta)$ và thành phần thứ hai với $(1 - \eta\varepsilon)$ mỗi vòng. Ổn định buộc $\eta < 2$. Lấy $\eta = 1,8$ và $\varepsilon = 0,05$. Hướng thứ nhất dao động đổi dấu mỗi bước với hệ số $-0,8$; hướng thứ hai mỗi vòng chỉ co 9%. Tỉ số $1/\varepsilon = 20$ chính là số điều kiện. Hình 1.4 vẽ đúng quỹ đạo đó: chi phí không nằm ở chỗ “đi xuống”, nó nằm ở chỗ đi ngang thung lũng.



Hình 1.4. Hạ gradient trên hàm toàn phương có số điều kiện 20: quỹ đạo nảy qua nảy lại theo hướng dốc trong khi bò rất chậm theo hướng phẳng. Chuẩn hoá đầu vào và các optimizer thích nghi đều nhắm vào việc làm hai trục của ellipse gần bằng nhau, tức là làm hình này tròn lại.

Khác biệt thứ hai, thực dụng hơn: khi không lồi, *cực tiểu nào* ta rơi vào là một phần của kết quả. Có bằng chứng thực nghiệm rằng nghiệm ở vùng dẹt tổng quát hoá tốt hơn nghiệm ở vùng nhọn, và batch lớn có xu hướng rơi vào vùng nhọn nếu không hiệu chỉnh gì [12]. Đây là quan sát đo

được và tái lập nhiều lần, nhưng cách giải thích vẫn còn tranh cãi và định nghĩa “dệt” phụ thuộc cách tham số hoá — hãy nhớ ở mức hiện tượng, đừng nhớ ở mức định lý.

E. Điều kiện dừng nào có nghĩa

Trong tối ưu cổ điển ta dừng khi chuẩn gradient nhỏ. Trong huấn luyện mạng, tiêu chí đó gần như vô dụng: gradient mini-batch không bao giờ nhỏ vì nhiều lấy mẫu, và ngay cả gradient toàn phần nhỏ cũng không nói gì về chất lượng mô hình. Thứ thực sự được dùng là dừng theo chỉ số trên tập *validation* — tức dừng khi mục tiêu ta *quan tâm*, không phải mục tiêu ta *tối ưu*, ngừng cải thiện.

→ **tiền đề A/01** — câu hỏi “validation đã cải thiện chưa” là câu hỏi thống kê về sai số lấy mẫu, được trả lời ở mục xác suất kế tiếp

Chuyển từ “điều kiện dừng của thuật toán” sang “điều kiện dừng của phép ước lượng ngoài mẫu” là một dịch chuyển tư duy lớn. Nó cũng là cầu nối sang **A/01/03**. Câu hỏi “validation đã ngừng cải thiện chưa” là câu hỏi thống kê, không phải câu hỏi giải tích.

F. Giới hạn và trường hợp hỏng

- **Giả định khả vi bị vi phạm thường xuyên hơn ta tưởng.** *ReLU* không khả vi tại không (vô hại, tập điểm có độ đo không), nhưng các phép toán rời rạc thật thì không cứu được: bước lọc bỏ các hộp trùng nhau và chỉ giữ hộp điểm cao nhất (*non-maximum suppression*) dùng *argmax*, phép lấy mẫu cứng, chỉ số nguyên trong **lượng tử hoá** (*quantization*) đều làm gradient bằng không ở mọi nơi. Cả một mảng kỹ thuật ở C và D — *surrogate loss*, *straight-through estimator*, *Gumbel-softmax* — tồn tại chỉ để vá chỗ này, và mỗi cách vá đưa vào một thiên lệch riêng.
- **hệ quả của C** — mọi cách vá chỗ không khả vi đều sinh ra từ đúng một sự thật: gradient bằng không ở mọi nơi trên một phép toán rời rạc
- **Autograd không cứu được sai số số học.** Gradient có thể đúng công thức nhưng vẫn tràn hoặc mất chính xác ở *half precision*. Triệu chứng là loss thành NaN sau vài trăm bước. Đó là lý do tồn tại của *loss scaling* trong huấn luyện hỗn hợp độ chính xác [13]. Ở chiều ngược lại, gradient check với float32 hay báo sai giả; phải kiểm ở double.
- **Dừng suy từ bài toán lồi sang mạng sâu.** Trực giác “quỹ đạo trơn tru đi xuống lòng chảo” mô tả sai cái đang xảy ra: quỹ đạo thật đi qua vùng có độ cong đổi dấu, và learning rate lớn ở giai đoạn đầu đóng vai trò **chính quy hoá** (*regularization*) chứ không chỉ là “đi nhanh” — giảm nó quá sớm làm mất tác dụng đó.

CẠM BÃY

Loss giảm không có nghĩa mô hình đang học thứ bạn muốn. Ba trường hợp hay gặp và cách phân biệt: loss giảm vì mô hình học thiên lệch của nhãn (kiểm bằng cách so với *baseline* đoán lớp phổ biến nhất); loss train giảm mà validation đi ngang vì **rò rỉ dữ liệu** (*data leakage*) làm hai tập không độc lập (kiểm bằng cách chia theo nhóm nguồn); loss giảm đều nhưng chỉ số nghiệp vụ đứng yên vì hàm mất mát không đại diện tốt cho chỉ số đó (kiểm bằng cách vẽ tương quan giữa hai đại lượng). Luôn xem loss như một *proxy* có sai lệch; **B/09** mổ xẻ sai lệch ấy.

CẦN NHỚ

Khớp shape trước, dấu và hệ số sau. Trước khi đổ lỗi cho tính không lỗi, hãy kiểm tra số điều kiện. Và đừng bao giờ dừng huấn luyện theo chuẩn gradient — dừng theo chỉ số trên tập validation, vì đó mới là đại lượng bạn thật sự quan tâm. [T1]

G. Kết nối

Mục này nhận trực tiếp từ [A/01/01-dai-so-tuyen-tinh](#) (Hessian và phổ của nó) và chuyển tiếp sang [A/01/03-xac-suat-va-thong-tin](#), nơi câu hỏi “validation ngừng cải thiện chưa” được trả lời bằng công cụ thống kê thay vì bằng mắt. Theo chiều ngang, đây là tiền đề của hai chỗ. Thứ nhất là toàn bộ chương optimizer và lịch trình learning rate ở D. Thứ hai là phần thiết kế hàm mất mát ở [B/09](#), nơi ta thấy nhiều lựa chọn loss thật ra là lựa chọn về *hình dạng gradient* chứ không về ý nghĩa thống kê.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao chế độ ngược tổn bộ nhớ còn chế độ xuôi thì không, và điều đó dẫn tới kỹ thuật gì khi mô hình quá lớn?
2. Bạn derive được $\partial L / \partial W$ mà không cần nhớ công thức. Lập luận đó dựa trên tính chất nào, và khi nào nó không đủ để xác định duy nhất đáp án?
3. Trong không gian nhiều triệu chiều, vì sao điểm dừng gần như chắc chắn là yên ngựa chứ không phải cực tiểu địa phương?
4. Vì sao “chuẩn gradient nhỏ” là điều kiện dừng vô nghĩa khi huấn luyện với mini-batch?
5. Tăng batch size gấp bốn thì nhiều gradient giảm bao nhiêu, và vì sao quan hệ đó khiến batch rất lớn kém hiệu quả về chi phí?
6. Loss giảm ổn định suốt 50 epoch nhưng mô hình vô dụng khi triển khai — ba nguyên nhân nào giải thích được chuyện đó, và mỗi nguyên nhân phân biệt bằng phép kiểm nào?

1.3 Xác suất, độ hợp lý và lý thuyết thông tin (Probability, Likelihood and Information Theory)

PHỤ THUỘC [A/01-toan-toi-thieu/02-vi-phan-va-toi-uu](#) (điều kiện dừng theo validation — mục này cung cấp công cụ để trả lời “validation đã thật sự cải thiện chưa”).

TÓM TẮT

Mục này làm hai việc: cho thấy gần như mọi **hàm mất mát** (*loss function*) là âm log **độ hợp lý** (*likelihood*) của một giả định phân phối được chọn ngầm, và cho bạn công cụ để biết một con số đo được đáng tin đến đâu. Đọc xong bạn không cần nhớ công thức loss nào nữa, vì suy ra được từ giả định. Bạn biết ngay loss đó hỏng ở đâu: đúng chỗ giả định vỡ. Và bạn tính được trong ba mươi giây xem chênh lệch một điểm phần trăm giữa hai mô hình có đáng bàn không. Nếu chỉ nhớ một điều: *mọi loss là một giả định phân phối trá hình, và mọi con số đo được là một ước lượng có sai số* — ai bỏ qua về thứ hai sẽ dành hàng tháng để đuổi theo nhiều.

KIẾN THỨC CẦN CÓ TRƯỚC

- **biến ngẫu nhiên, kỳ vọng, phương sai** — biến ngẫu nhiên là một đại lượng mà giá trị phụ thuộc kết quả của một phép thử; kỳ vọng là giá trị trung bình dài hạn của nó; phương sai đo mức dao động quanh kỳ vọng, và căn bậc hai của phương sai là độ lệch chuẩn.

- **phân phối (distribution)** — quy luật gán xác suất cho các giá trị mà biến ngẫu nhiên có thể nhận. Chọn một phân phối là đưa ra một giả định về cách dữ liệu được sinh ra.
- **độ hợp lý (likelihood)** — với một bộ tham số cho trước, xác suất mà mô hình gán cho *dữ liệu đã quan sát được*. Lưu ý nó là hàm của tham số, không phải hàm của dữ liệu.
- **prior** — niềm tin về tham số *trước khi* nhìn dữ liệu, viết dưới dạng một phân phối. Trong thực hành, một prior tương đương với một số hạng phạt cộng vào hàm mất mát.
- **hàm mất mát (loss function)** — một con số đo mức sai của dự đoán so với nhãn; huấn luyện là đi tìm tham số làm nó nhỏ nhất.
- **softmax** — phép biến một vector số thực bất kỳ thành một bộ số dương cộng lại bằng một, để đọc như xác suất của từng lớp. “Đọc như” không có nghĩa “là”.
- **tập test và tập validation** — validation là phần dữ liệu dùng để chọn siêu tham số và quyết định khi nào dừng; test là phần chỉ được chạm vào *một lần* ở cuối để báo cáo. Dùng test như validation làm mọi con số trở nên lạc quan giả.
- **độ chính xác (accuracy)** — tỉ lệ mẫu được dự đoán đúng. Nó là trung bình của các biến chỉ báo đúng hay sai, nên bản thân nó cũng là một ước lượng có sai số.

A. Đặt vấn đề

Hai câu nói kết thúc rất nhiều tranh luận kỹ thuật một cách sai: “mô hình A đạt 90,2%, mô hình B đạt 89,1%, vậy A tốt hơn” và “loss của tôi là **entropy chéo** (*cross-entropy*) vì mọi người dùng *cross-entropy*”. Câu thứ nhất là một phát biểu thống kê không kèm sai số. Câu thứ hai là một lựa chọn mô hình xác suất được nhớ như một quy ước. Mục này chứa cả hai bằng cùng một bộ công cụ.

Học xong mục này thì làm được gì mà trước đó không làm được? Bốn việc: suy ra hàm mất mát từ giả định về nhiễu thay vì tra bảng; dự đoán trước loss sẽ hỏng ở đâu; tính khoảng tin cậy và quyết định một thí nghiệm có đủ mẫu không *trước khi* chạy; và đọc đúng hướng của **phân kỳ KL** (*Kullback–Leibler divergence*) trong một paper để biết mô hình sẽ trải mỏng hay chọn một mode. Nền lý thuyết ở [14], [15]; phần thông tin ở [16].

TRỰC GIÁC

Cross-entropy là hoá đơn tiền điện của việc mã hoá sự thật bằng bảng mã của mô hình. Mô hình tin chắc vào điều đúng thì sự kiện thật rơi vào một ký hiệu ngắn và hoá đơn nhỏ; mô hình tin chắc vào điều sai thì sự kiện thật rơi vào một ký hiệu dài kinh khủng, đúng bằng $-\log q$ của một xác suất gần không. **Entropy** là hoá đơn tối thiểu không tránh được; KL là phần phụ trội phải trả vì dùng sai bảng mã. Cách nhìn này giải thích ngay vì sao mô hình bị phạt nặng bất đối xứng khi *tự tin mà sai*, chứ không phải khi chỉ đơn giản là sai.

B. Sai số lấy mẫu: một tập test 500 mẫu đáng tin đến đâu

Độ chính xác đo trên tập test là trung bình của các biến Bernoulli, nên sai số chuẩn là $\sqrt{p(1-p)/n}$. Nói bằng lời: *đại lượng này đo mức dao động của con số nếu bạn rút một mẫu 500 phần tử khác từ cùng phân phối* — không hơn.

Tính tay. Với $n = 500$, $p = 0,90$: sai số chuẩn $= \sqrt{0,9 \cdot 0,1/500} = \sqrt{0,00018} \approx 0,0134$, tức 1,34 điểm phần trăm. Khoảng tin cậy 95% xấp xỉ $90\% \pm 2,6\%$, tức từ 87,4% tới 92,6% — rộng hơn năm điểm.

Ở đây có một tình tế quan trọng, và bỏ qua nó thì sai theo cả hai chiều. Khi hai mô hình được đánh giá trên *cùng* một tập test, chúng sai ở phần lớn cùng những mẫu khó, nên phép so sánh đúng là so sánh *ghép cặp*: cái cần đếm là số mẫu hai mô hình bất đồng, và độ bất định của hiệu số nhỏ hơn nhiều so với độ bất định của từng con số riêng lẻ. Nếu trong 500 mẫu chỉ có 20 mẫu bất

đồng và 15 nghiêng về A thì kết luận còn yếu; nếu 60 mẫu bất đồng và 45 nghiêng về A thì hiệu số một điểm có thể có ý nghĩa. Công cụ đúng là kiểm định ghép cặp (McNemar) hoặc *bootstrap* trên chính tập test, không phải hai khoảng tin cậy vẽ cạnh nhau [17].

Còn hai nguồn bất định nữa mà công thức trên *không* bắt được, và chúng thường lớn hơn:

- **Bất định do chọn tập test.** Công thức chỉ trả lời “nếu rút 500 mẫu khác từ cùng phân phối”. Dòng công trình dựng lại tập test của ImageNet theo đúng quy trình gốc cho thấy độ chính xác tụt hàng điểm [18] — tức bản thân quy trình thu thập cũng là một nguồn ngẫu nhiên.
- **Nhãn sai.** Các tập chuẩn phổ biến có tỉ lệ nhãn sai đủ để đảo thứ hạng ở phần đỉnh bảng [19]. Nó đặt một trần cứng lên mọi phép so sánh, và không phương pháp thống kê nào vượt qua được trần đó.

C. Likelihood, MLE, MAP: nguồn gốc chung của mọi hàm mất mát

Giả sử ta mô hình hoá dữ liệu bằng một phân phối có tham số θ . Ước lượng độ hợp lý cực đại (*maximum likelihood estimation, MLE*) chọn θ cực đại $\prod_i p(y_i | x_i; \theta)$, tương đương cực tiểu $-\sum_i \log p(y_i | x_i; \theta)$. Nói bằng lời: *huấn luyện chính là tìm bộ tham số làm cho dữ liệu quan sát được trở nên ít bất ngờ nhất*. Chọn dạng phân phối nào thì ra loss này — và đó là toàn bộ nội dung Bảng 1.3.

Bảng 1.3. Hàm mất mát nào cũng là một giả định phân phối. Cột cuối là cột đáng giá nhất: giả định vỡ ở đâu thì loss hỏng ở đó.

Loss	Giả định phân phối	Cho ra ước lượng gì	Hỏng khi nào
MSE	Gaussian, phương sai hằng số	Trung bình có điều kiện	Có ngoại lai; nhiễu phụ thuộc tín hiệu; mục tiêu đa phương thức thì cho ra trung bình mờ
L1	Laplace	Trung vị có điều kiện	Cần ước lượng trung bình; gradient hằng số gây dao động gần nghiệm
Cross-entropy	Categorical	Xác suất từng lớp	Nhãn sai hoặc mất cân bằng cực đoan; phạt rất nặng khi tự tin mà sai
Phạt L_2	Cực đại hậu nghiệm (<i>maximum a posteriori, MAP</i>) với prior Gaussian	Nghiệm co về 0	Khi thang đo các tham số khác nhau nhiều

Bảng trên giải thích một chuyện hay gây bối rối: vì sao hồi quy độ sâu bằng MSE cho ra biên vật thể mờ. Ở biên, độ sâu thật là *đa phương thức*: hoặc thuộc vật trước, hoặc thuộc nền. MLE dưới giả định Gaussian đơn phương thức buộc mô hình trả về trung bình của hai đáp án đúng. Kết quả là một giá trị sai nằm ở giữa. Không siêu tham số nào cứu được; phải đổi giả định phân phối, và đó chính là động cơ của các mô hình sinh có điều kiện ở C.

↪ **tiền đề B/09** — mọi lựa chọn hàm mất mát ở chương thiết kế loss bắt đầu bằng việc chọn — có ý thức hay không — một giả định phân phối

MAP thêm một prior: cực đại $p(\theta | \text{dữ liệu}) \propto p(\text{dữ liệu} | \theta) p(\theta)$, tương đương thêm một số hạng phạt vào loss. Với nền SLAM đây là chỗ nổi quen thuộc: một prior trên pose *chính* là một số

hạng phạt trong hàm chi phí, và damping của Levenberg–Marquardt hoạt động y hệt một prior Gaussian quanh điểm hiện tại — cùng một cơ chế, hai cái tên.

Cần trung thực một chỗ: *không phải mọi loss đều là âm log độ hợp lý*. **Focal loss** [20] là phép định lại trọng số theo độ khó; **làm mượt nhãn** (label smoothing) [21] cố ý làm sai lệch mục tiêu; các loss dạng biên (triplet, contrastive) tối ưu một thứ tự chứ không một mật độ. Đó là kỹ thuật, không phải hệ quả của một mô hình sinh — và chính vì thế xác suất mà mô hình xuất ra sau khi huấn luyện bằng chúng không còn được **hiệu chuẩn** (calibrated).

D. Entropy, cross-entropy, KL và sự bất đối xứng

Ba đại lượng, một quan hệ:

$$H(p) = - \sum_i p_i \log p_i, \quad H(p, q) = - \sum_i p_i \log q_i, \quad D_{\text{KL}}(p||q) = H(p, q) - H(p).$$

Vì $H(p)$ không phụ thuộc tham số mô hình, *cực tiểu cross-entropy chính là cực tiểu KL từ dữ liệu tới mô hình, và cũng chính là MLE* — ba cách nói của một việc.

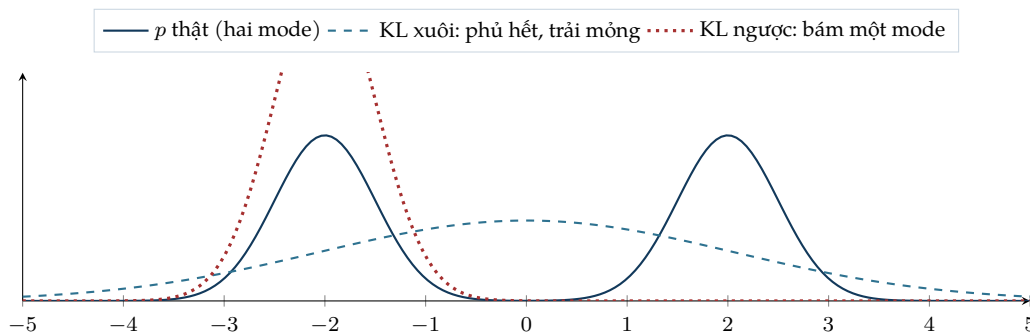
Một phép kiểm tra rẻ tiền rơi ra từ đây. Với bài toán 1000 lớp, mô hình khởi tạo hợp lý phải cho loss ban đầu xấp xỉ $\ln 1000 \approx 6,91$. Nếu là 20 thì khởi tạo hoặc tầng cuối sai; nếu là 2 thì có rò rỉ nhãn. Một dòng kiểm tra, bắt được cả một lớp bug.

KL không đối xứng, và đây không phải khiếm khuyết toán học mà là một lựa chọn về hành vi.

Ví dụ tính tay với hai phân phối Bernoulli $p = (0,5; 0,5)$ và $q = (0,9; 0,1)$:

$$D_{\text{KL}}(p||q) = 0,5 \ln \frac{0,5}{0,9} + 0,5 \ln \frac{0,5}{0,1} \approx 0,511, \quad D_{\text{KL}}(q||p) = 0,9 \ln \frac{0,9}{0,5} + 0,1 \ln \frac{0,1}{0,5} \approx 0,368.$$

Hai con số khác nhau vì hai đại lượng phạt hai kiểu sai lầm khác nhau. Hình 1.5 cho thấy hệ quả rõ hơn mọi lời giải thích: cùng một phân phối thật hai đỉnh, hai hướng KL cho ra hai nghiệm hoàn toàn khác.



Hình 1.5. Hai hướng của KL cho hai hành vi trái ngược khi mô hình quá đơn giản để khớp phân phối thật. KL xuôi $D_{\text{KL}}(p||q)$ phạt nặng chỗ p có khối lượng mà q gần bằng không, nên nghiệm bị ép phủ cả hai mode và đặt khối lượng vào vùng trống ở giữa. KL ngược $D_{\text{KL}}(q||p)$ phạt chỗ q đặt khối lượng mà p không có, nên nghiệm chọn một mode và bỏ mode kia.

Sự khác biệt ấy giải thích ba hiện tượng sẽ gặp lại:

→ **cùng nguyên lý với C** — chiều của KL được tối ưu quyết định mô hình sinh trái mông qua mọi mode hay bám lấy một mode — cùng cơ chế ở VAE, chưng cất và huấn luyện theo độ hợp lý

- **Mô hình sinh huấn luyện theo độ hợp lý sinh ra mẫu mờ.** MLE dùng KL xuôi, nên mô hình buộc phải phủ cả những vùng trống giữa các mode. Mẫu rút từ vùng ấy không giống bất kỳ mẫu thật nào.
- **Posterior collapse trong VAE** [22] — số hạng KL ngược tới prior khiến mô hình thà bỏ hẳn vài chiều tiềm ẩn còn hơn trái mông.

- **“Dark knowledge” trong chưng cất tri thức (*knowledge distillation*)** [23] — học trò khớp phân phối thầy theo chiều xuôi, nên bị buộc phải phủ cả các đuôi xác suất của những lớp *không phải* đáp án; chính các đuôi đó mang thông tin về quan hệ giữa các lớp.

E. Giới hạn và trường hợp hồng

- **Giả định độc lập và cùng phân phối gần như luôn sai trong thị giác.** Các khung hình liên tiếp của một video giống nhau; nhiều ảnh của cùng một bệnh nhân cũng vậy; nhiều mẫu chụp cùng buổi thì cùng ánh sáng. Một tập test 500 ảnh trích từ 20 video có kích thước mẫu *hiệu dụng* gần 20 hơn gần 500. Khoảng tin cậy thật vì thế rộng gấp nhiều lần. Chia tập theo nhóm (theo video, theo người, theo thiết bị) quan trọng hơn mọi tinh chỉnh mô hình.
- **Xác suất mà mạng xuất ra không phải xác suất thật.** Mạng hiện đại có xu hướng quá tự tin: trong nhóm dự đoán độ tin cậy 0,99, tỉ lệ đúng thực tế thấp hơn đáng kể, và điều này được đo lặp lại nhiều lần [24].

↔ **hệ quả của E** — mạng quá tự tin không phải lỗi cài đặt mà là hệ quả của việc tối ưu cross-entropy chứ không tối ưu độ hiệu chuẩn

Đừng dùng *softmax* làm ngưỡng ra quyết định khi chưa hiệu chuẩn; E bàn *temperature scaling*, *ensemble* và *conformal prediction* [25] như các cách vá với chi phí khác nhau.

- **Khung độ hợp lý có giới hạn khái niệm.** Nó tối ưu độ khớp phân phối, còn thứ ta quan tâm thường là một chỉ số nghiệp vụ bất đối xứng — bỏ sót một người đi bộ tệ hơn nhiều so với một báo động giả. Không prior nào biến MLE thành thứ tối ưu chỉ số đó; phải đặt vào hàm chi phí một cách tường minh, và đó là nội dung của B/09.

CẠM BẤY

“Cải thiện 0,3% trên tập test” gần như luôn là nhiều — nhưng cách sửa phổ biến, chạy nhiều seed rồi báo trung bình, *vẫn* chưa đủ nếu bạn đã dùng chính tập test đó để chọn siêu tham số. Khi tập test bị nhìn nhiều lần, nó biến thành tập validation và mọi khoảng tin cậy tính trên nó đều quá lạc quan. Đây là dạng rò rỉ tinh vi nhất, vì không có dòng code nào sai cả.

CẦN NHỚ

Trước khi hỏi “mô hình nào tốt hơn”, hãy tính $\sqrt{p(1-p)/n}$ — ba mươi giây, và nó loại bỏ phần lớn tranh luận vô ích. Trước khi chọn loss, hãy hỏi “tôi đang giả định nhiều phân phối gì” — vì loss sẽ hồng đúng chỗ giả định đó vỡ. [T1]

F. Kết nối

Mục này khép lại phần “ngôn ngữ chung” của chương, và là tiền đề của ba nhánh xa nhau. B/09 về thiết kế hàm mất mát, nơi ta cố ý lệch khỏi độ hợp lý. C về mô hình sinh, nơi hướng của KL quyết định hành vi mẫu sinh ra. Và toàn bộ E về đánh giá và độ tin cậy. Theo chiều đối lập, nó bổ khuyết cho A/01/02: mục kia nói gradient descent đi tới đâu, mục này nói làm sao biết chỗ đó có thật sự tốt hơn hay không.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao KL không đối xứng, và điều đó thay đổi hành vi cụ thể nào của một mô hình sinh?
2. Với 500 mẫu test và độ chính xác 90%, khoảng tin cậy 95% rộng bao nhiêu? Vì sao con số đó *không* trực tiếp bác bỏ được chênh lệch một điểm giữa hai mô hình đánh giá trên cùng tập test?

3. Tập test gồm 500 khung hình trích từ 20 video. Kích thước mẫu hiệu dụng gần con số nào hơn, và hệ quả với cách chia dữ liệu là gì?
4. Vì sao hồi quy độ sâu bằng MSE cho ra biên mờ, và vì sao đối optimizer không sửa được?
5. Loss ban đầu của bài 1000 lớp phải xấp xỉ bao nhiêu, và nếu nó bằng 2 thì bạn nghi ngờ điều gì?
6. Focal loss và label smoothing không phải âm log độ hợp lý của mô hình nào. Điều đó gây hệ quả gì cho việc đọc đầu ra softmax?

1.4 Hình học biến đổi và xử lý tín hiệu (Transformation Geometry and Signal Processing)

PHỤ THUỘC [A/01-toan-toi-thieu/01-dai-so-tuyen-tinh](#) (SVD — dùng để chiếu một ma trận nhiều về $SO(3)$). Mục này là tiền đề bắt buộc của cả chương [A/02-vat-ly-tao-anh](#).

TÓM TẮT

Mục này gộp hai chủ đề vì chúng chia nhau một vai trò: cả hai mô tả cấu trúc mà dữ liệu ảnh *đã có sẵn* trước khi mô hình nhìn thấy nó. Đọc xong bạn biết vì sao một mạng dự đoán phép quay bằng bốn số khó huấn luyện hơn bằng sáu số. Bạn biết vì sao CNN không thật sự bất biến tịnh tiến. Và bạn biết vì sao một dòng resize sai trong dataloader làm mất vài điểm độ chính xác, mà không sinh ra exception nào. Nếu chỉ nhớ một điều, hãy nhớ hai câu hỏi phòng vệ. *Trước mọi lần giảm độ phân giải: đã lọc thông thấp chưa? Trước mọi lần cho mạng xuất ra phép quay: biểu diễn này có liên tục không?* Hai câu đó bắt được phần lớn lỗi im lặng ở khâu hình học và tín hiệu.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tư thế (pose) và $SE(3)$** — tư thế là vị trí cộng hướng của một vật cứng trong không gian; tập mọi tư thế tạo thành nhóm $SE(3)$, còn tập mọi phép quay thuần túy là $SO(3)$, biểu diễn bằng ma trận 3×3 trực giao có định thức bằng một.
- **đa tạp (manifold)** — một tập hợp mà quanh mỗi điểm nó “trông giống” một không gian phẳng nhiều chiều, nhưng toàn cục thì cong. Hệ quả thực hành: không cộng trực tiếp hai phân tử được, phải cộng ở không gian tiếp tuyến.
- **convolution và kernel** — kernel là một cửa sổ nhỏ chứa các trọng số; convolution là trượt cửa sổ đó khắp ảnh và tại mỗi vị trí lấy tổng có trọng số của vùng lân cận. Đây là phép toán nền của mạng CNN.
- **CNN** — mạng nơ-ron mà phần lớn các tầng là convolution; nó được thiết kế dựa trên giả định rằng cấu trúc ảnh mang tính cục bộ và không đổi khi dịch chuyển.
- **stride và pooling** — stride là bước nhảy khi trượt kernel — stride 2 nghĩa là chỉ lấy một trên hai vị trí, tức giảm độ phân giải một nửa. Pooling cũng giảm độ phân giải bằng cách lấy giá trị lớn nhất hoặc trung bình của một vùng.
- **tần số không gian** — đo bằng “chu kỳ trên pixel”: một hoa văn 0,25 chu kỳ/pixel lặp lại sau mỗi bốn pixel. Tần số cao ứng với chi tiết mảnh, tần số thấp ứng với cấu trúc lớn.
- **biến đổi Fourier** — cách viết lại một tín hiệu thành tổng các sóng sin ở nhiều tần số khác nhau, để nhìn xem tín hiệu chứa những tần số nào và mạnh yếu ra sao.
- **nội suy (interpolation)** — khi thay đổi kích thước hoặc xoay ảnh, giá trị pixel mới phải được ước lượng từ các pixel cũ lân cận; cách ước lượng đó gọi là nội suy, và mỗi lần nội suy là một lần làm mờ thêm.

A. Đặt vấn đề

Bỏ qua cấu trúc có sẵn của dữ liệu ảnh thì mạng phải học lại nó từ đầu, tốn nhãn và tốn tham số; hiểu sai cấu trúc ấy thì bạn tự tay phá dữ liệu ở khâu tiền xử lý và không bao giờ tìm ra nguyên

nhân. Với người đã làm SLAM, nửa đầu mục này là ôn tập nhanh có đối trọng tâm: câu hỏi không còn là “ $SE(3)$ hoạt động thế nào” mà là “vì sao mạng khó xuất ra một phép quay”. Nửa sau thì hầu như luôn là chỗ hỏng thật, kể cả với người có kinh nghiệm.

Học xong mục này thì làm được gì mà trước đó không làm được? Chọn đúng biểu diễn đầu ra cho mọi bài toán ước lượng tư thế bằng mạng; kiểm tra một pipeline tiền xử lý và chỉ ra chỗ đang sinh hoa văn giả; và giải thích bằng ngôn ngữ tần số vì sao một mô hình huấn luyện ở 224×224 sụt điểm khi đưa vào ảnh độ phân giải khác.

TRỰC GIÁC

Aliasing không phải chuyện làm ảnh xấu đi; nó là chuyện làm ảnh *nói dối*. Nan hoa bánh xe quay ngược trong phim là ví dụ thuần khiết: máy quay không làm mờ bánh xe, nó tạo ra một chuyển động không hề tồn tại. Khi bạn resize ảnh bằng *nearest-neighbor* rồi đưa vào mạng, mạng nhận được những hoa văn cũng không hề có trên vật thật. Nó học các hoa văn ấy một cách hoàn toàn nghiêm túc. Rồi camera triển khai có độ phân giải khác, hoa văn giả đổi kiểu, và mô hình sập — mà không ai chạm vào trọng số nào.

B. Hình học: ba thao tác là đủ

Toạ độ đồng nhất (*homogeneous coordinates*) tồn tại vì một lý do thực dụng: nó biến phép tịnh tiến, vốn là phép cộng, thành phép nhân ma trận. Nhờ đó cả chuỗi biến đổi gộp được thành một tích duy nhất. Phép chiếu phối cảnh cũng viết được ở dạng tuyến tính, rồi chia cho thành phần cuối. Toàn bộ hình học nhiều view ở [A/02/03](#) xây trên tiện lợi đó.

Nhóm quay $SO(3)$ và nhóm chuyển động cứng $SE(3)$ là các **đa tạp** (*manifold*) chứ không phải không gian vector: cộng hai ma trận quay không cho ra ma trận quay. Hệ quả thực hành — và là toàn bộ phần **đại số Lie** (*Lie algebra*) bạn cần — gói trong ba thao tác:

1. $\exp(\delta)$ đưa một vector ba chiều (trục nhân góc) thành một ma trận quay;
2. $\log(R)$ làm ngược lại, cho ra “sai lệch quay” ở dạng vector cộng được;
3. cập nhật $R \leftarrow R \exp(\delta)$ thay cho $R \leftarrow R + \delta$, tức là *cộng ở không gian tiếp tuyến rồi mới quay về đa tạp*.

Nếu bạn đã viết một solver **hiệu chỉnh chùm tia** (*bundle adjustment*) thì bạn đã cài đúng ba thao tác này; điều đáng nói là chúng không biến mất khi chuyển sang mạng nơ-ron, chỉ đổi chỗ. Nền lý thuyết ở mức kỹ sư: [\[26\]](#), [\[27\]](#).

Chỗ chúng xuất hiện lại là bài toán cho mạng *xuất ra* một phép quay, và đây là một trong ít chỗ mà lý thuyết cho lời khuyên rất rõ. Không có phép tham số hoá liên tục nào của $SO(3)$ bằng ba hoặc bốn số mà không có điểm kỳ dị hoặc điểm gián đoạn; mà một mạng liên tục theo đầu vào thì không thể khớp một hàm mục tiêu gián đoạn, nên nó cứ xử tệ đúng quanh vùng gián đoạn. Bảng [1.4](#) so bốn lựa chọn.

Bảng 1.4. Bốn cách cho mạng xuất ra một phép quay. Cột cuối quyết định lựa chọn.

Biểu diễn	Số đầu ra	Cơ chế đưa về $SO(3)$	Hỏng khi nào
Góc Euler	3	Nhân ba phép quay trực	<i>Gimbal lock</i> ; gián đoạn ở biên $\pm\pi$; trung bình vô nghĩa
Quaternion	4	Chuẩn hoá về chuẩn 1	Phủ hai lần (q và $-q$ cùng một phép quay) nên hàm mục tiêu gián đoạn; phải xử lý dấu
6D (hai vector)	6	Gram–Schmidt trực giao hoá	Không có điểm gián đoạn — lựa chọn mặc định hiện nay; chỉ tốn thêm vài phép tính
Ma trận 9 số	9	Chiếu về $SO(3)$ bằng SVD	Cũng liên tục, nhưng SVD trong vòng huấn luyện đắt hơn và đạo hàm phức tạp hơn

↔ **trường hợp riêng của C** — mọi bài ước lượng tư thế bằng mạng là một trường hợp riêng của việc chọn biểu diễn liên tục cho đầu ra trên đa tạp

Cách sáu số cho mạng xuất ra hai vector ba chiều rồi trực giao hoá. Nó được phát biểu rõ trong dòng công trình về “biểu diễn quay liên tục” của Zhou và cộng sự. Cách chín số rồi chiếu bằng SVD tương đương về tinh thần, và dùng đúng kết quả xấp xỉ hạng thấp ở [A/01/01](#).

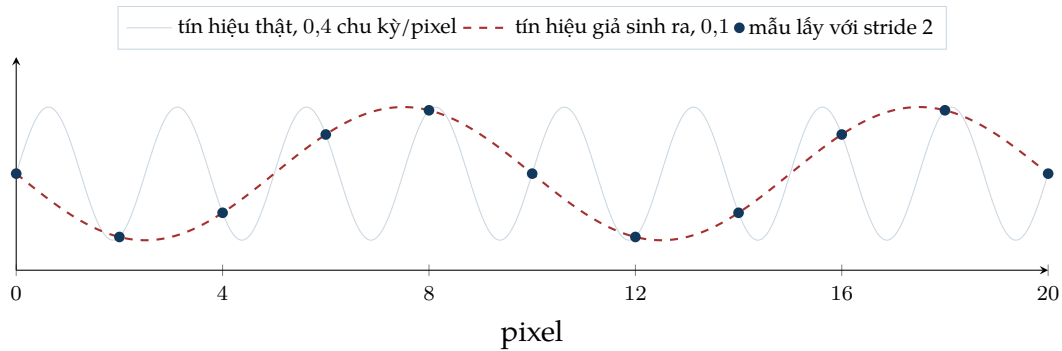
Một chi tiết dễ nhầm và tốn thời gian debug: *lấy trung bình các phép quay*. Trung bình cộng của các ma trận quay không phải ma trận quay; trung bình quaternion phụ thuộc dấu bạn chọn; trung bình góc Euler vô nghĩa qua chỗ chuyển $\pm\pi$. Cách đúng là trung bình trên không gian tiếp tuyến quanh một điểm neo rồi ánh xạ về, hoặc chiếu trung bình ma trận về $SO(3)$ bằng SVD. Sai lầm này xuất hiện lại nguyên vẹn khi người ta gộp dự đoán của nhiều mô hình hay nhiều *test-time augmentation*.

C. Tín hiệu: mọi lỗi im lặng đều là lỗi lấy mẫu

Convolution là phép toán duy nhất vừa tuyến tính vừa **đẳng biến tịnh tiến** (*translation equivariance*): dịch ảnh rồi lọc cũng bằng lọc rồi dịch. Đó là toàn bộ lý do CNN tồn tại. Ta *cài sẵn* giả định “một cạnh vẫn là một cạnh khi dịch sang phải mười pixel”, thay vì bắt mạng học nó từ dữ liệu. Định lý convolution nói convolution trong miền không gian là phép nhân từng điểm trong miền Fourier; nói bằng lời, *mỗi kernel là một bộ lọc tần số* — kernel làm mịn là thông thấp, kernel Sobel là thông cao. Từ đó cả kiến trúc đọc được gọn: một CNN sâu là chuỗi lọc xen kẽ các bước giảm mẫu, và mỗi bước giảm mẫu vớt bỏ một nửa dải tần.

Chính chỗ vớt bỏ ấy là nơi mọi thứ hỏng. **Định lý lấy mẫu** (*sampling theorem*) nói tần số lấy mẫu phải lớn hơn hai lần tần số cao nhất có trong tín hiệu; nếu không, thành phần tần số cao không biến mất mà *gập* xuống thành tần số thấp giả.

Ví dụ tính tay. Hoa văn 0,4 chu kỳ trên pixel, lấy mẫu cách hai pixel (stride 2): tần số lấy mẫu 0,5, tần số Nyquist 0,25; thành phần 0,4 vượt ngưỡng nên gập về $|0,4 - 0,5| = 0,1$ chu kỳ trên pixel — một hoa văn thô gấp bốn lần, không hề tồn tại trên vật thật. Hình 1.6 vẽ đúng phép tính đó.



Hình 1.6. Aliasing không làm mất thông tin một cách trung thực, nó *bịa* thông tin. Các điểm lấy mẫu cách hai pixel nằm chính xác trên một hình sin tần số thấp hơn bốn lần; mọi tầng phía sau đều không có cách nào biết hoa văn thô ấy là giả.

→ **tiền đề A/02** — khử khảm và giảm mẫu màu 4:2:0 trong chuỗi ISP là đúng bài toán lấy mẫu này, chỉ đổi tên

Từ đó rút ra quy tắc bất di bất dịch của mọi phép giảm mẫu: **lọc thông thấp trước, rồi mới lấy mẫu thưa**. Hệ quả ở hai chỗ:

- **Trong dataloader** — thu nhỏ ảnh phải dùng nội suy vùng (INTER_AREA) hoặc làm mờ Gaussian trước; *nearest-neighbor* gần như luôn sai khi thu nhỏ. Đây là một dòng code, và nó đáng vài điểm độ chính xác.
- **Trong kiến trúc** — tầng stride-2 và *max-pooling* lấy mẫu thưa mà không lọc trước, nên CNN *không* bất biến dịch chuyển như người ta hay nói: dịch ảnh một pixel có thể đổi hẳn dự đoán. Dòng công trình “làm cho CNN bất biến dịch trở lại” của Zhang chèn một bộ lọc mờ ngay trước mỗi bước giảm mẫu. Cách đó khôi phục phần lớn tính bất biến. Kết quả đã được đo và tái lập, dù mức lợi ích tùy bài toán.

Góc nhìn Fourier còn giải thích hai hiện tượng nữa. Thứ nhất là *spectral bias*. Mạng khớp thành phần tần số thấp trước, tần số cao sau, nên chi tiết mảnh xuất hiện muộn trong huấn luyện. Khi mục tiêu vốn nhiều tần số cao — chẳng hạn hàm mật độ của một cảnh 3D trong *NeRF* — người ta phải mã hoá đầu vào bằng đặc trưng Fourier để mạng học nổi [28].

→ **tiền đề C** — đẳng biến tịnh tiến là thiên lệch quy nạp mà cả dòng CNN dựa vào, và việc nó không đúng hoàn toàn là một trong các động cơ dẫn tới *attention*

Thứ hai là chuyện thang đo: một mô hình huấn luyện ở 224×224 thấy các cạnh ở một dải tần nhất định; đưa vào ảnh 4K rồi crop, cùng vật thể xuất hiện ở dải tần khác hẳn, và mô hình sụt điểm dù “vẫn là ảnh của cùng cái xe”.

▮ MẠCH 3 — TIỀN XỬ LÝ ẢNH, VÀ CÁI GIÁ CỦA MỖI BƯỚC

- 1 Vấn đề** ảnh vào có kích thước bất kỳ, mạng cần kích thước cố định.

Giải pháp resize.

Giả định nội suy giữ nguyên nội dung ngữ nghĩa.

Giá phải trả đổi phổ tần số của ảnh; thu nhỏ mà không lọc trước thì sinh hoa văn giả.
- 2 Vấn đề** resize làm méo tỉ lệ khung hình.

Giải pháp *letterbox* — giữ tỉ lệ, đệm phần thiếu.

Giá phải trả đưa vào vùng đệm nhân tạo mà mạng có thể học như tín hiệu, và mọi toạ độ hộp phải được biến đổi ngược đúng cách — một chỗ sinh bug rất phổ biến.
- 3 Vấn đề** muốn dữ liệu phong phú hơn nên xoay, lật, cắt.

Giả định	phép biến đổi không đổi nhãn.
Vấn đề mới	giả định đó sai cho biển báo có chữ, cho ảnh y tế có tính trái-phải, và cho mọi bài toán phải dự đoán hướng; ngoài ra mỗi lần xoay là một lần nội suy, tức lại đổi phổ.
4 Vấn đề	muốn augmentation mạnh hơn nên chồng nhiều phép liên tiếp.
Giả định	các phép nội suy chồng nhau không tích lũy hại.
Giá phải trả	sai — mỗi lần nội suy làm mờ thêm.
Giải pháp	gộp các phép biến đổi thành một ma trận rồi lấy mẫu <i>một lần</i> duy nhất, đúng như cách bạn gộp chuỗi $SE(3)$ thành một phép biến đổi trước khi warp.

D. Giới hạn và trường hợp hỏng

- **Bức tranh Fourier giả định tuyến tính và bất biến dịch**, mà mạng thật có phi tuyến sau mỗi tầng. “Mỗi tầng là một bộ lọc tần số” là trực giác hướng dẫn tốt, không phải mô tả chính xác — đừng suy diễn định lượng từ nó.
- **Hình học ở trên giả định phép biến đổi cứng và camera lý tưởng**. *Rolling shutter* phá giả định “một ảnh ứng với một pose” vì mỗi hàng pixel chụp ở một thời điểm khác; ống kính có méo làm phép biến đổi giữa hai ảnh không còn là homography ngay cả với mặt phẳng; vật thể không cứng thì $SE(3)$ không mô tả nổi. Mỗi giả định vỡ ở một chỗ và đòi một cách vá khác nhau — chủ đề của [A/02/03](#).

CẦN NHỚ

Hai câu hỏi phòng vệ, hỏi mỗi lần: *trước khi giảm độ phân giải — đã lọc thông thấp chưa?* và *trước khi cho mạng xuất ra phép quay — biểu diễn này có liên tục không?* Chúng rẻ như một dòng code và bắt được phần lớn lỗi im lặng ở khâu hình học và tín hiệu. [T1]

E. Kết nối

Mục này là nền trực tiếp của cả chương [A/02-vat-ly-tao-anh](#): phần lấy mẫu giải thích vì sao **khử khảm** (*demosaiçing*) là một bài toán khôi phục chứ không phải nội suy tầm thường, và phần hình học là toàn bộ ngôn ngữ của [A/02/03](#). Theo chiều ngang, tính bất biến dịch chuyển là giả định thiết kế của dòng CNN ở C, và việc nó *không* đúng hoàn toàn là một trong các động cơ dẫn tới *attention*; còn phần biểu diễn quay quay lại ở mọi bài toán ước lượng tư thế bằng mạng.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao không tồn tại cách cho mạng xuất ra phép quay bằng bốn số mà không có điểm gián đoạn, và cách sáu số sửa được gì?
2. Hoa văn 0,45 chu kỳ/pixel bị lấy mẫu với stride 2 sẽ hiện ra ở tần số nào? Tính tay.
3. Vì sao CNN không thật sự bất biến dịch chuyển, dù convolution thì có? Chỗ phá vỡ nằm ở tầng nào?
4. Bạn chồng năm phép augmentation hình học liên tiếp, mỗi phép một lần nội suy. Hại gì xảy ra và sửa thế nào?
5. Vì sao lấy trung bình cộng các ma trận quay là sai, và cách đúng dựa trên kết quả nào ở mục đại số tuyến tính?
6. Một mô hình huấn luyện ở 224×224 sụ điểm khi chạy trên ảnh 4K đã crop về 224×224 — giải thích bằng ngôn ngữ tần số thì chuyển gì đã xảy ra?

1.5 Thực hành

Phần lý thuyết ở trên chỉ trở thành kiến thức dùng được sau khi bạn tự tay chạm vào một lần. Khối dưới đây được thiết kế để tốn ít thời gian nhất mà chạm được vào nhiều điểm nhất; hãy làm mini project ngay cả khi thấy đã hiểu, vì câu hỏi kết thúc nó là câu hỏi mà lý thuyết một mình không trả lời được.

Repo và tài nguyên. Cho phần lý thuyết, *Mathematics for Machine Learning* [2] có PDF miễn phí và đúng liều lượng cho mục tiêu ở đây. Khi cần đổi chiều một đạo hàm ma trận, matrixcalculus.org trả lời nhanh hơn tra sách; cần chắc chắn hơn nữa thì `sympy` cho phép derive tay có kiểm chứng ký hiệu. Đạo hàm tự viết phải kiểm bằng `torch.autograd.gradcheck` trên đầu vào kiểu double — kiểm bằng float32 hay báo sai giả. Ghi chú lan truyền ngược của CS231n vẫn là bản giải thích đồ thị tính toán tốt nhất ở mức nhập môn, dù danh mục kiến trúc trong cùng khoá học thì nên đọc như lịch sử tư duy. Nhóm tài nguyên này ít bị bán rã — sách toán và công cụ ký hiệu bền hơn nhiều so với tên mô hình dẫn đầu bảng, vốn sẽ cũ trong 6–12 tháng kể từ mốc chốt tháng 9/2026.

Câu hỏi kiểm tra hiểu. Ba câu, mỗi câu chạm một mục khác nhau:

- Vì sao KL divergence không đối xứng, và sự bất đối xứng đó khiến một mô hình sinh trái mỏng hay chọn một mode?
- Số điều kiện lớn của một ma trận trọng số nói gì về độ ổn định huấn luyện — và nó *không* nói gì?
- Với tập test 500 mẫu và độ chính xác 90%, khoảng tin cậy 95% rộng bao nhiêu, và chênh lệch một điểm phần trăm giữa hai mô hình có nghĩa không? Câu trả lời khác nhau ra sao giữa hai phép đo độc lập và một phép so sánh ghép cặp trên cùng tập test?

Bài tập phụ. Cài PCA và *whitening* bằng NumPy thuần trên CIFAR-10 — tự tính ma trận hiệp phương sai, tự phân rã, hiển thị 16 vector riêng đầu tiên dưới dạng ảnh — rồi trả lời: vì sao whitening gần như không được dùng trong pipeline thị giác hiện đại? Ba lý do đáng tự tìm trước khi đọc gợi ý: chuẩn hoá theo tầng bên trong mạng đã làm một việc tương đương nhưng thích nghi hơn; whitening khuếch đại đúng những hướng phương sai nhỏ vốn chủ yếu là nhiễu cảm biến; và thống kê whitening tính trên tập huấn luyện trở thành một cái bẫy dịch chuyển miền khi triển khai trên camera khác. Nửa ngày, và nó xa SLAM — bài chính của chương nằm ở khối mini project bên dưới.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Phép chiếu vuông góc	Phép hạ một vector xuống không gian con gần nó nhất, sao cho phần dư vuông góc với không gian đó. Mọi bài bình phương tối thiểu <i>là</i> một phép chiếu; phương trình chuẩn tắc chỉ là cách viết lại điều kiện vuông góc.
•• SVD	Phân rã giá trị kỳ dị: cách viết một ma trận bất kỳ thành xoay, kéo dãn theo vài trục, rồi xoay lần nữa. Dùng để đọc hạng, để xấp xỉ hạng thấp, và để ép một ma trận nhiễu về $SO(3)$.
••• Số điều kiện	Tỉ số giữa trị kỳ dị lớn nhất và nhỏ nhất. Nó là hệ số khuếch đại sai số của <i>bài toán</i> , nên đổi solver hay đổi optimizer không cứu được. Nó vô nghĩa khi so giữa hai cách chọn đơn vị đo khác nhau.
•• Xấp xỉ hạng thấp	Giữ lại k trị kỳ dị lớn nhất để có ma trận hạng k gần bản gốc nhất. Đứng sau PCA, sau phép chiếu về $SO(3)$, và sau LoRA.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
• Hạng hiệu dụng	Entropy của phổ trị kỳ dị đã chuẩn hoá. Cần nó vì với số dấu phẩy động hầu như không trị kỳ dị nào bằng đúng không, nên đếm số trị khác không là vô nghĩa.
•• Vì sao lập $A^T A$ làm mất chữ số có nghĩa?	Vì trị kỳ dị bị bình phương, nên số điều kiện cũng bình phương. Bạn mất khoảng một nửa số chữ số; QR hoặc SVD giải thẳng trên A thì không.
••• Lan truyền ngược	Thuật toán tính gradient bằng cách đi ngược đồ thị phép tính đúng một lượt. Nó rẻ vì mỗi lượt trả về đạo hàm theo <i>mọi</i> tham số, nhưng phải giữ lại giá trị trung gian của lượt xuôi nên tốn bộ nhớ.
••• Lan truyền ngược so với lượt ngược	Lan truyền ngược là <i>thuật toán</i> ; lượt ngược là <i>một lần chạy</i> thuật toán đó trên một batch. Hai từ này hay bị dùng lẫn.
••• Quy tắc khớp shape	Gradient của một đại lượng phải có đúng shape của chính nó, nên thường chỉ còn một cách ghép hợp lệ. Khớp shape trước, dấu và hệ số sau; phần còn lại kiểm bằng gradcheck ở kiểu double.
•• Điểm yên ngựa	Điểm có gradient bằng không nhưng độ cong âm theo ít nhất một hướng. Trong không gian nhiều triệu chiều, gần như mọi điểm dừng đều là yên ngựa chứ không phải cực tiểu địa phương.
•• Vì sao chuẩn hoá và Adam làm huấn luyện nhanh hơn?	Cả hai kéo các hướng về cùng thang đo, tức hạ số điều kiện một cách gián tiếp. Khi các trục gần bằng nhau, một learning rate chung mới vừa cho mọi hướng.
•• Loss thành NaN sau vài trăm bước — nguyên nhân?	Learning rate quá lớn, hoặc tràn số ở độ chính xác 16 bit. Phân biệt: hạ learning rate mười lần mà vẫn NaN thì là tràn số, hãy bật loss scaling hoặc đổi sang bf16.
••• Độ hợp lý	Xác suất mà mô hình gán cho dữ liệu đã quan sát, xem như hàm của tham số. Cực đại nó tương đương cực tiểu âm log của nó — và đó chính là gần hết các hàm mất mát đang dùng.
••• Sai số chuẩn	Với độ chính xác đo trên n mẫu, nó bằng $\sqrt{p(1-p)/n}$. Với $n = 500$ và $p = 0,9$ thì bằng 1,34 điểm, nên khoảng tin cậy 95% rộng hơn năm điểm. Nó phình to khi các mẫu không độc lập.
•• Phân kỳ KL	Chi phí phụ trội khi mã hoá dữ liệu của phân phối này bằng bảng mã của phân phối kia. Không đối xứng: chiều xuôi ép mô hình phủ mọi mode, chiều ngược ép nó bám lấy một mode.
•• Vì sao hồi quy độ sâu bằng MSE cho ra biên mờ?	Vì ở biên, độ sâu thật là đa phương thức, còn MSE giả định Gaussian đơn phương thức. Mô hình buộc phải trả về trung bình của hai đáp án đúng, tức một giá trị sai ở giữa.
• Loss ban đầu của bài 1000 lớp bằng 2 — nghĩa là gì?	Nghi có rò rỉ nhãn. Khởi tạo hợp lý phải cho khoảng $\ln 1000 \approx 6,9$; nếu ngược lại nó bằng 20 thì lỗi nằm ở khởi tạo hoặc ở tầng cuối.
••• Aliasing	Hiện tượng thành phần tần số cao <i>gấp</i> xuống thành tần số thấp giả khi lấy mẫu quá thưa. Nó không làm mất thông tin một cách trung thực mà bịa thêm cấu trúc không có thật.
•• Đẳng biến tịnh tiến	Tính chất dịch ảnh rồi lọc cũng bằng lọc rồi dịch. Convolution có tính chất này, nhưng bước stride và pooling phá nó, nên CNN không thật sự bất biến với dịch chuyển.
•• Vì sao không cho mạng xuất ra quaternion?	Vì q và $-q$ là cùng một phép quay, nên hàm mục tiêu gián đoạn, mà một mạng liên tục thì không khớp nổi hàm gián đoạn. Cách sáu số rồi trực giao hoá thì liên tục.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
PyTorch, JAX	Toàn bộ <i>autograd</i> là hiện thực của vi phân chế độ ngược trên đồ thị tính toán — đúng mạch ở mục 2	Hạ tầng, ổn định từ 2017; ít bán rã nhất trong bảng này
Ceres, g2o, GTSAM	Bình phương tối thiểu phi tuyến với phân rã thừa và damping — phép chiếu cộng điều hoà số học của mục 1	Hạ tầng SLAM; chính là chỗ nói với nền sẵn có của bạn
LoRA và họ tinh chỉnh hiệu quả tham số	Giả thiết ma trận cập nhật trọng số có hạng thấp, tức định lý xấp xỉ hạng thấp	Cách tinh chỉnh mặc định cho mô hình lớn tính tới 9/2026
AdamW	Tách weight decay khỏi bước thích nghi — hệ quả trực tiếp của việc Adam đổi thang đo từng toạ độ	Optimizer mặc định của phần lớn mô hình lớn hiện nay
Đặc trưng Fourier trong NeRF và họ biểu diễn ngầm	Mã hoá đầu vào bằng nhiều tần số, vì mạng vốn khớp tần số thấp trước	Ý tưởng còn sống trong các biến thể dựng cảnh gần đây

MINI PROJECT SLAM · Đo số điều kiện của bài toán tư thế trên hai chuỗi EuRoC · vài giờ tới một ngày

Mục tiêu	biến ba khái niệm trừu tượng nhất của chương — phép chiếu, SVD, số điều kiện — thành ba con số đọc được trên chính hệ SLAM bạn đang chạy. Câu cần trả lời: chuỗi nào khó hơn, và khó vì <i>hình học chuyển động</i> chứ không vì thuật toán.
Dữ liệu	hai chuỗi EuRoC có đặc tính chuyển động khác hẳn nhau — một chuỗi tịnh tiến rộng và một chuỗi có nhiều đoạn xoay gần tại chỗ — cùng chân trị tư thế đi kèm tập dữ liệu. Không cần dữ liệu tự quay.
Các bước	(1) với mỗi <i>keyframe</i> , lấy chính ma trận Jacobian mà solver của bạn đã dựng cho bài tối ưu tư thế, hoặc dựng lại bằng cách xếp các hàng ràng buộc sai số tái chiếu của những điểm đang bám được. (2) Cân lại thang đo hai khối biến trước khi so sánh — tịnh tiến tính bằng mét, quay tính bằng radian — vì nếu không thì số điều kiện chỉ phản ánh lựa chọn đơn vị. (3) Tính phổ trị kỳ dị bằng SVD và ghi $\kappa = \sigma_{\max}/\sigma_{\min}$ theo thời gian cho cả hai chuỗi. (4) Ở mỗi chỗ phải chuẩn hoá một ma trận quay ước lượng, hãy làm bằng SVD: đặt cả ba trị kỳ dị bằng một. Ghi lại độ lệch trục giao <i>trước</i> khi chiếu, như một chỉ báo sức khoẻ số học. (5) Tính <i>RPE</i> bằng <i>evo</i> trên cửa sổ trượt để có một đường sai số cùng trục thời gian.
Xong khi	bạn có một hình duy nhất, vẽ hai đường κ theo thời gian trên cùng trục. Trên đó bạn chỉ ra các đoạn κ vọt lên, và đối chiếu chúng với đoạn chuyển động gần thuần quay hoặc điểm gần đồng phẳng. Cuối cùng bạn nêu một ngưỡng κ mà vượt qua thì RPE của sổ trượt tăng rõ trên cả hai chuỗi.
Nối với	A/01/01 cho SVD và số điều kiện; A/01/04 cho phép chiếu về $SO(3)$; A/02/03 cho lý do hình học khiến cấu hình gần thuần quay là suy biến. Đường κ theo thời gian sinh ra ở đây được dùng lại làm biến giải thích trong mini project của chương 2.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- Vì chi phí nằm ở *thứ tự* nhân các Jacobian, không ở quy tắc chuỗi. Nhân ngược từ phía đầu ra vô hướng cho mọi đạo hàm trong một lượt; nhân theo chiều xuôi thì tốn một lượt cho mỗi tham số.
- Vì số điều kiện đo tỉ số giữa trục dài nhất và trục ngắn nhất của bài toán *sau khi đã chọn đơn vị*; đổi đơn vị chỉ co giãn các trục chứ không đổi lượng thông tin mà dữ liệu ràng buộc. Nên chỉ so sánh được khi cách tham số hoá giữ nguyên.
- Vì đặt hai khoảng tin cậy độc lập cạnh nhau là phép so sánh sai. Trên cùng một tập test, hai mô hình sai ở phần lớn cùng những mẫu khó, nên chỉ các mẫu *bất đồng* mới mang thông tin — và kiểm định ghép cặp trên chúng có độ bất định nhỏ hơn nhiều.

- A4. Vì không tồn tại phép tham số hoá liên tục của $SO(3)$ bằng ba hay bốn số: quaternion phủ hai lần nên hàm mục tiêu gián đoạn, mà một mạng liên tục theo đầu vào thì không khớp nổi hàm gián đoạn. Sáu số rồi trực giao hoá thì liên tục.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn phải tìm chỗ thấp nhất của một thung lũng, giữa sương mù dày, và chỉ nhìn được mặt đất ngay dưới chân. Cách duy nhất là sờ xem phía nào dốc xuống rồi bước một bước về phía đó. Chương này là ba câu hỏi nảy ra từ đúng tình huống ấy.

Thung lũng này hình gì? Nhiều thung lũng thật ra là một khe rất hẹp và rất dài: đi ngang thì dốc đứng, đi dọc thì gần như bằng phẳng. Không biết điều đó thì bạn cứ nảy qua nảy lại giữa hai vách mà chẳng tiến xuống bao nhiêu. Có một phép tính cho biết khe hẹp gấp bao nhiêu lần chiều dài của nó, và con số ấy dự báo trước bạn sẽ phải chờ lâu cỡ nào.

Bước bao nhiêu là vừa? Bước dài thì văng qua vách bên kia, bước ngắn thì cả đời không tới đáy. Làm sao biết mình đã xuống thật? Cái đồng hồ đo độ cao trong tay có sai số. Nếu nó sai chừng hai mét mà bạn khoe vừa xuống thêm một mét, thì bạn chưa khoe được gì.

Cái giá: mọi mẹo trong chương chỉ làm khe rộng ra một chút cho dễ đi, chứ không ai vén được sương mù.

Chỗ không rút gọn được: vì sao dừng ở một cái đáy rộng lại cho kết quả bền hơn dừng ở đáy nhọn thì tới nay chưa có lời giải thích vừa gọn vừa đúng — và ngay cả chữ “rộng” cũng đổi nghĩa khi ta đổi cách viết các con số mô tả mô hình.

Một câu: *Toán ở chương này chỉ để trả lời ba câu: thung lũng hẹp theo hướng nào, bước bao nhiêu thì an toàn, và khi nào cái đồng hồ đo mới thật sự nói rằng bạn đã xuống thấp hơn.*

1.6 Kết nối

Chương này là tiền đề kỹ thuật của cả cây, nhưng nó nổi ra ngoài theo ba hướng rất khác nhau. Hướng thứ nhất đi sang [A/02-vat-ly-tao-anh](#): phần lấy mẫu và phần hình học của mục 4 được dùng gần như liên tục ở đó, nên đây là nhánh nên đọc tiếp ngay. Hướng thứ hai đi sang Phần B, nơi các đại lượng ở đây đổi vai. Số điều kiện và bộ nhớ kích hoạt không còn là tính chất toán học; chúng trở thành *ràng buộc thiết kế*, đứng cạnh ràng buộc về nhân và về độ trễ. Hướng thứ ba đi sang E: toàn bộ mục 3 là bộ công cụ mà chương đánh giá dùng hằng ngày, và nếu bỏ qua nó thì mọi bảng kết quả về sau đều đọc sai.

Tổng kết chương

Chương này đã dựng bốn công cụ và, quan trọng hơn, đã gắn mỗi công cụ vào một câu hỏi cụ thể sẽ quay lại. Không gian con và phép chiếu cho ta cách đọc mọi bài bình phương tối thiểu; phổ và số điều kiện cho ta ngôn ngữ chung giữa “bài toán suy biến” trong ước lượng trạng thái và “landscape khó tối ưu” trong huấn luyện. Bộ máy vi phân cho ta quy tắc khớp shape trước rồi mới lo dấu, cùng hiểu biết vì sao lan truyền ngược đất về bộ nhớ chứ không về phép tính. Xác suất và lý thuyết thông tin cho thấy hàm mất mát không phải phát minh rời rạc. Nó là âm log độ hợp lý của một giả định phân phối. Kèm theo đó là thói quen tính sai số trước khi tin một con số. Hình học và tín hiệu cho ta cấu trúc có sẵn của dữ liệu ảnh cùng hai câu hỏi phòng vệ: đã lọc trước khi giảm mẫu chưa, và biểu diễn quay này có liên tục không.

Còn bỏ ngỏ khá nhiều, và bỏ ngỏ có chủ ý. Chương này không đụng tới lý thuyết học thống kê — vì sao học được từ mẫu hữu hạn — đó là nội dung của Phần B. Nó không đụng tới tối ưu hoá lồi ở mức chứng minh, không đụng tới giải tích ngẫu nhiên hay lý thuyết ma trận ngẫu nhiên, và cố ý chỉ chạm Lie group ở đúng ba thao tác dùng được. Nếu sau này bạn thấy cần sâu hơn ở một nhánh, đó là dấu hiệu tốt: nhu cầu thật xuất hiện trước, việc học đến sau. Nhánh nên đọc tiếp

ngay là [A/02-vat-ly-tao-anh](#), nơi phần lấy mẫu và phần hình học của mục 4 được dùng liên tục.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Mục 1 nói số điều kiện là tính chất của *bài toán* nên đối solver không cứu được, còn mục 2 nói chuẩn hoá và Adam làm huấn luyện nhanh hơn nhờ hạ số điều kiện. Hai phát biểu này mâu thuẫn ở chỗ nào, và giả định nào khác nhau khiến cả hai cùng đúng?
2. Bài bình phương tối thiểu của mục 1 và âm log độ hợp lý Gauss của mục 3 là cùng một thứ viết hai cách. Nếu đổi giả định nhiều sang một phân phối đuôi nặng, công cụ nào của mục 1 — phương trình chuẩn tắc, phép chiếu, số điều kiện — còn dùng được và cái nào mất?
3. Bản đồ độ sâu của bạn mờ dọc theo mọi biên vật thể. Mục 3 và mục 4 cho hai nguyên nhân khác hẳn nhau cho cùng triệu chứng đó; hai nguyên nhân là gì, và phép thử nào phân biệt chúng mà không cần huấn luyện lại?
4. Bạn cho mạng xuất sáu số rồi trực giao hoá bằng SVD để có một phép quay. Thao tác đó nằm trong đồ thị tính toán, nên mục 2 đòi hỏi thêm điều gì mà mục 4 chưa nói tới, và chuyện gì xảy ra khi hai trị kỳ dị gần bằng nhau?
5. Ghép điều kiện dừng của mục 2 với sai số chuẩn của mục 3: vì sao “val loss không giảm thêm sau nhiều epoch” là quy tắc dừng có nghĩa hơn “chuẩn gradient nhỏ”, và quy tắc đó vẫn hỏng trong trường hợp nào?
6. Whitening bằng PCA và chuẩn hoá bên trong mạng đều nhằm kéo các hướng về cùng thang đo. Vì sao chỉ cái sau sống sót trong pipeline hiện đại, và lý do đó liên quan thế nào tới việc phổ trị riêng thay đổi trong lúc huấn luyện?
7. Đường κ theo thời gian trong mini project chỉ đọc được sau khi bạn cân lại thang đo giữa khối tịnh tiến và khối quay. Nếu bỏ bước cân đó, đường κ có còn vọt lên ở các đoạn gần thuần quay không, và khi đó bạn đang đo cái gì?

Vật lý tạo ảnh và mô hình camera (Image Formation Physics and Camera Models)

PHỤ THUỘC A/01-toan-toi-thieu/04-hinh-hoc-va-tin-hieu. Cần ba thứ từ đó: tọa độ đồng nhất, $SE(3)$, và quy tắc lấy mẫu. Đây là chương duy nhất của Phần A có phụ thuộc bắt buộc.

TÓM TẮT

Chương này mở hộp đen giữa thế giới thật và tensor đầu vào: photon và nhiễu, chuỗi xử lý trong máy ảnh, mô hình camera cùng hình học nhiều view, và các cảm biến ngoài RGB. Đọc xong bạn thiết kế được augmentation có cơ sở vật lý thay vì theo cảm tính. Bạn cập nhật đúng ma trận nội tại sau mọi phép biến đổi ảnh. Và khi hiệu năng tụt lúc triển khai, bạn đi ngược đường đi của tín hiệu để tìm nguyên nhân, thay vì thử đổi mô hình. Nếu chỉ nhớ một điều: *phần lớn khoảng cách giữa kết quả thí nghiệm và kết quả triển khai được quyết định trước khi mô hình nhìn thấy dữ liệu*. Đây là phần bị bỏ sót nhiều nhất, và là thứ phân biệt người làm thị giác máy tính với người dùng CNN.

KIẾN THỨC CẦN CÓ TRƯỚC

- **ảnh số** — một mảng số hai chiều (ảnh xám) hoặc ba chiều (ảnh màu, chiều thứ ba là kênh). Giá trị mỗi ô gọi là pixel, và trong ảnh 8 bit thông thường nó nằm trong khoảng 0–255.
- **photon và phép đếm** — ánh sáng tới cảm biến dưới dạng các hạt rời rạc; giá trị pixel về bản chất là một phép *đếm* số hạt bắt được, nên nó mang sẵn một mức dao động thống kê không loại bỏ được.
- **ISP (image signal processor)** — chuỗi xử lý bên trong máy ảnh biến số đo thô của cảm biến thành ảnh xem được: nội suy màu, cân bằng trắng, nén dải sáng, khử nhiễu, nén file. Nó được thiết kế cho mắt người chứ không cho mô hình.
- **RAW so với sRGB** — RAW là số đo gần như thô, tuyến tính theo lượng ánh sáng; sRGB là ảnh đã qua ISP, phi tuyến và phụ thuộc nội dung. Mọi tính toán mang ý nghĩa vật lý chỉ đúng ở miền tuyến tính.
- **ma trận nội tại K** — ma trận 3×3 chứa tiêu cự tính bằng pixel và tọa độ tâm ảnh; nó biến một tia trong hệ tọa độ camera thành tọa độ pixel. Nó thuộc về camera, còn tư thế thì thuộc về lần chụp.
- **tăng cường dữ liệu (data augmentation)** — sinh thêm mẫu huấn luyện bằng cách biến đổi mẫu có sẵn — lật, xoay, đổi màu, thêm nhiễu — để dạy mô hình rằng những biến đổi đó không làm đổi nhãn.
- **domain gap (dịch chuyển miền)** — hiện tượng phân phối dữ liệu lúc triển khai khác phân phối lúc huấn luyện, khiến mô hình tụt hiệu năng dù bản thân mô hình không đổi.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao nhân đôi giá trị pixel trên ảnh sRGB *không* tương đương phơi sáng gấp đôi — và sai lệch đó lớn cỡ nào?
- Q2. Cùng một cảm biến và cùng một cảnh: vì sao tăng ISO làm ảnh sáng lên mà không làm nó đáng tin hơn chút nào?

- Q3.** Bạn resize ảnh còn một nửa trước khi đưa vào mạng, không có exception nào được báo và loss vẫn giảm — vậy cái gì vừa hỏng?
- Q4.** Một mô hình đạt 95% khi thử nghiệm và 80% khi triển khai, dù vẫn là ảnh của cùng loại vật thể: phần lớn mười lăm điểm ấy mất ở đâu trên đường đi của tín hiệu?

Đây là phần bị bỏ sót nhiều nhất trong mọi lộ trình học thị giác, và là thứ phân biệt người làm computer vision với người dùng CNN. Lý do bỏ sót thì dễ hiểu: các tập dữ liệu chuẩn phát hành dưới dạng JPEG đã xử lý xong, nên toàn bộ chuỗi vật lý phía trước trở nên vô hình. Nhưng nó không biến mất. Nó trở thành một tập giả định ngầm mà bạn thừa hưởng, không biết mình đang giữ. Và nó xuất hoá đơn đúng lúc bạn mang mô hình ra khỏi phòng thí nghiệm.

Bốn mục của chương xếp theo *đường đi của tín hiệu*, từ photon tới quyết định, và mỗi mục trả lời một câu hỏi mà mục trước để lại:

- **Mục 1 — ánh sáng và nhiễu:** tín hiệu bắt đầu từ đâu và nó mang sẵn bao nhiêu bất định. Trả lời “vì sao ảnh thiếu sáng làm mô hình sập, ngoài chuyện tối hơn”.
- **Mục 2 — ISP và không gian màu:** tín hiệu ấy bị biến đổi thế nào trước khi thành ảnh. Trả lời “vì sao augmentation trên sRGB không tương đương thao tác vật lý thật”.
- **Mục 3 — camera và hình học nhiều view:** ánh xạ giữa pixel và thế giới ba chiều. Trả lời “thông tin nào bị xoá lúc chụp và mua lại bằng cách nào”.
- **Mục 4 — cảm biến khác và domain gap:** cùng khung phân tích áp cho depth, thermal, event, LiDAR, radar, rồi tổng kết thành luận điểm chung của cả chương.

Thứ tự này nên giữ. Mục 2 dùng mô hình nhiễu của mục 1, còn mục 4 tổng quát hoá cả ba mục trước. Riêng mục 3, với người đã làm SLAM, phần lớn là ôn tập. Ở đó chỉ có hai chỗ đáng đọc kỹ: quy tắc cập nhật K sau resize và crop, và đoạn nói mô hình học được che giấu các cấu hình suy biến.

Bản đồ chương — Vật lý tạo ảnh và mô hình camera

TÓM TẮT

Bản đồ này nói chương 2 gồm bốn mục nào, và vì sao chúng xếp theo đường đi của tín hiệu từ photon tới quyết định. Nó cũng nói mục nào phụ thuộc mục nào, và phần nào là cốt lõi so với phần nào chỉ cần biết là có. Đọc xong bạn quyết định được trong một phút nên vào mục nào trước cho nhu cầu hiện tại. Nếu chỉ nhớ một điều: thứ tự bốn mục là thứ tự nhân quả của tín hiệu thật, nên đọc lệch thứ tự thì sẽ phải quay lại. Luận điểm gộp của cả chương là domain gap sinh ra dọc chuỗi này, trước khi kiến trúc kịp có ý kiến.

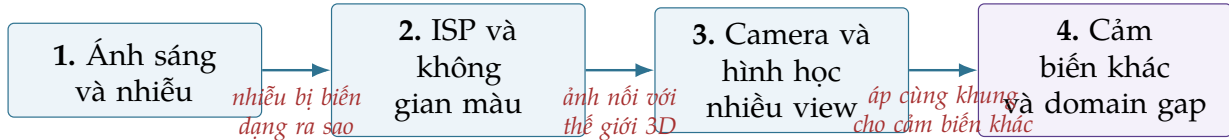
A. Chương này có gì

- **01-anh-sang-va-nhiều** — phơi sáng, dải động, nhiễu shot so với nhiễu đọc, mô hình phương sai affine, vì sao ảnh thiếu sáng làm mô hình sập.
- **02-isp-va-khong-gian-mau** — khử nhiễu, cân bằng trắng, gamma, ánh xạ tone, nén JPEG; RAW so với sRGB; các không gian màu RGB, HSV, YUV, Lab và augmentation nào có nghĩa vật lý.
- **03-camera-va-hinh-hoc-nhiều-view** — pinhole, ma trận nội tại, méo và hiệu chuẩn; ràng buộc epipolar, homography, tam giác hoá, PnP.

- **04-cam-bien-khac-va-domain-gap** — depth, thermal, event, LiDAR, radar và inductive bias riêng của mỗi loại; nguồn gốc domain gap.

B. Vì sao chia và xếp thứ tự như vậy

Bốn mục được xếp theo *đường đi vật lý của tín hiệu*, không theo độ khó hay theo lịch sử. Photon tới cảm biến ở mục 1. Chuỗi xử lý trong máy ảnh biến đổi tín hiệu ấy ở mục 2. Mục 3 nối ảnh kết quả lại với hình học ba chiều. Mục 4 áp cùng khung phân tích cho các cảm biến khác, rồi tổng kết thành luận điểm về domain gap. Cách xếp này có một lợi ích cụ thể: mỗi mục trả lời đúng câu hỏi mà mục trước để lại, nên mạch đọc không bị đứt. Hình 2.1 tóm tắt quan hệ đó.



Hình 2.1. Bốn mục của chương 2 xếp theo đường đi của tín hiệu. Mục 4 (tô khác màu) không phải một chủ đề mới mà là phần tổng quát hoá: nó áp lại khung phân tích của ba mục đầu cho các cảm biến khác, rồi rút ra luận điểm chung về domain gap.

C. Phụ thuộc

Cả chương phụ thuộc [A/01/04-hinh-hoc-va-tin-hieu](#). Phần toạ độ đồng nhất và $SE(3)$ là ngôn ngữ của mục 3. Phần lấy mẫu và aliasing giải thích vì sao khử khảm là bài toán khôi phục, và vì sao undistort làm mờ ảnh. Bên trong chương, mục 2 dùng mô hình nhiễu của mục 1, và mục 4 tổng quát hoá cả ba mục trước; mục 3 thì tương đối độc lập với mục 1 và 2, nên có thể đọc trước nếu nhu cầu hiện tại là hình học.

D. Cốt lõi hay tham khảo

- **Cốt lõi — phải nắm chắc:** quan hệ $SNR = \sqrt{N}$ và mô hình $Var = aI + b$ (mục 1); tính phi tuyến của gamma cùng hệ quả với augmentation (mục 2); quy tắc cập nhật ma trận nội tại sau resize và crop, cùng công thức sai số độ sâu $Z^2/(fB)$ (mục 3); luận điểm domain gap và mạch bốn bước thu hẹp nó (mục 4).
- **Tham khảo — biết là có, tra khi cần:** chi tiết các thuật toán khử khảm, công thức chuyển đổi giữa các không gian màu, mô hình méo bậc cao, và thông số kỹ thuật cụ thể của từng dòng cảm biến. Nhóm cuối cùng này bán rã nhanh nhất trong cả chương.

E. Riêng với người đọc đã làm SLAM

Mục 3 phần lớn là ôn tập, nhưng có ba chỗ mới đáng dừng: quy tắc cập nhật K khi ảnh bị resize hoặc crop trong dataloader (bug im lặng phổ biến nhất khi ghép mạng vào pipeline hình học); sự khác biệt giữa hình học cổ điển “hồng ồn ào” và mô hình học được “hồng im lặng”; và lý do các mô hình độ sâu đơn ảnh dự đoán độ sâu tương đối. Ngược lại, mục 1 và mục 2 gần như hoàn toàn mới với người xuất thân từ ước lượng trạng thái. Chúng giải thích một trải nghiệm quen thuộc mà kinh nghiệm SLAM không giải thích nổi: vì sao hệ thống mất track ngay khi trời tối.

I 2.1 Ánh sáng, cảm biến và mô hình nhiễu (Light, Sensors and Noise Models)

PHỤ THUỘC A/01-toan-toi-thieu/03-xac-suat-va-thong-tin (kỳ vọng, phương sai — ở đây dùng cho một biến đếm Poisson); A/01/04-hinh-hoc-va-tin-hieu nếu muốn hiểu vì sao khử nhiễu và giảm mẫu tương tác với nhau.

TÓM TẮT

Mục này truy nguyên chuỗi nhân quả từ số photon rơi vào cảm biến tới lúc mô hình mất độ chính xác, và cho bạn một mô hình nhiễu định lượng dùng được: phương sai affine theo cường độ. Đọc xong bạn giải thích được vì sao ảnh thiếu sáng phá mô hình theo cách mà nhiễu Gauss thêm vào ảnh sRGB không mô phỏng nổi. Bạn cũng biết khi nào tăng ISO là vô ích, và vùng cháy sáng khác vùng chìm tối ở chỗ nào. Nếu chỉ nhớ một điều: *nhiều ảnh phụ thuộc tín hiệu*. Tỉ số tín hiệu trên nhiễu chỉ tăng theo $\sqrt{N}$, với N là số photon. Mọi kỹ thuật “làm sáng ảnh” vì thế chỉ khuếch đại; chỉ thu thêm ánh sáng thật mới thêm thông tin.

KIẾN THỨC CẦN CÓ TRƯỚC

- **photon và electron** — photon là hạt ánh sáng; cảm biến ánh biến mỗi photon bắt được thành một electron tích trong một “giếng” ứng với một pixel, rồi đếm số electron đó. Vì vậy giá trị pixel về bản chất là một *phép đếm*.
- **phơi sáng (exposure)** — tổng lượng ánh sáng mà cảm biến thu được trong một lần chụp; nó do thời gian mở cửa trập và độ mở của ống kính quyết định.
- **ISO** — mức khuếch đại áp lên tín hiệu *sau khi* đã đo. Tăng ISO làm ảnh sáng lên nhưng không làm cảm biến thu thêm ánh sáng.
- **tỉ số tín hiệu trên nhiễu (SNR)** — tỉ số giữa độ lớn của tín hiệu thật và độ lớn trung bình của nhiễu. SNR càng nhỏ thì càng khó phân biệt chi tiết thật với nhiễu.
- **phân phối Poisson** — phân phối của một phép đếm các sự kiện độc lập xảy ra với tốc độ trung bình cố định. Tính chất định nghĩa của nó là phương sai bằng kỳ vọng — đếm được trung bình N thì dao động cỡ $\sqrt{N}$.
- **ISP (image signal processor)** — chuỗi xử lý bên trong máy ảnh biến số đo thô của cảm biến thành ảnh xem được: nội suy màu, cân bằng trắng, nén dải sáng, khử nhiễu, nén file.
- **RAW so với sRGB** — RAW là số đo gần như thô của cảm biến, còn tuyến tính theo ánh sáng; sRGB là ảnh đã qua ISP, phi tuyến, đã bị chỉnh cho hợp mắt người. Mọi tính toán vật lý chỉ đúng trên RAW.
- **track trong SLAM** — trạng thái hệ thống còn bám được vào bản đồ và ước lượng được tư thế camera. “Mất track” nghĩa là không tìm đủ điểm khớp giữa ảnh hiện tại và bản đồ.

A. Đặt vấn đề

Có một câu hỏi mà hầu hết người làm deep learning cho thị giác không trả lời được, dù nó quyết định phần lớn thất bại thực tế: *ảnh thiếu sáng khác ảnh đủ sáng ở chỗ nào, ngoài chuyện tối hơn?* Câu trả lời phổ biến — “nó nhiễu hơn” — đúng nhưng nông tới mức vô dụng, vì nó gợi ý một cách sửa sai: thêm nhiễu Gauss vào tập huấn luyện. Cách sửa đó gần như không giúp gì, và lý do nằm ở vật lý của phép đếm photon.

Với người đã cài SLAM, đây là mảnh còn thiếu của một trải nghiệm quen thuộc. Bạn đã thấy hệ thống chạy hoàn hảo ban ngày và mất **track** ngay khi trời tối, và đã quy nó cho “ít feature”. Đúng, nhưng chưa đủ: ít feature chỉ là triệu chứng cuối của một chuỗi bắt đầu từ số photon rơi vào mỗi giếng điện tích trong khoảng phơi sáng. Mục này đi ngược chuỗi đó tới tận gốc, vì gốc mới là chỗ can thiệp được.

Học xong mục này thì làm được gì mà trước đó không làm được? Đo và fit mô hình nhiễu của một camera cụ thể bằng vài chục tấm ảnh; quyết định đúng giữa “tăng thời gian phơi sáng”

và “tăng ISO” cho một robot đang chạy; và thiết kế **tăng cường dữ liệu** (*data augmentation*) mô phỏng điều kiện thiếu sáng theo cách có cơ sở vật lý thay vì theo cảm tính.

TRỰC GIÁC

Cảm biến là một lưới xô hứng mưa, mở nắp trong đúng một khoảng thời gian. Số giọt rơi vào một cái xô là biến ngẫu nhiên: cùng cơn mưa, cùng khoảng thời gian, hai cái xô cạnh nhau vẫn đếm ra hai số khác nhau — và độ dao động ấy là tính chất của mưa, không phải khuyết tật của xô. Đó là *shot noise*, và không cảm biến nào loại bỏ được. Tăng ISO không phải hứng thêm mưa; nó là đổi sang thước đo có vạch to hơn, để cái xô gần cạn cũng đọc ra một con số. Con số ấy vẫn dựa trên đúng vài giọt đã hứng, nên không đáng tin hơn chút nào.

B. Ba núm điều khiển ánh sáng, và chỉ hai núm thêm thông tin

Lượng ánh sáng tới cảm biến được điều khiển bởi ba núm, nhưng chúng *không* tương đương nhau — đây là điểm bị hiểu sai nhiều nhất.

Bảng 2.1. Ba núm điều khiển phơi sáng. Cột “thêm photon?” là cột phân biệt bản chất; cột cuối quyết định lựa chọn trên robot.

Núm	Thêm photon?	Cơ chế	Hỏng khi nào
Thời gian phơi sáng	Có	Mở cửa trập lâu hơn, thu nhiều photon hơn theo tỉ lệ tuyến tính	Vật chuyển động hoặc camera rung $\Rightarrow$ nhoè; với robot di động đây thường là cái giá đắt hơn nhiều
Khẩu độ	Có	Mở rộng lỗ thu sáng	Giảm độ sâu trường ảnh nên vật ngoài mặt phẳng lấy nét bị mờ; nhiều camera nhúng có khẩu cố định
ISO (khuếch đại)	Không	Nhân tín hiệu đã đo lên trước hoặc trong khi số hoá	Không thêm thông tin — ánh sáng lên và nhiễu sáng lên cùng hệ số; chỉ có ích khi read noise đang chi phối

$\hookrightarrow$ cùng nguyên lý với $B/03$ — độ khó không biến mất mà bị đẩy đi chỗ khác: giảm nhiễu bằng phơi sáng lâu thì trả giá bằng ảnh mờ do chuyển động

Chuỗi đánh đổi này giải thích vì sao ảnh ban đêm từ một robot di động luôn tệ theo *hai* chiều cùng lúc: vừa nhiễu vừa nhoè. Nó cũng giải thích vì sao mô phỏng chỉ một trong hai chiều là không đủ để mô hình chịu được ban đêm thật.

C. Hai nguồn nhiễu, và chế độ nào đang chi phối

Số photon thu được trong một khoảng thời gian tuân theo phân phối Poisson, mà tính chất định nghĩa của Poisson là phương sai bằng kỳ vọng. Nếu trung bình một điểm ảnh thu N photon thì độ lệch chuẩn là $\sqrt{N}$, nên

$$\text{SNR} = \frac{N}{\sqrt{N}} = \sqrt{N}.$$

Nói bằng lời: *muốn ảnh sạch gấp đôi thì phải thu gấp bốn lần ánh sáng* — một quan hệ lợi ích giảm dần đặt trần cứng lên mọi thứ phía sau. Ví dụ tính tay: điểm thu 10 000 photon có SNR bằng 100; điểm thu 100 photon có SNR bằng 10. Chênh một trăm lần về ánh sáng chỉ đổi lấy mười lần về SNR. Đó là lý do vùng bóng đổ luôn là chỗ mô hình sai trước tiên, ngay trong một tấm ảnh mà mắt người thấy là “đủ sáng”.

Nguồn thứ hai, *read noise*, đến từ mạch đọc và bộ chuyển đổi tương tự sang số. Nó gần như độc lập với mức tín hiệu, nên mô hình hoá bằng một hằng số cộng vào phương sai. Gộp lại, phương sai đo được ở miền tuyến tính có dạng affine:

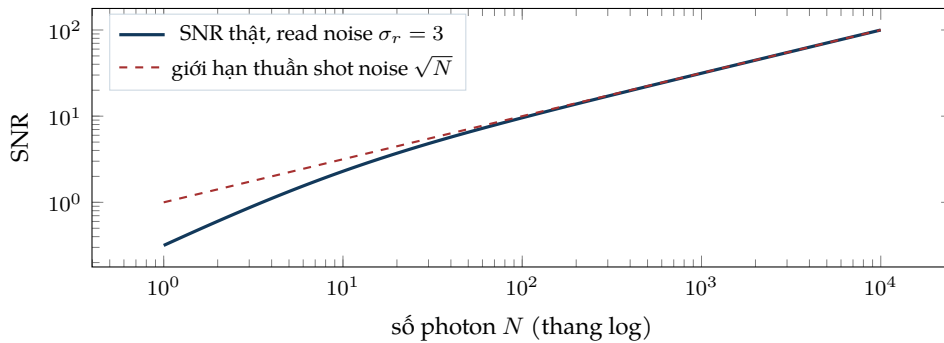
$$\text{Var}(I) = aI + b.$$

Nói bằng lời: *phần tỉ lệ với cường độ là nhiễu photon, phần hằng số là nhiễu của mạch điện*.

↪ **trường hợp riêng của $A/01$** — mô hình nhiễu này là một giả định phân phối, nên chọn hàm mất mát cho bài khử nhiễu chính là chọn mô hình nhiễu

Đây chính là mô hình mà mini project cuối chương yêu cầu bạn fit. Nó khớp rất tốt trên dữ liệu thật: đo được, không phải phỏng đoán.

Hình 2.2 cho thấy hai chế độ và chỗ giao nhau của chúng.



Hình 2.2. Hai chế độ nhiễu. Ở mức sáng (bên phải) hai đường trùng nhau: shot noise chi phối, và không cải tiến mạch điện nào giúp được. Ở mức rất tối (bên trái) đường thật tụt hẳn xuống dưới: read noise chi phối, và đây là vùng duy nhất mà cảm biến tốt hơn tạo khác biệt thật — một khác biệt phần cứng mà không kỹ thuật huấn luyện nào bù được.

Kiểm chứng bằng hai phép tính tay với $\sigma_r = 3$ electron. Ở mức 100 photon, tổng độ lệch chuẩn là $\sqrt{100 + 9} \approx 10,4$ — gần như thuần shot noise, tăng ISO chẳng ích gì. Ở mức 4 photon, tổng là $\sqrt{4 + 9} \approx 3,6$ so với tín hiệu 4, tức SNR khoảng 1,1: tín hiệu và nhiễu ngang nhau, ảnh chỉ còn là hạt.

D. Dải động: thông tin bị xoá vĩnh viễn theo hai kiểu khác nhau

Dải động (*dynamic range*) của cảm biến là tỉ số giữa mức bão hoà của giếng điện tích và sà nhiễu đọc, thường quy ra “stop” (mỗi stop là một lần gấp đôi). Cảnh thật thì có dải động lớn hơn nhiều: một hành lang có cửa sổ ban ngày dễ dàng vượt khả năng ghi của cảm biến điện thoại trong một lần phơi sáng. Hệ quả là hai chế độ mất mát *không đối xứng*:

- **Vùng cháy sáng** — bão hoà ở giá trị lớn nhất, mọi khác biệt bên trong bị xoá hoàn toàn. Không hậu xử lý nào, không mạng nào khôi phục được, vì thông tin không còn ở đó.
- **Vùng chìm tối** — không bị xoá hẳn nhưng nằm dưới sà nhiễu; kéo lên được, nhưng kéo lên chỉ khuếch đại nhiễu.

Sự bất đối xứng ấy có một hệ quả thực hành ít người tận dụng: khi phải chọn, *phơi sáng thiếu một chút an toàn hơn phơi sáng thừa*. Nó cũng giải thích vì sao mọi hệ thống thị giác ngoài trời cần

điều khiển phơi sáng thông minh hơn quy tắc “giữ trung bình ảnh ở giữa”. Và vì sao HDR ghép nhiều mức phơi sáng gần như bắt buộc cho xe tự lái. Cái giá của HDR là bóng ma chuyển động khi ghép — lại một đánh đổi mới.

I Mạch 1 — ảnh thiếu sáng phá mô hình theo đúng thứ tự này

- 1 Vấn đề** ít photon.

Hệ quả SNR tụt theo $\sqrt{N}$, nên vùng tối gần như chỉ còn nhiễu.

Giải pháp cảm biến hoạt động đúng — đây là giới hạn vật lý, không phải lỗi kỹ thuật.
- 2 Vấn đề** ảnh quá tối để nhìn.

Giải pháp khuếch đại bằng ISO và bằng đường cong tone của ISP.

Giá phải trả nhiễu được khuếch đại cùng hệ số, và vì đường cong *gamma* dốc ở vùng tối, nhiễu ở bóng đổ bị kéo giãn *mạnh hơn* nhiễu ở vùng sáng.

Vấn đề mới nhiễu vốn đã phụ thuộc tín hiệu, sau bước này còn phụ thuộc theo một cách phi tuyến.
- 3 Vấn đề** ảnh nhiễu trông xấu.

Giải pháp ISP bật khử nhiễu mạnh.

Giá phải trả chi tiết tần số cao bị xoá cùng với nhiễu, nên góc và cạnh — đúng thứ mà cả ORB lẫn tầng đầu của CNN dựa vào — biến mất.

Vấn đề mới mô hình đối mặt với một phân phối ảnh chưa từng thấy: mịn nhân tạo ở vùng phẳng, nhiễu có tương quan không gian ở vùng còn lại.
- 4 Vấn đề** muốn mô hình chịu được ban đêm nên thêm nhiễu Gauss vào ảnh sRGB khi huấn luyện.

Giải định nhiễu là cộng, độc lập từng pixel, không phụ thuộc cường độ.

Hệ quả cả ba giả định đều sai sau khi qua ISP, nên augmentation kiểu này mô phỏng một thứ không tồn tại.

Giải pháp gỡ xử lý ảnh sRGB ngược về miền tuyến tính, thêm nhiễu shot cộng read đúng mô hình affine, rồi cho chạy lại qua ISP mô phỏng — ý tưởng của dòng *Unprocessing Images for Learned Raw Denoising* [29].

E. Giới hạn và trường hợp hỏng

- **Mô hình affine bỏ qua vài hiệu ứng thật.** Dòng tối phụ thuộc nhiệt độ, nên cùng một camera cho tham số khác nhau giữa sáng sớm và giữa trưa trên xe. Có nhiễu cố định theo vị trí (*fixed-pattern noise*) không ngẫu nhiên và không giảm khi lấy trung bình nhiều khung. Điểm ảnh chết và điểm nóng bị ISP nội suy đè lên, tạo cấu trúc giả.
- **Mô hình chỉ đúng ở miền tuyến tính.** Sau khử khảm và gamma thì nhiễu không còn độc lập giữa các pixel lẫn giữa các kênh — chủ đề của mục tiếp theo.
- **Nhiều thật khó mô phỏng đủ tốt để thay dữ liệu thật.** Các tập dữ liệu chụp cặp ảnh nhiễu-sạch trên điện thoại thật, chẳng hạn SIDD [30], tồn tại chính vì mô phỏng chưa đủ; một bộ khử nhiễu huấn luyện thuần trên nhiễu tổng hợp thường tụt rõ khi đem sang ảnh thật.

CẠM BẦY

“Tăng độ sáng” bằng cách nhân giá trị pixel không phải là mô phỏng phơi sáng nhiều hơn — sai theo cả hai nghĩa. Sai về phi tuyến (mục sau tính ra con số cụ thể), và sai về thống kê: phơi sáng thật gấp đôi làm SNR tăng $\sqrt{2}$ lần, còn nhân giá trị lên gấp đôi giữ nguyên

SNR. Một mô hình học từ augmentation kiểu này sẽ tin rằng ảnh sáng và ảnh tối có cùng mức tin cậy — đúng cái niềm tin làm nó sập lúc trời tối.

CẦN NHỚ

Nhiều ảnh phụ thuộc tín hiệu: $\text{Var} = aI + b$, phần thứ nhất là photon, phần thứ hai là mạch điện. Hệ quả duy nhất cần mang theo: *chỉ có thu thêm ánh sáng thật mới thêm thông tin*; mọi thứ khác — ISO, kéo tone, khử nhiễu, augmentation độ sáng — đều chỉ phân phối lại cái đã có, và thường xoá bớt. [T1]

F. Kết nối

Mục này là gốc của cả chương. Nó chuyển tiếp trực tiếp sang [A/02/02-isp-va-khong-gian-mau](#), nơi ta theo dấu tín hiệu qua các tầng xử lý và thấy nhiễu bị biến dạng ra sao; và là tiền đề của [A/02/04](#), vì “mỗi cảm biến có đặc tính nhiễu riêng” chỉ có nghĩa khi đã hiểu đặc tính nhiễu của cảm biến RGB. Theo chiều ngang nó nối với [A/01/03](#): mô hình nhiễu ở đây là một giả định phân phối, nên chọn loss cho bài khử nhiễu chính là chọn mô hình nhiễu.

↔ **hệ quả của E** — nhiều dạng nhiễu trong các benchmark robustness là mô phỏng thô của đúng chuỗi nhân quả này, nên điểm số trên chúng phải đọc thận trọng

Và nó nối với E: nhiều dạng **corruption** trong các benchmark robustness thực chất là mô phỏng thô của chuỗi nhân quả mô tả ở đây, nên điểm số trên chúng nên đọc thận trọng.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nhiễu shot phụ thuộc tín hiệu. Điều đó phá vỡ giả định nào của mô hình nhiễu Gauss cộng, và hệ quả với hàm mất mát là gì?
2. Vì sao tăng ISO không cải thiện SNR trong khi kéo dài thời gian phơi sáng thì có? Cái giá của lựa chọn thứ hai với một robot đang di chuyển là gì?
3. Vùng cháy sáng và vùng chìm tối hỏng theo hai kiểu khác nhau. Kiểu nào cứu được, và điều đó gợi ý gì về chiến lược phơi sáng?
4. Ở mức tín hiệu rất thấp, nguồn nhiễu nào chi phối, và làm sao nhận ra chế độ đó từ dữ liệu đo được?
5. Vì sao thêm nhiễu Gauss vào ảnh sRGB là cách mô phỏng ban đêm tồi, và quy trình đúng gồm những bước nào?
6. Bạn đo $\text{Var} = aI + b$ trên cùng một camera ở ISO 100 và ISO 6400. Hai tham số thay đổi thế nào, và vì sao?

2.2 ISP, RAW so với sRGB và không gian màu (Image Signal Processing and Color Spaces)

PHỤ THUỘC [A/02-vat-ly-tao-anh/01-anh-sang-va-nhiu](#) (mô hình nhiễu affine — mục này theo dấu nhiễu ấy qua từng tầng xử lý); [A/01/04-hinh-hoc-va-tin-hieu](#) (lấy mẫu — khử nhiễu là một bài toán lấy mẫu).

TÓM TẮT

Mục này mở hộp đen giữa cảm biến và tensor đầu vào: một tấm ảnh sRGB đã đi qua khử nhiễu, cân bằng trắng, gamma, ánh xạ tone và nén JPEG trước khi mô hình nhìn thấy nó, và mỗi tầng phá

một giả định khác nhau. Đọc xong bạn biết vì sao “tăng độ sáng” trên sRGB không tương đương tăng phơi sáng (tính ra được con số: gấp đôi phơi sáng chỉ làm giá trị sRGB tăng khoảng 37%), vì sao giả thiết nhiễu độc lập từng pixel sai sau khứ khảm, và augmentation màu nào có nghĩa vật lý. Nếu chỉ nhớ một điều: *ISP là một phép biến đổi phi tuyến, phụ thuộc nội dung, và không khả nghịch* — nên mọi phép mô phỏng vật lý phải làm ở miền tuyến tính rồi mới dựng ảnh lại, chứ không làm thẳng trên sRGB.

KIẾN THỨC CẦN CÓ TRƯỚC

- **ISP (image signal processor)** — chuỗi xử lý bên trong máy ảnh biến số đo thô của cảm biến thành ảnh xem được. Nó được thiết kế cho mắt người chứ không cho mô hình.
- **RAW so với sRGB** — RAW là số đo gần như thô, còn *tuyến tính* theo lượng ánh sáng — gấp đôi ánh sáng thì gấp đôi giá trị. sRGB là ảnh đã qua ISP: phi tuyến, phụ thuộc nội dung, và không khôi phục lại RAW được một cách chính xác.
- **ma trận lọc màu Bayer và khứ khảm** — mỗi pixel của cảm biến chỉ đo được *một* trong ba màu, sắp theo một hoa văn cố định gọi là Bayer; khứ khảm là bước nội suy hai màu còn thiếu tại mỗi pixel.
- **cân bằng trắng (white balance)** — nhân mỗi kênh màu với một hệ số để vật màu trắng trông trắng dưới nguồn sáng hiện tại. Hệ số này được ước lượng từ chính cảnh, nên nó thay đổi theo nội dung ảnh.
- **gamma** — phép nén dải sáng bằng hàm mũ khoảng $1/2,2$, để các mã số được phân bố đều theo cảm nhận của mắt thay vì đều theo năng lượng ánh sáng. Đây là chỗ ảnh mất tính tuyến tính.
- **ánh xạ tone (tone mapping)** — điều chỉnh độ sáng và tương phản, thường *cục bộ* — cùng một mức sáng ở hai vùng khác nhau của ảnh có thể bị xử lý khác nhau.
- **JPEG và DCT** — JPEG chia ảnh thành các khối 8×8 , biến mỗi khối thành các hệ số tần số bằng phép biến đổi cosin rời rạc (DCT), rồi làm tròn thô các hệ số tần số cao. Đây là nén mất mát: thông tin bị bỏ đi không lấy lại được.
- **tăng cường dữ liệu (data augmentation)** — sinh thêm mẫu huấn luyện bằng cách biến đổi mẫu có sẵn — lật, xoay, đổi màu, thêm nhiễu — nhằm dạy mô hình rằng những biến đổi đó không làm đổi nhân.

A. Đặt vấn đề

Người mới coi ảnh đầu vào là “dữ liệu thô”. Nó không phải. Giữa photon và mảng số mà `cv2.imread` trả về có một chuỗi xử lý dài, phần lớn do nhà sản xuất thiết kế cho *mắt người*, không cho mô hình. Chuỗi ấy phi tuyến, phụ thuộc nội dung ảnh, thay đổi theo từng hãng máy, và phần lớn là không khả nghịch. Mọi giả định ngầm mà ta mang từ vật lý — nhiễu độc lập từng pixel, giá trị pixel tỉ lệ với ánh sáng, cùng vật thể cho cùng màu — đều bị một tầng nào đó trong chuỗi phá vỡ.

Học xong mục này thì làm được gì mà trước đó không làm được? Ba việc: thiết kế augmentation có cơ sở vật lý thay vì chọn theo thói quen; dự đoán trước cái gì hỏng khi huấn luyện trên JPEG rồi triển khai trên luồng RAW hoặc trên một hãng camera khác; và biết khi nào phải bỏ công lấy RAW thay vì tiếp tục làm việc trên sRGB.

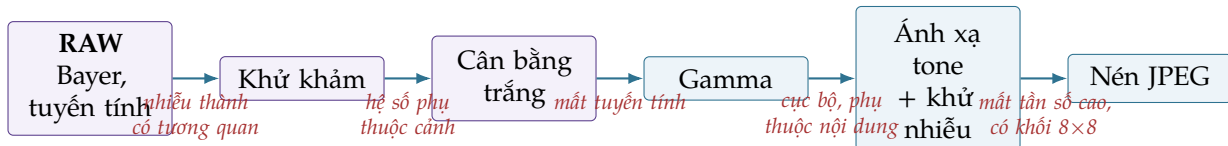
TRỰC GIÁC

ISP giống một phiên dịch viên có phong cách riêng. Bạn đưa cho anh ta một biên bản đo đạc khô khan (ảnh RAW) và anh ta trả lại một bản dịch *dễ nghe*: cân đối lại màu cho hợp mắt, nhấn chỗ này, làm dịu chỗ kia, cắt bớt chi tiết mà anh ta cho là không ai để ý. Bản dịch đẹp hơn bản gốc và hầu hết người đọc thích nó hơn. Nhưng nếu công việc của bạn là *đo*

đặc thì rắc rối bắt đầu. Đổi hãng máy, đổi firmware, đổi chế độ chụp đều là đổi phiên dịch viên. Số liệu của bạn đổi theo, dù hiện thực ngoài kia không đổi chút nào.

B. Chuỗi ISP: mỗi tầng phá một giả định

Hình 2.3 vẽ chuỗi điển hình và đánh dấu chỗ mỗi giả định vỡ.



Hình 2.3. Chuỗi ISP điển hình. Ba khối đầu (tô tím) còn ở miền tuyến tính — đây là nơi duy nhất mô hình nhiễu $\text{Var} = aI + b$ còn đúng. Từ gamma trở đi, mọi phép mô phỏng vật lý làm trực tiếp trên giá trị pixel đều sai. Nhân dưới mũi tên là giả định bị phá ở bước đó.

↔ cùng nguyên lý với $A/01$ — giảm mẫu màu 4:2:0 trong JPEG là aliasing có chủ ý — cùng cơ chế với bước giảm mẫu trong kiến trúc mạng

Đi qua từng tầng, mỗi tầng để lại một hệ quả cụ thể cho việc huấn luyện:

- **Khử nhiễu** — cảm biến chỉ đo một kênh màu tại mỗi pixel qua ma trận lọc màu Bayer, nên hai phần ba giá trị màu là *nội suy*. Hai hệ quả. Thứ nhất, chi tiết màu tần số cao sinh hoa văn giả: *moiré* và “răng cưa màu”. Thứ hai, nhiễu trở nên có tương quan cả về không gian lẫn giữa các kênh. Giả định “nhiều độc lập từng pixel” chết ngay ở tầng đầu tiên.
- **Cân bằng trắng** — nhân mỗi kênh với một hệ số ước lượng từ chính cảnh. Hệ quả: cùng một vật thể cho ra màu khác nhau tùy cảnh xung quanh; mọi đặc trưng dựa trên màu tuyệt đối đều không đáng tin, và *hệ số này thay đổi giữa hai khung hình liên tiếp* khi cảnh đổi.
- **Gamma** — nén dải sáng bằng hàm mũ khoảng $1/2,2$ để phân bố mã đều theo cảm nhận của mắt. Đây là chỗ mất tuyến tính, và là chỗ mọi tính toán vật lý phải dừng lại.
- **Ánh xạ tone và khử nhiễu** — *cục bộ* và phụ thuộc nội dung: cùng một mức sáng ở hai vùng khác nhau của ảnh có thể bị xử lý khác nhau.

↔ ràng buộc bởi $A/02/03$ — hình học trực tiếp và ghép ảnh ở mục camera dựa trên giả định này, nên chính sách ISP đặt một trần lên độ chính xác của chúng

Điều này phá vỡ giả định **độ sáng không đổi** (*brightness constancy*), mà **dòng quang** (*optical flow*) và SLAM trực tiếp lại dựa vào. Đó chính là lý do các hệ thống VO trực tiếp phải hiệu chuẩn quang trắc — một chỗ nổi rất thật với nền của bạn.

- **Nén JPEG** — chuyển sang YCbCr, giảm mẫu kênh màu (thường 4:2:0, tức độ phân giải màu chỉ còn một nửa mỗi chiều), rồi lượng tử hoá hệ số DCT trên từng khối 8×8 . Hệ quả: mất tần số cao, xuất hiện cấu trúc khối, và vật nhỏ có màu là dấu hiệu chính — đèn giao thông xa chẳng hạn — mất đúng cái dấu hiệu ấy.

C. Vì sao augmentation trên sRGB không tương đương thao tác vật lý

Đây là hệ quả quan trọng nhất của mục này, và nó tính ra được bằng số.

Ví dụ tính tay. Xấp xỉ đường cong sRGB bằng $V = L^{1/2,2}$ với L là ánh sáng tuyến tính. Tăng gấp đôi thời gian phơi sáng nghĩa là $L \rightarrow 2L$, nên

$$V' = (2L)^{1/2,2} = 2^{0,4545} L^{1/2,2} \approx 1,37 V.$$

Nói bằng lời: *gấp đôi ánh sáng thật chỉ làm giá trị pixel sRGB tăng khoảng 37%, không phải 100%*. Nhân giá trị sRGB lên gấp đôi tương ứng với tăng ánh sáng thật $2^{2,2} \approx 4,6$ lần — một phép augmentation mạnh hơn nhiều so với ý định.

Sai lệch về hệ số còn là phần nhẹ. Phần nặng là *hành vi của nhiễu*: phơi sáng thật gấp đôi làm SNR tăng $\sqrt{2}$ lần, còn nhân giá trị pixel thì nhân cả tín hiệu lẫn nhiễu nên SNR không đổi. Mô hình học từ augmentation kiểu này rút ra một quy luật sai — rằng ánh sáng và ảnh tối đáng tin như nhau.

I MẠCH 2 — từ “augmentation trông có vẻ hợp lý” tới quy trình đúng

- 1 Vấn đề** muốn mô hình chịu được thay đổi ánh sáng.

Giải pháp nhân giá trị pixel sRGB với một hệ số ngẫu nhiên.

Giả định giá trị pixel tỉ lệ với ánh sáng.

Giá phải trả giả định sai vì gamma, nên biên độ thực tế lệch vài lần và hành vi nhiễu hoàn toàn sai.
- 2 Vấn đề** vậy làm ở miền tuyến tính.

Giải pháp gỡ gamma ($L = V^{2,2}$), nhân hệ số, áp gamma lại.

Hệ quả bây giờ hệ số đúng nghĩa vật lý.

Vấn đề mới nhiễu vẫn không đúng: ánh sáng lên nhưng vẫn mang nhiễu của mức phơi sáng cũ.
- 3 Vấn đề** cần cả nhiễu đúng.

Giải pháp gỡ toàn bộ ISP về RAW xấp xỉ: gỡ gamma, gỡ cân bằng trắng, gỡ ma trận màu. Mô phỏng phơi sáng và nhiễu shot cộng read ở đó, rồi dựng lại ảnh qua một ISP mô phỏng [29].

Giá phải trả phải biết hoặc đoán tham số ISP của camera; kết quả là xấp xỉ, không phải RAW thật.
- 4 Vấn đề** khi nào thì bỏ hẳn sRGB và lấy RAW thật?

Giải pháp khi bài toán là bài toán tín hiệu mức thấp — khử nhiễu, HDR, chụp thiếu sáng, đo đặc quang học — hoặc khi phải triển khai qua nhiều hãng camera khác nhau.

Giá phải trả dữ liệu nặng gấp nhiều lần, hầu như không có mô hình tiền huấn luyện nào nhận RAW, và pipeline phải tự viết.

D. Không gian màu: chọn cái nào cho việc gì

Bảng 2.2. Bốn không gian màu thường gặp. Cột cuối là cột quyết định.

Không gian	Cơ chế	Mạnh khi nào	Hỏng khi nào
RGB	Ba kênh gắn với cảm biến và màn hình	Mặc định; mọi mô hình tiền huấn luyện dùng nó	Phụ thuộc thiết bị; khoảng cách Euclid giữa hai màu không tương ứng cảm nhận
HSV	Tách sắc độ, độ bão hoà, độ sáng	Ngưỡng màu thủ công, augmentation sắc độ rõ tiền	Sắc độ vô định khi bão hoà thấp (vùng xám, vùng tối) nên nhiều sắc độ nổ tung; không đều theo cảm nhận
YUV / YCbCr	Tách độ chói khỏi hai kênh màu	Nén và truyền video; xử lý riêng độ chói	Đã có giảm mẫu màu sẵn trong hầu hết luồng thật, nên chi tiết màu mất trước khi bạn chạm vào
Lab	Xấp xỉ đều theo cảm nhận quanh một điểm trắng	Đo khác biệt màu, tô màu ảnh, chuẩn hoá phong cách	Phụ thuộc điểm trắng giả định; chuyển đổi phi tuyến nên nhiều bị biến dạng khó lường

↔ **tiền đề** D — phân biệt augmentation mô phỏng miền với augmentation chính quy hoá ở chương huấn luyện bắt nguồn từ đúng tiêu chí này

Từ bảng trên rút ra tiêu chí thực dụng cho augmentation màu: một phép biến đổi *có nghĩa vật lý* khi nó tương ứng với một thứ có thể thật sự xảy ra ở khâu tạo ảnh.

- **Có nghĩa vật lý:** thay đổi hệ số cân bằng trắng (nhân từng kênh ở miền tuyến tính — tương ứng đổi nguồn sáng); thay đổi phơi sáng ở miền tuyến tính kèm điều chỉnh nhiễu; thêm nhiễu shot cộng read ở miền tuyến tính; thay đổi chất lượng nén JPEG; làm mờ chuyển động dọc theo một hướng hợp lý.
- **Ít hoặc không có nghĩa vật lý:** xoay sắc độ biên độ lớn (không nguồn sáng nào làm vậy); nhiễu Gauss cường độ cố định trên sRGB; đảo kênh màu; thay đổi độ tương phản độc lập từng kênh. Chúng vẫn có ích như một dạng chính quy hoá. Nhưng đừng nhầm chúng với mô phỏng điều kiện triển khai. Khi báo cáo, hãy nói rõ đó là chính quy hoá, không phải mô phỏng miền.

E. Giới hạn và trường hợp hỏng

- **Gỡ xử lý chỉ là xấp xỉ.** ISP thật có các bước không công bố, phụ thuộc firmware, và một số bước cục bộ thì về nguyên tắc không khả nghịch. Ảnh “RAW tổng hợp” luôn khác RAW thật, và độ lệch ấy đủ lớn để mô hình khử nhiễu tụt điểm — lý do các tập dữ liệu chụp thật vẫn cần thiết [30].
- **Bỏ ISP không phải lúc nào cũng tốt hơn.** ISP là kết quả của hàng chục năm kỹ thuật. Với các bài toán ngữ nghĩa mức cao, ảnh sRGB đã xử lý thường cho kết quả tốt hơn RAW thô. Lý do đơn giản: mô hình tiền huấn luyện đã quen với thống kê của ảnh sRGB. RAW đáng công khi bài toán là bài toán tín hiệu mức thấp.
- **Không gian màu không giải quyết được vấn đề gốc.** Chuyển sang Lab hay HSV không lấy lại được thông tin đã bị lượng tử hoá hay giảm mẫu; nó chỉ sắp xếp lại thông tin còn sót. Nếu chi tiết màu đã mất ở bước 4:2:0 thì mọi phép đổi toạ độ đều vô ích.

CẠM BẤY

Huấn luyện trên JPEG chất lượng 90 rồi triển khai trên luồng ít nén hoặc RAW là dịch chuyển miền theo chiều *ít người ngờ*: mô hình đã học rằng vùng phẳng thì thật sự phẳng (vì lượng tử hoá DCT xoá nhiều ở đó) và biên có cấu trúc khối. Ảnh ít nén mang lại nhiều tần số cao mà mô hình chưa từng thấy, và nó thường hỏng trước ở các bài toán mức thấp — phân đoạn biên, đếm vật nhỏ — chứ không phải ở phân loại toàn ảnh. Cách phòng vệ rẻ nhất là đưa mức nén thành một tham số augmentation ngay từ đầu.

CẦN NHỚ

Ảnh sRGB không phải phép đo, nó là một *bản dịch phi tuyến phụ thuộc nội dung* của phép đo. Mọi thao tác mang ý nghĩa vật lý — phơi sáng, nhiễu, cân bằng trắng — phải làm ở miền tuyến tính rồi mới dựng lại ảnh; làm thẳng trên sRGB thì bạn đang mô phỏng một hiện tượng không tồn tại. [T1]

F. Kết nối

Mục này tiếp nối trực tiếp [A/02/01](#): mô hình nhiễu affine chỉ đúng ở ba khối đầu của Hình 2.3, và toàn bộ phần còn lại của chuỗi là lý do vì sao nó không quan sát được trên ảnh sRGB. Nó là tiền đề của [A/02/04](#), nơi ta thấy mỗi loại cảm biến có một “ISP” riêng với các tầng méo riêng. Theo chiều ngang, nó nối với ba chỗ. Với [A/01/04](#), vì giảm mẫu màu 4:2:0 chính là aliasing có chủ ý. Với B về thiết kế pipeline dữ liệu. Và với D về augmentation, nơi ta phân biệt rạch ròi augmentation *mô phỏng miền* với augmentation *chính quy hoá* — một phân biệt bắt nguồn từ đúng mục này.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao augmentation “tăng độ sáng” trên ảnh sRGB không tương đương tăng phơi sáng thật? Nêu cả sai lệch về hệ số lẫn sai lệch về nhiễu.
2. Nếu train trên JPEG chất lượng 90 và triển khai trên luồng RAW, hỏng ở đâu trước và vì sao?
3. Sau khử nhiễu, giả định nào của mô hình nhiễu ở mục trước không còn đúng? Hệ quả với một mô hình khử nhiễu là gì?
4. Vì sao ánh xạ tone cục bộ phá vỡ giả định độ sáng không đổi, và các hệ VO trực tiếp xử lý chuyện đó bằng cách nào?
5. Khi nào thì đáng bỏ công lấy RAW thay vì tiếp tục dùng sRGB, và cái giá cụ thể là gì?
6. Kể hai phép augmentation màu có nghĩa vật lý và hai phép không có. Với hai phép sau, chúng vẫn có thể có ích — ích ở khía cạnh nào?

2.3 Mô hình camera và hình học nhiều view (Camera Models and Multi-View Geometry)

PHỤ THUỘC [A/01-toan-toi-thieu/04-hinh-hoc-va-tin-hieu](#) (toạ độ đồng nhất, $SE(3)$, nội suy và lấy mẫu — undistort và warp đều là phép lấy mẫu lại).

TÓM TẮT

Mục này nối pixel với thế giới thực: mô hình pinhole, ma trận nội tại, méo ống kính và hiệu chuẩn, rồi ràng buộc epipolar, homography, tam giác hoá và PnP. Đọc xong bạn biết ma trận nội tại đổi thế nào khi resize hay crop ảnh. Quên chỗ đó là bug im lặng số một khi ghép mạng vào pipeline

hình học. Bạn cũng tính được sai số độ sâu ứng với một pixel sai lệch thị sai, và gọi tên được các cấu hình suy biến làm tam giác hoá vô nghĩa. Nếu chỉ nhớ một điều: *camera không đo điểm, nó đo tia*. Độ sâu bị xoá ngay lúc chụp. Mọi phép khôi phục 3D vì thế phải mua lại thông tin đó, bằng một điểm nhìn thứ hai, một chuyển động, hoặc một prior học từ dữ liệu.

KIẾN THỨC CẦN CÓ TRƯỚC

- **pixel và toạ độ ảnh** — ảnh là một lưới ô vuông; toạ độ (u, v) đếm cột và hàng, gốc quy ước ở góc trên trái. Đơn vị của mọi đại lượng “trong ảnh” — tiêu cự, thị sai — đều là pixel, không phải mét.
- **toạ độ đồng nhất** — thêm một thành phần vào vector toạ độ để phép tịnh tiến và phép chiếu viết được dưới dạng nhân ma trận: điểm (x, y) viết thành $(x, y, 1)$, và hai vector tỉ lệ với nhau chỉ cùng một điểm.
- **ma trận nội tại K (intrinsics)** — ma trận 3×3 chứa tiêu cự tính bằng pixel (f_x, f_y) và toạ độ tâm ảnh (c_x, c_y) ; nó biến một tia trong hệ toạ độ camera thành toạ độ pixel. Nó thuộc về *camera*, không thuộc về *cảnh*.
- **ma trận ngoại tại (extrinsics)** — phép quay và tịnh tiến đưa toạ độ thế giới về hệ toạ độ camera — tức là tư thế của camera. Nó thuộc về *lần chụp*, không thuộc về camera.
- **méo ống kính (distortion)** — sai lệch làm đường thẳng ngoài đời cong trên ảnh, mạnh nhất ở rìa; mô hình phổ biến dùng vài hệ số đa thức theo bán kính. “Undistort” là lấy mẫu lại ảnh để bù sai lệch đó.
- **hiệu chuẩn (calibration)** — quá trình ước lượng K và các hệ số méo bằng cách chụp một vật đã biết hình dạng, thường là bàn cờ, ở nhiều tư thế.
- **thị sai (disparity) và đường cơ sở (baseline trong stereo)** — với hai camera đặt cách nhau một khoảng B (đường cơ sở), cùng một điểm 3D rơi vào hai vị trí lệch nhau d pixel trên hai ảnh; d gọi là thị sai và độ sâu là $Z = fB/d$.
- **RANSAC** — thủ tục ước lượng chịu được dữ liệu rác: rút ngẫu nhiên tập nhỏ điểm, dựng giả thuyết, đếm số điểm khớp, lặp nhiều lần và giữ giả thuyết được ủng hộ nhất. Cần nó vì ghép đặc trưng luôn có một tỉ lệ ghép sai.

A. Đặt vấn đề

Với người đã làm SLAM, phần lớn nội dung dưới đây là ôn tập. Trọng tâm vì thế được đặt ở chỗ khác: *cái gì thay đổi khi một mạng nơ-ron được ghép vào pipeline hình học*. Ba câu hỏi cụ thể dẫn dắt cả mục. Thứ nhất, mô hình học được có được phép “quên” hình học không — và nếu không thì hình học vào mô hình bằng đường nào? Thứ hai, những phép biến đổi ảnh mà ta dùng vô tư khi huấn luyện (resize, crop, warp) làm gì với các đại lượng hình học đi kèm? Thứ ba, khi hình học suy biến, mô hình học được có sập theo không, hay nó “đoán” và cho ra một con số trông hợp lý nhưng sai?

Học xong mục này thì làm được gì mà trước đó không làm được? Cập nhật đúng ma trận nội tại sau mọi phép biến đổi ảnh; đọc một paper 3D và biết ngay nó giả định biết K hay không, cần bao nhiêu view, và suy biến ở cấu hình nào; và ước lượng bằng con số xem một hệ stereo cho sai số độ sâu bao nhiêu ở tầm mà bạn quan tâm.

TRỰC GIÁC

Camera là một cái máy đóng dấu: nó lấy tất cả những gì nằm trên một tia xuất phát từ tâm quang học và đóng chúng chồng lên nhau vào đúng một pixel. Phép đóng dấu ấy không mất mát về *hướng* — một pixel xác định chính xác một tia — nhưng xoá sạch thông tin về *khoảng cách dọc theo tia*. Vì vậy mọi kỹ thuật 3D đều là một cách mua lại đúng chiều thông tin đã bị xoá. Cách thứ nhất là đóng dấu lần thứ hai từ chỗ khác: stereo hoặc chuyển động.

Cách thứ hai là dùng hiểu biết sẵn có về thế giới, chẳng hạn kích thước vật quen thuộc hay mặt đất phẳng. Một mạng ước lượng độ sâu đơn ảnh đã học đúng loại hiểu biết thứ hai; nó không phá vỡ định luật nào cả.

B. Pinhole, nội tại, và chỗ mọi người làm sai

Mô hình pinhole nói rằng một điểm 3D trong hệ toạ độ camera chiếu xuống ảnh theo

$$u = f_x \frac{X}{Z} + c_x, \quad v = f_y \frac{Y}{Z} + c_y.$$

Nói bằng lời: *toạ độ pixel chỉ phụ thuộc tỉ số giữa các toạ độ ngang và độ sâu* — nhân cả cảnh lên gấp đôi và đẩy ra xa gấp đôi thì ảnh không đổi một pixel nào. Đó là gốc rễ của tính **nhập nhằng thang đo** (*scale ambiguity*) trong mọi hệ đơn ảnh và đơn camera. Nó cũng là lý do các mô hình độ sâu đơn ảnh hiện đại dự đoán độ sâu *tương đối* chứ không phải mét [31].

↪ **hệ quả của C** — mô hình độ sâu đơn ảnh dự đoán độ sâu *tương đối* chính vì phép chiếu xoá thang đo tuyệt đối

Ví dụ tính tay. Với $f = 500$ pixel, một vật cao 1 m ở khoảng cách 10 m cao $500 \times 1/10 = 50$ pixel trên ảnh. Cùng vật đó ở 50 m chỉ còn 10 pixel — và đây là lý do vật ở xa không phải “khó nhận dạng hơn một chút” mà là *gần như không còn thông tin*: mọi kiến trúc, mọi augmentation đều không tạo ra được các pixel không tồn tại.

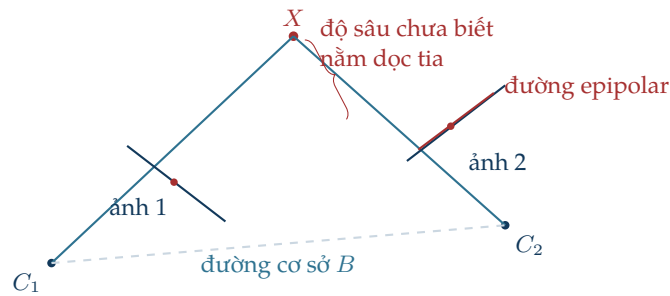
Chỗ mọi người làm sai, và làm sai rất im lặng, là quên rằng K gắn với *hệ toạ độ pixel* nên nó phải đổi theo mọi phép biến đổi ảnh:

- **Thu nhỏ ảnh với hệ số s** — mọi phần tử liên quan pixel nhân với s : $f'_x = sf_x$, $f'_y = sf_y$, $c'_x = sc_x$, $c'_y = sc_y$ (với quy ước tâm pixel, còn có một lượng hiệu chỉnh nửa pixel; sai ở đó ít khi chết người nhưng sai ở hệ số s thì chết ngay).
- **Cắt ảnh từ điểm (x_0, y_0)** — tiêu cự *không đổi*, chỉ tâm ảnh dịch: $c'_x = c_x - x_0$, $c'_y = c_y - y_0$. Đây là chỗ hay bị bỏ sót nhất vì crop trông “vô hại”.
- **Letterbox** — vừa co vừa đệm, nên phải áp cả hai luật trên theo đúng thứ tự.
- **Lật ngang** — $c'_x = W - 1 - c_x$, và nếu bài toán có tính trái-phải thì nhân cũng phải lật theo.

Méo ống kính thì được ước lượng cùng K trong bước hiệu chuẩn, thường bằng phương pháp bàn cờ của Zhang [32]. Chỉ số sức khoẻ của một lần hiệu chuẩn là *sai số tái chiếu trung bình*; theo kinh nghiệm thực hành, dưới nửa pixel là bình thường với camera phổ thông, miễn bàn cờ được chụp ở đủ nhiều tư thế. Vượt quá một pixel là dấu hiệu có gì đó sai: ảnh mờ, bàn cờ không phẳng, hoặc mô hình méo không đủ bậc. Lưu ý một điều mà lý thuyết ít nói: hiệu chuẩn *trôi*. Nhiệt độ, rung động, và nhất là ống kính có lấy nét tự động đều làm K thay đổi theo thời gian, nên một bộ tham số hiệu chuẩn từ năm ngoái không đáng tin bằng bạn nghĩ.

C. Nhiều view: bốn công cụ và điều kiện dùng của chúng

Khi có từ hai ảnh trở lên, thông tin về độ sâu quay lại — nhưng chỉ dưới những điều kiện rất cụ thể. Hình 2.4 minh hoạ ràng buộc cơ bản nhất.



Hình 2.4. Ràng buộc epipolar. Một pixel ở ảnh 1 xác định một tia; ảnh của cả tia đó trên ảnh 2 là một *đường thẳng*. Giá trị thực dụng: bài toán tìm điểm tương ứng co từ tìm kiếm hai chiều xuống một chiều — và nếu biết K của cả hai camera thì ràng buộc này còn cho ra tư thế tương đối giữa hai lần chụp.

Bốn công cụ hay dùng, đặt cạnh nhau trong Bảng 2.3; cột cuối là cột phải thuộc.

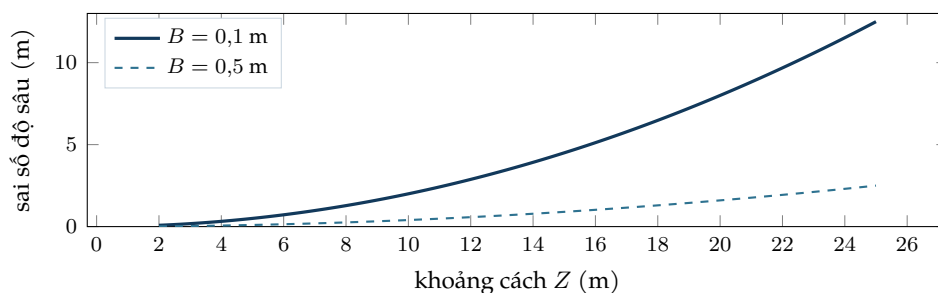
Bảng 2.3. Bốn công cụ hình học nhiều view. Biết “hỏng khi nào” quan trọng hơn biết công thức, vì các cấu hình suy biến xuất hiện thường xuyên trong dữ liệu thật.

Công cụ	Cơ chế	Giả định	Hỏng khi nào
Homography	Ánh xạ 3×3 giữa hai ảnh của cùng mặt phẳng	Cảnh là một mặt phẳng, <i>hoặc</i> camera chỉ xoay tại chỗ	Cảnh có độ sâu thật và camera có tịnh tiến — lúc đó không tồn tại homography đúng, và ước lượng cho ra một phép biến đổi “trung bình” sai ở mọi nơi
Ma trận cơ bản / bản chất	Ràng buộc epipolar giữa hai view	Cảnh cứng; ma trận bản chất còn cần biết K	Camera chỉ xoay không tịnh tiến (tư thế tương đối không xác định được thang đo lẫn hướng tịnh tiến); cảnh phẳng làm bài toán suy biến
Tam giác hoá	Giao hai tia từ hai tâm quang học	Đã biết tư thế tương đối và ghép điểm đúng	Đường cơ sở ngắn so với khoảng cách: hai tia gần song song nên giao điểm cực nhạy với nhiễu
PnP	Tư thế từ các cặp điểm 3D–2D	Đã biết K và có ít nhất ba cặp không thẳng hàng	Các điểm 3D gần đồng phẳng hoặc gần thẳng hàng; ghép sai điểm (phải bọc trong RANSAC)

Cấu hình suy biến của tam giác hoá đáng được định lượng, vì nó quyết định thiết kế phần cứng. Từ $Z = fB/d$, đạo hàm cho

$$\left| \frac{\partial Z}{\partial d} \right| = \frac{Z^2}{fB},$$

nghĩa là *sai số độ sâu tăng theo bình phương khoảng cách*. Lấy $f = 500$ pixel và $B = 0,1$ m. Một pixel sai lệch thị sai gây sai số 0,5 m ở khoảng cách 5 m. Cũng một pixel ấy gây sai số 2 m ở 10 m, và 8 m ở 20 m. Hình 16.4 vẽ đường cong đó. Muốn đo xa hơn thì phải tăng đường cơ sở hoặc tiêu cự, và cả hai đều kéo theo cái giá riêng: đường cơ sở dài làm vùng nhìn chung hẹp lại và làm bài toán ghép điểm khó hơn, tiêu cự dài làm trường nhìn hẹp lại.



Hình 2.5. Sai số độ sâu ứng với một pixel sai lệch thị sai, với $f = 500$ pixel. Quan hệ bình phương nghĩa là mọi hệ stereo đều có một “tâm dùng được” rất rõ; ngoài tầm đó, con số độ sâu trả về vẫn có nhưng không còn nghĩa. Đây là lý do một hệ stereo đường cơ sở ngắn không dùng được cho vật ở xa dù thuật toán khớp điểm hoàn hảo.

I Mạch 3 — khôi phục 3D: mỗi bước gỡ một nút thắt và tạo một nút mới

- 1 Vấn đề** một ảnh không cho biết độ sâu.

Giải pháp chụp thêm ảnh thứ hai từ vị trí khác và dùng ràng buộc epipolar.

Giả định cảnh cứng và có tính tiến giữa hai lần chụp.

Giá phải trả mất thang đo tuyệt đối nếu chỉ có một camera; phải ghép điểm được, tức cảnh phải có kết cấu.
- 2 Vấn đề** ghép điểm luôn có một tỉ lệ ghép sai.

Giải pháp bọc mọi ước lượng trong RANSAC.

Giá phải trả chi phí tăng, và có ngưỡng phải chọn; ngưỡng quá chặt thì loại cả điểm đúng, quá lỏng thì mô hình sai vẫn được chấp nhận.
- 3 Vấn đề** hai view cho kết quả nhiễu và không nhất quán toàn cục.

Giải pháp gộp nhiều view lại và tinh chỉnh đồng thời — đúng là **hiệu chỉnh chùm tia** (*bundle adjustment*), và COLMAP [33] là bản cài đặt tham chiếu.

Giá phải trả bài toán tối ưu lớn, cần khởi tạo tốt, và vẫn nhập nhằng thang đo nếu không có đường cơ sở thật hay cảm biến khác.
- 4 Vấn đề** cảnh không có kết cấu, hoặc chỉ có một ảnh, hoặc không biết K .

Giải pháp thay ràng buộc hình học bằng prior học từ dữ liệu — độ sâu đơn ảnh [31], hoặc dòng hồi quy trực tiếp bản đồ điểm từ ảnh không cần nội tại như DUST3R [34].

Giá phải trả kết quả chỉ đáng tin trong phạm vi phân phối dữ liệu huấn luyện, và mô hình *không* báo cho bạn biết khi nó ra ngoài phạm vi đó — nó vẫn trả về một con số trông hợp lý.

D. Giới hạn và trường hợp hỏng

- **Nhập nhằng thang đo là định luật, không phải khuyết điểm.** Một camera đơn không bao giờ tự cho ra mét. Mọi hệ thống trông như làm được điều đó đều đang mượn thông tin từ nơi khác: IMU, bánh xe, chiều cao camera đã biết, hay kích thước điển hình của vật thể học từ dữ liệu.
- **Rolling shutter phá giả định “một ảnh ứng với một tư thế”** vì mỗi hàng pixel chụp ở một thời điểm khác nhau; hiệu ứng này lớn hẳn lên khi camera quay nhanh, và làm cả hiệu chuẩn lẫn tam giác hoá lệch có hệ thống.

- **Undistort và warp là phép lấy mẫu lại**, nên chúng chịu đúng bài toán aliasing ở [A/01/04](#): warp ảnh nhiều lần liên tiếp làm mờ tích lũy; nên gộp các phép biến đổi thành một rồi lấy mẫu một lần.
- **Mô hình học được che giấu cấu trúc suy biến**. Hình học cổ điển thất bại một cách *ồn ào* — ma trận suy biến, RANSAC không tìm đủ inlier. Mô hình học được thất bại một cách *im lặng*: nó vẫn trả về độ sâu cho một bức tường trắng trơn. Với hệ thống an toàn thì đây là khác biệt quan trọng nhất giữa hai cách tiếp cận, và là lý do người ta vẫn giữ một tầng kiểm tra hình học phía sau mô hình.

↔ **đối lập với** E — ước lượng bất định và *conformal prediction* tồn tại để mô hình học được cũng biết “hông ồn ào” như hình học cổ điển

CẠM BẦY

Resize ảnh mà quên cập nhật K là bug im lặng phổ biến nhất khi ghép mạng vào pipeline hình học. Nó không gây exception và không làm loss tăng vọt. Nó chỉ làm mọi phép chiếu lệch một hệ số không đổi. Triệu chứng hiện ra ở tận cuối chuỗi, dưới dạng “quỹ đạo bị co lại” hay “độ sâu sai một hệ số”. Phép kiểm rẻ nhất: chiếu vài điểm 3D đã biết xuống ảnh đã biến đổi và nhìn bằng mắt xem chúng có rơi đúng chỗ không, trước khi tin bất kỳ con số nào phía sau.

CẦN NHỚ

Camera đo tia, không đo điểm. Độ sâu bị xoá ngay lúc chụp, và chỉ mua lại được bằng một điểm nhìn thứ hai hoặc bằng prior. Và K gắn với hệ toạ độ pixel — mọi resize, crop, letterbox, lật đều phải cập nhật nó. [T1]

E. Kết nối

Mục này dùng trực tiếp toạ độ đồng nhất và $SE(3)$ từ [A/01/04](#), và phần lấy mẫu của mục đó giải thích vì sao undistort làm mờ ảnh. Nó là tiền đề của [A/02/04](#), nơi các cảm biến khác được so sánh chính bằng “chúng đo trực tiếp cái gì mà camera phải suy ra”. Theo chiều ngang, đây là nền của toàn bộ nhánh 3D ở C: ước lượng độ sâu, dựng lại cảnh, biểu diễn trường bức xạ [28]. Tất cả các phương pháp đó đều bắt đầu bằng một bước ước lượng tư thế bằng hình học cổ điển. Sách nền đầy đủ: [35], và bản trình bày dễ vào hơn ở [36].

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Bạn resize ảnh còn một nửa rồi crop 64 pixel từ mép trái — ma trận nội tại mới bằng bao nhiêu, và bước nào không làm đổi tiêu cự?
2. Vì sao homography chỉ đúng cho cảnh phẳng hoặc camera xoay tại chỗ, và nếu áp nó cho cảnh có độ sâu thì sai lệch hiện ra ở đâu?
3. Với $f = 500$ pixel và $B = 0,1$ m, sai số độ sâu ứng với một pixel thì sai ở khoảng cách 20 m là bao nhiêu, và điều đó đặt giới hạn gì lên thiết kế hệ stereo?
4. Camera chỉ xoay tại chỗ giữa hai lần chụp — tam giác hoá hỏng ở chỗ nào, và vì sao đây lại là cấu trúc rất hay gặp trong dữ liệu thật?
5. Một mạng ước lượng độ sâu đơn ảnh không vi phạm định luật vật lý nào — vậy thông tin về độ sâu mà nó dùng đến từ đâu, và điều đó dự báo nó hỏng ở loại cảnh nào?
6. Vì sao hình học cổ điển “hông ồn ào” còn mô hình học được “hông im lặng”, và hệ quả với thiết kế một hệ thống an toàn là gì?

I 2.4 Cảm biến ngoài RGB và nguồn gốc domain gap (Beyond RGB: Sensors and the Origin of Domain Gap)

PHỤ THUỘC A/02-vat-ly-tao-anh/01-anh-sang-va-nhieu (mô hình nhiễu) và A/02/02-isp-va-khong-gian-mau (chuỗi xử lý) — mục này áp dụng cùng cách phân tích đó cho các cảm biến khác, rồi rút ra kết luận chung về domain gap.

TÓM TẮT

Mục này làm hai việc. Thứ nhất, so sánh năm loại cảm biến — depth, thermal, event, LiDAR, radar — trên bốn tiêu chí: đo trực tiếp cái gì, giả định gì, mạnh khi nào, hỏng khi nào. Thứ hai, trả lời câu hỏi lớn nhất của cả chương: khoảng cách giữa hiệu năng khi huấn luyện và khi triển khai sinh ra ở đâu. Đọc xong bạn chọn được cảm biến theo ràng buộc thay vì theo thói quen. Bạn biết mỗi loại đòi hỏi thay đổi gì trong kiến trúc và hàm mất mát. Và bạn biết chỗ nào trong pipeline đáng đầu tư công sức để thu hẹp domain gap. Nếu chỉ nhớ một điều: *phần lớn domain gap trong thị giác sinh ra ở khâu tạo ảnh, không ở kiến trúc*. Đổi backbone vì thế hiếm khi chữa được thứ mà một thay đổi ở khâu thu thập hoặc tiền xử lý chữa được ngay.

KIẾN THỨC CẦN CÓ TRƯỚC

- **domain gap (dịch chuyển miền)** — hiện tượng phân phối dữ liệu lúc triển khai khác phân phối lúc huấn luyện, khiến mô hình tụt hiệu năng dù không có gì trong mô hình thay đổi. “Miền” ở đây nghĩa là một điều kiện thu thập: một hãng camera, một thời điểm trong ngày, một bệnh viện.
- **inductive bias (thiên lệch quy nạp)** — những giả định được cài sẵn vào mô hình hoặc vào dữ liệu, cho phép nó tổng quát hoá từ số mẫu hữu hạn. Ví dụ: *convolution* cài sẵn giả định cấu trúc ảnh mang tính cục bộ và không đổi khi dịch chuyển.
- **camera độ sâu (depth camera)** — cảm biến trả về khoảng cách tới bề mặt tại mỗi pixel, bằng một trong ba cơ chế: chiếu hoa văn hồng ngoại rồi đo biến dạng (structured light), đo thời gian ánh sáng đi và về (time-of-flight), hoặc dùng hai camera và thị sai (stereo).
- **camera nhiệt (thermal)** — cảm biến đo bức xạ hồng ngoại mà vật tự phát ra, phụ thuộc nhiệt độ và độ phát xạ của vật liệu; nó không cần nguồn sáng ngoài nên hoạt động trong bóng tối hoàn toàn.
- **event camera** — cảm biến mà mỗi pixel báo cáo *độc lập và bất đồng bộ* mỗi khi độ sáng log tại pixel đó đổi quá một ngưỡng; đầu ra là một dòng sự kiện thưa theo thời gian, không phải khung hình.
- **LiDAR** — cảm biến phát tia laser và đo thời gian phản hồi để lấy khoảng cách trực tiếp; đầu ra là một đám mây điểm 3D thưa, không phụ thuộc ánh sáng môi trường.
- **radar** — cảm biến dùng sóng vô tuyến; nó đo được khoảng cách và, nhờ hiệu ứng Doppler, đo trực tiếp cả *vận tốc xuyên tâm* của vật thể trong một lần đo.
- **độ phân giải góc** — khả năng phân biệt hai vật nằm sát nhau theo hướng nhìn. Cảm biến có độ phân giải góc thấp thì hai vật gần nhau ở xa sẽ trộn thành một.

A. Đặt vấn đề

Hai câu hỏi tưởng như tách rời nhưng có chung một câu trả lời. Câu thứ nhất: khi bài toán không giải được bằng camera RGB — trời tối, sương mù, vật thể trong suốt, cần vận tốc tức thời — thì đổi sang cảm biến nào, và đổi kiến trúc thế nào cho khớp? Câu thứ hai: vì sao một mô hình đạt 95% khi thử nghiệm lại chỉ còn 80% khi triển khai, dù dữ liệu “vẫn là ảnh của cùng loại vật thể”?

Câu trả lời chung là: *mỗi cảm biến kèm theo một tập giả định riêng về thế giới, và hiệu năng sập đúng lúc một giả định bị vi phạm*. Với câu hỏi thứ nhất, việc chọn cảm biến chính là việc chọn tập giả định nào chấp nhận được. Với câu hỏi thứ hai, khoảng cách train–production chính là khoảng

cách giữa hai tập giả định: tập lúc thu thập dữ liệu và tập lúc triển khai. Cả hai tập ấy được ấn định ở khâu tạo ảnh, trước khi kiến trúc kịp có ý kiến.

Học xong mục này thì làm được gì mà trước đó không làm được? Chọn cảm biến dựa trên ràng buộc và chế độ hồng thay vì theo thói quen; sửa lại pipeline dữ liệu khi phải xử lý depth, thermal hay event thay vì nhồi chúng vào một mạng RGB không đổi; và khi gặp một cú tụt hiệu năng lúc triển khai, biết tìm nguyên nhân ở đúng chỗ thay vì thử đổi mô hình.

TRỰC GIÁC

Mỗi cảm biến là một loại nhân chứng với một kiểu mù riêng. Camera RGB là nhân chứng nhìn rất tinh nhưng chỉ làm việc khi có đèn, và kể lại màu sắc theo cách đã được biên tập. LiDAR là nhân chứng đo khoảng cách chính xác nhưng nhìn thế giới qua vài chục đường kẻ ngang và không phân biệt nổi chữ trên biển báo. Radar nhìn xuyên sương và nói ngay được vật đang lao tới nhanh cỡ nào, nhưng không phân biệt được cái vỏ lon với cái nắp cống. Ghép nhiều nhân chứng lại không cho ra một nhân chứng hoàn hảo. Nó cho ra một bài toán mới: tin ai khi hai lời khai mâu thuẫn. Đó mới là phần khó thật của hợp nhất cảm biến.

B. Năm cảm biến, cùng một khung phân tích

Bảng 2.4. Năm loại cảm biến ngoài RGB. Cột “đo trực tiếp cái gì” quyết định inductive bias mà cảm biến mang lại; cột cuối quyết định bạn có dùng được nó không.

Cảm biến	Đo trực tiếp cái gì	Mạnh khi nào	Hồng khi nào
Depth (ToF / structured light / stereo)	Khoảng cách tới bề mặt tại mỗi pixel	Trong nhà, tầm gần, cần hình học tuyệt đối mà không cần tam giác hoá	Bề mặt bóng, trong suốt, hoặc rất tối (không phản xạ đủ); ánh nắng lấn át nguồn hồng ngoại; ToF bị nhiễu đa đường ở góc tường. Pixel thiếu dữ liệu <i>không</i> phân bố ngẫu nhiên mà tương quan với vật liệu
Thermal	Bức xạ hồng ngoại vật tự phát	Bóng tối hoàn toàn, khói, phát hiện người và động cơ	Không có kết cấu bề mặt nên nhận dạng chi tiết rất khó; “thời điểm cân bằng nhiệt” lúc bình minh và hoàng hôn làm tương phản biến mất; bù đồng đều cảm biến làm ảnh đứng hình chớp nhoáng theo chu kỳ
Event	Thay đổi độ sáng log tại từng pixel, bất đồng bộ	Chuyển động rất nhanh, dải động cực rộng, độ trễ micro giây, tiêu thụ điện thấp	Cảnh tĩnh gần như không sinh dữ liệu; không có cường độ tuyệt đối; nhiễu nền tăng vọt ở mức sáng thấp; mọi kiến trúc ảnh thông thường phải đổi vì đầu vào là dòng sự kiện thưa
LiDAR	Khoảng cách theo tia laser	Hình học 3D chính xác, không phụ thuộc ánh sáng, tầm trung và xa	Mưa, sương, bụi làm tán xạ; bề mặt phản xạ gương; mật độ điểm giảm theo bình phương khoảng cách nên vật ở xa chỉ còn vài điểm; đất và cỏ bộ phận chuyển động
Radar	Khoảng cách và vận tốc xuyên tâm	Thời tiết xấu, tầm xa, cần vận tốc ngay lập tức	Độ phân giải góc rất thấp nên hai vật gần nhau trộn làm một; phản xạ nhiều đường tạo mục tiêu ma; vật kim loại nhỏ có thể sáng hơn cả một chiếc xe

Hai hệ quả cho phía mô hình, và cả hai đều bị bỏ qua thường xuyên:

- **Dữ liệu thiếu không phải nhiễu ngẫu nhiên.** Với camera độ sâu, các pixel không có giá trị tập trung ở kính, ở bề mặt bóng, ở mép vật thể — tức là tương quan mạnh với chính thứ ta muốn dự đoán. Cho mạng nhận thêm một kênh mặt nạ “chỗ nào thiếu” nghe có vẻ vô hại, nhưng nó *rò rỉ nhãn*: mô hình có thể học “chỗ nào thiếu độ sâu thì chỗ đó là cửa kính” và đạt điểm cao mà không hiểu gì về cửa kính. Đây là biến thể của cùng một cạm bẫy sẽ gặp lại ở B dưới tên *shortcut learning*.

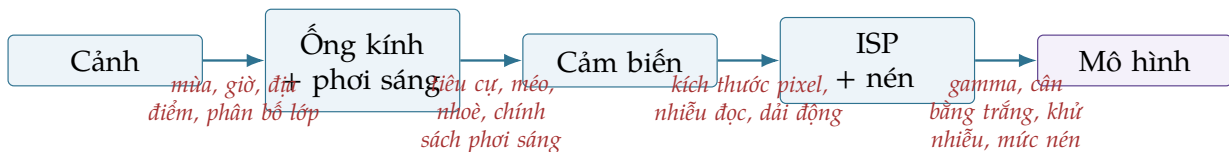
↔ **trường hợp riêng của B** — mô hình bám vào một dấu hiệu tương quan với nhãn thay vì cơ chế thật — ở đây dấu hiệu đó là chính cái mặt nạ dữ liệu thiếu

- **Cấu trúc dữ liệu quyết định kiến trúc.** Đám mây điểm LiDAR không có lưới đều nên convolution hai chiều không áp thẳng được; hoặc phải chiếu về một lưới (mất thông tin, sinh aliasing — đúng bài toán ở A/01/04), hoặc phải dùng kiến trúc làm việc trực tiếp trên tập điểm. Dòng sự kiện của event camera còn xa lưới hơn nữa. Việc “chỉ đổi cảm biến, giữ nguyên mạng” hầu như không bao giờ là lựa chọn đúng.

↔ **ràng buộc bởi C** — cấu trúc dữ liệu quyết định kiến trúc dùng được: lưới đều cho convolution, tập điểm cho kiến trúc trên tập, dòng sự kiện cho biểu diễn thời gian

C. Domain gap sinh ra ở đâu

Đây là luận điểm trung tâm của cả chương A/02. Hình 2.6 đặt các nguồn dịch chuyển vào đúng chỗ chúng sinh ra dọc theo đường đi của tín hiệu.



Hình 2.6. Bốn nhóm nguồn dịch chuyển, và cả bốn đều nằm *trước* mô hình. Đổi kiến trúc chỉ tác động vào khối cuối cùng, nên nó là công cụ yếu để chữa một khoảng cách sinh ra ở ba khối đầu. Ngược lại, ghi lại và kiểm soát bốn nhóm này lúc thu thập dữ liệu là công cụ mạnh — và rẻ hơn nhiều.

Luận điểm “phần lớn domain gap sinh ở khâu tạo ảnh” có bằng chứng ủng hộ, nhưng cần phát biểu cho đúng mức. Bằng chứng gồm: các bộ benchmark đo độ bền trước các biến dạng ảnh cho thấy độ chính xác tụt rất mạnh trước những biến dạng vốn là mô phỏng thô của hiệu ứng cảm biến và nén [37]; việc dựng lại một tập test theo đúng quy trình gốc vẫn làm mọi mô hình tụt hàng điểm [18]; và các tập dữ liệu được thiết kế riêng để đo dịch chuyển miền cho thấy khoảng cách giữa các miền lớn hơn nhiều so với khoảng cách giữa các kiến trúc [38]. Điều *chưa* được chứng minh là một phát biểu định lượng phổ quát, kiểu “ x phần trăm khoảng cách đến từ khâu tạo ảnh”. Con số đó phụ thuộc bài toán. Ai nêu một tỉ lệ cụ thể cho mọi trường hợp là đang nói quá. Hãy giữ nó ở dạng *chiều hướng đã được kiểm chứng nhiều lần*, không phải một hằng số.

Mạch 4 — thu hẹp domain gap, theo thứ tự chi phí tăng dần

1 Vấn đề

Giải pháp

Hệ quả

mô hình tụt điểm khi triển khai, chưa biết vì sao.

trước hết *đo* chứ đừng sửa: giữ một tập nhỏ dữ liệu thu ở chính điều kiện triển khai và tính độ chính xác trên đó, tách theo từng điều kiện (thiết bị, giờ trong ngày, địa điểm).

bạn biết khoảng cách nằm ở nhóm điều kiện nào trước khi tiêu một giờ nào cho mô hình.

2 Vấn đề

khoảng cách đến từ khác biệt về cảm biến và ISP.

- Giải pháp** làm cho tiền xử lý *giống nhau* ở hai đầu — cùng cách resize, cùng mức nén, cùng chính sách phơi sáng — và mô phỏng phần còn lại bằng augmentation có cơ sở vật lý ở miền tuyến tính.
- Giá phải trả** tốn công dựng pipeline gỡ xử lý; đổi lại là cách rẻ nhất trong các cách thật sự hiệu quả.
- 3 Vấn đề** vẫn còn khoảng cách vì cảnh khác hẳn.
- Giải pháp** thu thêm dữ liệu ở đúng miền đích, dù chỉ vài trăm mẫu có nhãn để tinh chỉnh.
- Giả định** bạn tiếp cận được miền đích — không phải lúc nào cũng đúng.
- Hệ quả** theo kinh nghiệm thực hành, một lượng nhỏ dữ liệu đúng miền thường có giá trị hơn một lượng lớn dữ liệu sai miền.
- 4 Vấn đề** không lấy được nhãn ở miền đích.
- Giải pháp** các phương pháp thích nghi miền không giám sát, hoặc huấn luyện trên nhiều miền để mô hình học phần bất biến.
- Giá phải trả** đây là nhóm phương pháp có kết quả *không ổn định* nhất: nhiều so sánh có kiểm soát cho thấy chúng khó vượt được một *baseline* được tune tử tế [39]. Hãy coi đây là lựa chọn cuối, không phải lựa chọn đầu.

D. Giới hạn và trường hợp hỏng

- **Hợp nhất cảm biến không phải phép cộng.** Ghép LiDAR với camera đòi hỏi hiệu chỉnh ngoại chính xác *và* đồng bộ thời gian chính xác; sai lệch một phần trăm giây ở tốc độ xe làm điểm 3D lệch cả mét. Khi hai cảm biến mâu thuẫn, hệ thống phải có quy tắc quyết định — và quy tắc đó là một thiết kế, không phải một thứ mạng tự học được từ dữ liệu bình thường.
- **Một cảm biến “tốt hơn” vẫn có thể làm hệ thống tệ đi** nếu chế độ hỏng của nó tương quan với chế độ hỏng của cảm biến kia. Camera và LiDAR cùng hỏng trong sương mù dày; thêm LiDAR không mua được sự độc lập mà bạn tưởng.
- **Không phải mọi khoảng cách đều thu hẹp được.** Nếu thông tin đã bị xoá ở khâu thu thập — vùng chói sáng, chi tiết màu bị giảm mẫu, vật thể chỉ còn ba pixel — thì không phương pháp thích nghi miền nào lấy lại được. Phân biệt “mô hình chưa học được” với “thông tin không còn ở đó” là bước chẩn đoán đầu tiên và hay bị bỏ qua nhất.

CẠM BÃY

Đừng chữa một vấn đề của khâu tạo ảnh bằng một thay đổi ở kiến trúc. Đây là phản xạ tự nhiên vì kiến trúc là thứ ta kiểm soát trực tiếp và thay đổi nhanh, còn đi thu thập lại dữ liệu thì chậm và chán. Nhưng giả sử tập huấn luyện chụp bằng một hãng máy còn triển khai bằng hãng khác. Khi đó mọi giờ dành cho việc đổi backbone đều tiêu vào sai chỗ. Tệ hơn, các thí nghiệm ấy vẫn cho những khác biệt nhỏ ngẫu nhiên, đủ để bạn tin rằng mình đang tiến bộ.

CẦN NHỚ

Mỗi cảm biến là một tập giả định về thế giới. Hệ thống sập đúng lúc một giả định bị vi phạm. Vì vậy hãy chọn cảm biến theo *chế độ hỏng*, không theo thông số đẹp. Và khi hiệu năng tụt lúc triển khai, hãy đi ngược đường đi của tín hiệu từ cảnh tới mô hình trước khi chạm vào mô hình. [T1]

E. Kết nối

Mục này khép lại chương [A/02](#) bằng cách tổng quát hoá hai mục đầu: mô hình nhiễu ở [A/02/01](#) và chuỗi xử lý ở [A/02/02](#) là trường hợp riêng cho cảm biến RGB của cùng một khung phân tích.

→ **tiền đề** E — đánh giá theo nhóm điều kiện — thiết bị, giờ trong ngày, địa điểm — thay cho một con số trung bình duy nhất, là hệ quả trực tiếp của luận điểm này

Nó nổi sang ba nhánh. Sang B, nơi domain gap được nhìn như một dạng ràng buộc thiết kế và *shortcut learning* được mổ xẻ. Sang D ở phần augmentation, nơi ta phân biệt augmentation mô phỏng miền với augmentation chính quy hoá. Và sang E, nơi đánh giá tách theo nhóm điều kiện trở thành kỷ luật bắt buộc, thay cho một con số trung bình duy nhất.

❏ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao các pixel thiếu dữ liệu của một camera độ sâu không được coi là nhiễu ngẫu nhiên, và điều đó tạo ra dạng rò rỉ nào khi bạn thêm một kênh mặt nạ?
2. Event camera mạnh ở dải động và độ trễ — vậy nó mất gì so với camera thường, và loại bài toán nào nó gần như vô dụng?
3. Vì sao thêm LiDAR vào một hệ camera không nhất thiết làm hệ bền hơn trong sương mù?
4. Luận điểm “phần lớn domain gap sinh ở khâu tạo ảnh” được ủng hộ bởi bằng chứng nào, và phát biểu nào của nó thì *chưa* được chứng minh?
5. Bạn phát hiện mô hình tụt 15 điểm khi triển khai — bước đầu tiên nên làm là gì, và vì sao nó không phải là đổi kiến trúc?
6. Làm sao phân biệt “mô hình chưa học được” với “thông tin không còn trong dữ liệu”, và vì sao phân biệt đó quyết định toàn bộ hướng xử lý tiếp theo?

❏ 2.5 Thực hành

Chương này khác các chương khác ở chỗ phần lớn nội dung chỉ thật sự vào đầu khi bạn cầm một cái camera thật. Hai bài dưới đây được thiết kế để mỗi bài cho ra một *con số* mà bạn tự đo được, chứ không phải một cảm giác.

Repo và SOTA. [opencv](#) cho hiệu chuẩn và undistort; [kornia](#) cho các phép thị giác khả vi (hữu ích khi cần đưa một phép biến đổi hình học vào trong đồ thị tính toán); [rawpy](#) cùng LibRaw để đọc ảnh RAW; [colour-science](#) cho chuyển đổi không gian màu đúng chuẩn; [COLMAP](#) [33] làm bản dựng lại 3D tham chiếu. Về việc gỡ ảnh sRGB ngược về RAW để mô phỏng nhiễu thật, xem dòng *Unprocessing Images for Learned Raw Denoising* [29] và các bản cài đặt lại; về dữ liệu nhiễu thật có cặp ảnh sạch, xem SIDD [30]. Sách nền: Szeliski [36] bản hai miễn phí, và [35] cho phần hình học nhiễu view. *Cảnh báo bán rã*: danh sách này chốt tại tháng 9/2026 và tên các mô hình dẫn đầu bảng sẽ cũ trong 6–12 tháng, trong khi các repo hạ tầng như OpenCV và COLMAP thì bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Ba câu, mỗi câu chạm một mục:

- Vì sao augmentation “tăng độ sáng” trên ảnh sRGB không tương đương tăng phơi sáng thật — sai ở hệ số bao nhiêu, và sai thế nào về hành vi của nhiễu?
- Nếu train trên JPEG chất lượng 90 và triển khai trên luồng RAW, hỏng ở đâu trước và vì sao?
- Nhiễu shot phụ thuộc tín hiệu — điều đó phá vỡ giả định nào của mô hình nhiễu Gauss, và hệ quả với việc chọn hàm mất mát cho bài khử nhiễu là gì?

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Shot noise	Dao động không tránh được của một phép đếm photon: phương sai bằng chính kỳ vọng. Hệ quả là tỉ số tín hiệu trên nhiễu chỉ tăng theo $\sqrt{N}$, nên vùng tối luôn là chỗ mô hình sai trước.
••• Mô hình nhiễu affine	$\text{Var}(I) = aI + b$: phần tỉ lệ với cường độ là nhiễu photon, phần hằng số là nhiễu mạch đọc. Fit được từ vài chục ảnh cảnh tĩnh, nhưng chỉ đúng ở miền tuyến tính.
•• Dải động	Tỉ số giữa mức bão hoà của giếng điện tích và sàn nhiễu đọc. Vượt trần thì cháy sáng và mất hẳn thông tin; dưới sàn thì chìm tối nhưng còn cứu được phần nào.
•• Vì sao tăng ISO không thêm thông tin?	Vì nó khuếch đại tín hiệu <i>đã đo</i> chứ không thu thêm photon, nên nhân cả tín hiệu lẫn nhiễu. Chỉ ở mức rất tối, nơi nhiễu mạch đọc chi phối, nó mới cải thiện được phần nào.
••• ISP	Chuỗi xử lý trong máy ảnh biến số đo thô của cảm biến thành ảnh xem được: khử khảm, cân bằng trắng, gamma, ánh xạ tone, nén. Nó phi tuyến, phụ thuộc nội dung, và không khôi phục ngược chính xác được.
••• Gamma	Phép nén dải sáng bằng hàm mũ khoảng $1/2,2$, để mã số phân bố đều theo cảm nhận của mắt. Đây là chỗ ảnh mất tính tuyến tính, nên mọi tính toán vật lý phải dừng lại trước bước này.
•• Khử khảm	Bước nội suy hai kênh màu còn thiếu tại mỗi pixel, vì cảm biến chỉ đo được một màu mỗi ô. Nó làm nhiễu có tương quan cả về không gian lẫn giữa các kênh, phá giả định nhiễu độc lập từng pixel.
••• Vì sao augmentation độ sáng phải làm ở miền tuyến tính?	Vì gamma bé cong thang sáng: gấp đôi phơi sáng chỉ làm giá trị sRGB tăng khoảng 1,37 lần. Nhân thẳng giá trị pixel còn sai cả về nhiễu, vì SNR không đổi trong khi lẽ ra phải tăng $\sqrt{2}$.
•• Train trên JPEG, chạy trên luồng ít nén — hỏng ở đâu trước?	Ở các bài toán mức thấp như phân đoạn biên hay đếm vật nhỏ, vì mô hình đã quen với vùng phẳng bị lượng tử hoá làm sạch. Phòng vệ rẻ nhất là đưa mức nén thành một tham số augmentation.
••• Mô hình pinhole	Toạ độ pixel chỉ phụ thuộc tỉ số giữa toạ độ ngang và độ sâu. Vì thế camera đo tia chứ không đo điểm: độ sâu và thang đo tuyệt đối bị xoá ngay lúc chụp.
••• Ma trận nội tại	Ma trận K cỡ 3×3 , chứa tiêu cự tính bằng pixel và toạ độ tâm ảnh; nó biến một tia thành toạ độ pixel. Nó gắn với hệ toạ độ ảnh, nên resize nhân mọi phần tử với hệ số, còn crop chỉ dịch tâm ảnh.
•• Ràng buộc epipolar	Một pixel ở ảnh này ứng với cả một đường thẳng ở ảnh kia. Giá trị thực dụng: bài tìm điểm tương ứng co từ tìm kiếm hai chiều xuống một chiều.
•• Homography	Ánh xạ 3×3 giữa hai ảnh của cùng một mặt phẳng. Nó chỉ đúng khi cảnh phẳng hoặc camera xoay tại chỗ; áp cho cảnh có độ sâu thật thì cho ra một phép biến đổi sai ở mọi nơi.
•• Quỹ đạo bị co lại một hệ số không đổi — nguyên nhân?	Gần như luôn là quên cập nhật K sau khi resize hoặc crop ảnh. Phân biệt với lỗi thuật toán: chiếu vài điểm 3D đã biết xuống ảnh đã biến đổi và nhìn xem chúng có rơi đúng chỗ không.
•• Vì sao một hệ stereo có tầm dùng được rất rõ?	Vì sai số độ sâu bằng $Z^2/(fB)$ trên mỗi pixel sai lệch thị sai, tức tăng theo bình phương khoảng cách. Với $f = 500$ và $B = 0,1$ m, một pixel là 0,5 m ở 5 m nhưng 8 m ở 20 m.
•• Event camera	Cảm biến mà mỗi pixel báo độc lập mỗi khi độ sáng log đổi quá một ngưỡng. Mạnh ở dải động và độ trễ micro giây, nhưng không cho cường độ tuyệt đối và gần như cảm trước cảnh tĩnh.
•• Pixel độ sâu bị thiếu tập trung ở kính và mép vật — hệ quả?	Dữ liệu thiếu không ngẫu nhiên mà tương quan với chính thứ cần dự đoán. Cho mạng thêm kênh mật nạ thiếu là rò rỉ nhãn: nó học “chỗ nào thiếu thì đó là kính”.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
● ● ● Domain gap	Khoảng cách giữa phân phối dữ liệu lúc huấn luyện và lúc triển khai. Phần lớn nó sinh ra ở khâu tạo ảnh — cảnh, quang học, cảm biến, ISP — tức trước khi kiến trúc kịp có ý kiến.
● ● Vì sao đổi backbone hiếm khi chứa được domain gap?	Vì kiến trúc là khối cuối cùng của chuỗi, còn nguyên nhân nằm ở bốn khối trước nó. Các thí nghiệm đổi kiến trúc vẫn cho khác biệt nhỏ ngẫu nhiên, đủ để bạn tưởng mình đang tiến bộ.
● Vì sao thêm LiDAR chưa chắc làm hệ bền hơn?	Vì chế độ hồng có thể tương quan: sương mù dày làm hỏng cả camera lẫn LiDAR. Hợp nhất cảm biến chỉ mua được sự độc lập khi hai cảm biến hồng vì hai lý do khác nhau.

● ● ● phải thuộc ● ● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenCV, COLMAP	Mô hình pinhole, hiệu chuẩn bàn cờ và hình học nhiều view gần như nguyên vẹn như trình bày ở mục 3	Hạ tầng; bền hơn nhiều so với tên mô hình dẫn đầu bảng
Dòng xử nhiều RAW kiểu <i>Unprocessing</i>	Gỡ ISP về miền tuyến tính rồi mô phỏng nhiều shot và read, đúng mạch của mục 1 và 2	Kỹ thuật chuẩn để sinh dữ liệu huấn luyện xử nhiều
Pipeline HDR trên điện thoại	Ghép nhiều mức phơi sáng để vượt dải động cảm biến — kèm cái giá là bóng ma chuyển động	Sản phẩm thật, nhiều hãng; chi tiết cài đặt thì không công bố
MiDaS và họ độ sâu đơn ảnh	Dự đoán độ sâu tương đối chứ không tuyệt đối, vì phép chiếu đã xóa thang đo	Thường được dùng làm prior cho các pipeline dựng cảnh
Benchmark robustness kiểu ImageNet-C	Mô phỏng thô các biến dạng sinh ra ở khâu tạo ảnh, để đo domain gap	Chỉ là xấp xỉ của hiệu ứng thật; đọc điểm số thận trọng

MINI PROJECT SLAM · Từ mô hình nhiễu của một camera thật tới ngưỡng gây tracking · một tới hai ngày

Mục tiêu	nổi thẳng vật lý cảm biến với hiệu năng SLAM: đo mô hình nhiễu của một camera thật, áp chính mô hình đó lên một chuỗi EuRoC, rồi tìm mức nhiễu mà tracking bắt đầu gây — thay vì chỉ nói “ảnh tối thì khó hơn”.
Dữ liệu	(i) khoảng ba mươi ảnh chụp cùng một cảnh tĩnh trên chân máy ở ISO 100, 800 và 6400, chụp RAW nếu máy cho phép; (ii) hai chuỗi EuRoC có chân trị ở hai mức khó, ví dụ một chuỗi dễ và một chuỗi có đoạn ảnh mờ.
Các bước	(1) Chọn vài chục vùng phẳng ở các mức sáng khác nhau, đo phương sai theo cường độ và fit $\text{Var} = aI + b$ cho từng mức ISO; bạn sẽ thấy cả a lẫn b tăng theo ISO, và thấy đường fit trên ảnh JPEG không thẳng như trên RAW. (2) Hiệu chuẩn camera bằng bàn cờ, báo cáo sai số tái chiếu, rồi thử bỏ bớt tư thế bàn cờ để thấy hiệu chuẩn cần <i>đa dạng tư thế</i> chứ không cần nhiều ảnh. (3) Áp mô hình nhiễu đã fit lên ảnh EuRoC ở bốn mức tương đương ISO. Làm ở miền tuyến tính: gỡ gamma xấp xỉ trước, áp lại sau. Ghi rõ trong báo cáo rằng đây là <i>xấp xỉ</i> , vì ảnh EuRoC đã qua xử lý nên không phải RAW thật. (4) Chạy hệ SLAM ba lần cho mỗi mức, ghi số đặc trưng bám được trên mỗi ảnh và ATE bằng <i>evo</i> . (5) Vẽ cả hai đại lượng theo mức nhiễu.
Xong khi	bạn có một hình duy nhất, trục hoành là mức nhiễu (hoặc ISO tương đương), hai trục tung là số đặc trưng bám được và ATE; trên đó bạn chỉ ra mức nhiễu mà số đặc trưng tụt xuống dưới ngưỡng buộc hệ khởi tạo lại, và ATE nhảy bậc ngay tại đó. Vẽ kèm độ lệch chuẩn của ba lần chạy, để chứng minh bậc nhảy lớn hơn nhiễu của chính harness.
Nổi với	A/02/01 cho mô hình nhiễu affine và quan hệ $\text{SNR} = \sqrt{N}$; A/02/02 cho lý do bắt buộc phải làm ở miền tuyến tính; A/01/03 để trả lời ba lần chạy đã đủ chưa. Đường số điều

kiện theo thời gian từ mini project chương 1 dùng được ngay làm biến giải thích cho các đoạn gãy.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì gamma. Ánh sáng thật $L \rightarrow 2L$ chỉ làm giá trị sRGB tăng $2^{1/2.2} \approx 1,37$ lần chứ không phải hai lần. Nặng hơn là hành vi nhiễu: phơi sáng thật gấp đôi làm SNR tăng $\sqrt{2}$, còn nhân giá trị pixel thì SNR không đổi.
- A2. Vì ISO khuếch đại tín hiệu *đã đo* chứ không thu thêm photon, nên nó nhân cả tín hiệu lẫn nhiễu với cùng một hệ số. Chỉ ở vùng cực tối, nơi read noise chi phối, nó mới cải thiện được phần nào.
- A3. Ma trận nội tại. Tiêu cự và tâm ảnh đều tính bằng pixel nên phải nhân theo hệ số resize; quên thì mọi phép chiếu lệch một hệ số không đổi, và triệu chứng chỉ hiện ra ở cuối chuỗi dưới dạng “quỹ đạo co lại” hay “độ sâu sai một hệ số”.
- A4. Phần lớn mất ở khâu tạo ảnh — cảnh, quang học và chính sách phơi sáng, cảm biến, ISP và nén — tức là ở bốn trạm nằm trước mô hình. Bằng chứng ủng hộ chiều hướng này khá vững, dù không có một tỉ lệ phổ quát cho mọi bài toán.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy nghĩ tới cái nhiệt kế điện tử treo trong phòng bạn. Bạn muốn biết phòng nóng bao nhiêu, nhưng thứ đọc được là con số trên màn hình — và giữa hai thứ đó có cả một chuỗi: cảm biến nằm ở chỗ nào, vỏ máy tự nóng lên bao nhiêu, rồi một chương trình nhỏ làm mượt số đo và làm tròn trước khi hiện lên.

Máy ảnh y hệt vậy, chỉ dài hơn. Nó đếm hạt sáng rơi vào từng ô nhỏ; chỗ tối đếm được ít nên con số nhảy nhót, còn chỗ quá sáng thì tràn và mọi khác biệt bên trong bị xoá sạch, không cách nào lấy lại. Rồi máy tự kéo sáng cho dễ nhìn, bôi mờ những chi tiết nó đoán là không ai để ý, và nén lại cho nhẹ. Cái đến tay bạn không phải một phép đo, mà là một bản đã được biên tập.

Mẹo của chương: muốn làm bất cứ phép tính nào mang ý nghĩa vật lý — giả lập trời tối hơn, thêm hạt nhiễu — thì phải gỡ ngược bản biên tập về gần số đo gốc, làm ở đó, rồi dựng lại.

Cái giá: gỡ ngược chỉ là gần đúng, phần đã bị xoá thì mất hẳn, và mỗi hãng máy biên tập một kiểu.

Chỗ không rút gọn được: câu “phần lớn khoảng cách giữa phòng thí nghiệm và thực tế sinh ra ở khâu chụp ảnh” nghe gọn nhưng không quy được thành một tỉ lệ chung: nó đổi theo từng bài toán, và ai đưa ra một con số phần trăm cho mọi trường hợp là đang nói quá.

Một câu: Ảnh không phải một phép đo mà là một bản đã biên tập, nên phần lớn khoảng cách giữa thí nghiệm và thực tế đã được quyết định xong trước khi mô hình kịp nhìn thấy gì.

2.6 Kết nối

Chương này khép lại Phần A và mở ra ba hướng. Hướng gần nhất là Phần B. Ở đó, các ràng buộc vật lý vừa nêu được nhìn lại như *ràng buộc thiết kế*: dải động của cảm biến, độ trễ do thời gian phơi sáng, và khác biệt giữa các hãng camera. Chúng đứng cạnh ràng buộc về nhân và về tính toán. Và chính chúng loại bỏ phần lớn phương án, trước khi độ chính xác kịp lên tiếng. Hướng thứ hai là C, nơi hình học nhiều view của mục 3 trở thành bước khởi tạo bắt buộc cho gần như mọi phương pháp dựng cảnh học được. Hướng thứ ba là E: luận điểm domain gap ở mục 4 biến thành một kỷ luật đo đạc cụ thể — tách kết quả theo nhóm điều kiện thay vì báo một con số trung bình.

Tổng kết chương

Chương này đã đi hết đường đi của tín hiệu và, ở mỗi trạm, chỉ ra một giả định bị phá vỡ. Ở cảm biến, phép đếm photon làm nhiều phụ thuộc tín hiệu. Tỷ số tín hiệu trên nhiễu vì thế chỉ tăng theo căn của lượng ánh sáng. Mọi kỹ thuật “làm sáng ảnh” chỉ khuếch đại, không thêm thông tin. Ở ISP, mỗi tầng phá một giả định khác nhau. Gamma phá tính tuyến tính. Khử nhiễu phá tính độc lập của nhiễu giữa các pixel. Ánh xạ tone cục bộ phá giả định độ sáng không đổi, còn nén JPEG xoá tần số cao. Vì vậy mọi phép mô phỏng vật lý phải làm ở miền tuyến tính, rồi mới dựng ảnh lại. Ở camera, phép chiếu xoá độ sâu, và K gắn với hệ tọa độ pixel. Mỗi phép biến đổi ảnh vì thế đòi một phép cập nhật K tương ứng. Quên thì không có lỗi nào được báo. Ở các cảm biến khác, mỗi loại mang một tập giả định riêng và một kiểu mù riêng, nên hợp nhất cảm biến là bài toán quyết định tin ai chứ không phải phép cộng.

Điểm gộp của cả bốn mục là luận điểm về domain gap: khoảng cách giữa phòng thí nghiệm và thực tế được ấn định phần lớn ở những trạm nói trên, tức là *trước* khi kiến trúc kịp có ý kiến. Luận điểm này có bằng chứng ủng hộ vững về chiều hướng, dù không có con số phổ quát; và hệ quả thực hành của nó rất cụ thể — khi hiệu năng tụt lúc triển khai, hãy đi ngược đường đi của tín hiệu trước khi chạm vào mô hình.

Còn bỏ ngỏ ba nhánh, và bỏ ngỏ có chủ ý. Thứ nhất là quang học ở mức thiết kế ống kính. Thứ hai là xử lý ảnh cổ điển ở mức thuật toán chi tiết. Thứ ba là toàn bộ phần thuật toán 3D hiện đại — dựng lại cảnh, biểu diễn ngầm, hợp nhất đa cảm biến — vốn thuộc C. Nhánh nên đọc tiếp là Phần B, nơi các ràng buộc vật lý ở đây được nhìn lại như một dạng ràng buộc thiết kế, cạnh các ràng buộc về dữ liệu, tính toán và độ trễ.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Mục 1 nói kéo dài phơi sáng là cách duy nhất thật sự thêm thông tin, còn mục 3 cần đặc trưng sắc nét để tam giác hoá. Với một robot đang di chuyển, hai đòi hỏi đó xung đột ở đâu, và bạn phân xử bằng đại lượng nào?
2. Bạn muốn sinh dữ liệu “chạy đêm” từ một tập ảnh sRGB chụp ban ngày. Chuỗi thao tác đúng đi qua những trạm nào của mục 1 và mục 2, và trạm nào trong chuỗi đó không đảo ngược được, khiến phép mô phỏng chỉ còn là xấp xỉ?
3. Hãy kể ba lỗi im lặng — mỗi mục 2, 3 và 4 một lỗi — tức lỗi không sinh exception và không làm loss huấn luyện xấu đi. Với mỗi lỗi, phép kiểm nào bắt được nó trước khi triển khai?
4. Vùng chói sáng ở mục 1 và pixel thiếu dữ liệu của camera độ sâu ở mục 4 đều là mất dữ liệu *không* ngẫu nhiên. Điểm chung về mặt thống kê là gì, và vì sao cùng một cách xử lý lại không dùng được cho cả hai?
5. Random resize crop gần như vô hại với bài phân loại nhưng làm hỏng nhãn của một mạng dự đoán độ sâu metric. Cơ chế nào gây ra chuyện đó, và sửa thể nào mà vẫn giữ được phép augmentation?
6. Luận điểm gộp của chương là phần lớn domain gap sinh ra trước khi mô hình nhìn thấy dữ liệu. Bạn thiết kế thí nghiệm nào có thể *bác bỏ* luận điểm đó, và kết quả nào sẽ đủ để bác bỏ?

TƯ DUY HỆ THỐNG

TÓM TẮT

Phần B trả lời một câu hỏi duy nhất: làm sao ra quyết định thiết kế đúng khi chưa có mô hình nào chạy. Chín chương là chín dạng ràng buộc — cách đọc một hệ thống lạ, nền lý thuyết của việc học, định nghĩa bài toán, chất lượng nhãn, chia dữ liệu, dịch chuyển phân phối, hàm mất mát, kiến trúc, và phần cứng. Đọc xong bạn loại được phần lớn phương án sai trước khi viết dòng code đầu tiên. Nếu chỉ nhớ một điều: đây là phần có chu kỳ bán rã dài nhất của cây — kiến trúc trong Phần C sẽ cũ, cách suy nghĩ ở đây thì không.

Toàn bộ phần này trả lời một câu hỏi duy nhất: **làm sao ra quyết định thiết kế đúng khi chưa có mô hình nào chạy?** Đó là câu hỏi bị bỏ qua nhiều nhất, vì nó không có code để chạy và không có con số để khoe, nhưng nó quyết định phần lớn kết quả cuối cùng. Một dự án chọn sai đơn vị phân tích ở tuần đầu, hoặc chia dữ liệu để lọt rò rỉ, sẽ cho ra những con số đẹp và vô nghĩa suốt sáu tháng sau đó, và không kiến trúc nào cứu được.

Cách đọc phần này khác với cách đọc một giáo trình. Mỗi chương là *một dạng ràng buộc*, và mọi kỹ thuật xuất hiện trong đó đều là câu trả lời cho một thất bại cụ thể đã xảy ra trước nó. Chương 3 đưa ra quy trình đọc một hệ thống lạ và nguyên lý xuyên suốt cả cây rằng độ khó không biến mất mà chỉ bị đẩy đi chỗ khác. Chương 4 dựng nền lý thuyết tối thiểu về việc học từ dữ liệu, và giới thiệu trục đánh đổi trung tâm giữa giả định và dữ liệu mà mọi phần sau sẽ gặp lại. Chương 5 tới 8 đi theo vòng đời của dữ liệu: định nghĩa bài toán, chất lượng nhãn, chia dữ liệu và giao thức đánh giá, rồi chuyện gì xảy ra khi phân phối lúc triển khai không còn giống lúc huấn luyện. Chương 9 tới 11 chuyển sang ba ràng buộc thiết kế còn lại: hàm mất mát phát biểu điều ta thật sự muốn, kiến trúc phát biểu giả định về lời giải, và phần cứng phát biểu giới hạn cuối cùng mà không lập luận nào vượt qua được.

Đây là phần thuộc tầng [T2], tức tầng có chu kỳ bán rã dài nhất trong cây. Kiến trúc trong phần C sẽ cũ; cách suy nghĩ trong phần B thì không.

Bảng 2.7. Bản đồ Phần B: các chương, nội dung, và mức đầu tư đề xuất.

Ch.	Thư mục	Nội dung	Mức
3	03-giai-phau-he-thong	Quy trình đọc một hệ thống lạ; nguyên lý độ khó bị đẩy đi chỗ khác	cốt lõi
4	04-nen-tang-hoc-tu-du-l	ERM, biểu diễn, manifold, inductive bias, trục đánh đổi giả định và dữ liệu	cốt lõi
5	05-thiet-ke-bai-toan	Đơn vị phân tích, định nghĩa nhãn, taxonomy, dạng đầu ra, population và sampling frame	cốt lõi
6	06-chat-luong-du-lieu	Nhiều nhãn, agreement, tài liệu hoá dữ liệu, phát hiện nhãn sai	cốt lõi
7	07-chia-du-lieu-va-danh	Leakage, chia theo nhóm và thời gian, chọn metric, đánh giá theo slice	cốt lõi
8	08-dich-chuyen-phan-pho	Các loại shift, domain adaptation và generalization, long-tail, OOD	cốt lõi
9	09-thiet-ke-ham-mat-mat	Loss như phát biểu về phân phối; khoảng cách giữa loss và metric	cốt lõi
10	10-thiet-ke-kien-truc	Inductive bias trong kiến trúc, receptive field, đa tỉ lệ, backbone/neck/head	cốt lõi
11	11-gioi-han-tinh-toan	Vì sao FLOPs không dự đoán được latency; roofline, bộ nhớ activation	cốt lõi

Đọc tuần tự. Chương 3 và 4 là tiền đề của bảy chương còn lại; chương 5 tới 8 đi theo vòng đời dữ liệu; chương 9 tới 11 là ba ràng buộc thiết kế còn lại.

Giải phẫu một hệ thống thị giác (Anatomy of a Vision System)

PHỤ THUỘC không bắt buộc; đây là chương mở đầu của Phần B. Đọc trước [A/02-vat-ly-tao-anh](#) nếu muốn hiểu vì sao “luồng dữ liệu” phải bắt đầu từ cảm biến chứ không từ tensor đầu vào.

TÓM TẮT

Chương này đưa ra ba thứ. Một quy trình bảy câu hỏi để mổ xẻ một repo hoặc paper lạ. Một bảng nói rõ paper, code và **ablation** mỗi nguồn che giấu điều gì. Và nguyên lý *độ khó không biến mất, chỉ bị đẩy đi chỗ khác*, công cụ dùng được suốt phần còn lại của cây. Đọc xong, chỉ sau một buổi bạn nói ra được các **chế độ hỏng** (*failure mode*) của một hệ thống mới gặp, thay vì chỉ liệt kê module của nó. Bạn cũng biết lúc nào nên đóng một paper lại thay vì đọc tiếp. Nếu chỉ nhớ một điều: đọc một hệ thống là đi tìm chỗ nó phải trả giá. Ràng buộc cứng loại bỏ phần lớn phương án trước khi độ chính xác kịp lên tiếng, nên hãy tìm ràng buộc trước và tìm ý tưởng sau.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model)** — một hàm có tham số, nhận ảnh làm đầu vào và trả về dự đoán (nhãn, hộp, mask). “Huấn luyện” nghĩa là đi tìm bộ tham số làm dự đoán sát nhãn nhất; “suy luận” (*inference*) là chạy hàm đó trên dữ liệu mới.
- **hệ thống thị giác và pipeline** — một hệ thống thật không chỉ có mô hình. Nó là một chuỗi khối nối tiếp — cảm biến, tiền xử lý, mô hình, hậu xử lý, quyết định — và chuỗi đó gọi là *pipeline*. Mỗi khối trong chuỗi gọi là một *module*. Phân biệt hai thứ này là điều kiện để đọc chương.
- **tín hiệu giám sát (supervision signal)** — thứ nói cho thuật toán biết dự đoán đúng là gì. Thường là nhãn do người gán nhãn tạo ra, nhưng cũng có thể là một ràng buộc vật lý, một cảm biến khác, hoặc chính dữ liệu (tự giám sát). Biết tín hiệu giám sát đến từ đâu là biết hệ thống học được điều gì và không học được điều gì.
- **giả định (assumption)** — một mệnh đề về thế giới mà hệ thống coi là đúng nhưng không kiểm tra — ví dụ “vật thể luôn nằm trên mặt đất” hay “ảnh đủ sáng”. Giả định không được nói ra vẫn có hiệu lực; hệ thống hỏng đúng lúc giả định bị vi phạm.
- **ablation** — thí nghiệm bỏ đi từng thành phần của hệ thống rồi đo lại kết quả, để biết thành phần nào thực sự đóng góp. Từ này không có bản dịch tiếng Việt thông dụng nên tài liệu giữ nguyên.
- **baseline** — phương pháp đối chứng đơn giản hơn mà phương pháp mới phải vượt qua thì mới có ý nghĩa. Nếu baseline không được tinh chỉnh cẩn thận thì mọi so sánh với nó đều vô giá trị.
- **độ trễ (latency) và FPS** — độ trễ là thời gian xử lý một khung hình, tính bằng mili-giây; FPS (*frames per second*) là số khung hình xử lý được mỗi giây. Hai đại lượng nghịch đảo nhau và là ràng buộc cứng phổ biến nhất trong hệ thống thời gian thực.
- **mAP** — thước đo phổ biến cho bài toán phát hiện vật thể, gộp độ chính xác và độ bao phủ trên nhiều ngưỡng thành một con số duy nhất. Ở chương này bạn chỉ cần biết nó là “điểm số trong bảng kết quả”; định nghĩa đầy đủ nằm ở Phần E.

CÂU HỎI MỞ ĐẦU

- Q1. Hai kỹ sư đọc cùng một repo trong cùng một buổi: vì sao một người ra về với danh sách module, còn người kia dự đoán được hệ thống sẽ sập trong một hành lang tường trắng?
- Q2. Nếu phần method là phần dài nhất của mọi paper, vì sao nó lại là phần đáng đọc *sau cùng*?
- Q3. Một paper báo cáo 45 ms còn bạn có 33 ms mỗi khung hình — vì sao phép tính hai dòng đó loại bỏ nhiều phương án hơn cả bảng kết quả?
- Q4. Nếu bảng ablation nói bỏ khối A không mất gì và bỏ khối B cũng không mất gì, bạn có được kết luận cả hai đều vô dụng không?

3.1 Đặt vấn đề

Đứng trước một repo hoặc một paper lạ, làm sao đi từ “không hiểu gì” tới “biết nó giả định gì và hỏng ở đâu” trong một buổi, thay vì mò mẫm hàng tuần? Câu hỏi này nghe như một câu hỏi về kỹ năng đọc, nhưng thực ra nó là câu hỏi về **cách phân loại**. Người mới đọc hệ thống theo đặc điểm bề mặt: tên module, tên tầng, con số trong bảng kết quả. Người có kinh nghiệm đọc theo cấu trúc sâu: bài toán gì, giả định gì, đánh đổi gì. Hai cách đọc cho ra hai bản đồ hoàn toàn khác nhau về cùng một hệ thống, và chỉ bản đồ thứ hai dùng được khi bạn phải sửa hoặc thay một khối.

Sự khác biệt này không phải chuyện học thuật. Bạn đã gặp nó ở phía SLAM. Hai người cùng đọc ORB-SLAM3 [40] trong một buổi. Người thứ nhất ra về với một danh sách bộ phận: có ORB extractor, có DBoW2, có **hiệu chỉnh chùm tia** (*bundle adjustment*) cục bộ, có **đóng vòng** (*loop closing*). Người thứ hai ra về với một câu đánh cược: hệ này tin rằng góc ảnh còn lặp lại được giữa hai khung hình liên tiếp. Từ câu đó suy ra ngay hai chỗ sập. Ảnh mờ làm góc nhoe đi nên không còn khớp được; bề mặt trơn thì ngay từ đầu đã không có góc nào để tìm. Người thứ hai cũng biết hệ bù lại bằng một tầng **tái định vị** (*relocalization*) đắt tiền. Chỉ anh ta dự đoán được chuyện gì xảy ra trong một hành lang tường trắng. Danh sách bộ phận không sinh ra dự đoán; bản đồ giả định thì có.

Cái làm cho việc đọc trở nên khó là không gian thiết kế quá rộng. Một hệ thống thị giác hiện đại có hàng chục lựa chọn gắn kết nhau, và paper hầu như luôn kể câu chuyện theo trục “chúng tôi nghĩ ra ý tưởng đẹp này”. Trục đó gần như không bao giờ là trục thật đã sinh ra thiết kế. Trục thật thường là một ràng buộc: chỉ có bằng ấy nhân, chỉ có bằng ấy mili-giây, chỉ có bằng ấy bộ nhớ. Vì vậy chương này không dạy bạn đọc nhanh hơn; nó dạy bạn đọc theo đúng trục, để hầu hết nội dung tự rơi vào chỗ của nó.

TRỰC GIÁC

Đọc một hệ thống thị giác giống khám một bệnh nhân chứ không giống đọc một cuốn catalogue. Catalogue liệt kê bộ phận; khám thì hỏi *bộ phận này gánh chức năng gì, và nếu nó hỏng thì triệu chứng gì hiện ra ở đâu*. Một người thuộc lòng tên hai trăm chi tiết máy vẫn không sửa được xe; người biết “nếu bơm xăng yếu thì máy chết ở vòng tua cao” thì sửa được. Thước đo của cả chương là bạn có phát biểu được các triệu chứng hay không.

3.2 Bảy câu hỏi để mổ xẻ (Seven Questions for Dissection)

A. Vì sao thứ tự của các câu hỏi là thông tin

Thứ tự dưới đây không tùy tiện: mỗi câu hỏi chỉ trả lời được khi câu trước đã có đáp án, và mỗi câu sinh ra vì câu trước để lại một chỗ hở. Nói gọn thì mạch đi từ *bài toán* (câu 1–2) sang *không gian thiết kế bị ràng buộc* (câu 3–4) rồi sang *đóng góp thật và chỗ hỏng* (câu 5–7). Đọc lệch thứ tự thì bạn dành công sức cho những chi tiết mà ràng buộc cứng đàng nào cũng xoá bỏ.

Mạch câu hỏi khi mổ xẻ một hệ thống lạ

- 1 Vấn đề** không biết bắt đầu từ đâu, nên bắt đầu từ chỗ dễ nhìn nhất là code.

Giải pháp tách ba tầng **bài toán** → **nguyên lý** → **cài đặt**, và trước khi đọc một dòng code, ép mình viết ra đúng một câu cho mỗi tầng.

Hệ quả nó ngăn bạn nhầm chi tiết framework với ý tưởng cốt lõi. Cách họ viết **data-loader** hay đặt tên **checkpoint** là chi tiết. Vì sao **tín hiệu giám sát** (*supervision signal*) đến từ nguồn này chứ không nguồn khác mới là ý tưởng.

Giá phải trả mất nửa giờ viết ba câu trông tầm thường; đổi lại không mất ba ngày đọc nhầm tầng.
- 2 Vấn đề** “bài toán” vẫn là một khái niệm mơ hồ, ai cũng gặt đầu nhưng mỗi người hiểu một kiểu.

Giải pháp định nghĩa nó bằng đúng ba thứ — đầu vào, đầu ra, và tín hiệu giám sát. Mọi thứ còn lại là lựa chọn triển khai.

Giả định ba thứ này xác định bài toán đủ chặt để hai người khác nhau viết ra cùng một bản mô tả.

Hệ quả nếu chưa trả lời được cả ba thì đọc tiếp là vô ích; hãy quay về phần dataset, vì câu trả lời nằm ở đó chứ không ở phần method.
- 3 Vấn đề** không gian thiết kế quá rộng để suy luận bằng trực giác.

Giải pháp truy tìm **ràng buộc cứng** (*hard constraint*) trước tiên: **độ trễ** (*latency*), bộ nhớ, lượng nhân có được, quyền riêng tư, năng lượng.

Giả định tác giả không tự do như phần method làm ra vẻ.

Hệ quả ràng buộc cứng loại bỏ phần lớn phương án trước khi độ chính xác kịp lên tiếng, nên nó giải thích nhiều lựa chọn thiết kế hơn toàn bộ phần method cộng lại. Con số “khoảng 90%” hay được nhắc là kinh nghiệm thực hành chứ không phải kết quả đo được; nhưng chiều của nhận định thì rất vững.
- 4 Vấn đề** thiết kế nào cũng trông đẹp khi đọc trong paper.

Giải pháp áp nguyên lý **độ khó không biến mất, chỉ bị đẩy đi chỗ khác** và hỏi thẳng: họ đã đẩy nó đi đâu — sang dữ liệu, sang hậu xử lý, sang thời gian huấn luyện, hay sang một giả định về đầu vào?

Giả định mọi hệ thống đều trả giá ở đâu đó cho cái nó làm dễ đi ở chỗ khác.

Hệ quả câu trả lời gần như luôn tồn tại; nếu bạn không tìm ra thì kết luận đúng là bạn chưa hiểu hệ thống.
- 5 Vấn đề** không biết phần nào trong bài là đóng góp thật.

Giải pháp đọc bảng **ablation** theo chiều ngược — bỏ thành phần nào thì hệ thống sập, chứ không phải thêm gì thì con số nhích lên.

Giả định ablation được chạy đàng hoàng và **baseline** được tinh chỉnh ngang với phương pháp đề xuất.

Vấn đề mới giả định đó thường sai; có cả một dòng công trình cho thấy phần lớn “tiến bộ” trong một số lĩnh vực biến mất khi baseline được tinh chỉnh tử tế [41], [42]. Chương 24 dạy cách kiểm tra chính giả định này.

↪ **tiền đề E/24** — độ tin cậy của một bảng ablation là điều kiện để quy trình đọc này còn giá trị

6 Vấn đề

Giải pháp

Hệ quả

paper hầu như không nói về chỗ nó hỏng.

đọc phần *limitations* và *failure case* trước phần kết quả, vì đó là chỗ tác giả không có động cơ phóng đại.

bạn bước vào phần kết quả với một danh sách nghi vấn sẵn có, nên các con số đẹp không còn quyền định khung suy nghĩ của bạn. Nguyên tắc phía sau: hiểu một hệ thống nghĩa là biết nó hỏng ở đâu, không phải biết nó chạy được gì.

7 Vấn đề

Giải pháp

Hệ quả

lỗi thật khi triển khai thường không nằm trong model.

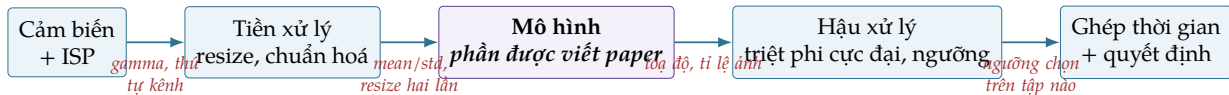
vẽ **sơ đồ luồng dữ liệu** (*data flow diagram*) đầy đủ, từ cảm biến tới quyết định cuối, và đánh dấu mọi đoạn nối giữa hai khối.

phần lớn lỗi hệ thống nằm ở các đoạn nối — sai thứ tự kênh màu, resize hai lần, đơn vị góc lệch nhau, *timestamp* lệch pha — chứ không nằm ở khối được viết paper [43]. Bất kỳ ai từng debug một pipeline SLAM đều đã sống qua điều này.

↪ **tiền đề A/02** — hai khối đầu của luồng — cảm biến và ISP — là vật lý tạo ảnh, và chúng quyết định phần lớn khoảng cách train/production

B. Luồng dữ liệu: chỗ paper im lặng

Câu hỏi thứ bảy đáng được vẽ ra chứ không chỉ nói suông, vì bản chất của nó là không gian: cái ta muốn thấy là tỉ lệ giữa phần được viết paper và phần còn lại của đường đi. Hình 3.1 cho thấy một luồng điển hình; khối tô đậm là khối duy nhất paper mô tả, và bốn mũi tên nối là nơi các lỗi tồn thời gian nhất sinh ra.



Hình 3.1. Luồng dữ liệu đầy đủ của một hệ thống thị giác. Paper mô tả một khối; các nhãn đỏ dưới mũi tên là những đoạn nối nơi lỗi triển khai thực tế hay sinh ra và không bao giờ xuất hiện trong bảng kết quả.

3.3 Ba nguồn thông tin: paper, code, ablation

Có đúng ba nguồn thông tin về một hệ thống lạ. Người mới thường chọn một nguồn theo thói quen — kỹ sư đọc code, sinh viên đọc paper — rồi tin rằng mình đã hiểu. Vấn đề là mỗi nguồn không chỉ *thiếu* thông tin, nó còn *che giấu* một loại thông tin cụ thể, và loại bị che luôn là loại bạn cần khi phải ra quyết định.

Bảng 3.1. Ba cách tiếp cận một hệ thống lạ, và cái mà mỗi cách che giấu. Cột “che giấu” là cột có giá trị nhất: nó nói cho bạn biết mình đang mù ở đâu.

Cách đọc	Cho bạn biết	Che giấu	Hỏng khi nào
Đọc paper	Ý định của tác giả, vị trí lịch sử của ý tưởng	Chi tiết thật sự quyết định kết quả: gán mẫu (<i>label assignment</i>), augmentation, lịch học	Khi câu chuyện được viết lại sau khi kết quả đã có
Đọc code	Sự thật về cái đang chạy, kể cả chỗ paper không kể	Vì sao chọn như vậy; đâu là thiết yếu, đâu là tàn dư lịch sử	Khi repo lớn tới mức bạn hiểu chi tiết mà mất tăng bài toán
Đọc ablation	Trọng số đóng góp thật của từng khối theo chính tác giả	Tương tác giữa các khối, chất lượng tinh chỉnh của baseline	Khi hai khối làm cùng một việc và che lẫn nhau

Bảng 3.1 cho thấy không nguồn nào đủ một mình, và quan trọng hơn, cho thấy cách phối hợp hiệu quả nhất: đọc ablation trước để biết phần nào đáng quan tâm, rồi đọc code của đúng phần đó, rồi mới quay lại paper để hiểu tác giả nghĩ gì — ngược hoàn toàn với thứ tự trang giấy. Lý do là ablation có tỉ lệ tín hiệu trên độ dài cao nhất, còn phần method có tỉ lệ thấp nhất vì nó được viết để thuyết phục người phản biện. Đây là dạng đọc chọn lọc mà Keshav mô tả dưới tên “đọc ba lượt” [44], chỉ khác là ta chọn điểm vào theo ablation thay vì theo phần mở đầu.

Cột cuối của bảng đáng được nói rõ hơn ở một điểm: ablation cho biết *đóng góp biên* của một khối khi các khối còn lại đã có mặt, chứ không cho biết giá trị tuyệt đối của nó. Hai khối làm cùng một việc sẽ che nhau — bỏ khối nào đi một mình cũng không mất gì, nên bảng kết luận cả hai đều vô dụng, trong khi bỏ cả hai thì hệ thống sập. Chuyện này xảy ra thường xuyên và giải thích vì sao ablation không thay được việc đọc code.

3.4 Độ khó không biến mất (Conservation of Difficulty)

A. Nguyên lý và ba ví dụ

Không có thiết kế nào làm cho một bài toán dễ đi; chỉ có thiết kế di chuyển phần khó sang một trục mà bạn sẵn lòng trả giá hơn. Nói gọn thì mỗi “cải tiến” là một phép đổi tiền tệ, và việc của bạn khi đọc là tìm ra tỉ giá.

• Ba chỗ độ khó hay bị đẩy tới, kèm ví dụ cụ thể:

- **Sang thời gian huấn luyện.** Mô hình *one-stage* bỏ bước sinh đề xuất vùng nên chạy nhanh hơn. Nhưng vẫn phải quyết định vị trí nào trên ảnh chịu trách nhiệm cho vật nào. Câu hỏi đó không biến mất, nó chỉ đổi tên thành bài toán gán mẫu và kéo theo cả một dòng heuristic [45], [46]. DETR bỏ được **triệt phi cực đại** (*non-maximum suppression*) và **anchor**, đổi lại số epoch cần để hội tụ tăng vọt [47].
- **Sang giả định về đầu vào.** SLAM trực tiếp bỏ được bước trích đặc trưng, đổi lại phải giả định **độ sáng không đổi** (*brightness constancy*) giữa các khung hình và chấp nhận một miền hội tụ hẹp hơn nhiều. Đây chính là trục *giả định mạnh với ít dữ liệu* đổi lại *giả định yếu với nhiều dữ liệu* mà chương 4 sẽ đặt tên.

→ cùng nguyên lý với B/04 — “độ khó không biến mất là phát biểu định tính của cùng một trục mà chương 4 định lượng

- **Sang dữ liệu và hậu xử lý.** Một hệ thống tự nhận là “đơn giản, không cần module X” thường đã chuyển gánh nặng sang chỗ khác. Chỗ đó có thể là một tập dữ liệu lớn hơn

nhiều lần. Cũng có thể là một chuỗi luật lọc đầu ra chiếm nửa số dòng trong repo mà paper không nhắc tới.

B. Ngân sách mili-giây: một phép tính hai dòng

Cách kiểm chứng nguyên lý này bằng số học rất đơn giản: lập một ngân sách và xem con số nào phải nhường chỗ. Giả sử camera chạy 30 Hz, tức mỗi khung hình có $1000/30 \approx 33$ ms. Nếu bạn dành 4 ms cho tiền xử lý, 3 ms cho hậu xử lý và 6 ms cho phần ghép thời gian thì mô hình chỉ còn

$$33 - (4 + 3 + 6) = 20 \text{ ms},$$

và sau khi chừa biên an toàn thì thực tế là khoảng 16 ms. Công thức chỉ nói một điều: *ngân sách của mô hình là phần dư sau khi mọi khối khác đã lấy phần của chúng*, nên nó luôn nhỏ hơn con số bạn tưởng. Bây giờ đọc một paper báo cáo 45 ms trên GPU máy chủ với **batch** bằng 8: con số đó vừa vượt ngân sách hơn hai lần, vừa được đo ở chế độ batch mà bạn không bao giờ có vì bạn xử lý từng khung hình đến. Phép tính hai dòng này loại bỏ cả một họ phương pháp nhanh hơn bất kỳ bảng mAP nào. Các con số trong ví dụ là do tôi đặt ra để minh họa cách tính, không trích từ một hệ thống cụ thể.

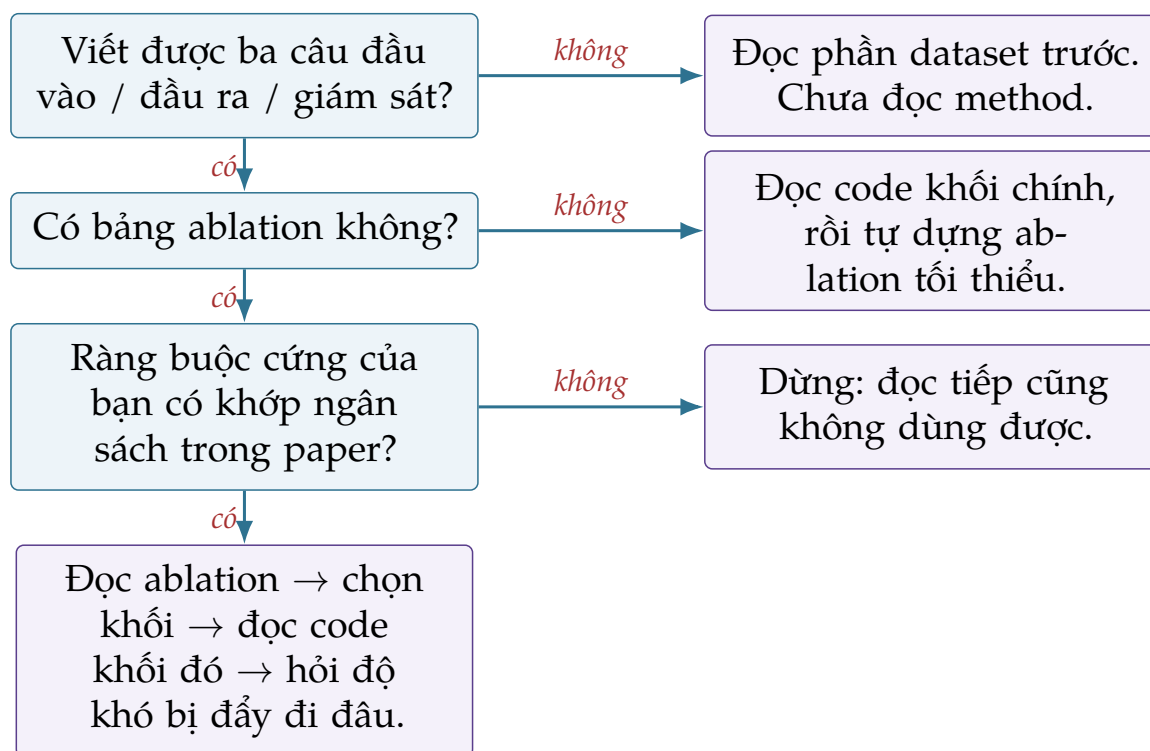
↔ **ràng buộc bởi B/11** — ngân sách mili-giây ở đây là dạng thô sơ của mô hình roofline, nơi ràng buộc thật là băng thông bộ nhớ chứ không phải số phép tính

3.5 Ba quy tắc quyết định

Bảy câu hỏi ở trên là quy trình; ba quy tắc dưới đây là các điểm dừng — những chỗ bạn được phép kết luận mà không cần đọc thêm. Giá trị của chúng nằm ở chỗ mỗi quy tắc cho phép bạn đóng một cuốn paper sớm.

1. **Chưa viết được ba câu đầu vào / đầu ra / tín hiệu giám sát?** Dừng đọc phần method, quay lại phần dataset. Đọc tiếp trong trạng thái đó chỉ nạp vào đầu các chi tiết không có chỗ để gắn.
2. **Nếu ràng buộc độ trễ siết đi năm lần thì khối nào bị thay đầu tiên?** Trả lời được câu này nghĩa là bạn đã hiểu kiến trúc theo nghĩa dùng được. Nó đòi hỏi hai thứ: biết chi phí tính toán phân bố ra sao, và biết khối nào có phương án thay thế rẻ hơn.
3. **Paper không có phần limitations?** Coi đó là tín hiệu cảnh báo về mức độ trung thực của phần còn lại, không phải dấu hiệu hệ thống hoàn hảo. Mọi hệ thống thật đều có chế độ hồng; im lặng về chúng là một lựa chọn biên tập.

Hình 3.2 gói ba quy tắc và mạch bảy câu hỏi thành một cây quyết định dùng được ngay khi mở một repo.



Hình 3.2. Cây quyết định khi mở một hệ thống lạ. Ba nút hỏi tương ứng ba quy tắc dừng; chỉ khi cả ba đều thoả thì việc đọc sâu mới có lãi.

3.6 Thực hành

Quy trình bảy câu hỏi chỉ trở thành phản xạ khi bạn chạy nó trên mã nguồn thật vài lần.

Repo và tài nguyên. Ba loại repo đáng mổ xẻ, theo thứ tự tăng dần độ phức tạp:

- **Tối giản, đọc hết trong một buổi** — [karpathy/minGPT](#), [lucidrains/vit-pytorch](#). Dùng để luyện tách ba tầng khi không có nhiều.
- **Production đầy đủ tầng** — [ultralytics/ultralytics](#), [open-mmlab/mmdetection](#), [facebookresearch/detectron2](#). Dùng để thấy một hệ thống thật được tổ chức ra sao.
- **Hệ thống nhiều khối ghép** — [nerfstudio-project/nerfstudio](#), [colmap/colmap](#). Dùng để thấy các đoạn nối trong Hình 3.1 mới là phần khó.

Danh sách tên mô hình dẫn đầu bảng chốt tại tháng 9/2026 và sẽ cũ trong sáu đến mười hai tháng; repo hạ tầng như COLMAP hay [timm](#) bền hơn nhiều, nên hãy chia thời gian theo tỉ lệ đó.

Câu hỏi kiểm tra hiểu. Trong repo bạn chọn, ranh giới giữa “bài toán”, “nguyên lý” và “chi tiết cài đặt” nằm ở đâu — chỉ đúng ba file hoặc ba hàm đánh dấu ranh giới đó. Độ khó của bài toán đã bị đẩy đi đâu, và bạn chứng minh bằng dòng code nào?

Mini project. Chọn một repo và viết *system datasheet* dài một trang: đầu vào, đầu ra, tín hiệu giám sát, ràng buộc cứng, ba giả định ngầm, ba chế độ hỏng. Ràng buộc của bài tập là không đọc paper, chỉ đọc code — đây chính là chỗ bạn phát hiện mình vẫn đang suy đoán từ tên biến thay vì từ hành vi. Vài giờ đến hai ngày.

3.7 Giới hạn và trường hợp hỏng

Quy trình này có ba chỗ dễ gây, và biết trước chúng thì bạn không tin nó quá mức.

- **Nó giả định ablation chạy đằng hoàng** — một giả định thường sai. Ablation có thể chạy trên một cấu hình, một *seed*; trong học tăng cường người ta đã cho thấy phương sai giữa các seed đôi khi lớn hơn khoảng cách giữa các phương pháp [48]. Kinh nghiệm benchmark SLAM của bạn nói cùng một điều: một hệ đa luồng cho hai kết quả khác nhau trên cùng một bag, nên một dòng trong bảng ablation không phải một phép đo.
- **Nó tối ưu cho việc hiểu, không cho việc tái hiện.** Rất nhiều hệ thống chỉ đạt con số đã công bố nhờ những chi tiết không được coi là ý tưởng: độ dài lịch học, cách khởi tạo, một phép chuẩn hoá đặt đúng chỗ. Bạn có thể hiểu hoàn hảo một hệ thống mà vẫn không tái hiện được nó.
- **Nó không áp dụng tốt cho công trình mà đóng góp chính là dữ liệu hoặc phép đo.** Với một paper giới thiệu benchmark, bảy câu hỏi phần lớn trả về câu trả lời rỗng; khung đọc đúng là khung của chương 7 và chương 24 — tập này lấy mẫu từ đâu, chia thế nào, và metric thường cho hành vi gì.

CẠM BÃY

Cạm bẫy phổ biến nhất là nhầm “tôi đọc hiểu từng dòng code” với “tôi hiểu hệ thống”. Đọc hết code cho bạn tăng cài đặt ở độ phân giải rất cao và tăng bài toán ở độ phân giải bằng không. Dấu hiệu nhận biết: bạn mô tả được hệ thống làm gì theo trình tự thời gian, nhưng không nói được điều gì sẽ hỏng nếu ảnh đầu vào tối đi hai *stop*. Nếu rơi vào trạng thái đó, đừng đọc thêm code — hãy quay lại câu hỏi số hai.

CẦN NHỚ

Hiểu một hệ thống nghĩa là biết nó *hỏng ở đâu*, không phải biết nó chạy được gì. Quy trình gọn nhất: viết ba câu đầu vào / đầu ra / giám sát → tìm ràng buộc cứng → đọc ablation ngược → hỏi độ khó đã bị đẩy đi đâu. Nếu chưa tìm ra chỗ độ khó bị đẩy tới, bạn chưa đọc xong. [T2]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Bài toán, nguyên lý, cài đặt	Ba tầng để tách một hệ thống ra trước khi đọc dòng code đầu tiên. Viết đúng một câu cho mỗi tầng thì bạn không còn nhầm một chi tiết framework với một ý tưởng cốt lõi.
••• Tín hiệu giám sát	Thứ nói cho thuật toán biết dự đoán đúng là gì: nhãn người gán, một ràng buộc vật lý, một cảm biến khác, hoặc chính dữ liệu. Biết nó đến từ đâu là biết hệ thống học được gì và không học được gì.
••• Ràng buộc cứng	Giới hạn mà tác giả không thể vượt qua: độ trễ, bộ nhớ, lượng nhân, quyền riêng tư, năng lượng. Nó loại bỏ phần lớn phương án trước khi độ chính xác kịp lên tiếng.
•• Vì sao ràng buộc cứng giải thích thiết kế nhiều hơn cả phần method?	Vì nó là ràng buộc loại trừ: nó xoá cả một họ phương án bất kể chúng chính xác tới đâu. Phần method chỉ xếp hạng những gì đã lọt qua.
••• Độ khó không biến mất	Nguyên lý trung tâm: không thiết kế nào làm bài toán dễ đi, nó chỉ đẩy phần khó sang dữ liệu, hậu xử lý, thời gian huấn luyện, hoặc một giả định về đầu vào. Không tìm ra chỗ bị đẩy tới nghĩa là chưa hiểu hệ thống.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
• Gán mẫu	Bước quyết định vị trí nào trên ảnh chịu trách nhiệm cho vật nào. Nó sinh ra đúng lúc mô hình một giai đoạn bỏ bước sinh đề xuất vùng, và là ví dụ điển hình của độ khó bị đẩy đi chỗ khác.
•• Ablation	Thí nghiệm bỏ đi từng thành phần rồi đo lại, để biết thành phần nào thực sự đóng góp. Cách đọc đúng là đọc ngược: bỏ cái gì thì hệ thống sập.
•• Đóng góp biên	Cái mà ablation thực sự đo: mức mất mát khi bỏ một khối trong khi các khối còn lại vẫn có mặt. Nó không phải giá trị tuyệt đối của khối đó.
••• Bỏ khối A không mất gì, bỏ khối B cũng không mất gì. Kết luận?	Không được kết luận cả hai vô dụng. Rất có thể chúng làm cùng một việc và che nhau; phép thử phân biệt là bỏ đồng thời cả hai, nếu hệ thống sập thì chúng dư thừa lẫn nhau chứ không thừa.
• Baseline	Phương pháp đối chứng đơn giản hơn mà phương pháp mới phải vượt qua. Nếu nó không được tinh chỉnh cẩn thận thì mọi so sánh với nó đều vô giá trị.
•• Vì sao đọc ablation trước rồi mới đọc code?	Ablation có tỉ lệ tín hiệu trên độ dài cao nhất, còn phần method thấp nhất vì nó được viết để thuyết phục người phản biện. Đọc ablation để chọn khối đáng quan tâm, rồi chỉ đọc code của khối đó.
•• Vì sao đọc limitations trước phần kết quả?	Vì đó là chỗ tác giả không có động cơ phóng đại. Bạn bước vào bảng số với một danh sách nghi vấn sẵn có, nên con số đẹp không định khung suy nghĩ của bạn.
• Paper không có mục limitations	Đọc như một tín hiệu cảnh báo về mức trung thực của phần còn lại, không phải dấu hiệu hệ thống hoàn hảo. Mọi hệ thống thật đều có chế độ hỏng; im lặng về chúng là một lựa chọn biến tập.
•• Sơ đồ luồng dữ liệu	Đường đi đầy đủ từ cảm biến tới quyết định cuối, có đánh dấu mọi đoạn nối. Vẽ nó ra là cách nhanh nhất thấy phần được viết paper chỉ chiếm một khối trong năm.
•• Đoạn nối	Chỗ ghép giữa hai khối: thứ tự kênh màu, chuẩn hoá, tỉ lệ toạ độ, timestamp. Phần lớn lỗi triển khai nằm ở đây chứ không nằm trong mô hình.
•• Ngân sách độ trễ	Thời gian cho mỗi khung hình, chia cho từng khối. Ngân sách của mô hình luôn là phần dư sau khi các khối khác đã lấy phần của chúng.
•• Vì sao camera 30 Hz chỉ chừa cho mô hình khoảng 16 ms?	Mỗi khung hình có 33 ms. Trừ 4 ms tiền xử lý, 3 ms hậu xử lý và 6 ms ghép thời gian còn 20 ms, rồi phải chừa biên an toàn.
•• System datasheet	Bản một trang mô tả một hệ thống: đầu vào, đầu ra, tín hiệu giám sát, ràng buộc cứng, ba giả định ngầm, ba chế độ hỏng. Viết được nó nghĩa là bạn đã đọc xong hệ thống.
•• Bạn kể được hệ thống làm gì theo trình tự nhưng không nói được nó hỏng khi nào	Bạn có tăng cài đặt ở độ phân giải rất cao và tăng bài toán bằng không. Phép thử: hỏi điều gì xảy ra nếu ảnh tối đi hai stop; trả lời không được thì quay lại ba câu đầu vào, đầu ra, giám sát.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Datasheets for datasets, model cards	Chính là “system datasheet” của chương này được chuẩn hoá thành tài liệu bắt buộc kèm mô hình và tập dữ liệu	Thông lệ ở nhiều nhóm lớn từ 2019, ít hết hạn

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
mmdetection , detectron2	Tách ba tầng bài toán / nguyên lý / cài đặt thành cấu hình, registry và module — đọc chúng là bài tập tự nhiên cho quy trình bày câu hỏi	Hạ tầng, bền
colmap/colmap	Hệ thống nhiều khối ghép, nơi các đoạn nối — hiệu chuẩn, thứ tự ảnh, ngưỡng khớp — quyết định kết quả hơn bất kỳ khối đơn lẻ nào	Hạ tầng, bền
Dòng DETR thời gian thực	Ví dụ sống của “độ khó bị đẩy đi chỗ khác”: bỏ triết phi cực đại và anchor, đổi lại chi phí hội tụ và cả một dòng nghiên cứu về thiết kế query	Tính tới 9/2026

MINI PROJECT SLAM · System datasheet cho ORB-SLAM3, chỉ đọc code · 1–2 ngày

Mục tiêu	biến quy trình bày câu hỏi thành một sản phẩm kiểm tra được, trên chính hệ thống bạn chạy hằng ngày. Bài tập sẽ lộ ra những chỗ bạn tưởng mình hiểu mà thực ra chỉ đang suy đoán từ tên biến.
Dữ liệu	mã nguồn ORB-SLAM3 [40] và một cấu hình EuRoC đang chạy được. Ràng buộc của bài tập: <i>không mở paper</i> , chỉ đọc code và file cấu hình.
Các bước	viết một trang gồm đúng sáu phần — (1) đầu vào, đầu ra, tín hiệu giám sát; (2) ràng buộc cứng đọc được từ code, ví dụ số đặc trưng tối đa, số mức kim tự tháp, ngưỡng số điểm khớp để coi là bám được; (3) sơ đồ luồng dữ liệu từ ảnh tới pose, đánh dấu mọi đoạn nối; (4) ba giả định ngầm, mỗi giả định kèm dòng code chứng minh; (5) ba chế độ hỏng suy ra từ ba giả định đó; (6) độ khó đã bị đẩy đi đâu.
Xong khi	bạn trả lời được bằng văn bản câu “nếu ràng buộc độ trễ siết đi năm lần thì khối nào bị thay đầu tiên”. Câu trả lời phải kèm phần trăm thời gian của từng khối, đo bằng bộ đếm bạn tự chèn chứ không bằng cảm giác. Kiểm tra chéo: ba chế độ hỏng bạn nêu phải tái hiện được ít nhất một cái trên dữ liệu thật (ví dụ làm mờ ảnh đầu vào hoặc giảm số đặc trưng) và quan sát đúng triệu chứng bạn đã dự đoán.
Nối với	B/05 dùng lại datasheet này khi thiết kế bài toán học máy đầu tiên gắn vào hệ; B/11 thay ước lượng phần trăm thời gian bằng mô hình roofline; E/24 cho cách đọc con số đo được khi harness có thành phần ngẫu nhiên.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì hai người đọc theo hai cách phân loại khác nhau: người thứ nhất theo đặc điểm bề mặt (tên module, tên tầng), người thứ hai theo cấu trúc sâu (bài toán, giả định, đánh đổi). Chỉ bản đồ giả định mới sinh ra dự đoán về triệu chứng.
- A2. Vì phần method có tỉ lệ tín hiệu trên độ dài thấp nhất — nó được viết để thuyết phục người phân biện — trong khi ablation và limitations nói thẳng phần nào đóng góp và phần nào hỏng. Đọc ablation trước để chọn điểm vào, rồi đọc code của đúng phần đó.
- A3. Vì ràng buộc cứng là ràng buộc loại trừ: nó xoá cả một họ phương án bất kể chúng chính xác tới đâu, còn bảng kết quả chỉ xếp hạng trong số những phương án đã lọt qua. Thêm nữa, 45 ms đó thường được đo ở chế độ batch mà hệ thống thời gian thực của bạn không bao giờ có.
- A4. Không. Ablation đo đóng góp biên khi các khối còn lại đã có mặt, nên hai khối dư thừa lẫn nhau sẽ cùng cho ra “bỏ đi không mất gì” trong khi bỏ cả hai thì hệ thống sập. Đây là lý do ablation không thay được việc đọc code.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn ngồi ăn ở một quán cơm nhỏ và muốn hiểu vì sao thực đơn của họ đúng như vậy. Bạn có thể hỏi chủ quán, và được nghe một câu chuyện đẹp về công thức gia truyền. Bạn có

thể xin vào bếp xem họ nấu, và thấy chính xác những gì đang xảy ra nhưng vẫn không hiểu vì sao. Nhưng nếu bạn đo cái bếp — chỉ có hai bếp lửa, một tủ lạnh nhỏ, và khách phải có đồ ăn trong tám phút — thì gần như cả thực đơn tự giải thích. Món nào cần hầm ba tiếng thì phải hầm sẵn từ sáng: công sức không mất đi, nó chỉ chuyển sang lúc khác.

Đọc một hệ thống máy tính lạ cũng vậy. Bài toán gốc là bạn có vài trăm nghìn dòng mã cùng một bài báo dài, còn thời gian thì chỉ một buổi. Mẹo là đừng bắt đầu từ ý tưởng, hãy bắt đầu từ những giới hạn mà người làm ra nó không thể vượt qua: máy phải trả lời trong bao nhiêu phần nghìn giây, có sẵn bao nhiêu dữ liệu đã được đánh dấu, bộ nhớ bao nhiêu. Những giới hạn ấy gạt bỏ phần lớn cách làm khác trước khi chuyện hay dở kịp được bàn tới.

Cái giá phải trả là bạn buộc phải đọc kỹ đúng phần trông nhàm chán nhất, và để dành phần hấp dẫn nhất tới cuối.

Chỗ không rút gọn được: chuyện “bỏ bộ phận nào ra thì máy vẫn chạy, nên bộ phận đó vô dụng” thì không rút gọn được mà vẫn đúng. Nếu hai bộ phận cùng làm một việc, bỏ riêng từng cái đều không thấy mất gì, mà bỏ cả hai thì máy đứng. Mọi cách nói ngắn hơn điều này đều dẫn tới kết luận sai.

Một câu: Muốn hiểu một cái máy lạ, hãy đo cái hộp mà người ta buộc phải nhét nó vào, đừng nghe câu chuyện về nó.

3.8 Kết nối

Chương này là tiền đề của cả Phần B; mỗi chương sau là một lát cắt chi tiết của một trong bảy câu hỏi.

- **Câu 2 (đầu vào / đầu ra / giám sát)** → [B/05-thiet-ke-bai-toan](#): ở đó việc chọn đơn vị phân tích và dạng đầu ra khoá chặt mọi thứ phía sau.
- **Câu 3 (ràng buộc cứng)** → [B/11-gioi-han-tinh-toan](#): ngân sách mili-giây trong ví dụ trên được thay bằng mô hình *roofline* nghiêm túc [49].
- **Câu 4 (độ khó bị đẩy đi đâu)** → [B/04-nen-tang-hoc-tu-du-lieu](#): ở đó nó mang tên chính thức là trục đánh đổi giả định ↔ dữ liệu.
- **Câu 5 (ablation đáng tin không)** → chương 24 ở Phần E.
- **Hai khối đầu của Hình 3.1** → [A/02-vat-ly-tao-anh](#), tiền đề vật lý của chúng.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao ràng buộc cứng loại bỏ nhiều phương án thiết kế hơn cả phần method cộng lại, và điều đó gợi ý gì về thứ tự đọc một paper?
2. Trong một repo bạn đã đọc, độ khó đã bị đẩy đi đâu — và bạn kiểm chứng bằng bằng chứng nào trong code?
3. Bảng ablation che giấu điều gì mà việc đọc code lộ ra, và trong tình huống cụ thể nào thì nó kết luận sai?
4. Vì sao nên đọc phần limitations trước phần kết quả, chứ không phải sau?
5. Nếu ngân sách độ trễ bị siết từ 33 ms xuống 8 ms mỗi khung hình, khối nào trong Hình 3.1 bị thay đầu tiên, và vì sao không phải khối mô hình?
6. Quy trình bảy câu hỏi hổng ở loại công trình nào, và khung đọc nào thay thế nó ở đó?

Nền tảng học từ dữ liệu (Foundations of Learning from Data)

PHỤ THUỘC B/03-giai-phau-he-thong cho khung “bài toán → nguyên lý → cài đặt”. Nguyên lý “độ khó không biến mất” ở chương đó, tại đây có tên chính thức là trục đánh đổi giả định ↔ dữ liệu. Nền toán: xác suất cơ bản và chuẩn vector.

TÓM TẮT

Chương này giải thích khe hở duy nhất mà từ đó mọi vấn đề của học máy sinh ra: ta tối ưu một trung bình trên mẫu hữu hạn nhưng muốn tốt trên cả phân phối thật. Nó đi từ *ERM* qua thất bại của *KNN* trên pixel, qua **lời nguyền số chiều** (*curse of dimensionality*) và **giả thuyết đa tạp** (*manifold hypothesis*), tới bốn nguồn của *inductive bias*, tới *double descent*, và kết ở *identifiability*. Đọc xong bạn trả lời được câu hỏi đắt tiền nhất trong mọi dự án, bằng phép đo chứ không bằng cảm tính: mô hình chưa đủ tốt thì nên thêm dữ liệu, thêm giả định, hay thêm **sức chứa** (*capacity*)? Bạn cũng nhận ra được những bài toán mà thêm dữ liệu chỉ là ném tiền qua cửa sổ. Nếu chỉ nhớ một điều: bạn không chọn được “không có giả định”, bạn chỉ chọn giả định nào, và trục duy nhất là *giả định mạnh với ít dữ liệu* đối lại *giả định yếu với nhiều dữ liệu*.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình, huấn luyện, suy luận** — mô hình là một hàm f có tham số, nhận đầu vào x và trả về dự đoán $f(x)$. Huấn luyện là đi tìm bộ tham số làm dự đoán sát nhất trên dữ liệu đã có; suy luận là chạy hàm đó trên dữ liệu mới. Cả chương này nói về khoảng cách giữa hai việc đó.
- **phân phối dữ liệu và giả định i.i.d.** — “phân phối” là quy luật xác suất sinh ra dữ liệu bạn sẽ gặp — ảnh nào hay xuất hiện, nhãn nào đi kèm. Giả định i.i.d. (*independent and identically distributed*) nói rằng mỗi mẫu được rút độc lập từ cùng một phân phối; đây là giả định ngầm của gần như mọi kết quả trong chương.
- **hàm mất mát và rủi ro (loss, risk)** — hàm mất mát ℓ đo một dự đoán sai bao nhiêu so với nhãn của một mẫu. Rủi ro là kỳ vọng của hàm mất mát trên toàn phân phối — tức mức sai trung bình trên thế giới thật. Ta muốn tối thiểu rủi ro nhưng chỉ đo được trung bình trên mẫu đã có.
- **tham số và sức chứa (capacity)** — tham số là các con số bên trong mô hình mà huấn luyện điều chỉnh. Sức chứa là mức độ phong phú của tập hàm mà mô hình biểu diễn được — càng nhiều tham số và càng ít ràng buộc thì sức chứa càng lớn, và mô hình càng dễ khớp mọi thứ, kể cả nhiễu.
- **overfitting và underfitting** — overfitting là khi mô hình khớp rất tốt dữ liệu huấn luyện nhưng kém trên dữ liệu mới — nó học thuộc chi tiết riêng của mẫu. Underfitting là khi nó kém ở cả hai, vì sức chứa hoặc thời gian huấn luyện chưa đủ. Phân biệt hai trạng thái này quyết định bạn nên thêm dữ liệu hay thêm mô hình.
- **bias** — ba nghĩa khác nhau — (a) trong thống kê, độ lệch hệ thống giữa kỳ vọng của bộ ước lượng và giá trị thật — đây là nghĩa dùng trong cụm “bias-variance”; (b) *inductive bias*, tập giả định mà mô hình mang sẵn về dạng lời giải, nghĩa dùng nhiều nhất trong chương này; (c) tham số cộng thêm b trong $Wx + b$ của một tầng tuyến tính — nghĩa này *không* xuất hiện ở đây. Khi đọc, luôn hỏi đang là nghĩa nào.
- **vector, chuẩn L_2 , số chiều** — một ảnh xám 32×32 được xem như một vector 1024 chiều bằng cách xếp các pixel thành một dãy số. Chuẩn L_2 của hiệu hai vector, $\|A - B\|_2 = \sqrt{\sum_i (A_i - B_i)^2}$, là

khoảng cách Euclid quen thuộc mở rộng lên nhiều chiều — “số chiều” ở đây chỉ đơn giản là số pixel.

- **SGD và batch** — thuật toán huấn luyện phổ biến nhất là *stochastic gradient descent*: lặp đi lặp lại việc lấy một nhóm nhỏ mẫu (một *batch*), tính hướng làm giảm hàm mất mát trên nhóm đó, rồi dịch tham số một bước theo hướng ngược lại. Chương này chỉ cần bạn biết rằng lựa chọn thuật toán tối ưu cũng là một giả định, không cần biết chi tiết cơ chế.

CÂU HỎI MỞ ĐẦU

- Q1. Nếu KNN trên pixel thất bại thảm hại còn KNN trên đặc trưng DINOv3 lại rất mạnh, mà thuật toán phân loại không đổi một dòng, thì “trí tuệ” của hệ thống thực ra nằm ở đâu?
- Q2. Một ảnh xám trông rõ ràng lại gần một cạnh dọc hơn chính cạnh đó dịch một pixel — vậy thứ gì đã hỏng trong khái niệm “khoảng cách”?
- Q3. Giáo trình nói tăng sức chứa thì sai số kiểm thử phải xấu đi. Vì sao những mạng có nhiều tham số hơn cả số mẫu vẫn tổng quát hoá tốt?
- Q4. Có bài toán nào mà gán nhãn thêm một triệu ảnh cũng không cải thiện được gì không, và làm sao biết trước điều đó thay vì biết sau khi đã tiêu tiền?

4.1 Đặt vấn đề

Bạn tối ưu trên một tập hữu hạn nhưng muốn hệ thống hoạt động tốt trên thế giới. Khoảng cách đó lấp bằng gì? Câu hỏi nghe hiển nhiên tới mức dễ bị bỏ qua, nhưng nó là khe hở duy nhất mà từ đó *mọi* vấn đề của ngành sinh ra: *overfitting*, *domain gap*, *shortcut learning*, mô hình tốt trên *benchmark* mà vô dụng khi triển khai. Hãy hiểu chính xác khe hở này rộng bao nhiêu và cái gì thu hẹp nó lại. Khi đó phần lớn kỹ thuật ở Phần C và Phần D không còn là một danh sách mẹo rời rạc. Chúng trở thành các câu trả lời khác nhau cho cùng một câu hỏi.

Chương này khác chương 3 ở chỗ nó không dạy cách đọc mà dạy cách *dự đoán*: vì sao một giả định lại giúp ích, và trong hoàn cảnh nào chính nó quay ra làm hại. Đó là điều kiện cần để trả lời câu hỏi thực dụng nhất trong mọi dự án — mô hình chưa đủ tốt, nên thêm dữ liệu, thêm giả định, hay thêm sức chứa?

Có một lý do riêng để một kỹ sư SLAM đọc kỹ chương này. Trong hình học đa tầm nhìn, bạn biết chính xác khi nào bài toán suy biến vì **ma trận thông tin** (*information matrix*) mất hạng. Học từ dữ liệu không cho sự rõ ràng đó miễn phí. Nhưng khái niệm identifiability ở cuối chương chính là bản dịch của trực giác “suy biến” sang ngôn ngữ thống kê. Nó cho bạn quyền nói “thêm dữ liệu sẽ không cứu được” bằng một lập luận, thay vì bằng một linh cảm.

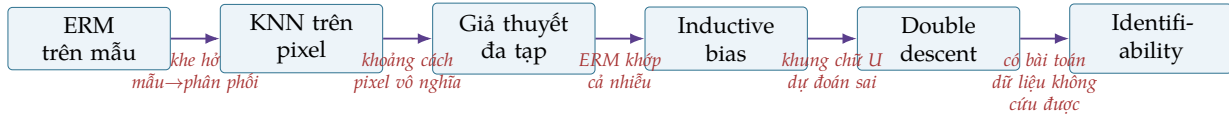
TRỰC GIÁC

Huấn luyện là chụp ảnh một địa hình từ vài trăm điểm quan sát rồi dựng lại toàn bộ bản đồ. Dữ liệu cho bạn các điểm; inductive bias là giả thiết về địa hình *giữa* các điểm — nó phẳng, nó liên tục, nó không đổi khi dịch ngang. Không có giả thiết nào thì có vô hạn địa hình đi qua đúng những điểm ấy và bạn không chọn được cái nào; giả thiết sai thì bạn dựng ra một bản đồ trơn tru, tự tin, và sai ở đúng những chỗ bạn chưa nhìn tới.

4.2 Từ ERM tới no free lunch (From ERM to No Free Lunch)

A. Bản đồ của mạch trước khi đi vào từng bước

Mạch dưới đây có tám bước, và mỗi bước tồn tại vì bước trước để lại đúng một nút thắt. Hình 4.1 cho thấy chuỗi nút thắt đó trước, để bạn đọc từng bước với bối cảnh sẵn có thay vì phải chờ tới cuối mới thấy hình dạng tổng thể.



Hình 4.1. Chuỗi nút thắt của chương. Mỗi khái niệm ra đời để gỡ nút thắt ghi dưới mũi tên đi vào nó; đọc theo chiều này thì không khái niệm nào là tùy tiện.

B. Tám bước

Mạch từ ERM tới no free lunch

- 1 Vấn đề** cái ta muốn tối thiểu thì không đo được. Ta muốn tối thiểu **rủi ro kỳ vọng** (*expected risk*) $R(f) = \mathbb{E}_{(x,y) \sim P} [\ell(f(x), y)]$ trên phân phối thật P , nhưng chỉ có n mẫu rút từ nó.

Giải pháp **tối thiểu hoá rủi ro thực nghiệm** (*empirical risk minimization*): tối thiểu $\hat{R}(f) = \frac{1}{n} \sum_i \ell(f(x_i), y_i)$ và hy vọng nó xấp xỉ $R(f)$ [50]. Công thức chỉ nói một điều: *ta thay một tích phân không tính được bằng một trung bình cộng đếm được*.

Giả định mẫu đại diện cho phân phối, và phân phối lúc triển khai vẫn là P .

Giá phải trả mọi vấn đề tổng quát hoá của ngành sinh ra từ khe hở giữa $\hat{R}$ và R .
 ↪ **đối lập với B/08** — chương dịch chuyển phân phối bỏ đúng giả định i.i.d. mà ERM dựa vào, nên mọi cận ở đây mất hiệu lực ở đó
- 2 Vấn đề** liệu bộ phân loại có phải chỗ đáng đầu tư không?

Giải pháp chạy thí nghiệm mở màn rẻ nhất: **k láng giềng gần nhất** (*k-nearest neighbours*) trên pixel thô, với khoảng cách L_1 hoặc L_2 . Nó không huấn luyện gì và đồn toàn bộ chi phí sang lúc suy luận. Trên ảnh, nó thất bại thảm hại.

Hệ quả bài học không phải “KNN dở”, vì cùng thuật toán ấy chạy trên đặc trưng của một mô hình **tự giám sát** (*self-supervised*) lại rất mạnh [51], [52]. Thất bại nằm ở **biểu diễn** (*representation*): khoảng cách pixel không tương ứng với khoảng cách ngữ nghĩa, và toàn bộ deep learning cho thị giác là nỗ lực học một không gian mà ở đó khoảng cách mới có nghĩa.
- 3 Vấn đề** vì sao khoảng cách pixel lại vô nghĩa đến vậy?

Giải pháp lời nguyên số chiều: trong không gian nhiều nghìn chiều, thể tích tập trung ở vỏ ngoài và mọi cặp điểm gần như cách đều nhau, nên “láng giềng gần nhất” mất phần lớn nội dung thông tin.

Vấn đề mới nếu vậy thì học trong không gian đó lẽ ra phải bất khả thi, mà thực tế thì không.

Giải pháp giả thuyết đa tạp: dữ liệu thật không rải đều khắp không gian mà nằm gần một đa tạp số chiều thấp hơn nhiều nhúng trong đó [53].
- 4 Vấn đề** ERM một mình sẽ khớp cả nhiễu. Một mạng đủ lớn học thuộc được cả nhãn ngẫu nhiên, nên đặt $\hat{R} = 0$ không nói gì về R [54].

Giải pháp thêm **inductive bias** — giả định về lời giải, thứ thu hẹp không gian hàm *trước khi* dữ liệu lên tiếng.

Giả định giả định đó đúng với bài toán của bạn.

- Hệ quả nhận thức then chốt: bạn không thể “không có giả định”; bạn chỉ chọn giả định nào, và việc không nói ra không làm nó biến mất.
- 5 Vấn đề** nên đặt bao nhiêu giả định?
- Giải pháp** nhìn nó như một trục liên tục: **giả định mạnh + ít dữ liệu** ở một đầu, **giả định yếu + nhiều dữ liệu** ở đầu kia.
- Giả định** bạn ước lượng đúng vị trí mình đang đứng trên trục đó, mà vị trí ấy do lượng dữ liệu quyết định chứ không do sở thích.
- Hệ quả** đây chính là nguyên lý “độ khó bị đẩy đi chỗ khác” của chương 3, lần này có tên và có trục. Bạn sẽ gặp lại nó nguyên vẹn hai lần nữa: ở CNN so với **ViT** tại chương 13, và ở hình học cổ điển so với mô hình **tiền thẳng** (*feed-forward*) tại chương 16.
- ↔ **tiền đề C/13** — CNN đứng ở đầu giả định mạnh, ViT ở đầu giả định yếu. Đó là toàn bộ lý do ViT cần nhiều dữ liệu hơn
- 6 Vấn đề** khung lý thuyết cổ điển dự đoán sai với mạng lớn.
- Giả định** khung ***bias–variance***: tăng sức chứa thì độ lệch hệ thống giảm nhưng kết quả dao động mạnh hơn giữa các lần huấn luyện, và sai số kiểm thử có dạng chữ U.
- Vấn đề mới** quan sát mâu thuẫn — mạng có nhiều tham số hơn cả số mẫu huấn luyện vẫn tổng quát hoá tốt.
- Giải pháp** ***double descent***: vượt qua **ngưỡng nội suy** (*interpolation threshold*), sai số kiểm thử tăng vọt rồi *giảm trở lại* [55], [56].
- Hệ quả** đừng suy luận về overfitting bằng trực giác lấy từ giáo trình cũ; hãy đo.
- 7 Vấn đề** có những bài toán mà thêm dữ liệu không cứu được.
- Giải pháp** khái niệm identifiability: nếu nhiều nghiệm khác nhau sinh ra cùng một quan sát thì mọi lượng dữ liệu đều vô ích.
- Hệ quả** hành động đúng không phải thêm mẫu mà là đổi bài toán, thêm ràng buộc, hoặc thêm cảm biến.
- 8 Vấn đề** vậy cuối cùng nên chọn kiến trúc nào?
- Hệ quả** kết luận của cả chương là ***no free lunch***: lấy trung bình trên mọi bài toán có thể, không thuật toán nào tốt hơn thuật toán nào [57]. Ở dạng nghiêm ngặt, định lý nói về một không gian bài toán mà thế giới thật không lấy mẫu từ đó, nên đừng dùng nó để phủ nhận việc so sánh. Ở dạng thực dụng: không có kiến trúc tốt nhất, chỉ có kiến trúc mà giả định của nó khớp với dữ liệu của bạn.

4.3 Vì sao khoảng cách pixel vô nghĩa (Why Pixel Distance Fails)

A. Một phép tính 4×4 làm được trên giấy

Bước hai của mạch trên dễ bị chấp nhận như một khẩu hiệu, nên hãy tính tay một ví dụ đủ nhỏ. Xét “ảnh” 4×4 một kênh.

- **Ba ảnh dùng trong ví dụ:**

- *A* — hai cột trái bằng 1, hai cột phải bằng 0: một cạnh dọc.
- *B* — chính *A* dịch sang phải một pixel (cột 2, 3 bằng 1). Về ngữ nghĩa, *A* và *B* là cùng một vật.

- G — ảnh xám đồng nhất, mọi pixel bằng 0,5: không cạnh, không cấu trúc, không vật thể.

Hiệu $A - B$ khác không ở cột 1 và cột 3, mỗi cột bốn pixel với độ lệch 1; còn $A - G$ lệch 0,5 ở cả mười sáu pixel. Do đó

$$\|A - B\|_2 = \sqrt{4 \cdot 1^2 + 4 \cdot 1^2} = \sqrt{8} \approx 2,83, \quad \|A - G\|_2 = \sqrt{16 \cdot 0,5^2} = 2,00.$$

Hai con số này nói một điều rất khó chịu: *ảnh xám trông rõ ràng gần A hơn chính A dịch một pixel*. Đây không phải tình huống bệnh lý được dựng lên. Dịch chuyển, đổi độ sáng và đổi nền là ba biến đổi phổ biến nhất trong ảnh thật. Cả ba đều giữ nguyên nội dung nhưng làm khoảng cách pixel nhảy vọt.

B. Lờn nguyên số chiều bằng con số

Với ảnh CIFAR-10, không gian có $32 \times 32 \times 3 = 3072$ chiều. Muốn phủ khối lập phương đơn vị trong không gian đó bằng một lưới rất thô, mười mức mỗi trục, bạn cần 10^{3072} điểm — một con số không có nghĩa vật lý nào. Ý nghĩa của phép tính: không tập dữ liệu nào “phủ” được không gian pixel, nên láng giềng gần nhất trong đó không bao giờ thực sự gần, và mọi phương pháp dựa vào khoảng cách thô đều đang đo một đại lượng gần như ngẫu nhiên. Đây cũng chính là lý do các công cụ chiếu xuống chiều thấp như *UMAP* [7] chỉ hữu ích khi áp lên *đặc trưng đã học*, không phải lên pixel.

4.4 Bốn nguồn của inductive bias (Four Sources of Inductive Bias)

Nói “thêm giả định” thì dễ, nhưng giả định được cài vào hệ thống ở đúng bốn chỗ, và biết nó nằm ở chỗ nào quyết định bạn sửa cái gì khi nó sai.

• Bốn nguồn, kèm giả định cụ thể mà mỗi nguồn phát biểu:

- **Kiến trúc** — *convolution* phát biểu hai điều: đặc trưng hữu ích thì cục bộ, và ý nghĩa của một mẫu hình không đổi khi nó dịch vị trí. Đó là lý do CNN học được từ vài chục nghìn ảnh trong khi mạng toàn kết nối cần nhiều hơn hẳn.
- **Hàm mất mát** — *sai số bình phương* (*squared error*) ngầm giả định nhiễu Gauss đồng nhất; *cross-entropy* ngầm giả định đầu ra là một phân phối phạm trù. Chọn loss là chọn mô hình nhiễu, dù bạn có nghĩ tới điều đó hay không.

↪ **trường hợp riêng của B/09** — cả chương thiết kế hàm mất mát là việc chọn một nguồn inductive bias duy nhất trong bốn nguồn ở đây

- **Dữ liệu** — mỗi phép **tăng cường dữ liệu** (*data augmentation*) là một phát biểu “nhân không đổi dưới phép này”. Lật ngang là phát biểu về đối xứng trái phải của thế giới, và nó sai ngay khi bài toán cần đọc chữ hoặc phân biệt tay trái với tay phải.
- **Thuật toán tối ưu** — *SGD* không dừng ở một nghiệm bất kỳ trong tập nghiệm; có bằng chứng thực nghiệm rằng *batch* lớn dẫn tới nghiệm “nhon” hơn kèm tổng quát hoá kém hơn [12]. Cơ chế vẫn còn tranh cãi, nên hãy coi đây là quan sát thực nghiệm, không phải định luật.

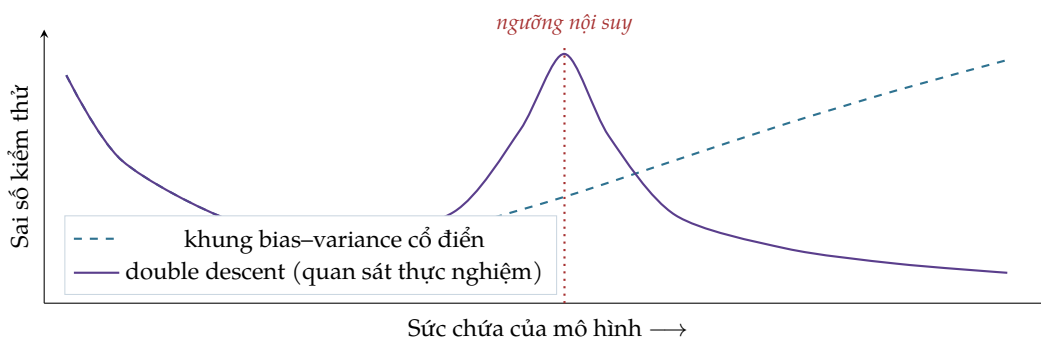
Bốn nguồn này không tương đương về chi phí sửa chữa, và đó mới là thông tin dùng được khi chẩn đoán (Bảng 4.1).

Bảng 4.1. Bốn nguồn inductive bias xếp theo chi phí sửa khi bạn phát hiện đã đặt sai giả định. Chẩn đoán nên đi từ trên xuống.

Nguồn	Giả định nó phát biểu	Chi phí sửa	Hòng khi nào
Augmentation	Nhân bất biến dưới một nhóm phép biến đổi	Một buổi, huấn luyện lại	Khi phép biến đổi thật sự đối nhân (chữ, trái/phải, màu chẩn đoán)
Thuật toán tối ưu	Nghiem “phẳng” tổng quát hoá tốt hơn	Vài lần chạy để dò lại	Khi thay đổi batch size hoặc lịch học làm dịch nghiệm mà không ai để ý
Hàm mất mát	Dạng phân phối của nhiễu và của đầu ra	Huấn luyện lại, thường phải sửa cả metric	Khi nhiễu lệch nặng, hoặc khi lớp mất cân bằng cực đoan
Kiến trúc	Cấu trúc không gian của tín hiệu (cục bộ, đẳng biến tịnh tiến)	Thiết kế lại và huấn luyện lại từ đầu	Khi quan hệ tầm xa mới là thứ quyết định, hoặc dữ liệu không có cấu trúc lưới

4.5 Chữ U cũ và hai lần đi xuống (Bias–Variance vs Double Descent)

Hình 12.12 đặt cạnh nhau điều giáo trình cổ điển dự đoán và điều thường quan sát được với mạng lớn. Cần đọc hình này cẩn thận, vì nó rất dễ bị hiểu thành “cứ tăng mô hình là tốt”.



Hình 4.2. Hai dự đoán về sai số kiểm thử khi sức chứa tăng. Trước ngưỡng nội suy — điểm mà mô hình vừa đủ sức khớp toàn bộ tập huấn luyện — hai khung trùng nhau; sau ngưỡng đó chúng đi ngược chiều. Trục không có đơn vị: đây là hình minh họa dạng hàm, không phải một phép đo.

Điều đáng rút ra không phải là “mô hình càng lớn càng tốt” — vùng nguy hiểm nhất trong hình là quanh ngưỡng nội suy, và một dự án ngân sách hẹp rất dễ dừng lại đúng ở đó. Điều đáng rút ra là lời khuyên cổ điển “train loss thấp hơn test loss nhiều thì phải giảm mô hình” không còn là suy luận hợp lệ; nó là một giả thuyết phải kiểm chứng bằng cách quét kích thước mô hình. Cũng cần nói thẳng rằng double descent hiện rõ trong một số cấu hình và mờ nhạt hoặc vắng mặt trong nhiều cấu hình khác, nên đừng trông đợi thấy nó trên mọi bài toán.

4.6 Khi thêm dữ liệu là vô ích (Identifiability)

Ví dụ sạch nhất nằm ngay trong lĩnh vực bạn đã làm. Ước lượng độ sâu từ một ảnh đơn: nhân toàn bộ cảnh và toàn bộ khoảng cách camera với cùng một hệ số $s > 0$ thì ảnh thu được giống hệt. Nói gọn thì phép chiếu đã xoá thông tin về tỉ lệ tuyệt đối, nên không hàm nào nhận ảnh làm

đầu vào phục hồi được tỉ lệ đó từ hình học thuần túy — thêm một triệu ảnh nữa cũng không đổi. Đó là lý do các công trình độ sâu đơn ảnh dùng hàm mất mát bất biến theo tỉ lệ, hoặc theo cả tỉ lệ lẫn dịch chuyển, thay vì hồi quy độ sâu tuyệt đối [31], [58].

Ở đây cần *phức tạp hoá* thay vì cắt gọn, vì bức tranh thật sự không đơn giản. Các mô hình đơn ảnh vẫn đoán được tỉ lệ ở mức nào đó — nhưng nhờ tiên nghiệm về kích thước quen thuộc của vật thể (người cao chừng ấy, xe rộng chừng ấy) chứ không nhờ hình học. Nói cách khác, phần không định danh được về mặt hình học được lấp bằng một inductive bias học từ dữ liệu, và bias ấy sập ngay khi cảnh chứa vật thể lạ kích thước. Vì vậy câu “mô hình này ước lượng được độ sâu tuyệt đối” vừa đúng vừa sai, và biết nó đúng theo nghĩa nào là điều kiện để dùng nó an toàn. Hệ SLAM đơn camera mang đúng tính chất này: bản đồ và quỹ đạo chỉ xác định tới một hệ số tỉ lệ. Cách xử lý đúng không phải thu thập thêm ảnh mà là thêm một phép đo khác: IMU, *stereo baseline*, hoặc một vật mốc đã biết kích thước [59].

↔ **tiền đề B/05** — learning curve phẳng có đúng hai nguyên nhân: hết tín hiệu trong dữ liệu, hoặc bài toán vốn không identifiable. Phân biệt được chúng thì mới biết nên tiêu tiền vào đâu

4.7 Ba đòn bẩy khi mô hình chưa đủ tốt (Three Levers)

Toàn bộ chương quy về một quyết định lặp đi lặp lại trong mọi dự án. Bảng 4.2 là dạng vận hành được của nó; cột cuối đáng đọc nhất vì nó cho bạn dấu hiệu nhận ra mình đang kéo nhầm đòn bẩy trước khi tiêu hết ngân sách.

Bảng 4.2. Ba nguồn cải thiện khi mô hình chưa đủ tốt, và dấu hiệu bạn đang chọn sai.

Đòn bẩy	Giả định ngầm	Hiệu quả nhất khi	Hỏng khi nào
Thêm dữ liệu	Bài toán identifiable và nhân nhất quán	<i>learning curve</i> theo kích thước dữ liệu chưa bão hoà	Curve đã phẳng từ lâu mà vẫn tiếp tục thu thập
Thêm giả định (kiến trúc, loss, augmentation)	Bạn thật sự biết một tính chất đúng của bài toán	Dữ liệu ít, tri thức miền rõ ràng	Thêm bias sai làm mô hình tệ hơn cả baseline
Thêm sức chứa	Đang <i>underfit</i> chứ không phải thiếu tín hiệu	Train loss còn cao và còn giảm đều	Train loss đã gần 0 mà vẫn phóng to mô hình

Ba đòn bẩy cạnh tranh cùng một ngân sách, nên có một thứ tự chẩn đoán rõ nhất:

1. Nhìn train loss để loại hoặc xác nhận underfit. Đây là bước rõ nhất vì bạn đã có sẵn số.
2. Vẽ learning curve theo kích thước dữ liệu để biết mình còn ở đoạn dốc hay đã bão hoà — quy trình cụ thể nằm ở [B/05-thiet-ke-bai-toan](#).
3. Chỉ khi cả hai bước trên đều nói “không” mới đặt câu hỏi về giả định, vì đó là đòn bẩy đòi hỏi hiểu biết miền và cũng là đòn bẩy dễ dùng sai nhất.

4.8 Thực hành

Chương này trừu tượng hơn các chương khác, nên bài tập được thiết kế để biến một mệnh đề trừu tượng — “vấn đề nằm ở biểu diễn” — thành một con số bạn tự đo được.

Repo và tài nguyên. [scikit-learn](#) cho KNN và baseline cổ điển; [facebookresearch/dinov3](#) để lấy đặc trưng tự giám sát mạnh làm đối chứng; [faiss](#) khi vét cạn không còn khả thi; [umap-learn](#) để nhìn cấu trúc đa tạp bằng mắt [7]. Sách nền: Goodfellow và cộng sự [53] cho manifold hypothesis, Murphy [14] cho phần thống kê. Tên mô hình nền tảng chốt tại tháng 9/2026 và sẽ cũ trong sáu đến mười hai tháng; [scikit-learn](#) và [faiss](#) bền hơn nhiều, nên hãy học giao diện chứ đừng học tên *checkpoint*.

Câu hỏi kiểm tra hiểu. Vì sao KNN trên pixel thất bại còn KNN trên đặc trưng DINOv3 lại rất mạnh, dù thuật toán phân loại không đổi một dòng — điều đó nói gì về vị trí thật sự của “trí tuệ” trong hệ thống? Double descent phá vỡ lời khuyên cổ điển nào mà có thể bạn vẫn đang áp dụng theo phản xạ? Nếu ai đó đề nghị gán nhãn thêm mười nghìn ảnh để cải thiện một mô hình ước lượng độ sâu tuyệt đối từ ảnh đơn, bạn phản biện thế nào trong ba câu?

Bài tập phụ. Nếu muốn một phiên bản không dính tới SLAM để khởi động, chạy KNN trên CIFAR-10 ở ba biểu diễn: pixel thô, *HOG*, và đặc trưng DINOv3 đóng băng. Sau đó vẽ độ chính xác theo k cho cả ba trên cùng một trục. Mini project SLAM ở cuối chương là phiên bản nghiêm túc của chính thí nghiệm này, trên dữ liệu của bạn.

4.9 Giới hạn và trường hợp hỏng

- **Giả định i.i.d. gần như luôn sai ở mức độ nào đó.** Mọi lập luận về tổng quát hoá trong chương này sụp đổ *hoàn toàn* khi phân phối dịch chuyển, chứ không suy giảm dần dần. Cả chương [B/08-dịch-chuyen-phan-phoi](#) tồn tại để xử lý phần sai đó.
- **Khung này im lặng về shortcut learning**, một chế độ hỏng nguy hiểm hơn overfitting. Mô hình có thể đạt $\hat{R}$ rất thấp và tổng quát hoá tốt trên tập kiểm thử cùng phân phối. Nó vẫn có thể đang dựa vào một tương quan giả: nền ảnh, *watermark*, hay kết cấu thay vì hình dạng. Tương quan đó biến mất thì mô hình sập [60]. ERM không phân biệt được đặc trưng nhân quả với đặc trưng chỉ tương quan; nó chỉ tối thiểu một trung bình.
- **Các khái niệm ở đây mô tả giới hạn chứ không phải quy trình.** Chúng nói khi nào một hướng đi là vô vọng, nhưng không nói kiến trúc nào nên thử trước. Hãy dùng chúng để loại bỏ những thí nghiệm không cần chạy.

CẠM BÃY

Cạm bẫy hay gặp nhất là dùng “no free lunch” như một cái cớ để không so sánh. Định lý phát biểu về trung bình trên *mọi* bài toán khả dĩ, kể cả những bài toán mà nhãn hoàn toàn ngẫu nhiên — một không gian mà thế giới thật không lấy mẫu từ đó. Trên lớp bài toán thực tế, có những kiến trúc tốt hơn hẳn những kiến trúc khác một cách ổn định, và câu “không có kiến trúc nào tốt nhất” không cho phép bạn bỏ qua việc đo. Cách dùng đúng: để đặt câu hỏi “giả định của mô hình này khớp với dữ liệu của tôi ở điểm nào”, chứ không phải để kết thúc tranh luận.

CẦN NHỚ

Bạn không chọn được “không có giả định” — bạn chỉ chọn giả định nào, và trực đánh đổi duy nhất là *giả định mạnh với ít dữ liệu* đổi lại *giả định yếu với nhiều dữ liệu*. Trước khi kéo bất kỳ đòn bẩy nào, kiểm tra hai điều theo thứ tự: bài toán có identifiable không (nếu không, mọi dữ liệu thêm vào đều lãng phí), và train loss đã thấp chưa (nếu chưa, bạn đang underfit chứ không thiếu dữ liệu). [T1]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
ERM	Tối thiểu sai số trung bình trên mẫu đã có, rồi hy vọng nó xấp xỉ sai số trên phân phối thật. Đây là thứ mọi thuật toán học có giám sát thực sự làm.
Rủi ro kỳ vọng	Sai số trung bình trên toàn phân phối thật, tức đại lượng ta muốn tối thiểu nhưng không đo được. Khe hở giữa nó và trung bình trên mẫu là nơi mọi vấn đề tổng quát hoá sinh ra.
Giả định i.i.d.	Mỗi mẫu được rút độc lập từ cùng một phân phối, và phân phối lúc triển khai vẫn là phân phối đó. Nó sập thì mọi lập luận về tổng quát hoá mất hiệu lực hoàn toàn chứ không suy giảm dần.
KNN trên pixel	Phép thử mở màn rẻ nhất: phân loại bằng láng giềng gần nhất, không huấn luyện gì. Nó thất bại thảm hại trên ảnh, và cùng thuật toán đó chạy trên đặc trưng tự giám sát lại rất mạnh.
Vì sao khoảng cách pixel không đo được ngữ nghĩa?	Vì nó đo chênh lệch độ sáng ở từng vị trí. Một ảnh xám trông rõ ràng cách một cạnh dọc 2,00 đơn vị, còn chính cạnh đó dịch một pixel thì cách 2,83 đơn vị.
Lời nguyên số chiều	Trong không gian nhiều nghìn chiều, mọi cặp điểm gần như cách đều nhau, nên “láng giềng gần nhất” mất phần lớn nội dung thông tin. Không tập dữ liệu nào phủ nổi không gian pixel.
Giả thuyết đa tạp	Dữ liệu thật không rải đều khắp không gian mà nằm gần một mặt có số chiều thấp hơn nhiều. Nó giải thích vì sao học vẫn khả thi dù lời nguyên số chiều có thật.
Inductive bias	Giả định về dạng lời giải, cài vào trước khi dữ liệu lên tiếng. Nó đến từ đúng bốn nguồn: kiến trúc, hàm mất mát, augmentation, và thuật toán tối ưu.
Augmentation	Mỗi phép biến đổi áp lên ảnh huấn luyện là một phát biểu “nhân không đổi dưới phép này”. Lật ngang sai ngay khi bài toán cần đọc chữ hoặc phân biệt tay trái với tay phải.
Vì sao thuật toán tối ưu cũng là một nguồn giả định?	Vì SGD không dừng ở một nghiệm bất kỳ trong tập nghiệm. Có bằng chứng thực nghiệm rằng batch lớn dẫn tới nghiệm “nhộn” hơn kèm tổng quát hoá kém hơn, dù cơ chế vẫn còn tranh cãi.
Trực giả định — dữ liệu	Một đầu là giả định mạnh với ít dữ liệu, đầu kia là giả định yếu với nhiều dữ liệu. Bạn không chọn được “không có giả định”, chỉ chọn giả định nào và đứng ở đâu trên trục.
Vì sao train loss bằng không không nói gì về tổng quát hoá?	Vì một mạng đủ lớn học thuộc được cả nhãn hoàn toàn ngẫu nhiên. Khả năng khớp dữ liệu huấn luyện không phải bằng chứng về khả năng đoán dữ liệu mới.
Double descent	Vượt qua ngưỡng nội suy, tức điểm mà mô hình vừa đủ sức khớp toàn bộ tập huấn luyện, sai số kiểm thử tăng vọt rồi giảm trở lại. Hệ quả: trực giác chữ U từ giáo trình cũ không còn là suy luận hợp lệ.
Identifiability	Bài toán identifiable là bài toán mà quan sát xác định nghiệm duy nhất. Nếu nhiều nghiệm khác nhau sinh ra cùng một quan sát thì mọi lượng dữ liệu thêm vào đều vô ích.
Vì sao thêm dữ liệu vô ích với độ sâu tuyệt đối từ ảnh đơn?	Nhân cả cảnh và khoảng cách camera với cùng một hệ số thì ảnh thu được giống hệt. Phép chiếu đã xoá tỉ lệ tuyệt đối, nên không hàm nào nhận ảnh làm đầu vào phục hồi được nó.
Ba đòn bẩy	Thêm dữ liệu, thêm giả định, thêm sức chứa. Chúng cạnh tranh cùng một ngân sách, nên phải chần đoán trước khi kéo.
Train loss còn cao và còn giảm đều. Kéo đòn bẩy nào?	Bạn đang underfit, nên thêm sức chứa hoặc huấn luyện lâu hơn. Phép phân biệt: nếu train loss đã gần không mà test vẫn kém thì vấn đề nằm ở dữ liệu hoặc ở giả định, không ở kích thước mô hình.
Tốt trên tập kiểm thử cùng phân phối rồi sập ngoài đời. Nghi ngờ gì?	Shortcut learning: mô hình dựa vào một tương quan giả như nền ảnh, watermark hay kết cấu. Phép phân biệt là dựng một tập kiểm thử phá vỡ đúng tương quan đó.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
• No free lunch	Lấy trung bình trên mọi bài toán khả dĩ, không thuật toán nào tốt hơn thuật toán nào. Dùng nó để hỏi giả định của mô hình khớp dữ liệu của bạn ở đâu, không phải để từ chối đo.

• • • phải thuộc • • nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
DINOv2, DINOv3	Bằng chứng trực tiếp cho luận điểm “vấn đề nằm ở biểu diễn”: KNN và probe tuyến tính trên đặc trưng đóng băng đạt mức mà pixel thô không bao giờ với tới	Mô hình dẫn đầu tính tới 9/2026
Scaling law (Kaplan, Chinchilla)	Dạng đo được của trực giả định $\leftrightarrow$ dữ liệu: khi giả định yếu đi thì dữ liệu và tính toán phải tăng theo quy luật lũy thừa để bù	Kết quả đã công bố, dùng để lập ngân sách huấn luyện
MiDaS và dòng ước lượng độ sâu đơn ảnh	Xử lý trực tiếp một bài toán không identifiable bằng cách bỏ tỉ lệ tuyệt đối ra khỏi hàm mất mát thay vì cố học nó	Hạ tầng ước lượng độ sâu, bền
faiss	Tìm láng giềng gần nhất ở quy mô lớn chỉ có nghĩa sau khi không gian đã được học — đây là hiện thực công nghiệp của bước hai trong mạch	Hạ tầng, bền

MINI PROJECT SLAM • Khoảng cách nào mới có nghĩa: ghép cặp mảnh ảnh quanh đặc trưng ORB • 1–2 ngày

Mục tiêu	kiểm chứng luận điểm trung tâm của chương ngay trên dữ liệu của chính bạn: thất bại nằm ở biểu diễn chứ không ở thuật toán. Cách kiểm chứng là giữ nguyên bộ phân loại và chỉ đổi không gian đo khoảng cách.
Dữ liệu	một chuỗi EuRoC bất kỳ. Cho ORB-SLAM3 chạy và xuất ra, với mỗi đặc trưng ORB, một mảnh ảnh 32×32 quanh nó cùng với id của điểm 3D mà nó được gán. Nhân của một <i>cặp</i> mảnh là nhị phân: cùng điểm 3D hay không. Lấy vài chục nghìn cặp, cân bằng dương/âm.
Các bước	chạy KNN trên tập cặp và đo độ chính xác ghép cặp theo k , lần lượt với ba biểu diễn: pixel thô; mô tả từ ORB 256 bit sẵn có, dùng khoảng cách Hamming; và đặc trưng từ một mô hình tự giám sát đóng băng. Chú ý dùng đúng khoảng cách của từng không gian, và chia tập theo <i>chuỗi con</i> chứ không theo cặp, nếu không hai mảnh của cùng một điểm 3D sẽ nằm ở cả hai phía.
Xong khi	bạn có một hình duy nhất với ba đường độ chính xác ghép cặp theo k , và ba câu giải thích chênh lệch trong đó ít nhất một câu nói về <i>khoảng cách</i> chứ không về “mô hình mạnh hơn”. Con số phụ bắt buộc: khoảng cách L_2 trung bình giữa hai mảnh của cùng một điểm 3D, đặt cạnh khoảng cách L_2 trung bình giữa hai mảnh ngẫu nhiên. Hai con số gần nhau nghĩa là bạn vừa tái hiện ví dụ 4×4 ở mục trên, lần này trên dữ liệu thật.
Nối với	B/03 cung cấp datasheet nói cho bạn biết đặc trưng và mô tả từ nằm ở khối nào; B/05 dùng chính tập cặp này làm ví dụ về đơn vị phân tích và rò rỉ theo chuỗi; B/07 cho giao thức chia tập chống rò rỉ.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

A1. Nằm ở biểu diễn, không ở bộ phân loại. Cùng một KNN chạy trên hai không gian khác nhau cho hai kết quả khác hẳn, nên phần tạo ra chênh lệch là ánh xạ từ pixel sang đặc trưng — và đó là thứ deep learning thực sự học.

- A2. Hồng ở chỗ khoảng cách Euclid trên pixel đo sự khác nhau về *giá trị sáng từng vị trí*, không đo sự khác nhau về *nội dung*. Dịch chuyển, đổi độ sáng và đổi nền đều làm nó nhảy vọt trong khi giữ nguyên ngữ nghĩa.
- A3. Vì chữ U cổ điển chỉ mô tả vùng trước ngưỡng nội suy. Sau ngưỡng đó, trong nhiều cấu hình sai số kiểm thử giảm trở lại — hiện tượng double descent — nên khoảng cách train/test lớn không còn tự động là bằng chứng phải thu nhỏ mô hình.
- A4. Có: các bài toán không identifiable, mà ví dụ sạch nhất là tỉ lệ tuyệt đối trong ước lượng độ sâu đơn ảnh. Biết trước bằng cách tìm một phép biến đổi tham số giữ nguyên quan sát; tìm được thì mọi dữ liệu thêm vào đều vô ích, và việc đúng là thêm ràng buộc hoặc thêm cảm biến.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn nấu một nồi canh to và muốn biết đã vừa miệng chưa. Bạn không thể uống hết nồi, nên bạn nếm một thìa. Toàn bộ chuyện máy học từ dữ liệu nằm gọn trong hành động đó: đo trên một thìa rồi kết luận về cả nồi.

Mẹo này chạy được nhờ hai điều. Thứ nhất, bạn khuấy đều trước khi nếm; nếu mức đúng chỗ vừa đổ muối vào thì thìa ấy nói dối, và mọi kết luận sau đó đều hồng chứ không phải chỉ lệch đi một chút. Thứ hai, khi thìa quá nhỏ so với nồi, bạn phải bù bằng hiểu biết có sẵn: bạn biết canh thường mặn ở đáy, biết loại rau này hút muối. Chính phần hiểu biết thêm vào ấy giúp bạn đoán đúng phần nồi mình chưa nếm. Điều quan trọng là bạn không thể chọn “không thêm hiểu biết nào” — bạn chỉ chọn thêm cái gì. Thêm đúng thì một thìa là đủ; thêm sai thì nếm mười thìa vẫn kết luận sai, và sai một cách rất tự tin.

Cái giá phải trả là mọi kết luận đều là một vụ đặt cược rằng thìa của bạn giống phần còn lại. Và có những câu hỏi mà nếm bao nhiêu cũng vô ích: nếm không cho biết nồi này được bắc lên bếp lúc mấy giờ. Gặp loại câu hỏi đó, việc đúng là đi tìm cái đồng hồ, chứ không phải nếm thêm.

Chỗ không rút gọn được: vì sao một cái máy khổng lồ, đủ sức học thuộc cả những chỗ dữ liệu bị ghi sai, mà vẫn đoán tốt trên dữ liệu mới, thì tới nay chưa ai nói gọn được mà vẫn đúng. Mọi lời giải thích ngắn hiện có đều bỏ sót một trường hợp ngược lại.

Một câu: Bạn luôn kết luận về cả nồi từ một thìa, nên việc quan trọng nhất không phải là nếm nhiều hơn mà là biết thìa của mình đại diện cho cái gì.

4.10 Kết nối

- **Bốn nguồn inductive bias** tách thành các chương riêng: hàm mất mát ở [B/09-thiet-ke-ham-mat-mat](#), kiến trúc ở [B/10-thiet-ke-kien-truc](#) rồi cả Phần C, dữ liệu và thuật toán tối ưu ở Phần D.
- **Chương đối lập trực tiếp** là [B/08-dich-chuyen-phan-phoi](#): ở đây ta giả định phân phối đứng yên, ở đó ta bỏ giả định ấy.
- **Learning curve** — công cụ quyết định trong Bảng 4.2 — được xây dựng cụ thể ở [B/05-thiet-ke-bai-toan](#), chương kế tiếp, nơi “đầu tư dữ liệu hay đầu tư mô hình” trở thành quyết định kinh tế có con số.
- **Ngược lên**, [B/03-giai-phau-he-thong](#) dạy cách phát hiện giả định của người khác; chương này dạy cách chọn giả định của mình.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao ERM là một xấp xỉ chứ không phải một phép giải, và chính xác đại lượng nào trong định nghĩa của nó là thứ ta không đo được?
2. KNN trên pixel thất bại còn KNN trên đặc trưng tự giám sát thành công. Kết luận đúng rút ra là gì, và kết luận sai mà nhiều người rút ra là gì?

3. Bốn nguồn inductive bias là gì, và xếp theo chi phí sửa chữa khi phát hiện đặt sai giả định thì thứ tự của chúng ra sao?
4. Vì sao “train loss thấp hơn test loss nhiều nên phải giảm kích thước mô hình” không còn là suy luận hợp lệ?
5. Trong lĩnh vực của bạn, bài toán nào mà thêm dữ liệu chắc chắn vô ích, và phép biến đổi cụ thể nào giữ nguyên quan sát chứng minh nó không identifiable?
6. Shortcut learning khác overfitting ở chỗ nào, và vì sao tập kiểm thử rút cùng phân phối không phát hiện được nó?

Thiết kế bài toán và định nghĩa dữ liệu (Problem Framing and Data Definition)

PHỤ THUỘC B/03-giai-phau-he-thong (bộ ba đầu vào / đầu ra / tín hiệu giám sát); B/04-nen-tang-hoc-tu-du-lieu (identifiability và ba đòn bẩy — chương này biến đòn bẩy “thêm dữ liệu” thành một quyết định có con số).

TÓM TẮT

Chương này nói về những quyết định bạn phải ra *trước khi* có mô hình nào chạy: một mẫu là gì, nhân được định nghĩa thế nào ở các ca biên, gộp hay tách lớp, dùng *bbox* hay *mask*, lấy mẫu từ đâu, và nên đầu tư tiếp vào dữ liệu hay vào mô hình. Đọc xong bạn tự chẩn đoán được nguyên nhân của phần lớn thất bại triển khai, kể cả những nguyên nhân không hiện lên trên bất kỳ metric nội bộ nào. Bạn cũng trả lời được câu “gán nhãn thêm bao nhiêu thì đáng” bằng một phép tính hai dòng trên *learning curve*, thay vì bằng cảm tính. Nếu chỉ nhớ một điều: **đơn vị phân tích** (*unit of analysis*) và dạng đầu ra khoá chặt mọi thứ phía sau, còn thiên lệch sinh ra ở khâu thu thập thì không hàm mất mát nào sửa được — chỉ có thu thập lại.

KIẾN THỨC CẦN CÓ TRƯỚC

- **nhãn (label) và tập dữ liệu** — nhãn là câu trả lời đúng gắn với một ảnh — tên lớp, tọa độ hộp, hay mask. Tập dữ liệu là tập các cặp (ảnh, nhãn) dùng để dạy và để đánh giá mô hình. Cả chương này nói về việc *định nghĩa* nhãn chứ không về việc học từ nó.
- **mẫu (sample)** — **hai nghĩa** — (a) một điểm dữ liệu đơn lẻ, “một mẫu ảnh”; (b) trong thống kê, cả tập được rút ra từ một quần thể, “mẫu gồm 500 ảnh”. Chương này dùng nghĩa (a) khi nói “số mẫu” và nghĩa (b) khi nói “lấy mẫu”; ngữ cảnh phân biệt được.
- **tập huấn luyện và tập kiểm thử** — tập dữ liệu được chia làm ít nhất hai phần: phần dùng để điều chỉnh tham số (huấn luyện) và phần giữ riêng, không cho mô hình nhìn thấy, dùng để ước lượng chất lượng trên dữ liệu mới (kiểm thử). Nếu một mẫu lọt vào cả hai phần thì con số kiểm thử lạc quan giả — hiện tượng gọi là rò rỉ dữ liệu.
- **các dạng nhãn không gian** — *bounding box* (bbox) là hình chữ nhật bao quanh vật, ghi bằng bốn số; *mask* là nhãn cho từng pixel, nói pixel đó thuộc vật nào; *polygon* là đường bao đa giác, xấp xỉ rẻ hơn của mask; *keypoint* là một tập điểm có ý nghĩa cố định (khớp vai, khuỷu tay); *hộp 3D* là hộp trong không gian ba chiều, cần thêm thông tin hình học.
- **lớp (class) và taxonomy** — lớp là một phạm trù mà nhãn có thể nhận (“xe”, “người”). Taxonomy là toàn bộ danh sách lớp cùng cách chúng gộp hay tách khỏi nhau — “xe” hay là “ô tô / xe tải / xe buýt”. Đây là một quyết định thiết kế, không phải một sự thật có sẵn trong thế giới.
- **quần thể và khung lấy mẫu** — quần thể (*population*) là toàn bộ tình huống hệ thống sẽ gặp khi chạy thật. Khung lấy mẫu (*sampling frame*) là phần của quần thể đó mà bạn thực sự chụp được. Hai thứ này gần như không bao giờ trùng nhau, và chênh lệch giữa chúng là chủ đề trung tâm của chương.
- **learning curve** — **hai nghĩa** — (a) đường cong sai số theo số vòng huấn luyện (epoch), dùng để xem mô hình đã hội tụ chưa; (b) đường cong sai số theo kích thước tập dữ liệu, dùng để quyết định có nên thu thập thêm dữ liệu không. Chương này chỉ dùng nghĩa (b), và đó là nghĩa ít người vẽ hơn hẳn.
- **IoU và mAP** — IoU (*intersection over union*) đo mức chồng lấn giữa dự đoán và nhãn: diện tích phần giao chia cho diện tích phần hợp, bằng 1 khi trùng khít. mAP là điểm tổng hợp cho bài

toán phát hiện, dựng trên IoU. Ở chương này chỉ cần biết chúng là các thước đo phụ thuộc vào dạng đầu ra bạn đã chọn.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao hai người gán nhãn giỏi, cùng đọc một guideline, vẫn cho ra hai tập nhãn khác nhau — và vì sao con số bất đồng đó lại là trần hiệu năng của mô hình?
- Q2. Một mô hình ổn định qua năm lần chia tập rồi sập ngay tuần đầu triển khai. Nếu không phải overfitting thì là gì?
- Q3. Đổi từ bbox sang mask nghe như một thay đổi kiến trúc; vì sao nó thực ra là quyết định đắt nhất của cả dự án?
- Q4. Câu hỏi “nên gán nhãn thêm hay nên đổi mô hình” có trả lời được bằng một con số thay vì bằng thâm niên không?

5.1 Đặt vấn đề

Trước khi có mô hình, bạn phải biến một nhu cầu mơ hồ thành một bài toán học máy. Mọi sai lầm ở bước này không sửa được ở bước sau — đó là điều làm cho chương này khác hẳn mọi chương kỹ thuật khác trong cây. Một **hàm mất mát** (*loss function*) chọn sai thì đổi loss; một kiến trúc chọn sai thì huấn luyện lại; nhưng một đơn vị phân tích chọn sai, hay một dạng đầu ra chọn sai, thì phải gán nhãn lại từ đầu, và trong hầu hết dự án điều đó đồng nghĩa với việc dự án dừng.

Người mới thường coi phần này là “việc chuẩn bị” và muốn đi nhanh qua để tới phần thú vị. Quan sát lặp lại — và cũng là quan sát chung trong kỹ thuật hệ thống học máy [43] — là điều ngược lại: khi một dự án thị giác thất bại, nguyên nhân hiếm khi nằm ở kiến trúc. Nó thường nằm ở một trong hai chỗ: nhãn được định nghĩa không nhất quán, hoặc dữ liệu được thu thập từ một quần thể khác với quần thể sẽ dùng hệ thống. Cả hai nguyên nhân này đều *vô hình* đối với mọi metric nội bộ: bạn có thể đạt **mAP** rất cao trên tập kiểm thử của chính mình và vẫn thất bại hoàn toàn khi triển khai, mà không con số nào trong bảng cảnh báo bạn.

Nếu bạn đã làm **benchmark** SLAM thì bạn có sẵn trực giác đúng cho chương này, chỉ là dưới tên khác. Một quỹ đạo **chân trị** (*ground truth*) trên EuRoC được định nghĩa trong một hệ quy chiếu cụ thể; nếu bạn đánh giá quỹ đạo camera bằng chân trị của thân máy bay, mọi con số đều ra, đều đẹp, và đều sai. Sai lầm không nằm ở thuật toán mà nằm ở định nghĩa của đại lượng được đo. Chương này là phiên bản tổng quát của bài học đó, áp cho toàn bộ khâu thiết kế bài toán.

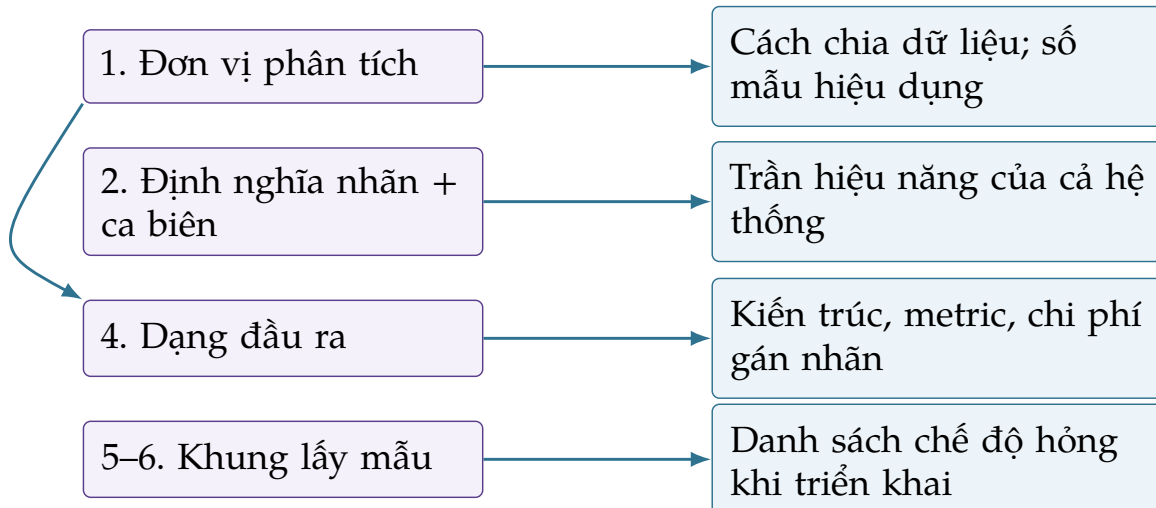
TRỰC GIÁC

Định nghĩa bài toán giống việc đúc một cái khuôn: mọi thứ đổ vào sau đó đều mang hình dạng của nó. Chọn đơn vị phân tích là chọn kích thước khuôn; chọn dạng đầu ra là chọn hình dạng khoang; định nghĩa nhãn kèm ca biên là mài các cạnh cho hai người thợ khác nhau đúc ra cùng một chi tiết. Bạn có thể đổi kim loại đổ vào — đổi kiến trúc, đổi loss, đổi **bộ tối ưu** (*optimizer*) — bao nhiêu lần tùy thích, nhưng đổi khuôn thì phải phá cả mẻ đã đúc.

5.2 Tám quyết định theo thứ tự ràng buộc (Eight Decisions, Ordered by Reach)

A. Quyết định nào khoá cái gì

Thứ tự của mạch dưới đây đi từ quyết định ràng buộc nhiều thứ nhất tới quyết định ràng buộc ít nhất. Đảo ngược thứ tự là sai lầm phổ biến: chọn dạng đầu ra trước khi biết đơn vị phân tích là gì thì bạn sẽ phát hiện mâu thuẫn sau khi đã gán vài nghìn ảnh. Hình 5.1 cho thấy tầm với của từng quyết định — đọc nó trước sẽ tiết kiệm cho bạn cả mạch phía sau.



Hình 5.1. Bốn quyết định có tầm với xa nhất và thứ mà mỗi cái khoá lại. Mũi tên cong bên trái nhắc rằng đơn vị phân tích còn ràng buộc cả dạng đầu ra: chọn video làm đơn vị thì nhãn theo khung hình mất phần lớn ý nghĩa.

B. Tám bước

I. Mạch thiết kế một bài toán thị giác

1 Vấn đề

“một mẫu” là gì, câu hỏi tưởng như không cần hỏi.

Giải pháp

chọn đơn vị phân tích một cách có ý thức — một khung hình, một video, một bệnh nhân, một ca chụp, một chuyến đi.

Hệ quả

nó quyết định luôn cách chia dữ liệu ở chương 7: chọn khung hình làm đơn vị rồi chia ngẫu nhiên là công thức tạo rò rỉ dữ liệu (*data leakage*). Hai khung hình cách nhau 33 ms gần như là cùng một ảnh, nên chúng sẽ nằm ở cả tập huấn luyện lẫn tập kiểm thử.

Giá phải trả

chọn đơn vị lớn hơn làm số mẫu hiệu dụng giảm mạnh, và bạn phải chấp nhận con số đó thay vì con số đẹp hơn nhưng vô nghĩa.

↪ **tiền đề B/07** — quy tắc chia dữ liệu ở chương đánh giá chỉ đúng khi đơn vị phân tích đã được chọn đúng ở đây

2 Vấn đề

nhãn nghe có vẻ hiển nhiên nhưng không phải.

Giả định

phần lớn cái ta gọi là “mô hình kém” thực ra là nhãn được định nghĩa không nhất quán: biên của mask đặt ở đâu khi vật nhoè, che khuất bao nhiêu phần trăm thì vẫn gán, vật nhỏ tới mức nào thì bỏ qua.

Giải pháp

viết định nghĩa nhãn kèm các **ca biên** trước khi gán mẫu đầu tiên, rồi thử nghiệm nó trên vài chục ảnh với hai người gán nhãn độc lập.

Hệ quả

mức đồng thuận giữa người gán nhãn (*inter-annotator agreement*) là trần hiệu năng thực tế của hệ thống; không mô hình nào vượt qua độ nhất quán của dữ liệu dạy nó một cách đáng tin.

- 3 Vấn đề** chia lớp thế nào cho đúng.
Giải pháp thiết kế *taxonomy*, tức quyết định gộp hay tách.
Giá phải trả gộp làm bài toán dễ và metric đẹp nhưng đầu ra kém hữu ích; tách làm đầu ra hữu ích hơn nhưng đẩy chi phí gán nhãn lên và làm các lớp hiếm quá thừa để học.
Hệ quả quy tắc thực dụng: tách khi và chỉ khi hai lớp dẫn tới hai *hành động* khác nhau ở phía sau hệ thống; nếu hệ thống phản ứng như nhau thì việc tách chỉ tạo công gán nhãn và bất đồng giữa người gán nhãn.
- 4 Vấn đề** chọn dạng đầu ra nào.
Giải pháp cân nhắc giữa nhãn ảnh, bbox, mask, *polygon*, *keypoint*, **hộp 3D** (*3D box*) — xem Bảng 5.1.
Giá phải trả đây là quyết định đắt nhất trong cả dự án, vì nó ràng buộc *đồng thời* kiến trúc, metric và chi phí gán nhãn. Đổi từ bbox sang mask giữa chừng nghĩa là gán lại từ đầu.
Hệ quả nên trả giá suy nghĩ ở đây bằng một thử nghiệm nhỏ, đừng trả giá bằng nửa ngân sách gán nhãn.
- 5 Vấn đề** bạn huấn luyện trên tập nào và triển khai cho ai?
Giải pháp phân biệt rành mạch **quần thể** (*population*) — toàn bộ tình huống hệ thống sẽ gặp khi chạy thật — với **khung lấy mẫu** (*sampling frame*), phần của quần thể đó mà bạn thực sự lấy mẫu được.
Giả định hai thứ trùng nhau, một giả định gần như luôn sai.
Hệ quả khoảng cách giữa chúng là nguồn thất bại số một khi triển khai, và nó không hiện lên trên bất kỳ metric nội bộ nào vì tập kiểm thử của bạn cũng rút từ chính khung lấy mẫu.
- 6 Vấn đề** thiên lệch được cài vào ngay từ khâu đầu.
Giả định *selection bias* khi thu thập: bạn chụp ban ngày vì tiện, dùng ba thiết bị vì chỉ có ba, lấy ảnh ở những địa điểm cho phép quay.
Giải pháp nguyên tắc cứng — thiên lệch sinh ra ở khâu thu thập thì không mô hình, loss, hay *reweighting* nào sửa được ở khâu cuối. Reweighting chỉ sửa được lệch *tỉ lệ* giữa các nhóm đã có mặt; nó không tạo ra được nhóm chưa từng xuất hiện.
Hệ quả chỉ có thu thập lại, nên chi phí của một quyết định lấy mẫu cầu thả là toàn bộ ngân sách dữ liệu.
- 7 Vấn đề** nên đầu tư vào dữ liệu hay vào mô hình?
Giải pháp một phép đo thay cho tranh luận — vẽ learning curve theo kích thước dữ liệu: huấn luyện cùng một cấu hình ở 10%, 25%, 50%, 100% dữ liệu và nhìn hình dạng đường cong.
Hệ quả curve còn dốc thì thu thập tiếp; curve đã phẳng thì tiền gán nhãn tiếp theo gần như bị đốt, và bạn phải đổi mô hình, đổi dạng đầu ra, hoặc đổi bài toán. Đây chính là đòn bẩy “thêm dữ liệu” của chương 4, lần này có quy trình đo cụ thể.
 ↪ **hệ quả của B/04** — bảng ba đòn bẩy nói có ba lựa chọn; learning curve nói *chọn cái nào*
- 8 Vấn đề** dữ liệu không miễn phí, mà thảo luận về nó thường không có đơn vị.
Giải pháp so *chi phí thu thập* với *giá trị biên mỗi mẫu* đọc từ chính đường cong ở bước trên.
Hệ quả nó biến một quyết định kỹ thuật thành quyết định kinh tế phát biểu được bằng con số — cách nghĩ dùng được ở mọi dự án thật, kể cả khi con số chỉ chính xác tới một bậc độ lớn.

I 5.3 Chọn dạng đầu ra (Choosing the Output Representation)

Trong tám bước trên, bước bốn đáng được mở rộng vì nó là bước duy nhất mà sai lầm không có đường lùi rẽ. Cột chi phí trong Bảng 5.1 là *tương đối* và chỉ nên đọc ở mức bậc độ lớn: các con số này phản ánh kinh nghiệm thực hành lưu truyền trong cộng đồng gắn nhãn, không phải một phép đo có kiểm soát. Chúng còn thay đổi mạnh khi có trợ lý gắn nhãn bằng mô hình nền tảng.

Bảng 5.1. Chọn dạng đầu ra. Cột “hóng khi nào” là cột quyết định: nó cho biết loại lỗi bạn tự nguyện chấp nhận vĩnh viễn khi chọn dạng đó.

Dạng	Giả định về bài toán	Chi phí nhãn (tương đối)	Hóng khi nào
Nhãn ảnh	Chỉ cần biết trong ảnh có gì	1×	Khi hệ thống cần định vị hoặc đếm
Bbox	Vật tách rời, hình chữ nhật đủ mô tả nó	5–10×	Vật dài, cong, hoặc chồng lấn: hộp chứa chủ yếu là nền
Mask	Cần biên chính xác tới mức pixel	20–50×	Biên mơ hồ (tóc, khối, kính) gây bất đồng lớn giữa người gắn nhãn
Polygon	Cần biên nhưng chấp nhận xấp xỉ đa giác	10–30×	Biên phức tạp bị làm trơn quá mức
Keypoint	Vật có cấu trúc cố định đã biết trước	10–20×	Vật không có cấu trúc chung, hoặc bị che một phần
Hộp 3D	Có hình học hoặc cảm biến phụ đồng bộ	50× trở lên	Lệch hiệu chuẩn (<i>calibration</i>) hoặc lệch thời gian giữa các cảm biến

Cách chọn đúng không phải là “chọn dạng giàu thông tin nhất mà ngân sách cho phép”, mà là đi ngược từ hành động phía sau: hệ thống của bạn *làm gì* với đầu ra? Nếu robot chỉ cần biết “có chướng ngại trong hành lang bên trái hay không” thì mask là lãng phí nhiều chục lần và còn làm chậm cả pipeline. Nếu nó phải bám theo mép một vật dài và cong thì bbox *sai về nguyên tắc*, không chỉ kém chính xác. Hình chữ nhật bao quanh một vật nằm chéo chứa phần lớn diện tích là nền. Hậu quả kép: bộ điều khiển phía sau không dùng được dự đoán đó, còn mọi metric dựa trên *IoU* thì vẫn khen nó.

↔ **tiền đề B/03** — đây chính là câu hỏi “đầu ra là gì của quy trình bảy câu hỏi, lần này phải trả lời cho hệ thống của chính bạn

Một dấu hiệu nhận biết mình chọn sai dạng đầu ra: bạn phải viết ngày càng nhiều luật hậu xử lý để dịch đầu ra của mô hình thành thứ hệ thống thật cần.

I 5.4 Quần thể và khung lấy mẫu (Population vs Sampling Frame)

Bước năm và sáu nói cùng một chuyện từ hai phía, và đáng được tách ra vì chúng là nguồn thất bại triển khai hay gặp nhất. Nói gọn thì có ba tập lồng nhau, không phải hai: quần thể là tập tất cả tình huống hệ thống sẽ gặp; khung lấy mẫu là tập tình huống bạn có khả năng chụp được; tập dữ liệu là một mẫu rút từ khung lấy mẫu, *không phải* từ quần thể. Tập kiểm thử của bạn cũng rút từ khung lấy mẫu, nên nó đo được sai số trong khung và mù hoàn toàn với phần quần thể nằm ngoài khung.

Điều này giải thích một hiện tượng gây bối rối: mô hình đạt kết quả tốt và ổn định qua nhiều lần chia tập, rồi sập khi triển khai. Không có gì mâu thuẫn — mọi phép chia đều nằm trong cùng một khung lấy mẫu, nên chúng đo lại cùng một thứ nhiều lần thay vì đo thêm thứ mới. Các

benchmark có kiểm soát về dịch chuyển phân phối được xây dựng chính vì lý do này [18], [38]. Bài học chung từ chúng: độ chính xác trong phân phối không dự đoán tốt độ chính xác ngoài phân phối.

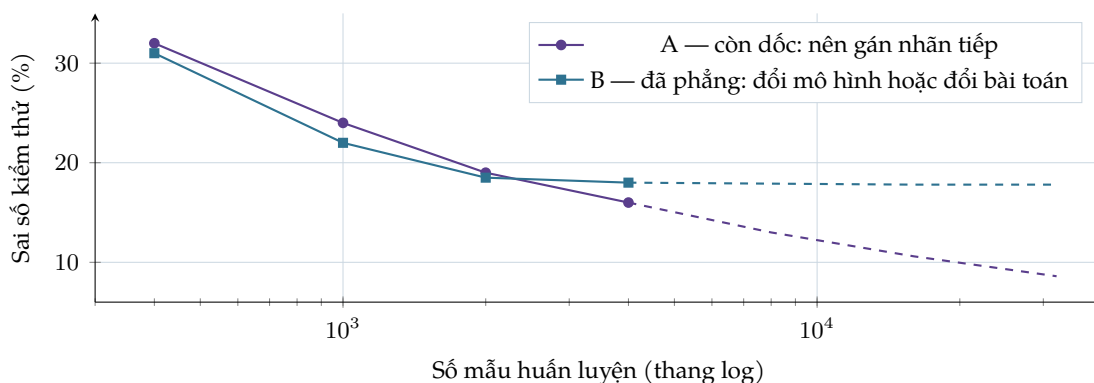
↔ **tiền đề B/08** — chênh lệch quần thể ↔ khung lấy mẫu là dạng tĩnh của dịch chuyển phân phối, chỉ khác ở chỗ nó có mặt ngay từ ngày đầu

Hệ quả hành động thì đơn giản và không dễ chịu. Trước khi thu thập, hãy viết ra bằng lời quần thể mục tiêu và liệt kê các trục biến thiên của nó: thiết bị, ánh sáng, mùa, địa điểm, người dùng. Sau đó đánh dấu trục nào khung lấy mẫu của bạn không phủ. Danh sách đó chính là danh sách chế độ hỏng đã biết trước của hệ thống, và nó nên nằm trong *datasheet* của tập dữ liệu [61] chứ không nằm trong đầu một người. Nếu một trục quan trọng không phủ được, quyết định đúng là thu hẹp phạm vi cam kết của sản phẩm, chứ không phải hy vọng mô hình tự tổng quát hoá.

5.5 Đường cong học theo kích thước dữ liệu (Data-Size Learning Curve)

A. Hai hình dạng, hai quyết định trái ngược

Câu hỏi “nên gán nhãn thêm hay nên đổi mô hình” thường được tranh luận bằng trực giác và thâm niên. Nó không cần phải như vậy: một buổi huấn luyện ở bốn tỉ lệ dữ liệu cho bạn câu trả lời kiểm chứng được. Điều quan trọng là bạn phân biệt hai trường hợp bằng *hình dạng* đường cong chứ không bằng một con số đơn lẻ (Hình 5.2).



Hình 5.2. Hai hình dạng learning curve theo kích thước dữ liệu, vẽ trên trục log. Đường liền là bốn điểm đo được ở 10%, 25%, 50%, 100% dữ liệu; đường đứt là ngoại suy. Đường A gần thẳng trên trục log — dấu hiệu quy luật lũy thừa còn hiệu lực; đường B đã chạm trần, và trần đó thường là mức nhất quán của nhãn chứ không phải giới hạn của mô hình. Số liệu là minh họa, không phải phép đo trên một tập cụ thể.

B. Phép tính hai dòng biến nó thành quyết định kinh tế

Với đường A, sai số giảm từ 24% ở 1000 mẫu xuống 16% ở 4000 mẫu. Giả sử dạng lũy thừa $e(n) \approx a n^{-b}$, lấy tỉ số hai điểm ta có

$$b = \frac{\log(24/16)}{\log(4000/1000)} = \frac{\log 1,5}{\log 4} \approx \frac{0,405}{1,386} \approx 0,29.$$

Số mũ b nói một điều duy nhất: *mỗi lần nhân đôi dữ liệu thì sai số nhân với 2^{-b}* . Với $b \approx 0,29$, nhân tư số mẫu lên 16,000 cho $e \approx 16 \cdot 4^{-0,29} \approx 10,6\%$, nghĩa là 12,000 ảnh gán nhãn thêm mua được khoảng 5,4 điểm. Nếu chi phí gán một ảnh là c thì bạn đang cân nhắc bỏ 12,000 c để đổi lấy 5,4 điểm, và bạn so nó với các phương án khác — thuê thêm một kỹ sư trong một tháng để thử kiến trúc mới, hay giảm c bằng trợ lý gán nhãn tự động. Với đường B thì phép tính không cần làm: b gần 0, nên mọi khoản đầu tư dữ liệu đều trả về gần 0.

Hai cảnh báo về công cụ này. Thứ nhất, ngoại suy quy luật lũy thừa ra ngoài dải đã đo là suy đoán và thường lạc quan quá mức, vì đường cong thật cuối cùng cũng phải chạm trần nhãn; hãy coi 10,6% là cận trên của kỳ vọng chứ không phải một dự báo. Thứ hai, mỗi điểm phải được huấn luyện với cùng một quy trình và có phương sai được ước lượng, nếu không bạn đang đọc nhiều như đọc xu hướng — đây chính là bài học từ benchmark có thành phần ngẫu nhiên mà bạn đã gặp ở phía SLAM.

5.6 Thực hành

Chương này chỉ chuyển thành kỹ năng khi bạn tự chịu hậu quả của một định nghĩa nhãn lỏng lẻo. Vì vậy cả hai bài tập đều xoay quanh một phép đo: mức bất đồng giữa hai người gán nhãn. Đó là con số hầu như không dự án nào đo, mà hầu như dự án nào cũng bị nó chặn.

Repo và tài nguyên. Ba nhóm công cụ:

- **Gán nhãn** — [HumanSignal/label-studio](#), [cvat-ai/cvat](#), Roboflow Annotate.
- **Trợ lý gán nhãn bằng mô hình nền tảng** — dòng SAM cho mask theo *prompt* [62] và dòng Grounding DINO cho box theo mô tả văn bản. Chúng làm dịch chuyển đáng kể cột chi phí trong Bảng 5.1, nhưng *không* làm dịch chuyển vấn đề nhất quán của định nghĩa nhãn.
- **Soát và tài liệu hoá dữ liệu** — [voxel51/fiftyone](#) để khám phá; datasheets [61] và *model cards* [63] để viết tài liệu kèm.

Danh sách tên mô hình chốt tại tháng 9/2026 và sẽ cũ trong sáu đến mười hai tháng; CVAT và FiftyOne bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Trong bài toán của bạn, hai người gán nhãn giỏi sẽ bất đồng ở đâu — và bạn xử lý bằng cách siết guideline hay bằng cách đổi định nghĩa nhãn? Nếu chuyển đơn vị phân tích từ khung hình sang video thì tập kiểm thử phải chia lại thế nào, và số mẫu hiệu dụng thay đổi bao nhiêu lần? Quần thể mục tiêu của bạn có trực biến thiên nào mà khung lấy mẫu hiện tại không phủ?

Bài tập phụ. Nếu bài toán của bạn không phải SLAM, phiên bản tổng quát của bài tập này là: viết *annotation guideline* hai trang, tự gán 200 ảnh, rồi nhờ một người khác gán 50 ảnh trong số đó. Đo mức bất đồng bằng **hệ số kappa của Cohen** (*Cohen's kappa*) [64] nếu là bài toán phân loại, hoặc bằng tỉ lệ khớp ở một ngưỡng IoU cố định nếu là phát hiện. Mini project SLAM ở cuối chương là phiên bản gắn với hệ thống của bạn.

5.7 Giới hạn và trường hợp hồng

- **Khung này giả định bạn có quyền quyết định về dữ liệu.** Rất nhiều dự án bắt đầu từ một tập đã tồn tại với nhãn đã gán; khi đó phần lớn tám bước trở thành công việc khảo cổ — đọc ngược từ dữ liệu ra định nghĩa nhãn ngầm mà người trước đã dùng. Việc này vẫn đáng làm và thường phát hiện những mâu thuẫn giải thích được vì sao mô hình kém, nhưng nó không thay thế được quyền thiết kế.
- **Nó giả định quần thể mục tiêu đứng yên đủ lâu.** Có những hệ thống mà môi trường đổi theo mùa, theo phiên bản phần cứng, hay theo hành vi người dùng. Ở đó “quần thể” là một mục tiêu di động. Bài toán đúng không phải thiết kế một lần cho tốt, mà là dựng một vòng lặp thu thập lại; đó là nội dung của [B/08-dich-chuyen-phan-phoi](#).
- **Mức đồng thuận là trần thực dụng, không phải trần lý thuyết.** Có trường hợp mô hình vượt từng người gán nhãn đơn lẻ vì nó học phân tín hiệu chung và trung bình hoá phần nhiễu

riêng của mỗi người; ngược lại, có những tập dữ liệu nổi tiếng mà chính nhãn kiểm thử chứa lượng lỗi đủ lớn để làm sai lệch bảng xếp hạng [19]. Hãy dùng nó như tín hiệu cảnh báo, đừng dùng như hằng số vật lý.

CẠM BẤY

Cạm bẫy tốn kém nhất trong chương này là tin rằng một thiên lệch thu thập có thể sửa được ở khâu huấn luyện. Reweighting, *resampling* hay các hàm mất mát cân bằng nhóm chỉ điều chỉnh được *tỉ lệ* giữa những nhóm đã có mặt trong dữ liệu. Giả sử hệ thống sẽ gặp điều kiện ban đêm mà tập dữ liệu không có một ảnh ban đêm nào. Không trọng số nào tạo ra được thông tin đó. Mô hình sẽ đưa ra dự đoán tự tin và sai, còn tập kiểm thử của bạn thì không hề một lời. Dấu hiệu nhận biết: cuộc thảo luận về “xử lý mất cân bằng dữ liệu” diễn ra hoàn toàn bằng ngôn ngữ hàm mất mát, không ai nhắc tới việc đi chụp thêm.

CẦN NHỚ

Đơn vị phân tích và dạng đầu ra là hai quyết định khoá chặt mọi thứ phía sau: sai ở đó thì phải gán nhãn lại từ đầu. Thiên lệch sinh ra ở khâu thu thập chỉ sửa được bằng cách thu thập lại — không loss nào, không trọng số nào chạm tới nó. Và trước khi tranh luận “thêm dữ liệu hay đổi mô hình”, hãy vẽ learning curve: số mẫu b trả lời câu hỏi đó bằng một con số. [T1]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Đơn vị phân tích	Thứ được tính là “một mẫu”: một khung hình, một video, một bệnh nhân, một chuyến đi. Nó quyết định luôn cách chia dữ liệu và số mẫu hiệu dụng, nên chọn sai là phải làm lại từ đầu.
••• Rò rỉ dữ liệu	Cùng một thông tin có mặt ở cả tập huấn luyện lẫn tập kiểm thử, làm con số kiểm thử lạc quan giả. Chọn khung hình làm đơn vị rồi chia ngẫu nhiên là công thức tạo ra nó.
••• Ca biên	Các trường hợp lấp lửng trong định nghĩa nhãn: biên vật nhoè, mức che khuất, kích thước tối thiểu. Phải viết ra giấy trước khi gán mẫu đầu tiên, vì sau đó sửa nghĩa là gán lại.
••• Mức đồng thuận giữa người gán nhãn	Tỉ lệ hai người gán độc lập cho ra cùng một nhãn. Đây là trần hiệu năng thực tế của hệ thống, vì mô hình chỉ học được từ những nhãn họ tạo ra.
•• Khi nào mô hình vượt được trần đó?	Khi mỗi người nhầm theo một kiểu riêng. Mô hình học phần tín hiệu chung và trung bình hoá phần nhiễu riêng của từng người, nên nhỉnh hơn từng người đơn lẻ.
•• Taxonomy	Danh sách lớp cùng cách chúng gộp hay tách. Quy tắc thực dụng: chỉ tách khi hai lớp dẫn tới hai hành động khác nhau ở phía sau hệ thống.
••• Dạng đầu ra	Nhãn ảnh, bbox, mask, polygon, keypoint, hay hộp 3D. Đây là quyết định đắt nhất dự án vì nó ràng buộc đồng thời kiến trúc, metric và chi phí gán nhãn.
••• Vì sao đổi dạng đầu ra giữa chừng lại đắt tới vậy?	Kiến trúc thì huấn luyện lại là xong, nhưng nhãn phải gán lại toàn bộ từ đầu. Đổi từ bbox sang mask là nhân chi phí gán nhãn lên vài lần.
•• Bbox	Hình chữ nhật bao quanh vật, ghi bằng bốn số. Nó sai về nguyên tắc với vật dài và cong, vì hộp bao một vật nằm chéo chứa phần lớn diện tích là nền.
•• Bạn phải viết ngày càng nhiều luật hậu xử lý. Triệu chứng của gì?	Dạng đầu ra chọn sai: bạn đang dịch đầu ra của mô hình thành thứ hệ thống thật cần. Phép phân biệt là hỏi hệ thống làm gì với đầu ra, rồi chọn dạng đi ngược từ đó.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Quần thể	Toàn bộ tình huống hệ thống sẽ gặp khi chạy thật. Đây là thứ bạn cam kết phục vụ, không phải thứ bạn đo được.
●●● Khung lấy mẫu	Phần của quần thể mà bạn thực sự chụp được. Tập kiểm thử cũng rút từ đây, nên nó mù hoàn toàn với phần quần thể nằm ngoài khung.
●●● Ổn định qua năm lần chia tập rồi sập khi triển khai. Nguyên nhân?	Chênh lệch giữa quần thể và khung lấy mẫu, không phải overfitting. Cả năm phép chia đều nằm trong cùng một khung nên chúng đo lại cùng một thứ; phép phân biệt là dựng một tập kiểm thử từ ngoài khung.
●●● Selection bias	Thiên lệch cài vào ngay khâu thu thập: chụp ban ngày vì tiện, dùng ba thiết bị vì chỉ có ba. Nó không sửa được ở khâu cuối, chỉ có thu thập lại.
●●● Vì sao reweighting không sửa được selection bias?	Reweighting chỉ chỉnh tỉ lệ giữa những nhóm đã có mặt trong dữ liệu. Nó không tạo ra được nhóm chưa từng xuất hiện, ví dụ ảnh ban đêm khi tập dữ liệu không có tấm nào.
●● Cả nhóm bàn “mất cân bằng dữ liệu” hoàn toàn bằng ngôn ngữ hàm mất mát	Dấu hiệu đang rơi vào bẫy reweighting. Phép kiểm tra: hỏi trong dữ liệu có bao nhiêu mẫu của nhóm thiểu; nếu là không thì việc phải làm là đi chụp thêm.
●●● Learning curve theo kích thước dữ liệu	Sai số kiểm thử vẽ theo số mẫu huấn luyện, đo ở 10, 25, 50 và 100 phần trăm dữ liệu. Nó biến câu “dữ liệu hay mô hình” thành một con số thay vì một cuộc tranh luận theo thâm niên.
●● Số mũ b của learning curve	Độ dốc trên trục log: mỗi lần nhân đôi dữ liệu thì sai số nhân với hai mũ trừ b. Với b khoảng 0,29 thì nhân tư dữ liệu mua được vài điểm; với b gần không thì mọi khoản đầu tư dữ liệu trả về gần không.
●● Datasheet của tập dữ liệu	Tài liệu ghi nguồn, quần thể mục tiêu, khung lấy mẫu, quy trình gán nhãn, mức đồng thuận và các lớp bị thiếu. Danh sách trục không phủ được chính là danh sách chế độ hỏng đã biết trước.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Datasheets for datasets, model cards	Chuẩn hoá đúng việc chương này yêu cầu: ghi lại quần thể mục tiêu, khung lấy mẫu, quy trình gán nhãn và các lớp bị thiếu	Thông lệ ở nhiều nhóm lớn từ 2019, ít hết hạn
Dòng SAM và Grounding DINO trong quy trình gán nhãn voxel51/fiftyone	Kéo chi phí của mask và bbox xuống một bậc, tức dịch chuyển trực tiếp cột chi phí trong bảng chọn dạng đầu ra	Đã thành mặc định tính tới 9/2026
	Công cụ soi chênh lệch giữa khung lấy mẫu và quần thể bằng cách lát cắt dữ liệu theo thiết bị, ánh sáng, địa điểm	Hạ tầng, bền
ImageNet-v2 và WILDS	Bảng chứng đo được rằng độ chính xác trong khung lấy mẫu không dự đoán tốt độ chính xác ngoài nó	Kết quả đã công bố

MINI PROJECT SLAM • Biến “có nên làm keyframe không” thành một bài toán học máy • 1–2 ngày

Mục tiêu đi hết tám quyết định của chương trên một bài toán bạn đã có sẵn trực giác kỹ thuật. Mục đích là thấy rõ chỗ nào trực giác đó *không* đủ để thay một định nghĩa viết ra giấy.

Dữ liệu	hai chuỗi EuRoC, một dễ và một khó (ví dụ một chuỗi MH và một chuỗi V2), cùng nhật ký của ORB-SLAM3 ghi lại từng khung hình và quyết định keyframe hiện tại của heuristic có sẵn.
Các bước	viết một tài liệu định nghĩa bài toán gồm sáu phần — (1) đơn vị phân tích: một khung hình, một đoạn, hay một cặp khung hình so với keyframe gần nhất, và hệ quả của lựa chọn đó lên cách chia dữ liệu; (2) định nghĩa nhãn dương/âm kèm ít nhất năm ca biên viết rõ (khung hình mờ nhưng nhiều đặc trưng, camera đứng yên, quay nhanh tại chỗ, vừa mất bám xong, vùng đã lập bản đồ dày); (3) taxonomy: nhị phân hay ba mức, và mỗi mức dẫn tới hành động khác nhau nào trong hệ; (4) dạng đầu ra và metric đi kèm; (5) khung lấy mẫu so với quần thể thật của bạn, ghi rõ trục nào EuRoC không phủ; (6) giao thức chia dữ liệu chống rò rỉ theo chuỗi. Sau đó tự gán 300 khung hình và nhờ một người khác gán 80 khung hình trong số đó theo đúng tài liệu, không được hỏi lại.
Xong khi	hai người gán độc lập theo tài liệu của bạn mà không hỏi thêm một câu nào, và bạn báo cáo được mức đồng thuận trên 80 khung hình chung bằng một con số (κ hoặc tỉ lệ khớp). Ngưỡng để coi là đạt: sau một vòng sửa tài liệu, mức đồng thuận phải tăng, và bạn liệt kê được ít nhất ba ca biên mà vòng đầu không nghĩ ra.
Nối với	B/03 cho biết keyframe nằm ở khối nào trong luồng dữ liệu và độ khó bị đẩy đi đâu khi bạn thay heuristic bằng mô hình; B/06 biến quy trình gán nhãn này thành một quy trình có ngân sách; B/07 kiểm tra giao thức chia dữ liệu ở phần (6); B/04 dùng chính tập nhãn này để vẽ learning curve và quyết định có đáng gán tiếp không.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì nhãn không phải một sự thật có sẵn mà là kết quả của một định nghĩa, và mọi định nghĩa đều dễ hờ các ca biên: biên vật nhòe, mức che khuất, kích thước tối thiểu. Mô hình học từ tập nhãn đó nên không thể nhất quán hơn chính dữ liệu dạy nó một cách đáng tin.
- A2. Là chênh lệch giữa quần thể và khung lấy mẫu. Cả năm lần chia tập đều nằm trong cùng một khung lấy mẫu, nên chúng đo lại cùng một thứ năm lần thay vì đo thêm thứ mới; phần quần thể nằm ngoài khung không xuất hiện trong bất kỳ phép chia nào.
- A3. Vì nó đổi *đồng thời* kiến trúc, metric và toàn bộ nhãn đã gán. Kiến trúc thì huấn luyện lại là xong; nhãn thì phải làm lại từ đầu với chi phí lớn hơn nhiều lần, và đó là thứ thường giết dự án.
- A4. Có: huấn luyện cùng một cấu hình ở bốn tỉ lệ dữ liệu, khớp $e(n) \approx a n^{-b}$ và đọc số mũ b . Với $b \approx 0,29$, nhân tư dữ liệu mua được vài điểm; với b gần 0, mọi khoản đầu tư dữ liệu tiếp theo đều trả về gần không.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn nhờ người khác đi chợ hộ. Nếu mẫu giấy chỉ ghi “mua ít rau”, bạn sẽ nhận về thứ khác với thứ mình hình dung; nhờ người khác thì lại nhận về thứ thứ ba. Không ai sai cả — mẫu giấy sai. Bài toán gốc của chương này đúng như vậy, chỉ khác là người đi chợ là một cái máy. Trước khi có cái máy nào chạy, bạn phải viết mẫu giấy đó: mỗi lần nó nhìn vào cái gì, cái gì được coi là đúng, và những trường hợp lấp lửng thì tính sao. Rau hơi héo có mua không? Bó to gấp đôi thì tính là một hay hai?

Mẹo gồm ba việc. Một, viết các trường hợp lấp lửng ra giấy trước khi bắt đầu chứ không phải sau. Hai, nhờ hai người cùng đi chợ với cùng mẫu giấy rồi xem họ mua khác nhau bao nhiêu; chênh lệch đó chính là mức tốt nhất cái máy có thể đạt, vì nó chỉ học được từ những gì hai người kia mang về. Ba, nhớ rằng bạn chỉ nhờ mua ở chợ đầu ngõ nên sẽ không bao giờ biết chợ khác bán gì; nếu nhà sắp chuyển đi nơi khác thì việc phải làm là đi chợ mới, chứ không phải viết lại mẫu giấy cho hay hơn.

Cái giá phải trả là viết mẫu giấy vừa tốn thời gian vừa nhầm chán, và phải làm xong trước khi được chạm vào phần thú vị.

Chỗ không rút gọn được: câu “cái máy không bao giờ giỏi hơn hai người đi chợ” gần đúng nhưng không đúng hẳn. Nếu mỗi người nhầm theo một kiểu riêng, cái máy học từ cả hai có thể trung bình hoá được phần nhầm và nhỉnh hơn từng người. Nói gọn hơn nữa là sai.

Một câu: *Cái máy chỉ làm được đúng cái mẫu giấy bạn viết, nên phần khó nhất của dự án nằm ở tờ giấy chứ không nằm ở cái máy.*

5.8 Kết nối

- **Ngược lên** — ba câu hỏi đầu của [B/03-giai-phau-he-thong](#) (đầu vào, đầu ra, giám sát) ở đây được trả lời cho *bài toán của bạn* thay vì cho bài toán của người khác; và đôn bẩy “thêm dữ liệu” trong bảng ba đôn bẩy của [B/04-nen-tang-hoc-tu-du-lieu](#) ở đây có được quy trình đo cụ thể.
- **Đi tiếp** — [B/06-chat-luong-du-lieu](#) nhận vấn đề nhất quán nhãn ở bước hai và biến nó thành quy trình có ngân sách; [B/07-chia-du-lieu-va-danh-gia](#) nhận đơn vị phân tích ở bước một và cho thấy chọn sai nó sinh ra rò rỉ thế nào; [B/08-dich-chuyen-phan-phoi](#) xử lý phần quần thể nằm ngoài khung lấy mẫu.
- **Sang ngang** — việc chọn dạng đầu ra ràng buộc luôn thước đo, nên Phần E về đánh giá đọc dễ hơn nhiều nếu bạn quay lại Bảng 5.1 khi đọc nó.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao chọn khung hình làm đơn vị phân tích rồi chia dữ liệu ngẫu nhiên lại tạo ra rò rỉ, và triệu chứng của rò rỉ đó hiện ra ở con số nào?
2. Vì sao mức bất đồng giữa hai người gán nhãn lại là trần hiệu năng thực tế, và trong hoàn cảnh nào mô hình vượt được trần đó?
3. Đưa ra một bài toán mà bbox không chỉ kém chính xác hơn mask mà còn sai về nguyên tắc. Sai ở đâu, và metric nào che giấu cái sai đó?
4. Một mô hình đạt kết quả ổn định qua năm lần chia tập rồi sập khi triển khai — cặp khái niệm quần thể / khung lấy mẫu giải thích hiện tượng đó thế nào, mà không cần dùng tới từ “overfitting”?
5. Vì sao reweighting không sửa được selection bias, và chính xác thì nó sửa được loại lệch nào, không sửa được loại nào?
6. Learning curve của bạn có số mũ $b \approx 0,05$. Kết luận gì, và bước tiếp theo là gì?
7. Trong bốn quyết định — đơn vị phân tích, taxonomy, dạng đầu ra, ngưỡng của metric — cái nào rẻ nhất để đổi sau khi đã gán nhãn xong, và vì sao?

Chất lượng dữ liệu và nhãn (Data and Label Quality)

PHỤ THUỘC B/05-thiet-ke-bai-toan (đơn vị phân tích (*unit of analysis*), thiên lệch chọn mẫu (*selection bias*)); B/04-nen-tang-hoc-tu-du-lieu (vì sao thêm dữ liệu không phải lúc nào cũng giúp). Chương này là tiền đề trực tiếp của B/07-chia-du-lieu-va-danh-gia.

TÓM TẮT

Chương này nói về chỗ mà trần hiệu năng của một hệ thống thị giác thật sự nằm: không ở kiến trúc, mà ở độ nhất quán của người gán nhãn. Đọc xong bạn đặt được một con số lên trần đó, trước khi huấn luyện mô hình đầu tiên. Bạn phân biệt được hai loại nhiễu nhãn có cơ chế gây hại khác hẳn nhau. Bạn tìm ra vài nghìn nhãn sai trong một trăm nghìn ảnh mà không phải xem hết bằng mắt. Nếu chỉ nhớ một điều: nhãn là một phép đo có sai số riêng, nên mức đồng thuận giữa người gán nhãn là trần trên của mọi con số bạn báo cáo. Và 3% nhiễu hệ thống hại hơn 10% nhiễu ngẫu nhiên, vì mô hình học được cái thứ nhất.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model), huấn luyện (training) và hàm mất mát (loss function)** — mô hình là một hàm có tham số, nhận một ảnh và trả về dự đoán. Hàm mất mát là con số đo mức lệch giữa dự đoán và nhãn của *một* mẫu, và huấn luyện chính là đi tìm bộ tham số làm trung bình con số đó nhỏ nhất. Cần biết vì cả chương bàn về chuyện nhãn sai; khi nhãn sai thì “làm dự đoán sát nhãn” không còn là mục tiêu đúng. Cần biết thêm vì cách tìm nhãn sai rẻ nhất là xếp hạng mẫu theo giá trị mất mát.
- **nhãn (label) và chân trị (ground truth)** — nhãn là đáp án gán cho mỗi mẫu: với phân loại là tên lớp, với phát hiện là khung bao kèm lớp, với phân vùng là mặt nạ từng pixel. *Chân trị* là toàn bộ tập nhãn được coi là đúng khi chấm điểm. Luận điểm trung tâm của chương là chân trị cũng chỉ là một phép đo có sai số riêng, không phải chân lý.
- **accuracy, precision, recall và IoU** — accuracy là tỉ lệ dự đoán đúng trên toàn bộ mẫu; precision là trong số lần mô hình báo “có” thì bao nhiêu phần đúng; recall là trong số ca “có” thật thì mô hình bắt được bao nhiêu phần. IoU là tỉ số giữa diện tích phần giao và diện tích phần hợp của hai vùng, bằng 1 khi trùng khít và 0 khi rời nhau. Ba con số đầu khác nhau rất xa khi một lớp hiếm; IoU là cách đo “hai người vẽ mask có giống nhau không”.
- **mức đồng thuận giữa người gán nhãn (inter-annotator agreement) và Cohen kappa** — cho hai người cùng gán một tập ảnh rồi đếm xem họ trùng nhau ở bao nhiêu phần trăm — đó là mức đồng thuận trần. Cohen kappa sửa con số đó bằng cách trừ đi phần trùng nhau chỉ nhờ may rủi, rồi chia cho phần dư địa còn lại: $\text{kappa} = 0$ nghĩa là chỉ ngang mức đoán bừa, $\text{kappa} = 1$ nghĩa là đồng ý hoàn toàn. Cần nó vì với lớp hiếm, hai người gán bừa vẫn đồng ý tới hơn 90%.
- **trùng lặp gần đúng (near-duplicate)** — hai ảnh khác nhau từng byte nhưng giống hệt nhau với mắt người — cùng một tấm bị nén lại, cắt cúp, đổi kích thước, thêm watermark, hoặc hai khung hình liền nhau trong video. Cần biết vì nếu một ảnh nằm ở tập huấn luyện còn bản sao gần đúng của nó nằm ở tập kiểm tra thì kết quả đo được là kết quả giả.
- **bias — phân biệt ba nghĩa** — (a) *độ lệch* thống kê: sai luôn về một hướng của một phép đo hay một tập dữ liệu so với thực tế; (b) *thiên lệch quy nạp (inductive bias)*: các giả định được cài sẵn vào kiến trúc hay hàm mất mát; (c) tham số cộng thêm b trong tầng tuyến tính $Wx + b$. Chương này luôn dùng nghĩa (a).

- **embedding** — vector vài trăm chiều mà một mạng đã huấn luyện sinh ra để biểu diễn một ảnh; hai ảnh giống nhau về nội dung cho hai vector gần nhau, đo bằng khoảng cách thông thường. Cần biết vì đây là cách duy nhất phát hiện được trùng lặp gần đúng — so từng byte thì không thấy gì.
- **dự đoán ngoài mẫu (out-of-sample) và kiểm định chéo (cross-validation)** — dự đoán ngoài mẫu là dự đoán cho một mẫu mà mô hình *không* được nhìn thấy lúc huấn luyện. Kiểm định chéo là cách lấy được dự đoán như vậy cho *mọi* mẫu: chia dữ liệu thành k phần, lần lượt giữ một phần ra ngoài, huấn luyện trên $k - 1$ phần còn lại rồi đoán cho phần bị giữ. Không có nó thì mọi phép “dùng mô hình soi nhãn” đều vô nghĩa.

CÂU HỎI MỞ ĐẦU

- Q1. Hai người gán nhãn giỏi chỉ đồng ý với nhau ở 85% số ảnh. Vậy con số accuracy 92% mà bạn vừa báo cáo đang đo cái gì — sự thật, hay sự bất đồng của con người?
- Q2. Vì sao 3% nhãn sai lại có thể tàn phá một mô hình nặng hơn 10% nhãn sai, dù ít hơn ba lần?
- Q3. Tập validation của ImageNet — thứ cả ngành dùng để xếp hạng suốt hơn một thập kỷ — chứa khoảng 6% nhãn sai. Bao nhiêu phần của bảng xếp hạng đó là cuộc đua khớp lỗi nhãn?
- Q4. Bạn có 100 nghìn ảnh, nghi vài nghìn nhãn sai, và không có ngân sách xem lại bằng mắt. Làm sao bắt chính mô hình chỉ ra chúng mà không để nó bao che cho lỗi nó đã học thuộc?

6.1 Đặt vấn đề

Ở chương trước, bài toán đã được định nghĩa: đơn vị phân tích là gì, đầu ra dạng nào, mẫu lấy từ đâu. Bây giờ có người ngồi gán nhãn, và câu hỏi trở nên khó chịu hơn nhiều: *nhãn của bạn đến từ con người, và con người thì không nhất quán — vậy trần hiệu năng của hệ thống nằm ở đâu, và làm sao nâng nó lên với một ngân sách hữu hạn?*

Câu hỏi này thường bị bỏ qua vì nó không có vẻ là một câu hỏi kỹ thuật: tập dữ liệu bị coi như hằng số của bài toán, còn công sức thì dồn hết vào phần *mô hình*. Cái giá không hiện ra ngay. Nó hiện ra ba tháng sau, dưới dạng một câu hỏi không trả lời được: đã thử mười kiến trúc, con số kẹt ở 91% và không nhúc nhích, tại sao? Nếu hai người gán nhãn giỏi chỉ đồng ý với nhau ở 90% số ảnh thì 91% chính là chỗ mọi thứ phải dừng, và mười kiến trúc kia chưa bao giờ là biến số quyết định.

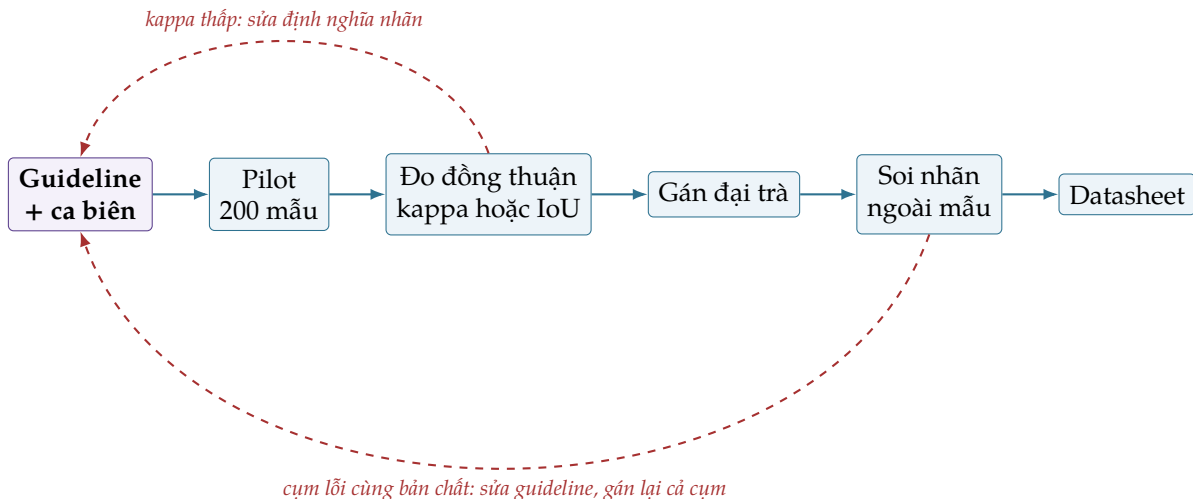
Người đọc đã làm SLAM thấy ngay chuyện này quen. Trong benchmark visual-inertial, **chân trị** (*ground truth*) là đầu ra của một hệ đo khác: Vicon, Leica hay RTK-GNSS. Hệ đó có sai số riêng và độ trễ riêng. Phép **hiệu chỉnh ngoại** (*extrinsic calibration*) nối nó với camera cũng chỉ là một ước lượng, và sai số của phép hiệu chỉnh cộng thẳng vào chân trị. Khi sai số quỹ đạo của thuật toán tụt xuống cùng bậc độ lớn với sai số của hệ tham chiếu, bảng so sánh giữa các thuật toán thôi không còn đo cái nó tự nhận là đo. Chất lượng nhãn trong thị giác máy tính là đúng hiện tượng đó, chỉ khác là ít ai nói ra.

TRỰC GIÁC

Coi tập nhãn như một hệ đo tham chiếu chứ không như chân lý. Mọi hệ đo có ba đặc trưng: một độ phân giải (chi tiết nhỏ hơn mức này thì nó không thấy), một sai số ngẫu nhiên (đo lại thì lệch), và một sai số hệ thống (lệch cùng một hướng mọi lần). Ba thứ này quyết định bạn được phép kết luận gì. Một thước kẹp chia vạch 0,1 mm không cho phép bạn tranh luận về chênh lệch 0,02 mm, dù có đo bao nhiêu lần đi nữa; và nếu thước bị cong, đo nhiều lần chỉ làm bạn tự tin hơn vào một con số sai.

I 6.2 Gán nhãn là một quy trình đo (Annotation as Measurement)

Nói gọn thì: chất lượng nhãn không phải thứ đo được ở khâu nghiệm thu, mà là thứ *được tạo ra* ở khâu quy trình. Kiểm tra cuối chỉ bắt được nhiều ngẫu nhiên; nó gần như mù với nhiều hệ thống, vì nhiều hệ thống trông hoàn toàn nhất quán nên trông giống như đúng. Hình 6.1 cho thấy điều làm nên khác biệt không phải các khối, mà là hai vòng phản hồi quay ngược về guideline.



Hình 6.1. Quy trình gán nhãn. Các khối là phần ai cũng làm; hai mũi tên đứt đỏ là phần quyết định chất lượng. Vòng trên bắt lỗi *định nghĩa nhãn* khi còn rẻ (200 mẫu); vòng dưới bắt lỗi *hệ thống* sau khi đã có mô hình. Bỏ hai vòng này thì quy trình biến thành dây chuyền một chiều, và mọi hiểu nhầm ban đầu được nhân lên đúng bằng kích thước tập dữ liệu.

A. Từ “có nhãn” tới “biết nhãn tốt tới đâu”

I Mạch 1 — bốn bước dựng trần hiệu năng

1 Vấn đề

chất lượng nhãn không tự có.

Giải pháp

viết guideline kèm ca biên trước khi gán mẫu đầu tiên, chạy pilot vài trăm mẫu, đọc lại chỗ bất đồng, rồi *sửa guideline* và gán lại. Sản phẩm chính của pilot là vòng sửa guideline chứ không phải mấy trăm nhãn kia.

Hệ quả

chất lượng được quyết định ở khâu hiệu chuẩn, không ở khâu đọc số.

2 Vấn đề

không biết nhãn tốt tới đâu.

Giải pháp

đo **mức đồng thuận giữa người gán nhãn** (*inter-annotator agreement*) bằng cách cho hai người gán chồng lên nhau một tập con.

Hệ quả

nếu hai người chỉ đồng ý ở 85% số mẫu thì accuracy 92% trên test không nói về sự thật, nó nói về mức mô hình khớp với *nhiều của người gán*. Đồng thuận là trần trên, và mọi phần trăm giành được phía trên trần đó đáng nghi hơn đáng mừng.

3 Vấn đề

không phải nhiễu nào cũng như nhau.

Giải pháp

tách **nhiều ngẫu nhiên** (*random label noise*) (người gán mệt, bấm nhầm, kéo lệch một biên — phân bố đều, chủ yếu làm chậm hội tụ) khỏi **nhiều hệ thống** (*systematic label noise*) (guideline mơ hồ ở một chỗ cụ thể: mọi người gán đều bỏ vật nhỏ hơn 20 pixel, đều coi xe đạp có người ngồi là “người”).

Hệ quả mạng học được cái thứ hai và tái tạo nó rất trung thành, nên 3% nhiễu hệ thống nguy hiểm hơn 10% nhiễu ngẫu nhiên; việc mạng sâu thừa sức thuộc lòng cả một tập nhãn ngẫu nhiên hoàn toàn [54] cho thấy **sức chứa** (*capacity*) không phải hàng rào bảo vệ nào cả.

4 Vấn đề

Giả định

Hệ quả

ta coi chân trị là chân lý.

nó cũng chỉ là một ước lượng, có sai số của thiết bị đo và của người gán.

khi sai số của mô hình tụt xuống dưới sai số của chân trị, mọi bảng so sánh mất ý nghĩa và bạn đang tối ưu nhiễu. Đây là lúc hành động đúng là đi cải thiện phép đo, không phải thử kiến trúc thứ mười một.

B. Cohen kappa, và vì sao “đồng ý 95%” có thể là con số rỗng

Đo đồng thuận bằng tỉ lệ đồng ý trần (*raw agreement*) có một lỗ hổng chết người: khi một lớp hiếm, hai người gán bữa cũng đồng ý gần như tuyệt đối. *Cohen's kappa* [64] sửa chuyện đó bằng cách trừ đi phần đồng thuận giải thích được bởi ngẫu nhiên:

$$\kappa = \frac{p_o - p_e}{1 - p_e}.$$

Công thức này nói: *trong khoảng cải thiện còn lại phía trên mức ngẫu nhiên, hai người gán đã đi được bao nhiêu phần*. Mẫu số $1 - p_e$ là toàn bộ dư địa; tử số là phần thực sự đạt được. Ở đây p_o là tỉ lệ đồng ý quan sát được, p_e là tỉ lệ đồng ý kỳ vọng nếu hai người gán độc lập nhưng giữ nguyên tần suất từng lớp của mình.

Hai trường hợp tính tay được, cùng cỡ mẫu 100 ảnh nhị phân, cho thấy toàn bộ vấn đề:

- **Lớp dương phổ biến.** Cùng dương 45, cùng âm 40, bất đồng 15 (A dương B âm 10, A âm B dương 5). Vậy $p_o = 0,85$; A nói dương 55% số lần, B nói dương 50%, nên $p_e = 0,55 \cdot 0,50 + 0,45 \cdot 0,50 = 0,50$ và $\kappa = 0,35/0,50 = 0,70$.
- **Lớp dương hiếm.** Cùng dương 3, cùng âm 92, bất đồng 5. Bây giờ $p_o = 0,95$ — nghe tuyệt vời — nhưng $p_e = 0,06 \cdot 0,05 + 0,94 \cdot 0,95 \approx 0,896$, nên $\kappa \approx 0,054/0,104 \approx 0,52$.

Bảng đồng thuận trông đẹp hơn hẳn lại cho kappa thấp hơn hẳn. Bài học không phải “kappa tốt hơn accuracy”. Với lớp hiếm, cả đồng thuận trần lẫn accuracy đều bị nền âm tính khổng lồ kéo lên, vì mẫu số của chúng là toàn bộ tập chứ không phải riêng phần đáng quan tâm. Hai con số vì thế nói dối theo cùng một cách. Đây chính là lý do B/07 phải dành hẳn một mục cho việc chọn metric.

→ **tiền đề B/07** — nền âm tính khổng lồ kéo cả accuracy lẫn đồng thuận trần lên; ở đó cùng cơ chế này làm AUROC nói dối khi lớp dương chỉ chiếm một phần trăm.

Kappa chỉ định nghĩa cho nhãn phân loại rời rạc. Với mask, bbox hay keypoint, hãy đo đồng thuận bằng chính metric hình học của bài toán: IoU giữa hai mask của hai người gán, hoặc sai số pixel giữa hai bộ keypoint. Và đọc *phân bố* của con số đó chứ đừng đọc trung bình, vì bất đồng giữa người gán dồn hết vào phần đuôi — chỗ vật nhỏ và biên mờ. Cách này không có nền lý thuyết gọn như kappa nhưng trả lời đúng câu hỏi cần: mô hình còn bao nhiêu chỗ trống trước khi chạm trần người.

6.3 Lỗi nhãn trong chính tập test chuẩn (Label Errors in Benchmarks)

Có một phản bác hợp lý: nhiễu nhãn nghe như vấn đề của dữ liệu tự thu thập, còn ImageNet hay COCO đã qua bao năm soi xét. Phản bác này sai theo một cách đã được đo: khảo sát mười tập

test chuẩn của học máy tìm thấy tỉ lệ lỗi nhãn trung bình vài phần trăm, riêng tập validation của ImageNet khoảng 6% [19]. Hệ quả sắc hơn con số: khi sửa lại các nhãn sai đó, *thứ tự xếp hạng giữa các mô hình có thể đảo*, và mô hình lớn hơn — thứ khớp tập test gốc tốt hơn — hoá ra cũng là thứ khớp nhiều nhãn tốt hơn.

Từ đó ra câu hỏi thực dụng: tìm nhãn sai trong 100 nghìn ảnh bằng cách nào? Câu trả lời là dùng chính mô hình làm cảm biến, ở ba mức từ rẻ tới đắt:

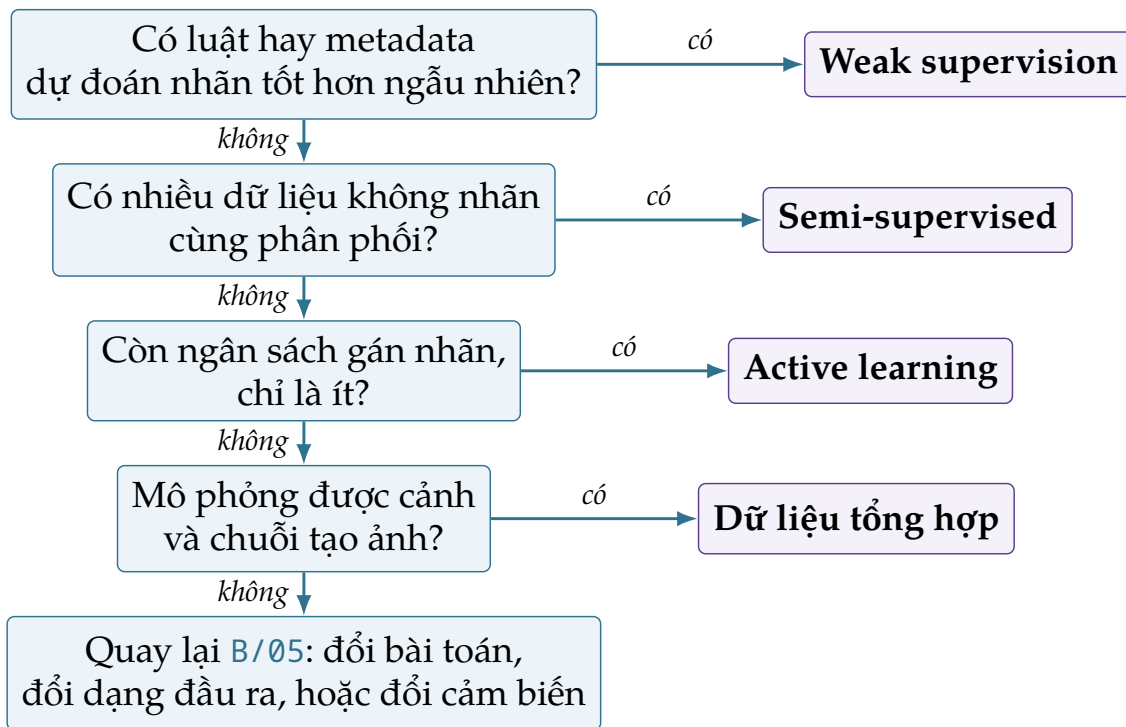
- **Lọc theo loss.** Xếp hạng mẫu theo giá trị mất mát sau khi huấn luyện; đuôi cao chứa cả nhãn sai lẫn mẫu khó. Rẻ nhất, nhiều nhất, nhưng thường đủ để bắt đầu.
- **Confident learning** [65] — ước lượng ma trận chung giữa nhãn quan sát được và nhãn tiềm ẩn từ xác suất dự đoán *ngoài mẫu*, rồi suy ra ngưỡng cắt từ chính ma trận đó thay vì đặt ngưỡng bằng tay. Đây là mức nên dùng mặc định.
- **influence function** — hỏi mẫu huấn luyện nào đang kéo dự đoán trên tập val đi sai. Đắt nhất, nhưng là công cụ duy nhất trả lời được câu hỏi “mẫu nào gây ra lỗi này”.

Cả ba đều giả định mô hình đủ tốt để sự bất đồng của nó mang tín hiệu; với một mô hình chưa học được gì, danh sách nghi vấn chỉ là danh sách ngẫu nhiên. Và cả ba đều đánh dấu nhầm các mẫu *khó nhưng đúng*, nên đây không phải bộ lọc tự động mà là bộ sắp thứ tự cho mắt người: nó biến “xem 100k ảnh” thành “xem 2k ảnh đáng ngờ nhất”. Ai để nó tự xoá mẫu sẽ vô tình xoá đúng phần đuôi khó — chính phần mô hình cần nhất — và tạo ra một tập dữ liệu sạch một cách giả tạo. Khi phát hiện một cụm lỗi cùng bản chất (mọi vật bị che quá nửa đều bị bỏ), đó là lỗi guideline chứ không phải lỗi người gán: sửa guideline rồi gán lại cả cụm, đúng vòng phản hồi dưới của Hình 6.1.

↔ **tiền đề E/24** — cùng một kỹ thuật “dùng mô hình soi dữ liệu”, ở đó dùng để phân rã lỗi theo loại thay vì để tìm nhãn sai.

6.4 Ngân sách hữu hạn: bốn đường thoát (Label-Efficient Learning)

Giả sử quy trình đã tốt và ta biết nhãn đáng tin tới đâu. Vẫn còn ràng buộc cuối: không đủ tiền gán nhãn. Cơ chế của bốn hướng thoát đều đơn giản, nên đó không phải chỗ đáng học. Chỗ đáng học là *mỗi hướng mua sự rẻ bằng một giả định khác nhau*. Hình 6.2 xếp bốn hướng theo đúng thứ tự để kiểm chứng của giả định đó.



Hình 6.2. Cây quyết định chọn hướng giảm chi phí nhãn. Thứ tự các câu hỏi không tùy tiện: nó đi từ hướng có giả định dễ kiểm chứng nhất (bạn *biết* mình có luật hay không) tới hướng có giả định khó kiểm chứng nhất (mô phỏng “đủ gần” là một phán đoán). Nhánh cuối cùng — không hướng nào áp dụng được — là một kết quả hợp lệ, và nó nói rằng vấn đề nằm ở khâu thiết kế bài toán chứ không ở khâu dữ liệu.

Bảng 6.1. Bốn cách giảm chi phí nhãn. Cột cuối đáng đọc nhất: mỗi hướng hòng theo một kiểu riêng, và cả bốn kiểu hòng đều tạo ra nhiều *hệ thống*.

Hướng	Cơ chế	Giả định	Hòng khi nào
giám sát yếu (<i>weak supervision</i>)	Sinh nhãn từ heuristic, luật, metadata	Heuristic tốt hơn ngẫu nhiên và sai một cách đo được	Nhiều bám đúng thiên lệch của heuristic → mô hình học lại chính heuristic đó
học chủ động (<i>active learning</i>)	Chọn mẫu đáng gán nhất theo độ bất định	Ước lượng bất định đáng tin	Bất định kém hiệu chuẩn; tập thu được lệch phân phối nên khó tái dùng cho mô hình khác
học bán giám sát (<i>semi-supervised learning</i>), <i>pseudo-labeling</i>	Khai thác dữ liệu không nhãn quanh cụm	Biên quyết định nằm ở vùng mật độ thấp	Mô hình khởi đầu lệch → vòng lặp tự củng cố lỗi, càng train càng tự tin sai
dữ liệu tổng hợp (<i>synthetic data</i>), sim2real	Render cảnh có nhãn miễn phí	Mô phỏng đủ gần ở những đặc trưng mô hình thật sự dùng	Khoảng cách miền nằm ở khâu tạo ảnh (nhiều cảm biến, ISP, quang sai) chứ không ở khâu hình học

Bảng 6.1 có một quy luật ngầm đáng nhớ: cả bốn hướng đều *đổi nhiều ngẫu nhiên đắt tiền lấy nhiều hệ thống rẻ tiền* — hợp lý khi bạn biết mình đang đổi gì, là bấy khi không biết. Đây chính là trục đánh đổi “thêm dữ liệu hay thêm giả định” ở B/04, lần này xuất hiện dưới dạng một quyết định về ngân sách. Riêng sim2real, chỗ hờ hiếm khi nằm ở hình học (render chính xác tuyệt đối) mà nằm ở chuỗi tạo ảnh: nhiễu đọc của cảm biến, đường cong đáp ứng, khử nhiễu và làm nét của ISP, ảnh mờ do chuyển động — đó là lý do A/02-vat-ly-tao-anh có giá trị rất thực tế ở đây.

↔ **ràng buộc bởi A/02** — khoảng cách giữa ảnh render và ảnh thật nằm ở chuỗi tạo ảnh, nên nó bị chặn bởi mức bạn mô phỏng được cảm biến và ISP.

↔ **cùng nguyên lý với B/04** — cả bốn hướng đều là “thêm giả định thay cho thêm dữ liệu”, đúng trực đánh đổi đã dựng ở chương nền tảng.

6.5 Ba vấn đề âm thầm (Duplicates, Missingness, Documentation)

Ba thứ còn lại ít hào nhoáng nhưng gây thiệt hại lớn, và chúng có chung một đặc điểm: không thứ nào báo lỗi.

- **trùng lặp gần đúng (*near-duplicate*)**. Khi kết quả đẹp một cách khó hiểu, đây nên là nghi ngờ đầu tiên. Ảnh crawl từ web có vô số bản resize, crop, nén lại, thêm watermark; video thì mọi frame liền kề gần như trùng nhau.
 - **Vì sao hash thất bại** — chỉ cần nén lại một lần là mã băm đổi hoàn toàn, trong khi ảnh với mắt người vẫn y hệt.
 - **Công cụ đúng** — khử trùng lặp trên không gian *embedding* với một ngưỡng khoảng cách, hoặc trên *perceptual hash*.
 - **Nổi sang nền có sẵn** — hai frame cách nhau 33 ms không phải hai quan sát độc lập, đúng như hai keyframe liền kề trong bundle adjustment không cho hai ràng buộc độc lập.
- **dữ liệu thiếu (*missing data*): ba loại, không phải một**. Cách dữ liệu bị thiếu tự nó mang thông tin.
 - **MCAR** (thiếu hoàn toàn ngẫu nhiên) — bỏ mẫu là an toàn, chỉ tốn dữ liệu.
 - **MAR** (thiếu phụ thuộc các biến *quan sát được*) — bù được bằng mô hình.
 - **MNAR** (thiếu phụ thuộc chính giá trị bị thiếu) — trường hợp độc: ảnh quá tối nên người gán bỏ qua, cảm biến depth trả về lỗ đúng ở bề mặt bóng và vật trong suốt. Áp cách xử lý của MCAR lên đây không loại được thiên lệch mà tạo ra một thiên lệch mới, và thiên lệch mới thì vô hình vì nó vừa được “dọn sạch”.
- **Trí nhớ của tổ chức**. Sáu tháng sau không ai còn nhớ tập dữ liệu này là gì: thư mục còn đó nhưng lý do chọn nguồn, tiêu chí loại ảnh, phiên bản guideline, mức đồng thuận đo được đều bốc hơi. *Datasheets for Datasets* [61] và *Model Cards* [63] là câu trả lời rẻ đến bất ngờ: vài trang về động cơ thu thập, quy trình, thành phần, lớp bị thiếu, và những cách dùng *không* nên dùng. Đây là một dạng cụ thể của khoản nợ kỹ thuật ẩn trong hệ thống học máy [43] — chi phí không nằm ở lúc viết mà ở lúc không viết.

Và cuối cùng, thứ không công cụ nào thay thế được: *ngồi xem 200 ảnh bằng mắt*. Đây là kinh nghiệm thực hành, không phải kết quả đo. Nhưng nó nhất quán tới mức đáng coi là quy tắc. Lần nào cũng lộ ra thứ mà không thống kê nào cho thấy: một lớp được gán bằng hai định nghĩa ở hai đợt, hay một watermark xuất hiện đúng ở các mẫu dương. Lý do đơn giản là thống kê chỉ đếm được thứ bạn đã nghĩ ra để đếm.

↔ **hệ quả của B/05** — mọi thứ chương này phải dọn dẹp đều bắt nguồn từ định nghĩa nhãn và khung lấy mẫu đã chốt ở khâu thiết kế bài toán.

6.6 Giới hạn và trường hợp hỏng

Toàn bộ chương này đứng trên vài giả định nên nói rõ. Khái niệm “trần đồng thuận” giả định có một sự thật khách quan mà người gán đang cố xấp xỉ; với các nhãn mang tính đánh giá — mức độ hư hỏng, độ thẩm mỹ, mức nguy hiểm — giả định đó sai, và bất đồng giữa người gán là tín hiệu chứ không phải nhiễu. Khi đó thứ cần làm không phải ép về một nhãn mà là giữ lại phân bố nhãn và huấn luyện với nhãn mềm.

Confident learning và mọi biến thể “dùng mô hình soi nhãn” giả định nhiều nhãn độc lập với đặc trưng đầu vào khi đã biết lớp thật; với nhiều hệ thống gắn với ngoại hình, giả định này vỡ đúng ở chỗ ta quan tâm nhất và phương pháp sẽ báo sạch một tập vẫn còn lệch. Đo đồng thuận cũng hỏng lạng lẽ khi hai người gán do cùng một người huấn luyện: họ chia sẻ cùng một hiểu nhầm nên kappa cao mà cả hai cùng sai — thuốc giải là để một người thứ ba, không đọc guideline của bạn, gán mù một tập nhỏ.

CẠM BÃY

Cạm bẫy phổ biến nhất là kiểm tra chất lượng bằng chính mô hình đã được train trên tập cần kiểm tra, rồi kết luận “dữ liệu sạch” vì loss thấp đều. Mô hình đã hấp thụ nhiều hệ thống và bây giờ đồng ý với nó — bạn đang hỏi bị cáo xem mình có tội không. Muốn tín hiệu có nghĩa, dự đoán dùng để soi nhãn phải là dự đoán *ngoài mẫu* (**kiểm định chéo** (*cross-validation*)), đúng như confident learning yêu cầu.

CẦN NHỚ

Mức đồng thuận giữa người gán nhãn là trần trên của mọi con số bạn báo cáo, nên hãy đo nó *trước* khi huấn luyện mô hình đầu tiên. Và 3% nhiễu hệ thống hại hơn 10% nhiễu ngẫu nhiên, vì mô hình học được cái thứ nhất và chỉ chậm hội tụ vì cái thứ hai. [T2]

6.7 Thực hành

Repo và tài nguyên. [cleanlab/cleanlab](#) (confident learning); [visual-layer/fastdup](#) và [idealo/imagededup](#) (trùng lặp gần đúng); [voxel51/fiftyone](#) (soát dữ liệu có hệ thống); [snorkel-team/snorkel](#) (weak supervision); [baal-org/baal](#) và [modAL](#) (active learning); [lightly-ai/lightly](#) (chọn mẫu theo embedding). Dữ liệu tổng hợp: Blender kèm [BlenderProc](#), hoặc NVIDIA Omniverse Replicator. Danh sách này chốt tại tháng 9/2026 và tên công cụ sẽ cũ trong 6–12 tháng; hạ tầng (FiftyOne, Blender) bền hơn nhiều so với tên phương pháp đang dẫn đầu.

Câu hỏi kiểm tra hiểu. Nhãn nhiễu ngẫu nhiên 10% và nhãn nhiễu hệ thống 3% — cái nào hại hơn và cơ chế gây hại khác nhau ở đâu? Nếu mức đồng thuận giữa người gán nhãn là 85% thì con số accuracy 92% trên test đang nói điều gì? Vì sao khử trùng lặp bằng mã băm byte gần như vô dụng với ảnh web?

Bài tập phụ. Làm hỏng 10% nhãn CIFAR-10 ngẫu nhiên, train một mạng nhỏ, dùng cleanlab tìm lại, báo cáo precision/recall của việc phát hiện. Lặp lại với nhiễu hệ thống (đổi lớp A thành B ở 3% số mẫu): trường hợp sau khó phát hiện hơn hẳn dù tỉ lệ nhiễu thấp hơn ba lần.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Chân trị	Tập nhãn được coi là đúng khi chấm điểm. Nó do thiết bị và con người tạo ra nên có sai số riêng, và khi sai số của mô hình tụt xuống dưới sai số đó thì mọi bảng so sánh mất ý nghĩa.
••• <i>Inter-annotator agreement</i>	Mức đồng thuận giữa người gán nhãn: tỉ lệ hai người gán độc lập trùng nhau trên cùng một tập con. Nó là trần trên của mọi con số bạn báo cáo, nên phải đo trước khi huấn luyện mô hình đầu tiên.
••• Nhiễu hệ thống	Một quy tắc sai được lặp lại nhất quán, ví dụ mọi vật nhỏ hơn 20 pixel đều bị bỏ. Mô hình học được nó và tái tạo nó trên dữ liệu mới, nên nó không tự lộ ra ở khâu nghiệm thu.
•• Nhiễu ngẫu nhiên	Sai sót rải đều: người gán mệt, bấm nhầm, kéo lệch một biên. Nó chủ yếu làm chậm hội tụ chứ không cài một thiên lệch vào mô hình.
•• Vì sao 3% nhiễu hệ thống hại hơn 10% nhiễu ngẫu nhiên?	Vì mạng học được quy tắc sai và áp lại nó cho dữ liệu mới. Nhiễu ngẫu nhiên thì trung bình về không, còn dung lượng lớn không cứu được: mạng sâu thừa sức thuộc lòng cả một tập nhãn ngẫu nhiên.
••• Cohen kappa	Mức đồng thuận sau khi trừ đi phần trùng nhau do may rủi, chia cho phần dư địa còn lại. Nó chỉ định nghĩa cho nhãn rời rạc; với mask hay keypoint phải đo bằng IoU hoặc sai số pixel.
•• Hai người gán đồng ý 95% mà kappa chỉ 0,52 — nguyên nhân?	Lớp dương quá hiếm, nên nền âm tính khổng lồ kéo tỉ lệ đồng ý trần lên. Phân biệt bằng cách nhìn tỉ lệ lớp: nếu lớp dương dưới vài phần trăm thì luôn đọc kappa, đừng đọc tỉ lệ đồng ý.
••• Confident learning	Ước lượng ma trận chung giữa nhãn quan sát được và nhãn tiềm ẩn, rồi suy ngưỡng cắt từ chính ma trận đó. Nó giả định nhiều độc lập với ngoại hình khi đã biết lớp thật, nên vỡ với nhiễu hệ thống gắn ngoại hình.
•• Vì sao phải soi nhãn bằng dự đoán ngoài mẫu?	Vì mô hình đã hấp thụ nhiễu hệ thống và bây giờ đồng ý với nó. Dự đoán dùng để soi nhãn phải là dự đoán ngoài mẫu, lấy bằng kiểm định chéo.
•• Weak supervision	Sinh nhãn từ heuristic, luật hoặc metadata thay vì từ người. Giá phải trả là nhiễu bám đúng thiên lệch của heuristic, nên mô hình học lại chính heuristic đó.
•• Active learning	Chỉ gán nhãn những mẫu mô hình bất định nhất. Giá phải trả là tập thu được lệch phân phối, nên khó tái dùng cho một mô hình khác.
• Pseudo-labeling	Dùng dự đoán của chính mô hình làm nhãn cho dữ liệu không nhãn. Nó giả định biên quyết định nằm ở vùng mật độ thấp, và hóng thành vòng lặp tự củng cố lỗi nếu mô hình khởi đầu lệch.
•• Sim2real	Huấn luyện trên ảnh render rồi triển khai trên ảnh thật. Khoảng cách hiếm khi nằm ở hình học mà nằm ở chuỗi tạo ảnh: nhiễu cảm biến, đường cong đáp ứng, khử nhiễu của ISP, ảnh mờ do chuyển động.
••• Trùng lặp gần đúng	Hai ảnh khác nhau từng byte nhưng giống hệt với mắt người: bản nén lại, cắt cúp, đổi kích thước, hay hai khung hình liên nhau. Phải khử trước khi chia dữ liệu.
•• Vì sao mã băm byte vô dụng với trùng lặp gần đúng?	Vì chỉ cần nén lại một lần là mã băm đổi hoàn toàn, trong khi nội dung ảnh không đổi. Công cụ đúng là khoảng cách trên embedding hoặc <i>perceptual hash</i> .
•• Mô hình mới vọt 8 điểm trên test mà không đổi gì lớn — nghi ngờ đầu tiên?	Trùng lặp gần đúng giữa train và test, thường do ảnh crawl web hoặc khung hình liên nhau. Phân biệt với “mô hình thật sự tốt” bằng cách khử trùng lặp theo embedding rồi đo lại: nếu con số tụt hẳn về mức cũ thì thủ phạm là bản sao.
• MNAR	Dữ liệu thiếu mà lý do thiếu phụ thuộc chính giá trị bị thiếu, ví dụ ảnh quá tối nên người gán bỏ qua. Xử lý nó như thiếu ngẫu nhiên sẽ tạo ra một thiên lệch mới đã được “dọn sạch” nên vô hình.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
• Datasheet for datasets	Vài trang ghi nguồn, quy trình gán nhãn, mức đồng thuận đo được, các lớp bị thiếu, và những cách dùng không nên dùng. Không có nó thì sáu tháng sau tập dữ liệu thành một thư mục bí ẩn.
•• Khi nào bắt đầu của người gán là tín hiệu?	Với nhãn mang tính đánh giá: mức hư hỏng, độ thẩm mỹ, mức nguy hiểm. Khi đó hãy giữ lại phân bố nhãn và huấn luyện với nhãn mềm thay vì ép về một nhãn.
•• Thứ mười kiến trúc mà con số kết ở 91% — đo gì tiếp theo?	Đo mức đồng thuận giữa hai người gán trên hai trăm mẫu. Nếu nó cũng quanh 90% thì bạn đã chạm trần và phải đầu tư vào nhãn; nếu nó 98% thì vấn đề đúng là ở mô hình.

••• phải thuộc •• nên nắm • biết là có

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
cleanlab	Cài đặt trực tiếp confident learning: ước lượng ma trận chung giữa nhãn quan sát và nhãn tiềm ẩn từ dự đoán ngoài mẫu	Chính nhóm này công bố khảo sát lỗi nhãn trên mười tập test chuẩn
FiftyOne (voxel51)	Biến “xem 200 ảnh bằng mắt” thành thao tác có hệ thống: lọc theo lát, sắp theo độ nghi ngờ, so hai phiên bản nhãn	Hạ tầng soát dữ liệu, bền hơn tên mô hình
Trợ lý gán nhãn bằng mô hình nền tảng (SAM 3, Grounding DINO)	Chính là giám sát yếu, chỉ khác là nguồn nhãn thô đến từ một mô hình lớn thay vì từ luật viết tay	Tên mô hình sẽ đổi nhanh; cơ chế và cái giá (nhiều hệ thống theo thiên lệch của mô hình nguồn) thì không
snorkel	Gộp nhiều luật nhiều thành nhãn xác suất, mô hình hoá luôn độ tin cậy của từng luật	Ý tưởng đã lan sang các pipeline lọc dữ liệu huấn luyện mô hình ngôn ngữ
Dataset card và model card trên Hugging Face Hub	Đúng nội dung của Datasheets và Model Cards, được đưa thành mục mặc định khi công bố tập dữ liệu	Bảng chứng rõ nhất rằng ý tưởng tài liệu hoá đã thành chuẩn ngành

MINI PROJECT SLAM • Đo mức đồng thuận của chính bạn trên bài toán đóng vòng • 1–2 ngày

Mục tiêu	biến “trần hiệu năng” từ một khái niệm thành một con số của riêng bạn, trên đúng quyết định mà mọi hệ đóng vòng phải đưa ra: hai khung hình này có phải cùng một chỗ hay không.
Dữ liệu	200 cặp khung hình rút từ hai chuỗi EuRoC (ví dụ MH_03 và V1_02), trộn ba nhóm với tỉ lệ ngang nhau: cặp thật sự cùng chỗ, cặp gần nhau về thời gian nhưng khác chỗ, và cặp ngẫu nhiên.
Các bước	viết guideline một trang <i>trước</i> khi gán mẫu đầu tiên, chốt rõ ca biên (bao nhiêu phần cảnh chồng lấn thì tính là cùng chỗ, lệch góc nhìn tới đâu thì thôi, ảnh mờ tới mức nào thì bỏ). Gán lần một. Chờ ba tới năm ngày. Gán lại đúng 200 cặp đó với thứ tự đã xáo trộn. Lập bảng 2×2 giữa hai lần gán, tính p_o , p_e và kappa bằng tay.
Xong khi	bạn có con số kappa của chính mình kèm bảng 2×2 , và một danh sách ít nhất mười cặp bạn tự mâu thuẫn với bản thân, mỗi cặp kèm đúng một dòng sửa guideline.
Nối với	B/07-chia-du-lieu-va-danh-gia dùng lại chính 200 cặp này làm tập test có nhãn cho mini project chương sau; E/24-thi-nghiem-va-ablation đọc kappa như trần trên khi so hai bộ phát hiện đóng vòng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Nó đo mức mô hình khớp với *nhiều của người gán*, không đo sự thật. 85% là trần: mọi phần trăm giành được phía trên nó chỉ nói rằng mô hình đã học luôn cả các hiểu nhầm nhất quán trong guideline.
- A2. Vì hai loại nhiễu có cơ chế gây hại khác hẳn. Nhiễu ngẫu nhiên phân bố đều nên chủ yếu làm chậm hội tụ; nhiễu hệ thống là một quy tắc sai được lặp lại, và mô hình học được quy tắc đó rồi tái tạo nó trên dữ liệu mới.
- A3. Để đảo thứ hạng: khi sửa lại nhãn, mô hình lớn hơn — thứ khớp tập test gốc tốt hơn — hoá ra cũng là thứ khớp nhiều nhãn tốt hơn. Một phần cuộc đua nhiều năm là cuộc đua khớp lỗi nhãn, không phải khớp thực tế.
- A4. Bằng dự đoán *ngoài mẫu*: chia k phần, mỗi mẫu chỉ được chấm bởi mô hình chưa nhìn thấy nó, rồi dùng confident learning để suy ngưỡng từ ma trận nhầm lẫn. Dùng mô hình đã train trên chính tập đó thì nó chỉ xác nhận nhiễu nó đã hấp thụ.

NÓI LẠI THẬT ĐƠN GIẢN

Tưởng tượng bạn thuê hai người chấm cùng một tập hai trăm bài văn, thang mười điểm. Chấm xong, bạn so hai bảng điểm và thấy trung bình họ lệch nhau một điểm rưỡi. Bây giờ bạn xây một cái máy chấm tự động và khoe rằng nó chỉ lệch nửa điểm so với bảng điểm của người thứ nhất. Nửa điểm đó không có nghĩa là máy giỏi hơn người. Nó chỉ có nghĩa là máy đã học được thói quen chấm của đúng một người, kể cả những chỗ người đó chấm sai.

Bài toán gốc là như vậy: thứ bạn dùng làm đáp án cũng do người làm ra, nên nó có sai số riêng. Mẹo thoát ra gồm hai bước rẻ tiền. Một, cho hai người chấm chồng lên nhau một phần nhỏ rồi đo xem họ lệch bao nhiêu; con số đó là mức tốt nhất mà máy có quyền đạt tới, và mọi thứ vượt qua nó nên bị nghi ngờ. Hai, viết trước cách xử lý các bài khó — bài lạc đề nhưng viết hay thì mấy điểm — rồi sửa bản hướng dẫn chấm, chứ không sửa từng bài một.

Cái giá là vài ngày làm việc nhằm chần chừ trước khi có mô hình đầu tiên, cộng với việc phải thừa nhận rằng một phần công sức trước đây của bạn đã đổ vào đuổi theo nhiễu.

Chỗ không rút gọn được: ranh giới giữa “bài bị chấm sai” và “bài khó nhưng chấm đúng” thì không nói gọn được. Mọi cách diễn đạt đơn giản đều biến nó thành một quy tắc máy móc, trong khi chính ranh giới đó là thứ mà cả người lẫn máy đang phải học.

Một câu: Trước khi hỏi mô hình giỏi tới đâu, hãy hỏi đáp án của bạn đúng tới đâu — vì con số thứ hai chặn trên con số thứ nhất.

6.8 Kết nối

Chương này là phần thực thi của [B/05-thiet-ke-bai-toan](#): ở đó ta quyết định nhãn là gì, ở đây ta đối mặt với việc nhãn *trở thành* gì sau khi qua tay người. Nó là tiền đề bắt buộc của [B/07-chia-du-lieu-va-danh-gia](#), vì trùng lặp gần đúng chính là một dạng rò rỉ và trần đồng thuận quyết định chênh lệch nào giữa hai mô hình mới đáng bàn. Nó đối lập hữu ích với [B/08-dich-chuyen-phan-phoi](#): ở đây nhãn sai vì con người không nhất quán, ở đó nhãn đúng nhưng thế giới đã đổi. Phần “dùng mô hình soi nhãn” quay lại đầy đủ ở [E/24-thi-nghiem-va-ablation](#).

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao mức đồng thuận giữa người gán nhãn là trần trên của accuracy, chứ không chỉ là một con số tham khảo?
2. Cùng một bảng đồng thuận cho $p_o = 0,95$ lại có thể cho kappa thấp hơn một bảng có $p_o = 0,85$. Cơ chế nào gây ra chuyện đó, và nó cảnh báo gì về accuracy?

3. Confident learning giả định gì về cấu trúc của nhiễu, và loại nhiễu nào làm giả định đó vỡ đúng ở chỗ ta cần nhất?
4. Vì sao dùng chính mô hình đã train trên tập đó để soi nhãn lại cho kết luận vô nghĩa, và sửa bằng cách nào?
5. Bạn có 500 giờ công gán nhãn: khi nào tiêu vào việc *gán lại* dữ liệu cũ có giá trị hơn gán thêm dữ liệu mới?
6. Trong cây quyết định ở Hình 6.2, vì sao câu hỏi về heuristic lại đứng trước câu hỏi về mô phỏng?

Chia dữ liệu và giao thức đánh giá (Data Splitting and Evaluation Protocol)

PHỤ THUỘC B/05-thiet-ke-bai-toan (đơn vị phân tích (*unit of analysis*) quyết định cách chia); B/06-chat-luong-du-lieu (trần đồng thuận và trùng lặp gần đúng). Chương này chuẩn bị trực tiếp cho E/24-thi-nghiem-va-ablation.

TÓM TẮT

Chương này nói về cách để có một con số dự báo được hiệu năng tương lai, và về mọi cách con số đó nói dối. Đọc xong, nhìn một bảng kết quả là bạn biết cách chia của nó giả định gì và metric của nó đang giấu loại lỗi nào. Bạn tính được chênh lệch bao nhiêu điểm mới đáng gọi là thật. Bạn thiết kế được một giao thức đánh giá dám đem ra tranh luận. Nếu chỉ nhớ một điều: mọi lỗi trong chương này đều làm kết quả *đẹp lên* chứ không báo lỗi, nên chúng chỉ bị phát hiện ở chỗ đắt nhất — sau khi triển khai — trừ khi bạn đi tìm chúng một cách có hệ thống.

KIẾN THỨC CẦN CÓ TRƯỚC

- **ba tập train / val / test, tham số và siêu tham số (hyperparameter)** — dữ liệu được cắt thành ba phần có ba việc khác nhau: *train* để thuật toán tự tìm ra các tham số của mô hình (trọng số); *val* để bạn chọn các siêu tham số — những số bạn đặt bằng tay trước khi huấn luyện như tốc độ học, số tầng, ngưỡng quyết định — và để biết khi nào nên dừng; *test* chỉ dùng đúng một lần ở cuối để đo. Nhầm vai trò của ba tập này là gốc của mọi vấn đề trong chương.
- **rò rỉ dữ liệu (data leakage)** — bất kỳ đường nào làm thông tin của tập kiểm tra lọt vào quá trình huấn luyện hoặc chọn mô hình — ảnh hưởng nằm hai bên, thống kê chuẩn hoá tính trên cả tập, hai khung hình liền nhau bị tách đôi, hoặc chính bạn xem kết quả test rồi đổi thiết kế. Đặc điểm nguy hiểm là nó không báo lỗi, chỉ làm con số đẹp lên.
- **metric và hàm mất mát (loss function)** — **không phải một** — hàm mất mát là đại lượng *khả vi* mà thuật toán tối ưu giảm xuống trong lúc huấn luyện, tính trên từng mẫu. Metric là đại lượng bạn báo cáo và dùng để ra quyết định, thường không khả vi nên không tối ưu trực tiếp được. Cả chương này nói về metric.
- **accuracy, precision, recall, TPR, FPR và tỉ lệ dương (prevalence)** — accuracy là tỉ lệ đúng trên toàn bộ mẫu. Precision là trong các lần báo “có” thì bao nhiêu phần đúng. Recall, còn gọi là TPR (tỉ lệ dương thật), là trong các ca “có” thật thì bắt được bao nhiêu phần. FPR là trong các ca “không” thật thì bao nhiêu phần bị báo nhầm thành “có”. Prevalence là tỉ lệ ca “có” thật trong toàn bộ dữ liệu — khi nó nhỏ, chẳng hạn 1%, bốn con số trên rời xa nhau tới mức đọc nhầm một cái là kết luận sai hoàn toàn.
- **đường cong ROC, đường cong PR, AUROC và AUPRC** — mọi bộ phân loại đều có một ngưỡng; kéo ngưỡng từ chặt sang lỏng thì được cả một dãy điểm làm việc thay vì một điểm. Vẽ dãy đó trên trục (FPR, TPR) ra đường cong ROC; trên trục (recall, precision) ra đường cong PR. Diện tích dưới mỗi đường là một con số tóm tắt, gọi lần lượt là AUROC và AUPRC.
- **metric riêng của từng bài toán: IoU, mAP, ATE, RPE** — IoU là tỉ số diện tích giao trên diện tích hợp của hai vùng, dùng để quyết định một hộp dự đoán có “khớp” hộp thật hay không; mAP là trung bình diện tích dưới đường PR lấy trên mọi lớp, và “mAP@0.5” nghĩa là coi khớp khi IoU ≥ 0.5 . Trong SLAM, ATE (*absolute trajectory error*) đo độ lệch của toàn bộ quỹ đạo so với quỹ đạo

thật, sau khi căn hai quỹ đạo về cùng hệ toạ độ. RPE (*relative pose error*) đo độ lệch của các đoạn dịch chuyển ngắn.

- **lát (slice), và mẫu (sample) — phân biệt hai nghĩa** — một lát là một tập con của tập kiểm tra được cắt theo một điều kiện có nghĩa vận hành: ảnh ban đêm, vật nhỏ, một camera cụ thể; đọc metric theo từng lát mới thấy được nhóm nào đang bị bỏ rơi. Từ mẫu thì có hai nghĩa: (a) một điểm dữ liệu đơn lẻ, ví dụ một ảnh; (b) một tập được rút ra từ tổng thể, như trong “lấy mẫu có hoàn lại”.
- **sai số chuẩn và khoảng tin cậy (confidence interval)** — sai số chuẩn ước lượng mức dao động của một con số nếu bạn lặp lại phép đo trên một tập khác cùng cỡ; khoảng tin cậy 95% là khoảng rộng khoảng hai lần sai số chuẩn về mỗi phía. Cần biết vì không có nó thì không cách nào phân biệt một cải tiến thật với nhiễu đo đạc.
- **mẫu số của một tỉ lệ** — con số nằm dưới gạch ngang. Nghe tầm thường, nhưng gần như mọi hiểu nhầm về metric là hiểu nhầm mẫu số: precision chia cho số lần hệ thống báo “có”, recall chia cho số ca thật sự dương, còn FPR chia cho số ca thật sự âm. Ba mẫu số khác nhau, nên ba con số kể ba câu chuyện khác nhau về cùng một lỗi.
- **ma trận nhầm lẫn (confusion matrix)** — bảng đếm bốn kết cục có thể xảy ra khi một hệ thống trả lời “có/không” cho một câu hỏi có đáp án “có/không”: báo có và đúng, báo có và sai, báo không và sai, báo không và đúng. Mọi metric phân loại trong chương đều là một phân số lấy từ bốn số này.
- **trung bình điều hoà (harmonic mean)** — nghịch đảo của trung bình cộng các nghịch đảo. Tính chất cần nhớ là nó bị con số nhỏ kéo xuống mạnh hơn trung bình cộng, nên nó thường cho sự cân bằng giữa hai đại lượng. F1 dùng nó chính vì lý do đó.
- **điểm vận hành (operating point)** — một ngưỡng cụ thể mà bạn chọn để biến điểm số liên tục của mô hình thành quyết định “có/không”. Đổi ngưỡng thì mọi metric đổi theo, nên báo cáo một metric mà không nói ngưỡng là báo cáo nửa thông tin.
- **bootstrap** — cách ước lượng độ dao động của một con số khi không có công thức giải tích: lấy mẫu có hoàn lại từ chính tập dữ liệu đã có, tính lại con số, lặp vài nghìn lần, rồi đọc phân vị. Cần nó vì mAP, HOTA hay AUPRC không phải trung bình của các mẫu độc lập nên không có công thức sai số chuẩn.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao một mô hình đạt 99% accuracy vẫn có thể vô dụng hoàn toàn, và bạn phát hiện điều đó bằng đúng hai con số nào?
- Q2. Bạn không hề chạm vào nhãn của tập test — chỉ tính mean và std trên toàn bộ dữ liệu trước khi chia. Vì sao con số cuối cùng đã hỏng?
- Q3. Vì sao thứ hạng trên leaderboard thường không tái lập được, dù không ai gian lận và không ai làm lộ tập test?
- Q4. Quỹ đạo SLAM của bạn có ATE rất đẹp nhưng chạy thật thì lệch dần theo thời gian. Bước căn chỉnh đã hấp thụ mất cái gì?
- Q5. Mô hình mới hơn mô hình cũ 1 điểm phần trăm trên 1000 ảnh. Phép tính nào cho biết bạn có quyền viết điều đó vào báo cáo hay không?
- Q6. Precision 0,95 nhưng recall 0,40 — hệ thống của bạn đang hành xử thế nào, và ba nguyên nhân nào giải thích được nó?
- Q7. Hai hệ thống có cùng $F_1 = 0,600$. Vì sao một trong hai vẫn có thể tốt hơn hẳn cái kia cho bài toán đóng vòng?
- Q8. Vì sao một mô hình có AUC hoàn hảo vẫn có thể sai hoàn toàn về xác suất, và bạn phát hiện điều đó bằng biểu đồ nào?

7.1 Đặt vấn đề

Bạn cần đúng một thứ từ giao thức đánh giá: *một con số dự báo được hiệu năng tương lai*. Không phải một con số đẹp, cũng không phải một con số so được với paper. Cái bạn cần là: khi bạn nói “mô hình này đạt 0,87”, hệ thống thật sự chạy quanh mức đó trên dữ liệu nó chưa từng thấy.

Điều làm chương này khó chịu là mọi lỗi ở đây đều làm kết quả đẹp lên. **Rò rỉ dữ liệu** (*data leakage*) không báo lỗi, không làm loss tăng, không làm gì cả ngoài việc tặng bạn thêm vài điểm phần trăm và một cảm giác tiến bộ. Không có bug nào có cơ chế tự vệ tốt hơn một bug làm bạn vui.

Người đã tự xây benchmark harness cho SLAM có sẵn phần lớn phản xạ cần thiết, chỉ là chưa gọi tên chúng: bạn biết một lần chạy không nói lên điều gì vì nhiều giữa các lần chạy có thể lớn hơn hiệu ứng cần đo; bạn biết chọn sequence nào để báo cáo là quyết định có sức nặng ngang việc sửa thuật toán; bạn biết một con số ATE trung bình có thể che một lần mất tracking. Chương này là ba trực giác đó viết lại bằng ngôn ngữ học máy, cộng thêm các dạng rò rỉ mà pipeline SLAM không có.

TRỰC GIÁC

Coi tập test như một *mẫu chuẩn niêm phong* trong phòng đo lường, không như một thước đo dùng bao nhiêu lần cũng được. Mẫu chuẩn giữ được giá trị chỉ khi nó ở trong tủ; mỗi lần bạn mở ra để chỉnh thiết bị, một chút đặc tính của thiết bị được in ngược vào cái mà bạn coi là chuẩn. Sau ba mươi lần “chỉnh theo mẫu chuẩn”, thiết bị khớp mẫu chuẩn hoàn hảo và sai với mọi thứ khác. Val là mẫu làm việc, dùng thoải mái và chấp nhận nó hao mòn; test là mẫu niêm phong, mở một lần ở cuối.

7.2 Ba vai trò và các đường rò rỉ (Splits and Leakage)

Việc tách train/val/test hay bị dạy như một thủ tục: cắt 70/15/15. Dạy như vậy thì người học nhớ tỉ lệ và quên mất phần quan trọng hơn — ba tập có *ba vai trò khác nhau về bản chất*.

- **Train** — ước lượng tham số của mô hình.
- **Val** — chọn siêu tham số và quyết định lúc nào dừng.
- **Test** — ước lượng hiệu năng, đúng một lần, ở cuối.

Nhằm vai trò thì mọi kết luận sau đó vô giá trị. Và nhằm vai trò không tạo ra thông báo lỗi nào.

A. Sáu đường rò rỉ, theo thứ tự khó phát hiện dần

Mạch 1 — rò rỉ và cách chặn từng đường

- Vấn đề** đo trên dữ liệu đã học là vô nghĩa.

Giải pháp tách ba tập với ba vai trò rõ ràng.

Giả định bạn thật sự tôn trọng vai trò đó, tức không bao giờ dùng test để quyết định bất cứ điều gì.

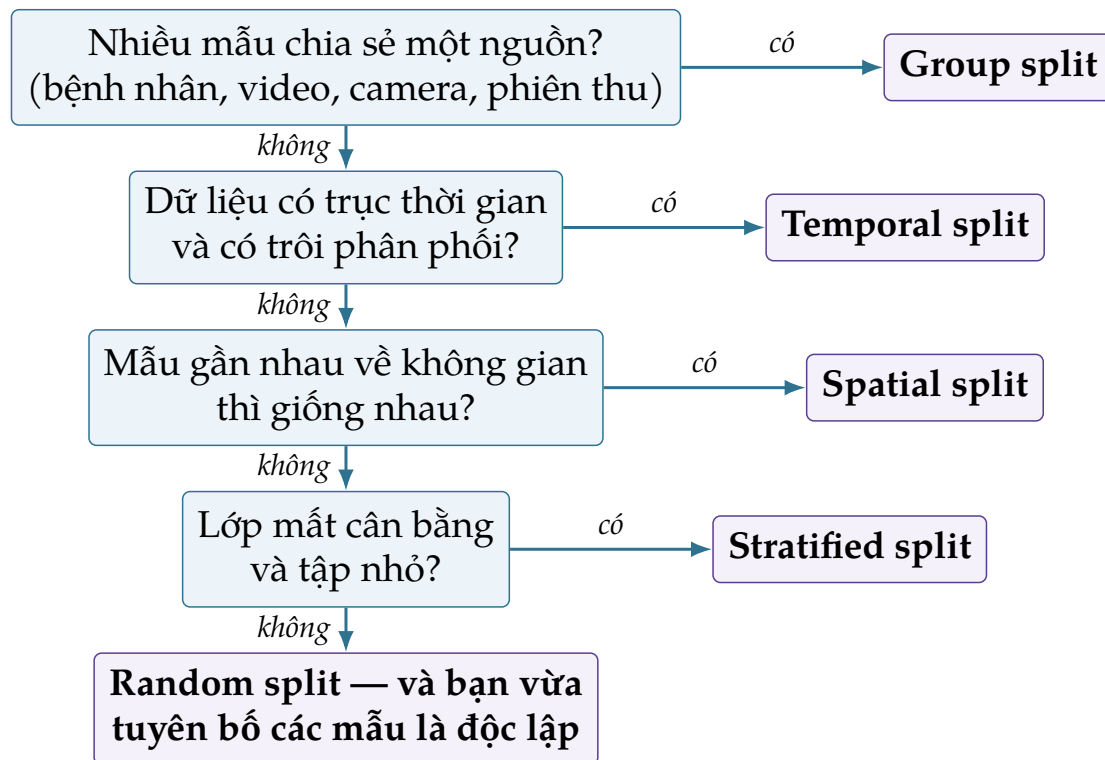
Hệ quả bước duy nhất ai cũng làm, và cũng là bước duy nhất không đủ.
- Vấn đề** rò rỉ qua khâu tiền xử lý.

Giải pháp tính mean/std, PCA, chọn đặc trưng, khớp bộ chuẩn hoá *sau* khi chia và *chỉ* trên train.

- Hệ quả thống kê của test đã đi vào mô hình — nhẹ, nhưng đủ để mọi chênh lệch nhỏ trở nên vô nghĩa.
- 3 Vấn đề** rò rỉ qua mẫu trùng.
- Giải pháp** khử **trùng lặp gần đúng** (*near-duplicate*) trước khi chia, bằng *embedding* chứ không bằng mã băm (*B/06*).
- Hệ quả với ảnh crawl web và frame video, đây là nguồn tạo kết quả đẹp mà giả phổ biến nhất.
- 4 Vấn đề** rò rỉ qua nhóm.
- Giải pháp** nếu nhiều mẫu chia sẻ một nguồn — cùng bệnh nhân, cùng video, cùng camera, cùng phiên thu — thì mọi mẫu của một nhóm phải nằm trọn một bên.
- Hệ quả bỏ qua điều này là cách rẻ nhất để tự tặng mình mười tới hai mươi điểm hiệu năng ảo; đây là kinh nghiệm được lặp lại rộng rãi trong y tế và video, không phải một con số đo chính xác.
- 5 Vấn đề** rò rỉ qua thời gian.
- Giải pháp** khi dữ liệu có trục thời gian, test phải nằm *sau* train.
- Hệ quả chia ngẫu nhiên trên dữ liệu có trôi cho phép mô hình nội suy giữa quá khứ và tương lai, một đặc quyền nó sẽ không có khi chạy thật.
- 6 Vấn đề** rò rỉ qua chính bạn.
- Vấn đề mới** ***adaptive overfitting***: mỗi lần bạn nhìn test rồi ra một quyết định, một chút thông tin từ test rò vào mô hình qua bộ não bạn.
- Hệ quả sau vài chục vòng như vậy, test đã trở thành val — lời giải thích tự nhiên cho việc thứ hạng leaderboard thường không tái lập được trên một tập test mới thu thập cùng quy trình [18].

B. Chọn cách chia là phát biểu một giả định, không phải chọn một hàm

Nhận thức then chốt của cả chương nằm ở đây. Khi bạn gọi `train_test_split` với `shuffle=True`, bạn không chọn một tiện ích; bạn đang tuyên bố rằng dữ liệu tương lai sẽ đến từ cùng nhóm, cùng thời điểm, cùng địa điểm với dữ liệu hiện tại. Đó là một phát biểu mạnh về thế giới, và nó thường sai. Hình 7.1 biến phát biểu đó thành một chuỗi câu hỏi trả lời được, còn Bảng 7.1 nói cái giá của mỗi lựa chọn.



Hình 7.1. Cây quyết định chọn cách chia. Thứ tự các câu hỏi là thứ tự mức thiệt hại: rò rỉ qua nhóm gây sai lệch lớn nhất nên phải loại trừ trước, còn *stratified* chỉ giảm phương sai chứ không chặn rò rỉ nào — nó đứng cuối vì nó là lựa chọn *cộng thêm*, không phải lựa chọn thay thế. Nhánh cuối cùng không phải mặc định an toàn mà là một tuyên bố cần được biện minh.

Bảng 7.1. Mỗi cách chia là một giả định về cách mô hình sẽ được dùng. Cột cuối nói điều gì xảy ra nếu giả định đó sai.

Cách chia	Giả định tương ứng	Dùng khi	Rủi ro nếu chọn sai
Random	Các mẫu độc lập với nhau	Ảnh rời rạc, không có cấu trúc nhóm	Rò rỉ qua nhóm và qua thời gian, không có dấu hiệu cảnh báo
Stratified	Cần giữ tỉ lệ lớp giữa các tập	Lớp mất cân bằng, tập nhỏ	Giảm phương sai ước lượng nhưng <i>không</i> chặn rò rỉ nhóm
Group	Sẽ gặp nhóm hoàn toàn mới	Y tế, video, đa camera, đa phiên thu	Nếu bỏ qua: hiệu năng ảo cao hàng chục điểm
Temporal	Sẽ dự đoán về tương lai	Có trôi phân phối theo thời gian	Đánh giá lạc quan giả tạo; mô hình được phép nội suy thời gian
Spatial	Sẽ gặp vùng địa lý mới	Ảnh vệ tinh, khảo sát hiện trường	Tự tương quan không gian làm hai bên split gần như trùng nhau

Một hệ quả ít được nói: cách chia phải khớp với đơn vị phân tích đã chọn ở B/05. Chọn frame làm đơn vị rồi chia ngẫu nhiên là công thức tạo rò rỉ. Hai frame cách nhau 33 ms không phải hai quan sát độc lập, đúng như hai keyframe liên kế không cho hai ràng buộc độc lập trong **hiệu chỉnh chùm tia** (*bundle adjustment*). Nếu đơn vị thật sự là “một video” hay “một phiên thu” thì test phải cắt ở cấp đó. Hệ quả là số mẫu độc lập thật nhỏ hơn nhiều so với con số bạn tưởng: một nghìn khung hình lấy từ mười video chỉ đáng giá mười mẫu. Điều đó nói rộng khoảng tin cậy ở cuối chương theo đúng căn bậc hai của tỉ lệ ấy.

↪ **hệ quả của B/05** — đơn vị phân tích chọn ở khâu thiết kế quyết định cách chia nào là hợp lệ; chọn frame rồi chia ngẫu nhiên là rõ ràng ngay từ định nghĩa.

↪ **tiền đề B/06** — khử trùng lặp gần đúng phải xong *trước* khi chia, nếu không thì mọi cách chia đều rõ ràng qua bản sao.

C. Khi dữ liệu quá ít để cắt một tập test tử tế

Kiểm định chéo (*cross-validation*) là câu trả lời cổ điển, và **nested cross-validation** là cách tune siêu tham số mà không làm ô nhiễm ước lượng cuối: vòng trong chọn siêu tham số, vòng ngoài chỉ để đo. Cái giá là chi phí tính toán nhân lên theo số fold. Với deep learning hiện đại, nhân năm lần một lượt huấn luyện kéo dài vài ngày là không khả thi. Đó là lý do *thực dụng*, không phải lý do lý thuyết, khiến kiểm định chéo hiếm gặp trong thị giác máy tính ngày nay. Biết điều này để không nhầm sự vắng mặt của nó với một chuẩn mực đúng.

I 7.3 Chọn metric khớp chi phí thật (Metric Selection)

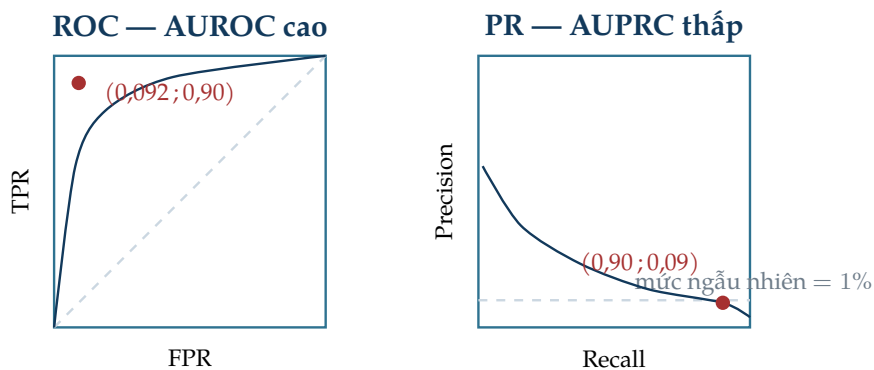
Giả sử đã chia đúng. Vẫn còn một cách nói dối tinh vi hơn: chọn sai thứ để đo. Metric là một hàm nén toàn bộ hành vi hệ thống xuống một số thực, và mọi phép nén đều mất thông tin — câu hỏi duy nhất đáng hỏi là *nó vứt đi loại lỗi nào*.

A. Ví dụ nhỏ nhất: accuracy 99% mà vô dụng

Tập test có 1000 ảnh, trong đó 10 ảnh thuộc lớp dương (1%).

- **Mô hình rỗng** — luôn trả lời “âm”. Accuracy $990/1000 = 99\%$, cao hơn phần lớn hệ thống thật, trong khi hoàn toàn vô dụng.
- **Mô hình có ích** — bắt được 9 trong 10 ca dương (*recall* 0,90) với tỉ lệ báo động giả 9,2% trên lớp âm, tức $0,092 \times 990 \approx 91$ báo động giả.
 - **Trên ROC** — điểm làm việc là $(FPR, TPR) = (0,092, 0,90)$, sát góc trên trái, trông xuất sắc.
 - **Người dùng thấy** — mô hình báo động $9 + 91 = 100$ lần, đúng 9 lần: **precision** = 9%.

Cơ chế rất rõ khi viết ra: FPR chia cho 990 mẫu âm nên một tỉ lệ nhỏ vẫn là rất nhiều mẫu, còn precision chia cho tổng số lần báo động nên nó cảm nhận đúng cái người dùng cảm nhận. Đó là toàn bộ lý do **AUPRC** phải thay **AUROC** khi lớp dương hiếm, và Hình 7.2 cho thấy cùng một điểm làm việc trông khác nhau đến mức nào trên hai không gian.



Hình 7.2. Cùng một mô hình, cùng một điểm làm việc, hai không gian đánh giá (sơ đồ minh họa, không phải số đo thật). Bên trái điểm đó nằm sát góc lý tưởng; bên phải chính nó nằm sát sàn. Khác biệt sinh ra vì trục tung bên trái chuẩn hoá theo 990 mẫu âm, còn bên phải chuẩn hoá theo 100 lần báo động — và người dùng sống ở trục bên phải.

B. Mỗi metric giấu loại lỗi nào

Bảng 7.2. Vài metric quen thuộc và loại lỗi mà mỗi cái làm cho vô hình. Cột giữa là lý do phải đọc ít nhất hai metric cùng lúc.

Metric	Giấu loại lỗi nào	Nên đọc kèm
Accuracy	Toàn bộ hành vi trên lớp hiếm	Precision/recall theo từng lớp
AUROC	Số lượng tuyệt đối báo động giả khi lớp dương hiếm	AUPRC ở đúng mức tỉ lệ dương thật
mAP@0.5	Sai lệch định vị: hộp lệch nhẹ vẫn được tính là đúng	mAP@0.5:0.95, hoặc phân rã lỗi kiểu TIDE [66]
mIoU	Lớp nhỏ và biên; một lớp chiếm phần lớn pixel có thể kéo con số lên	IoU từng lớp, và Panoptic Quality nếu cần tách thực thể [67]
MOTA	Việc liên tục đổi ID: MOTA cao vẫn tương thích với danh tính bị hoán loạn	IDF1 và HOTA [68]
PSNR, SSIM	Chất lượng cảm nhận; ảnh mờ trung bình hoá thường thắng	LPIPS [69], và với mô hình sinh thì FID [70]
ATE sau alignment	Trôi toàn cục, vì phép căn chỉnh hấp thụ đúng thành phần đó	RPE trên nhiều độ dài đoạn [71]

C. Metric hình học và cái bẫy của phép căn chỉnh

Dòng cuối Bảng 7.2 đáng nói kỹ vì người đọc sống ở đó. **sai số quỹ đạo tuyệt đối** (*absolute trajectory error*) được tính *sau* khi căn chỉnh quỹ đạo ước lượng vào quỹ đạo tham chiếu bằng một phép biến đổi cứng, hoặc một phép biến đổi có tỉ lệ trong trường hợp đơn mắt. Phép căn chỉnh này chính là thứ hấp thụ sai lệch toàn cục: một quỹ đạo trôi tỉ lệ đều đặn sẽ bị kéo về gần tham chiếu và cho ra ATE dễ chịu, trong khi lỗi cục bộ vẫn còn nguyên. **sai số tương đối** (*relative pose error*) thì ngược lại — nó đo tính nhất quán trên các đoạn ngắn và mù với sai lệch toàn cục.

Không cái nào sai; sai là đọc một cái rồi kết luận. Nguyên tắc tổng quát: *mỗi metric là một phép chiếu, và bạn cần ít nhất hai phép chiếu không song song để định vị được lỗi.*

↔ **cùng nguyên lý với C/16** — trong thị giác 3D, cặp ATE-RPE và cặp Chamfer-độ phủ đều là hai phép chiếu bù nhau của cùng một sai số.

Một khía cạnh nữa thường bị đẩy ra ngoài giao thức rồi phải trả giá sau là **calibration**: mô hình có thể xếp hạng đúng mà vẫn tự tin sai lệch một cách hệ thống, và mạng hiện đại nổi tiếng là quá tự tin [24]. Nếu đầu ra đi vào một quyết định có ngưỡng, hoặc vào một bộ hợp nhất cảm biến coi nó như xác suất, thì calibration là một phần của tính đúng đắn chứ không phải chuyện phụ — khai triển đầy đủ ở E/25-do-tin-cay-va-van-hanh.

↔ **tiền đề E/25** — một đầu ra xếp hạng đúng nhưng lệch hiệu chuẩn sẽ phá mọi ngưỡng quyết định đặt phía sau nó.

I 7.4 Bốn ô của ma trận nhầm lẫn (Reading the Confusion Matrix)

Từ đây tới hết chương, trọng tâm đổi. Phần trên dạy *chọn* metric; phần này dạy *đọc* metric. Đây là hai kỹ năng khác nhau. Chọn đúng metric mà đọc sai con số thì vẫn ra quyết định sai.

Mọi thứ bắt đầu từ một bảng bốn ô. Một bộ phân loại nhị phân nhận một mẫu và trả lời “có” hoặc “không”. Chân trị cũng chỉ có hai giá trị. Ghép hai bên lại được đúng bốn kết cục, không thừa không thiếu. Mọi metric phân loại mà bạn từng gặp đều là một phân số lắp từ bốn số này.

Người đọc đã sống với bảng này nhiều năm mà chưa gọi tên nó. Trong đòng vòng của SLAM, bước kiểm chứng hình học nhận một cặp khung hình ứng viên và quyết định “đây là cùng một chỗ” hay “không phải”. Đó chính là một bộ phân loại nhị phân. Ngưỡng số điểm inlier chính là **điểm vận hành** (*operating point*) của nó. Cả chương này vì thế nói về đúng thứ bạn đang chỉnh bằng tay.

A. Bốn ô, gọi bằng lời thường

- **TP — dương thật** (*true positive*). Hệ thống báo “có” và đúng là có. Đây là công việc hệ thống được thuê để làm.
- **FP — dương giả** (*false positive*). Hệ thống báo “có” nhưng thật ra không có. Đây là báo động giả. Người dùng nhìn thấy nó ngay.
- **FN — âm giả** (*false negative*). Hệ thống báo “không” nhưng thật ra có. Đây là ca bị bỏ sót. Người dùng *không* nhìn thấy nó, và đó mới là chỗ nguy hiểm.
- **TN — âm thật** (*true negative*). Hệ thống báo “không” và đúng là không. Ô này thường đông đảo tới mức vô nghĩa, và chính nó là nguồn của phần lớn ảo giác về hiệu năng.

Ghi nhớ theo một quy tắc: chữ thứ hai là *cái hệ thống nói*, chữ thứ nhất là *hệ thống có nói đúng không*. “False positive” nghĩa là hệ thống nói dương, và nó sai.

Hình 7.3 vẽ bốn ô đó cùng một bộ số cụ thể. Bộ số này đến từ một bộ phân loại tổng hợp mà chúng ta sẽ dùng lại suốt phần còn lại của chương: 20 000 mẫu, trong đó 200 mẫu dương, tức tỉ lệ nền 1%. Ở ngưỡng 0,50, nó cho TP 134, FP 1043, FN 66 và TN 18 757. Đây là số sinh từ mô phỏng, không phải số đo trên một hệ thống thật; giá trị của nó nằm ở chỗ mọi con số phía sau đều tính ra được từ đúng bốn số này.

		CHÂN TRỊ		
		thật sự có (200 ca)	thật sự không (19 800 ca)	
HỆ THỐNG BÁO	báo “có” (1177 lần)	TP = 134 báo có, đúng là có <i>việc hệ thống được thuê để làm</i>	FP = 1043 báo có, thật ra không <i>báo động giả — ai cũng thấy</i>	mẫu số của precision mọi lần hệ thống mở miệng
	báo “không” (18 823 lần)	FN = 66 báo không, thật ra có <i>bỏ sót — không ai thấy</i>	TN = 18 757 báo không, đúng là không <i>ô khổng lồ và gần như vô nghĩa</i>	
		mẫu số của recall mọi ca thật sự có	mẫu số của FPR mọi ca thật sự không	

Hình 7.3. Bốn ô của ma trận nhầm lẫn, với số từ bộ phân loại tổng hợp ở ngưỡng 0,50 (20 000 mẫu, 1% dương; số mô phỏng, không phải số đo). Ba dấu ngoặc là ba mẫu số khác nhau: precision chia theo hàng trên, recall chia theo cột trái, FPR chia theo cột phải. Cùng một ô FP nằm trong hai phân số có mẫu số lệch nhau gần hai mươi lần, và đó là toàn bộ lý do hai metric kể hai câu chuyện khác nhau.

Hai cái bẫy nhỏ nhưng tốn thời gian

Trước khi đi tiếp, hai chỗ vấp rất hay xảy ra khi đọc bảng của người khác.

Thứ nhất là *chiều của bảng*. Một số thư viện đặt hàng là lớp thật và cột là lớp dự đoán; một số tài liệu làm ngược lại. Nếu đọc nhầm chiều thì precision và recall hoán vị cho nhau, và bạn sẽ kết

luận ngược hoàn toàn. Cách kiểm tra trong ba giây: cộng theo hàng, xem tổng có bằng số mẫu thật của từng lớp không. Nếu có, hàng là chân trị.

Thứ hai là *ai được tính vào mẫu số*. Ở phát hiện vật thể, một dự đoán không khớp hộp nào là FP, nhưng một hộp thật không được khớp là FN — và không có ô TN nào cả, vì số “vùng ảnh không chứa vật” là vô hạn. Đây là lý do accuracy không tồn tại trong phát hiện vật thể, và cũng là lý do mọi metric ở đó đều xây trên precision và recall chứ không xây trên bốn ô đầy đủ.

B. Bốn ô là bốn chi phí, và chính chênh lệch chi phí mới chọn metric

Bốn ô không chỉ là bốn cách đếm. Chúng là bốn loại thiệt hại có giá khác nhau trong đời thực, và cái giá đó do bài toán quyết định chứ không do thuật toán.

- **Đóng vòng trong SLAM.** Một vòng sai (FP) khâu hai chỗ khác nhau lại thành một, làm cong toàn bộ bản đồ, và thường không hồi phục được. Một vòng bị bỏ lỡ (FN) chỉ có nghĩa là sai số tích lũy chưa được xoá lần này; lát nữa qua chỗ đó lần nữa vẫn còn cơ hội. Chi phí lệch hẳn về một phía, có thể tới vài bậc độ lớn — đây là đánh giá định tính từ cơ chế, không phải một con số đo được.
- **Sàng lọc y tế.** Ngược lại hoàn toàn. Một ca bỏ sót (FN) là một người bệnh về nhà. Một báo động giả (FP) chỉ tốn thêm một lần xét nghiệm.
- **Phát hiện vật cản cho robot.** FN là va chạm. FP là robot phanh vô cớ. Cả hai đều tổn, nhưng tổn theo hai đơn vị khác nhau, và người thiết kế phải quy đổi.

Từ đây rút ra nguyên tắc chi phối cả phần còn lại của chương. *Metric nào đáng nhìn phụ thuộc vào ô nào đấy*. Nếu FP đắt thì precision là con số quan trọng. Nếu FN đắt thì recall là con số quan trọng. Nếu cả hai cùng đắt và đắt ngang nhau, bạn cần một con số gộp — và sẽ thấy ở mục sau rằng gộp cũng có giá của nó.

Còn một điều nữa cần nói thẳng. Ô TN gần như không bao giờ đắt. Nó chỉ đông. Trong bảng trên, TN chiếm 93,8% tổng số mẫu. Bất kỳ metric nào có TN trong tử số hoặc trong mẫu số đều bị ô này chi phối, và accuracy là ví dụ tiêu biểu. Đó là lý do accuracy nói dối một cách hệ thống trên dữ liệu lệch lớp.

C. Bắc cầu: kiểm chứng đóng vòng là một bộ phân loại nhị phân

Đặt bảng bốn ô lên đúng hệ thống mà người đọc đang bảo trì. Tầng kiểm chứng hình học của đóng vòng nhận một cặp khung hình ứng viên, chạy RANSAC trên các cặp điểm khớp, đếm số inlier, và so với một ngưỡng. Ánh xạ như sau.

- **Mẫu** là một cặp khung hình ứng viên, không phải một khung hình. Đơn vị phân tích ở đây là cặp, và điều đó ảnh hưởng tới cả cách chia dữ liệu lẫn cách bootstrap về sau.
- **Lớp dương** là “hai khung này thật sự nhìn cùng một chỗ”. Định nghĩa của nó do bạn đặt — thường bằng một ngưỡng khoảng cách trên chân trị quỹ đạo — nên nó là một quyết định thiết kế, không phải một sự thật khách quan.
- **Điểm số** là số inlier sau RANSAC, và **ngưỡng inlier** chính là điểm vận hành.
- **TP** là một vòng đúng được chấp nhận; **FP** là một vòng sai được chấp nhận; **FN** là một vòng đúng bị từ chối; **TN** là một cặp không liên quan bị từ chối, và ô này chiếm gần như toàn bộ dữ liệu.

Đặt xong ánh xạ này thì mọi thứ trong chương trở nên áp dụng được trực tiếp. Nâng ngưỡng inlier từ 20 lên 40 là dịch điểm vận hành sang phải trên đường cong. Việc đó tăng precision, giảm recall, và không đổi một chút nào bản thân bộ mô tả hay bộ ghép cặp. Nếu bạn muốn cải thiện

thật sự thì phải làm cả đường cong dịch lên, chứ không phải trượt dọc theo nó — và đó là hai loại công việc hoàn toàn khác nhau.

7.5 Các tỉ lệ dẫn xuất và mẫu số của từng cái (Derived Rates and Denominators)

Gần như mọi hiểu nhầm về metric phân loại là hiểu nhầm mẫu số. Người ta nhớ tử số vì nó là “số ca đúng”, rồi quên hỏi con số đó được chia cho cái gì. Mục này đi qua từng tỉ lệ theo đúng bốn bước: định nghĩa bằng lời, câu hỏi nó trả lời, cao và thấp nghĩa là gì, và điều nó *không* nói.

A. Precision — đọc theo hàng “hệ thống đã báo”

Precision là tỉ lệ đúng trong những lần hệ thống báo “có”. Mẫu số là $TP + FP$, tức toàn bộ số lần hệ thống mở miệng.

$$\text{precision} = \frac{TP}{TP + FP}.$$

Câu hỏi nó trả lời, ở ngôi thứ hai: *trong những cái tôi báo là dương, bao nhiêu phần là đúng?* Nói cách khác, nếu bây giờ hệ thống báo động, bạn có nên tin không.

Precision cao nghĩa là hệ thống ít khi làm phiền bạn vô cớ: mỗi lần nó nói, gần như chắc chắn có chuyện thật. Precision thấp nghĩa là bạn phải tự lọc lại phần lớn những gì nó đưa ra, và niềm tin vào hệ thống bốc hơi rất nhanh.

Điều precision *không* nói: nó không nói gì về số ca bị bỏ sót. Một hệ thống chỉ báo đúng một ca duy nhất trong cả năm, và báo đúng, có precision bằng 1,0. Mẫu số của precision không hề chứa FN, nên FN có thể lớn tùy ý mà precision không nhúc nhích.

B. Recall — đọc theo cột “thật sự có”

Recall, còn gọi là độ nhạy (*sensitivity*) hoặc TPR, là tỉ lệ bắt được trong số các ca thật sự dương. Mẫu số là $TP + FN$, tức toàn bộ số ca dương có trong dữ liệu.

$$\text{recall} = \frac{TP}{TP + FN}.$$

Câu hỏi nó trả lời: *trong những cái thật sự dương, tôi bắt được bao nhiêu phần?*

Recall cao nghĩa là ít thứ lọt lưới. Recall thấp nghĩa là hệ thống im lặng ở phần lớn các ca thật, và sự im lặng đó không để lại dấu vết nào trong log.

Điều recall *không* nói: nó không nói gì về số lần báo bừa. Một hệ thống luôn luôn báo “có” cho mọi mẫu có recall bằng 1,0. Mẫu số của recall không chứa FP.

Hai đoạn trên gộp lại thành một câu đáng thuộc: *precision và recall bỏ qua hai ô khác nhau, nên một mình cái nào cũng vô nghĩa*. Precision mù với FN. Recall mù với FP. Chỉ khi đọc cùng nhau chúng mới khoá được cả bốn ô.

C. Các tỉ lệ còn lại, xếp theo mẫu số

Bảng 7.3 liệt kê các tỉ lệ còn lại. Cột quan trọng nhất là cột mẫu số; cột cuối là cái bẫy đặc trưng của từng cái.

Bảng 7.3. Các tỉ lệ dẫn xuất từ bốn ô. Số ở cột giá trị lấy từ bộ phân loại tổng hợp ở ngưỡng 0,50 (TP 134, FP 1043, FN 66, TN 18 757). Đọc bảng theo cột mẫu số: hai tỉ lệ khác mẫu số thì không so sánh trực tiếp được với nhau.

Tỉ lệ	Mẫu số là gì	Giá trị	Nó không nói điều gì
Precision	mọi lần báo “có”	0,114	Số ca bỏ sót. Precision 1,0 vẫn tương thích với việc bỏ sót 99% ca thật.
Recall = TPR = độ nhạy	mọi ca thật sự dương	0,670	Số lần báo bừa. Luôn báo “có” cho recall 1,0.
Độ đặc hiệu (<i>specificity</i>) = TNR	mọi ca thật sự âm	0,947	Nó gần như luôn đẹp khi lớp âm đông; 0,947 ở đây vẫn ứng với 1043 báo động giả.
FPR = 1 – specificity	mọi ca thật sự âm	0,053	Số lượng báo động giả. 5,3% của 19 800 là 1043 lần.
FNR = 1 – recall	mọi ca thật sự dương	0,330	Cùng thông tin với recall, chỉ đổi hướng; không phải một metric mới.
NPV (giá trị dự đoán âm)	mọi lần báo “không”	0,9965	Nó gần như luôn sát 1 khi lớp dương hiếm, nên hầu như không mang thông tin.
Accuracy	toàn bộ mẫu	0,945	Bị ô TN chi phối. Mô hình rỗng luôn báo “không” đạt 0,990, cao hơn.
Balanced accuracy	trung bình của recall và specificity	0,808	Nó chứa được lệch lớp nhưng vẫn không phản ánh chi phí thật khi FP và FN lệch giá.
Prevalence (tỉ lệ nền)	toàn bộ mẫu	0,010	Không phải metric của mô hình. Nó là tính chất của <i>dữ liệu</i> , và nó quyết định cách đọc mọi dòng trên.

Ba nhận xét đáng rút ra từ bảng.

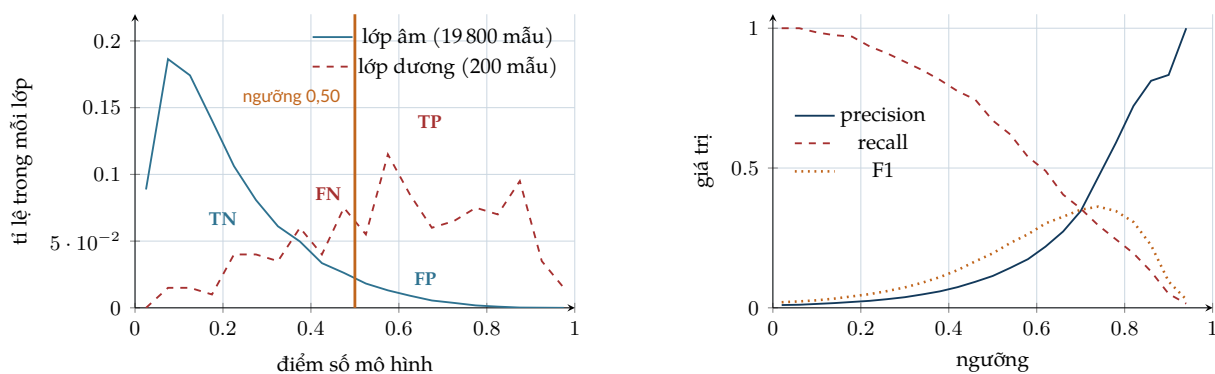
- **Specificity và precision hay bị lẫn.** Cả hai đều “liên quan tới báo động giả”, nhưng specificity chia FP cho số ca âm (19 800), còn precision chia FP cho số lần báo (1177). Mẫu số lệch gần hai mươi lần, nên specificity 0,947 nghe rất đẹp trong khi precision 0,114 thì thảm hại. Cùng một FP, hai câu chuyện.
- **NPV gần như vô dụng khi lớp dương hiếm.** Nếu chỉ 1% mẫu là dương thì đoán “không” đã đúng 99% số lần. NPV cao không chứng minh điều gì.
- **Prevalence không phải metric nhưng phải báo cáo cùng metric.** Precision phụ thuộc trực tiếp vào nó. Cùng một mô hình, đem sang tập có tỉ lệ nền thấp hơn mười lần thì precision tụt gần mười lần, dù mô hình không đổi một chút nào. Recall, specificity và FPR thì không đổi, vì mẫu số của chúng nằm trong một lớp duy nhất.

Nhận xét thứ ba là thứ hay cắn người nhất khi triển khai. Bạn đo precision trên tập test cân bằng nhân tạo, thấy 0,85, rồi đem ra hiện trường nơi tỉ lệ nền thấp hơn hai chục lần và precision rơi xuống 0,2. Không có gì hỏng cả; bạn chỉ đã đo một mẫu số khác.

D. Ngưỡng dịch chuyển bốn ô như thế nào

Bốn ô không phải một tính chất cố định của mô hình. Chúng là một tính chất của mô hình *cộng với* một ngưỡng. Đổi ngưỡng thì bốn số chảy từ ô này sang ô kia theo một quy luật cứng: siết ngưỡng chặt lại thì TP chảy sang FN và FP chảy sang TN; nới lỏng thì ngược lại.

Hình 7.4 cho thấy điều đó trên chính bộ phân loại tổng hợp. Bên trái là phân bố điểm số của hai lớp, bên phải là precision, recall và F1 khi ngưỡng chạy từ lỏng sang chặt. Hai đường precision và recall đi ngược chiều nhau trên toàn dải, và không có ngưỡng nào làm cả hai cùng đẹp — đó là hình dạng của một sự đánh đổi, không phải một khuyết điểm cần sửa.

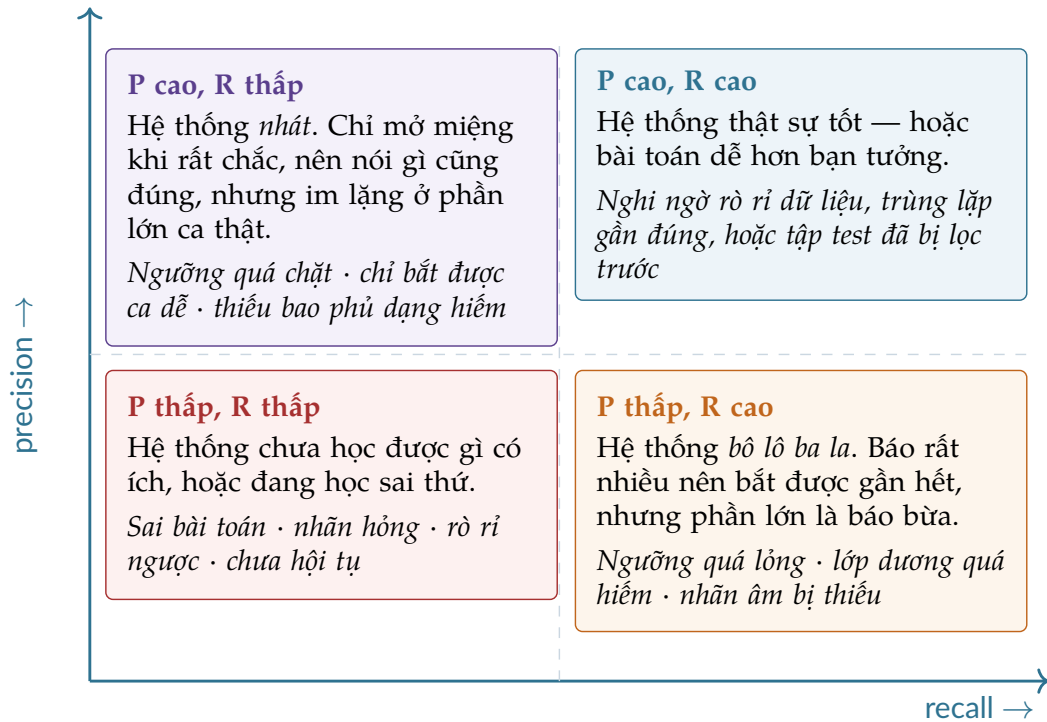


Hình 7.4. Ngưỡng dịch chuyển bốn ô (bộ phân loại tổng hợp, 20 000 mẫu, 1% dương). Bên trái: mỗi đường được chuẩn hoá trong lớp của nó, nên đừng đọc chiều cao như số lượng — đường đỏ đại diện 200 mẫu, đường xanh 19 800 mẫu. TN và FP nằm trên đường xanh (lớp âm); FN và TP nằm trên đường đỏ (lớp dương). Kéo ngưỡng sang phải thì FP giảm và FN tăng. Bên phải: precision và recall đi ngược chiều trên toàn dải, còn F1 đạt cực đại quanh ngưỡng 0,74 — một điểm không có lý do gì để trùng với điểm tối ưu theo chi phí thật của bạn.

Một hệ quả thực dụng: ngưỡng 0,5 không có gì thiêng liêng. Nó chỉ là giá trị mặc định của hàm `argmax` khi có hai lớp. Chọn ngưỡng là một quyết định thiết kế riêng, tách hẳn khỏi việc huấn luyện, và nó phải được ra dựa trên chi phí của FP và FN chứ không dựa trên thói quen.

7.6 Bốn góc precision–recall (The Four Corners)

Đây là phần trả lời trực tiếp câu hỏi “precision cao mà recall thấp thì nghĩa là gì”. Hai con số, mỗi con số cao hoặc thấp, cho bốn tổ hợp. Mỗi tổ hợp mô tả một *hành vi* của hệ thống, và mỗi hành vi lại có vài nguyên nhân khác nhau. Việc của bạn là đi từ tổ hợp về nguyên nhân, và điều đó cần một phép thử chứ không cần trực giác.



Hình 7.5. Bốn góc của mặt phẳng precision–recall và hành vi ứng với mỗi góc. Dòng in nghiêng trong mỗi ô là các nguyên nhân thường gặp — chúng khác nhau về cách sửa, nên phải phân biệt bằng phép thử chứ không đoán. Lưu ý góc trên phải cũng là một cảnh báo: kết quả quá đẹp trên bài toán khó là dấu hiệu của rò rỉ trước khi là dấu hiệu của tài năng.

A. Từ tổ hợp về nguyên nhân, kèm phép thử

Bảng 7.4 là phần dùng được nhất của mục này. Cột cuối là cột đáng giá: nó biến một suy đoán thành một việc làm được trong nửa giờ.

Bảng 7.4. Bốn tổ hợp precision–recall: hệ thống đang hành xử thế nào, vì sao, và làm gì để biết là vì sao. Mỗi ô ở cột giữa liệt kê các nguyên nhân *cạnh tranh nhau*; phép thử ở cột phải là cách tách chúng.

Tổ hợp	Nguyên nhân thường gặp	Phép thử để phân biệt
P cao, R cao	(a) mô hình thật sự tốt; (b) rò rỉ dữ liệu; (c) tập test đã bị lọc bỏ ca khó; (d) bài toán dễ hơn tưởng	So với một baseline ngu ngốc (đoán theo tỉ lệ nền, hoặc một quy tắc thủ công ba dòng). Nếu baseline cũng đẹp thì là (d). Chia lại theo nhóm và đo lại; nếu con số sập thì là (b). Đếm phân bố độ khó của tập test so với dữ liệu hiện trường để bắt (c).
P cao, R thấp	(a) ngưỡng đặt quá chặt; (b) mô hình chỉ học được dạng dễ, mù với dạng hiếm; (c) tập huấn luyện thiếu bao phủ; (d) định nghĩa nhãn dương hẹp hơn thực tế	Quét toàn dải ngưỡng và vẽ đường PR. Nếu nới ngưỡng làm recall tăng mạnh mà precision giảm ít thì là (a) — sửa bằng một dòng cấu hình. Nếu đường PR phẳng lì và tụt ngay thì không phải ngưỡng, mà là (b) hoặc (c). Phân biệt (b) với (c) bằng cách đọc recall theo từng lát.
P thấp, R cao	(a) ngưỡng quá lỏng; (b) lớp dương quá hiếm nên precision bị mẫu số chìm; (c) nhãn âm bị thiếu, cái mô hình bắt đúng lại bị tính là FP; (d) mô hình bám vào một đặc trưng tất	Siết ngưỡng và xem precision có leo lên không — nếu có thì là (a). Nếu precision vẫn thấp ở mọi ngưỡng, tính lại precision kỳ vọng của một bộ đoán ngẫu nhiên (bằng đúng tỉ lệ nền) để xem có phải (b). Với (c), bốc tay 50 mẫu FP và tự gán nhãn lại; tỉ lệ “FP thật ra đúng” nói ngay có phải nhãn thiếu hay không.
P thấp, R thấp	(a) mô hình chưa hội tụ hoặc có bug; (b) nhãn hỏng hoặc mâu thuẫn; (c) đặc trưng đầu vào không mang thông tin về nhãn; (d) rò rỉ ngược — tập huấn luyện chứa thứ mâu thuẫn với tập test	Kiểm tra mô hình có <i>học thuộc</i> nổi một batch nhỏ không; nếu không thì là (a). Đo mức đồng thuận giữa người gán nhãn trên một mẫu con để bắt (b), xem B/06-chat-luong-du-lieu . Nếu mô hình học thuộc được train mà chết trên test, và chia lại vẫn thế, thì nghi (c) hoặc (d).

B. Ba tình huống hai con số này đánh lừa bạn

Bốn góc ở trên giả định precision và recall được tính đúng. Có ba cách rất phổ biến để chúng được tính *sai* mà không ai nhận ra.

- **Accuracy 99% trên dữ liệu lệch 99:1.** Đây là bẫy kinh điển và nó vẫn bắt được người, vì con số 99 nghe như một lời khen. Kiểm tra chỉ mất một phút: tính accuracy của mô hình rỗng luôn trả lời lớp đa số. Nếu con số của bạn không cao hơn nó một cách rõ ràng thì bạn chưa có gì cả. Trong ví dụ đang chạy, mô hình rỗng đạt 0,990 còn mô hình thật đạt 0,945 — *thấp hơn*, dù mô hình thật rõ ràng có ích.
- **Precision đẹp vì tập test đã bị lọc.** Rất hay xảy ra khi tập đánh giá được xây từ đầu ra của một hệ thống cũ: bạn chỉ gán nhãn những ứng viên mà hệ thống cũ đề xuất. Khi đó mọi ca mà hệ thống cũ bỏ sót đều biến mất khỏi mẫu số của recall, và recall bạn đo là recall *có điều kiện đã lọt qua tầng trước*. Nó luôn đẹp hơn recall đầu-cuối, đôi khi đẹp hơn rất nhiều. Dấu hiệu nhận biết: bạn không thể trả lời câu hỏi “các mẫu âm trong tập test đến từ đâu”.
- **Recall đẹp vì chân trị thiếu.** Nếu chân trị bỏ sót một số ca dương thật, thì mẫu số của recall nhỏ hơn thực tế và recall bị thổi lên. Đồng thời, những ca mô hình bắt đúng nhưng chân trị không ghi lại bị tính thành FP, nên precision bị chìm xuống một cách oan uổng. Đây là lý do một mô hình mạnh đôi khi trông tệ hơn mô hình yếu: nó tìm ra những thứ mà người gán nhãn đã bỏ qua. Phép thử duy nhất đáng tin là bốc tay vài chục FP và gán nhãn lại chúng.

Trường hợp thứ ba có một biểu hiện quen thuộc trong SLAM. Khi bạn đánh giá tầng lấy ứng viên đóng vòng bằng nhãn “cùng chỗ” sinh tự động từ chân trị quỹ đạo, ngưỡng khoảng cách bạn chọn *chính là* định nghĩa của lớp dương. Nới ngưỡng từ 1 m lên 3 m thì recall của cùng một hệ thống thay đổi, dù hệ thống không đổi. Con số không mô tả hệ thống; nó mô tả cặp hệ thống–định nghĩa.

7.7 Gộp hai số làm một, và cái giá của việc gộp (F1, F-beta, MCC)

Hai con số khó xếp hạng. Khi phải chọn giữa mười cấu hình, người ta muốn một con số. Mọi cách gộp đều làm mất thông tin, nên câu hỏi đúng vẫn là câu hỏi cũ: nó vứt đi cái gì.

A. Vì sao F1 dùng trung bình điều hoà chứ không phải trung bình cộng

F1 là **trung bình điều hoà** (*harmonic mean*) của precision và recall:

$$F_1 = \frac{2PR}{P+R} = \left(\frac{P^{-1} + R^{-1}}{2} \right)^{-1}.$$

Lý do chọn trung bình điều hoà rất cụ thể: nó bị con số nhỏ kéo xuống. Trung bình cộng thì không. Lấy một hệ thống có precision 0,95 và recall 0,40. Trung bình cộng cho 0,675, nghe khá ổn. F1 cho 0,563, và con số đó phản ánh đúng hơn cái mà người dùng gặp: một hệ thống bỏ sót 60% ca thật.

Nói chặt hơn, trung bình điều hoà luôn nhỏ hơn hoặc bằng trung bình cộng, và khoảng cách giữa hai cái càng lớn khi hai số càng lệch nhau. Bằng nhau chỉ khi $P = R$. Vì vậy F1 thưởng cho sự cân bằng và phạt sự lệch — đúng thứ ta muốn khi *không* có lý do ưu tiên một phía. F1 xuất phát từ thước đo F trong truy hồi thông tin [72].

B. Cái F1 che mất, và F-beta chữa được tới đâu

Ba đoạn số sau nói hết vấn đề của F1. Xét hai hệ thống:

- Hệ A: $P = 0,60, R = 0,60$.
- Hệ B: $P = 0,90, R = 0,45$.

Cả hai đều có $F_1 = 0,600$. Nhưng đây là hai hệ thống hoàn toàn khác nhau. Hệ B đúng chín trên mười lần nó lên tiếng, đổi lại bỏ sót 55% ca thật. Hệ A sai bốn trên mười lần lên tiếng, nhưng bắt được nhiều hơn hẳn. Nếu bạn đang làm đóng vòng, hệ B tốt hơn hẳn; nếu bạn đang sàng lọc y tế, hệ A tốt hơn hẳn. F1 nói chúng bằng nhau. Đó là ý nghĩa của câu “F1 che mất bất đối xứng chi phí”.

F-beta cho phép nói ra sự bất đối xứng đó:

$$F_\beta = \frac{(1 + \beta^2)PR}{\beta^2 P + R}.$$

Cách nhớ ý nghĩa của β : β là số lần recall quan trọng hơn precision. $\beta = 2$ nghĩa là bạn coi một ca bỏ sót đắt gấp đôi một báo động giả; $\beta = 0,5$ nghĩa là ngược lại. Áp vào hai hệ trên: F_2 cho A là 0,600 và cho B là 0,500, nên nó chọn A. $F_{0,5}$ cho A là 0,600 và cho B là 0,750, nên nó chọn B. Cùng một cặp hệ thống, hai kết luận ngược nhau, và cái quyết định là β chứ không phải dữ liệu.

Bài học không phải “hãy dùng F-beta”. Bài học là: chọn β chính là phát biểu tỉ lệ chi phí giữa FP và FN, nên hãy phát biểu nó ra thành lời trước, rồi mới chọn số. Nếu bạn không nói được “một vòng sai đắt bằng bao nhiêu vòng bỏ lỡ” thì bạn cũng chưa đủ dữ kiện để chọn β .

C. Macro, micro, weighted — ba cách gộp cho ra ba kết luận

Với nhiều lớp, còn một tầng gộp nữa: gộp qua các lớp. Ba cách phổ biến, và chúng có thể xếp hạng ngược nhau.

Ví dụ nhỏ nhất, tính tay được. Ba lớp, 1000 mẫu: A có 900, B có 90, C có 10. Mô hình dự đoán như sau. Trong 900 mẫu A, đúng 891, còn 9 bị gọi thành B. Trong 90 mẫu B, đúng 54, còn 36 bị gọi thành A. Trong 10 mẫu C, đúng 1, còn 9 bị gọi thành A.

Viết ra thành bảng thì tính tay được trong hai phút. Bảng 7.5 là ma trận nhầm lẫn của ví dụ đó, kèm precision và recall từng lớp.

Bảng 7.5. Ma trận nhầm lẫn ba lớp của ví dụ tính tay. Hàng là lớp thật, cột là lớp dự đoán. Đọc theo hàng ra recall, đọc theo cột ra precision — và chính hai hướng đọc đó là gốc của chênh lệch giữa macro và micro.

thật	dự đoán			tổng	recall	precision	F1
	A	B	C				
A	891	9	0	900	0,990	0,952	0,971
B	36	54	0	90	0,600	0,857	0,706
C	9	0	1	10	0,100	1,000	0,182
tổng cột	936	63	1	1000			

Tính F1 từng lớp: lớp A được 0,971, lớp B được 0,706, lớp C được 0,182. Ba cách gộp cho ba con số:

- **Micro** — gộp bốn ô của mọi lớp lại rồi mới tính. Ở bài toán một nhãn, micro-F1 bằng đúng accuracy: $946/1000 = 0,946$.
- **Weighted** — trung bình F1 các lớp, trọng số theo số mẫu mỗi lớp: 0,939.
- **Macro** — trung bình cộng F1 các lớp, mỗi lớp một phiếu: 0,619.

Chênh lệch giữa 0,946 và 0,619 không phải sai số. Nó là hai câu hỏi khác nhau. Micro hỏi “một mẫu ngẫu nhiên có được xử lý đúng không”, nên lớp đông quyết định. Macro hỏi “một lớp ngẫu nhiên có được phục vụ tử tế không”, nên lớp C với 10 mẫu có quyền ngang lớp A với 900 mẫu. Weighted nằm giữa nhưng gần micro tới mức thường không thêm thông tin.

Quy tắc chọn rất đơn giản: *nếu bạn quan tâm tới lớp hiếm thì phải báo macro*. Và luôn báo cả hai. Một cặp micro cao, macro thấp là chữ ký của “con số tổng đang che một lớp bị bỏ rơi”, đúng chủ đề của mục kế tiếp trong chương.

D. MCC và kappa — vì sao chúng ổn định hơn accuracy

hệ số tương quan Matthews (*Matthews correlation coefficient*) dùng cả bốn ô trong một công thức đối xứng [73]:

$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$

Nó là hệ số tương quan giữa hai biến nhị phân “dự đoán” và “chân trị”, nên nó nằm trong $[-1, 1]$: 1 là hoàn hảo, 0 là không hơn đoán mò, âm là tệ hơn đoán mò. Tính chất quan trọng nhất: MCC chỉ cao khi cả bốn ô đều tốt. Không ô nào bị bỏ ra ngoài công thức, khác hẳn precision, recall và F1 — cả ba đều không đụng tới TN [74].

Điều đó cho MCC một tính chất mà accuracy không có. Mô hình rỗng luôn trả lời “không” đạt accuracy 0,990 nhưng MCC bằng đúng 0, vì nó không tương quan gì với chân trị. Mô hình thật ở ngưỡng 0,50 đạt accuracy 0,945 — thấp hơn — nhưng MCC 0,261. Hai con số accuracy xếp hạng ngược, MCC xếp hạng đúng.

hệ số kappa của Cohen (*Cohen's kappa*) [64] đi theo cùng ý tưởng nhưng phát biểu khác: nó đo mức đồng thuận vượt trên mức đồng thuận ngẫu nhiên tính từ tỉ lệ nền. Trên cùng ví dụ, kappa bằng 0,181. Cả hai đều là cách trả lời câu hỏi “con số này hơn đoán mò bao nhiêu”, và cả hai vì thế miễn nhiễm với trò 99% accuracy.

Cái giá phải trả cũng nên nói thẳng: MCC và kappa khó diễn giải hơn hẳn. Không ai nói được “MCC 0,26 nghĩa là người dùng gặp bao nhiêu báo động giả”. Chúng tốt để xếp hạng và để phát

hiện ảo giác, không tốt để *báo cáo cho người vận hành*. Trong thực tế, cặp precision–recall vẫn là thứ đi vào báo cáo, còn MCC là thứ bạn nhìn để tự kiểm.

7.8 Đường cong, và cái đường cong giấu đi (PR and ROC Curves)

Một điểm vận hành cho bốn ô. Nhưng ngưỡng là thứ bạn tự chọn, nên báo cáo một điểm duy nhất là báo cáo một lựa chọn chứ không phải một mô hình. Đường cong sinh ra để tách hai thứ đó: nó cho thấy *toàn bộ* các điểm vận hành mà mô hình có thể đạt tới.

A. Hai đường cong, thực là gì và một điểm nghĩa là gì

- **Đường ROC** — trục hoành là FPR, trục tung là TPR (tức recall). Một điểm trên đường là một ngưỡng cụ thể, và nó nói: “ở ngưỡng này, tôi bắt được TPR phần các ca dương, và làm phiền FPR phần các ca âm”. Cả hai trục chuẩn hoá *trong lớp*, nên đường ROC không phụ thuộc vào tỉ lệ nền.
- **Đường PR** — trục hoành là recall, trục tung là precision. Một điểm nói: “ở ngưỡng này, tôi bắt được recall phần các ca dương, và trong những gì tôi báo thì precision phần là đúng”. Trục tung chuẩn hoá theo *số lần báo*, nên đường PR phụ thuộc mạnh vào tỉ lệ nền.

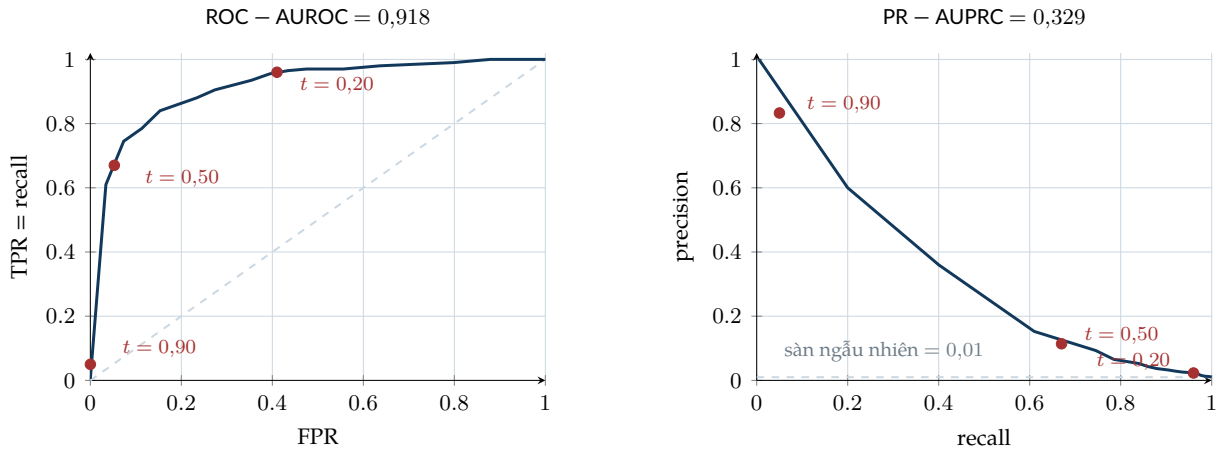
Đường của một bộ đoán ngẫu nhiên nằm ở hai chỗ khác nhau, và đây là chi tiết hay bị quên. Trên ROC, đoán ngẫu nhiên là đường chéo từ (0, 0) tới (1, 1), tức AUROC = 0,5, bất kể tỉ lệ nền. Trên PR, đoán ngẫu nhiên là một đường nằm ngang ở đúng độ cao bằng tỉ lệ nền. Với tỉ lệ nền 1%, sàn ngẫu nhiên của đường PR là 0,01. Vì thế AUPRC 0,33 không phải “kém hơn một nửa” như trục giác mách bảo — nó gấp ba mươi ba lần sàn.

B. Vì sao ROC trông đẹp giả tạo khi lớp dương hiếm

Đây là ý quan trọng nhất của cả mục, và nó nằm gọn trong một câu: *FPR có mẫu số rất lớn, nên FP tăng mạnh mà FPR gần như không nhúc nhích*.

Chạy con số cho cụ thể, vẫn trên bộ phân loại tổng hợp. Ở ngưỡng 0,50, FPR bằng 0,053. Nghe nhỏ. Nhưng 0,053 nhân với 19 800 mẫu âm ra 1043 lần báo động giả, trong khi chỉ có 134 lần báo đúng. Người vận hành nhận được 1177 cảnh báo mỗi ngày và 89% trong số đó là rác. Trục ROC không có cách nào hiển thị điều này, vì nó chia cho 19 800 chứ không chia cho 1177.

Hình 7.6 vẽ cả hai đường cho cùng một mô hình. AUROC bằng 0,918 — một con số bạn sẽ vui vẻ viết vào báo cáo. AUPRC bằng 0,329 trên cùng dữ liệu, cùng mô hình, cùng mọi thứ. Không cái nào sai. Chúng trả lời hai câu hỏi khác nhau, và chỉ một trong hai là câu hỏi mà người dùng đang hỏi.



Hình 7.6. Cùng một mô hình, cùng một dữ liệu, hai đường cong (bộ phân loại tổng hợp: 20 000 mẫu, 200 dương). Ba điểm đỏ là cùng ba ngưỡng t trên cả hai hình. Ngưỡng 0,50 nằm gần góc lý tưởng trên ROC nhưng nằm sát sàn trên PR, vì trục tung bên trái chia cho 19 800 mẫu âm còn bên phải chia cho 1177 lần báo động. Sàn ngẫu nhiên của ROC là đường chéo; sàn ngẫu nhiên của PR là đường ngang ở độ cao bằng tỉ lệ nền.

Quan hệ giữa hai không gian đã được chứng minh chặt: một đường cong chi phối trên ROC thì cũng chi phối trên PR và ngược lại, nhưng thứ hạng theo diện tích thì *không* bảo toàn [75]. Trên dữ liệu lệch lớp, đường PR phân biệt các mô hình rõ hơn hẳn [76]. Giới thiệu cổ điển về ROC và cách đọc nó vẫn là bài của Fawcett [77].

C. Chọn điểm vận hành bằng ma trận chi phí, không bằng thói quen

Nếu bạn gán được giá cho FP và FN thì việc chọn ngưỡng không còn là chuyện khẩu vị. Gọi c_{FP} và c_{FN} là chi phí một báo động giả và một ca bỏ sót. Chi phí kỳ vọng ở một ngưỡng là

$$\mathcal{C}(t) = c_{FP} \cdot FP(t) + c_{FN} \cdot FN(t).$$

Quét t qua toàn dải, tính $\mathcal{C}$, lấy điểm nhỏ nhất. Đó là toàn bộ thuật toán. Nó tầm thường về mặt tính toán và gần như không ai làm, vì phần khó là gán giá chứ không phải quét ngưỡng.

Với đóng vòng SLAM, phép tính này ra kết quả rất lệch. Đặt $c_{FN} = 1$ — một vòng bỏ lỡ mất một cơ hội xóa sai số. Đặt $c_{FP} = 100$ — một vòng sai lầm cong bản đồ và thường phải chạy lại. Tỉ lệ 100:1 là một ước lượng bậc thang từ cơ chế, không phải số đo; nhưng ngay cả khi bạn tin nó chỉ đúng trong vòng một bậc, kết luận vẫn không đổi: điểm vận hành phải nằm hẳn ở phần precision cao của đường cong, chấp nhận recall thấp. Đó chính xác là lý do ORB-SLAM3 đòi nhiều khung hình liên tiếp cùng đồng ý trước khi chấp nhận một vòng [40]. Cũng vì thế bộ từ vựng nhị phân luôn đi kèm bước kiểm chứng thời gian và hình học, không ai dùng nó trần [78].

D. Một đường cong đơn điệu vẫn ứng với hai hệ thống rất khác nhau

Đường cong tóm tắt tốt hơn một điểm, nhưng nó vẫn là một phép nén. Hai chỗ nó giấu thông tin:

- **Diện tích bằng nhau, hình dạng khác nhau.** Một mô hình có precision rất cao ở recall thấp rồi tụt dốc, và một mô hình precision trung bình đều trên toàn dải, có thể cho cùng một AUPRC. Nếu bạn chỉ vận hành ở vùng recall thấp thì mô hình thứ nhất tốt hơn nhiều. *Hãy đọc đường cong ở vùng bạn thật sự vận hành, đừng đọc diện tích.*
- **Cùng một đường, hai tập lỗi khác nhau.** Đường cong chỉ đếm số lượng, không nói *mẫu* nào sai. Hai mô hình cùng đường PR có thể sai trên hai nhóm mẫu hoàn toàn tách biệt. Cách phát hiện là đọc metric theo lát, hoặc đơn giản là đếm số mẫu mà hai mô hình bất đồng.

I 7.9 Xác suất có đáng tin không (Calibration)

Ba mục vừa rồi chỉ dùng *thứ hạng* của điểm số. Đường ROC, đường PR, AUC — tất cả đều không đổi nếu bạn ép điểm số qua một hàm tăng nghiêm ngặt bất kỳ. Điều đó có một hệ quả khó chịu: *một mô hình có AUC hoàn hảo vẫn có thể sai hoàn toàn về xác suất*.

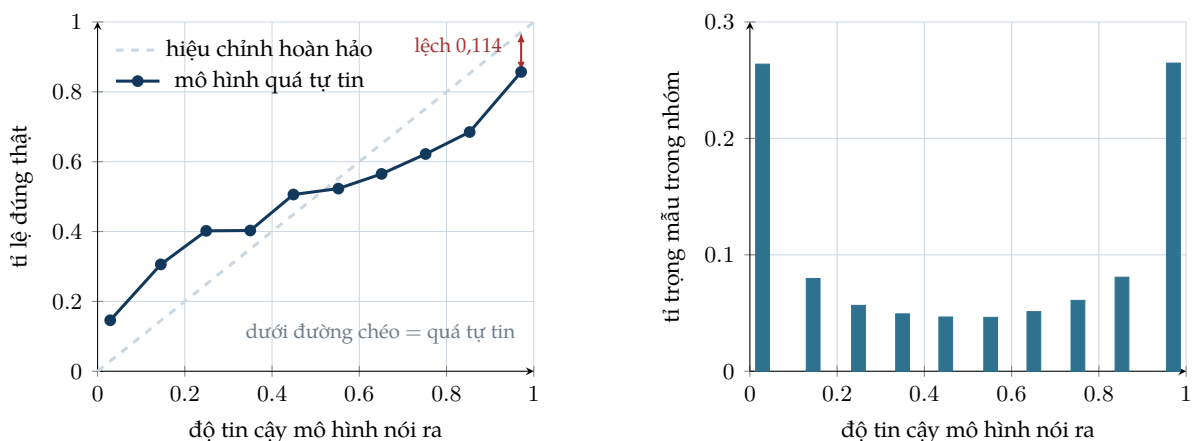
Chuyện này quan trọng khi đầu ra không dừng ở một quyết định nhị phân. Điểm số có thể đi vào một bộ hợp nhất cảm biến, vào một ma trận thông tin, hay vào một quy tắc “chỉ hành động khi độ tin cậy trên 0,9”. Trong cả ba trường hợp, giá trị số của nó là một phần của tính đúng đắn.

A. Biểu đồ độ tin cậy: đọc thế nào

biểu đồ độ tin cậy (*reliability diagram*) được vẽ theo ba bước. Chia các dự đoán thành các nhóm theo độ tin cậy mô hình nói ra — ví dụ mười nhóm, $[0; 0,1)$, $[0,1; 0,2)$, v.v. Trong mỗi nhóm, tính hai số: độ tin cậy trung bình mô hình *nói*, và tỉ lệ đúng *thật*. Vẽ cặp đó lên mặt phẳng. Mô hình hiệu chỉnh tốt nằm trên đường chéo.

Đọc lệch: điểm nằm *dưới* đường chéo nghĩa là mô hình nói mạnh hơn thực tế, tức quá tự tin; nằm trên là quá dè dặt. Mạng nơ-ron hiện đại lệch xuống một cách hệ thống ở vùng độ tin cậy cao [24].

Hình 7.7 vẽ một trường hợp như vậy. Ở nhóm mà mô hình nói 0,97, tỉ lệ đúng thật chỉ 0,86. Điều đáng chú ý nhất: mô hình này có AUROC *y* hết mô hình hiệu chỉnh đúng, vì nó chỉ bị làm sắc lại chứ không đổi thứ hạng. Mọi metric dựa trên thứ hạng đều không thấy gì.



Hình 7.7. Biểu đồ độ tin cậy của một mô hình quá tự tin (dữ liệu tổng hợp, 20 000 mẫu). Bên trái: đường nằm dưới đường chéo ở nửa phải, nghĩa là mô hình nói mạnh hơn thực tế; nhóm “0,97” chỉ đúng 0,86. Bên phải là tỉ trọng mẫu trong từng nhóm, cần thiết vì ECE lấy trung bình có trọng số theo cột này — một nhóm lệch nặng nhưng gần như rỗng thì đóng góp không đáng kể. ECE của mô hình này là 0,115.

B. ECE, Brier, và phân rã Brier

- **ECE** (*expected calibration error*) — lấy độ lệch tuyệt đối giữa hai số trong mỗi nhóm, rồi trung bình có trọng số theo số mẫu trong nhóm [79]. Trong hình trên, ECE bằng 0,115. Đọc là: “trung bình, độ tin cậy mô hình nói ra lệch khoảng 11 điểm phần trăm so với tần suất đúng thật”. Nhược điểm phải biết: ECE phụ thuộc vào cách chia nhóm, nên hai bài báo dùng số nhóm khác nhau không so được với nhau.
- **Điểm Brier** — trung bình bình phương sai lệch giữa xác suất dự đoán và nhãn 0/1 [80]. Nó là một hàm mất mát bình phương, nên nó phạt cả việc xếp hạng sai lẫn việc hiệu chỉnh sai. Ở ví dụ trên, Brier của mô hình là 0,188 còn của mô hình hiệu chỉnh đúng là 0,173.

- **Phân rã Brier** — và đây là chỗ nó trở nên hữu ích thật sự. Điểm Brier tách được thành ba phần: độ tin cậy (*reliability*), độ phân giải (*resolution*) và độ bất định nền (*uncertainty*) [81]. Viết gọn: $\text{Brier} = \text{reliability} - \text{resolution} + \text{uncertainty}$. Phần *reliability* đo đúng cái ECE đo, và càng nhỏ càng tốt. Phần *resolution* đo khả năng mô hình tách các mẫu thành các nhóm có tần suất khác nhau, và càng lớn càng tốt. Phần *uncertainty* chỉ phụ thuộc tỉ lệ nền, không phụ thuộc mô hình.

Phân rã này đáng nhớ vì nó cho một chẩn đoán rõ. Brier tệ do *reliability* lớn thì bạn chỉ cần hiệu chỉnh lại đầu ra, một việc rẻ và hậu kiểm được. Brier tệ do *resolution* nhỏ thì mô hình thật sự không phân biệt nổi các mẫu, và không phép hậu xử lý nào cứu được.

Cách sửa và cách vận hành phần này — *temperature scaling*, hiệu chỉnh theo lát, đo lại sau khi triển khai — thuộc về [E/25-do-tin-cay-va-van-hanh](#). Ở đây chỉ cần nhớ một điều: *AUC cao không cho phép bạn tin con số xác suất*.

7.10 Nhiều lớp và nhiều nhãn (Multi-class and Multi-label)

Bốn ô là chuyện của hai lớp. Với nhiều lớp, ma trận nhầm lẫn thành một bảng $K \times K$, và cách đọc nó là một kỹ năng riêng.

A. Đọc theo hàng và đọc theo cột là hai câu hỏi khác nhau

Quy ước phổ biến: hàng là lớp thật, cột là lớp dự đoán. Khi đó ô (i, j) là số mẫu thuộc lớp i bị gọi thành lớp j . Từ đó:

- **Đọc theo hàng i** — “những mẫu thật sự thuộc lớp i đã đi đâu”. Chia ô đường chéo cho tổng hàng ra *recall của lớp i* . Hàng có ô lớn lệch khỏi đường chéo cho biết lớp i đang bị nuốt bởi lớp nào.
- **Đọc theo cột j** — “những mẫu bị gọi là lớp j thật ra là gì”. Chia ô đường chéo cho tổng cột ra *precision của lớp j* . Cột dày cho biết lớp j là một “hố đen” mà mọi thứ rơi vào.
- **Cặp lớp bị lẫn** — nhìn cặp ô đối xứng (i, j) và (j, i) . Nếu cả hai đều lớn thì hai lớp đó thật sự khó phân biệt, và có thể định nghĩa nhãn đang chồng lấn. Nếu chỉ một chiều lớn thì đó là thiên lệch một phía, thường do mất cân bằng số lượng.
- **Lớp hiếm biến mất** — một hàng có tổng bằng vài chục trong bảng vài chục nghìn mẫu thì mọi con số tổng đều mù với nó. Đây là lý do phải báo macro cùng micro, như ở mục trước.

B. Top-1 và top-5

Top-1 là accuracy thông thường: nhãn có điểm cao nhất có đúng không. Top-5 hỏi nhẹ hơn: nhãn đúng có nằm trong năm nhãn điểm cao nhất không. Chênh lệch giữa hai con số là một chẩn đoán trực tiếp. Top-5 cao mà top-1 thấp nghĩa là mô hình *biết* vùng đáp án nhưng không xếp hạng nổi trong vùng đó — thường là các lớp cực giống nhau, hoặc nhãn mơ hồ ở chính chân trị. Cả hai cùng thấp nghĩa là mô hình chưa khoanh được vùng.

Cần một cảnh báo: top-5 có ý nghĩa vì ImageNet có 1000 lớp [82]. Với 8 lớp, top-5 gần như luôn bằng 1 và không mang thông tin nào.

C. Nhiều nhãn: subset accuracy so với Hamming

Bài toán nhiều nhãn cho phép một mẫu mang nhiều nhãn cùng lúc. Hai cách đo phổ biến nằm ở hai cực:

- **Subset accuracy** — chỉ tính đúng khi *toàn bộ* tập nhãn khớp chính xác. Rất nghiêm. Với 20 nhãn, một mẫu sai một nhãn cũng bị tính là sai hoàn toàn. Con số này rơi rất nhanh khi số nhãn tăng, và nó phạt như nhau dù bạn sai một nhãn hay sai cả hai mươi.
- **Hamming loss** — tỉ lệ ô nhãn sai trên tổng số ô, tính trên toàn bộ ma trận mẫu $\times$ nhãn. Rất dễ dài. Nếu mỗi mẫu chỉ có 2 trong 20 nhãn là dương, thì đoán “tất cả đều âm” đã cho Hamming loss 0,10, nghe như một kết quả tốt.

Hai cái đó đóng khung khoảng đúng, nên cách hành xử đúng là báo cả hai, cộng với macro-F1 theo nhãn. Riêng Hamming loss cần luôn kèm tỉ lệ nền trung bình, vì nếu không thì nó lặp lại đúng bấy accuracy 99% ở quy mô lớn hơn.

7.11 Metric của từng bài toán thị giác, và điều mỗi cái che giấu (Task-specific Metrics)

Các mục trên là bộ khung chung. Mỗi bài toán thị giác lại đóng gói bộ khung đó theo một cách riêng, và mỗi lần đóng gói lại giấu thêm một thứ. Mục này đi qua năm họ metric mà người đọc sẽ gặp, theo cùng một câu hỏi: *con số này tăng thì thật ra cái gì đã tốt lên*.

A. Phát hiện vật thể: IoU, AP, mAP và các biến thể

Ở phát hiện vật thể, trước khi đếm được TP và FP, phải quyết định một hộp dự đoán có “khớp” một hộp thật hay không. Đó là việc của IoU, tỉ số diện tích giao trên diện tích hợp. Khớp hay không là một quyết định nhị phân với ngưỡng riêng, nên bài toán có *hai* ngưỡng chứ không phải một: ngưỡng độ tin cậy, và ngưỡng IoU.

- **AP** — diện tích dưới đường PR của một lớp, quét theo ngưỡng độ tin cậy với ngưỡng IoU cố định. **mAP** là trung bình AP trên các lớp, tức một phép trung bình macro và vì thế lớp hiếm có quyền ngang lớp đông.
- **AP50 so với AP75** — cùng một mô hình, hai ngưỡng IoU. AP50 dễ dài với sai lệch định vị; AP75 đòi hộp bám sát. Chênh lệch giữa hai con số là một chẩn đoán: AP50 cao mà AP75 thấp nghĩa là mô hình *tìm* được vật nhưng *khoanh* không chuẩn.
- **AP theo kích thước (APs, APm, APl)** — COCO chia vật thể theo diện tích và báo riêng [83]. Đây là cách chia lát đã được đóng sẵn vào metric, và nó là chỗ ẩn nhiều thông tin nhất.
- **AR@k** — recall trung bình khi mỗi ảnh chỉ được giữ k đề xuất. Nó đo chất lượng của tầng sinh ứng viên, tách khỏi tầng phân loại [84]. Cùng tinh thần với Recall@K trong truy hồi ở mục C dưới đây.

Điều mAP che giấu, nói thẳng: “*mAP tăng 1 điểm*” có thể là vật lớn tốt lên trong khi vật nhỏ tệ đi. mAP là trung bình trên các lớp và ngưỡng, nên hai thay đổi ngược chiều có thể triệt tiêu nhau và để lại một con số tổng nhích lên. Nếu ứng dụng của bạn sống bằng vật nhỏ ở xa — đúng trường hợp phát hiện vật cản cho robot — thì con số tổng đang nói dối bạn. Cách chặn rất rẻ: luôn báo APs, APm, APl cùng mAP, và dùng một công cụ phân rã lỗi như TIDE khi cần biết lỗi thuộc loại nào [66], [85].

B. Phân đoạn: mIoU, Dice, boundary F và PQ

- **mIoU** — IoU tính theo pixel cho từng lớp rồi lấy trung bình *theo lớp*. Vì lấy trung bình theo lớp chứ không theo pixel, nó công bằng hơn accuracy pixel; nhưng nó vẫn giấu lớp hiếm theo một cách khác. Một lớp chỉ xuất hiện trong 2% số ảnh vẫn có một phiếu đầy đủ, nên IoU của nó dao động dữ dội giữa các lần chạy và kéo mIoU nhảy theo mà không có nguyên nhân thật.

- **Dice, tức F1 theo pixel** — $\text{Dice} = 2|A \cap B|/(|A| + |B|)$. Nó và IoU là hàm đơn điệu của nhau nên xếp hạng giống nhau, nhưng Dice luôn cho số cao hơn. Đừng so Dice của bài này với IoU của bài kia.
- **Boundary F** — đo riêng độ khớp của *đường biên* thay vì của cả vùng [86]. Cần nó vì IoU bị chi phối bởi phần lõi của vùng: một vật lớn có biên sai vài pixel gần như không làm IoU nhúc nhích, trong khi với một vật nhỏ thì đúng sai lệch ấy là toàn bộ câu chuyện. Đây là điểm đã được nêu từ lâu trong bàn luận về thước đo phân đoạn [87].
- **PQ (*panoptic quality*)** — tách thành hai phần nhân nhau: chất lượng nhận dạng (RQ, chính là F1 ở mức thực thể) và chất lượng phân đoạn (SQ, IoU trung bình của các thực thể đã khớp) [67]. Cấu trúc tích này rất đáng học: nó cho phép nói “PQ tụt vì RQ chứ không vì SQ”, tức vì đếm sai số thực thể chứ không vì khoanh xấu.

C. Truy hồi và nhận dạng địa điểm — và đóng vòng SLAM

Ở truy hồi, hệ thống trả về một danh sách xếp hạng chứ không một câu trả lời. Metric vì thế phải nói rõ “xét bao nhiêu ứng viên”.

- **Recall@K** — tỉ lệ truy vấn mà có ít nhất một kết quả đúng nằm trong K ứng viên đầu. Đây là metric chuẩn của nhận dạng địa điểm. Chú ý mẫu số: nó chia cho *số truy vấn*, không chia cho số ứng viên.
- **Precision@K** — tỉ lệ đúng trong K ứng viên đầu. Nó và Recall@K trả lời hai câu khác nhau, và với đóng vòng thì cái thứ hai mới là cái quan trọng, vì tầng sau phải xử lý cả K ứng viên.
- **mAP truy hồi** — trung bình AP trên các truy vấn, dùng khi một truy vấn có nhiều kết quả đúng.
- **Đường recall-precision theo K** — tăng K thì recall tăng và precision giảm. Đây chính là đường PR của mục trước, chỉ tham số hoá bằng K thay vì bằng ngưỡng.

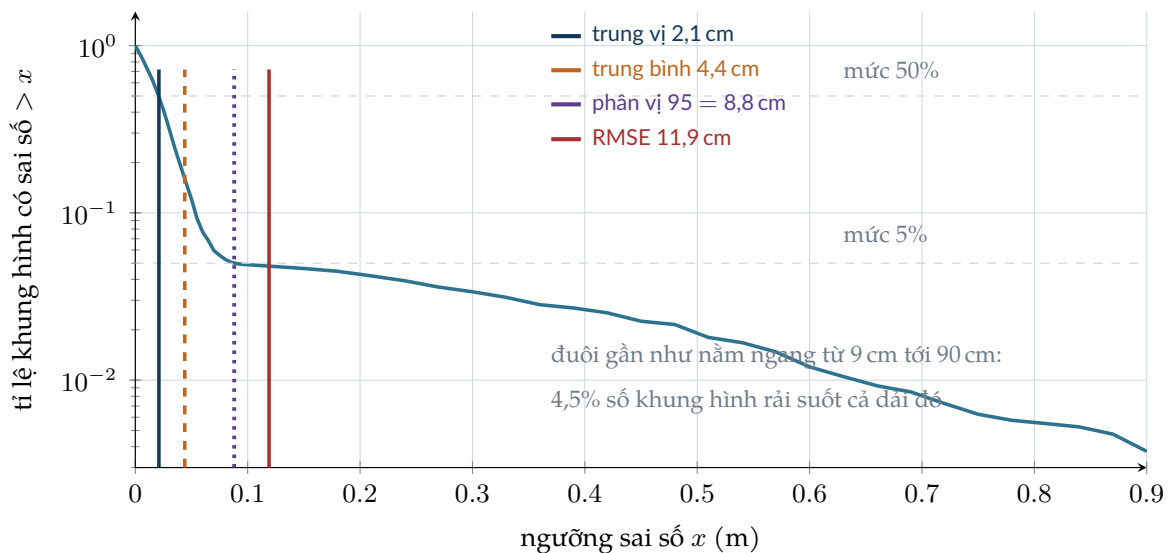
Nói thẳng sang đóng vòng. Tầng lấy ứng viên (DBoW2, NetVLAD, hay bất cứ thứ gì bạn dùng) được đánh giá bằng Recall@K, và điều đó *đúng*, vì việc của nó chỉ là không để lọt. Nhưng hệ thống đóng vòng đầy đủ thì phải được đánh giá bằng precision ở điểm vận hành cuối, sau kiểm chứng hình học. Hai tầng, hai metric, và lẫn lộn chúng là một lỗi thường gặp.

Lý do đã nói ở trên nhưng đáng lặp lại: *một vòng sai đắt hơn nhiều một vòng bỏ lỡ*. Vòng bỏ lỡ chỉ có nghĩa là sai số chưa được xoá; vòng sai khau hai nơi khác nhau thành một và làm hỏng bản đồ, thường không hồi phục. Cho nên điểm vận hành của hệ đóng vòng phải lệch hẳn về phía precision — gần góc trên trái của Hình 7.5 — và recall thấp là một lựa chọn có chủ ý, không phải một thất bại.

D. Hồi quy và quỹ đạo: MAE, RMSE, trung vị, phân vị

- **MAE** — trung bình trị tuyệt đối của sai số. Mỗi mẫu đóng góp tuyến tính.
- **RMSE** — căn của trung bình bình phương sai số. Bình phương làm mẫu lỗi lớn đóng góp gấp bội, nên RMSE là một thước đo *bị đuôi chi phối*.
- **Trung vị và phân vị** — trung vị nói “một mẫu điển hình sai bao nhiêu”; phân vị 95 và 99 nói “ngày tệ nhất tệ tới đâu”.

Hình 7.8 cho thấy vì sao ba nhóm này phải đi cùng nhau. Dữ liệu là sai số vị trí theo từng khung hình của một chuỗi tổng hợp: phần lớn thời gian bám tốt, và 4,5% số khung có một đợt bùng. Trung vị là 2,1 cm, trung bình 4,4 cm, còn RMSE lên tới 11,9 cm. RMSE gấp gần sáu lần trung vị, và toàn bộ chênh lệch đó do 4,5% số khung tạo ra.



Hình 7.8. Cùng một chuỗi tổng hợp (4000 khung), vẽ theo lối “bao nhiêu phần khung hình tệ hơn mức x ” với trục tung log. Đường tụt rất nhanh tới mức 5% rồi *gần như nằm ngang* suốt từ 9 cm tới 90 cm — đó chính là cái đuôi. Bốn đường dọc là bốn cách tóm tắt cùng phân bố này. Trung vị cắt đường ở mức 50%, phân vị 95 cắt ở mức 5%, còn RMSE nằm hẳn bên phải phân vị 95, tức nó mô tả một khung hình bạn gặp ít hơn một lần trong hai mươi. Báo một mình RMSE là báo về cái đuôi và im lặng về hành vi thường ngày; báo một mình trung vị thì ngược lại.

Với quỹ đạo SLAM, hệ quả rất cụ thể và đáng thành thói quen.

- **Báo APE kèm trung vị và phân vị, không chỉ RMSE.** Công cụ [MichaelGrupp/evo](#) in sẵn cả bộ; vấn đề nằm ở việc người ta chỉ chép mỗi dòng RMSE vào bảng. Hai hệ có cùng RMSE có thể là một hệ luôn lệch đều và một hệ hoàn hảo trừ hai giây bung ra. Hướng dẫn đánh giá quỹ đạo của Zhang và Scaramuzza nói rõ điểm này [88]; thư mục metric của TUM RGB-D là nơi cặp APE–RPE được chuẩn hoá [71].
- **Luôn báo tỉ lệ hoàn thành chuỗi cùng APE.** Đây là chỗ dễ tự lừa nhất. Một hệ mất bám ở giây thứ 30 của chuỗi 120 giây, rồi chỉ nộp phần quỹ đạo nó còn giữ, có thể cho APE *rất đẹp* — vì nó chỉ được chấm trên đoạn dễ. So sánh một hệ hoàn thành 100% với một hệ hoàn thành 25% bằng APE là so sánh vô nghĩa. Quy tắc tối thiểu: mọi bảng APE phải có một cột tỉ lệ hoàn thành ngay bên cạnh.

E. Ghép cặp đặc trưng: matching score, MMA, tỉ lệ inlier

- **Matching score** — tỉ lệ điểm đặc trưng phát hiện được mà tìm đúng điểm tương ứng, chuẩn hoá theo số điểm phát hiện được [89].
- **MMA (mean matching accuracy)** — tỉ lệ cặp khớp có sai số chiều dưới một ngưỡng pixel, lấy trung bình trên các cặp ảnh; nó được báo theo từng ngưỡng chứ không một số [90].
- **Tỉ lệ inlier** — tỉ lệ cặp khớp sống sót sau RANSAC. Đây là con số bạn nhìn hằng ngày trong log.

Và đây là câu hỏi mà người đọc thật sự cần trả lời: *vì sao ba con số này tăng mà APE không đổi*. Có bốn lời giải thích, và chúng cần bốn phép thử khác nhau.

- **Bão hoà.** Bộ ước lượng đã có thừa ràng buộc. Thêm cặp khớp đúng thứ 200 không dịch chuyển nghiệm khi 80 cặp đã đủ. Phép thử: giảm số cặp giữ lại xuống một nửa rồi đo lại APE; nếu APE không đổi thì bạn đang ở vùng bão hoà.

- **Đúng nhưng không đa dạng.** Cặp khớp mới tập trung vào vùng đã có nhiều điểm, hoặc ở cùng một mặt phẳng độ sâu. Hình học của bài toán không được cải thiện. Phép thử: đo phân bố không gian của các điểm khớp, và đo số điều kiện của ma trận thông tin trước và sau.
- **Nút thắt nằm ở chỗ khác.** Nếu sai số bị chi phối bởi hiệu chỉnh, độ trễ thời gian, hay mô hình IMU, thì cải thiện tầng khớp không đụng tới nút thắt. Phép thử: thay tầng khớp bằng chân trị (dùng chính chân trị để sinh cặp khớp hoàn hảo) và xem APE tụt tới đâu. Con số đó là trần lý thuyết của mọi cải tiến ở tầng khớp.
- **Metric ghép cặp đo trên phân bố khác.** Matching score và MMA thường đo trên các cặp ảnh có góc nhìn cách xa nhau, còn tracking khung-tới-khung thì góc nhìn gần như nhau. Cải thiện ở chế độ khó không chuyển thành cải thiện ở chế độ dễ. Phép thử: đo lại MMA đúng trên phân bố cặp khung hình mà hệ thống thật sự dùng.

Bài học tổng quát vượt ra ngoài SLAM: *một metric trung gian chỉ có giá trị khi nó nằm trên đường tới hạn của metric cuối.* Trước khi tối ưu một metric trung gian, hãy đo xem nó có nằm trên đường tới hạn không — bằng đúng phép thử “thay bằng chân trị” ở trên.

F. Bảng tóm tắt: mỗi metric trả lời câu hỏi nào

Bảng 7.6 gom năm họ trên vào một chỗ, theo đúng bốn bước của cả khối: nó trả lời câu hỏi nào, mẫu số là gì, và nó im lặng về điều gì.

Bảng 7.6. Metric của từng bài toán, đọc theo cùng một khuôn. Cột “im lặng về” là cột đáng đọc trước, vì nó nói cho bạn biết phải đọc thêm metric nào nữa.

Metric	Câu hỏi nó trả lời	Mẫu số	Nó im lặng về
mAP	Trung bình chất lượng xếp hạng phát hiện trên các lớp	mỗi lớp một phiếu	Vật nhỏ so với vật lớn; chất lượng định vị nếu chỉ dùng ngưỡng IoU 0,5
AR@k	Tầng sinh ứng viên có để lọt vật nào không	số vật thật	Chất lượng phân loại và xếp hạng ở tầng sau
mIoU	Mỗi lớp được khoanh khớp tới đâu, tính theo pixel	mỗi lớp một phiếu	Đường biên; và lớp hiếm có IoU dao động mạnh kéo con số nhảy
Boundary F	Đường biên khớp tới đâu	số điểm biên	Phần lõi của vùng; một vùng lệch hẳn vị trí vẫn có thể có biên trơn
PQ	Vừa đếm đúng thực thể vừa khoanh đúng chưa	thực thể	Cần tách RQ và SQ mới biết hồng ở đếm hay ở khoanh
Recall@K	Ứng viên đúng có nằm trong K kết quả đầu không	số truy vấn	Chi phí xử lý K ứng viên đó ở tầng sau; và precision cuối cùng
APE (ATE)	Quỹ đạo lệch bao nhiêu sau khi căn chỉnh	số khung hình được chấm	Trôi toàn cục đã bị căn chỉnh hấp thụ; và phần chuỗi bị mất bám
RPE	Các đoạn dịch chuyển ngắn có nhất quán không	số đoạn	Sai lệch toàn cục
Tỉ lệ inlier	Bao nhiêu cặp khớp sống sót sau RANSAC	số cặp khớp đề xuất	Hình học có được cải thiện không; và nút thắt có nằm ở tầng khớp không

7.12 Bảng chẩn đoán ngược (Reading Numbers Backwards)

Đây là đích đến của cả khối. Mọi mục trên đi từ định nghĩa tới ý nghĩa. Trong công việc thật, bạn đi ngược: nhìn thấy một bộ số lạ, rồi phải tìm ra nó nghĩa là gì. Bảng 7.7 tổ chức việc đó theo ba cột — triệu chứng, các cách giải thích, và phép thử tách chúng ra.

Cách dùng bảng: đọc cột giữa như một danh sách nghi phạm, không phải một câu trả lời. Gần như mọi triệu chứng đều có từ hai tới bốn nguyên nhân khác nhau, và chúng đòi những cách sửa khác hẳn nhau. Cột phải là thứ biến nghi ngờ thành kết luận.

Bảng 7.7. Từ con số ngược về nguyên nhân. Cột phải luôn là một việc làm được trong một buổi, không phải một lời khuyên chung chung.

Bạn thấy	Các cách giải thích	Phép thử để biết là cách nào
Accuracy 0,99 mà recall lớp hiếm bằng 0	Mô hình đã học chiến lược “luôn đoán lớp đa số”; hoặc lớp hiếm quá ít mẫu để học; hoặc hàm mất mát không có trọng số lớp	Tính accuracy của mô hình rỗng. Nếu bằng nhau thì mô hình đúng là rỗng. Sau đó thử cân bằng lại hàm mất mát và xem recall lớp hiếm có nhúc nhích không; nếu vẫn 0 thì vẫn đề là số mẫu chứ không phải trọng số.
AUROC 0,95 mà AUPRC 0,25	Lớp dương hiếm — đây là hành vi <i>bình thường</i> , không phải lỗi; hoặc mô hình xếp hạng tốt ở vùng giữa mà tệ ở đỉnh danh sách	So AUPRC với tỉ lệ nền: nếu AUPRC gấp hàng chục lần tỉ lệ nền thì mô hình thật sự có ích và bạn chỉ đang đọc nhầm thang đo. Vẽ đường PR và nhìn riêng vùng recall thấp để bắt trường hợp thứ hai.
mAP tăng còn mIoU giảm	Cải thiện ở tầng phát hiện thực thể đi kèm suy giảm ở tầng phân loại pixel; hoặc hai metric được tính trên hai tập lớp khác nhau; hoặc thay đổi hậu xử lý làm hộp chặt hơn còn mặt nạ thô hơn	Đọc AP và IoU theo từng lớp rồi so cặp. Nếu chỉ vài lớp đảo chiều thì là thay đổi thật ở các lớp đó. Kiểm tra danh sách lớp của hai phép đánh giá có trùng nhau không — đây là nguyên nhân tầm thường nhưng rất hay gặp.
Val loss giảm mà metric không đổi	Mô hình đang cải thiện độ tin cậy chứ không đổi thứ hạng; hoặc metric có ngưỡng nên thay đổi nhỏ chưa vượt ngưỡng; hoặc metric bão hoà	Vẽ biểu đồ độ tin cậy trước và sau: nếu ECE giảm mà AUC không đổi thì đúng là chỉ hiệu chỉnh đổi. Quét lại toàn dải ngưỡng; nếu đường PR dịch lên mà điểm vận hành cố định thì là chuyện ngưỡng.
Precision khi triển khai thấp hơn hẳn trên tập test	Tỉ lệ nền ngoài đời thấp hơn trong tập test; hoặc phân phối đầu vào đã dịch chuyển; hoặc tập test đã bị lọc trước bởi một hệ thống cũ	Đo tỉ lệ nền thật ở hiện trường và tính lại precision kỳ vọng theo tỉ lệ đó. Nếu con số khớp thì nguyên nhân là mẫu số, không phải mô hình. Nếu vẫn lệch thì sang B/08-dịch-chuyen-phan-phoi .
F1 đứng yên trong khi cả precision lẫn recall đều đổi	Hai thay đổi bù trừ nhau — một hệ thống khác hẳn đang trộn sau một con số không đổi	Vẽ điểm vận hành cũ và mới trên cùng mặt phẳng PR. Rồi tính F_2 và $F_{0,5}$: nếu chúng xếp hạng ngược nhau thì hai hệ thật sự khác, và β đúng theo chi phí của bạn mới là trọng tài.
Recall@1 cao mà đóng vòng vẫn sai	Tầng lấy ứng viên tốt nhưng tầng kiểm chứng hình học yếu; hoặc nhãn “cùng chỗ” sinh tự động quá lỏng; hoặc lỗi nằm ở tối ưu sau khi chấp nhận vòng	Đếm riêng bốn tầng: ứng viên, ghép đặc trưng, kiểm chứng, tối ưu. Đo precision <i>sau</i> kiểm chứng chứ không trước. Siết ngưỡng khoảng cách của nhãn tự động xuống rồi đo lại Recall@1; nếu nó sập thì nhãn đang quá lỏng.
APE đẹp mà chạy thật thấy trôi	Phép căn chỉnh đã hấp thụ trôi toàn cục; hoặc chuỗi không hoàn thành và chỉ đoạn dễ được chấm	Đọc RPE trên nhiều độ dài đoạn. Kiểm tra tỉ lệ hoàn thành chuỗi trước mọi việc khác — đây là kiểm tra rẻ nhất và hay bị bỏ qua nhất.
APE trung vị tốt mà RMSE tệ	Có một hoặc vài đợt bùng ngắn; hoặc vài khung hình bị lệch mốc thời gian	Vẽ sai số theo thời gian. Nếu là vài đỉnh nhọn thì đọc log ở đúng các mốc đó. Nếu là bậc thang thì nghi mốc thời gian hoặc một lần khởi tạo lại.
Tỉ lệ inlier tăng mà APE không đổi	Bảo hoà ràng buộc; hoặc cặp khớp mới không đa dạng về hình học; hoặc nút thất nằm ở hiệu chỉnh, IMU hay độ trễ	Giảm nửa số cặp khớp và đo lại APE. Thay tầng khớp bằng cặp khớp sinh từ chân trị để lấy trần lý thuyết. Nếu trần đó cũng không tốt hơn hiện tại thì tầng khớp không phải nút thắt.
Macro-F1 thấp hơn micro-F1 rất nhiều	Các lớp hiếm đang bị bỏ rơi — đây gần như luôn là nguyên nhân	Xếp F1 theo lớp từ thấp lên và nhìn năm lớp đáy cùng số mẫu của chúng. Nếu chúng đều là lớp ít mẫu thì đã rõ; nếu có lớp nhiều mẫu nằm trong đáy thì đó là chuyện định nghĩa nhãn.
Metric nhảy mạnh giữa các lần chạy cùng cấu hình	Phương sai do huấn luyện (hạt giống, thứ tự dữ liệu, kernel không tắt định); hoặc tập đánh giá quá nhỏ; hoặc metric nhạy ngưỡng	Chạy lặp năm lần và đo độ lệch chuẩn. So nó với sai số chuẩn tính từ cỡ tập test. Nếu độ lệch giữa các lần chạy lớn hơn hẳn thì nguồn là huấn luyện, không phải cỡ mẫu.

Bạn thấy	Các cách giải thích	Phép thử để biết là cách nào (tiếp)
Mô hình mới thắng ở mAP nhưng thua ở APs	Cải thiện chỉ có ở vật lớn; hoặc thay đổi độ phân giải đầu vào; hoặc gán mẫu đối làm vật nhỏ mất mẫu dương	Báo tách APs, APm, API. Nếu chỉ APs giảm thì kiểm tra độ phân giải và quy tắc gán mẫu trước khi kiểm tra bất cứ thứ gì khác.
Top-5 cao mà top-1 thấp	Các lớp cực giống nhau; hoặc nhãn chân trị mơ hồ với ảnh nhiễu vật	Đọc ma trận nhầm lẫn và tìm các cặp ô đối xứng lớn. Bốc tay 30 ảnh sai và tự gán nhãn lại: nếu bạn cũng phân vân thì đó là trần đồng thuận, không phải lỗi mô hình.
Precision cao ở mọi ngưỡng nhưng recall trần thấp	Mô hình mù với một dạng con của lớp dương; hoặc tăng trước đã lọc mất dạng đó	Đọc recall theo lát. Tìm lát nào có recall gần 0. Rồi kiểm tra dạng đó có xuất hiện trong tập huấn luyện với đủ số lượng không.
ECE tốt mà mô hình vẫn ra quyết định tệ	Hiệu chỉnh tốt trên toàn cục nhưng lệch theo từng lát; hoặc vấn đề là độ phân giải chứ không phải độ tin cậy	Tính ECE riêng theo từng lát. Phân rã điểm Brier: nếu phần reliability nhỏ mà resolution cũng nhỏ thì mô hình không tách được mẫu, và hiệu chỉnh không cứu được.
Hai mô hình cùng AUPRC nhưng người dùng thích hẳn một cái	Hai đường cong cùng diện tích, khác hình dạng; hoặc chúng sai trên hai nhóm mẫu khác nhau	So hai đường cong tại đúng điểm vận hành đang dùng, không so diện tích. Đếm số mẫu hai mô hình bất đồng và xem chúng thuộc lát nào.

7.13 Mọi con số phải kèm một khoảng (Every Number Needs an Interval)

Cả khối trên đọc từng con số như thể nó chính xác. Nó không. Mỗi metric trong chương là một ước lượng tính từ một mẫu hữu hạn, nên nó có một khoảng dao động, và khoảng đó thường lớn hơn mức người ta tưởng.

Phần công cụ chung — sai số chuẩn, so sánh bất cặp, phương sai do huấn luyện — nằm ở mục sau của chính chương này, và kỷ luật thí nghiệm đầy đủ nằm ở [E/24-thi-nghiem-va-ablation](#). Ở đây chỉ nói ba điều riêng của việc *đọc* metric.

- **Mẫu số nhỏ thì khoảng rất rộng, và mẫu số của metric theo lớp luôn nhỏ.** Recall của một lớp có 10 mẫu dương chỉ nhận được 11 giá trị rời rạc, cách nhau 0,1. Nói “recall lớp C tăng từ 0,1 lên 0,2” nghĩa là *một mẫu* đổi kết quả. Đó không phải một cải tiến, đó là một mẫu.
- **Metric không phải trung bình thì không có công thức sai số chuẩn.** mAP, HOTA, PQ, AUPRC đều là hàm phi tuyến của toàn bộ tập, nên công thức $\sqrt{p(1-p)/n}$ không áp dụng được. Cách đúng là *bootstrap* [91]: lấy mẫu có hoàn lại từ chính tập test, tính lại metric, lặp vài nghìn lần, rồi đọc phân vị 2,5 và 97,5. Nó tốn vài phút CPU và trả về đúng thứ bạn cần.
- **Bootstrap phải lấy mẫu theo đúng đơn vị phân tích.** Nếu tập test gồm nhiều khung hình từ ít video, thì lấy mẫu theo khung hình sẽ cho khoảng hẹp giả tạo, vì các khung trong cùng video không độc lập. Phải lấy mẫu theo *video*. Đây là cùng một nguyên tắc đã chi phối cách chia dữ liệu ở đầu chương, xuất hiện lại ở tầng đo độ bất định.

Quy tắc chốt lại: *đo sần nhiều trước, nói về cải tiến sau*. Không có sần nhiều thì mọi so sánh trong bảng chẩn đoán ở trên đều có thể đang chẩn đoán một dao động ngẫu nhiên.

7.14 Con số tổng che nhóm bị bỏ rơi (Slice-based and Behavioral Evaluation)

Ngay cả metric đúng, tính trên tập chia đúng, vẫn là một trung bình — và trung bình che các nhóm nhỏ một cách hoàn hảo. Hai công cụ bù lại chỗ đó, và cả hai đều rẻ:

- **slice-based evaluation** — cắt tập test theo các lát có nghĩa vận hành rồi đọc metric trên từng lát.
 - **Lát nào** — ban đêm, vật nhỏ, mức che khuất cao, từng camera, từng địa điểm, từng đợt thu.
 - **Điều kiện tiên quyết** — phải có metadata cho từng mẫu, nên hãy thu thập nó *từ lúc gán nhãn* (B/06); không có metadata thì không có lát nào cắt được.
 - **Giá trị** — theo kinh nghiệm thực hành, đây là bước có tỉ lệ “phát hiện vấn đề trên mỗi giờ công” cao nhất trong toàn bộ quy trình đánh giá. Khi nhóm tệ nhất mới là thứ quan trọng, có cả một dòng phương pháp tối ưu trực tiếp cho nó [92].
- **kiểm thử hành vi (behavioral testing)** — kiểm tra các tính chất mà bạn *biết* hệ thống phải có, theo kiểu unit test.
 - **Bất biến** — xoay ảnh vài độ thì nhãn không được đổi.
 - **Có hướng** — che vật thể đi thì độ tin cậy phải giảm, không được tăng.
 - **Vì sao cần** — loại test này không cho con số so với paper, nhưng nó bắt đúng loại lỗi mà tập test không bắt được, vì tập test chia sẻ mọi thiên lệch với tập train.

7.15 Chênh lệch 0,3% có thật không? (Uncertainty in Reported Numbers)

Đây là chỗ người đọc có lợi thế sẵn: bạn đã biết chạy cùng một cấu hình ba lần cho ba con số khác nhau, và phần lớn “cải tiến” được báo cáo trong một chiều tối nằm gọn trong phổ đó. Cần biến trực giác này thành phép tính, và có hai nguồn phương sai hoàn toàn khác nhau.

→ **tiền đề E/24** — kỷ luật chạy lặp và báo cáo độ lệch ở đây chính là điều kiện tối thiểu để một bảng ablation có nghĩa.

- **Phương sai do tập test hữu hạn.** Với 1000 ảnh và accuracy 92%, sai số chuẩn là $\sqrt{0,92 \cdot 0,08/1000} \approx 0,0086$, nên **khoảng tin cậy (confidence interval)** 95% rộng khoảng $\pm 1,7$ điểm phần trăm. Công thức này nói: *nếu lấy một tập test khác cùng cỡ từ cùng phân phối, con số sẽ nhảy trong khoảng đó*. Một chênh lệch 0,3 điểm nằm sâu bên trong và không nói lên điều gì.
 - **Nhưng phải trung thực về một điểm tinh tế** — khi hai mô hình đo trên cùng một tập test, phép so sánh là bất cập, và chỉ những mẫu chúng bất đồng mới mang thông tin. Bất đồng ở 40 mẫu, chia 25 nghiêng về A và 15 về B: chênh lệch quan sát được là 1,0 điểm với sai số chuẩn cỡ $\sqrt{40}/1000 \approx 0,63$ điểm — chặt hơn khoảng tin cậy rời rạc ở trên, nhưng vẫn chưa đủ để tuyên bố A thắng.
 - **Bài học kép** — khoảng tin cậy rời rạc là ước lượng bảo thủ cho so sánh bất cập; và ngay cả phép so sánh chặt hơn cũng thường không cứu được một chênh lệch một điểm.
- **Phương sai do quá trình huấn luyện.** Hạt giống ngẫu nhiên, thứ tự dữ liệu, tính không tắt định của kernel GPU — nguồn này không giảm đi khi bạn thu thập thêm ảnh.
 - **Cách xử lý đúng** — chạy lặp, báo cáo trung bình kèm độ lệch, không chọn lần chạy tốt nhất; đúng kỷ luật bạn đã áp dụng cho benchmark harness của mình. Với metric phức tạp như mAP hay HOTA (không có công thức sai số chuẩn giải tích), dùng **bootstrap**: lấy mẫu có hoàn lại từ chính tập test vài nghìn lần rồi đọc phân vị.
 - **Bối cảnh** — việc báo cáo phương sai theo hạt giống từng bị chỉ ra là điểm yếu có hệ thống của cả lĩnh vực [48]. Trong một số nhánh, phần lớn “tiến bộ” biến mất khi **baseline** được tune ngang bằng [41], [42].

7.16 Giới hạn và trường hợp hỏng

Giao thức chặt tới đâu cũng chỉ đo được cái mà tập test đại diện. Nếu tập test lấy từ cùng một khung lấy mẫu với tập train, mọi thiên lệch chung giữa hai bên vẫn vô hình — chia đúng chỉ chặn rõ ràng, không chặn thiên lệch chung. Đây là ranh giới giữa chương này và chương sau: nội dung ở đây giả định phân phối không đổi, còn khi phân phối đổi thì mọi ước lượng ở đây đều lạc quan.

Nguyên tắc “chỉ chạm test một lần” cũng không sống sót nguyên vẹn: một dự án hai năm sẽ chạm test nhiều lần, không tránh được. Cách xử lý trung thực là ghi lại số lần chạm và thay tập test khi nó đã hao mòn — nghĩa là phải giữ sẵn ngân sách thu thập một tập test mới. Cuối cùng, chia theo nhóm và theo thời gian đều làm tập test nhỏ đi và phương sai lớn lên: một cái giá thật, và với dữ liệu ít bạn đôi khi buộc phải chọn giữa ước lượng thiên lệch có phương sai nhỏ và ước lượng không thiên lệch có phương sai lớn.

CẠM BẤY

Cạm bẫy tốn kém nhất không phải chia sai mà là *chọn tập test sau khi đã nhìn kết quả*. Nó xuất hiện dưới những dạng trông vô hại: bỏ vài sequence “bị lỗi thu”, thêm một tập test thứ hai vì tập đầu “không đại diện”, hay chọn checkpoint theo test vì val nhỏ quá. Cả ba đều là cùng một hành vi, và cả ba đều chuyển test thành val mà không ai ghi vào nhật ký.

CẦN NHỚ

Cách chia là một phát biểu về cách mô hình sẽ được dùng, và metric là một phép chiếu làm mất thông tin. Nên hai câu hỏi phải hỏi trước mọi bảng kết quả là: *cách chia này giả định gì*, và *metric này đang giấu loại lỗi nào*. Với một tập test 1000 mẫu, chênh lệch dưới khoảng một điểm phần trăm chưa phải là chênh lệch. [T2]

7.17 Thực hành

Repo và tài nguyên. [cocodataset/cocoapi](#) là định nghĩa tham chiếu của mAP — khi hai bài báo báo cáo mAP khác nhau, thủ phạm thường là cài đặt chứ không phải mô hình; [Lightning-AI/torchmetrics](#); [voxel51/fiftyone](#) (đánh giá theo lát); [dbolya/tide](#) (phân rã lỗi detection); [MichaelGrupp/evo](#) (ATE/RPE); [photosynthesis-team/piq](#) và [richzhang/PerceptualSimilarity](#) (LPIPS); [GaParmar/clean-fid](#). Danh sách chốt tại tháng 9/2026 và sẽ cũ trong 6–12 tháng; riêng nhóm hạ tầng metric (cocoapi, torchmetrics, evo) bền hơn hẳn tên các mô hình dẫn đầu bảng.

Câu hỏi kiểm tra hiểu. mAP@0.5 và mAP@0.5:0.95 khác nhau ở chỗ nào, và loại cải tiến nào chỉ hiện lên ở cái thứ hai? Vì sao AUROC gây hiểu lầm khi lớp dương chiếm 1%, trong khi AUPRC thì không? Bước căn chỉnh trước khi tính ATE hấp thụ mất loại lỗi nào, và metric nào bù lại chỗ đó?

Bài tập phụ. Tự cài mAP từ đầu — matching theo IoU, xây đường cong precision–recall, nội suy — rồi so với [pycocotools](#) trên cùng một file dự đoán. Chênh lệch sẽ xuất hiện, và việc truy ra nó đến từ quy tắc nội suy, từ cách xử lý mẫu iscrowd, hay từ giới hạn số phát hiện mỗi ảnh, dạy nhiều hơn bất kỳ bài đọc nào về metric.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Rò rỉ dữ liệu	Bất kỳ đường nào làm thông tin của tập test lọt vào huấn luyện hoặc chọn mô hình. Đặc điểm nguy hiểm là nó không báo lỗi, chỉ làm con số đẹp lên.
●●● Vì sao không được chọn checkpoint theo tập test?	Vì như vậy test đã đóng vai trò của val. Train ước lượng tham số, val chọn siêu tham số và quyết định lúc dừng, test chỉ được đo đúng một lần ở cuối.
●●● Vì sao chuẩn hoá trước khi chia đã là rò rỉ?	Vì mean và std tính trên cả tập test đã đi vào mô hình. Rò rỉ không cần tới nhãn — chỉ cần một thông tin nào đó của test tham gia vào huấn luyện.
●●● Chia theo nhóm	Mọi mẫu chung một nguồn — cùng bệnh nhân, cùng video, cùng camera — nằm trọn một bên. Hai khung hình cách nhau 33 ms không phải hai quan sát độc lập, nên bỏ qua nó là cách rẻ nhất để tự tặng mình hàng chục điểm hiệu năng ảo.
●● Chia theo thời gian	Test nằm sau train trên trục thời gian. Cần khi dữ liệu có trôi; chia ngẫu nhiên ở đây cho mô hình đặc quyền nội suy giữa quá khứ và tương lai, thứ nó không có khi chạy thật.
●●● Chia lại tập năm lần đều cho cùng con số, nhưng triển khai thì sập — nguyên nhân?	Chia đúng chỉ chặn rò rỉ, nó không chặn thiên lệch chung. Nếu train và test cùng lấy từ một khung lấy mẫu thì mọi cách chia đều cho cùng một con số lạc quan. Phân biệt bằng vài trăm mẫu thu ở điều kiện triển khai thật rồi đo lại.
●●● Adaptive overfitting	Test hao mòn dần vì mỗi lần bạn nhìn nó rồi ra quyết định là một lần thông tin rò qua bộ não bạn. Sau vài chục vòng, test đã trở thành val.
●● Thứ hạng leaderboard không tái lập trên tập test mới — nguyên nhân?	Adaptive overfitting ở quy mô cộng đồng: hàng nghìn cấu hình được thử trên một tập test cố định. Phân biệt với tiến bộ thật bằng cách thu một tập test mới theo đúng quy trình cũ rồi đo lại.
●● Prevalence	Tỉ lệ mẫu thuộc lớp dương trong dữ liệu. Khi nó nhỏ, accuracy, precision, recall và FPR rời xa nhau tới mức đọc nhầm một con số là kết luận sai hoàn toàn.
●●● AUPRC	Diện tích dưới đường cong precision–recall. Phải dùng thay AUROC khi lớp dương hiếm, vì nó chuẩn hoá theo số lần báo động chứ không theo số mẫu âm.
●●● Vì sao AUROC gây hiểu lầm khi lớp dương chiếm 1%?	Vì FPR chia cho 990 mẫu âm, nên 9% báo động giả trông nhỏ mà thực ra là 91 lần báo nhầm. Precision chia cho tổng 100 lần báo động và cho ra 9%, đúng cái người dùng cảm nhận.
●● Accuracy 99% trên tập có 1% mẫu dương — đọc thêm gì?	Precision và recall ở đúng điểm làm việc. Một mô hình luôn trả lời “âm” cũng đạt 99%, nên chỉ hai con số đó mới tách được mô hình có ích khỏi mô hình rỗng.
●●● Bốn ô TP, FP, FN, TN	TP báo có và đúng; FP báo có nhưng sai, tức báo động giả ai cũng thấy; FN báo không nhưng thật ra có, tức ca bỏ sót không ai thấy; TN báo không và đúng. Chữ thứ hai là cái hệ thống nói, chữ thứ nhất là nó có nói đúng không.
●●● Mẫu số của precision, recall và FPR	Precision chia cho mọi lần báo “có”. Recall chia cho mọi ca thật sự dương. FPR chia cho mọi ca thật sự âm. Ba mẫu số khác nhau là lý do cùng một ô FP cho ba con số nghe khác hẳn nhau.
●●● Precision cao, recall thấp — hệ thống đang làm gì?	Nó nhất: chỉ báo khi rất chắc nên nói gì cũng đúng, nhưng bỏ sót phần lớn ca thật. Nguyên nhân cạnh tranh: ngưỡng quá chặt, chỉ bắt được dạng dễ, hoặc thiếu bao phủ. Quét toàn dải ngưỡng để tách nguyên nhân đầu khỏi hai cái sau.
●●● Precision thấp, recall cao — hệ thống đang làm gì?	Nó báo bừa: bắt được gần hết nhưng phần lớn cảnh báo là rác. Nguyên nhân: ngưỡng quá lỏng, tỉ lệ nền quá thấp, hoặc nhãn âm bị thiếu nên cái bắt đúng lại bị tính là FP. Bóc tay 50 mẫu FP và gắn nhãn lại để tách nguyên nhân cuối.
●● Vì sao F1 dùng trung bình điều hoà?	Vì trung bình điều hoà bị con số nhỏ kéo xuống, còn trung bình cộng thì không. Với $P = 0,95$ và $R = 0,40$, trung bình cộng cho 0,675 còn F1 cho 0,563 — con số sau phản ánh đúng hơn việc bỏ sót 60% ca thật.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Macro-F1 thấp hơn micro-F1 rất nhiều — nghĩa là gì?	Các lớp hiếm đang bị bỏ rơi. Micro cho mỗi mẫu một phiếu nên lớp đông quyết định; macro cho mỗi lớp một phiếu. Trong ví dụ ba lớp của chương, micro là 0,946 còn macro chỉ 0,619.
●●● Vì sao ROC trông đẹp giả tạo khi lớp dương hiếm?	Vì FPR có mẫu số rất lớn. FPR 0,053 nghe nhỏ, nhưng nhân với 19 800 mẫu âm ra 1043 báo động giả trong khi chỉ có 134 báo đúng. Đường PR chia cho số lần báo nên nó nhìn thấy điều đó.
●● Biểu đồ độ tin cậy	Chia dự đoán theo độ tin cậy thành nhóm, rồi vẽ độ tin cậy trung bình mô hình nói ra so với tỉ lệ đúng thật. Nằm dưới đường chéo là quá tự tin. Một mô hình quá tự tin có AUC y hết mô hình hiệu chỉnh đúng, vì thứ hạng không đổi.
●● Đọc ma trận nhầm lẫn nhiều lớp theo hàng và theo cột	Theo hàng: mẫu thật sự thuộc lớp này đã đi đâu, chia ra recall. Theo cột: mẫu bị gọi là lớp này thật ra là gì, chia ra precision. Cặp ô đối xứng cùng lớn nghĩa là hai lớp thật sự khó phân biệt.
●● mAP@0.5:0.95	mAP lấy trung bình trên nhiều ngưỡng IoU thay vì chỉ 0,5. Nó là chỗ duy nhất mà cải tiến về độ chính xác định vị hiện ra, vì ở ngưỡng 0,5 hộp lệch nhẹ vẫn được tính là đúng.
● HOTA	Metric theo dõi đối tượng cân bằng giữa phát hiện và giữ đúng danh tính. Cần nó vì MOTA cao vẫn hoàn toàn tương thích với việc ID bị hoán loạn liên tục.
●●● ATE	Sai số quỹ đạo tuyệt đối, tính sau khi căn quỹ đạo ước lượng vào quỹ đạo thật. Nó đo độ lệch toàn cục, nên phải đọc kèm RPE để thấy tính nhất quán trên các đoạn ngắn.
●●● Phép căn chỉnh trước khi tính ATE hấp thụ mất gì?	Trôi toàn cục. Một quỹ đạo trôi tỉ lệ đều đặn bị kéo về sát tham chiếu và cho ra ATE dễ chịu, trong khi lỗi cục bộ vẫn còn nguyên và chỉ RPE mới thấy.
●● Calibration	Mức khớp giữa độ tin cậy mô hình nói ra và tần suất nó đúng thật. Mạng hiện đại thường quá tự tin, nên mọi ngưỡng quyết định đặt sau đầu ra đều lệch theo.
●● Slice-based evaluation	Cắt tập test theo nhóm có nghĩa vận hành — ban đêm, vật nhỏ, từng camera — rồi đọc metric trên từng lát. Điều kiện là có metadata cho từng mẫu, nên phải thu thập từ lúc gán nhãn.
● Kiểm thử hành vi	Kiểm tra các tính chất bạn biết chắc phải đúng, theo kiểu unit test: xoay ảnh vài độ thì nhãn không đổi, che vật thì độ tin cậy phải giảm. Nó bắt loại lỗi mà tập test không bắt được.
●●● Vì sao đóng vòng phải lệch hẳn về precision?	Vì một vòng sai khâu hai nơi khác nhau thành một và làm cong bản đồ, thường không hồi phục; còn một vòng bỏ lỡ chỉ là một cơ hội xoá sai số chưa dùng tới. Recall thấp ở đây là lựa chọn có chủ ý.
●●● Vì sao RMSE của quỹ đạo phải đi kèm trung vị và phân vị?	Vì bình phương làm cái đuôi chi phối. Trong ví dụ của chương, trung vị 2,1 cm còn RMSE 11,9 cm, và toàn bộ chênh lệch đo 4,5% số khung tạo ra.
●● Bootstrap cho metric không phải trung bình	Lấy mẫu có hoàn lại từ chính tập test, tính lại mAP hoặc AUPRC, lặp vài nghìn lần, đọc phân vị. Phải lấy mẫu theo đúng đơn vị phân tích — theo video chứ không theo khung hình — nếu không khoảng sẽ hẹp giả tạo.
●●● Hai mô hình chênh 0,3 điểm trên 1000 ảnh — kết luận?	Chưa kết luận được gì. Sai số chuẩn của accuracy 92% trên 1000 mẫu khoảng 0,86 điểm, nên khoảng tin cậy 95% rộng chừng $\pm 1,7$ điểm. So sánh bất cập chặt hơn nhưng thường vẫn không cứu được một điểm.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
cocodataset/cocoapi	Là định nghĩa vận hành của mAP: quy tắc ghép theo IoU, nội suy đường PR, giới hạn số phát hiện mỗi ảnh	Hạ tầng; hai bài báo lệch mAP thường lệch vì cài đặt chứ không vì mô hình
WILDS (p-lambda/wilds)	Chia theo nhóm và theo miền thật — bệnh viện khác, năm khác, vùng khác — thay vì chia ngẫu nhiên	Bằng chứng rằng “chia theo nhóm” đã thành chuẩn thiết kế benchmark
TIDE (dbolya/tide)	Phân rã một con số mAP thành các loại lỗi tách biệt, đúng ý “metric là phép chiếu”	Cùng dòng với công trình phân tích lỗi detection sớm hơn [85]
ImageNet-v2	Thu một tập test mới theo đúng quy trình gốc để đo phần hiệu năng đã mất do tập test cũ hao mòn	Cách duy nhất đã biết để định lượng adaptive overfitting [18]
MichaelGrupp/evo	Chuẩn hoá việc báo cáo ATE cùng RPE và ghi rõ kiểu căn chỉnh đã dùng	Hạ tầng đánh giá SLAM; chính chỗ bẫy căn chỉnh trở nên hữu hình

MINI PROJECT SLAM · Định lượng rõ ràng do chia ngẫu nhiên theo khung hình · 1–2 ngày

Mục tiêu	biến câu “chia ngẫu nhiên trên video là rõ ràng” thành một con số điểm phần trăm đo được trên chính dữ liệu bạn dùng hằng ngày.
Dữ liệu	ba tới bốn chuỗi EuRoC. Nhãn “cùng chỗ / khác chỗ” lấy từ mini project chương B/06-chat-luong-du-lieu , mở rộng tự động bằng chân trị quỹ đạo: hai khung hình cách nhau dưới 1 m và lệch góc dưới 30° thì gán là cùng chỗ.
Các bước	dựng một bộ phân loại cặp thật đơn giản — một ngưỡng trên điểm tương đồng DBoW2, hoặc hồi quy logistic trên khoảng cách cosine giữa hai vectơ mô tả toàn ảnh. Huấn luyện và đo cùng bộ đó ba lần với ba cách chia: ngẫu nhiên theo khung hình, theo đoạn thời gian liền mạch trong cùng chuỗi, và theo chuỗi (giữ trọn một chuỗi ra ngoài). Mỗi cách chạy năm lần với năm hạt giống, ghi cùng một metric (AUPRC, hoặc precision tại recall 0,9).
Xong khi	bạn viết được một câu dạng “chia ngẫu nhiên theo khung hình cho AUPRC cao hơn chia theo chuỗi X điểm, khoảng tin cậy $\pm Y$ ”, trong đó Y tính từ năm lần chạy chứ không đoán.
Nối với	B/06-chat-luong-du-lieu là nguồn nhãn; B/08-dich-chuyen-phan-phoi vì chia theo chuỗi chính là mô phỏng dịch chuyển miền ở quy mô nhỏ; E/24-thi-nghiem-va-ablation cho cách đọc khoảng tin cậy đó.

MINI PROJECT SLAM · Đường PR của tầng lấy ứng viên đóng vòng, và điểm vận hành theo chi phí · 1–2 ngày

Mục tiêu	biến câu “một vòng sai đắt hơn một vòng bỏ lỡ” thành một ngưỡng cụ thể, chọn bằng phép tính chứ bằng cảm giác.
Dữ liệu	ba tới bốn chuỗi EuRoC. Nhãn “cùng chỗ” sinh từ chân trị quỹ đạo: hai khung hình cách nhau dưới 1 m và lệch góc dưới 30°. Ghi lại luôn kết quả với ngưỡng 2 m để đo xem con số nhạy thế nào với chính định nghĩa nhãn.
Các bước	lấy điểm tương đồng của tầng lấy ứng viên mà hệ bạn đang dùng (DBoW2 hoặc tương đương) cho mọi cặp khung hình đủ xa nhau về thời gian. Quét ngưỡng trên điểm đó, ghi TP, FP, FN, TN ở từng ngưỡng, vẽ cả đường PR lẫn đường ROC trên cùng một trang. Sau đó đặt $c_{FN} = 1$ và thử ba giá trị $c_{FP} \in \{10, 100, 1000\}$, rồi tìm ngưỡng cực tiểu hoá $c_{FP} \cdot FP + c_{FN} \cdot FN$ cho từng giá trị. Cuối cùng lập toàn bộ với nhãn ngưỡng 2 m.
Xong khi	bạn có một hình hai bảng cạnh nhau cho thấy ROC gần như không đổi trong khi PR đổi rõ; một bảng ba dòng “chi phí c_{FP} — ngưỡng tối ưu — precision và recall tại đó”; và một câu nói rõ ngưỡng tối ưu dịch bao nhiêu khi định nghĩa nhãn đổi từ 1 m sang 2 m.

Nối với [B/06-chat-luong-du-lieu](#) cho chất lượng nhãn tự động; mini project trước của chính chương này cho cách chia dữ liệu khi đo; [E/25-do-tin-cay-va-van-hanh](#) cho việc biến điểm tương đồng thành một xác suất đáng tin trước khi đặt ngưỡng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì với tỉ lệ dương 1%, một mô hình luôn trả lời “âm” đã đạt 99% . Hai con số cần thêm là precision và recall ở đúng điểm làm việc — hoặc tương đương, AUPRC thay cho AUROC.
- A2. Vì thống kê của tập test đã đi vào mô hình qua bộ chuẩn hoá. Rõ ràng không cần nhãn: chỉ cần một thông tin nào đó của test tham gia vào quá trình huấn luyện hay chọn mô hình là con số đã lạc quan.
- A3. Vì adaptive overfitting: cả cộng đồng thử hàng nghìn cấu hình trên một tập test cố định, nên phần thắng cuối cùng có một phần là khớp riêng tập đó. Thu một tập test mới cùng quy trình làm phần đó bốc hơi.
- A4. Hấp thụ trôi toàn cục: phép biến đổi cứng (hoặc có tỉ lệ, với đơn mắt) kéo quỹ đạo trôi đều đặn về sát tham chiếu. RPE bù lại chỗ đó vì nó đo tính nhất quán trên các đoạn ngắn và mù với sai lệch toàn cục.
- A5. Với 1000 ảnh, khoảng tin cậy 95% của một con số 92% rộng khoảng $\pm 1,7$ điểm. So sánh bất cặp chặt hơn: nếu hai mô hình bất đồng ở 40 mẫu chia 25/15 thì sai số chuẩn của chênh lệch cỡ 0,63 điểm — vẫn chưa đủ để tuyên bố thắng.
- A6. Nó đang *nhất*: chỉ mở miệng khi rất chắc, nên nói gì cũng đúng, nhưng im lặng ở 60% ca thật. Ba nghi phạm là ngưỡng quá chặt, mô hình chỉ bắt được dạng dễ, và tập huấn luyện thiếu bao phủ dạng hiếm. Quét toàn dải ngưỡng để tách nghi phạm thứ nhất khỏi hai cái còn lại.
- A7. Vì F1 gộp hai số bằng một phép trung bình đối xứng nên nó che mất bất đối xứng chi phí. Hệ $P = 0,90$, $R = 0,45$ và hệ $P = R = 0,60$ cùng cho $F_1 = 0,600$; với đóng vòng, nơi một vòng sai đắt hơn nhiều một vòng bỏ lỡ, hệ đầu tốt hơn hẳn. $F_{0,5}$ nói ra điều đó, F1 thì không.
- A8. Vì AUC chỉ dùng thứ hạng của điểm số, mà thứ hạng không đổi khi bạn ép điểm số qua một hàm tăng bất kỳ. Biểu đồ độ tin cậy phát hiện chỗ lệch: điểm nằm dưới đường chéo nghĩa là mô hình nói mạnh hơn thực tế. ECE và phân rã điểm Brier định lượng phần lệch đó.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy nghĩ về một kỳ thi. Bạn có ba tập giấy: sách bài tập để học, một đề thi thử để biết mình yếu chỗ nào, và một đề thi thật nghiêm phong để chấm điểm cuối cùng. Điểm của đề thật chỉ có nghĩa khi thí sinh chưa từng nhìn thấy nó.

Bài toán gốc là mọi cách để đề bị lộ mà không ai cố ý làm lộ. Bạn photo đề thật lẫn vào sách bài tập rồi phát nhầm một trang. Bạn ra đề thử bằng đúng các câu của đề thật, chỉ đổi số. Bạn cho lớp này thi trước rồi mới thi lớp sau. Hoặc tình vi nhất: bạn cho thí sinh làm đề thật ba mươi lần, mỗi lần xem điểm rồi chỉnh cách ôn. Sau ba mươi lần, điểm cao không còn nói lên năng lực nữa, nó chỉ nói rằng họ đã quen với đúng tờ đề đó.

Mẹo có ba phần. Một, quyết định trước ai vào tập nào theo đúng cách bạn sẽ gặp người mới: nếu ngoài đời bạn sẽ gặp học sinh trường khác thì đề thật phải là học sinh trường khác. Hai, đừng đọc mỗi con số tổng — xem điểm theo từng nhóm, và luôn hỏi chênh lệch nửa điểm giữa hai thí sinh có lớn hơn sai số của chính kỳ thi hay không.

Ba, và đây là phần dễ quên nhất: mỗi lần thấy một tỉ lệ, hãy hỏi nó chia cho cái gì. “Chín mươi phần trăm” là chín mươi phần trăm của *cái gì*. Một người gác cổng chặn nhầm 5% số khách vào toà nhà nghe như rất giỏi; nhưng nếu mỗi ngày có hai vạn khách và chỉ hai trăm kẻ gian, thì 5% ấy là một nghìn người bị chặn oan, so với hơn một trăm kẻ gian bắt được. Cùng một sai lầm, chia cho hai thứ khác nhau, cho ra hai câu chuyện trái ngược. Con số nào cũng chỉ có nghĩa cùng với mẫu số của nó.

Cái giá là đề thi thật trở thành thứ tiêu hao: dùng ít lần thôi, và phải để dành ngân sách ra một đề mới.

Chỗ không rút gọn được: việc “chênh lệch này có thật không” thì không rút gọn được thành một quy tắc. Đo bằng khoảng dao động của từng con số thì quá dè dặt, mà chỉ đếm những chỗ hai bên trả lời khác nhau thì lại quá dễ dãi khi hai bên vốn rất giống nhau.

Một câu: Mọi kỹ thuật trong chương chỉ bảo vệ một thứ duy nhất: một con số đo trên phần dữ liệu mà bạn chưa từng chạm vào, kể cả bằng ánh mắt.

7.18 Kết nối

Chương này là hệ quả trực tiếp của [B/05-thiet-ke-bai-toan](#): đơn vị phân tích chọn ở đó quyết định cách chia hợp lệ ở đây. Nó dựa trên [B/06-chat-luong-du-lieu](#) ở hai chỗ — trùng lặp gần đúng là một đường rò rỉ, và trần đồng thuận quyết định chênh lệch nào đáng bàn. Nó là tiền đề của [B/08-dich-chuyen-phan-phoi](#), chương gỡ bỏ đúng giả định mà toàn bộ chương này đứng lên: rằng test và thực tế cùng một phân phối. Phần calibration nối sang [E/25-do-tin-cay-va-van-hanh](#), còn kỷ luật thí nghiệm và ablation được khai triển đầy đủ ở [E/24-thi-nghiem-va-ablation](#).

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao chuẩn hoá dữ liệu trước khi chia là rò rỉ, dù bạn không hề dùng nhãn của test?
2. Trong Hình 7.1, vì sao câu hỏi về nhóm đứng trước câu hỏi về mất cân bằng lớp?
3. Một mô hình đạt accuracy 99% trên tập có 1% mẫu dương. Cần thêm đúng hai con số nào để biết nó có ích hay không?
4. Vì sao ATE và RPE phải đọc cùng nhau, và phép căn chỉnh làm mất thông tin gì?
5. Adaptive overfitting không tạo ra bug, không tạo ra cảnh báo. Vậy dấu hiệu sớm nào cho thấy tập test của bạn đã hao mòn?
6. Hai mô hình chênh nhau 1 điểm phần trăm trên 1000 ảnh. Phép tính nào cho biết chênh lệch đó có ý nghĩa không, và vì sao khoảng tin cậy rời rạc lại là ước lượng bảo thủ ở đây?
7. Chạy lặp với nhiều hạt giống làm giảm nguồn phương sai nào, và *không* làm giảm nguồn nào?
8. Precision 0,95 và recall 0,40: hệ thống đang hành xử thế nào, và phép thử nào tách “ngưỡng quá chặt” khỏi “mô hình mù với dạng hiếm”?
9. Vì sao AUROC gần như không nhúc nhích khi số báo động giả tăng gấp mười lần trên dữ liệu có 1% mẫu dương, và metric nào thì có?

Phân phối, dịch chuyển và ngoài phân phối (Distribution Shift and OOD)

PHỤ THUỘC B/04-nen-tang-hoc-tu-du-lieu (giả định IID và vì sao mọi bảo đảm học máy đứng trên nó); B/05-thiet-ke-bai-toan (thiên lệch chọn mẫu (*selection bias*), khung lấy mẫu); B/07-chia-du-lieu-va-danh-gia (chương này gỡ bỏ đúng giả định mà giao thức ở đó dựa vào).

TÓM TẮT

Chương này nói về chuyện xảy ra khi dữ liệu thật không còn cùng phân phối với dữ liệu huấn luyện — tức là gần như luôn luôn. Đọc xong bạn thay được câu “bị domain shift” bằng một chẩn đoán cụ thể chỉ ra nên gán nhãn lại, nên chỉnh đầu ra, hay nên thu thập thêm; bạn dựng được một quy trình giám sát trôi phân phối; và bạn biết vì sao mô hình lại mong manh đến vậy. Nếu chỉ nhớ một điều: ba loại dịch chuyển — $p(x)$ đổi, $p(y)$ đổi, $p(y | x)$ đổi — cho cùng một triệu chứng nhưng đòi ba cách chữa loại trừ nhau, và chỉ loại thứ ba mới bắt buộc phải gán nhãn lại.

KIẾN THỨC CẦN CÓ TRƯỚC

- **phân phối xác suất và ký hiệu** $p(\cdot)$ — $p(x)$ mô tả những ảnh nào hay xuất hiện và những ảnh nào hiếm; $p(y)$ mô tả tỉ lệ các lớp; $p(y | x)$ là “cho trước ảnh này, nhãn đúng của nó có khả năng là gì”; $p(x | y)$ là “ảnh của lớp này trông thế nào”. Toàn bộ chương xoay quanh đúng một câu hỏi: trong bốn thứ đó, thứ nào đã thay đổi.
- **giả định IID** — giả định rằng mỗi mẫu được rút ra một cách độc lập từ cùng một phân phối, cho cả dữ liệu huấn luyện lẫn dữ liệu tương lai. Mọi bảo đảm lý thuyết của học máy đứng trên giả định này, và chương này nói về chuyện xảy ra khi nó vỡ.
- **ba loại dịch chuyển phân phối** — *covariate shift* — $p(x)$ đổi (camera mới, ánh sáng khác) nhưng quan hệ ảnh–nhãn $p(y | x)$ vẫn đúng. *Label shift* — tỉ lệ các lớp $p(y)$ đổi trong khi ảnh của mỗi lớp vẫn y hệt. *Concept drift* — chính $p(y | x)$ đổi, tức cùng một ảnh nay phải gán nhãn khác. Ba loại cho cùng một triệu chứng nhưng đòi ba cách chữa loại trừ nhau.
- **mô hình, huấn luyện và tối thiểu hoá rủi ro thực nghiệm (empirical risk minimization, ERM)** — mô hình là hàm có tham số biến ảnh thành dự đoán; huấn luyện là tìm tham số làm trung bình mất mát trên tập huấn luyện nhỏ nhất — đúng cái nguyên tắc gọn ấy có tên là ERM. Nó là *baseline* mà mọi phương pháp phức tạp hơn trong chương này phải vượt qua.
- **tiên nghiệm (prior) và tỉ số cược (odds)** — tiên nghiệm là xác suất của một lớp *trước khi* nhìn ảnh, tức tỉ lệ lớp đó trong dữ liệu; tỉ số cược của một xác suất p là $p/(1 - p)$, một cách viết khác của cùng thông tin nhưng nhân chia được. Cần biết vì phép chữa rẻ nhất trong cả chương chỉ là nhân tỉ số cược với tỉ lệ giữa hai tiên nghiệm.
- **đuôi dài (long-tail)** — **phân biệt với mất cân bằng lớp** — mất cân bằng lớp là vài lớp có ít mẫu hơn hẳn các lớp khác. Đuôi dài là tình huống có *rất nhiều* lớp mà mỗi lớp chỉ có dăm bảy mẫu, nên vấn đề không còn là tỉ lệ mà là không đủ dữ liệu để học được lớp đó là gì. Phân biệt này quan trọng vì cách chữa của trường hợp đầu không dùng được cho trường hợp sau.
- **ngoài phân phối (OOD), softmax, độ tin cậy và hiệu chuẩn (calibration)** — softmax biến các điểm số thô ở đầu ra thành một bộ số dương cộng lại bằng 1, đọc như xác suất; giá trị lớn nhất trong đó gọi là độ tin cậy của mô hình. Hiệu chuẩn tốt nghĩa là khi mô hình nói 0.8 thì nó đúng khoảng 80% số lần. Một đầu vào *ngoài phân phối* là đầu vào khác hẳn mọi thứ mô hình từng thấy

lúc huấn luyện, và “phát hiện OOD” là cố nhận ra nó để hệ thống nói “tôi không chắc” thay vì trả lời bừa.

- **embedding, augmentation và corruption** — embedding là vector vài trăm chiều biểu diễn một ảnh, ảnh giống nhau cho vector gần nhau — dùng ở đây để so hai tập dữ liệu mà không cần nhãn. Augmentation là cố ý biến dạng ảnh *huấn luyện* (lật, đổi màu, thêm nhiễu, làm mờ) để mô hình bớt phụ thuộc chi tiết không quan trọng; corruption là cùng loại biến dạng đó nhưng áp lên tập *kiểm tra* để đo độ bền. Cùng một phép biến đổi, hai mục đích ngược nhau.

CÂU HỎI MỞ ĐẦU

- Q1. Mô hình đạt 95% trên tập test và 60% ở khách hàng đầu tiên. Ba câu hỏi nào tách được nguyên nhân, và câu nào cho phép chữa mà không cần huấn luyện lại một tham số nào?
- Q2. Vì sao một mạng có thể phân biệt bò với lạc đà rất giỏi mà thực ra chưa bao giờ nhìn con vật?
- Q3. Nếu hàng trăm phương pháp tổng quát hoá miền đều tốt hơn ERM trong bài báo của chính chúng, vì sao khi tune công bằng thì không cái nào thắng?
- Q4. Vì sao độ tin cậy của mô hình lại là tín hiệu cảnh báo tệ nhất, đúng vào lúc bạn cần nó nhất?

8.1 Đặt vấn đề

Mọi kết quả lý thuyết ở B/04 — từ chặn tổng quát hoá tới chính khái niệm “sai số kỳ vọng” — đứng trên một giả định duy nhất: dữ liệu huấn luyện và dữ liệu tương lai được rút độc lập từ cùng một phân phối. Trong thị giác máy tính, giả định đó gần như luôn sai. Camera được thay sau mười tám tháng, firmware ISP cập nhật, mùa đổi, khách hàng mới ở một thành phố khác, một loại xe mới xuất hiện trên đường.

Sai lầm đắt nhất của người mới ở đây không phải chọn sai phương pháp thích nghi, mà là chẩn đoán quá thô. Hệ thống tụt tám điểm sau khi triển khai, và kết luận duy nhất được đưa ra là “bị domain shift” — một câu vừa đúng vừa vô dụng, vì nó không dẫn tới hành động nào. Ba nguyên nhân hoàn toàn khác nhau đều cho ra cùng triệu chứng ấy, và chúng đòi ba cách chữa loại trừ lẫn nhau: một cái cần gán nhãn lại, một cái chỉ cần vài phép nhân ở đầu ra, một cái cần thu thập thêm dữ liệu miền mới.

Có một trục giác từ robotics giúp đóng khung cả chương. Một hệ định vị được điều chỉnh cho một mô hình nhiễu cụ thể chạy tuyệt vời trong đúng điều kiện nó được điều chỉnh; đổi cảm biến hay đổi bề mặt sàn thì nó không báo lỗi, nó chỉ *sai một cách tự tin*. Mạng nơ-ron còn tệ hơn ở chỗ nó không có **phần dư** (*residual*) để bạn nhìn vào, nên nó không có cách nào tự nói rằng mình đang ở ngoài vùng hợp lệ. Phần lớn chương này là các cách xây dựng nhân tạo cái phần dư ấy.

→ **hệ quả của B/04** — giả định IID là nền của mọi bảo đảm tổng quát hoá; gỡ nó ra thì các bảo đảm đó mất hiệu lực, chứ không phải yếu đi.

TRỰC GIÁC

Hãy hình dung một robot định vị bằng cách đếm số bước từ cửa phòng thay vì đọc bản đồ. Trong toà nhà nó được hiệu chỉnh, kết quả chính xác tuyệt đối và không có cách nào phân biệt nó với một hệ định vị thật. Sang toà nhà khác — cùng loại hành lang, cùng loại cửa, chỉ khác chiều dài — nó sai hoàn toàn ngay bước đầu tiên. Nó không hỏng; nó đã học đúng thứ tương quan mạnh nhất với đáp án trong dữ liệu nó thấy. Đây chính là cơ chế của **shortcut learning**, và nó giải thích vì sao độ chính xác cao trên tập test không hề bảo đảm hệ thống hiểu bài toán.

8.2 Ba loại dịch chuyển (Covariate, Label and Concept Shift)

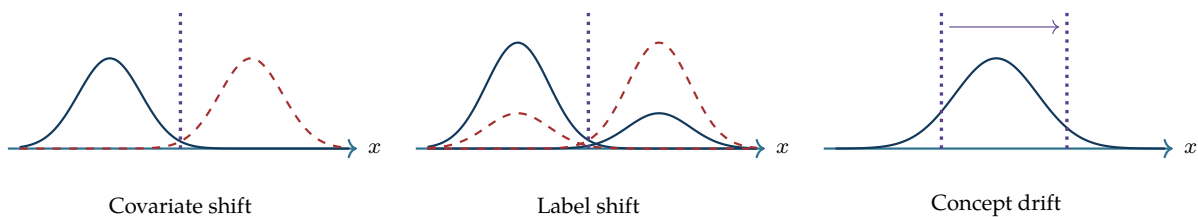
Muốn thay “bị shift” bằng một chẩn đoán dùng được, phải tách bài toán theo cấu trúc của phân phối chung $p(x, y)$. Nói gọn thì có ba cách phân rã nó, và mỗi cách cho một loại dịch chuyển với một cách chữa riêng.

A. Phân biệt bằng cấu trúc của $p(x, y)$

Bảng 8.1. Ba loại dịch chuyển. Cột cuối là lý do phải phân biệt: cách chữa của loại này vô dụng với loại kia.

Loại	Cái gì đổi	Dấu hiệu	Cách xử lý
dịch chuyển hiệp biến (<i>covariate shift</i>)	$p(x)$ đổi, $p(y x)$ giữ nguyên	Ảnh trông khác đi nhưng quan hệ ảnh–nhãn vẫn đúng	Reweighting theo mật độ, augmentation, domain adaptation
dịch chuyển nhãn (<i>label shift</i>)	$p(y)$ đổi, $p(x y)$ giữ nguyên	Tỉ lệ lớp đổi; ảnh của mỗi lớp vẫn y hệt	Hiệu chỉnh tiên nghiệm ngay ở đầu ra, không cần train lại
trôi khái niệm (<i>concept drift</i>)	$p(y x)$ đổi	Cùng một ảnh nay phải gán nhãn khác	Gán nhãn lại và huấn luyện lại; không có đường tắt

Hình 8.1 vẽ ba loại đó trên một trục đặc trưng duy nhất, và điểm đáng nhìn là chúng làm biến dạng những bộ phận khác nhau của cùng một bức tranh: hai loại đầu di chuyển *dữ liệu* quanh một biên quyết định đứng yên, còn loại thứ ba di chuyển *chính biên quyết định*. Đó là lý do hai loại đầu chữa được bằng số học trong khi loại thứ ba đòi nhãn mới.



Hình 8.1. Ba loại dịch chuyển trên một trục đặc trưng. Nét liền xanh là phân phối lúc huấn luyện, nét đứt đỏ là lúc triển khai, nét chấm tím là biên quyết định. Hai panel đầu: dữ liệu dịch chuyển, biên đứng yên. Panel cuối: dữ liệu y nguyên, biên dịch chuyển — và đó là lý do duy nhất trong ba trường hợp mà nhãn cũ trở thành nhãn sai.

B. Ví dụ tính tay: hiệu chỉnh tiên nghiệm khi có label shift

Label shift đáng có một ví dụ số vì nó là trường hợp duy nhất chữa được gần như miễn phí, và cũng là trường hợp hay bị bỏ qua nhất. Giả sử bạn huấn luyện một bộ phân loại nhị phân trên tập cân bằng (tỉ lệ dương 50%), còn khi triển khai tỉ lệ dương thật chỉ là 5%. Mô hình trả về $p = 0,60$ cho một mẫu.

- Đổi sang **tỉ số cược** (*odds*): $0,60/0,40 = 1,50$.
- Vì $p(x | y)$ không đổi, **tỉ số độ hợp lý** (*likelihood ratio*) giữ nguyên và chỉ phần tiên nghiệm cần sửa. Nhân với $\frac{0,05/0,95}{0,50/0,50} \approx 0,0526$, được 0,079.
- Đổi ngược về xác suất: $p \approx 0,079/1,079 \approx 0,073$.

Phép tính này nói điều gì? Rằng một dự đoán “khá chắc là dương” thật ra là “7% khả năng dương”, và toàn bộ sự chênh lệch đến từ việc mô hình mang theo tiên nghiệm của tập huấn

luyện chứ không phải của thế giới. Không train lại một tham số nào; chỉ cần biết tỉ lệ lớp mới. Ngược lại, nếu vấn đề thật là concept drift thì phép nhân này không những vô ích mà còn làm sai lệch thêm, vì giả định $p(x | y)$ bất biến đã vỡ ngay từ đầu.

↔ **tiền đề E/25** — hiệu chỉnh tiên nghiệm là một thao tác vận hành: nó sửa đầu ra theo tỉ lệ lớp thật, và chỉ dùng được khi đầu ra đã hiệu chuẩn.

8.3 Biết là đã trôi, chưa biết có hại (Shift Detection)

Trước khi chữa phải phát hiện, và phát hiện rẻ hơn nhiều so với người ta tưởng. Hai công cụ thực dụng, cộng một giới hạn phải luôn nhớ:

- **Khoảng cách trên không gian *embedding*** — so phân phối đặc trưng của dữ liệu huấn luyện với dữ liệu đang chạy. Rẻ, chạy liên tục được, nhưng con số khoảng cách tự nó khó diễn giải.
- **Bộ phân loại phân biệt miền** — huấn luyện một bộ phân loại nhị phân để tách “mẫu train” khỏi “mẫu production”. Đây là dụng cụ đo tốt hơn vì kết quả đọc được ngay:
 - $AUC \approx 0,5$ — hai tập không phân biệt được, chưa có dịch chuyển đáng kể.
 - $AUC \approx 0,95$ — chỉ cần nhìn ảnh cũng biết nó đến từ đâu.
 - **Lợi ích kèm theo** — các đặc trưng mà bộ phân biệt dựa vào chính là mô tả cụ thể của dịch chuyển, nên nó vừa báo động vừa chẩn đoán.
- **Giới hạn chung của cả hai** — chúng trả lời “phân phối có đổi không”, không phải “việc đó có làm hỏng hệ thống không”. Hai điều đó tách rời theo cả hai chiều: đổi tông màu toàn cục là thay đổi lớn về thống kê nhưng vô hại nếu mô hình không dùng màu; ngược lại một thay đổi bé xíu đúng vào đặc trưng mô hình dựa vào lại đủ làm sập nó.

Hệ quả thực hành: tín hiệu dịch chuyển phải luôn được đối chiếu với một metric thật trên một tập nhỏ có nhãn thu ở miền đích. Giám sát dịch chuyển mà không có mẫu có nhãn nào ở miền đích là giám sát mù. Vì thế ngân sách nên đổ vào việc lấy vài trăm nhãn mỗi tháng từ production, trước khi đổ vào thuật toán thích nghi.

8.4 Thích nghi hay tổng quát hoá (Domain Adaptation vs Generalization)

Đến khi phải hành động, có một phân đôi quyết định toàn bộ chiến lược, và nó phụ thuộc vào đúng một câu hỏi: *bạn có mẫu từ miền đích hay không?*

Mạch 1 — từ thích nghi tới tổng quát hoá

1 Vấn đề

Giải pháp

Giả định

Giá phải trả

có mẫu miền đích (dù không nhãn), và chỉ cần chạy tốt ở đúng miền đó.

thích nghi miền (*domain adaptation*): căn chỉnh phân phối đặc trưng giữa hai miền, hoặc tự huấn luyện bằng nhãn giả trên miền đích.

hai miền chia sẻ cùng quan hệ $p(y | x)$, tức đây là covariate shift chứ không phải concept drift.

mô hình bị chuyên biệt hoá cho một phân phối cụ thể; miền thứ ba xuất hiện thì làm lại từ đầu.

2 Vấn đề

Giải pháp

chưa từng thấy miền đích và không biết nó sẽ là gì.

tổng quát hoá miền (*domain generalization*): học biểu diễn bất biến qua nhiều miền huấn luyện, với hy vọng cái bất biến đó cũng đúng ở miền chưa thấy [93].

- Giá phải trả** phải hy sinh hiệu năng trung bình để lấy độ bền — một biểu diễn chỉ giữ phần bất biến thì bỏ đi thông tin có ích ở miền hiện tại.
- Giả định** tồn tại một cơ chế nhân quả bất biến và bạn có đủ miền huấn luyện đa dạng để nhận ra nó.
- Vấn đề mới** giả định đó có được thoả trên dữ liệu thật không?
- Hệ quả** đây là chỗ cần trung thực: một khảo sát có kiểm soát cho thấy khi mọi phương pháp tổng quát hoá miền được tune công bằng với cùng ngân sách, không phương pháp nào vượt được ERM thuần một cách nhất quán [39]. Kết quả này không nói “đừng làm DG”; nó nói rằng phần lớn cải tiến báo cáo được đến từ quy trình chọn mô hình, không từ thuật toán. Đây đúng là *adaptive overfitting* ở B/07, lần này xuất hiện ở cấp độ cả một dòng nghiên cứu.
- 3 Vấn đề** cái ta thật sự quan tâm là nhóm tẻ nhất, không phải trung bình.
- Giải pháp** tối ưu trực tiếp cho nhóm tẻ nhất thay vì cho trung bình [92].
- Giả định** bạn biết trước các nhóm là gì — nghĩa là quay lại đúng vấn đề metadata ở B/07: không có nhãn nhóm thì không có phương pháp nhóm nào dùng được.

Đằng sau cả bốn bước là một nguyên nhân chung đã nêu ở hộp trực giác: *shortcut learning* [60]. Mô hình không cố hiểu bài toán; nó tìm đặc trưng dễ nhất tương quan đủ mạnh với nhãn trong tập huấn luyện. Nếu ảnh của lớp “bò” toàn chụp trên đồng cỏ thì cỏ là đặc trưng rẻ hơn hình dạng con bò rất nhiều; nếu ảnh bệnh lý đến từ một máy chụp và ảnh bình thường từ máy khác, thì dấu hiệu của máy là đặc trưng rẻ nhất trong toàn bộ ảnh. Đây là lời giải thích thống nhất cho hiện tượng “chính xác trên test, ngu ngốc ngoài đời”, và nó nổi ngược về B/05: shortcut hầu như luôn bắt nguồn từ một thiên lệch chọn mẫu ở khâu thu thập, nên nó không phải lỗi của mô hình mà là lỗi được cài vào từ trước khi mô hình tồn tại.

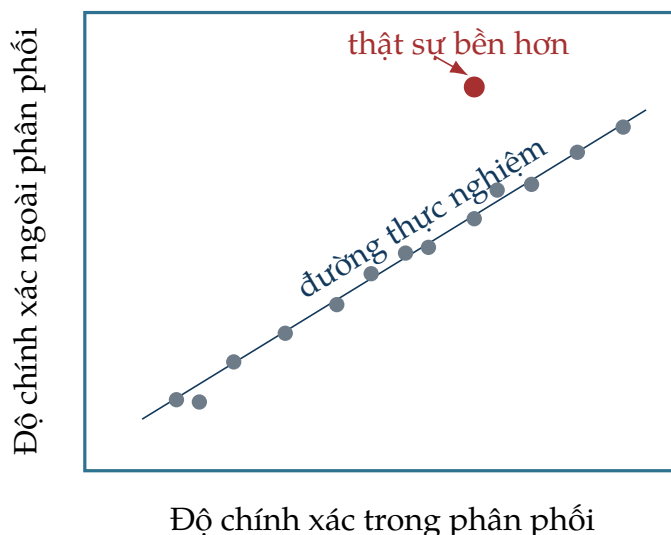
→ **hệ quả của B/05** — thiên lệch sinh ở khâu thu thập thì không hàm mất mát nào sửa được ở khâu cuối — shortcut chỉ là cách nó lộ ra.

8.5 Đo độ bền và quyền nói “tôi không biết” (Robustness and OOD Detection)

A. Benchmark corruption, và đường thẳng làm chuẩn nghiêm khắc

Nếu độ bền quan trọng thì nó phải được đo, không phải được hy vọng. Cách rẻ nhất là các bộ benchmark corruption tổng hợp: làm nhiễu, mờ, nén, đổi thời tiết ở nhiều mức nghiêm trọng rồi đọc metric theo từng loại và từng mức [37]. Giá trị thật của chúng nằm ở *cấu trúc* chứ không ở con số tổng — loại corruption làm sập mạnh nhất chính là loại phá huỷ đặc trưng mô hình đang dùng. Nếu mờ chuyển động làm sập trong khi đổi màu không ảnh hưởng, mô hình đang sống nhờ cấu trúc tần số cao, và đó là thông tin hành động được.

Giới hạn thì cũng rõ: corruption tổng hợp không thay thế được dịch chuyển thật. Vì thế các tập dữ liệu như WILDS được xây từ dịch chuyển xảy ra tự nhiên trong ứng dụng: bệnh viện khác, năm khác, vùng địa lý khác [38]. Và có một quan sát thực nghiệm hữu ích, hơi bất ngờ, được vẽ ở Hình 8.2: trên nhiều họ dịch chuyển tự nhiên, độ chính xác ngoài phân phối gần như là hàm tuyến tính của độ chính xác trong phân phối [18], [94].



Hình 8.2. “Accuracy on the line” (sơ đồ minh họa theo mô tả trong [94], không phải số đo lại). Hầu hết mô hình và hầu hết “phương pháp tăng độ bền” chỉ trượt dọc theo một đường thẳng: cải thiện ngoài phân phối đến hoàn toàn từ cải thiện trong phân phối. Chỉ điểm đỏ — nằm *trên* đường — mới xứng đáng gọi là tăng độ bền. Đường thẳng này vì thế là một *baseline* nghiêm khắc mà mọi tuyên bố về độ bền phải vượt qua.

B. Cho mô hình quyền nói “tôi không biết”

↔ **ràng buộc bởi E/24** — đường thẳng đó là chuẩn so sánh: một ablation về độ bền chỉ có nghĩa khi nó tách được điểm ra khỏi đường.

Hướng bổ sung, không thay thế: **phát hiện ngoài phân phối** (*out-of-distribution detection*) nhằm nhận ra đầu vào nằm ngoài phân phối huấn luyện để hệ thống chuyển sang chế độ an toàn thay vì trả lời bừa. Ba điều cần biết, theo thứ tự quan trọng:

- **Đường cơ sở đơn giản đến bất ngờ** — dùng chính giá trị softmax cực đại làm điểm số [95]. Đây vừa là tin tốt (rẻ, không cần train lại) vừa là cảnh báo: nó cho thấy phần lớn công trình sau đó chỉ cải thiện tiệm tiến trên một tín hiệu vốn yếu.
- **Giới hạn thực nghiệm** — độ tin cậy của mạng sâu vốn đã kém hiệu chuẩn ngay trong phân phối, và nó xuống cấp thêm đúng dưới dịch chuyển, tức đúng lúc ta cần nó nhất [96].
- **Giới hạn nguyên tắc** — không thể có bộ phát hiện OOD phổ quát: “ngoài phân phối” chỉ có nghĩa khi đã cố định một phân phối tham chiếu, nên mọi bộ phát hiện đều là bộ phát hiện của *một loại* bất thường mà bạn đã ngầm chọn.

8.6 Đuôi dài và vòng phản hồi (Long-Tail and Feedback Loops)

Hai hiện tượng cuối không phải dịch chuyển theo nghĩa thời gian, nhưng chúng phá cùng một giả định.

- **Long-tail khác mất cân bằng lớp ở cơ chế, không chỉ ở mức độ.**
 - **Mất cân bằng lớp** — vài lớp có ít mẫu hơn hẳn; tăng trọng số hay lấy mẫu lại giúp được, vì vấn đề nằm ở hàm mất mát.
 - **Đuôi dài** — *rất nhiều* lớp, mỗi lớp cực hiếm; vấn đề không còn là trọng số mà là không đủ mẫu để học biểu diễn. Tăng trọng số cho một lớp có bốn mẫu chỉ làm mô hình thuộc lòng bốn mẫu đó nhanh hơn.

- **Hướng đi có cơ sở** — tách hai việc vốn bị gộp: học biểu diễn trên toàn bộ dữ liệu (phần đầu phân phối cung cấp tín hiệu), rồi học riêng bộ phân loại với dữ liệu cân bằng lại; hoặc đổi hẳn bài toán sang truy hồi theo độ tương đồng, nơi thêm một lớp mới chỉ là thêm một mẫu tham chiếu.
- **vòng phản hồi (*feedback loop*) là thứ nguy hiểm nhất chương, vì nó vô hình với mọi phép đo ngoại tuyến.**
 - **Cơ chế** — hệ thống quyết định cái gì được nhìn thấy: ảnh nào tới tay người kiểm duyệt, vùng nào được quét kỹ, mẫu nào được gửi đi gắn nhãn. Dữ liệu thu về sau đó đã bị chính mô hình lọc.
 - **Vì sao không đo được** — mô hình tiếp theo học trên dữ liệu ấy sẽ củng cố đúng thiên lệch của mô hình trước. Mọi metric ngoại tuyến vẫn xác nhận nó tốt lên, vì tập đánh giá cũng đi qua cùng cái phễu. Đây là một dạng cụ thể của khoản nợ kỹ thuật ẩn trong hệ thống học máy [43].
 - **Cách phá vòng** — cố tình giữ một luồng dữ liệu không bị mô hình lọc: một tỉ lệ nhỏ mẫu chọn ngẫu nhiên, thu thập bất kể mô hình nói gì.

Từ đó suy ra hành động chủ động cuối cùng, và cũng là hành động rẻ nhất trong cả chương: *tự xây một bộ test đối kháng*. Không phải đối kháng theo nghĩa nhiều tối ưu hoá. Ý ở đây là vài trăm ảnh được chọn có chủ ý để lộ đúng shortcut bạn nghi ngờ: cùng vật thể trên nền khác, cùng cảnh chụp bằng camera khác, đúng lớp hiếm ở điều kiện xấu. Vài trăm ảnh như vậy thường nói nhiều hơn cả một bộ benchmark corruption, vì chúng nhắm vào giả thuyết cụ thể của bạn thay vì rải đều.

↔ **đối lập với B/07** — tập test chuẩn được thiết kế để *đại diện*; bộ test đối kháng được thiết kế để *phá*, nên hai thứ trả lời hai câu hỏi khác nhau.

8.7 Giới hạn và trường hợp hỏng

Cách phân loại ba dịch chuyển ở đầu chương gọn gàng hơn thực tế: dịch chuyển thật hầu như luôn là hỗn hợp của cả ba, và không có phép kiểm định nào tách chúng ra sạch sẽ khi bạn không có nhãn ở miền đích. Thứ tự chẩn đoán thực dụng là bắt đầu bằng cái rẻ nhất — kiểm tra tiên nghiệm lớp có đổi không, vì nếu có thì hiệu chỉnh gần như miễn phí — rồi mới tới các giả thuyết đắt hơn.

Hiệu chỉnh theo tiên nghiệm giả định $p(x | y)$ bất biến, một giả định thường bị vi phạm âm thầm: khi tỉ lệ lớp đổi vì camera mới, thì ảnh của mỗi lớp cũng đã đổi cùng lúc, và bạn có cả label shift lẫn covariate shift chồng lên nhau. Reweighting theo tỉ số mật độ chỉ hoạt động khi vùng mà phân phối nguồn thật sự có mẫu của phân phối đích nằm trong giá đỡ của phân phối nguồn; với miền thật sự mới, tỉ số mật độ tiến ra vô cực ở đúng vùng bạn quan tâm và phương sai của ước lượng nổ tung.

CẠM BÃY

Cạm bẫy hay gặp nhất là dùng chính độ tin cậy của mô hình làm cảnh báo dịch chuyển. Nghe hợp lý — mô hình gặp thứ lạ thì phải kém tự tin — nhưng đó chính là điều mạng sâu *không* làm: dưới dịch chuyển, nó thường vẫn tự tin và tự tin sai [96]. Một hệ thống giám sát chỉ báo động khi confidence tụt sẽ im lặng đúng trong những kịch bản hỏng nguy hiểm nhất.

CẦN NHỚ

“Bị domain shift” không phải một chẩn đoán. Chẩn đoán là trả lời được $p(x)$ đổi, $p(y)$ đổi, hay $p(y | x)$ đổi. Ba câu trả lời đó dẫn tới ba hành động loại trừ nhau, và chỉ cái cuối cùng bắt buộc phải gán nhãn lại. Muốn chẩn đoán được thì phải có một dòng nhỏ dữ liệu *có nhãn* từ miền đích; không có nó thì mọi thuật toán thích nghi đều là đoán mò. [T2]

8.8 Thực hành

Repo và tài nguyên. [bethgelab/imagecorruptions](#) cùng ImageNet-C và ImageNet-R cho corruption tổng hợp; [RobustBench/robustbench](#) để so với các *baseline* đã được đo lại cẩn thận; [p-lambda/wilds](#) cho dịch chuyển thật thay vì nhân tạo; [Jingkang50/OpenOOD](#) và [kkirchheim/pytorch-ood](#) cho OOD detection; [evidentlyai/evidently](#) và [SeldonIO/alibi-detect](#) để giám sát trôi phân phối trong production. Danh sách chốt tại tháng 9/2026 và tên phương pháp dẫn đầu sẽ cũ trong 6–12 tháng; các tập dữ liệu (WILDS, ImageNet-C) và hạ tầng giám sát bền hơn nhiều so với bảng xếp hạng.

Câu hỏi kiểm tra hiểu. Accuracy trên production tụt 8%: ba phép đo nào phân biệt được covariate shift với concept drift, và phép nào rẻ nhất nên làm trước? Vì sao tăng trọng số cho lớp hiếm không giải quyết được đuôi dài? Vì sao một bộ phát hiện OOD “phổ quát” là khái niệm không nhất quán?

Bài tập phụ. Đo accuracy trên 15 loại corruption ở 5 mức nghiêm trọng, vẽ bản đồ nhiệt, tìm loại làm sập mạnh nhất, rồi giải thích nó bằng ngôn ngữ của chuỗi tạo ảnh ở [A/02-vat-ly-tao-anh](#): corruption đó phá huỷ đặc trưng nào, và điều đó nói gì về thứ mô hình đang thật sự dùng?

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Giả định IID	Mỗi mẫu được rút độc lập từ cùng một phân phối, cho cả dữ liệu huấn luyện lẫn dữ liệu tương lai. Mọi bảo đảm lý thuyết của học máy đứng trên nó, nên khi nó vỡ thì các bảo đảm mất hiệu lực chứ không chỉ yếu đi.
••• Covariate shift	$p(x)$ đổi nhưng quan hệ ảnh–nhân giữ nguyên: ảnh trông khác, quy luật vẫn đúng. Chữa bằng thích nghi miền, augmentation, hoặc đánh trọng số lại theo mật độ.
••• Label shift	Tỉ lệ các lớp đổi trong khi ảnh của mỗi lớp vẫn y hệt. Đây là trường hợp duy nhất chữa được gần như miễn phí, bằng cách chỉnh lại tiên nghiệm ở đầu ra.
••• Concept drift	Chính $p(y x)$ đổi: cùng một ảnh nay phải gán nhãn khác. Không có đường tắt — phải gán nhãn lại rồi huấn luyện lại.
•• Vì sao label shift chữa được mà không cần huấn luyện lại?	Vì $p(x y)$ không đổi nên tỉ số độ hợp lý giữ nguyên, chỉ phân tiên nghiệm sai. Nhân tỉ số cực của đầu ra với tỉ lệ giữa hai tiên nghiệm là xong.
•• Mô hình cho 0,60 nhưng tỉ lệ dương thật chỉ 5% — xác suất đúng?	Khoảng 0,073. Tỉ số cực 1,5 nhân với 0,0526 ra 0,079, đổi ngược về xác suất là 7,3%. Một dự đoán “khá chắc” hoá ra chỉ là 7%.
••• Bộ phân biệt miền	Một bộ phân loại nhị phân tách “mẫu train” khỏi “mẫu production”. AUC quanh 0,5 nghĩa là chưa có dịch chuyển đáng kể; AUC 0,95 nghĩa là nhìn ảnh cũng biết nó đến từ đâu.
••• Vì sao phát hiện được dịch chuyển vẫn chưa đủ?	Vì nó trả lời “có đổi không”, không trả lời “có hại không”. Đổi tông màu toàn cục là thay đổi lớn mà vô hại; một thay đổi bé đúng vào đặc trưng mô hình dựa vào lại đủ làm sập nó.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
•• Thích nghi miền	Căn chỉnh phân phối đặc trưng, hoặc tự huấn luyện bằng nhân giả, khi bạn đã có mẫu từ miền đích. Nó chuyên biệt hoá mô hình cho một phân phối, nên miền thứ ba xuất hiện thì phải làm lại.
•• Tổng quát hoá miền	Học biểu diễn bất biến qua nhiều miền huấn luyện, dùng khi chưa từng thấy miền đích. Giá phải trả là hy sinh hiệu năng trung bình để lấy độ bền.
•• Vì sao tổng quát hoá miền không thắng nổi ERM?	Vì phần lớn cải tiến báo cáo được đến từ quy trình chọn mô hình, không từ thuật toán. Cho mọi phương pháp cùng ngân sách tune và cùng thủ tục chọn thì lợi thế biến mất.
••• Shortcut learning	Mô hình học đặc trưng dễ nhất mà tương quan đủ mạnh với nhãn: cổ thay cho hình dạng con bò, dấu hiệu máy chụp thay cho bệnh lý. Đây là lời giải thích thống nhất cho “chính xác trên test, ngu ngốc ngoài đời”.
•• Accuracy on the line	Quan sát rằng trên nhiều dịch chuyển tự nhiên, độ chính xác ngoài phân phối gần như tuyến tính theo độ chính xác trong phân phối. Dùng nó làm chuẩn: chỉ điểm nằm trên đường mới đáng gọi là tăng độ bền.
•• Benchmark corruption	Đo lại metric qua nhiều loại nhiễu, mờ, nén, thời tiết ở nhiều mức. Giá trị nằm ở bản đồ nhiệt theo từng loại, vì loại làm sập mạnh nhất chính là loại phá đặc trưng mô hình đang dùng.
••• OOD detection	Nhận ra đầu vào nằm ngoài phân phối huấn luyện để hệ thống nói “tôi không chắc”. Không thể có bộ phát hiện phổ quát, vì “ngoài phân phối” chỉ có nghĩa khi đã cố định một phân phối tham chiếu.
••• Vì sao độ tin cậy của mô hình là cảnh báo tệ?	Vì mạng sâu vốn đã quá tự tin ngay trong phân phối, và xuống cấp thêm đúng dưới dịch chuyển. Hệ giám sát chỉ báo động khi độ tin cậy tụt sẽ im lặng đúng lúc nguy hiểm nhất.
••• Đuôi dài	Rất nhiều lớp mà mỗi lớp chỉ có dăm bảy mẫu. Khác mất cân bằng lớp ở chỗ nút thắt là thiếu dữ liệu để học biểu diễn, nên tăng trọng số chỉ làm mô hình thuộc lòng vài mẫu đó nhanh hơn.
•• Vòng phản hồi	Mô hình quyết định cái gì được nhìn thấy, nên dữ liệu thu về sau đó đã bị chính nó lọc. Phá vòng bằng cách giữ một tỉ lệ nhỏ mẫu chọn ngẫu nhiên, thu bắt kể mô hình nói gì.
•• Accuracy production tụt 8% — ba phép đo nào, làm cái nào trước?	Đo tỉ lệ lớp, đo AUC của bộ phân loại phân biệt miền, và gán tay vài chục mẫu để xem nhãn cũ còn đúng không. Làm phép đầu trước vì nếu là label shift thì chữa gần như miễn phí.
•• Mọi metric ngoại tuyến đều tốt lên mà người dùng vẫn phàn nàn — vì sao?	Vòng phản hồi: tập đánh giá đi qua đúng cái phễu mà mô hình đã lọc. Phân biệt bằng một tập giữ riêng thu ngẫu nhiên, không qua mô hình; nếu con số ở đó không nhích thì vòng lặp đã khoá.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
WILDS (p-lambda/wilds)	Đóng gói dịch chuyển tự nhiên theo bệnh viện, theo năm, theo vùng, và yêu cầu báo cáo hiệu năng nhóm tẻ nhất thay vì trung bình	Bằng chứng rằng phân biệt các loại dịch chuyển đã thành chuẩn thiết kế benchmark
ImageNet-C, ImageNet-R, RobustBench	Đo độ bền theo từng loại corruption và từng mức, đúng ý “cấu trúc quan trọng hơn con số tổng”	Hạ tầng đánh giá; RobustBench tồn tại vì các báo cáo tự đo không tái lập được

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenOOD	So các phương pháp phát hiện ngoài phân phối trên cùng giao thức, kể cả <i>baseline</i> softmax cực đại	Cho thấy khoảng cách giữa baseline đơn giản và phương pháp mới hẹp hơn ấn tượng từ từng bài báo
evidentlyai/evidently, SeldonIO/alibi-detect	Giám sát production bằng đúng hai công cụ của chương: khoảng cách phân phối trên embedding và bộ phân loại phân biệt miền	Hạ tầng vận hành, ít phụ thuộc mô hình cụ thể
Mô hình nền tảng huấn luyện trên dữ liệu web quy mô lớn (dòng CLIP, DINOv2/v3)	Được cho là nhắc hẩn đường “accuracy on the line” nhờ đa dạng dữ liệu, chứ không nhờ một thuật toán bền hơn	Đây là cách đọc phổ biến tính tới 9/2026, không phải một kết luận đã khép lại

MINI PROJECT SLAM · Chẩn đoán loại dịch chuyển khi đổi tập dữ liệu · 1–2 ngày

Mục tiêu	lấy đúng bộ phân loại “cùng chỗ” đã dựng ở chương trước, chạy nó trên một tập dữ liệu khác, rồi trả lời bằng bằng chứng số: mức sụt là covariate shift, label shift, hay concept drift.
Dữ liệu	EuRoC làm miền nguồn; TUM-VI (chuỗi room hoặc corridor) hoặc OpenLORIS làm miền đích. Sinh nhân cùng chỗ ở miền đích theo <i>đúng</i> quy tắc hình học đã dùng ở B/07-chia-du-lieu-va-danh-gia , không đổi ngưỡng.
Các bước	bốn phép đo, theo thứ tự từ rẻ tới đắt. (1) Đo metric ở cả hai miền, ghi mức sụt. (2) Huấn luyện một bộ phân loại phân biệt miền trên embedding của hai tập và ghi AUC của nó. (3) Đo tỉ lệ cặp dương ở hai miền, áp hiệu chỉnh tiên nghiệm, ghi phần hiệu năng lấy lại được. (4) Gán tay 50 cặp ở miền đích và đếm tỉ lệ bất đồng giữa nhãn hình học và nhãn tay.
Xong khi	bạn kết luận được một loại dịch chuyển kèm ba con số chống lưng: AUC của bộ phân biệt miền, phần trăm hiệu năng lấy lại nhờ hiệu chỉnh tiên nghiệm, và tỉ lệ bất đồng ở bước (4). Tỉ lệ bất đồng cao chính là bằng chứng của concept drift, vì định nghĩa “cùng chỗ” đã đổi theo cấu trúc cảnh.
Nối với	B/07-chia-du-lieu-va-danh-gia cho bộ đo và cách chia; E/25-do-tin-cay-va-van-hanh vì nếu kết luận là concept drift thì bước kế tiếp là thiết kế chế độ hồng an toàn chứ không phải tinh chỉnh thêm.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Ba câu hỏi: tỉ lệ lớp có đổi không; ảnh có trông khác không; cùng một ảnh nay có phải gán nhãn khác không. Câu đầu — label shift — chưa được gần như miễn phí bằng hiệu chỉnh tiên nghiệm ở đầu ra, nên phải kiểm tra trước tiên vì nó rẻ nhất.
- A2. Vì nó học nền chứ không học con vật: bò gần như luôn đứng trên cỏ, lạc đà trên cát. Đặc trưng nền rẻ hơn hình dạng rất nhiều và tương quan đủ mạnh với nhãn trong tập huấn luyện, nên ERM chọn đúng nó.
- A3. Vì phần lớn cải tiến báo cáo được đến từ quy trình chọn mô hình — mỗi bài chọn siêu tham số theo cách thuận lợi cho phương pháp của mình. Cho mọi phương pháp cùng ngân sách tune và cùng thủ tục chọn thì lợi thế đó biến mất [39].
- A4. Vì mạng sâu vốn đã quá tự tin ngay trong phân phối, và độ tin cậy xuống cấp thêm đúng dưới dịch chuyển. Một hệ giám sát chỉ báo động khi confidence tụt sẽ im lặng đúng trong những kịch bản hồng nguy hiểm nhất.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn bán hàng ở khu phố A suốt ba năm, quen tới mức nhìn khách bước vào là đoán được họ sẽ mua gì. Rồi bạn chuyển sang khu phố B và tỉ lệ đoán đúng tụt hẳn. Chỉ có đúng ba lý do, và ba cách chữa khác hẳn nhau.

Một, khách trông khác đi — ăn mặc khác, đến vào giờ khác — nhưng quy luật cũ vẫn đúng: ai vội thì mua nhanh. Cái này chỉ cần thời gian làm quen với kiểu khách mới. Hai, tỉ lệ khách đổi: khu cũ toàn dân văn phòng, khu mới toàn sinh viên. Quy luật không sai một chút nào, chỉ là bạn đang kỳ vọng sai tỉ lệ, nên chỉnh lại kỳ vọng là xong và gần như không tốn gì. Ba, nghĩa của dấu hiệu đã đổi: ở khu cũ, khách đứng lâu trước quầy nghĩa là sắp mua; ở khu mới, đứng lâu nghĩa là đang đợi bạn. Chỉ trường hợp thứ ba mới bắt bạn quan sát lại từ đầu.

Mẹo là đừng gộp ba thứ đó thành một câu than “khu này khó bán”. Hỏi lần lượt: khách có khác không, tỉ lệ có khác không, dấu hiệu có còn nghĩa cũ không — và hỏi cái rõ nhất trước, vì cái rõ nhất cũng là cái chữa nhanh nhất.

Cái giá là bạn cần vài chục đơn hàng thật ở khu mới mới trả lời được ba câu hỏi ấy. Không có chúng thì bạn chỉ đang đoán, và đoán một cách rất tự tin.

Chỗ không rút gọn được: vì sao máy lại bám vào một dấu hiệu ăn may thay vì dấu hiệu đúng thì không có cách nói gọn nào không gây hiểu lầm. Nói “máy lười” là sai: nó chọn đúng thứ gắn chặt nhất với đáp án trong đồng dữ liệu nó được xem, và nếu chỉ nhìn đồng đó thì không ai — kể cả người — phân biệt được đâu là dấu hiệu thật.

Một câu: Sự hiệu năng không phải một chẩn đoán; chẩn đoán là nói được cái gì đã đổi, vì mỗi thứ đổi có đúng một cách chữa.

8.9 Kết nối

Chương này đóng lại Phần B ở phía dữ liệu bằng cách gỡ bỏ giả định mà [B/04-nen-tang-hoc-tu-du-lieu](#) phải giả sử và [B/07-chia-du-lieu-va-danh-gia](#) phải dựa vào. Nó nối ngược về [B/05-thiet-ke-bai-toan](#) qua khái niệm shortcut, vốn chỉ là thiên lệch chọn mẫu nhìn từ phía mô hình, và về [B/06-chat-luong-du-lieu](#) theo quan hệ đối lập: ở đó nhấn sai vì con người, ở đây nhấn đúng nhưng thế giới đã đổi. Nó là tiền đề của [B/09-thiet-ke-ham-mat-mat](#), nơi một số dịch chuyển được xử lý ngay trong hàm mất mát, và của [E/25-do-tin-cay-va-van-hanh](#), nơi OOD detection và calibration trở thành một phần của thiết kế chế độ hồng an toàn.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một hệ thống có thể có dịch chuyển phân phối rất lớn theo phép đo mà hiệu năng không giảm chút nào, và ngược lại?
2. Bạn đo được tỉ lệ lớp dương đã đổi từ 50% xuống 5%. Vì sao đây là tin tốt, và phép sửa cụ thể gồm mấy bước?
3. Tổng quát hoá miền phải hy sinh cái gì để lấy độ bền, và kết quả của [39] buộc bạn nghi ngờ điều gì trong chính thí nghiệm của mình?
4. Trong Hình 8.2, vì sao một điểm nằm *trên* đường thẳng mới đáng gọi là tăng độ bền, còn một điểm nằm xa về bên phải thì không?
5. Vì sao dùng confidence của mô hình làm cảnh báo dịch chuyển lại im lặng đúng lúc nguy hiểm nhất?
6. Đuôi dài và mất cân bằng lớp khác nhau ở cơ chế nào, và vì sao sự khác nhau đó khiến reweighting thất bại?
7. Một vòng phản hồi làm mọi metric ngoại tuyến tốt lên trong khi hệ thống tụt đi. Cơ chế nào cho phép điều đó, và biện pháp rõ nhất để phá vòng là gì?

Thiết kế hàm mất mát (Loss Function Design)

PHỤ THUỘC B/04-nen-tang-hoc-tu-du-lieu (ERM và *inductive bias* — loss là một trong bốn chỗ cài giả định); B/05-thiet-ke-bai-toan (đầu ra và tín hiệu giám sát); A/01-toan-toi-thieu/03-xac-suat-va-thong-tin (KL, entropy, ước lượng độ hợp lý cực đại).

TÓM TẮT

Chương này nói về khoảng cách giữa ba thứ hay bị gọi chung là “mục tiêu”: điều bạn thật sự muốn, phép đo bạn báo cáo, và hàm khả vi bạn thực sự tối ưu. Nó cho một nguyên lý duy nhất sinh ra hầu hết công thức loss: mọi loss là một phát biểu về phân phối. Các họ loss còn lại ra đời vì phát biểu mặc định đó không khớp với một thực tế nào đó. Đọc xong, bạn chọn được loss bằng cách trả lời một câu hỏi về dữ liệu thay vì tra bảng công thức. Bạn chẩn đoán được “loss giảm mà kết quả không tốt lên”. Và bạn nhìn ra chỗ một loss bị *phẳng*, nơi mô hình mù hoàn toàn, trước khi nó kịp gây hại. Nếu chỉ nhớ một điều: loss không phải công thức mà là một phát biểu về nhiễu, về mẫu nào đáng được chú ý, và về loại sai lầm nào đắt hơn; nếu bạn không nói được phát biểu đó thành một câu tiếng Việt thì bạn mới sao chép loss chứ chưa chọn nó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình và tham số** — “mô hình” ở đây là một hàm số có hàng triệu con số điều chỉnh được gọi là tham số (*parameter*, hay trọng số); huấn luyện nghĩa là tìm bộ con số đó. Siêu tham số (*hyperparameter*) thì ngược lại: bạn đặt bằng tay trước khi huấn luyện và thuật toán không tự sửa.
- **gradient và hạ gradient** — gradient là vector đạo hàm của một hàm theo mọi tham số, tức hướng làm hàm đó tăng nhanh nhất. Huấn luyện là lặp đi lặp lại việc đi ngược hướng gradient một bước nhỏ. Hệ quả quan trọng cho cả chương: nơi nào hàm mất mát *phẳng*, gradient bằng 0 và mô hình không nhận được tín hiệu học nào từ chỗ đó.
- **logit và softmax** — logit là điểm số thô mà mạng xuất ra cho mỗi lớp, có thể âm hoặc lớn tùy ý. Softmax là phép biến đổi exp rồi chuẩn hoá, đưa các logit về một bộ số dương cộng lại bằng 1 để đọc được như xác suất.
- **nhãn one-hot** — cách mã hoá nhãn đúng thành một vector toàn số 0 trừ một số 1 ở vị trí lớp đúng. Nó phát biểu “lớp này đúng tuyệt đối, mọi lớp khác sai tuyệt đối” — một phát biểu mạnh hơn thực tế, và vài kỹ thuật trong chương ra đời chính vì điều đó.
- **độ hợp lý (likelihood)** — với một mô hình xác suất và một tập dữ liệu, độ hợp lý là xác suất mà mô hình gán cho chính dữ liệu đã quan sát. “Cực đại hoá độ hợp lý” nghĩa là chọn tham số làm dữ liệu thật trở nên khả dĩ nhất có thể.
- **metric so với hàm mất mát** — metric là con số bạn *báo cáo* (accuracy, mAP, IoU) — thường không khả vi nên không dùng để học được. Hàm mất mát là con số bạn *tối ưu*. Toàn bộ chương này nói về khoảng cách giữa hai thứ đó.
- **IoU và Dice** — IoU (*intersection over union*) là diện tích phần giao chia cho diện tích phần hợp của hai vùng — thước đo chuẩn cho hộp và mặt nạ. Dice là biến thể lấy hai lần phần giao chia cho tổng hai diện tích; cùng tinh thần, khác mẫu số.
- **hiệu chỉnh xác suất (calibration)** — mức độ mà xác suất mô hình xuất ra khớp với tần suất đúng thật sự: nếu mô hình nói “chắc 80%” trên một nghìn mẫu thì khoảng tám trăm mẫu phải đúng. Một mô hình phân loại chính xác vẫn có thể hiệu chỉnh rất tệ, và điều đó quan trọng khi có ngưỡng ra quyết định ở hạ nguồn.

CÂU HỎI MỞ ĐẦU

- Q1. Hai mô hình có cùng giá trị hàm mất mát trên cùng một tập, vì sao một cái dùng được để ra quyết định có ngưỡng còn cái kia thì không?
- Q2. Nếu bạn chưa bao giờ ngồi chọn giả định về phân phối của nhiễu, vì sao chỉ cần gỡ MSE Loss là bạn đã chọn xong rồi?
- Q3. Vì sao một mô hình phân vùng đạt độ chính xác pixel 99% lại có thể chưa từng đoán đúng một pixel tiền cảnh nào?
- Q4. Hai hộp dự đoán cùng trượt khỏi hộp thật, một cái trượt sát và một cái trượt rất xa. Vì sao thước đo chuẩn của bài toán không phân biệt được chúng, và mô hình học được gì từ chỗ đó?
- Q5. Vì sao cổ tình bôi bẩn nhãn đúng — nói rằng lớp đúng chỉ đúng 90% — lại làm mô hình tốt lên ở một khía cạnh và tệ đi ở khía cạnh khác?

9.1 Đặt vấn đề

Mô hình tối ưu chính xác cái bạn viết ra, không phải cái bạn nghĩ trong đầu. Câu này nghe hiển nhiên tới mức nhàm, nhưng nó là nguồn của phần lớn những buổi chiều mà loss giảm đều đặn còn hệ thống thì tệ đi. Gốc rễ nằm ở chỗ có *ba* thứ khác nhau thường bị gọi chung là “mục tiêu”, và chúng nằm trên một chuỗi xấp xỉ mà mỗi bước đều rò rỉ:

- **Mục tiêu thực tế** — giảm số vụ va chạm của con robot. Hầu như không bao giờ khả vi, thường còn không đo được trong lúc huấn luyện: nó là biến cố hiếm, quan sát được sau nhiều tháng, và phụ thuộc cả những khối không phải mạng nơ-ron.
- **Metric đánh giá** — mAP trên tập test. Đo được ngay, nhưng là hàm bậc thang theo tham số: thay trọng số một lượng vô cùng nhỏ thì nó hoặc đứng yên hoặc nhảy một bậc, nên *gradient* của nó bằng 0 gần như mọi nơi và vô dụng cho việc học.
- **Hàm mất mát khả vi** — một tổ hợp *focal* cộng GloU cộng vài số hạng nữa. Trơn, có gradient khác 0 ở mọi chỗ ta cần, và ta *hy vọng* đẩy nó xuống thì kéo metric lên.

Chữ “hy vọng” ở dòng cuối là một phát biểu toán học có tên: tính **nhất quán** (*consistency*) của **hàm thay thế** (*surrogate loss*) — điều kiện để nghiệm tối ưu của hàm thay thế trùng với nghiệm tối ưu của mục tiêu gốc. Khi điều kiện đó không thành lập, bạn đang tối ưu sai thứ một cách có hệ thống, và không có lượng dữ liệu nào cứu được. Đây là chỗ mà chương này nổi thẳng về chương 4: hàm mất mát là một trong bốn chỗ cài *inductive bias*, và cũng như ba chỗ kia, giả định cài vào có thể sai.

→ **tiền đề B/04** — bốn nguồn của inductive bias — kiến trúc, hàm mất mát, dữ liệu, thuật toán tối ưu — được liệt kê ở đó; chương này khai triển nguồn thứ hai.

Câu hỏi dẫn dắt do đó không phải “có những loss nào” mà là: *làm sao viết ra đúng thứ bạn muốn, khi thứ bạn muốn thường không khả vi?*

TRỰC GIÁC

Gradient chỉ nhìn thấy *độ dốc cục bộ* của loss, nên chỗ nào loss phẳng thì chỗ đó mô hình mù hoàn toàn. Hãy đọc mọi hàm mất mát như một bản đồ độ dốc: nó quyết định mẫu nào còn “phát ra tín hiệu” và mẫu nào đã tàng hình. **Hinge loss** cố tình làm phẳng vùng đã vượt lề, nên mẫu phân đúng biến mất khỏi tầm nhìn; **entropy chéo** (*cross-entropy*) không bao giờ phẳng, nên một mẫu đã đúng với xác suất 0,99 vẫn tiếp tục kéo mô hình. Focal loss là hành động ngược lại: *cố ý* bơm vùng phẳng vào chỗ dễ để tín hiệu của mẫu khó nổi lên.

Nhìn theo bản đồ độ dốc thì ba công thức trông rất khác nhau kia trở thành ba lựa chọn của cùng một câu hỏi: “tôi muốn ai còn nói được?”

9.2 Loss là một phát biểu xác suất (Loss as a Probabilistic Statement)

A. Một nguyên lý sinh ra hầu hết công thức

Có hàng chục công thức loss trong các thư viện, và học thuộc chúng là cách tệ nhất để dùng chúng. Có một nguyên lý sinh ra gần hết: **cực đại hoá độ hợp lý** (*maximum likelihood estimation*). Giả sử bạn mô hình hoá dữ liệu bằng một phân phối có điều kiện $p_\theta(y | x)$; khi đó tối thiểu hoá $-\sum_i \log p_\theta(y_i | x_i)$ chính là ước lượng độ hợp lý cực đại, và mọi thứ còn lại chỉ là chọn họ phân phối:

Gaussian $\Rightarrow \frac{1}{2}\|y - \hat{y}\|^2$ (MSE), Laplace $\Rightarrow |y - \hat{y}|$ (MAE), Categorical $\Rightarrow -\log \hat{p}_y$ (CE).

Ý nghĩa của ba dòng trên, nói gọn: *bạn không cần nhớ công thức nào cả, chỉ cần trả lời một câu — tôi giả định phần dư phân bố thế nào?* Nếu bạn tin sai số là Gaussian thì bạn đã chọn MSE dù không biết mình chọn; nếu dữ liệu có ngoại lai nặng đuôi thì giả định Gaussian sai, và MSE để một mẫu hỏng kéo cả mô hình vì nó phạt bậc hai [14].

B. Chính quy hoá là prior, không phải mẹo

Nguyên lý này mở rộng tự nhiên. Thay *MLE* bằng **cực đại hoá hậu nghiệm** (*maximum a posteriori*) thì thêm một số hạng $-\log p(\theta)$: prior Gaussian cho phạt L2, prior Laplace cho phạt L1 và tính thừa đi kèm [15]. Điều này biến câu “thêm *weight decay* cho đỡ *overfitting*” — một câu vô nghĩa về mặt nội dung — thành một phát biểu kiểm chứng được: *tôi tin rằng trước khi nhìn dữ liệu, trọng số nhỏ khả dĩ hơn trọng số lớn*. Và như mọi niềm tin, nó có thể sai: với mạng có **chuẩn hoá** (*normalization*), phạt L2 lên trọng số mất phần lớn nghĩa hình học vì thang của trọng số bị chuẩn hoá triệt tiêu ngay sau đó.

Một hệ quả ít được nói: nếu loss là log-likelihood thì đầu ra của mô hình là *tham số của một phân phối*, không phải một con số. Đi theo hướng đó ta được *heteroscedastic loss* — mạng xuất ra cả $\hat{y}$ lẫn $\log \sigma^2$ và tối thiểu hoá $\frac{(y - \hat{y})^2}{2\sigma^2} + \frac{1}{2} \log \sigma^2$. Số hạng thứ nhất cho phép mô hình giảm phạt ở mẫu nó tự nhận là khó; số hạng thứ hai ngăn nó tuyên bố mọi thứ đều khó. Kết quả là mô hình học luôn độ bất định của chính mình, và độ bất định đó dùng được ở hạ nguồn [97]. Đó đúng là thứ một bộ lọc hợp nhất cảm biến cần, và là chỗ nối tự nhiên nhất giữa chương này với nền tảng ước lượng trạng thái bạn đã có.

C. Từ “chọn công thức” tới “phát biểu giả định”

Mạch 1 — hồi quy, đọc theo giả định chứ không theo tên

1 Vấn đề

Giải pháp

Hệ quả

có quá nhiều công thức loss để nhớ, và không có tiêu chí chọn.

coi mọi loss là $-\log p_\theta(y | x)$ và chỉ chọn họ phân phối.

câu hỏi “dùng MSE hay MAE” trở thành “phần dư của tôi có đuôi nặng không”, một câu trả lời được bằng cách vẽ biểu đồ phần dư thay vì bằng cách thử cả hai rồi chọn con số đẹp hơn.

2 Vấn đề

giả định Gaussian sai khi có ngoại lai, nhưng MAE có gradient hằng nên hội tụ chậm quanh nghiệm.

- Giải pháp** **hàm Huber** (*Huber loss*) — bậc hai trong bán kính δ , tuyến tính ngoài bán kính đó.
- Giả định** ngoại lai là hiếm và nằm xa; δ tách được hai chế độ.
- Giá phải trả** thêm một siêu tham số, và nếu δ đặt sai thì bạn được cái tệ nhất của cả hai. Người làm SLAM nhận ra ngay đây chính là hàm robust quen thuộc trong **hiệu chỉnh chùm tia** (*bundle adjustment*), và trực giác từ đó chuyển sang nguyên vẹn.
- 3 Vấn đề** đôi khi ta không cần một điểm mà cần một khoảng.
- Giải pháp** **quantile loss** — phạt bất đối xứng để mô hình học phân vị thứ τ thay vì kỳ vọng; ba đầu ra ở $\tau = 0,1; 0,5; 0,9$ cho một khoảng dự đoán mà không cần giả định Gaussian nào.
- Vấn đề mới** nếu loss là giả định phân phối thì *prior cũng vậy* → chính quy hoá thôi là mẹo và trở thành một phát biểu có thể sai, có thể kiểm chứng, và có thể gỡ ra khi sai.

9.3 Phân loại: chọn loss là chọn hành vi (Classification: Choosing Behaviour)

A. Hai bản đồ độ dốc trên cùng một bài toán

Bảng 9.1. Hinge và cross-entropy: cùng một bài toán, hai hành vi hoàn toàn khác nhau.

Tiêu chí	Hinge (multiclass SVM)	Cross-entropy (softmax)
Cơ chế	Phạt cho tới khi vượt lề Δ , sau đó gradient tắt hẳn	Phạt – log xác suất đúng, không bao giờ tắt
Giả định ngầm	Chỉ cần phân đúng với một lề an toàn là đủ	Ta cần cả xác suất, không chỉ nhãn
Đầu ra	Điểm số, không diễn giải được như xác suất	Xác suất dùng được ở hạ nguồn
Ngoại lai đã phân đúng	Bị bỏ qua hoàn toàn	Vẫn kéo mô hình
Mạnh khi nào	Cần biên quyết định ổn định, ít quan tâm độ tin cậy	Cần ngưỡng hoá, hợp nhất, hoặc chưng cất về sau
Hổng khi nào	Khi quyết định hạ nguồn cần một xác suất có nghĩa	Nhãn nhiều: mô hình bị kéo vô hạn về phía nhãn sai

Bài học tổng quát vượt ra ngoài hai công thức: *chọn loss là chọn hành vi*. Đặc tính “không bao giờ tắt” của cross-entropy là ưu điểm khi ta cần xác suất được hiệu chỉnh, và là nhược điểm chí mạng khi trong tập có 5 % nhãn sai — mô hình sẽ dành phần lớn sức chứa cuối buổi huấn luyện để ghi nhớ đúng những nhãn sai đó [19].

B. Ba cách định hình lại sự chú ý

Cross-entropy cộng đều đóng góp của mọi mẫu. Trong một bài phát hiện vật thể một giai đoạn, số *anchor* nên có thể gấp mười nghìn lần số anchor có vật; dù mỗi anchor nên chỉ đóng góp một lượng nhỏ, tổng của chúng nhấn chìm tín hiệu. Ba sửa chữa phổ biến tấn công ba chỗ khác nhau, và nhằm lẫn giữa chúng là lỗi rất hay gặp.

1 Mạch 2 — ba cơ chế khác nhau cho ba triệu chứng khác nhau

- 1 Vấn đề** mẫu dễ áp đảo về số lượng.

Giải pháp *focal loss*: nhân CE với $(1 - p_t)^\gamma$ [20].

Giả định đã dễ thì đã học xong, không cần học thêm.

Giá phải trả một siêu tham số γ , và nếu nhân nhiều thì focal *khuếch đại* nhiều, vì một mẫu có nhãn sai luôn trông y hệt một mẫu khó.
- 2 Vấn đề** lớp hiếm ít mẫu — đây là chuyện khác hẳn “mẫu dễ nhiều”.

Giải pháp *class-balanced loss* — cân theo số mẫu hiệu dụng của từng lớp.

Giả định nguyên nhân là tần suất, không phải độ khó hình ảnh.

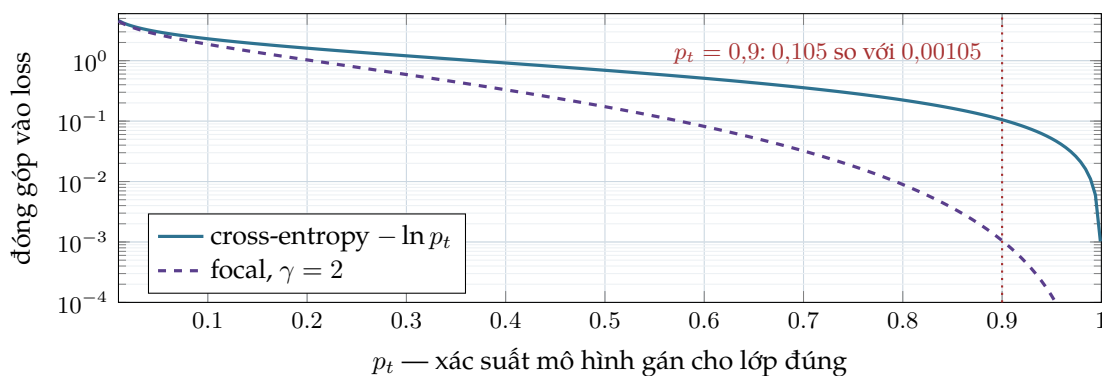
Hệ quả trên một tập đuôi dài mà mọi lớp đều dễ về mặt thị giác, focal không giúp gì còn class-balanced thì giúp; trên một tập cân bằng nhưng có vài mẫu cực khó thì ngược lại.
- 3 Vấn đề** mô hình quá tự tin, xác suất không dùng được.

Giải pháp *làm mượt nhãn* (label smoothing): thay nhãn one-hot bằng $(1 - \varepsilon)$ cho lớp đúng và $\varepsilon/(C - 1)$ cho phần còn lại [21].

Hệ quả nó đặt trần hữu hạn cho logit đúng nên chấm dứt cuộc đua logit vô hạn.

Giá phải trả nó cũng làm co cụm các lớp trong không gian đặc trưng, và điều đó *làm hại* nếu bạn định dùng chính đặc trưng đó cho truy hồi hoặc chứng cất.

Hình 9.1 vẽ ra điều mà hộp trực giác vừa nói bằng lời: nó cho thấy chính xác vùng nào của trục xác suất bị focal loss “tắt tiếng”.



Hình 9.1. Vì sao focal loss “tắt tiếng” của mẫu dễ. Ở $p_t = 0,9$, hệ số $(1 - p_t)^2 = 0,01$ chia đóng góp cho một trăm lần; ở $p_t = 0,1$ hệ số là 0,81, gần như không đổi. Tỷ lệ chú ý giữa mẫu khó và mẫu dễ do đó bị nhân lên khoảng tám mươi lần. Đó cũng là lý do một nhãn sai được khuếch đại đúng chừng ấy: với mô hình, nhãn sai trông y hệt một mẫu cực khó.

C. Chiều của KL quyết định mô hình sinh ra thứ gì

Một trục thứ tư ít được nêu nhưng quyết định hành vi của cả họ mô hình sinh: *đo khoảng cách giữa hai phân phối theo chiều nào*. $KL(p \parallel q)$ (*forward*, tức cross-entropy quen thuộc) phạt rất nặng khi q gán xác suất gần 0 vào chỗ p có khối lượng, nên nghiệm tối ưu *phủ hết các mode* và bôi trơn qua mọi khả năng. $KL(q \parallel p)$ (*reverse*) chỉ phạt khi q đặt khối lượng vào chỗ p không có, nên nghiệm tối ưu *bám lấy một mode* và bỏ qua phần còn lại. Hệ quả rất cụ thể: hướng phủ mode sinh ra thứ trung bình mờ nhạt, hướng tìm mode sinh ra thứ sắc nét nhưng thiếu đa dạng. Chương 17 dùng lại đúng trục này để giải thích vì sao VAE mờ còn GAN thì sắc mà hay sập mode.

↔ **tiền đề C/17** — bốn họ mô hình sinh khác nhau ở chỗ chúng chọn chiều nào của KL, hoặc thay KL bằng một định nghĩa “giống” khác hẳn.

9.4 Khi đầu ra có cấu trúc (Structured Outputs)

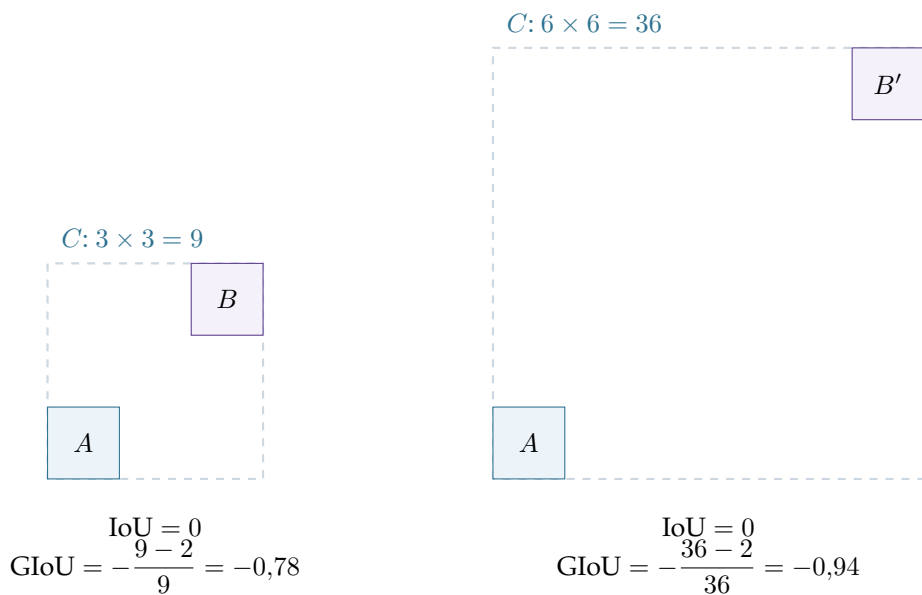
Nếu đầu ra là một mặt nạ, một hộp, một chuỗi hay một tập hợp thì trung bình theo từng phần tử là xấp xỉ tồi của thứ ta thật sự quan tâm. Nguyên tắc chung: *loss phải tôn trọng cấu trúc của đầu ra*, và cách làm điển hình là tìm một xấp xỉ khả vi cho chính metric của bài toán.

A. Phân vùng: vì sao Dice không bị đánh lừa

Với **phân vùng** (*segmentation*), CE theo từng pixel bị chi phối bởi lớp nền. Lấy ví dụ nhỏ nhất tính tay được: ảnh 100×100 với đúng 100 pixel tiền cảnh, tức 1 %. Một mô hình đoán “toàn bộ là nền” đạt độ chính xác pixel 99 % và CE thấp một cách đáng khen, trong khi hệ số **Dice** (*Dice coefficient*) $= 2|A \cap B|/(|A| + |B|) = 2 \cdot 0/(0 + 100) = 0$ — bằng không tuyệt đối [98]. Dice không bị đánh lừa vì nó chỉ đếm phần giao trên tổng diện tích hai mặt nạ, và pixel nền đúng không được cộng điểm. Cái giá là gradient của Dice phụ thuộc *toàn ảnh* chứ không tách được theo từng pixel, nên nó ồn hơn khi *batch* nhỏ và có mẫu số tiến về 0 khi ảnh không chứa tiền cảnh nào. Trong thực hành người ta hầu như luôn dùng tổ hợp CE cộng Dice, còn **Tversky** (*Tversky loss*) là bản tổng quát cho phép đặt giá khác nhau cho dương tính giả và âm tính giả — tức là chọn mặt nạ “béo” hay “gầy” ngay từ công thức, trước khi nhìn kết quả.

B. Hộp: lỗ hổng gradient của IoU và cách GIoU và nó

Với **hộp giới hạn** (*bounding box*), L1 trên bốn tọa độ không phải thứ ta đo, vì metric là IoU. Nhưng IoU có một lỗ hổng chí mạng và nó tính tay được, như Hình 9.2 cho thấy.



Hình 9.2. Hai dự đoán rời hoàn toàn khỏi hộp thật *A*. IoU bằng 0 trong cả hai trường hợp, nên loss phẳng và mô hình không biết *B* tốt hơn *B'*. GIoU trừ đi phần thừa của hộp bao nhỏ nhất *C* và nhờ vậy phân biệt được hai tình huống, cho gradient chỉ đúng hướng.

Công thức là $\text{GIoU} = \text{IoU} - (|C| - |A \cup B|)/|C|$ [99], và ý nghĩa của số hạng thứ hai bằng lời: phạt theo mức độ “lãng phí” của hộp bao nhỏ nhất chứa cả hai hộp — càng xa nhau thì hộp bao càng rộng, càng bị phạt nặng. Đây là ví dụ mẫu mực cho cả chương: người ta không phát minh GIoU vì nó đẹp, mà vì họ đi tìm đúng chỗ loss bị phẳng.

C. Chuỗi và tập hợp

Cùng một nguyên tắc sinh ra hai họ nữa. CTC xử lý chuỗi không căn chỉnh bằng cách cộng xác suất trên mọi cách căn chỉnh hợp lệ, biến một bài toán không khả vi vì thiếu nhân từng khung thành một bài toán khả vi. Và **ghép cặp Hungary** (*Hungarian matching*) xử lý đầu ra dạng tập hợp: DETR bỏ hẳn NMS bằng cách ghép một-một tối ưu giữa dự đoán và nhãn *trước* khi tính loss, khiến loss trở nên bất biến với thứ tự [47]. Cả hai minh hoạ cùng một câu. Khi metric của bài toán không có xấp xỉ khả vi sẵn, bạn phải tự thiết kế một cái. Việc thiết kế nằm ở chỗ gỡ đúng điểm không khả vi, không phải ở chỗ làm công thức phức tạp hơn.

9.5 Ba nguồn giám sát ngoài nhãn lớp (Supervision Beyond Class Labels)

Ba họ loss còn lại chia nhau một ý tưởng: tín hiệu giám sát có thể đến từ quan hệ giữa các mẫu, từ vật lý, hoặc từ một mô hình khác.

- **Từ quan hệ giữa các mẫu — *metric learning*.** Thay vì hỏi “ảnh này thuộc lớp nào”, ta hỏi “hai ảnh này có giống nhau không”, và tối ưu để cặp dương gần hơn cặp âm một lẽ nào đó [100], [101]. Hai chi tiết quyết định thành bại, và cả hai đều là chỗ hay hỏng:
 - **Mẫu âm** — nếu toàn mẫu âm dễ thì loss về 0 mà biểu diễn chẳng học được gì; nhưng khai thác mẫu âm khó quá tay thì bạn chủ yếu đang huấn luyện trên các dương giả bị gán nhầm là âm.
 - **Nhiệt độ (*temperature*)** — nó điều khiển độ tập trung của phân phối tương tự; nhiệt độ thấp biến *InfoNCE* thành “chỉ quan tâm mẫu âm khó nhất”, tức là đưa cả loss về sát chế độ hỏng vừa nói.

Đây là nền của toàn bộ tự giám sát hiện đại.

↔ **tiền đề C/18** — mọi phương pháp tự giám sát hiện đại đều là một lựa chọn cụ thể của cặp “mẫu dương là gì” và “mẫu âm lấy từ đâu”.

- **Từ vật lý — ràng buộc hình học nhúng vào mục tiêu.** Thị giác máy tính có sẵn một kho tri thức không cần học lại: **sai số tái chiếu** (*reprojection error*), **nhất quán trắc quang** (*photometric consistency*) giữa hai khung nhìn, prior trơn cho trường độ sâu. Đưa chúng vào loss là cách nói “đừng học lại hình học, tôi biết rồi”,

↔ **cùng nguyên lý với C/16** — sai số tái chiếu trong hình học đa khung nhìn và mất mát tự giám sát cho độ sâu là cùng một hàm, chỉ khác chỗ ai giữ biến nào cố định.

và đó chính là cây cầu giữa SLAM cổ điển với học sâu — lý do các phương pháp ước lượng độ sâu tự giám sát huấn luyện được mà không cần một nhãn độ sâu nào. Giả định ngầm — bề mặt Lambert, không có vật chuyển động, phơi sáng không đổi — chính là danh sách các chế độ hỏng của chúng.

- **Từ một mô hình khác. Mất mát tri giác (*perceptual loss*)** so sánh đặc trưng của một mạng đã huấn luyện thay vì so sánh pixel [69], [102]; nó chứa một hiện tượng rất cụ thể. Khi nhiều đáp án đều đúng, loss trung bình theo pixel kéo đầu ra về *trung bình của các đáp án* — mà trung bình của nhiều ảnh sắc nét là một ảnh mờ. Cùng họ, ***style loss*** qua ma trận Gram cho thấy thống kê bậc hai của đặc trưng mang thông tin kết cấu còn bậc một mang thông tin bố cục [103]. Và **chưng cất tri thức (*knowledge distillation*)** [23] khai thác đúng chỗ nhãn one-hot lãng phí: phân phối mềm của *teacher* nói rằng con mèo này hơi giống con cáo, một thông tin về cấu trúc lớp mà nhãn cứng vứt bỏ hoàn toàn.

9.6 Cân bằng nhiều thành phần (Balancing Multi-Term Losses)

Khi một loss có bốn số hạng, siêu tham số khó nhất của cả hệ thống không nằm trong paper mà nằm trong file cấu hình. Ba triệu chứng cần nhận ra:

- **Thang đo lệch nhau** — một số hạng cỡ 10^{-3} và một số hạng cỡ 10^1 , nên số hạng nhỏ chỉ là trang trí. Kiểm tra bằng cách log riêng từng thành phần và chuẩn của gradient mà nó sinh ra, vì một loss nhỏ vẫn có thể có gradient lớn.
- **Xung đột gradient** — hai số hạng đẩy trọng số về hai hướng có tích vô hướng âm, nên tiến bộ ở nhánh này là thoái bộ ở nhánh kia. Đây là bản chất của học đa nhiệm chứ không phải lỗi cài đặt, nên đừng đi tìm bug.
- **Trọng số không chuyển được** — bộ trọng số tinh chỉnh trên một tập dữ liệu thường vô nghĩa trên tập khác, vì thang của từng thành phần phụ thuộc phân bố dữ liệu.

↪ **ràng buộc bởi E/24** — hai bộ trọng số chỉ so sánh được khi mọi thứ khác trong giao thức thí nghiệm được giữ cố định — nếu không, bạn đang đo nhiễu.

Có cách đặt trọng số có nguyên lý thay vì dò tay: cho mô hình tự học một σ_k cho mỗi nhánh, rồi cân bằng theo $1/\sigma_k^2$ [97]. Nhưng hãy trung thực rằng đây là kinh nghiệm thực hành chứ không phải định lý. Trong nhiều hệ thống thật, một vòng tìm kiếm ngẫu nhiên trên hai ba trọng số vẫn thắng nó.

Hai khung liên quan đầy ý tưởng đi xa hơn:

- **DRO** — tối thiểu hoá loss của *nhóm tệ nhất* thay vì loss trung bình [92]. Đây là thứ bạn muốn khi trung bình đẹp che giấu một nhóm hỏng hoàn toàn.
- **Cost-sensitive learning** — thổi giá vờ rằng dương tính giả và âm tính giả đắt như nhau. Nếu bỏ sót một người đi bộ đất gấp nghìn lần một cảnh báo thừa, con số đó phải nằm trong loss chứ không phải trong một cuộc họp.

9.7 Thực hành

Repo và tài nguyên. [qubvel-org/segmentation_models.pytorch](https://github.com/qubvel-org/segmentation_models.pytorch) cho Dice, Tversky và focal đã cài sẵn; [KevinMusgrave/pytorch-metric-learning](https://github.com/KevinMusgrave/pytorch-metric-learning) cho toàn bộ họ *contrastive* và các chiến lược khai thác mẫu âm; [lightly-ai/lightly](https://github.com/lightly-ai/lightly) cho InfoNCE; [open-mmlab/mmdetection](https://github.com/open-mmlab/mmdetection) cho GIoU/DIoU/CIoU và Hungarian matching. Với loss nhiều thành phần, hãy đọc code chứ đừng đọc paper: trọng số thật luôn nằm trong config. Danh sách này chốt tại tháng 9/2026 và tên mô hình dẫn đầu sẽ cũ trong 6–12 tháng, trong khi các repo hạ tầng kể trên bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Vì sao một mô hình đạt hinge loss bằng 0 vẫn có thể hiệu chỉnh xác suất rất tệ? Focal loss và class-balanced loss chứa cùng một triệu chứng bằng hai cơ chế khác nhau — khác ở đâu, và tập dữ liệu nào làm lộ ra sự khác biệt? Trong một loss bốn thành phần, làm sao biết thành phần nào đang chi phối gradient?

Mini project. Trên CIFAR-10-LT, huấn luyện cùng một *backbone* với ba cấu hình: CE, CE cộng label smoothing, và focal. Báo cáo ba con số cho mỗi cấu hình — độ chính xác tổng, độ chính xác nhóm đuôi, và ECE. Ba con số kể ba câu chuyện khác nhau, và chỗ chúng mâu thuẫn nhau mới là chỗ đáng viết ra.

9.8 Giới hạn và trường hợp hỏng

Giới hạn lớn nhất là giới hạn khái niệm: loss chỉ là hàm thay thế, và không có gì bảo đảm nó nhất quán với metric, càng không bảo đảm metric nhất quán với mục tiêu thực tế. Khi khe hở đó đủ rộng, mô hình sẽ khai thác nó — đây là *Goodhart's law* phiên bản học máy: khi một phép đo trở thành mục tiêu, nó thôi là một phép đo tốt. Điều đáng sợ không phải mô hình thất bại, mà là nó *thành công* theo cách bạn không tưởng tượng nổi: một mô hình phân vùng đạt Dice cao nhờ phình mặt nạ ra vì bạn đặt Tversky phạt âm tính giả nặng gấp bốn; một mô hình phát hiện đạt mAP cao nhờ đổ ra hàng trăm hộp điểm thấp mà metric không phạt.

Ba trường hợp hỏng cụ thể đáng thuộc:

- **Loss giảm mà metric không lên** — hoặc hàm thay thế không nhất quán, hoặc bạn đang đo trên một phân bố khác với phân bố huấn luyện (chương 8). Hai nguyên nhân này cần hai hành động ngược nhau, nên phải phân biệt trước khi sửa.
- **Loss vượt rồi đột ngột thành NaN** — hầu như luôn là một thành phần có mẫu số tiến về 0: Dice trên ảnh không có tiền cảnh, chia cho σ trong loss heteroscedastic, log của xác suất bằng 0. Phải truy nguyên từ bước *đầu tiên* xuất hiện chứ không phải từ chỗ nó nổ.
- **Loss đúng công thức nhưng sai giả định** — focal trên tập nhãn nhiều, MSE trên dữ liệu đuôi nặng, contrastive với mẫu âm toàn là dương giả. Đây là loại lỗi không bao giờ lộ ra trong đường cong huấn luyện.

CẠM BẮY

Trước khi tin bất kỳ đường cong loss nào, hãy kiểm tra giá trị tại lúc khởi tạo. Với C lớp và mô hình chưa học gì, cross-entropy kỳ vọng phải bằng $\ln C$: $\ln 10 \approx 2,30$ cho CIFAR-10, $\ln 1000 \approx 6,91$ cho ImageNet. Nếu epoch 0 in ra 0,4 thì bạn đang rò rỉ nhãn hoặc đánh giá nhầm tập; nếu in ra 15 thì logit đang nổ hoặc chỉ số nhãn lệch. Phép kiểm tra này mất ba mươi giây và bắt được nhiều lỗi hơn bất kỳ thứ gì khác cùng chi phí. Kèm theo nó là hai thói quen: tắt hết chính quy hoá để nhìn riêng *data loss*, và log từng thành phần riêng thay vì chỉ log tổng.

CẦN NHỚ

Loss không phải là công thức mà là một *phát biểu*: về phân phối của nhiễu, về việc mẫu nào đáng được chú ý, và về việc loại sai lầm nào đắt hơn. Nếu bạn không nói được phát biểu đó thành một câu tiếng Việt thì bạn chưa chọn loss — bạn mới sao chép nó. Và ở chỗ nào loss phẳng, ở chỗ đó mô hình mù. [T2]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Hàm thay thế	Tiếng Anh là <i>surrogate loss</i> : hàm khả vi ta thực sự tối ưu, đứng thay cho metric vốn không khả vi. Nó chỉ hợp lệ khi nghiệm tối ưu của nó trùng nghiệm tối ưu của mục tiêu gốc; điều đó gần như không bao giờ được chứng minh trong thị giác.
••• Vì sao không tối ưu thẳng metric như mAP?	Vì metric là hàm bậc thang theo tham số: đổi trọng số một lượng vô cùng nhỏ thì nó hoặc đứng yên hoặc nhảy một bậc. Gradient của nó bằng 0 gần như mọi nơi nên vô dụng cho việc học.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
••• Cực đại hoá độ hợp lý	Chọn tham số làm dữ liệu đã quan sát trở nên khả dĩ nhất. Nó sinh ra hầu hết công thức loss: nhiều Gaussian cho MSE, Laplace cho MAE, Categorical cho entropy chéo.
•• Hàm Huber	Bậc hai trong bán kính δ , tuyến tính ngoài bán kính đó, nên một ngoại lai không kéo được cả mô hình. Giả định là ngoại lai hiếm và nằm xa; đặt δ sai thì bạn được cái tệ nhất của cả MSE lẫn MAE.
•• Heteroscedastic loss	Mạng xuất ra cả giá trị dự đoán lẫn phương sai của chính nó, và phương sai đó vào thẳng mẫu số của số hạng phạt. Kết quả là mô hình học luôn độ bất định của mình, dùng được cho khâu hợp nhất cảm biến ở hạ nguồn.
•• Vì sao hinge và cross-entropy hành xử khác nhau?	Hinge tắt gradient ngay khi mẫu vượt lề, nên mẫu đã đúng biến mất khỏi tầm nhìn. Cross-entropy không bao giờ tắt, nên nó cho xác suất hiệu chỉnh tốt hơn nhưng bị nhãn sai kéo vô hạn.
••• Focal loss	Nhân entropy chéo với $(1 - p_t)^\gamma$ để dim đóng góp của mẫu dễ. Ở $p_t = 0,9$ với $\gamma = 2$, hệ số là 0,01, tức chia đóng góp cho một trăm lần. Nó hỏng khi nhãn nhiều, vì nhãn sai trông y hệt mẫu khó.
•• Vì sao focal loss không chứa được tập đuôi dài?	Vì focal cân theo <i>độ khó của từng mẫu</i> , còn đuôi dài là vấn đề <i>tần suất của từng lớp</i> . Nếu mọi lớp đều dễ về thị giác thì focal không đổi gì; phải dùng loss cân theo số mẫu hiệu dụng.
•• Label smoothing	Thay nhãn one-hot bằng $(1 - \epsilon)$ cho lớp đúng và chia đều ϵ cho phần còn lại. Nó đặt trần hữu hạn cho logit đúng nên hiệu chỉnh xác suất tốt lên, nhưng làm các lớp co cụm lại nên hại cho truy hồi và chưng cất.
• Reverse KL	Chỉ phạt khi mô hình đặt khối lượng vào chỗ dữ liệu không có, nên nghiệm tối ưu bám lấy một mode. Hệ quả: đầu ra sắc nét mà thiếu đa dạng, ngược hẳn với forward KL vốn phủ hết mode và cho ảnh mờ.
••• Hệ số Dice	Hai lần phần giao chia cho tổng diện tích hai mặt nạ. Nó không cộng điểm cho pixel nền đúng, nên đoán “toàn nền” cho 99% độ chính xác pixel nhưng Dice bằng 0. Mẫu số tiến về 0 khi ảnh không có tiền cảnh.
••• GIoU	IoU trừ đi tỉ lệ phần rỗng của hộp bao nhỏ nhất chứa cả hai hộp. Nó vá đúng chỗ IoU bị phẳng: với hai hộp rời nhau, IoU đều bằng 0 còn GIoU cho $-0,78$ và $-0,94$, nên gradient lại chỉ đúng hướng.
•• Ghép cặp Hungary	Ghép một-một tối ưu giữa dự đoán và nhãn <i>trước</i> khi tính loss, khiến loss bất biến với thứ tự. Đây là thứ cho phép DETR bỏ hẳn bước triết phi cực đại.
•• InfoNCE	Loss đối chiếu một mẫu dương với nhiều mẫu âm trong một không gian nhúng. Hai thứ quyết định thành bại là nguồn mẫu âm và nhiệt độ; mẫu âm quá dễ thì loss về 0 mà không học được gì.
•• Vì sao loss theo pixel cho ảnh mờ?	Khi nhiều đáp án đều đúng, trung bình bình phương kéo đầu ra về <i>trung bình của các đáp án</i> , mà trung bình của nhiều ảnh sắc nét là một ảnh mờ. Mất mát tri giác và loss đối kháng ra đời để thoát khỏi chính hiện tượng này.
• Chưng cất tri thức	Huấn luyện theo phân phối mềm của một mô hình lớn thay vì theo nhãn cứng. Phân phối mềm chứa thông tin về sự tương tự giữa các lớp mà nhãn one-hot vứt bỏ hoàn toàn.
•• Vì sao trọng số của loss nhiều thành phần khó chỉnh?	Vì thang của từng số hạng phụ thuộc phân bố dữ liệu, nên bộ trọng số tinh chỉnh trên tập này thường vô nghĩa trên tập khác. Ngoài ra hai số hạng có thể xung đột gradient, và đó là bản chất của học đa nhiệm chứ không phải lỗi cài đặt.
••• Epoch 0 in ra loss 6,5 trên CIFAR-10 — nghĩa là gì?	Sai, vì giá trị kỳ vọng lúc khởi tạo phải là $\ln 10 \approx 2,30$. Cao hơn nhiều nghĩa là logit đang nổ hoặc chỉ số nhãn lệch; thấp bất thường thì ngược lại, nghi ngờ rò rỉ nhãn.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Loss giảm mượt rồi đột ngột thành NaN — nguyên nhân?	Gần như luôn là một thành phần có mẫu số tiến về 0: Dice trên ảnh không có tiền cảnh, chia cho σ trong loss heteroscedastic, hoặc log của xác suất bằng 0. Truy nguyên từ bước <i>đầu tiên</i> xuất hiện, không phải từ chỗ nó nổ.
●● Loss giảm đều mà mAP đứng yên — nguyên nhân?	Hoặc hàm thay thế không nhất quán với metric, hoặc bạn đang đo trên một phân bố khác với phân bố huấn luyện. Phân biệt bằng cách đo lại metric ngay trên chính tập huấn luyện: nếu ở đó metric cũng đứng yên thì lỗi nằm ở loss.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
mmdetection , detectron2	Focal loss cho phân loại anchor và họ GloU/DIoU/CIoU cho hồi quy hộp là mặc định của gần như mọi detector một giai đoạn	Hạ tầng, ít hết hạn; ý tưởng loss bền hơn tên kiến trúc
Dòng DETR (Deformable DETR, DINO-DETR)	Ghép cặp Hungary biến phát hiện vật thể thành bài toán dự đoán tập hợp và bỏ được triết phi cực đại	Phần được kế thừa rộng rãi là loss, không phải kiến trúc
nnU-Net	Tổ hợp entropy chéo cộng Dice là cấu hình mặc định, chính vì lý do mất cân bằng nền đã phân tích ở chương	Vẫn là đối chứng mạnh cho ảnh y sinh tính tới 9/2026
CLIP, DINOv2/DINOv3	Toàn bộ tín hiệu huấn luyện là contrastive: InfoNCE với nhiệt độ học được, không có nhân phân lớp nào	Nền của phần lớn mô hình nền tảng thị giác hiện nay
Bộ mã hoá tự động của các mô hình khuếch tán tiềm ẩn	Mất mát tri giác cộng một số hạng đối kháng là lý do ảnh giải mã không bị mờ như khi chỉ dùng MSE	Đây đúng là hiện tượng “trung bình của nhiều đáp án” đã nêu trong chương

MINI PROJECT SLAM · Hai hàm mất mát cho cùng một bài ghép cặp đặc trưng ORB · 1–2 ngày

Mục tiêu	kiểm chứng bằng số luận điểm trung tâm của chương — chọn loss là chọn hành vi — trên đúng bài toán bạn đang làm: quyết định hai mảnh ảnh quanh hai đặc trưng ORB có ứng với cùng một điểm 3D hay không.
Dữ liệu	một chuỗi EuRoC; sinh cặp dương và cặp âm từ chân trị pose cộng độ sâu stereo, hoặc đơn giản hơn là lấy các ghép cặp đã sống sót qua kiểm định hình học của chính ORB-SLAM3 làm cặp dương. Cắt mảnh 32×32 quanh mỗi đặc trưng; vài chục nghìn cặp là đủ.
Các bước	huấn luyện một CNN nhỏ theo hai cách, giữ nguyên backbone và mọi siêu tham số, chỉ đổi đầu ra và hàm mất mát: (1) đầu phân loại nhị phân với entropy chéo trên nhãn “cùng điểm / khác điểm”; (2) đầu nhúng với loss có biên — triplet hoặc InfoNCE. Sau đó tính khoảng cách trong không gian nhúng cho tập kiểm tra và vẽ hai biểu đồ phân bố, một cho cặp đúng và một cho cặp sai. Lặp lại phiên bản (2) với khai thác mẫu âm khó và vẽ lại.
Xong khi	bạn có ba con số cho mỗi cấu hình — tỉ lệ dương giả tại ngưỡng cho recall 95%, diện tích chồng lấn của hai phân bố, và khoảng cách giữa phân vị 5% của cặp sai với phân vị 95% của cặp đúng — và nói được loss nào tách tốt hơn ở <i>đuôi</i> , tức đúng vùng mà quyết định ghép cặp thật sự xảy ra.
Nổi với	C/18 cho vai trò của mẫu âm và nhiệt độ; B/10 vì kết quả phụ thuộc cả backbone chứ không riêng loss; B/11 nếu bạn định chạy bộ mô tả này thời gian thực.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì hai mô hình có thể cùng giá trị loss mà khác nhau ở độ hiệu chỉnh xác suất: hinge tất gradient ngay khi vượt lề nên điểm số của nó không có nghĩa xác suất, còn cross-entropy là proper scoring rule nên xác suất của nó đọc được. Quyết định có ngưỡng cần thứ hai, không cần thứ nhất.
- A2. Vì MSE là log-likelihood của một mô hình nhiễu Gaussian. Bạn không tránh được việc chọn giả định; bạn chỉ chọn nó một cách vô thức, và giả định đó sai ngay khi phần dư có đuôi nặng.
- A3. Vì entropy chéo theo pixel cộng điểm cho mọi pixel nên đoán đúng, mà nên chiếm 99%. Dice không cộng điểm cho nên nên nó bằng 0 tuyệt đối trong đúng tình huống đó — hai hàm đo hai thứ khác nhau trên cùng một dự đoán.
- A4. Vì IoU bằng 0 cho mọi cặp hộp rời nhau, bất kể xa gần: loss phẳng nên gradient bằng 0 và mô hình không nhận được tín hiệu nào về hướng phải đi. GIoU trừ thêm phần rỗng của hộp bao nhỏ nhất, cho $-0,78$ so với $-0,94$ trong ví dụ ở chương, nên hướng lại xuất hiện.
- A5. Label smoothing đặt một trần hữu hạn cho logit đúng nên chấm dứt cuộc đua logit vô hạn, và độ hiệu chỉnh xác suất tốt lên. Cái giá là các lớp bị co cụm trong không gian đặc trưng, nên chính đặc trưng đó dùng cho truy hồi hay chứng cất thì tệ đi.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn muốn một đứa trẻ dọn phòng cho sạch, nhưng “sạch” thì không đo được, nên bạn treo thưởng theo số món đồ được cất vào tủ. Đứa trẻ rất chăm và tuyệt đối theo chữ nghĩa: nó nhét cả rác vào tủ, còn những góc bạn quên tính điểm thì nó không đụng tới.

Máy học đúng như vậy. Thứ bạn thật sự muốn — robot không đâm vào ai — không đo được trong lúc dạy, còn máy thì chỉ hiểu một thứ duy nhất là một con số càng nhỏ càng tốt.

Mẹo thì đơn giản: đừng đi nhớ công thức, hãy trả lời một câu hỏi về dữ liệu của bạn — “sai số của tôi thường trông thế nào?”. Nếu sai số thường nhỏ và cân đối thì ra một công thức; nếu thỉnh thoảng có một mẫu hỏng nặng thì ra công thức khác. Mỗi câu trả lời là một công thức.

Cái giá gồm hai phần. Thứ nhất, máy tối ưu đúng con số bạn viết chứ không phải ý bạn định, nên chỗ nào bạn viết cầu thả thì nó thắng còn bạn thua. Thứ hai, chỗ nào con số không thay đổi thì máy hoàn toàn mù: hai cái hộp cùng đoán trượt, một cái sát và một cái rất xa, mà cùng bị chấm không điểm, thì máy không có cách nào biết cái nào tốt hơn.

Chỗ không rút gọn được: không ai chứng minh được rằng đẩy con số thay thế xuống thì thứ bạn thật sự muốn cũng tốt lên. Với hầu hết công thức đang dùng trong thị giác máy tính, đây là niềm tin dựa trên kinh nghiệm chứ không phải một định lý. Tôi không biết cách nói đơn giản nào che được chỗ hờ này mà không nói dối.

Một câu: Máy làm đúng thứ bạn viết ra, nên phần khó không nằm ở chỗ dạy máy mà ở chỗ viết cho đúng.

9.9 Kết nối

Chương này là hệ quả trực tiếp của chương 4: loss là một trong bốn chỗ cài inductive bias, và mọi thứ vừa bàn chỉ là những cách khác nhau để phát biểu “tôi biết gì về lời giải”. Nó là *đối tác* của chương 10: kiến trúc cài giả định về *dạng* của hàm, loss cài giả định về *mục tiêu*; một giả định cài sai ở chỗ này thường phải trả giá ở chỗ kia. Và nó là *tiền đề ngược* của chương 7 và 8: nếu giao thức đánh giá sai thì việc loss có nhất quán với metric hay không cũng thành vô nghĩa.

Đọc tiếp theo bốn hướng. [C/12-co-che-huan-luyen](#) cho biết gradient của những loss này thực sự chảy thế nào và vì sao vài loss khó tối ưu dù đúng về nguyên lý. [C/14-bai-toan-cv-loi](#) là nơi focal, GIoU, Dice và Hungarian matching xuất hiện trong ngữ cảnh thật, kèm lý do lịch sử vì sao từng cái ra đời. [C/17-mo-hinh-sinh](#) phát triển đoạn về chiều của KL thành bốn triết lý khác nhau của chữ “giống dữ liệu thật” [104]. Cuối cùng, [E/25-do-tin-cay-va-van-hanh](#) nhặt lại sợi chỉ hiệu chỉnh xác suất mà label smoothing và *proper scoring rule* vừa thả ra [24].

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao “loss giảm nhưng mAP không tăng” là triệu chứng có *nhiều* nguyên nhân, và ba phép đo nào phân biệt được chúng?
2. Focal loss giả định gì về quan hệ giữa “mẫu khó” và “mẫu có nhãn sai”? Giả định đó bị vi phạm khi nào và hậu quả định lượng là bao nhiêu?
3. Cho hai hộp dự đoán cùng có IoU bằng 0 với hộp thật, vì sao GIoU phân biệt được chúng còn IoU thì không, và hai con số cụ thể là bao nhiêu?
4. Nếu bạn thay MSE bằng MAE mà kết quả tệ đi, điều đó nói gì về phân bố phần dư của bài toán?
5. Trong loss bốn thành phần, vì sao log giá trị từng thành phần là chưa đủ, và bạn cần log thêm cái gì?
6. Label smoothing cải thiện hiệu chỉnh xác suất nhưng làm hại điều gì, và vì sao điều đó quan trọng nếu bạn định dùng đặc trưng cho truy hồi?
7. Nêu một trường hợp trong hệ thống của bạn mà mục tiêu thực tế, metric và loss lệch nhau — và cái giá của khe hở đó là gì?

Nguyên lý thiết kế kiến trúc (Architecture Design Principles)

PHỤ THUỘC B/04-nen-tang-hoc-tu-du-lieu (inductive bias và trục “giả định mạnh + ít dữ liệu ↔ giả định yếu + nhiều dữ liệu” — chương này chính là trục đó áp cho kiến trúc); B/09-thiet-ke-ham-mat-mat (chỗ cài giả định thứ hai); A/01-toan-toi-thieu/04-hinh-hoc-va-tin-hieu (convolution, lấy mẫu).

TÓM TẮT

Chương này giải thích vì sao không thể dùng một MLP khổng lồ cho mọi thứ, và vì sao mỗi kiến trúc thực chất là một tập giả định về cấu trúc dữ liệu được đông cứng thành mã nguồn. Nó xây năm câu hỏi để mổ xẻ một kiến trúc lạ, kèm phép tính tay ra con số cho từng câu: giả định nào được cài vào, mô hình nhìn được bao xa, thông tin định tuyến cố định hay theo nội dung, gradient chảy thế nào, và bốn thước đo chi phí rơi vào đâu. Đọc xong, bạn dự đoán được một lớp lỗi trước khi huấn luyện, tức trường tiếp nhận không phủ hết vật thể. Bạn giải thích được vì sao thêm độ sâu và thêm độ rộng có giá hoàn toàn khác nhau. Và bạn nhận ra khi nào một bảng so sánh kiến trúc thực ra đang so sánh công thức huấn luyện. Nếu chỉ nhớ một điều: kiến trúc không làm mô hình “thông minh hơn”, nó quyết định *ai chạm được tới ai, sau bao nhiêu bước, và với chi phí nào*.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tensor và feature map** — tensor là mảng số nhiều chiều — ảnh đầu vào là tensor $C \times H \times W$ (kênh, cao, rộng). Feature map là tensor ở giữa mạng: mỗi “kênh” của nó là một bản đồ cho biết một loại đặc trưng xuất hiện mạnh ở vị trí nào.
- **tầng nối đầy đủ và convolution** — tầng nối đầy đủ (*fully connected*) nhân toàn bộ đầu vào với một ma trận — mọi đầu ra nhìn thấy mọi đầu vào. Convolution trượt một cửa sổ nhỏ (kernel) khắp ảnh và dùng *cùng* bộ trọng số ở mọi vị trí, nên mỗi đầu ra chỉ nhìn thấy một vùng nhỏ.
- **kernel, stride, padding** — kernel là kích thước cửa sổ trượt (thường 3×3); stride là bước nhảy giữa hai vị trí liên tiếp (stride 2 làm ảnh nhỏ đi một nửa); padding là số pixel đệm thêm ở viền để kiểm soát kích thước đầu ra.
- **hàm kích hoạt (activation function)** — hàm áp lên từng phần tử giữa hai tầng tuyến tính, ví dụ ReLU $\max(0, x)$. Không có nó thì xếp chồng bao nhiêu tầng tuyến tính cũng chỉ tương đương một tầng tuyến tính — đó là lý do nó bắt buộc phải có.
- **lan truyền ngược và luồng gradient** — để cập nhật trọng số ở tầng đầu, đạo hàm phải nhân dồn ngược qua tất cả các tầng phía sau. Chuỗi phép nhân dài đó có thể tiêu biến hoặc bùng nổ, và đó là lý do “đường về của gradient” là một tiêu chí thiết kế thật sự chứ không phải ẩn dụ.
- **tham số với FLOPs** — tham số là số trọng số phải lưu; FLOPs là số phép nhân-cộng phải làm cho một lần chạy. Hai đại lượng độc lập với nhau: convolution có ít tham số nhưng nhiều FLOPs vì cùng bộ trọng số được dùng lại ở mọi vị trí.
- **inductive bias** — tập giả định mà bạn cài sẵn vào hệ thống về hình dạng của lời giải, trước khi nhìn dữ liệu. Nó đến từ bốn chỗ: kiến trúc, hàm mất mát, augmentation, và thuật toán tối ưu. Bạn không thể “không có giả định” — chỉ có thể chọn giả định nào.
- **batch** — số mẫu được xử lý cùng lúc trong một bước cập nhật. Nó ảnh hưởng tới bộ nhớ, tới độ ổn định của gradient, và — như phần cuối chương cho thấy — tới cả độ trễ đo được.

CÂU HỎI MỞ ĐẦU

- Q1. Một MLP đủ rộng xấp xỉ được mọi hàm liên tục — vậy vì sao không một hệ thống thị giác nghiêm túc nào dùng MLP cho ảnh?
- Q2. Trước ResNet, mạng 56 tầng có sai số *huấn luyện* cao hơn mạng 20 tầng. Vì sao đó chắc chắn không phải *overfitting*, và cái gì thực sự hỏng?
- Q3. Nếu *attention* mạnh hơn convolution ở mọi mặt lý thuyết, vì sao thể hệ ViT tiếp theo lại phải cắt nó về thành cục bộ trong từng cửa sổ?
- Q4. Hai mạng cùng số tham số, một sâu gấp đôi và một rộng gấp đôi — vì sao cái rẻ hơn về tham số lại là cái đắt hơn về thời gian chạy?
- Q5. Bạn đọc một bảng nói kiến trúc mới hơn kiến trúc cũ 3 điểm mAP. Vì sao con số đó có thể không nói gì về kiến trúc cả?

10.1 Đặt vấn đề

Một mạng nơ-ron đủ rộng xấp xỉ được mọi hàm liên tục; định lý xấp xỉ phổ quát nói vậy từ những năm 1980. Nếu thế thì vì sao không dùng một **MLP** khổng lồ cho mọi thứ và để dữ liệu lo phần còn lại? Câu trả lời không nằm ở khả năng biểu diễn mà nằm ở *chi phí học*: định lý nói hàm đúng *tồn tại* trong không gian giả thuyết, nó không nói bạn tìm được hàm đó với lượng dữ liệu và lượng tính toán bạn có. Kiến trúc là công cụ thu hẹp không gian giả thuyết lại quanh những hàm khả dĩ, và mọi lần thu hẹp đều là một phát biểu về cấu trúc của dữ liệu.

Ba ví dụ để thấy phát biểu đó cụ thể tới mức nào:

- **Convolution** phát biểu rằng đặc trưng hữu ích thì *cục bộ*, và một đặc trưng có nghĩa ở góc trên trái thì cũng có nghĩa y như vậy ở góc dưới phải.
- **Pooling** phát biểu rằng vị trí chính xác ít quan trọng hơn sự hiện diện.
- **Attention** phát biểu rằng quan hệ giữa hai vị trí nên do *nội dung* quyết định chứ không do khoảng cách.

Nếu phát biểu khớp với dữ liệu, bạn được tổng quát hoá tốt với ít mẫu; nếu không khớp, bạn đã cài sẵn một sai lệch mà không lượng dữ liệu nào gỡ được. Đây chính xác là trục ở chương 4: giả định mạnh đối lấy ít dữ liệu. Toàn bộ lịch sử kiến trúc thị giác đọc được như một chuỗi các lần *nói lòng* giả định, mỗi lần dữ liệu và tính toán rẽ đi một bậc.

↪ **tiền đề B/04** — trục đánh đổi giả định–dữ liệu là thứ giải thích vì sao ViT cần tiền huấn luyện quy mô lớn còn CNN thì không.

Giá trị của chương: sau khi đọc, bạn không cần nhớ tên kiến trúc nào cũng đọc được một bài báo kiến trúc lạ, vì bạn có năm câu hỏi cố định và ba phép tính tay để tự trả lời chúng.

TRỰC GIÁC

Hãy nghĩ về kiến trúc như hệ thống ống dẫn chứ không như một cái hộp đen: nó quyết định *thông tin nào có thể chạm tới thông tin nào, sau bao nhiêu bước*. Một MLP là bể chứa — mọi thứ chạm được mọi thứ ngay lập tức, nên nó không có cấu trúc để dựa vào và phải trả bằng tham số. Convolution là mạng ống ngăn nối các điểm lân cận, nên hai pixel ở hai góc ảnh chỉ “nói chuyện” được sau nhiều tầng. Nếu số tầng không đủ, chúng *không bao giờ* nói chuyện được, dù bạn huấn luyện bao lâu. **Kết nối tắt** (*residual connection*) là đường ống phụ chạy song song, đặt ở đó không phải để mang thêm thông tin xuôi mà để **gradient** có lối về ngấn. Đọc kiến trúc theo hình dạng đường ống thì receptive field, attention và skip

connection thôi là ba khái niệm rời rạc và trở thành ba câu trả lời cho cùng một câu hỏi: ai chạm được tới ai.

10.2 Từ template matching tới convolution (From Template Matching to Convolution)

A. Điểm xuất phát nhìn được bằng mắt

Điểm xuất phát đáng ngạc nhiên là bộ phân loại tuyến tính $f = Wx + b$. Cách đọc bề ích không phải đại số mà là hình học: mỗi hàng của W là một **mẫu chuẩn** (*template*) có cùng số chiều với ảnh, và điểm số của một lớp là tích vô hướng giữa ảnh và mẫu chuẩn đó. Ý nghĩa bằng lời: *mô hình chấm điểm bằng cách hỏi ảnh này giống bản mẫu của lớp đến đâu*. Vẽ các hàng của W ra thành ảnh sau khi huấn luyện trên CIFAR-10 là bài tập rất đáng làm. Bạn sẽ thấy mẫu “ngựa” có hai đầu. Lý do là mỗi lớp chỉ có *một* mẫu chuẩn, nên nó buộc phải trung bình mọi tư thế lại. Giới hạn của mô hình lộ ra từ hình ảnh đó, trước cả khi bạn đọc con số accuracy.

B. Ba lần thu hẹp không gian giả thuyết

Mạch 1 — vì sao convolution ra đời, và nó lấy đi cái gì

- 1 Vấn đề** một mẫu chuẩn cho mỗi lớp là quá ít, và biên quyết định chỉ là siêu phẳng.

Giải pháp xếp chồng nhiều tầng tuyến tính xen phi tuyến, tức MLP.

Giả định hàm cần học liên tục và mạng đủ rộng.

Giá phải trả số tham số nổ tung — ảnh $224 \times 224 \times 3 = 150\,528$ chiều nối đầy đủ vào 1000 nơ-ron là ≈ 150 triệu tham số cho *một* tầng.

Vấn đề mới tệ hơn cả chi phí, mô hình không hề biết hai pixel cạnh nhau thì liên quan, nên phải học lại sự thật đó từ dữ liệu, và học lại nó riêng cho từng vị trí.
- 2 Vấn đề** quá nhiều tham số và không có chút tri thức nào về ảnh.

Giải pháp **chia sẻ trọng số** (*weight sharing*) cộng tính cục bộ, tức convolution [105].

Hệ quả ta được **đẳng biến tịnh tiến** (*translation equivariance*) gần như miễn phí, và số tham số giảm hàng nghìn lần: một tầng conv 3×3 từ 3 kênh sang 64 kênh chỉ có $3 \cdot 3 \cdot 3 \cdot 64 = 1728$ trọng số, *bất kể ảnh to bao nhiêu*.

Giá phải trả bất biến với xoay và với tỉ lệ thì *không* được cài vào, nên mạng vẫn phải học chúng từ dữ liệu hoặc từ *augmentation* — tức chỗ cài giả định thứ ba của chương 4; và quan hệ giữa hai vùng xa nhau trở nên đắt đỏ.

Vấn đề mới nếu thông tin chỉ lan ra từng chút qua mỗi tầng thì *mô hình nhìn được bao xa?*

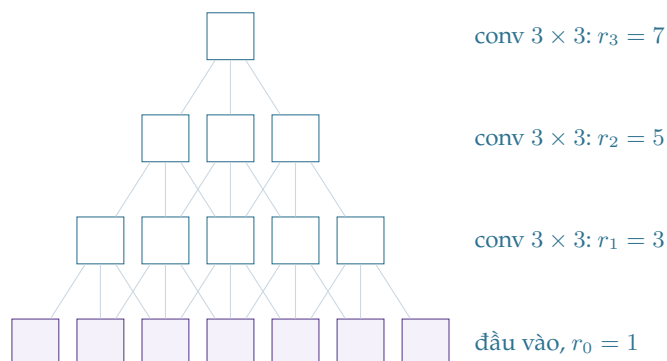
Giải pháp tính receptive field bằng tay *trước* khi huấn luyện, thay vì phát hiện vấn đề sau ba ngày.

C. Receptive field: con số tính được trước khi train

Kích thước đầu ra của một tầng conv là $(W - F + 2P)/S + 1$, và **trường tiếp nhận** (*receptive field*) tích lũy theo hai công thức truy hồi rất đơn giản:

$$j_l = j_{l-1} \cdot s_l, \quad r_l = r_{l-1} + (k_l - 1) j_{l-1}, \quad j_0 = 1, r_0 = 1.$$

Diễn giải bằng lời: mỗi tầng mở rộng tầm nhìn thêm đúng $(k - 1)$ lần bước nhảy hiện tại, và mỗi lần lấy mẫu xuống thì bước nhảy nhân lên. Hệ quả là stride mở rộng tầm nhìn theo cấp số nhân, còn độ sâu chỉ mở rộng theo cấp số cộng. Hình 10.1 minh họa trường hợp nhỏ nhất tính được trong đầu.



Hình 10.1. Ba tầng 3×3 stride 1 (vẽ theo một chiều cho gọn): một nơ-ron ở tầng trên cùng nhìn thấy đúng 7 pixel đầu vào, tô đậm. Cùng tầm nhìn với một tầng 7×7 nhưng chỉ tốn $3 \cdot 9C^2 = 27C^2$ tham số thay vì $49C^2$, và có ba phi tuyến thay vì một — đây chính là lập luận của VGG.

Vì sao con số này quan trọng đến thế? Vì *receptive field* nhỏ hơn vật thể là nguyên nhân của một lớp lỗi rất khó chẩn đoán. Mô hình sẽ sai có hệ thống trên vật lớn, và trên những gì cần ngữ cảnh rộng: phân biệt một con thuyền với một cái xe đòi hỏi nhìn thấy nước hay đường. Triệu chứng thì trông giống hệt “thiếu dữ liệu” hoặc “mô hình nhỏ quá”, và cả hai chẩn đoán đó đều dẫn tới hành động sai.

↔ **đối lập với B/08** — cùng triệu chứng “sai có hệ thống trên một nhóm”, nhưng ở đó nguyên nhân là dịch chuyển phân phối chứ không phải giới hạn kiến trúc; phép đo phân biệt hai thứ là khác nhau.

Điều dễ chịu là receptive field tính được trong hai phút bằng giấy bút.

Nhưng ở đây phải phức tạp hoá một chút, vì cắt gọn sẽ thành sai. Với ResNet-18 ở tầng conv cuối, con số lý thuyết rơi vào cỡ vài trăm pixel, lớn hơn ảnh 224×224 , nên thoát nhìn có vẻ không có vấn đề gì. Hiệu chỉnh quan trọng là **trường tiếp nhận hiệu dụng** (*effective receptive field*) nhỏ hơn nhiều: đóng góp của các pixel giảm dần theo dạng gần Gaussian tính từ tâm, và bán kính hiệu dụng chỉ tăng cỡ căn bậc hai của số tầng. Đây là kết quả đã được công bố (dòng công trình về *effective receptive field*), và hệ quả thực dụng là hãy coi con số lý thuyết như một chặn trên rất lỏng chứ đừng coi là mô tả hành vi thật.

10.3 Bốn cách nhìn xa hơn, bốn cái giá (Widening the Receptive Field)

Khi receptive field không đủ, có bốn đòn bẩy và không cái nào miễn phí.

Bảng 10.1. Bốn cách mở rộng receptive field: cơ chế, cái được, và chế độ hỏng riêng của từng cách.

Cách	Cơ chế	Được	Hỏng khi nào
Tăng độ sâu	Cộng dồn $(k - 1)j$ qua nhiều tầng	Cả tầm nhìn lẫn độ trừu tượng	Khó tối ưu nếu không có residual; độ trễ tăng <i>tuần tự</i> , không song song hoá được
Lấy mẫu xuống	Nhân bước nhảy j lên	Rẻ nhất, mở rộng theo cấp số nhân	Mất chi tiết không gian vĩnh viễn; vật nhỏ biến mất trước khi tới tầng sâu
<i>Dilated conv</i> [106]	Giãn kernel mà giữ nguyên độ phân giải	Vừa xa vừa chi tiết — đúng thứ phân vùng cần	<i>Gridding artifact</i> : điểm lấy mẫu không phủ liên tục nên có tần số bị bỏ sót
Attention toàn cục [107]	Mọi vị trí chạm mọi vị trí trong một bước	Quan hệ xa tùy ý, do nội dung quyết định	Chi phí bậc hai theo số <i>token</i> ; bỏ bớt inductive bias nên đối dữ liệu

Cột chi phí của attention đáng làm thành con số vì nó giải thích gần hết thiết kế của các ViT hiện đại. Ở 224×224 với patch 16×16 ta có $14 \times 14 = 196$ token và ma trận tương tự $196^2 \approx 3,8 \cdot 10^4$ phần tử — hoàn toàn chấp nhận được. Nhưng phân vùng cần đặc trưng ở 56×56 , tức 3136 token, và $3136^2 \approx 9,8 \cdot 10^6$: gấp 256 lần. Đó là lý do *vật lý*, không phải thẩm mỹ, khiến Swin chia attention thành cửa sổ cục bộ rồi dịch cửa sổ giữa các tầng [108]. Nói cách khác, Swin *cài lại* tính cục bộ mà ViT vừa gỡ bỏ, sau khi cả ngành đã học được rằng gỡ hết thì quá đắt.

↔ **ràng buộc bởi B/11** — con số $256 \times$ này vẫn chưa phải toàn bộ câu chuyện: thời gian chạy phụ thuộc lưu lượng bộ nhớ chứ không chỉ số phép tính.

10.4 Residual và luồng gradient (Residual Connections and Gradient Flow)

A. Nghịch lý mà residual thực sự giải quyết

Đây là điểm hay bị hiểu sai nhất trong toàn chương. Trước ResNet, người ta quan sát một nghịch lý: mạng 56 tầng có sai số *huấn luyện* cao hơn mạng 20 tầng [109]. Đây không thể là *overfitting* — overfitting làm sai số test cao chứ không làm sai số train cao. Mạng sâu hơn về nguyên tắc biểu diễn được mọi thứ mạng nông biểu diễn được (chỉ cần các tầng thừa học hàm đồng nhất), nên vấn đề nằm ở chỗ SGD *không tìm được* nghiệm đó. Kết nối tắt $y = x + F(x)$ sửa đúng chỗ đó: hàm đồng nhất trở thành mặc định (chỉ cần $F \rightarrow 0$), và gradient có một đường về thẳng không bị nhân qua chuỗi Jacobian. Nói gọn thì *residual giải quyết bài toán tối ưu trước*, còn *độ sâu chỉ là hệ quả của việc bài toán tối ưu đã dễ đi*.

B. Chuẩn hoá đặt ở đâu, và nó giả định gì

Hệ quả kéo dài tới hôm nay: mọi kiến trúc sâu đều có dạng “một luồng chính cộng dồn, các khối chỉ ghi thêm một lượng nhỏ vào luồng đó”. Chỗ đặt *normalization* là chi tiết đi kèm và có hệ quả rất thực. Đặt sau phép cộng (*post-norm*) thì luồng chính bị chuẩn hoá lại ở mỗi tầng, đường về của gradient không còn sạch, và *transformer* sâu cần khởi động học suất rất cẩn thận mới không phân kỳ; đặt trước khối (*pre-norm*) thì luồng chính không bị đung vào, huấn luyện ổn định hơn

nhiều nhưng thường mất một chút chất lượng cuối. Bản thân họ chuẩn hoá cũng là những giả định khác nhau.

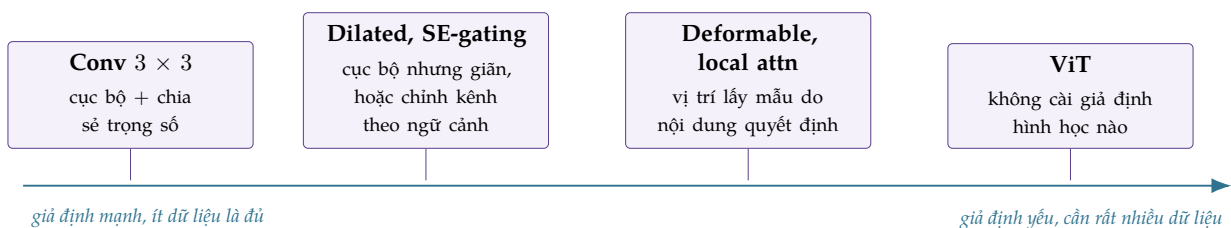
Bảng 10.2. Ba họ chuẩn hoá: mỗi họ là một giả định khác nhau về chỗ nào thống kê đáng tin.

Họ	Chuẩn hoá theo chiều nào	Giả định	Hỏng khi nào
BatchNorm [4]	Trên cả batch, từng kênh	Batch đủ lớn và đồng nhất; thống kê lúc suy luận giống lúc huấn luyện	Batch bằng 2 (cấu hình huấn luyện detection); dữ liệu suy luận lệch phân bố; suy luận trực tuyến từng mẫu
LayerNorm [110]	Trong từng mẫu, trên toàn bộ đặc trưng	Các kênh trong một mẫu có thang so sánh được	Ảnh có kênh với ý nghĩa vật lý rất khác nhau
GroupNorm [111]	Trong từng mẫu, theo nhóm kênh	Có cấu trúc nhóm tự nhiên giữa các kênh	Số nhóm chọn tùy tiện; mất một phần lợi ích chính quy hoá của BN

Ở đây cần trung thực về mức độ chắc chắn: lý do *vì sao* chuẩn hoá giúp vẫn còn tranh luận. Lời giải thích “giảm internal covariate shift” của bài gốc đã bị phản bác bằng thực nghiệm [112], và cách hiểu được ủng hộ hơn hiện nay là nó làm mất loss trơn hơn. Điều *đã được đo* là hành vi ở cột cuối bảng; điều còn là giả thuyết là cơ chế.

10.5 Định tuyến cố định hay theo nội dung (Fixed vs Content-Based Routing)

Convolution trộn thông tin theo một mẫu hình *cố định*: cùng một kernel áp lên mọi vị trí, bất kể nội dung. Đây vừa là sức mạnh (ít tham số, tổng quát hoá tốt) vừa là giới hạn cứng. Ba thể hệ giải pháp nới lỏng dần sự cố định đó, và xếp chúng lên một quang phổ như Hình 10.2 là cách gọn nhất để nhớ cả họ.



Hình 10.2. Quang phổ giả định của kiến trúc thị giác: đi từ trái sang phải, mỗi bậc gỡ bớt một ràng buộc hình học và đổi lấy nhu cầu dữ liệu cao hơn. Đây chính là trục của chương 4, lần này áp cho kiến trúc thay vì cho toàn hệ thống.

Ba bậc giữa và bậc cuối, đọc theo cái được và cái mất:

- **Channel gating** kiểu Squeeze-and-Excitation chỉ học một trọng số cho mỗi kênh dựa trên ngữ cảnh toàn cục [113]. Rất rẻ, và nó chứng minh rằng chỉ một chút định tuyến động đã đủ tạo khác biệt.

- **Deformable convolution** để mạng học luôn *vị trí lấy mẫu* thay vì cố định lưới [114]. Mạnh khi vật thể biến dạng nhiều; khó tối ưu hơn và khó tăng tốc trên phần cứng vì truy cập bộ nhớ không đều.
- **Self-attention** tính lại toàn bộ mẫu hình tròn cho từng đầu vào một. Cái mất không chỉ là FLOPs mà còn là *lượng dữ liệu tối thiểu*: ViT huấn luyện từ đầu trên ImageNet-1k thua CNN, và chỉ vượt lên khi có tiền huấn luyện quy mô lớn hoặc khi được thêm lại một ít inductive bias qua chưng cất và augmentation mạnh [115], [116].

Có một hướng ngược lại đáng nhắc: thay vì nói lỏng giả định, *cài thêm toán học vào kiến trúc* — tăng tối ưu khả vi, vòng lặp tối ưu được trải phẳng, ràng buộc hình học cứng đặt thẳng vào đồ thị tính toán. Tất cả đều nói “đừng học lại thứ tôi đã biết”. Chúng mạnh khi dữ liệu ít và mô hình toán đúng; chúng cứng nhắc và sai một cách khó gỡ khi giả định toán không khớp. Người làm hình học sẽ thấy trực này quen thuộc: đây đúng là cuộc tranh luận giữa pipeline SLAM cổ điển và các mô hình **tiền thẳng** (*feed-forward*), và chương 16 sẽ mở nó ra đầy đủ.

↪ **đối lập với C/16** — hình học cổ điển cài toán vào hệ thống, mô hình tiến thẳng để mạng học lại toán đó — cùng một bài toán, hai vị trí đặt giả định.

10.6 Bốn thước đo chi phí (Four Cost Metrics)

A. Ba trục phân tích được bằng giấy bút

Có bốn thước đo và chúng *không* tỉ lệ với nhau: số tham số, FLOPs, bộ nhớ, và độ trễ thực đo. Trước khi chạy bất cứ thứ gì, ba trục kiến trúc cơ bản đã phân tích được.

Bảng 10.3. Nhân đôi theo ba trục: chi phí không đi cùng nhau, nên “cùng ngân sách FLOPs” không có nghĩa là “cùng giá”.

Nhân đôi	Tham số	FLOPs	Bộ nhớ kích hoạt	Độ trễ (batch 1)
Độ sâu	×2	×2	×2	×2 và <i>tuần tự</i> — không giấu được bằng song song hoá
Độ rộng	×4	×4	×2	< ×4: kernel to hơn dùng GPU hiệu quả hơn
Độ phân giải	×1	×4	×4	≈ ×4; là trục dễ tràn bộ nhớ nhất

Bảng 10.3 giải thích vài điều thoạt nhìn nghịch lý. Tăng độ rộng làm tham số tăng *bậc hai* — vì một tầng conv có $k^2 C_{in} C_{out}$ trọng số và cả hai chỉ số cùng nhân đôi — trong khi tăng độ sâu chỉ tăng tuyến tính; xét theo tham số thì sâu rẻ hơn rộng. Nhưng xét theo độ trễ ở batch 1 thì ngược lại: các tầng phải chạy nối tiếp, còn độ rộng thì trải ra được trên hàng nghìn lõi. Đây là lý do các mạng thiết kế cho thiết bị biên thường nông và rộng hơn ta tưởng. Và vì ba trục không tương đương, dồn hết ngân sách vào một trục là lựa chọn tồi. Mở rộng cả ba cùng lúc theo một tỉ lệ cố định — **compound scaling** — cho kết quả tốt hơn [117].

Điểm nối trực tiếp sang chương 11 nằm ở cột cuối: *FLOPs thấp không có nghĩa là nhanh*. MobileNet dùng convolution tách theo chiều sâu để giảm FLOPs hơn một bậc so với ResNet-50 [118]. Nhưng ở batch 1 trên GPU nó thường không nhanh hơn tương ứng, đôi khi còn chậm hơn. Lý do là mỗi FLOP của nó vẫn phải kéo theo gần như cùng lượng dữ liệu như trước, nên phép tính chuyển từ chế độ nghẽn tính toán sang chế độ nghẽn băng thông. Chương 11 biến câu đó thành hai con số byte cụ thể.

↔ **hệ quả của D/20** — mọi kỹ thuật nén mô hình ở đó đều nhằm vào một trong bốn thước đo này, và chọn nhầm thước đo là lý do phổ biến nhất khiến nén xong không nhanh hơn.

B. Các mô típ lặp lại: cách đọc nhanh nhất một kiến trúc lạ

- **Tách ba tầng chức năng** — *backbone* trích đặc trưng đa tỉ lệ, *neck* trộn các tỉ lệ (ví dụ FPN [119]), *head* biến đặc trưng thành đầu ra của bài toán. Nhận ra ba tầng này là bước đầu tiên khi mở một repo lạ, vì nó cho biết chỗ nào thay được.
- **Encoder-decoder với skip connection** [120] — dùng khi đầu ra cần độ phân giải cao: nhánh xuống lấy ngữ nghĩa, skip mang chi tiết không gian đã bị mất khi lấy mẫu xuống.
- **Bottleneck** — giảm số kênh ở giữa để cắt chi phí bậc hai của tầng 3×3 , rồi phục hồi.
- **Đa tỉ lệ và coarse-to-fine** — xử lý thô trước rồi tinh dần, để chi phí tập trung vào chỗ cần.

Chín mươi phần trăm kiến trúc thị giác là các mô típ này lặp lại theo thứ tự khác nhau, và khi bạn thấy chúng thì phần còn lại của bài báo đọc rất nhanh.

10.7 Thực hành

Repo và tài nguyên. [huggingface/pytorch-image-models](#) (timm) là nơi đối chiếu hàng trăm backbone trên cùng một khung; [TylerYep/torchinfo](#) để in bảng hình dạng và tham số từng tầng; [facebookresearch/fvcore](#) để đếm FLOPs; [lutzroeder/netron](#) để nhìn đồ thị tính toán thật thay vì tin hình vẽ trong paper; [Fangyh09/pytorch-receptive-field](#) để đối chiếu với phép tính tay của bạn. Danh sách này chốt tại tháng 9/2026 và tên các mô hình dẫn đầu sẽ cũ trong 6–12 tháng, còn các repo hạ tầng kể trên bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Tính tay receptive field của ResNet-18 ở tầng cuối — nó có phủ hết ảnh 224×224 không, và nếu có thì vì sao mô hình vẫn hỏng trên vật rất lớn? Vì sao tăng độ sâu rẻ hơn tăng độ rộng xét theo tham số nhưng đắt hơn xét theo độ trễ ở batch 1? Với batch 32 ảnh 512×512 , tầng nào chiếm nhiều bộ nhớ kích hoạt nhất, và vì sao câu trả lời không phải là tầng có nhiều tham số nhất?

Mini project. Chọn ba backbone trong timm có cùng mức accuracy ImageNet nhưng khác họ (một CNN cổ điển, một mạng *depthwise*, một ViT nhỏ). Tính tay tham số và FLOPs, viết ra dự đoán thứ tự độ trễ trước khi đo, rồi đo thật trên GPU và trên CPU. Mọi chỗ dự đoán sai là một bài học của chương 11; ghi lại cụ thể sai bao nhiêu lần và vì sao.

10.8 Giới hạn và trường hợp hỏng

Khung “kiến trúc là tập giả định” rất mạnh nhưng có ba chỗ nó nói dối.

- **Nó giả vờ rằng ta biết mình đang cài giả định gì.** Trong thực tế nhiều lựa chọn kiến trúc là kết quả của tìm kiếm thực nghiệm rồi được kể lại như một lập luận có nguyên lý. Bằng chứng rõ nhất là ConvNeXt [121]: khi lấy một ResNet và áp lần lượt các chi tiết huấn luyện lẫn thiết kế của ViT, phần lớn khoảng cách vốn được quy cho “attention” biến mất.
- **Mọi phép đếm ở đây là chặn, không phải hành vi thật.** Receptive field hiệu dụng nhỏ hơn lý thuyết rất nhiều, và FLOPs không dự đoán được độ trễ. Ai tối ưu theo con số lý thuyết mà không đo sẽ tối ưu nhầm hướng — chương 11 tồn tại đúng vì lý do này.
- **“Giả định mạnh thì cần ít dữ liệu” chỉ đúng khi giả định khớp.** Ảnh y sinh, ảnh vệ tinh và camera gắn trên người đều có xoay tự do. Với chúng, đẳng biến tịnh tiến của conv không

giúp gì cả, nên mạng phải tiêu sức chứa để học lại phép xoay — thứ mà một biểu diễn phù hợp hơn cho không mất gì. Cùng logic, nếu ảnh đã được cắt căn giữa và luôn cùng tỉ lệ thì phần lớn lợi thế của conv so với MLP biến mất.

CẠM BẤY

Đừng so sánh hai kiến trúc khi công thức huấn luyện của chúng khác nhau — và chúng gần như luôn khác nhau. Số epoch, augmentation, optimizer, độ phân giải, EMA của trọng số, độ dài warm-up: mỗi thứ có thể tạo chênh lệch lớn hơn chênh lệch giữa hai backbone. Khi đọc một bảng kết quả, câu hỏi đầu tiên không phải “kiến trúc nào cao hơn” mà là “hai dòng này có được huấn luyện như nhau không”; câu trả lời nằm ở phụ lục, không nằm ở phần method.

CẦN NHỚ

Kiến trúc không làm mô hình “thông minh hơn”, nó quyết định *ai chạm được tới ai, sau bao nhiêu bước, và với chi phí nào*. Ba câu hỏi tương ứng — receptive field, cách định tuyến, bốn thước đo chi phí — đủ để đọc gần như mọi kiến trúc thị giác, kể cả kiến trúc chưa ra đời.

[T2]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Inductive bias	Tập giả định về lời giải được cài sẵn trước khi nhìn dữ liệu. Nó đến từ bốn chỗ: kiến trúc, hàm mất mát, augmentation, và thuật toán tối ưu. Bạn không thể “không có giả định”, chỉ có thể chọn giả định nào.
●● Template matching	Cách đọc bộ phân loại tuyến tính: mỗi hàng của ma trận trọng số là một ảnh mẫu, và điểm số là tích vô hướng với ảnh đầu vào. Vẽ các hàng đó ra sẽ thấy mẫu “ngựa” có hai đầu, vì một lớp chỉ có một mẫu chuẩn.
●●● Vì sao không dùng một MLP khổng lồ cho ảnh?	Định lý xấp xỉ phổ quát chỉ nói hàm đúng <i>tồn tại</i> , không nói bạn tìm được nó. Một tầng nối đầy đủ từ ảnh $224 \times 224 \times 3$ sang 1000 nơ-ron đã là 150 triệu tham số, và mô hình vẫn không biết hai pixel cạnh nhau thì liên quan.
●●● Đăng biến tịnh tiến	Dời vật trong ảnh thì bản đồ đặc trưng dời theo đúng chừng ấy. Convolution có tính chất này gần như miễn phí nhờ chia sẻ trọng số, nhưng <i>không</i> có tính chất tương ứng cho xoay hay đổi tỉ lệ.
●●● Trường tiếp nhận	Vùng ảnh đầu vào ảnh hưởng tới một vị trí đầu ra. Tính bằng hai công thức truy hồi: bước nhảy nhân với stride, bán kính cộng thêm $(k - 1)$ lần bước nhảy hiện tại. Nhỏ hơn vật thể là một lớp lồi tính được trước khi huấn luyện.
●● Trường tiếp nhận hiệu dụng	Vùng thực sự đóng góp, nhỏ hơn con số lý thuyết rất nhiều vì đóng góp giảm dần theo dạng gần Gaussian từ tâm, và chỉ tăng cỡ căn bậc hai của số tầng. Con số lý thuyết chỉ là chặn trên rất lỏng.
●● Dilated convolution	Giãn khoảng cách giữa các điểm lấy mẫu của kernel để mở rộng tầm nhìn mà không giảm độ phân giải. Đúng thứ phân vùng cần, nhưng các điểm lấy mẫu không phủ liên tục nên sinh gridding artifact.
●● Vì sao attention đắt ở bài toán dày đặc?	Chi phí bậc hai theo số token. Ở 224×224 với patch 16×16 chỉ có 196 token, nhưng đặc trưng 56×56 cần 3136 token, đắt hơn 256 lần. Đó là lý do vật lý khiến Swin cắt attention về từng cửa sổ.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Kết nối tắt	Cộng thẳng đầu vào vào đầu ra của một khối, khiến hàm đồng nhất thành mặc định và mở một đường về sạch cho gradient. Nó giải quyết bài toán <i>tối ưu</i> trước; độ sâu chỉ là hệ quả.
●●● Vì sao mạng sâu hơn lại có sai số huấn luyện cao hơn?	Quan sát này có thật trước ResNet: mạng 56 tầng thua mạng 20 tầng ngay trên tập huấn luyện. Vì SGD không tìm được nghiệm, chứ không phải vì mạng không biểu diễn nổi. Đây chắc chắn không phải overfitting: overfitting làm sai số <i>kiểm thử</i> cao, không làm sai số huấn luyện cao.
●● Ba họ chuẩn hoá	BatchNorm chuẩn hoá theo chiều batch nên giả định batch đủ lớn và đồng nhất. LayerNorm và GroupNorm chuẩn hoá trong từng mẫu nên bỏ được giả định đó. Vì sao chuẩn hoá giúp thì tới nay vẫn còn tranh luận.
● Squeeze-and-Excitation	Học một trọng số cho mỗi kênh dựa trên ngữ cảnh toàn ảnh, rồi nhân lại vào kênh đó. Rất rẻ, và nó cho thấy chỉ một chút định tuyến theo nội dung đã đủ tạo khác biệt.
● Deformable convolution	Để mạng học luôn <i>vị trí lấy mẫu</i> thay vì cố định trên lưới. Mạnh khi vật thể biến dạng nhiều, nhưng truy cập bộ nhớ không đều nên khó tăng tốc trên phần cứng.
●●● Thước đo chi phí	Có bốn cái: số tham số, FLOPs, bộ nhớ, và độ trễ thực đo. Chúng không tỉ lệ với nhau, nên “cùng ngân sách FLOPs” không có nghĩa là “cùng giá”.
●● Vì sao nên mở rộng cả ba trục cùng lúc?	Vì ba trục có giá khác nhau: độ rộng làm tham số tăng bậc hai, độ sâu tăng tuyến tính nhưng cộng thẳng vào độ trễ tuần tự, độ phân giải nhân bộ nhớ lên bốn khi tăng gấp đôi. Dồn hết vào một trục thì chạm trần của trục đó trước.
●● Backbone–neck–head	Ba tầng chức năng của gần như mọi hệ thống thị giác: trích đặc trưng đa tỉ lệ, trộn các tỉ lệ, rồi biến đặc trưng thành đầu ra của bài toán. Nhận ra ba tầng này là bước đầu tiên khi mở một repo lạ.
●● Mô hình sai có hệ thống trên vật rất lớn — nguyên nhân?	Hai khả năng: trường tiếp nhận hiệu dụng không phủ hết vật, hoặc tập huấn luyện thiếu vật cỡ đó. Phân biệt bằng cách cắt vật lớn thành mảnh nhỏ rồi đo lại: nếu điểm số hồi phục thì lỗi nằm ở tầm nhìn, không ở dữ liệu.
●● Huấn luyện detection với batch 2, kết quả tụt bất thường — nguyên nhân?	Nhiều khả năng là BatchNorm: thống kê tính trên hai mẫu thì quá nhiều, và thống kê chạy dừng lúc suy luận lệch hẳn. Đổi sang GroupNorm hoặc đóng băng BatchNorm rồi đo lại.
● Bảng nói kiến trúc mới hơn 3 điểm mAP — kiểm tra gì trước?	Kiểm tra phụ lục xem hai dòng có cùng công thức huấn luyện không: số epoch, augmentation, optimizer, độ phân giải, độ dài warm-up. ConvNeXt cho thấy phần lớn khoảng cách CNN–ViT biến mất khi giữ công thức cố định.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
timm	Nó tồn tại chính vì bài học “phải giữ công thức huấn luyện cố định mới so được kiến trúc”	Hạ tầng bền; là nơi duy nhất so backbone công bằng
ConvNeXt	Bằng chứng thực nghiệm trực tiếp rằng phần lớn khoảng cách CNN–ViT đến từ công thức huấn luyện chứ không từ attention	Kết quả này thay đổi cách đọc mọi bảng so sánh kiến trúc
Dòng Swin và các ViT phân cấp	Cài lại tính cục bộ và cấu trúc đa tỉ lệ vào ViT, đúng vì lý do bậc hai theo số token đã tính trong chương	Là kiến trúc mặc định cho bài toán dày đặc tính tới 9/2026

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Segment Anything (SAM)	Phân bổ chi phí cực lệch: bộ mã hoá ảnh rất nặng chạy một lần, còn phần giải mã mặt nạ rất nhẹ chạy nhiều lần	Minh hoạ sạch nhất cho việc tách backbone-head theo ràng buộc độ trễ
Họ EfficientNet và các biến thể sau nó	Mở rộng hợp nhất theo cả ba trục thay vì dồn ngân sách vào một trục	Ý tưởng còn sống dù tên mô hình đã cũ

MINI PROJECT SLAM · Đo trường tiếp nhận thực nghiệm của backbone frontend · vài giờ tới một ngày

Mục tiêu	kiểm chứng bằng số cái chặn trên lòng đã nêu trong chương: đặt con số trường tiếp nhận lý thuyết tính bằng giấy bút cạnh con số đo được bằng cách che ảnh, và xem chúng lệch nhau bao nhiêu lần.
Dữ liệu	vài trăm ảnh xám 752×480 từ một chuỗi EuRoC, cùng đúng backbone bạn định dùng cho frontend.
Các bước	(1) tính tay trường tiếp nhận lý thuyết ở ba tầng có độ sâu khác nhau bằng hai công thức truy hồi trong chương; (2) chọn một vị trí đầu ra cố định; (3) trượt một ô che 16×16 khắp ảnh và mỗi lần ghi lại mức thay đổi của vectơ đặc trưng tại vị trí đó; (4) vẽ bản đồ nhạy cảm và lấy bán kính chứa 90% tổng thay đổi; (5) lặp cho cả ba tầng.
Xong khi	bạn có một bảng ba dòng — tầng, trường tiếp nhận lý thuyết, bán kính 90% đo được — cùng tỉ số giữa hai cột, và trả lời được bằng số rằng backbone này có nhìn đủ xa cho vật thể lớn nhất xuất hiện trong chuỗi của bạn hay không.
Nối với	B/11 vì bản đồ nhạy cảm tốn hàng nghìn lượt xuôi và là bài tập tốt để đo thông lượng; C/13 nếu bạn muốn lặp lại phép đo trên một ViT để so hai họ trên cùng thước đo; E/25 vì đây chính là một dạng bản đồ nổi bật.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì định lý chỉ nói hàm đúng *tồn tại* trong không gian giả thuyết, không nói bạn tìm được nó với lượng dữ liệu và tính toán bạn có. Một tầng nổi đầy đủ từ ảnh $224 \times 224 \times 3$ sang 1000 nơ-ron đã là 150 triệu tham số, và mô hình vẫn không biết hai pixel cạnh nhau thì liên quan.
- A2. Vì overfitting làm sai số *test* cao chứ không làm sai số *train* cao. Mạng sâu về nguyên tắc biểu diễn được mọi thứ mạng nông biểu diễn được, nên cái hỏng là bài toán tối ưu: SGD không tìm được nghiệm đó. Kết nối tắt biến hàm đồng nhất thành mặc định và mở một đường về sạch cho gradient.
- A3. Vì chi phí bậc hai theo số token. Ở 224×224 với patch 16×16 chỉ có 196 token, nhưng bài toán dày đặc cần đặc trưng ở 56×56 , tức 3136 token và đắt hơn 256 lần. Swin cài lại tính cục bộ mà ViT vừa gỡ bỏ, sau khi đã biết rằng gỡ hết thì quá đắt.
- A4. Vì tham số của một tầng conv là $k^2 C_{in} C_{out}$, nên nhân đôi độ rộng nhân tham số lên bốn còn nhân đôi độ sâu chỉ nhân lên hai. Nhưng các tầng phải chạy nối tiếp, còn các kênh trải ra được trên hàng nghìn lõi — nên ở batch 1, độ sâu là thứ không giấu được.
- A5. Vì hai dòng trong bảng gần như luôn khác nhau ở số epoch, augmentation, optimizer, độ phân giải và độ dài warm-up — những thứ có thể tạo chênh lệch lớn hơn chênh lệch kiến trúc. ConvNeXt là bằng chứng: giữ công thức cố định thì phần lớn khoảng cách biến mất.

NÓI LẠI THẬT ĐƠN GIẢN

Có hai cách xây nhà. Cách thứ nhất là đổ bê tông tự do: tạo được bất kỳ hình dạng nào, nhưng tốn rất nhiều vật liệu và thời gian. Cách thứ hai là lắp bằng một bộ gạch có sẵn, mỗi viên chỉ khớp với viên bên cạnh theo vài kiểu nhất định. Bộ gạch giới hạn hình dạng, nhưng bạn xây nhanh hơn với ít vật liệu hơn hẳn.

Mạng nơ-ron cũng vậy. Nối mọi điểm ảnh với mọi thứ là bê tông tự do: chỉ một tầng đã tốn hàng trăm triệu con số cần chỉnh. Hoặc bạn cài sẵn một hiểu biết rất tầm thường — hai điểm ảnh cạnh

nhau thì liên quan tới nhau, và một hoa văn có nghĩa ở góc này thì cũng có nghĩa y hệt ở góc kia. Cài xong, số thứ phải học giảm hàng nghìn lần.

Cái giá là gì? Vì mỗi viên gạch chỉ nhìn thấy vùng nhỏ quanh nó, muốn hai đầu bức ảnh “nói chuyện” được với nhau thì phải xếp rất nhiều lớp gạch chồng lên. Xếp không đủ lớp thì hai đầu đó không bao giờ gặp nhau, dù bạn dạy máy bao lâu — và đây là con số tính được bằng giấy bút trước khi bắt đầu. Còn nếu ảnh của bạn bị xoay lung tung thì bộ gạch không giúp gì, bạn tốn công hơn cả khi dùng bê tông.

Chỗ không rút gọn được: vì sao việc chuẩn hoá các con số giữa các lớp lại làm việc học dễ đi thì tới nay vẫn chưa có lời giải thích vừa gọn vừa đúng; cách giải thích trong bài báo gốc đã bị chính thực nghiệm bác bỏ.

Một câu: Kiến trúc là bộ gạch bạn chọn: chọn khớp với dữ liệu thì xây nhanh với rất ít vật liệu, chọn lệch thì tốn hơn cả xây tự do.

10.9 Kết nối

Chương này là *trục của chương 4 áp cho kiến trúc*: quang phổ ở Hình 10.2 chính là trục “giả định mạnh + ít dữ liệu ↔ giả định yếu + nhiều dữ liệu”. Nó *đối tác* với chương 9: kiến trúc cài giả định về dạng của hàm, loss cài giả định về mục tiêu. Nó là *tiền đề trực tiếp* của chương 11, vì cột “độ trễ” trong mọi bảng ở đây chỉ có nghĩa khi bạn hiểu vì sao FLOPs không dự đoán được thời gian.

Sang Phần C, [C/13-kien-truc-backbone](#) đọc lịch sử CNN, attention và ViT như một chuỗi các lần gỡ nút thắt — đó là bản kể chi tiết của quang phổ trong chương này. [C/14-bai-toan-cv-loi](#) cho thấy tách backbone/neck/head hoạt động ra sao trên bài toán thật và vì sao đa tỉ lệ là bắt buộc chứ không phải trang trí. [C/16-thi-giac-3d](#) mở ra cuộc tranh luận “nhúng toán vào kiến trúc hay để mạng học”. Và [D/20-toi-uu-va-nen-mo-hinh](#) nhặt lại chỗ này khi bàn về việc thay đổi kiến trúc *sau* huấn luyện.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao định lý xấp xỉ phổ quát *không* phải lý do để dùng MLP cho ảnh — giới hạn thật nằm ở khả năng biểu diễn hay ở chi phí học?
2. Residual connection giải quyết vấn đề gì *trước tiên*, và bằng chứng thực nghiệm nào cho thấy đó không phải vấn đề overfitting?
3. Một mô hình phân vùng hồng có hệ thống trên vật rất lớn: hai chẩn đoán nào giải thích được triệu chứng đó, và phép đo nào phân biệt chúng?
4. Vì sao Swin phải chia attention thành cửa sổ, và phép tính nào về số token cho thấy đó là bắt buộc chứ không phải một lựa chọn thẩm mỹ?
5. Nếu dữ liệu của bạn có xoay tự do, đẳng biến tịnh tiến của convolution còn giá trị gì, và bạn sẽ cài giả định thiếu vào chỗ nào trong bốn chỗ của chương 4?
6. Hai mạng cùng số tham số, một sâu gấp đôi và một rộng gấp đôi: chênh lệch độ trễ ở batch 1 khác chênh lệch ở batch 64 ra sao, và vì sao?
7. BatchNorm hồng ở ba tình huống nào, và mỗi tình huống vi phạm giả định nào của nó?

Giới hạn tính toán và hệ thống (Compute and Systems Limits)

PHỤ THUỘC B/10-thiet-ke-kien-truc (bốn thước đo chi phí — chương này biến cột “độ trễ” ở đó thành con số); B/03-giai-phau-he-thong (ràng buộc cứng); hữu ích nhưng không bắt buộc: D/19-numpy-toi-pytorch nếu bạn muốn tự đo.

TÓM TẮT

Chương này trả lời một câu hỏi rất cụ thể: vì sao bạn giảm FLOPs một nửa mà mô hình không nhanh hơn. Hầu hết phép toán trong một mạng thị giác không chờ đơn vị tính toán; chúng chờ bộ nhớ. Đại lượng phân biệt hai chế độ đó là cường độ số học, tức số FLOP làm được trên mỗi byte phải chuyển. Nó tính được bằng giấy bút trong hai phút. Đọc xong bạn ước lượng được bộ nhớ và độ trễ trước khi viết code. Bạn cũng đọc được một bảng “FLOPs thấp hơn ba lần” mà không bị nó dẫn sai hướng, và chọn đúng biện pháp cho đúng chế độ nghẽn thay vì thử tất cả. Nếu chỉ nhớ một điều: FLOPs là thuộc tính của mô hình, còn thời gian là thuộc tính của cặp mô hình–máy. Cây cầu giữa chúng là cường độ số học đem so với điểm gãy roofline của chính cỗ máy bạn chạy: khoảng 150 FLOP/byte trên A100, khoảng 330 trên Jetson Orin.

KIẾN THỨC CẦN CÓ TRƯỚC

- **FLOPs** — số phép nhân–cộng dấu phẩy động cần cho một lần chạy mô hình. Nó đo *khối lượng công việc tính toán*, không đo thời gian. Phân biệt hai thứ này là toàn bộ nội dung của chương. Có một nhập nhằng phổ biến: nhiều bài báo ghi “FLOPs” nhưng thực ra đếm số phép nhân–cộng, tức một nửa con số theo định nghĩa chặt.
- **băng thông bộ nhớ (memory bandwidth)** — số byte mỗi giây chuyển được giữa bộ nhớ chính của thiết bị và chip tính toán, ví dụ 2000 GB/s trên một GPU máy chủ và khoảng 200 GB/s trên một mô-đun nhúng. Đây là một hằng số vật lý của máy, không phải thứ tối ưu phần mềm thay đổi được.
- **cường độ số học (arithmetic intensity)** — tỉ số giữa số FLOP thực hiện và số byte phải chuyển để thực hiện chúng. Nó trả lời câu “mỗi byte kéo về được dùng để làm bao nhiêu việc”, và so nó với một hằng số của máy là ra ngay bạn đang bị giới hạn bởi tính toán hay bởi bộ nhớ.
- **độ trễ (latency)** — thời gian từ lúc một đầu vào đi vào hệ thống tới lúc có kết quả cho *chính* đầu vào đó. Đây là đại lượng mà hệ thống thời gian thực quan tâm.
- **thông lượng (throughput)** — số mẫu xử lý xong trên một đơn vị thời gian khi được phép gom nhiều mẫu lại cùng lúc. Hai đại lượng này xung đột nhau: gom mẫu làm thông lượng tăng và độ trễ xấu đi.
- **kích hoạt (activation)** — các tensor trung gian sinh ra ở giữa mạng trong *lượt xuôi* (một lần chạy mô hình từ đầu vào tới đầu ra). *Lượt ngược* là lần chạy tính gradient theo chiều ngược lại, do thuật toán lan truyền ngược điều khiển. Nó cần đúng các tensor đó, nên khi huấn luyện ta phải giữ lại tất cả. Vì vậy kích hoạt là phần bộ nhớ tỉ lệ với số mẫu trong một batch và với độ phân giải ảnh, khác hẳn tham số vốn cố định.
- **kernel (trong ngữ cảnh GPU)** — một chương trình nhỏ chạy trên GPU thực hiện một phép toán, ví dụ “một tầng convolution” hoặc “một phép cộng tensor”. CPU phải *phóng* từng kernel và mỗi lần phóng tốn một khoản thời gian cố định. Đừng nhầm với “kernel” nghĩa cửa sổ convolution ở chương 10 — hai nghĩa hoàn toàn khác nhau.

- **GPU utilization và MFU** — “utilization” chỉ cho biết GPU có đang bận hay không, kể cả khi nó bận chờ bộ nhớ; MFU (*model FLOPs utilization*) là tỉ lệ giữa FLOPs thật sự làm được và đỉnh lý thuyết của chip. Một GPU có thể util 95% mà MFU chỉ 10%, và chính khoảng cách giữa hai con số đó là manh mối chẩn đoán mạnh nhất bạn có.

CÂU HỎI MỞ ĐẦU

- Q1. Hai mô hình cùng số FLOPs, vì sao một cái chạy nhanh gấp năm lần cái kia trên cùng một GPU?
- Q2. Vì sao mô hình vừa khít bộ nhớ khi suy luận lại tràn khi huấn luyện, dù trọng số không đổi một byte nào?
- Q3. Điểm gây roofline của một mô-đun nhúng *cao gấp đôi* của một GPU máy chủ — điều đó nói gì về danh sách tối ưu bạn mang từ máy chủ xuống thiết bị?
- Q4. Bạn tối ưu mô hình suốt một tuần và GPU vẫn nhàn rỗi bốn mươi phần trăm thời gian. Vì sao lỗi có thể hoàn toàn không nằm trong mô hình?
- Q5. Bạn đo được lượt xuôi của một mạng lớn mất 0,2 ms. Vì sao con số đó gần như chắc chắn là đối trá, và đối trá theo cơ chế nào?

11.1 Đặt vấn đề

Bạn thay một khối trong mô hình bằng một khối rẻ hơn một bậc về **FLOPs**, đo lại, và độ trễ không nhúc nhích — đôi khi còn tệ hơn. Đây không phải chuyện hiếm và cũng không phải lỗi cài đặt: nó là hệ quả trực tiếp của một sự thật vật lý mà bảng so sánh trong paper không bao giờ nhắc tới. Bộ xử lý hiện đại *tính* nhanh hơn khả năng *nạp dữ liệu* của nó hàng trăm lần, nên phần lớn các phép toán trong một mạng nơ-ron không chờ đơn vị tính toán mà chờ bộ nhớ. Khi bạn đang bị giới hạn bởi băng thông mà lại đi tối ưu số phép nhân, bạn đang tối ưu đúng cách nhưng nhầm chỗ.

Đọc nhanh, nếu bạn chỉ có năm phút. Có một đại lượng duy nhất quyết định bạn đang bị giới hạn bởi cái gì: **cường độ số học** (*arithmetic intensity*) — số phép tính thực hiện được trên mỗi byte phải chuyển qua bộ nhớ. Đem so nó với một hằng số của phần cứng (**điểm gây** (*ridge point*) = đỉnh tính toán chia cho băng thông) là ra ngay chế độ nghẽn, và mỗi chế độ có một tập biện pháp riêng mà các tập kia hoàn toàn vô ích. Trên Jetson AGX Orin, điểm gây nằm quanh 330 FLOP/byte, nghĩa là gần như mọi thứ trừ *convolution* dày và *GEMM* lớn đều nghẽn bộ nhớ.

Giá trị của chương này rất cụ thể. Bạn dự đoán được *trước khi viết một dòng code* rằng một tối ưu sẽ giúp hay không. Bạn ước lượng được bộ nhớ và độ trễ bằng giấy bút với sai số một bậc. Và bạn đọc được một bảng “FLOPs thấp hơn ba lần” mà không bị nó dẫn đi sai hướng. Nếu bạn đã từng đo latency của một pipeline SLAM trên thiết bị nhúng, chương này chủ yếu là đặt tên và công thức hoá những thứ bạn đã cảm nhận bằng tay.

TRỰC GIÁC

Hãy hình dung GPU như một nhà máy khổng lồ với một cửa nhập hàng rất hẹp. Bên trong có hàng nghìn thợ (**CUDA core** và **lõi tensor** (*tensor core*)) làm việc nhanh kinh khủng; nhưng nguyên liệu chỉ vào được qua một cửa duy nhất với lưu lượng cố định. **Cường độ số học** chính là số thao tác nhà máy làm được trên mỗi kilogram nguyên liệu nhập vào. Nếu công đoạn của bạn làm nhiều việc trên mỗi kilogram — như nhân hai ma trận lớn — thì thợ bận rộn và cửa nhập hàng thành thoi. Nếu nó chỉ làm một hai phép trên mỗi kilogram — như **BatchNorm** hay hàm kích hoạt — thì thợ đứng chơi và cả nhà máy chạy đúng bằng

tốc độ cửa. Giảm số việc phải làm trong trường hợp thứ hai không rút ngắn được gì cả, vì nút thất nằm ở cửa. Đây là toàn bộ nội dung của mô hình *roofline*, và cũng là lý do **hợp nhất kernel** (*kernel fusion*) — gộp nhiều công đoạn để nguyên liệu chỉ phải vào cửa một lần — lại là biện pháp mạnh đến vậy.

11.2 Roofline và cường độ số học (Roofline and Arithmetic Intensity)

A. Một công thức và một hằng số

Với một phép toán bất kỳ, định nghĩa **cường độ số học** (*arithmetic intensity*) là

$$I = \frac{\text{số FLOP thực hiện}}{\text{số byte phải chuyển giữa bộ nhớ và chip}}.$$

Nói gọn thì I đo mức độ “đáng công vận chuyển” của một phép toán. Hiệu năng đạt được bị chặn bởi hai thứ cùng lúc — đỉnh tính toán của chip và băng thông nhân với cường độ — nên [49]

$$P_{\text{đạt được}} = \min(P_{\text{đỉnh}}, B \cdot I).$$

Câu diễn giải: nếu mỗi byte không mang theo đủ việc để làm, chip sẽ chạy theo nhịp của bộ nhớ chứ không theo nhịp của chính nó. Chỗ hai chặn cắt nhau, $I^* = P_{\text{đỉnh}}/B$, là **điểm gãy** (*ridge point*): bên trái nó bạn nghẽn bộ nhớ, bên phải bạn nghẽn tính toán. Điểm gãy là một hằng số của máy, không phải của mô hình, và nó đáng thuộc lòng cho từng thiết bị bạn dùng.

- **NVIDIA A100 80 GB** — đỉnh khoảng 312 TFLOP/s fp16 (lỗi tensor, dày), băng thông HBM2e 2039 GB/s. Điểm gãy ≈ 153 FLOP/byte.
- **Jetson AGX Orin 64 GB** — đỉnh khoảng 68 TFLOP/s fp16 (suy ra từ con số công bố 275 TOPS INT8 thừa; tôi chưa tự đo), băng thông LPDDR5 204,8 GB/s. Điểm gãy ≈ 332 FLOP/byte.

Con số thứ hai là điều đáng ngạc nhiên nhất của cả chương: *điểm gãy của thiết bị nhúng cao gấp đôi máy chủ*, vì bộ nhớ LPDDR chậm hơn nhiều so với đà tăng của đơn vị tính toán. Hệ quả thực hành rất nặng: một tối ưu chứng minh được là hữu ích trên máy chủ có thể vô ích trên Orin, và ngược lại, kernel fusion trên Orin thường đáng giá hơn nhiều so với việc cắt FLOPs.

B. Ví dụ nhỏ nhất tính tay được

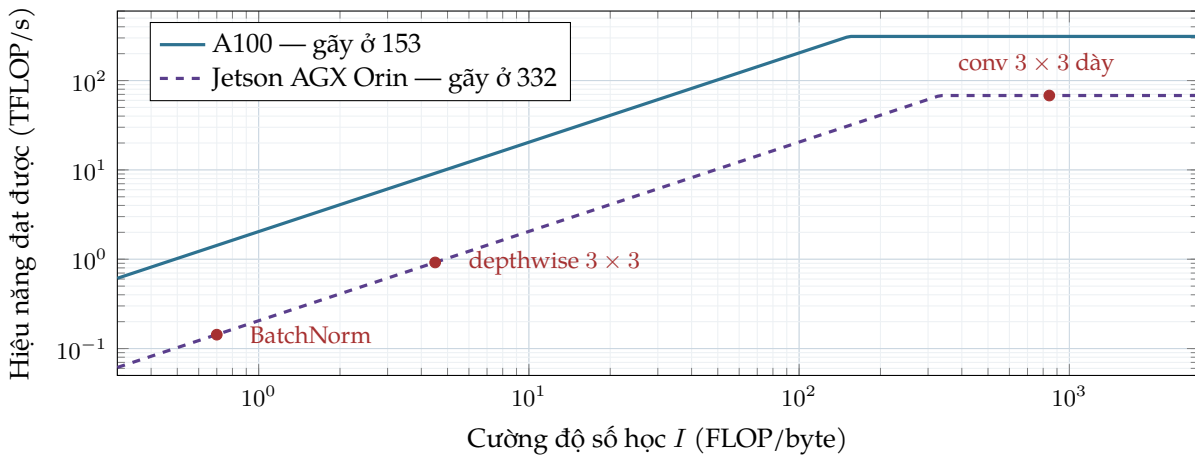
Lấy một *feature map* $256 \times 56 \times 56$ ở fp16 (2 byte mỗi phần tử), tức 802 816 phần tử và 1,61 MB mỗi lần đọc hoặc ghi. So sánh ba phép toán chạy trên chính nó, batch bằng 1.

- **Convolution 3×3 dày, 256 $\rightarrow$ 256 kênh.**
 - FLOPs = $2 \cdot 9 \cdot 256 \cdot 256 \cdot 56 \cdot 56 \approx 3,70 \cdot 10^9$.
 - Byte = vào 1,61 MB + ra 1,61 MB + trọng số $9 \cdot 256 \cdot 256 \cdot 2 = 1,18 \text{ MB} \approx 4,4 \text{ MB}$.
 - $I \approx 843$ FLOP/byte — nằm bên phải điểm gãy của *cả hai* máy, tức compute-bound thật sự.
- **BatchNorm trên cùng tensor đó.**
 - FLOPs ≈ 4 phép trên mỗi phần tử $\approx 3,2 \cdot 10^6$; byte ≈ 3 lượt qua tensor $\approx 4,8 \text{ MB}$.
 - $I \approx 0,7$ FLOP/byte — thấp hơn convolution hơn một nghìn lần. Đây là câu trả lời cho câu hỏi “vì sao BatchNorm memory-bound còn conv thì compute-bound”, và cũng là lý do mọi trình biên dịch đều cố gộp BatchNorm vào conv liền trước nó.
- **Convolution 3×3 tách theo chiều sâu (depthwise), 256 kênh.**

- FLOPs = $2 \cdot 9 \cdot 256 \cdot 56 \cdot 56 \approx 1,45 \cdot 10^7$ — ít hơn bản dày đúng 256 lần.
- Byte $\approx 3,2$ MB — chỉ ít hơn bản dày 1,4 lần.
- $I \approx 4,5$ FLOP/byte. Nằm sâu bên trái điểm gãy, nên thời gian chạy quyết định bởi 3,2 MB kia chứ không bởi 14 triệu phép tính.

Ba dòng trên đủ để trả lời câu hỏi mở đầu chương, và đáng làm cho tới cùng. Hãy thay một convolution dày 3×3 bằng cặp *depthwise* 3×3 cộng *pointwise* 1×1 , đúng công thức của MobileNet [118]. Tổng phép tính là $1,45 \cdot 10^7 + 4,11 \cdot 10^8 \approx 4,3 \cdot 10^8$ FLOP, tức ít hơn 8,7 lần. Nhưng tổng lưu lượng bộ nhớ lại là $3,2 + 3,3 \approx 6,5$ MB, nhiều hơn 1,5 lần so với 4,4 MB của bản dày. Lý do là bây giờ có hai kernel, nên tensor trung gian phải ghi ra rồi đọc lại. Trên một máy nghèo bộ nhớ, thời gian tỉ lệ với byte chứ không với FLOP, nên phép “tối ưu” này làm mô hình *chậm đi* khoảng một rưỡi lần trong khi bảng số liệu khoe giảm FLOPs gần chín lần. Không có gì mâu thuẫn ở đây cả; chỉ là hai đại lượng khác nhau bị gọi chung là “chi phí”.

Hình 11.1 đặt cả ba phép toán lên mái nhà roofline của hai máy để thấy vị trí tương đối của chúng.



Hình 11.1. Roofline của hai máy và vị trí ba phép toán (chấm đặt trên mái của Orin). Phần dốc bên trái là vùng nghèo băng thông, phần phẳng bên phải là vùng nghèo tính toán. Điểm gãy của Orin nằm xa bên phải hơn A100, nên trên thiết bị những có *nhiều* phép toán rơi vào vùng nghèo bộ nhớ hơn. Các đỉnh lấy từ thông số công bố, không phải đo thực tế.

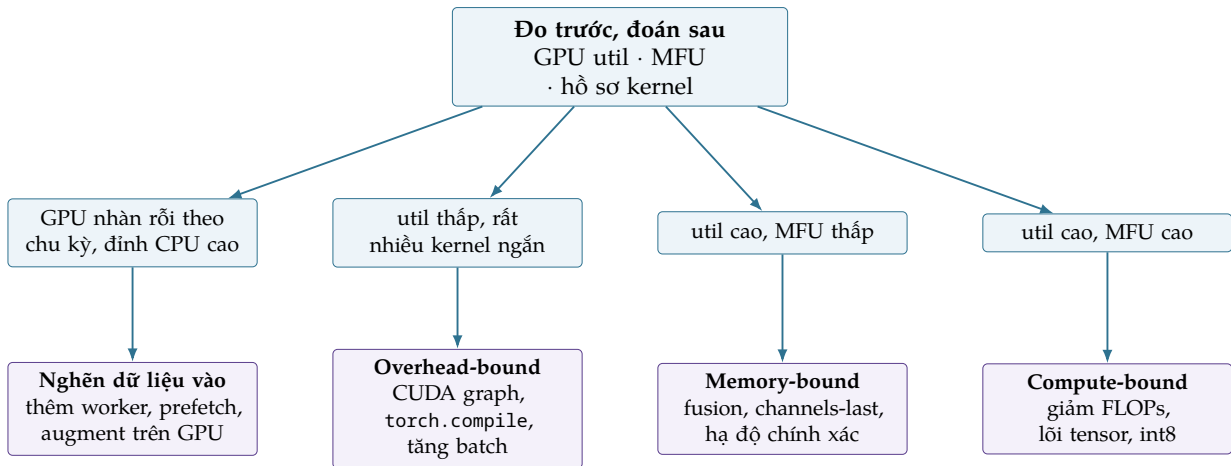
C. Ba chế độ nghèo và biện pháp tương ứng

Roofline mới chỉ phân biệt hai chế độ. Thực tế có chế độ thứ ba, và nó là chế độ phổ biến nhất ở suy luận batch nhỏ trên thiết bị nhúng: GPU không chờ tính toán cũng không chờ bộ nhớ mà chờ CPU gửi việc cho nó. Bảng 11.1 là bảng đáng dán lên tường.

Bảng 11.1. Ba chế độ nghèo: triệu chứng, biện pháp đúng, và biện pháp hoàn toàn vô ích.

Chế độ	Triệu chứng đo được	Biện pháp đúng	Vô ích
Compute-bound	GPU util cao, MFU cao (> 50%)	Giảm FLOPs thật, dùng lõi tensor, lượng tử hoá xuống int8	Kernel fusion
Memory-bound	GPU util cao nhưng MFU thấp; thời gian tỉ lệ với kích thước tensor	Kernel fusion, bố cục <i>channels-last</i> , giảm số lượt đọc/ghi, hạ độ chính xác để giảm byte	Giảm FLOPs
Overhead-bound	GPU util thấp, hồ sơ đầy kernel ngắn dưới $10 \mu s$, khoảng trống giữa các kernel	<i>CUDA graph</i> , <i>torch.compile</i> , gộp kernel, tăng batch	Đổi kiến trúc, giảm FLOPs

Hình 12.14 biến bảng đó thành một quy trình chẩn đoán theo thứ tự, kèm một nhánh thứ tư mà người làm thị giác gặp thường xuyên hơn cả ba nhánh kia cộng lại.



Hình 11.2. Cây chẩn đoán: bốn nhánh, và điều quan trọng nhất là biện pháp của nhánh này gần như vô dụng ở nhánh kia. Nhánh trái ngoài cùng — nghẽn ở đường dữ liệu vào — là nhánh đặc thù của thị giác máy tính và hay bị bỏ sót nhất.

11.3 Cùng FLOPs, khác nhiều lần độ trễ (Equal FLOPs, Unequal Latency)

Phần trước đã cho một nguyên nhân. Còn ba nguyên nhân nữa, và cộng lại chúng giải thích được gần hết các trường hợp dự đoán sai.

1. **Cường độ số học khác nhau** — đã tính ở trên: hai mạng cùng FLOPs có thể chênh nhau hai bậc về lưu lượng bộ nhớ. Đây là nguyên nhân lớn nhất và cũng dễ ước lượng nhất bằng giấy bút.
2. **Số lượng kernel và chi phí phóng kernel** — mỗi lần phóng một kernel tốn cỡ 5–10 μs trên máy chủ và nhiều hơn trên nền CPU yếu của thiết bị nhúng (đây là bậc độ lớn thường được đo, không phải một con số chuẩn). Một mạng nhẹ kiểu MobileNet có thể có hai trăm kernel ở batch 1, tức một sàn cỡ 1–2 ms mà không lượng FLOPs nào ảnh hưởng tới. Với ngân sách 33 ms cho cả pipeline, cái sàn đó không hề nhỏ.
3. **Bố cục bộ nhớ và điều kiện dùng lỗi tensor** — lỗi tensor chỉ được kích hoạt khi tensor ở đúng bố cục (*NHWC*, tức *channels-last*) và khi số kênh chia hết cho một bội số nhất định (8 với fp16). Sai một trong hai điều kiện thì kernel rơi xuống đường chậm và mất nhiều lần hiệu năng *trong khi FLOPs không đổi một chút nào*. Một mạng có số kênh 100 thay vì 96 hoặc 104 có thể chậm hơn đáng kể mà biểu đồ FLOPs không hề lộ gì.
4. **Mức độ song song có sẵn** — đây là chỗ nổi thẳng về Bảng ba trực ở chương 10

↪ **hệ quả của B/10** — bảng ba trực ở chương kiến trúc giờ đọc được bằng đơn vị thời gian thay vì bằng đơn vị FLOPs.

: hai mạng cùng FLOPs, một sâu và hẹp, một nông và rộng, có độ trễ rất khác nhau ở batch 1 vì các tầng phải chạy nối tiếp còn các kênh thì trải ra được trên hàng nghìn lõi. Ở batch 64 thì khoảng cách này thu hẹp lại, vì batch cung cấp nguồn song song thay thế.

Điểm chung của cả bốn: *FLOPs là một đại lượng của mô hình, độ trễ là một đại lượng của cặp mô hình–máy*. Không có gì bảo đảm hai đại lượng đó tỉ lệ với nhau, và trong họ mạng nhẹ chúng thường xuyên đi ngược nhau.

11.4 Bộ nhớ: kích hoạt so với tham số (Memory: Activations vs Parameters)

A. Vì sao mô hình vừa lúc suy luận lại tràn lúc huấn luyện

Đây là câu hỏi ai cũng gặp trong tuần đầu tiên. Bộ nhớ lúc huấn luyện gồm bốn phần rất khác nhau về cách chúng lớn lên:

- **Tham số** — cố định, không phụ thuộc batch. N tham số ở fp32 tốn $4N$ byte.
- **Gradient** — cùng kích thước với tham số, cũng cố định.
- **Trạng thái optimizer** — với Adam có hai moment nên thêm $8N$ byte. Cộng cả ba: **16 byte cho mỗi tham số**, một con số đáng thuộc.
- **Kích hoạt (activation)** — các tensor trung gian phải giữ lại để dùng trong lượt ngược. Đây là phần *tỉ lệ với batch và với bình phương độ phân giải*, và nó là phần gây tràn.

Ba phần đầu là lý do một mô hình 25 triệu tham số cần khoảng 410 MB chỉ để tồn tại trong vòng huấn luyện. Phần thứ tư là lý do cùng mô hình đó tràn 24 GB khi bạn nâng batch hoặc độ phân giải.

B. Tính tay bộ nhớ kích hoạt của ResNet-50

Hãy làm thật với ResNet-50 ở 224×224 , fp16. Đếm số phần tử của các đầu ra convolution cho một ảnh, theo từng tầng ($56^2 = 3136$, $28^2 = 784$, $14^2 = 196$, $7^2 = 49$):

- **Stem**: $64 \cdot 112^2 = 0,80$ M, sau max-pool $64 \cdot 3136 = 0,20$ M.
- **Stage 1** (3 khối ở 56×56 , kênh $64/64/256$): mỗi khối $(64+64+256) \cdot 3136 = 1,20$ M $\Rightarrow$ 3,61 M, cộng nhánh chiều 0,80 M.
- **Stage 2** (4 khối ở 28×28 , $128/128/512$): mỗi khối $768 \cdot 784 = 0,60$ M $\Rightarrow$ 2,41 M.
- **Stage 3** (6 khối ở 14×14 , $256/256/1024$): mỗi khối $1536 \cdot 196 = 0,30$ M $\Rightarrow$ 1,81 M.
- **Stage 4** (3 khối ở 7×7 , $512/512/2048$): mỗi khối $3072 \cdot 49 = 0,15$ M $\Rightarrow$ 0,45 M.

Tổng $\approx 10,1$ triệu phần tử mỗi ảnh, tức 20,2 MB ở fp16. Với batch 32: ≈ 650 MB — và đó mới chỉ là đầu ra của các tầng convolution. Trong thực tế còn phải giữ đầu ra của BatchNorm, mặt nạ của ReLU, các tensor của nhánh cộng; hệ số nhân thực nghiệm thường rơi vào khoảng hai tới ba lần, cho $\approx 1,3$ –2 GB. Đối chiếu với 410 MB của phía tham số: ở batch 32, kích hoạt chiếm gấp ba tới năm lần, và mọi biện pháp tiết kiệm bộ nhớ phải nhắm vào nó. (Hệ số hai–ba lần là kinh nghiệm thực hành, không phải con số tôi đo được cho đúng mô hình này.)

Hai quan sát rơi ra ngay từ phép tính này. Thứ nhất, *tầng chiếm nhiều kích hoạt nhất là tầng đầu chứ không phải tầng có nhiều tham số nhất*: stem một mình đã bằng gần một nửa cả stage 4 cộng lại, trong khi tham số thì tập trung gần hết ở stage 3 và 4. Thứ hai, kích hoạt tỉ lệ với $H \cdot W$. Nâng độ phân giải từ 224 lên 448 do đó nhân bộ nhớ kích hoạt lên bốn lần, trong khi tham số không đổi một byte. Đó chính là cột thứ ba của bảng ba trục ở chương 10, giờ đã thành con số.

C. Đổi tính toán lấy bộ nhớ

Khi kích hoạt là nút thắt, có một cách đổi rất sạch: *gradient checkpointing* — chỉ lưu một số điểm mốc trên đồ thị và tính lại phần ở giữa trong lượt ngược [9]. Đặt các mốc cách nhau $\sqrt{N}$ tầng thì bộ nhớ giảm từ $O(N)$ xuống $O(\sqrt{N})$. Cái giá là một lượt xuôi phụ, cỡ +30% thời gian huấn luyện. Ba biện pháp cùng họ nhắm ba phần bộ nhớ khác nhau, nên phải biết mình đang tràn vì phần nào rồi mới chọn:

- **Gradient checkpointing** — cắt phần kích hoạt, trả bằng thời gian tính lại [9].

- **Độ chính xác hỗn hợp** — cắt một nửa số byte của mỗi kích hoạt [13].
- **Phân mảnh trạng thái optimizer** — cắt phần $16N$ byte của tham số, gradient và hai moment, bằng cách chia chúng qua nhiều thiết bị [122].

→ **ràng buộc bởi $C/12$** — bộ nhớ kích hoạt chặn batch tối đa, mà batch lại thay đổi động học của tối ưu — nên một ràng buộc phần cứng đi thẳng vào chất lượng mô hình.

I 11.5 Độ trễ, thông lượng và ngân sách thời gian thực (Latency, Throughput, Real-Time Budgets)

Hai mục tiêu này xung đột nhau và hầu hết tranh cãi về “mô hình nào tốt hơn” thực ra là tranh cãi về việc đang tối ưu cái nào. **Thông lượng** (*throughput*) là số mẫu xử lý được mỗi giây khi bạn được phép gom batch lớn; **độ trễ** (*latency*) là thời gian từ lúc một khung hình tới lúc có kết quả cho *chính* khung đó. Tăng batch gần như luôn cải thiện thông lượng — vì nó nâng cường độ số học và pha loãng chi phí phóng kernel — và gần như luôn làm xấu độ trễ, vì khung đầu tiên phải chờ khung cuối cùng của batch. Cấu hình tối ưu cho huấn luyện do đó thường là cấu hình tệ nhất cho robot.

Với hệ thống thời gian thực, cách làm đúng là *chia ngân sách mili-giây trước khi thiết kế*, giống hệt cách bạn phân bổ ngân sách cho các khối của một pipeline SLAM.

→ **trường hợp riêng của $B/03$** — ngân sách mili-giây là một ràng buộc cứng, và ràng buộc cứng loại bỏ phương án trước khi độ chính xác kịp lên tiếng.

Bảng 11.2 là một cách chia ví dụ cho 30 Hz; con số cụ thể tùy hệ thống, nhưng kỷ luật thì không đổi: mỗi khối có một trần, và khối nào vượt trần phải bị cắt chứ không được “mượn” của khối khác.

Bảng 11.2. Một cách chia ngân sách 33 ms cho pipeline nhận thức 30 Hz (ví dụ minh họa, không phải số đo).

Khối	Trần (ms)	Điều xảy ra khi vượt trần
Thu ảnh + ISP	4	Không cắt được; là ràng buộc cứng của cảm biến
Tiền xử lý, chuyển bố cục	2	Gộp vào kernel đầu của mạng, hoặc làm trên GPU
Backbone	12	Trục đầu tiên bị cắt; hạ độ phân giải hoặc đổi backbone
Head + hậu xử lý	7	NMS trên CPU là thủ phạm quen thuộc
Hợp nhất, quyết định	3	Thường bị quên trong lúc đo mô hình
Dự phòng (jitter, throttling)	5	Không có nó thì hệ thống đúng 90% thời gian

Dòng cuối cùng đáng nói riêng. Trên thiết bị nhúng, năng lượng là ràng buộc thứ nhất chứ không phải thứ hai, và **giảm xung do nhiệt** (*thermal throttling*) khiến mọi phép đo ngắn nói dối: mô hình đo được 25 ms trong ba mươi giây đầu có thể thành 40 ms sau mười phút chạy liên tục trong vỏ kín. Đây là kinh nghiệm thực hành phổ biến trên Jetson và là lý do benchmark phải chạy đủ dài để nhiệt bão hòa.

→ **ràng buộc bởi $E/24$** — một phép đo tốc độ là một thí nghiệm, nên nó chịu đúng các yêu cầu về làm nóng, lặp lại và báo cáo phân vị.

Nó cũng là một phần lời giải thích vì sao nhiều phương pháp dẫn đầu bảng xếp hạng không dùng được ngoài đời: bảng xếp hạng đo trên máy chủ có tản nhiệt và điện không giới hạn.

11.6 Nút cổ chai không nằm ở GPU (Bottlenecks Outside the GPU)

Đặc thù của thị giác máy tính so với xử lý ngôn ngữ: đầu vào rất nặng và phải được giải mã. Giải một ảnh JPEG 1080p trên một lõi CPU tốn cỡ 5–15 ms, và augmentation còn tốn thêm. Với tám worker, bạn cấp được cỡ vài trăm tới hơn một nghìn ảnh mỗi giây — trong khi một GPU máy chủ nuốt ResNet-50 ở tốc độ cao hơn thế. Kết quả là GPU nhàn rỗi theo chu kỳ, và mọi giờ bạn bỏ ra tối ưu mô hình đều không thay đổi gì. Các lỗi thoát theo thứ tự chi phí tăng dần: tăng số worker và bật prefetch; chuyển augmentation lên GPU; chuyển sang định dạng dữ liệu tuần tự đọc theo khối thay vì hàng triệu file nhỏ; giải mã JPEG bằng phần cứng.

Một nút cổ chai họ hàng là đường truyền giữa host và thiết bị. Một batch 32 ảnh $3 \times 224 \times 224$ ở fp32 là 19 MB. Qua PCIe 4.0 x16 với thông lượng thực tế cỡ 25 GB/s, phép sao chép tốn khoảng 0,8 ms. Con số đó không lớn, nhưng nếu bạn quên dùng bộ nhớ ghim và sao chép bất đồng bộ thì nó nằm trọn trên đường tới hạn. Trên Jetson thì câu chuyện khác hẳn và theo hướng có lợi: CPU và GPU dùng chung bộ nhớ vật lý, nên bản sao đó có thể biến mất hoàn toàn nếu bạn cấp phát đúng loại bộ nhớ — một tối ưu không tồn tại trên máy chủ.

↔ **hệ quả của D/21** — đây đúng là loại tối ưu mà runtime triển khai khai thác, và là lý do cùng một mô hình phải được đo lại trên từng nền tảng đích.

11.7 Các con số cần thuộc lòng (Numbers Worth Memorizing)

Đây là phần duy nhất của chương thực sự là một danh sách. Nó đáng thuộc vì nó cho phép ước lượng bậc độ lớn ngay trong đầu, giữa một cuộc họp.

- **Kích thước dữ liệu.** fp32 = 4 byte, fp16/bf16 = 2, int8 = 1. Một triệu tham số fp32 = 4 MB; huấn luyện với Adam = 16 MB.
- **Công thức chi phí.** Tích chập: $FLOPs = 2k^2 C_{in} C_{out} H_{out} W_{out}$ cho mỗi ảnh. Tầng nối đầy đủ: $2 C_{in} C_{out}$. Activation tỉ lệ với $batch \times H \times W$ — nhân đôi độ phân giải là nhân bốn bộ nhớ.
- **Hằng số của máy.** Điểm gãy roofline của phần cứng hiện đại nằm trong khoảng 100–350 FLOP/byte; đọc 1 GB từ bộ nhớ tốn 0,5 ms ở 2 TB/s và 5 ms ở 200 GB/s; phóng một kernel tốn cỡ 5 μ s; PCIe 4.0 x16 cho cỡ 25 GB/s thực tế.
- **Ngưỡng chẩn đoán.** MFU dưới 30% thì gần như chắc chắn bạn *không* compute-bound, nên đừng đụng vào FLOPs. Kernel ngắn hơn 10 μ s là dấu hiệu overhead-bound.
- **Ngân sách thời gian.** 30 Hz = 33 ms mỗi khung; 60 Hz = 16,7 ms; một ảnh JPEG 1080p mất 5–15 ms để giải mã trên một lõi.

11.8 Thực hành

Repo và tài nguyên. torch.profiler kèm trace viewer của TensorBoard là điểm khởi đầu; NVIDIA Nsight Systems cho bức tranh toàn hệ thống và Nsight Compute cho từng kernel; [XuehaiPan/nvtop](#) để nhìn nhanh; [facebookresearch/fvcore](#) để đếm FLOPs; [triton-lang/triton](#) khi bạn cần tự viết kernel hợp nhất. Bài đọc ngắn gọn nhất về ba chế độ nghề là “Making Deep Learning Go Brrrr From First Principles” của Horace He. Danh sách này chốt tại tháng 9/2026; khác với các chương về mô hình, phần lớn nội dung ở đây dựa trên vật lý phần cứng nên bán rã dài hơn nhiều — chỉ các con số tuyệt đối của từng đời chip là sẽ cũ đi.

Câu hỏi kiểm tra hiểu. Mô hình của bạn đạt 30 % MFU: nghẽn ở đâu, và ba phép đo nào phân biệt được ba khả năng? Vì sao BatchNorm memory-bound trong khi convolution 3×3 compute-bound, và điều đó gợi ý gì về việc nên hợp nhất cái gì với cái gì? Nếu chuyển từ A100 sang Orin, những tối ưu nào bạn đang có sẽ mất tác dụng và những tối ưu nào trở nên đáng giá hơn?

Mini project. Hồ sơ hoá một vòng huấn luyện thật và lập bảng thời gian ở năm chỗ: dataloader, lượt xuôi, lượt ngược, bước optimizer, và đồng bộ. Yêu cầu bắt buộc: *viết dự đoán ra giấy trước khi mở profiler*, rồi ghi lại mức sai của từng dự đoán. Bản ghi mức sai đó có giá trị hơn bảng số liệu.

11.9 Giới hạn và trường hợp hỏng

Roofline là một mô hình *thượng giới hạn* và nó nói dối theo những cách có thể liệt kê được. Nó giả định phép toán chồng lấn hoàn hảo giữa tính toán và truyền dữ liệu, giả định cache không tồn tại hoặc luôn trúng, và giả định chỉ có một nút thất tại một thời điểm. Trong thực tế một kernel có thể vừa nghẽn băng thông vừa nghẽn số thanh ghi, và một mô hình có thể compute-bound ở tầng này nhưng overhead-bound ở tầng khác trong cùng một lượt. Roofline dùng để *chọn hướng điều tra*, không dùng để dự đoán con số.

Ba trường hợp hỏng cụ thể đáng cảnh giác. Thứ nhất, *đỉnh công bố không phải đỉnh đạt được*: các con số marketing thường tính ở chế độ thừa hoặc ở độ chính xác thấp nhất, và khoảng cách với thực tế có thể là hai lần — vì vậy hãy đo đỉnh của chính máy mình một lần rồi dùng con số đó mãi. Thứ hai, *benchmark ngắn nói dối* vì nhiệt chưa bão hoà, vì xung chưa ổn định, và vì lần chạy đầu tiên còn gánh chi phí biên dịch kernel; phép đo đúng phải có giai đoạn làm nóng và phải lấy phân vị chứ không chỉ lấy trung bình. Thứ ba, *đo sai chỗ*: lệnh gọi GPU là bất đồng bộ, nên đo bằng đồng hồ của CPU mà quên đồng bộ hoá sẽ cho ra những con số nhanh đến phi lý — đây là lỗi phổ biến nhất trong toàn bộ lĩnh vực đo hiệu năng học sâu.

CẠM BÃY

Trực giác về hiệu năng gần như luôn sai cho tới khi đo, kể cả trực giác của người có kinh nghiệm. Nhưng “cứ đo đi” là lời khuyên chưa đủ, vì đo mà không dự đoán trước thì bạn chỉ thu được một con số chứ không cải thiện được trực giác. Quy trình đúng có ba bước và bước đầu là bước hay bị bỏ: *viết dự đoán ra trước*, đo, rồi ghi lại mức sai. Sau mười lần như vậy bạn sẽ biết mình có thiên kiến hệ thống ở đâu — hầu hết mọi người đánh giá thấp chi phí bộ nhớ và đánh giá cao ảnh hưởng của FLOPs.

CẦN NHỚ

Một câu để mang theo: *FLOPs là thuộc tính của mô hình, thời gian là thuộc tính của cặp mô hình–máy, và cây cầu giữa hai thứ đó là cường độ số học*. Trước khi tối ưu bất cứ điều gì, hãy tính I cho khối bạn định sửa. Rồi đem nó so với điểm gãy của máy bạn đang chạy. Nếu I nằm bên trái điểm gãy, mọi nỗ lực giảm FLOPs sẽ không đem lại gì. [T2]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• FLOPs	Số phép nhân–cộng dấu phẩy động cho một lần chạy. Nó đo khối lượng công việc tính toán, không đo thời gian, và đó là toàn bộ nội dung của chương.
••• Cường độ số học	Số FLOP làm được trên mỗi byte phải chuyển giữa bộ nhớ và chip. Nó trả lời câu “mỗi byte kéo về được dùng để làm bao nhiêu việc”, và là đại lượng duy nhất phân biệt nghẽn tính toán với nghẽn bộ nhớ.
••• Điểm gãy roofline	Đỉnh tính toán chia cho băng thông, một hằng số của <i>máy</i> chứ không của mô hình. Khoảng 150 FLOP/byte trên A100 và khoảng 330 trên Jetson Orin; bên trái nó bạn nghẽn bộ nhớ, bên phải nghẽn tính toán.
•• Vì sao định fp16 công bố không dùng thẳng được?	Vì con số marketing thường tính ở chế độ thừa hoặc ở độ chính xác thấp nhất, và khoảng cách với thực tế có thể tới hai lần. Hãy đo định của chính máy mình một lần bằng một GEMM lớn, rồi dùng con số đó mãi.
••• Vì sao BatchNorm memory-bound còn convolution compute-bound?	Vì cường độ số học chênh hơn một nghìn lần: convolution 3×3 dày ở 256 kênh cho khoảng 843 FLOP/byte, còn BatchNorm trên cùng tensor chỉ khoảng 0,7. Đó là lý do trình biên dịch luôn cố gộp BatchNorm vào convolution liền trước.
••• Memory-bound	Chế độ mà thời gian chạy do lưu lượng bộ nhớ quyết định chứ không do số phép tính. Triệu chứng là GPU util cao nhưng MFU thấp; chứa bằng hợp nhất kernel và bố cục channels-last, còn giảm FLOPs thì vô ích.
•• Overhead-bound	Chế độ mà GPU chờ CPU gửi việc. Triệu chứng là util thấp với rất nhiều kernel ngắn dưới $10 \mu s$; chứa bằng CUDA graph, torch.compile hoặc tăng batch, còn đổi kiến trúc thì vô ích.
••• Vì sao depthwise cộng pointwise giảm FLOPs mà không nhanh hơn?	Vì nó giảm FLOPs 8,7 lần nhưng <i>tăng</i> lưu lượng bộ nhớ từ 4,4 MB lên 6,5 MB: hai kernel nên tensor trung gian phải ghi ra rồi đọc lại. Trên máy nghẽn bộ nhớ, thời gian đi theo byte.
•• Channels-last (NHWC)	Bố cục xếp kênh liền nhau trong bộ nhớ. Lỗi tensor chỉ được kích hoạt khi tensor ở đúng bố cục này và số kênh chia hết cho 8 ở fp16; sai một điều kiện thì kernel rơi xuống đường chậm dù FLOPs không đổi.
•• Chi phí phóng kernel	Mỗi lần CPU gửi một kernel cho GPU tốn cỡ $5\text{--}10 \mu s$, và nhiều hơn trên CPU yếu của thiết bị nhúng. Một mạng nhẹ hai trăm kernel do đó có sần thời gian 1–2 ms mà không lượng FLOPs nào ảnh hưởng tới.
••• Bộ nhớ kích hoạt	Các tensor trung gian của lượt xuôi, phải giữ lại vì lượt ngược cần tới. Nó tỉ lệ với batch và với bình phương độ phân giải, nên nó mới là thứ chặn batch chứ không phải số trọng số.
•• Trạng thái optimizer	Với Adam là hai moment, cộng với trọng số và gradient thành 16 byte cho mỗi tham số ở fp32. Phần này cố định, không phụ thuộc batch, nên nó chỉ phổ biến khi mô hình rất lớn mà rất nông.
•• Gradient checkpointing	Chỉ lưu một số điểm mốc trên đồ thị rồi tính lại phần ở giữa trong lượt ngược. Đặt mốc cách nhau $\sqrt{N}$ tầng thì bộ nhớ giảm từ $O(N)$ xuống $O(\sqrt{N})$, trả bằng cỡ +30% thời gian huấn luyện.
•• Tăng batch xong thông lượng tăng nhưng độ trễ đuôi xấu hẳn — vì sao?	Vì hai đại lượng này xung đột. Gộp batch nâng cường độ số học và pha loãng chi phí phóng kernel, nên thông lượng tăng; nhưng khung hình đầu phải chờ khung cuối của batch, nên độ trễ xấu đi. Cấu hình tốt cho huấn luyện thường là cấu hình tệ nhất cho robot.
•• Ngân sách mili-giây	Chia trần thời gian cho từng khối <i>trước</i> khi thiết kế: 33 ms cho 30 Hz, 16,7 ms cho 60 Hz. Khối nào vượt trần phải bị cắt chứ không được mượn của khối khác, và phải chừa phần dự phòng cho jitter và giảm xung.
• Giảm xung do nhiệt	Thiết bị tự hạ tần số khi nóng, nên một mô hình đo 25 ms trong ba mươi giây đầu có thể thành 40 ms sau mười phút chạy trong vỏ kín. Vì vậy benchmark phải chạy đủ dài để nhiệt bão hoà.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Vì sao dataloader có thể là nút cổ chai của cả hệ?	Giải mã một ảnh JPEG 1080p tốn 5–15 ms trên một lõi CPU, nên tám worker chỉ cấp được vài trăm tới hơn một nghìn ảnh mỗi giây. Con số đó thấp hơn tốc độ nuốt của một GPU máy chủ, và mọi tối ưu mô hình khi đó đều vô nghĩa.
●●● GPU util 95 % nhưng MFU chỉ 12 % — nghĩa là gì?	Bạn memory-bound: GPU bận, nhưng nó chờ bộ nhớ chứ không bận tính. Phân biệt với compute-bound bằng chính MFU, và phân biệt với overhead-bound bằng util — overhead-bound thì util thấp chứ không cao.
●● GPU nhàn rỗi theo chu kỳ, đỉnh CPU cao — nguyên nhân?	Nghẽn ở đường dữ liệu vào, không ở mô hình. Thêm worker và bật prefetch rồi đo lại; nếu thời gian một bước giảm theo số worker thì đúng là nhánh này.
●●● Lượt xuôi của một mạng lớn đo được 0,2 ms — lỗi ở đâu?	Bạn quên đồng bộ hoá trước khi bấm giờ. Lệnh gọi GPU là bất đồng bộ nên đồng hồ CPU trả về ngay khi lệnh được xếp hàng, chứ không phải khi phép tính xong.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
FlashAttention	Hệ quả trực tiếp của phân tích roofline: nó không giảm FLOPs mà giảm số lượt đọc/ghi bộ nhớ — đúng biện pháp của chế độ memory-bound	Ví dụ sạch nhất trong ngành về việc đo đúng đại lượng [123]
torch.compile, TensorRT, ONNX Runtime	Hợp nhất kernel và ghi lại đồ thị thành CUDA graph — biện pháp của chế độ memory-bound và overhead-bound	Hạ tầng bền; là nơi phần lớn tốc độ “miễn phí” đến từ
NVIDIA DALI và các định dạng dữ liệu tuần tự	Tồn tại vì nút cổ chai giải mã ảnh trên CPU, không vì bất kỳ lý do nào liên quan tới mô hình	Đúng nhánh trái ngoài cùng của cây chẩn đoán trong chương
Gradient checkpointing trong PyTorch và Hugging Face	Đổi một lượt xuôi phụ lấy bộ nhớ kích hoạt $O(\sqrt{N})$	Bật bằng một cờ; là lý do nhiều mô hình lớn huấn luyện được trên phần cứng nhỏ [9]
DeepSpeed ZeRO và FSDP	Cắt đúng phần 16 byte mỗi tham số của trạng thái optimizer bằng cách phân mảnh qua nhiều thiết bị	Nhắm một phần bộ nhớ khác hẳn với checkpointing — biết mình tràn vì phần nào mới chọn đúng [122]

MINI PROJECT SLAM · Roofline đo thật cho Jetson Orin của bạn, với ba khối trong pipeline SLAM · 1–2 ngày

Mục tiêu	thay mọi con số công bố trong chương này bằng con số đo được của chính máy bạn. Đáng ngờ nhất là đỉnh fp16: ở đây nó chỉ được <i>suy ra</i> từ thông số 275 TOPS INT8 thừa, chưa hề được đo. Có mái nhà đo thật rồi thì dùng nó để phân loại ba khối trong pipeline của bạn.
Dữ liệu	một chuỗi EuRoC chạy qua pipeline hiện tại; cố định xung bằng jetson_clocks và chạy đủ lâu cho nhiệt bão hoà <i>trước khi</i> bắt đầu ghi số.
Các bước	(1) đo đỉnh tính toán bằng một GEMM fp16 đủ lớn lặp nhiều lần; (2) đo bằng thông thực bằng một phép sao chép tensor lớn hơn hẳn cache; (3) vẽ mái nhà và tính điểm gãy đo được, đặt cạnh con số 332 FLOP/byte suy ra từ thông số; (4) chọn ba khối trong pipeline — ví dụ trích đặc trưng ORB trên GPU, một tầng convolution của backbone, và một bước chuẩn hoá hoặc đổi kích thước theo phần tử — rồi đếm bằng tay số FLOP và số byte của từng khối; (5) đặt ba điểm lên đồ thị và đối chiếu thời gian dự đoán với thời gian đo bằng Nsight Systems.

- Xong khi** bạn có một đồ thị roofline với đỉnh và bằng thông *đo được*, và ba khối của bạn nằm trên đó với toạ độ tính bằng tay. Mỗi khối kèm một câu kết luận dạng “khối này memory-bound ở I FLOP/byte, nên hợp nhất kernel đáng giá còn cắt FLOPs thì không”. Cuối cùng, bạn ghi được sai số giữa thời gian roofline dự đoán và thời gian đo thật.
- Nối với** [D/20](#) và [D/21](#) để biến kết luận thành hành động; [E/24](#) cho cách báo cáo đúng (làm nóng, lặp lại, phân vị thay vì trung bình); [B/10](#) nếu kết luận buộc bạn đổi backbone thay vì đổi kernel.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì FLOPs không đo thứ chi phối thời gian. Hai mô hình cùng FLOPs có thể chênh nhau hai bậc về cường độ số học. Chúng cũng có thể chênh hàng trăm lần về số kernel phải phóng, và khác nhau ở chỗ có kích hoạt được lõi tensor hay không. Ba nguyên nhân này độc lập với nhau, và không cái nào xuất hiện trong con số FLOPs.
- A2. Vì trọng số chỉ là một trong bốn phần bộ nhớ. Huấn luyện thêm gradient, thêm trạng thái optimizer (Adam giữ hai moment, tổng cộng 16 byte mỗi tham số), và quan trọng nhất là kích hoạt phải giữ lại cho lượt ngược — phần duy nhất tỉ lệ với batch và với bình phương độ phân giải.
- A3. Rằng phần lớn danh sách đó sẽ đối thứ tự. Điểm gây cao hơn nghĩa là nhiều phép toán rơi vào vùng nghẽn bộ nhớ hơn, nên hợp nhất kernel và giảm số lượt đọc/ghi trở nên đáng giá hơn, còn cắt FLOPs kém hiệu quả hơn hẳn so với trên máy chủ.
- A4. Vì có một nhánh chẩn đoán thứ tư nằm ngoài GPU: giải mã ảnh và augmentation trên CPU. Một ảnh JPEG 1080p tốn 5–15 ms trên một lõi, nên tám worker chỉ cấp được vài trăm tới hơn một nghìn ảnh mỗi giây — thấp hơn tốc độ nuốt của một GPU máy chủ.
- A5. Vì lệnh gọi GPU là bất đồng bộ: đồng hồ CPU trả về ngay khi lệnh được xếp hàng chứ không phải khi phép tính xong. Không đồng bộ hoá trước khi bấm giờ là lỗi đo phổ biến nhất trong toàn bộ lĩnh vực này.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn gọi thợ sửa ống nước. Việc thật sự chỉ mất năm phút vặn cờ lê, nhưng anh ta phải lái xe bốn mươi phút mới tới nhà bạn. Đoán hoá đơn bằng cách đếm số phút vặn cờ lê thì bạn sai gần mười lần. Thuê thợ khéo gấp đôi thì hoá đơn gần như không đổi, vì phần lớn thời gian nằm ở đường đi chứ không ở tay nghề.

Máy tính chạy mạng nơ-ron y hệt vậy. Con số mọi người hay khoe, tức số phép tính phải làm, chính là số phút vặn cờ lê. Thứ tốn thời gian là chuyển dữ liệu từ bộ nhớ vào chỗ tính, tức quãng đường lái xe. Nên phép đo đáng giá không phải “bao nhiêu việc” mà là “mỗi chuyến đi làm được bao nhiêu việc”.

Nếu mỗi chuyến chở về đủ việc làm cả buổi thì cắt bớt việc mới giúp. Nếu mỗi chuyến chỉ làm được một hai thao tác thì cắt việc chẳng đổi gì, phải gộp nhiều thao tác vào chung một chuyến. Còn nếu xe cứ đứng chờ vì bạn gọi đặt lịch từng việc một, thì phải gộp các cuộc gọi.

Cái giá là bạn buộc phải đo. Trực giác về tốc độ gần như luôn sai, và tỉ lệ giữa thời gian đi với thời gian làm khác nhau ở mỗi máy: kết luận đúng trên máy chủ có thể sai hẳn trên thiết bị nhúng.

Chỗ không rút gọn được: một phép tính có thể vừa kẹt ở đường đi vừa kẹt ở chỗ khác cùng lúc, nên bức tranh “chỉ có đúng một nút thắt” là một xấp xỉ. Nó đủ tốt để chọn hướng điều tra, nhưng không đủ để dự đoán ra con số.

Một câu: Thời gian chạy không được quyết định bởi bạn làm bao nhiêu việc, mà bởi bạn phải đi lấy dữ liệu bao nhiêu lần — nên trước khi cắt việc, hãy đếm số chuyến đi.

11.10 Kết nối

Chương này là *mặt sau* của chương 10: mọi đánh đổi kiến trúc ở đó đều có một cái giá vật lý được định lượng ở đây, và cột “độ trễ” trong bảng ba trục chỉ đọc được sau khi có khái niệm cường độ số học. Nó cũng là *trường hợp cụ thể* của nguyên lý ràng buộc cứng ở chương 3: ngân sách mili-giây và ngân sách năng lượng loại bỏ phương án trước khi độ chính xác kịp lên tiếng, và đó là lý do phần lớn thiết kế trong hệ thống nhúng trông “bảo thủ” hơn trong paper.

Đọc tiếp: [D/20-toi-uu-va-nen-mo-hinh](#) biến các quan sát ở đây thành hành động — lượng tử hoá, tĩa, chưng cất — và mỗi kỹ thuật ở đó nhắm vào một chế độ nghẽn khác nhau, nên chương này là điều kiện để chọn đúng. [D/21-inference-va-trien-khai](#) bàn về trình biên dịch và runtime, tức là nơi kernel fusion và CUDA graph thực sự xảy ra. [C/12-co-che-huan-luyen](#) giải thích vì sao batch lớn không chỉ là chuyện bộ nhớ mà còn thay đổi động học của tối ưu, nên bạn không được tự do tăng batch để lấy thông lượng. Và [E/24-thi-nghiem-va-ablation](#) dạy cách đo cho tử tế — phân vị, lặp lại, làm nóng — những thứ mà phần “giới hạn” ở trên vừa chỉ ra là bắt buộc.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao thay convolution dày bằng depthwise cộng pointwise giảm FLOPs gần chín lần mà vẫn có thể làm mô hình chậm đi, và hai con số byte nào chứng minh điều đó?
2. Điểm gãy roofline của Jetson Orin cao hơn của A100. Điều đó có nghĩa Orin “tốt hơn” ở khía cạnh nào và “tệ hơn” ở khía cạnh nào, và nó đổi hướng tối ưu của bạn ra sao?
3. Mô hình vừa khít GPU khi suy luận nhưng tràn khi huấn luyện với cùng batch: bốn thành phần bộ nhớ nào đang tồn tại, và thành phần nào chịu trách nhiệm?
4. Với ResNet-50 batch 32, vì sao tầng chiếm nhiều kích hoạt nhất lại không phải tầng có nhiều tham số nhất?
5. GPU util của bạn là 95 % nhưng MFU chỉ 12 %. Đó là chế độ nào, và vì sao “giảm FLOPs” là biện pháp sai?
6. Vì sao một benchmark 30 giây trên thiết bị nhúng có thể cho con số tốt hơn thực tế vận hành tới gần hai lần?
7. Bạn đo được lượt xuôi mất 0,2 ms trên một mạng lớn — lỗi đo nào gần như chắc chắn đã xảy ra?



THUẬT TOÁN VÀ MÔ HÌNH

TÓM TẮT

Phần C đi qua bảy họ thuật toán và mô hình, mỗi họ đọc như một chuỗi kỹ thuật ra đời đúng theo thứ tự để gỡ nút thắt của nhau: cơ chế huấn luyện, các họ backbone, ba bài toán thị giác lõi, thị giác chuỗi và video, thị giác 3D, mô hình sinh, và pretraining. Đọc xong bạn đoán được một kiến trúc lạ sẽ hỏng ở đâu mà không cần chạy thử. Nếu chỉ nhớ một điều: đừng đọc phần này như danh mục kiến trúc — hãy đọc như chuỗi vấn đề, vì chuỗi vấn đề còn giá trị lâu sau khi tên mô hình đã cũ.

Đây là phần dài nhất của cây, và cũng là phần dễ đọc sai nhất. Đọc sai nghĩa là đọc nó như một danh mục: một danh sách kiến trúc kèm năm ra đời và điểm số trên bảng xếp hạng. Đọc như vậy thì kiến thức hết hạn cùng lúc với bảng xếp hạng. Cách đọc đúng là coi mỗi họ mô hình như *một chuỗi kỹ thuật ra đời theo đúng thứ tự để gỡ nút thắt của nhau*: mỗi bước giải quyết một khiếm khuyết cụ thể của bước trước và gần như luôn tạo ra một vấn đề mới, và chính chuỗi vấn đề đó mới là thứ còn giá trị sau mười năm.

Bảy chương được sắp theo thứ tự phụ thuộc chứ không theo độ hấp dẫn. Chương 12 là cơ chế huấn luyện, đi từ vi mô tới vĩ mô, và là chương quyết định nhiều nhất tới năng lực thực hành: nếu không hiểu vì sao gradient chết và vì sao learning rate là siêu tham số quan trọng nhất thì mọi kiến trúc phía sau chỉ là hộp đen. Chương 13 là các họ backbone, đọc như ba mạch song song — dòng CNN, dòng chuỗi tới attention, và ViT cùng bài học của nó về inductive bias. Chương 14 và 15 áp các backbone đó vào các lớp bài toán thật, trước là ảnh tĩnh rồi tới chuỗi và video, nơi thời gian thêm vào một chiều ràng buộc mới. Chương 16 là thị giác 3D, nơi hình học cổ điển và học sâu gặp nhau trên cùng một bài toán qua bốn thế hệ. Chương 17 là các mô hình sinh và low-level vision, chứa một trong vài kết quả *có tính lý thuyết* hiếm hoi của cả cây là trade-off giữa perceptual và distortion. Chương 18 khép lại bằng câu chuyện thống nhất phía sau mười năm qua: nguồn giám sát ngày càng rẻ, từ nhãn người gán tới chính bức ảnh, tới văn bản đi kèm, tới việc ghép thị giác vào một mô hình ngôn ngữ đã biết suy luận.

Xuyên suốt bảy chương, hãy giữ hai câu hỏi trong đầu ở mỗi kỹ thuật mới: *nó giả định gì, và độ khó đã bị đẩy đi đâu*. Khi một phương pháp trông như bữa trưa miễn phí thì gần như chắc chắn là bạn chưa tìm ra chỗ trả giá.

Bảng 11.5. Bản đồ Phần C: các chương, nội dung, và mức đầu tư đề xuất.

Ch.	Thư mục	Nội dung	Mức
12	12-co-che-huan-luyen	Bốn mục: cơ học của gradient; làm cho training chạy được; tối ưu hoá; tổng quát hoá và chẩn đoán	cốt lõi
13	13-kien-truc-backbone	Ba mạch: dòng CNN; từ chuỗi tới attention; ViT và bài học inductive bias	cốt lõi
14	14-bai-toan-cv-loi	Bốn mục: detection hai giai đoạn, một giai đoạn, segmentation, các bài toán lân cận	cốt lõi
15	15-thi-giac-khong-thoi-	Ba mục: đầu ra độ dài thay đổi; biểu diễn video; flow, tracking và ràng buộc streaming	cốt lõi
16	16-thi-giac-3d	Ba mục: bốn thế hệ 3D; biểu diễn và depth và SLAM; NeRF, 3DGS, hợp nhất cảm biến	cốt lõi
17	17-mo-hinh-sinh	Bốn mục: bốn họ sinh; tối ưu trên pixel; diffusion; low-level vision	tham khảo có chọn lọc
18	18-pretraining-va-found	Hai mục: nguồn giám sát ngày càng rẻ; kinh tế học của pretraining	cốt lõi

Chương 12 trước hết và đọc kỹ nhất. Chương 13 là nền cho 14 tới 16. Chương 17 đọc theo nhu cầu. Chương 18 nên đọc sớm hơn vị trí của nó nếu bạn đang phải chọn giữa train từ đầu và dùng mô hình có sẵn.

Cơ chế huấn luyện, từ vi mô tới vĩ mô (Training Mechanics, from Micro to Macro)

PHỤ THUỘC A/01-toan-toi-thieu (quy tắc chuỗi, phương sai, số điều kiện); B/09-thiet-ke-ham-mat-mat (biết mất mát là gì trước khi hỏi cách tối thiểu hoá nó). Chương này là tiền đề cho toàn bộ Phần C: mọi kiến trúc ở các chương sau đều được huấn luyện bằng bộ máy mô tả ở đây.

TÓM TẮT

Cả chương là câu trả lời cho *một* câu hỏi: làm sao đưa **gradient** đi qua một mạng sâu mà không làm nó chết hoặc nổ. Gần như mọi thủ thuật trong một file cấu hình huấn luyện — từ cách khởi tạo, chọn hàm kích hoạt, đặt normalization, tối warmup và **mixed precision** — tồn tại vì lý do đó. Đọc xong bốn mục, bạn tự viết được lượt ngược cho một tầng mới và đặt được siêu tham số có căn cứ. Bạn cũng chẩn đoán được một lần chạy hỏng trong vài phút, và đưa được mô hình lên nhiều GPU đúng thứ tự. Nếu chỉ nhớ một điều: đây là chương duy nhất trong cây mà kỹ năng chỉ hình thành qua tay — đọc hiểu hết mà chưa từng cố ý làm hỏng một lần chạy thì coi như chưa học.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model)** — một hàm có tham số, nhận đầu vào và trả về dự đoán; “huấn luyện” nghĩa là đi tìm bộ tham số làm dự đoán sát nhất.
- **tham số (parameter, weight)** — các con số bên trong mô hình mà huấn luyện được phép thay đổi; một mạng thị giác cỡ vừa có hàng chục triệu con số như vậy.
- **hàm mất mát (loss function)** — hàm biến dự đoán và nhãn thành một số duy nhất đo mức sai; toàn bộ huấn luyện là bài toán làm giảm số đó.
- **gradient** — vectơ đạo hàm riêng của hàm mất mát theo từng tham số; nó chỉ hướng làm mất mát TĂNG nhanh nhất, nên ta đi ngược lại.
- **batch và epoch** — “batch” là nhóm mẫu đi qua mạng cùng lúc; một “epoch” là khi mô hình đã đi hết tập huấn luyện một lượt. Một epoch gồm nhiều bước cập nhật, mỗi bước một batch.
- **learning rate** — độ dài bước mà thuật toán tối ưu đi theo hướng ngược gradient ở mỗi lần cập nhật; đây là siêu tham số quan trọng nhất trong cả chương.
- **kích hoạt và hàm kích hoạt** — “kích hoạt” là giá trị đầu ra của một tầng giữa mạng; “hàm kích hoạt” là phép phi tuyến sinh ra nó, ví dụ ReLU. Tên gần giống nhưng là hai thứ khác nhau.
- **normalization** — phép chuẩn hoá lại phân bố của kích hoạt ở giữa mạng về trung bình 0 độ lệch chuẩn 1; nó tồn tại để giữ tín hiệu không trôi khi đi qua nhiều tầng.
- **overfitting** — mô hình khớp rất tốt dữ liệu huấn luyện nhưng kém trên dữ liệu mới; đây là chủ đề của mục 4.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao một mạng 10^9 tham số huấn luyện được trong vài ngày, trong khi vi phân thuận cần 10^9 lượt chạy để có cùng lượng thông tin?
- Q2. Nếu gradient chỉ là quy tắc chuỗi — thứ đã có từ thế kỷ XVII — vì sao suốt hai mươi năm người ta không huấn luyện nổi mạng sâu?

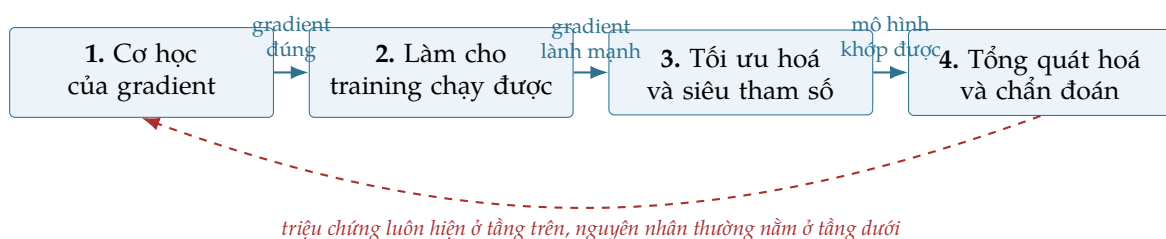
- Q3.** Bầy *optimizer* quen thuộc thường được dạy như bầy lựa chọn trong một menu. Vì sao thực ra chúng là bầy liên tiếp, và cái nào sinh ra chỉ để sửa lỗi do chính cái trước tạo ra?
- Q4.** Một mạng đủ sức học thuộc cả một tập ảnh với nhãn xáo ngẫu nhiên — vậy vì sao nó vẫn tổng quát hoá tốt khi nhãn là chân trị?
- Q5.** *bf16* có dải động rộng đúng bằng *fp32*. Vậy vì sao huấn luyện ở *bf16* vẫn buộc phải giữ một bản trọng số *fp32*?

12.1 Vì sao chương này chia làm bốn mục

Bốn mục dưới đây không phải bốn chủ đề song song. Chúng là bốn tầng của cùng một câu hỏi, xếp từ vi mô tới vĩ mô. Và *mỗi tầng chỉ có nghĩa khi tầng dưới nó đã đúng*. Đó là lý do thứ tự đọc ở đây là bắt buộc chứ không phải gợi ý.

- 1. Cơ học của gradient** hỏi: gradient được tính thế nào, và vì sao cách tính đó rẻ. Đây là tầng vi mô nhất — một phép toán, một tầng, một đồ thị. Không nắm tầng này thì ba tầng sau chỉ là các quy tắc phải học thuộc.
- 2. Làm cho training chạy được** hỏi: gradient đúng rồi thì làm sao nó sống sót qua năm mươi tầng. Ở đây xuất hiện đại lượng trung tâm của cả chương — hệ số nhân của tích các Jacobian — và ba hướng tấn công vào nó.
- 3. Tối ưu hoá và siêu tham số** giả định gradient đã lành mạnh và hỏi tiếp: nên đi bước dài bao nhiêu, theo hướng nào, ở độ chính xác số nào. Đây là tầng mà bầy *optimizer* quen thuộc hiện ra như một chuỗi vá lỗi chứ không như một menu.
- 4. Tổng quát hoá và chẩn đoán** là tầng vĩ mô nhất: mô hình chạy tốt trên dữ liệu huấn luyện rồi, còn ngoài đời thì sao, và khi mọi thứ không như kỳ vọng thì tìm nguyên nhân theo trình tự nào.

Hình 12.1 cho thấy vì sao thứ tự này không đảo được. Mỗi tầng tiêu thụ đúng thứ mà tầng dưới sản xuất ra. Một lỗi ở tầng dưới lại luôn hiện ra thành triệu chứng ở tầng trên. Đó chính là lý do người ta hay tune learning rate để chữa một bug ở lượt ngược.



Hình 12.1. Bốn tầng của chương, và chiều mà lỗi lan truyền. Mũi tên liền là quan hệ “tầng sau giả định tầng trước đã đúng”; mũi tên đứt là điều khiến việc gỡ lỗi khó: một bug ở tầng 1 hầu như luôn biểu hiện thành một đường cong xấu ở tầng 4.

Bản đồ chương 12 — Cơ chế huấn luyện

TÓM TẮT

Chương 12 gồm bốn mục và chúng *không* song song: đây là bốn tầng của cùng một câu hỏi, xếp từ vi mô tới vĩ mô, mỗi tầng giả định tầng dưới đã đúng. Đọc theo đúng thứ tự 1 → 2 → 3 → 4; đọc nhảy cóc thì mục 3 và mục 4 biến thành danh sách quy tắc phải học thuộc. Nếu chỉ nhớ một điều: câu hỏi trung tâm của cả chương là *làm sao đưa gradient đi qua một mạng sâu mà không làm nó chết hoặc nổ*. Mọi kỹ thuật trong chương là một câu trả lời cho đúng câu hỏi đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **gradient** — vectơ đạo hàm riêng của hàm mất mát theo từng tham số; nó chỉ hướng làm mất mát TĂNG nhanh nhất, nên ta đi ngược lại.
- **lan truyền ngược (backpropagation)** — thuật toán tính gradient bằng cách đi ngược đồ thị phép tính một lượt duy nhất; nội dung của mục 1.
- **siêu tham số (hyperparameter)** — con số do người đặt trước khi huấn luyện chứ không học từ dữ liệu — learning rate, batch size, cường độ weight decay; nội dung của mục 3.
- **tổng quát hoá (generalization)** — khả năng mô hình chạy tốt trên dữ liệu chưa từng thấy, chứ không chỉ trên dữ liệu đã huấn luyện; nội dung của mục 4.

Có gì trong chương này

Bảng 12.1. Bốn mục của chương 12: câu hỏi trung tâm, phụ thuộc, và mức độ đầu tư.

Mục	Câu hỏi trung tâm	Phụ thuộc	Mức độ
1. Cơ học của gradient	Gradient được tính thế nào, vì sao cách đó rẻ, và nó tồn bộ nhớ ở đâu?	A/01	Cốt lõi [T1]
2. Làm cho training chạy được	Gradient đúng rồi thì làm sao nó sống sót qua năm mươi tầng?	Mục 1	Cốt lõi [T1]
3. Tối ưu hoá và siêu tham số	Đi bước dài bao nhiêu, theo hướng nào, ở độ chính xác số nào?	Mục 2	Cốt lõi [T2]
4. Tổng quát hoá và chẩn đoán	Mô hình khớp được dữ liệu rồi, còn ngoài đời thì sao — và hỏng thì tìm ở đâu?	Mục 3, B/07	Cốt lõi [T2]

Vì sao lại chia như vậy

Cách chia này theo *độ trừu tượng của đối tượng được nói tới*, chứ không theo thứ tự lịch sử hay theo mức độ phổ biến. Mục 1 nói về một phép toán và một đồ thị; mục 2 nói về một mạng; mục 3 nói về một quá trình huấn luyện; mục 4 nói về quan hệ giữa quá trình đó và thế giới bên ngoài. Lợi ích của cách chia này là mỗi mục có một đại lượng trung tâm rõ ràng. Lần lượt: tích vector-Jacobian, hệ số nhân của tích các Jacobian, độ dài bước đi, và khoảng cách train-val. Gặp một kỹ thuật lạ, bạn biết ngay nó thuộc tầng nào.

Cách chia này cũng làm lộ ra một quan hệ mà cách chia theo chủ đề che mất: *triệu chứng luôn xuất hiện ở tầng trên, nguyên nhân thường nằm ở tầng dưới*. Một lỗi ở lượt ngược (tầng 1) biểu hiện thành một đường cong học xấu (tầng 4). Đây là lý do mục 4 dành phần lớn dung lượng cho một cây chẩn đoán đi từ trên xuống. Thấy một đường cong lạ thì phản xạ đúng là đi xuống, không đi ngang.

Đọc theo thứ tự nào

1. Nếu bạn mới bắt đầu: đọc tuần tự $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$, và làm mini project của chương *ngay sau mục 4* chứ không để dành — nó là chỗ kiến thức chuyển thành phản xạ.
2. Nếu bạn đã huấn luyện mạng vài lần và muốn lấp lỗ hổng: đọc mục 3 và mục 4 trước, quay lại mục 1 khi gặp câu hỏi về bộ nhớ hoặc khi phải viết một tầng mới.
3. Nếu bạn đang gỡ một lần chạy hỏng *ngay bây giờ*: vào thẳng cây chẩn đoán và bảng triệu chứng ở mục 4, rồi đi ngược lên tầng mà cây chỉ tới.

Cốt lõi và tham khảo

Cả bốn mục đều thuộc phần cốt lõi của cây, đây là một trong ít chương như vậy. Trong nội bộ chương, ba phần đáng quay lại nhiều lần là: ba mẫu gradient cộng quy tắc rẽ nhánh (mục 1), lập luận phương sai dẫn tới khởi tạo He (mục 2), và mạch bảy bước của optimizer (mục 3). Ba phần có thể đọc lướt lần đầu và tra lại khi cần là: chi tiết công thức lượt ngược của BatchNorm, bảng bốn biến thể normalization, và bảng bốn mức song song hoá dữ liệu.

Nối vào phần còn lại của cây

Chương này tiêu thụ [A/01-toan-toi-thieu](#) và [B/09-thiet-ke-ham-mat-mat](#); nó cấp đầu vào cho *tần bộ* Phần C, vì mọi kiến trúc ở các chương sau đều được huấn luyện bằng bộ máy này. Ba nhánh nối ngang đáng nhớ: chi phí bộ nhớ kích hoạt nối sang [B/11-gioi-han-tinh-toan](#); dải động của fp16 và bf16 nối sang [D/20-toi-uu-va-nen-mo-hinh](#); và nhận xét “phần lớn cải tiến kiến trúc hoá ra là cải tiến công thức huấn luyện” nối sang [E/24-thi-nghiem-va-ablation](#).

12.2 Cơ học của gradient (Mechanics of the Gradient)

PHỤ THUỘC [A/01-toan-toi-thieu](#) (quy tắc chuỗi, Jacobian, chuẩn ma trận); hữu ích nhưng không bắt buộc: [B/11-gioi-han-tinh-toan](#) nếu muốn hiểu vì sao chi phí bộ nhớ ở cuối mục này lại là ràng buộc thật chứ không phải chi tiết phụ.

TÓM TẮT

Mục này giải thích vì sao thứ tự áp dụng quy tắc chuỗi quyết định chi phí huấn luyện, và vì sao **lan truyền ngược** (*backpropagation*) rẻ hơn vì phân thuận khoảng tám bậc độ lớn trên một mạng thật. Đọc xong bạn tự viết được lượt ngược cho một tầng bất kỳ. Bạn cũng tự bắt được lỗi của nó bằng kiểm tra gradient số, và ước lượng được bộ nhớ mà kiến trúc sẽ ngốn. Nếu chỉ nhớ một điều: lan truyền ngược không phải công thức riêng của mạng nơ-ron, nó là thuật toán vi phân cho mọi đồ thị khả vi, nên mọi kiến trúc lạ chỉ còn là một đồ thị mới.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model)** — một hàm có tham số, nhận đầu vào và trả về dự đoán; “huấn luyện” nghĩa là đi tìm bộ tham số làm dự đoán sát nhãn nhất.
- **tham số (parameter, weight)** — các con số bên trong mô hình mà quá trình huấn luyện được phép thay đổi; một mạng thị giác cỡ vừa có hàng chục triệu con số như vậy.
- **bias** — **phân biệt hai nghĩa** — trong công thức $Wx + b$, b là *tham số cộng* của một tầng. Từ “bias” trong thống kê lại nghĩa là *độ lệch hệ thống* của một bộ ước lượng. Mục này chỉ dùng nghĩa thứ nhất.

- **hàm mất mát (loss function)** — hàm biến dự đoán và nhả thành một số duy nhất đo mức sai; mọi thứ trong huấn luyện đều nhằm giảm số đó.
- **gradient** — vectơ đạo hàm riêng của hàm mất mát theo từng tham số; nó chỉ hướng làm mất mát TĂNG nhanh nhất, nên ta đi ngược lại.
- **Jacobian** — ma trận đạo hàm riêng của một hàm nhiều đầu vào, nhiều đầu ra; nó là dạng tổng quát của gradient và là đối tượng chính mà lượt ngược thao tác.
- **tensor** — mảng số nhiều chiều — một ảnh RGB là tensor ba chiều, một lô ảnh là tensor bốn chiều; đây là đơn vị dữ liệu mà mọi phép toán trong mạng làm việc với.
- **batch (lô)** — nhóm mẫu được đưa qua mạng cùng lúc để tận dụng phần cứng song song; kích thước lô ảnh hưởng tới cả tốc độ lẫn chất lượng, và ta sẽ thấy vì sao ở các mục sau.
- **kích hoạt (activation)** — giá trị đầu ra của một tầng ở giữa mạng, tức dữ liệu đang trên đường đi; đừng nhầm với “hàm kích hoạt” là phép phi tuyến sinh ra nó.

A. Đặt vấn đề

Một mạng hiện đại là hàm hợp của vài nghìn phép toán sơ cấp. Nó phụ thuộc vào 10^7 đến 10^9 tham số vô hướng. Mỗi bước huấn luyện lại cần đạo hàm của *một* số duy nhất — giá trị **hàm mất mát (loss function)** — theo *toàn bộ* chỗ tham số đó. Câu hỏi không phải “quy tắc chuỗi có áp dụng được không”, vì nó luôn áp dụng được. Câu hỏi là: áp dụng quy tắc chuỗi *theo thứ tự nào* để chi phí là một lượt đi thay vì một trăm triệu lượt. Toàn bộ nội dung của **lan truyền ngược (backpropagation)** nằm trong câu hỏi thứ tự đó, chứ không nằm trong bản thân đạo hàm.

Bạn đã gặp mọi nguyên liệu của bài toán này ở phía tối ưu hoá phi tuyến trong SLAM: dựng Jacobian, nhân dồn các Jacobian cục bộ, giải hệ chuẩn tắc. Nhưng hình dạng bài toán ở đó ngược hẳn. Trong bundle adjustment, số dư (*residual*) rất nhiều còn tham số tương đối ít, nên việc dựng tường minh ma trận J rồi làm việc với $J^T J$ là hợp lý. Trong huấn luyện mạng, đầu ra chỉ có *một* chiều còn đầu vào có hàng trăm triệu chiều. **Forward-mode autodiff** lan truyền một đạo hàm theo hướng: mỗi lượt cho bạn đạo hàm theo đúng một hướng đầu vào, nên cần 10^8 lượt. **Reverse-mode autodiff** lan truyền độ nhạy ngược từ đầu ra: mỗi lượt cho bạn đạo hàm theo *tất cả* đầu vào, nên cần đúng một lượt cho mỗi đầu ra vô hướng. Tỉ số 10^8 giữa hai cách là lý do tồn tại của lan truyền ngược [8]. Cùng một quy tắc chuỗi, hai thứ tự áp dụng, chi phí lệch nhau tám bậc độ lớn.

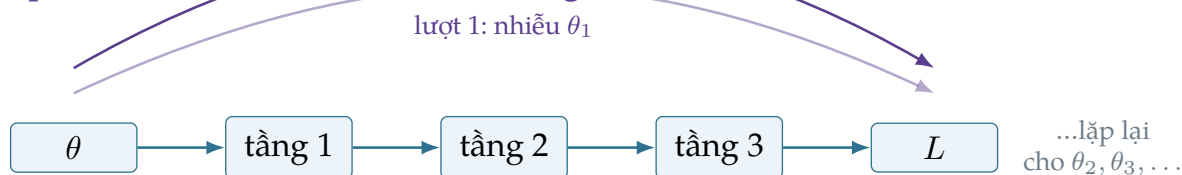
Mục này cho bạn ba việc cụ thể. Bạn tự viết được lượt ngược cho một tầng chưa có trong thư viện. Bạn tự bắt được lỗi của nó bằng kiểm tra số. Và bạn ước lượng được bộ nhớ mà một kiến trúc sẽ ngốn. Trước đó cả ba đều phải tin vào framework.

Bảng 12.2. Hai chế độ vi phân tự động. Chúng dùng đúng cùng một quy tắc chuỗi và chỉ khác nhau ở thứ tự nhân các Jacobian, nhưng thứ tự đó quyết định toàn bộ chi phí.

Chế độ	Cơ chế	Chi phí cho n đầu vào, m đầu ra	Hỏng / không dùng khi
Forward-mode	Lan truyền đạo hàm theo một hướng đầu vào, cùng chiều lượt xuôi	n lượt; bộ nhớ hằng số	n lớn — huấn luyện mạng có 10^8 tham số là bất khả thi
Reverse-mode	Lan truyền độ nhạy của một đầu ra, ngược chiều lượt xuôi	m lượt; bộ nhớ tỉ lệ tổng kích hoạt	m lớn (cần cả Jacobian đầy đủ), hoặc bộ nhớ không đủ giữ kích hoạt

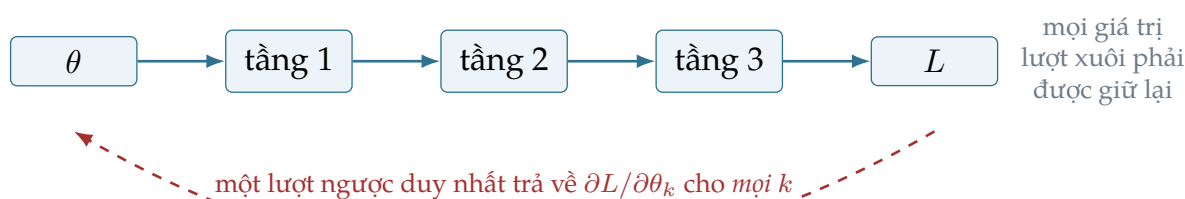
Bảng 12.2 nói gọn lại: huấn luyện là bài toán “một đầu ra, rất nhiều đầu vào”, và đó chính xác là hình dạng mà reverse-mode được sinh ra để phục vụ.

Vi phân thuận: một lượt cho *mỗi* hướng đầu vào



Chi phí: n lượt cho n tham số — với $n = 10^8$ là bất khả thi. Bộ nhớ: hằng số.

Vi phân ngược: một lượt xuôi, rồi *một* lượt ngược cho tất cả



Chi phí: 1 lượt — nhưng bộ nhớ tỉ lệ với tổng kích hoạt của cả mạng.

Hình 12.2. Cùng một quy tắc chuỗi, hai thứ tự nhân, hai chi phí lệch nhau tám bậc độ lớn. Vi phân thuận đi cùng chiều lượt xuôi nên mỗi lượt chỉ trả lời được cho một hướng đầu vào; vi phân ngược đi ngược lại nên một lượt phủ mọi tham số. Điểm đáng nhớ nằm ở dòng cuối mỗi nửa hình: cái ta tiết kiệm được ở thời gian phải trả lại bằng bộ nhớ.

Hình 12.2 cho thấy điều mà bảng không nói được: hai chế độ dùng *cùng* một đồ thị và cùng các Jacobian cục bộ, chỉ khác chiều duyệt — và chính chiều duyệt sinh ra cả lợi thế lẫn cái giá.

Nhận thức then chốt cần mang ra khỏi mục này là: lan truyền ngược không phải công thức riêng của mạng nơ-ron. Nó là thuật toán chung cho *mọi* đồ thị có hướng không chu trình mà mỗi nút khả vi. Khi bạn đã tin điều đó, một kiến trúc lạ không còn là một thứ phải học thuộc; nó chỉ là một đồ thị mới, và bạn đã biết cách vi phân mọi đồ thị.

TRỰC GIÁC

Hãy nghĩ về forward pass như một dây chuyền sản xuất: nguyên liệu đi qua ba mươi trạm, mỗi trạm biến đổi một chút, cuối dây chuyền ra một sản phẩm có một con số sai lệch so với chuẩn. Forward-mode giống việc rung nhẹ *một* nút vặn rồi chạy lại cả dây chuyền để xem sai lệch đổi bao nhiêu. Cách đó đúng, nhưng phải lặp lại đúng bằng số nút vặn. Reverse-mode làm ngược: bắt đầu từ sản phẩm lỗi, hỏi trạm cuối “một đơn vị sai ở đây tương ứng với bao nhiêu sai ở đầu vào của anh”, rồi chuyển đúng câu hỏi đó ngược lên trạm trước. Một lượt đi ngược duy nhất trả lời cho mọi nút vặn trên toàn dây chuyền, vì mỗi trạm chỉ cần biết *quy tắc quy đổi cục bộ* của riêng nó.

B. Đồ thị tính toán và tích vector-Jacobian (computational graph, VJP)

Hình thức hoá của ẩn dụ trên rất gọn. Biểu diễn phép tính thành một đồ thị: nút là giá trị trung gian, cạnh là phụ thuộc. Lượt xuôi duyệt đồ thị theo thứ tự tô-pô và tính giá trị; lượt ngược duyệt theo thứ tự tô-pô đảo và tính *adjoint* $\bar{v} = \partial L / \partial v$ cho từng nút. Mỗi nút chỉ cần biết một thứ: cách biến adjoint ở đầu ra của nó thành adjoint ở đầu vào của nó. Nếu nút thực hiện $y = g(x)$ thì quy

tắc đó là

$$\bar{x} = \left(\frac{\partial y}{\partial x} \right)^\top \bar{y},$$

tức là một **tích vector-Jacobian** (*vector-Jacobian product, VJP*). Điểm quan trọng về mặt kỹ thuật là ta *không bao giờ* dựng ma trận Jacobian ở giữa. Xét một tầng affine $y = Wx + b$ với $W \in \mathbb{R}^{4096 \times 4096}$. Jacobian theo W là ma trận $4096 \times 16,8$ triệu, khoảng $6,9 \cdot 10^{10}$ phần tử. Không thể lưu nổi. Nhưng VJP của nó chỉ là ba biểu thức tầm thường:

$$\bar{x} = W^\top \bar{y}, \quad \bar{W} = \bar{y} x^\top, \quad \bar{b} = \bar{y},$$

trong đó $\bar{W}$ là một tích ngoài với đúng 16,8 triệu phần tử — bằng kích thước của chính W . Tỉ số 4096 giữa “dựng Jacobian” và “áp dụng Jacobian” là lý do mọi framework *autodiff* đều cài đặt VJP chứ không cài đặt đạo hàm theo nghĩa ma trận. Với một batch X xếp theo hàng, quy tắc trên trở thành $\bar{W} = \bar{Y}^\top X$, và bạn nên coi đây là công thức phải viết lại được từ đầu bất cứ lúc nào.

↔ **trường hợp riêng của A/01** — lan truyền ngược chỉ là quy tắc chuỗi của giải tích nhiều biến, áp dụng theo thứ tự làm chi phí rẻ nhất

↔ **tiền đề C/12/02** — chính tích các Jacobian cục bộ ở đây là đại lượng mà ba hướng chống gradient tiêu biến nhắm vào

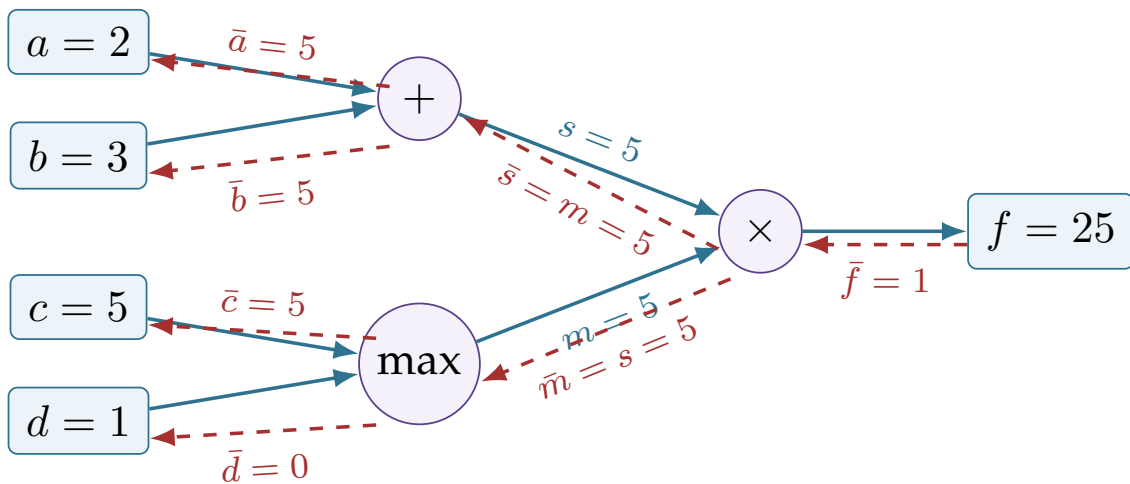
C. Ba mẫu gradient lặp lại (the three gradient patterns)

Nghe qua thì việc phải derive backward cho từng loại tầng là công việc vô tận. Thực tế thì không, vì gần như mọi đồ thị đều được ghép từ ba mẫu, và ba mẫu đó có tên gọi trực quan.

- **Cộng là bộ phân phối** (*add = distributor*) — nếu $y = u + v$ thì $\bar{u} = \bar{v} = \bar{y}$. Nói gọn: adjoint đi qua nút cộng mà không đổi giá trị, chỉ nhân bản ra mọi nhánh.
- **Nhân là bộ hoán đổi** (*multiply = swapper*) — nếu $y = uv$ thì $\bar{u} = v \bar{y}$ và $\bar{v} = u \bar{y}$. Mỗi nhánh nhận adjoint nhân với giá trị của *nhánh kia*; đây chính là lý do lượt ngược buộc phải nhớ các giá trị của lượt xuôi, và do đó là gốc rễ của chi phí bộ nhớ ở mục G.
- **Max là bộ định tuyến** (*max = router*) — nếu $y = \max(u, v)$ thì toàn bộ adjoint đi về nhánh thắng, nhánh thua nhận 0. Nói gọn: max không trộn, nó chọn đường.
- **Rẽ nhánh là bộ cộng dồn** (*fan-out = accumulator*) — mẫu thứ tư, ít được nêu tên nhưng gây nhiều lỗi nhất. Nếu một giá trị được dùng ở hai chỗ trong đồ thị, adjoint của nó là *tổng* các adjoint quay về từ hai chỗ đó.

Mẫu thứ tư đáng được nói thêm một câu vì hệ quả thực hành của nó rất rộng: *lượt ngược của broadcasting là một phép cộng theo đúng trục đã được broadcast*. Cộng một vector bias shape $(C,)$ vào một *tensor* shape (N, C, H, W) chính là rẽ nhánh giá trị bias ra $N \cdot H \cdot W$ chỗ, nên $\bar{b} = \sum_{n,h,w} \bar{y}_{n,c,h,w}$. Một mình quy tắc này giải thích phần lớn lỗi shape mà người mới gặp khi tự viết lượt ngược.

Hình 12.3 chạy cả ba mẫu trên một ví dụ đủ nhỏ để tính tay. Với $f = (a + b) \cdot \max(c, d)$ và $a = 2, b = 3, c = 5, d = 1$, lượt xuôi cho $s = 5, m = 5, f = 25$. Lượt ngược đặt $\bar{f} = 1$; nút nhân hoán đổi nên $\bar{s} = m = 5$ và $\bar{m} = s = 5$; nút cộng phân phối nên $\bar{a} = \bar{b} = 5$; nút max định tuyến nên $\bar{c} = 5$ và $\bar{d} = 0$. Kiểm tra bằng trực giác: tăng d một chút không đổi gì cả vì d đang thua, đúng như $\bar{d} = 0$; nhưng nếu d vượt c thì đạo hàm nhảy bậc, và đó chính là chỗ kiểm tra gradient bằng số sẽ báo động giả ở phần sau.



Hình 12.3. Ba mẫu gradient trên một đồ thị bốn nút. Mũi tên liền là lượt xuôi, mũi tên đứt là lượt ngược. Nút cộng phân phối adjoint, nút nhân hoán đổi giá trị hai nhánh, nút max định tuyến toàn bộ adjoint về nhánh thắng và cắt nhánh thua về 0.

D. Bốn backward đáng tự viết một lần trong đời

Việc tự derive gradient cho vài tầng cụ thể là bài tập kiểu CS231n, và nó nằm ở tầng bán rã [T2]–[T3]: không phải kiến thức vĩnh cửu, nhưng cũng không phải chi tiết framework sẽ mất giá sau một năm. Giá trị của nó là sau khi làm xong, bạn đọc bất kỳ tầng mới nào cũng tự hỏi “VJP của cái này là gì” thay vì “thư viện có hỗ trợ không”.

Tầng affine đã nêu ở trên. **ReLU** là mẫu max trá hình: $\bar{x} = \bar{y} \cdot \mathbb{I}[x > 0]$, và lượt ngược chỉ cần một mặt nạ bit. **Softmax ghép cross-entropy** là ví dụ đẹp nhất về việc ghép hai phép toán làm backward súp đổ: tính riêng thì Jacobian của softmax là $\text{diag}(p) - pp^\top$, một ma trận $K \times K$ phải nhân với adjoint; ghép lại thì tất cả rút gọn thành $\bar{z} = p - y$, vừa rẻ hơn vừa ổn định hơn về số học. **Max pooling** định tuyến adjoint về vị trí argmax nên phải lưu chỉ số argmax từ lượt xuôi — một chi phí bộ nhớ thật — trong khi average pooling chỉ phân phối $1/k^2$ và không cần nhớ gì.

BatchNorm là bài khó nhất và cũng bổ ích nhất, vì nó là tầng đầu tiên mà gradient của một mẫu phụ thuộc vào *mọi mẫu khác trong batch*. Chuỗi phép toán là trung bình $\rightarrow$ phương sai $\rightarrow$ chuẩn hoá $\rightarrow$ scale-and-shift, và vì μ lẫn σ^2 đều là hàm của cả batch nên adjoint chạy ngược qua ba đường song song rồi cộng lại. Kết quả gọn có dạng

$$\bar{x}_i = \frac{\gamma}{N\sigma} \left(N \bar{\hat{x}}_i - \sum_j \bar{\hat{x}}_j - \hat{x}_i \sum_j \bar{\hat{x}}_j \hat{x}_j \right),$$

với $\hat{x}$ là giá trị đã chuẩn hoá. Hai tổng trong ngoặc chính là hai đường phụ thuộc qua μ và qua σ^2 . Công thức này giải thích luôn một câu hỏi hay bị hỏi: vì sao BatchNorm làm lượt ngược tốn thêm bộ nhớ. Vì để tính được nó, lượt xuôi phải giữ lại $\hat{x}$ và σ cho mỗi tầng BN, chứ không thể vứt đi sau khi dùng.

↪ **ràng buộc bởi C/12/04** — việc gradient một mẫu phụ thuộc cả batch là gốc của mọi rắc rối BatchNorm trong huấn luyện phân tán

E. Kiểm tra gradient bằng số (numerical gradient check)

Derive tay thì sai lúc nào không biết, và điều tệ là một gradient sai *vẫn làm loss giảm* — chỉ là giảm chậm hơn hoặc hội tụ về chỗ khác. Nên cần một phép kiểm tra độc lập. Công cụ là **sai phân hữu**

hạn (*finite difference*) đối xứng

$$\frac{\partial f}{\partial x_k} \approx \frac{f(x + he_k) - f(x - he_k)}{2h},$$

chứ không phải sai phân một phía. Lý do là khai triển Taylor: sai phân một phía có sai số bậc $O(h)$, sai phân đối xứng triệt tiêu số hạng bậc hai nên sai số bậc $O(h^2)$. Với $h = 10^{-5}$, đó là chênh lệch giữa sai số cỡ 10^{-5} và cỡ 10^{-10} — đủ để phân biệt “code sai” với “xấp xỉ số”.

Tiêu chí so sánh phải là sai số tương đối $|a - n| / \max(|a|, |n|)$ chứ không phải sai số tuyệt đối, vì độ lớn gradient thay đổi hàng chục bậc giữa các tầng. Kinh nghiệm thực hành phổ biến: dưới 10^{-7} là tốt, quanh 10^{-4} là đáng nghi trừ khi hàm có điểm gãy, trên 10^{-2} gần như chắc chắn sai. Ba điều kiện phải thoả thì con số đó mới có nghĩa. Thứ nhất, phải tính bằng *double*: với float32, epsilon máy khoảng $1,2 \cdot 10^{-7}$, nên phép trừ hai giá trị gần nhau ở tử số mất gần hết chữ số có nghĩa và bạn đang đo nhiễu làm tròn chứ không đo đạo hàm. Thứ hai, phải tắt mọi nguồn ngẫu nhiên — dropout, augmentation, thứ tự batch — hoặc cố định seed, vì f phải là một hàm tắt định thì sai phân mới có nghĩa. Thứ ba, chỉ kiểm vài chục toạ độ ngẫu nhiên, không kiểm hết, vì mỗi toạ độ tốn hai lượt xuôi.

Điểm gãy là cái bẫy lớn nhất. Giả sử x_k rất gần 0 ở đầu vào một ReLU. Khi đó $x_k + h$ và $x_k - h$ nằm hai bên điểm gãy. Sai phân số trả về một giá trị trung gian, còn gradient giải tích trả về 0 hoặc 1. Kiểm tra “thất bại” dù code hoàn toàn đúng. Cách xử lý là dùng ít điểm dữ liệu, chọn toạ độ ở xa điểm gãy, và diễn giải một vài lần thất bại cô lập một cách thận trọng thay vì lao đi viết lại backward. Kỹ năng này không hề mất giá theo thời gian: mỗi lần bạn viết một autograd.Function riêng hay một kernel CUDA cho một phép toán mới, việc đầu tiên nên làm là chạy `torch.autograd.gradcheck` với đầu vào double.

F. Vectorization, và vì sao nó không chỉ là chuyện tốc độ

Bỏ vòng lặp Python khỏi loss và gradient là bài tập kinh điển. Lợi ích hiển nhiên là tốc độ, nhưng chi phí không nằm ở FLOPs. Nó nằm ở phần trên đầu của mỗi lời gọi: thông dịch Python cộng chi phí phóng một kernel, cỡ vài micro-giây. Một nghìn phép toán tí hon vì thế tiêu tốn vài mili-giây trước khi làm phép nhân nào. Đây đúng là hiện tượng bạn đã đo khi tối ưu SLAM: nghẽn ở số lời gọi chứ không ở số phép tính. Nhưng lợi ích thứ hai mới đáng giá lâu dài: viết dạng vector buộc bạn phát biểu rõ shape của mọi thứ, do đó phát biểu rõ *ai rẽ nhánh sang ai*, và khi đó backward gần như tự lộ ra từ ba mẫu ở trên. Đoạn dưới là toàn bộ forward và backward của softmax ghép cross-entropy cho một batch.

```
# Z: (N, K) logits, y: (N,) nhãn dung
Z = Z - Z.max(axis=1, keepdims=True) # on định so: tranh exp tràn
P = np.exp(Z); P /= P.sum(axis=1, keepdims=True)
loss = -np.log(P[np.arange(N), y]).mean()

dZ = P.copy()
dZ[np.arange(N), y] -= 1.0 # dZ = p - y
dZ /= N # vì loss lay trung bình theo batch
dW = X.T @ dZ # VJP của tang affine
db = dZ.sum(axis=0) # backward của broadcast = cong theo trục
```

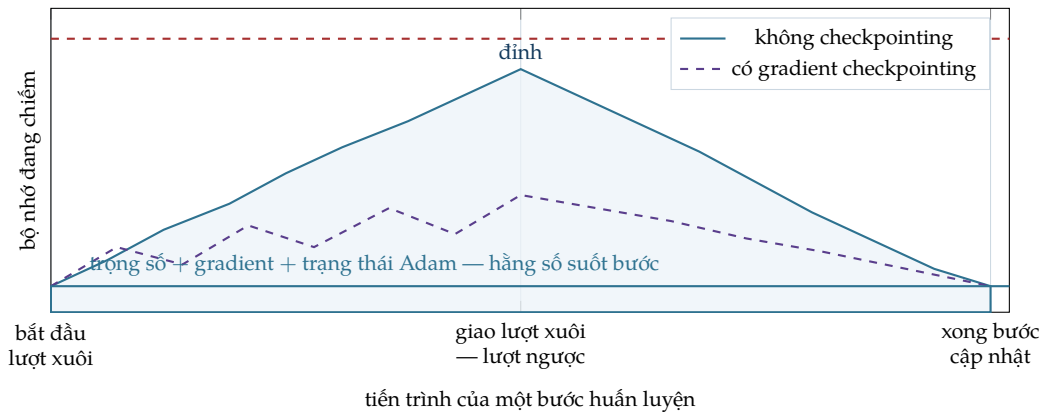
Hai dòng cuối minh hoạ chính xác hai quy tắc đã nêu: tích ngoài cho ma trận trọng số, và tổng theo trục batch cho bias.

G. Cái giá của reverse mode: bộ nhớ kích hoạt

Reverse-mode rẻ về thời gian nhưng không miễn phí, và cái giá của nó là bộ nhớ. Mẫu nhân đòi hỏi giá trị của nhánh kia. Nên lượt ngược cần phần lớn giá trị trung gian của lượt xuôi. Bộ nhớ vì thế tăng theo tổng kích thước **kích hoạt** (*activation*) của cả mạng, không theo số tham số. Một

phép tính cụ thể cho thấy bậc độ lớn. Lấy đầu ra của một tầng conv ở batch 32, 64 kênh, độ phân giải 112×112 , lưu ở fp32. Nó chiếm $32 \cdot 64 \cdot 112 \cdot 112 \cdot 4 \approx 1,03 \cdot 10^8$ byte, tức khoảng 98 MiB cho một tầng. Một backbone vài chục tầng nhanh chóng đưa con số này lên vài GB. Nên trong phần lớn bài huấn luyện thị giác, thứ quyết định batch size tối đa là bộ nhớ chứ không phải tính toán.

→ **ràng buộc bởi B/11** — bộ nhớ kích hoạt là đại lượng chặn batch size, và chương giới hạn tính toán do nó cùng với băng thông trên mô hình roofline



Hình 12.4. Bộ nhớ trong một bước huấn luyện. Phần không đổi là trọng số, gradient và trạng thái optimizer; phần dâng lên rồi rút xuống là kích hoạt, và nó đạt đỉnh đúng ở chỗ lượt xuôi giao lượt ngược — vì đó là lúc mọi giá trị trung gian đã được sinh ra mà chưa có giá trị nào được giải phóng. Hai hệ quả đọc thẳng từ hình: batch size tối đa bị quyết định bởi một thời điểm chứ không phải bởi mức trung bình, và checkpointing hạ đỉnh bằng cách vớt rồi tính lại, đổi lấy phần diện tích tăng thêm ở lượt ngược. Các con số là minh họa định tính, không phải số đo.

Hình 12.4 giải thích một điều mà con số 98 MiB ở trên không nói: cái chặn bạn không phải tổng bộ nhớ trung bình mà là *đỉnh tức thời* tại điểm giao giữa hai lượt. Đó cũng là lý do một mô hình chạy được ở batch 16 có thể sập ở batch 17 chứ không xuống cấp dần.

Đây là một đánh đổi thời gian-bộ nhớ quen thuộc, và nó có lời giải quen thuộc. **Gradient checkpointing** [9] chỉ lưu một tập con các nút, rồi khi lượt ngược cần một giá trị đã bị vớt thì tính lại nó từ điểm lưu gần nhất; với cách chọn điểm lưu hợp lý, bộ nhớ giảm từ $O(L)$ xuống $O(\sqrt{L})$ với giá khoảng một lượt xuôi phụ, tức chừng 30 phần trăm thời gian. Ở đầu kia của phổ, forward-mode có bộ nhớ hằng số nhưng cần một lượt cho mỗi hướng đầu vào, nên nó không dùng để huấn luyện mà dùng cho tích Jacobian-vector, chẳng hạn khi cần tích Hessian-vector trong các phương pháp bậc hai.

H. Giới hạn và trường hợp hỏng

Khung đồ thị có ba chỗ gãy đáng biết trước.

Thứ nhất là *điểm không khả vi*. ReLU tại 0 và max khi hai nhánh bằng nhau không có đạo hàm; framework chọn một quy ước, thường là 0 cho ReLU và “nhánh chỉ số nhỏ hơn thắng” cho max. Quy ước đó vô hại về mặt tối ưu hoá nhưng có hệ quả về tính tái lập: khi hai giá trị bằng nhau tới từng bit, việc ai thắng phụ thuộc thứ tự duyệt, và thứ tự duyệt có thể phụ thuộc lịch trình luồng. Đây đúng là loại nguồn bất định mà bạn đã truy trong hệ SLAM đa luồng, chỉ là xuất hiện ở một tầng khác.

Thứ hai là *đồ thị bị cắt mà không ai báo*. Một phép gán tại chỗ ghi đè giá trị mà lượt ngược cần sẽ cho gradient sai; PyTorch bắt được phần lớn trường hợp này bằng bộ đếm phiên bản tensor, nhưng một kernel tự viết thì không có ai bảo vệ và bạn nhận gradient sai một cách im lặng. Cùng loại đó là việc vô tình gọi `detach()` hoặc đặt một nhánh trong `no_grad()`: loss vẫn giảm, chỉ có một mô-đun không bao giờ học, và triệu chứng duy nhất là một đường cong hơi tệ hơn kỳ vọng.

Thứ ba là *ổn định số*. $\log(0)$, $\exp$ tràn, chia cho phương sai gần 0: ba nguồn NaN kinh điển. Chúng không phải lỗi của autodiff mà là lỗi của biểu thức bạn viết ra. Cách chữa luôn là viết lại biểu thức ở dạng ổn định: trừ max trước khi lấy $\exp$, dùng $\log\text{sumexp}$, cộng ε vào mẫu số. Kẹp gradient sau khi nó đã thành NaN thì quá muộn. Cùng họ với nó là điều khiển luồng phụ thuộc dữ liệu: một câu lệnh rẽ nhánh theo giá trị tensor là hợp lệ, nhưng gradient chỉ chảy qua nhánh đã chọn, nên nếu bạn tưởng mình đang huấn luyện cả hai nhánh thì bạn nhầm.

CẠM BÃY

Cái bẫy sinh ra nhiều gradient sai âm thầm nhất không phải công thức khó, mà là quên rằng *broadcast ở lượt xuôi là rẽ nhánh, nên lượt ngược phải cộng theo trục đã broadcast*. Nguy hiểm ở chỗ nếu bạn quên phép cộng đó, shape của kết quả nhiều khi vẫn hợp lệ nhờ chính broadcast, nên không có exception nào được ném ra; bạn chỉ nhận một gradient sai hệ số N và một quá trình huấn luyện chậm hơn đáng kể. Cách phòng duy nhất đáng tin là kiểm tra bằng số trên một ví dụ có $N = 3$ và $N = 1$ rồi so hai kết quả.

CẦN NHỚ

Lan truyền ngược = duyệt đồ thị theo thứ tự tô-pô đảo, mỗi nút áp một tích vector-Jacobian cục bộ, với ba mẫu cộng-nhân-max cộng thêm quy tắc “rẽ nhánh thì cộng dồn”. Nếu bạn gọi tên được Jacobian cục bộ của một khối, bạn vi phân được khối đó — kể cả khối bạn chưa từng thấy. [T1]

I. Kết nối

Mục này cho bạn công cụ tính gradient; mục tiếp theo, [C/12/02-lam-cho-training-chay-duoc](#), hỏi câu kè: gradient tính đúng rồi thì làm sao để nó không chết hoặc nổ khi đi qua năm mươi tầng — và câu trả lời bắt đầu chính từ tích các Jacobian cục bộ vừa nêu ở đây. Chi phí bộ nhớ kích hoạt ở cuối mục là tiền đề trực tiếp cho [B/11-gioi-han-tinh-toan](#) (roofline, chế độ nghẽn bộ nhớ) và cho phần huấn luyện phân tán ở [C/12/04-tong-quat-hoa-va-chan-doan](#). Ý “mọi kiến trúc chỉ là một đồ thị” được dùng lại nguyên vẹn ở [C/13-kien-truc-backbone](#): khi đọc ResNet hay Transformer, thứ đáng hỏi không phải sơ đồ khối mà là đường đi của adjoint qua sơ đồ đó.

I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao reverse-mode rẻ hơn forward-mode đúng bằng tỉ số giữa số đầu vào và số đầu ra, và trong tình huống nào tỉ số đó đảo chiều khiến forward-mode thắng?
2. Vì sao không bao giờ dựng ma trận Jacobian của một tầng affine, dù về mặt toán học nó tồn tại?
3. Kiểm tra gradient bằng số báo sai số tương đối $3 \cdot 10^{-3}$ trên một mạng có ReLU — hai nguyên nhân khả dĩ là gì, và bạn phân biệt chúng bằng cách nào?
4. Vì sao gradient của BatchNorm với một mẫu lại phụ thuộc vào các mẫu khác trong batch, và điều đó kéo theo hệ quả gì cho huấn luyện phân tán?
5. Bạn viết một kernel CUDA cho một phép toán mới và lượt ngược cho gradient sai đúng hệ số bằng batch size. Chỗ nào trong code là nghi phạm số một?
6. Gradient checkpointing đổi cái gì lấy cái gì, và khi nào phép đổi đó *không* đáng?

I 12.3 Làm cho training chạy được (Making Training Work)

PHỤ THUỘC C/12/01-co-hoc-cua-gradient (tích các Jacobian cục bộ — toàn bộ mục này là hệ quả của nó); A/01-toan-toi-thieu (phương sai, giá trị kỳ dị, số điều kiện).

TÓM TẮT

Mục này trả lời câu hỏi trung tâm của cả chương: làm sao đưa *gradient* qua năm mươi tầng mà nó không chết cũng không nổ. Ba nhóm kỹ thuật tưởng như rời rạc — hàm kích hoạt, khởi tạo trọng số, normalization — thực ra nhắm vào đúng một đại lượng duy nhất. Đọc xong bạn nhìn một file cấu hình huấn luyện và nói được từng dòng trong đó đang giữ đại lượng nào ổn định, thay vì chép nguyên recipe của người khác. Nếu chỉ nhớ một điều: adjoint đi ngược qua L tầng là tích của L Jacobian, nên mọi thứ trong mục này chỉ là các cách khác nhau để ép hệ số nhân của tích đó về gần 1.

KIẾN THỨC CẦN CÓ TRƯỚC

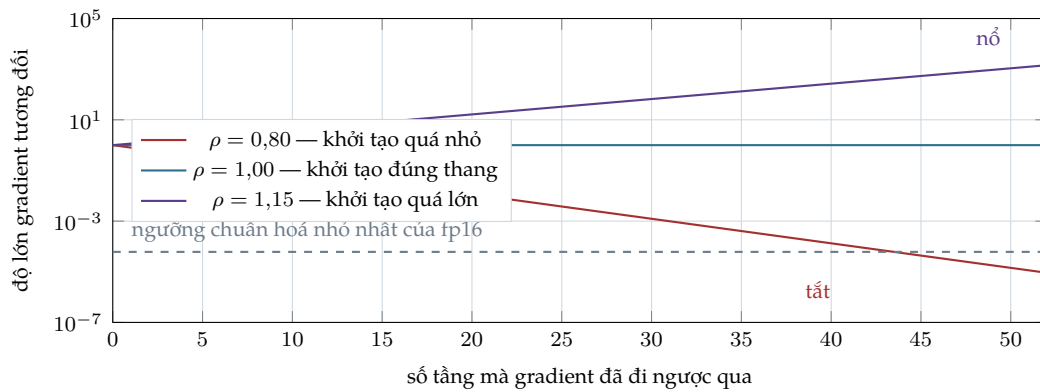
- **tầng (layer)** — một khối phép toán có tham số riêng; “mạng sâu” nghĩa là nhiều tầng nối tiếp, và toàn bộ khó khăn của mục này sinh ra từ chữ “nối tiếp” đó.
- **kích hoạt (activation)** — giá trị đầu ra của một tầng giữa mạng. “Hàm kích hoạt” là phép phi tuyến tạo ra nó, ví dụ ReLU — hai khái niệm khác nhau nhưng tên gần giống.
- **gradient** — vectơ đạo hàm riêng của hàm mất mát theo từng tham số; đi ngược nó thì mất mát giảm. Gradient “tiêu biến” nghĩa là nó nhỏ tới mức tầng đó không học được nữa.
- **phương sai (variance)** — đại lượng đo độ tản mát của một tập số quanh trung bình của chúng; cả lập luận khởi tạo trọng số trong mục này chỉ là theo dõi phương sai đi qua từng tầng.
- **số điều kiện (condition number)** — tỉ số giữa độ cong lớn nhất và nhỏ nhất của mặt mất mát; càng lớn thì đường đồng mức càng dẹt và thuật toán tối ưu càng đi zigzag.
- **batch (lô)** — nhóm mẫu đi qua mạng cùng lúc. Với BatchNorm, kích thước lô không chỉ là chuyển tốc độ mà còn thay đổi *chính hàm số* mà mạng biểu diễn.
- **chế độ huấn luyện và suy luận (train / eval mode)** — cùng một mô hình hành xử khác nhau ở hai chế độ: BatchNorm và dropout dùng thống kê batch khi huấn luyện, dùng thống kê tích lũy khi suy luận. Quên chuyển chế độ là bug kinh điển ở cuối mục.
- **bias — phân biệt hai nghĩa** — trong $Wx + b$, b là tham số cộng của một tầng, và “đẩy bias xuống rất âm” ở phần ReLU là nói về nó. Nghĩa thống kê — độ lệch hệ thống của một bộ ước lượng — không dùng trong mục này.

A. Đặt vấn đề

Gradient đã tính đúng, kiểm tra bằng số đã qua, code chạy không lỗi. Vậy mà loss vẫn nằm im ở mức phỏng đoán ngẫu nhiên, hoặc bay lên NaN ở bước thứ ba trăm. Đây là trạng thái mà mọi người từng huấn luyện mạng sâu đều đã sống qua, và nó không phải một lỗi lập trình. Nó là biểu hiện của nút thắt lịch sử lớn nhất của ngành: *vanishing and exploding gradient* — gradient tắt dần hoặc nổ tung khi đi ngược qua nhiều tầng. Giữa đầu thập niên 1990 và khoảng 2012, người ta biết cách viết mạng sâu và biết cách tính gradient của nó. Chỉ có điều không ai huấn luyện được nó. Gần như mọi “trick” trong một file cấu hình huấn luyện hiện đại tồn tại vì lý do đó.

Cơ chế thì rất gọn, và nó suy trực tiếp ra từ mục trước. Adjoint đi ngược qua L tầng là một tích của L Jacobian cục bộ. Nếu giá trị kỳ dị điển hình của mỗi Jacobian là ρ , thì độ lớn gradient ở tầng đầu tiên tỉ lệ với ρ^L . Đây là hàm mũ, nên không có vùng an toàn rộng: với $L = 50$, $\rho = 0,9$ cho $0,9^{50} \approx 5 \cdot 10^{-3}$ và $\rho = 1,1$ cho $1,1^{50} \approx 117$. Một sai lệch 10 phần trăm ở mỗi tầng biến thành hai bậc độ lớn ở toàn mạng theo hướng này và hai bậc theo hướng kia.

→ hệ quả của C/12/01 — tích các Jacobian là kết quả trực tiếp của việc adjoint đi ngược qua từng tầng một



Hình 12.5. Vì sao không có vùng an toàn rộng cho khởi tạo. Ba đường chỉ khác nhau ở hệ số ρ của một tầng — lệch 20 phần trăm theo hai chiều — nhưng sau 50 tầng chúng cách nhau tám bậc độ lớn, vì lũy thừa thắng mọi thứ khác. Đường $\rho = 0,80$ chui xuống dưới ngưỡng biểu diễn của fp16 từ khoảng tầng thứ 40, tức các tầng đầu mạng không nhận được tín hiệu nào cả. Đây là đồ thị của một mô hình đơn giản hoá (mọi Jacobian có cùng phổ, độc lập nhau) nên nó cho thấy *chiều* và *tốc độ* của hiện tượng, không dự đoán giá trị cụ thể.

Hình 12.5 là toàn bộ lý do ba mục tiếp theo tồn tại: khoảng cách giữa “huấn luyện được” và “không huấn luyện được” chỉ là vài phần trăm ở hệ số của một tầng.

Do đó bài toán không phải “làm cho ρ tốt” mà là “làm cho ρ bằng 1 và giữ nó ở đó suốt quá trình huấn luyện”.

Có đúng ba hướng tấn công vào cùng con số ρ đó, và toàn bộ mục này là ba hướng ấy.

- **Sửa hệ số danh nghĩa** — chọn hàm kích hoạt có đạo hàm không nhỏ đi (mục C).
- **Đặt điểm làm việc ban đầu** — chọn khởi tạo sao cho phương sai tín hiệu giữ nguyên qua mỗi tầng (mục D).
- **Hồi tiếp trong lúc chạy** — chèn phép chuẩn hoá ở giữa mạng để ép lại phân bố mỗi khi nó trôi (mục E).

Nhưng trước cả ba, có một việc rẻ hơn nhiều phải làm trước ở đầu vào. Giá trị cụ thể của cả mục: bạn sẽ đọc được một cấu hình huấn luyện lạ và chỉ ra chính xác dòng nào đang giữ ρ gần 1, do đó biết dòng nào có thể bỏ và dòng nào không.

TRỰC GIÁC

Coi mạng sâu như một chuỗi năm mươi bộ khuếch đại nối tiếp nhau, mỗi bộ có hệ số khuếch đại danh nghĩa là 1 nhưng lệch chuẩn vài phần trăm. Không ai chỉnh từng bộ để tổng thể đúng bằng 1 — sai số nhân dồn sẽ thắng. Cái người ta làm trong điện tử là mắc hồi tiếp âm quanh mỗi tầng để hệ số của tầng đó bị ép về giá trị mong muốn bất kể linh kiện trôi. Ba hướng trong mục này là ba kiểu hồi tiếp khác nhau lên cùng một đại lượng: hàm kích hoạt sửa hệ số danh nghĩa, khởi tạo đặt điểm làm việc ban đầu, normalization là vòng hồi tiếp thật sự vì nó đo phân bố đang chạy rồi kéo nó về.

B. Chuẩn hoá đầu vào (input normalization): việc rẻ nhất, làm trước tiên

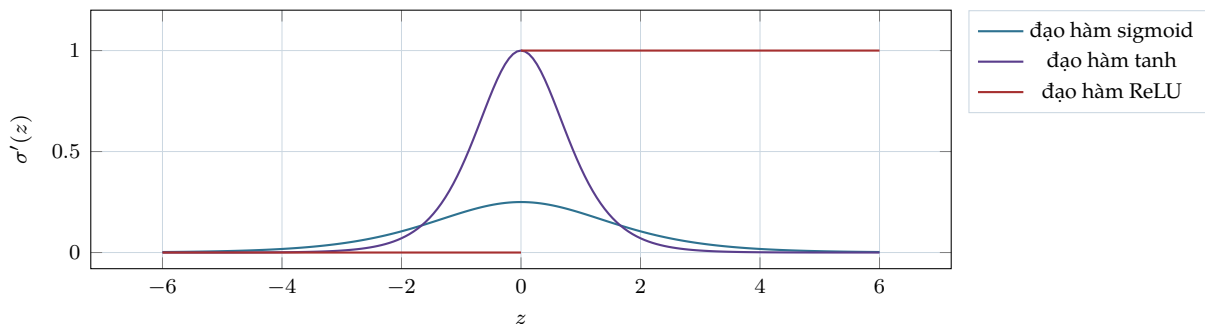
Trước khi nói về chiều sâu, hãy nhìn một tầng tuyến tính duy nhất. Hessian của mất mát bình phương theo trọng số của tầng đó tỉ lệ với $\mathbb{E}[xx^\top]$, tức ma trận moment bậc hai của đầu vào. Giả sử một đặc trưng nằm trong $[0, 1]$ còn đặc trưng kia trong $[0, 1000]$. Hai trị riêng của Hessian khi đó lệch nhau khoảng 10^6 , và đường đồng mức của mất mát là những ellipse cực dẹt. Gradient descent trên một ellipse dẹt đi zigzag. Bước học phải đủ nhỏ để không phân kỳ theo trục dọc,

nên nó nhỏ đến vô lý theo trực giác. Tốc độ hội tụ khi đó do **số điều kiện** (*condition number*) κ chi phối, theo hệ số $\left(\frac{\kappa-1}{\kappa+1}\right)^t$. Với $\kappa = 1000$, hệ số đó là 0,998 mỗi bước — cần khoảng 350 bước chỉ để giảm sai số đi một nửa.

Trung bình và chia độ lệch chuẩn theo từng kênh đưa κ từ 10^6 về vài đơn vị với chi phí gần bằng không. Đây là lý do vì sao nó là dòng đầu tiên của mọi pipeline, chứ không phải vì một lý do thẩm mỹ nào. Bước xa hơn về lý thuyết là *whitening* bằng PCA, đưa κ về đúng 1; nhưng với ảnh thì nó gần như không bao giờ được dùng, vì hai lý do có thể tính ra được. Thứ nhất là chi phí. Một ảnh $3 \times 224 \times 224$ có 150 528 chiều, nên ma trận hiệp phương sai của nó có $150\,528^2 \approx 2,27 \cdot 10^{10}$ phần tử. Riêng việc lưu nó đã tốn khoảng 90 GB ở fp32, chưa nói tới phân rã. Thứ hai, và quan trọng hơn, whitening trộn mọi pixel với mọi pixel nên nó *phá cấu trúc không gian*: sau khi whiten, hai pixel cạnh nhau không còn cạnh nhau theo nghĩa thống kê, và toàn bộ giả định **chia sẻ trọng số** (*weight sharing*) của *convolution* mất cơ sở. Đây là một ví dụ sạch về nguyên lý “một phép biến đổi tối ưu theo một tiêu chí có thể phá huỷ đúng cấu trúc mà phần còn lại của hệ thống đang dựa vào”.

C. Hướng 1 — hàm kích hoạt (kích hoạt function), đọc như một chuỗi sửa lỗi

Hàm kích hoạt (*activation function*) đóng góp trực tiếp vào ρ : Jacobian của nó là một ma trận đường chéo chứa $\sigma'(z)$, nên nếu σ' nhỏ thì ρ nhỏ. Sigmoid có σ' cực đại bằng 0,25 tại $z = 0$ và tiến nhanh về 0 ở hai đầu. Ngay cả ở điều kiện tốt nhất, mười tầng sigmoid nhân vào nhau cho hệ số $0,25^{10} \approx 10^{-6}$; ở điều kiện thực tế, nơi phần lớn nơ-ron bão hoà, con số còn tệ hơn nhiều. Một phép nhân hai dòng như vậy giải thích vì sao mạng sâu không huấn luyện được trước 2010, thuyết phục hơn bất kỳ lời giải thích định tính nào.



Hình 12.6. Đạo hàm của ba hàm kích hoạt — đây là đại lượng thật sự đi vào tích các Jacobian, chứ không phải bản thân hàm. Đỉnh của đạo hàm sigmoid chỉ là 0,25 và tụt nhanh về 0 ở hai đầu; đạo hàm tanh đạt 1 nhưng cũng bão hoà; ReLU giữ đúng 1 trên toàn nửa dương và trả về 0 trên nửa âm, đối tính bão hoà lấy nguy cơ nơ-ron chết.

Hình 12.6 vẽ đúng đại lượng cần nhìn: không phải hình dạng hàm kích hoạt mà là hình dạng *đạo hàm* của nó, vì chỉ đạo hàm mới đi vào tích Jacobian. Bảng 12.3 đọc tiếp bức tranh đó theo trục thời gian.

Bảng 12.3. Chuỗi hàm kích hoạt đọc như một chuỗi sửa lỗi. Cột cuối là cột đáng nhớ: mỗi bước sửa được một khiếm khuyết và để lại một khiếm khuyết mới.

Hàm	Cơ chế	Sửa được gì	Vấn hòng ở đâu
Sigmoid	Ép về $(0, 1)$	— (khởi điểm lịch sử)	Bão hoà hai đầu giết gradient; đầu ra không quanh 0 nên gradient của cả tầng cùng dấu
Tanh	Ép về $(-1, 1)$	Đầu ra quanh 0, hết hiện tượng cùng dấu	Vẫn bão hoà, $\sigma' \leq 1$
ReLU	$\max(0, x)$	Không bão hoà phía dương ($\sigma' = 1$), tính cực rẻ	Nơ-ron chết vĩnh viễn nếu rơi hẳn sang phía âm; không quanh 0
Leaky ReLU, PReLU, ELU	Cho phía âm một độ dốc nhỏ	Nơ-ron chết	Thêm siêu tham số; lợi ích không ổn định giữa các bài toán
GELU, SiLU	Cổng mượt theo giá trị đầu vào	Đạo hàm liên tục, hợp với transformer và lịch học dài	Đắt hơn ReLU vài chục phần trăm; khác biệt nhỏ trên CNN

Bảng 12.3 đọc theo chiều dọc thì thấy rõ đây không phải một danh sách lựa chọn ngang hàng mà là một chuỗi nhân quả. Điều đáng chú ý là bước tiến lớn nhất — từ tanh sang ReLU — không đến từ một hàm “tốt hơn” theo nghĩa toán học mà từ việc *bỏ đi* tính bão hoà. Đáng chú ý không kém là ReLU tạo ra một chế độ hỏng hoàn toàn mới không tồn tại trước đó: một nơ-ron nhận bước cập nhật lớn đẩy bias xuống rất âm sẽ cho 0 trên mọi dữ liệu, do đó gradient của nó bằng 0 mãi mãi, và nó chết không hồi phục. Tỷ lệ nơ-ron chết (*dead ReLU*) vì thế là một chỉ báo sức khoẻ đáng đo, và ta sẽ gặp lại nó ở mục sau.

D. Hướng 2 — khởi tạo trọng số (weight initialization), hay toán đơn giản giải bài toán thật

Lập luận phương sai là ví dụ đẹp nhất trong ngành về việc vài dòng đại số giải quyết một vấn đề đã chặn cả một lĩnh vực. Xét $y = Wx$ với n đầu vào, các w_{ij} độc lập cùng phân bố trung bình 0, và x độc lập với W . Khi đó

$$\text{Var}(y_j) = n \text{Var}(w) \text{Var}(x).$$

Muốn **phương sai** (*variance*) tín hiệu không đổi qua mỗi tầng, cần $n \text{Var}(w) = 1$, tức $\text{Var}(w) = 1/n_{\text{in}}$. Nhưng lượt ngược cũng phải giữ phương sai, và ở đó vai trò của n_{in} và n_{out} đổi chỗ; không thoả cả hai cùng lúc được nên Xavier/Glorot chọn trung hoà $\text{Var}(w) = 2/(n_{\text{in}} + n_{\text{out}})$ [124]. Đó là lời giải cho mạng tanh.

Với ReLU, lập luận trên thiếu một hệ số. ReLU xoá một nửa số đầu ra về 0, nên phương sai sau kích hoạt chỉ còn một nửa phương sai trước nó; hệ số mỗi tầng là $1/2$ chứ không phải 1. Sau 30 tầng, tín hiệu bị nhân với $2^{-30} \approx 10^{-9}$ — tức là tắt hẳn, dù bạn đã “khởi tạo đúng” theo Xavier. Sửa lại thì hiển nhiên: nhân đôi phương sai, $\text{Var}(w) = 2/n_{\text{in}}$. Đó là khởi tạo He/Kaiming [125], và chính hệ số 2 đó là thứ cho phép huấn luyện mạng ReLU ba mươi tầng lần đầu tiên. Hãy để ý mạch lập luận: một giả định (“ReLU không đổi phương sai”) bị vi phạm, hậu quả được lượng hoá ra một con số, và lời sửa là một hằng số. Đây là dạng suy luận mà bạn nên tái tạo được, chứ không phải hai công thức để nhớ.

↔ **tiền đề A/01** — toàn bộ lập luận chỉ dùng một tính chất của phương sai: phương sai của tổng các biến độc lập bằng tổng phương sai

E. Hướng 3 – normalization, vòng hồi tiếp thật sự

Hai hướng trên chỉ đặt điều kiện *ban đầu*. Sau vài nghìn bước, trọng số đã đổi và phân bố kích hoạt trôi khỏi chỗ được thiết kế. **Chuẩn hoá** (*normalization*) đo phân bố đang chạy rồi ép nó về trung bình 0 và độ lệch chuẩn 1. Sau đó nó trả lại quyền chính cho mạng, qua hai tham số học được γ và β . Bốn biến thể khác nhau ở đúng một điểm: *lấy trung bình trên trục nào*.

Bảng 12.4. Bốn biến thể normalization. Chúng chỉ khác nhau ở tập chỉ số dùng để tính thống kê, và chính lựa chọn đó quyết định chúng hỏng khi nào.

Loại	Trung bình theo	Sinh ra để giải quyết	Hỏng khi nào
BatchNorm [4]	Batch $\times$ không gian, riêng từng kênh	Ổn định phân bố giữa mạng, cho phép dùng learning rate lớn hơn hẳn	Batch nhỏ (thống kê nhiễu); lệch train/eval; phải đồng bộ giữa các GPU
LayerNorm [110]	Toàn bộ đặc trưng của <i>một</i> mẫu	Batch nhỏ hoặc bằng 1; chuỗi độ dài thay đổi; không phụ thuộc mẫu khác	Trên CNN thì giác thường kém BN; trộn chung các kênh có ý nghĩa rất khác nhau
GroupNorm [111]	Nhóm kênh của một mẫu	Detection/segmentation với batch một tới hai ảnh mỗi GPU	Thêm siêu tham số số nhóm; không có lợi thế nào khi batch đủ lớn
InstanceNorm	Mỗi kênh của mỗi mẫu	Style transfer: xoá thống kê style khỏi đặc trưng	Ở bài toán khác, thứ bị xoá lại là thông tin có ích (độ sáng, tương phản)

Bảng 12.4 nên đọc cùng một nhận xét về mặt cơ chế: câu chuyện gốc của BatchNorm là “giảm internal covariate shift”, nhưng lời giải thích đó đã bị chất vấn nghiêm túc. Santurkar và cộng sự bơm nhiễu để *tăng* covariate shift một cách cố ý, và BN vẫn hoạt động tốt. Họ đề xuất tác dụng thật là làm mất mát mát trơn hơn. Gradient bớt thay đổi đột ngột, nên bước đi dài hơn mà không phân kỳ [112]. Đây là một trường hợp đáng nhớ khi đọc paper. Kỹ thuật đúng, nhưng lời giải thích đi kèm nó lúc ra đời có thể sai. Học thuộc lời giải thích thay vì cơ chế thì bạn sẽ suy luận sai ở tình huống kế tiếp.

Hệ quả kỹ thuật quan trọng nhất của BatchNorm là nó *ghép các mẫu trong batch lại với nhau*. Ta đã thấy điều đó ở công thức backward trong mục trước: gradient của một mẫu phụ thuộc mọi mẫu cùng batch. Từ đó suy ra gần như mọi chế độ hỏng của nó. Batch nhỏ thì thống kê nhiễu, nên gradient nhiễu. Lúc suy luận không có batch nên phải dùng thống kê chạy tích lũy trong huấn luyện, và bất kỳ chênh lệch nào giữa hai chế độ đều thành sai lệch hệ thống. Trong huấn luyện phân tán, mỗi tiến trình tính thống kê riêng nên mô hình thực chất khác nhau ở mỗi GPU trừ khi bạn dùng SyncBN. LayerNorm và GroupNorm né toàn bộ họ vấn đề này bằng cách không dùng tới trục batch — và đó là toàn bộ lý do transformer dùng LayerNorm, chứ không phải vì LayerNorm “hiện đại hơn”.

→ **ràng buộc bởi C/14** — detection chạy một tới hai ảnh mỗi GPU, nên chính ràng buộc batch nhỏ ở đó quyết định lựa chọn GroupNorm

(1) conv → chuẩn hoá → hàm kích hoạt — thứ tự mặc định



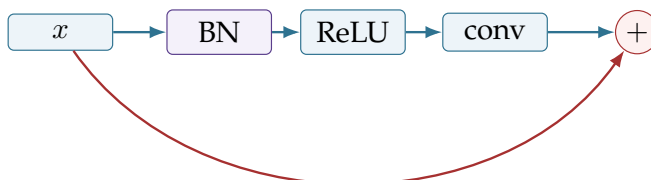
Đầu vào ReLU có trung bình 0, nên khoảng một nửa số nơ-ron hoạt động ở mọi batch. Đây là cấu hình mà lập luận phương sai của He giả định.

(2) conv → hàm kích hoạt → chuẩn hoá



BN phải chuẩn hoá một phân bố *không âm*, nên β phải gánh việc dời tâm và γ làm việc trên một phổ hẹp hơn. Chạy được, nhưng lãng phí hai tham số.

(3) tiền kích hoạt trong khối residual: BN → ReLU → conv



Gradient từ đầu ra về x đi qua một đường không bị chuẩn hoá chặn, nên thành phần bằng 1 trong Jacobian được giữ nguyên qua rất nhiều khối.

đường tắt *sạch*: không có phép toán nào chen vào

Hình 12.7. Ba cách đặt phép chuẩn hoá quanh hàm kích hoạt, và hệ quả lên luồng gradient. Điều hình này nói mà câu văn khó nói: vị trí của khối BN không phải chi tiết thẩm mỹ. Nó quyết định phân bố mà ReLU nhìn thấy, tức hàng 1 so với hàng 2. Nó cũng quyết định đường tắt có sạch hay không, tức hàng 3. Ở hàng 3, mọi phép toán đã bị đẩy vào *nhánh residual*, nên đường cộng từ x tới điểm $+$ hoàn toàn trống. Đó là lý do biến thể tiền kích hoạt huấn luyện được ở độ sâu lớn hơn hẳn.

Hình 12.7 cũng là chỗ ba hướng của mục này gặp nhau trên một sơ đồ duy nhất: hàng 1 nói về quan hệ giữa chuẩn hoá và hàm kích hoạt, hàng 3 nói về quan hệ giữa chuẩn hoá và kết nối tắt.

Còn một hướng thứ tư mà mục này cố tình không kể: **kết nối tắt** (*residual connection*). Nó tấn công đúng cùng đại lượng ρ , nhưng bằng cách thêm một đường cộng để Jacobian có dạng $I + \partial F / \partial x$ — luôn có một thành phần bằng 1 không thể triệt tiêu. Vì nó là một lựa chọn *kiến trúc* chứ không phải một thủ thuật huấn luyện, nó thuộc về [C/13/01-dong-cnn](#); nhưng khi đọc chương đó, hãy nhớ nó là thành viên thứ tư của đúng gia đình này.

↔ cùng nguyên lý với [C/13/01](#) — kết nối tắt giữ tích Jacobian gần 1 bằng một đường cộng, tức cùng mục tiêu với ba hướng ở đây nhưng thực hiện ở tầng kiến trúc

F. Giới hạn và trường hợp hỏng

Ba hướng trên không cộng dồn vô hạn, và chúng có những chỗ giao thoa gây bất ngờ.

Khởi tạo đúng chỉ đúng ở bước 0. He/Kaiming giả định trọng số độc lập cùng phân bố và đầu vào đã chuẩn hoá; sau vài nghìn bước cả hai giả định đều sai, nên khởi tạo không thay được normalization. Ngược lại, ở các mạng rất sâu có kết nối tắt, khởi tạo quan trọng theo một kiểu khác. Nhánh residual cần được khởi tạo nhỏ hoặc bằng 0, để mạng bắt đầu gần với ánh xạ đồng nhất. Bỏ chi tiết đó đi thì mạng sâu vẫn khó huấn luyện, dù đã có đủ BN.

Normalization che giấu lỗi. Một mạng có BN ở khắp nơi vẫn huấn luyện được dù khởi tạo sai, dù đầu vào chưa chuẩn hoá, dù learning rate lệch một bậc — chỉ là kém hơn đáng kể. Đây là tin tốt cho tiến độ và tin xấu cho việc học chẩn đoán, vì nó xoá mất các triệu chứng mà lẽ ra bạn phải học đọc.

Cuối cùng, các con số như ρ^L hay 2^{-30} trong mục này là ước lượng bậc độ lớn dựa trên giả định các Jacobian độc lập và có phổ đều. Giả định đó thô: trong thực tế các Jacobian tương quan với nhau, và phổ của chúng không đều. Chúng dùng để cho thấy *hàm mũ thắng mọi thứ khác*, chứ không dùng để dự đoán một giá trị cụ thể.

CẠM BẤY

Bug kinh điển của cả nhóm này: quên gọi `model.eval()` khi đánh giá. BatchNorm khi đó dùng thống kê của chính batch kiểm thử thay vì thống kê chạy. Kết quả vì thế phụ thuộc vào việc bạn xếp mẫu nào chung batch. Tệ nhất là nó thường *tốt hơn* thực tế, vì mô hình được nhìn trộm thống kê của tập kiểm thử. Triệu chứng nhận dạng: điểm số đôi khi bạn đổi batch size lúc đánh giá, hoặc điểm số tụt hẳn khi triển khai với batch bằng 1. Cùng một dòng lệnh cũng tắt dropout, nên hai lỗi này luôn đi cùng nhau.

CẦN NHỚ

Cả ba hướng — hàm kích hoạt, khởi tạo, normalization — chỉ nhắm vào đúng một đại lượng: hệ số nhân của tích các Jacobian qua các tầng. Mọi câu hỏi dạng “kỹ thuật X có cần thiết không” nên được dịch lại thành “trong cấu hình của tôi, cái gì đang giữ hệ số đó gần 1”. [T1]

G. Kết nối

Mục này lấy đầu vào từ [C/12/01-co-hoc-cua-gradient](#): toàn bộ lập luận chỉ là hệ quả số học của việc adjoint là một tích Jacobian. Nó chuyển tiếp sang [C/12/03-toi-uu-hoa](#). Ở đó ta giả định gradient đã lành mạnh, rồi hỏi tiếp nên đi bước dài bao nhiêu. Warmup sẽ xuất hiện ngay tại đó, như hệ quả trực tiếp của việc thống kê BN chưa ổn định ở những bước đầu. Thành viên thứ tư của gia đình, kết nối tắt, nằm ở [C/13/01-dong-cnn](#). Lựa chọn giữa BatchNorm và GroupNorm gắn chặt với ràng buộc batch size trong detection, nên nó được nhắc lại ở [C/14-bai-toan-cv-loi](#); còn chi phí bộ nhớ của việc lưu $\hat{x}$ cho lượt ngược thuộc về [B/11-gioi-han-tinh-toan](#).

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một sai lệch 10 phần trăm ở hệ số mỗi tầng lại là vấn đề nghiêm trọng, trong khi sai lệch 10 phần trăm ở learning rate thì không?
2. Whitening đưa số điều kiện về 1, tức tối ưu theo tiêu chí ở đầu mục. Vậy vì sao gần như không ai whiten ảnh?
3. Hệ số 2 trong khởi tạo He đến từ đâu, và nếu bạn thay ReLU bằng GELU thì lập luận đó thay đổi thế nào?
4. BatchNorm hỏng theo ba kiểu khác nhau; cả ba đều suy ra từ một tính chất duy nhất của nó. Tính chất đó là gì?
5. Vì sao lời giải thích “internal covariate shift” bị nghi ngờ, và vì sao việc học thuộc một lời giải thích sai lại nguy hiểm hơn là không có lời giải thích nào?
6. Bạn thay BatchNorm bằng LayerNorm trong một CNN thị giác và kết quả tệ đi — cách giải thích của bạn là gì, và phép đo nào kiểm chứng được nó?

12.4 Tối ưu hoá và siêu tham số (Optimization and Hyperparameters)

PHỤ THUỘC [C/12/02-lam-cho-training-chay-duoc](#) (gradient phải lành mạnh trước khi bàn tới bước đi); [A/01-toan-toi-thieu](#) (số điều kiện, hội tụ của gradient descent).

TÓM TẮT

Mục này đọc bảy *optimizer* quen thuộc như một chuỗi vá lỗi liên tiếp chứ không như một menu lựa chọn, rồi chỉ ra siêu tham số nào thật sự quyết định kết quả. Đọc xong bạn nhìn một đường cong học và đoán được đang sai ở đâu, đặt được *learning rate* và lịch của nó có căn cứ thay vì mò, và giải thích được vì sao *mixed precision* cần *loss scaling*. Nếu chỉ nhớ một điều: optimizer chỉ định hình lại bước đi, nó không cứu được một learning rate sai — nên hãy tune learning rate trước, batch size sau, và chọn optimizer cuối cùng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model)** — một hàm có tham số, nhận đầu vào và trả về dự đoán; huấn luyện là đi tìm bộ tham số làm dự đoán sát nhất.
- **gradient** — vectơ đạo hàm riêng của hàm mất mát theo từng tham số; nó chỉ hướng làm mất mát TĂNG nhanh nhất, nên mọi thuật toán ở đây đi ngược lại nó.
- **learning rate** — độ dài bước mà thuật toán tối ưu đi theo hướng ngược gradient ở mỗi lần cập nhật; quá lớn thì nghiệm bật ra ngoài, quá nhỏ thì hội tụ chậm tới mức vô dụng.
- **optimizer (thuật toán tối ưu)** — quy tắc biến gradient thành bước cập nhật thực tế; toàn bộ mục này là lịch sử bảy quy tắc như vậy và lý do từng cái ra đời.
- **siêu tham số (hyperparameter)** — con số do người đặt trước khi huấn luyện chứ không được học từ dữ liệu — learning rate, batch size, cường độ weight decay.
- **bước và epoch (step / epoch)** — một “bước” là một lần cập nhật tham số dựa trên một lô dữ liệu; một “epoch” là khi mô hình đã đi hết toàn bộ tập huấn luyện một lượt.
- **weight decay** — cơ chế kéo các trọng số về gần 0 sau mỗi bước, nhằm hạn chế mô hình bám quá sát dữ liệu huấn luyện; cách nó được cài đặt là toàn bộ khác biệt giữa Adam và AdamW.
- **fp32, fp16, bf16** — ba định dạng số thực với số bit khác nhau. Điều quan trọng không phải “bao nhiêu bit” mà là bit đó chia thế nào giữa *mũ* (quyết định dải giá trị biểu diễn được) và *định trị* (quyết định độ chính xác).

A. Đặt vấn đề

Giả sử gradient đã tính đúng và không còn chết hay nổ. Câu hỏi còn lại nghe rất đơn giản: biết hướng dốc rồi thì đi bao xa? Nhưng nó khó vì ba lý do chồng lên nhau. Thứ nhất, mất mát bị điều kiện xấu — có hướng dốc gấp trăm lần hướng khác, nên một bước dài theo hướng này là bước ngắn vô vọng theo hướng kia. Thứ hai, gradient bạn có không phải gradient thật mà là ước lượng từ một minibatch, nên nó nhiễu. Thứ ba, mọi thứ phi logic, nên không có lý thuyết nào nói cho bạn bước đi tối ưu.

Toàn bộ lịch sử *optimizer* trong deep learning là câu trả lời cho ba khó khăn ấy, và nó là ví dụ tốt nhất trong tài liệu này về việc *một danh sách thực ra là một mạch lập luận*. Bảy cái tên dưới đây thường được dạy như bảy lựa chọn ngang hàng trong một menu. Chúng không ngang hàng: mỗi cái ra đời để sửa một khiếm khuyết cụ thể của cái trước, và mỗi cái tạo ra một khiếm khuyết mới cho cái sau. Nếu bạn đọc được mạch đó thì bạn không cần nhớ công thức, vì bạn suy lại được chúng; nếu bạn chỉ nhớ danh sách thì bạn sẽ chọn optimizer theo thói quen và không bao giờ biết mình đang trả giá gì.

TRỰC GIÁC

Momentum thường được kể bằng ẩn dụ hòn bi lăn xuống dốc, nhưng ẩn dụ đó không nói cho bạn biết *vì sao* nó hiệu quả. Cách nhìn đúng hơn với người đã lọc tín hiệu IMU: chuỗi gradient theo thời gian là một tín hiệu, và momentum là một bộ lọc thông thấp một cực đặt lên tín hiệu đó. Trong một rãnh hẹp, thành phần vuông góc với rãnh đổi dấu mỗi bước — đó là thành phần tần số cao, và bộ lọc dập nó. Thành phần dọc theo rãnh giữ nguyên

dấu qua nhiều bước — đó là thành phần một chiều, và bộ lọc để nó cộng dồn tới hệ số $1/(1 - \mu)$. Momentum không “thêm quán tính”; nó tách tín hiệu khỏi dao động, đúng như khi bạn lọc bias con quay ra khỏi rung động khung máy.

B. Bảy bước của một mạch lập luận duy nhất (SGD → AdamW)

Trước khi vào mạch, hãy cố định một con số để có cảm giác về mức độ nghiêm trọng. Với một mặt bậc hai có số điều kiện κ , gradient descent hội tụ với hệ số $\left(\frac{\kappa-1}{\kappa+1}\right)$ mỗi bước. Với $\kappa = 1000$, hệ số đó là 0,998, nên cần khoảng 350 bước chỉ để giảm sai số một nửa. Mọi thứ trong mạch dưới đây là những cách khác nhau để không phải trả cái giá đó.

→ **tiền đề** A/01 — số điều kiện và hệ số hội tụ của gradient descent trên mặt bậc hai là kết quả chuẩn của giải tích số

I Mạch 1 — tiến hoá của optimizer

- 1 Vấn đề** gradient descent ngẫu nhiên đi zigzag trong rãnh hẹp và bò rất chậm theo hướng thoải.

Giải pháp **hạ gradient ngẫu nhiên** (*stochastic gradient descent, SGD*), $\theta \leftarrow \theta - \eta g$, là khởi điểm và cũng là chuẩn so sánh; ưu điểm thật của nó là nhiễu *minibatch* đóng vai trò **chính quy hoá ngầm** (*implicit regularization*).

Giá phải trả bước học η phải nhỏ hơn nghịch đảo độ cong lớn nhất để không phân kỳ, nên nó tự động quá nhỏ cho mọi hướng còn lại.

Vấn đề mới tốc độ bị khoá vào κ , và một giá trị η duy nhất phải phục vụ mọi tham số trong mạng.
- 2 Vấn đề** dao động vuông góc với rãnh làm phí gần hết công.

Giải pháp **momentum**: $v \leftarrow \mu v + g$, $\theta \leftarrow \theta - \eta v$. Thành phần đổi dấu triệt tiêu trong tổng trượt, thành phần cùng dấu cộng dồn tới hệ số $1/(1 - \mu)$, tức gấp 10 lần khi $\mu = 0,9$.

Hệ quả tốc độ hội tụ chuyển từ phụ thuộc κ sang phụ thuộc $\sqrt{\kappa}$, một cải thiện có chứng minh cho trường hợp bậc hai.

Vấn đề mới momentum đã tích lũy không dừng lại được ngay: nghiệm bị vọt lố qua điểm cực tiểu, và trong một rãnh cong thì vọt lố nghĩa là văng ra thành.
- 3 Vấn đề** vọt lố vì gradient được đo ở nơi ta *đang* đứng chứ không phải nơi ta *sắp* tới.

Giải pháp **Nesterov**: tính gradient tại điểm nhìn trước $\theta - \eta \mu v$. Hiệu giữa gradient ở điểm nhìn trước và gradient tại chỗ là một xấp xỉ của số hạng bậc hai. Nó hoạt động như lực hãm tỉ lệ với tốc độ thay đổi của gradient, tức phanh trước khi đâm vào thành chứ không sau.

Giá phải trả thêm một lần bookkeeping, thường được cài bằng phép đổi biến nên chi phí thực tế bằng không.

Vấn đề mới vẫn còn nguyên vấn đề gốc: một η chung cho tất cả tham số.
- 4 Vấn đề** các tham số có độ lớn gradient lệch nhau hàng bậc, nên không tồn tại một η đúng cho tất cả.

Giải pháp **AdaGrad**: tích lũy $G \leftarrow G + g^2$ theo từng toạ độ và bước theo $\eta g/(\sqrt{G} + \epsilon)$. Tham số hiếm được cập nhật nhận bước lớn, tham số hay được cập nhật nhận bước nhỏ; ý tưởng sinh ra từ các bài toán lỗi thừa và rất hợp lý ở đó.

Vấn đề mới G là một tổng đơn điệu tăng, nên bước học hiệu dụng tắt dần về 0. Trong một bài toán lỗi thì đó là tính năng; trong một mạng sâu chạy hàng trăm nghìn bước thì đó là cái chết: mô hình ngừng học trong khi loss còn cao.

5 Vấn đề mẫu số của AdaGrad nhớ quá lâu.

Giải pháp **RMSProp**: thay tổng bằng trung bình trượt phân rã, $G \leftarrow \beta G + (1 - \beta)g^2$. Bộ nhớ trở thành hữu hạn với chân trời khoảng $1/(1 - \beta)$ bước, tức chừng 1000 bước khi $\beta = 0,999$, nên bước học hiệu dụng không tắt.

Vấn đề mới hướng đi vẫn là gradient thô nên vẫn nhiễu; và trung bình trượt khởi tạo từ 0 nên trong những bước đầu G bị đánh giá thấp, mẫu số quá nhỏ, bước đi quá dài đúng vào lúc mô hình dễ tổn thương nhất.

6 Vấn đề cần cả hai: hướng đã lọc nhiễu và thang bước riêng cho từng tham số.

Giải pháp **Adam** [5] giữ hai trung bình trượt: m cho gradient và v cho bình phương gradient. Nó hiệu chỉnh độ chệch bằng $\hat{m} = m/(1 - \beta_1^t)$ và $\hat{v} = v/(1 - \beta_2^t)$, rồi bước theo $\eta \hat{m}/(\sqrt{\hat{v}} + \epsilon)$. Hiệu chỉnh độ chệch không phải chi tiết trang trí. Ở bước $t = 1$ với $\beta_1 = 0,9$ và $\beta_2 = 0,999$, ta có $m = 0,1g$ và $\sqrt{v} = 0,0316|g|$. Bước chưa hiệu chỉnh vì thế dài gấp khoảng 3,2 lần bước mong muốn.

Giá phải trả hai trạng thái phụ cho mỗi tham số: với mô hình 100 triệu tham số ở fp32, tổng bộ nhớ là $4P$ cho trọng số, $4P$ cho gradient và $8P$ cho trạng thái, tức 1,6 GB trước khi tính một byte kích hoạt nào.

Vấn đề mới hình phạt L2 cộng vào gradient bị chia cho $\sqrt{v}$ giống hệt phần còn lại, nên tham số nào có lịch sử gradient lớn thì bị suy giảm ít đi — nó không còn là weight decay theo nghĩa ban đầu nữa.

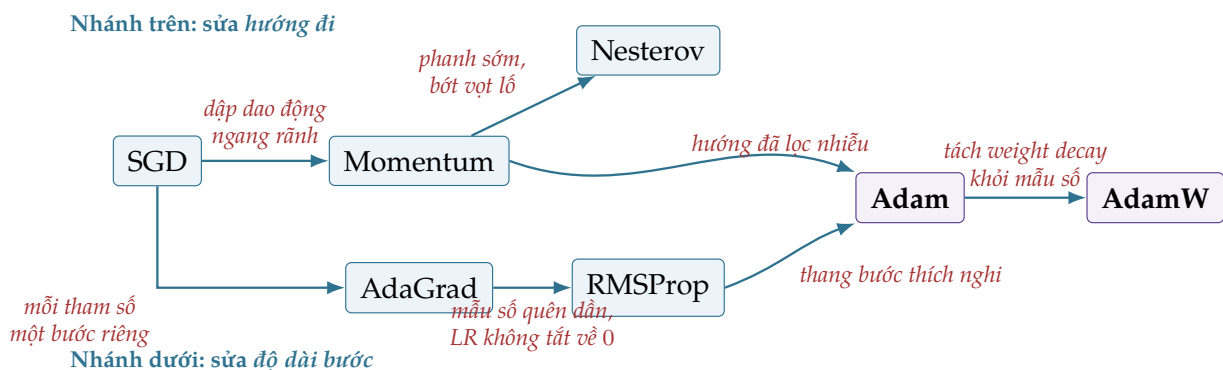
7 Vấn đề weight decay trong Adam không phải weight decay.

Giải pháp **AdamW** [11] tách hẳn hai thứ: $\theta \leftarrow \theta - \eta(\hat{m}/(\sqrt{\hat{v}} + \epsilon) + \lambda\theta)$, tức phần suy giảm đi thẳng vào trọng số, không qua mẫu số thích nghi.

Hệ quả suy giảm trở lại thành một phép co đều, độc lập với độ lớn gradient, và đây là lý do AdamW là mặc định thực tế cho transformer lẫn cho các recipe CNN hiện đại.

Giá phải trả λ không còn so sánh được với λ của Adam cũ, nên mọi giá trị chép từ code cũ đều sai thang.

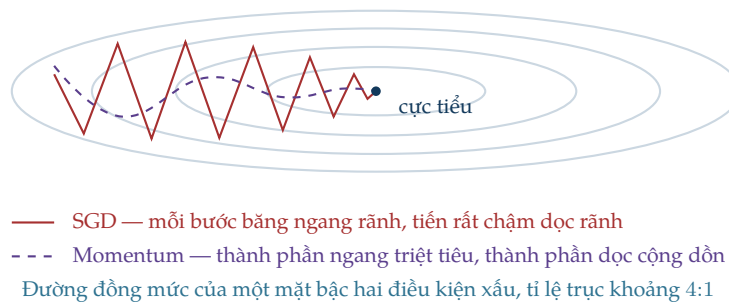
Vấn đề mới phần suy giảm nay tỉ lệ với η , nên nó tương tác với lịch học: giảm η theo cosine thì cũng đang giảm dần cường độ chính quy hoá, một hiệu ứng ít người nói ra.



Hình 12.8. Tiến hoá của optimizer không phải một chuỗi thẳng mà là hai nhánh độc lập rồi hợp lại. Nhánh trên sửa hướng của bước đi bằng cách lọc nhiễu theo thời gian; nhánh dưới sửa độ dài bước riêng cho từng tham số. Adam là chỗ hai nhánh gặp nhau, và AdamW là bản vá cho khiếm khuyết mà chính phép hợp nhất đó sinh ra. Mỗi mũi tên ghi đúng khiếm khuyết mà bước sau gỡ được.

Hình 12.8 nên được nhìn trước khi đọc mạch bảy bước, vì nó cho thấy ngay điều mà đọc tuần tự khó thấy: Adam không phải “bước tiến thứ sáu” mà là phép hợp nhất hai dòng giải quyết hai vấn đề khác nhau, và đó cũng là lý do nó thừa hưởng nhược điểm của cả hai.

Hai nhận xét khế lại mạch. Thứ nhất, hãy để ý rằng không bước nào trong bảy bước trên giải quyết vấn đề phi lồi; tất cả đều xử lý điều kiện xấu và nhiễu. Optimizer không tìm cực tiểu toàn cục và không hứa điều đó. Thứ hai, cả họ thích nghi có một cái giá chung ít được nêu: chúng *che giấu* một learning rate sai. Adam với η lệch một bậc vẫn cho đường cong loss trông chấp nhận được, trong khi SGD với sai lệch tương tự thì phân kỳ ngay và bạn biết mình sai. Đây là lý do đáng giữ SGD với momentum làm phép thử chẩn đoán, ngay cả khi bản chạy cuối dùng AdamW.



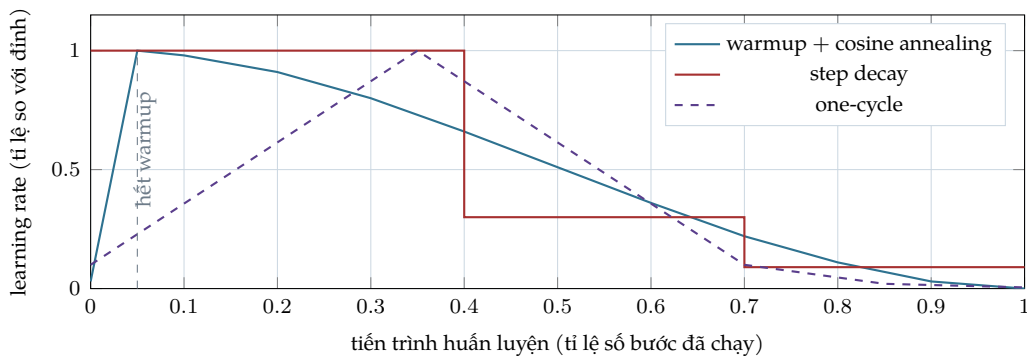
Hình 12.9. Vì sao momentum thắng trong rãnh hẹp. Thành phần vuông góc với rãnh đổi dấu mỗi bước nên triệt tiêu trong tổng trượt; thành phần dọc rãnh giữ nguyên dấu nên cộng dồn. Tỉ lệ trục trong hình chỉ khoảng 4:1 cho dễ nhìn; trong mạng thật số điều kiện lớn hơn hai tới ba bậc, nên hiệu ứng mạnh hơn nhiều.

Hình 12.9 là toàn bộ trực giác hình học của bước hai trong mạch, và cũng cho thấy vì sao Nesterov cần thiết: đường đứt nét vẫn vọt qua tâm ở lần tiếp cận đầu tiên nếu μ đủ lớn.

C. Learning rate là siêu tham số quan trọng nhất, không có đối thủ

Nếu chỉ được tune một thứ, tune learning rate. Điều này nghe như khẩu hiệu, nhưng nó có cơ sở. η là đại lượng duy nhất nhân trực tiếp vào mọi bước cập nhật. Sai một bậc ở η đổi hẳn bản chất của quá trình; sai một bậc ở weight decay chỉ làm kết quả kém đi vài phần trăm. Kỹ năng đáng đầu tư nhất là nhận dạng *hình dạng* đường cong học, vì nó tiết kiệm hàng tuần chạy thử: loss tăng vọt hoặc thành NaN trong vài chục bước là η quá cao rõ ràng; loss tụt rất nhanh rồi phẳng ở mức cao và nhiễu mạnh là η hơi cao, hệ đang nảy quanh một lòng chảo mà không xuống được; loss giảm gần như tuyến tính và không có dấu hiệu bão hòa sau nhiều epoch là η quá thấp. Cách rẻ nhất để định vị là chạy một lượt quét: tăng η theo cấp số nhân trong vài trăm bước và vẽ loss theo η ; vùng dốc nhất trước điểm nổ là vùng đáng dùng.

warmup tăng η tuyến tính từ gần 0 lên giá trị đích, trong vài trăm tới vài nghìn bước. Thoạt nhìn nó là một thủ thuật tùy tiện. Thực ra nó có ít nhất hai lý do rõ ràng, và cả hai đều đã xuất hiện ở trên. Thứ nhất, mẫu số thích nghi của Adam ở những bước đầu được ước lượng từ rất ít mẫu, nên phương sai của $\hat{v}$ rất lớn; một toạ độ vô tình có $\hat{v}$ nhỏ sẽ nhận một bước khổng lồ và phá trọng số ngay khi chưa có gì để cứu. Thứ hai, thống kê chạy của BatchNorm chưa hội tụ trong vài trăm bước đầu, nên hàm mà bạn đang tối ưu vẫn còn đang thay đổi dưới chân. Warmup mua thời gian cho cả hai bộ ước lượng đó. **Lịch learning rate** thì giải quyết một vấn đề khác: SGD với η cố định không hội tụ về một điểm mà tiến tới một phân bố dừng có phương sai tỉ lệ với η ; giảm η là thu nhỏ bán kính quả cầu mà nghiệm đang lang thang trong đó. Ba lịch phổ biến là step decay, cosine annealing [126] và one-cycle; khác biệt giữa chúng nhỏ hơn nhiều so với khác biệt giữa “có lịch” và “không có lịch”.



Hình 12.10. Ba lịch learning rate trên cùng một trục. Điều hình cho thấy mà lời văn khó nói: chúng khác nhau ở *chỗ* tiêu tiền, không ở tổng lượng. Step decay giữ nguyên bước dài rất lâu rồi cắt đột ngột. Cosine hạ liên tục nên tránh được cú sốc ở mỗi mốc cắt. One-cycle thì cố tình bắt đầu thấp và *tăng* lên đỉnh ở giữa, để đi qua vùng nhiễu nhanh hơn. Đoạn dốc đứng ở đầu đường cosine là warmup: nó tồn tại vì bộ ước lượng $\hat{v}$ của Adam và thống kê BatchNorm đều chưa đáng tin ở vài trăm bước đầu.

Hình 12.10 cũng cho thấy vì sao so sánh hai bài báo dùng hai lịch khác nhau là việc rất dễ sai: hai mô hình cùng số epoch có thể đã trải qua hai quá trình rất khác nhau.

D. Đổi batch size thì đổi gì theo (linear scaling rule)

quy tắc tỉ lệ tuyến tính (*linear scaling rule*) nói rằng nhân batch size lên k lần thì nhân learning rate lên k lần. Quy tắc này đã được kiểm chứng trên ImageNet với warmup đủ dài, tới batch 8192 [127]. Lập luận đơn giản: một bước với batch kB nên tiến xa bằng k bước với batch B , và trung bình của k gradient liên tiếp xấp xỉ gradient của batch gộp khi trọng số chưa đổi nhiều.

Nhưng nguyên nhân sâu hơn mới đáng nhớ, vì nó nói rằng đây không chỉ là chuyện tốc độ. Phương sai của gradient minibatch tỉ lệ nghịch với batch size, nên batch lớn cho gradient sạch hơn — và nhiễu gradient chính là một dạng chính quy hoá ngầm. Batch quá lớn không chỉ “khó tune hơn”: nó *lấy đi* một nguồn chính quy hoá, và các quan sát trên ImageNet cho thấy vượt một ngưỡng nhất định thì độ chính xác kiểm thử tụt dù loss huấn luyện vẫn tốt [12]. Smith và Le đưa ra một cách nhìn thống nhất: giảm η và tăng batch size là hai cách làm cùng một việc, đều là giảm mức nhiễu η/B [10]. Cách nhìn đó giải thích cả linh cảm “tăng batch thì phải tăng LR”, lẫn lý do quy tắc vớ. Khi η đã lớn tới mức không tăng được nữa vì lý do ổn định, mỗi liên hệ đứt.

→ **đối lập với C/12/04** — nhiễu gradient vừa là thứ quy tắc tỉ lệ tuyến tính muốn bảo toàn, vừa là nguồn chính quy hoá ngầm mà batch lớn xoá mất

E. Theo dõi sức khoẻ huấn luyện (training health), vì loss là chỉ báo quá chậm

Đường cong loss cho biết *đã* có vấn đề, thường sau hàng nghìn bước. Ba phép đo dưới đây rẻ như nhau nhưng phát hiện sớm hơn nhiều, vì chúng đo trạng thái bên trong chứ không đo kết quả.

- **Tỉ lệ cập nhật trên trọng số** (*update-to-weight ratio*), theo từng tầng: $\|\eta \Delta \theta\| / \|\theta\|$.
 - Kinh nghiệm thực hành phổ biến: khoảng 10^{-3} là lành mạnh; trên 10^{-2} là mỗi bước làm đổi trọng số vài phần trăm, tức quá lớn; dưới 10^{-4} là tầng đó thực chất đang đứng yên.
 - Vì sao dùng nó thay cho chuẩn gradient thô: nó không thứ nguyên, nên so sánh được giữa các tầng có thang hoàn toàn khác nhau.
- **Phân bố kích hoạt theo chiều sâu** — vẽ độ lệch chuẩn của kích hoạt từng tầng. Co dần theo chiều sâu là vanishing đang hình thành; phình lên là exploding. Đây chính là đại lượng ρ^L của mục trước, quan sát trực tiếp.

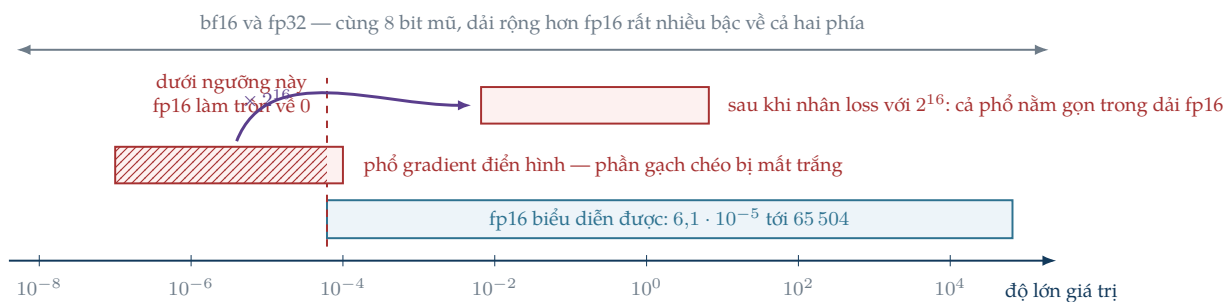
- **Tỉ lệ ReLU chết** — phần trăm nơ-ron cho 0 trên toàn bộ một batch xác thực. Vài phần trăm là bình thường; ba mươi phần trăm ở một tầng nghĩa là η đã có lúc quá cao, kể cả khi hiện tại nó đã ổn.

Cả ba chỉ báo chỉ tốn vài dòng hook và nên được ghi lại mặc định, vì giá trị của chúng nằm ở chỗ chúng *đi trước* loss chứ không đi sau.

F. Mixed precision: chuyển của dải động, không phải của độ chính xác

Chuyển sang **fp16** làm training ra NaN, và nguyên nhân thường bị hiểu nhầm là “thiếu độ chính xác”. Thủ phạm thật là *dải động*. fp16 có 5 bit mũ và 10 bit định trị, nên số chuẩn hoá nhỏ nhất là khoảng $6,1 \cdot 10^{-5}$ và số lớn nhất là 65 504. Gradient trong mạng thị giác thường nằm quanh 10^{-7} tới 10^{-4} ; phần lớn phổ đó rơi xuống dưới ngưỡng chuẩn hoá của fp16 và bị làm tròn về 0. Mạng không nổ, nó *tê liệt*: một phần các tầng đơn giản là không nhận được tín hiệu nào.

Lời sửa của **loss scaling** [13] tinh tế ở chỗ nó không đụng vào định dạng số. Nhân loss với một hệ số S cỡ 2^{16} trước khi gọi backward; vì backward tuyến tính theo adjoint đầu vào, mọi gradient bị nhân đúng S lần và cả phổ dịch lên vùng biểu diễn được; trước khi optimizer bước, chia gradient lại cho S . Bản động của kỹ thuật này nhân S lên gấp đôi sau mỗi vài nghìn bước sạch và chia đôi ngay khi phát hiện inf hoặc NaN, bỏ luôn bước đó — tức là nó tự dò biên trên của dải động.



Hình 12.11. Vì sao fp16 làm huấn luyện ra NaN, và vì sao loss scaling chữa được. Trục là độ lớn giá trị theo thang log. Dải fp16 (nền xanh) không bao phủ phần dưới của phổ gradient điển hình, nên phần gạch chéo bị làm tròn về 0 — mạng không nổ mà *tê liệt*. Nhân loss với 2^{16} dịch toàn bộ phổ sang phải mà không đổi hình dạng của nó, đưa cả phổ vào vùng biểu diễn được; chia lại trước bước cập nhật thì không mất gì. bf16 giải bài toán theo hướng khác: nó giữ nguyên số bit mũ của fp32 nên dải đủ rộng từ đầu, đổi lại chỉ còn 7 bit định trị.

Hình 12.11 làm rõ một điểm mà hai chữ “mixed precision” che mắt: vấn đề nằm ở *vị trí* của phổ gradient so với dải biểu diễn được, không nằm ở số chữ số có nghĩa. Đó cũng là lý do lời sửa là một phép nhân vô hướng chứ không phải một thuật toán mới.

bf16 giải bài toán theo hướng khác: giữ 8 bit mũ như fp32 nên dải động y hệt fp32 và không cần loss scaling, đổi lại chỉ còn 7 bit định trị.

Chính 7 bit định trị đó dẫn tới điểm kỹ thuật đáng nhớ nhất của cả mục. Độ chính xác tương đối của bf16 là khoảng $2^{-8} \approx 4 \cdot 10^{-3}$. Nhưng ta vừa nói tỉ lệ cập nhật trên trọng số lành mạnh là khoảng 10^{-3} . Nghĩa là bước cập nhật *nhỏ hơn một đơn vị cuối cùng của trọng số*: nếu bạn cộng nó vào một trọng số lưu ở bf16, phép cộng làm tròn về đúng giá trị cũ và bước đi biến mất hoàn toàn. Đây là lý do mọi cài đặt mixed precision nghiêm túc giữ một bản **master weights** ở fp32 cho optimizer, dù forward và backward chạy ở nửa độ chính xác; và cũng là lý do bảng kế toán bộ nhớ không giảm một nửa như người ta hy vọng.

↪ cùng nguyên lý với D/20 — lượng tử hoá lúc suy luận hỏi đúng câu này: bit chia thế nào giữa mũ và định trị.

Cuối cùng, **cắt ngưỡng gradient** (*gradient clipping*) theo chuẩn toàn cục là bắt buộc với RNN và thường cần với transformer: nó chặn bước đi khi mất mát có vách dựng đứng, nơi gradient tại một điểm không còn nói gì đúng về hàm ở khoảng cách một bước.

G. Giới hạn và trường hợp hỏng

Linear scaling rule vỡ ở batch lớn, và nó vỡ theo kiểu khó phát hiện: loss huấn luyện vẫn đẹp, chỉ có độ chính xác kiểm thử tụt. Nếu bạn chỉ theo dõi loss huấn luyện, bạn sẽ kết luận rằng batch lớn hoạt động tốt.

Loss scaling che giấu NaN thật. Bộ chia tỉ lệ động bỏ qua mọi bước có tràn số. Giả sử mô hình của bạn sinh NaN vì một lỗi thật, chẳng hạn chia cho 0 hoặc lấy log của số âm. Bạn sẽ thấy một quá trình huấn luyện tưởng như bình thường, nhưng thực chất nó bỏ một tỉ lệ lớn số bước. Đáng ghi lại số bước bị bỏ như một chỉ báo riêng.

Weight decay áp nhằm chỗ là lỗi phổ biến và tốn kém: các tham số γ, β của normalization và các bias không nên bị suy giảm, vì chúng không phải “độ phức tạp” theo nghĩa mà weight decay muốn phạt, và co chúng về 0 làm hỏng chính phép chuẩn hoá. Phần lớn recipe tốt loại các nhóm này ra khỏi decay một cách tường minh.

Cuối cùng, những con số trong mục này thuộc ba mức tin cậy khác nhau và nên được đối xử khác nhau. Hệ số $1/(1 - \mu)$, tỉ lệ 3,2 của bước Adam đầu tiên, và các ngưỡng của fp16 là suy dẫn hoặc tra cứu, nên chắc chắn. Kết quả linear scaling tới batch 8192 là một phép đo đã công bố trên một cấu hình cụ thể, không phải một định luật. Còn ngưỡng 10^{-3} cho tỉ lệ cập nhật trên trọng số chỉ là kinh nghiệm thực hành, được truyền lại rộng rãi. Tôi chưa thấy nghiên cứu hệ thống nào xác định lại nó cho các kiến trúc hiện đại. Hãy dùng nó như một cờ báo động, đừng dùng như mục tiêu để tối ưu.

CẠM BÃY

`torch.optim.Adam(..., weight_decay= $\lambda$ )` *không* phải AdamW: nó cộng $\lambda\theta$ vào gradient rồi cho cả cục đi qua mẫu số thích nghi, đúng cái lỗi mà AdamW sinh ra để sửa. Muốn weight decay tách rời thì phải dùng AdamW. Đây là lỗi mà rất nhiều code cũ vẫn mắc, và nó đặc biệt khó phát hiện. Hậu quả không phải một exception mà là điểm số thấp hơn vài phần trăm. Mức đó vừa đủ nhỏ để bạn đổ cho kiến trúc, vừa đủ lớn để mất một vị trí trên bảng xếp hạng.

CẦN NHỚ

Optimizer chỉ định hình lại bước đi; nó không cứu được một learning rate sai. Thứ tự đầu tư công sức: learning rate và lịch của nó trước, batch size cùng quy tắc tỉ lệ sau, chọn optimizer cuối cùng. Bấy cái tên trong mạch không phải bấy lựa chọn, mà là bấy lần vá liên tiếp trên cùng một vấn đề. [T2]

H. Kết nối

Mục này giả định gradient đã lành mạnh, tức giả định toàn bộ [C/12/02-lam-cho-training-chay-duoc](#). Nó chuyển tiếp sang [C/12/04-tong-quat-hoa-va-chan-doan](#). Ở đó, nhiều gradient của quy tắc tỉ lệ tuyến tính được nhìn lại như một công cụ chính quy hoá. Cũng ở đó, bảng kế toán bộ nhớ của trạng thái Adam trở thành lý do tồn tại của ZeRO. Phần dải động của fp16 và bf16 là tiền đề trực tiếp cho [D/20-toi-uu-va-nen-mo-hinh](#) (lượng tử hoá), nơi cùng một câu hỏi được đặt cho suy luận thay vì cho huấn luyện, và cho [B/11-gioi-han-tinh-toan](#) về phía băng thông. Nếu bạn muốn thấy các lựa chọn trong mục này

ảnh hưởng thế nào tới kết luận của một paper, [E/24-thi-nghiem-va-ablation](#) là chương nói rằng phần lớn “cải tiến kiến trúc” hoá ra là cải tiến lịch học.

❏ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Momentum sửa được gì mà SGD không làm được, và nó tạo ra vấn đề mới nào buộc Nesterov phải ra đời?
2. AdaGrad và RMSProp khác nhau đúng một phép toán. Phép toán đó là gì, và vì sao khác biệt nhỏ ấy lại là ranh giới giữa “dùng được cho mạng sâu” và “không dùng được”?
3. Vì sao AdamW khác Adam cộng L2, và bạn kỳ vọng khác biệt thể hiện ở đại lượng đo được nào?
4. Vì sao warmup cần thiết với Adam hơn là với SGD, nếu trả lời bằng tính chất của bộ ước lượng $\hat{v}$?
5. Bạn tăng batch size gấp 8 và nhân learning rate lên 8; loss huấn luyện tốt hơn nhưng accuracy kiểm thử tụt 1 phần trăm — hai cách giải thích là gì, và phép đo nào phân biệt chúng?
6. Vì sao huấn luyện ở bf16 vẫn cần giữ trọng số fp32, dù bf16 có dải động bằng fp32?
7. Đường cong học giảm nhanh rồi phẳng ở mức cao — ba nguyên nhân khả dĩ là gì, và ba phép đo nào phân biệt chúng?

❏ 12.5 Tổng quát hoá và chẩn đoán (Generalization and Diagnosis)

PHỤ THUỘC [C/12/03-toi-uu-hoa](#) (nhiều gradient, kế toán bộ nhớ của trạng thái Adam); [B/04-nen-tang-hoc-tu-du-lieu](#) và [B/07-chia-du-lieu-va-danh-gia](#) (vì sao “tổng quát hoá” chỉ có nghĩa khi tập chia đúng).

TÓM TẮT

Mục này xử lý khoảng cách giữa “mạng khớp được dữ liệu huấn luyện” và “mạng dùng được”, rồi biến phần còn lại thành quy trình chẩn đoán. Đọc xong bạn xếp được các công cụ **chính quy hoá** (*regularization*) theo hiệu quả thực tế trong thị giác, thay vì theo độ nổi tiếng. Bạn biết đưa mô hình lên nhiều GPU theo đúng thứ tự tăng dần độ phức tạp. Và bạn có một quy trình bắt lỗi cho kết luận trong vài phút thay vì vài ngày. Nếu chỉ nhớ một điều: trước khi chạy full, hãy *overfit một batch 20–50 mẫu tới loss gần 0*. Phép thử này tách bạch “vòng lặp huấn luyện có bug” khỏi “mô hình không tổng quát hoá được”. Phần lớn thời gian bị mất là do người ta nhầm hai thứ đó với nhau.

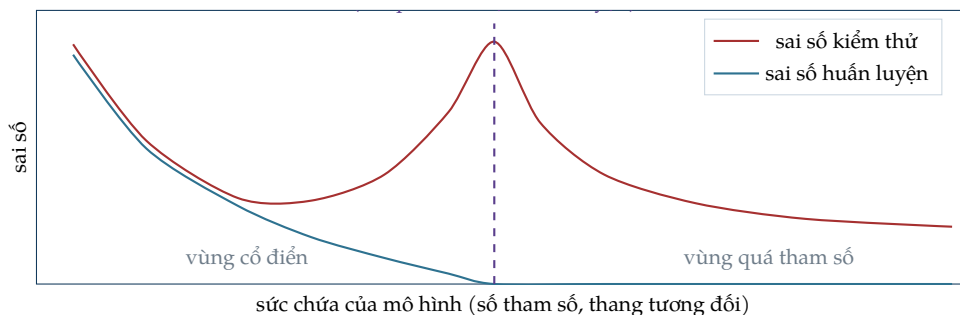
KIẾN THỨC CẦN CÓ TRƯỚC

- **tập huấn luyện, xác thực, kiểm thử** — ba phần rời nhau của dữ liệu: phần để học, phần để chọn siêu tham số và dùng đúng lúc, phần chỉ dùng đúng một lần để báo cáo. Trộn lẫn ba phần là cách phổ biến nhất để tự lừa mình.
- **overfitting** — mô hình khớp rất tốt dữ liệu huấn luyện nhưng kém trên dữ liệu mới, vì nó đã học cả những chi tiết chỉ đúng với tập huấn luyện.
- **chính quy hoá (regularization)** — mọi kỹ thuật nhằm thu hẹp khoảng cách giữa kết quả huấn luyện và kết quả thực tế — bằng cách ràng buộc mô hình, thêm nhiễu, hoặc thêm thông tin.
- **augmentation** — sinh thêm mẫu huấn luyện bằng cách biến đổi mẫu có sẵn (lật, cắt, đổi màu); mỗi phép biến đổi là một tuyên bố “nhân không đổi dưới phép này”.
- **sức chứa (capacity)** — độ phong phú của lớp hàm mà mô hình biểu diễn được, thường xấp xỉ bằng số tham số; lý thuyết cổ điển gắn sức chứa với overfitting, còn thực nghiệm hiện đại thì không.

- **rò rỉ dữ liệu (data leakage)** — thông tin của tập đánh giá lọt vào quá trình huấn luyện, khiến điểm số đẹp giả tạo — ví dụ hai khung hình liên tiếp của cùng một video nằm ở hai bên ranh giới chia tập.
- **tiền trình và GPU** — trong huấn luyện phân tán, mỗi GPU thường chạy một tiến trình riêng giữ một bản mô hình; “đồng bộ” nghĩa là các tiến trình trao đổi gradient để giữ các bản đó giống nhau.
- **EMA (trung bình trượt lũy thừa)** — một bản trọng số phụ được cập nhật bằng trung bình có trọng số giảm dần theo thời gian; nó mượt hơn trọng số hiện thời và thường cho kết quả đánh giá tốt hơn.

A. Đặt vấn đề

Loss huấn luyện xuống đẹp, độ chính xác trên tập kiểm thử thì không. Đây là bài toán cũ nhất của học máy, nhưng cách hiểu nó trong deep learning đã thay đổi hẳn và việc mang nguyên trực giác cũ sang là một nguồn suy luận sai lớn. Trực giác cổ điển nói: mô hình quá nhiều tham số so với dữ liệu thì học thuộc lòng, nên hãy giảm sức chứa mô hình. Nhưng một mạng hiện đại khớp hoàn hảo được cả một tập ImageNet với nhãn đã bị xáo ngẫu nhiên [54]. Sức chứa của nó dư sức học thuộc. Vậy mà chính nó vẫn tổng quát hoá tốt khi nhãn là chân trị. Các cận sai số dựa trên sức chứa do đó gần như rỗng, và hiện tượng *double descent* còn cho thấy tăng sức chứa vượt qua điểm nội suy có thể làm sai số kiểm thử *giảm* trở lại [55], [56].



Hình 12.12. Hiện tượng giảm kép (*double descent*), vẽ định tính. Trực giác cổ điển chỉ mô tả đúng nửa trái: sai số kiểm thử đi xuống rồi đi lên khi mô hình lớn dần. Nhưng ngay tại ngưỡng nội suy — chỗ mô hình vừa đủ sức khớp hết dữ liệu huấn luyện — sai số kiểm thử đạt *đỉnh* rồi *giảm trở lại* khi ta tiếp tục tăng sức chứa. Đây là lý do câu “mô hình quá lớn thì sẽ overfit” không dùng được làm quy tắc, và là lý do phải chọn công cụ chính quy hoá theo đo đặc chứ không theo lý thuyết sức chứa. Hình là minh hoạ định tính; vị trí đỉnh và độ sâu của lần giảm thứ hai phụ thuộc tác vụ và lượng nhiều nhãn.

Hình 12.12 nói rõ điều mà một câu trích dẫn không nói được: hai trực giác đối nghịch nhau đều đúng, chỉ là đúng ở hai nửa khác nhau của cùng một trục.

Hệ quả thực hành rất cụ thể: đừng chọn công cụ theo lý thuyết sức chứa, hãy chọn theo hiệu quả đo được trong đúng lĩnh vực của bạn. Và vì phần lớn các lần “mô hình không tổng quát hoá” thực ra là “vòng lặp huấn luyện có lỗi”, phần thứ hai của mục này quan trọng không kém phần thứ nhất. Giá trị cụ thể: sau mục này bạn có một cây quyết định để chẩn đoán một lần chạy hỏng, và một bảng triệu chứng để tra thay vì đoán.

↔ **tiền đề B/07** — khoảng cách train-val chỉ có nghĩa nếu tập được chia đúng; chia sai thì mọi kết luận ở đây sụp.

TRỰC GIÁC

Chính quy hoá thường được kể như “ràng buộc mô hình cho nó bớt phức tạp”. Nhưng công cụ mạnh nhất trong thị giác lại là augmentation, và nó không ràng buộc gì cả: nó

thêm thông tin. Hãy nghĩ theo kiểu người hiệu chuẩn cảm biến: khi bạn lắc camera qua nhiều tư thế lúc hiệu chuẩn, bạn không làm mô hình camera đơn giản đi, bạn đang nói cho bộ ước lượng biết đại lượng nào *không được phép* thay đổi kết quả. Augmentation làm đúng việc đó với mạng: mỗi phép biến đổi là một câu tuyên bố “nhân không đổi dưới phép này”. Vì thế nó thắng các công cụ ràng buộc sức chứa — nó trả lời một câu hỏi mà chúng không trả lời được.

B. Bộ công cụ chính quy hoá, xếp theo hiệu quả thực tế trong thị giác

Thứ tự dưới đây là thứ tự *hiệu quả thực hành trong computer vision*, không phải thứ tự nổi tiếng. Nó khác đáng kể thứ tự mà sách giáo khoa trình bày.

- **Augmentation** — gần như luôn cho lợi ích lớn nhất trên mỗi giờ công bỏ ra, vì nó là công cụ duy nhất trong danh sách *thêm thông tin* thay vì lấy bớt tự do. Chi tiết ở mục C.
- **Early stopping** — dừng khi loss xác thực bắt đầu xấu. Gần như miễn phí, không có siêu tham số ngoài kiên nhẫn, và trong nhiều recipe nó là thứ đang âm thầm gánh phần lớn công việc chính quy hoá.
- **Weight decay** — vẫn hữu ích, nhưng cơ chế thật của nó trong mạng có normalization không phải cơ chế trong sách. Với một tầng conv đứng ngay trước BatchNorm, nhân W với một hằng số *không đổi* hàm số mà mạng biểu diễn, vì BN chuẩn hoá lại ngay sau đó. Vậy weight decay không thu hẹp lớp hàm; cái nó thật sự điều khiển là *độ lớn của bước học hiệu dụng*, vì bước tương đối $\|\Delta W\|/\|W\|$ tăng khi $\|W\|$ bị co lại. Đây là lý do weight decay và learning rate không độc lập với nhau như tên gọi gợi ý.
- **EMA của trọng số** — giữ một bản trung bình trượt của trọng số và dùng nó để đánh giá. Rẻ, gần như luôn cải thiện một chút, và cơ chế thì đã nói ở mục trước: SGD ở cuối quá trình lang thang trong một quả cầu bán kính tỉ lệ với η , nên lấy trung bình là đứng vào tâm quả cầu đó thay vì một điểm ngẫu nhiên trên nó.
- **Stochastic depth** — bỏ ngẫu nhiên cả một khối residual trong lúc huấn luyện [128]. Hợp với mạng rất sâu; nó là “dropout ở cấp khối” và không phá thống kê không gian như dropout cấp phần tử.
- **Dropout** — *đã mất phần lớn vai trò* trong CNN thị giác hiện đại [129]. Có hai lý do. BatchNorm đã bơm sẵn nhiễu qua thống kê batch, nên nhiễu thêm là thừa. Và hai kỹ thuật xung khắc nhau: dropout làm phương sai kích hoạt lúc huấn luyện khác lúc suy luận, đúng đại lượng mà BN đang cố cố định. Nó vẫn sống khoẻ ở tầng đầu ra của transformer và ở các mô hình ngôn ngữ.

Điểm đáng nhớ của cả danh sách không phải thứ tự mà là *vì sao* thứ tự đó lệch với sách vở: sách xếp theo mức độ ràng buộc lớp hàm, thực tế xếp theo lượng thông tin mà công cụ mang thêm vào hoặc lượng nhiễu mà nó khớp với chế độ suy luận.

C. Augmentation là nơi bạn phát biểu invariance

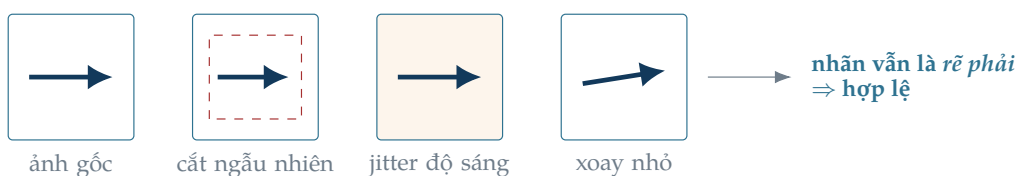
Câu hỏi “augmentation nào là đúng” không có câu trả lời chung, nhưng có một cách nhìn thống nhất làm nó trở thành câu hỏi trả lời được: *mỗi phép augmentation là một tuyên bố về tính bất biến mà bạn muốn mô hình có*. Lật ngang là tuyên bố “lớp không đổi khi lật trái phải”. Jitter màu là tuyên bố về bất biến với điều kiện chiếu sáng. Cắt ngẫu nhiên là tuyên bố “đối tượng có thể ở bất kỳ đâu và bất kỳ tỉ lệ nào”. Từ đó suy ra ngay quy tắc chọn: hãy đưa vào đúng những phép biến đổi thật sự xảy ra trong dữ liệu triển khai của bạn, và loại bỏ những phép phá huỷ nhân.

Chính vì vậy augmentation có ý nghĩa vật lý mạnh hơn augmentation ngẫu nhiên. Nếu camera của bạn có nhiều đọc, nhiều bắn photon và một đường ống ISP cụ thể, thì mô phỏng đúng chuỗi đó (xem [A/02-vat-ly-tao-anh](#)) tạo ra các mẫu nằm trên đúng đa tạp mà mô hình sẽ gặp; còn cộng nhiều Gauss vào ảnh đã qua ISP thì tạo ra các mẫu không tồn tại trong thực tế.

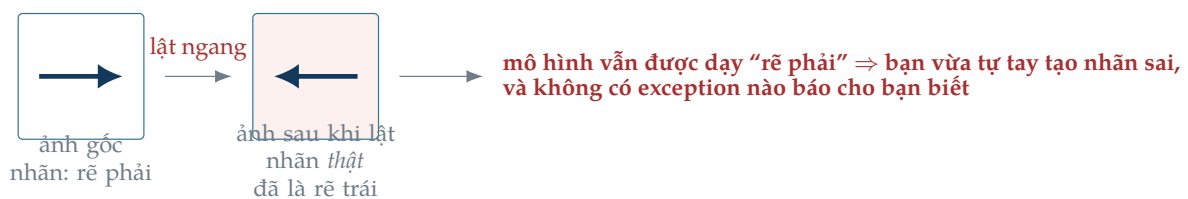
→ hệ quả của [A/02](#) — chuỗi tạo ảnh vật lý quyết định phép biến đổi nào sinh ra mẫu có thật, phép nào sinh ra mẫu không bao giờ gặp.

Nhóm hiện đại thì mở rộng ý tưởng theo hai hướng khác nhau: *Mixup* và *CutMix* trộn hai mẫu và trộn cả nhãn theo cùng tỉ lệ [130], [131], tức tuyên bố một giả định về tính tuyến tính giữa các lớp chứ không còn về bất biến hình học; *RandAugment* thì tự động hoá việc chọn cường độ và số phép [132], giảm không gian tune xuống hai con số; còn copy-paste và mosaic ghép các đối tượng vào bối cảnh mới, đặc biệt hiệu quả cho detection vì chúng tăng số đối tượng mỗi ảnh.

Giữ nhãn — mỗi phép này là một tuyên bố “lớp không đổi dưới phép biến đổi này”



Phá nhãn — cùng một phép, sai bài toán



Hình 12.13. Augmentation là nơi bạn phát biểu tính bất biến, nên nó chỉ đúng khi tuyên bố đó đúng. Hàng trên: bốn phép biến đổi không đụng tới thứ quyết định nhãn, nên chúng dạy mô hình bỏ qua các chiều biến thiên vô nghĩa. Hàng dưới: cùng một phép lật ngang, nhưng ở bài toán mà hướng chính là lớp, nó tạo ra một cặp ảnh–nhãn mâu thuẫn. Đây cũng là lý do việc chọn augmentation phải bắt đầu từ câu hỏi “cái gì được phép đổi mà nhãn không đổi”, chứ không phải từ danh sách mặc định của thư viện.

Hình 12.13 cho thấy vì sao không tồn tại một bộ augmentation “tốt” chung: cùng một phép biến đổi vừa là công cụ mạnh nhất vừa là nguồn nhãn sai, tùy vào bài toán.

Chỗ hỏng thì rất cụ thể và đáng thuộc: augmentation phá nhãn. Lật ngang một ảnh chữ số hoặc biến báo rẽ trái là tạo ra một mẫu có nhãn sai. Jitter màu trong bài toán mà màu chính là lớp (đèn giao thông, phân loại quả chín) cũng vậy. Xoay lớn trong ảnh trên cao thì hợp lý, trong ảnh chụp bởi camera gắn xe thì không, vì trọng lực là một tiên nghiệm thật.

D. Từ một GPU tới nhiều GPU

Bốn bước dưới đây xếp theo đúng thứ tự tăng dần độ phức tạp và mức tiết kiệm bộ nhớ. Nguyên tắc là chỉ leo lên bước sau khi bước trước đã hết dư địa. Trước khi đọc bảng, hãy cố định bảng kế toán. Với P tham số huấn luyện ở fp32 bằng Adam, mỗi GPU phải giữ $4P$ byte trọng số, $4P$ byte gradient và $8P$ byte trạng thái Adam. Tổng là $16P$ byte, tức 4,8 GB khi $P = 300$ triệu — trước khi tính một byte kích hoạt nào.

Bảng 12.5. Bốn mức song song hoá dữ liệu. Cột bộ nhớ tính cho N tiến trình và bỏ qua kích hoạt; cột cuối là cột đáng đọc kỹ nhất.

Bước	Cơ chế	Bộ nhớ mỗi GPU	Hỏng khi nào
DataParallel	Một tiến trình, phát tán mô hình sang các GPU mỗi bước	$16P$	Nghẽn ở GIL và ở việc sao chép mô hình; GPU 0 quá tải. Coi như đã lỗi thời
DDP	Mỗi GPU một tiến trình, all-reduce gradient sau backward	$16P$	Seed và sampler không đặt đúng thì các tiến trình thấy cùng dữ liệu; BatchNorm không đồng bộ nên mỗi GPU là một mô hình hơi khác
Gradient accumulation	Chia batch thành micro-batch, cộng dồn gradient rồi mới bước	$16P$, nhưng kích hoạt giảm theo micro-batch	Không giảm bộ nhớ trạng thái; thống kê BatchNorm vẫn tính theo micro-batch nên batch hiệu dụng lớn mà thống kê thì không
ZeRO / FSDP [122]	Chia nhỏ trạng thái (giai đoạn 1), gradient (2), rồi cả trọng số (3) giữa các tiến trình	$8P + 8P/N$ tới $16P/N$	Lưu lượng giao tiếp tăng theo giai đoạn; nghẽn khi mạng liên kết chậm hoặc mô hình nhiều tầng nhỏ

Ba cạm bẫy trong Bảng 12.5 đáng nêu tên riêng vì chúng không gây lỗi mà chỉ gây kết quả kém. Thứ nhất là *seed và sampler*: nếu mọi tiến trình dùng cùng seed cho việc xáo dữ liệu, bạn không huấn luyện trên N lần dữ liệu mà trên cùng một dòng dữ liệu N lần. Thứ hai là *BatchNorm*. Mỗi tiến trình tính thống kê trên batch cục bộ của nó. Với detection chạy một tới hai ảnh mỗi GPU, thống kê nhiều tới mức vô dụng. Đây chính là lý do GroupNorm và SyncBN tồn tại, và cũng là vấn đề đã gặp ở mục 2 của chương này. Thứ ba là *cách gộp loss*: lấy tổng thay vì trung bình làm learning rate hiệu dụng bị nhân với số tiến trình, một lỗi im lặng có hậu quả đúng bằng việc đặt sai learning rate một bậc.

↪ hệ quả của C/12/03 — bảng kế toán $16P$ byte cho trọng số, gradient và trạng thái Adam là lý do tồn tại của ZeRO

Song song với bốn bước trên còn một trục thứ hai đã nhắc ở mục 1: *gradient checkpointing* [9] đổi khoảng 30 phần trăm thời gian tính lấy việc giảm bộ nhớ kích hoạt từ $O(L)$ xuống $O(\sqrt{L})$. Trong thị giác, kích hoạt thường lớn hơn trọng số rất nhiều, nên với một GPU đơn thì checkpointing hầu như luôn là bước nên thử trước ZeRO.

E. Chính quy hoá ngầm: chỗ ngành chưa có câu trả lời

Cần biết mục này để *không suy luận sai*, chứ không phải để dùng. Câu hỏi mở là: vì sao một mạng quá tham số, đủ sức học thuộc nhiều, lại chọn đúng nghiệm tổng quát hoá được khi ta huấn luyện nó bằng SGD? Các mảnh câu trả lời hiện có đều là mảnh: nhiều của SGD thiên vị các nghiệm ở vùng phẳng của mặt mất mát; các thuật toán bậc nhất có xu hướng tìm nghiệm chuẩn nhỏ trong các bài toán tuyến tính quá tham số; khởi tạo và lịch học đều để lại dấu vết trên nghiệm cuối. Chưa có lý thuyết nào ghép các mảnh đó lại thành một lời giải thích đầy đủ cho mạng sâu thật.

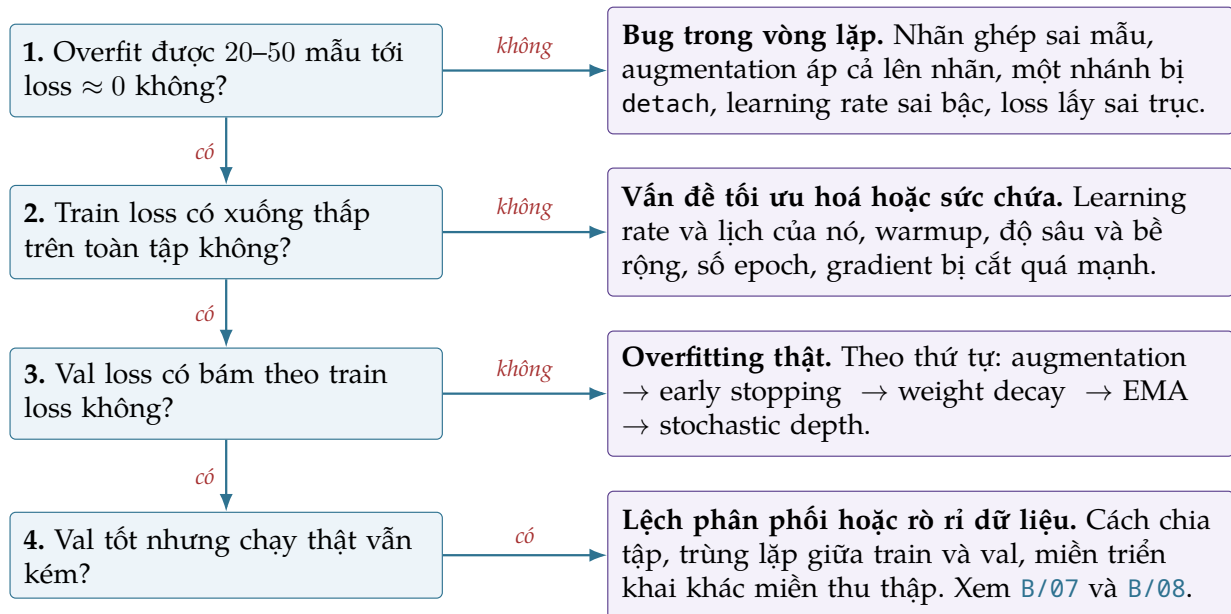
Hệ quả cho cách bạn đọc và nói: đừng kết luận “mô hình này quá lớn nên sẽ overfit” như một mệnh đề chắc chắn, vì nó thường sai trên thực nghiệm; và cũng đừng kết luận ngược lại rằng sức chứa không quan trọng. Phát biểu trung thực là: quá trình huấn luyện, chứ không chỉ lớp hàm, quyết định nghiệm — và ta chưa mô tả được “chứ không chỉ” đó một cách định lượng.

F. Phản xạ debug số một: overfit một batch nhỏ

Trước khi chạy full, lấy 20 tới 50 mẫu, tắt mọi augmentation và mọi chính quy hoá, rồi huấn luyện tới khi loss huấn luyện gần 0. Nếu không đạt được điều đó, dừng lại: bug nằm trong vòng lặp, không nằm ở khả năng tổng quát hoá, và mọi phút bỏ ra để tune siêu tham số lúc này là phí.

Sức mạnh của phép thử nằm ở chỗ nó *loại bỏ đúng các biến gây nhiễu*. Trên 20 mẫu, một mạng có hàng triệu tham số thừa sức nhớ hết, nên nếu nó không nhớ được thì tín hiệu không đi từ dữ liệu tới loss hoặc gradient không đi ngược lại được. Nó kiểm tra toàn tuyến — đọc dữ liệu, tiền xử lý, ghép nhãn, hàm mất mát, backward, bước cập nhật — trong vài phút, thay vì để bạn suy đoán từ một đường cong sau ba giờ. Đây cũng là lý do nó phải là việc đầu tiên, không phải việc cuối cùng.

Hình 12.14 biến phép thử đó thành một cây quyết định đầy đủ. Điểm đáng chú ý là mỗi nhánh dẫn tới một *họ* nguyên nhân khác hẳn nhau, nên trả lời sai câu hỏi đầu tiên khiến bạn đi tìm ở nhầm họ suốt phần còn lại.



Hình 12.14. Cây chẩn đoán một lần chạy không đạt kỳ vọng. Bốn câu hỏi phải hỏi theo đúng thứ tự này, vì mỗi câu chỉ có nghĩa khi câu trước đã trả lời “có”; bỏ qua câu số một là nguyên nhân phổ biến nhất của việc tune siêu tham số cho một hệ thống đang có bug.

G. Phá hỏng có chủ đích: xây thư viện triệu chứng

Kỹ năng cao nhất của cả chương không phải biết cách sửa mà là biết *nhìn một đường cong và đoán ra nguyên nhân*. Kỹ năng đó không đến từ đọc; nó đến từ việc cố ý làm hỏng từng thứ một và ghi lại hình dạng đường cong tương ứng. Bảng 12.6 là tám phiên bản hỏng đáng chạy, và cột cuối là cột biến bảng này từ một danh sách thành một công cụ chẩn đoán.

Bảng 12.6. Tám lỗi cố ý và triệu chứng của chúng. Chạy cả tám, vẽ tám đường cong vào một hình duy nhất, và giữ hình đó.

Phá hỏng	Triệu chứng trên đường cong	Phân biệt bằng phép đo nào
Learning rate $\times 100$	Loss vọt lên rồi thành NaN trong vài chục bước, hoặc phẳng ngay ở mức phỏng đoán ngẫu nhiên	Tỉ lệ update trên trọng số ở bước đầu vượt 10^{-1} ; chuẩn gradient tăng theo bước
Learning rate $\div 100$	Loss giảm gần như tuyến tính, không có dấu hiệu bão hoà khi hết lịch	Tỉ lệ update trên trọng số dưới 10^{-5}
Quên shuffle	Loss răng cưa với chu kỳ đúng bằng một epoch	Vẽ loss theo từng bước thay vì theo epoch; hình răng cưa lộ ra ngay
Quên eval()	Val loss thấp bất thường và <i>đổi</i> khi đổi batch size lúc đánh giá	Chạy val hai lần với batch 1 và batch 64, so kết quả
Nhân lệch một chỉ số	Train loss giảm bình thường; val accuracy quanh mức ngẫu nhiên	Ma trận nhầm lẫn lệch hẳn khỏi đường chéo một cột
Quên chuẩn hoá đầu vào	Hội tụ chậm hơn rõ rệt; nếu có BatchNorm thì gần như không thấy gì	Histogram của batch đầu vào; thống kê kích hoạt tầng đầu
Khởi tạo toàn 0	Loss đứng đúng mức ngẫu nhiên, không nhích	Các hàng của W giống hệt nhau sau vài bước: đối xứng chưa bị phá
Dropout 0,9	Train loss <i>cao hơn</i> val loss; cả hai hội tụ rất chậm	Khoảng cách train trừ val mang dấu âm — điều gần như không bao giờ xảy ra khi mọi thứ đúng

H. Giới hạn và trường hợp hỏng

Phép thử “overfit một batch nhỏ” có một chế độ hỏng đáng biết, và nó là loại chế độ hỏng khó chịu nhất: phép thử *qua* trong khi hệ thống vẫn sai. Nếu nhân của bạn bị hoán vị theo một quy tắc cố định — ví dụ chỉ số lệch một — thì trên 20 mẫu, mạng học thuộc ánh xạ sai đó dễ dàng và loss vẫn về 0. Cùng lý do, một bug trong tiền xử lý áp giống nhau cho mọi mẫu cũng không bị bắt. Nói cách khác, phép thử chứng minh “tín hiệu chảy được từ đầu vào tới loss và ngược lại”, không chứng minh “tín hiệu chảy đúng thứ”. Bổ sung rẻ nhất là nhìn trực tiếp vài chục mẫu đã qua toàn bộ pipeline, kèm nhãn, ngay trước khi vào mô hình.

↪ **tiền đề E/24** — ablation chạy trên một hệ thống còn bug là vô nghĩa, nên phép thử này phải đứng trước mọi thí nghiệm so sánh

Bảng triệu chứng cũng có giới hạn: nó được lập trên một cấu hình cụ thể, và một số triệu chứng bị normalization che mất — “quên chuẩn hoá đầu vào” gần như vô hình khi mạng có BatchNorm ở tầng đầu. Đây là mặt trái đã nói ở mục 2: normalization làm hệ thống bền hơn với lỗi và vì thế xoá bớt các triệu chứng mà bạn muốn học đọc. Cuối cùng, thứ tự ưu tiên các công cụ chính quy hoá ở mục B chỉ là *kinh nghiệm thực hành trong thị giác*. Nó được tổng hợp từ các recipe phổ biến, không phải từ một nghiên cứu so sánh có kiểm soát. Ở lĩnh vực khác, chẳng hạn mô hình ngôn ngữ, thứ tự đó khác.

CẠM BẤY

Rõ rĩ dữ liệu là cách phổ biến nhất để có một đường cong xác thực đẹp và một hệ thống vô dụng, và nó gần như không bao giờ trông giống một lỗi. Ba dạng hay gặp trong thị giác: chia tập theo khung hình thay vì theo chuỗi, khiến hai khung hình cách nhau 1/30 giây nằm ở hai bên ranh giới; augmentation hoặc chuẩn hoá được tính trên *toàn bộ* dữ liệu trước khi chia; và các ảnh trùng lặp hoặc gần trùng nằm ở cả hai phía. Dấu hiệu nhận biết: khoảng cách train–val nhỏ bất thường, và điểm số tụt mạnh ngay khi bạn đánh giá trên dữ liệu thu ở một buổi khác.

CẦN NHỚ

Trước khi tune bất cứ thứ gì, overfit 20–50 mẫu tới loss gần 0. Đây là ranh giới tách “vòng lặp có bug” khỏi “mô hình không tổng quát hoá được”. Hai họ nguyên nhân đó cần hai bộ công cụ hoàn toàn khác nhau. Nhắm chúng với nhau là cách tốn thời gian nhất trong nghề. [T1]

I. Kết nối

Mục này khép lại chương 12 và mở sang hai hướng. Về phía dữ liệu, “tổng quát hoá” chỉ có nghĩa khi tập được chia đúng, nên [B/07-chia-du-lieu-va-danh-gia](#) là tiền đề bắt buộc của cả mục, và [B/08-dich-chuyen-phan-phoi](#) là chương giải thích nhánh cuối cùng của cây chẩn đoán. Về phía kiến trúc, thư viện triệu chứng ở mục G là công cụ bạn sẽ dùng liên tục khi đọc [C/13-kien-truc-backbone](#). Bài học lớn nhất của chương đó là ConvNeXt: phần lớn chênh lệch giữa các kiến trúc hoá ra là chênh lệch công thức huấn luyện. Bài học ấy chỉ có nghĩa nếu bạn phân biệt được hai nguồn. Cuối cùng, sự nghi ngờ có kỷ luật với một bảng so sánh là nội dung của [E/24-thi-nghiem-va-ablation](#).

I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao lập luận “mô hình nhiều tham số hơn số mẫu thì sẽ học thuộc lòng” không còn dùng được, và điều đó thay đổi cách bạn chọn công cụ chính quy hoá thể nào?
2. Trong một mạng có BatchNorm, weight decay thật sự điều khiển đại lượng nào? Vì sao nó không thu hẹp lớp hàm?
3. Augmentation nào bạn phải *bỏ* nếu bài toán là phân loại đèn giao thông, và vì sao?
4. Gradient accumulation cho batch hiệu dụng lớn hơn nhưng không sửa được một vấn đề của BatchNorm. Vấn đề đó là gì?
5. Phép thử “overfit một batch nhỏ” qua được nhưng hệ thống vẫn sai — tình huống cụ thể nào gây ra chuyện đó, và phép kiểm tra bổ sung nào bắt được nó?
6. Bạn thấy train loss cao hơn val loss — hai nguyên nhân khả dĩ là gì?
7. ZeRO giai đoạn 3 giảm bộ nhớ xuống $16P/N$; nó đổi lấy cái gì, và khi nào phép đổi đó không đáng?

I 12.6 Thực hành

Chương này khác các chương khác ở một điểm: đọc hiểu không tạo ra kỹ năng. Hai mục dưới đây là phần đọc và phần tự vấn; phần tay chân nằm ở khối mini project cuối chương, và đó mới là chỗ kiến thức chuyển thành phản xạ.

Repo và tài nguyên. Để hiểu autodiff từ gốc, [karpathy/micrograd](#) là bản cài đặt reverse-mode gọn trong khoảng một trăm dòng và đọc hết trong một buổi; [tinygrad/tinygrad](#) là bước tiếp theo, đủ nhỏ để đọc nhưng đã có đầy đủ khái niệm của một framework thật. Để tự derive gradient theo lối bài tập, bộ assignment của CS231n vẫn là tài liệu tốt nhất, và [labmlai/annotated_deep_learning_paper_implementations](#) cho bạn bản cài đặt có chú giải của phần lớn kỹ thuật nêu trong chương. Về augmentation, [alumentations-team/alumentations](#) và `transforms.v2` là hai lựa chọn mặc định. Về huấn luyện phân tán, đọc trực tiếp tài liệu DDP và FSDP của PyTorch trước, rồi mới tới [microsoft/DeepSpeed](#) nếu cần các giai đoạn ZeRO sâu hơn. Danh sách này chốt tại tháng 9/2026, và tên các mô hình dẫn đầu sẽ cũ trong sáu tới mười hai tháng. Nhưng phần lớn nội

dung chương này là hạ tầng chứ không phải mô hình, nên nó bền hơn hẳn mức trung bình của cây.

Câu hỏi kiểm tra hiểu. Vì sao AdamW khác Adam cộng L2, và bạn kỳ vọng khác biệt đó lớn cỡ nào trong thực tế? Một đường cong học giảm rất nhanh rồi phẳng ở mức cao — nêu ba nguyên nhân khả dĩ và ba phép đo phân biệt chúng. Vì sao BatchNorm làm lượt ngược tốn thêm bộ nhớ, và điều đó gợi ý gì khi bạn phải chọn giữa BatchNorm và GroupNorm ở một bài detection chạy hai ảnh mỗi GPU?

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Lan truyền ngược	Thuật toán tính gradient bằng cách đi ngược đồ thị phép tính đúng một lượt. Nó rẻ hơn vì phân thuận khoảng tám bậc độ lớn, vì một lượt trả về đạo hàm theo <i>mọi</i> tham số. Nó không bao giờ dựng ma trận Jacobian, chỉ áp tích vector-Jacobian ở từng nút.
•• Lượt xuôi, lượt ngược	Lượt xuôi tính giá trị và ghi lại kết quả trung gian; lượt ngược dùng chúng để tính gradient. Cả hai là <i>một lần chạy</i> của lan truyền ngược, đừng nhầm với chính thuật toán.
••• Ba mẫu gradient	Cộng phân phối adjoint đều cho mọi nhánh. Nhân hoán đổi: mỗi nhánh nhận adjoint nhân giá trị nhánh kia. Max định tuyến toàn bộ về nhánh thắng. Mẫu thứ tư hay bị quên: rẽ nhánh thì cộng dồn, nên lượt ngược của broadcast là một phép cộng theo trục.
••• Bộ nhớ kích hoạt	Toàn bộ đầu ra trung gian của lượt xuôi, phải giữ lại vì lượt ngược cần tới. Nó đạt đỉnh ở chỗ hai lượt giao nhau, nên nó mới là thứ chặn batch chứ không phải số trọng số. Gradient checkpointing hạ đỉnh xuống $O(\sqrt{L})$ với giá khoảng 30 phần trăm thời gian.
••• Vì sao gradient tắt hoặc nổ khi mạng sâu?	Vì adjoint bị nhân dồn qua từng tầng. Nếu hệ số mỗi tầng là ρ thì gradient ở tầng đầu tỉ lệ với ρ^L ; $\rho = 0,9$ qua 50 tầng còn $5 \cdot 10^{-3}$.
••• Khởi tạo He	Đặt $\text{Var}(w) = 2/n_{\text{in}}$ để phương sai tín hiệu không đổi qua mỗi tầng. Hệ số 2 đến từ việc ReLU xóa một nửa số đầu ra, nên nó đúng cho ReLU chứ không đúng cho tanh.
••• BatchNorm	Chuẩn hoá kích hoạt theo thống kê của cả batch, rồi trả quyền chính lại cho mạng qua γ và β . Vì nó ghép các mẫu trong batch lại với nhau, nó hỏng khi batch nhỏ, khi train và eval dùng hai nguồn thống kê, và khi các GPU không đồng bộ.
•• Val loss thấp bất thường, và đổi khi đổi batch size lúc đánh giá — nguyên nhân?	Quên gọi <code>model.eval()</code> , nên BatchNorm dùng thống kê của chính batch kiểm thử. Phân biệt với rò rỉ dữ liệu: rò rỉ cho điểm cao ổn định, còn lỗi này đổi điểm khi bạn đổi batch size lúc đánh giá.
••• Vì sao momentum thắng trong rãnh hẹp?	Thành phần vuông góc với rãnh đổi dấu mỗi bước nên triệt tiêu trong tổng trượt. Thành phần dọc rãnh giữ nguyên dấu nên cộng dồn tới hệ số $1/(1 - \mu)$, gấp 10 lần khi $\mu = 0,9$.
••• AdamW	Adam với weight decay tách khỏi mẫu số thích nghi, cộng thẳng vào bước cập nhật. Nhờ đó suy giảm là một phép co đều, không phụ thuộc lịch sử gradient. <code>Adam(weight_decay=...)</code> <i>không</i> phải AdamW.
••• Vì sao learning rate là siêu tham số quan trọng nhất?	Vì nó là đại lượng duy nhất nhân trực tiếp vào mọi bước cập nhật. Sai một bậc ở nó đổi hẳn bản chất quá trình; sai một bậc ở weight decay chỉ mất vài phần trăm điểm số. Warmup tồn tại cũng vì lý do này: $\hat{v}$ của Adam ở vài trăm bước đầu quá nhiều để tin.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
•• Vì sao fp16 làm huấn luyện tê liệt?	Gradient thị giác thường nằm quanh 10^{-7} tới 10^{-4} , dưới ngưỡng chuẩn hoá $6,1 \cdot 10^{-5}$ của fp16, nên bị làm tròn về 0. Loss scaling nhân loss với 2^{16} để dịch cả phổ vào vùng biểu diễn được, rồi chia lại trước bước cập nhật.
••• Loss thành NaN trong vài chục bước đầu — nguyên nhân?	Learning rate quá lớn, hoặc tràn số ở fp16. Phân biệt: hạ learning rate mười lần mà vẫn NaN thì là tràn số, hãy thử bf16 hoặc bật loss scaling.
••• Loss giảm gần tuyến tính và không bão hoà khi hết lịch — nguyên nhân?	Learning rate quá nhỏ. Phân biệt với mô hình thiếu sức chứa: đo tỉ lệ cập nhật trên trọng số, dưới 10^{-5} là learning rate, còn quanh 10^{-3} mà loss vẫn cao thì là sức chứa hoặc dữ liệu.
•• Loss răng cưa với chu kỳ đúng bằng một epoch — nguyên nhân?	Quên shuffle, nên mô hình gặp lại đúng thứ tự dữ liệu mỗi vòng. Phân biệt với learning rate quá cao: vẽ loss theo từng bước, răng cưa có chu kỳ đều là shuffle, còn nhiều không chu kỳ là learning rate.
••• Train loss giảm bình thường nhưng val accuracy quanh mức ngẫu nhiên — nguyên nhân?	Nhân bị lệch chỉ số hoặc ghép sai mẫu: mạng học thuộc một ánh xạ sai nhưng nhất quán. Phân biệt bằng ma trận nhầm lẫn — nó lệch hẳn khỏi đường chéo đúng một cột.
•• Augmentation	Sinh thêm mẫu bằng cách biến đổi mẫu có sẵn. Mỗi phép là một tuyên bố “nhân không đổi dưới phép này”, nên nó thêm thông tin chứ không ràng buộc sức chứa. Nó hỏng khi phép biến đổi phá nhân, ví dụ lật ngang một biến báo rẽ.
•• ZeRO và FSDP	Chia trọng số, gradient và trạng thái optimizer giữa các tiến trình, theo ba giai đoạn tăng dần. Nó là hệ quả trực tiếp của bảng kế toán $16P$ byte, và trả giá bằng lưu lượng giao tiếp.
••• Overfit một batch nhỏ	Huấn luyện 20–50 mẫu tới loss gần 0 sau khi tắt mọi augmentation. Không đạt thì bug nằm trong vòng lặp chứ không ở khả năng tổng quát hoá. Nó vẫn qua nếu nhân bị hoán vị theo một quy tắc cố định.
• Double descent	Vượt qua ngưỡng nội suy, sai số kiểm thử đạt đỉnh rồi <i>giảm trở lại</i> khi mô hình lớn thêm. Hệ quả: trực giác chữ U của giáo trình cũ không dùng được cho mạng quá tham số.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
PyTorch autograd, JAX	Toàn bộ cơ chế tính gradient là hiện thực của vi phân ngược trên đồ thị: mỗi phép toán khai báo một tích vector-Jacobian	Hạ tầng bền; API gần như không đổi từ 2017, nên đây là phần ít mất giá nhất của chương
FlashAttention	Tính lại theo ô thay vì lưu ma trận $T \times T$ — chính là ý tưởng gradient checkpointing áp cho attention, đổi phép tính lấy bộ nhớ	Mặc định trong phần lớn thư viện transformer hiện nay
DeepSpeed ZeRO, PyTorch FSDP	Hệ quả trực tiếp của bảng kế toán $16P$ byte: chia trọng số, gradient và trạng thái Adam giữa các tiến trình	Ba giai đoạn ZeRO tương ứng ba dòng của bảng đó
AdamW làm mặc định trong timm , transformers , và phần lớn recipe thị giác	Tách weight decay khỏi mẫu số thích nghi — đúng bước cuối của mạch bày optimizer	Lỗi Adam cộng L2 vẫn còn trong nhiều mã nguồn cũ
<code>torch.amp</code> , bf16 trên GPU thể hệ Ampere trở đi	Loss scaling và master weights fp32 — hai hệ quả trực tiếp của phân tích dải động	Bf16 làm loss scaling gần như không cần, nhưng không bỏ được master weights

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
RandAugment, Mixup và CutMix trong <code>timm</code> và <code>transforms.v2</code>	Augmentation như nơi phát biểu bất biến, và như công cụ chính quy hoá mạnh nhất trong thị giác	Chính các thành phần recipe này là thứ ConvNeXt dùng để bắt kịp ViT ở chương 13

MINI PROJECT SLAM · Bảng triệu chứng cho mạng chấm điểm ghép đặc trưng · 1–2 ngày

Mục tiêu	tạo một thư viện triệu chứng dùng được suốt sự nghiệp: huấn luyện một mạng nhỏ dự đoán độ tin cậy của một cặp ghép đặc trưng ORB, rồi cố ý phá nó tám cách để học nhận ra từng dạng hỏng <i>chỉ bằng hình dạng đường loss</i> , trước khi cần tới chúng trong một hệ thật.
Dữ liệu	một chuỗi EuRoC bất kỳ, chẳng hạn MH_01. Trích cặp ghép ORB giữa các khung hình liên tiếp; gán nhãn dương khi sai số tái chiếu dưới ngưỡng theo chân trị pose, âm khi vượt. Vài chục nghìn cặp là quá đủ; đầu vào có thể chỉ là khoảng cách Hamming, chênh lệch góc, chênh lệch tỉ lệ và toạ độ chuẩn hoá.
Các bước	chạy một lần baseline tới khi loss xác thực phẳng và ghi lại ba chỉ báo sức khoẻ; rồi chạy tám phiên bản hỏng có chủ đích — learning rate nhân 100, learning rate chia 100, quên shuffle, quên eval(), nhãn lệch một chỉ số, chuẩn hoá đầu vào bằng thống kê của chuỗi khác, khởi tạo toàn hằng số, tắt chuẩn hoá giữa mạng; vẽ cả chín đường loss vào <i>một</i> hình; cuối cùng che nhãn và tự đoán từng đường là lỗi nào.
Xong khi	bạn có bảng ánh xạ chín đường cong sang chín nguyên nhân, nhận đúng ít nhất bảy trên chín khi che nhãn, và với mỗi cặp dễ nhầm nếu được một phép đo phân biệt — tỉ lệ cập nhật trên trọng số, hoặc mức đổi điểm số khi đổi batch size lúc đánh giá.
Nối với	C/12/04 cho cây chẩn đoán mà bảng này bổ sung số liệu; C/12/03 cho ba chỉ báo sức khoẻ dùng làm phép đo phân biệt; tập cặp ghép đã gán nhãn tạo ở đây được dùng lại nguyên vẹn cho mini project của chương 13.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì vi phân ngược trả về đạo hàm theo *mọi* đầu vào chỉ trong một lượt duy nhất, còn vi phân thuận cho đạo hàm theo đúng một hướng mỗi lượt. Bài toán huấn luyện có một đầu ra và 10^9 đầu vào, tức đúng hình dạng mà vi phân ngược phục vụ; tỉ số chi phí giữa hai cách chính là tỉ số giữa số đầu vào và số đầu ra.
- A2. Vì quy tắc chuỗi cho ra một *tích* của L Jacobian, và tích lũy thừa thì hoặc tắt hoặc nổ — một hệ số 0,9 qua 50 tầng còn $5 \cdot 10^{-3}$. Cái thiếu suốt hai mươi năm không phải công thức đạo hàm mà là ba cách giữ hệ số đó gần 1: hàm kích hoạt không bão hoà, khởi tạo theo lập luận phương sai, và chuẩn hoá.
- A3. Vì chúng nằm trên hai nhánh giải quyết hai vấn đề khác nhau — momentum lọc *hướng*, AdaGrad và RMSProp chỉnh *độ dài bước* cho từng tham số — rồi hợp lại ở Adam. AdamW là cái sinh ra chỉ để sửa lỗi của phép hợp nhất đó: khi hình phạt L2 bị chia cho mẫu số thích nghi, nó không còn là weight decay nữa.
- A4. Vì nghiệm không do lớp hàm quyết định một mình, mà do *quá trình* tìm nghiệm quyết định. Nhiều của SGD, khởi tạo và lịch học đều thiên vị những nghiệm nhất định. Ngành gọi đó là chính quy hoá ngầm và chưa mô tả được nó một cách định lượng — nên các cận sai số dựa trên sức chứa gần như rỗng.
- A5. Vì dải động và độ chính xác là hai chuyện khác nhau. bf16 chỉ có 7 bit định trị, tức độ chính xác tương đối khoảng $4 \cdot 10^{-3}$. Bước cập nhật lành mạnh lại chỉ bằng 10^{-3} độ lớn trọng số. Cộng một số nhỏ như vậy vào trọng số bf16 thì phép cộng làm tròn về giá trị cũ, và bước đi biến mất hoàn toàn.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn phải chỉnh một cây đàn piano khổng lồ có mười triệu sợi dây, và mỗi lần chỉnh xong bạn chỉ được nghe đúng một hợp âm rồi nhận đúng một con số: nghe chênh bao nhiêu. Nếu thử từng dây một — vặn nhẹ, gảy lại, nghe xem con số đổi thế nào — thì bạn cần mười triệu lần gảy chỉ để biết nên vặn theo hướng nào. Không ai làm nổi.

Mẹo của cả chương là đừng đi xuôi mà đi ngược. Sau khi gảy một lần, bạn bắt đầu từ tiếng chênh ở đầu ra và lần ngược qua từng cơ cấu truyền lực, hỏi mỗi cái đúng một câu rất hẹp: kéo đầu ra của anh nhích một li thì đầu vào của anh phải nhích bao nhiêu. Mỗi cơ cấu chỉ cần biết quy đổi nội bộ của riêng nó, và một vòng đi ngược duy nhất trả lời cho cả mười triệu dây.

Cái giá có ba phần. Thứ nhất, muốn đi ngược được thì lượt đi xuôi phải ghi nhớ trạng thái của mọi cơ cấu, nên thứ chặn bạn thường là chỗ để ghi chứ không phải thời gian. Thứ hai, lực truyền qua ba mươi khớp nối: mỗi khớp nuốt một chút thì tới cuối chẳng còn gì, mỗi khớp thêm một chút thì nó giật đứt dây — gần như mọi thủ thuật còn lại chỉ để giữ mỗi khớp truyền đúng bằng lực nhận vào. Thứ ba, vặn bao nhiêu mỗi lần: mạnh tay thì vọt qua, nhẹ tay thì cả đời không xong.

Chỗ không rút gọn được: vì sao một cách chỉnh nằm ở vùng “phẳng” — vùng mà vặn lệch đi một chút vẫn nghe được — lại chơi hay hơn trên những bản nhạc chưa từng thử, thì tới nay chưa có lời giải thích vừa gọn vừa đúng. Mọi cách nói đơn giản hiện có đều bỏ sót một phần ví dụ, nên đây là chỗ nên biết là mình chưa biết.

Một câu: Đi xuôi một lần để ghi nhớ, đi ngược một lần để quy trách nhiệm cho từng núm vặn, rồi vặn thật nhẹ — và mọi thứ còn lại chỉ là giữ cho lực truyền qua ba mươi khớp nối không tắt cũng không giật.

12.7 Kết nối

Chương này đứng ở giữa cây và có hai chiều nối rõ ràng. Ngược lên, nó tiêu thụ [A/01-toan-toi-thieu](#) cho quy tắc chuỗi và lập luận phương sai, và [B/09-thiet-ke-ham-mat-mat](#) cho thứ mà toàn bộ bộ máy này đang tối thiểu hoá. Xuôi xuống, nó là tiền đề của *mọi* chương còn lại trong Phần C: không có nó thì các kiến trúc ở chương 13 tới 18 chỉ là sơ đồ khối không chạy được. Hai nhánh nối ngang đáng đi ngay sau chương này. Thứ nhất là [B/11-gioi-han-tinh-toan](#): ở đó bộ nhớ kích hoạt và dải động số được đặt vào mô hình roofline. Thứ hai là [E/24-thi-nghiem-va-ablation](#): ở đó bạn học cách không để một bảng so sánh dẫn tới kết luận sai, vì bảng ấy thường chỉ phản ánh khác biệt công thức huấn luyện.

Tổng kết chương

Ta đã đi từ một phép nhân Jacobian cục bộ tới việc chia trạng thái optimizer giữa nhiều GPU, và điều đáng giữ lại là toàn bộ hành trình đó có một sợi chỉ duy nhất. Mục 1 cho thấy vì sao đạo hàm rẻ và cái giá của nó là bộ nhớ. Mục 2 cho thấy tích các Jacobian là đại lượng phải kiểm soát, và ba nhóm kỹ thuật tưởng rời rạc đều nhắm vào nó. Mục 3 cho thấy bảy optimizer là bảy lần vá liên tiếp, và learning rate vẫn là siêu tham số không có đối thủ. Mục 4 cho thấy phần lớn các lần “mô hình không tổng quát hoá” thực ra là lỗi trong vòng lặp, và đưa ra quy trình tách hai thứ đó.

Còn bỏ ngỏ hai chỗ lớn. Thứ nhất là chính quy hoá ngầm: ta chưa biết vì sao SGD lại chọn được nghiệm tổng quát hoá tốt trong một không gian đầy nghiệm học thuộc, nên mọi kết luận về sức chứa mô hình đều nên phát biểu thận trọng. Thứ hai là chương này cố tình bỏ qua kết nối tất, thành viên thứ tư của họ “giữ tích Jacobian gần 1”, vì nó là một lựa chọn kiến trúc; nó mở đầu chương tiếp theo. Đọc tiếp [C/13-kien-truc-backbone](#) với một câu hỏi trong đầu: trong mỗi bước tiến hoá kiến trúc, bao nhiêu phần là kiến trúc thật và bao nhiêu phần là công thức huấn luyện vừa học ở đây?

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Sợi chỉ của cả chương là giữ tích các Jacobian không chết cũng không nổ. Khởi tạo, normalization và lịch learning rate tấn công tích đó ở ba chỗ khác nhau nào, và vì sao có cả ba vẫn chưa đủ để bỏ warmup?
2. BatchNorm xuất hiện ở cả bốn mục với bốn vai khác nhau: gradient của một mẫu phụ thuộc cả batch, ảnh hưởng tới số điều kiện, tương tác với weight decay, và giới hạn của gradient accumulation. Bốn điều đó suy ra từ đúng một tính chất — tính chất nào, và nó ép bạn chọn gì khi huấn luyện phân tán?
3. Bộ nhớ kích hoạt của mục 1, trạng thái optimizer của mục 3 và phân mảnh ZeRO của mục 4 là ba khoản chi bộ nhớ khác nhau. Khi một lần chạy hết bộ nhớ, bạn cắt khoản nào trước, và mỗi lựa chọn đòi lấy cái gì?
4. Loss huấn luyện phẳng ở mức cao có thể do hình dạng landscape, do công thức huấn luyện, hoặc do một lỗi trong vòng lặp dữ liệu. Ba phép thử nào tách ba nguyên nhân đó, và vì sao phép “overfit một batch” phải chạy trước hai phép còn lại?
5. Vì sao một cải tiến đo trong công thức huấn luyện cũ có thể biến mất khi đo lại trong công thức mới, và điều đó ràng buộc thế nào cách bạn thiết kế ablation cho những kỹ thuật trong chương này?
6. Mục 4 thừa nhận ta chưa biết vì sao SGD chọn được nghiệm tổng quát hoá tốt. Với khoảng trống đó, quyết định thực hành nào trong chương vẫn giữ nguyên căn cứ, và quyết định nào chỉ còn là kinh nghiệm?
7. Bạn hạ mixed precision từ fp32 xuống bf16 và loss xấu đi một chút. Ba nguyên nhân nằm ở ba mục khác nhau của chương là gì, và mỗi nguyên nhân phân biệt bằng phép đo nào?

Các họ kiến trúc backbone (Backbone Architecture Families)

PHỤ THUỘC C/12-co-che-huan-luyen (toàn chương — không có nó thì mọi kiến trúc ở đây chỉ là sơ đồ khối); B/10-thiet-ke-kien-truc (trực đánh đổi giả định-dữ liệu); B/11-gioi-han-tinh-toan (FLOPs không phải độ trễ).

TÓM TẮT

Chương này đọc lịch sử kiến trúc như *một cuộc tranh luận kéo dài về việc nên cài bao nhiêu giả định vào mô hình*. Mỗi thể hệ trả lời khác nhau, vì lượng dữ liệu sẵn có ở thời điểm đó khác nhau. Đọc xong bạn chọn được *backbone* theo ràng buộc thật của mình — dữ liệu, độ trễ, độ phân giải — thay vì theo bảng xếp hạng, và đọc được cấu trúc một repo thị giáic lạ trong mười phút. Nếu chỉ nhớ một điều: mọi kiến trúc trong chương là một điểm trên cùng một trục. Một đầu là “cài sẵn nhiều giả định, cần ít dữ liệu”, đầu kia là “cài ít giả định, cần nhiều dữ liệu”. Bài học đắt nhất: phần lớn chênh lệch từng được gán cho kiến trúc hoá ra thuộc về công thức huấn luyện.

KIẾN THỨC CẦN CÓ TRƯỚC

- **backbone** — phần mạng làm nhiệm vụ trích đặc trưng từ ảnh, dùng chung được cho nhiều bài toán; phần sinh đầu ra cụ thể được lắp thêm phía sau.
- **inductive bias** — tập giả định mà mô hình mang sẵn trước khi nhìn dữ liệu; nhiều giả định đúng thì học nhanh với ít mẫu, giả định sai thì chặn luôn nghiệm đúng.
- **convolution** — phép nhân một cửa sổ trọng số nhỏ trượt khắp ảnh; vì cùng bộ trọng số dùng cho mọi vị trí, nó cài sẵn hai giả định — quan hệ gần là quan trọng, và mẫu hình có ý nghĩa như nhau ở mọi vị trí.
- **trường tiếp nhận (receptive field)** — vùng pixel đầu vào ảnh hưởng tới một giá trị đầu ra; nó lớn dần theo chiều sâu trong CNN và bằng toàn ảnh ngay từ tầng đầu trong attention toàn cục.
- **attention** — cơ chế cho mỗi vị trí tự chọn lấy thông tin từ mọi vị trí khác bằng trọng số học được, thay vì chờ thông tin lan dần qua từng tầng.
- **FLOPs** — số phép nhân-cộng mà mô hình thực hiện; nó là chỉ số đại diện phổ biến cho chi phí, nhưng *không* tỉ lệ với độ trễ thật vì còn phụ thuộc băng thông bộ nhớ.
- **pretraining** — huấn luyện trước trên một tập rất lớn để học đặc trưng chung, rồi mới huấn luyện tiếp trên tập nhỏ của bài toán đích.

CÂU HỎI MỞ ĐẦU

- Q1. Nếu **attention** cho mọi vị trí nói chuyện trực tiếp với nhau ngay từ tầng đầu tiên, vì sao suốt mười năm người ta vẫn dùng **convolution** — thứ cổ tình chỉ cho phép nhìn một cửa sổ 3×3 ?
- Q2. Một mạng phẳng 56 tầng có sai số *huấn luyện* cao hơn mạng 20 tầng, dù nó thừa sức chứa để bắt chước mạng nông bằng cách để các tầng thừa làm ánh xạ đồng nhất. Chuyện gì đã xảy ra?
- Q3. MobileNet giảm số phép nhân khoảng 8,7 lần. Vì sao độ trễ đo được trên GPU lại không giảm 8,7 lần?
- Q4. Nếu ConvNeXt cho thấy CNN vẫn ngang ViT khi dùng cùng công thức huấn luyện, thì mười năm “cải tiến kiến trúc” đã thực sự cải tiến cái gì?

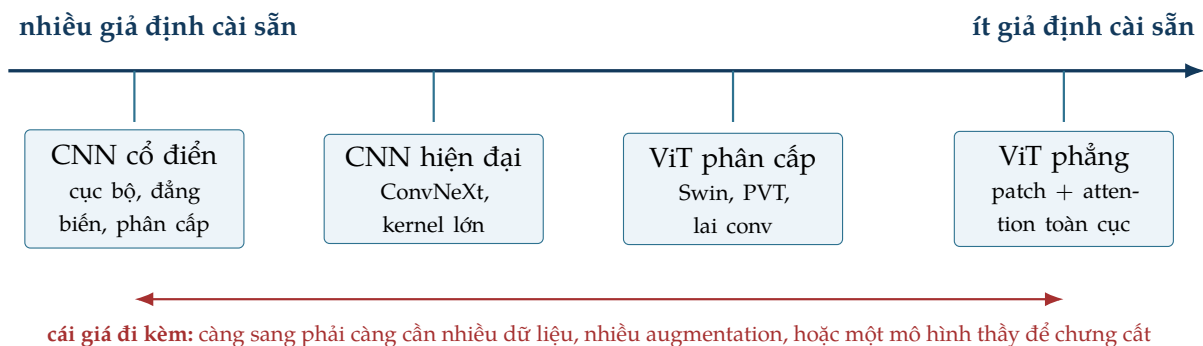
Q5. Vì sao cùng một kiến trúc lại thua CNN khi có một triệu ảnh và thắng khi có ba trăm triệu ảnh — điều gì trong mô hình thay đổi theo lượng dữ liệu?

13.1 Vì sao chương này chia làm ba mạch

Ba mục của chương không phải ba họ kiến trúc song song mà là ba lần trả lời cùng một câu hỏi, trên ba loại dữ liệu và ở ba thời điểm khác nhau. Đọc theo thứ tự này thì mục sau luôn có sẵn khái niệm mà mục trước dựng lên.

1. **Mạch 1 — dòng CNN.** Bắt đầu từ câu trả lời “cài nhiều giả định”, rồi theo dõi mười lăm năm gõ từng nút thắt. Mục này dựng lên các khái niệm nền của cả chương: trường tiếp nhận, chi phí tính toán, kết nối tắt, và bài học ConvNeXt về recipe.
2. **Mạch 2 — từ chuỗi tới attention.** Chuyển sang loại dữ liệu mà convolution không phục vụ được — dữ liệu có thứ tự và quan hệ xa — và theo dõi chuỗi RNN, LSTM, attention, transformer, rồi các cách chống chi phí bậc hai. Đây là mục cung cấp cơ chế cho mạch 3.
3. **Mạch 3 — ViT và inductive bias.** Mang attention vào ảnh, tức chọn câu trả lời “cài rất ít giả định”, rồi đo cái giá. Mục này chỉ đọc được sau hai mục trước, vì nó liên tục so sánh với cả hai.

Hình 13.1 đặt cả ba mạch lên một trục duy nhất. Điều đáng chú ý là trục này không có điểm “tốt nhất”: vị trí đúng phụ thuộc vào lượng dữ liệu bạn có, và đó chính là nội dung của cuộc tranh luận.



Hình 13.1. Bốn họ kiến trúc đặt trên trục “lượng giả định cài sẵn”. Trục này không có điểm tối ưu tuyệt đối: vị trí đúng phụ thuộc lượng dữ liệu có sẵn mà bạn nắm trong tay, nên cùng một bài toán có thể đổi câu trả lời khi tập dữ liệu lớn lên.

Bản đồ chương 13 — Các họ kiến trúc backbone

TÓM TẮT

Chương 13 gồm ba mục, và chúng là ba lần trả lời *cùng một câu hỏi*: nên cài sẵn bao nhiêu giả định vào mô hình. Ba lần đó diễn ra trên ba loại dữ liệu và ở ba thời điểm khác nhau. Thứ tự 1 → 2 → 3 là bắt buộc vì mục 3 liên tục so sánh với cả hai mục trước. Nếu chỉ nhớ một điều: mọi kiến trúc trong chương nằm trên một trục duy nhất giữa “nhiều giả định, ít dữ liệu” và “ít giả định, nhiều dữ liệu”, và không có điểm nào trên trục đó là tốt nhất một cách tuyệt đối.

KIẾN THỨC CẦN CÓ TRƯỚC

- **backbone** — phần mạng trích đặc trưng từ ảnh, dùng chung cho nhiều bài toán; phần sinh đầu ra cụ thể (nhân, hộp, mặt nạ) được lắp thêm phía sau.
- **inductive bias** — tập giả định mà mô hình mang sẵn trước khi nhìn dữ liệu; nó là đại lượng trung tâm mà cả chương xoay quanh.
- **convolution** — phép nhân một cửa sổ trọng số nhỏ trượt khắp ảnh, cài sẵn giả định “quan hệ gần là quan trọng” và “mẫu hình có ý nghĩa như nhau ở mọi vị trí”.
- **attention** — cơ chế cho mỗi vị trí lấy thông tin trực tiếp từ mọi vị trí khác bằng trọng số học được, không cần thông tin lan dần qua nhiều tầng.

Có gì trong chương này**Bảng 13.1.** Ba mục của chương 13: câu hỏi trung tâm, phụ thuộc và mức độ đầu tư.

Mục	Câu hỏi trung tâm	Phụ thuộc	Mức độ
1. Dòng CNN	Mỗi thể hệ CNN gỡ nút thắt nào, và đẩy độ khó sang trục nào?	C/12	Cốt lõi [T2]
2. Từ chuỗi tới attention	Xử lý quan hệ xa thế nào, và cái giá của việc rút ngắn đường đi thông tin là gì?	Mục 1, C/12/02	Cốt lõi [T2]
3. ViT và inductive bias	Vứt bỏ giả định thị giác thì mất gì, và trả lại bằng cách nào?	Mục 1 và 2	Cốt lõi [T2]

Vì sao lại chia như vậy

Cách chia phổ biến trong sách là chia theo *họ kiến trúc*: CNN, RNN, Transformer. Cách đó che mất điều quan trọng nhất. Ba họ này trả lời cùng một câu hỏi thiết kế, chứ không phục vụ ba thể giới tách biệt. Ở đây ta chia theo *loại dữ liệu và lượng giả định*: mục 1 là dữ liệu lưới với nhiều giả định; mục 2 là dữ liệu có thứ tự, nơi giả định lưới không dùng được; mục 3 là quay lại dữ liệu lưới nhưng với rất ít giả định.

Lợi ích của cách chia này lộ ra ở hai chỗ nổi. Thứ nhất, cổng của LSTM ở mục 2 và kết nối tắt của ResNet ở mục 1 hoá ra là cùng một mẹo — tạo một đường cộng cho gradient — và cách chia theo họ kiến trúc sẽ giấu mất điều đó. Thứ hai, mục 3 chỉ có nghĩa như một *thí nghiệm đối chứng* cho mục 1: nó đo giá trị của các giả định mà mục 1 mô tả, bằng đơn vị là lượng dữ liệu.

Đọc theo thứ tự nào

1. Đọc tuần tự 1 → 2 → 3. Mục 3 tham chiếu liên tục sang hai mục kia nên đọc trước sẽ hệt.
2. Nếu bạn chỉ cần chọn một backbone cho một dự án đang chạy: đọc mục 1 phần bảng dòng CNN, rồi nhảy tới mục 3 phần G (tách backbone, neck, head) và phần giới hạn. Quay lại mục 2 khi phải xử lý ảnh độ phân giải cao.
3. Nếu bạn đến từ phía SLAM hoặc hệ thống nhúng: đọc mục 1 phần C (ba phép tính tay) trước tiên, vì đó là chỗ FLOPs, cường độ số học và độ trễ thật gặp nhau.

Cốt lõi và tham khảo

Ba phần đáng quay lại nhiều lần: ba phép tính tay ở mục 1 (xếp chồng 3×3 , nút thắt 1×1 , depthwise separable); lập luận $\sqrt{d}$ và bảng ba hướng chống chi phí bậc hai ở mục 2; và trục giả định–dữ liệu cùng sơ đồ backbone–neck–head ở mục 3. Ba phần đọc lướt lần đầu rồi tra lại khi cần: danh sách biến thể convolution, chi tiết các họ mã hoá vị trí, và phần mô hình không gian trạng thái — phần cuối này thay đổi nhanh nhất nên cũng mất giá nhanh nhất.

Nối vào phần còn lại của cây

Chương này tiêu thụ toàn bộ [C/12-co-che-huan-luyen](#) và trực đánh đổi ở [B/10-thiet-ke-kien-truc](#); nó cấp đầu vào cho [C/14-bai-toan-cv-loi](#) (head cụ thể lắp lên backbone), [C/16-thi-giac-3d](#) và [C/18-pretraining-va-foundation](#). Hai nhánh ngang đáng nhớ: phần FLOPs và cường độ số học nổi sang [B/11-gioi-han-tinh-toan](#) cùng [D/20-toi-uu-va-nen-mo-hinh](#); còn bài học ConvNeXt — rằng recipe giải thích phần lớn chênh lệch được gán cho kiến trúc — là tiền đề trực tiếp của [E/24-thi-nghiem-va-ablation](#).

13.2 Dòng CNN: gỡ từng nút thắt (The CNN Lineage)

PHỤ THUỘC [C/12-co-che-huan-luyen](#) (đặc biệt mục 2 — kết nối tất là thành viên thứ tư của họ “giữ tích Jacobian gần 1”); [B/11-gioi-han-tinh-toan](#) (FLOPs, cường độ số học, chế độ ngهن bộ nhớ).

TÓM TẮT

Mục này đọc mười lăm năm lịch sử *CNN* như một chuỗi gỡ nút thắt: mỗi kiến trúc ra đời để phá đúng một rào cản của kiến trúc trước, và tạo ra rào cản mới cho kiến trúc sau. Đọc xong bạn nhìn một *backbone* lạ và gọi tên được nó đang tối ưu cho ràng buộc nào: tham số, độ sâu, FLOPs, hay độ trễ thật. Nhờ đó bạn chọn backbone theo ràng buộc của mình, thay vì theo bảng xếp hạng. Nếu chỉ nhớ một điều: bài học lớn nhất của cả dòng nằm ở mắt xích cuối, ConvNeXt. Rất nhiều “cải tiến kiến trúc” trong lịch sử hoá ra là cải tiến *công thức huấn luyện*. Nên mọi so sánh kiến trúc không kiểm soát recipe đều đáng ngờ.

KIẾN THỨC CẦN CÓ TRƯỚC

- **convolution** — phép nhân một cửa sổ trọng số nhỏ, gọi là kernel hay filter, trượt khắp ảnh; vì cùng bộ trọng số dùng cho mọi vị trí nên số tham số không phụ thuộc kích thước ảnh.
- **kênh (channel)** — chiều thứ ba của một bản đồ đặc trưng — ảnh RGB có 3 kênh đầu vào, còn một tầng giữa mạng có thể có 256 kênh, mỗi kênh là một loại đặc trưng mà mạng học được.
- **stride** — bước nhảy của cửa sổ khi trượt; stride 2 làm bản đồ đặc trưng nhỏ đi một nửa mỗi chiều, nên nó là cách chính để giảm độ phân giải và tăng tầm nhìn.
- **trường tiếp nhận (receptive field)** — vùng pixel đầu vào ảnh hưởng tới một giá trị đầu ra; nó lớn dần theo chiều sâu, và cả mục này xoay quanh việc mở rộng nó cho rẻ.
- **pooling** — phép gộp một vùng nhỏ thành một giá trị (lấy lớn nhất hoặc lấy trung bình), dùng để giảm độ phân giải mà không tốn tham số.
- **FLOPs** — số phép nhân-cộng mà mô hình thực hiện; đây là chỉ số đại diện phổ biến cho chi phí, nhưng *không* tỉ lệ với độ trễ thật.
- **cường độ số học (arithmetic intensity)** — số phép tính thực hiện trên mỗi byte dữ liệu đọc từ bộ nhớ; thấp thì kernel bị ngهن ở bảng thông chứ không ở khả năng tính toán, và giảm FLOPs lúc đó gần như vô ích.
- **recipe (công thức huấn luyện)** — toàn bộ các lựa chọn không thuộc kiến trúc — optimizer, lịch học, số epoch, augmentation — và bài học cuối mục là chúng giải thích phần lớn chênh lệch giữa các kiến trúc.

A. Đặt vấn đề

Toàn bộ chương 13 là một cuộc tranh luận kéo dài về một câu hỏi: *nên cài sẵn bao nhiêu giả định vào mô hình?* **Convolution** là một câu trả lời rất mạnh cho câu hỏi đó, và nó cài sẵn đúng ba giả định. Thứ nhất là **tính cục bộ (locality)** — một pixel chỉ liên quan trực tiếp tới các pixel lân cận, nên trọng số bằng 0 ở ngoài cửa sổ $k \times k$. Thứ hai là **chia sẻ trọng số (weight sharing)**, tương đương

với **đẳng biến tịnh tiến** (*translation equivariance*) — một mẫu hình có ý nghĩa như nhau dù nó nằm ở đâu trong ảnh. Thứ ba là **phân cấp** (*hierarchy*) — đặc trưng phức tạp được ghép từ đặc trưng đơn giản, nên tăng dần độ trừu tượng và giảm dần độ phân giải.

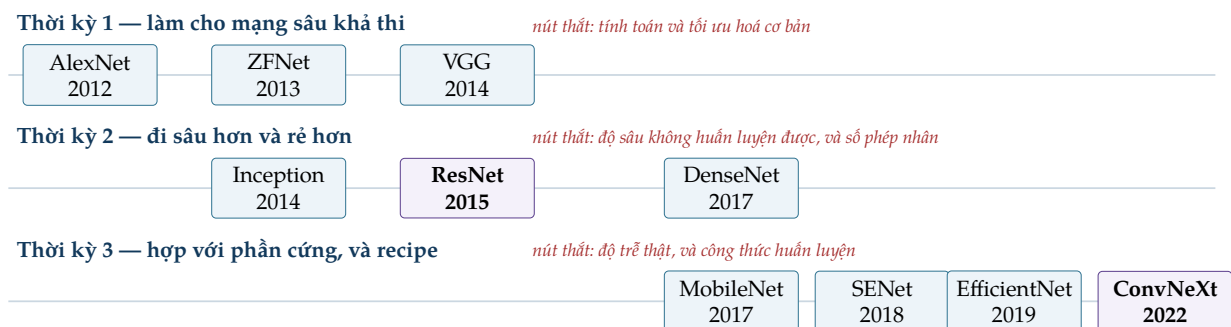
Ba giả định này đến từ vật lý của việc tạo ảnh chứ không từ toán học, và chúng đúng đủ tốt để một mạng conv cần ít dữ liệu hơn hẳn một mạng dày đặc cùng sức chứa. Nhưng chúng cũng khoá lại một không gian thiết kế, và lịch sử CNN chính là lịch sử của việc lần lượt phát hiện ra rào cản nào trong không gian đó đang chặn, rồi gỡ nó. Giá trị cụ thể của mục này: cầm một backbone lạ lên, bạn trả lời được ba câu mà không cần chạy thử. Nó đang tiết kiệm cái gì? Nó trả giá ở đâu? Và ràng buộc nào khiến người ta chọn nó?

TRỰC GIÁC

Đừng đọc dòng CNN như một cuộc thi xem ai chính xác hơn. Hãy đọc nó như lịch sử tối ưu một đường ống sản xuất, nơi mỗi thế hệ tìm ra *nút cổ chai hiện thời* rồi mở rộng đúng chỗ đó. Mở đúng nút cổ chai thì thông lượng tăng vọt, và nút cổ chai lập tức chuyển sang chỗ khác. Đó là lý do mỗi bước tiến trong bảng dưới đây lại sinh ra một vấn đề mới ở trực khác. Người đọc lịch sử theo cách này đoán được bước tiếp theo; người đọc theo cách “ai chính xác hơn” chỉ nhớ được tên.

B. Dòng thời gian: ba thời kỳ, ba nút thắt khác nhau

Nhìn từ xa, mười lăm năm này chia làm ba thời kỳ rõ rệt, và *nút thắt của mỗi thời kỳ nằm ở một trực khác nhau*. Thời kỳ đầu vướng ở tính khả thi. Thời kỳ giữa vướng ở khả năng tối ưu hoá. Thời kỳ sau vướng ở phần cứng và ở công thức huấn luyện.



Hình 13.2. Dòng CNN đặt trên trục thời gian và tách theo nút thắt được gỡ. Hai mắt xích được tô đậm vì chúng thay đổi cách ngành suy nghĩ chứ không chỉ thay đổi con số: ResNet cho thấy rào cản của mạng sâu là *tối ưu hoá* chứ không phải sức chứa, còn ConvNeXt cho thấy phần lớn khoảng cách giữa các kiến trúc là khoảng cách công thức huấn luyện.

Bảng 13.2 đọc cùng Hình 13.2 theo chiều dọc; cột cuối là cột đáng nhớ, vì nó cho thấy không bước nào miễn phí.

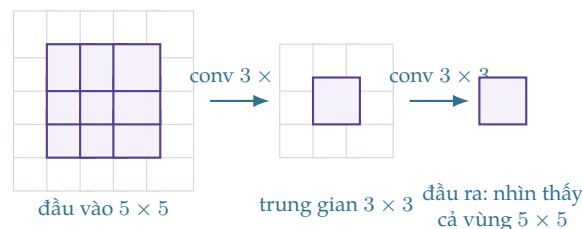
Bảng 13.2. Dòng CNN đọc theo mạch gỡ nút thắt. Cột cuối cho thấy mỗi bước tiến đẩy độ khó sang một trục khác chứ không xoá nó.

Bước	Gỡ nút thắt gì	Cái giá / vấn đề mới
LeNet	Chứng minh conv + pooling + huấn luyện gradient học được đặc trưng thị giác [105]	Chỉ chạy ở quy mô chữ số; chưa có dữ liệu và phần cứng để mở rộng
AlexNet	Chứng minh mạng sâu + GPU + ReLU + dropout là khả thi ở quy mô ImageNet [133]	Kiến trúc làm bằng tay, filter lớn 11×11 , rất nhiều tham số ở tầng FC
ZFNet	Trực quan hoá để thấy mạng học gì, từ đó chỉnh filter và stride	Đóng góp là phương pháp quan sát, không phải kiến trúc mới
VGG	Xếp chồng 3×3 : cùng trường tiếp nhận, ít tham số hơn, nhiều phi tuyến hơn [134]	Tầng FC chiếm phần lớn tham số; tổng FLOPs rất cao
Inception	Nhiều tỉ lệ song song; 1×1 làm nút thắt giảm chiều trước phép tính nặng [21], [135]	Kiến trúc phức tạp, nhiều siêu tham số nhánh, khó chỉnh và khó mở rộng
ResNet	Vấn đề <i>tối ưu hoá</i> của mạng sâu, bằng đường cộng đồng nhất [109]	Vẫn nhiều FLOPs; nhánh tắt buộc giữ thêm kích hoạt nên tốn bộ nhớ hơn
DenseNet	Tái dùng đặc trưng tối đa, ít tham số hơn ở cùng độ chính xác [136]	Phép nối làm bộ nhớ và băng thông phình ra nếu cài đặt ngây thơ
MobileNet	Chi phí tính toán trên thiết bị yếu, bằng depthwise separable [118]	Giảm FLOPs nhiều hơn giảm độ trễ thật, vì cường độ số học tụt
SENet	Mạng không tự điều chỉnh được tầm quan trọng giữa các kênh [113]	Thêm một điểm đồng bộ toàn cục; lợi ích nhỏ dần khi backbone đã mạnh
EfficientNet	Việc mở rộng mô hình bị làm tùy tiện theo một chiều [117]	Tối ưu cho FLOPs, không tối ưu cho một phần cứng cụ thể
ConvNeXt	Nghi vấn “ViT thắng nhờ kiến trúc”, bằng cách hiện đại hoá recipe của CNN [121]	Cho thấy phần lớn chênh lệch trước đó là do công thức huấn luyện, làm mọi so sánh kiến trúc cũ trở nên đáng ngờ

C. Ba phép tính tay giải thích ba bước quan trọng nhất

Ba bước có ảnh hưởng lớn nhất trong bảng đều rút gọn được thành một phép tính ngắn. Tự làm ba phép tính này một lần thì bạn nhớ chúng mãi.

VGG — vì sao xếp chồng 3×3 . Hai tầng 3×3 nối tiếp có cùng trường tiếp nhận với một tầng 5×5 ; ba tầng thì bằng 7×7 . Nhưng chi phí tham số khác hẳn. Với C kênh vào và ra, một tầng 5×5 tốn $25C^2$ tham số, còn hai tầng 3×3 chỉ tốn $2 \cdot 9C^2 = 18C^2$ — ít hơn 28 phần trăm. So 7×7 với ba tầng 3×3 thì là $49C^2$ đối với $27C^2$, ít hơn 45 phần trăm. Và ta còn được thêm hai lần phi tuyến. Nói gọn: cùng tầm nhìn, ít tham số hơn, nhiều khả năng biểu diễn hơn — một trong những phép đổi hiểm hơi tốt ở cả ba mặt.

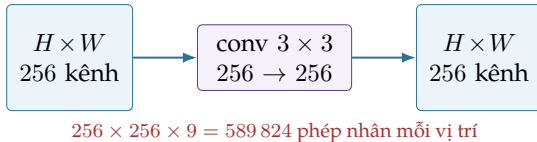


Hình 13.3. Hai tầng 3×3 nối tiếp cho một ô đầu ra nhìn thấy đúng vùng 5×5 của đầu vào. Cùng trường tiếp nhận với một tầng 5×5 , nhưng chỉ tốn $18C^2$ tham số thay vì $25C^2$. Và có thêm một lần phi tuyến ở giữa.

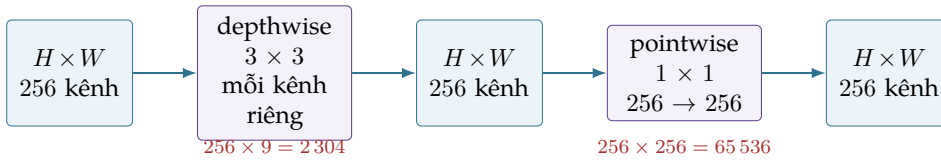
Hình 13.3 là toàn bộ lập luận của VGG dưới dạng hình. Điều nó *không* nói ra là nút thắt kế tiếp: tầng kết nối đầy đủ đầu tiên của VGG-16 có $7 \cdot 7 \cdot 512 \cdot 4096 \approx 103$ triệu tham số, tức khoảng ba phần tư toàn bộ mô hình nằm trong một tầng duy nhất không phải conv. Đó là lý do các thế hệ sau thay tầng FC bằng *global average pooling*.

Inception — vì sao 1×1 là nút thắt hiệu quả. Giả sử cần một conv 3×3 từ 256 kênh sang 256 kênh. Chi phí trực tiếp là $256 \cdot 256 \cdot 9 = 589\,824$ phép nhân-cộng cho mỗi vị trí không gian. Bây giờ chèn hai tầng 1×1 : giảm $256 \rightarrow 64$, làm 3×3 trong không gian 64 kênh, rồi mở lại $64 \rightarrow 256$. Tổng chi phí là $256 \cdot 64 + 64 \cdot 64 \cdot 9 + 64 \cdot 256 = 16\,384 + 36\,864 + 16\,384 = 69\,632$, tức rẻ hơn 8,5 lần. Ý nghĩa của phép tính: phần đắt nhất của conv là tương tác giữa *mọi* kênh vào với *mọi* kênh ra, nhân với k^2 ; nếu ta làm phép tính không gian ở một không gian kênh hẹp thì cái k^2 chỉ nhân với một số nhỏ. Đây cũng chính là khối *bottleneck* của ResNet, và là ý tưởng nền của phần lớn kiến trúc hiệu quả về sau.

conv 3×3 thường: mọi kênh vào nói chuyện với mọi kênh ra, ở mọi vị trí



depthwise separable: tách phần không gian khỏi phần kênh



Nhưng bước depthwise đọc mỗi phần tử để làm vòn vẹn 9 phép nhân, nên cường độ số học của nó tụt xuống mức nghèo bằng thông.

Hình 13.4. Vì sao depthwise separable rẻ, và vì sao “rẻ” theo FLOPs không có nghĩa là “nhanh”. Conv thường trả tiền cho tích của ba thứ cùng lúc: kênh vào, kênh ra, và diện tích của số. Phép tách chia nó thành hai bước, mỗi bước chỉ trả cho *hai* trong ba thứ đó — đây là toàn bộ nguồn gốc của tỉ số $1/C_{\text{out}} + 1/k^2$. Cái giá nằm ở dòng cuối: bước depthwise có rất ít phép tính trên mỗi byte đọc vào, nên nó rơi vào phía nghèo bộ nhớ và thời gian thực tế không giảm theo tỉ lệ FLOPs.

Hình 13.4 bày ra cả hai vế của phép đổi trên cùng một sơ đồ, và đó là lý do nó đáng vẽ: con số 8,7 và lời cảnh báo về độ trễ luôn phải đọc cùng nhau.

MobileNet — và vì sao FLOPs không phải độ trễ. Depthwise separable tách một conv thành hai: một conv $k \times k$ chạy độc lập trên từng kênh, rồi một conv 1×1 trộn kênh. Tỉ số chi phí so với conv thường là $\frac{1}{C_{\text{out}}} + \frac{1}{k^2}$; với $k = 3$ và $C_{\text{out}} = 256$ thì bằng $0,0039 + 0,111 \approx 0,115$, tức rẻ hơn khoảng 8,7 lần. Nhưng độ trễ thực đo được hầu như không bao giờ giảm 8,7 lần, và lý do là thứ bạn đã gặp khi tối ưu CUDA: phần depthwise đọc mỗi phần tử đầu vào để làm vòn vẹn $k^2 = 9$ phép nhân, nên **cường độ số học** (*arithmetic intensity*) của nó cực thấp và nó rơi vào chế độ nghèo bằng thông bộ nhớ. Giảm FLOPs của một kernel vốn đã nghèo ở bộ nhớ thì gần như không giảm thời gian. Đây là ví dụ rõ nhất trong cả cây về việc một chỉ số đại diện — FLOPs — đo sai thứ mà ta quan tâm.

↪ **ràng buộc bởi B/11** — mô hình roofline nói chính xác khi nào giảm FLOPs không giảm thời gian: khi kernel đã nằm ở phía nghèo bằng thông

D. ResNet: nút thắt là tối ưu hoá, không phải sức chứa

Đóng góp của ResNet hay bị kể sai thành “mạng sâu hơn thì mạnh hơn”. Bằng chứng gốc nói điều khác và mạnh hơn nhiều: một mạng phẳng 56 tầng có *sai số huấn luyện* cao hơn một mạng phẳng 20 tầng [109]. Sai số huấn luyện, không phải sai số kiểm thử. Nên đây không phải overfitting, và cũng không phải thiếu sức chứa. Mạng sâu hơn về nguyên tắc biểu diễn được mọi thứ mạng

nông biểu diễn được: chỉ cần các tầng thừa làm ánh xạ đồng nhất. Vấn đề là gradient descent không tìm ra nghiệm đó.

Kết nối tắt $y = x + F(x)$ làm hai việc cùng lúc, và cả hai đều đã xuất hiện ở chương 12. Về mặt biểu diễn, ánh xạ đồng nhất trở thành phương án *mặc định* — chỉ cần $F \approx 0$, điều mà khởi tạo nhỏ cho sẵn. Về mặt gradient, Jacobian của khối là $I + \partial F / \partial x$, nên tích các Jacobian qua nhiều khối luôn còn một thành phần bằng 1 không thể triệt tiêu; đây chính xác là thành viên thứ tư của họ kỹ thuật “giữ ρ gần 1” mà mục C/12/02 đã mở ngoặc để dành.

Khối phẳng



$\frac{\partial y}{\partial x} = J_2 J_1$ — mọi thừa số đều có thể nhỏ hơn 1
đường tắt: x đi thẳng, không qua phép toán nào

Khối residual



Hệ quả kép: ánh xạ đồng nhất trở thành phương án *mặc định* (chỉ cần $F \approx 0$), và gradient luôn có một đường về không bị nhân đôi.

$\frac{\partial y}{\partial x} = I + \frac{\partial F}{\partial x}$ — số hạng I không bao giờ triệt tiêu

Hình 13.5. Kết nối tắt sửa vấn đề gì, nhìn từ phía lượt ngược. Ở khối phẳng, đạo hàm của đầu ra theo đầu vào là một tích thuần tuý, nên nếu mỗi thừa số hơi nhỏ hơn 1 thì sau vài chục khối nó tắt — đúng cơ chế ở C/12/02. Ở khối residual, đường cộng thêm một số hạng I vào Jacobian, và số hạng đó không phụ thuộc trọng số nên không thể bị nhân cho tiêu biến. Đây là lý do bằng chứng gốc của ResNet là sai số *huấn luyện* chứ không phải sai số kiểm thử: thứ được sửa là khả năng tối ưu hoá, không phải sức chứa.

Hình 13.5 cũng cho thấy cái giá đã nhắc ở bảng: mũi tên đường tắt buộc tensor x phải sống tới tận điểm cộng, nên bộ nhớ kích hoạt của một mạng residual cao hơn một mạng phẳng cùng độ sâu.

Cái giá thì cụ thể: nhánh tắt buộc giữ tensor x sống tới tận chỗ cộng, nên bộ nhớ kích hoạt tăng so với một mạng phẳng cùng độ sâu.

↪ cùng nguyên lý với C/12/02 — kết nối tắt, hàm kích hoạt, khởi tạo và chuẩn hoá đều nhắm vào một đại lượng: hệ số nhân của tích các Jacobian

E. Các biến thể convolution và mục đích riêng

Đây là một bộ sưu tập chứ không phải một chuỗi, nên đọc theo kiểu tra cứu. Điểm chung: mỗi biến thể *nới lỏng hoặc siết chặt* đúng một trong ba giả định gốc ở mục A.

- **Depthwise separable** — tách tương tác không gian khỏi tương tác kênh.
 - **Được:** chi phí giảm khoảng k^2 lần, như phép tính ở mục C.
 - **Mất:** cường độ số học tụt nên độ trễ thật giảm ít hơn nhiều; và giả định “không gian và kênh tách rời được” không phải lúc nào cũng đúng.
- **Group convolution** — chia kênh thành g nhóm, mỗi nhóm chỉ nói chuyện trong nội bộ. Ra đời từ ràng buộc bộ nhớ của AlexNet trên hai GPU, sau đó thành công cụ giảm tham số. **Mất:**

thông tin không đi qua ranh giới nhóm, nên cần một phép trộn kênh (ví dụ shuffle hoặc 1×1) để bù.

- **Dilated convolution** [106] — giãn khoảng cách giữa các điểm lấy mẫu trong cửa sổ, nên trường tiếp nhận tăng theo cấp số nhân mà không cần giảm độ phân giải. Sinh ra cho segmentation, nơi giảm độ phân giải là mất mát trực tiếp. *Mất*: hiện tượng lưới — các ô lấy mẫu không phủ kín nên xuất hiện hoa văn răng lược nếu xếp chồng nhiều tầng cùng hệ số giãn.
- **Deformable convolution** [114] — học luôn cả *vị trí* lấy mẫu thay vì cố định trên lưới, tức nới lỏng giả định cục bộ theo một cách có điều khiển. *Mất*: truy cập bộ nhớ không đều nên rất khó tối ưu trên GPU, và là một trong những lý do phổ biến khiến một mô hình không xuất được sang runtime suy luận.
- **Transposed convolution** — tăng độ phân giải ở nhánh giải mã. *Mất*: hiện tượng bàn cờ khi kích thước kernel không chia hết cho stride; nhiều cài đặt hiện đại thay nó bằng *upsample rồi conv* thường để tránh hẳn.

F. ConvNeXt và bài học đắt nhất của cả mục

Khi ViT vượt CNN trên ImageNet, cách đọc tự nhiên là “*attention* tốt hơn convolution”. ConvNeXt kiểm chứng cách đọc đó bằng một thí nghiệm có kỷ luật. Nhóm tác giả giữ nguyên convolution, rồi lần lượt áp từng thành phần của recipe hiện đại và đo sau mỗi bước [121]. Danh sách gồm lịch học dài hơn, AdamW, augmentation mạnh, stochastic depth, LayerNorm thay Batch-Norm, kernel lớn hơn, và tỉ lệ tăng theo kiểu transformer. Kết quả là một CNN thuần bắt kịp ViT cùng hạng.

Kết luận đúng không phải “CNN thắng” mà là: *phần lớn khoảng cách được gán cho kiến trúc thực ra thuộc về công thức huấn luyện*. Điều này thay đổi cách bạn đọc mọi paper kiến trúc về sau. Câu hỏi đầu tiên khi thấy một bảng so sánh không còn là “kiến trúc nào cao hơn”. Nó thành “hai dòng này có cùng recipe không, và nếu không thì ai được tune kỹ hơn”. Đây cũng là lý do chương [E/24-thi-nghiem-va-ablation](#) tồn tại.

↪ **hệ quả của E/24** — một bảng so sánh chỉ có nghĩa khi hai dòng được huấn luyện bằng cùng công thức và được tune ngang nhau

G. Giới hạn và trường hợp hồng

Ba giới hạn đáng mang theo. Thứ nhất, *FLOPs không phải độ trễ, và số tham số cũng không*. Ba đại lượng này tương quan yếu với nhau và với thời gian chạy thật; MobileNet và DenseNet là hai ví dụ ngược chiều nhau về đúng điểm này. Bất kỳ lựa chọn backbone nào cũng phải đo trên đúng phần cứng và đúng chế độ batch mà bạn sẽ triển khai.

Thứ hai, *độ chính xác ImageNet là một chỉ số đại diện, không phải mục tiêu*. Tương quan giữa nó và chất lượng ở tác vụ khác là có thật, nhưng không chặt. Tương quan đó yếu đi rõ rệt với các tác vụ dự đoán dày như segmentation. Ở đó, độ phân giải và đặc trưng đa tỉ lệ quan trọng hơn khả năng phân loại toàn ảnh.

Thứ ba, các con số trong mục này thuộc hai loại khác nhau. Các phép tính tham số và FLOPs ở mục C là số học thuần túy nên chính xác. Ngược lại, các phát biểu kiểu “rẻ hơn 8,7 lần nhưng độ trễ giảm ít hơn nhiều” là mô tả định tính của một hiện tượng đã được quan sát rộng rãi; mức giảm cụ thể phụ thuộc GPU, thư viện và kích thước tensor tới mức tôi không nêu một con số nào ở đây.

CẠM BÃY

Cạm bẫy phổ biến nhất khi chọn backbone là so sánh hai mô hình được huấn luyện bằng hai recipe khác nhau rồi kết luận về kiến trúc. Một ResNet-50 theo recipe 2015 và một ResNet-50 theo recipe 2022 chênh nhau vài điểm phần trăm trên ImageNet. Khoảng cách đó lớn hơn khoảng cách giữa nhiều cặp kiến trúc mà người ta viết hẳn một paper để so. Trước khi tin bất kỳ bảng nào, hãy kiểm tra ba thứ: số epoch, bộ augmentation, và optimizer cùng lịch học.

CẦN NHỚ

Mỗi bước trong dòng CNN gỡ đúng một nút thắt và đẩy độ khó sang một trục khác: tham số $\rightarrow$ khả năng tối ưu hoá $\rightarrow$ chi phí tính toán $\rightarrow$ độ trễ trên phần cứng thật $\rightarrow$ công thức huấn luyện. Khi cầm một backbone mới, hỏi nó đang đứng ở nút nào trong chuỗi đó — câu trả lời cho bạn biết nó sẽ hỏng ở đâu. [T2]

H. Kết nối

Mục này giả định toàn bộ [C/12-co-che-huan-luyen](#), và trả lại cho chương đó thành viên thứ tư của họ “giữ tích Jacobian gần 1”. Nó đi thẳng vào [C/13/02-tu-chuoi-toi-attention](#), nơi câu hỏi “bao nhiêu giả định là đủ” được đặt cho dữ liệu có thứ tự. Rồi vào [C/13/03-vit-va-inductive-bias](#), nơi ba giả định ở mục A bị gỡ bỏ có chủ đích để xem chuyện gì xảy ra. Phần FLOPs và cường độ số học nối trực tiếp sang [B/11-gioi-han-tinh-toan](#) và [D/20-toi-uu-va-nen-mo-hinh](#); còn bài học ConvNeXt là tiền đề của [E/24-thi-nghiem-va-ablation](#). Nếu bạn quan tâm việc lắp backbone vào một hệ thống thật, [C/14-bai-toan-cv-loi](#) là chương nói về phần neck và head gắn lên nó.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Ba giả định mà convolution cài sẵn là gì, và mỗi biến thể ở mục E nới lỏng giả định nào?
2. Vì sao xếp chồng hai tầng 3×3 tốt hơn một tầng 5×5 ở cả ba mặt, và trong tình huống nào nó không tốt hơn?
3. Bằng chứng gốc của ResNet là sai số *huấn luyện* chứ không phải kiểm thử. Vì sao chi tiết đó thay đổi hoàn toàn kết luận rút ra?
4. MobileNet giảm FLOPs khoảng 8,7 lần nhưng độ trễ giảm ít hơn nhiều — giải thích bằng cường độ số học thì thế nào, và phép đo nào xác nhận chẩn đoán đó?
5. Nếu ConvNeXt đúng khi nói recipe quan trọng hơn kiến trúc, điều đó thay đổi thứ tự bạn đọc một paper kiến trúc thế nào?
6. Deformable convolution mạnh hơn conv thường ở đâu, và vì sao nó thường bị loại khi triển khai lên thiết bị nhúng?

13.3 Từ chuỗi tới attention (From Sequences to Attention)**PHỤ THUỘC**

[C/12/02-lam-cho-training-chay-duoc](#) (tích các Jacobian — vanishing gradient ở đây là đúng hiện tượng đó, chỉ trái theo thời gian); [C/13/01-dong-cnn](#) (khái niệm trường tiếp nhận và inductive bias).

TÓM TẮT

Mục này đi từ mạng hồi quy tới **attention** và tới các mô hình không gian trạng thái, đọc như một chuỗi gỡ nút thắt chứ không như một danh sách kiến trúc. Đọc xong bạn giải thích được vì sao **transformer** bỏ được tính tuần tự, và vì sao nó phải trả bằng chi phí bậc hai. Bạn cũng nói được ba hướng tấn công chi phí đó khác nhau ở tầng nào: toán, kiến trúc, hay hệ thống. Nếu chỉ nhớ một điều: LSTM và ResNet giải cùng một bài toán bằng cùng một mẹo. Cả hai tạo một *đường cộng* để gradient đi qua mà không bị nhân dồn. Nhận ra điều đó thì hai kiến trúc trông rất khác nhau trở thành một ý tưởng duy nhất.

KIẾN THỨC CẦN CÓ TRƯỚC

- **chuỗi (sequence)** — dữ liệu có thứ tự, ví dụ một câu gồm nhiều từ hoặc một video gồm nhiều khung hình; “thứ tự là thông tin” nghĩa là hoán vị các phần tử sẽ đổi ý nghĩa.
- **token** — một phần tử của chuỗi sau khi đã được cắt và mã hoá thành vectơ số — một từ, một mảnh từ, hoặc với ảnh là một ô vuông 16×16 pixel.
- **embedding** — vectơ số biểu diễn một token; các token có nghĩa gần nhau được đặt gần nhau trong không gian này, và toàn bộ mô hình chỉ làm việc với vectơ chứ không với ký tự.
- **trạng thái ẩn (hidden state)** — vectơ mà mạng hồi quy mang theo từ bước thời gian này sang bước sau; nó là toàn bộ “trí nhớ” của mạng, nên mọi thứ không nhét được vào nó là mất.
- **BPTT (backpropagation through time)** — cách áp lan truyền ngược cho mạng hồi quy: trải mạng ra thành T bản sao theo thời gian rồi lan truyền ngược qua cả T bước, nên một chuỗi dài 100 hành xử như một mạng sâu 100 tầng.
- **attention** — cơ chế cho mỗi vị trí tự chọn lấy thông tin từ mọi vị trí khác bằng một trọng số học được, thay vì phải chờ thông tin truyền dần qua từng bước.
- **độ phức tạp bậc hai** — chi phí tỉ lệ với T^2 khi độ dài chuỗi là T ; gấp đôi độ dài thì chi phí gấp bốn, nên đây là rào cản chính khi mang attention sang ảnh độ phân giải cao.
- **HBM** — bộ nhớ băng thông cao gắn ngoài chip GPU — rất lớn nhưng chậm hơn nhiều so với bộ nhớ trên chip; số lần đọc ghi HBM thường quyết định tốc độ hơn cả số phép nhân.

A. Đặt vấn đề

Convolution giả định dữ liệu nằm trên một lưới đều và quan hệ quan trọng là quan hệ gần. Nhưng nhiều dữ liệu có *thứ tự* chứ không có lưới, và quan hệ quan trọng có thể ở rất xa: từ đầu câu quyết định nghĩa từ cuối câu, khung hình đầu video quyết định cách hiểu khung hình cuối. Câu hỏi của mục này là: cấu trúc nào xử lý được loại quan hệ đó, và mỗi câu trả lời phải trả giá bằng gì?

Giá trị cụ thể của mục: nhìn một kiến trúc xử lý chuỗi, bạn trả lời được ba câu. Thông tin đi từ vị trí i tới vị trí j qua bao nhiêu bước? Chi phí tăng theo độ dài như thế nào? Và cái gì trong kiến trúc đang giữ vai trò “biết thứ tự”? Ba câu đó đủ để phân loại mọi biến thể mà bạn sẽ gặp.

TRỰC GIÁC

Hãy đo mọi kiến trúc trong mục này bằng một đại lượng duy nhất: *độ dài đường đi của thông tin* giữa hai vị trí xa nhau. Trong mạng hồi quy, thông tin phải đi qua T trạm nối tiếp, và mỗi trạm nhân nó với một ma trận. Giống như truyền một tin nhắn qua dây chuyền thì thầm dài 100 người. Trong attention, mọi vị trí nói chuyện trực tiếp với mọi vị trí, đường đi bằng 1 — giống như thay dây chuyền thì thầm bằng một phòng họp mà ai cũng nghe thấy ai. Toàn bộ ưu điểm lẫn toàn bộ chi phí của transformer nằm gọn trong phép đổi đó: phòng họp không tam sao thất bản, nhưng số cặp người phải nghe nhau tăng theo bình phương số.

B. Mạch từ RNN tới state space model

Sáu bước dưới đây là một chuỗi liên tục, và mỗi bước chỉ hiểu được khi đã thấy vấn đề mà bước trước để lại.

I Mạch 2 – từ chuỗi tới attention

- 1 Vấn đề** dữ liệu có thứ tự, nhưng mạng dày đặc và convolution đều không có khái niệm “bước trước”.

Giải pháp **mạng hồi quy** (recurrent neural network, RNN): giữ một trạng thái ẩn $h_t = f(Wh_{t-1} + Ux_t)$ và cập nhật nó theo từng bước, nên cùng một bộ trọng số dùng lại cho mọi độ dài chuỗi.

Vấn đề mới huấn luyện bằng BPTT nghĩa là lan truyền ngược qua T tầng dùng chung trọng số. Gradient vì thế tỉ lệ với lũy thừa T của cùng một ma trận. Đó đúng là hiện tượng vanishing và exploding của mục C/12/02, nhưng tệ hơn: thừa số lặp lại y hệt nhau nên không có triệt tiêu ngẫu nhiên nào cứu.
- 2 Vấn đề** trí nhớ dài không tồn tại: sau vài chục bước tín hiệu đã tắt.

Giải pháp **LSTM** [137] và GRU dùng **cổng** (gate) để tạo một trạng thái ô $c_t = f_t \odot c_{t-1} + i_t \odot g_t$.

Hệ quả đạo hàm $\partial c_t / \partial c_{t-1} = f_t$, nên khi cổng quên mở gần 1 thì gradient chảy qua mà không bị nhân với ma trận trọng số. Đây chính xác là mẹo của kết nối tắt trong ResNet, chỉ khác là hệ số của đường cộng ở đây học được thay vì cố định bằng 1.

Vấn đề mới tính tuần tự vẫn còn nguyên: h_t cần h_{t-1} , nên không song song hoá được theo thời gian, và độ dài dùng được vẫn giới hạn.
- 3 Vấn đề** tuần tự nghĩa là không tận dụng được GPU, và thông tin xa vẫn phải đi qua mọi trạm trung gian.

Giải pháp **attention**. Mỗi vị trí sinh một truy vấn q ; mọi vị trí sinh khoá k và giá trị v . Đầu ra là tổ hợp $\text{softmax}(qK^\top / \sqrt{d})V$, tức truy cập trực tiếp mọi vị trí trong một bước. **Transformer** [107] bỏ hẳn phần hồi quy và chỉ giữ attention cộng mạng truyền thẳng theo vị trí, nên toàn bộ chuỗi được xử lý song song.

Giá phải trả chi phí và bộ nhớ bậc hai theo độ dài, và mất hoàn toàn khái niệm thứ tự vì phép tính bất biến với hoán vị.
- 4 Vấn đề** mô hình không còn biết token nào đứng trước token nào.

Giải pháp **mã hoá vị trí** (positional encoding): cộng hoặc trộn thêm thông tin vị trí vào embedding.

Giả định vị trí có thể mã hoá thành một vectơ mà mô hình học cách dùng.

Hệ quả bốn họ với đánh đổi khác nhau — tuyệt đối dạng sin, tuyệt đối học được, tương đối, và RoPE quay embedding theo góc tỉ lệ vị trí [138]. Tuyệt đối đơn giản nhưng ngoại suy kém ra ngoài độ dài đã thấy khi huấn luyện; tương đối và RoPE ngoại suy tốt hơn vì chúng mã hoá khoảng cách chứ không mã hoá chỉ số. Với ảnh còn cần biến thể hai chiều, vì vị trí là một cặp (x, y) chứ không phải một số.
- 5 Vấn đề** chi phí bậc hai chặn đường tới ảnh độ phân giải cao.

Giải pháp ba hướng tấn công ở ba tầng khác nhau — xấp xỉ toán học, giới hạn phạm vi kiến trúc, và tối ưu truy cập bộ nhớ — xem Bảng 13.3.

Hệ quả đây là một bài tập rất tốt về việc *cùng một vấn đề có thể tấn công ở nhiều tầng trừu tượng*, và ba lời giải không loại trừ nhau: người ta dùng FlashAttention *bên trong* Swin.

6 Vấn đề

Giải pháp

Giá phải trả

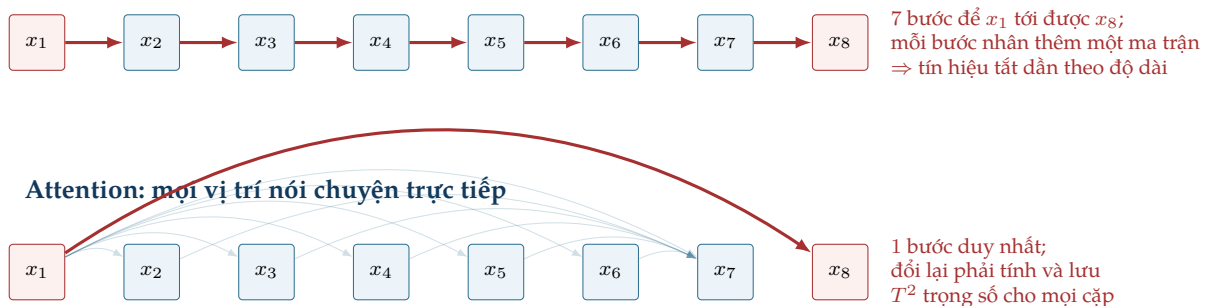
cả ba hướng trên đều và chứ không xoá được bậc hai.

mô hình không gian trạng thái (*state space model, SSM*) như Mamba [139] quay lại dạng hồi quy tuyến tính, với tham số phụ thuộc đầu vào. Nó tính được bằng phép quét song song. Nhờ vậy chi phí tuyến tính theo độ dài mà vẫn huấn luyện song song được.

đây là hướng đang lên, chưa phải kết luận: tính tới thời điểm viết, SSM chưa thay thế attention trong thị giác, và phần lớn kết quả mạnh nhất vẫn thuộc về các kiến trúc dựa trên attention.

↔ **cùng nguyên lý với C/13/01** — cổng của LSTM và kết nối tắt của ResNet là cùng một mẹo — tạo một đường cộng để gradient không bị nhân dồn

Mạng hồi quy: thông tin bò từng bước



Hình 13.6. Đại lượng đáng dùng để so mọi kiến trúc chuỗi: *độ dài đường đi của thông tin* giữa hai vị trí xa nhau. Ở mạng hồi quy nó bằng khoảng cách giữa hai vị trí, nên tín hiệu phải sống sót qua từng ấy phép nhân ma trận — đây chính là vanishing gradient của mục C/12/02, trải theo trục thời gian. Ở attention nó luôn bằng 1, và cái giá là các cạnh mờ trong hình: mọi cặp đều phải có một trọng số, nên số cạnh tăng theo T^2 . LSTM nằm giữa hai thái cực: đường đi vẫn dài, nhưng cổng quên tạo một đường cộng nên tín hiệu không bị nhân dồn.

Hình 13.6 là dạng hình học của ẩn dụ ở đầu mục, và nó cũng cho thấy vì sao ba bước đầu của mạch phải đi theo đúng thứ tự đó.

C. Vì sao chia cho $\sqrt{d}$ — một phép tính hai dòng

Chi tiết $\sqrt{d}$ trong công thức attention trông như một hệ số làm đẹp. Nhưng bỏ nó đi thì mô hình không huấn luyện được, và lý do tính ra được. Giả sử các thành phần của q và k độc lập, trung bình 0, phương sai 1. Tích vô hướng $q \cdot k = \sum_{i=1}^d q_i k_i$ là tổng của d số hạng độc lập, mỗi số hạng trung bình 0 và phương sai 1. Nên nó có phương sai d , tức độ lệch chuẩn $\sqrt{d}$. Với $d = 64$, độ lệch chuẩn của các logit là 8.

Ý nghĩa của con số đó: softmax trên một tập logit có độ lệch chuẩn 8 gần như luôn dồn toàn bộ khối lượng vào một phần tử, tức nó bão hoà thành một vectơ gần one-hot. Mà đạo hàm của softmax ở vùng bão hoà xấp xỉ 0 — đúng hiện tượng đã gặp với sigmoid ở C/12/02. Chia cho $\sqrt{d}$ đưa độ lệch chuẩn về 1, giữ softmax ở vùng có gradient. Nói gọn: $\sqrt{d}$ không phải chuẩn hoá cho đẹp, nó là điều kiện để attention có gradient khi d lớn.

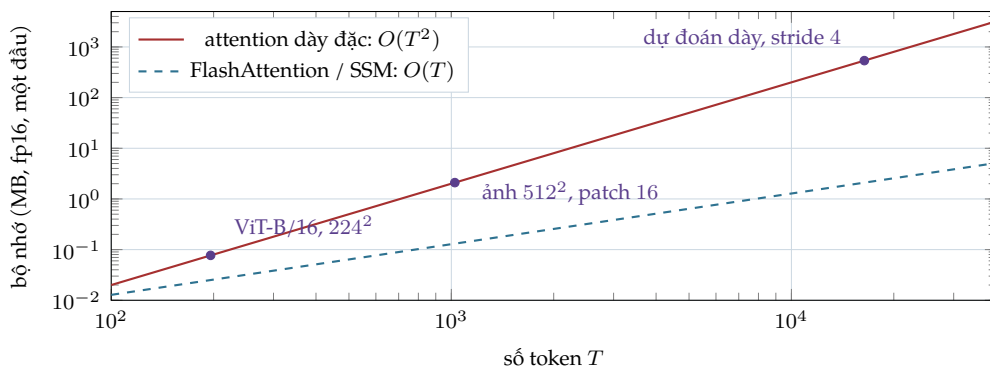
↔ **trường hợp riêng của C/12/02** — đây đúng là hiện tượng bão hoà của sigmoid, chỉ khác là xảy ra ở softmax và nguyên nhân là thang của tích vô hướng

D. Chi phí bậc hai: con số cụ thể trước, giải pháp sau

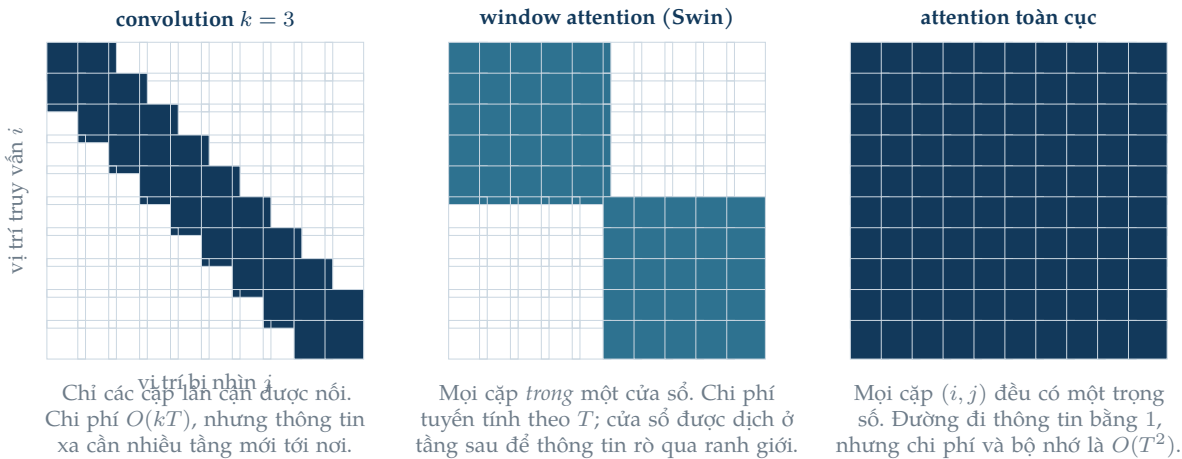
Trước khi bàn giải pháp, hãy làm phép tính để biết vấn đề nghiêm trọng tới mức nào. Ma trận attention có kích thước $T \times T$ cho mỗi đầu và mỗi tầng.

- **Phân loại ảnh, ViT-B/16 ở 224×224 :** $T = (224/16)^2 = 196$ token, nên ma trận có $196^2 = 38\,416$ phần tử — không đáng kể.
- **Ảnh 512×512 , patch 16:** $T = 1024$, ma trận có hơn một triệu phần tử. Vẫn chạy được.
- **Dự đoán dày ở stride 4 trên ảnh 512×512 :** $T = 128^2 = 16\,384$, ma trận có $2,68 \cdot 10^8$ phần tử, tức khoảng 537 MB ở fp16 — cho một đầu, một tầng. Nhân với 12 đầu là 6,4 GB chỉ để chứa các trọng số attention của một tầng.

Con số cuối là toàn bộ lý do các biến thể ở Bảng 13.3 tồn tại, và cũng là lý do ViT phải không dùng trực tiếp được cho segmentation. Hình 13.7 vẽ cùng ba mốc đó trên trục log để thấy chỗ đường cong bẻ gãy ngân sách.



Hình 13.7. Bộ nhớ cần để chứa ma trận attention theo số token, thang log hai chiều. Ba chấm là ba chế độ dùng thật. Khoảng cách giữa hai đường là lý do tồn tại của cả ba hướng tấn công ở Bảng 13.3; lưu ý FlashAttention nằm trên đường tuyến tính về bộ nhớ nhưng vẫn làm đủ $O(T^2)$ phép tính.



Hình 13.8. Ba kiến trúc, cùng một cách nhìn: ô (i, j) được tô nghĩa là vị trí i được phép lấy thông tin trực tiếp từ vị trí j . Convolution là một dải hẹp quanh đường chéo — đó chính là “tính cục bộ” phát biểu dưới dạng ma trận. Window attention là các khối trên đường chéo, tức một thoả hiệp: dày đặc trong cửa sổ, rộng ngoài cửa sổ. Attention toàn cục tô kín, và diện tích phần tô chính là chi phí phải trả. Nhìn ba lưới cạnh nhau thì “inductive bias” thôi không còn là một khái niệm trừu tượng: nó chính là các ô bị bỏ trắng vì kiến trúc quyết định trước rằng chúng không đáng quan tâm.

Hình 13.8 nên được xem trước khi đọc Bảng 13.3: hai trong ba hướng tấn công ở đó chính là hai cách khác nhau để *bớt tô* lưới bên phải, còn hướng thứ ba giữ nguyên lưới nhưng đổi cách nó được tính.

Bảng 13.3. Ba hướng tấn công chi phí bậc hai, ở ba tầng trừu tượng khác nhau. Chúng không loại trừ nhau và trên thực tế hay được dùng chồng lên nhau.

Tầng tấn công	Đại diện	Cơ chế	Mất gì / hỏng khi nào
Toán học	Linear attention	Đổi thứ tự nhân ma trận nhờ một xấp xỉ của softmax, biến $O(T^2d)$ thành $O(Td^2)$	Chất lượng giảm ở tác vụ cần attention “sắc”, tức cần chọn đúng một vị trí trong nhiều vị trí giống nhau
Kiến trúc	Swin, window attention [108]	Chỉ cho attention trong cửa sổ cố định, rồi dịch cửa sổ giữa các tầng để thông tin rò qua	Quan hệ xa cần nhiều tầng mới nổi được; xuất hiện lại một dạng “đường đi dài” mà attention sinh ra để xoá
Hệ thống	FlashAttention [123]	Chia ô và tính softmax trực tuyến để không bao giờ dựng ma trận $T \times T$ trong HBM	Không giảm FLOPs và không giảm độ phức tạp tiệm cận; lợi ích phụ thuộc kích thước SRAM của GPU cụ thể

Bảng 13.3 đáng đọc kỹ vì nó dạy một thói quen chung hơn cả nội dung: khi gặp một rào cản, hãy hỏi nó có thể bị tấn công ở tầng nào. FlashAttention là ví dụ sạch nhất — nó không sửa thuật toán, nó sửa *cách thuật toán chạm vào bộ nhớ*, và với người đã tối ưu kernel CUDA thì đây là câu chuyện quen thuộc: tăng cường độ số học bằng cách giữ dữ liệu trên chip và giảm số vòng đọc ghi ra bộ nhớ ngoài.

↔ **ràng buộc bởi B/11** — FlashAttention chỉ có ý nghĩa trong mô hình roofline: nó dời kernel từ phía ngiên bằng thông sang phía ngiên tính toán

E. Giới hạn và trường hợp hỏng

Attention không phải luôn thắng, và ba chỗ nó thua đáng nhớ.

Thứ nhất, *với chuỗi ngắn và dữ liệu ít, mạng hồi quy vẫn cạnh tranh*. Chi phí bậc hai chỉ là vấn đề khi T lớn; khi T nhỏ, phần chi phí thật nằm ở mạng truyền thẳng chứ không ở attention, còn transformer thì lại cần nhiều dữ liệu hơn vì nó cài ít giả định hơn.

Thứ hai, *mã hoá vị trí là chỗ rò rỉ giả định*. Mọi phép ngoại suy ra độ dài chưa từng thấy khi huấn luyện đều là ngoại suy, và mô hình có thể xuống cấp lạng lẽ. Với ảnh, đổi độ phân giải đầu vào mà không nội suy lại positional embedding là một lỗi phổ biến, cho ra kết quả tệ mà không báo lỗi.

Thứ ba, *linear attention và window attention đánh đổi ở hai chỗ khác nhau*, nên đừng coi chúng thay thế được nhau. Linear attention mất khả năng chọn sắc; window attention giữ được độ sắc trong cửa sổ nhưng mất quan hệ xa. Nếu tác vụ của bạn cần cả hai — ví dụ theo vết một đối tượng nhỏ qua toàn ảnh — thì cả hai đều không đủ và bạn phải quay lại kiến trúc phân cấp.

Về mức độ tin cậy: các con số bộ nhớ ở mục D là số học thuần tuý nên chắc chắn. Còn phát biểu “SSM chưa thay thế attention trong thị giác” chỉ là đánh giá của tôi tính tới tháng 9/2026. Nó dựa trên việc các kết quả dẫn đầu vẫn dùng attention, và có thể sai trong vòng một năm.

CẠM BẤY

Cạm bẫy thực hành hay gặp nhất khi mang một ViT sang bài toán của mình: đổi kích thước ảnh đầu vào mà quên nội suy positional embedding. Số token thay đổi, nhưng bảng vị trí

đã học vẫn có kích thước cũ; tùy cài đặt mà bạn nhận một exception — trường hợp may mắn — hoặc một phép cắt/dệm âm thầm cho ra một mô hình hoạt động nhưng kém hẳn. Dấu hiệu nhận biết: điểm số tụt đúng một bậc ngay khi đổi độ phân giải, trong khi mọi thứ khác giữ nguyên.

CẦN NHỚ

Do mọi kiến trúc chuỗi bằng ba câu: thông tin đi từ i tới j qua bao nhiêu bước, chi phí tăng theo T như thế nào, và cái gì đang giữ vai trò “biết thứ tự”. RNN cho đường đi $O(T)$ và chi phí $O(T)$; attention cho đường đi $O(1)$ và chi phí $O(T^2)$; mọi biến thể là một điểm nằm giữa hai thái cực đó. [T2]

F. Kết nối

Bước hai của mạch — cổng tạo đường cộng — là cùng một ý tưởng với kết nối tắt ở [C/13/01-dong-cnn](#), và cả hai đều là hệ quả của lập luận tích Jacobian ở [C/12/02-lam-cho-training-chay-duoc](#); nếu chỉ nhớ một liên kết trong cả chương 13 thì nên nhớ liên kết này. Mục tiếp theo, [C/13/03-vit-va-inductive-bias](#), mang attention vào ảnh và hỏi cái giá của việc vứt bỏ tính cục bộ. Phần FlashAttention nối trực tiếp sang [B/11-gioi-han-tinh-toan](#) (cường độ số học, roofline) và sang [D/20-toi-uu-va-nen-mo-hinh](#). Còn attention với vai trò cơ chế lựa chọn trong bài toán thị giác cụ thể xuất hiện lại ở [C/14-bai-toan-cv-loi](#) qua DETR [47].

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao vanishing gradient trong RNN tệ hơn trong một mạng truyền thẳng cùng độ sâu, dù cơ chế là như nhau?
2. Cổng của LSTM và kết nối tắt của ResNet giống nhau ở điểm cốt lõi nào, và khác nhau ở điểm nào?
3. Nếu bỏ hệ số $1/\sqrt{d}$ trong attention thì hỏng ở đâu, và vì sao hậu quả nặng hơn khi d lớn?
4. FlashAttention không giảm FLOPs — vậy nó nhanh hơn nhờ đâu, và trên phần cứng nào lợi ích của nó nhỏ đi?
5. Ba hướng tấn công chi phí bậc hai nằm ở ba tầng khác nhau; với bài toán segmentation ảnh 1024×1024 thì bạn chọn hướng nào trước, và vì sao?
6. Bạn fine-tune một ViT ở độ phân giải gấp đôi lúc pretrain và kết quả tệ đi. Nghi phạm số một là gì?

13.4 ViT và bài học về inductive bias (Vision Transformers and Inductive Bias)

PHỤ THUỘC [C/13/01-dong-cnn](#) (ba giả định mà convolution cài sẵn); [C/13/02-tu-chuoi-toi-attention](#) (attention, mã hoá vị trí, chi phí bậc hai).

TÓM TẮT

Mục này dùng *Vision Transformer* làm thí nghiệm sạch nhất cho một câu hỏi trung tâm của cả cây: cài sẵn giả định vào mô hình thì được gì và mất gì. ViT vứt bỏ hai giả định mà convolution cài sẵn, và cái giá lộ ra rất rõ ràng dưới dạng *lượng dữ liệu cần có*. Đọc xong bạn quyết định được khi nào

nên dùng ViT, khi nào nên dùng CNN, khi nào nên lai. Bạn cũng đọc được mã nguồn của một hệ detection trong mười phút, nhờ cách tách *backbone*, *neck* và *head*. Nếu chỉ nhớ một điều: *inductive bias là một khoản vay trả trước cho dữ liệu* — bạn hoặc trả bằng giả định, hoặc trả bằng mẫu, và ViT chỉ đơn giản chọn cách trả thứ hai.

KIẾN THỨC CẦN CÓ TRƯỚC

- **inductive bias** — tập giả định mà mô hình mang sẵn trước khi nhìn dữ liệu, ví dụ “pixel gần nhau thì liên quan”; nó thu hẹp không gian giả thuyết nên học nhanh hơn với ít dữ liệu, nhưng nếu giả định sai thì nó chặn luôn nghiệm đúng.
- **patch** — ô vuông nhỏ được cắt ra từ ảnh, thường 16×16 pixel, rồi làm phẳng thành một vector để coi như một token; đây là cách ViT biến ảnh thành chuỗi.
- **đẳng biến và bất biến (equivariance vs invariance)** — đẳng biến tịnh tiến nghĩa là dịch ảnh thì bản đồ đặc trưng dịch theo đúng như vậy; bất biến nghĩa là dịch ảnh thì đầu ra *không đổi*. Convolution cho đẳng biến, còn phép gộp toàn cục ở cuối mới tạo ra bất biến.
- **pretraining và fine-tuning** — huấn luyện trước trên một tập rất lớn để học đặc trưng chung, rồi huấn luyện tiếp trên tập nhỏ của bài toán đích; đây là cách phổ biến nhất để bù lượng dữ liệu mà một mô hình ít giả định đòi hỏi.
- **chưng cất (distillation)** — huấn luyện mô hình học sinh bắt chước đầu ra của một mô hình thầy đã tốt, thay vì chỉ học từ nhãn cứng; nó chuyển được cả các giả định ngầm của thầy sang trò.
- **backbone, neck, head** — ba tầng chức năng của một hệ thị giác: backbone trích đặc trưng từ ảnh, neck hợp nhất đặc trưng ở nhiều tỉ lệ, head sinh đầu ra theo đúng bài toán (nhân lớp, hộp, mặt nạ).
- **FPN (feature pyramid network)** — một kiểu neck ghép đặc trưng độ phân giải cao nhưng ngữ nghĩa yếu ở tầng nông với đặc trưng ngữ nghĩa mạnh nhưng thô ở tầng sâu, để phát hiện được cả vật nhỏ lẫn vật lớn.

A. Đặt vấn đề

Suốt chương 13 ta lặp lại một câu hỏi: nên cài sẵn bao nhiêu giả định vào mô hình? Câu hỏi đó rất khó trả lời bằng lý luận, vì hai vế của nó nằm ở hai đơn vị khác nhau. Một bên là “mô hình biết trước điều gì”; bên kia là “có bao nhiêu dữ liệu”. *Vision Transformer* (ViT) [115] biến câu hỏi đó thành một thí nghiệm đo được: lấy một kiến trúc gần như không có giả định thị giác nào, huấn luyện nó ở nhiều quy mô dữ liệu khác nhau, và xem điểm giao cắt nằm ở đâu.

Kết quả là câu trả lời sạch nhất mà ngành có: với dữ liệu ít, ViT thua CNN rõ rệt; với dữ liệu rất nhiều, ViT vượt. Giá trị cụ thể của mục này là biến quan sát đó thành một quy tắc quyết định dùng được. Quy tắc ấy chạy trên ba dữ kiện: bạn có bao nhiêu ảnh có nhãn, bạn có mô hình pretrain nào, và ràng buộc độ phân giải của bạn là gì. Mục này cũng cho bạn một cách đọc mã nguồn nhanh cho bất kỳ hệ thị giác nào.

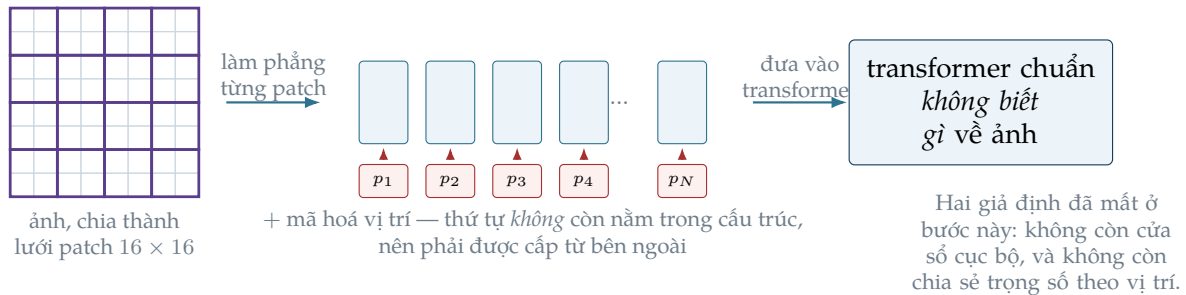
→ **tiền đề B/10** — trực đánh đổi giả định–dữ liệu được phát biểu ở chương thiết kế kiến trúc; mục này là phép đo thực nghiệm cho chính trực đó

B. ViT bỏ đi cái gì

ViT cắt ảnh thành các patch 16×16 , làm phẳng mỗi patch thành một vector, cộng mã hoá vị trí, rồi đưa cả dãy vào một transformer chuẩn. Phép biến đổi đó vứt bỏ hai trong ba giả định của convolution nêu ở C/13/01.

- **Tính cục bộ biến mất.** Ngay từ tầng đầu tiên, mọi patch nói chuyện với mọi patch. Không có gì trong kiến trúc nói rằng hai patch cạnh nhau liên quan nhiều hơn hai patch ở hai góc ảnh — mô hình phải *học* điều đó từ dữ liệu.

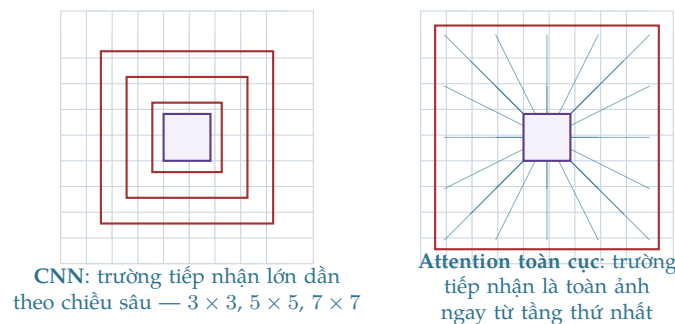
- **Đăng biến tịnh tiến biến mất.** Convolution dùng chung một bộ trọng số cho mọi vị trí nên dịch ảnh thì đặc trưng dịch theo. ViT gán một mã vị trí riêng cho từng ô, nên cùng một vật ở hai vị trí khác nhau đi qua hai đường tính toán hơi khác nhau.
- **Phân cấp cũng gần như biến mất** ở bản ViT gốc: mọi tầng chạy ở cùng một độ phân giải token, không có chuỗi giảm dần độ phân giải như CNN. Đây là giả định thứ ba, và chính nó sẽ được các biến thể phân cấp lấy lại đầu tiên.



Hình 13.9. Ba bước biến một ảnh thành thứ mà transformer xử lý được, và chỗ inductive bias rơi ra. Việc cắt patch vẫn giữ lại một chút tính cục bộ: 256 điểm ảnh trong một ô bị gộp cứng. Nhưng *quan hệ giữa* các patch thì không còn được mã hoá ở đâu trong kiến trúc. Nó phải được cấp bằng một bảng vị trí học được. Đó là lý do đổi độ phân giải đầu vào mà quên nội suy bằng này làm mô hình xuống cấp lạng lẽ: bạn vừa cấp sai đúng cái thông tin mà kiến trúc không tự có.

Hình 13.9 cho thấy điều mà câu “ViT coi ảnh như một chuỗi” còn giấu: chuỗi đó không có thứ tự tự nhiên, và toàn bộ hiểu biết về không gian hai chiều bị dồn vào một bảng tham số phụ.

Cần nói ngay một điều để tránh hiểu quá đà: *ViT không phải không có giả định*. Bản thân việc cắt thành patch là một giả định cục bộ khá mạnh — 256 pixel trong một ô bị gộp thành một token và không bao giờ được xét riêng nữa. Ngoài ra, các phân tích attention cho thấy ở những tầng đầu, nhiều đầu tự học ra hành vi gần như cục bộ, tức mô hình khám phá lại một phần giả định đã bị vứt. Phát biểu đúng là: ViT giảm mạnh lượng giả định cài sẵn, chứ không xoá sạch.

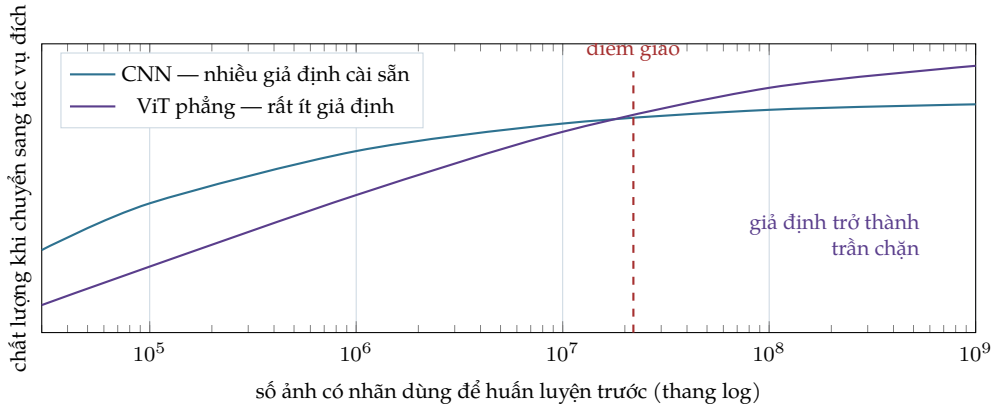


Hình 13.10. Hai cách phủ sóng thông tin. CNN mở rộng trường tiếp nhận dần dần và do đó *cài sẵn* rằng quan hệ gần quan trọng hơn quan hệ xa; attention toàn cục nối mọi vị trí ngay lập tức và không cài giả định nào về khoảng cách, đổi lại phải học quan hệ đó từ dữ liệu và trả chi phí bậc hai. Swin và các biến thể cửa sổ nằm giữa hai thái cực này.

Hình 13.10 là cách gọn nhất để thấy cả ưu điểm lẫn cái giá: bên trái, thông tin xa cần nhiều tầng nhưng mỗi tầng rẻ và có sẵn tiên nghiệm đúng cho ảnh tự nhiên; bên phải, thông tin xa có ngay nhưng phải trả bằng T^2 cặp quan hệ và bằng việc mô hình không được cho biết cặp nào đáng quan tâm.

C. Cái giá hiện ra dưới dạng lượng dữ liệu

Kết quả gốc của ViT rất dễ tóm tắt: huấn luyện từ đầu trên ImageNet-1k thì nó thua các ResNet cùng hạng; pretrain trên một tập lớn hơn nhiều bậc rồi fine-tune thì nó vượt [115]. Đây là minh chứng trực tiếp cho trục đánh đổi giả định–dữ liệu: giả định mà bạn không cài vào kiến trúc thì phải mua bằng mẫu.



Hình 13.11. Trục đánh đổi giả định–dữ liệu, vẽ định tính. Ở phía trái, các giả định mà convolution cài sẵn là món quà miễn phí: chúng thu hẹp không gian giả thuyết đúng chỗ, nên CNN học được từ ít mẫu. Ở phía phải, chính các giả định ấy thành trần — chúng cấm mô hình biểu diễn những quan hệ mà dữ liệu đã đủ để dạy nó. Điều quan trọng nhất trong hình là có một điểm giao chứ không phải vị trí của nó: vị trí phụ thuộc tác vụ, recipe và quy mô mô hình, và ba cách trả nợ giả định ở mục C đều là cách kéo đường tím sang trái.

Hình 13.11 biến một câu định tính thành một hình có thể tranh luận được: câu hỏi “CNN hay ViT” thực chất là câu hỏi “tôi đang đứng ở đâu trên trục hoành”.

Điều làm câu chuyện thú vị là có ít nhất ba cách trả khoản nợ đó, và chúng trả bằng ba loại tiền khác nhau.

- **Trả bằng dữ liệu** — pretrain trên tập cực lớn, rồi fine-tune. Hiệu quả nhất về chất lượng, nhưng chỉ khả thi nếu bạn có tập đó hoặc có checkpoint của người khác; và nó đẩy toàn bộ chi phí sang một pha huấn luyện mà bạn không kiểm soát.
- **Trả bằng chính quy hoá và chứng cất** — DeiT cho thấy với augmentation mạnh, lịch học dài và một thầy CNN để chứng cất, ViT huấn luyện được trên riêng ImageNet-1k [116]. Đáng chú ý là thầy CNN chuyển sang trò không chỉ nhãn mà cả *giả định* của mình — đây là một cách cài inductive bias qua cửa sau.
- **Trả bằng kiến trúc** — lắp lại một phần giả định đã bỏ, tức các biến thể phân cấp ở mục D. Rẻ nhất về dữ liệu, nhưng đánh mất sự thuần khiết và một phần khả năng mở rộng theo quy mô.

→ **hệ quả của C/18** — pretraining tự giám sát là cách rẻ nhất để trả khoản nợ giả định; các backbone mạnh nhất hiện nay đều đến từ hướng này.

→ **cùng nguyên lý với C/12/04** — augmentation mạnh và chứng cất đều bơm thêm giả định vào mô hình, chỉ khác đường đi.

D. Phản ứng thứ nhất: ViT phân cấp

ViT phẳng có hai vấn đề thực dụng ngoài chuyện dữ liệu: nó chỉ sinh ra đặc trưng ở một tỉ lệ duy nhất, trong khi detection và segmentation cần nhiều tỉ lệ; và chi phí bậc hai chặn đường tới độ phân giải cao, như đã tính ở C/13/02. Swin giải cả hai bằng một thay đổi: giới hạn attention

trong các cửa sổ cố định rồi *dịch* cửa sổ ở tầng kế tiếp để thông tin rò qua ranh giới, đồng thời gộp patch dần để tạo kim tự tháp đặc trưng [108]. Chi phí trở thành tuyến tính theo số token, và đầu ra có đúng dạng mà FPN cần. PVT đi theo hướng gần giống nhưng giảm số khoá và giá trị thay vì giới hạn cửa sổ. Còn lại ghép đơn giản nhất là thay phép cắt patch bằng vài tầng convolution làm “stem”. Cách này được biết là làm giai đoạn huấn luyện đầu ổn định hơn hẳn. Đây là kinh nghiệm thực hành rất phổ biến, không phải một kết quả tôi kiểm chứng lại.

Cái giá thì cụ thể: cửa sổ tạo lại một dạng “đường đi dài” cho quan hệ xa, đúng thứ mà attention sinh ra để xoá; và kiến trúc nhiều giai đoạn khó mở rộng theo quy mô một cách trơn tru hơn so với một chồng khối đồng nhất.

E. Phản ứng thứ hai: ConvNeXt, và vì sao nó là phản chứng quan trọng

Hướng ngược lại không sửa transformer mà sửa cách so sánh. ConvNeXt giữ nguyên convolution rồi lần lượt áp từng thành phần recipe của transformer, đo sau mỗi bước, và kết quả là CNN bắt kịp [121]. Kết luận đã nêu ở C/13/01 và nhắc lại ở đây vì nó thay đổi cách đọc chính mục này: phần lớn khoảng cách được gán cho “attention thay convolution” thực ra thuộc về công thức huấn luyện. Nói cách khác, thí nghiệm ViT vẫn đúng về trục giả định–dữ liệu, nhưng *độ lớn* của hiệu ứng kiến trúc đã bị phóng đại trong cách trình bày đương thời.

F. Ba dạng tổ chức và ba lớp bài toán

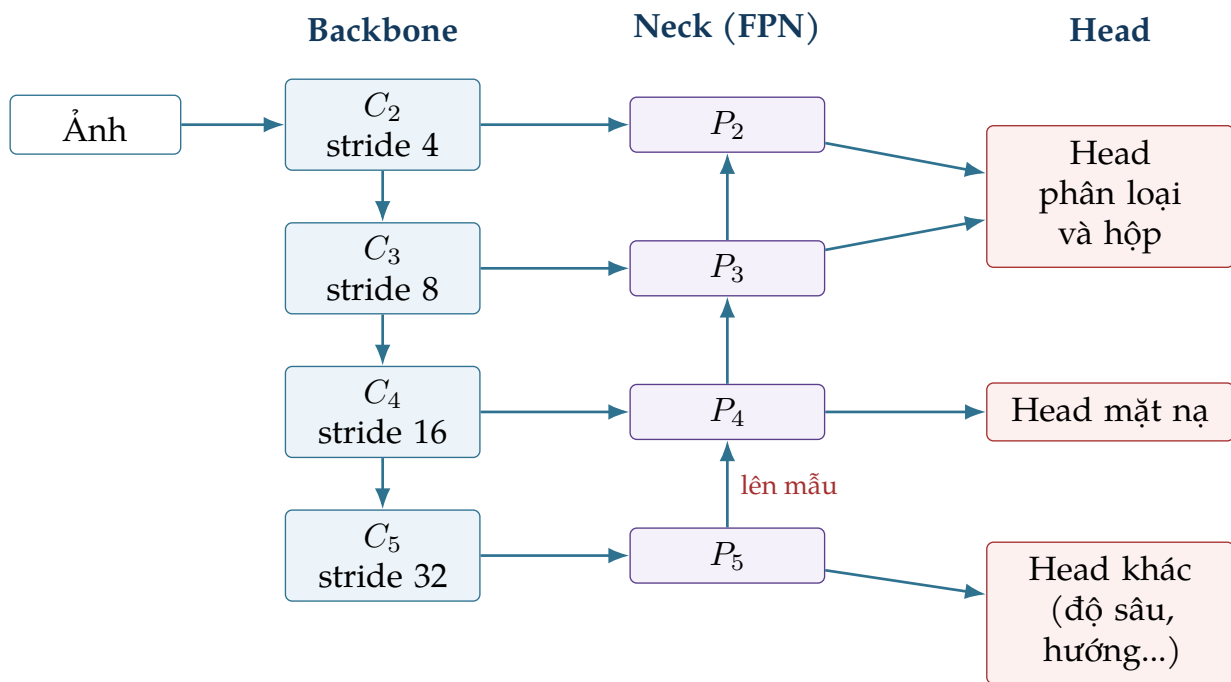
Chọn sai dạng tổ chức là lỗi thiết kế cơ bản, không phải lỗi tinh chỉnh. Không có siêu tham số nào cứu được một encoder-only đang bị bất sinh chuỗi tự hồi quy.

Bảng 13.4. Ba dạng tổ chức của một mô hình dựa trên transformer, và lớp bài toán tương ứng.

Dạng	Cơ chế	Hợp với bài toán nào	Hỏng khi nào
Encoder-only	Mọi token thấy mọi token, một lượt	Hiểu và dự đoán từ toàn bộ đầu vào: phân loại, dự đoán dài, trích đặc trưng cho hệ khác	Cần sinh chuỗi có độ dài chưa biết trước, hoặc cần sinh tự hồi quy
Decoder-only	Attention có mặt nạ nhân quả, sinh từng token dựa trên các token đã sinh	Sinh tự hồi quy: ngôn ngữ, chuỗi hành động, mã hoá ảnh thành token rồi sinh	Suy luận chậm vì tuần tự; lãng phí khi đầu ra là một nhãn duy nhất
Encoder-decoder	Encoder đọc đầu vào, decoder sinh đầu ra và tra cứu encoder bằng cross-attention	Biến đổi chuỗi sang chuỗi; trong thị giác là DETR với tập truy vấn học được [47]	Nhiều tham số và nhiều đường nối hơn; hội tụ chậm hơn ở bài toán nhỏ

G. Tách backbone, neck, head – cách đọc code nhanh nhất

Ba khái niệm này không phải lý thuyết mà là quy ước kỹ thuật, và chúng có giá trị thực dụng rất cao: gần như mọi repo detection và segmentation hiện đại đều tổ chức theo đúng ba khối này, nên nắm được chúng là đọc được cấu trúc một repo lạ trong mười phút thay vì một ngày.



Hình 13.12. Ba khối chức năng của một hệ thị giác dày. Backbone sinh đặc trưng ở nhiều stride; neck hợp nhất chúng theo chiều từ sâu về nông để mọi tỉ lệ đều có ngữ nghĩa mạnh; head chuyển đặc trưng thành đầu ra của bài toán. Thay backbone là thao tác rẻ nhất và hay làm nhất; thay neck ảnh hưởng chủ yếu tới vật nhỏ; thay head là đổi bài toán.

Hình 14.1 cũng giải thích vì sao ViT phẳng khó lắp vào hệ detection: nó chỉ sinh ra một cột duy nhất chứ không sinh ra bốn mức C_2 tới C_5 , nên hoặc phải chế thêm một neck đặc biệt, hoặc phải chuyển sang biến thể phân cấp. Đây là một ràng buộc kỹ thuật rất thực tế và thường quyết định lựa chọn backbone hơn cả điểm số ImageNet.

↔ **tiền đề C/14** — các head của detection và segmentation giả định có sẵn bốn mức đặc trưng, nên dạng đầu ra của backbone quyết định head nào lắp được

H. Giới hạn và trường hợp hỏng

Bốn chỗ hỏng đáng nhớ, xếp theo tần suất gặp trong thực hành.

Thứ nhất, *ViT huấn luyện từ đầu trên tập nhỏ*. Đây là cách phổ biến nhất để kết luận sai rằng “transformer không hợp bài toán của tôi”. Nếu bạn có vài nghìn ảnh, kết luận đúng là bạn không đủ dữ liệu để trả khoản nợ giả định, chứ không phải kiến trúc kém.

Thứ hai, *kích thước patch so với kích thước đối tượng*. Một patch 16×16 nghiền mọi đối tượng nhỏ hơn 16 pixel thành một phần của một token duy nhất. Với ảnh camera gắn xe, người đi bộ ở xa chỉ chiếm mười pixel. Đây không phải chi tiết nhỏ mà là giới hạn cứng. Nó giải thích vì sao các hệ dự đoán dày dùng patch nhỏ hoặc dùng conv stem.

Thứ ba, *mã hoá vị trí khi đổi độ phân giải*, đã nói ở mục trước và nhắc lại vì nó thực sự hay xảy ra.

Thứ tư, *mất đẳng biến tịnh tiến là có thật và đo được*. Cùng một vật ở hai vị trí có thể cho đặc trưng hơi khác nhau. Trong phần lớn trường hợp, augmentation cắt ngẫu nhiên đã bù đủ. Nhưng ở các bài toán đo lường chính xác như ước lượng tư thế hay đo khoảng cách, đây là một nguồn sai số hệ thống đáng kiểm tra chứ không nên giả định là không có.

Về độ tin cậy: quan hệ “ít dữ liệu thì CNN thắng, nhiều dữ liệu thì ViT thắng” là kết quả đã công bố và được lặp lại rộng rãi; còn mức dữ liệu cụ thể ở điểm giao cắt phụ thuộc tác vụ, recipe và quy mô mô hình tới mức tôi không nêu một con số nào ở đây.

CẠM BẤY

Cạm bẫy tư duy lớn nhất của mục này là đọc “ViT không có inductive bias” theo nghĩa đen rồi suy ra “ViT là mô hình trung lập, không giả định gì”. Không đúng. Phép cắt patch là một giả định cục bộ rất mạnh. Mã hoá vị trí là một giả định về cách biểu diễn không gian. Và cách chọn dữ liệu pretrain là một giả định khổng lồ về thế giới mà mô hình sẽ gặp. Phát biểu chính xác là ViT *dịch chuyển* giả định từ kiến trúc sang dữ liệu và recipe — và giả định nằm trong dữ liệu thì khó nhìn thấy hơn nhiều so với giả định nằm trong kiến trúc.

CẦN NHỚ

Inductive bias là khoản vay trả trước cho dữ liệu: giả định bạn không cài vào kiến trúc thì phải mua bằng mẫu, bằng chứng cất, hoặc bằng augmentation. Khi chọn giữa CNN, ViT và các biến thể lai, câu hỏi đầu tiên không phải “cái nào mạnh hơn” mà là “tôi có bao nhiêu dữ liệu và bao nhiêu tiên nghiệm đáng tin về bài toán này”. [T2]

I. Kết nối

Mục này khép lại chương 13 bằng cách trả lời câu hỏi mở ở [C/13/01-dong-cnn](#): ba giả định của convolution đáng giá bao nhiêu, đo bằng lượng dữ liệu cần để thay thế chúng. Nó dựa hoàn toàn vào cơ chế attention ở [C/13/02-tu-chuoi-toi-attention](#), và cách tách backbone–neck–head ở mục G là tiền đề trực tiếp cho [C/14-bai-toan-cv-loi](#), nơi các head cụ thể được thiết kế. Về phía dữ liệu, việc trả nợ giả định bằng pretraining là nội dung của [C/18-pretraining-va-foundation](#), còn việc trả bằng augmentation đã nằm ở [C/12/04-tong-quat-hoa-va-chan-doan](#). Cuối cùng, bài học ConvNeXt nối sang [E/24-thi-nghiem-va-ablation](#), chương duy nhất trong cây dạy cách không bị một bảng so sánh đánh lừa.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. ViT bỏ đi hai giả định nào của convolution, và mỗi giả định bị bỏ gây ra hậu quả đo được nào?
2. Vì sao nói ViT vẫn có inductive bias, dù nó bỏ tính cục bộ và đẳng biến tịnh tiến?
3. Ba cách bù lại lượng dữ liệu mà ViT đòi hỏi là gì, và mỗi cách trả giá bằng loại tài nguyên nào?
4. Vì sao Swin lắp vào một hệ detection dễ hơn ViT phẳng — câu trả lời nằm ở khối nào trong sơ đồ backbone–neck–head?
5. Nếu ConvNeXt cho thấy recipe quan trọng hơn kiến trúc, điều đó thay đổi cách bạn đọc một bảng so sánh backbone thể nào?
6. Bạn cần phát hiện người đi bộ cách xa 80 mét, chỉ chiếm khoảng 12 pixel chiều cao. Vì sao ViT patch 16 là lựa chọn tệ, và hai cách sửa là gì?
7. Trong hệ của bạn, thay backbone, thay neck và thay head có tác động khác nhau thế nào — và vì sao thứ tự thử nên là backbone trước?

13.5 Thực hành

Chương này dễ đọc thụ động và khó nhớ, vì phần lớn nội dung là tên riêng. Hai mục dưới đây là phần đọc và phần tự vấn; phần chuyển nó thành thứ dùng được nằm ở khối mini project cuối chương, nơi bạn tự tạo ra bảng đo của chính mình thay vì tin bảng của người khác.

Repo và SOTA (chốt tháng 9/2026). Backbone dùng thật: [facebookresearch/dinov3](#) phủ dãi từ khoảng 21 triệu tới vài tỉ tham số nhưng có giấy phép riêng, nên nếu cần điều khoản thoáng thì DINOv2 (Apache 2.0) là lựa chọn thay thế [52]; ConvNeXt-V2 và Swin lấy qua [timm](#); ngoài ra EVA-02 và SigLIP. Cài đặt tối giản để *đọc* chứ không để chạy production: [lucidrains/vit-pytorch](#) và [karpathy/nanoGPT](#). Attention nhanh: [Dao-AILab/flash-attention](#). Mô hình không gian trạng thái: [state-spaces/mamba](#). Danh sách tên mô hình này chốt tại tháng 9/2026 và sẽ cũ trong sáu tới mười hai tháng; ngược lại, các repo hạ tầng như [timm](#) hay COLMAP bền hơn nhiều, nên hãy phân bổ thời gian học theo đúng tỉ lệ đó.

Câu hỏi kiểm tra hiểu. Vì sao ViT cần nhiều dữ liệu hơn CNN, và ba cách bù lại — pretraining, augmentation mạnh, lai ghép — trả giá bằng loại tài nguyên nào? FlashAttention không giảm FLOPs, vậy nó nhanh hơn nhờ đâu, và trên phần cứng nào lợi ích của nó thu hẹp lại? Nếu ConvNeXt cho thấy recipe quan trọng hơn kiến trúc, điều đó thay đổi thứ tự bạn đọc một paper kiến trúc như thế nào?

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ba giả định của convolution	Tính cục bộ: chỉ pixel lân cận liên quan trực tiếp. Chia sẻ trọng số, tức đẳng biến tịnh tiến. Và phân cấp: đặc trưng phức tạp ghép từ đặc trưng đơn giản. Cả chương là chuyện thêm hoặc bớt ba giả định này.
•• Trường tiếp nhận	Vùng pixel đầu vào ảnh hưởng tới một giá trị đầu ra. Trong CNN nó lớn dần theo chiều sâu; trong attention toàn cục nó bằng cả ảnh ngay từ tầng đầu.
•• Vì sao xếp chồng hai tầng 3×3 thay một tầng 5×5 ?	Cùng trường tiếp nhận, nhưng tốn $18C^2$ tham số thay vì $25C^2$, và có thêm một lần phi tuyến ở giữa. Một trong ít phép đổi tốt ở cả ba mặt.
•• Vì sao nút thất 1×1 rẻ hơn gần chín lần?	Vì phần đắt nhất của conv là tích của kênh vào, kênh ra và diện tích của số. Giảm kênh xuống 64 rồi mới làm 3×3 thì k^2 chỉ nhân với một số nhỏ: 589 824 phép nhân còn 69 632.
••• Kết nối tắt	Đường cộng $y = x + F(x)$ đưa ánh xạ đồng nhất thành phương án mặc định. Jacobian thành $I + \partial F / \partial x$, nên tích qua nhiều khối luôn còn một thành phần bằng 1. Giá phải trả: phải giữ x sống tới điểm cộng, nên tốn thêm bộ nhớ.
•• Depthwise separable	Tách conv thành một bước không gian chạy riêng từng kênh, rồi một bước 1×1 trộn kênh. Chi phí còn $1/C_{\text{out}} + 1/k^2$ lần, khoảng 0,115 với $k = 3$ và $C_{\text{out}} = 256$.
••• Giảm 8,7 lần FLOPs mà độ trễ gần như không đổi — nguyên nhân?	Bước depthwise đọc mỗi phần tử để làm vôn vẹn chín phép nhân, nên nó nghẽn ở băng thông chứ không ở khả năng tính toán. Phân biệt với nghẽn tính toán: đo thông lượng bộ nhớ, nếu đã sát trần thì giảm FLOPs vô ích.
• Dilated convolution	Giãn khoảng cách giữa các điểm lấy mẫu, nên trường tiếp nhận tăng mà không phải giảm độ phân giải. Sinh ra cho segmentation; hòng thành hoa văn răng lược nếu xếp chồng nhiều tầng cùng hệ số giãn.
•• Cổng của LSTM	Cơ chế cho trạng thái ô đi qua theo dạng cộng: $c_t = f_t \odot c_{t-1} + \dots$. Khi cổng quên mở gần 1, gradient chảy qua mà không bị nhân với ma trận trọng số — đúng mero của kết nối tắt, chỉ khác là hệ số học được.
••• Attention	Mỗi vị trí sinh truy vấn, mọi vị trí sinh khoá và giá trị; đầu ra là tổ hợp có trọng số của mọi vị trí. Đường đi thông tin bằng 1 thay vì T , đổi lại chi phí và bộ nhớ $O(T^2)$.
••• Vì sao attention chia cho $\sqrt{d}$?	Tích vô hướng d chiều có độ lệch chuẩn $\sqrt{d}$, tức 8 khi $d = 64$. Softmax trên logit lệch chuẩn 8 bão hoà thành gần one-hot, và ở vùng đó đạo hàm bằng 0.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Mã hoá vị trí	Thông tin vị trí được cấp từ bên ngoài, vì phép attention bất biến với hoán vị. Bốn họ: sin tuyệt đối, học được, tương đối, và RoPE. Loại tuyệt đối ngoại suy kém ra ngoài độ dài đã thấy khi huấn luyện.
●● Window attention	Chỉ cho attention trong một cửa sổ cố định, rồi dịch cửa sổ ở tầng sau để thông tin rò qua ranh giới. Chi phí thành tuyến tính theo T ; giá phải trả là quan hệ xa cần nhiều tầng mới nổi được.
● FlashAttention	Chia ô và tính softmax trực tuyến để không bao giờ dựng ma trận $T \times T$ trong HBM. Số FLOPs không đổi và độ phức tạp tiệm cận cũng không đổi; cái giảm là số vòng đọc ghi bộ nhớ ngoài.
●●● ViT	Cắt ảnh thành patch, coi mỗi patch là một token, rồi đưa vào transformer chuẩn. Nó vứt bỏ tính cục bộ và đẳng biến tịnh tiến, nên thua CNN khi dữ liệu ít và vượt khi dữ liệu rất nhiều.
●●● Vì sao ViT cần nhiều dữ liệu hơn CNN?	Vì giả định không cài vào kiến trúc thì phải mua bằng mẫu. Ba cách trả: pretrain trên tập rất lớn, chung cất từ thầy CNN cộng augmentation mạnh, hoặc lắp lại giả định bằng kiến trúc phân cấp.
●● Đối độ phân giải đầu vào rồi ViT tệ hẳn — nguyên nhân?	Quên nội suy bằng mã hoá vị trí: số token đổi nhưng bảng vẫn kích thước cũ. Phân biệt với thiếu fine-tune: lỗi này làm điểm tụt ngay cả khi bạn fine-tune lại đủ lâu.
●● ViT phẳng hết bộ nhớ ở dự đoán dày stride 4 — nguyên nhân?	Số token là $128^2 = 16\,384$, nên ma trận attention chiếm khoảng 537 MB cho một đầu, một tầng. Lỗi sửa là chuyển sang biến thể cửa sổ hoặc phân cấp.
●●● ConvNeXt	Một CNN được hiện đại hoá bằng chính công thức huấn luyện của ViT, và nó bắt kịp ViT cùng hạng. Bài học: phần lớn chênh lệch từng được gán cho kiến trúc thực ra thuộc về recipe.
●●● Backbone, neck, head	Backbone trích đặc trưng ở nhiều stride, neck hợp nhất chúng theo chiều từ sâu về nông, head sinh đầu ra của bài toán. Đây là cách đọc một repo detection nhanh nhất, và là lý do ViT phẳng khó lắp: nó chỉ sinh một mức đặc trưng.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
timm	Kho backbone chuẩn của ngành: ConvNeXt, Swin, EVA, ResNet hiện đại hoá — và là nơi thấy rõ nhất rằng cùng kiến trúc với recipe khác nhau cho điểm số khác nhau	Hạ tầng bền; đáng đầu tư hơn bất kỳ tên mô hình nào
DINOv2 và DINOv3	Backbone ViT trả khoản nợ giả định bằng pretraining tự giám sát ở quy mô rất lớn — đúng cách thứ nhất trong ba cách ở mạch 3	DINOv2 giấy phép Apache 2.0; DINOv3 giấy phép riêng, chốt 9/2026
Swin và các biến thể phân cấp	Lắp lại tính cục bộ và đa tỉ lệ mà ViT phẳng đánh mất, để sinh ra bốn mức đặc trưng mà FPN cần	Vẫn là backbone mặc định cho nhiều hệ dự đoán dày
flash-attention	Tấn công chi phí bậc hai ở tầng hệ thống: không dựng ma trận $T \times T$ trong HBM, giữ nguyên FLOPs	Đã thành mặc định trong phần lớn thư viện transformer
ConvNeXt-V2	Bằng chứng đang sống rằng convolution vẫn cạnh tranh khi được huấn luyện bằng recipe hiện đại	Đọc kèm bảng ablation của nó là cách nhanh nhất để thấy từng thành phần recipe đáng bao nhiêu

MINI PROJECT SLAM · Chọn backbone cho bộ mô tả đặc trưng dưới ngân sách độ trễ của Orin · 1–2 ngày

Mục tiêu	biến câu hỏi “backbone nào tốt hơn” thành một quyết định có bằng chứng, dưới đúng ràng buộc của bạn. Và thấy tận mắt rằng thứ hạng theo FLOPs có thể ngược thứ hạng theo độ trễ thật.
Dữ liệu	dùng lại tập cặp ghép đã gán nhãn từ mini project chương 12, nhưng thay đầu vào thủ công bằng miếng ảnh 32×32 cắt quanh mỗi góc ORB trên EuRoC. Ba backbone cùng hạng lấy từ timm : một CNN cổ điển (ResNet-18), một CNN hiện đại hoặc hướng thiết bị nhúng (ConvNeXt-T hoặc MobileNetV3), và một ViT phân cấp (Swin-T).
Các bước	fine-tune cả ba ở cùng ngân sách epoch và cùng công thức huấn luyện — đây là điều kiện bắt buộc, nếu không thì bạn đang đo recipe chứ không đo kiến trúc; đo trên Jetson Orin ở batch bằng 1, tức đúng chế độ mà SLAM chạy, sau khi khoá xung nhịp, lấy cả trung vị lẫn phân vị 95; lập bảng sáu cột gồm độ chính xác ghép, số tham số, FLOPs, độ trễ trung vị, độ trễ p_{95} , bộ nhớ đỉnh.
Xong khi	bạn chỉ ra được backbone nào thắng ở một ngân sách cụ thể, chẳng hạn 8 ms mỗi khung hình. Bạn giải thích thứ hạng bằng hai đại lượng: trường tiếp nhận so với kích thước miếng ảnh, và cường độ số học so với ngưỡng roofline của Orin. Và bạn chỉ ra được ít nhất một cặp trong bảng có thứ hạng FLOPs ngược thứ hạng độ trễ.
Nối với	B/11 cho mô hình roofline dùng để giải thích cột độ trễ; C/12/03 cho việc giữ công thức huấn luyện ngang nhau giữa ba lần chạy; C/14 cho bước tiếp theo là lắp backbone đã chọn vào một head thật.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì cửa sổ 3×3 không phải một giới hạn mà là một *giả định*: quan hệ gần quan trọng hơn quan hệ xa, và mẫu hình có ý nghĩa như nhau ở mọi vị trí. Giả định đó đúng với ảnh tự nhiên, nên nó thay thế cho dữ liệu. Attention toàn cục không giả định gì về khoảng cách, nên phải học quan hệ đó từ mẫu và trả thêm chi phí $O(T^2)$.
- A2. Gradient descent không tìm ra nghiệm “các tầng thừa làm ánh xạ đồng nhất”, dù nghiệm đó tồn tại. Đây là vấn đề tối ưu hoá, không phải sức chứa — và kết nối tắt sửa nó bằng cách biến ánh xạ đồng nhất thành phương án mặc định, đồng thời để lại một thành phần bằng 1 trong tích các Jacobian.
- A3. Vì phần depthwise đọc mỗi phần tử đầu vào để làm vôn vện $k^2 = 9$ phép nhân, nên cường độ số học của nó rất thấp và kernel rơi vào phía nghèo bằng thông bộ nhớ. Giảm FLOPs của một kernel vốn đã nghèo ở bộ nhớ thì gần như không giảm thời gian.
- A4. Cải tiến công thức huấn luyện — optimizer, lịch học, số epoch, augmentation — nhiều hơn là cải tiến kiến trúc. Điều đó không xoá bỏ tiến bộ về kiến trúc, nhưng nó nói rằng mọi bảng so sánh không kiểm soát recipe đều không tách được hai nguồn ấy, nên câu hỏi đầu tiên khi đọc bảng phải là câu hỏi về recipe.
- A5. Không có gì trong mô hình thay đổi — thứ thay đổi là *ai trả tiền cho các giả định*. Với ít dữ liệu, giả định cài sẵn của convolution là món quà miễn phí; với rất nhiều dữ liệu, chính giả định đó thành trần, còn mô hình ít giả định thì học ra được các quan hệ mà convolution cấm nó biểu diễn.

NÓI LẠI THẬT ĐƠN GIẢN

Giả sử bạn thuê một người thợ mới và giao việc phân loại đồ vật qua ảnh. Bạn có hai cách. Cách thứ nhất là dặn trước thật nhiều quy tắc: hai chỗ cạnh nhau trong ảnh thường liên quan tới nhau; một cái cốc vẫn là cái cốc dù nó nằm ở góc nào của bàn; nhìn nét nhỏ trước rồi mới ghép thành hình lớn. Cách thứ hai là không dặn gì cả, chỉ đưa cho họ thật nhiều ví dụ và để họ tự rút ra. Cả chương là câu chuyện về việc chọn giữa hai cách đó, và phát hiện quan trọng nhất là chúng thay thế được cho nhau. Ai chỉ có vài nghìn tấm ảnh thì dặn trước nhiều quy tắc là món hời, vì quy tắc làm thay phần việc mà ví dụ đáng ra phải làm. Ai có hàng trăm triệu tấm thì chính những quy tắc ấy lại thành xiềng: chúng cấm người thợ nhận ra các mối liên hệ mà bạn không nghĩ tới khi dặn.

Có hai cái giá phải nhớ. Một là bỏ bớt quy tắc thì phải trả bằng ví dụ, hoặc bằng việc mượn một người thợ đã được luyện sẵn ở nơi khác. Hai là “rẻ hơn trên giấy” không có nghĩa là “nhanh hơn trong xương”: một cách làm cắt được tám phần chín số phép tính vẫn có thể chạy chậm y như cũ, vì thời gian thật đi vào việc lấy vật liệu ra khỏi kho chứ không vào việc gia công. Và bất ngờ cuối cùng: phần lớn khác biệt giữa hai người thợ hoá ra không nằm ở quy tắc họ được dạy, mà ở cách họ được luyện tập.

Một câu: Quy tắc dạy trước là khoản vay trả trước cho ví dụ — ít ví dụ thì nên dạy nhiều, nhiều ví dụ thì chính lời dạy thành xiềng, và phần lớn chênh lệch bạn thấy trên bảng xếp hạng lại đến từ cách luyện tập chứ không từ lời dạy.

13.6 Kết nối

Chương này nhận đầu vào từ [C/12-co-che-huan-luyen](#): không có bộ máy huấn luyện thì các kiến trúc ở đây chỉ là sơ đồ khối. Nó cũng nhận từ [B/10-thiet-ke-kien-truc](#), nơi trực đánh đổi giả định-dữ liệu được phát biểu lần đầu. Xuôi xuống, nó cấp backbone cho [C/14-bai-toan-cv-loi](#), [C/16-thi-giac-3d](#) và [C/17-mo-hinh-sinh](#), và đặt sẵn câu hỏi mà [C/18-pretraining-va-foundation](#) trả lời: nếu giả định phải mua bằng dữ liệu, thì mua ở đâu cho rẻ nhất. Hai nhánh ngang nên đọc ngay sau chương này. Thứ nhất là [B/11-gioi-han-tinh-toan](#), để biến các phép tính FLOPs ở đây thành dự đoán độ trễ có căn cứ. Thứ hai là [E/24-thi-nghiem-va-ablation](#), để không bị chính các bảng so sánh trong chương này dẫn tới kết luận quá mạnh.

Tổng kết chương

Ta đã đi qua ba mạch và về cùng một chỗ. Mạch 1 cho thấy dòng CNN là một chuỗi gỡ nút thắt, mỗi bước đẩy độ khó sang một trục khác, và mất xích cuối — ConvNeXt — buộc ta xét lại toàn bộ các so sánh trước đó. Mạch 2 cho thấy attention mua được đường đi thông tin ngắn nhất có thể bằng chi phí bậc hai, và ba hướng chống chi phí đó nằm ở ba tầng trừu tượng khác nhau. Mạch 3 cho thấy khi vứt bỏ giả định thị giác, cái giá hiện ra rất sạch dưới dạng lượng dữ liệu cần có — và có ít nhất ba loại tiền để trả khoản đó.

Còn bỏ ngỏ ba chỗ. Thứ nhất, chương này chỉ nói về *trích đặc trưng*; việc biến đặc trưng thành đầu ra của một bài toán cụ thể — hộp, mặt nạ, khoảng cách — là nội dung của [C/14-bai-toan-cv-loi](#). Thứ hai, cách rẻ nhất để trả khoản nợ giả định là dùng một mô hình đã pretrain, và toàn bộ chủ đề đó nằm ở [C/18-pretraining-va-foundation](#). Thứ ba, mô hình không gian trạng thái là hướng đang chuyển động nhanh nhất trong ba mạch. Chương này đánh giá rằng nó chưa thay thế attention trong thị giác. Đó là đánh giá tính tới tháng 9/2026, không phải một kết luận bền. Đọc tiếp [C/14-bai-toan-cv-loi](#) với câu hỏi: khi lắp một head cụ thể lên các backbone này, ràng buộc nào của backbone sẽ trở thành ràng buộc của cả hệ thống?

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Trục duy nhất của chương là “cài sẵn nhiều giả định, cần ít dữ liệu” đổi lại “cài ít giả định, cần nhiều dữ liệu”. Hãy đặt ResNet, Swin và ViT phẳng lên trục đó, rồi nói cái gì trong mô hình thực sự thay đổi khi dữ liệu tăng, khiến thứ hạng đảo chiều?
2. Kết nối tắt của ResNet, cổng của LSTM và chuẩn hoá theo tầng cùng phục vụ một mục tiêu đã đặt ra ở [C/12-co-che-huan-luyen](#). Mục tiêu đó là gì, và vì sao attention giải quyết nó theo một cách khác hẳn ba thứ kia?
3. Nếu ConvNeXt đúng khi nói recipe quan trọng hơn kiến trúc, thì các bảng so sánh trong chính chương này còn đọc được ở mức nào, và bạn đòi thêm cột nào trước khi tin một dòng?

4. FLOPs xuất hiện ở cả ba mục nhưng không mục nào coi nó là độ trễ. MobileNet, FlashAttention và attention cửa sổ mỗi cái phá vỡ quan hệ FLOPs–độ trễ theo kiểu riêng nào, và đại lượng chung nào giải thích được cả ba?
5. Bạn cần một backbone cho ảnh rất lớn, vật thể chỉ vài chục pixel, và ngân sách bộ nhớ chặt. Ràng buộc nào loại bớt phương án trước, và vì sao câu trả lời đôi khi bạn có hay không có một mô hình pretrain?
6. Cả ba mục đều kể lịch sử như một chuỗi gỡ nút thắt. Nếu bạn tin nút thắt hiện tại nằm ở dữ liệu chứ không ở kiến trúc, thì quan sát nào trong chính chương này ủng hộ niềm tin đó, và quan sát nào phản bác?

Bài toán CV lõi: nhận dạng, phát hiện, phân vùng (Core Vision Tasks: Recognition, Detection, Segmentation)

PHỤ THUỘC C/13-kien-truc-backbone (backbone, stride, attention); B/09-thiet-ke-ham-mat-mat (loss đa nhiệm, mất cân bằng); B/07-chia-du-lieu-va-danh-gia (mAP, mIoU, đánh giá theo lát). [T3]

TÓM TẮT

Chương này lắp các “đầu” bài toán lên trên backbone của chương trước, và đọc toàn bộ lịch sử detection lẫn segmentation theo một trục duy nhất: từng bước nuốt các heuristic bên ngoài vào bên trong mạng, tức biến một khối không nhận gradient thành một khối được tối ưu cho đúng mục tiêu cuối. Đọc xong bạn phát biểu được một bài toán thì giác mới dưới dạng đúng, chọn được họ mô hình từ ràng buộc chứ không từ bảng xếp hạng, và đọc được một bảng so sánh detector với đúng mức hoài nghi. Nếu chỉ nhớ một điều: những chi tiết không được viết trong abstract — cách gán mẫu, tỉ lệ mẫu dương/âm, lịch huấn luyện — thường giải thích chênh lệch kết quả nhiều hơn hình vẽ kiến trúc ở trang đầu.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình và huấn luyện** — mô hình là một hàm có hàng triệu tham số biến ảnh thành dự đoán; huấn luyện là chỉnh dần các tham số đó để dự đoán khớp nhãn, bằng cách đi ngược hướng *gradient* của hàm mất mát.
- **hàm mất mát (loss function)** — một số duy nhất đo mức sai của dự đoán. Đây là thứ định nghĩa “đúng” nghĩa là gì, nên đối hàm mất mát là đối bài toán chứ không phải đối mô hình.
- **backbone, neck, head** — backbone biến ảnh thành lưới đặc trưng; neck trộn các lưới ở nhiều độ phân giải; head là phần nhỏ phía trên sinh ra đầu ra của một bài toán cụ thể. Ba bài toán khác nhau thường dùng chung backbone và chỉ khác head.
- **chân trị (ground truth)** — đáp án do người gán cho mỗi ảnh — khung, mask, hay nhãn lớp. Mô hình chỉ học được đúng những gì nhãn nói, kể cả khi nhãn sai.
- **IoU** — diện tích giao chia diện tích hợp của hai vùng; bằng 1 khi trùng khít, 0 khi rời nhau. Nó là đơn vị đo cơ bản của cả detection lẫn segmentation.
- **mAP và mIoU** — hai thước đo tổng: mAP cho detection (diện tích dưới đường precision-recall, trung bình theo lớp và theo ngưỡng IoU), mIoU cho segmentation (IoU trung bình theo lớp).
- **heuristic** — một quy tắc viết tay, không có tham số học được, do đó không nhận gradient và không tự chỉnh theo lỗi cuối. Ví dụ: NMS, ngưỡng IoU khi gán mẫu, công thức chọn tầng của FPN.

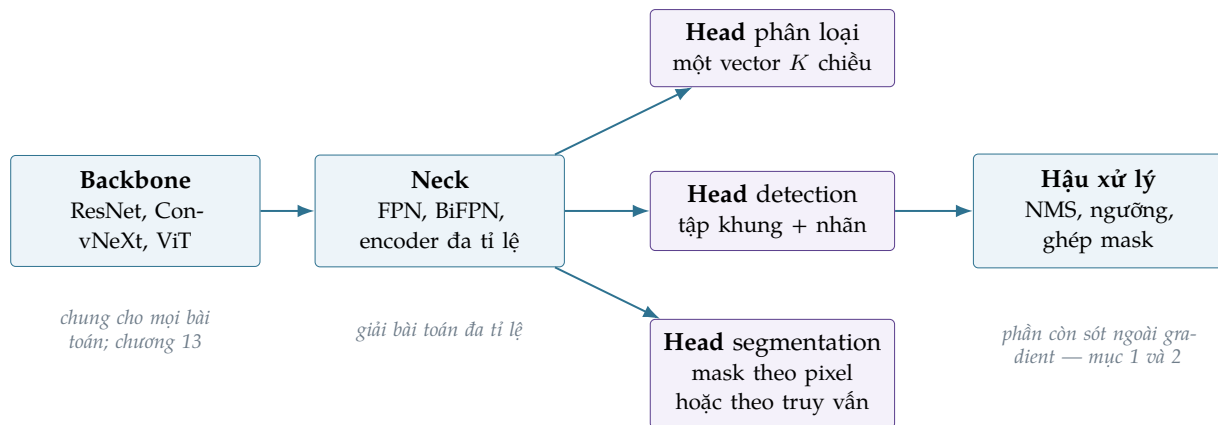
CÂU HỎI MỞ ĐẦU

- Q1. Vì sao thay đúng một quy tắc gán mẫu — một hàm chục dòng nằm trong code, không có trong abstract — lại đổi kết quả nhiều hơn thay cả backbone?
- Q2. R-CNN chạy backbone hai nghìn lần mỗi ảnh, Fast R-CNN chạy một lần. Vì sao bản một lần lại *chính xác hơn* chứ không chỉ nhanh hơn?
- Q3. NMS chỉ là một vòng lặp xóa khung trùng. Vì sao suốt mười năm không ai “nuốt” được nó vào mạng, trong khi selective search thì nuốt được ngay?

- Q4.** Nếu bỏ đúng một phép làm tròn toạ độ mà chất lượng mask nhảy vọt, điều đó nói gì về việc bạn nên tin phần nào của một paper?
- Q5.** Semantic, instance và panoptic segmentation từng là ba code base riêng. Việc chúng gộp được vào một mô hình là dấu hiệu của cái gì?

Chương trước cho bạn cái máy trích đặc trưng; chương này hỏi phải gắn cái gì lên trên nó để trả lời những câu hỏi mà thế giới thật đặt ra. Ba câu hỏi kinh điển — *ảnh này là gì, có những vật gì ở đâu, mỗi pixel thuộc về cái gì* — trông như ba mức độ chi tiết tăng dần của cùng một bài toán. Nhưng chỉ câu thứ nhất có đầu ra kích thước cố định, và chính điều đó chia đôi chương này: mọi khó khăn kỹ thuật của detection và segmentation đều bắt nguồn từ việc đầu ra là một *tập hợp* có lực lượng thay đổi, thứ mà bộ máy gradient không biết chấm điểm trực tiếp.

Lịch sử của lĩnh vực này, đọc theo đúng trục, là một câu chuyện duy nhất: **từng bước nuốt các heuristic bên ngoài vào bên trong mạng**. Selective search bị RPN nuốt, thiết kế đa tỉ lệ thủ công bị FPN nuốt, lấy mẫu cứng bị focal loss nuốt, NMS bị Hungarian matching nuốt. Mỗi lần nuốt, một khối vốn không nhận được gradient trở thành một khối được tối ưu cho đúng mục tiêu cuối, và gần như lần nào hệ thống cũng vừa nhanh hơn vừa chính xác hơn. Đọc theo mạch đó, bạn không chỉ nhớ được lịch sử mà còn đoán được bước tiếp theo của bất kỳ dòng mô hình nào: hãy liệt kê những gì vẫn còn nằm ngoài.



Hình 14.1. Giải phẫu chung của một hệ thống thị giác hiện đại. Ba đầu bài toán khác nhau dùng chung backbone và neck; cái phân biệt chúng là *dạng đầu ra* và do đó là hàm mất mát. Khối hậu xử lý bên phải là phần duy nhất không nhận gradient, và toàn bộ chương này là câu chuyện thu nhỏ khối đó lại.

Hình 14.1 là bản đồ nên giữ trong đầu suốt chương: mỗi khi gặp một mô hình mới, hãy hỏi nó thay đổi khối nào trong bốn khối này.

Chương chia làm bốn mục theo đúng thứ tự phụ thuộc đó. Mục 1 đi hết dòng hai giai đoạn, nơi mẫu hình “nuốt heuristic” hiện ra rõ nhất và nơi các khái niệm nền — anchor, RoI, NMS — được định nghĩa cho cả chương. Mục 2 bỏ giai đoạn đề xuất để lấy tốc độ, và đúng lúc đó một vấn đề mới lộ ra mà mục 1 chưa hề gặp: mất cân bằng foreground–background, thứ được giải quyết ở tầng hàm mất mát chứ không ở tầng kiến trúc. Mục 2 cũng là nơi đặt bài học trung tâm của cả chương: cách gán mẫu cho các vị trí dự đoán — thành phần hiểm khi nằm trong abstract và thường nằm trong code — giải thích phần lớn chênh lệch giữa hai detector, nhiều hơn cả kiến trúc. Mục 3 chuyển sang segmentation. Ở đó cùng một mâu thuẫn “ngữ nghĩa sâu chống lại định vị chính xác” được tấn công từ bốn phía. Mục kết thúc bằng một lần đổi phát biểu, hợp nhất semantic, instance và panoptic. Lần đổi phát biểu đó mượn nguyên ý tưởng dự đoán tập hợp của mục 2, nên hai mục phải đọc theo thứ tự. Mục 4 gom các bài toán lân cận mà kỹ sư gặp hằng ngày: pose, OCR, retrieval, cùng hai chế độ phân bố dữ liệu mà mọi con số tổng đều che giấu.

Nếu bạn chỉ có thời gian cho một phần, hãy đọc mục 2. Nó chứa cái mà roadmap gọi là bài học lặp lại của cả lĩnh vực, và nó thay đổi cách bạn đọc mọi bảng kết quả detection về sau.

Bản đồ chương 14 — Bài toán CV lõi (Core Vision Tasks)

TÓM TẮT

Chương 14 gồm bốn mục, chia theo *dạng đầu ra* chứ không theo thứ tự lịch sử: hai mục đầu về đầu ra dạng tập khung (detection), mục ba về đầu ra dạng mask, mục bốn về những dạng đầu ra còn lại mà kỹ sư gặp hằng ngày. Đọc bản đồ này để biết mục nào là cốt lõi phải nắm chắc, mục nào tra khi cần, và thứ tự phụ thuộc giữa chúng. Nếu chỉ nhớ một điều: mục 2 là mục cốt lõi của cả chương, vì nó chứa bài học “cách gán mẫu quyết định nhiều hơn kiến trúc”.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mục (section) và chương** — chương là một chủ đề lớn; mỗi mục là một file riêng đọc độc lập được. Số thứ tự trong tên thư mục phản ánh *thứ tự nên đọc*, không phải thứ tự lịch sử.
- **cốt lõi và tham khảo** — “cốt lõi” là phần phải nắm chắc và quay lại nhiều lần; “tham khảo” là phần biết-là-có, đọc kỹ khi thật sự gặp bài toán đó.
- **detection và segmentation** — detection trả về danh sách khung chữ nhật kèm nhãn; segmentation trả về nhãn cho từng pixel. Khác biệt cốt lõi là *dạng đầu ra*, và nó kéo theo hàm mất mát lẫn thước đo.
- **gán mẫu (label assignment)** — quy tắc quyết định vị trí dự đoán nào phải học vật nào. Nó nằm trong định nghĩa của hàm mất mát chứ không phải trong kiến trúc — lý do nó là bài học trung tâm của chương.

Có gì trong chương này

- **Mục 1 — Detection hai giai đoạn (cốt lõi).** R-CNN → Fast R-CNN → Faster R-CNN với RPN → FPN → Mask R-CNN với RoIAlign. Đây là nơi định nghĩa các khái niệm nền dùng lại suốt chương: anchor, RoI, NMS, IoU, loss đa nhiệm.
- **Mục 2 — Một giai đoạn và cuộc đánh đổi tốc độ (cốt lõi, đọc kỹ nhất).** YOLO/SSD, mất cân bằng nền-vật thể, focal loss, anchor-free, và trực thật sự quan trọng: **gán mẫu (label assignment)** từ IoU cố định qua ATSS tới SimOTA, rồi DETR bỏ NMS bằng Hungarian matching.
- **Mục 3 — Phân vùng ảnh và sự hợp nhất (cốt lõi).** FCN, U-Net, DeepLab với atrous/ASPP, SegFormer, Mask2Former, rồi bước dịch sang mô hình nền tảng nhận prompt.
- **Mục 4 — Các bài toán lân cận (tham khảo có trọng điểm).** Pose, OCR, metric learning và truy hồi, fine-grained và long-tail. Đọc lướt để biết là có, quay lại khi thật sự gặp.

Vì sao chia như vậy

Ba câu hỏi kinh điển của thị giác — *ảnh này là gì, có gì ở đâu, mỗi pixel thuộc về cái gì* — trông như ba mức chi tiết của cùng một bài toán, nhưng chỉ câu đầu có đầu ra kích thước cố định. Chương chia theo đúng ranh giới đó: mục 1 và 2 xử lý đầu ra dạng tập hợp khung, mục 3 xử lý đầu ra dạng mask, mục 4 gom các dạng còn lại. Cách chia này quan trọng vì *dạng đầu ra* kéo theo hàm mất mát, metric và chế độ hồng — ba thứ khó đổi hơn kiến trúc rất nhiều.

Thứ tự đọc và phụ thuộc

1. Đọc [C/13-kien-truc-backbone](#) trước; toàn chương giả định bạn hiểu stride, trường tiếp nhận và feature map đa tỉ lệ.
2. Mục 1 trước mục 2: mất cân bằng nền–vật thể chỉ lộ ra *sau khi* bỏ giai đoạn đề xuất, nên đọc ngược thứ tự thì mất luôn nguyên nhân.
3. Mục 2 trước mục 3: khung truy vấn của Mask2Former mượn nguyên ý tưởng dự đoán tập hợp của DETR.
4. Mục 4 độc lập, đọc lúc nào cũng được.
5. Đọc [B/09-thiet-ke-ham-mat-mat](#) song song với mục 2 nếu phần focal loss chưa rõ, và [B/07-chia-du-lieu-va-danh-gia](#) song song với mục 4.

Nếu chỉ có một buổi

Đọc phần đặt vấn đề của mục 1, rồi đọc trọn mục 2. Mục 2 chứa bài học lặp lại của cả lĩnh vực — rằng những chi tiết không được viết trong abstract thường quyết định kết quả nhiều hơn kiến trúc — và bài học đó thay đổi cách bạn đọc mọi bảng kết quả về sau.

14.1 Detection hai giai đoạn nuốt dần heuristic (Two-Stage Detection)

PHỤ THUỘC [C/13-kien-truc-backbone/01-dong-cnn](#) (feature map, stride, trường tiếp nhận (*receptive field*)); [B/09-thiet-ke-ham-mat-mat](#) (loss đa nhiệm); [B/05-thiet-ke-bai-toan](#) (bộ ba đầu vào – đầu ra – tín hiệu giám sát).

TÓM TẮT

Mục này theo dấu bốn thể hệ detection hai giai đoạn: R-CNN, Fast R-CNN, Faster R-CNN với FPN, và Mask R-CNN. Chúng không phải bốn ý tưởng rời rạc, mà là bốn lần thay một bước thủ công bằng một khối học được. Đọc xong, cầm bất kỳ paper detection nào bạn cũng chỉ ra được hai thứ: khối nào nằm trong tầm với của gradient, và khối nào là heuristic cố định. Từ đó bạn đoán được bước cải tiến hợp lý tiếp theo. Đó là kỹ năng dự đoán, không phải ghi nhớ. Nếu chỉ nhớ một điều: ranh giới “cái gì nhận được gradient” dịch xuống qua từng thể hệ. Hai thứ sống sót được là triết phi cực đại và cách gán mẫu. Chúng sống sót vì lý do cấu trúc, không phải ngẫu nhiên, nên gỡ chúng đòi hỏi đổi phát biểu bài toán chứ không đổi kiến trúc.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model)** — một hàm có tham số, ánh xạ ảnh đầu vào thành dự đoán; “huấn luyện” là tìm bộ tham số làm dự đoán khớp nhãn nhất có thể.
- **gradient** — vector đạo hàm của hàm mất mát theo từng tham số; nó chỉ hướng phải chỉnh tham số để lỗi giảm. Một khối “không nhận được gradient” là khối không tự chỉnh được, dù nó gây ra lỗi.
- **backbone** — phần mạng dùng chung để biến ảnh thành *feature map* — một lưới vector đặc trưng, mỗi ô mô tả nội dung của một vùng ảnh. Phần gần phía trên để trả lời một bài toán cụ thể gọi là *head*.
- **stride** — tỉ lệ thu nhỏ giữa ảnh và feature map. Stride 16 nghĩa là một ô đặc trưng ứng với một vùng 16×16 pixel ảnh, nên mọi toạ độ tính trên feature map đều thô tới 16 pixel.
- **IoU** — diện tích giao chia diện tích hợp của hai khung chữ nhật; bằng 1 khi trùng khít, 0 khi rời nhau. Đây là thước đo “khung dự đoán có đúng chỗ không”.

- **anchor** — một khung chữ nhật mẫu, kích thước và tỉ số cạnh định sẵn, rải đều trên mọi vị trí của feature map. Mô hình không đoán khung từ số không mà đoán *độ lệch* so với anchor gần nhất — một bài toán dễ hơn nhiều.
- **proposal (đề xuất vùng)** — một vùng ảnh ứng viên “có thể chứa vật gì đó”, sinh ra ở giai đoạn một để giai đoạn hai chỉ phải phân loại vài trăm vùng thay vì hàng chục nghìn.
- **NMS** — bước hậu xử lý xoá các khung trùng lặp: giữ khung điểm cao nhất, xoá mọi khung có IoU với nó vượt ngưỡng, lặp lại. Nó là thuật toán viết tay, không học được.
- **mAP** — thước đo chuẩn của detection: với mỗi lớp, vẽ đường precision-recall rồi lấy diện tích dưới đường đó, sau đó trung bình trên mọi lớp và trên nhiều ngưỡng IoU.

Đặt vấn đề

Phân loại ảnh có một đặc tính dễ chịu mà ta ít khi để ý: đầu ra của nó có kích thước cố định. Một ảnh vào, một vector K chiều ra, và toàn bộ bộ máy huấn luyện bằng *gradient* chạy trơn tru vì **hàm mất mát** (*loss function*) được định nghĩa trên một đối tượng có hình dạng biết trước. **Phát hiện vật thể** (*object detection*) phá vỡ đúng đặc tính đó. Số vật thể trong ảnh thay đổi từ không tới hàng trăm, mỗi vật cần một nhãn và một vị trí, nên đầu ra là một **tập hợp** (*set*) có lực lượng thay đổi. Không có cách hiển nhiên nào viết một hàm mất mát khả vi trên tập hợp, và đây chính là nút thắt mà toàn bộ lịch sử detection xoay quanh.

Cách né đầu tiên, và trong nhiều năm là cách duy nhất, là biến bài toán tập hợp thành nhiều bài toán phân loại: đề xuất trước một tập ứng viên vùng ảnh, rồi phân loại từng ứng viên độc lập. Mẹo này giải quyết được vấn đề hình dạng đầu ra, nhưng nó đẩy độ khó sang chỗ khác — sang câu hỏi *ai sinh ra các ứng viên đó*. Trong R-CNN, câu trả lời là một thuật toán phân đoạn thủ công tên là *selective search*, thứ không có tham số nào học được và không bao giờ nhận được một chút *gradient* nào từ lỗi cuối cùng.

Học xong mục này bạn làm được gì mà trước đó không làm được? Bạn đọc một paper detection bất kỳ và chỉ ra ngay được ba thứ: khối nào nằm trong tầm với của gradient, khối nào là heuristic cố định, và vì thế bước cải tiến hợp lý tiếp theo là gì. Đó là một kỹ năng dự đoán, không phải kỹ năng ghi nhớ.

TRỰC GIÁC

Hãy hình dung hệ thống như một xưởng máy mà một số máy được nối vào bảng điều khiển trung tâm, còn một số thì quay tay. Máy nối vào bảng điều khiển tự chỉnh lại khi sản phẩm cuối bị lỗi; máy quay tay thì không, dù nó có gây ra lỗi hay không. “Nuốt một heuristic vào mạng” theo nghĩa cơ học chính xác là *nối thêm một máy vào bảng điều khiển*: biến một bước không khả vi, không tham số, thành một khối khả vi nhận được gradient từ hàm mất mát detection. Vì thế mỗi lần nuốt, hệ thống không chỉ nhanh hơn (bớt một công đoạn rồi) mà còn chính xác hơn (thêm một khối được tối ưu cho đúng mục tiêu cuối).

A. Bốn bước nuốt

I MẠCH 1 — detection hai giai đoạn

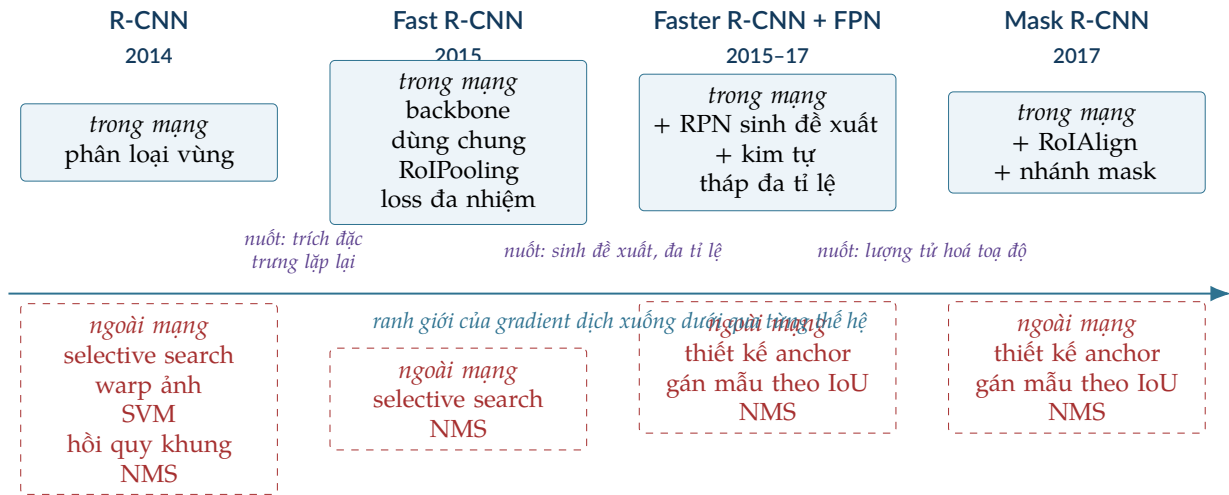
1 Vấn đề

không thể chấm điểm một tập hợp có lực lượng thay đổi.

Giải pháp

R-CNN cắt khoảng hai nghìn vùng ứng viên do *selective search* sinh ra. Mỗi vùng được warp về kích thước cố định rồi cho qua CNN để lấy đặc trưng. Một SVM phân loại, và một bộ hồi quy riêng tinh chỉnh khung [140].

- Giả định** các vùng đáng quan tâm đều nằm trong tập ứng viên; nếu selective search bỏ sót thì không gì cứu được.
- Giá phải trả** mỗi ảnh phải chạy CNN hai nghìn lần, huấn luyện chia làm ba pha rời rạc, và không pha nào tối ưu cho mục tiêu của pha kia.
- 2 Vấn đề** hai nghìn lượt chạy CNN cho cùng một ảnh là lãng phí gần như toàn bộ.
- Giải pháp** Fast R-CNN đảo thứ tự: chạy *backbone* **một lần** trên cả ảnh, rồi với mỗi vùng ứng viên chỉ cắt phần tương ứng trên feature map bằng *RoIPooling* và cho qua vài tầng **kết nối đầy đủ** (*fully connected*) [141]. Phân loại và hồi quy khung được gộp thành một **hàm mất mát đa nhiệm** (*multi-task loss*) duy nhất, huấn luyện một pha.
- ↔ **ràng buộc bởi B/09** — trọng số giữa hạng thức phân loại và hạng thức hồi quy là một siêu tham số ẩn, và chương thiết kế hàm mất mát giải thích vì sao nó không thể chọn tùy tiện.
- Giả định** đặc trưng dùng chung cho mọi vùng vẫn đủ tốt cho từng vùng — đúng khi các vùng chồng lấn nhiều, và chúng chồng lấn thật.
- Vấn đề mới** khi phần tính toán nặng đã bị loại bỏ, selective search trở thành nút cổ chai.
- 3 Vấn đề** bộ sinh đề xuất là công đoạn quay tay cuối cùng còn nặng.
- Giải pháp** Faster R-CNN thay nó bằng **mạng đề xuất vùng** (*region proposal network, RPN*), một mạng con trượt trên chính feature map đã có, tại mỗi vị trí dự đoán độ khách quan và độ lệch khung so với một tập *anchor* định sẵn [142]. Vì RPN dùng chung backbone, chi phí sinh đề xuất gần như bằng không.
- Giả định** tập anchor phủ đủ dày dãi tỉ lệ và tỉ số cạnh của vật thể trong dữ liệu.
- Giá phải trả** anchor trở thành một **siêu tham số** (*hyperparameter*) nhạy và *mang theo thống kê của tập dữ liệu gốc*; đổi miền ảnh thì phải thiết kế lại.
- 4 Vấn đề** một feature map duy nhất không phục vụ nổi mọi tỉ lệ. Feature ở tầng sâu có ngữ nghĩa mạnh nhưng độ phân giải thô, feature nông thì ngược lại; vật nhỏ vì thế bị thiệt kép.
- Giải pháp** **kim tự tháp đặc trưng** (*feature pyramid network, FPN*) dựng một kim tự tháp bằng nhánh *top-down* cộng với kết nối ngang, để mọi tầng đều vừa sâu về ngữ nghĩa vừa đúng độ phân giải [119].
- Hệ quả** đây là lần đầu “đa tỉ lệ” được đưa vào kiến trúc thay vì đưa vào pipeline dưới dạng chạy ảnh ở nhiều kích thước.
- ↔ **tiền đề C/13/01** — stride và trường tiếp nhận của backbone chính là thứ tạo ra mâu thuẫn ngữ nghĩa–độ phân giải mà FPN sinh ra để gỡ.
- 5 Vấn đề** RoIPooling làm tròn toạ độ, và việc làm tròn đó phá hỏng mọi bài toán cân độ chính xác dưới pixel.
- Giải pháp** Mask R-CNN thay bằng *RoIAlign*: không làm tròn, lấy mẫu song tuyến tại các điểm có toạ độ thực, rồi thêm một nhánh dự đoán **mặt nạ** (*mask*) nhị phân song song với nhánh phân loại [143].
- Hệ quả** nhánh mask gần như miễn phí về ý tưởng nhưng chỉ hoạt động tốt *sau khi* lỗi lượng tử hoá được gỡ bỏ — một minh hoạ sắc nét rằng chi tiết cài đặt có thể chặn đứng cả một bài toán mới.

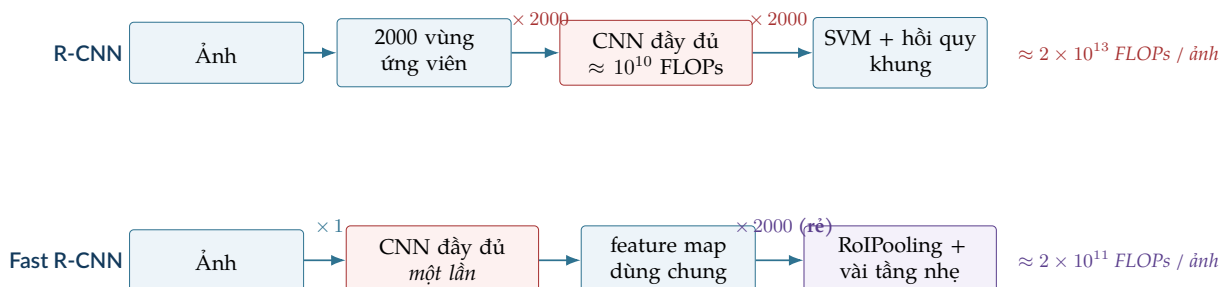


Hình 14.2. Dòng thời gian của họ hai giai đoạn, vẽ theo trục “cái gì nhận được gradient”. Khối trên đường kẻ nằm trong tầm với của hàm mất mát detection và tự chỉnh; khối dưới đường kẻ là heuristic cố định. Nhấn tím giữa hai cột ghi rõ mỗi bước nuốt được heuristic nào. Qua bốn thể hệ, phần dưới teo dần nhưng NMS và cách gán mẫu thì ở lại — đó là dư địa mà Mục 2 khai thác.

Hình 14.2 cho thấy điều mà một bảng thời gian không cho thấy: cái còn sót lại sau bốn bước không phải là ngẫu nhiên. **Triệt phi cực đại** (*non-maximum suppression, NMS*) sống sót vì nó thao tác trên *quan hệ giữa các dự đoán* chứ không trên một dự đoán đơn lẻ, mà loss của kiến trúc này lại chấm điểm từng dự đoán độc lập. Cách **gán mẫu** (*label assignment*) sống sót vì nó là một phần của *định nghĩa* loss chứ không phải một khối tính toán — không có gì để “nuốt” theo nghĩa thông thường. Hai chỗ đó đòi hỏi đổi phát biểu bài toán, và đó đúng là việc DETR làm ở mục sau.

B. Ba phép tính nhỏ giải thích ba bước nhảy

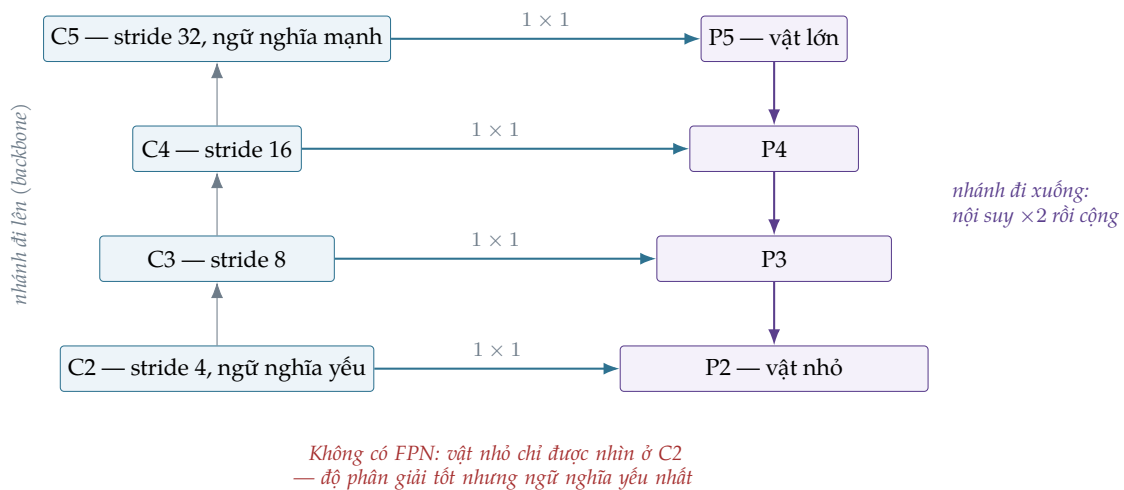
- **Vì sao chia sẻ feature map lại nhanh hơn hai bậc.** Giả sử backbone tốn khoảng 10^{10} FLOPs cho một ảnh 224×224 .
 - **R-CNN** chạy nó cho từng vùng trong $N = 2000$ vùng: 2×10^{13} FLOPs mỗi ảnh.
 - **Fast R-CNN** chạy backbone một lần trên ảnh lớn hơn (giả sử đắt gấp mười, 10^{11}), rồi mỗi vùng chỉ qua vài tầng nhẹ cỡ 10^8 , tổng 2×10^{11} .
 - **Tỉ số** khoảng $2 \times 10^{13} / 3 \times 10^{11} \approx 70$ lần, và thực tế còn lớn hơn vì R-CNN phải ghi/đọc đặc trưng ra đĩa. *Ý nghĩa:* chi phí chuyển từ $O(N)$ lượt backbone sang $O(1)$ lượt backbone cộng $O(N)$ lượt đầu nhẹ — đó mới là điều cần nhớ, không phải con số.



Hình 14.3. Cùng số vùng ứng viên, cùng backbone, khác nhau ở *thứ tự* phép cắt và phép trích đặc trưng. R-CNN đặt phép cắt trước nên khối đắt nhất bị nhân với N ; Fast R-CNN đặt phép cắt sau nên khối đắt chỉ chạy một lần và chỉ phần nhẹ mới bị nhân. Toàn bộ khoảng cách hai bậc độ lớn nằm ở chỗ đổi chỗ hai mũi tên, không ở chỗ nào khác.

Hình 14.3 là dạng nhìn thấy được của phép tính vừa rồi, và nó cũng là mẫu hình đáng nhớ: khi một pipeline lập một khối đất, hãy hỏi liệu có đầy được khối đó ra ngoài vòng lặp hay không.

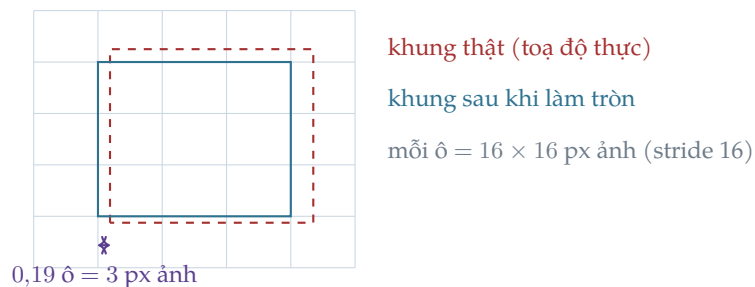
Vì sao làm tròn lại đầu đến thế. Backbone thông dụng có *stride* 16, nên một pixel feature ứng với ô 16×16 pixel ảnh. Khung có cạnh trái tại $x = 19$ rơi vào tọa độ feature $19/16 = 1,1875$; RoIPooling làm tròn xuống 1, tức dịch biên $0,1875 \times 16 = 3$ pixel ảnh gốc, rồi chia ô pooling làm tròn lần thứ hai. Ba pixel là nhiều với người cao 200 pixel, nhưng là 10% kích thước với một vật 30 pixel. Đó là lý do RoIAlign gần như không đổi gì ở ngưỡng **giao trên hợp** (*intersection over union, IoU*) lỏng nhưng cải thiện mạnh ở ngưỡng chặt và với nhánh mask [143].



Hình 14.4. FPN gộp hai thứ vốn nằm ở hai đầu đối lập của backbone. Đi lên, độ phân giải giảm còn ngữ nghĩa mạnh dần; đi xuống, ngữ nghĩa của tầng sâu được nội suy trả về từng mức và cộng vào đặc trưng nông qua kết nối ngang 1×1 . Kết quả là mọi mức P_i đều vừa đủ sâu vừa đủ mịn, nên vật nhỏ không còn bị thiệt kép.

- Hình 14.4 làm rõ vì sao đây là một bước “nuốt heuristic”: trước FPN, cách xử lý đa tỉ lệ là chạy *cả ảnh* ở nhiều kích thước rồi gộp kết quả — một vòng lặp bên ngoài mạng, tốn gấp nhiều lần. FPN đưa đúng chức năng đó vào trong một nhánh có tham số học được.

FPN gán vùng vào tầng nào. Công thức trong bài gốc là $k = \lfloor k_0 + \log_2(\sqrt{wh}/224) \rfloor$ với $k_0 = 4$. Nó nói gì: vật càng nhỏ thì càng được đẩy về tầng có độ phân giải cao. Khung 112×112 cho $\log_2(112/224) = -1$ nên $k = 3$; khung 448×448 cho $k = 5$. Đáng chú ý là công thức này vẫn là heuristic thủ công nằm ngoài gradient — lại một vòng nữa của cùng mẫu hình.



Hình 14.5. Vì sao RoIPooling làm hỏng bài toán mask. Trên lưới đặc trưng stride 16, việc làm tròn tọa độ khung về ô nguyên dịch biên đi tới vài pixel trên ảnh gốc; sai số này cố định, không giảm khi huấn luyện lâu hơn, và tỉ lệ nghịch với kích thước vật. RoIAlign lấy mẫu song tuyến tại tọa độ thực nên không sinh ra khoảng lệch màu tím.

Hình 14.5 là một lời nhắc quen thuộc với người làm hình học. Đây đúng là lỗi lượng tử hoá mà mọi hệ bám đặc trưng dưới pixel đều phải xử lý. Cách chữa cũng giống hệt: nội suy thay vì làm tròn, đúng như một bộ *tracker* KLT tinh chỉnh vị trí đặc trưng.

C. Giới hạn và trường hợp hỏng

Họ hai giai đoạn hỏng theo ba kiểu rất đặc trưng, và mỗi kiểu có một triệu chứng quan sát được:

- **Cảnh đông đúc.** Khi hai vật cùng lớp chồng lấn nhiều hơn ngưỡng NMS, một trong hai chắc chắn bị xoá. *Triệu chứng:* hạ ngưỡng thì mất vật, nâng ngưỡng thì sinh trùng lặp — không có giá trị nào cứu được cả hai, vì vấn đề nằm ở phát biểu chứ không ở tham số.
- **Miền anchor lệch.** Tập anchor mã hoá phân bố tỉ lệ và tỉ số cạnh của dữ liệu huấn luyện. Đưa sang ống kính góc rộng, ảnh *fisheye*, hay lớp vật dài mảnh thì recall của RPN sụt trước khi bộ phân loại kịp nói gì. *Triệu chứng:* recall giai đoạn đề xuất thấp trong khi độ chính xác trên các đề xuất còn lại vẫn cao.
- **Độ trễ không ổn định.** Số đề xuất qua ngưỡng thay đổi theo ảnh, nên thời gian giai đoạn hai dao động theo nội dung cảnh. *Triệu chứng:* phân vị 99 của độ trễ cao hơn nhiều so với trung vị — một tính chất rất khó chịu khi bạn phải cam kết ngân sách mili-giây cho mỗi khung hình, đúng kiểu ràng buộc cứng đã bàn ở [B/03-giai-phau-he-thong](#).

CẠM BẦY

Rất nhiều so sánh “kiến trúc A tốt hơn B 2 mAP” thực chất là so sánh hai công thức huấn luyện: số epoch, **tăng cường dữ liệu** (*data augmentation*), tỉ lệ mẫu dương/âm trong mỗi *batch*, ngưỡng IoU khi gán mẫu. Trước khi tin một cải tiến kiến trúc, hãy tìm xem phần gán mẫu và recipe có được giữ nguyên không — và trong đa số paper, thông tin đó nằm trong code chứ không nằm trong bài.

↔ **hệ quả của E/24** — muốn tách phần đóng góp của một khối khỏi phần đóng góp của recipe thì phải ablate đúng một thứ — đó là phương pháp, không phải trực giác.

Công cụ như [dbolya/tide](#) tách mAP thành các loại lỗi thành phần [66], [85], và thường phát hiện rằng hai mô hình “chênh 2 mAP” hỏng theo hai kiểu hoàn toàn khác nhau.

CẦN NHỚ

Mỗi bước tiến của detection hai giai đoạn là một heuristic bị kéo vào trong tầm với của gradient. Muốn đoán bước tiếp theo của một dòng mô hình bất kỳ, kể cả *SLAM*, hãy liệt kê những gì còn nằm *ngoài* vòng tối ưu. Rồi hỏi cái nào trong số đó viết lại được thành một khối khả vi. [T3]

D. Kết nối

Mục tiếp theo ([C/14-bai-toan-cv-loi/02-detection-mot-giai-doan](#)) là mặt đối lập: bỏ hẳn giai đoạn đề xuất để đổi lấy tốc độ, và trả giá bằng một vấn đề mất cân bằng mà mục này chưa gặp. [C/13-kien-truc-backbone/01-dong-cnn](#) là tiền đề: RoIPooling, stride và trường tiếp nhận chỉ hiểu được sau khi nắm cơ học của *convolution*. [B/09-thiet-ke-ham-mat-mat](#) giải thích vì sao gộp phân loại và hồi quy vào một loss đa nhiệm không hiển nhiên đúng. [E/24-thi-nghiem-va-ablation](#) là nơi kiểm chứng cạm bẫy vừa nêu.

❏ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao Fast R-CNN nhanh hơn R-CNN nhiều bậc trong khi cả hai dùng cùng backbone và cùng số vùng ứng viên?

2. RoIAlign khác RoIPooling ở đúng một chỗ. Vì sao chỗ đó ảnh hưởng tới nhánh mask mạnh hơn nhiều so với nhánh phân loại?
3. Miền ảnh cụ thể nào làm tập anchor mặc định của COCO sụt recall, và triệu chứng quan sát được ở đó là gì?
4. Vì sao NMS và cách gán mẫu là hai thứ duy nhất sống sót qua cả bốn thế hệ? Hai lý do đó khác nhau ở điểm nào?
5. Nếu bạn phải cam kết một ngân sách mili-giây cố định cho mỗi khung hình, kiến trúc hai giai đoạn gây khó khăn gì mà một giai đoạn không gây?

14.2 Một giai đoạn và cuộc đánh đổi tốc độ (One-Stage Detection and the Speed Trade-off)

PHỤ THUỘC C/14-bai-toan-cv-loi/01-detection-hai-giai-doan (anchor, NMS, RoI); B/09-thiet-ke-ham-mat-mat (focal loss, mất cân bằng lớp); C/13/02-tu-chuoi-toi-attention (attention, cần cho phần DETR).

TÓM TẮT

Mục này trả lời ba câu hỏi. Vì sao bỏ giai đoạn đề xuất vùng lại làm lộ ra mất cân bằng nền–vật thể? Vì sao vấn đề đó được chữa ở tầng hàm mất mát chứ không ở tầng kiến trúc? Và vì sao chuỗi ATSS → SimOTA → Hungarian matching mới là trực tiến hoá thật của detection hiện đại? Đọc xong, trong bất kỳ bảng so sánh detector nào bạn cũng tách được phần chênh lệch *do kiến trúc* khỏi phần chênh lệch *do gán mẫu và công thức huấn luyện*. Bạn cũng biết tìm thông tin đó ở đâu khi paper không nói. Nếu chỉ nhớ một điều: cách gán mẫu cho 10^5 vị trí dự đoán là một phần của *định nghĩa bài toán tối ưu*. Hai detector gán mẫu khác nhau vì thế là hai bài toán khác nhau. So sánh chúng rồi quy kết cho kiến trúc là sai ngay từ tiền đề.

KIẾN THỨC CẦN CÓ TRƯỚC

- **hàm mất mát (loss function)** — một số duy nhất đo mức sai của dự đoán so với nhãn; toàn bộ việc huấn luyện là chỉnh tham số để số đó nhỏ đi. Đổi hàm mất mát là đổi *mục tiêu*, không phải đổi mô hình.
- **cross-entropy** — hàm mất mát chuẩn cho phân loại: $-\log p_t$ với p_t là xác suất mô hình gán cho lớp đúng. Đoán đúng chắc chắn thì mất mát gần 0; đoán sai chắc chắn thì mất mát rất lớn.
- **mẫu dương / mẫu âm** — với mỗi vị trí dự đoán, “dương” nghĩa là vị trí đó được coi là chứa vật thể và phải học nhãn của vật đó; “âm” nghĩa là nền. Ai được coi là dương là do bước gán mẫu quyết định.
- **gán mẫu (label assignment)** — quy tắc quyết định mỗi trong hàng trăm nghìn vị trí dự đoán phải học điều gì. Nó không phải một khối tính toán mà là một phần của *định nghĩa* hàm mất mát — đây là ý trung tâm của cả mục.
- **anchor và anchor-free** — có anchor là dự đoán độ lệch so với các khung mẫu định sẵn; anchor-free là dự đoán trực tiếp từ mỗi ô lưới, chẳng hạn bốn khoảng cách tới bốn cạnh của khung.
- **NMS** — hậu xử lý xoá khung trùng: giữ khung điểm cao nhất rồi xoá mọi khung chồng lấn với nó quá ngưỡng IoU. Vì nó xét quan hệ *giữa* các dự đoán nên không thể huấn luyện cùng phần còn lại.
- **truy vấn (query) trong DETR** — một vector học được đóng vai “một chỗ trống để điền một vật thể”. Mô hình có N truy vấn cố định, mỗi truy vấn tự đi hỏi feature map rồi trả về một khung cộng một nhãn.
- **epoch** — một lượt đi hết toàn bộ tập huấn luyện. “Hội tụ sau 500 epoch” nghĩa là phải quét dữ liệu 500 lần mới đạt kết quả tốt — rất đắt.

Đặt vấn đề

Giai đoạn đề xuất vùng trong Faster R-CNN giải quyết được một vấn đề rất cụ thể: nó *lọc* hàng chục nghìn vị trí ứng viên xuống còn vài trăm, và mọi thứ phía sau chỉ phải làm việc trên một tập nhỏ, đã tương đối cân bằng giữa mẫu dương và mẫu âm. Cái giá là một vòng lặp nữa qua bộ nhớ và một lượng tính toán tỉ lệ với số đề xuất. Với robot phải chạy 30 khung hình mỗi giây trên một máy nhúng, cái giá đó thường không trả nổi. Câu hỏi tự nhiên là: bỏ hẳn giai đoạn đề xuất, cho mạng dự đoán thẳng trên lưới đặc trưng, thì mất gì?

Học xong mục này bạn làm được gì mà trước đó không làm được? Đọc phần thực nghiệm của một paper detection, bạn biết ngay cần kiểm tra ba dòng nào trong code trước khi tin bằng kết quả.

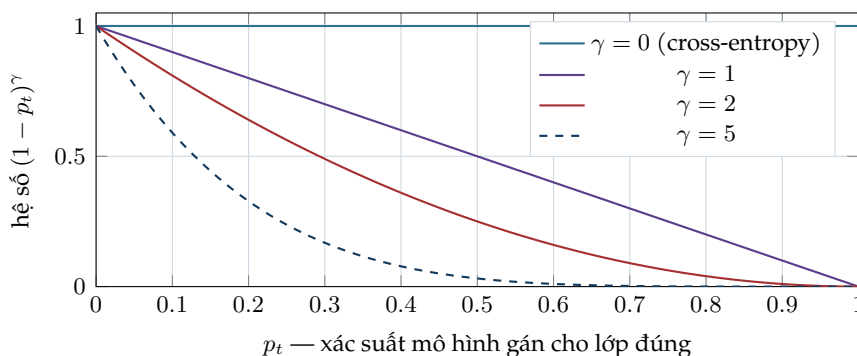
YOLO và SSD trả lời bằng cách làm đúng thể [144], [145]. Kết quả là tốc độ tăng vọt, nhưng một vấn đề mới xuất hiện mà kiến trúc hai giai đoạn chưa từng phải đối mặt, bởi vì bộ lọc đề xuất đã âm thầm che nó đi: **mất cân bằng nền–vật thể** (*foreground–background imbalance*). Bài học lớn nhất của mục này nằm ở chỗ vấn đề đó được giải quyết ở tầng **hàm mất mát** (*loss function*), không phải ở tầng kiến trúc. Câu chuyện sau đó còn cho thấy một điều nữa: phần lớn khoảng cách giữa các detector hiện đại nằm ở những chi tiết *không được viết trong abstract*.

A. Mất cân bằng, và vì sao nó là vấn đề của loss (Class Imbalance)

Hãy đếm. Một mạng một giai đoạn với **kim tự tháp đặc trưng** (*feature pyramid network, FPN*) năm tầng, ảnh 800×1333 , chín *anchor* mỗi vị trí, sinh ra cỡ 10^5 dự đoán mỗi ảnh. Một ảnh COCO trung bình có bảy vật thể; kể cả khi mỗi vật khớp vài anchor thì số mẫu dương vẫn chỉ vài chục. Tỉ lệ âm trên dương do đó vào khoảng 10^3 tới 10^4 .

Con số đó chưa đủ để thấy vấn đề. Cái đáng sợ nằm ở tổng gradient. Giả sử sau vài epoch, mạng đã học được rằng gần như mọi thứ là nền. Mỗi anchor nền khi đó có xác suất đúng $p_t = 0,99$, đóng góp **entropy chéo** (*cross-entropy*) $-\log 0,99 \approx 0,01$. Nghe như không đáng kể — nhưng 10^5 anchor nền cho tổng $10^5 \times 0,01 = 1000$. Trong khi đó ba mươi anchor dương khó, mỗi cái $p_t = 0,5$ và loss 0,69, chỉ cho tổng 21. Tín hiệu học thật bị nhấn chìm dưới tỉ lệ 50:1 bởi những mẫu mà mạng đã làm đúng rồi. Đây là ví dụ nhỏ nhất giải thích vì sao “hạ learning rate” hay “train lâu hơn” không cứu được: bản thân hàm mất mát đang chỉ sai hướng.

Cách chữa cổ điển là **hard negative mining** — SSD giữ tỉ lệ âm/dương 3:1 bằng cách chỉ lấy các mẫu âm có loss cao nhất. Cách này hoạt động nhưng đưa thêm một heuristic rời rạc, không khả vi vào giữa vòng huấn luyện. **Focal loss** thay nó bằng một hệ số điều biến trơn: $FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log p_t$ [20]. Với $\gamma = 2$, mẫu dễ $p_t = 0,99$ bị nhân với $(0,01)^2 = 10^{-4}$. Mẫu khó $p_t = 0,5$ chỉ bị nhân 0,25. Lập lại phép đếm ở trên: tổng đóng góp của nền tụt từ 1000 xuống 0,1, trong khi mẫu dương khó chỉ giảm bốn lần. Tỉ lệ đảo ngược hoàn toàn.



Hình 14.6. Hệ số điều biến của focal loss. Điều quan trọng không phải mẫu khó được đề cao, mà là mẫu *đã đúng* bị hạ xuống gần như bằng không — với $\gamma = 2$, một anchor nền chắc chắn đóng góp ít hơn bốn bậc so với cross-entropy. Vì γ chỉ nén chứ không đảo thứ tự, nó không thể sửa được một tập nhãn sai.

Hình 14.6 cũng cho thấy giới hạn. Hệ số luôn dương và đơn điệu. Nên khi một mẫu bị gán nhãn sai, mô hình dự đoán *đúng theo thực tế nhưng sai theo nhãn*, focal loss coi nó là mẫu khó và *tăng* trọng số cho nó. Trên dữ liệu nhiều nhãn, đây là chế độ hổng thật, không phải lý thuyết.

↪ **trường hợp riêng của B/09** — focal loss là một ca cụ thể của nguyên tắc tổng quát “sửa mất cân bằng ở tầng hàm mất mát thay vì ở tầng lấy mẫu dữ liệu”.

Kết luận rút ra từ RetinaNet vượt xa bản thân hàm loss. Suốt vài năm, khoảng cách giữa một giai đoạn và hai giai đoạn được giải thích bằng “kiến trúc hai giai đoạn tinh vi hơn”. Hoá ra phần lớn khoảng cách đó là hệ quả của việc giai đoạn đề xuất đã ngầm cân bằng lại tập mẫu. Sửa ở tầng loss là đủ.

B. Bỏ anchor, và câu hỏi thật sự là gán mẫu (Label Assignment)

TRỰC GIÁC

Kiến trúc là thí sinh; cách gán mẫu là *đáp án*. Một detector một giai đoạn có 10^5 ô trên lưới, và gán mẫu chính là việc điền vào đáp án cho từng ô: ô này phải nói “có xe ở đây”, ô kia phải nói “không”. Đối cách gán mẫu không phải là đối thí sinh, mà là đối đáp án — tức đối hần bài toán tối ưu. Vì thế so hai detector dùng hai cách gán mẫu khác nhau rồi kết luận về kiến trúc cũng vô nghĩa như so điểm hai thí sinh làm hai đề khác nhau.

I MẠCH 2 — một giai đoạn, từ anchor tới tập hợp

1 Vấn đề

Giải pháp

Giá phải trả

anchor là một siêu tham số thủ công mang theo thông kê của tập dữ liệu.

họ *anchor-free* dự đoán trực tiếp từ mỗi vị trí trên lưới: FCOS dự đoán bốn khoảng cách từ điểm đó tới bốn cạnh của khung [45], CenterNet dự đoán một heatmap tâm vật cộng kích thước [146].

không còn tập anchor, nhưng bây giờ phải trả lời một câu hỏi mới và khó hơn: *vị trí nào trên lưới chịu trách nhiệm cho vật nào*. Bài toán không biến mất, nó đổi tên thành gán mẫu.

2 Vấn đề

Giải pháp

Hệ quả

gán mẫu theo ngưỡng IoU cố định là một lựa chọn tùy tiện. Ngưỡng 0,5 khiến vật nhỏ chỉ nhận được một hai mẫu dương trong khi vật lớn nhận hàng chục, tức mô hình được huấn luyện với mức giám sát rất khác nhau tùy kích thước.

ATSS — viết tắt của *adaptive training sample selection* — chọn ngưỡng thích nghi cho từng vật: lấy k ứng viên gần tâm nhất ở mỗi tầng, tính trung bình và độ lệch chuẩn IoU của chúng, dùng $\mu + \sigma$ làm ngưỡng [46].

bài báo cho thấy sau khi thống nhất cách gán mẫu, khác biệt giữa detector có anchor và không anchor gần như biến mất — một trong những kết quả ablation quan trọng nhất của cả lĩnh vực.

3 Vấn đề

Giải pháp

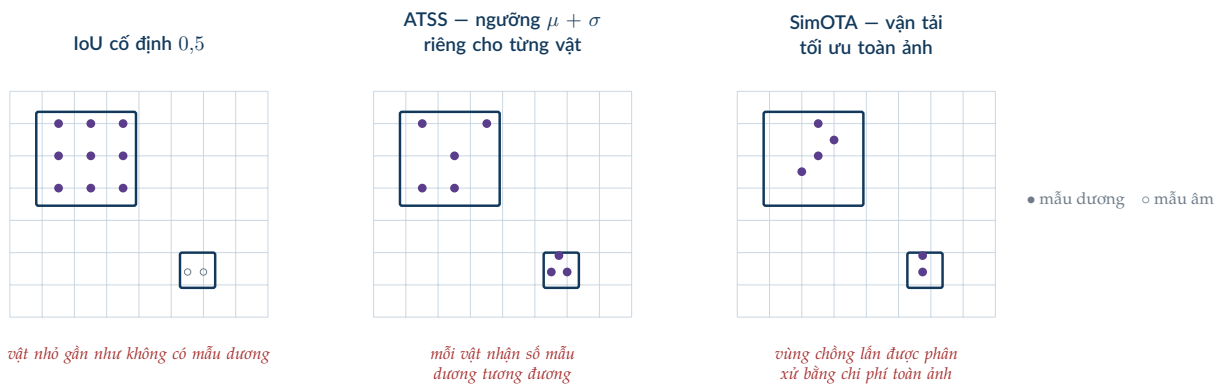
Giá phải trả

gán mẫu từng vật độc lập là tối ưu cục bộ. Khi hai vật chồng lấn, một vị trí lưới có thể tốt cho cả hai, và quyết định tham lam cho từng vật riêng lẻ dẫn tới xung đột.

optimal transport assignment (OTA) và bản rút gọn SimOTA phát biểu việc gán mẫu thành một bài toán vận tải tối ưu: mỗi ground truth là một nguồn cung k nhãn dương, mỗi vị trí lưới là một điểm cầu, chi phí là tổ hợp loss phân loại và loss hồi quy [147].

thêm chi phí tính toán trong lúc huấn luyện, không thêm gì lúc suy luận.

- 4 Vấn đề** NMS là heuristic cuối cùng, và nó là heuristic duy nhất thao tác trên quan hệ giữa các dự đoán. Soft-NMS làm mềm bằng cách giảm điểm thay vì xóa hẳn [148], nhưng vẫn là hậu xử lý.
- Giải pháp** DETR đổi phát biểu: dự đoán một tập hợp cố định N truy vấn, ghép chúng với **chân trị** (*ground truth*) bằng **ghép cặp Hungary** (*Hungarian matching*) một-một, và loss chỉ được định nghĩa sau khi ghép [47]. Đây là ý tưởng **dự đoán tập hợp** (*set prediction*) mà Mục 3 sẽ mượn nguyên vẹn sang segmentation. Vì mỗi vật chỉ có đúng một dự đoán được thưởng, mô hình học cách tự triệt tiêu trùng lặp; NMS biến mất khỏi pipeline.
- Giá phải trả** hội tụ cực chậm — bản gốc cần khoảng 500 epoch trong khi Faster R-CNN quen dùng lịch 12–36 epoch — và kém trên vật nhỏ.
- 5 Vấn đề** vì sao DETR chậm hội tụ? Hai nguyên nhân được các bản sửa chỉ ra: **attention** dày đặc trên toàn feature map khởi đầu gần như đồng đều nên mất rất lâu để tập trung, và phép ghép một-một dao động giữa các epoch nên tín hiệu giám sát cho mỗi truy vấn không ổn định.
- Giải pháp** Deformable DETR cho mỗi truy vấn chỉ lấy mẫu một số ít điểm quanh một điểm tham chiếu học được, giảm cả chi phí lẫn thời gian hội tụ xuống cỡ 50 epoch [114], [149]; DINO thêm truy vấn khứ nhiễu (cho mạng các khung ground truth đã bị nhiễu và bất khôi phục) để ổn định hoá tín hiệu [150].
- Hệ quả** tới 2026, dòng thực dụng như RT-DETR và RF-DETR đã đạt được cả tốc độ lẫn độ chính xác mà không cần NMS.



Hình 14.7. Cùng một ảnh, cùng một mạng, cùng một tập vị trí dự đoán — ba quy tắc gán mẫu cho ba tập mẫu dương khác nhau, và do đó ba hàm mất mát khác nhau. Khung đậm là hai vật thật, một lớn một nhỏ. Với ngưỡng IoU cố định, vật lớn nuốt gần hết ngân sách giám sát còn vật nhỏ hầu như không được dạy; ATSS cân lại bằng ngưỡng tính riêng cho từng vật; SimOTA thêm một bước phân xử toàn ảnh nên vị trí tranh chấp không bị gán cho cả hai. Vị trí các chấm ở đây là minh hoạ nguyên lý, không phải kết quả chạy thật.

Hình 14.7 cho thấy thứ mà một bảng số không cho thấy: cái thay đổi giữa ba cột không phải mô hình mà là *đề bài* đưa cho mô hình.

Bảng 14.1. Bốn cách trả lời câu hỏi “vị trí nào chịu trách nhiệm cho vật nào”. Đây là thành phần hiểm khi nằm trong abstract nhưng thường giải thích phần lớn chênh lệch giữa hai detector.

Cách gán	Cơ chế	Giả định	Hỏng khi nào
IoU cố định	Anchor nào có IoU vượt ngưỡng thì là dương	Ngưỡng tốt cho mọi kích thước và tỉ lệ	Vật nhỏ nhận rất ít mẫu dương; đối phân bố kích thước là phải chỉnh lại tay
Centerness / FCOS	Điểm trong vùng trung tâm của khung là dương, có trọng số theo độ lệch tâm	Tâm hình học là điểm giàu thông tin nhất	Vật hình khuyên, hình chữ L, hoặc bị che ở giữa — tâm rơi vào nền
ATSS	Ngưỡng $\mu + \sigma$ tính riêng cho từng vật từ k ứng viên gần nhất	Phân bố IoU của ứng viên là đơn đỉnh	Cảnh rất đông: các ứng viên “gần nhất” thuộc về vật khác
OTA / SimOTA	Vận tải tối ưu toàn ảnh trên chi phí phân loại + hồi quy	Chi phí ghép phản ánh đúng chất lượng dự đoán	Nhạy với chi phí khởi đầu khi mô hình còn kém; tốn thêm thời gian huấn luyện

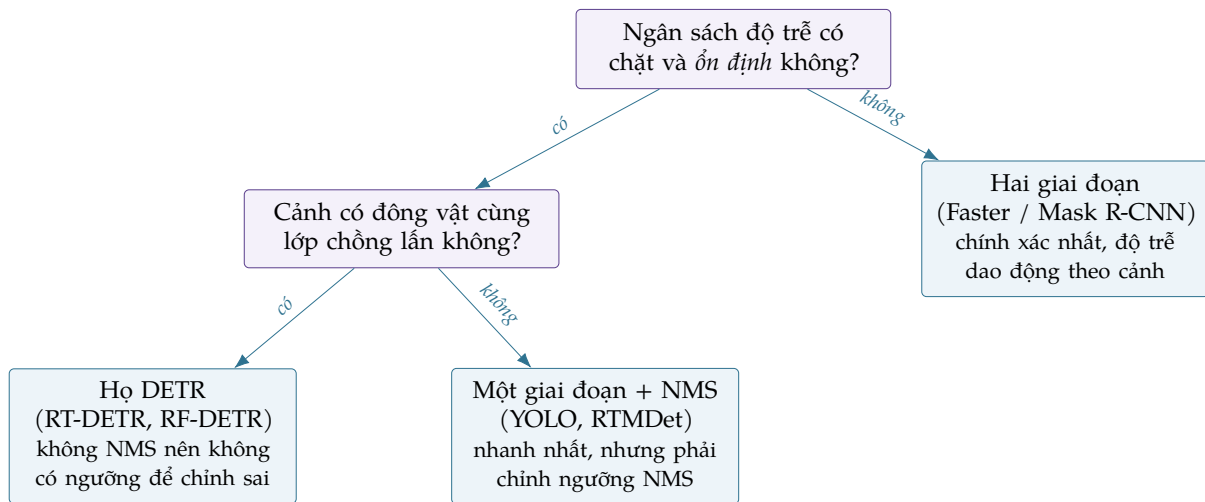
Bảng 14.1 nên đọc cùng với một quan sát khó chịu: bốn dòng này thay đổi *hàm mất mát*, không thay đổi mô hình. Hai detector khác nhau ở dòng gán mẫu là hai bài toán tối ưu khác nhau, nên so sánh chúng rồi quy kết chênh lệch cho “kiến trúc” là một suy luận sai ngay từ tiền đề.

↔ cùng nguyên lý với C/14/03 — Mask2Former mượn nguyên phép ghép cặp một–một này sang miền mask, và nhờ đó hợp nhất được semantic, instance và panoptic.

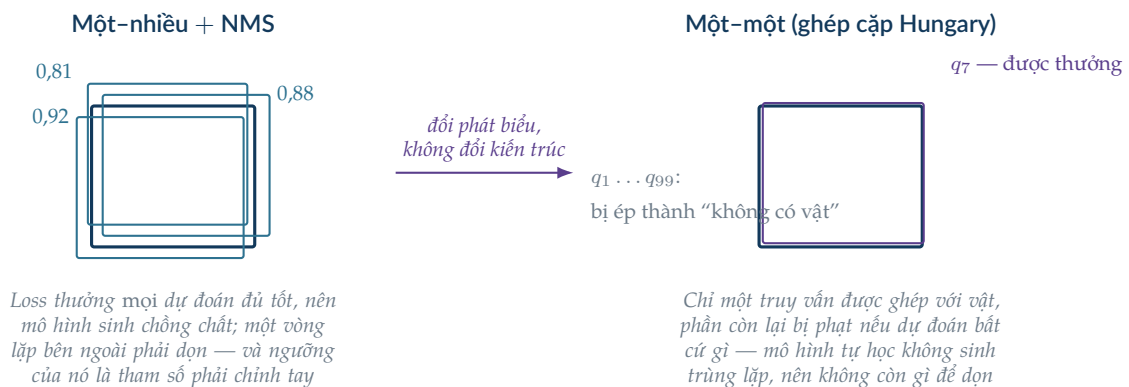
Hungarian matching của DETR, nhìn qua lăng kính này, chỉ là dòng thứ năm — một cách gán mẫu một–một thay vì một–nhiều — và chính điều đó, chứ không phải Transformer, mới là thứ khiến NMS trở nên thừa.

CẠM BÃY

Điểm số mà NMS dùng để xếp hạng là điểm *phân loại*, nhưng thứ ta cần giữ lại là khung *định vị* tốt nhất. Hai đại lượng này tương quan yếu hơn ta tưởng, nên một khung tự tin về nhãn nhưng lệch vị trí vẫn có thể xóa mất một khung đúng hơn. Các bản vá là dự đoán thêm một nhánh ước lượng IoU hoặc centerness rồi nhân vào điểm; nhưng nếu bạn thấy mAP ở ngưỡng IoU chặt (AP_{75}) tụt nhiều hơn ở ngưỡng lỏng (AP_{50}), hãy nghi ngờ đúng chỗ này trước khi đổi backbone.



Hình 14.8. Cây quyết định thực dụng: ràng buộc loại bỏ phương án trước khi độ chính xác kịp lên tiếng. Hai nhánh trái đều là một giai đoạn, nhưng khác nhau ở chỗ NMS còn hay không — và trong cảnh đông đúc, đó là khác biệt quyết định chứ không phải vài điểm mAP.



Hình 14.9. Vì sao NMS biến mất khi đổi cách ghép. Bên trái, hàm mất mát chấm điểm từng dự đoán độc lập nên nhiều dự đoán cùng đúng và cùng được thưởng; việc chọn một trong số đó phải làm ở hậu xử lý, bằng một heuristic có ngưỡng. Bên phải, phép ghép một-một biến “sinh trùng lặp” thành hành vi bị phạt ngay trong hàm mất mát, nên bước dọn không còn lý do tồn tại. Các số tin cậy chỉ để minh họa.

Hình 14.9 cũng giải thích cái giá: vì mỗi vật chỉ có đúng một truy vấn được thưởng, tín hiệu giám sát cho mỗi truy vấn thưa hơn hẳn, và phép ghép còn dao động giữa các epoch — hai lý do khiến DETR bản gốc hội tụ chậm.

↪ **ràng buộc bởi B/03** — cây quyết định này là hiện thân của nguyên tắc đọc hệ thống: ràng buộc cứng loại phương án trước, độ chính xác chỉ lên tiếng sau.

Hình 14.8 tóm lại mục này dưới dạng dùng được ngay: câu hỏi đầu tiên không phải “mô hình nào mAP cao nhất” mà “ràng buộc nào của tôi là cứng” — đúng cách đọc hệ thống ở B/03-giai-phau-he-thong.

C. Giới hạn và trường hợp hỏng

Detector một giai đoạn hỏng ở ba chỗ đáng nhớ, mỗi chỗ có một triệu chứng quan sát được:

- **Nhãn nhiễu gặp focal loss.** Focal loss giả định nhãn sạch; trên dữ liệu gán bằng đám đông hoặc bán tự động, nó khuếch đại đúng những mẫu bị gán sai và có thể tệ hơn cross-entropy

thường. *Triệu chứng*: loss huấn luyện đứng yên trong khi một nhóm rất nhỏ mẫu chiếm gần hết *gradient*.

- **Gán nhãn tinh vi ở giai đoạn đầu.** SimOTA và họ hàng dựa vào chi phí ghép, mà chi phí đó vô nghĩa khi mô hình còn kém, nên chúng phải kèm một giai đoạn khởi động. *Triệu chứng*: phân kỳ hoặc hội tụ về nghiệm suy biến trong vài epoch đầu — và chi tiết khắc phục nằm trong code chứ không trong bài.
- **Ngân sách truy vấn cố định của DETR.** Ảnh chứa nhiều vật hơn N thì phần dư bị mất im lặng, không có cảnh báo nào. *Triệu chứng*: recall giảm đều theo mật độ vật thể; không lộ ra trên COCO nhưng lộ ra ngay khi bạn đếm người trong đám đông.

Với người làm hệ thống thời gian thực, còn một giới hạn nữa ít được nói: bỏ NMS không có nghĩa là bỏ hậu xử lý không xác định. Một giai đoạn có thời gian chạy gần như cố định theo nội dung cảnh, đó là ưu điểm lớn nhất của nó so với hai giai đoạn; nhưng nếu bạn thêm lại NMS trên đầu ra (như hầu hết bản triển khai YOLO làm), thời gian NMS lại tỉ lệ với số khung sống sót, và phương sai độ trễ quay về.

CẦN NHỚ

Trước khi tin bất kỳ so sánh detector nào, hãy hỏi hai câu: hai bên dùng cách gán mẫu nào, và hai bên chạy bao nhiêu epoch với augmentation gì. Rất nhiều “cải tiến kiến trúc” công bố thực chất là cải tiến gán mẫu, và nó bị chôn trong code. [T3]

D. Kết nối

[C/14-bai-toan-cv-loi/01-detection-hai-giai-doan](#) là tiền đề trực tiếp: mất cân bằng chỉ lộ ra khi bỏ giai đoạn đề xuất, nên mục này chỉ có nghĩa sau mục kia. [B/09-thiet-ke-ham-mat-mat](#) là nơi focal loss được đặt cạnh các cách xử lý mất cân bằng khác, và là nơi trả lời câu hỏi tổng quát “khi nào nên sửa ở loss thay vì sửa ở dữ liệu”. Ý tưởng dự đoán tập hợp bằng Hungarian matching quay lại nguyên vẹn ở [C/14-bai-toan-cv-loi/03-segmentation](#) dưới dạng khung truy vấn của Mask2Former — đó là trường hợp hiếm hoi một ý tưởng chuyển miền mà không mất gì. [E/24-thi-nghiem-va-ablation](#) cung cấp phương pháp để tự kiểm chứng khẳng định trung tâm của mục này.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao mất cân bằng foreground–background hầu như không xuất hiện trong Faster R-CNN nhưng lại nghiêm trọng trong SSD, dù cả hai dùng cùng anchor?
2. Hệ số điều biến với $\gamma = 2$ bằng bao nhiêu tại $p_t = 0,9$ và tại $p_t = 0,3$? Hai con số đó giải thích thế nào việc focal loss nguy hiểm trên dữ liệu nhiều nhãn?
3. ATSS cho thấy điều gì về câu hỏi “anchor hay anchor-free”? Kết quả đó thay đổi cách bạn đọc một paper detection như thế nào?
4. DETR bỏ được NMS nhờ Hungarian matching chứ không nhờ Transformer. Hãy giải thích tại sao, và điều gì sẽ xảy ra nếu ta ghép một–nhiều thay vì một–một?
5. Bạn đo được AP_{50} cao nhưng AP_{75} thấp — hai giả thuyết nào giải thích được, và thí nghiệm nào phân biệt chúng?
6. Trên một robot có ngân sách 20 ms mỗi khung hình, vì sao phương sai độ trễ có thể quan trọng hơn độ trễ trung bình?

14.3 Phân vùng ảnh và sự hợp nhất (Segmentation and Unification)

PHỤ THUỘC C/13-kien-truc-backbone/01-dong-cnn (stride, trường tiếp nhận (*receptive field*), upsampling); C/14-bai-toan-cv-loi/02-detection-mot-giai-doan (dự đoán tập hợp bằng Hungarian matching — cần cho phần Mask2Former).

TÓM TẮT

Mục này theo một mâu thuẫn duy nhất: mạng phân loại giỏi *nhờ* vứt bỏ thông tin không gian, còn phân vùng lại cần đúng thông tin vừa bị vứt. Bốn kiến trúc lớn — FCN, U-Net, DeepLab, SegFormer — là bốn cách khác nhau để lấy lại thông tin đó, mỗi cách trả một loại giá khác nhau; Mask2Former thì không lấy lại mà đối hân phát biểu, và nhờ vậy hợp nhất được semantic, instance và panoptic. Đọc xong bạn chọn được kiến trúc phân vùng từ ràng buộc (bộ nhớ, lượng nhân, yêu cầu về biên) thay vì từ bảng xếp hạng, và đọc được mIoU với đúng mức hoài nghi. Nếu chỉ nhớ một điều: mọi kiến trúc segmentation đều là một câu trả lời cho “chi tiết không gian đã mất ở đâu, lấy lại bằng cách nào” — và mIoU cao không hề bảo đảm biên đúng hay vật nhỏ không bị bỏ sót.

KIẾN THỨC CẦN CÓ TRƯỚC

- **semantic segmentation** — gán cho *mỗi pixel* một nhãn lớp. Hai chiếc xe cạnh nhau đều là “xe” và không được tách ra.
- **instance segmentation** — ngoài nhãn lớp còn tách từng cá thể: xe thứ nhất và xe thứ hai là hai mask khác nhau.
- **panoptic segmentation** — gộp cả hai: các lớp đếm được thì tách cá thể, các lớp không đếm được (bầu trời, mặt đường) thì chỉ cần nhãn lớp.
- **mask** — một lưới nhị phân cùng kích thước ảnh, đánh dấu pixel nào thuộc về vùng đang nói tới. Đây là “dạng đầu ra” của mọi bài toán trong mục này.
- **output stride** — tỉ lệ thu nhỏ giữa ảnh và feature map. Output stride 32 nghĩa là ảnh 1024×1024 còn lại lưới 32×32 , nên mỗi ô đại diện cho 32×32 pixel và biên chỉ định vị được tới độ thô đó.
- **encoder-decoder** — encoder là phần đi xuống, thu nhỏ dần độ phân giải để lấy ngữ nghĩa; decoder là phần đi lên, khôi phục độ phân giải. *Skip connection* là đường nối tắt từ một tầng encoder sang tầng decoder cùng cỡ.
- **trường tiếp nhận (receptive field)** — vùng ảnh gốc mà một ô đặc trưng “nhìn thấy”. Muốn hiểu ngữ cảnh rộng thì cần trường tiếp nhận lớn, mà cách rẻ nhất để mở rộng nó lại là thu nhỏ độ phân giải — đó chính là mâu thuẫn của mục này.
- **mIoU** — thước đo chuẩn của segmentation: với mỗi lớp lấy diện tích giao chia diện tích hợp giữa vùng dự đoán và vùng thật, rồi trung bình trên các lớp.

Đặt vấn đề

Detection trả lời “có cái gì, ở khoảng nào”. Nhiều ứng dụng cần chặt hơn: một khung chữ nhật quanh một con đường không nói được đâu là mặt đường có thể đi. Segmentation đẩy độ phân giải của câu trả lời xuống tận pixel, và ngay lập tức va vào một mâu thuẫn mà cả một thập kỷ nghiên cứu xoay quanh: mạng phân loại tốt *nhờ* vứt bỏ thông tin không gian — pooling và stride là cơ chế tạo ra bất biến tịnh tiến — trong khi segmentation lại cần đúng thông tin không gian vừa bị vứt bỏ. Ta muốn ngữ nghĩa sâu và định vị chính xác cùng lúc, còn kiến trúc phân loại được thiết kế để đánh đổi cái sau lấy cái trước.

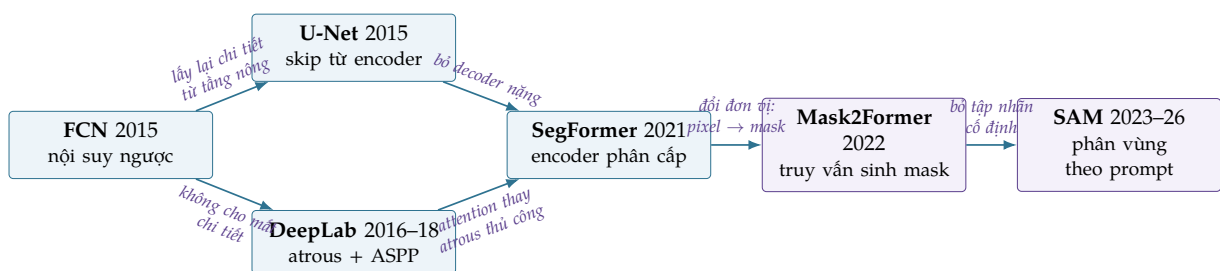
Một phép tính nhỏ cho thấy mâu thuẫn này cụ thể tới mức nào. Backbone phân loại điển hình có **output stride** 32: ảnh 1024×1024 ra feature map 32×32 . Mỗi ô đặc trưng đại diện cho một mảng 32×32 pixel. Dù bạn nội suy lên lại bằng phép gì, thông tin để phân biệt hai pixel kề nhau bên trong cùng một ô đã không còn tồn tại; sai số định vị biên có sàn cỡ chục pixel. Toàn bộ mạch dưới đây là các cách khác nhau để phá cái sàn đó, và mỗi cách trả một loại giá khác nhau.

Học xong mục này bạn làm được gì mà trước đó không làm được? Cho một yêu cầu cụ thể — “bám mép đường tới 2 pixel”, “chạy trên Jetson với 4 GB”, “chỉ có 200 ảnh gán nhãn” — bạn suy ra được kiến trúc nào khả thi và vì sao, trước khi chạy thí nghiệm nào.

TRỰC GIÁC

Semantic segmentation cổ điển hỏi mỗi pixel: “bạn thuộc lớp nào?” — một trăm nghìn câu hỏi độc lập, và mô hình không có chỗ nào để nói “ba nghìn pixel này cùng thuộc *một* vật”. Mask2Former đảo chiều điểm danh: thay vì hỏi từng pixel, nó đưa ra một trăm cái tên (truy vấn) và bắt mỗi cái tên tự nhận “tôi là vùng nào, và tôi là cái gì”. Đảo chiều câu hỏi là lý do duy nhất khiến semantic, instance và panoptic gộp được vào một mô hình: một vật thể trở thành một đơn vị được nói tới, thay vì một tính chất phát sinh từ hậu xử lý.

A. Bốn cách phá sản độ phân giải, rồi một lần đổi phát biểu (From FCN to Mask2Former)



Hình 14.10. Dòng thời gian phân vùng ảnh, đọc theo nút thất được gỡ ở mỗi bước. Hai nhánh giữa (U-Net và DeepLab) tấn công *cùng một* vấn đề mất chi tiết từ hai phía đối lập nên có thể kết hợp; hai bước cuối không gỡ nút thất kỹ thuật mà đổi phát biểu bài toán — đơn vị được nói tới chuyển từ pixel sang mask, rồi tập nhãn cố định bị bỏ hẳn.

Hình 14.10 đáng nhìn trước khi đọc mạch bên dưới, vì nó cho thấy ngay một điều mà đọc tuần tự khó thấy: bốn bước đầu là kỹ thuật, hai bước cuối là đổi phát biểu.

Mạch 3 — segmentation và sự hợp nhất

1 Vấn đề

mạng phân loại không sinh được đầu ra theo pixel.

Giải pháp

FCN vứt các tầng fully connected, coi chúng như conv 1×1 , rồi dùng transposed conv để nâng độ phân giải trở lại kích thước ảnh [151].

Hệ quả

lần đầu tiên toàn bộ pipeline là một mạng duy nhất huấn luyện đầu-cuối.

Vấn đề mới

biên rất thô, vì chi tiết đã mất ở đường xuống chứ không phải ở đường lên — nội suy không thể tạo lại thứ đã bị bỏ.

2 Vấn đề

chi tiết mất ở đường xuống thì phải lấy lại từ đường xuống.

Giải pháp

U-Net nối thẳng feature ở mỗi mức của encoder sang mức tương ứng của decoder [120]. Feature nông giữ biên sắc, feature sâu giữ ngữ nghĩa, decoder hợp nhất hai loại.

Giả định

biên trong ảnh nông và biên ngữ nghĩa trùng nhau — đúng với ảnh y tế và ảnh vệ tinh, kém đúng khi vật có hoa văn mạnh bên trong.

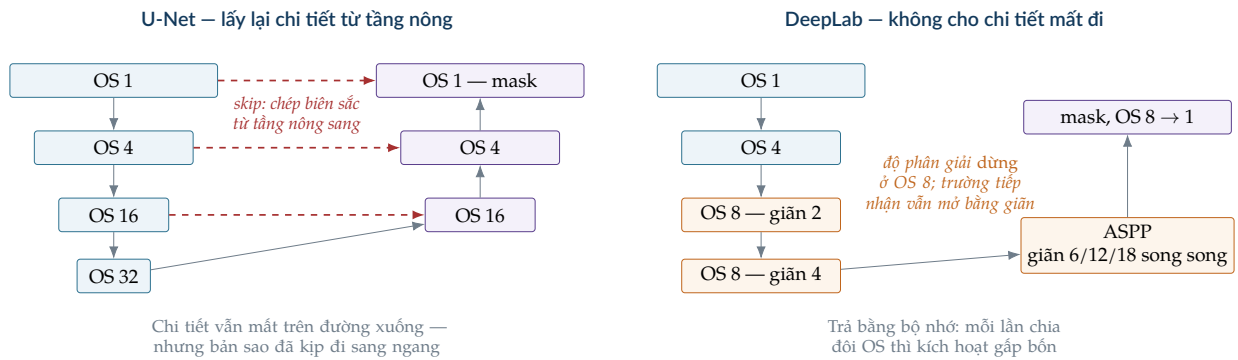
Hệ quả

đây vẫn là kiến trúc mặc định của ảnh y tế sau mười năm, và MIC-DKFZ/nnUNet cho thấy một U-Net cấu hình tự động vẫn rất khó bị đánh bại.

3 Vấn đề

một hướng tấn công khác: đừng để mất chi tiết ngay từ đầu.

- Giải pháp** **tích chập giãn** (*atrous / dilated convolution*) chèn khoảng trống giữa các trọng số, mở rộng trường tiếp nhận mà không giảm độ phân giải [106]; DeepLab dùng nó để giữ output stride ở mức 8 hoặc 16, và ghép nhiều tỉ lệ giãn song song thành *ASPP* [152].
- Giá phải trả** bộ nhớ và tính toán, vì kích hoạt trung gian to gấp bốn lần mỗi khi giảm output stride một nửa ở cả hai chiều.
- Vấn đề mới** hiệu ứng lưới (gridding) khi các tỉ lệ giãn có ước chung, làm mất mẫu ở một số vị trí.
- 4 Vấn đề** decoder nặng và thiết kế đa tỉ lệ thủ công.
- Giải pháp** SegFormer dùng encoder Transformer phân cấp sinh sẵn đặc trưng ở bốn tỉ lệ, rồi ghép chúng bằng một decoder chỉ gồm vài tầng MLP [153].
- Hệ quả** attention lo phần ngữ cảnh xa mà trước đây phải mua bằng atrous, nên decoder được phép rẻ.
- Giá định** đủ dữ liệu để Transformer bù lại phần inductive bias đã bỏ.
 ↪ **ràng buộc bởi** C/13/03 — bỏ bớt thiên lệch quy nạp thì phải mua lại bằng dữ liệu — đó là lý do SegFormer đòi dữ liệu hơn U-Net chứ không phải vì nó “phức tạp hơn”.
- 5 Vấn đề** semantic, instance và panoptic là ba bài toán với ba code base, ba loss, ba metric.
- Giải pháp** Mask2Former phát biểu cả ba như **phân loại mặt nạ** (*mask classification*): một tập cố định **truy vấn** (*query*), mỗi truy vấn sinh ra một mask nhị phân cộng một nhãn lớp, ghép với ground truth bằng Hungarian matching — đúng ý tưởng dự đoán tập hợp của DETR, chuyển sang miền mask [67], [154].
 ↪ **tiền đề** C/14/02 — khung truy vấn ở đây chỉ là dự đoán tập hợp của DETR chuyển sang miền mask; chưa hiểu vì sao ghép cặp một-một bỏ được NMS thì không hiểu được vì sao ba bài toán hợp nhất được.
- Đóng góp kỹ thuật then chốt là **attention có mặt nạ** (*masked attention*): mỗi truy vấn chỉ chú ý vào vùng mà nó đang dự đoán, khiến hội tụ nhanh hơn nhiều so với attention toàn cục.
- Hệ quả** ba bài toán chỉ khác nhau ở cách hậu xử lý tập mask, không khác nhau ở mô hình. Khi ba bài toán hợp nhất được, thường là dấu hiệu ta đã tìm ra phát biểu đúng.
- 6 Vấn đề** tập nhãn cố định là ràng buộc cuối cùng. Mọi mô hình ở trên chỉ biết những lớp có trong tập huấn luyện.
- Giải pháp** mô hình nền tảng nhận prompt — SAM phân vùng theo điểm, khung hay vùng do người chỉ, không cần biết vật đó tên gì [62]; các hệ open-vocabulary như Grounding DINO nhận mô tả văn bản; tới 2026, SAM 3 nhận prompt ở mức khái niệm và chạy được cả trên video.
- Giá phải trả** mô hình lớn, khó kiểm soát hành vi ở biên, và điều quan trọng với kỹ sư: đầu ra không kèm một khái niệm “lớp” ổn định để hệ thống phía sau dựa vào.

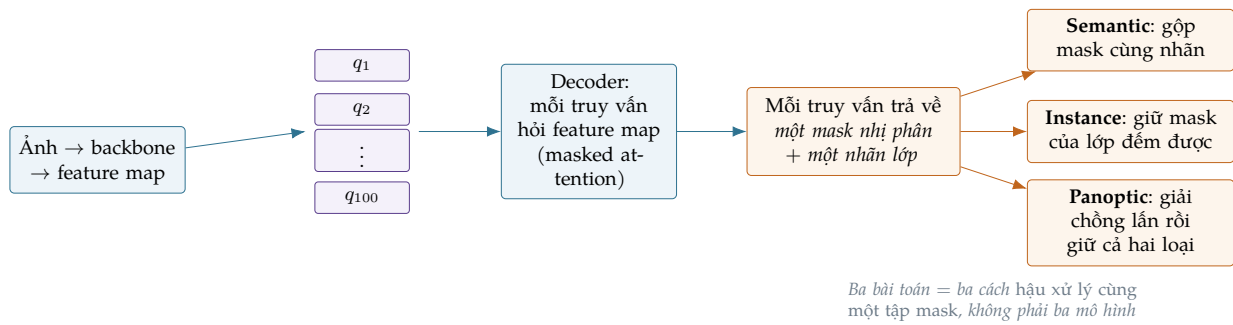


Hình 14.11. Hai lời giải đối lập cho cùng một vấn đề. U-Net chấp nhận để chi tiết mất trên đường xuống rồi chép nó sang ngang bằng skip connection; DeepLab từ chối để mất, giữ output stride (OS) ở mức 8 và mua trường tiếp nhận bằng tích chập giãn thay vì bằng thu nhỏ. Vì hai cơ chế độc lập nhau nên chúng cộng được: nhiều kiến trúc thực dụng dùng cả hai.

Hình 14.11 là lý do nên đọc mạch trên theo trục “chi tiết mất ở đâu” chứ không theo trục thời gian: hai bước liền nhau trong lịch sử lại là hai hướng tấn công khác nhau, không phải cái sau thay thế cái trước.

Bảng 14.2. Năm cách lấy lại độ phân giải, cùng cái giá và chế độ hồng riêng. Ba dòng đầu tấn công cùng một vấn đề từ ba phía khác nhau, nên chúng có thể kết hợp; dòng cuối đối phát biểu chứ không đối cơ chế.

Cách	Cơ chế	Mạnh khi nào	Hồng khi nào
FCN	Nội suy ngược từ feature sâu	Vùng lớn, biên không quan trọng	Biên thô; vật mảnh (dây, cột) biến mất hẳn
U-Net	Skip connection encoder → decoder	Ít dữ liệu, biên rõ, ảnh y tế	Vật có kết cấu nội bộ mạnh: skip mang cả nhiều lên
Atrous / ASPP	Giữ output stride nhỏ, mở trường tiếp nhận bằng giãn	Cần ngữ cảnh xa mà vẫn cần biên	Bộ nhớ tăng mạnh; gridding nếu tỉ lệ giãn không nguyên tố cùng nhau
SegFormer	Encoder phân cấp + decoder MLP	Dữ liệu nhiều, cần ngữ cảnh toàn ảnh	Dữ liệu ít; chi phí attention ở độ phân giải cao
Mask2Former	Truy vấn sinh mask + Hungarian matching	Cần instance/panoptic trong một mô hình	Số truy vấn cố định; ảnh nhiều instance hơn số truy vấn thì mất im lặng



Hình 14.12. Khung truy vấn của Mask2Former. Mô hình không hỏi từng pixel “bạn thuộc lớp nào” mà đưa ra một tập cố định truy vấn, mỗi truy vấn tự nhận một vùng và một nhãn; huấn luyện ghép các truy vấn với chân trị bằng ghép cặp Hungary như DETR. Vì đơn vị được nói tới là *một mask kèm một nhãn*, cả ba bài toán phân vùng chỉ còn khác nhau ở bước đọc tập mask đó, nên chúng dùng chung một mô hình và một hàm mất mát.

Hình 14.12 làm rõ điều mà hộp trực giác ở đầu mục chỉ nói bằng lời: cái được hợp nhất không phải ba kiến trúc mà là ba *cách phát biểu đầu ra*.

B. Một phép tính về cái giá của độ phân giải (The Resolution Budget)

Bảng 14.2 nói “bộ nhớ tăng mạnh” — con số cụ thể đáng ghi nhớ. Với ảnh 1024×1024 và một tầng đặc trưng 256 kênh: ở output stride 16, feature map là $64 \times 64 \times 256 \approx 1,05$ triệu phần tử, tức 4,2 MB ở fp32. Hạ xuống output stride 8, kích thước không gian nhân đôi ở cả hai chiều nên số phần tử nhân bốn: 16,8 MB cho *một* tensor. **Lượt ngược** (*backward pass*) phải giữ kích hoạt của mọi tầng, nên hệ số bốn này nhân vào toàn bộ phần sâu của mạng. Đây là lý do rất thực tế khiến hệ thống triển khai hiếm khi chạy ở output stride 4. Nó cũng giải thích vì sao U-Net rẻ hơn nhiều so với cảm giác ban đầu: U-Net chỉ giữ đặc trưng phân giải cao ở các tầng *ngang*, nơi số kênh còn ít.

Ba tầng cùng giãn 2



một nửa số pixel không bao giờ được nhìn — hiệu ứng lưới

Ba tầng giãn 1, 2, 3



mọi pixel đều đóng góp, cùng trường tiếp nhận

(một chiều, kernel 3×1 , tâm ở giữa; ô tô đậm là pixel thực sự ảnh hưởng tới đầu ra ở tâm)

Hình 14.13. Hiệu ứng lưới của tích chập giãn, vẽ trên một hàng pixel. Khi mọi tầng dùng cùng tỉ lệ giãn chẵn, các vị trí lấy mẫu luôn rơi vào cùng một lớp chẵn lẻ, nên trường tiếp nhận trông thì rộng mà thực chất thừa và bỏ sót một nửa tín hiệu. Chọn dãy tỉ lệ nguyên tố cùng nhau làm các lớp dư khác nhau phủ lẫn nhau, và trường tiếp nhận trở lại đặc.

Hình 14.13 là ví dụ nhỏ nhất cho thấy vì sao “mở trường tiếp nhận” và “nhìn thấy mọi pixel” là hai chuyện khác nhau.

Về atrous: một kernel 3×3 với tỉ lệ giãn r có trường tiếp nhận hiệu dụng $3 + 2(r - 1)$. Xếp chồng ba tầng giãn 1, 2, 4 cho trường tiếp nhận $3 \rightarrow 7 \rightarrow 15$ mà không hề giảm độ phân giải. Nhưng nếu ba tầng cùng giãn 2, các vị trí lấy mẫu luôn rơi vào cùng một lớp chẵn lẻ. Một nửa số pixel khi đó không bao giờ được nhìn tới. Đó là hiệu ứng gridding, và cách chữa là chọn dãy tỉ lệ giãn nguyên tố cùng nhau.

C. Giới hạn và trường hợp hỏng

Chế độ hỏng nguy hiểm nhất của segmentation không nằm ở mô hình mà ở thước đo. **giao trên hợp trung bình** (*mean intersection over union, mIoU*) trung bình theo lớp, nên một lớp hiếm nhưng quan trọng (người đi bộ, vật cản nhỏ) đóng góp bằng một lớp phủ kín ảnh (bầu trời), điều này

tốt; nhưng bên trong mỗi lớp, IoU trung bình theo *diện tích*, nên sai vài pixel ở biên của một vùng lớn gần như không bị phạt. Hệ quả là một mô hình có mIoU 0,85 vẫn có thể cho biên răng cưa tới mức không bám nổi mép đường. Nó cũng có thể bỏ sót hoàn toàn một vật cản 20×20 pixel mà chỉ mất một phần nghìn điểm. Nếu ứng dụng của bạn quan tâm tới biên hay tới vật nhỏ, mIoU là thước đo sai và phải bổ sung metric theo biên hoặc theo instance.

→ **hệ quả của B/07** — một con số tổng luôn che giấu chế độ khó; cách chữa duy nhất là báo cáo theo lát, ở đây là lát theo kích thước vật và lát theo độ dài biên.

Ba chế độ hỏng còn lại đáng nêu tên riêng:

- **Vật mảnh.** Dây điện, chân bàn, cột — biến mất ở output stride lớn, và không có hậu xử lý nào lấy lại được thứ đã không được lấy mẫu. *Dấu hiệu:* IoU của lớp đó gần 0 trong khi mIoU tổng vẫn cao.
- **Nhiều instance cùng lớp chồng lấn.** Các mô hình theo truy vấn chia sẻ một ngân sách cố định, nên phần vượt ngân sách mất im lặng — đúng chế độ hỏng của DETR ở Mục 2, xuất hiện lại ở miền mask.
- **Khái niệm không ổn định ở mô hình nhận prompt.** Cùng một ảnh, hai prompt gần như đồng nghĩa cho ra hai phân vùng ở hai mức chi tiết khác nhau (cái cửa, hay tay nắm cửa?). Hệ thống hạ nguồn giả định “một vật, một mask” sẽ hỏng theo cách rất khó tái hiện, vì lỗi phụ thuộc câu chữ chứ không phụ thuộc ảnh.

CẦN NHỚ

Đừng nhớ danh sách kiến trúc; hãy nhớ rằng mọi kiến trúc segmentation đều là một câu trả lời cho cùng một câu hỏi “chi tiết không gian đã mất ở đâu, và lấy lại bằng cách nào”. Skip lấy lại từ tầng nông, atrous không cho mất, attention học lại quan hệ xa, query đổi hẳn đơn vị được nói tới. [T3]

D. Kết nối

[C/14-bai-toan-cv-loi/02-detection-mot-giai-doan](#) là tiền đề của phần Mask2Former: nếu chưa hiểu vì sao Hungarian matching bỏ được NMS thì khung truy vấn ở đây chỉ là một kiến trúc lạ. [C/13-kien-truc-backbone/03-vit-va-inductive-bias](#) giải thích cái giá dữ liệu mà SegFormer phải trả. [C/18-pretraining-va-foundation](#) là nơi bàn kỹ về bước dịch chuyển sang mô hình nền tảng và kinh tế học của nó. [C/15-thi-giac-khong-thoi-gian/03-flow-tracking-va-rang-buoc](#) nối tiếp trực tiếp: một mô hình segmentation per-frame có mIoU cao vẫn cho video nhấp nháy, và lý do là mô hình này chưa hề có khái niệm thời gian.

❏ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao nội suy ngược từ feature ở output stride 32 không thể tạo lại biên sắc, dù ta dùng phép nội suy tốt đến đâu?
2. U-Net và DeepLab tấn công cùng một vấn đề từ hai phía nào, và trong tình huống nào thì cách của U-Net tốt hơn hẳn?
3. Ba tầng atrous cùng tỉ lệ giãn 2 gây ra chuyện gì, và vì sao chọn tỉ lệ nguyên tố cùng nhau lại chữa được?
4. Vì sao việc semantic, instance và panoptic hợp nhất được vào một mô hình lại là bằng chứng cho thấy phát biểu bài toán đúng, chứ không chỉ là tiện lợi kỹ thuật?
5. Mô hình của bạn đạt mIoU 0,85 nhưng bỏ sót vật cản nhỏ trên đường. Vì sao metric không phát hiện được, và bạn thay bằng metric nào?

14.4 Các bài toán lân cận, mỗi cái một đánh đổi riêng (Neighbouring Tasks)

PHỤ THUỘC C/14-bai-toan-cv-loi/01-detection-hai-giai-doan và 03-segmentation (RoIAlign, heatmap, độ phân giải); B/07-chia-du-lieu-va-danh-gia (slice, metric theo nhóm); B/08-dich-chuyen-phan-phoi (long-tail).

TÓM TẮT

Mục này gom bốn họ bài toán mà kỹ sư gặp hằng ngày nhưng không nằm gọn trong trục khung/pixel: ước lượng tư thế, OCR, học biểu diễn cho truy hồi, và hai chế độ phân bố dữ liệu (fine-grained, long-tail). Điểm chung của cả bốn là quyết định quan trọng nhất không phải chọn kiến trúc mà là *chọn dạng đầu ra*: điểm, heatmap, chuỗi, hay embedding. Dạng đầu ra kéo theo luôn cả loss, metric và chế độ hỏng. Đọc xong bạn phát biểu được một bài toán thì giác lạ dưới dạng đúng ngay từ đầu, thay vì phát hiện mình chọn sai sau khi đã huấn luyện xong. Nếu chỉ nhớ một điều: chọn sai dạng đầu ra thì không kiến trúc nào cứu được, và triệu chứng của nó luôn xuất hiện ở chỗ mô hình phải biểu diễn sự *mơ hồ* — khớp bị che, ký tự nhoè, hai người rất giống nhau.

KIẾN THỨC CẦN CÓ TRƯỚC

- **dạng đầu ra** — kiểu dữ liệu mà mô hình phải sinh ra — một nhãn, một khung, một mask, một điểm, một chuỗi ký tự, hay một vector. Nó quyết định luôn hàm mất mát và thước đo, nên đây là quyết định thiết kế đắt giá nhất.
- **hồi quy (regression) và phân loại (classification)** — hồi quy là dự đoán một số thực (tọa độ x); phân loại là chọn một trong các lựa chọn rời rạc (ô lưới nào chứa khớp). Hai cách này hỏng theo kiểu rất khác nhau khi câu trả lời mơ hồ.
- **keypoint và pose** — keypoint là một điểm có ý nghĩa trên vật (khủy tay, góc biến số); pose là tập hợp các keypoint của một đối tượng cùng quan hệ giữa chúng.
- **heatmap** — một bản đồ xác suất cùng cỡ với feature map, giá trị cao ở nơi mô hình tin rằng có keypoint. Khác với một cặp tọa độ, heatmap biểu diễn được “có hai khả năng”.
- **embedding** — một vector số chiều cố định biểu diễn một ảnh, học sao cho hai ảnh cùng danh tính thì gần nhau. Nhận dạng trở thành đo khoảng cách, nên thêm một người mới không cần huấn luyện lại.
- **tập lớp mở (open-set)** — tình huống mà các lớp lúc chạy thật không có trong tập huấn luyện — lý do phân loại softmax không dùng được và metric learning ra đời.
- **long-tail** — phân bố dữ liệu mà vài lớp chiếm gần hết số mẫu còn hàng trăm lớp chỉ có vài chục mẫu; accuracy tổng bị các lớp đông thống trị nên che kín phần đuôi.
- **lát (slice)** — một tập con dữ liệu chia theo một thuộc tính có ý nghĩa — theo lớp, theo kích thước vật, theo điều kiện chụp. Báo cáo metric theo lát là cách duy nhất thấy được các chế độ khó.

Đặt vấn đề

Ba mục trước đi theo hai trục quen thuộc: nhãn ở mức khung và nhãn ở mức pixel. Nhưng phần lớn hệ thống thị giác thật lại đứng ở chỗ khác. Cánh tay robot cần *tọa độ khớp* chứ không cần khung; hệ đọc biển số cần một *chuỗi ký tự* đi qua ba khối nối tiếp; hệ nhận dạng khuôn mặt cần trả lời “có phải cùng một người” cho những người chưa từng có trong tập huấn luyện, tức đầu ra không thể là một nhãn trong tập cố định. Mục này gom bốn họ bài toán như vậy.

↪ **tiền đề B/05** — chọn dạng đầu ra chính là bước thứ hai trong bộ ba đầu vào – đầu ra – tín hiệu giám sát; sai ở bước này thì mọi lựa chọn sau đó đều sai theo.

Chúng không nối thành một mạch lịch sử duy nhất, nhưng chúng chia sẻ một điểm chung đáng học: ở mỗi họ, quyết định quan trọng nhất là *chọn dạng đầu ra*, và chọn sai dạng đầu ra thì không kiến trúc nào cứu được.

Học xong mục này bạn làm được gì mà trước đó không làm được? Nhận một yêu cầu mơ hồ kiểu “đọc số công-tơ từ camera” hay “nhận ra người quen ở cửa”, bạn viết ngay được bộ ba đầu vào – đầu ra – tín hiệu giám sát cùng metric đúng. Kèm theo là danh sách chỗ hệ thống sẽ hỏng.

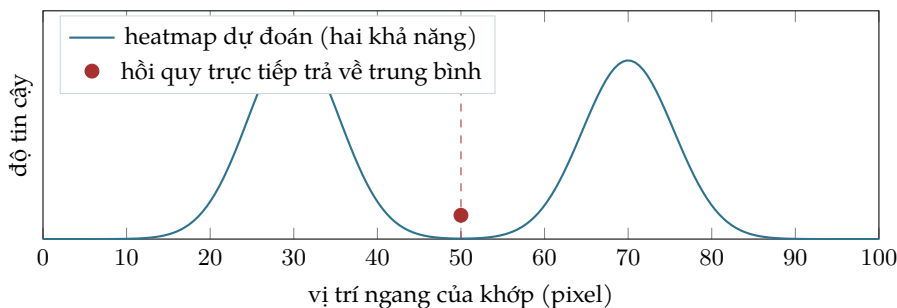
TRỰC GIÁC

Phân loại xây một cánh cửa cho mỗi lớp: muốn thêm lớp thì phải xây thêm cửa, tức huấn luyện lại. Metric learning không xây cửa mà vẽ *bản đồ*: mọi ảnh được đặt vào một không gian sao cho khoảng cách phản ánh “cùng danh tính hay không”. Thêm một người mới chỉ là ghim thêm một điểm mốc lên bản đồ đã có. Sức mạnh và điểm yếu đều nằm ở đó: bạn không phải xây lại gì cả, nhưng nếu tấm bản đồ vẽ sai thì mọi điểm mốc ghim lên nó đều sai theo, và không có cách nào sửa cục bộ.

A. Keypoint và pose: hai lần chọn dạng (Keypoint and Pose Estimation)

Ước lượng tư thế phải chọn hai lần. Lần thứ nhất là *dạng đầu ra của một khớp*, và chỉ có hai phương án.

- **Hồi quy trực tiếp hai số (x, y)** — hiển nhiên và rẻ nhất. Nhưng mạng phải nén một bản đồ tin cậy hai chiều thành một điểm, qua các tầng fully connected. Cách này không biểu diễn được sự mơ hồ: khi một khớp bị che, câu trả lời đúng là “có hai khả năng”, còn hồi quy trả về trung bình của hai khả năng — một điểm mà cả hai giả thuyết đều bác bỏ.
- **Dự đoán một bản đồ nhiệt (heatmap)** — một bản đồ xác suất cho mỗi khớp, lấy argmax làm vị trí [155]. Heatmap giữ được sự mơ hồ. Huấn luyện cũng ổn định hơn nhiều vì bài toán trở thành phân loại theo pixel. Cái giá là bộ nhớ, và một lỗi lượng tử hoá đi kèm.



Hình 14.14. Vì sao dạng đầu ra quyết định chế độ hỏng. Khi một khớp bị che, câu trả lời đúng là “một trong hai chỗ này”. Heatmap biểu diễn được điều đó bằng hai đỉnh, và tầng sau có thể chọn hoặc hoãn quyết định. Hồi quy trực tiếp bị hàm mất mát bình phương kéo về trung bình của hai khả năng — một điểm mà cả hai giả thuyết đều bác bỏ. Đây đúng là lý do một bộ ước lượng trạng thái không bao giờ thay một phân bố đa đỉnh bằng kỳ vọng của nó.

Hình 14.14 là ví dụ nhỏ nhất cho luận điểm trung tâm của mục này.

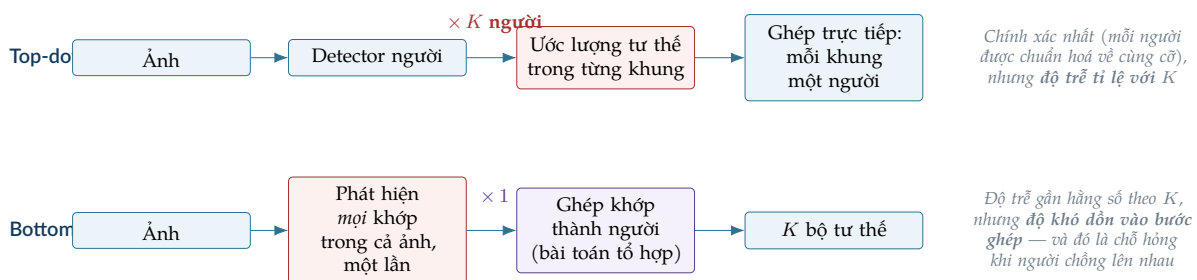
Lỗi lượng tử hoá đó tính được bằng tay. Với heatmap ở stride 4 trên ảnh 256×192 , lưới đầu ra là 64×48 ; argmax chỉ định vị được tới bội số của 4 pixel, nên sai số hệ thống là ± 2 pixel ngay cả khi mô hình hoàn hảo. Với người cao 200 pixel trong ảnh, 2 pixel là 1% — chấp nhận được; nhưng với bàn tay 40 pixel thì đó là 5%, và mọi bản triển khai nghiêm túc đều phải thêm một nhánh hồi quy độ lệch dưới pixel. Về bộ nhớ: 17 khớp $\times 64 \times 48 \times 4$ byte ≈ 209 KB cho một người — không đáng kể; nhưng ở độ phân giải 384×288 và 133 điểm (cả bàn tay và mặt) thì con số nhân lên hơn hai chục lần, và đó là lúc lựa chọn bắt đầu đau.

Lần chọn thứ hai là *cách xử lý nhiều người*. **top-down** chạy detector trước rồi ước lượng tư thế trong từng khung. Cách này chính xác nhất, vì mỗi người được chuẩn hoá về cùng kích thước.

Nhưng chi phí tỉ lệ tuyến tính với số người, nên độ trễ phụ thuộc nội dung cảnh — một tính chất tệ hại cho hệ thời gian thực. **bottom-up** phát hiện mọi khớp trong cả ảnh một lần rồi ghép chúng thành người, chẳng hạn bằng trường liên kết từng phần trong OpenPose [156]: thời gian gần như hằng số theo số người, nhưng bước ghép là một bài toán tổ hợp và nó chính là chỗ hỏng khi người chồng lên nhau.

→ cùng nguyên lý với C/15/03 — top-down và bottom-up lặp lại đúng dạng của tracking-by-detection và joint tracking: tách rời thì dễ gỡ lỗi nhưng chi phí tỉ lệ số đối tượng, gộp chung thì chi phí ổn định nhưng độ khó dồn vào một bước liên kết mờ ám.

Đây là cùng một cấu trúc đánh đổi mà bạn sẽ gặp lại ở tracking: tách rời thì dễ gỡ lỗi nhưng chi phí tỉ lệ với số đối tượng; gộp chung thì ổn định về chi phí nhưng đẩy độ khó vào một bước liên kết mờ ám.



Hình 14.15. Cùng một bài toán, hai chỗ đặt độ khó. Top-down mua độ chính xác bằng cách chạy lại khối nặng cho từng người, nên chi phí — và quan trọng hơn, *phương sai* của chi phí — phụ thuộc nội dung cảnh. Bottom-up cố định chi phí phát hiện nhưng phải giải một bài toán ghép tổ hợp mà không có lời giải chắc chắn. Đây đúng là cấu trúc đánh đổi sẽ trở lại ở tracking dưới tên tracking-by-detection và joint tracking.

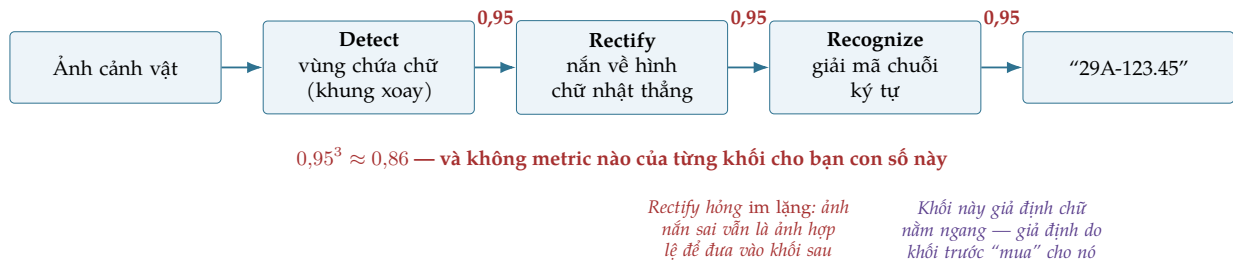
Hình 14.15 cho thấy vì sao lựa chọn này là lựa chọn hệ thống chứ không phải lựa chọn độ chính xác: với ngân sách mili-giây cố định, cột phải mới là cột quyết định.

B. OCR: một pipeline hoàn chỉnh trong một hệ thống nhỏ (Optical Character Recognition)

OCR đáng được mổ xẻ không phải vì nó khó nhất mà vì nó là hệ thống nhỏ nhất chứa đủ mọi thứ: phát hiện, biến đổi hình học, mô hình chuỗi, và một thước đo cấp hệ thống. Pipeline điển hình gồm ba khối nối tiếp:

1. **Detect** vùng chứa chữ. Thường là segmentation hoặc detection với khung xoay, vì chữ hiếm khi nằm ngang.
2. **Rectify** nắn vùng đó về một hình chữ nhật thẳng, bằng phép biến đổi phối cảnh hoặc một lưới biến dạng học được.
3. **Recognize** giải mã ảnh đã nắn thành chuỗi ký tự.

Điều đáng học nằm ở chỗ nối. Thứ nhất, khối rectify tồn tại vì khối recognize giả định chữ nằm ngang và đọc từ trái sang phải; đó là một giả định được đưa vào để làm khối cuối rẻ đi, đúng theo nguyên lý “độ khó không biến mất, chỉ bị đẩy đi chỗ khác”. Thứ hai, lỗi của ba khối nhân lên chứ không cộng lại: nếu mỗi khối đúng 95% thì hệ thống đúng $0,95^3 \approx 86\%$, và bạn không bao giờ đọc được con số đó từ metric của từng khối. Thứ ba, thước đo đúng của OCR là tỉ lệ khớp cả chuỗi hoặc khoảng cách chỉnh sửa, chứ không phải độ chính xác từng ký tự — một biến số sai một ký tự là một biến số sai. Bất kỳ ai từng dựng một pipeline thì giác nhiều tầng đều nhận ra hai bài học này, và OCR là nơi rẻ nhất để học chúng.

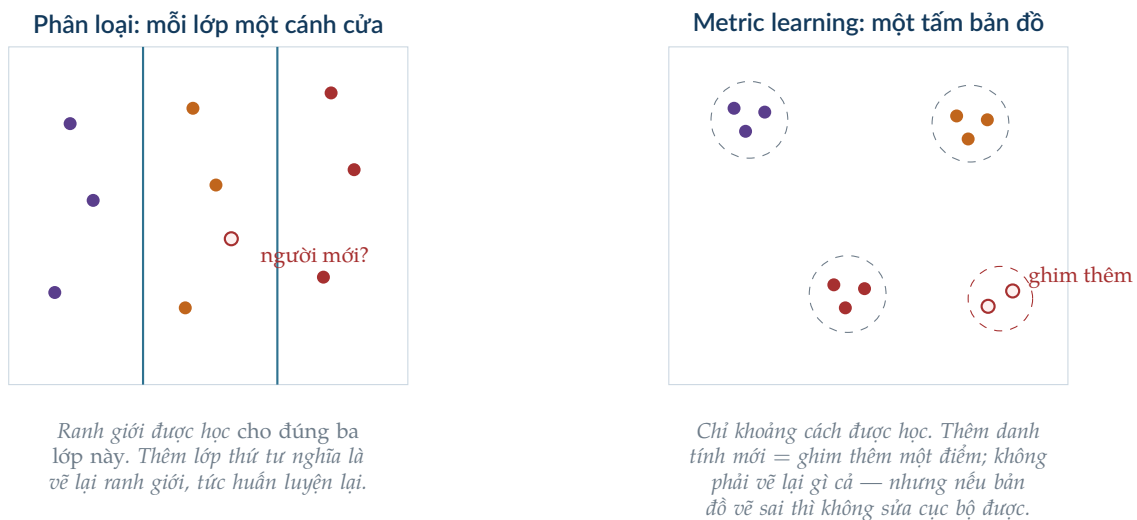


Hình 14.16. OCR là hệ thống nhỏ nhất chứa đủ mọi bệnh của một pipeline nhiều tầng. Độ chính xác của các khối *nhân* với nhau chứ không cộng, nên ba khối rất tốt vẫn cho một hệ thống tầm thường; khối giữa tồn tại chỉ để mua một giả định cho khối cuối; và thước đo đúng là khớp cả chuỗi chứ không phải độ chính xác từng ký tự — một biến số sai một ký tự là một biến số sai.

Hình 14.16 đáng nhớ vì nó áp dụng cho mọi pipeline nhiều tầng, kể cả pipeline SLAM: nếu bạn chỉ theo dõi metric của từng khối, bạn không bao giờ nhìn thấy con số mà người dùng thật sự trải nghiệm.

C. Học biểu diễn cho truy hồi (Metric Learning and Retrieval)

Nhận dạng khuôn mặt, re-identification và tìm kiếm ảnh có chung một ràng buộc mà phân loại không đáp ứng được: tập lớp *mở*. Không thể huấn luyện một softmax trên tập nhân viên của công ty rồi mỗi lần có người mới lại huấn luyện lại. Giải pháp là học một hàm *embedding* f sao cho khoảng cách trong không gian đích phản ánh quan hệ đồng nhất, rồi việc nhận dạng trở thành tìm láng giềng gần nhất. FaceNet huấn luyện bằng *triplet loss* — kéo cặp cùng danh tính lại gần hơn cặp khác danh tính ít nhất một biên α [100]; ArcFace thay bằng một biên góc cộng vào logit của softmax, vừa dễ huấn luyện hơn vừa cho ranh giới hình học rõ ràng trên mặt cầu đơn vị [157].



Hình 14.17. Vì sao tập lớp mở buộc phải đổi dạng đầu ra. Bên trái, mô hình học *ranh giới*, nên số lớp là một phần của kiến trúc và mọi lớp mới đòi một lần huấn luyện lại. Bên phải, mô hình chỉ học một phép đo khoảng cách, nên danh tính trở thành dữ liệu chứ không còn là tham số. Cái giá là chất lượng phụ thuộc hoàn toàn vào hình dạng của không gian nhúng — thứ không sửa được cho riêng một truy vấn.

Hình 14.17 giải thích luôn vì sao tầng tìm láng giềng xấp xỉ lại quan trọng đến thế: khi danh tính là dữ liệu, việc tra cứu trở thành một phần của hệ thống chứ không còn là một phép nhân ma trận.

Cái giá của cách tiếp cận này gồm ba phần. Thứ nhất, chất lượng phụ thuộc *hoàn toàn* vào không gian nhúng; không có cơ chế nào sửa được một truy vấn cụ thể mà không động vào toàn cục. Thứ hai, việc lấy mẫu quan trọng ngang hàm loss: phần lớn cặp ngẫu nhiên đã thoả biên nên không sinh gradient, và mọi phương pháp thực dụng đều phải khai thác mẫu khó — lại là một chi tiết nằm trong code. Thứ ba, ở quy mô lớn phải thêm một tầng *approximate nearest neighbour* (ANN), và tầng đó có sai số riêng: recall của ANN đặt trần lên recall của cả hệ thống, độc lập với chất lượng embedding.

CẠM BẮY

Một khảo sát có kiểm soát đã chỉ ra rằng phần lớn “tiền bộ” công bố trong metric learning biến mất khi so sánh công bằng [42]. Công bằng ở đây nghĩa là cùng backbone, cùng cách lấy mẫu, cùng quy trình chọn siêu tham số. Bài học không chỉ dành cho metric learning: khi một lĩnh vực có nhiều bậc tự do nằm ngoài phần được viết trong bài (cách lấy mẫu, augmentation, lịch huấn luyện), bảng kết quả đo lường công sức tinh chỉnh nhiều hơn đo lường ý tưởng.

D. Fine-grained và long-tail: hai chế độ mà accuracy tổng che giấu

Hai chế độ khó cuối cùng không phải là bài toán mới mà là hai kiểu phân bố dữ liệu khiến mọi con số tổng trở nên vô nghĩa. **phân loại chi tiết** (*fine-grained classification*) là khi khác biệt giữa các lớp nhỏ hơn khác biệt bên trong một lớp: hai loài chim khác nhau ở màu một mảng lông, trong khi cùng một loài trông rất khác giữa tư thế bay và tư thế đậu. Hệ quả kỹ thuật là mô hình cần độ phân giải cao ở đúng vùng phân biệt và cần bất biến mạnh với tư thế — hai yêu cầu kéo ngược nhau. **phân bố đuôi dài** (*long-tail distribution*) là khi vài lớp chiếm phần lớn dữ liệu còn hàng trăm lớp chỉ có vài chục mẫu; accuracy tổng bị các lớp đầu thống trị nên có thể rất cao trong khi mô hình gần như không bao giờ dự đoán các lớp đuôi.

→ cùng nguyên lý với B/08 — long-tail ở đây và dịch chuyển phân phối ở chương 8 có chung một cơ chế: phân bố lúc đánh giá không giống phân bố mà hệ thống thật sự gặp.

Điểm chung của hai chế độ là chúng *vô hình* với một con số tổng. Cách phát hiện duy nhất là chia dữ liệu theo lát và báo cáo metric trên từng lát — theo lớp, theo kích thước vật, theo điều kiện chụp. Đây chính là kỹ thuật ở B/07-*chia-du-lieu-va-danh-gia*, và lý do nó được nhắc lại ở đây là vì detection và segmentation có thêm một chiều lát riêng: kích thước vật. Bộ công cụ như *dbolya/tide* tách mAP thành lỗi phân loại, lỗi định vị, lỗi trùng lặp và lỗi nền [66], [85]; đọc bảng đó thường có ích hơn đọc thêm mười paper kiến trúc.

Bảng 14.3. Bốn họ bài toán lân cận, đọc theo trục “dạng đầu ra quyết định mọi thứ còn lại”.

Bài toán	Dạng đầu ra	Metric đúng	Hỏng khi nào
Pose	Heatmap hoặc toạ độ	PCK, OKS	Khớp bị che: hồi quy trả về trung bình của hai khả năng
OCR	Chuỗi ký tự	Khớp cả chuỗi, khoảng cách chỉnh sửa	Lỗi nhân qua ba khối; rectify hỏng im lặng
Retrieval	Embedding	Recall@ <i>k</i> , mAP truy hồi	Không gian nhúng sai thì không sửa cục bộ được; recall của ANN đặt trần
Fine-grained / long-tail	Nhãn (như phân loại)	Accuracy theo lớp và theo lát	Con số tổng bị các lớp đầu thống trị, che kín các lớp đuôi

Bảng 14.3 nên đọc từ cột thứ hai sang: chọn cột hai thì cột ba và cột bốn gần như bị quyết định theo, và đó chính là lý do bước chọn dạng đầu ra đáng dành thời gian hơn bước chọn backbone.

E. Giới hạn và trường hợp hỏng

Bốn họ trên chia nhau bốn chế độ hỏng, và điểm chung là cả bốn đều *im lặng*:

- **Pose — khớp bị che.** Heatmap trả một đỉnh yếu ở vị trí sai, và hệ thống hạ nguồn không phân biệt được “dự đoán yếu” với “không có”. *Cách chữa duy nhất*: truyền độ tin cậy đi cùng toạ độ, để tầng sau quyết định. Đây đúng là nguyên tắc mà bộ lọc trạng thái trong SLAM đã dùng từ lâu — không ai truyền một phép đo mà không truyền hiệp phương sai của nó.
- **OCR — nhân lỗi và thất bại im lặng.** Ba khối nhân xác suất đúng với nhau, và khối rectify hỏng vẫn sinh ra một ảnh hợp lệ để đưa vào khối nhận dạng, nên không có ngoại lệ nào được ném ra.
- **Retrieval — gallery trôi theo thời gian.** Chất lượng giảm dần mà không có tín hiệu nào lúc chạy, vì hệ thống không bao giờ biết câu trả lời đúng. *Cách phát hiện*: giữ một tập truy vấn cố định và đo lại định kỳ.
- **Fine-grained và long-tail — chính bạn.** Đo bằng một con số tổng rồi tin rằng hệ thống hoạt động. *Cách chữa*: báo cáo theo lát, không bao giờ chỉ báo cáo một số.

CẦN NHỚ

Ở mỗi bài toán lân cận, câu hỏi đầu tiên không phải “dùng kiến trúc nào” mà là “đầu ra nên có dạng gì” — điểm, heatmap, chuỗi, hay embedding. Dạng đầu ra quyết định loss, metric, chế độ hỏng và cả cách hệ thống hạ nguồn phải đối xử với sự mơ hồ. [T3]

F. Kết nối

[B/05-thiet-ke-bai-toan](#) là tiền đề khái niệm: chọn dạng đầu ra chính là bước hai trong bộ ba đầu vào – đầu ra – tín hiệu giám sát. [B/07-chia-du-lieu-va-danh-gia](#) và [B/08-dich-chuyen-phan-phoi](#) là chỗ xử lý nghiêm túc fine-grained và long-tail. [C/15-thi-giac-khong-thoi-gian/03-flow-tracking-va-rang-buoc](#) nối trực tiếp với phần top-down/bottom-up: tracking lặp lại đúng cấu trúc đánh đổi ở quy mô thời gian. [C/18-pretraining-va-foundation](#) giải thích vì sao không gian nhúng học từ pretraining quy mô lớn thường tốt hơn không gian học riêng cho một bài toán retrieval nhỏ.

❏ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao hồi quy trực tiếp toạ độ khớp lại xử lý sai trường hợp khớp bị che, trong khi heatmap thì không?
2. Với heatmap stride 4, sai số lượng tử hoá là bao nhiêu pixel, và khi nào con số đó bắt đầu quan trọng?
3. Top-down chính xác hơn nhưng vì sao lại là lựa chọn tệ cho một hệ thống có ngân sách mili-giây cố định?
4. Ba khối OCR mỗi khối đúng 95%. Hệ thống đúng bao nhiêu, và vì sao metric từng khối không cho bạn biết điều đó?
5. Metric learning cho phép thêm lớp mới không cần huấn luyện lại. Cái giá của khả năng đó là gì, và nó lộ ra ở đâu khi hệ thống chạy thật?
6. Vì sao accuracy tổng che giấu cả fine-grained lẫn long-tail, và bạn thay nó bằng cách báo cáo nào?

14.5 Thực hành

Chương này là chương dễ thực hành nhất của cả cây, vì mọi khẳng định trong đó đều kiểm chứng được trong vài giờ trên một GPU tầm trung. Hãy tận dụng: đọc mười paper detection không dạy bạn được bằng một lần tự tay đổi cách gán mẫu và nhìn con số nhúc nhích.

Repo và tài nguyên. Về detection, [roboflow/rf-detr](#) là dòng DETR thực dụng nhất tại thời điểm viết (giấy phép Apache 2.0, backbone DINOv2 nên khá bền với vật bị che); [ultralytics](#) YOLO11/YOLO26 nhanh và hợp với thiết bị biên nhưng dùng giấy phép **AGPL-3.0**, hãy kiểm tra kỹ trước khi đưa vào sản phẩm đóng; [lyuwenyu/RT-DETR](#) và RTMDet trong [open-mmlab/mmdetection](#) (MIT) là hai lựa chọn trung dung. Về segmentation, SAM 3 cho phân vùng theo prompt trên cả ảnh lẫn video, Mask2Former và SegFormer cho tập nhãn đóng, còn [MIC-DKFZ/nnUNet](#) là baseline y tế rất khó đánh bại. Open-vocabulary: Grounding DINO, Grounded-SAM. Khung làm việc: [facebookresearch/detectron2](#), [open-mmlab/mmdetection](#), [qubvel-org/segmentation_models.pytorch](#). Soi lỗi: [voxel51/fiftyone](#) để nhìn tận mắt, [dbolya/tide](#) để tách mAP thành các loại lỗi. Danh sách này chốt tại tháng 9/2026 và tên mô hình dẫn đầu sẽ cũ trong 6–12 tháng, trong khi các repo hạ tầng (detectron2, mmdetection, fiftyone) bền hơn nhiều — hãy đầu tư thời gian theo tỉ lệ đó.

Câu hỏi kiểm tra hiểu. Vì sao DETR bản gốc hội tụ chậm hơn Faster R-CNN nhiều lần, và hai bản sửa lớn tấn công vào nguyên nhân nào? Nếu hai detector chênh nhau 2 mAP, bạn làm thế nào để biết chênh lệch đó đến từ kiến trúc hay đến từ cách gán mẫu và công thức huấn luyện? Khi nào một mô hình nền tảng zero-shot vẫn thua một mô hình nhỏ fine-tune, và dấu hiệu nào cho bạn biết điều đó trước khi đo?

Mini project (vài giờ đến hai ngày). Huấn luyện RF-DETR và một biến thể YOLO trên cùng một tập dữ liệu tự thu, với cùng ngân sách thời gian chứ không phải cùng số epoch. So sánh mAP, độ trễ trung bình và phương sai độ trễ, dung lượng mô hình, và giấy phép. Kết thúc bằng một khuyến nghị viết cho một tình huống triển khai cụ thể, trong đó nêu rõ ràng buộc nào loại bỏ phương án nào.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Anchor	Khung chữ nhật mẫu, kích thước và tỉ số cạnh định sẵn, rải đều trên mọi vị trí của feature map. Mô hình đoán <i>độ lệch</i> so với anchor gần nhất chứ không đoán khung từ số không. Nó mã hoá thông kê hình dạng của tập dữ liệu gốc, nên đối miền ảnh là phải thiết kế lại.
••• RPN	Mạng con trượt trên chính feature map của backbone, mỗi vị trí trả về độ khách quan và độ lệch khung. Nó thay selective search, và vì dùng chung backbone nên chi phí sinh đề xuất gần như bằng không.
•• Vì sao Fast R-CNN vừa nhanh hơn vừa chính xác hơn R-CNN?	Nhanh hơn vì backbone chạy một lần thay vì hai nghìn lần, đưa chi phí từ $O(N)$ lượt backbone về $O(1)$. Chính xác hơn vì phân loại và hồi quy khung gộp vào một hàm mất mát huấn luyện một pha, nên mọi khối cùng tối ưu cho mục tiêu cuối.
•• RoIAlign	Phép cắt đặc trưng theo vùng, lấy mẫu song tuyến tại toạ độ thực thay vì làm tròn về ô nguyên. Ở stride 16, làm tròn dịch biên tới 3 pixel ảnh, đủ để phá nhánh mask; bỏ nó đi thì nhánh mask mới chạy được.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Vì sao vật nhỏ bị thiết kế trước khi có FPN?	Tầng sâu có ngữ nghĩa mạnh nhưng độ phân giải thô, tầng nông thì ngược lại. Vật nhỏ cần cả hai mà không tầng nào cho đủ. FPN gộp chúng bằng nhánh đi xuống cộng kết nối ngang 1×1 .
●● Recall đề xuất thấp mà độ chính xác trên đề xuất vẫn cao — nguyên nhân?	Tập anchor lệch miền: nó không phủ dài tỉ lệ hay tỉ số cạnh của dữ liệu mới. Phân biệt với detector kém bằng cách đo riêng recall của giai đoạn một; nếu recall đó thấp thì lỗi nằm ở anchor, không ở bộ phân loại.
●●● Triệt phi cực đại (NMS)	Bước hậu xử lý giữ khung điểm cao nhất rồi xoá mọi khung chồng lấn quá ngưỡng IoU. Nó không học được vì nó xét quan hệ <i>giữa</i> các dự đoán. Hổng trong cảnh đông: hai vật cùng lớp chồng nhau thì chắc chắn mất một.
●●● Gán mẫu	Quy tắc quyết định vị trí dự đoán nào phải học vật nào. Nó nằm trong định nghĩa hàm mất mát chứ không phải trong kiến trúc, nên đổi nó là đổi bài toán tối ưu. Đây là thành phần giải thích phần lớn chênh lệch giữa các detector.
●●● Focal loss	Cross-entropy nhân thêm hệ số $(1 - p_t)^\gamma$ để nén đóng góp của mẫu đã đúng. Với $\gamma = 2$, một anchor nền chắc chắn bị hạ hơn bốn bậc, nên 10^5 mẫu nền hết chi phối gradient. Hổng trên nhãn nhiễu: mẫu gán sai trông y hệt mẫu khó.
●● Vì sao chỉ detector một giai đoạn cần focal loss?	Vì giai đoạn đề xuất đã ngâm lọc hàng chục nghìn vị trí xuống còn vài trăm, tức đã cân bằng lại tập mẫu. Bỏ nó đi thì tỉ lệ nền trên vật quay về cỡ 10^3 tới 10^4 , và mẫu dễ nhấn chìm tín hiệu học.

●●● phải thuộc ●● nên nắm ● biết là có

I Active recall — phần 2 (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●● ATSS	Cách gán mẫu chọn ngưỡng IoU riêng cho từng vật: lấy k ứng viên gần tâm nhất, tính trung bình và độ lệch chuẩn IoU của chúng, dùng $\mu + \sigma$ làm ngưỡng. Nó cho thấy sau khi thống nhất gán mẫu, khác biệt anchor với anchor-free gần như biến mất.
●● SimOTA	Cách gán mẫu phát biểu việc chọn mẫu dương thành bài toán vận tải tối ưu trên toàn ảnh, chi phí là tổ hợp loss phân loại và loss hồi quy. Ưu điểm là phân xử được vùng chồng lấn; nhược điểm là vô nghĩa ở giai đoạn đầu huấn luyện nên phải kèm bước khởi động.
●●● Ghép cặp Hungary	Phép gán một-một giữa N truy vấn và các vật thật, tìm phép ghép có tổng chi phí nhỏ nhất. Vì chỉ một truy vấn được thưởng cho mỗi vật, mô hình tự học không sinh trùng lặp, và NMS hết lý do tồn tại. Giá là hội tụ chậm.
● AP ₅₀ cao nhưng AP ₇₅ thấp — nguyên nhân?	Định vị kém, không phải phân loại kém. Thường vì NMS xếp hạng bằng điểm phân loại, nên khung tự tin về nhãn nhưng lệch vị trí xoá mất khung đúng hơn. Phân biệt bằng TIDE: nếu lỗi định vị chiếm phần lớn thì đúng là nguyên nhân này.
●● Output stride	Tỉ lệ thu nhỏ giữa ảnh và feature map. Output stride 32 nghĩa là mỗi ô đặc trưng đại diện cho một mảng 32×32 pixel, nên sai số định vị biên có sàn cỡ chục pixel. Hạ nó một nửa thì lượng kích hoạt gấp bốn.
●● Vì sao tích chập giãn không làm mất độ phân giải?	Vì nó chèn khoảng trống giữa các trọng số thay vì thu nhỏ ảnh. Kernel 3×3 giãn r có trường tiếp nhận hiệu dụng $3 + 2(r - 1)$, trong khi số ô đầu ra giữ nguyên. Cái giá là bộ nhớ, vì độ phân giải không giảm.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Vì sao ba tầng cùng giãn 2 bỏ sót nửa số pixel?	Vì mọi vị trí lấy mẫu rơi vào cùng một lớp chẵn lẻ, nên trường tiếp nhận trông rộng mà thực chất thừa. Đó là hiệu ứng lưới, và cách chữa là chọn dãy tỉ lệ giãn nguyên tố cùng nhau.
●●● Mask classification	Phát biểu coi đầu ra là một tập truy vấn, mỗi truy vấn trả một mask nhị phân cộng một nhãn lớp, ghép với chân trị bằng ghép cặp Hungary. Nhờ nó semantic, instance và panoptic chỉ còn khác nhau ở cách đọc tập mask, dùng chung một mô hình.
●● Heatmap	Bản đồ xác suất cho mỗi keypoint, lấy argmax làm vị trí. Nó biểu diễn được sự mơ hồ khi khớp bị che, điều mà hồi quy tọa độ không làm được. Giá là bộ nhớ, cộng sai số lượng tử hoá ± 2 pixel ở stride 4.
● Số tổng đẹp — mAP cao, mIoU 0,85 — nhưng lớp vật nhỏ gần như bằng 0?	Vì cả hai thước đo đều trung bình theo diện tích bên trong mỗi lớp, nên vật tí hon đóng góp gần như bằng không, và một lớp hiếm bị các lớp lớn che lấp. Phân biệt bằng cách báo cáo theo lát kích thước vật; nếu chỉ lát nhỏ nhất sập thì đúng là nguyên nhân này.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
<code>detectron2</code> , <code>mmdetection</code>	Hiện thực trực tiếp mạch hai giai đoạn: RPN, RoIAlign, FPN, và các bộ gán mẫu thay thế được	Hạ tầng, bền hơn tên mô hình dẫn đầu rất nhiều
Họ RT-DETR / RF-DETR (2024–2026)	Dùng dự đoán tập hợp và ghép cặp Hungary để bỏ hẳn NMS, cộng attention biến dạng để chữa hội tụ chậm	Dòng detector không NMS thực dụng tính tới 9/2026
Ultralytics YOLO	Bộ gán mẫu động kiểu SimOTA/TAL là một trong những thứ đóng góp nhiều nhất vào chênh lệch giữa các phiên bản	Được cho là vậy dựa trên code; ghi chú phát hành không phải lúc nào cũng nêu rõ
Segment Anything (SAM, và SAM 3 tính tới 9/2026)	Segmentation theo prompt — bỏ ràng buộc tập nhãn cố định, giữ nguyên đầu ra dạng mask	Đang được dùng làm bộ gán nhãn bán tự động trong nhiều pipeline dữ liệu
<code>nnUNet</code>	U-Net với skip connection, cấu hình tự động theo tập dữ liệu; vẫn là baseline rất khó đánh bại trong ảnh y tế	Bằng chứng rằng kiến trúc 2015 chưa hề lỗi thời khi ràng buộc là ít dữ liệu
<code>tide</code> , <code>fiftyone</code>	Tách mAP thành lỗi phân loại, định vị, trùng lặp, nền — hiện thực của ý “một con số tổng che giấu chế độ hồng”	Công cụ chẩn đoán, không phải mô hình

MINI PROJECT SLAM · Lọc vật thể động bằng segmentation trước khi vào frontend SLAM · 1–2 ngày

Mục tiêu	trả lời một câu hỏi kỹ thuật cụ thể thay vì một câu hỏi học thuật: đưa một mô hình phân vùng có sẵn vào trước frontend để vứt bỏ đặc trưng nằm trên người đi lại, thì quỹ đạo có tốt lên <i>vượt sân nhiều</i> của chính harness của bạn không?
Dữ liệu	một chuỗi EuRoC có người di chuyển trong khung hình, hoặc một chuỗi tự quay trong hành lang có người qua lại; chân trị lấy từ hệ bắt chuyển động hoặc từ một lần chạy tham chiếu. Mô hình phân vùng dùng sẵn, không huấn luyện lại — đây là bài toán tích hợp, không phải bài toán huấn luyện.
Các bước	(1) chạy mô hình phân vùng ngoại tuyến trên cả chuỗi, lưu mask nhị phân của các lớp động (người, xe) ra đĩa để không làm nhiều phép đo thời gian; (2) chèn một bước lọc vào ngay sau khối trích đặc trưng: bỏ mọi đặc trưng có tâm rơi vào mask; (3) chạy cấu

hình gốc và cấu hình có lọc, mỗi cấu hình ít nhất ba lần, ghi ATE bằng **evo**; (4) ghi thêm hai số phụ mỗi lần chạy: số đặc trưng còn lại trung bình mỗi khung hình, và số khung hình mất bám.

Xong khi bạn có một bảng ATE trung bình và độ lệch chuẩn cho cả hai cấu hình trên ít nhất ba lần chạy mỗi bên, và phát biểu được một trong hai kết luận có bằng chứng: “cải thiện lớn hơn hai lần độ lệch chuẩn nên không phải nhiều”, hoặc “cải thiện nằm trong sàn nhiều nên chưa kết luận được”. Kèm một câu giải thích dựa trên hai số phụ: nếu ATE tệ đi, có phải vì mask ăn mất quá nhiều đặc trưng ở vùng giàu kết cấu không?

Nối với E/24 cho cách đọc chệnh lệch trong nhiều; B/07 cho giao thức đánh giá và việc không được chọn lần chạy đẹp nhất; kết quả của bài này là baseline cho mini project của chương 15.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì quy tắc gán mẫu nằm trong *định nghĩa* hàm mất mát chứ không trong mô hình: đổi nó là đổi bài toán tối ưu, trong khi đổi backbone chỉ đổi cách giải cùng một bài toán. ATSS cho thấy sau khi thống nhất gán mẫu, khác biệt anchor / anchor-free gần như biến mất.
- A2. Vì chạy backbone một lần cho phép gộp phân loại và hồi quy khung vào một hàm mất mát duy nhất huấn luyện một pha; ở R-CNN ba pha rời rạc, mỗi pha tối ưu cho mục tiêu của riêng nó nên không pha nào tối ưu cho kết quả cuối.
- A3. Vì NMS thao tác trên *quan hệ giữa các dự đoán*, trong khi hàm mất mát của các kiến trúc đó chấm điểm từng dự đoán độc lập — không có chỗ để gán gradient. Gỡ được nó đòi hỏi đổi phát biểu sang dự đoán tập hợp với ghép cặp một-một, chứ không phải thêm một khối mới.
- A4. Rằng chi tiết cài đặt có thể là biến quyết định chứ không phải chi tiết phụ, nên phần đáng soi kỹ nhất của một paper thường là code và bảng ablation, không phải sơ đồ kiến trúc ở trang đầu.
- A5. Rằng ta đã tìm được phát biểu đúng. Khi ba bài toán trở thành ba cách hậu xử lý cùng một tập mask, đơn vị mà mô hình nói tới là một mask kèm một nhãn. Đơn vị đó khớp với cấu trúc thật của bài toán, không còn là quy ước của từng code base.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn làm rơi chùm chìa khoá trong một căn phòng và nhờ một người máy tìm hộ. Cách hiển nhiên là bảo nó “nhìn cả phòng rồi nói chìa nằm ở đâu”, nhưng người máy không làm được: có hôm một chùm chìa, có hôm ba chùm, có hôm không có chùm nào, mà nó thì chỉ biết trả lời những câu hỏi có hình dạng cố định.

Mẹo của cả chương này là chia phòng thành hàng trăm nghìn ô vuông nhỏ rồi hỏi mỗi ô đúng hai câu giống hệt nhau: “trong ô này có thứ gì không” và “nếu có thì mép của nó cách đây bao xa”. Bây giờ mọi câu hỏi đều cùng một dạng, nên người máy học được. Toàn bộ lịch sử của lĩnh vực là chuyện đưa dần những việc con người vẫn phải làm tay vào bên trong phần mà máy tự chỉnh được. Những việc đó là: khoanh trước vùng đáng nhìn, chọn cỡ ô, và dọn các câu trả lời trùng nhau.

Cái giá có hai khoản. Thứ nhất, hàng chục ô cạnh nhau cùng nói “có”, nên phải có một bước dọn trùng, và bước dọn ấy chính là chỗ hai chùm chìa nằm sát nhau bị nhập làm một. Thứ hai, cái luật quyết định ô nào *được phép* nói “có” cho chùm chìa nào là do người đặt ra, nằm ngoài phần máy học — và đổi luật đó thường làm kết quả thay đổi nhiều hơn đổi cả cái máy.

Chỗ không rút gọn được: vì sao một luật chọn ô lại tốt hơn luật khác thì tới nay chủ yếu vẫn là kết quả đo được chứ chưa có lời giải thích gọn; ai nói chắc chắn về điều này là đang nói quá.

Một câu: Chia bài toán thành hàng trăm nghìn câu hỏi giống hệt nhau thì máy học được — nhưng luật chấm điểm cho những câu hỏi ấy mới là thứ quyết định kết quả.

Tổng kết chương

Ta đã đi từ R-CNN, nơi mạng chỉ làm mỗi việc phân loại một mẫu ảnh, tới các hệ thống mà gần như mọi quyết định đều nằm trong tầm với của gradient. Trục xuyên suốt là ranh giới giữa phần học được và phần cố định, và mỗi thế hệ đẩy ranh giới đó xuống thấp hơn. Hai thứ còn sót lại là NMS và cách gán mẫu. Chúng sót lại không ngẫu nhiên: một cái thao tác trên quan hệ *giữa* các dự đoán, cái kia nằm trong chính định nghĩa của loss. Cả hai chỉ gỡ được bằng cách đổi phát biểu bài toán. DETR và Mask2Former là hai lần đổi phát biểu như vậy, và điều đáng chú ý là chúng dùng chung một ý tưởng: dự đoán một tập hợp và ghép một-một.

Bài học thực dụng thì nằm ở chỗ khác và quan trọng hơn: phần quyết định kết quả thường không phải phần được viết trong bài. Cách gán mẫu, tỉ lệ mẫu dương/âm, lịch huấn luyện, ngưỡng NMS — những thứ này giải thích chênh lệch mAP nhiều hơn hình vẽ kiến trúc trên trang đầu. Mang thói quen đó sang mọi chương sau: trước khi tin một cải tiến, hỏi xem có gì khác đã thay đổi cùng lúc.

Còn bỏ ngỏ hai hướng. Thứ nhất, mọi thứ trong chương này chạy trên một ảnh tĩnh; thêm chiều thời gian sinh ra ba vấn đề hoàn toàn mới và đó là [C/15-thì-giac-khong-thoi-gian](#). Thứ hai, khi tập nhãn không còn cố định, câu hỏi chuyển từ “kiến trúc nào” sang “pretraining trên nguồn giám sát nào”, và đó là [C/18-pretraining-va-foundation](#).

I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Trục của chương là nuốt dần heuristic vào trong mạng. Selective search, RoIPooling và NMS bị nuốt ở ba thời điểm rất khác nhau — điều gì ở mỗi heuristic quyết định nó dễ hay khó nuốt?
2. Quy tắc gán mẫu xuất hiện ở cả detection hai giai đoạn, một giai đoạn và Hungarian matching của DETR. Ba cách gán đó khác nhau ở giả định nào về quan hệ giữa dự đoán và chân trị, và giả định nào hỏng khi vật thể chen chúc?
3. Mất cân bằng lớp trong detection và mất cân bằng lớp trong segmentation đều được chữa bằng cách đổi hàm mất mát, nhưng vì hai lý do khác nhau. Hai lý do đó là gì, và vì sao cùng một liều focal loss lại nguy hiểm ở một bên hơn bên kia?
4. Hệ của bạn chạy detector rồi mới cắt vùng cho head tư thế hoặc OCR. Kiểu hỏng nào của giai đoạn đầu bị thước đo của từng khối che giấu, và bạn đo cả chuỗi bằng cách nào?
5. Hộp, mặt nạ, heatmap khớp và nhúng metric learning là bốn cách phát biểu đầu ra. Với mỗi cách, chỗ nào trong đó là một phép lượng tử hoá hay làm tròn, và cái giá của nó lộ ra ở loại vật thể nào?
6. Vì sao việc ba bài segmentation gộp được vào một mô hình lại là bằng chứng về *phát biểu bài toán*, trong khi việc detection vẫn tách hai nhánh anchor và anchor-free thì không nói lên điều tương tự?
7. Bạn nhận một bảng so sánh detector với mọi dòng cùng backbone. Ba chi tiết nào không nằm trong bảng vẫn đủ sức đảo thứ tự, và bạn hỏi tác giả câu nào trước?

Thị giác chuỗi và không-thời gian (Sequential and Spatio-Temporal Vision)

PHỤ THUỘC C/13-kien-truc-backbone (CNN, attention); C/14-bai-toan-cv-loi (detection, segmentation — tracking dựng thẳng trên detection); B/11-gioi-han-tinh-toan (ngân sách bộ nhớ và tính toán); B/07-chia-du-lieu-va-danh-gia (chọn metric, rò rỉ dữ liệu). [T3]

TÓM TẮT

Chương này hỏi: thêm chiều thời gian vào ảnh thì phát sinh những vấn đề nào mà ảnh tĩnh không có? Câu trả lời là đúng ba vấn đề, và ba mục của chương tương ứng với ba vấn đề đó. Thứ nhất, đầu ra có độ dài thay đổi — đây là nơi *attention* ra đời. Thứ hai, chi phí bộ nhớ nhân theo số khung hình. Thứ ba, mô hình chạy trực tuyến không được nhìn khung hình tương lai. Đọc xong bạn thiết kế được một hệ thị giác thời gian thực bắt đầu từ ràng buộc chứ không từ bảng xếp hạng: biết kiến trúc nào bị loại ngay, ngân sách bộ nhớ phải chia thế nào, và metric nào không đánh lừa. Nếu chỉ nhớ một điều: hai bài toán khó nhất của thị giác thời gian là tương ứng điểm và liên kết dữ liệu. Đó đúng là hai bài toán mà một hệ SLAM giải ở mỗi khung hình. Phần lớn chương này vì thế là dịch từ vựng, không phải học lại từ đầu.

KIẾN THỨC CẦN CÓ TRƯỚC

- **khung hình (frame) và clip** — khung hình là một ảnh tại một thời điểm; clip là một đoạn gồm T khung liên tiếp, và là đơn vị đầu vào của mô hình video — tương đương “một ảnh” của mô hình ảnh.
- **trực tuyến (online / streaming)** — chế độ chạy mà dữ liệu đến dần theo thời gian và phải trả kết quả ngay, khác với chế độ ngoại tuyến được xem toàn bộ video rồi mới quyết định.
- **nhân quả (causal)** — tính chất của một mô hình chỉ dùng thông tin tới thời điểm hiện tại. Mô hình hai chiều — nhìn cả trước lẫn sau — không nhân quả, nên không dùng được cho hệ trực tuyến.
- **độ trễ (latency) và thông lượng (throughput)** — độ trễ là thời gian từ lúc khung hình vào tới lúc có kết quả; thông lượng là số khung xử lý được mỗi giây. Hai đại lượng này độc lập: gộp batch làm thông lượng tăng nhưng độ trễ tăng theo.
- **optical flow** — trường vector cho biết mỗi pixel dịch chuyển đi đâu giữa hai khung hình — biểu diễn tường minh của chuyển động, và là dạng dày đặc của bài toán tương ứng điểm.
- **tracking và track ID** — tracking là giữ danh tính của từng đối tượng qua thời gian; track ID là mã số phải giữ nguyên suốt thời gian đối tượng xuất hiện. Đánh tráo mã số giữa hai đối tượng gọi là *ID switch*.
- **attention** — cơ chế cho bên cần thông tin tính một bộ trọng số trên các phần của bên có thông tin rồi lấy tổ hợp có trọng số — tức hoãn việc nén dữ liệu lại cho tới khi biết đang cần gì.

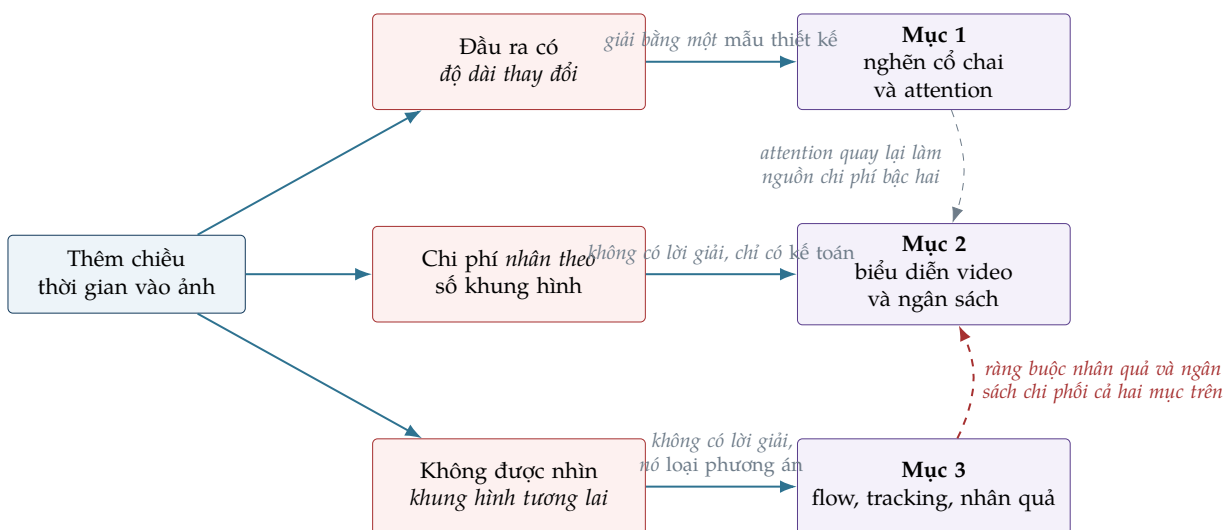
CÂU HỎI MỞ ĐẦU

- Q1. Vì sao một mô hình đạt mIoU cao trên từng khung hình vẫn cho ra video nhấp nháy, và vì sao không có thước đo phổ biến nào bắt được lỗi đó?

- Q2.** *attention* thường được kể như một phát minh của Transformer. Vậy nó thực sự ra đời để giải quyết nút thắt nào, và vì sao nút thắt đó xuất hiện lần đầu ở bài toán mô tả ảnh chứ không ở phân loại?
- Q3.** Một hệ tracking có thể đánh tráo danh tính đối tượng liên tục mà chỉ số MOTA gần như không suy chuyển. Công thức nào cho phép chuyển đó xảy ra?
- Q4.** Vì sao “thời gian là trục thứ ba của tensor” là một câu nói sai, dù về mặt lập trình nó hoàn toàn đúng?
- Q5.** Nếu bạn tăng gấp bốn độ dài clip mà giữ nguyên bộ nhớ, bạn phải trả bằng gì — và vì sao ba cách trả giá đó không hề tương đương?

Chương 14 xử lý từng ảnh như một ốc đảo. Đó là một đơn giản hoá rất tiện nhưng rất xa thực tế. Camera không sinh ra ảnh rời rạc mà sinh ra một dòng liên tục. Phần lớn thông tin hữu ích chỉ tồn tại *giữa* các khung hình, không nằm trong khung nào cả. Một vật đang tiến lại gần, một người vừa ngã, cùng một chiếc xe hay hai chiếc khác nhau — không câu nào trong số đó trả lời được từ một ảnh tĩnh.

Thêm chiều thời gian không phải là thêm một chiều nữa cho tensor. Nó sinh ra đúng ba vấn đề mới, và ba vấn đề đó độc lập với nhau tới mức mỗi cái đáng một mục riêng. Thứ nhất, **đầu ra có độ dài thay đổi**: mô tả một cảnh cần một câu, không phải một nhãn, và việc phải nén cả ảnh vào một vector trước khi sinh câu là nút thắt đã khai sinh ra *attention* trong thị giác. Thứ hai, **chi phí nhân theo số khung hình**. Ngân sách bộ nhớ của một mô hình ảnh vốn đã chật, giờ bị chia tiếp cho ba chiều: độ dài clip, độ phân giải, batch size. Mọi lựa chọn kiến trúc vì thế là lựa chọn hy sinh chiều nào. Thứ ba, **ràng buộc nhân quả**: hệ chạy trực tuyến không được nhìn khung hình tương lai, và ràng buộc đó loại bỏ nguyên một lớp kiến trúc trước khi độ chính xác kịp lên tiếng.



Hình 15.1. Bản đồ chương: ba mục không phải ba chủ đề rời mà là ba *loại vấn đề* khác nhau mà chiều thời gian sinh ra, và ba loại câu trả lời khác nhau về bản chất. Mũi tên đứt cho thấy ba nhánh không độc lập: ràng buộc nhân quả và ngân sách bộ nhớ quyết định cách chữa nào ở mục 3 là khả thi, còn *attention* của mục 1 quay lại chính là nguồn chi phí bậc hai của mục 2.

Hình 15.1 nên xem trước khi đọc, vì nó cho biết mục nào trả lời loại câu hỏi nào — và vì thế mục nào đáng đọc kỹ với bài toán cụ thể của bạn.

Ba mục của chương lần lượt xử lý ba vấn đề đó, theo thứ tự phụ thuộc chứ không theo thứ tự lịch sử. Mục 1 đi từ sinh mô tả ảnh kiểu CNN–RNN tới “Show, Attend and Tell”. Nên đọc nó *trước* phần Transformer của chương 13. Lý do là nó cho thấy *attention* ra đời để gỡ một nút thắt cụ thể, không phải để làm kiến trúc đẹp hơn. Mục 2 chuyển sang biểu diễn video và biến ngân

sách bộ nhớ thành những con số tính được bằng tay, vì đó là ràng buộc mà mọi quyết định phía sau phải sống chung. Mục 3 gom ba bài toán mà một kỹ sư robot gặp hằng ngày — optical flow, tracking, và tính nhất quán thời gian — rồi kết thúc bằng ràng buộc nhân quả, thứ chi phối tất cả.

Với người đọc đến từ hình học và SLAM, chương này gần chuyên môn hơn bất kỳ chương nào khác trong Phần C, và cách đọc hiệu quả nhất là đọc như một cuốn từ điển đối chiếu. Optical flow là bài toán tương ứng điểm ở dạng dày đặc, với đúng bài toán khâu độ và đúng ma trận cấu trúc mà bạn dùng để chọn đặc trưng góc. Tracking là bài toán liên kết dữ liệu, với đúng bộ đôi dự đoán bằng bộ lọc trạng thái cộng gán tối ưu. Còn ràng buộc nhân quả và ngân sách mili-giây thì bạn đã sống cùng nó hằng ngày. Cái mới ở đây không phải cấu trúc bài toán mà là các thành phần học được đặt vào từng vị trí, cùng với câu hỏi quen thuộc: mỗi lần thay một khối hình học bằng một khối học được thì ta mua được độ bền gì và mất đi tính chất gì.

Bảng 15.1. Từ điển đối chiếu cho người đến từ SLAM. Cột giữa và cột phải là cùng một bài toán với hai truyền thống thuật ngữ khác nhau; chỉ cột cuối là thứ thực sự mới.

Bài toán	Tên trong SLAM / hình học	Tên trong thị giác học sâu, và cái mới
Tương ứng điểm	Ghép đặc trưng, KLT, tìm kiếm dọc đường epipolar	Optical flow dày đặc; cái mới là học được prior về chuyển động nên vùng không kết cấu cũng có đáp án
Liên kết dữ liệu	Ghép đặc trưng với điểm bản đồ, công lọc Mahalanobis, RANSAC	Tracking-by-detection; cái mới là dùng thêm đặc trưng ngoại hình học được làm chi phí ghép
Dự đoán trạng thái	Bộ lọc Kalman, mô hình chuyển động hằng vận tốc	Vẫn là Kalman trong đa số bộ tracker; cái mới nằm ở phần đo, không ở phần dự đoán
Chọn đặc trưng tốt	Giá trị riêng nhỏ nhất của ma trận cấu trúc	Cùng ma trận đó xuất hiện trong Lucas–Kanade; mô hình học sâu thay tiêu chí này bằng khối tương quan
Nhận diện nơi đã qua	Túi từ trực quan, kiểm chứng hình học	Nhúng toàn cục cộng bước quay lại đặc trưng — đúng cấu trúc của attention
Ngân sách thời gian thực	Giới hạn số vòng lặp tối ưu, kích thước cửa sổ trượt	Số bước tinh chỉnh của RAFT, độ dài clip, cửa sổ nhân quả

Bảng 15.1 nên đọc trước cả ba mục. Nó cho biết chỗ nào bạn được phép đọc lướt vì cấu trúc bài toán đã quen. Nó cũng chỉ ra chỗ phải đọc kỹ: nơi một thành phần học được thay vào, mang theo giả định mới.

Bản đồ chương 15 — Thị giác chuỗi và không-thời gian

TÓM TẮT

Chương 15 chia làm ba mục theo đúng ba vấn đề mà chiều thời gian sinh ra, chứ không theo thứ tự lịch sử của các kiến trúc. Ba vấn đề đó là: đầu ra có độ dài thay đổi, chi phí bộ nhớ nhân theo số khung hình, và ràng buộc nhân quả. Bản đồ này nói mục nào cốt lõi, mục nào tra khi cần, và mục nào nên đọc chen vào giữa chương 13. Nếu chỉ nhớ một điều: mục 3 là mục gần chuyên môn của một kỹ sư SLAM nhất và cũng là mục dùng được ngay nhất, còn mục 1 nên đọc *trước* phần Transformer của chương 13.

KIẾN THỨC CẦN CÓ TRƯỚC

- **chương và mục** — chương là một chủ đề lớn, mỗi mục là một file đọc độc lập được. Số trong tên thư mục là thứ tự nên đọc, không phải thứ tự lịch sử.
- **trực tuyến (streaming) và ngoại tuyến** — trực tuyến là xử lý dữ liệu đến dần và trả kết quả ngay; ngoại tuyến là được xem toàn bộ video rồi mới quyết định. Phần lớn số liệu công bố là ngoại tuyến.
- **nhân quả (causal)** — chỉ dùng thông tin tới thời điểm hiện tại, không nhìn khung hình tương lai — ràng buộc cứng của mọi hệ chạy trên robot.
- **ngân sách bộ nhớ ba chiều** — lượng bộ nhớ khi huấn luyện video tỉ lệ với độ dài clip $\times$ độ phân giải² $\times$ batch size, nên tăng một chiều buộc phải giảm chiều khác.

Có gì trong chương này

- **Mục 1 — Đầu ra độ dài thay đổi và sự ra đời của attention (cốt lõi, đọc sớm)**. Sinh mô tả ảnh kiểu CNN-RNN, nghiền cổ chai của một vector duy nhất, và “Show, Attend and Tell”. Giá trị lớn nhất của mục không phải bài toán captioning mà là việc nó cho thấy *động cơ* của attention.
- **Mục 2 — Biểu diễn video và cái giá của thời gian (cốt lõi)**. Gộp theo khung, hai luồng, tích chập 3D, bản tách (2+1)D, SlowFast, video transformer — và ba phép tính bộ nhớ quyết định tất cả.
- **Mục 3 — Flow, tracking và các ràng buộc riêng (cốt lõi, gần chuyên môn nhất)**. Lucas-Kanade tới RAFT, định vị hành động theo khoảng thời gian, tracking như bài toán liên kết dữ liệu, bầy của MOTA và cách HOTA sửa nó, tính nhất quán thời gian, và ràng buộc nhân quả.

Vì sao chia như vậy

Cách chia hiển nhiên sẽ là chia theo bài toán — captioning, phân loại hành động, tracking — nhưng cách đó che mất điều đáng học. Ba mục ở đây chia theo *ba vấn đề cấu trúc* mà chiều thời gian sinh ra, và mỗi vấn đề có một dạng câu trả lời khác hẳn. Vấn đề thứ nhất được giải bằng một mẫu thiết kế, tức attention. Vấn đề thứ hai không có lời giải, chỉ có kế toán. Vấn đề thứ ba cũng không có lời giải; nó chỉ loại bỏ phương án. Nhìn theo trục này thì các kiến trúc cụ thể trở thành hệ quả, và bạn không phải nhớ chúng như một danh sách.

Thứ tự đọc và phụ thuộc

1. Đọc mục 1 *trước* [C/13-kien-truc-backbone/02-tu-chuoi-toi-attention](#): hiểu nút thắt trước thì self-attention thành hệ quả hiển nhiên thay vì một phát minh khó hiểu.
2. Mục 2 trước mục 3: ngân sách bộ nhớ ở mục 2 là thứ quyết định cách chữa nào cho hiện tượng nhấp nháy là khả thi ở mục 3.
3. Mục 3 giả định bạn đã đọc [C/14-bai-toan-cv-loi](#), vì tracking-by-detection dựng thẳng trên một detector.
4. Đọc [B/07-chia-du-lieu-va-danh-gia](#) song song với mục 3 nếu phần metric và rò rỉ dữ liệu theo clip còn mờ.
5. [C/16-thi-giac-3d](#) là chương tiếp nối tự nhiên: nó nâng bài toán tương ứng điểm từ mặt phẳng ảnh lên không gian ba chiều.

Nếu chỉ có một buổi

Đọc mục 3, và trong mục 3 đọc kỹ hai chỗ. Thứ nhất là bài toán khâu độ với ma trận cấu trúc: nó nổi thẳng vào việc chọn đặc trưng mà bạn đã làm. Thứ hai là hộp cạm bẫy về MOTA: nó thay đổi ngay cách bạn báo cáo kết quả tracking. Phần còn lại có thể đọc lướt và quay lại khi cần.

15.1 Đầu ra độ dài thay đổi và sự ra đời của attention (Variable-Length Outputs and the Birth of Attention)

PHỤ THUỘC C/13-kien-truc-backbone/02-tu-chuoi-toi-attention (RNN, LSTM, attention — mục này là động cơ của mục đó, nên đọc mục này trước sẽ dễ hơn); B/05-thiet-ke-bai-toan (dạng đầu ra).

TÓM TẮT

Mục này kể lại đúng một sự kiện: **attention** không sinh ra từ mong muốn làm kiến trúc đẹp hơn, mà sinh ra để gỡ một nút thắt rất cụ thể — việc nén cả một ảnh vào một vector duy nhất trước khi sinh câu mô tả. Đọc xong bạn hiểu *vì sao* self-attention của Transformer là hiển nhiên chứ không bí ẩn, và bạn nhận ra được cùng một nút thắt ở những nơi khác, kể cả trong pipeline SLAM của chính mình. Nếu chỉ nhớ một điều: ở đâu một hệ thống nén thông tin có cấu trúc thành một vector cố định *trước khi* biết sẽ cần phần nào, ở đó có một nút thắt. Attention là cách chuẩn để gỡ nó, bằng cách hoãn phép nén tới lúc biết đang cần gì.

KIẾN THỨC CẦN CÓ TRƯỚC

- **token** — một đơn vị rời rạc của chuỗi đầu ra — thường là một từ hoặc một mảnh từ. Mô hình sinh câu bằng cách dự đoán token tiếp theo, lặp lại tới khi gặp token kết thúc.
- **RNN và LSTM** — mạng xử lý chuỗi bằng cách mang theo một *trạng thái ẩn* (một vector) từ bước này sang bước sau. LSTM là biến thể có cổng, giúp trạng thái giữ được thông tin qua nhiều bước hơn.
- **encoder-decoder** — encoder biến đầu vào thành biểu diễn số; decoder sinh đầu ra từ biểu diễn đó. Ở đây encoder là CNN nhìn ảnh, decoder là mạng sinh câu.
- **feature map (lưới đặc trưng)** — đầu ra không gian của CNN: một lưới, ví dụ 14×14 , mỗi ô là một vector mô tả một vùng ảnh. Giữ nguyên lưới nghĩa là giữ nguyên thông tin “ở đâu”.
- **global average pooling** — phép lấy trung bình mọi ô của lưới đặc trưng để ra một vector duy nhất. Rất tiện cho phân loại, nhưng nó xóa sạch thông tin vị trí.
- **attention** — cơ chế cho một bên (bên cần thông tin) tính một bộ trọng số trên các phần của bên kia (bên có thông tin), rồi lấy tổ hợp có trọng số. Trọng số phụ thuộc vào việc đang cần gì, nên phép nén được “hoãn” tới lúc có truy vấn.
- **self-attention** — trường hợp hai bên là cùng một tập: mỗi phần tử tự tính trọng số trên mọi phần tử khác của chính chuỗi đó. Đây là khối cơ bản của Transformer.

Đặt vấn đề

Toàn bộ chương 14 giả định một điều mà ta không để ý: đầu ra tuy có thể là tập hợp nhưng *mỗi phần tử* của nó vẫn có hình dạng cố định — một khung là bốn số, một mask là một lưới nhị phân. **Sinh mô tả ảnh** (*image captioning*) phá vỡ cả điều đó: đầu ra là một câu, dài ngắn tùy ảnh, và mỗi từ phụ thuộc vào các từ trước. Đây là lần đầu thị giác máy tính phải sinh ra một chuỗi, và nó buộc phải mượn bộ máy của xử lý ngôn ngữ.

Giải pháp đầu tiên nghe rất hợp lý. Cho ảnh qua CNN, lấy vector đặc trưng ở tầng áp chót, dùng nó làm trạng thái khởi tạo cho một **mạng hồi quy** (*recurrent neural network, RNN*) hoặc **LSTM** [137], rồi để mạng đó sinh từng **token** một. Kiến trúc này chạy được và cho ra những câu nghe khá trôi chảy. Nhưng nó chứa một giả định chết người: rằng *một* vector cố định — thường 512 hay 2048 số — đủ để trả lời mọi câu hỏi mà mọi từ trong câu sẽ đặt ra. Khi mạng đang sinh từ “con chó”, nó cần biết chi tiết ở góc dưới bên trái; khi sinh từ “bãi cỏ”, nó cần chi tiết ở nền. Vector kia phải chứa sẵn cả hai, và cả những chi tiết cho mọi câu khác mà nó không biết trước.

Học xong mục này bạn làm được gì mà trước đó không làm được? Bạn phát hiện được nút thắt kiểu “nén trước, dùng sau” trong một kiến trúc bất kỳ, kể cả kiến trúc không dính gì tới ngôn ngữ. Bạn cũng biết attention là công cụ chuẩn để gỡ nó, và biết cái giá phải trả khi gỡ.

TRỰC GIÁC

Hãy tưởng tượng bạn phải kể lại một bộ phim cho người khác, nhưng luật chơi là: xem xong, bạn chỉ được ghi *một* câu tóm tắt vào giấy, rồi trả lại bằng phim, và từ đó chỉ được nhìn tờ giấy để kể suốt hai mươi phút. Bạn sẽ kể trôi chảy, nhưng mọi chi tiết đều là bịa hợp lý. Attention là việc bỏ luật đó đi: mỗi khi kể đến một cảnh, bạn được phép tua lại băng và nhìn đúng cảnh ấy. Không có gì huyền bí — toàn bộ ý tưởng chỉ là *hoãn việc nén lại cho tới khi biết mình đang cần gì*.

A. Từ một vector tới một cái nhìn có chọn lọc

I Mạch 1 — đầu ra độ dài thay đổi

- 1 Vấn đề** đầu ra là một câu, không phải một nhãn.

Giải pháp gán một bộ giải mã chuỗi lên trên bộ trích đặc trưng: CNN cho vector, RNN/LSTM sinh từng token, huấn luyện bằng **entropy chéo** (*cross-entropy*) trên từng vị trí token.

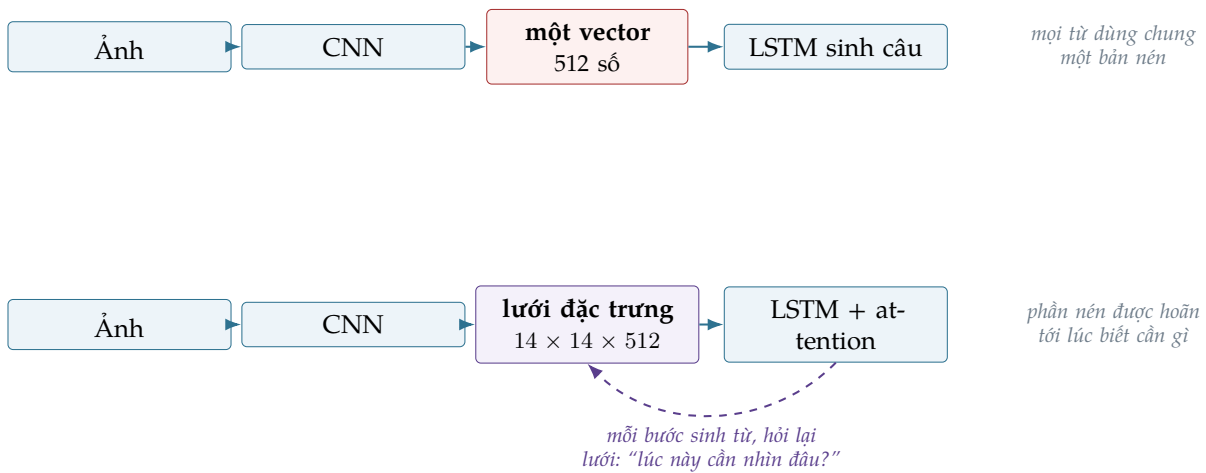
Giả định đặc trưng toàn cục của ảnh chứa đủ thông tin cho toàn bộ câu.

Hệ quả lần đầu tiên một hệ thị giác sinh được đầu ra có độ dài thay đổi.
- 2 Vấn đề** một vector duy nhất là nghẽn cổ chai thông tin. Mọi chi tiết không gian bị nén trước khi ta biết sẽ cần chi tiết nào; và nén *có mất mát* thì không đảo ngược được.

Giải pháp “Show, Attend and Tell” giữ nguyên lưới đặc trưng không gian [158]. Ở mỗi bước sinh từ, bộ giải mã tính một phân bố trọng số trên các ô lưới. Tổ hợp có trọng số của các ô đó là đầu vào cho bước ấy.

Hệ quả đây là nơi attention xuất hiện lần đầu trong thị giác, và nó ra đời để gỡ một nút thắt cụ thể chứ không phải vì đẹp. Ý tưởng gốc đến từ dịch máy, nơi cùng một nghẽn cổ chai xuất hiện ở dạng vector câu nguồn [159].
- 3 Vấn đề** nếu attention hữu ích cho quan hệ giữa *câu và ảnh*, tại sao không dùng cho quan hệ giữa *các phần của cùng một ảnh*?

Giải pháp đó chính là bước đi tới self-attention và **transformer** [107], và đó là lý do nên đọc mục này *trước* C/13-kien-truc-backbone/02-tu-chuoi-toi-attention: khi đã thấy nút thắt, self-attention trở thành hệ quả hiển nhiên thay vì một phát minh từ trên trời rơi xuống.



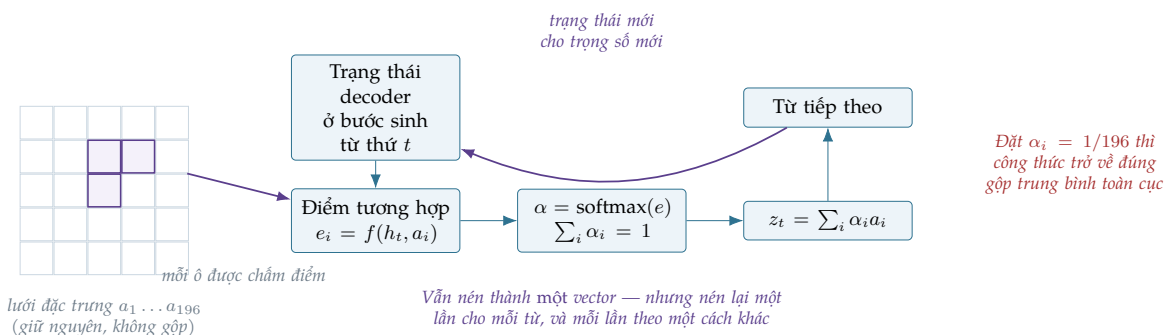
Hình 15.2. Cùng một pipeline, khác nhau đúng một chỗ. Nhánh trên nén lưới đặc trưng thành một vector trước khi biết câu sẽ nói gì; nhánh dưới giữ nguyên lưới và để bộ giải mã truy vấn lại nó ở mỗi bước. Mỗi tên dứt là toàn bộ nội dung của attention — một vòng phản hồi từ nơi cần thông tin về nơi có thông tin.

B. Nút thắt lớn tới mức nào — một phép tính

Hình 15.2 sẽ thuyết phục hơn nhiều khi kèm con số. Lưới đặc trưng của một CNN trước tầng gộp toàn cục có kích thước điển hình $14 \times 14 \times 512$, tức $196 \times 512 = 100\,352$ số. Sau **gộp trung bình toàn cục** (*global average pooling*) chỉ còn 512 số. Tỉ số nén là 196:1, và điều quan trọng không phải tỉ số mà là *cái gì bị mất*: phép gộp lấy trung bình theo không gian, nên nó xoá sạch thông tin “đặc trưng này nằm ở đâu”. Một câu mô tả cần đúng thông tin đó để nói “con chó bên trái người đàn ông”.

↔ hệ quả của C/13/02 — self-attention là bước tiếp theo hiển nhiên khi đem chính cơ chế này áp cho quan hệ giữa các phần bên trong một đầu vào, thay vì giữa hai đầu vào khác nhau.

Nói gọn thì attention thay phép trung bình đều bằng phép trung bình có trọng số, với trọng số phụ thuộc vào việc bộ giải mã đang cần gì: gọi a_i là vector đặc trưng của ô thứ i , và α_i là trọng số attention tại bước sinh từ hiện tại. Đầu vào của bước đó là $z = \sum_i \alpha_i a_i$, với $\sum_i \alpha_i = 1$. Công thức này nói một điều rất đơn giản: *vẫn nén thành một vector, nhưng nén lại một lần cho mỗi từ, và mỗi lần nén theo một cách khác nhau*. Trường hợp $\alpha_i = 1/196$ với mọi i chính là gộp trung bình toàn cục — mô hình cũ là một trường hợp riêng của mô hình mới.



Hình 15.3. Cơ chế attention viết ra thành luồng tính. Điều đáng chú ý không phải công thức mà là vòng phản hồi: trạng thái của bộ giải mã quyết định trọng số, trọng số quyết định vector đưa vào bước sau, và bước sau lại đổi trạng thái. Mô hình cũ là trường hợp riêng khi trọng số bị ép bằng nhau, nên đây là một phép tổng quát hoá chứ không phải một kiến trúc cạnh tranh.

Hình 15.3 cũng cho thấy chi phí: phần chấm điểm phải chạy lại cho *mỗi* từ trên *mọi* ô lưới, nên tổng chi phí tỉ lệ với tích số ô nhân số bước sinh — mằm mông của vấn đề bậc hai ở mục sau.

Bảng 15.2. Ba cách đưa thông tin ảnh vào bộ giải mã chuỗi.

Cách	Cơ chế	Giá phải trả	Hỏng khi nào
Một vector toàn cục	Gộp trung bình rồi khởi tạo trạng thái	Rẻ nhất, một lần tính	Câu cần chi tiết cục bộ hoặc quan hệ không gian; mô hình bị chi tiết hợp lý
Soft attention	Trung bình có trọng số, khả vi	Chi phí tỉ lệ số ô $\times$ số từ	Ảnh nhiều vật giống nhau: trọng số dàn đều, không chọn được
Hard attention	Lấy mẫu rồi rạc một ô	Rẻ khi chạy, nhưng không khả vi	Phải huấn luyện bằng ước lượng gradient có phương sai cao; kém ổn định

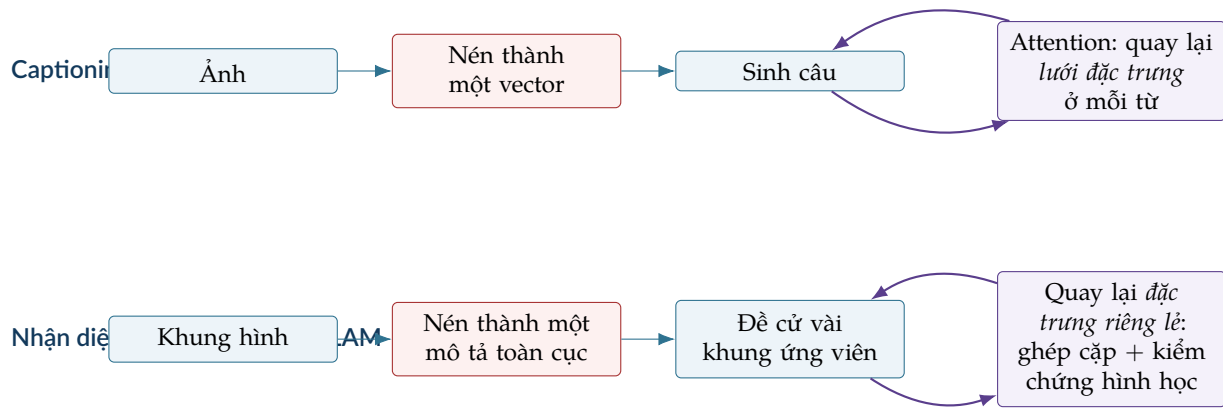
↔ cùng nguyên lý với C/14/02 — mỗi truy vấn của DETR “hỏi” feature map xem vật của nó ở đâu — đúng vòng phản hồi từ nơi cần thông tin về nơi có thông tin.

C. Cùng một nút thắt trong pipeline SLAM

Chỗ nổi này đáng dừng lại, vì nó cho thấy nút thắt trên không phải chuyện riêng của ngôn ngữ. Một hệ *SLAM* nhận diện nơi đã đi qua bằng cách nén cả khung hình thành một mô tả toàn cục — túi từ trực quan hoặc một vector nhúng — rồi so mô tả đó với cơ sở dữ liệu [40], [59]. Bước nén ấy có *đúng* tính chất của nghẽn cổ chai: nó xảy ra trước khi biết sẽ cần kiểm chứng điều gì, nên nó vứt bỏ hình học.

Và giải pháp mà cộng đồng SLAM dùng cũng có đúng cấu trúc của attention: sau khi mô tả toàn cục đề cử vài ứng viên, hệ thống *quay lại* các đặc trưng riêng lẻ để ghép cặp và kiểm chứng bằng ràng buộc hình học. Nói cách khác, mô tả toàn cục chỉ dùng để *chọn nơi cần nhìn kỹ*, chứ không dùng để kết luận — y hệt vai trò của phân bố α trong công thức trên. Nếu bạn thấy self-attention khó hình dung, hãy nghĩ nó như bước kiểm chứng đặc trưng đó, chỉ khác là trọng số được học thay vì được tính bằng RANSAC.

↔ cùng nguyên lý với C/16 — nhận diện nơi đã đi qua nén khung hình thành một mô tả toàn cục rồi phải quay lại đặc trưng để kiểm chứng hình học — cùng cấu trúc “đề cử rồi nhìn kỹ” của attention.



Cùng một cấu trúc: bản nén toàn cục chỉ dùng để chọn nơi cần nhìn kỹ, không dùng để kết luận. Khác biệt duy nhất là trọng số quay lại được học (attention) hay được tính bằng ràng buộc hình học (RANSAC).

Hình 15.4. Nghẽn cổ chai “nén trước, dùng sau” không phải chuyện riêng của ngôn ngữ. Hệ SLAM gặp đúng nó khi nén một khung hình thành một mô tả toàn cục để tra cứu, và giải đúng theo cách của attention: dùng bản nén để đề cử, rồi quay về nguồn thông tin đầy đủ để kiểm chứng. Nhận ra được cấu trúc này thì self-attention thôi trông giống một phát minh xa lạ.

Hình 15.4 là lý do mục này nên đọc trước phần Transformer: khi mẫu thiết kế đã quen, phần công thức chỉ còn là chi tiết.

D. Giới hạn và trường hợp hỏng

- **Thước đo của captioning rất yếu.** Các độ đo dựa trên trùng khớp n -gram thường cho câu “an toàn”: đúng ngữ pháp và giống câu tham chiếu. Vì thế một mô hình sinh mô tả chung chung có thể ghi điểm cao hơn một mô hình mô tả đúng nhưng diễn đạt khác. *Hệ quả thực tế:* cải tiến trên bảng xếp hạng không bảo đảm cải tiến khi người dùng thật đọc.
- **Mô hình bịa chi tiết.** Vì bộ giải mã được huấn luyện để sinh câu *hợp lý*, nó lấp chỗ trống bằng thứ thường đúng — “người đàn ông cầm ván lướt sóng” cho một ảnh bãi biển không có ván. Nút thắt càng chặt thì hiện tượng này càng nặng, nên đây là triệu chứng chẩn đoán được.
- **Bản đồ attention không phải lời giải thích.** Rất dễ nhìn α rồi tin rằng mô hình “đang nhìn vào đó vì lý do đó”. Các kiểm tra tính đúng đắn cho thấy nhiều loại bản đồ nổi bật vẫn trông hợp lý ngay cả khi trọng số mô hình bị làm hỏng [160], nên trọng số attention là *manh mối chẩn đoán*, không phải bằng chứng.
- **Chi phí tăng theo tích số ô $\times$ số bước.** Với ảnh phân giải cao hoặc câu dài, phần attention có thể vượt chi phí của chính CNN. Đây là mầm mống của vấn đề bậc hai ở mục sau, khi số ô còn nhân thêm với số khung hình.

CẦN NHỚ

Nghẽn cổ chai thông tin xuất hiện ở *mọi* chỗ mà hệ thống nén dữ liệu có cấu trúc thành một vector cố định trước khi biết sẽ cần phần nào. Attention không phải một loại tầng mạng, nó là một mẫu thiết kế: *giữ nguyên nguồn, và hoãn việc nén tới lúc có truy vấn*. Nhận ra mẫu này rồi thì Transformer, cross-attention trong DETR, và cả bước kiểm chứng hình học trong SLAM đều là cùng một thứ. [T2]

E. Kết nối

[C/13-kien-truc-backbone/02-tu-chuoi-toi-attention](#) là nơi hình thức hoá đầy đủ; mục này là động cơ của nó, nên thứ tự đọc tự nhiên là mục này trước.

[C/14-bai-toan-cv-loi/02-detection-mot-giai-doan](#) dùng lại chính cơ chế này dưới dạng truy vấn của DETR: mỗi truy vấn “hỏi” feature map xem vật của nó ở đâu. [C/15-thi-giac-khong-thoi-gian/02-bieu-dien-video](#) là bước kế tiếp trực tiếp: khi nguồn không chỉ là một lưới không gian mà là một khối không-thời gian, chi phí attention nhân lên và trở thành ràng buộc thiết kế chính.

❏ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao nén ảnh thành một vector *trước* khi sinh câu lại là một quyết định tồi, trong khi nén ảnh thành một vector trước khi phân loại thì hoàn toàn ổn?
2. Chứng minh bằng một câu rằng gộp trung bình toàn cục là trường hợp riêng của soft attention. Điều đó nói gì về quan hệ giữa hai kiến trúc?
3. Hard attention rẻ hơn khi chạy nhưng ít được dùng. Vì sao?
4. Vì sao không nên đọc bản đồ attention như một lời giải thích về nguyên nhân?
5. Trong một hệ thống bạn đã làm, nghẽn cổ chai kiểu “nén trước, dùng sau” nằm ở đâu, và hệ thống đó bù lại bằng bước “quay lại nguồn” nào?

❏ 15.2 Biểu diễn video và cái giá của thời gian (Video Representation and the Cost of Time)

PHỤ THUỘC [C/13/01-dong-cnn](#) (conv, số tham số, FLOPs); [C/13/02-tu-chuoi-toi-attention](#) (chi phí bậc hai của attention); [B/11-gioi-han-tinh-toan](#) (bảng thông bộ nhớ, ngân sách tính toán).

TÓM TẮT

Mục này trả lời câu hỏi “thêm chiều thời gian vào ảnh thì phải trả giá gì”. Câu trả lời ngắn: bộ nhớ, và nó bị chia cho ba chiều cùng lúc — độ dài clip, độ phân giải, và *batch* — nên tăng một chiều buộc phải giảm chiều khác. Bốn họ kiến trúc (gộp theo khung, hai luồng, tích chập 3D và bản tách (2+1)D, video transformer) là bốn cách khác nhau để mua thông tin thời gian với ít bộ nhớ nhất có thể. Đọc xong bạn tính được ngân sách bộ nhớ của một cấu hình video *trước khi* chạy, và biết chiều nào nên hy sinh cho bài toán của mình. Nếu chỉ nhớ một điều: trước khi tin bất kỳ mô hình video nào, hãy chạy baseline “mỗi khung một lần rồi gộp”. Trên rất nhiều tập dữ liệu, baseline đó gần bằng mô hình chuyển động. Chênh lệch còn lại mới là thứ đáng trả tiền.

KIẾN THỨC CẦN CÓ TRƯỚC

- **clip** — một đoạn video ngắn gồm T khung hình liên tiếp (hoặc lấy mẫu thưa), là đơn vị mà mô hình video nhận vào — tương đương “một ảnh” của mô hình ảnh.
- **kích hoạt (activation)** — tensor giá trị trung gian mà mỗi tầng mạng sinh ra — khác với *hàm kích hoạt*, là phép phi tuyến áp lên nó. Khi huấn luyện phải giữ các tensor này trong bộ nhớ để tính gradient ở *lượt ngược* (*backward pass*), nên chúng — chứ không phải tham số — thường là thứ làm tràn bộ nhớ.
- **batch size** — số mẫu xử lý cùng lúc trong một bước cập nhật. Batch lớn cho ước lượng gradient ổn định hơn nhưng tốn bộ nhớ tỉ lệ thuận.
- **batch normalization** — phép chuẩn hoá kích hoạt bằng trung bình và phương sai *tính trong batch*. Batch quá nhỏ thì hai thống kê này rất nhiễu, nên mô hình video hay phải đổi sang chuẩn hoá theo nhóm hoặc theo tầng.
- **FLOPs** — số phép tính dấu phẩy động cần cho một lần chạy; dùng để so chi phí tính toán độc lập với phần cứng. Nó không tỉ lệ chặt với thời gian chạy thật, vì bộ nhớ mới hay là nút thắt.

- **optical flow** — trường vector cho biết mỗi pixel dịch đi đâu giữa hai khung hình liên tiếp — biểu diễn tường minh của chuyển động, dùng làm đầu vào cho nhánh chuyển động của kiến trúc hai luồng.
- **token và chi phí bậc hai** — Transformer chia đầu vào thành n token rồi tính quan hệ giữa mọi cặp, nên chi phí tỉ lệ n^2 . Với video, n nhân thêm số khung hình, và n^2 nhân thêm bình phương số đó.
- **inductive bias** — giả định cài sẵn trong kiến trúc, ví dụ tích chập giả định tính cục bộ và **đẳng biến tịnh tiến** (*translation equivariance*). Bỏ bớt giả định thì mô hình linh hoạt hơn nhưng cần nhiều dữ liệu hơn để tự học lại điều đã bị bỏ.

Đặt vấn đề

Cách hiển nhiên nhất để xử lý video là coi nó như một khối ba chiều và dùng **tích chập ba chiều** (*3D convolution*) thay cho tích chập hai chiều. Cách này đúng về mặt hình thức và sai về mặt kinh tế: số phần tử phải giữ trong bộ nhớ nhân với số khung hình, số tham số của mỗi kernel nhân với chiều dài thời gian, và ngân sách vốn đã chật của một mô hình ảnh lập tức vỡ. Vì thế toàn bộ lĩnh vực này không phải là câu chuyện “kiến trúc nào tốt hơn” mà là câu chuyện “mua thông tin thời gian bằng cách nào cho rẻ”.

Nhưng còn một lý do sâu hơn khiến ta không nên coi thời gian là trục không gian thứ ba. Trục thời gian có *hướng*: đảo ngược nó thì “mở cửa” thành “đóng cửa”, trong khi lật ngược trục x của một ảnh vẫn cho một ảnh hợp lệ của cùng một cảnh. Trục thời gian cũng có *đơn vị khác*: 16 khung hình ở 30 Hz là nửa giây, còn 16 pixel là một chi tiết nhỏ — không có lý do gì để cùng một kernel $3 \times 3 \times 3$ là kích thước hợp lý ở cả ba trục. Hai bất đối xứng này là nguyên nhân sâu xa của mọi thiết kế tách trục mà ta sẽ gặp.

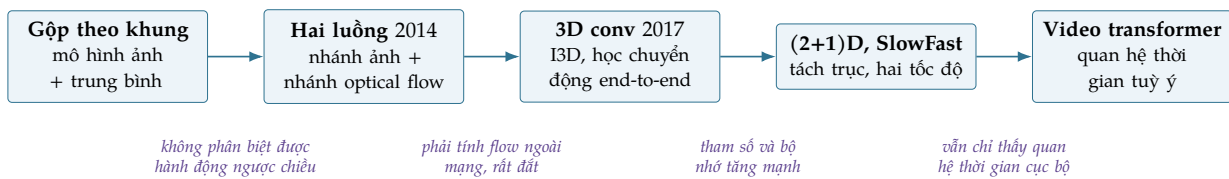
Học xong mục này bạn làm được gì mà trước đó không làm được? Cầm một yêu cầu như “phân loại hành động trên clip 4 giây, chạy được trên GPU 8 GB”, bạn suy ra ngay cấu hình khả thi và thứ phải hy sinh. Không phải thử ngẫu nhiên tới khi hết bộ nhớ.

TRỰC GIÁC

Ngân sách bộ nhớ của video giống một tấm chăn ngắn kéo theo ba hướng: độ dài clip, độ phân giải, và batch size. Tích của ba đại lượng đó gần như là một hằng số do phần cứng ấn định, nên mọi quyết định kiến trúc thực chất là quyết định kéo chăn về phía nào. Điều đáng nói là ba hướng ấy không tương đương về mặt khoa học: giảm batch làm hỏng thống kê của **chuẩn hoá theo batch** (*batch normalization*), giảm độ phân giải làm mất vật nhỏ, còn giảm độ dài clip làm mất chính thứ ta đang muốn học. Không có hướng nào miễn phí, và biết trước điều đó tiết kiệm hàng tuần thí nghiệm.

→ **ràng buộc bởi B/11** — tích độ dài clip $\times$ độ phân giải² $\times$ batch bị chặn bởi bộ nhớ thiết bị — cùng một trần đã chi phối mọi lựa chọn ở chương giới hạn tính toán.

A. Bốn họ biểu diễn, và nút thắt mà mỗi họ gỡ



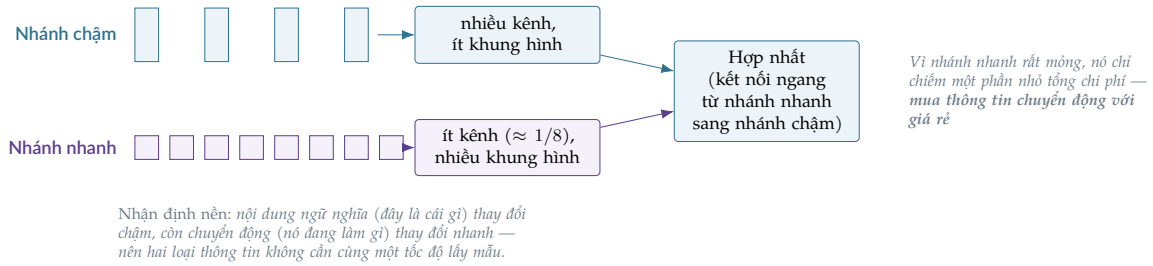
Hình 15.5. Dòng thời gian biểu diễn video, đọc theo nút thắt được gỡ. Nhân tím dưới mỗi mũi tên là *vấn đề của bước trước* — đó là thứ khiến bước sau ra đời. Chú ý rằng đây không phải một thang chất lượng: bước đầu tiên vẫn là baseline mạnh và bắt buộc phải chạy.

Hình 15.5 cho thấy mạch phát triển, còn Bảng 15.3 cho thấy cái mà mạch đó che đi: mỗi họ giả định một điều khác nhau về bản chất của chuyển động, và chính giả định — chứ không phải số tầng — quyết định nó hỏng ở đâu.

Bảng 15.3. Năm cách biểu diễn video. Cột chi phí ghi theo bậc độ lớn tương đối so với một mô hình ảnh chạy trên một khung hình, không phải số đo thực nghiệm.

Cách	Giả định	Chi phí	Được	Hỏng khi nào
Mỗi khung + gộp	Thời gian không quan trọng lắm	$\sim T \times$	Rẻ, tái dùng nguyên mô hình ảnh	Không phân biệt được hành động ngược chiều (mở/đóng, nhặt/đặt)
Hai luồng	Chuyển động tách được thành một nhánh riêng [161]	$\sim 2T \times + \text{flow}$	Rõ ràng, hiệu quả với ít dữ liệu	Phải tính optical flow ngoài mạng; flow sai thì nhánh đó thành nhiễu
3D conv	Không-thời gian đối xứng [162]	$\sim 3T \times$	Học chuyển động đầu-cuối	Tham số và bộ nhớ tăng mạnh; giả định đối xứng vốn sai
(2+1)D	Trục thời gian tách được khỏi trục không gian [163]	$\sim 1,5T \times$	Cùng số tham số nhưng thêm một hàm kích hoạt, dễ tối ưu hơn	Chuyển động thực sự xoắn không-thời gian (xoáy, biến dạng nhanh)
Video transformer	Quan hệ thời gian tùy ý, không cần cục bộ	bậc hai theo số token	Mạnh nhất khi đủ dữ liệu	Chi phí bậc hai theo T ; rất đói dữ liệu

Một họ nữa nằm ngoài bảng vì nó là một ý tưởng chéo chứ không phải một loại tích chập: **SlowFast** chạy *hai* nhánh trên cùng video — một nhánh tốc độ khung thấp nhưng nhiều kênh để nắm ngữ nghĩa, một nhánh tốc độ khung cao nhưng ít kênh để nắm chuyển động [164]. Ý nghĩa của thiết kế này: *nội dung ngữ nghĩa thay đổi chậm, còn chuyển động thay đổi nhanh, nên hai loại thông tin không cần cùng một tốc độ lấy mẫu*. Nhánh nhanh rất mỏng: bài báo dùng khoảng một phần tám số kênh. Nó vì thế chỉ chiếm một phần nhỏ tổng chi phí. Đây là ví dụ đẹp của việc dùng cấu trúc bài toán để mua thông tin rẻ hơn.



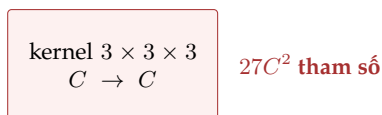
Hình 15.6. SlowFast dùng cấu trúc của tín hiệu để mua thông tin rẻ hơn. Thay vì tăng đồng loạt tần số lấy mẫu cho toàn mạng — cách tốn kém nhất — nó tách thành hai nhánh có tần số và bề dày kênh ngược nhau, rồi hợp nhất. Nếu nhận định nền sai với dữ liệu của bạn (ngữ nghĩa thay đổi nhanh, chẳng hạn cảnh cắt dựng liên tục) thì lợi thế chi phí biến mất.

Hình 15.6 đáng đặt cạnh Hình 15.7: cả hai đều là ví dụ của cùng một chiến lược — đọc ra một bất đối xứng thật của bài toán rồi đưa nó vào kiến trúc, thay vì trả tiền đồng đều cho mọi chiều.

B. Ba phép tính quyết định mọi thứ

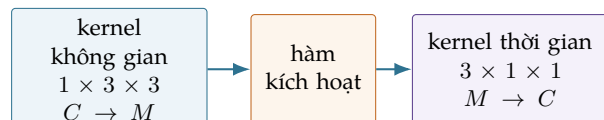
- **Bộ nhớ của một tensor kích hoạt.** Một clip 16 khung, 224×224 , 64 kênh, fp32: $16 \times 224 \times 224 \times 64 \times 4 \text{ byte} \approx 205 \text{ MB}$ — cho một tensor, ở batch bằng 1. Lướt ngược phải giữ kích hoạt của mọi tầng, nên con số này nhân với số tầng nông. Đây là lý do thực tế vì sao mô hình video hầu như luôn chạy ở batch nhỏ, và vì sao chuẩn hoá theo batch trở thành vấn đề (xem [C/12-co-che-huan-luyen/02-lam-cho-training-chay-duoc](#)).
- **Vì sao (2+1)D không tốn thêm tham số.** Một kernel 3D $3 \times 3 \times 3$ từ C kênh sang C kênh có $27C^2$ tham số. Bản tách gồm hai kernel. Kernel không gian $1 \times 3 \times 3$ đi từ C sang M kênh, tốn $9CM$; kernel thời gian $3 \times 1 \times 1$ đi từ M về C , tốn $3MC$. Tổng là $12CM$. Cho hai vế bằng nhau: $M = 2,25C$. Ý nghĩa: với cùng ngân sách tham số, ta mua được một hàm kích hoạt bổ sung ở giữa và một bài toán tối ưu dễ hơn — lợi ích đến từ dạng của hàm, không từ sức chứa (capacity) [163].

Tích chập 3D



Một khối duy nhất trộn cả ba trục, ngầm giả định thời gian đối xứng với không gian

Bản tách (2+1)D



tổng $12CM$; cho bằng $27C^2$ thì $M = 2,25C$

Cùng ngân sách tham số, nhưng mua thêm được một hàm kích hoạt ở giữa — lợi ích đến từ dạng của hàm, không từ sức chứa

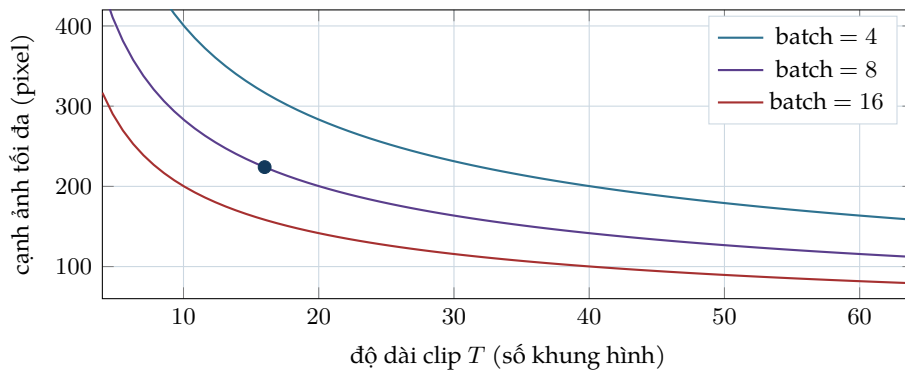
Hình 15.7. Phân rã (2+1)D vẽ theo khối. Kernel 3D duy nhất được tách thành một kernel chỉ tác động theo không gian rồi một kernel chỉ tác động theo thời gian, với số kênh trung gian M chọn sao cho tổng tham số bằng bản gốc. Phép tách phản ánh đúng bất đối xứng của bài toán: trục thời gian có hướng và có đơn vị khác, nên không có lý do để cùng một kernel đối xứng là hợp lý ở cả ba trục.

Hình 15.7 làm rõ vì sao đây không phải mẹo tiết kiệm: nếu chỉ để tiết kiệm thì người ta đã giảm số kênh, chứ không giữ nguyên tham số rồi chèn thêm một phi tuyến.

- **Vì sao attention trên video đắt tới mức đó.** Một ViT chia ảnh 224×224 thành $14 \times 14 = 196$ token, nên ma trận attention có $196^2 \approx 3,8 \times 10^4$ phần tử. Với clip 16 khung, số token thành 3136 và ma trận thành $9,8 \times 10^6$ — gấp 256 lần, đúng bằng 16^2 .

↪ ràng buộc bởi C/13/03 — video transformer bỏ thêm một tầng thiên lệch quy nạp nữa — lần này theo trục thời gian — nên nó đối dữ liệu hơn cả ViT trên ảnh.

Ý nghĩa: chi phí bậc hai theo số token nghĩa là bậc hai theo độ dài clip, nên mọi video transformer thực dụng đều phải phân tách attention (theo thời gian rồi theo không gian) hoặc giới hạn cửa sổ.



Hình 15.8. Đường đẳng ngân sách bộ nhớ. Vì lượng kích hoạt tỉ lệ với $T \cdot R^2 \cdot B$, một ngân sách cố định cho $R \propto 1/\sqrt{TB}$. Điểm xanh đậm là cấu hình tham chiếu (16 khung, 224 px, batch 8). Hình cho thấy điều mà một bảng số không cho thấy: muốn gấp đôi độ dài clip, bạn chỉ phải giảm cạnh ảnh khoảng 30%, nhưng muốn gấp bốn thì phải giảm một nửa — và đường cong dốc rất nhanh ở phía trái.

Hình 15.8 nên dùng như một công cụ chứ không phải một minh họa: khi ai đó đề nghị “tăng độ dài clip lên 64 khung”, hãy chỉ ngay vào đường cong và hỏi họ định trả bằng độ phân giải hay bằng batch.

C. Giới hạn và trường hợp hỏng

- **Rất nhiều tập dữ liệu không thực sự cần thời gian.** Nhiều tập dữ liệu hành động phổ biến có thể đoán đúng phần lớn chỉ từ *bối cảnh* của một khung hình — thấy bể bơi thì đoán “bơi”. Đây là quan sát được lặp lại rộng rãi trong lĩnh vực, và hệ quả là một mô hình thời gian tinh vi có thể trông tốt vì lý do hoàn toàn khác với lý do bạn nghĩ.
- **Optical flow là một phụ thuộc ngoài mạng.** Nhánh chuyển động của kiến trúc hai luồng đòi hỏi flow được tính trước, thường tốn hơn cả mô hình chính. Chất lượng flow khi đó thành trần cho toàn hệ thống — đúng mẫu “đẩy độ khó sang chỗ khác”.
- **Batch nhỏ phá thống kê chuẩn hoá.**
 ⇔ hệ quả của C/12/02 — chuẩn hoá theo batch ước lượng trung bình và phương sai từ chính batch, nên batch nhỏ làm ước lượng nhiễu — đúng cơ chế đã bàn ở phần làm cho huấn luyện chạy được.
 Mô hình video thường chạy batch 2–8 trên mỗi GPU, nên batch norm cho ước lượng phương sai rất nhiễu; đây là lý do các kiến trúc video hay dùng chuẩn hoá theo nhóm hoặc theo tầng [111].
- **Lấy mẫu khung là một siêu tham số ẩn.** Lấy 16 khung liên tiếp, hay 16 khung rải đều trên 4 giây, là hai bài toán khác nhau; và cách lấy mẫu lúc đánh giá thường khác lúc huấn luyện, nên số công bố phụ thuộc vào một chi tiết ít khi được nêu rõ.

CẠM BẮY

Trước khi so sánh bất kỳ hai kiến trúc video nào, hãy chạy baseline rẻ nhất: lấy một khung hình ngẫu nhiên, chạy mô hình ảnh, rồi trung bình dự đoán trên vài khung. Nếu baseline này đạt gần bằng mô hình thời gian, thì tập dữ liệu của bạn không đo cái bạn tưởng nó đo, và mọi so sánh giữa các mô hình thời gian phía trên đều là nhiễu. Đây là bài kiểm tra rẻ nhất và bị bỏ qua nhiều nhất trong cả lĩnh vực.

CẦN NHỚ

Ngân sách bộ nhớ của video chia cho ba chiều nhân với nhau: độ dài clip $\times$ độ phân giải² $\times$ batch. Mọi lựa chọn kiến trúc vì thế là lựa chọn hy sinh chiều nào. Và trước khi trả tiền cho chiều thời gian, hãy chứng minh rằng bài toán của bạn thật sự cần nó, bằng baseline một khung hình. [T2]

D. Kết nối

[B/11-gioi-han-tinh-toan](#) là nơi ngân sách bộ nhớ và băng thông được xử lý tổng quát; mục này chỉ là trường hợp riêng nhưng khắc nghiệt nhất. [C/13-kien-truc-backbone/03-vit-va-inductive-bias](#) giải thích vì sao video transformer đối dữ liệu: bỏ inductive bias theo trục thời gian còn tốn dữ liệu hơn bỏ theo trục không gian. [C/15-thi-giac-khong-thoi-gian/03-flow-tracking-va-rang-buoc](#) tiếp nối trực tiếp bằng nhánh optical flow — thứ mà kiến trúc hai luồng dựa vào — và bằng ràng buộc nhân quả, thứ loại bỏ phần lớn các kiến trúc trong bảng trên khi bạn phải chạy streaming.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao coi thời gian như trục không gian thứ ba là một giả định sai, và sai đó biểu hiện thành lỗi gì trong thực tế?
2. Tính lượng bộ nhớ cho một tensor kích hoạt clip 32 khung, 112×112 , 128 kênh, fp16. Nếu vượt ngân sách, bạn cắt chiều nào trước và vì sao?
3. $(2+1)D$ có cùng số tham số với 3D conv nhưng thường tốt hơn. Lợi ích đó đến từ đâu, nếu không phải từ sức chứa của mô hình?
4. Vì sao chi phí attention trên clip 16 khung gấp 256 lần trên một ảnh, chứ không phải 16 lần?
5. SlowFast dựa trên nhận định nào về bản chất của tín hiệu? Nếu nhận định đó sai với dữ liệu của bạn thì kiến trúc hồng thế nào?
6. Baseline “một khung hình rồi gộp” đạt 92% so với mô hình 3D đạt 94%. Bạn kết luận gì, và bước tiếp theo là gì?

15.3 Flow, tracking và các ràng buộc riêng (Optical Flow, Tracking and Their Constraints)

PHỤ THUỘC [C/15-thi-giac-khong-thoi-gian/02-bieu-dien-video](#) (ngân sách bộ nhớ); [B/07-chia-du-lieu-va-danh-gia](#) (rõ ràng dữ liệu, chọn metric); [A/01-toan-toi-thieu](#) (bình phương tối thiểu, điều kiện của ma trận).

TÓM TẮT

Mục này bàn ba thứ mà một kỹ sư SLAM đã sống cùng hằng ngày, chỉ dưới tên khác: **dòng quang** (*optical flow*) là bài toán tương ứng điểm, **bám nhiều mục tiêu** (*multi-object tracking*) là bài toán liên kết dữ liệu, và ràng buộc nhân quả là ràng buộc cứng của mọi hệ chạy thời gian thực. Đọc xong bạn ánh xạ được các kỹ thuật học sâu ở đây về đúng chỗ tương ứng trong pipeline hình học của mình. Bạn chọn được metric tracking không đánh lừa. Và bạn biết kiến trúc nào bị loại ngay từ đầu khi không được nhìn khung hình tương lai. Nếu chỉ nhớ một điều: trong cả flow lẫn tracking, chỗ hệ thống sống hay chết là bước *liên kết* chứ không phải bước phát hiện — và cả hai lĩnh vực đều có một metric phổ biến che giấu đúng chỗ đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **optical flow** — trường vector (u, v) cho biết mỗi pixel của khung này nằm ở đâu trong khung kia. “Dày đặc” nghĩa là tính cho mọi pixel, khác với việc chỉ bám vài trăm đặc trưng.
- **gradient ảnh** I_x, I_y, I_t — tốc độ thay đổi độ sáng theo trục ngang, trục dọc và theo thời gian tại một pixel. Ba đại lượng này là toàn tập dữ liệu mà phương trình flow cổ điển có.
- **bài toán khẩu độ (aperture problem)** — khi chỉ nhìn qua một cửa sổ nhỏ, chỉ thành phần chuyển động vuông góc với cạnh là quan sát được; thành phần dọc theo cạnh hoàn toàn vô hình. Đây là thiếu thông tin thật, không phải lỗi thuật toán.
- **liên kết dữ liệu (data association)** — bước quyết định phép đo nào ở khung này ứng với đối tượng nào đã biết. Trong tracking đó là ghép hộp với track; trong SLAM đó là ghép đặc trưng với điểm bản đồ. Gán sai gây hậu quả nặng và khó rút lại.
- **track ID** — mã số mà hệ thống gán cho một đối tượng và phải giữ nguyên suốt thời gian nó xuất hiện. *ID switch* là khi mã số bị đánh tráo giữa hai đối tượng — lỗi liên kết điển hình.
- **bộ lọc Kalman** — bộ ước lượng đệ quy giữ vị trí và vận tốc cùng độ bất định của chúng; nó dự đoán vị trí khung tới rồi cập nhật khi có phép đo mới. Trong tracking, nó cung cấp dự đoán để ghép cặp.
- **thuật toán Hungary** — thuật toán giải bài toán gán tối ưu: cho ma trận chi phí giữa m dự đoán và n phát hiện, tìm phép ghép một-một có tổng chi phí nhỏ nhất.
- **MOTA và HOTA** — hai thước đo tracking. MOTA cộng số phát hiện sót, phát hiện thừa và số lần đổi mã rồi chia cho tổng số vật thật. HOTA tách riêng chất lượng phát hiện (DetA) và chất lượng liên kết (AssA) rồi lấy trung bình nhân của chúng.
- **nhân quả / streaming** — ràng buộc rằng quyết định tại thời điểm t chỉ được dùng dữ liệu tới t , không được dùng khung hình tương lai. Nó loại bỏ mọi kiến trúc hai chiều và mọi phép làm mượt nhìn về phía trước.

Đặt vấn đề

Hai mục trước xử lý video như một khối cần phân loại. Nhưng phần lớn ứng dụng thật lại cần thứ tinh hơn nhiều: *cùng một điểm vật lý ở khung này nằm ở đâu trong khung kia*. Trả lời câu đó cho mọi pixel là optical flow; trả lời cho từng vật thể qua thời gian là tracking; và trong cả hai trường hợp, khó khăn thật không nằm ở việc nhìn thấy mà ở việc *ghép cặp*. Đây chính xác là cấu trúc của bài toán mà một hệ SLAM giải mỗi khung hình, nên mục này ít giới thiệu khái niệm mới hơn là dịch từ vựng.

Thêm vào đó là một ràng buộc mà ảnh tĩnh không có: mô hình chạy trực tuyến không được nhìn khung hình tương lai, và cũng không được nhìn quá lâu vào quá khứ nếu ngân sách mili-giây có hạn. Ràng buộc này loại bỏ nguyên một lớp kiến trúc trước khi độ chính xác kịp lên tiếng — đúng nghĩa **ràng buộc cứng ở B/03-giai-phau-he-thong**.

Học xong mục này bạn làm được gì mà trước đó không làm được? Bạn đọc một paper flow hay tracking và biết ngay nó thay khối nào trong pipeline hình học quen thuộc, nó mua độ bền bằng giả định gì, và nó có chạy được dưới ràng buộc nhân quả của bạn hay không.

TRỰC GIÁC

Nhìn một cạnh thẳng đang chuyển động qua một cái ống hút: bạn chỉ thấy được thành phần chuyển động *vuông góc* với cạnh, còn thành phần dọc theo cạnh thì hoàn toàn vô hình. Đó không phải hạn chế của mắt hay của thuật toán, mà là thiếu thông tin thật sự: một phương trình, hai ẩn. Mọi thứ trong optical flow — lấy cửa sổ, làm trơn, kim tự tháp, khối tương quan — đều là các cách khác nhau để mượn thêm ràng buộc từ nơi khác. Và ma trận đo mức độ “mượn được hay không” chính là ma trận mà bạn đã dùng để chọn đặc trưng góc trong SLAM.

A. Optical flow: từ Lucas–Kanade tới RAFT

Giả định gốc là **độ sáng không đổi** (*brightness constancy*): cùng một điểm vật lý giữ nguyên giá trị sáng giữa hai khung. Viết $I(x, y, t) = I(x + u, y + v, t + 1)$ rồi khai triển Taylor bậc nhất cho phương trình ràng buộc

$$I_x u + I_y v + I_t = 0.$$

Câu này nói gì bằng lời: *tốc độ thay đổi độ sáng theo thời gian tại một pixel phải bằng đúng phần độ sáng bị “trôi vào” do chuyển động*. Một phương trình, hai ẩn — đó là **bài toán khẩu độ** (*aperture problem*) ở dạng đại số.

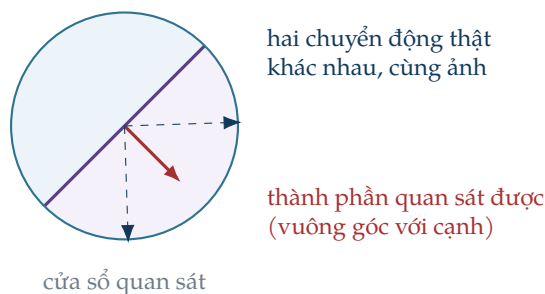
Lucas–Kanade mượn ràng buộc bằng cách giả định flow không đổi trong một cửa sổ nhỏ, cho N phương trình và hai ẩn, rồi giải bình phương tối thiểu. Phương trình chuẩn có ma trận hệ số

$$A^T A = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix},$$

và đây đúng là ma trận cấu trúc trong tiêu chuẩn chọn đặc trưng góc. Ý nghĩa: bài toán giải được khi và chỉ khi cửa sổ có gradient theo *hai* hướng độc lập. Ví dụ nhỏ nhất: một cửa sổ nằm trên một cạnh dọc lý tưởng có $I_x = g$ và $I_y = 0$ ở mọi pixel, nên $A^T A = \begin{bmatrix} Ng^2 & 0 \\ 0 & 0 \end{bmatrix}$ — suy biến, giá trị riêng nhỏ nhất bằng 0, và thành phần flow dọc theo cạnh không xác định.

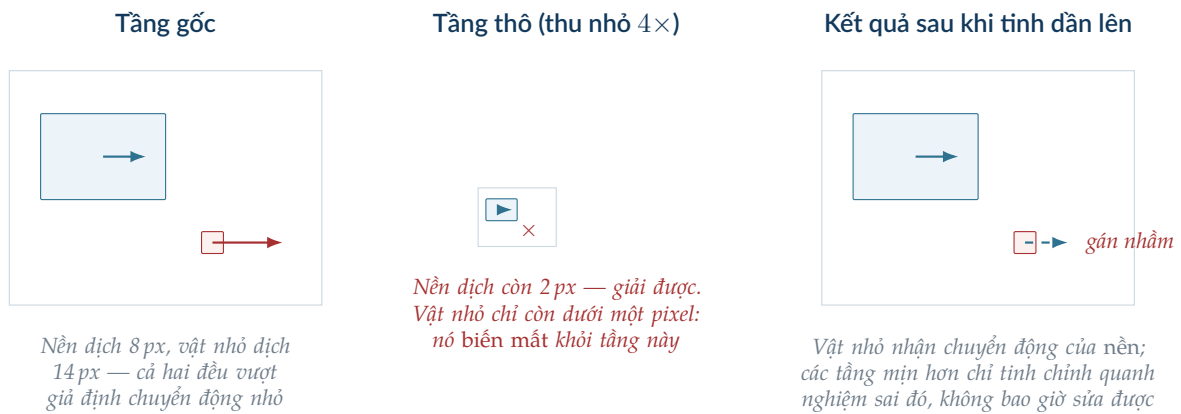
→ **tiền đề A/01** — đây đúng là câu hỏi điều kiện của một hệ bình phương tối thiểu: nghiệm ổn định khi giá trị riêng nhỏ nhất đủ xa 0, và suy biến khi nó bằng 0.

Chọn “good features to track” chính là chọn các cửa sổ mà giá trị riêng nhỏ nhất của ma trận này đủ lớn; nói cách khác, việc chọn đặc trưng trong SLAM và tính giải được của flow là *cùng một điều kiện*.



Hình 15.9. Bài toán khẩu độ. Hai chuyển động thật rất khác nhau (mũi tên đứt) tạo ra cùng một hình ảnh bên trong cửa sổ, vì chỉ thành phần vuông góc với cạnh là quan sát được. Về đại số, đó là việc ma trận $A^T A$ suy biến; về thực hành, đó là lý do mọi bộ bám đặc trưng đều phải chọn góc chứ không chọn cạnh.

Hình 15.9 cũng giải thích luôn giả định thứ ba của Lucas–Kanade — *chuyển động nhỏ* — vì khai triển Taylor bậc nhất chỉ đúng khi dịch chuyển nhỏ hơn cỡ một pixel gradient. Cách chữa cổ điển là kim tự tháp thô–tối–mịn: giảm ảnh xuống bốn lần thì chuyển động 8 pixel thành 2 pixel. Cái giá là một chế độ hồng đặc trưng: vật *nhỏ* và *nhạy* biến mất ở tầng thô, nên hệ thống ước lượng chuyển động của nền thay vì của vật, và sai lầm đó không bao giờ được sửa ở các tầng mịn hơn.



Hình 15.10. Chế độ hồng đặc trưng của chiến lược thô-tới-mịn. Thu nhỏ ảnh làm chuyển động lớn trở nên nhỏ — đó là mục đích — nhưng nó cũng làm vật nhỏ biến mất trước khi kịp được ước lượng. Vì mỗi tầng chỉ *tinh chỉnh* nghiệm của tầng thô hơn, một sai lầm ở tầng thô là sai lầm vĩnh viễn. Đây chính là lý do RAFT bỏ kim tự tháp và tính tương quan toàn cặp một lần.

Hình 15.10 nên đọc như một cảnh báo quen thuộc với người làm bám đặc trưng: mọi sơ đồ nhiều tầng chỉ tinh chỉnh chứ không sửa lại đều thừa hưởng đúng chế độ hồng này, kể cả bộ tối ưu bạn tự viết.

Bảng 15.4. Bốn thể hệ optical flow. Cột cuối là cột đáng giá nhất: mỗi phương pháp mượn ràng buộc từ một nguồn khác nhau, nên nó hồng khi đúng nguồn đó cạn.

Phương pháp	Mượn ràng buộc từ đâu	Mạnh khi nào	Hồng khi nào
Lucas–Kanade	Giả định flow hằng trong cửa sổ nhỏ	Chuyển động nhỏ, kết cấu giàu, rất rẻ	Vùng trơn, cạnh dài, chuyển động lớn; thay đổi độ sáng phá giả định gốc
LK + kim tự tháp	Thêm giả định trơn qua các tỉ lệ	Chuyển động lớn của vùng lớn	Vật nhỏ chuyển động nhanh biến mất ở tầng thô
FlowNet, PWC-Net	Học từ dữ liệu; PWC-Net đưa kim tự tháp, warping, khối chi phí thành kiến trúc [165], [166]	Cân bằng tốt giữa chính xác và tốc độ	Lệch miền so với dữ liệu tổng hợp dùng để huấn luyện
RAFT	Khối tương quan <i>toàn cặp</i> + tinh chỉnh lặp bằng GRU [167]	Chuyển động lớn, biên sắc, khái quát tốt	Bộ nhớ của khối tương quan; số vòng lặp là ngân sách phải tự chọn

RAFT đáng chú ý riêng vì hai lý do rất gần với kinh nghiệm tối ưu phi tuyến. Thứ nhất, nó bỏ chiến lược thô-tới-mịn bằng cách tính tương quan giữa *mọi* cặp pixel một lần, nhờ đó chuyển động lớn không còn cần kim tự tháp. Cái giá đo được: ở độ phân giải 1/8 của ảnh 1024×436 , lưới là $128 \times 54 \approx 6900$ điểm, nên khối toàn cặp có $6900^2 \approx 4,8 \times 10^7$ phần tử — khoảng 190 MB ở fp32, trước khi tính bất cứ thứ gì. Thứ hai, nó tinh chỉnh nghiệm bằng một số vòng lặp cố định thay vì giải một lần; điều này khiến *số vòng lặp* trở thành *núm vặn độ trễ*, đúng như việc bạn giới hạn số bước Gauss–Newton trong một bộ tối ưu thời gian thực. Đây là chỗ nổi đáng nhớ: mô hình học sâu ở đây không thay thế tư duy tối ưu lặp, nó mượn lại tư duy đó.

B. Nhấn nằm trên khoảng thời gian, không nằm trên khung hình

Nhận dạng hành động trên clip đã cất sẵn là bài toán dễ; *temporal action localization* mới là bài toán thật: cho một video dài, tìm *khoảng* nào chứa hành động gì. Việc nhấn nằm trên khoảng kéo theo hai hệ quả mà người quen bài toán ảnh dễ vấp.

- **Metric phải đổi.** Ta không đánh giá theo khung hình mà theo khoảng, với ngưỡng *temporal IoU*. Ranh giới hành động vốn mơ hồ ngay cả với người gán nhãn — “bắt đầu ngồi xuống” là khung nào? — nên độ chính xác ở ngưỡng tIoU chặt phản ánh sự bất đồng của người gán nhãn nhiều hơn phản ánh chất lượng mô hình.
- **Chia dữ liệu phải theo video, không theo clip.** Hai clip cắt từ cùng một video có nền, ánh sáng và diễn viên giống nhau; để chúng nằm hai bên tập train/test là rò rỉ dữ liệu kinh điển, và nó thối phồng kết quả một cách rất khó phát hiện.

↪ **trường hợp riêng của B/07** — hai clip cắt từ cùng một video là một nhóm phụ thuộc; chia ngẫu nhiên theo clip là rò rỉ dữ liệu kinh điển, chỉ đổi trục từ không gian sang thời gian.

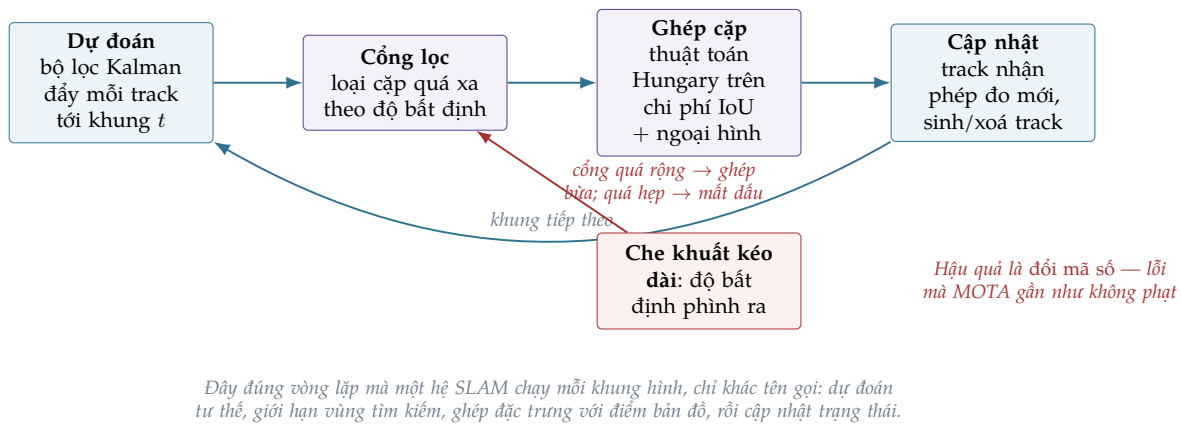
C. Tracking chính là liên kết dữ liệu

Có hai triết lý. *Tracking-by-detection* tách hẳn hai việc: một detector chạy độc lập trên mỗi khung, rồi một khối liên kết nối các phát hiện qua thời gian bằng một mô hình chuyển động cộng một độ đo tương đồng. *Joint detection-tracking* huấn luyện cả hai cùng lúc, mạnh hơn trên giấy nhưng khó gỡ lỗi vì không còn ranh giới rõ để đặt điểm dừng. Với hệ thống sản xuất, cách thứ nhất thường thắng vì lý do kỹ thuật hệ thống chứ không phải vì độ chính xác: bạn thay được detector mà không phải huấn luyện lại phần liên kết.

Khối liên kết là chỗ mà người làm SLAM sẽ thấy quen tới mức bất ngờ. SORT dùng đúng bộ đôi kinh điển — bộ lọc Kalman dự đoán vị trí khung tới, rồi thuật toán Hungary ghép cặp dự đoán với phát hiện theo chi phí IoU [168]. ByteTrack thêm một quan sát đơn giản mà hiệu quả: đừng vứt các phát hiện điểm thấp, hãy dùng chúng ở vòng ghép thứ hai, vì một vật bị che một phần thường cho điểm thấp chứ không phải không có [169]. Về mặt cấu trúc, đây là cùng bài toán **liên kết dữ liệu** (*data association*) mà một hệ SLAM giải khi ghép đặc trưng với điểm bản đồ: có dự đoán từ mô hình chuyển động, có cổng lọc theo khoảng cách thống kê, có một bài toán gán tối ưu, và có hậu quả nghiêm trọng khi gán sai.

↪ **cùng nguyên lý với C/16** — ghép hộp với track và ghép đặc trưng với điểm bản đồ là cùng một bài toán gán tối ưu có cổng lọc theo độ bất định; chỉ khác hậu quả khi gán sai.

Khác biệt lớn nhất là ở SLAM, một liên kết sai làm hỏng bản đồ và rất khó rút lại, trong khi ở tracking nó chỉ đổi một mã số — nhưng chính vì hậu quả nhẹ hơn nên metric mới dễ che giấu nó.



Hình 15.11. Tracking là một vòng lặp liên kết dữ liệu, không phải một mạng nơ-ron. Ba trong bốn khối không chứa tham số học được nào; phần học được nằm ở detector cấp phép đo và ở đặc trưng ngoại hình dùng làm chi phí ghép. Vì thế cải thiện detector và cải thiện liên kết là hai việc khác nhau, và một thước đo gộp chúng vào một số sẽ che mất việc nào đang hỏng.

Hình 15.11 là lý do hộp cam bẫy ngay dưới đây quan trọng đến vậy: nếu vòng lặp này hỏng mà thước đo không phản ánh, bạn sẽ dành công sức tối ưu nhầm khối.

CẠM BÃY

MOTA cộng ba loại lỗi rồi chia cho tổng số vật thể thật: $MOTA = 1 - (FN + FP + IDSW) / GT$. Vấn đề nằm ở việc FN và FP được đếm *trên từng khung hình cho từng vật*, còn mỗi lần đổi mã số chỉ được đếm *một lần*. Ví dụ: 1000 vật–khung, 50 FN, 30 FP, 40 lần đổi mã cho $MOTA = 0,88$; cải thiện khối liên kết để chỉ còn 20 lần đổi mã — tức tốt lên gấp đôi — chỉ nâng MOTA lên 0,90. Nói cách khác, một hệ thống liên tục đánh tráo danh tính vẫn có thể khoe MOTA cao. HOTA sửa điều này bằng cách tách riêng chất lượng phát hiện và chất lượng liên kết, rồi lấy trung bình nhân $\sqrt{DetA \cdot AssA}$. Hai thành phần khi đó có trọng số ngang nhau, và không thành phần nào bù đắp được cho thành phần kia [68], [170].

D. Nhất quán thời gian và ràng buộc nhân quả

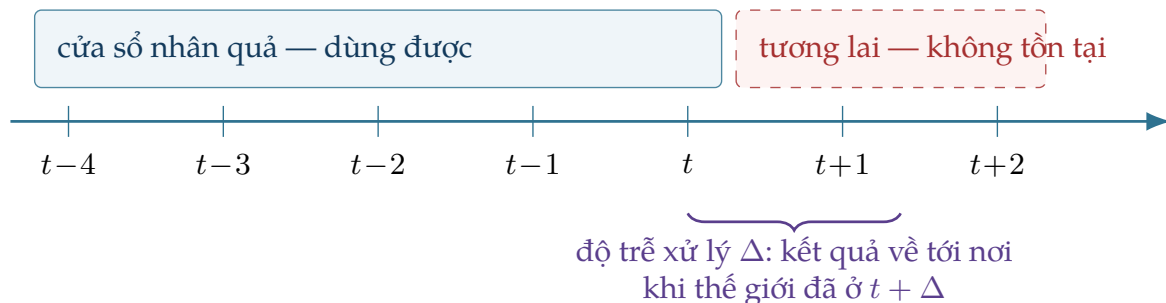
Một mô hình phân vùng chạy độc lập trên từng khung có thể đạt mIoU cao mà vẫn cho video nhấp nháy. Lý do là hàm mất mát không có số hạng nào phạt việc dự đoán đổi qua đổi lại giữa hai khung gần như giống nhau. Ba cách xử lý, với ba mức chi phí khác nhau:

Bảng 15.5. Ba cách chữa hiện tượng nhấp nháy, xếp theo mức xâm lấn vào hệ thống.

Cách	Cơ chế	Chi phí	Hỏng khi nào
Làm mượt hậu xử lý	Lọc dự đoán qua vài khung, có thể warp bằng flow	Rẻ, nhưng thêm độ trễ	Chuyển cảnh và chuyển động nhanh: làm mượt thành làm nhòe
Đưa thời gian vào mô hình	Mô hình nhận nhiều khung, hoặc mang trạng thái ẩn	Đắt nhất, chạm vào ngân sách ở Mục 2	Trạng thái ẩn có thể “dính” vào một sai lầm và tự củng cố
Loss nhất quán thời gian	Phạt sai khác giữa dự đoán của hai khung liên tiếp sau khi warp	Trung dung, chỉ tốn lúc huấn luyện	Warp dựa vào flow, nên flow sai thì loss dạy điều sai

↔ **ràng buộc bởi C/15/02** — cách chữa “đưa thời gian vào mô hình” nghe hấp dẫn nhất nhưng chính là cách đắt nhất, vì nó tiêu vào đúng ngân sách ba chiều đã cạn ở mục trước.

Cả ba đều bị một ràng buộc lớn hơn chi phối: hệ thống chạy trực tuyến *không được nhìn khung hình tương lai*. Ràng buộc này loại thẳng mọi kiến trúc hai chiều, mọi phép làm mượt trung tâm, và mọi lịch trình đánh giá “xem toàn bộ video rồi mới quyết định” — và nó loại chúng trước khi độ chính xác được đem ra bàn.



Hình 15.12. Ràng buộc nhân quả, vẽ trên trục thời gian. Vùng xanh là toàn bộ thông tin mà một hệ streaming được phép dùng để quyết định tại thời điểm t ; vùng đỏ là thứ mà mọi mô hình hai chiều lạng lẽ giả định là có. Ngoặc tím nhắc thêm một điều thường bị quên: ngay cả quyết định đúng cũng đến muộn mất Δ , nên hệ thống hạ nguồn phải hoặc dự đoán bù, hoặc chấp nhận hành động theo một trạng thái đã cũ.

Hình 15.12 nói thêm một điều mà người làm robot cảm nhận rõ hơn ai hết: ràng buộc nhân quả không chỉ là “không được nhìn tương lai” mà còn là “kết quả luôn nói về quá khứ”. Vì thế, khi so sánh hai mô hình cho hệ thời gian thực, đại lượng đáng so không phải độ chính xác trên khung hình t . Đáng so là độ chính xác *tại thời điểm quyết định được dùng*. Cách đánh giá này vẫn còn ít được chuẩn hoá, và nhiều bảng kết quả công bố không nói gì về nó.

E. Giới hạn và trường hợp hồng

- **Giả định độ sáng không đổi bị vi phạm thường xuyên hơn ta nghĩ.** Tự động phơi sáng, đèn nhấp nháy, phản xạ gương và bề mặt bóng đều làm sai giả định gốc của flow — và đây đúng là những cảnh làm hỏng cả các hệ thống hình học dựa trên độ sáng. Mô hình học sâu chịu đựng tốt hơn nhưng không miễn nhiễm, vì dữ liệu huấn luyện của chúng phần lớn là ảnh tổng hợp phơi sáng cố định.
- **Che khuất kéo dài là kẻ thù của mọi bộ bám.** Mô hình chuyển động dự đoán được vài khung, sau đó độ bất định lớn tới mức cổng lọc hoặc quá rộng (ghép bù) hoặc quá hẹp (mất dấu). Không có cách chỉnh tham số nào thoát khỏi đánh đổi này; chỉ có thêm thông tin — ngoại hình, hoặc một cảm biến khác.
- **Metric đánh lừa theo hai kiểu.** MOTA che giấu lỗi liên kết như đã nói; còn mIoU theo khung che giấu hiện tượng nhấp nháy. Cả hai đều là ví dụ của cùng một sai lầm: chọn một con số tổng không đo được đại lượng mà hệ thống hạ nguồn thực sự cần.
- **Lệch miền của dữ liệu flow.** Vì nhãn flow dày đặc gần như không thể gán bằng tay, các mô hình được huấn luyện chủ yếu trên dữ liệu tổng hợp; chất lượng trên cảnh thật vì thế phụ thuộc vào mức độ giống nhau giữa hai miền, và điều đó thay đổi theo camera của bạn.

CẦN NHỚ

Trong flow cũng như trong tracking, phần khó không phải phát hiện mà là *liên kết* — và cả hai lĩnh vực đều có một metric phổ biến che giấu đúng phần đó. Hãy chọn metric tách riêng chất lượng phát hiện khỏi chất lượng liên kết (HOTA cho tracking, sai số theo từng

chế độ chuyển động cho flow), rồi mới chọn mô hình. Ràng buộc nhân quả thì đứng trên tất cả: nó loại phương án trước khi độ chính xác kịp lên tiếng. [T2]

F. Kết nối

[C/15-thi-giac-khong-thoi-gian/02-bieu-dien-video](#) là tiền đề về chi phí: mọi cách chữa nhấp nháy bằng “đưa thời gian vào mô hình” đều phải trả bằng ngân sách ba chiều ở đó. [B/07-chia-du-lieu-va-danh-gia](#) là nơi bàn tổng quát về việc chọn metric và về rò rỉ do nhóm phụ thuộc, mà chia dữ liệu theo clip là một ca điển hình. [C/16-thi-giac-3d](#) nối tiếp trực tiếp: flow là bài toán tương ứng hai chiều, còn chương 16 nâng nó lên tương ứng ba chiều và ràng buộc epipolar. [C/14-bai-toan-cv-loi/04-bai-toan-lan-can](#) chứa cấu trúc đánh đổi song song: top-down và bottom-up ở pose lặp lại đúng dạng của tracking-by-detection và joint tracking.

I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một phương trình độ sáng không đủ để giải ra flow, và ba giả định của Lucas–Kanade mượn ràng buộc còn thiếu từ đâu?
2. Viết ma trận $A^T A$ cho một cửa sổ nằm trên cạnh dọc lý tưởng và giải thích vì sao nó suy biến. Điều đó liên hệ thế nào với việc chọn đặc trưng góc?
3. Kim tự tháp thô–tối–mịn chữa được chuyển động lớn nhưng sinh ra chế độ hỏng nào, và nó biểu hiện ra sao trong một cảnh giao thông cụ thể?
4. RAFT bỏ kim tự tháp bằng khối tương quan toàn cặp. Cái giá tính được bằng con số là gì, và số vòng lặp đóng vai trò gì với một hệ thời gian thực?
5. Một hệ thống có MOTA 0,90 nhưng đổi mã số liên tục. Giải thích bằng công thức vì sao điều đó có thể xảy ra, và HOTA sửa nó thế nào?
6. Ràng buộc nhân quả loại bỏ những loại kiến trúc và loại đánh giá nào? Vì sao độ trễ Δ khiến “độ chính xác trên khung t ” là một đại lượng không đủ?
7. Bạn có mô hình phân vùng mIoU cao nhưng video nhấp nháy, và ngân sách tính toán đã cạn. Bạn chọn cách chữa nào trong ba cách, và chấp nhận hỏng ở đâu?

I 15.4 Thực hành

Chương này có một đặc điểm may mắn: mọi khẳng định trong đó đều kiểm chứng được trên một video 60 giây và một GPU tầm trung. Hãy tận dụng điều đó. Và hãy tự tay làm hỏng hệ thống: các bài dưới đây được thiết kế để bạn quan sát *metric nào tụt trước*. Đó mới là thứ dạy bạn về metric.

Repo và tài nguyên. Tracking: [ifzhang/ByteTrack](#) (đơn giản, mạnh, đọc được trong một buổi), [NirAharon/BoT-SORT](#), và [mikel-brostrom/boxmot](#) nếu muốn thử nhiều bộ liên kết trên cùng một detector. Phân vùng và bám trong video: SAM 3. Bám điểm dày: [facebookresearch/co-tracker](#). Optical flow: [princeton-vl/RAFT](#) và [autonomousvision/unimatch](#). Nhận dạng hành động: [open-mmlab/mmdetection](#), VideoMAE-V2, InternVideo. Video qua mô hình ngôn ngữ-thị giác: Qwen3-VL (định vị trong video dài), Molmo2 (mở cả dữ liệu, theo dõi đối tượng qua khung hình). Đánh giá: [JonathonLuiten/TrackEval](#) — đây là thứ đáng đầu tư thời gian nhất trong danh sách, vì nó tính HOTA đúng cách và tách được lỗi phát hiện khỏi lỗi liên kết. Danh sách này chốt tại tháng 9/2026 và các tên mô hình dẫn đầu sẽ cũ trong 6–12 tháng, trong khi hạ tầng đánh giá như TrackEval thì bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Vì sao MOTA có thể cao trong khi hệ thống liên tục đổi mã số đối tượng, và HOTA sửa điều đó bằng cơ chế nào? Một mô hình phân vùng đạt mIoU 0,85 nhưng video nhấp nháy — ba cách xử lý là gì, và mỗi cách trả giá bằng gì? Nếu bạn phải chạy trực tuyến với ngân sách 30 ms mỗi khung hình, những họ kiến trúc nào trong chương này bị loại ngay từ đầu, và vì lý do gì?

Mini project (vài giờ đến hai ngày). Ghép một detector với ByteTrack trên một video 60 giây, đánh giá bằng TrackEval để có cả MOTA lẫn HOTA. Sau đó *cố tình làm hỏng* detector bằng cách hạ dần ngưỡng tin cậy, và ghi lại metric nào tụt trước, tụt bao nhiêu. Kết thúc bằng một đoạn giải thích vì sao thứ tự tụt lại như vậy — đó là bài học thật của bài tập này.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Ngăn cổ chai thông tin	Chỗ mà hệ thống nén dữ liệu có cấu trúc thành một vector cố định <i>trước khi</i> biết sẽ cần phần nào. Trong sinh mô tả ảnh, đó là việc nén lưới $14 \times 14 \times 512$ thành 512 số, tỉ lệ 196:1, xoá sạch thông tin “đặc trưng này nằm ở đâu”.
●●● Attention	Phép trung bình có trọng số trên các phần của nguồn, với trọng số phụ thuộc vào thứ đang cần. Nó vẫn nén thành một vector, nhưng nén lại một lần cho mỗi bước đầu ra. Gộp trung bình toàn cục là trường hợp riêng khi mọi trọng số bằng nhau.
●● Vì sao gộp trung bình toàn cục làm hỏng sinh mô tả ảnh?	Vì nó lấy trung bình theo không gian, nên xoá thông tin vị trí. Câu “con chó bên trái người đàn ông” cần đúng thông tin đó. Triệu chứng là mô hình bịa chi tiết hợp lý thay vì mô tả cái có thật.
●●● Kích hoạt	Tensor giá trị trung gian mà mỗi tầng sinh ra, phải giữ trong bộ nhớ vì lượt ngược cần tới. Một clip 16 khung $224 \times 224 \times 64$ ở fp32 chiếm khoảng 205 MB cho <i>một</i> tensor ở batch 1. Đây mới là thứ làm tràn bộ nhớ, không phải số trọng số.
●●● Ngân sách bộ nhớ	Lượng kích hoạt tỉ lệ với độ dài clip $\times$ độ phân giải ² $\times$ batch. Vì tích của ba đại lượng bị phần cứng chặn, mọi lựa chọn kiến trúc video thực chất là chọn hy sinh chiều nào.
●● (2+1)D	Tách kernel 3D thành một kernel chỉ theo không gian rồi một kernel chỉ theo thời gian. Chọn số kênh trung gian $M = 2,25 C$ thì tổng tham số bằng bản 3D. Lợi ích đến từ một hàm kích hoạt thêm vào giữa, không từ sức chứa.
●● SlowFast	Hai nhánh chạy trên cùng video: nhánh chậm ít khung nhưng nhiều kênh để nắm ngữ nghĩa, nhánh nhanh nhiều khung nhưng ít kênh để nắm chuyển động. Nhận định nền là ngữ nghĩa đối chậm còn chuyển động đối nhanh; nếu nhận định đó sai thì lợi thế chi phí biến mất.
●● Vì sao chi phí video transformer là bậc hai theo độ dài clip?	Vì attention tính quan hệ giữa mọi cặp token, và số token tỉ lệ với số khung. Một ảnh có 196 token cho ma trận $3,8 \times 10^4$ ô; clip 16 khung có 3136 token cho $9,8 \times 10^6$ ô, gấp 256 lần.
●● Mô hình 3D chỉ hơn baseline một khung hình 2 điểm — nghĩa là gì?	Nhiều khả năng tập dữ liệu đoán đúng được từ bối cảnh một khung, tức nó không đo thông tin thời gian. Phân biệt bằng cách chạy thêm trên dữ liệu cân thứ tự, ví dụ phân biệt mở với đóng; nếu baseline sập ở đó thì phép đo mới có nghĩa.
●●● Dòng quang	Trường vector cho biết mỗi pixel của khung này nằm ở đâu trong khung kia. Giả định gốc là độ sáng không đổi, nên nó hỏng khi tự động phơi sáng, đèn nhấp nháy hay bề mặt bóng làm giá trị sáng đổi.

●●● phải thuộc ●● nên nắm ● biết là có

Active recall — phần 2 (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Bài toán khẩu độ	Một phương trình độ sáng nhưng hai ẩn, nên qua một cửa sổ nhỏ chỉ đo được thành phần chuyển động vuông góc với cạnh. Đây là thiếu thông tin thật, không phải lỗi thuật toán, nên mọi cách chữa đều là mượn thêm ràng buộc từ nơi khác.
●● Vì sao phải chọn góc chứ không chọn cạnh để bám?	Vì nghiệm bình phương tối thiểu của Lucas-Kanade cần ma trận $A^T A$ không suy biến, tức cửa sổ phải có gradient theo hai hướng độc lập. Trên một cạnh lý tưởng, giá trị riêng nhỏ nhất bằng 0 và flow dọc cạnh không xác định.
●● RAFT	Bộ ước lượng dòng quang dùng khối tương quan toàn cặp cộng một số vòng tinh chỉnh cố định bằng GRU. Nó bỏ được kim tự tháp nên xử lý tốt chuyển động lớn; giá là khối tương quan cỡ 190 MB ở 1/8 độ phân giải, và số vòng lặp thành nùm vận độ trễ.
●● Vì sao kim tự tháp thô-tới-mịn làm mất vật nhỏ chuyển động nhanh?	Vì thu nhỏ ảnh làm vật nhỏ tụt xuống dưới một pixel ở tầng thô. Nó nhận chuyển động của nền, và các tầng mịn hơn chỉ tinh chỉnh quanh nghiệm sai đó nên không bao giờ sửa lại được.
●●● Liên kết dữ liệu	Bước quyết định phép đo nào ở khung này ứng với đối tượng nào đã biết. Trong tracking là ghép hộp với track, trong SLAM là ghép đặc trưng với điểm bản đồ. Đây là chỗ hệ thống hỏng, chứ không phải bước phát hiện.
●● HOTA	Thước đo tracking tách riêng chất lượng phát hiện (DetA) và chất lượng liên kết (AssA), rồi lấy trung bình nhân $\sqrt{\text{DetA} \cdot \text{AssA}}$. Vì là trung bình nhân, không thành phần nào bù đắp được cho thành phần kia.
●● Vì sao MOTA che được lỗi đổi mã số?	Vì phát hiện sót và phát hiện thừa đếm trên từng khung cho từng vật, còn mỗi lần đổi mã chỉ đếm một lần. Với 1000 vật-khung, giảm số lần đổi mã từ 40 xuống 20 chỉ nâng MOTA từ 0,88 lên 0,90.
●●● Cửa sổ nhân quả	Toàn bộ dữ liệu tới thời điểm hiện tại mà một hệ trực tuyến được phép dùng. Nó loại thẳng mọi kiến trúc hai chiều và mọi phép làm mượt nhìn về phía trước. Thêm nữa, độ trễ xử lý khiến câu trả lời luôn nói về một quá khứ gần.
●● mIoU từng khung cao nhưng video nhấp nháy — nghĩa là gì?	Mô hình chạy độc lập trên từng khung và hàm mất mát không phạt việc đổi dự đoán giữa hai khung gần giống nhau. Phân biệt với lỗi phát hiện bằng cách đo độ ổn định giữa các khung liên tiếp, không đo mIoU.
● Bám mất dấu sau khi vật bị che vài giây — nguyên nhân?	Độ bất định của mô hình chuyển động phình ra, nên cổng lọc hoặc quá rộng và ghép bừa, hoặc quá hẹp và mất dấu. Không có giá trị ngưỡng nào thoát khỏi đánh đổi này; chỉ thêm thông tin ngoại hình mới cứu được.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
RAFT và các bản kế thừa	Khối tương quan toàn cặp cộng tinh chỉnh lặp; số vòng lặp làm nùm vận độ trễ	Vẫn là kiến trúc tham chiếu của dòng quang học sâu tính tới 9/2026
ByteTrack, BoT-SORT	Tracking-by-detection: bộ lọc Kalman dự đoán, thuật toán Hungary ghép cặp, và ý “dừng vút phát hiện điểm thấp”	Được dùng rộng rãi trong hệ đếm và giám sát thực tế
TrackEval	Hiện thực HOTA — tách chất lượng phát hiện khỏi chất lượng liên kết thay vì gộp vào một số	Hạ tầng đánh giá; bền hơn mọi tên mô hình trong danh sách này

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
SAM 3 (2026) cho video	Phân vùng theo prompt cộng bám danh tính qua khung hình — đúng bài toán liên kết dữ liệu, chỉ khác là đơn vị được liên kết là mask	Ranh giới giữa segmentation và tracking đang mờ dần
co-tracker	Bám điểm dày qua thời gian; nối trực tiếp bài toán tương ứng điểm với bài toán giữ danh tính	Hữu ích khi bạn cần tương ứng dài hạn chứ không chỉ giữa hai khung liên tiếp
Kiến trúc SlowFast và các bản kế thừa	Ý “ngữ nghĩa thay đổi chậm, chuyển động thay đổi nhanh nên không cần cùng tốc độ lấy mẫu”	Vẫn xuất hiện trong nhiều mô hình video hai nhánh; tên mô hình dẫn đầu thì đổi liên tục

MINI PROJECT SLAM · Dòng quang hay mô tả đặc trưng — ngưỡng chuyển động nào làm mỗi bên gây? · 1–2 ngày

Mục tiêu	đo trực tiếp thứ mà cả chương chỉ nói bằng lời: hai cách giải bài toán tương ứng điểm — bám theo dòng quang và ghép theo mô tả đặc trưng — gây ở đâu, và ngưỡng gây đó nằm ở độ lớn chuyển động nào. Đây chính là quyết định liên kết dữ liệu mà frontend SLAM của bạn phải đưa ra ở mỗi khung hình.
Dữ liệu	một chuỗi EuRoC có dải chuyển động rộng (chuỗi “khó” như V2_03 là lựa chọn tốt vì nó có cả đoạn chậm lẫn đoạn lắc mạnh và mờ). Chân trị tương ứng điểm không có sẵn, nên dùng <i>kiểm tra vòng</i> làm thước đo thay thế: ghép từ khung t sang $t + 1$ rồi ghép ngược lại, cặp nào quay về đúng vị trí ban đầu trong ngưỡng một pixel thì tính là ghép đúng.
Các bước	(1) trích một tập đặc trưng góc chung cho cả hai phương pháp trên mỗi khung hình, để hai bên xuất phát từ cùng một tập điểm; (2) nhánh A: bám các điểm đó sang khung sau bằng dòng quang (KLT có kim tự tháp, hoặc RAFT rồi lấy mẫu tại các điểm); nhánh B: tính mô tả đặc trưng và ghép theo khoảng cách gần nhất có kiểm tra tỉ số; (3) với mỗi cặp khung hình, tính độ lớn chuyển động trung vị (từ chân trị tư thế, hoặc từ chính trung vị dòng quang) và tỉ lệ ghép đúng theo kiểm tra vòng; (4) chia các cặp khung hình thành các nhóm theo độ lớn chuyển động và vẽ tỉ lệ ghép đúng theo nhóm cho cả hai nhánh.
Xong khi	bạn có một đồ thị tỉ lệ ghép đúng theo độ lớn chuyển động, hai đường cho hai phương pháp. Bạn chỉ ra được bằng số ngưỡng mà mỗi đường bắt đầu sụt, tính bằng pixel dịch chuyển trung vị. Và bạn giải thích được cơ chế: dòng quang gây vì khai triển bậc nhất hết hiệu lực và kim tự tháp làm mất vật nhỏ, còn mô tả đặc trưng gây vì mờ chuyển động phá kết cấu cục bộ.
Nối với	C/15/03 cho cơ chế của cả hai bên; A/01 cho lý do ma trận cấu trúc quyết định điểm nào bám được; E/24 cho cách trình bày kết quả; dùng lại đúng chuỗi và đúng harness của mini project chương 14.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì hàm mất mát của mô hình per-frame không có số hạng nào phạt việc dự đoán đổi qua đổi lại giữa hai khung gần như giống nhau; và mIoU tính độc lập trên từng khung nên trung bình của nhiều khung nhấp nháy vẫn cao. Muốn thấy lỗi đó phải đo sự ổn định *giữa* các khung, thứ chưa được chuẩn hoá thành một thước đo phổ biến.
- A2. Nó ra đời để gỡ nghẽn cổ chai thông tin: cả ảnh bị nén vào một vector trước khi biết câu sẽ nói gì. Nút thắt này không xuất hiện ở phân loại vì ở đó đầu ra chỉ có một câu hỏi duy nhất và một vector là đủ; nó chỉ lộ ra khi đầu ra gồm nhiều bước, mỗi bước cần một phần khác nhau của ảnh.
- A3. Vì $MOTA = 1 - (FN + FP + IDSW)/GT$ đếm phát hiện sót và phát hiện thừa *trên từng khung cho từng vật*, nhưng mỗi lần đổi mã số chỉ được đếm một lần. Với nghìn vật–khung, giảm một nửa số lần đổi mã chỉ nâng MOTA vài phần trăm điểm. HOTA sửa bằng trung bình nhân $\sqrt{DetA \cdot AssA}$, nên không thành phần nào bù cho thành phần kia.

- A4. Vì trục thời gian có hướng: đảo ngược nó thì “mở” thành “đóng”, trong khi lật trục x vẫn cho một ảnh hợp lệ. Nó lại còn có đơn vị khác hẳn hai trục kia. Vậy không có lý do gì để cùng một kernel $3 \times 3 \times 3$ là kích thước hợp lý ở cả ba trục. Hai bất đối xứng đó là nguyên nhân sâu xa của mọi thiết kế tách trục.
- A5. Bằng độ phân giải hoặc bằng batch, theo quan hệ $R \propto 1/\sqrt{TB}$. Ba cách không tương đương: giảm batch làm hỏng thống kê của chuẩn hoá theo batch, giảm độ phân giải làm mất vật nhỏ vĩnh viễn, còn giảm độ dài clip thì làm mất chính thông tin thời gian mà bạn đang trả tiền để có.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn ngồi trước một cái bàn nhỏ, và cứ mỗi một phần ba mươi giây lại có người đặt thêm lên bàn một tấm ảnh chụp cùng một hành lang. Nhìn một tấm ảnh, bạn nói được “có một người” — nhưng để nói “người đó đang bước tới” hoặc “vẫn là người lúc này chứ không phải người khác”, bạn buộc phải đặt vài tấm cạnh nhau và so.

Mẹo thì đơn giản: so từng chỗ trên tấm này với chỗ tương ứng trên tấm kia. Hoặc so từng điểm nhỏ, để biết mọi thứ dịch đi đâu; hoặc so cả một vùng, để nói được “cái hộp này” ở tấm trước với “cái hộp kia” ở tấm sau. Việc nói đó, chứ không phải việc nhìn thấy, mới là phần khó.

Cái giá gồm ba khoản, và cả ba đều đắt. Thứ nhất, mặt bàn có hạn: muốn để bốn mươi tấm thay vì mười tấm thì phải in ảnh nhỏ lại, mà ảnh nhỏ thì mất những vật bé. Thứ hai, các tấm ảnh của tương lai chưa được chụp, nên bạn chỉ được dùng những gì đã có tới lúc này — mọi cách làm cần “xem hết rồi mới nói” đều bị loại ngay. Thứ ba, đến khi bạn nói xong thì người trong ảnh đã bước thêm mấy bước, nên câu trả lời của bạn luôn nói về một quá khứ gần.

Chỗ không rút gọn được: ngưỡng chuyển động mà một cách nối bắt đầu sai phụ thuộc vào kết cấu bề mặt, độ mờ và cách phơi sáng của đúng cái camera bạn dùng, nên không có con số chung để nhớ — chỗ này bắt buộc phải tự đo.

Một câu: Thêm thời gian vào ảnh không tặng bạn một chiều dữ liệu, nó gửi cho bạn ba hoá đơn: bộ nhớ nhân lên, câu trả lời luôn muộn, và phần khó nhất là nối đúng vật ở tấm này với vật ở tấm kia.

Tổng kết chương

Ba vấn đề mà chiều thời gian sinh ra hoá ra có ba câu trả lời rất khác nhau về bản chất. Đầu ra độ dài thay đổi được giải bằng một *mẫu thiết kế*: giữ nguyên nguồn thông tin và hoãn việc nén lại cho tới khi có truy vấn — và mẫu thiết kế đó, dưới cái tên attention, sau này chiếm lĩnh cả lĩnh vực. Chi phí bộ nhớ không có lời giải, chỉ có kế toán: tích của độ dài clip, độ phân giải bình phương và batch size gần như là hằng số, nên mọi kiến trúc là một cách chọn hy sinh. Ràng buộc nhân quả cũng không có lời giải, nó chỉ đơn giản loại bỏ phương án — và loại bỏ trước khi ta kịp bàn tới độ chính xác.

Bài học chung nhất thì nằm ở phần liên kết. Trong optical flow cũng như trong tracking, phần khó không phải nhìn thấy mà là ghép cặp, và trong cả hai lĩnh vực đều tồn tại một thước đo phổ biến che giấu đúng phần khó đó. Đây là biến thể của bài học ở chương 14: chi tiết không được viết trong abstract thường quyết định kết quả. Ở đây nó xuất hiện dưới dạng thước đo, không phải dưới dạng cách gán mẫu. Thói quen cần mang theo là như nhau: trước khi tin một con số, hãy hỏi nó cộng những loại lỗi nào lại với nhau và với trọng số bao nhiêu.

Còn bỏ ngỏ hai hướng. Thứ nhất, toàn chương này làm việc trên mặt phẳng ảnh; khi bài toán tương ứng điểm được nâng lên không gian ba chiều với ràng buộc epipolar và độ sâu, ta bước sang [C/16-thị-giác-3d](#) — và đó là chỗ mà nền hình học của bạn phát huy trực tiếp nhất. Thứ hai, mọi mô hình ở đây đều cần nhãn thời gian đắt đỏ; [C/18-pretraining-va-foundation](#) bàn về việc lấy giám sát từ chính cấu trúc thời gian của video, thứ có sẵn miễn phí.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Ba vấn đề riêng của chiều thời gian — đầu ra độ dài thay đổi, chi phí bộ nhớ, ràng buộc nhân quả — có xung đột với nhau không? Chỉ ra một thiết kế mà chữa vấn đề này thì làm nặng thêm vấn đề kia.
2. Mục 1 sinh ra attention để tránh nghẽn “nén trước, dùng sau”, còn mục 2 cho thấy chính attention nở chi phí trên clip dài. Hai mục vì thế đẩy về hai hướng ngược nhau; bạn hoà giải thế nào cho một mô hình chạy trực tuyến?
3. Tương ứng điểm và liên kết dữ liệu là hai bài toán mà một hệ SLAM đã giải ở mỗi khung hình. Với mỗi bài, phần nào của từ vựng SLAM dịch thẳng sang được và phần nào không, và chỗ không dịch được bắt nguồn từ giả định nào?
4. Nhấp nháy không xuất hiện trong mIoU, đổi danh tính gần như không xuất hiện trong MOTA, và độ trễ không xuất hiện trong “độ chính xác trên khung t ”. Ba lỗi hổng đó có chung một nguyên nhân về cách xây thước đo không, và nó là gì?
5. Cửa sổ trượt, trạng thái hồi quy và lấy mẫu thưa theo thời gian là ba cách trả giá cho việc kéo dài clip. Chúng hổng theo ba kiểu khác nhau nào, và loại chuyển động nào phơi bày từng kiểu?
6. Baseline chỉ nhìn một khung hình rồi gộp lại gần bằng mô hình thời gian đắt hơn nhiều. Trước khi kết luận rằng thời gian không quan trọng, bạn kiểm ba khả năng nào ở dữ liệu và ở thước đo?

Thị giác 3D và hình học (3D Vision and Geometry)

PHỤ THUỘC A/02-vat-ly-tao-anh/03-camera-va-hinh-hoc-nhieu-view (mô hình chiếu, hiệu chỉnh); B/04-nen-tang-hoc-tu-du-lieu (đánh đổi giả định $\leftrightarrow$ dữ liệu); B/07-chia-du-lieu-va-danh-gia (giao thức đánh giá, ATE/RPE); C/13-kien-truc-backbone (transformer, thiên lệch quy nạp).

TÓM TẮT

Đây là lĩnh vực có mạch lịch sử rõ nhất trong computer vision: cùng một bài toán — suy hình dạng 3D và pose camera từ ảnh 2D — được giải lại từ đầu bốn lần, mỗi lần đổi một loại giả định lấy một loại độ bền. Chương này dựng khung bốn thể hệ đó, rồi đi vào các quyết định con (biểu diễn, độ sâu, SLAM, học trên point cloud) và hai thứ mới nhất là *neural rendering* và hợp nhất cảm biến. Đọc xong bạn chọn được thể hệ đúng cho dữ liệu đang có và biết cách phát hiện lúc một mô hình học đang nói dối về hình học. Nếu chỉ nhớ một điều: mỗi lần bỏ bớt một giả định về thế giới, bạn phải bù bằng dữ liệu, và bạn mất một phần khả năng tự chẩn đoán. Đó là cái giá ít được nói tới nhất của thị giác 3D hiện đại.

KIẾN THỨC CẦN CÓ TRƯỚC

- **thị giác 3D (3D vision)** — nhánh của computer vision lo việc suy hình dạng ba chiều và chuyển động camera từ ảnh hai chiều — ngược lại với nhánh nhận dạng, vốn chỉ hỏi “trong ảnh có gì”.
- **phép chiếu phối cảnh (perspective projection)** — cách camera biến một điểm 3D thành một pixel: chia toạ độ ngang và dọc cho độ sâu. Chính phép chia này làm mất chiều thứ ba.
- **prior (prior)** — những gì mô hình “tin sẵn” về thế giới trước khi nhìn dữ liệu hiện tại — có thể do người viết cài cứng (mô hình vật lý) hoặc do mạng học từ dữ liệu lớn.
- **metric (đơn vị mét)** — một kết quả gọi là metric khi nó cho ra số đo tuyệt đối tính bằng mét, chứ không chỉ đúng sai khác một hệ số tỉ lệ chưa biết.
- **SLAM và VIO** — hai hệ thống ước lượng vị trí robot theo thời gian thực trong lúc dựng bản đồ, dùng camera (SLAM) hoặc camera cộng cảm biến quán tính (VIO). Chương này giả sử bạn đã quen chúng và chỉ đặt chúng vào bức tranh chung.
- **neural rendering** — họ phương pháp học một biểu diễn cảnh bằng cách bắt nó render ra ảnh giống các ảnh đã chụp, thay vì bằng cách đo hình học trực tiếp.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao thêm bao nhiêu dữ liệu cũng không cho một camera đơn biết vật thể trước mặt lớn thật sự bao nhiêu mét?
- Q2. Nếu một *transformer* suy được pose và hình học chỉ trong một lượt xuôi, vì sao COLMAP vẫn chưa bị thay thế?
- Q3. Vì sao NeRF phải mô tả cảnh bằng khói mờ thay vì bằng bề mặt cứng, trong khi thứ tồn tại thật là bề mặt?
- Q4. Vì sao một hệ hợp nhất camera–LiDAR có thể chạy hoàn hảo lúc robot đứng yên rồi hỏng đúng lúc nó xoay nhanh, dù không một tham số nào thay đổi?

Vì sao chương này chia làm ba mục

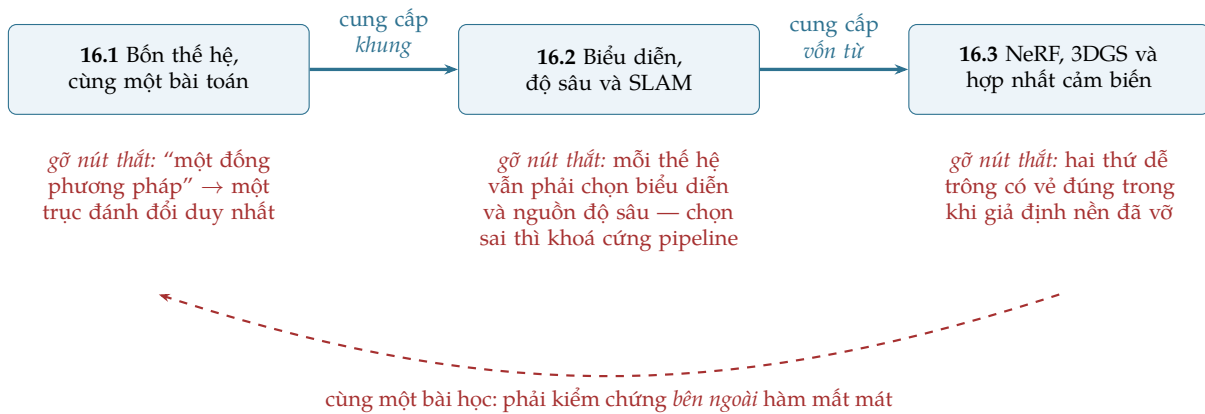
Chương được chia theo ba tầng câu hỏi, và thứ tự là thứ tự phụ thuộc chứ không phải thứ tự lịch sử.

Mục thứ nhất dựng *khung*: bốn thể hệ giải cùng một bài toán, mỗi thể hệ thay một đoạn khác nhau của cùng một pipeline. Không có khung này thì mọi thứ còn lại rơi vào tình trạng “một đồng phương pháp”, và ta không trả lời được câu hỏi thực dụng nhất — với dữ liệu đang có thì nên dùng cái gì. Khung này cũng là chỗ trực đánh đổi “giả định mạnh + ít dữ liệu $\leftrightarrow$ giả định yếu + nhiều dữ liệu” của chương 4 hiện ra rõ nhất trong toàn bộ CV.

Mục thứ hai đi vào các *quyết định con* mà mọi thể hệ đều phải đưa ra: lưu cảnh bằng biểu diễn nào, lấy độ sâu từ đâu và cái gì cố định tỉ lệ, SLAM đứng ở đâu trong bức tranh, và làm sao đưa mạng nơ-ron lên một tập điểm không thứ tự. Đây là tầng “vốn từ”: không nắm nó thì không đọc được mục ba.

Mục thứ ba đào sâu đúng hai thứ xa nền tảng hình học cổ điển nhất — neural rendering (NeRF, 3DGS) và hợp nhất camera–LiDAR–IMU. Hai chủ đề này được gộp vào một mục vì chúng chia chung một bài học: cả hai đều dễ trông có vẻ hoạt động trong khi giả định nền đã bị vi phạm, và cả hai đều đòi một cơ chế kiểm chứng *bên ngoài* hàm mất mát của chính chúng.

Hình 16.1 vẽ ba mục đó cùng nút thắt mà mỗi mục gỡ.



Hình 16.1. Ba mục của chương và quan hệ phụ thuộc giữa chúng. Thứ tự là thứ tự *phụ thuộc*, không phải thứ tự lịch sử: mục 16.1 dựng khung để hai mục sau không biến thành danh sách phương pháp rời rạc, mục 16.2 cung cấp vốn từ cho mục 16.3. Mũi tên đứt phía dưới là điều mà cả ba mục cùng dẫn tới, và cũng là điều dễ bị bỏ sót nhất khi đọc riêng từng mục.

Với người đọc đang làm visual-inertial SLAM thật, ba mục này có mật độ thông tin mới rất khác nhau: mục một chủ yếu sắp xếp lại thứ đã biết, mục hai bổ sung phần học sâu trên dữ liệu 3D, còn mục ba mới là phần thực sự mới. Nếu thời gian eo hẹp, đọc mục một để lấy khung rồi nhảy thẳng sang mục ba.

Bản đồ chương 16 — Thị giác 3D và hình học

TÓM TẮT

Chương 16 gồm ba mục, chia theo ba tầng câu hỏi chứ không theo thứ tự lịch sử: khung bốn thể hệ, các quyết định con, rồi hai chủ đề mới nhất. Bản đồ này nói mục nào là cốt lõi, mục nào có thể tra khi cần, và đọc theo trình tự nào tùy theo nền sẵn có của bạn. Nếu chỉ nhớ một điều: mục 16.1 là khung để đọc hai mục còn lại — bỏ qua nó thì phần sau biến thành một danh sách phương pháp rời rạc.

KIẾN THỨC CẦN CÓ TRƯỚC

- **SfM / MVS** — hai bước của pipeline hình học cổ điển: suy pose camera và cấu trúc thừa từ ảnh (SfM), rồi dựng bề mặt dày đặc khi đã biết pose (MVS).
- **neural rendering** — học một biểu diễn cảnh bằng cách bắt nó dựng ra ảnh giống các ảnh đã chụp, thay vì đo hình học trực tiếp.
- **feed-forward geometry** — mô hình suy pose và hình học trong *một* lần chạy mạng, không cần vòng tối ưu và không cần pose cho trước.
- **point cloud / voxel / mesh** — ba cách lưu hình dạng 3D: tập điểm rời rạc, lưới ô đều, và bề mặt tam giác.

Ba mục và vai trò của từng mục

- **16.1 — Bốn thể hệ, cùng một bài toán (cốt lõi).** Khung của cả chương: hình học cổ điển, lai học-hình học, neural rendering, feed-forward geometry. Trục đánh đổi “giả định mạnh + ít dữ liệu ↔ giả định yếu + nhiều dữ liệu”. Đọc mục này trước mọi thứ khác.
- **16.2 — Chọn biểu diễn 3D, độ sâu và SLAM (cốt lõi phần biểu diễn và độ sâu; phần SLAM là tham khảo với người đã làm SLAM).** Vốn từ: voxel, point cloud, mesh, trường ẩn, Gaussian; bốn nguồn độ sâu và cái gì cố định tỉ lệ; PointNet và bất biến hoán vị.
- **16.3 — NeRF, 3DGS và hợp nhất cảm biến (cốt lõi).** Cơ chế dựng ảnh thể tích, vì sao phải dùng mật độ liên tục, 3DGS đổi gì lấy gì, ba giả định nền dễ vỡ; early đối lại late fusion và ngân sách sai số hiệu chỉnh.

Trình tự đọc

1. Nếu bạn **chưa có nền hình học nhiều view**: đọc mục *camera và hình học nhiều view* ở [A/02-vat-ly-tao-anh](#) trước, rồi 16.1 → 16.2 → 16.3 theo đúng thứ tự.
2. Nếu bạn **đã làm SLAM hoặc SfM**: đọc 16.1 để lấy khung (nhẹ, phần lớn là sắp xếp lại thứ đã biết), lướt mục 16.2 phần D về SLAM, đọc kỹ 16.2 phần C và E (độ sâu học được, point cloud) và toàn bộ 16.3 — đó là phần có mật độ thông tin mới cao nhất với bạn.
3. Khối *Thực hành* nằm ở file chương, sau ba mục.

Chương này phụ thuộc gì và được dùng lại ở đâu

- **Phụ thuộc lên trên:** [A/02-vat-ly-tao-anh](#) (mô hình chiếu, hiệu chỉnh, phơi sáng); [B/04-nen-tang-hoc-tu-du-lieu](#) (đánh đổi giả định ↔ dữ liệu, tính xác định được); [B/07-chia-du-lieu-va-danh-gia](#) (ATE/RPE, giao thức); [C/13-kien-truc-backbone](#) (transformer).
- **Được dùng lại ở dưới:** [C/17-mo-hinh-sinh/02-toi-uu-tren-pixel](#) dùng lại nguyên ý tưởng tối ưu trên chính đối tượng cần tạo ra; [C/17-mo-hinh-sinh/04-low-level-vision](#) dùng lại

khái niệm bài toán ill-posed và vai trò của prior; E/25-do-tin-cay-va-van-hanh dùng lại yêu cầu “cơ chế phát hiện sai từ bên ngoài”.

16.1 Bốn thế hệ, cùng một bài toán (Four Generations of 3D Reconstruction)

PHỤ THUỘC A/02-vat-ly-tao-anh/03-camera-va-hinh-hoc-nhieu-view (mô hình chiếu, ràng buộc epipolar); B/04-nen-tang-hoc-tu-du-lieu (trục “giả định mạnh + ít dữ liệu” ↔ “giả định yếu + nhiều dữ liệu”); B/03-giai-phau-he-thong (tách bài toán → nguyên lý → cài đặt).

TÓM TẮT

Bài toán khôi phục hình học 3D từ ảnh đã được giải lại từ đầu bốn lần. Bốn thế hệ đó không thay thế nhau: mỗi thế hệ thay một *đoạn khác nhau* của cùng một pipeline. Vì vậy chúng chồng lớp được, và câu hỏi “VGGT có thay được COLMAP không” là câu hỏi sai. Đọc xong bạn làm được ba việc. Chọn thế hệ đúng cho dữ liệu đang có: bao nhiêu ảnh, có pose hay không, cảnh loại gì. Ghép hai thế hệ thành một pipeline, một bên khởi tạo một bên tinh chỉnh. Và kiểm tra được lúc một mô hình tiến thẳng đang nói dối. Nếu chỉ nhớ một điều: đi từ thế hệ một tới thế hệ bốn, thứ mất đi không chỉ là độ chính xác metric mà là *khả năng tự chẩn đoán*. Hiệu chỉnh chùm tia trả về phần dư và hiệp phương sai; mạng tiến thẳng không trả về gì cả. Vì vậy đưa nó vào hệ thống thật là phải tự dựng lại cơ chế phát hiện sai từ bên ngoài.

KIẾN THỨC CẦN CÓ TRƯỚC

- **SfM (structure from motion)** — quy trình cổ điển suy đồng thời hình dạng 3D của cảnh và vị trí camera từ một tập ảnh chụp cùng cảnh ở nhiều góc.
- **MVS (multi-view stereo)** — bước sau SfM: khi đã biết pose camera, dựng bề mặt dày đặc bằng cách khớp từng vùng ảnh giữa nhiều view.
- **pose camera (camera pose)** — vị trí và hướng của camera trong không gian, sáu bậc tự do; “biết pose” là biết ảnh được chụp từ đâu, nhìn về đâu.
- **tam giác hoá (triangulation)** — tìm điểm 3D bằng giao của hai hay nhiều tia nhìn xuất phát từ các camera đã biết pose; đây là chỗ chiều bị mất khi chiếu được lấy lại.
- **hiệu chỉnh chùm tia (bundle adjustment)** — bài toán bình phương tối thiểu phi tuyến. Nó tinh chỉnh đồng thời mọi pose và mọi điểm 3D sao cho sai số chiếu lại nhỏ nhất. Bạn đã quen nó; điều cần nhớ ở đây là nó trả về *phần dư* và *hiệp phương sai*.
- **RANSAC** — thủ tục lọc điểm sai bằng cách thử ngẫu nhiên nhiều giả thuyết nhỏ rồi giữ giả thuyết được nhiều dữ liệu ủng hộ nhất. Nó giả định điểm sai là *ngẫu nhiên*, không phải sai đồng loạt theo một kiểu.
- **point map** — ảnh mà mỗi pixel chứa một toạ độ 3D thay vì một màu; đây là dạng đầu ra của các mô hình tiến thẳng hiện đại, thay cho “point cloud rời rạc”.
- **disparity** — độ lệch ngang của cùng một điểm giữa ảnh trái và ảnh phải trong hệ stereo; nó tỉ lệ nghịch với độ sâu.

A. Đặt vấn đề

Bài toán gốc của thị giác 3D không đổi trong bốn mươi năm: cho một tập ảnh 2D của cùng một cảnh, hãy khôi phục hình dạng ba chiều của cảnh và vị trí camera lúc chụp. Nó khó vì phép chiếu phối cảnh làm mất một chiều — mỗi pixel chỉ nói “có gì đó nằm đâu đó trên tia này”, chứ không nói nó xa bao nhiêu. Chỉ khi nhiều tia từ nhiều vị trí camera khác nhau cắt nhau thì độ sâu mới được xác định, và cả ngành 3D là các cách khác nhau để lấy lại thông tin đã mất đó.

Điều đáng học ở lĩnh vực này không phải thuật toán cụ thể mà là mạch lịch sử của nó: đây là chỗ duy nhất trong computer vision mà ta thấy một bài toán được giải lại từ đầu bốn lần, mỗi lần đổi một loại giả định lấy một loại độ bền. Thế hệ đầu viết thẳng mô hình vật lý vào bài toán tối ưu, nên chỉ cần vài chục ảnh nhưng đòi cảnh phải cư xử đúng như mô hình. Thế hệ cuối vứt gần hết mô hình đó đi. Nó thay bằng một *prior* học từ hàng triệu cảnh, nên chạy được ở chỗ mô hình vật lý bó tay. Cái giá là không còn cách nào kiểm chứng kết quả từ bên trong. Đây chính xác là trục “giả định mạnh + ít dữ liệu $\leftrightarrow$ giả định yếu + nhiều dữ liệu” của chương 4, và không chỗ nào trong CV nó hiện ra sạch sẽ như ở đây.

$\leftrightarrow$ **trường hợp riêng của B/04** — bốn thế hệ 3D là hiện thân cụ thể nhất của trục “giả định mạnh + ít dữ liệu $\leftrightarrow$ giả định yếu + nhiều dữ liệu”; ở đây ta đo được cái giá bằng centimét.

Học mục này thì làm được gì mà trước đó không làm được? Trả lời được ba câu hỏi thực dụng: (i) với dữ liệu bạn đang có — bao nhiêu ảnh, có pose hay không, cảnh loại gì — thế hệ nào là lựa chọn đúng; (ii) khi ghép hai thế hệ vào một pipeline thì ai làm khởi tạo, ai làm tinh chỉnh; (iii) làm thế nào phát hiện lúc một mô hình **tiền thẳng** (*feed-forward*) đang nói dối, việc mà **hiệu chỉnh chùm tia** (*bundle adjustment*) làm hộ bạn miễn phí còn mạng học thì không.

Với người đọc đang làm *visual-inertial SLAM*, ba thế hệ đầu phần lớn là sân nhà. Phần lạ nằm ở hai thế hệ sau — *neural rendering* và *feed-forward geometry* — nên chương này viết kỹ đúng hai chỗ đó.

TRỰC GIÁC

Bốn thế hệ là bốn cách trả lời câu hỏi “căn phòng này rộng bao nhiêu”. Thế hệ hình học mang thước ra đo từng cạnh rồi cộng sai số theo quy tắc rõ ràng: đúng tới milimét, và nếu các số đo mâu thuẫn nhau thì phần dư của phép bình phương tối thiểu tự nó tố cáo. Thế hệ lai vẫn dùng thước, nhưng nhờ mắt chỉ chỗ đặt thước cho nhanh. *Neural rendering* không đo phòng: nó dựng một mô hình phòng bằng sương mù có màu rồi chỉnh đám sương cho tới khi ảnh chụp mô hình trùng khít mọi ảnh thật — đúng về mặt *nhìn*, chưa chắc đúng về mặt *chạm*. Thế hệ tiền thẳng liếc một cái rồi nói “khoảng bốn nhân năm mét”, vì đã ở trong hàng vạn căn phòng: nhanh, thường đúng, và khi sai thì sai rất tự tin vì không có phép đo nào để mâu thuẫn với nó.

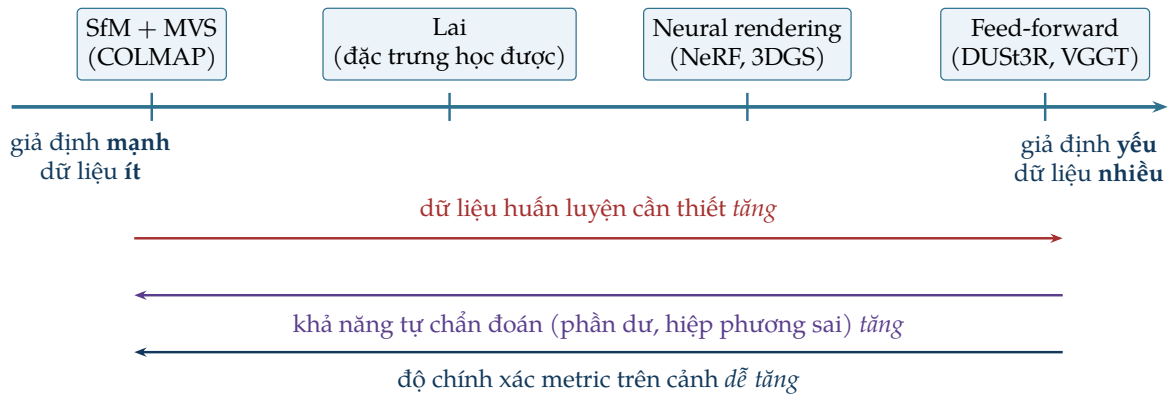
B. Bốn thế hệ đặt cạnh nhau

Bảng 16.1 đặt bốn thế hệ trên cùng bốn tiêu chí. Cột đáng đọc kỹ nhất là cột cuối: mỗi thế hệ ra đời vì thế hệ trước hỏng ở một chỗ cụ thể, rồi lại hỏng ở một chỗ khác.

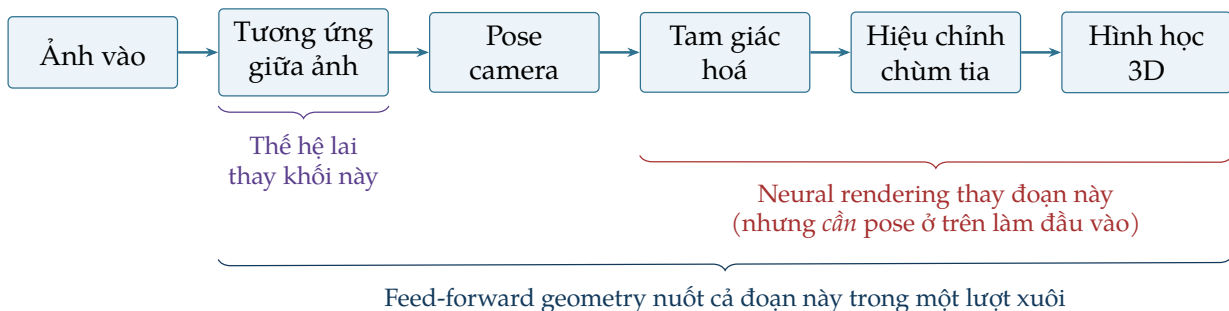
Bảng 16.1. Bốn thể hệ giải bài toán khôi phục hình học 3D từ ảnh. Không thể hệ nào thay thế hoàn toàn thể hệ trước; trên thực tế chúng chồng lên nhau.

Thể hệ	Cơ chế	Giả định	Mạnh khi	Hồng khi
Hình học cổ điển (SfM/MVS, COLMAP)	Tìm tương ứng, ước lượng pose, tam giác hoá, hiệu chỉnh chùm tia	Cảnh tĩnh, đủ texture, đủ chồng lấn giữa các view	Cần độ chính xác metric và cần kiểm chứng được	Ít texture, ít ảnh, bề mặt phản chiếu hoặc trong suốt
Lai học + hình học	Học phần tương ứng hoặc phần độ sâu, giữ nguyên phần tối ưu hình học	Hình học vẫn đúng; chỉ khâu nhận dạng cần học	Có prior mạnh nhưng vẫn cần ràng buộc cứng	Khi khâu học sai một cách <i>có hệ thống</i> : RANSAC không lọc nổi
Neural rendering (NeRF, 3DGS)	Tối ưu một biểu diễn riêng cho từng cảnh bằng loss render khả vi	Đã biết pose; cảnh tĩnh; phơi sáng nhất quán	Cần chất lượng ảnh ở góc nhìn mới	Không có pose sẵn; phải tối ưu lại từ đầu cho mỗi cảnh
Feed-forward geometry (DUST3R, VGGT)	Một <i>transformer</i> suy ra pose + độ sâu + point map trong một lượt xuôi	Prior học từ dữ liệu lớn khái quát được sang cảnh mới	Ít ảnh, không có pose, cần kết quả nhanh	Cảnh dễ mà cần độ chính xác metric: thua hiệu chỉnh chùm tia

Bảng cho biết *cái gì*, hai hình sau cho biết *tại sao*. Hình 16.2 đặt bốn thể hệ lên một trục duy nhất và cho thấy ba đại lượng cùng biến thiên theo trục đó; Hình 16.3 cho thấy mỗi thể hệ thay một *đoạn* khác nhau của cùng một pipeline. Hình thứ hai giải thích vì sao câu hỏi “VGGT có thay được COLMAP không” là câu hỏi sai — câu hỏi đúng là “thay được đoạn nào”.



Hình 16.2. Một trục duy nhất giải thích cả bốn thể hệ. Đi sang phải, ta bỏ bớt giả định về thể giới và bù bằng dữ liệu; đổi lại, hai thứ cùng giảm — độ chính xác metric ở cảnh dễ, và quan trọng hơn, khả năng hệ thống tự biết mình đang sai.



Hình 16.3. Cùng một pipeline, ba cách thay thế khác nhau. Thể hệ lai thay đúng khối dễ hỏng nhất mà giữ nguyên phần tối ưu; neural rendering nằm *sau* khâu pose nên nó là khách hàng của SfM chứ không phải đối thủ; chỉ thể hệ tiến thẳng mới thực sự cạnh tranh trực tiếp với COLMAP.

C. Vì sao thế hệ sau ra đời

I Mạch 1 — bốn thế hệ, cùng một bài toán

- 1 Vấn đề** ảnh mất một chiều, và không có cách nào lấy lại nó từ một ảnh đơn lẻ.

Giải pháp viết thẳng mô hình chiếu vào một bài toán bình phương tối thiểu: tìm tương ứng giữa các ảnh, ước lượng pose tương đối, tam giác hoá điểm, rồi tinh chỉnh toàn bộ bằng hiệu chỉnh chùm tia [33], [35].

Giả định cảnh tĩnh, bề mặt *Lambertian* đủ texture, nhiễu đo là Gauss độc lập.

Hệ quả mô hình sinh ảnh đã được cài cứng, nên phương pháp cần rất ít dữ liệu: vài chục ảnh là ra kết quả metric. Quan trọng hơn, nó trả về *phần dư* cùng *ma trận hiệp phương sai*, tức là tự nói cho ta biết nó tin kết quả tới đâu.
- 2 Vấn đề** khâu yếu nhất của thế hệ một là tìm tương ứng: nó dựa vào *descriptor* thủ công, và descriptor thủ công chết ở thay đổi ánh sáng lớn, góc nhìn lệch nhiều, vùng hoa văn lặp lại.

Giải pháp thay đúng khối đó bằng một mạng học — detector/descriptor học được, matcher học được, hoặc **khớp dày đặc** (*dense matching*) — và giữ nguyên phần tối ưu hình học phía sau.

Giả định hình học vẫn đúng; chỉ phần “nhận dạng” mới cần học.

Giá phải trả mất tính tái lập bit-for-bit và mất khả năng suy luận giải tích về sai số của khâu đầu.

Vấn đề mới mạng học sai theo kiểu *tương quan*. Ở một mặt tường lặp hoa văn, nó không cho ra *outlier* ngẫu nhiên mà cho ra một tập tương ứng sai *nhất quán*. RANSAC không lọc được loại lỗi đó, vì nó giả định outlier là ngẫu nhiên.

↔ **hệ quả của B/08** — sai đồng loạt của khâu học là lệch phân phối biểu hiện ở tầng hình học; đó là lý do một bộ lọc thống kê không cứu được nó.
- 3 Vấn đề** ngay cả khi hình học đúng, đầu ra của SfM/MVS là point cloud hoặc mesh — render lại thì thùng lỗ, không có hiệu ứng phụ thuộc góc nhìn.

Giải pháp bỏ luôn ý định khôi phục bề mặt; tham số hoá cảnh bằng một trường liên tục và tối ưu trường đó sao cho ảnh render khớp mọi ảnh quan sát [28].

Giả định pose đã biết trước, cảnh tĩnh, phơi sáng và cân bằng trắng nhất quán.

Hệ quả chất lượng ảnh ở góc nhìn mới nhảy vọt.

Giá phải trả hàm mất mát chỉ ràng buộc *ảnh render*, không ràng buộc *hình học*.

Vấn đề mới mỗi cảnh phải tối ưu lại từ đầu, và pose vẫn phải xin từ thế hệ một.
- 4 Vấn đề** cả ba thế hệ trên đều gậy ở cùng một điều kiện: quá ít ảnh, hoặc chồng lấn không đủ để tam giác hoá. Ba tấm ảnh chụp ba góc một căn phòng, mỗi cặp chồng lấn 20 %, thì COLMAP thường không khởi tạo nổi.

Giải pháp huấn luyện một transformer trên rất nhiều cảnh có chân trị 3D, cho nó nhận thẳng một tập ảnh không pose và xuất ra *point map* trong một hệ toạ độ chung [34]; dòng kế tiếp (DUST3R → MAST3R → VGGT) mở rộng từ hai ảnh lên hàng chục ảnh trong một lượt xuôi.

Giả định cảnh mới nằm trong phân phối mà prior đã học.

Hệ quả bài toán đổi bản chất: từ ước lượng có ràng buộc cứng thành hồi quy.

Giá phải trả độ chính xác metric ở cảnh dễ thua hiệu chỉnh chùm tia, và — điểm quan trọng nhất — không còn phần dư hình học để phát hiện lúc nó sai.

Đọc theo mạch này thì thấy cả bốn bước là cùng một phép đánh đổi lặp lại: mỗi lần bỏ bớt một giả định về thế giới, ta phải bù bằng dữ liệu, và mất đi một phần khả năng tự kiểm tra — đúng ba mũi tên của Hình 16.2.

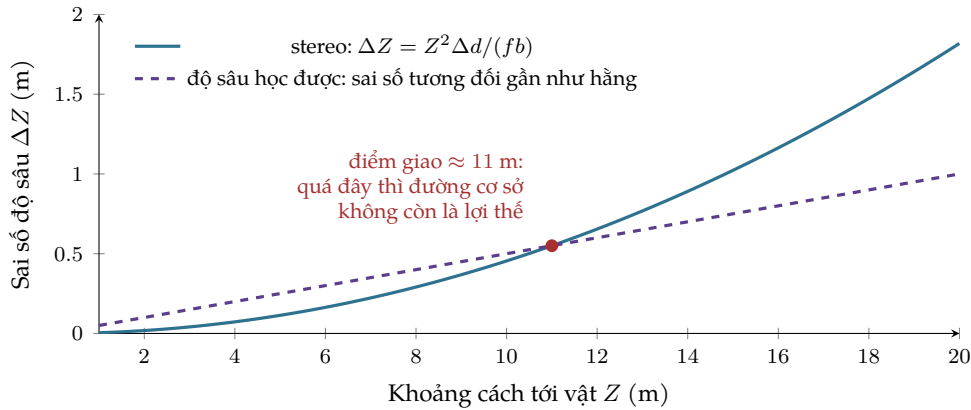
D. Một phép tính nhỏ: vì sao “chính xác metric” là đặc sản của thế hệ một

Nói gọn thì: sai số độ sâu của phương pháp hình học có một *công thức*, còn sai số của mạng học thì không. Với một cặp *stereo* đã hiệu chỉnh, độ sâu suy ra từ **độ lệch ngang** (*disparity*) theo $Z = fb/d$, nên

$$\Delta Z = \frac{Z^2}{fb} \Delta d.$$

Câu này nói: sai khớp điểm *một chút* ở gần thì hầu như không sao, nhưng cùng sai số đó ở xa thì bị khuếch đại theo bình phương khoảng cách. Lấy con số của một bộ EuRoC quen thuộc, $f \approx 400$ pixel và $b \approx 0,11$ m: ở $Z = 10$ m thì $d = 4,4$ pixel, và $\Delta d = 0,2$ pixel cho $\Delta Z \approx 0,45$ m (4,5%); ở $Z = 2$ m thì cùng Δd đó chỉ cho $\Delta Z \approx 1,8$ cm.

Hình 16.4 vẽ hai đường sai số đó cạnh nhau và cho thấy chúng cắt nhau ở đâu.



Hình 16.4. Hai loại sai số độ sâu có *hình dạng* khác nhau, và đó mới là điều đáng nhớ. Sai số stereo tăng theo Z^2 nên rất tốt ở gần và vô dụng ở xa; sai số của một mạng học được tăng gần như tuyến tính vì nó là sai số *tương đối* do prior quyết định, không do hình học. Vì vậy tồn tại một khoảng cách mà quá nó, phép đo hình học thua phép đoán có học. *Con số minh họa* ($f = 400$ px, $b = 0,11$ m, $\Delta d = 0,2$ px, sai số tương đối 5%), không phải số đo trên một hệ cụ thể.

Ba hệ quả rút ra:

- **Có một “chân trời” độ sâu.** Vượt qua nó, độ sâu stereo vô dụng — không phải vì thuật toán kém mà vì hình học.
- **Cách duy nhất đẩy chân trời ra xa là tăng fb .** Bundle adjustment trên một chuỗi ảnh chính là cách tăng **đường cơ sở** (*baseline*) hiệu dụng bằng thời gian; đây là lý do SLAM đơn mắt vẫn có độ sâu tốt khi robot đi đủ xa.
- **Mạng suy độ sâu từ một ảnh không có phương trình này.** Sai số của nó do prior quyết định, nên không tuân theo Z^2 và cũng không hưởng lợi khi ta đi thêm một quãng để mở rộng đường cơ sở. Trộn hai loại sai số này vào cùng một bộ lọc mà không mô hình hoá đúng thì hiệp phương sai sẽ sai.

E. Giới hạn và trường hợp hồng

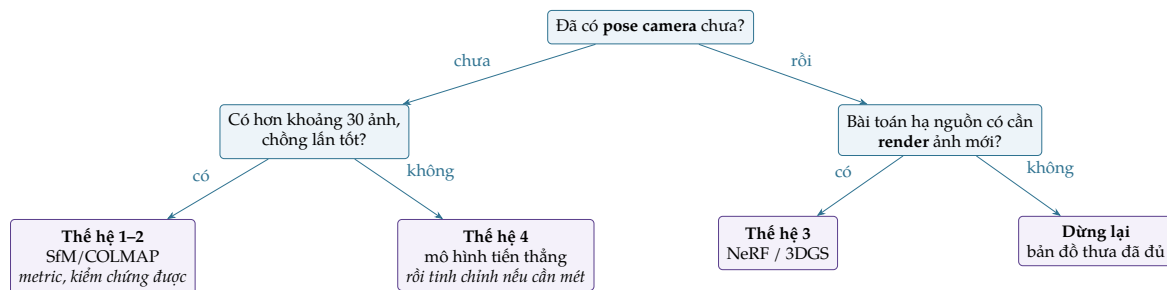
Giới hạn lớn nhất của cách kể “bốn thế hệ” là nó gợi ý một sự thay thế tuần tự, trong khi thực tế là chồng lớp. Pipeline mạnh nhất hiện nay thường lai, gồm ba chặng. Thứ nhất, dùng feed-forward geometry để có khởi tạo pose và point map thô, không cần khởi tạo thủ công. Thứ hai, ném khởi

tạo đó vào hiệu chỉnh chùm tia cổ điển để lấy lại độ chính xác metric. Thứ ba, dựng biểu diễn render lên trên. Thế hệ bốn giải bài toán *khởi tạo* (vốn là chỗ SfM hay chết), thế hệ một giải bài toán *tinh chỉnh*.

Các chế độ hỏng riêng:

- **Thế hệ một** hỏng khi giả định vật lý bị vi phạm — mặt kính, mặt nước, tường trắng, cảnh động — và nó hỏng *ôn ào*: số *inlier* tụt, phần dư tăng, hệ thống biết mình đang hỏng.
- **Thế hệ hai** kế thừa toàn bộ chỗ hỏng đó và thêm **lệch phân phối** (*domain shift*) của khâu học.
- **Thế hệ ba** hỏng ở ba chỗ: pose đầu vào sai, phơi sáng tự động thay đổi giữa các ảnh, hoặc cảnh có vật chuyển động. Riêng pose thì chỉ cần sai vài phần mười độ: ảnh render mờ đi, và tối ưu bù lại bằng vật chất giả lơ lửng.
- **Thế hệ bốn** hỏng khi cảnh ra khỏi phân phối huấn luyện — và đây là chế độ nguy hiểm nhất vì nó *im lặng*.

Gộp lại, bốn chế độ hỏng đó cho một quy tắc chọn dùng được ngay: Hình 16.5 hỏi theo đúng thứ tự các ràng buộc, từ ràng buộc cứng nhất trở đi.



Hình 16.5. Chọn thế hệ nào, hỏi theo đúng thứ tự. Câu hỏi đầu tiên không phải “phương pháp nào mới nhất” mà “tôi đã có pose chưa” — vì neural rendering nằm sau khâu pose nên nó không bao giờ là câu trả lời khi chưa có pose. Nhánh phải dưới cùng là nhánh hay bị bỏ quên nhất: rất nhiều bài toán robot chỉ cần định vị, và ở đó bản đồ thừa cổ điển vẫn là lựa chọn đúng.

CẠM BẮY

Cạm bẫy phổ biến nhất khi so sánh thế hệ bốn với COLMAP là so trên chính loại cảnh mà thế hệ bốn được huấn luyện. Nhiều bộ *benchmark* 3D lớn đã được dùng làm dữ liệu huấn luyện cho các mô hình tiến thẳng gần đây, nên con số “tốt hơn COLMAP” trên các bộ đó không nói được gì về cảnh của bạn. Phép thử trung thực duy nhất là chạy trên dữ liệu bạn tự quay. Và cách phát hiện lỗi duy nhất là kiểm tra chéo bằng một ràng buộc bên ngoài. Ba ràng buộc rẻ nhất: chiều dài một vật đã biết, một đoạn quỹ đạo có chân trị, hoặc phần dư sau khi ép kết quả tiến thẳng qua một vòng hiệu chỉnh chùm tia.

↪ **cùng nguyên lý với C/17/04** — khôi phục ảnh cũng đối ràng buộc cứng lấy prior học được, và cũng mất khả năng tự phát hiện sai theo đúng cách này.

↪ **tiền đề A/02** — mọi phép tam giác hoá đứng trên mô hình chiếu và giả định hiệu chỉnh; sai ở tầng đó thì không thể hệ nào cứu được.

CẦN NHỚ

Đi từ thế hệ một tới thế hệ bốn, thứ mất đi không chỉ là độ chính xác metric mà là *khả năng tự chẩn đoán*. Hiệu chỉnh chùm tia trả về phần dư và hiệp phương sai, nên hệ thống biết lúc nào nó không tin được; một mạng tiến thẳng trả về point map với cùng vẽ tự tin dù cảnh

nằm trong hay ngoài phân phối. Khi đưa thể hệ bốn vào hệ thống thật, việc đầu tiên phải làm là dựng lại một cơ chế phát hiện sai từ bên ngoài. [T3]

F. Kết nối

Mục này là khung để đọc hai mục còn lại của chương. Mục kế tiếp, [C/16-thi-giac-3d/02-bieu-dien-3d-va-depth](#), đi vào các quyết định con bên trong mỗi thể hệ; mục cuối, [C/16-thi-giac-3d/03-nerf-3dgs-va-hop-nhat](#), đào sâu đúng hai thứ lạ nhất với người làm SLAM.

Trục đánh đổi ở đây là trường hợp riêng, cụ thể nhất, của [B/04-nen-tang-hoc-tu-du-lieu](#). Cơ chế “prior thay ràng buộc cứng” sẽ trở lại nguyên vẹn dưới tên bài toán *ill-posed* ở [C/17-mo-hinh-sinh/04-low-level-vision](#). Mô hình chiếu và các giả định hiệu chỉnh nằm ở [A/02-vat-ly-tao-anh/03-camera-va-hinh-hoc-nhieu-view](#); còn nếu khâu tương ứng hỏng vì domain shift thì [B/08-dich-chuyen-phan-phoi](#) là chỗ đọc tiếp.

! Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao thể hệ lai “chỉ học khâu tương ứng” lại nguy hiểm hơn thể hệ cổ điển ở đúng một loại cảnh, dù RANSAC vẫn được giữ nguyên?
2. Neural rendering đúng *sau* khâu pose trong pipeline. Điều đó nói gì về câu “NeRF thay thế SfM”?
3. Một mạng tiến thẳng cho ra point map đẹp trên video bạn quay. Bạn có ba mươi phút để kiểm tra xem nó đúng hay chỉ trông đúng — bạn làm gì?
4. Sai số độ sâu stereo tăng theo Z^2 . Vì sao sai số của độ sâu đơn ảnh học được lại *không* tuân theo quy luật đó, và điều này ảnh hưởng thế nào tới việc gán trọng số cho nó trong một bộ lọc?
5. Nếu phải chọn đúng một thể hệ để đưa vào robot chạy 24/7 ngoài trời, bạn chọn thể hệ nào và dựng thêm cơ chế an toàn nào?

! 16.2 Chọn biểu diễn 3D, độ sâu và SLAM (3D Representations, Depth and SLAM)

PHỤ THUỘC [C/16-thi-giac-3d/01-bon-the-he-3d](#) (bốn thể hệ); [B/10-thiet-ke-kien-truc](#) (thiết kế theo bất biến); [B/07-chia-du-lieu-va-danh-gia](#) (ATE/RPE và giao thức đánh giá); [A/01-toan-toi-thieu/03-xac-suat-va-thong-tin](#) (tính xác định được của tham số).

TÓM TẮT

Mục này nói về ba quyết định con nằm bên trong mọi hệ thống 3D: lưu cảnh bằng biểu diễn nào, lấy độ sâu từ đâu, và học sâu làm gì được trên dữ liệu không phải lưới. Đọc xong bạn làm được ba việc. Chọn biểu diễn cho một pipeline mới mà không phải viết lại nó sáu tháng sau. Nhìn một mô-đun độ sâu là biết nó có quyền xuất hiện trong phần dư metric hay không. Và đọc một mạng trên point cloud để biết nó mù với cái gì. Nếu chỉ nhớ hai điều: biểu diễn quyết định *phép toán nào* *re*, nên phải chọn nó theo khâu cuối cùng của pipeline chứ không theo khâu đầu tiên. Và độ sâu đơn ảnh không phải phép đo mà là suy đoán có học, với một tỉ lệ trôi giữa các khung hình — đó chính là chỗ nó phá hỏng một bộ ước lượng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **voxel** — ô lập phương trong một lưới đều 3D — “pixel của không gian ba chiều”. Đơn giản nhưng số ô tăng theo lũy thừa ba của độ phân giải.
- **point cloud (point cloud)** — một tập toạ độ 3D không có thứ tự và không có kết nối giữa các điểm; đây là dạng dữ liệu thô của LiDAR và của stereo dày đặc.
- **mesh (mesh)** — biểu diễn bề mặt bằng các đỉnh nối thành tam giác; chuẩn của đồ hoạ máy tính, vì nó cho biết *đâu là mặt* chứ không chỉ *đâu có vật chất*.
- **SDF (signed distance function)** — một hàm gán cho mỗi điểm trong không gian khoảng cách có dấu tới bề mặt gần nhất — âm ở trong, dương ở ngoài. Bề mặt chính là tập điểm có giá trị 0, nên nó được định nghĩa rất sạch.
- **trường ẩn (implicit field)** — cách mô tả hình dạng bằng một *hàm* (thường là một mạng nhỏ) thay vì bằng một danh sách phần tử; SDF là ví dụ điển hình.
- **đường cơ sở (baseline)** — khoảng cách giữa hai tâm camera trong một hệ stereo; nó là thứ duy nhất cấp đơn vị mét cho phép đo độ sâu.
- **ATE và RPE** — hai chỉ số đánh giá quỹ đạo: ATE đo sai lệch tuyệt đối sau khi căn chỉnh toàn cục hai quỹ đạo, RPE đo sai lệch trên các đoạn ngắn nên không cần căn chỉnh. Bạn đã quen hai chỉ số này; ở đây chỉ dùng chúng làm ví dụ về việc “phép căn chỉnh che giấu điều gì”.
- **bất biến hoán vị (permutation invariance)** — tính chất của một hàm mà kết quả không đổi khi ta xáo trộn thứ tự các phần tử đầu vào; bắt buộc phải có nếu đầu vào là một *tập*.

A. Đặt vấn đề

Trong ảnh 2D không ai phải chọn biểu diễn: ảnh là một lưới pixel, hết chuyện. Trong 3D thì không có biểu diễn mặc định nào cả, và lựa chọn đó quyết định nhiều hơn cả việc chọn kiến trúc mạng. Lý do rất cụ thể: **biểu diễn quyết định phép toán nào rẻ**. Trên lưới *voxel*, tích chập là rẻ nhưng bộ nhớ tăng theo lũy thừa ba. Trên *point cloud*, bộ nhớ rẻ nhưng “lân cận” không có sẵn nên mọi phép cục bộ phải trả tiền tìm kiếm. Trên *mesh*, bề mặt là tường minh nên render và mô phỏng vật lý rẻ, nhưng thay đổi *topology* thì gần như không khả vi. Một khi đã chọn biểu diễn, bạn đã chọn luôn tập bài toán còn giải được hiệu quả — và đây là quyết định khó đảo ngược nhất trong một pipeline 3D.

Câu chuyện thứ hai là nguồn của độ sâu. Bốn thể hệ ở mục trước đều cần độ sâu từ đâu đó. Có ba cách lấy nó: một ảnh, hai ảnh, hoặc cảm biến chủ động. Chúng khác nhau không phải ở độ chính xác mà ở chỗ *cái nào xác định được tỉ lệ tuyệt đối*. Cuối cùng, ta đặt SLAM và VIO vào đúng vị trí: không phải một chủ đề riêng mà là hệ thống duy nhất bắt buộc giải cả ba bài toán — biểu diễn, độ sâu, tối ưu — dưới ràng buộc thời gian thực.

Học mục này thì làm được gì mà trước đó không làm được? Ba việc: chọn được biểu diễn cho một pipeline mới mà không phải viết lại nó sáu tháng sau; nhìn một mô-đun độ sâu và nói ngay nó có quyền xuất hiện trong phần dư metric hay không; và đọc được một mô hình học trên point cloud để biết nó mù với cái gì.

TRỰC GIÁC

Chọn biểu diễn 3D giống chọn hệ toạ độ để viết một bài toán cơ học. Bài toán không đổi, nhưng viết trong hệ Descartes thì trọng lực đơn giản còn chuyển động tròn phức tạp; trong hệ cực thì ngược lại. Voxel là hệ Descartes của 3D: đều đặn, dễ lập trình, và lãng phí khủng khiếp vì phần lớn không gian là rỗng. Point cloud là “danh sách các chỗ có vật chất”: gọn đúng bằng lượng thông tin thật, nhưng bạn phải tự dựng lại khái niệm hàng xóm mỗi lần cần tới nó.

B. Năm biểu diễn và cái giá của mỗi cái

Năm biểu diễn dưới đây không ngang hàng nhau. Chúng chia làm ba nhóm theo *nguồn gốc*, và biết nhóm nào giúp đoán ngay chỗ hỏng:

- **Biểu diễn đo được** — voxel và point cloud, ra thẳng từ cảm biến. Chỗ hỏng của chúng là chỗ hỏng của cảm biến: mật độ không đều, vùng khuất, nhiễu phụ thuộc khoảng cách.
- **Biểu diễn đồ hoạ** — mesh. Chỗ hỏng là bước tái dựng bề mặt đứng trước nó.
- **Biểu diễn được tối ưu** — **trường ẩn** (*implicit field*) và **Gaussian splatting**. Chúng không tồn tại cho tới khi chạy xong một vòng tối ưu trên chính cảnh đó, nên chỗ hỏng của chúng là chỗ hỏng của hàm mất mát đã sinh ra chúng.

Bảng 16.2. Năm biểu diễn 3D. Cột cuối là cột quyết định: mỗi biểu diễn hỏng ở một chỗ khác nhau, và chỗ đó thường lộ ra muộn.

Biểu diễn	Cơ chế lưu	Được	Mất / hỏng khi
Voxel	Lưới đều 3D, mỗi ô một giá trị	Conv 3D dùng được nguyên xi; lân cận cho sẵn	Bộ nhớ theo lũy thừa ba; hỏng ngay khi cần độ phân giải cao
Point cloud	Tập điểm không thứ tự	Gọn, hợp với LiDAR và stereo	Không có cấu trúc lân cận; phải bất biến hoán vị; mật độ không đều
Mesh	Đỉnh + mặt	Chuẩn của đồ hoạ và mô phỏng; bề mặt tường minh	Tối ưu topology gần như không khả vi; mặt tự cắt nhau
Implicit field (SDF, occupancy)	Một hàm $\mathbb{R}^3 \rightarrow \mathbb{R}$, thường là MLP	Độ phân giải liên tục; bề mặt là tập mức 0, định nghĩa sạch	Render chậm vì phải lấy mẫu nhiều lần dọc tia; khó chỉnh sửa cục bộ
Gaussian splatting	Tập Gaussian dị hướng có màu và độ đục	Rasterize được nên render rất nhanh	Dung lượng lớn; hình học không tường minh nên trích mesh khó

Hình 16.6 vẽ cùng một mặt cong bằng cả năm biểu diễn, đặt cạnh nhau — đây là cách nhanh nhất để thấy mỗi cái giữ lại gì và vứt đi gì.

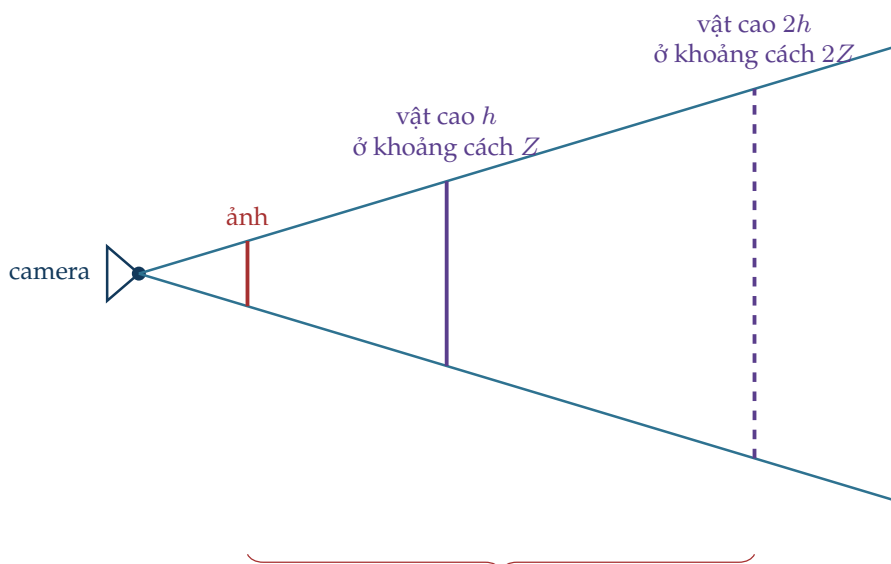


Hình 16.6. Cùng một mặt cong, năm biểu diễn. Nhìn hàng chú thích dưới cùng thì thấy ngay điều mà bảng không nói được: mỗi biểu diễn giữ lại một loại thông tin và vứt đi loại khác. Point cloud biết ở đâu có vật chất nhưng không biết đâu là mặt; voxel biết cả không gian rỗng nhưng trả tiền cho nó; mesh biết mặt nhưng khoá cứng topology; SDF định nghĩa mặt sạch nhất nhưng phải dò dọc tia mới render được; Gaussian render nhanh nhất nhưng không có mặt để mà trích ra.

Con số làm cho dòng đầu bảng thành cụ thể. Một lưới 512^3 chứa khoảng $1,34 \times 10^8$ ô; lưu một số thực 32-bit cho mỗi ô đã tốn 512 MiB, và một tầng conv 3D với 32 kênh ở cùng độ phân giải cần 16 GiB chỉ để giữ *activation*. Nâng lên 1024^3 thì nhân tám. Trong khi đó một quét LiDAR dày đặc của cùng cảnh có chừng 2×10^5 điểm, tức 2,4 MB nếu lưu ba tọa độ float — ít hơn hai bậc độ lớn. Toàn bộ dòng **tích chập thưa** (*sparse convolution*) tồn tại chỉ vì khoảng cách này: hơn 99% ô của lưới đều là rỗng, nên trả tiền cho chúng là trả tiền cho hư không.

C. Độ sâu đến từ đâu, và cái gì cố định tỉ lệ

Với ảnh đơn, bài toán độ sâu không phải khó — nó *không xác định được*. Nhân toàn bộ cảnh với hệ số s và nhân luôn mọi vị trí camera với s thì mọi ảnh thu được giống hệt nhau. Hình 16.7 là toàn bộ chứng minh, vẽ ra trong một hình.



Hai cảnh khác nhau, **một ảnh duy nhất**. Không phép đo nào trong ảnh phân biệt được chúng.

Hình 16.7. Tính không xác định được của tỉ lệ trong ảnh đơn. Nhân cảnh với s và nhân quãng đường camera với cùng s cho ra đúng bộ ảnh cũ; vì vậy *không lượng dữ liệu nào* chứa được, chỉ prior mới chứa được. Đây cũng chính là lý do SLAM đơn mắt có một bậc tự do tỉ lệ, và vì sao chỉ cần thêm một gia tốc kế — một cảm biến biết đơn vị mét — là bậc tự do đó biến mất.

Đây đúng là hiện tượng **không xác định được** (*non-identifiability*) ở chương 4: dữ liệu không chứa thông tin để phân biệt hai giả thuyết.

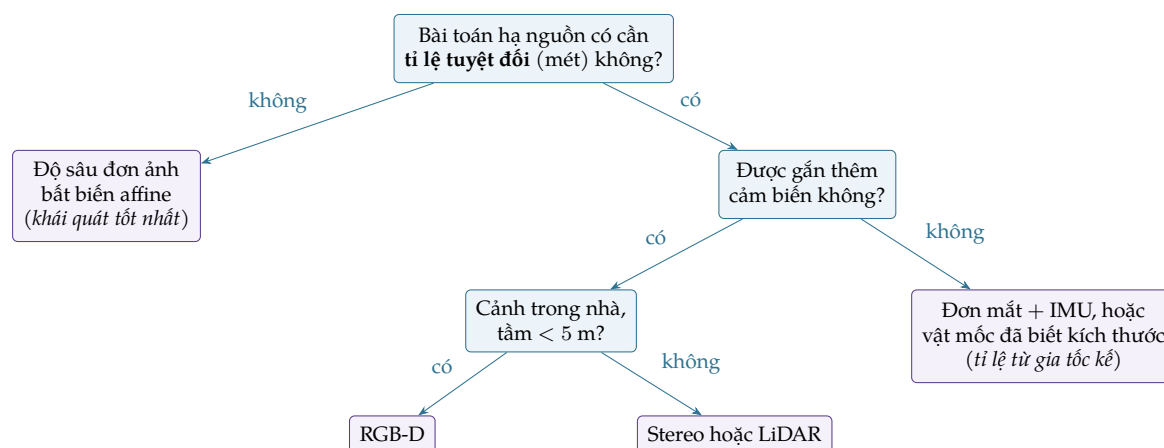
↪ **trường hợp riêng của B/04** — scale ambiguity là một ca sạch của tính không xác định được: hai tham số khác nhau sinh ra cùng một phân phối quan sát, nên thêm dữ liệu không giúp gì.

Một mạng độ sâu đơn ảnh phá thế bế tắc bằng cách học kích thước điển hình của vật thể và mối liên hệ với tiêu cự; nghĩa là con số nó trả về là một *suy đoán có học*, không phải một phép đo. Nó vì thế sai theo kiểu rất đặc trưng ở đúng những cảnh phá vỡ prior kích thước: mô hình thu nhỏ, ảnh chụp từ trên cao, nội soi, ảnh macro.

Bốn nguồn độ sâu, đọc theo cái mà mỗi nguồn dùng để phá thế bế tắc:

- **Đơn ảnh, metric** — phá bằng prior kích thước. Cho ra mét ngay, nhưng chỉ đúng trong miền đã học.
- **Đơn ảnh, bất biến affine** — không phá, mà *bỏ* tỉ lệ đi: huấn luyện trên nhiều tập dữ liệu hỗn tạp với hàm mất mát bất biến với phép biến đổi affine của độ sâu, nên mô hình chỉ phải học thứ tự và hình dạng tương đối [31], [58]. Khái quát rất tốt; đổi lại mỗi khung hình còn dư hai ẩn (tỉ lệ và độ dịch) và hai ẩn đó *không nhất quán giữa các khung hình*.
- **Stereo** — phá bằng vật lý: đường cơ sở đã biết cho tỉ lệ. Cần hiệu chỉnh chính xác, cần texture để khớp, và có chân trời độ sâu tính được từ $\Delta Z = Z^2 \Delta d / (fb)$.
- **RGB-D** — phá bằng cảm biến chủ động: chiếu ánh sáng ra và đo phản hồi, nên có tỉ lệ tuyệt đối ngay cả trên tường trắng. Chết dưới nắng, trên mặt gương, trên vật hấp thụ hồng ngoại; có sai lệch phụ thuộc khoảng cách cần hiệu chuẩn riêng.

Hình 16.8 gói bốn lựa chọn đó thành một cây quyết định dùng được ngay.



Hình 16.8. Cây quyết định chọn nguồn độ sâu. Câu hỏi đầu tiên không phải “mô hình nào chính xác nhất” mà “bài toán của tôi có thật sự cần mét không”. Rất nhiều bài toán chỉ cần độ sâu tương đối: tránh vật cản gần, sắp thứ tự lớp, tách nền. Ở đó lựa chọn bất biến affine vừa rẻ vừa bền hơn hẳn.

D. SLAM và VIO đứng ở đâu

Bạn đọc chương này đã sống trong SLAM nên mục này không nhắc lại cơ chế; nó chỉ đặt SLAM vào bức tranh bốn thế hệ. SLAM là thế hệ một chạy dưới ràng buộc thời gian thực, và toàn bộ kiến trúc *frontend/backend* là hệ quả trực tiếp của ràng buộc đó: frontend phải quyết định tham lam trong vài mili-giây mỗi khung hình nên chỉ dùng ràng buộc cục bộ, còn backend là nơi hình học thực sự được ép đúng trên một cửa sổ hoặc trên toàn đồ thị [40], [59]. **Đóng vòng** (*loop closure*) không phải một tính năng phụ mà là thứ duy nhất đối *bậc* của sai số: không có nó, sai số tích lũy tăng đơn điệu theo quãng đường; có nó, sai số bị chặn bởi độ chính xác của phép đóng vòng.

Bốn thế hệ chạm vào SLAM ở ba chỗ khác nhau:

- **Thế hệ lai thay frontend** — đặc trưng và khớp học được, backend giữ nguyên. Đây là hướng đã thành thực dụng.
- **Thế hệ lai, bản triệt để hơn** — dòng DROID-SLAM dùng **dòng quang** (*optical flow*) dày đặc học được nhưng vẫn nhét một tầng hiệu chỉnh chùm tia khả vi vào giữa, nên nó vẫn là thế hệ hai chứ không phải thế hệ bốn.
- **Thế hệ ba thay bản đồ** — giữ tracker cổ điển nhưng lưu bản đồ bằng trường bức xạ hoặc Gaussian. Đánh đổi bản đồ thưa lấy khả năng render: đáng giá cho robot cần nhìn lại cảnh, đáng ngờ cho robot chỉ cần định vị.

Về đánh giá chỉ cần một câu nhắc: *ATE* cần một phép căn chỉnh toàn cục trước khi đo, vì quỹ đạo ước lượng sống trong một hệ tọa độ tự do — với đơn mắt thì tự do cả tỉ lệ. Chính phép căn chỉnh đó hấp thụ một phần *drift* toàn cục. *RPE* thì đo trên các đoạn ngắn nên không cần căn chỉnh, và vì thế nhạy với lỗi cục bộ mà *ATE* làm phẳng đi [71]. Hai chỉ số phải đọc cùng nhau, và mọi thảo luận sâu hơn về giao thức — chia dữ liệu, số lần lặp, khoảng tin cậy — nằm ở [B/07-chia-du-lieu-va-danh-gia](#).

↔ **ràng buộc bởi B/07** — phép căn chỉnh của *ATE* là một bước tiền xử lý của giao thức đánh giá, nên nó quyết định lỗi nào còn nhìn thấy được.

E. Học sâu trên point cloud

Muốn đưa mạng nơ-ron lên point cloud thì vấp ngay vào một trở ngại cấu trúc: đầu vào là một *tập*, không phải chuỗi hay lưới. Với n điểm có $n!$ cách đánh số cùng mô tả một hình dạng, nên bất kỳ hàm nào không bất biến với hoán vị đều đang học lại cùng một hình dạng hàng giai thừa lần.

I Mạch 2 — từ tập điểm tới mạng dùng được

- 1 Vấn đề** đầu vào là tập không thứ tự, mà MLP thường thì không bất biến hoán vị.

Giải pháp ép mọi điểm qua cùng một MLP chia sẻ trọng số rồi gộp bằng một **hàm đối xứng** (*symmetric function*) — lấy max theo từng chiều đặc trưng [171].

Giả định hình dạng mô tả được đủ tốt bằng một vectơ đặc trưng toàn cục.

Hệ quả bất biến hoán vị ở đây là bất biến *theo cấu trúc*, không phải học được từ **tăng cường dữ liệu** (*data augmentation*), nên nó đúng tuyệt đối chứ không đúng gần đúng.
- 2 Vấn đề** max toàn cục vứt bỏ mọi cấu trúc cục bộ: đầu ra chỉ phụ thuộc một tập điểm “tới hạn” rất nhỏ — những điểm đang giữ giá trị lớn nhất ở mỗi chiều.

Giá phải trả thêm hay bớt điểm bên trong bao lồi của tập tới hạn không đổi đầu ra: rất bền với thay đổi mật độ, nhưng mù với chi tiết bề mặt mịn.

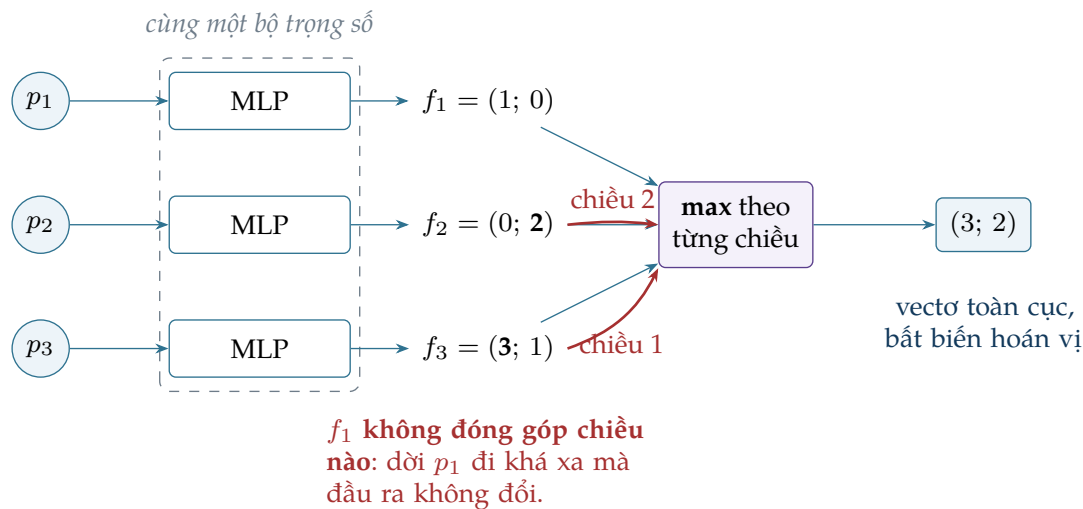
Giải pháp lồng PointNet vào chính nó theo phân cấp không gian — nhóm điểm theo bán kính, chạy PointNet trên từng nhóm, rồi lặp lại ở tỉ lệ lớn hơn [172].

Vấn đề mới chi phí truy vấn lân cận thành nút cổ chai, và kết quả nhạy với việc chọn bán kính.
- 3 Vấn đề** ở quy mô một cảnh đường phố với hàng trăm nghìn điểm, truy vấn lân cận là quá đắt.

Giải pháp quay lại lưới nhưng chỉ giữ ô không rỗng (*sparse convolution*), hoặc gộp điểm theo cột dọc thành *pillar* rồi rơi về một bài toán 2D quen thuộc [173].

Giả định cảnh ngoài trời gần như phẳng theo phương thẳng đứng nên nén trục z mất ít thông tin.

Giá phải trả giả định đó sai ngay trong nhà, nơi vật thể chồng lên nhau theo chiều cao.



Hình 16.9. Hàm đối xứng của PointNet, vẽ trên đúng ba điểm của ví dụ tính tay. Mọi điểm đi qua cùng một MLP rồi được gộp bằng phép max theo từng chiều, nên xáo trộn thứ tự ba điểm cho ra cùng một vector $(3; 2)$ — tính bất biến ở đây là bất biến theo cấu trúc, không phải học được. Nhưng hình cũng phơi ra cái giá: đầu ra chỉ do *tập điểm tới hạn* quyết định (ở đây là f_2 và f_3), còn f_1 hoàn toàn vô hình. Đúng cái mẹo giữ cho mạng bất biến cũng là đúng cái mẹo làm nó mù.

Hình 16.9 vẽ đúng cơ chế đó trên ba điểm.

Tính tay để thấy hàm đối xứng thật sự làm gì. Lấy ba điểm đã qua MLP thành các đặc trưng hai chiều $f_1 = (1; 0)$, $f_2 = (0; 2)$, $f_3 = (3; 1)$. Gộp bằng max theo từng chiều cho $(3; 2)$. Đảo thứ tự ba điểm bằng bất kỳ hoán vị nào vẫn cho $(3; 2)$ — đó là toàn bộ mẹo của PointNet. Nhưng phép tính đó cũng phơi ra ngay giới hạn: chiều thứ nhất chỉ do f_3 đóng góp, chiều thứ hai chỉ do f_2 ; điểm f_1 hoàn toàn không ảnh hưởng, và bạn có thể dời nó đi khá xa mà vector toàn cục không nhúc nhích. *Đúng cái mẹo giữ cho mạng bất biến cũng là đúng cái mẹo làm nó mù.* Cùng cơ chế này sẽ xuất hiện lại ở [C/17-mo-hình-sinh/02-toi-uu-tren-pixel](#), nơi ma trận Gram vút bỏ vị trí không gian để giữ lại “phong cách” — hai kỹ thuật trông không liên quan, một nguyên lý.

↪ **cùng nguyên lý với C/17/02** — ma trận Gram gộp trên vị trí đúng như PointNet gộp trên điểm: một hàm đối xứng mua tính bất biến bằng chính thông tin bị gộp đi.

↪ **trường hợp riêng của B/10** — bất biến hoán vị là một ví dụ của việc cài thiên lệch quy nạp thẳng vào kiến trúc thay vì học nó từ dữ liệu.

F. Giới hạn và trường hợp hỏng

Điểm yếu chung của cả mục này là các biểu diễn không chuyển đổi qua lại mà không mất mát. Điểm sang mesh cần một bước tái dựng bề mặt vốn tự nó là bài toán ước lượng có prior riêng; mesh sang SDF cần mesh kín, điều gần như không đúng với mesh quét thực tế; Gaussian sang mesh là bài toán mở. Vì vậy lỗi thiết kế đắt nhất trong một pipeline 3D là chọn biểu diễn theo khâu đầu tiên thay vì theo khâu cuối cùng cần dùng kết quả.

Hai chế độ hỏng đáng nhớ riêng:

- **Độ sâu học được:** nguy hiểm nhất không phải sai số lớn mà là sai số *không nhất quán theo thời gian* — mỗi khung hình một tỉ lệ khác nhau trong khi từng khung hình đều trông hợp lý.
- **Point cloud:** thay đổi mật độ giữa lúc huấn luyện và lúc chạy. Mô hình huấn luyện trên LiDAR 64 tia thường tụt mạnh trên 32 tia, và triệu chứng là *mất vật ở xa trước tiên* — đúng nơi mật độ giảm nhanh nhất.

CẠM BẦY

Cạm bẫy quen thuộc với người làm SLAM: lấy độ sâu đơn ảnh làm phép đo trong một bộ ước lượng có trọng số. Nó không những sai tỉ lệ, mà tỉ lệ đó còn *trôi giữa các khung hình*. Sai số của nó vì thế có tương quan mạnh theo thời gian. Đó là vi phạm đúng giả định nhiều trắng độc lập mà bộ lọc dựa vào, và hệ quả là hiệp phương sai bị đánh giá quá lạc quan. Có hai cách dùng an toàn. Cách thứ nhất: coi nó như ràng buộc *tương đối* bên trong một khung hình, ví dụ thứ tự độ sâu hay tính phẳng của mặt. Cách thứ hai: ước lượng tường minh một tỉ lệ và một độ dịch cho mỗi khung hình, rồi đưa hai ẩn đó vào bài toán tối ưu. Cách sai là nhét mét thẳng vào phần dư.

CẦN NHỚ

Biểu diễn quyết định phép toán nào rẻ, và vì thế quyết định bài toán nào bạn còn giải được về sau: hãy chọn nó theo khâu *cuối cùng* của pipeline, không theo khâu đầu tiên. Còn về độ sâu: chỉ có baseline và cảm biến chủ động cho ra mét; mọi thứ khác cho ra một suy đoán có học, và suy đoán đó không được phép xuất hiện trong phần dư metric mà không kèm hai ẩn tỉ lệ và độ dịch. [T4]

G. Kết nối

Mục này cung cấp vốn từ cho [C/16-thi-giac-3d/03-nerf-3dgs-va-hop-nhat](#), nơi hai biểu diễn cuối bảng được đào sâu như hai câu trả lời cho cùng một bài toán render. Bất biến hoán vị của PointNet là trường hợp riêng của nguyên tắc thiết kế theo bất biến ở [B/10-thiet-ke-kien-truc](#); tính không xác định được của tỉ lệ là trường hợp riêng của [B/04-nen-tang-hoc-tu-du-lieu](#). Phần ATE/RPE nổi thẳng sang [B/07-chia-du-lieu-va-danh-gia](#), còn tụt hiệu năng khi đổi mật độ cảm biến là một ca của [B/08-dich-chuyen-phan-phoi](#).

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao bộ nhớ voxel không phải vấn đề kỹ thuật có thể tối ưu đi mà là hệ quả cấu trúc của biểu diễn?
2. Độ sâu đơn ảnh bất biến affine tổng quát tốt hơn độ sâu metric. Cái giá của tính tổng quát đó hiện ra ở đâu khi ghép nhiều khung hình?
3. PointNet bất biến hoán vị “theo cấu trúc”. Vì sao điều đó mạnh hơn hẳn việc học tính bất biến bằng augmentation, và nó khiến mạng mù với cái gì?
4. Nếu loop closure bị tắt, sai số của SLAM đổi bậc như thế nào, và chỉ số nào trong hai chỉ số ATE/RPE phản ánh sự thay đổi đó rõ hơn?
5. Một mô hình phát hiện vật thể trên LiDAR 64 tia được đưa sang cảm biến 32 tia. Bạn dự đoán triệu chứng đầu tiên là gì, và vì sao?
6. Hình 16.7 nói rằng không lượng dữ liệu nào phá được tính không xác định của tỉ lệ. Vậy vì sao một mạng “metric depth” vẫn cho ra mét, và điều đó ngầm giả định gì?

16.3 NeRF, 3DGS và hợp nhất cảm biến (Neural Rendering and Sensor Fusion)

PHỤ THUỘC [C/16-thi-giac-3d/01-bon-the-he-3d](#) (thế hệ ba nằm ở đâu trong pipeline); [C/16-thi-giac-3d/02-bieu-dien-3d-va-depth](#) (trường ẩn và Gaussian trong bảng biểu diễn); [A/02-vat-ly-tao-anh](#) (phơi sáng, cân bằng trắng, hiệu chỉnh ngoại).

TÓM TẮT

Mục này giải thích cơ chế thật của *neural rendering*: vì sao NeRF phải biểu diễn cảnh bằng mật độ liên tục chứ không bằng bề mặt, và 3DGS đổi cái gì lấy tốc độ. Nửa sau chuyển sang bài toán hợp nhất camera–LiDAR–IMU. Đọc xong bạn quyết định được khi nào một bản đồ render được là đáng giá cho robot, và khi nào bản đồ thừa cổ điển vẫn đúng hơn. Bạn cũng tính được bằng con số vì sao *early fusion* nhạy với lệch hiệu chỉnh còn *late fusion* thì không. Nếu chỉ nhớ một điều: hàm mất mát của neural rendering chỉ ràng buộc *ảnh render*, không ràng buộc *hình học*. Nên PSNR cao không chứng minh bản đồ đúng. Nếu robot sắp va chạm dựa trên bản đồ đó, phải đánh giá bằng độ chính xác hình học so với một quét chân trị.

KIẾN THỨC CẦN CÓ TRƯỚC

- **novel view synthesis (dựng ảnh ở góc nhìn mới)** — bài toán: cho một số ảnh đã chụp của một cảnh, hãy tạo ra ảnh của cảnh đó từ một vị trí camera chưa từng chụp.
- **radiance field (trường bức xạ)** — một hàm gán cho mỗi điểm trong không gian và mỗi hướng nhìn một màu và một độ đặc; đây là “cảnh” mà NeRF học, thay cho bề mặt.
- **volume rendering (dựng ảnh thể tích)** — cách tổng hợp một pixel bằng cách lấy mẫu nhiều điểm dọc theo tia nhìn rồi cộng dồn màu có trọng số theo độ đục tích lũy — thay vì tìm giao điểm với một bề mặt cứng.
- **splat / Gaussian splatting** — “splat” là vết loang một hạt để lại khi bị chiếu xuống mặt ảnh. 3DGS mô tả cảnh bằng hàng triệu hạt Gaussian mờ, và render là chiếu chúng xuống rồi trộn theo thứ tự độ sâu.
- **rasterize** — chuyển một hình học đã chiếu thành pixel bằng cách quét trực tiếp trên lưới ảnh; nhanh hơn nhiều so với đi dò dọc từng tia, và là thứ GPU sinh ra để làm.
- **PSNR** — chỉ số đo sai lệch ảnh dựa trên bình phương trung bình theo pixel, tính bằng decibel; cao là “gần ảnh gốc theo nghĩa số học”, không phải “trông giống theo nghĩa thị giác”.
- **hiệu chỉnh ngoại (extrinsic calibration)** — phép xác định vị trí và hướng tương đối giữa hai cảm biến; sai một độ ở đây biến thành sai vài chục centimét ở tâm hai chục mét.
- **early / late fusion** — hai tầng hợp nhất cảm biến: gộp ở mức đặc trưng (sớm, giàu thông tin) hoặc gộp ở mức kết quả từng nhánh (muộn, bền hơn).

A. Đặt vấn đề

Giả sử SfM đã chạy xong: bạn có pose của bốn mươi tám ảnh và một point cloud thừa. Bây giờ hãy render cảnh đó từ một góc nhìn chưa từng chụp. Kết quả sẽ tệ, và tệ theo cách rất đặc trưng: thủng lỗ ở chỗ không có điểm, viền răng cưa ở biên vật thể, mặt kính và mặt kim loại sai hoàn toàn vì màu của chúng phụ thuộc góc nhìn còn point cloud chỉ lưu được một màu cho mỗi điểm. Vấn đề nằm ở chỗ pipeline cổ điển tối ưu cho *độ chính xác hình học* rồi hy vọng ảnh đẹp theo sau, trong khi cái ta cần ở đây là ngược lại.

Neural rendering đảo ngược thứ tự đó bằng một ý tưởng đơn giản tới mức khó chịu: *nếu thứ ta muốn là ảnh, hãy đặt ảnh vào hàm mất mát*. Tham số hoá cảnh bằng một biểu diễn nào đó, viết một bộ render **khả vi** (*differentiable*) từ biểu diễn đó ra ảnh, rồi hạ **gradient** trên tham số của biểu diễn cho tới khi ảnh render trùng khít mọi ảnh quan sát. Không có bước tìm tương ứng, không tam giác hoá, không ràng buộc epipolar nào được viết ra — hình học nếu có xuất hiện thì xuất hiện như một *tác dụng phụ* của việc khớp ảnh. Câu “nếu có” đó là toàn bộ nội dung mục này.

Học mục này thì làm được gì mà trước đó không làm được? Ba việc rất cụ thể: đánh giá được một đề xuất kiểu “thay bản đồ SLAM bằng Gaussian” thay vì phải tin hay không tin; đọc được một bài neural rendering và chỉ ra ngay giả định nào trong ba giả định nền của nó bị dữ liệu robot vi phạm; và biến câu “hiệu chỉnh phải chính xác” thành một ngân sách sai số tính bằng centimét ở một tầm xa cụ thể.

TRỰC GIÁC

Vì sao NeRF dựng khối chứ không dựng tường? Vì đạo hàm. Nếu cảnh gồm các bề mặt cứng, dịch một bề mặt đi một phần nghìn mét thì một pixel hoặc đổi màu hẳn, hoặc không đổi gì cả. Hàm mất mát khi đó là bậc thang: gradient bằng không gần như mọi nơi, và **hạ gradient** (*gradient descent*) không biết vận về hướng nào. Thay bề mặt cứng bằng một trường mật độ liên tục thì mỗi pixel trở thành tổng có trọng số của hàng chục điểm dọc theo tia, và *mọi* điểm đều nhận được một phần gradient. Đây là phép đổi một công tắc bật/tắt thành một chiết áp: chỉ chiết áp mới nói được nên vận theo chiều nào.

B. NeRF: cảnh là một hàm, ảnh là tích phân dọc tia

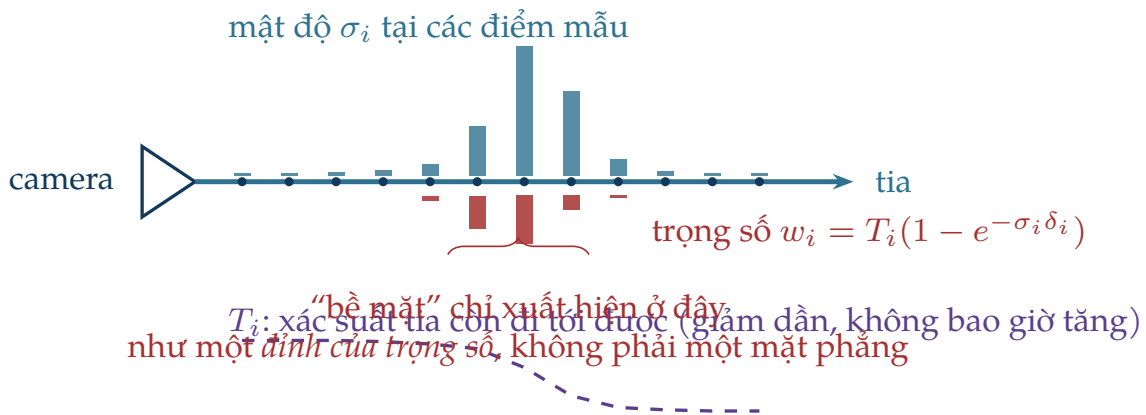
NeRF biểu diễn cảnh bằng một MLP nhận toạ độ 3D và hướng nhìn, trả về mật độ σ và màu c [28]. Để render một pixel, lấy mẫu N điểm dọc tia đi qua pixel đó rồi tổng hợp theo công thức dựng ảnh thể tích rồi raster hoá:

$$C(\mathbf{r}) = \sum_{i=1}^N T_i (1 - e^{-\sigma_i \delta_i}) c_i, \quad T_i = \exp\left(-\sum_{j<i} \sigma_j \delta_j\right).$$

Công thức này thực sự nói gì: T_i là xác suất tia đi tới được điểm thứ i mà chưa bị chặn, còn $1 - e^{-\sigma_i \delta_i}$ là xác suất nó dừng lại đúng ở đó; vậy màu pixel là kỳ vọng của màu tại điểm tia dừng lại. Điều đáng nhớ là mọi số hạng đều khả vi theo σ và c , nên gradient của sai số ảnh chảy ngược được về mọi điểm mẫu — đó là toàn bộ lý do NeRF hoạt động; phần MLP chỉ là một lựa chọn triển khai, và mục sau sẽ thay nó.

→ **tiền đề C/12/01** — toàn bộ cơ chế đứng trên một điều duy nhất: mọi nút trên đường từ tham số tới ảnh phải khả vi.

Hình 16.10 vẽ đúng ba đại lượng trong công thức trên một tia duy nhất.

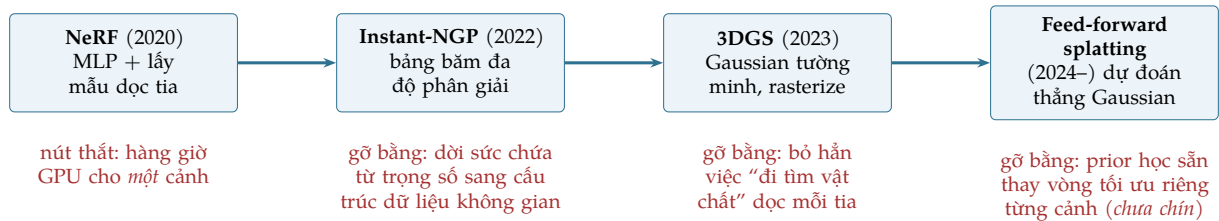


Hình 16.10. Dựng ảnh thể tích, vẽ trên một tia. Mạng không trả lời câu hỏi “mặt ở đâu” mà trả lời “mật độ tại điểm này bao nhiêu” ở hàng chục điểm dọc tia; màu pixel là tổng có trọng số của màu tại các điểm đó. Ba hàng trong hình giải thích công thức: mật độ σ_i quyết định tia dừng lại bao nhiêu ở mỗi điểm, T_i là phần tia còn sống sót tới đó, và tích của hai thứ cho trọng số. Vì mọi số hạng đều liên tục, *mọi* điểm mẫu đều nhận được gradient — đó chính là điều mà một bề mặt cứng không làm được.

Một chi tiết dễ bỏ qua nhưng quyết định chất lượng là **mã hoá vị trí** (*positional encoding*): nhồi thẳng toạ độ vào MLP cho ra ảnh mờ nhoè, vì MLP có thiên lệch mạnh về hàm tần số thấp. Ánh xạ toạ độ qua một dãy sin/cos nhiều tần số trước khi đưa vào mạng là cách trả lại cho mạng khả năng biểu diễn chi tiết sắc nét — cùng câu chuyện *spectral bias* đã gặp ở B/10-thiet-ke-kien-truc.

→ **hệ quả của B/10** — mã hoá vị trí là cách bù cho một thiên lệch quy nạp có sẵn của MLP, chứ không phải một mẹo kỹ thuật rời rạc.

Cái giá là chi phí tính toán, và nó lớn hơn trực giác nhiều. Một ảnh 800×600 có 480 000 tia; với 64 điểm mẫu thô cộng 128 điểm mẫu tinh, ta cần khoảng $9,2 \times 10^7$ lần chạy MLP cho *một* khung hình. Nhân với hàng chục nghìn bước huấn luyện thì thành nhiều giờ GPU cho một cảnh duy nhất — và đây là nút thắt sinh ra cả dòng công trình tiếp theo, tóm tắt ở Hình 16.11.



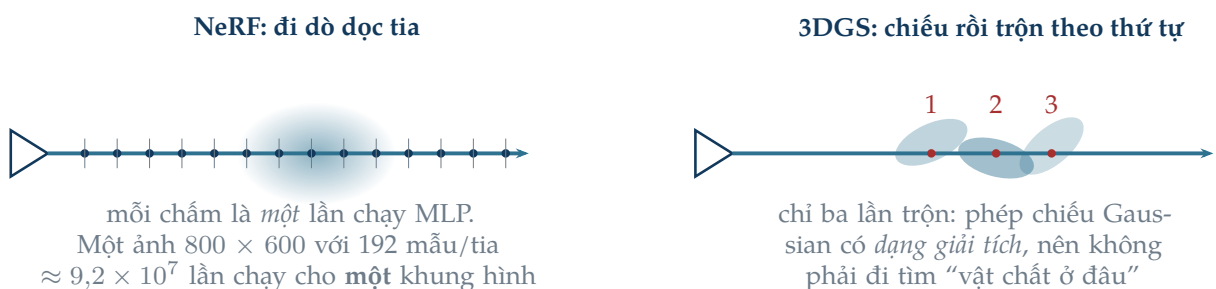
Hình 16.11. Dòng thời gian của neural rendering, đọc theo nút thắt. Mỗi bước không phải “phiên bản tốt hơn” mà là câu trả lời cho đúng một nút cổ chai của bước trước; biết nút thắt nào đang cản trong bài toán của mình thì biết nên dừng ở mốc nào.

Instant-NGP tấn công đúng chi phí đó. Nó chuyển phần lớn sức chứa của mô hình từ trọng số MLP sang một bảng băm đa độ phân giải đặt trong không gian, để MLP chỉ còn nhỏ xíu [174]. Thời gian huấn luyện tụt từ hàng giờ xuống hàng chục giây. Đây là một mẫu hình sẽ gặp lại nhiều lần: khi mạng phải học một hàm có cấu trúc không gian rõ ràng, đưa cấu trúc đó vào *cấu trúc dữ liệu* rẻ hơn nhiều so với bất trọng số học lại nó.

C. 3DGS: đối biểu diễn ẩn lấy biểu diễn tường minh

Ngay cả với bảng băm, NeRF vẫn phải đi dọc từng tia và lấy mẫu — vẫn “đi tìm vật chất ở đâu” cho mỗi pixel mỗi khung hình. 3D Gaussian splatting bỏ hẳn bước tìm kiếm đó: cảnh là một tập hữu hạn Gaussian dị hướng, mỗi cái có vị trí, ma trận hiệp phương sai, độ đục và màu phụ thuộc góc nhìn; render là chiếu từng Gaussian xuống mặt phẳng ảnh, sắp xếp theo độ sâu, rồi trộn theo thứ tự [175]. Toán tổng hợp màu vẫn đúng công thức trên, nhưng phép chiếu có dạng giải tích nên bộ rasterize chạy được ở tốc độ thời gian thực.

Hình 16.12 đặt hai cách tính cùng một pixel cạnh nhau, và cho thấy khoản tiết kiệm đến từ đâu.



Hình 16.12. Cùng một tia, hai cách tính màu pixel. Bên trái, NeRF phải lấy mẫu dày dọc tia vì nó *không biết* trước vật chất nằm ở đâu — toàn bộ chi phí đến từ việc đi tìm. Bên phải, 3DGS đã lưu sẵn vật chất thành các nguyên thể tường minh, nên chỉ cần chiếu chúng xuống mặt ảnh, sắp xếp theo độ sâu và trộn ba lần. Toán tổng hợp màu ở hai bên là một; thứ thay đổi là ai đi tìm — và đó là toàn bộ khoản chênh hai tới ba bậc độ lớn về tốc độ render.

Bảng 16.3. Ba mốc của neural rendering. Cả ba chia chung một tập giả định — pose đã biết, cảnh tĩnh, phơi sáng nhất quán — nên hồng chung ở những chỗ đó, và hồng riêng ở cột cuối.

Phương pháp	Cơ chế	Chi phí	Hồng khi / mất gì
NeRF (MLP)	MLP + lấy mẫu dọc tia + tích phân thể tích	Huấn luyện hàng giờ; render < 1 fps; mô hình vài MB	Chậm cả hai đầu; không chỉnh sửa được cục bộ
Instant-NGP	Bảng băm đa độ phân giải thay phần lớn trọng số	Huấn luyện hàng chục giây; tốn VRAM	Va chạm băm gây nhiễu ở vùng chi tiết
3D Gaussian splatting	Tập Gaussian tường minh, rasterize có sắp xếp	Render thời gian thực; mô hình vài trăm MB	Dung lượng lớn; bề mặt không tường minh; nhạy với nhiễu tạo từ SfM

Tính tay dung lượng một lần cho nhớ. Mỗi Gaussian lưu 3 số vị trí, 3 số tỉ lệ, 4 số quaternion, 1 số độ đục và 48 hệ số **hàm điều hoà cầu** (*spherical harmonics*) bậc ba cho màu: tổng 59 số thực 32-bit, tức 236 byte. Một cảnh phòng khách dựng tốt thường cần cỡ một triệu Gaussian, tức khoảng 236 MB — so với vài megabyte cho trọng số của một NeRF MLP. Đó chính là “giá phải trả” hay được nhắc chung chung: đổi khoảng hai bậc độ lớn dung lượng lấy khoảng hai tới ba bậc độ lớn tốc độ render.

Cái mất thứ hai tinh tế hơn và quan trọng hơn với người làm robot: **hình học kém tường minh**. Trong một trường *SDF*, bề mặt là tập mức 0 — định nghĩa sạch, trích mesh bằng *marching cubes* là chuyện cơ học. Trong 3DGS, “bề mặt” không được định nghĩa ở đâu cả: tối ưu chỉ đòi ảnh render đúng, nên nó thoải mái đặt các Gaussian dẹt lơ lửng trước một bức tường nếu làm vậy khớp ảnh tốt hơn. Muốn trích mesh dùng được, người ta phải thêm ràng buộc ép Gaussian bám bề mặt (dẹt hoá, gấn pháp tuyến) — tức là trả lại đúng phần ràng buộc hình học mà bước đầu đã cố tình bỏ đi.

D. Ba giả định chung, và điều gì xảy ra khi chúng sai

Cả họ neural rendering đứng trên ba giả định, và với dữ liệu quay bằng camera robot thì cả ba đều dễ vỡ:

- **Pose đã biết và chính xác.** Sai lệch vài phần mười độ làm các ảnh mâu thuẫn nhau; vì tối ưu luôn tìm cách giảm loss, nó bù bằng cách sinh ra vật chất giả lơ lửng — hiện tượng *floater*. Đây là lý do 3DGS “nhảy với khối tạo từ SfM”: rác vào thì rác ra, chỉ khác là rác ra trông rất đẹp.
- **Cảnh tĩnh.** Người đi qua khung hình để lại vệt mờ hoặc khối đục lơ lửng ở giữa phòng.
- **Quá trình tạo ảnh nhất quán.** Camera bật phơi sáng và cân bằng trắng tự động vi phạm giả định này liên tục, và mô hình buộc phải giải thích thay đổi độ sáng bằng cách nhét thêm hình học sai. **Ảnh mờ do chuyển động** (*motion blur*) cũng thuộc nhóm này: một khung hình mờ không phải quan sát nhiều của cảnh tĩnh mà là tích phân của cảnh dọc một quãng chuyển động, và mô hình render không có chỗ nào biểu diễn điều đó.

CẠM BÃY

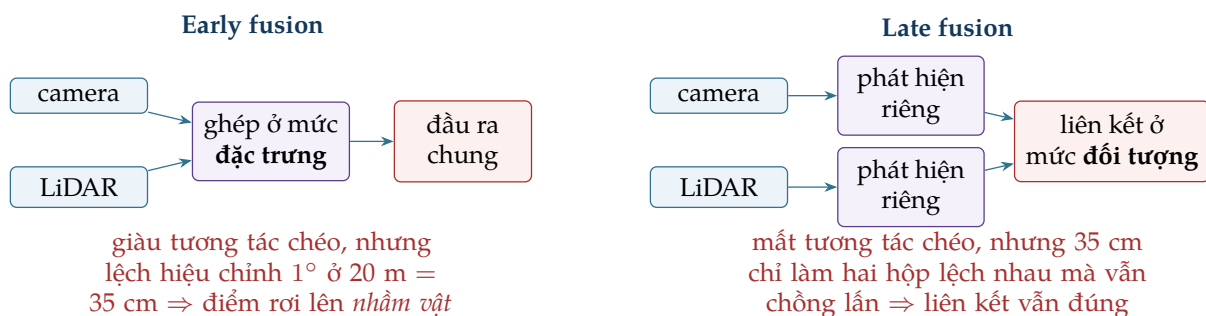
PSNR cao trên các view giữ lại *không* chứng minh hình học đúng. Hàm mất mát chỉ ràng buộc ảnh render; hai cấu hình mật độ rất khác nhau có thể sinh ra cùng bộ ảnh, đặc biệt khi màu được phép phụ thuộc hướng nhìn — mô hình có thể “vẽ” thứ đáng lẽ phải “đứng”. Triệu chứng chẩn đoán: chất lượng đẹp ở các view nội suy gần quỹ đạo chụp và sập ngay khi ngoại suy ra ngoài. Nếu bạn định dùng đầu ra làm bản đồ để robot va chạm hay bám bề mặt, hãy đánh giá bằng độ chính xác hình học so với một quét chân trị, không phải bằng PSNR hay *LPIPS* [69].

Với người làm SLAM, hệ quả thực hành gọn lại thành một câu hỏi: *bạn có thực sự cần render không?* Nếu chỉ cần định vị, bản đồ thừa của SLAM cổ điển vẫn là lựa chọn đúng — rẻ hơn nhiều bậc và có sai số kiểm chứng được. Neural rendering đáng đưa vào khi bài toán hạ nguồn thật sự cần ảnh: kiểm tra trực quan từ xa, sinh dữ liệu huấn luyện từ góc nhìn mới, mô phỏng cảm biến, hoặc định vị lại bằng cách so ảnh render với ảnh thật.

→ cùng nguyên lý với C/17/02 — cùng một ý tưởng “đặt thứ ta muốn vào hàm mất mát rồi tối ưu trên chính đối tượng cần tạo ra”, chỉ khác biến tối ưu là ảnh thay vì cảnh.

E. Hợp nhất camera, LiDAR và IMU

Ba cảm biến này bù trừ cho nhau gần như hoàn hảo: camera cho ngữ nghĩa và độ phân giải góc cao nhưng không có tỉ lệ; LiDAR cho hình học metric thừa và bền với ánh sáng; IMU cho chuyển động tần số cao và quan sát được phương trọng lực nhưng trôi rất nhanh khi tích phân. Câu hỏi thiết kế là hợp nhất ở tầng nào (Hình 16.13).



Hình 16.13. Cùng một sai số hiệu chỉnh, hai hậu quả khác hẳn nhau. Early fusion liên kết ở mức pixel nên sai số biến thành *đầu vào rác*; late fusion liên kết ở mức đối tượng, nơi vài chục centimet vẫn nằm trong kích thước của vật, nên sai số chỉ thành nhiễu đầu ra. Đó là toàn bộ nội dung của cuộc đánh đổi “giàu thông tin” đổi lại “bền vững”.

Con số trong hình tính ra như sau: sai số hiệu chỉnh ngoại xoay 1° chuyển thành sai số ngang $\tan 1^\circ \approx 17,5$ mm trên mỗi mét tầm xa, nên ở 20 m là 35 cm. Lệch đồng bộ thời gian sinh ra đúng loại lỗi đó, nhưng âm thầm hơn vì nó chỉ xuất hiện khi có chuyển động. Robot quay $100^\circ/\text{s}$ với lệch nhãn thời gian 5 ms cho sai lệch góc $0,5^\circ$. Con số này tỉ lệ thuận với tốc độ quay. Nghĩa là hệ thống chạy hoàn hảo khi đứng yên, rồi hỏng đúng lúc đang xoay nhanh — đúng lúc bạn cần nó nhất. Đây là lý do một nhận định lặp đi lặp lại trong thực hành: ở các hệ đa cảm biến, phần lớn thời gian sửa lỗi nằm ở đồng bộ thời gian và hiệu chỉnh ngoại chứ không ở thuật toán hợp nhất.

CẦN NHỚ

Sai số hiệu chỉnh và sai số đồng bộ không phải nhiều — chúng là sai số *có hệ thống*, phụ thuộc tầm xa và tốc độ, nên không trung bình về không khi gom nhiều quan sát. Hệ quả bắt buộc: chúng phải được đưa vào mô hình như biến cần ước lượng (hiệu chỉnh trực tuyến, ước lượng độ trễ t_d) chứ không phải được hy vọng là nhỏ. Và với neural rendering, điều tương đương là: hàm mất mát ràng buộc ảnh, nên hình học phải được kiểm tra bằng một phép đo bên ngoài. [T4]

F. Giới hạn và trường hợp hỏng

Bên cạnh bài toán tối ưu riêng từng cảnh (per-scene optimization), người triển khai 3DGS và NeRF trên robot và hệ thống nhúng phải đối mặt với ba rủi ro kỹ thuật rất cụ thể:

- *Bùng nổ bộ nhớ VRAM khi mở rộng bản đồ (VRAM Explosion)*: Một phòng làm việc nhỏ có thể sinh ra từ 2 tới 5 triệu hạt Gaussian, ngốn từ 1.5 tới 3 GB VRAM trên GPU. Khi robot di chuyển qua toà nhà hoặc môi trường ngoài trời rộng lớn, việc lưu trữ toàn bộ Gaussian trong bộ nhớ đồ hoạ dẫn tới tràn RAM GPU (OOM) lập tức. Mặc dù các kỹ thuật nén (như Scaffold-GS hay lượng tử hoá thuộc tính) giảm dung lượng đáng kể, chúng làm tăng chi phí giải nén thời gian thực và làm mờ các chi tiết hình học mảnh.
- *Vật cản ma (Phantom Obstacles) và hình học suy biến*: Do hàm mất mát chỉ tối ưu trên ảnh 2D, tại những vùng camera ít quan sát hoặc góc nhìn hẹp, 3DGS có xu hướng sinh ra các Gaussian hình cây kim dài (needle-like Gaussians) hoặc các đám mây lơ lửng trước ống kính nhằm hạ loss một cách cơ hội. Đối với một hệ thống robot tự hành lấy bản đồ này để lập đường đi, những đám mây giả đó sẽ bị hiểu nhầm là vật cản thực tế, làm robot dừng đột ngột hoặc lập lộ trình vòng vèo vô ích.
- *Tính phi bất biến trước biến thiên ánh sáng*: Cả NeRF và 3DGS sử dụng hàm cầu điều hoà (Spherical Harmonics) để mô tả tính phản xạ phụ thuộc hướng nhìn (view-dependent appearance). Khi môi trường có ánh sáng biến động mạnh (đèn bật/tắt, bóng râm thay đổi, phơi sáng camera tự động nhảy), mô hình sẽ nhầm lẫn giữa sự thay đổi bức xạ và sự thay đổi hình học, từ đó “khắc” các vệt sáng tối vĩnh viễn vào vị trí của các hạt 3D.

Về hợp nhất, chế độ hỏng đáng sợ nhất là hỏng *im lặng* của một nhánh. Mưa làm LiDAR sinh điểm giả; đường hầm làm camera mất hết đặc trưng; rung động làm IMU bão hoà. Một hệ hợp nhất không kiểm tra tính nhất quán giữa các nhánh sẽ vẫn cho ra ước lượng trông bình thường, với hiệp phương sai cũng trông bình thường. Vì vậy phần đáng đầu tư trong một hệ thật thường không phải bộ hợp nhất mà là bộ chẩn đoán từng nhánh: phân dư theo từng cảm biến, kiểm tra chi bình phương, và một cơ chế hạ cấp có kiểm soát khi một nhánh bị loại.

↪ **hệ quả của E/25** — hỏng im lặng của một nhánh cảm biến là trường hợp cụ thể của yêu cầu chung: hệ thống phải biết lúc nào nó không đáng tin.

G. Kết nối

Mục này khép lại chương 16 và nối ngược lên [C/16-thi-giac-3d/01-bon-the-he-3d](#): neural rendering là thể hệ ba, đứng *sau* khâu pose, nên nó tiêu thụ đầu ra của thể hệ một chứ không thay thế. Ý tưởng “đặt cái ta muốn vào hàm mất mát rồi hạ gradient trên chính đối tượng cần tạo ra” sẽ trở lại nguyên vẹn ở [C/17-mo-hinh-sinh/02-toi-uu-tren-pixel](#), chỉ khác là biến tối ưu ở đó là ảnh chứ không phải cảnh. Các giá định về quá trình tạo ảnh thuộc [A/02-vat-ly-tao-anh](#); việc chẩn đoán từng nhánh cảm biến và hạ cấp có kiểm soát thuộc [E/25-do-tin-cay-va-van-hanh](#).

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao biểu diễn cảnh bằng mật độ liên tục là điều kiện cần để gradient descent hoạt động, trong khi bề mặt cứng thì không?
2. 3DGS nhanh hơn NeRF nhờ đổi cái gì lấy cái gì, và vì sao cái mất đó lại đặc biệt bất tiện cho robot?
3. Một cảnh có PSNR rất cao trên các view giữ lại nhưng bản đồ hình học sai. Cơ chế nào cho phép hai điều đó cùng đúng, và bạn thiết kế phép thử nào để phát hiện?
4. Vì sao một hệ early fusion có thể chạy hoàn hảo khi robot đứng yên và hỏng khi robot xoay nhanh, dù không tham số nào thay đổi?
5. Nếu buộc phải chọn giữa một tuần cải tiến thuật toán hợp nhất và một tuần dừng lại quy trình hiệu chỉnh cùng đồng bộ thời gian, bạn chọn cái nào và dựa trên bằng chứng gì?
6. Hình 16.11 đọc theo nút thắt. Trong bài toán của bạn, nút thắt đang cản là thời gian dựng, tốc độ render, hay dung lượng bản đồ — và điều đó chỉ về mốc nào?

16.4 Thực hành

Chương này là chương dễ thực hành nhất của Phần C, vì hạ tầng mã nguồn mở đã chín và một chiếc điện thoại là đủ để tạo dữ liệu. Một lưu ý về bản rã trước khi vào danh sách: danh sách dưới đây chốt tại **tháng 9/2026**, và các tên mô hình dẫn đầu bảng sẽ cũ trong 6–12 tháng. Các repo hạ tầng thì bền hơn nhiều: COLMAP, Open3D, evo, gsplat. Hãy đầu tư kỹ năng vào nhóm sau.

Repo và tài nguyên. SfM tham chiếu: [colmap/colmap](#) (giấy phép BSD; lưu ý bản 4.x đổi backend đọc ảnh nên không tái lập bit-for-bit với 3.x) [33]. *Feed-forward geometry*: [facebookresearch/vgg](#) (CVPR 2025 best paper; nhiều pipeline không cần COLMAP dựng trên nó), DUST3R/MASt3R [34]. *Splatting và rendering*: [nerfstudio-project/gsplat](#) (Apache 2.0, mặc định hợp lý cho nghiên cứu), [graphdeco-inria/gaussian-splatting](#) (bản gốc, giấy phép hạn chế nhất — đọc kỹ trước khi dùng cho sản phẩm), [nerfstudio-project/nerfstudio](#), NVIDIA vkSplatting. *Độ sâu đơn ảnh*: Depth Anything V2, Apple MoGe. *SLAM*: ORB-SLAM3 [40], DROID-SLAM; đánh giá bằng [MichaelGrupp/evo](#). *Point cloud*: [isl-org/Open3D](#), [open-mmlab/mmdetection3d](#).

Câu hỏi kiểm tra hiểu. Vì sao ATE cần bước căn chỉnh còn RPE thì không, và mỗi cái nhạy với loại lỗi nào? 3DGS nhanh hơn NeRF nhờ đổi cái gì lấy cái gì? Feed-forward geometry tốt hơn COLMAP ở tình huống nào và tệ hơn ở tình huống nào — và bạn nhận ra mình đang ở tình huống nào bằng dấu hiệu gì *trước khi chạy*?

Mini project (vài giờ đến hai ngày). Quay một video 60 giây quanh một vật thể. Chạy hai đường: (a) COLMAP → gsplat, (b) một mô hình tiền thẳng → gsplat. So ba đại lượng: thời gian tường, PSNR trên các view giữ lại, và độ lệch pose giữa hai đường. Giá trị của bài này không nằm ở con số mà ở chỗ bạn buộc phải quyết định *đường nào là tham chiếu* — và phát hiện ra rằng câu hỏi đó không có câu trả lời hiển nhiên.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• SfM	Quy trình cổ điển suy đồng thời pose camera và cấu trúc 3D thừa từ một tập ảnh: tìm tương ứng, ước lượng pose, tam giác hoá, tinh chỉnh. Nó là thể hệ một và vẫn là chuẩn đối chiếu của mọi phương pháp mới.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
••• Hiệu chỉnh chùm tia	Bài toán bình phương tối thiểu phi tuyến, tính chỉnh đồng thời mọi pose và mọi điểm 3D sao cho sai số chiều lại nhỏ nhất. Điều đáng nhớ nhất: nó trả về phần dư và hiệp phương sai, nên hệ thống tự biết lúc nào nó sai.
•• Point map	Ảnh mà mỗi pixel chứa một toạ độ 3D thay vì một màu. Đây là dạng đầu ra của các mô hình tiến thẳng như DUST3R hay VGGT, thay cho đám mây điểm rời rạc.
••• Vì sao mô hình tiến thẳng hồng “im lặng”?	Vì nó là một bài hồi quy, không phải một bài ước lượng có ràng buộc. Không còn phần dư hình học nào để mâu thuẫn với kết quả, nên cảnh trong hay ngoài phân phối nó đều trả lời với cùng vẻ tự tin.
•• Thị sai	Độ lệch ngang của cùng một điểm giữa ảnh trái và ảnh phải trong hệ stereo. Nó tỉ lệ nghịch với độ sâu: $Z = fb/d$.
•• Vì sao sai số độ sâu stereo tăng theo Z^2?	Vì $Z = fb/d$ nên $\Delta Z = Z^2 \Delta d / (fb)$. Hệ quả thực hành: mọi hệ stereo có một “chân trời”, và cách duy nhất đẩy nó ra xa là tăng tiêu cự hoặc đường cơ sở.
••• Scale ambiguity	Phóng to cả cảnh lẫn quãng đường camera cùng một hệ số cho ra đúng bộ ảnh cũ. Vì vậy tỉ lệ tuyệt đối không suy ra được từ ảnh, dù thêm bao nhiêu dữ liệu; chỉ prior hoặc một cảm biến biết mét mới phá được.
•• Voxel	Ô lập phương trong một lưới đều 3D. Ưu điểm là lân cận cho sẵn nên conv 3D dùng được ngay; nhược điểm là số ô tăng theo lũy thừa ba, và hơn 99% ô là rỗng.
•• SDF	Hàm gán cho mỗi điểm khoảng cách có dấu tới bề mặt gần nhất — âm ở trong, dương ở ngoài. Bề mặt là tập điểm bằng 0, nên trích lưới tam giác là chuyện cơ học; đổi lại render phải dò dọc tia.
•• Point cloud	Tập toạ độ 3D không có thứ tự và không có kết nối. Gọn và hợp với LiDAR, nhưng không có sẵn khái niệm lân cận, nên mọi phép cục bộ đều phải trả tiền tìm kiếm.
•• PointNet	Mạng cho tập điểm: mọi điểm qua cùng một MLP rồi gộp bằng phép max theo từng chiều. Phép max không quan tâm thứ tự nên tính bất biến hoán vị là đúng tuyệt đối, không phải học được.
• Bản đồ dựng ra trông đẹp nhưng mọi khoảng cách sai cùng một hệ số — nguyên nhân?	Scale ambiguity chưa được phá: pipeline đơn mắt không có cảm biến nào biết đơn vị mét. Phép phân biệt với lỗi hiệu chỉnh: đo vài vật ở các tầm khác nhau — sai một hệ số <i>không đổi</i> là lỗi tỉ lệ, sai tăng dần theo tầm là lỗi hiệu chỉnh.
• Mô hình LiDAR 64 tia tụt mạnh trên 32 tia, mất vật ở xa trước — nguyên nhân?	Mật độ điểm lúc chạy khác lúc huấn luyện, và vùng xa là nơi mật độ giảm nhanh nhất. Phân biệt với lỗi hiệu chỉnh: lỗi hiệu chỉnh làm lệch vị trí hộp ở mọi tầm, không làm mất hẳn vật ở xa.
••• Volume rendering	Cách tính màu một pixel bằng cách lấy mẫu nhiều điểm dọc tia rồi cộng dồn màu có trọng số theo độ đục tích lũy. Nó thay câu hỏi “bề mặt ở đâu” bằng câu hỏi “mật độ tại điểm này bao nhiêu”.
••• Vì sao NeRF dùng mật độ liên tục, không dùng bề mặt?	Vì đạo hàm. Bề mặt cứng làm hàm mất mát thành bậc thang, gradient bằng không gần như mọi nơi. Mật độ liên tục biến mỗi pixel thành tổng có trọng số, nên mọi điểm mẫu đều nhận được gradient.
•• Positional encoding	Ánh xạ toạ độ qua một dãy sin/cos nhiều tần số trước khi đưa vào MLP. Không có nó thì ảnh mờ nhoè, vì MLP thiên lệch mạnh về hàm tần số thấp.
••• 3D Gaussian splatting	Biểu diễn cảnh bằng hàng triệu Gaussian dị hướng có màu và độ đục, render bằng cách chiếu rồi trộn theo thứ tự độ sâu. Được tốc độ thời gian thực; mất dung lượng (236 byte mỗi Gaussian) và mất bề mặt tường minh.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Vì sao early fusion nhạy với lệch hiệu chỉnh hơn late fusion?	Vì nó liên kết ở mức pixel. Lệch 1° ở 20 m là 35 cm, đủ để điểm LiDAR rơi lên nhầm vật và biến đặc trưng ghép thành rác. Late fusion liên kết ở mức đối tượng, nơi 35 cm vẫn nằm trong kích thước vật.
● PSNR trên view giữ lại rất cao nhưng bản đồ hình học sai — nghĩa là gì?	Mô hình đã “vẽ” thay vì “dựng”: nó khớp ảnh bằng màu phụ thuộc hướng nhìn. Phép phân biệt: ngoại suy ra ngoài quỹ đạo chụp, hoặc so với một quét chân trị.
● Hệ hợp nhất chạy hoàn hảo khi robot đứng yên, hỏng khi xoay nhanh — nguyên nhân?	Lệch đồng bộ thời gian, vì sai số của nó tỉ lệ với tốc độ quay: 5 ms ở $100^\circ/\text{s}$ là $0,5^\circ$. Phân biệt với lệch hiệu chỉnh ngoại: lệch hiệu chỉnh sai cả khi đứng yên.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
COLMAP	Hiện thực tham chiếu của thể hệ một: tìm tương ứng, ước lượng pose, hiệu chỉnh chùm tia. Vẫn là chuẩn để đối chiếu mọi phương pháp mới	Hạ tầng bền; phần lớn dữ liệu huấn luyện của thể hệ bốn cũng do nó tạo ra
ORB-SLAM3	Hiệu chỉnh chùm tia dưới ràng buộc thời gian thực, với tách frontend/backend và đóng vòng để chặn sai số	Chuẩn đối chứng của SLAM đặc trưng thưa
DUST3R / MAST3R / VGGT	Thể hệ bốn: point map trong một hệ tọa độ chung, không cần pose đầu vào	Mạnh ở khởi tạo và ảnh thưa; vẫn thường được nối tiếp bằng một vòng hiệu chỉnh chùm tia
gsplat, nerfstudio	Neural rendering thực dụng: tối ưu biểu diễn cảnh bằng loss render khả vi, tiêu thụ pose từ SfM	Hạ tầng; phần lớn pipeline dựng cảnh hiện nay đi qua đây
Depth Anything V2, MoGe	Độ sâu đơn ảnh bất biến affine — bỏ tỉ lệ để đổi lấy khả năng khái quát	Dùng làm prior tương đối thì tốt; dùng làm phép đo metric thì phải kèm hai ẩn tỉ lệ và độ dịch
Open3D, mmdetection3d	Học sâu trên point cloud: bất biến hoán vị, phân cấp không gian, tích chập thưa	Hạ tầng cho mảng LiDAR

MINI PROJECT SLAM · Hai đường dựng cảnh 3D và điểm giao của chúng · 1–2 ngày

Mục tiêu	tìm bằng số cái ngưỡng mà ở dưới nó thể hệ tiến thẳng thắn hình học cổ điển, và ở trên nó thì thua. Đo trên chính camera và chính loại cảnh bạn đang chạy SLAM, không phải trên benchmark của paper.
Dữ liệu	một video 60 giây quay quanh một vật thể hoặc một góc phòng, quay bằng đúng camera bạn dùng cho SLAM (giữ nguyên phơi sáng thủ công nếu có). Trích ra bốn tập ảnh với $N = 8, 15, 30, 60$ khung hình phân bố đều, và giữ riêng 10 % khung hình làm view kiểm.
Các bước	với mỗi N : (1) chạy đường A — COLMAP → gsplat — và đường B — một mô hình 3D tiến thẳng (DUST3R/MAST3R/VGGT) → gsplat; (2) ghi thời gian tường của từng khâu; (3) ghi PSNR trên view kiểm; (4) ghi độ lệch pose giữa hai đường sau khi căn chỉnh Sim3; (5) ghi lại N nhỏ nhất mà mỗi đường còn khởi tạo được.
Xong khi	bạn vẽ được hai đường PSNR theo N trên cùng một trục và chỉ ra điểm giao bằng con số, kèm bảng thời gian tường và N tối thiểu của từng đường. Câu trả lời phải có dạng “dưới $N = \dots$ đường tiến thẳng thắn, trên đó COLMAP thắng, và nó tốn thêm ... phút”.

Nối với [C/16/01](#) giải thích vì sao điểm giao đó *phải* tồn tại; [B/07](#) cho cách tách view kiểm cho đúng; [E/24](#) cho việc lặp lại nhiều lần trước khi tin một chênh lệch. Bảng số bạn tạo ra ở đây là baseline cho mini project của chương sau.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì phóng to cả cảnh lẫn quãng đường camera cùng một hệ số cho ra chuỗi ảnh y hệt — đây là tính không xác định được, dữ liệu không mang thông tin đó. Chỉ prior kích thước, hoặc một cảm biến biết đơn vị mét (đường cơ sở stereo, LiDAR, gia tốc kế), mới phá được.
- A2. Vì hai bên giải hai bài toán khác nhau. Mô hình tiến thẳng mạnh ở *khởi tạo*: ít ảnh, chồng lấn kém, không pose. Hiệu chỉnh chùm tia mạnh ở *tinh chỉnh*. Quan trọng hơn, chỉ nó trả về phần dư và hiệp phương sai, nên chỉ nó cho ta biết lúc nào kết quả không đáng tin.
- A3. Vì đạo hàm. Bề mặt cứng làm hàm mất mát thành bậc thang, gradient bằng không gần như mọi nơi nên không có hướng để đi. Mật độ liên tục biến mỗi pixel thành tổng có trọng số của hàng chục điểm dọc tia, và mọi điểm đều nhận được một phần gradient.
- A4. Vì lệch đồng bộ thời gian chỉ hiện ra khi có chuyển động và tỉ lệ thuận với tốc độ quay: 5 ms ở $100^\circ/\text{s}$ là $0,5^\circ$, ở tầm 20 m thành khoảng 17 cm. Early fusion liên kết ở mức pixel nên độ lệch đó biến điểm LiDAR thành rơi lên nhầm vật, tức là đầu vào rác.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy nghĩ tới cái bóng của một vật in trên tường. Nhìn cái bóng, bạn không có cách nào biết vật đó to bằng nào và cách tường bao xa: một cái cốc nhỏ để gần và một cái thùng lớn để xa đổ ra đúng cùng một cái bóng. Đó chính xác là chuyện xảy ra khi máy ảnh chụp một cảnh — ảnh là cái bóng, và chiều sâu bị mất khi chụp.

Cả chương này là các cách lấy lại chiều đã mất, và chỉ có ba cách. Cách thứ nhất là đi vòng sang chỗ khác chụp thêm vài cái bóng nữa rồi ghép chúng lại; chỗ nào các cái bóng chỉ về cùng một điểm thì đó là vật thật. Cách này chính xác tới từng centimét, và quan trọng hơn, nếu các cái bóng mâu thuẫn nhau thì bạn biết ngay là mình đã sai ở đâu đó. Cách thứ hai là nặn một mô hình bằng đất sét trong bóng tối, rồi soi đèn từ đúng những hướng đã chụp và chỉnh đất sét cho tới khi mọi cái bóng khớp. Cách thứ ba là đưa ba tấm ảnh cho một người đã nhìn hàng triệu cảnh và hỏi luôn.

Cái giá thì rất rõ. Cách thứ nhất cần bạn đi thêm một vòng và cần cảnh đứng yên. Cách thứ hai cho mô hình đổ bóng đúng ở mọi hướng đã thử. Nhưng nó chưa chắc đúng khi soi từ một hướng mới, vì bạn chỉ ép nó khớp *cái bóng*, không ép nó khớp *hình dạng*. Cách thứ ba trả lời trong một giây và không cần đi đâu cả, nhưng khi gặp một cảnh lạ nó vẫn trả lời tự tin y hệt, và không có cái bóng nào mâu thuẫn để tố cáo nó.

Chỗ không rút gọn được: chỗ nói về sai lệch khi lắp camera và cảm biến với nhau. Cách nói đơn giản tự nhiên là “phải lắp cho thật chuẩn”, nhưng nói vậy là sai trọng tâm: vấn đề không phải sai số lớn hay nhỏ, mà là nó *lặp lại y hệt* ở mọi phép đo, nên gom thêm bao nhiêu phép đo cũng không triệt tiêu được nó. Đây là chỗ phải nói đủ dài mới không sai.

Một câu: Ảnh làm mất chiều sâu, và bạn chỉ có hai cách lấy lại: đi thêm một vòng để nhìn từ chỗ khác, hoặc đoán bằng kinh nghiệm — chỉ cách thứ nhất mới tự nói cho bạn biết lúc nào nó sai.

Kết nối

Chương này khép lại phần hình học của Phần C. [C/17-mô-hình-sinh](#) lấy lại nguyên vẹn ý tưởng “đặt cái ta muốn vào hàm mất mát rồi tối ưu trên chính đối tượng cần tạo ra”, lần này trên pixel; [B/07-chia-du-lieu-va-danh-gia](#) hoàn thiện phần giao thức đánh giá mà chương này chỉ nhắc qua; [E/25-do-tin-cay-va-van-hanh](#) biến “cơ chế phát hiện sai từ bên ngoài” thành thiết kế cụ thể.

Tổng kết chương

Ta đã đi từ một bài toán duy nhất — lấy lại chiều đã mất khi chiếu — qua bốn cách giải nó, và thấy chúng không xếp thành một chuỗi tiến bộ mà thành một trục đánh đổi. Ở một đầu trục là hình học cổ điển: giả định nhiều, dữ liệu ít, độ chính xác metric cao, và quan trọng nhất là tự biết lúc nào mình sai. Ở đầu kia là mô hình tiến thẳng: giả định ít, dữ liệu nhiều, chạy được ở chỗ hình học bó tay, và im lặng khi sai. Neural rendering không nằm trên trục đó mà nằm ở một tầng khác — nó không cạnh tranh với SfM mà tiêu thụ đầu ra của SfM, và nó tối ưu cho một mục tiêu khác hẳn là chất lượng ảnh.

Ba chủ đề còn bỏ ngỏ, và cả ba đều là chỗ nên theo dõi trong một hai năm tới: thứ nhất, các mô hình tiến thẳng dự đoán thẳng biểu diễn render được, tức là gộp thể hệ ba và bốn làm một; thứ hai, neural rendering cho cảnh động, nơi giả định cảnh tĩnh bị gỡ bỏ; thứ ba, và thực dụng nhất với robot, cách gắn một ước lượng bất định tin cậy được vào đầu ra của các mô hình học. Chừng nào chưa có nó, thể hệ bốn vẫn chỉ dùng được ở vai trò khởi tạo, không phải vai trò quyết định.

I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Luận điểm gộp là mỗi lần bỏ một giả định, bạn bù bằng dữ liệu *và* mất một phần khả năng tự chẩn đoán. Theo dấu khoanh mất thứ hai qua bốn thể hệ: ở mỗi thể hệ, tín hiệu tự chẩn đoán nào biến mất?
2. Tính không xác định của tỉ lệ ở mục 1 và mục 2, so với việc mô hình ở mục 3 vẫn dựng được cảnh trông rất đúng — hai chỗ này có mâu thuẫn không? Chỉ ra thông tin nào đã lén vào thay cho ràng buộc còn thiếu.
3. Bạn có ba nguồn độ sâu cho cùng một pixel: stereo, độ sâu đơn ảnh học được, và LiDAR thừa. Ba luật sai số ở đây khác hẳn nhau; luật nào khiến việc gán trọng số theo nghịch đảo phương sai trở nên không hợp lệ, và bạn xử lý thế nào?
4. Hình học cổ điển hổng ồn ào còn mô hình học được hổng im lặng. Với một hệ dùng cả hai, bạn dựng cơ chế nào để buộc phần im lặng phát ra tiếng, và cơ chế đó tiêu tốn cái gì?
5. Loop closure ở mục 2 và tinh chỉnh toàn cục sau khâu pose ở mục 1 đều là ràng buộc toàn cục. Vì sao neural rendering ở mục 3 hầu như không có thứ tương đương, và hệ quả với một bản đồ dựng suốt nhiều giờ là gì?
6. Bạn nghi hệ hợp nhất hổng vì hiệu chỉnh ngoại tại hoặc lệch đồng bộ thời gian chứ không vì thuật toán. Phép thử nào tách hai nguyên nhân đó, và vì sao chuyển động xoay là điều kiện thử tốt hơn chuyển động thẳng?

Mô hình sinh, style transfer và low-level vision (Generative Models and Low-Level Vision)

PHỤ THUỘC B/09-thiet-ke-ham-mat-mat (hướng đo khoảng cách phân phối — điều kiện tiên quyết thật sự của chương này); C/12-co-che-huan-luyen (huấn luyện ổn định, quy mô); C/13-kien-truc-backbone (U-Net, transformer); A/02-vat-ly-tao-anh (nhiều thật, dài sáng động).

TÓM TẮT

Chương này gom bốn chủ đề thường bị dạy rời nhau — mô hình sinh, chuyển phong cách, diffusion, và khôi phục ảnh — vì chúng chia chung đúng một câu hỏi: khi nhiều đầu ra cùng hợp lệ, cái gì quyết định chọn cái nào? Bốn họ mô hình sinh là bốn câu trả lời, chuyển phong cách là câu trả lời viết thẳng lên pixel, diffusion là câu trả lời hiện đang thắng, và low-level vision là nơi câu hỏi trở thành vấn đề đúng sai chứ không còn là thẩm mỹ. Đọc xong bạn nhìn một mô hình sinh là đọc ra kiểu thất bại của nó trước khi chạy. Nếu chỉ nhớ một điều: chất lượng đầu ra khác nhau là vì *định nghĩa “giống dữ liệu thật”* khác nhau, không phải vì mô hình này thông minh hơn mô hình kia — và mọi cơ chế làm ảnh “đẹp hơn” đều mua độ trung thành bằng độ đa dạng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình sinh (generative model)** — mô hình học *phân phối* của dữ liệu, để có thể lấy mẫu ra dữ liệu mới; khác với mô hình phân biệt, vốn chỉ học ánh xạ từ đầu vào sang nhãn.
- **phân phối dữ liệu (data distribution)** — cách nghĩ về tập ảnh thật như các mẫu rút ra từ một phân phối xác suất trên không gian ảnh; “sinh ảnh mới” nghĩa là rút thêm một mẫu từ phân phối đó.
- **hàm mất mát (loss function)** — đại lượng số mà quá trình huấn luyện cực tiểu hoá. Toàn chương này là một minh hoạ kéo dài cho nguyên tắc: chọn hàm mất mát là chọn trước kiểu sai lầm mà mô hình sẽ mắc.
- **không gian ẩn (latent space)** — không gian chiều thấp mà mô hình dùng để mã hoá một ảnh; nó vừa là nơi “ý nghĩa” được sắp xếp, vừa là mẹo giúp diffusion chạy được trên GPU thường.
- **style transfer (chuyển phong cách)** — bài toán giữ nội dung của một ảnh nhưng khoác lên nó “chất” của một ảnh khác — nét cọ, bảng màu, kết cấu.
- **ill-posed (đặt không chính)** — bài toán có nhiều nghiệm cùng hợp lệ vì thông tin đã bị xoá; mọi bài toán khôi phục ảnh đều thuộc loại này.
- **FID / LPIPS / PSNR** — ba chỉ số đánh giá ở ba tầng khác nhau: FID so hai *phân phối* ảnh, LPIPS so hai ảnh trong không gian đặc trưng học được, PSNR so hai ảnh theo từng pixel. Mục 17.4 nói rõ mỗi cái mù với gì.

CÂU HỎI MỞ ĐẦU

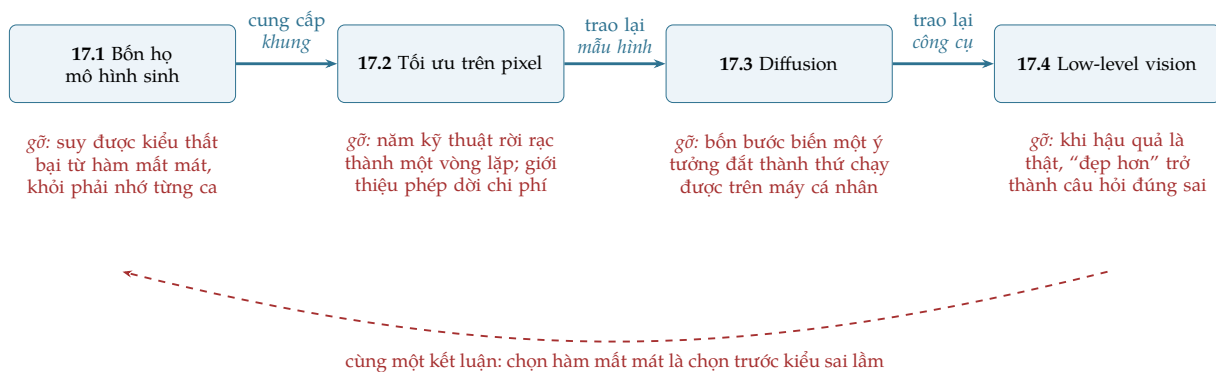
- Q1. Vì sao mô hình tối ưu MSE *luôn* cho ảnh mờ, và vì sao đó là một kết quả toán học chứ không phải lỗi cài đặt?
- Q2. VAE hỏng vì mờ, GAN hỏng vì nghèo — hai triệu chứng trái ngược nhau. Vì sao chúng lại là hai mặt của cùng một quyết định thiết kế?

- Q3.** Vì sao diffusion, thứ phải chạy mạng hàng chục tới hàng trăm lần cho một ảnh, lại đánh bại GAN vốn chỉ cần một lượt xuôi?
- Q4.** Vì sao chỉnh một pipeline sinh ảnh tới lúc trông “đẹp nhất” lại là cách chắc chắn nhất để làm hỏng nó khi dùng đầu ra làm dữ liệu huấn luyện?

Vì sao chương này chia làm bốn mục

Bốn mục là bốn tầng của cùng một câu hỏi, và thứ tự là thứ tự phụ thuộc.

Hình 17.1 vẽ bốn mục cùng nút thắt mà mỗi mục gỡ.



Hình 17.1. Bốn mục của chương, xếp theo thứ tự phụ thuộc chứ không theo mức độ phổ biến. Mục 17.2 nằm giữa lý thuyết và diffusion không phải ngẫu nhiên: nó là ví dụ sạch nhất cho việc hàm mục tiêu quyết định tất cả, và nó giới thiệu mẫu hình “dời chi phí sang lúc huấn luyện” mà mục 17.3 dùng lại nguyên vẹn. Mũi tên đứt là kết luận chung mà cả bốn mục đều dẫn tới.

Mục thứ nhất dựng khung: bốn họ mô hình sinh là bốn định nghĩa khác nhau của cụm từ “giống dữ liệu thật”. Khung này quan trọng hơn bất kỳ chi tiết kiến trúc nào, vì nó cho phép suy ra kiểu thất bại từ hàm mất mát — ảnh mờ, mất mode, hay chậm — thay vì phải ghi nhớ từng trường hợp.

Mục thứ hai tách riêng một ý tưởng cắt ngang: đóng băng mạng, coi ảnh là biến, hạ gradient trên chính pixel. Nó được đặt ở đây, giữa lý thuyết và diffusion, vì hai lý do: nó là ví dụ sạch nhất về việc hàm mục tiêu quyết định tất cả (đổi một dòng là được một kỹ thuật khác), và nó giới thiệu mẫu hình “dời chi phí từ lúc chạy sang lúc huấn luyện” mà mục ba sẽ dùng lại nguyên vẹn.

Mục thứ ba là họ đang thắng. Nó không chỉ trình bày nguyên lý mà đi qua bốn bước kỹ thuật đã biến diffusion từ một ý tưởng thành sản phẩm chạy được trên máy cá nhân — và mỗi bước là một đánh đổi cụ thể mà người dùng phải chỉnh bằng tay.

Mục thứ tư đưa cùng bộ khái niệm sang một bối cảnh mà hậu quả là thật: khôi phục ảnh. Ở đây “sắc nét hơn” đồng nghĩa với “bịa nhiều hơn”, và điều đó không phải một lời than phiền mà là một kết quả có chứng minh. Mục này đặt cuối vì nó cần cả ba mục trước để nói cho đủ.

Bản đồ chương 17 — Mô hình sinh, style transfer và low-level vision

TÓM TẮT

Chương 17 gồm bốn mục, xếp theo thứ tự phụ thuộc chứ không theo mức độ phổ biến: khung bốn họ mô hình sinh, ý tưởng tối ưu trên pixel, diffusion, rồi khôi phục ảnh. Bản đồ này nói mục nào cốt lõi, mục nào tra khi cần, và trình tự đọc tùy mục tiêu của bạn. Nếu chỉ nhớ một điều: đọc mục 17.1 trước — nó cho phép *suy ra* kiểu thất bại của một mô hình từ hàm mất mát, và ba mục còn lại đều dùng lại khả năng đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình sinh (generative model)** — mô hình học phân phối dữ liệu để lấy mẫu ra dữ liệu mới, khác với mô hình dự đoán nhãn.
- **diffusion** — họ mô hình sinh bằng cách học khử nhiễu dần, từ nhiễu trắng về ảnh; hiện là họ thống trị.
- **style transfer** — giữ nội dung của ảnh này, khoác “chất” của ảnh kia.
- **low-level vision** — nhóm bài toán ảnh sang ảnh: khử nhiễu, siêu phân giải, khử nhoè, HDR.

Bốn mục và vai trò của từng mục

- **17.1 — Bốn họ và cuộc đánh đổi ba chiều (cốt lõi).** Autoregressive, VAE, GAN, diffusion là bốn định nghĩa của “giống dữ liệu thật”. Đây là khung để suy ra kiểu thất bại, và là mục đắt giá nhất của cả chương.
- **17.2 — Tối ưu trên pixel thay vì trên trọng số (cốt lõi về ý tưởng, tham khảo về chi tiết style transfer).** Một vòng lặp mười dòng cho ra năm kỹ thuật; ma trận Gram; mẫu hình dờ chi phí sang lúc huấn luyện.
- **17.3 — Diffusion và các bước làm cho nó khả thi (cốt lõi).** Quá trình thuận/ngược, score function, bộ lấy mẫu, dẫn hướng, khuếch tán trong không gian ẩn, ControlNet.
- **17.4 — Low-level vision và trade-off không thể tránh (cốt lõi với người làm hệ thống thật).** Ill-posed, đánh đổi perceptual-distortion, các chỉ số và chỗ mù của từng cái, dữ liệu sinh dùng để huấn luyện.

Trình tự đọc

1. **Mục tiêu là hiểu nguyên lý:** 17.1 → 17.3 → 17.4, bỏ qua 17.2 ở lần đọc đầu.
2. **Mục tiêu là dùng được công cụ sinh ảnh:** 17.1 (chỉ phần bảng và tam giác) → 17.3 toàn bộ → 17.4 phần đánh giá.
3. **Mục tiêu là làm hệ thống thật có khôi phục ảnh:** 17.4 trước, rồi quay lên 17.1 để hiểu vì sao “mờ” là nghiệm đúng.
4. Khối *Thực hành* nằm ở file chương, sau bốn mục.

Chương này phụ thuộc gì và được dùng lại ở đâu

- **Phụ thuộc lên trên:** [B/09-thiet-ke-ham-mat-mat](#) (hai hướng của KL — điều kiện tiên quyết thật sự); [C/12-co-che-huan-luyen](#); [C/13-kien-truc-backbone](#) (U-Net, transformer); [A/02-vat-ly-tao-anh](#) (nhiều thật); [C/16-thi-giac-3d](#) (vai trò của prior trong bài toán thiếu ràng buộc).
- **Được dùng lại ở dưới:** [D/20-toi-uu-va-nen-mo-hinh](#) (dời chi phí sang lúc huấn luyện, chung cất); [C/18-pretraining-va-foundation](#) (nguồn giám sát và dữ liệu sinh);

E/25-do-tin-cay-va-van-hanh (hồng im lặng, trực quan hoá và các phép kiểm tra tính đúng đắn).

17.1 Bốn họ và cuộc đánh đổi ba chiều (Four Families of Generative Models)

PHỤ THUỘC B/09-thiet-ke-ham-mat-mat (hướng đo khoảng cách phân phối, forward đối lại reverse KL); A/01-toan-toi-thieu/03-xac-suat-va-thong-tin (likelihood, entropy tương đối); C/13-kien-truc-backbone (U-Net, transformer).

TÓM TẮT

Bốn họ mô hình sinh — *autoregressive*, VAE, GAN, *diffusion* — là bốn định nghĩa khác nhau của cụm từ “giống dữ liệu thật”. Chất lượng đầu ra khác nhau chính vì định nghĩa khác nhau, không phải vì mô hình này thông minh hơn mô hình kia. Đọc xong bạn nhìn một mô hình sinh là đọc ra ngay nó đang tối ưu đại lượng gì. Từ đó suy được nó sẽ mờ, sẽ thiếu đa dạng, hay sẽ chậm. Và bạn biết mình đang mua cái gì bằng cái gì. Nếu chỉ nhớ một điều: ảnh mờ của VAE và *mode collapse* của GAN không phải hai lỗi kỹ thuật khác nhau mà là hai mặt của cùng một lựa chọn — đo khoảng cách phân phối theo hướng nào.

KIẾN THỨC CẦN CÓ TRƯỚC

- **likelihood (độ hợp lý)** — xác suất mà một mô hình gán cho dữ liệu đã quan sát; “cực đại hoá likelihood” là chỉnh tham số sao cho mô hình thấy dữ liệu thật là chuyện bình thường chứ không phải chuyện lạ.
- **KL divergence (phân kỳ Kullback–Leibler)** — một cách đo khoảng cách giữa hai phân phối; nó *không đối xứng*, và chính sự không đối xứng đó sinh ra khác biệt giữa “mờ” và “mất mode”.
- **mode (mode của phân phối)** — một vùng tập trung nhiều xác suất — ví dụ “ảnh mèo” và “ảnh chó” là hai mode. Phủ hết mode nghĩa là sinh ra được cả hai loại.
- **mode collapse** — hiện tượng mô hình chỉ sinh ra một phần nhỏ của phân phối (chỉ vài kiểu ảnh) trong khi vẫn đạt điểm cao theo tiêu chí huấn luyện.
- **biến ẩn (latent variable)** — vectơ chiều thấp mà mô hình dùng làm “bản tóm tắt” của một mẫu; sinh ảnh là lấy ngẫu nhiên một vectơ như vậy rồi giải mã.
- **discriminator (bộ phân biệt)** — mạng thứ hai trong GAN, có việc duy nhất là phân biệt mẫu thật với mẫu do mô hình sinh; nó đóng vai một hàm mất mát *học được*.
- **score matching** — cách huấn luyện bằng việc khớp gradient của log mật độ thay vì khớp mật độ; đây là nền toán của diffusion.

A. Đặt vấn đề

Một mô hình sinh phải làm được một việc nghe rất mơ hồ: tạo ra mẫu mới “giống” dữ liệu huấn luyện nhưng không phải bản sao của nó. Muốn tối ưu bất cứ thứ gì thì phải viết “giống” thành một con số, và đây là chỗ bốn họ mô hình rẽ nhánh — chúng viết ra bốn con số khác nhau. Autoregressive đo bằng **độ hợp lý** (*likelihood*) chính xác của từng pixel theo thứ tự; VAE đo bằng một cận dưới của likelihood; GAN đo bằng “một bộ phân biệt học được không tách nổi mẫu giả khỏi mẫu thật”; diffusion đo bằng mức khớp của *score* ở mọi mức nhiễu. Bốn định nghĩa, bốn hình dạng lỗi.

Điều làm chủ đề này đáng học kỹ là các chế độ lỗi có thể *suy ra* chứ không phải ghi nhớ. Khi biết một mô hình tối ưu trung bình bình phương theo pixel, bạn dự đoán được nó cho ảnh mờ trước khi chạy nó; khi biết một mô hình tối ưu một hàm mục tiêu dạng trò chơi hai người, bạn dự đoán được nó không hội tụ theo nghĩa thông thường.

Học mục này thì làm được gì mà trước đó không làm được? Đọc một bài báo mô hình sinh và trả lời trong ba mươi giây: mô hình này sẽ mờ, thiếu đa dạng, hay chậm — và vì sao bắt buộc phải như vậy. Sau đó chọn được họ mô hình cho một bài toán cụ thể thay vì chọn theo cái đang nổi.

TRỰC GIÁC

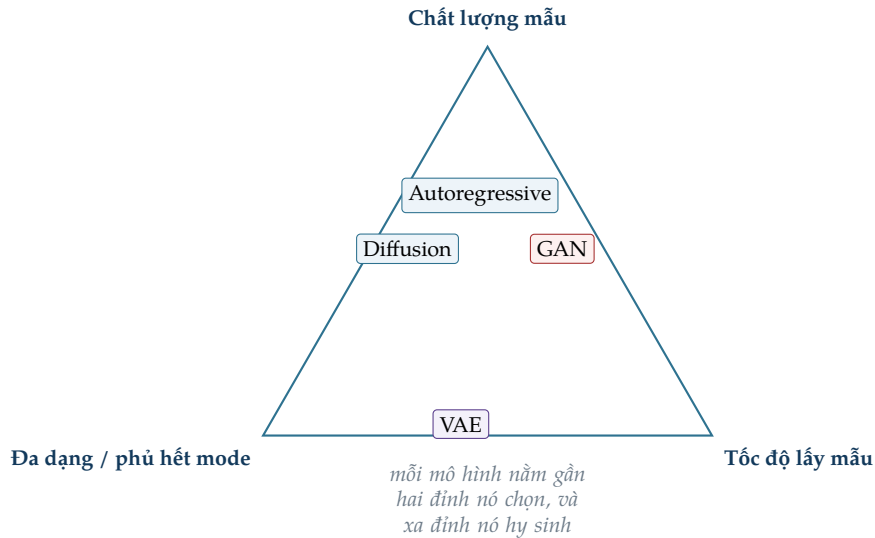
Bốn họ giống bốn cách chấm điểm một người làm tranh giả. *Autoregressive* bắt vẽ từng nét theo đúng xác suất của nét tiếp theo: chấm được điểm rất chính xác, nhưng vẽ chậm khủng khiếp vì mỗi nét phải chờ nét trước. *VAE* bắt nén bức tranh thành vài chục con số rồi giải nén: nhanh, nhưng phần giải nén trả về *trung bình* của mọi bức tranh khớp với mấy con số đó, nên nét bị nhoè. *GAN* thuê một giám định viên và chỉ đòi lừa được ông ta: nét rất sắc vì giám định viên soi nét — nhưng nếu ông ta chỉ quen soi hoa văn ở góc trái thì kẻ làm giả sẽ chỉ học vẽ giỏi đúng cái góc đó. *Diffusion* bắt đầu từ một tờ giấy đầy nhiễu và tẩy dần từng nấc, mỗi nấc chỉ phải đoán lùi một chút. Cách này chậm vì nhiều nấc. Bù lại mỗi nấc là một bài dễ, nên nó không bao giờ “vẽ hỏng”.

B. Bốn họ đặt cạnh nhau

Bảng 17.1. Bốn họ mô hình sinh, đọc theo định nghĩa của “giống”. Cột cuối suy ra được từ cột thứ hai — đó là điểm của cả bảng.

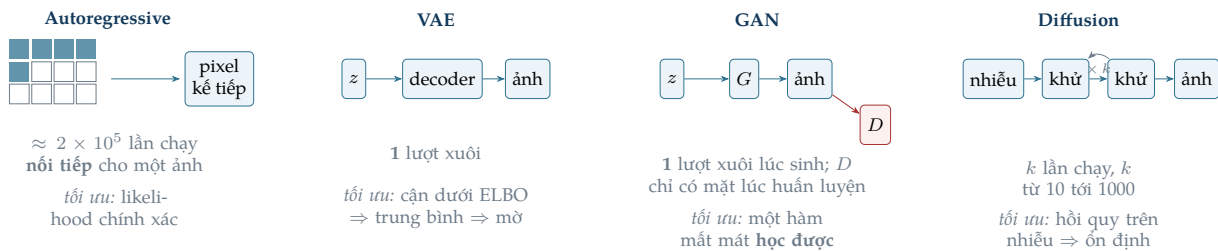
Họ	Định nghĩa “giống”	Được	Mất / hỏng khi
Auto-regressive (PixelRN-N/CNN)	Likelihood chính xác, phân rã theo quy tắc chuỗi	Tính được likelihood; huấn luyện ổn định	Sinh cực chậm vì tuần tự theo pixel; lệch giữa lúc huấn luyện và lúc sinh
VAE	Cận dưới của likelihood (ELBO)	Nhanh; có không gian ẩn có nghĩa, nội suy được	Ảnh mờ — hệ quả trực tiếp của loss trung bình; <i>posterior collapse</i>
GAN	“Một discriminator học được không phân biệt nổi”	Ảnh sắc nét ở cùng ngân sách tính toán	Huấn luyện bất ổn; mode collapse; không có likelihood để chẩn đoán
Diffusion	Khớp score của phân phối ở mọi mức nhiễu	Chất lượng và đa dạng cao; huấn luyện ổn định như hồi quy	Sinh chậm vì nhiều bước; nhạy với lựa chọn bộ lấy mẫu

Ba đại lượng trong cột ba và bốn không độc lập nhau. Trong tài liệu, người ta hay gọi tình thế này là *thế lưỡng nan ba chiều* của mô hình sinh: chất lượng mẫu, độ phủ hết các mode, và tốc độ lấy mẫu — ba thứ mà cho tới nay chưa họ nào đạt đồng thời cả ba ở mức tốt nhất. Cần nói rõ để trung thực: đây là một *cách gọi tên* một quan sát thực nghiệm lặp lại, không phải một định lý; nó khác hẳn về địa vị với đánh đổi perceptual-distortion ở mục 17.4, vốn là một kết quả có chứng minh. Hình 17.2 đặt bốn họ lên tam giác đó.



Hình 17.2. Thế lưỡng nan ba chiều. GAN mua chất lượng và tốc độ bằng độ phủ mode; VAE mua tốc độ và độ phủ bằng chất lượng; diffusion và autoregressive mua chất lượng cùng độ phủ bằng tốc độ. Toàn bộ tiến bộ kỹ thuật của mảng này — *sampler distillation, flow matching, latent diffusion* — là các nỗ lực kéo một điểm về gần tâm hơn.

Hình 17.3 bổ sung cho tam giác một thứ mà tam giác không nói được: *cơ chế* sinh ra mỗi vị trí trên đó.



Hình 17.3. Bốn cơ chế lấy mẫu, đặt cạnh nhau để thấy chi phí nằm ở đâu. Hai họ ở giữa sinh ảnh trong một lượt xuôi duy nhất; hai họ ở hai đầu phải chạy mạng nhiều lần, và đó chính là đỉnh “tốc độ lấy mẫu” mà chúng hy sinh trong Hình 17.2. Điểm đáng chú ý nhất là bộ phân biệt D của GAN: nó *không* tham gia lúc sinh, nó chỉ tồn tại lúc huấn luyện — vai trò của nó là một hàm mất mát học được, không phải một khối tính toán.

C. Vì sao họ sau ra đời

I MẠCH 1 — bốn định nghĩa của “giống dữ liệu thật”

- 1 Vấn đề** muốn mô hình hoá một phân phối trên không gian ảnh, mà không gian đó quá lớn để đếm.
- Giải pháp** phân rã theo quy tắc chuỗi: $p(x) = \prod_i p(x_i \mid x_{<i})$, rồi học từng thừa số bằng một mạng.
- Giả định** có một thứ tự tự nhiên trên pixel (thường là quét từ trái sang phải, trên xuống dưới — một giả định đã sai ngay từ đầu với ảnh).
- Hệ quả** likelihood tính được chính xác, nên có thể so sánh mô hình bằng một con số có ý nghĩa.
- Giá phải trả** lấy mẫu là tuần tự: một ảnh 256×256 màu cần 196 608 lần chạy mạng nối tiếp nhau; ở 1 ms mỗi lần thì hơn ba phút cho *một* ảnh.

- 2 Vấn đề** sinh tuần tự quá chậm để dùng được.
- Giải pháp** đưa vào một biến ẩn z chiều thấp, học đồng thời bộ mã hoá và bộ giải mã, tối ưu cận dưới ELBO [22]; lấy mẫu chỉ còn một lượt xuôi.
- Hệ quả** thêm một phần thưởng ngoài dự tính: không gian ẩn có cấu trúc, nội suy giữa hai điểm cho ra biến đổi mượt.
- Giá phải trả** bộ giải mã Gauss làm cho phần tái tạo của ELBO trở thành sai số bình phương trung bình theo pixel. Mà nghiệm tối ưu của bình phương trung bình là *kỳ vọng có điều kiện*, tức trung bình của mọi ảnh hợp lệ. Trung bình của nhiều ảnh sắc nét lệch nhau vài pixel là một ảnh mờ.
- **hệ quả của C/17/04** — cùng phép tính này quay lại ở khôi phục ảnh, nơi “mờ” trở thành nghiệm tối ưu PSNR và là một nửa của đánh đổi perceptual-distortion.
- Vấn đề mới** ảnh mờ không sửa được bằng cách huấn luyện lâu hơn; nó là nghiệm đúng của bài toán đã đặt ra.
- 3 Vấn đề** muốn ảnh sắc thì phải bỏ hàm mất mát trung bình, nhưng ta không biết viết công thức nào cho “trông thật”.
- Giải pháp** đừng viết — hãy *học* nó: cho một mạng thứ hai làm bộ phân biệt và huấn luyện bộ sinh để đánh lừa nó [176].
- Hệ quả** bộ phân biệt phạt bất cứ dấu hiệu nào lộ ra chất giả, kể cả dấu hiệu ta không nghĩ ra được, nên ảnh sắc nét [177].
- **cùng nguyên lý với C/17/02** — “không viết được hàm mất mát thì học nó” cũng chính là ý tưởng của mất mát tri giác, chỉ khác ở chỗ mạng chấm điểm được đóng băng thay vì huấn luyện đối kháng.
- Giá phải trả** mục tiêu không còn là cực tiểu hoá một hàm cố định mà là điểm cân bằng của một trò chơi hai người; “hội tụ” theo nghĩa thông thường không tồn tại, và mô hình có thể đạt mục tiêu bằng cách chỉ sinh ra một phần nhỏ của phân phối — *mode collapse*.
- Vấn đề mới** không có likelihood nên mất luôn công cụ chẩn đoán tự nhiên.
- 4 Vấn đề** muốn sắc nét như GAN nhưng ổn định như một bài hồi quy.
- Giải pháp** chẻ bài toán khó thành một chuỗi bài toán dễ: thêm nhiều dần cho tới khi ảnh thành nhiễu trắng, rồi học khử nhiễu ngược lại từng nấc [178], [179].
- Giả định** chuỗi đủ mịn để mỗi nấc chỉ cần một bước nhỏ.
- Hệ quả** hàm mất mát trở thành bình phương trung bình trên *nhiều* chứ không trên ảnh. Huấn luyện vì thế ổn định như mọi bài hồi quy. Đó là lý do diffusion đánh bại GAN: không phải bằng ý tưởng đẹp hơn, mà bằng việc dễ huấn luyện hơn ở quy mô lớn.
- Giá phải trả** sinh chậm vì phải chạy hàng chục tới hàng trăm nấc. Mục 17.3 dành trọn cho việc gỡ đúng cái giá này.

D. Ví dụ nhỏ nhất: vì sao mờ và mode collapse là hai mặt của một đồng xu

Lấy phân phối thật là hỗn hợp hai Gauss cách nhau: $p = \frac{1}{2}\mathcal{N}(-1, 0,25) + \frac{1}{2}\mathcal{N}(+1, 0,25)$. Bây giờ ép mô hình phải là *một* Gauss duy nhất $q = \mathcal{N}(\mu, \sigma^2)$. Mô hình quá nghèo để khớp — đúng tình huống thực tế, chỉ là ở chiều thấp.

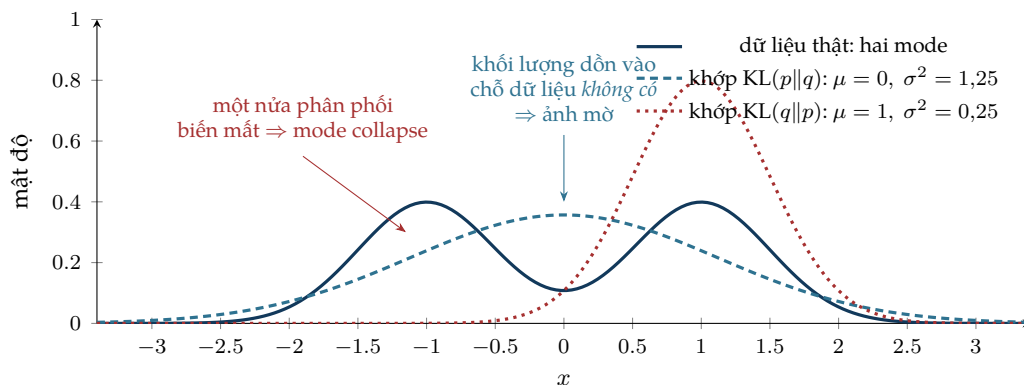
- **Nếu tối ưu $KL(p \parallel q)$** (hướng “thuận”, hướng mà cực đại hoá likelihood dùng): nghiệm là Gauss khớp mô-men của p , tức $\mu = 0$ và $\sigma^2 = 0,25 + 1 = 1,25$. Mô hình *trải rộng phủ cả hai mode*, và đặt nhiều khối lượng nhất ở $x = 0$ — đúng chỗ dữ liệu thật gần như không có gì. Dịch sang ảnh: đây chính là ảnh mờ.

- **Nếu tối ưu $KL(q \parallel p)$ (hướng “ngược”):** phạt rất nặng việc đặt khối lượng ở nơi p nhỏ, nên nghiệm co về đúng một mode, $\mu \approx \pm 1, \sigma^2 \approx 0,25$. Mẫu sinh ra đều hợp lệ và sắc nét — nhưng một nửa phân phối biến mất. Dịch sang ảnh: đây chính là mode collapse.

Hai dòng trên là toàn bộ lời giải thích. VAE mờ và GAN thiếu đa dạng không phải hai lỗi cần sửa bằng hai mẹo khác nhau; chúng là hai nghiệm của cùng một bài toán, khác nhau ở hướng đo khoảng cách. Chi tiết về hai hướng KL nằm ở [B/09-thiet-ke-ham-mat-mat](#); ở đây chỉ cần nhớ rằng chọn hàm mất mát là chọn kiểu thất bại.

↪ **tiền đề B/09** — hai hướng của KL là điều kiện tiên quyết thật sự của cả chương: không nắm nó thì “mờ” và “mất mode” trông như hai lỗi rời rạc.

Hình 17.4 vẽ đúng ba phân phối của ví dụ trên.



Hình 17.4. Toàn bộ chương gói trong một hình. Cùng một phân phối thật hai mode, cùng một họ mô hình nghèo (một Gauss duy nhất), chỉ đổi hướng đo khoảng cách thì ra hai nghiệm trái ngược. Hướng thuận phạt việc bỏ sót mode nên nghiệm trải rộng và đặt nhiều khối lượng nhất ở $x = 0$, đúng chỗ dữ liệu thật gần như trống — dịch sang ảnh, đó là ảnh mờ của VAE. Hướng ngược phạt việc đặt khối lượng ở nơi mật độ thật thấp nên nghiệm co về đúng một mode — dịch sang ảnh, đó là mode collapse của GAN. Số trong hình tính tay được từ ví dụ ở mục D.

E. Giới hạn và trường hợp hỏng

Bảng và tam giác ở trên đơn giản hoá một bức tranh thật ra phức tạp hơn, và có ba chỗ cần nói thẳng:

- **Ranh giới giữa các họ đang mờ đi.** Diffusion trong không gian ẩn dùng một autoencoder kiểu VAE; *flow matching* là diffusion viết lại dưới dạng phương trình vi phân thường; mô hình sinh ảnh bằng token tự hồi quy đang quay lại mạnh mẽ. Bốn họ là bốn *nguyên lý*, không phải bốn hộp kín.
- **Likelihood cao không đồng nghĩa mẫu đẹp.** Một mô hình có thể đạt likelihood tốt nhờ mô hình hoá tốt các chi tiết tần số cao vô nghĩa mà vẫn sinh ra ảnh xấu; ngược lại GAN không có likelihood nhưng mẫu đẹp. Hai đại lượng đo hai thứ khác nhau.
- **Đánh giá là chỗ yếu nhất của cả mảng.** Nhìn vài chục mẫu được chọn lọc không nói được gì về độ phủ mode, mà độ phủ mới là thứ hay hỏng. Mục 17.4 nói kỹ về chỗ này.

CẠM BẦY

Đừng đánh giá một mô hình sinh bằng những mẫu đẹp nhất của nó. Chế độ hỏng đắt nhất — mode collapse và thu hẹp phân phối — *không nhìn thấy được* trong mẫu chọn lọc, vì các mẫu còn lại vẫn đẹp, chỉ là chúng giống nhau. Phép thử tối thiểu: sinh vài nghìn mẫu với cùng điều kiện rồi đo độ đa dạng (khoảng cách trung bình theo cặp trong không gian đặc

trưng, số cụm sau khi gom nhóm), và đối chiếu với cùng số lượng mẫu thật. Nếu bạn định dùng mẫu sinh làm dữ liệu huấn luyện thì phép thử này là bắt buộc chứ không phải tùy chọn — xem [C/17-mo-hinh-sinh/04-low-level-vision](#).

CẦN NHỚ

Mỗi họ mô hình sinh là một định nghĩa của “giống dữ liệu thật”, và *kiểu thất bại của mô hình được quyết định bởi định nghĩa đó, không phải bởi kiến trúc*. Trung bình theo pixel cho ảnh mờ; hướng ngược của KL cho mode collapse; chuỗi bài toán dễ cho huấn luyện ổn định nhưng lấy mẫu chậm. Biết mô hình tối ưu đại lượng gì là biết trước nó sẽ hỏng thế nào. [T2]

F. Kết nối

Mục này là khung cho cả chương 17: [C/17-mo-hinh-sinh/02-toi-uu-tren-pixel](#) tách riêng một ý tưởng cắt ngang cả bốn họ (đóng băng mạng, tối ưu trên chính ảnh); [C/17-mo-hinh-sinh/03-diffusion](#) đào sâu họ thứ tư và các bước biến nó thành sản phẩm; [C/17-mo-hinh-sinh/04-low-level-vision](#) cho thấy cùng bộ khái niệm này quay lại ở bài toán khôi phục ảnh. Hướng KL và hệ quả “mờ đổi lại thu hẹp” thuộc [B/09-thiet-ke-ham-mat-mat](#). Ý tưởng “không viết được hàm mất mát thì học nó” cũng chính là ý tưởng của **mất mát tri giác** (*perceptual loss*) [102]. Nó cũng là ý tưởng của bộ phân biệt trong *pix2pix* [180], một cầu nối trực tiếp sang bài toán ảnh sang ảnh.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao ảnh mờ của VAE là *nghiệm đúng* của bài toán đã đặt ra chứ không phải dấu hiệu huấn luyện chưa đủ?
2. GAN không có likelihood. Điều đó lấy đi của bạn khả năng chẩn đoán cụ thể nào khi mô hình hoạt động kém?
3. Vì sao diffusion huấn luyện ổn định hơn GAN, và lý do đó có liên quan gì tới việc mục tiêu của nó là một bài hồi quy?
4. Bạn có một mô hình sinh và một trăm mẫu trông rất đẹp; phép thử ba mươi phút nào cho biết nó có bị thu hẹp phân phối hay không?
5. Nếu ràng buộc cứng của bạn là sinh 30 ảnh mỗi giây trên một GPU nhúng, tam giác ở Hình 17.2 chỉ về đâu, và bạn chấp nhận mất gì?

17.2 Tối ưu trên pixel thay vì trên trọng số (Optimizing Pixels, Not Weights)

PHỤ THUỘC [C/12-co-che-huan-luyen/01-co-hoc-cua-gradient](#) (gradient chảy về đâu là do ta chọn); [C/17-mo-hinh-sinh/01-bon-ho-mo-hinh-sinh](#) (hàm mất mát quyết định kiểu thất bại); [C/13-kien-truc-backbone/01-dong-cnn](#) (bản đồ đặc trưng theo tầng).

TÓM TẮT

Có một ý tưởng đơn giản nối liền năm kỹ thuật trông chẳng liên quan gì tới nhau: đóng băng một mạng đã huấn luyện, coi *ảnh* là biến cần tối ưu, rồi hạ gradient trên chính pixel. Đổi hàm mục tiêu là được một kỹ thuật khác — đảo ngược đặc trưng, DeepDream, trực quan hoá lớp, mẫu đối kháng, và chuyển phong cách. Đọc xong bạn viết được cả năm bằng cùng một vòng lặp mười dòng, và hiểu vì sao chuyển phong cách phiên bản tiến thẳng là ví dụ sạch nhất của mẫu hình “đẩy chi phí từ lúc

chạy sang lúc huấn luyện”. Nếu chỉ nhớ một điều: tối ưu trên pixel tìm ra đúng những hướng mà mạng và mắt người *bất đồng*; khi hai bên tình cờ đồng ý ta gọi kết quả là nghệ thuật, khi bất đồng ta gọi nó là mẫu đối kháng — cùng một cơ chế.

KIẾN THỨC CẦN CÓ TRƯỚC

- **lan truyền ngược (backpropagation)** — thuật toán tính đạo hàm của một đại lượng đầu ra theo mọi nút của một đồ thị tính toán khả vi. Nó không phân biệt “tham số” với “dữ liệu” — đó là tiền đề của cả mục này.
- **bản đồ đặc trưng (feature map)** — tensor đầu ra của một tầng tích chập: một chồng “ảnh” nhỏ hơn, mỗi kênh phản ứng với một loại mẫu hình thị giác.
- **đóng băng (freeze)** — đặt cờ không tính gradient cho trọng số của một mạng, để nó chỉ còn đóng vai một hàm chấm điểm cố định.
- **ma trận Gram** — ma trận tích trong giữa các kênh của một bản đồ đặc trưng, cộng dồn trên mọi vị trí không gian; nó mô tả “kênh nào hay bật cùng kênh nào” mà bỏ hẳn thông tin vị trí.
- **mẫu đối kháng (adversarial example)** — ảnh bị thêm một nhiễu rất nhỏ, mắt người không thấy, nhưng đủ để mạng phân loại sai hoàn toàn.
- **hàm mục tiêu (objective function)** — toàn bộ đại lượng được tối ưu, gồm phần đo sai lệch (**hàm mất mát (loss function)**) cộng các số hạng chính quy hoá. Mục này dùng đúng từ “hàm mục tiêu” vì phần chính quy hoá ở đây quyết định kết quả không kém phần đo sai lệch.
- **chính quy hoá (regularization)** — phần cộng thêm vào hàm mục tiêu để loại bỏ các nghiệm “đúng về số nhưng vô lý về mặt ảnh”, ví dụ phạt biến thiên toàn phần để ép ảnh mượt.
- **feed-forward (tiền thẳng)** — chạy dữ liệu qua mạng đúng một lần từ đầu tới cuối, không lặp — đối lập với một vòng tối ưu nhiều trăm bước.

A. Đặt vấn đề

Trong huấn luyện thông thường, dữ liệu là hằng số và trọng số là ẩn số: ta cố định ảnh, tính gradient của hàm mất mát theo trọng số, rồi cập nhật trọng số. Nhưng **lan truyền ngược (backpropagation)** không quan tâm ta gọi cái gì là ẩn số — nó tính đạo hàm dọc theo một đồ thị khả vi, và đồ thị đó có đầu vào cũng là một nút. Vậy hoàn toàn có thể làm ngược lại: cố định trọng số, tính gradient theo *pixel đầu vào*, rồi cập nhật ảnh.

Câu hỏi thú vị không phải “làm được không” mà “để làm gì”. Câu trả lời: một mạng đã huấn luyện là một hàm chấm điểm ngầm định của thế giới thị giác — nó đã học được cái gì là mèo, cái gì là hoa văn, cái gì là biên. Tối ưu trên pixel là cách ép hàm chấm điểm đó *nói ra* thứ nó biết, dưới dạng một bức ảnh. Hình 17.5 là toàn bộ sự khác biệt, vẽ trong một hình.

Học mục này thì làm được gì mà trước đó không làm được? Bốn việc: cài chuyển phong cách, DeepDream, trực quan hoá lớp và tấn công đối kháng bằng cùng một đoạn code; giải thích được vì sao ma trận Gram bắt được “phong cách”; nhận ra mẫu hình dờ chi phí sang lúc huấn luyện khi gộp lại nó ở **chưng cất tri thức (knowledge distillation)**; và có một công cụ chẩn đoán mạnh — muốn biết mạng của bạn thật sự nhìn cái gì, hãy tối ưu một ảnh làm nó phát điên rồi nhìn ảnh đó.



Hình 17.5. Cùng một đồ thị khả vi, hai cách dùng. Lan truyền ngược không phân biệt “dữ liệu” với “tham số” — nó chỉ tính đạo hàm dọc đồ thị; việc chọn nút nào là biến tối ưu hoàn toàn nằm ở người viết. Đây là lý do năm kỹ thuật ở Bảng 17.2 dùng chung một đoạn code.

↪ trường hợp riêng của `C/12/01` — tối ưu trên pixel chỉ là lan truyền ngược chạy tới tận nút đầu vào thay vì dừng ở nút tham số.

TRỰC GIÁC

Hãy hình dung mạng đã huấn luyện là một giám khảo hội họa rất nhất quán nhưng không biết nói — ông ta chỉ chấm điểm, không giải thích. Tối ưu trên pixel là điêu khắc một bức tượng bằng cách liên tục hỏi “sửa chỗ nào thì điểm tăng” rồi sửa đúng chỗ đó. Bức tượng cuối cùng phản ánh chính xác thứ mà giám khảo quan tâm — kể cả những thứ vô lý mà ông ta quan tâm nhưng con người thì không. Khi những thứ đó trùng với thẩm mỹ của ta, ta gọi kết quả là chuyển phong cách; khi chúng lệch hẳn, ta gọi kết quả là mẫu đối kháng.

B. Một vòng lặp, năm kỹ thuật

Vòng lặp chung ngắn tới mức viết ra được trọn vẹn:

```
# Đóng băng mạng; biến tối ưu là ảnh, không phải net.parameters()
for p in net.parameters():
    p.requires_grad_(False)
x = init_image.clone().requires_grad_(True)
opt = torch.optim.LBFGS([x])
def step():
    opt.zero_grad()
    loss = objective(net, x) # đổi objective -> đổi kỹ thuật
    loss.backward() # gradient chảy về tận x
    return loss
for _ in range(300):
    opt.step(step)
```

Chỉ dòng `objective` thay đổi giữa năm kỹ thuật ở Bảng 17.2. Đây không phải một sự trùng hợp thú vị mà là một bài học về cách đọc tài liệu: khi năm phương pháp chia chung một khung, học khung rẻ hơn học năm phương pháp gấp nhiều lần.

Bảng 17.2. Năm kỹ thuật cùng một vòng lặp, khác nhau đúng ở hàm mục tiêu.

Kỹ thuật	Hàm mục tiêu	Nó tiết lộ điều gì / hỏng khi nào
Đảo ngược đặc trưng	Khớp bản đồ đặc trưng của một ảnh mục tiêu ở một tầng	Tầng càng sâu càng mất thông tin vị trí; ảnh khôi phục ngày càng “đúng nội dung, sai bố cục”
DeepDream	Cực đại hoá chuẩn của activation ở một tầng	Cho thấy tầng đó thích hoa văn gì; kết quả lặp lại một mô-típ tới mức bệnh hoạn
Trực quan hoá lớp	Cực đại hoá điểm số của một lớp + phạt chính quy hoá	Không có phạt thì ra nhiều có điểm số cao — bằng chứng trực tiếp rằng mạng không nhìn như người
Mẫu đối kháng	Giảm điểm số lớp đúng với ràng buộc $\ \delta\ $ nhỏ	Tồn tại nhiều vô hình đối hẩn dự đoán; đây là chế độ hỏng của <i>mọi</i> mạng, không phải của một mạng kém
Chuyển phong cách	Nội dung ở tầng sâu + thống kê phong cách ở nhiều tầng nông	Phong cách nắm được là <i>kết cấu</i> , không phải bố cục hay ngữ nghĩa

C. Neural Style Transfer: vì sao ma trận Gram lại là “phong cách”

Bài toán của Gatys: giữ *nội dung* của ảnh A và *phong cách* của ảnh B [103]. Nội dung dễ định nghĩa — bản đồ đặc trưng ở một tầng sâu, vì tầng sâu giữ “có gì ở đâu” mà bỏ bớt chi tiết bề mặt. Phong cách mới là chỗ khó: nó phải là thứ mô tả “chất” của bức tranh mà không mô tả nội dung, tức là phải *độc lập với vị trí*.

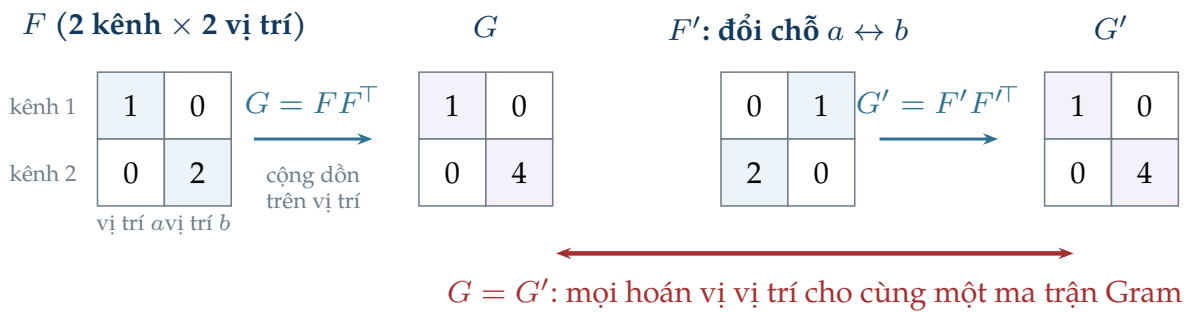
Lời giải là **ma trận Gram** (*Gram matrix*). Gọi $F \in \mathbb{R}^{C \times N}$ là bản đồ đặc trưng của một tầng, với C kênh và N vị trí không gian; đặt $G = FF^\top$, tức $G_{ij} = \sum_p F_{ip}F_{jp}$. Công thức này nói gì: G_{ij} đo mức độ hai kênh i và j cùng bật lên tại cùng một vị trí, cộng dồn trên toàn ảnh — nó là thống kê *đồng xuất hiện* của các đặc trưng, và phép cộng dồn trên p đã vứt bỏ hoàn toàn thông tin vị trí.

Tính tay trên ma trận 2×2 . Hai kênh, hai vị trí:

$$F = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix} \Rightarrow G = FF^\top = \begin{pmatrix} 1 & 0 \\ 0 & 4 \end{pmatrix}, \quad F' = \begin{pmatrix} 0 & 1 \\ 2 & 0 \end{pmatrix} \Rightarrow G' = \begin{pmatrix} 1 & 0 \\ 0 & 4 \end{pmatrix}.$$

Đổi chỗ hai vị trí không gian mà G không đổi. Đó vừa là sức mạnh — “phong cách” đúng là thứ không phụ thuộc chỗ đặt — vừa là giới hạn: *Gram không phân biệt được bố cục*. Nó nắm được kết cấu nét cọ, bảng màu, độ thô của hạt; nó không nắm được “chân dung thì mất ở trên miệng”. Mọi thất bại quen thuộc của chuyển phong cách đều suy ra từ đây: khuôn mặt bị loang màu, bầu trời mọc nét cọ ở giữa mặt người, cấu trúc lớn của tranh gốc biến mất.

Hình 17.6 vẽ đúng phép tính đó.



Hình 17.6. Vì sao ma trận Gram bắt được “phong cách” — và vì sao nó không bao giờ bắt được bố cục. Phép cộng dồn trên các vị trí không gian biến F thành một thống kê *đồng xuất hiện giữa các kênh*, và phép cộng thì không quan tâm thứ tự. Hai bản đồ đặc trưng khác hẳn nhau về chỗ đặt cho ra cùng một G . Đó vừa là định nghĩa vừa là giới hạn của “phong cách”: nét cọ và bảng màu thì giữ được, “mắt nằm trên miệng” thì không.

Đáng dừng lại một nhịp ở chỗ này, vì đây là lần thứ hai ta gặp cùng một mẹo: G là một *tổng* trên các vị trí, tức một hàm đối xứng — đúng cấu trúc mà PointNet dùng khi lấy max trên các điểm (C/16-thi-giac-3d/02-bieu-dien-3d-va-depth). Hai lĩnh vực không liên quan, một nguyên lý: muốn bất biến với thứ tự hay vị trí thì gộp bằng một phép toán đối xứng, và cái giá luôn luôn là mất chính cái thông tin thứ tự đó.

↔ cùng nguyên lý với C/16/02 — PointNet lấy max trên tập điểm, Gram lấy tổng trên vị trí: cùng một hàm đối xứng, cùng một cái giá.

D. Dời chi phí từ lúc chạy sang lúc huấn luyện

Bản gốc của Gatys phải chạy vài trăm bước tối ưu cho *mỗi* ảnh, mất từ vài chục giây tới vài phút trên GPU — không dùng được cho video hay cho ứng dụng tương tác. Cách gỡ là một mẫu hình tổng quát, không riêng gì chuyển phong cách.

I. Mạch 2 — ba nấc của cùng một bài toán

1 Vấn đề

mỗi ảnh cần một vòng tối ưu riêng.

Giải pháp

huấn luyện một mạng **tiền thẳng** (*feed-forward*) để làm đúng việc mà vòng tối ưu làm, dùng chính hàm mất mát của Gatys làm tín hiệu giám sát — tức là **mất mát tri giác** (*perceptual loss*) [102].

Hệ quả

lúc chạy chỉ còn một lượt xuôi, nhanh hơn ba bậc độ lớn.

Giá phải trả

phải huấn luyện một mạng cho *mỗi* phong cách, và mất khả năng tinh chỉnh trọng số nội dung/phong cách sau khi huấn luyện.

2 Vấn đề

một mạng cho một phong cách là không mở rộng được.

Giải pháp

AdaIN: chuẩn hoá đặc trưng của ảnh nội dung về trung bình–độ lệch chuẩn 0–1 theo từng kênh, rồi *gán lại* trung bình và độ lệch chuẩn theo từng kênh của ảnh phong cách [181].

Giả định

điều đáng nói nằm ở giả định ngầm: nếu chỉ cần khớp thống kê bậc nhất theo kênh đã ra phong cách, thì phần lớn nội dung của ma trận Gram (thống kê bậc hai) là dư thừa — một phát hiện thực nghiệm khá bất ngờ.

Hệ quả

một mạng duy nhất làm được phong cách tùy ý, thời gian thực.

Giá phải trả

chất lượng thấp hơn bản tối ưu; các phong cách có cấu trúc lớn (tranh có hình khối rõ) bị mất nhiều nhất.

3 Vấn đề

ta vẫn chỉ có một đánh đổi chứ chưa có bữa trưa miễn phí.

Hệ quả

mẫu hình chung: độ khó không biến mất, nó chuyển từ lúc chạy sang lúc huấn luyện — và cùng mẫu hình đó sẽ xuất hiện ở chừng cất tri thức (*D/20-toi-uu-va-nen-mo-hinh*) và ở việc chừng cất bộ lấy mẫu diffusion (*C/17-mo-hinh-sinh/03-diffusion*).

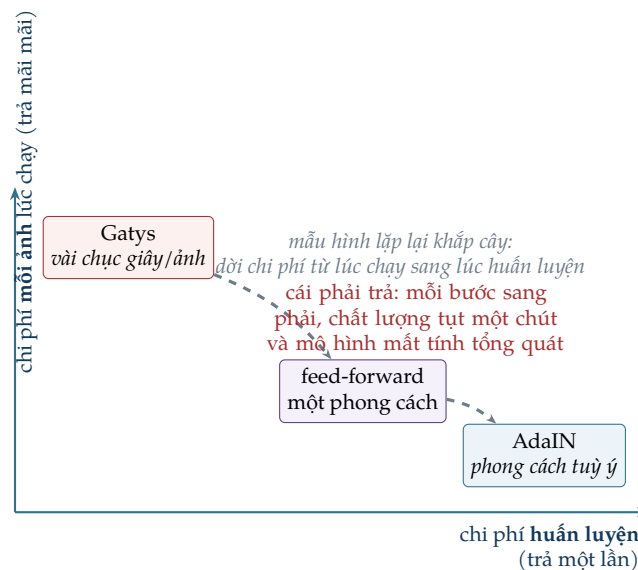
→ cùng nguyên lý với *D/20* — chừng cất tri thức là chính mẫu hình này ở tầng mô hình: trả một lần lúc huấn luyện để khỏi trả mãi lúc suy luận.

→ cùng nguyên lý với *C/16/03* — neural rendering làm đúng việc này trên một cảnh 3D thay vì trên một ảnh.

Bảng 17.3. Ba nấc của chuyển phong cách, đọc theo chỗ chi phí nằm ở đâu.

Cách làm	Chi phí mỗi ảnh	Chi phí huấn luyện	Hỏng khi nào
Gatys (tối ưu trên pixel)	Vài chục giây tới vài phút	Không có	Không dùng được cho video hay tương tác
Tiến thẳng một phong cách	Một lượt xuôi	Một lần huấn luyện cho mỗi phong cách	Số phong cách lớn; muốn đổi trọng số sau khi huấn luyện
AdaIN (phong cách tùy ý)	Một lượt xuôi	Một lần huấn luyện duy nhất	Phong cách có cấu trúc lớn; chất lượng dưới bản tối ưu

Hình 17.7 đặt ba nấc lên một mặt phẳng để thấy chi phí đã đi đâu.



Hình 17.7. Ba nấc của chuyển phong cách, vẽ trên mặt phẳng “ai trả chi phí”. Không nấc nào rẻ hơn nấc trước một cách miễn phí: cái được lấy đi ở trục dọc bị trả lại ở trục ngang, và một phần nhỏ nữa bị trả bằng chất lượng. Đây là cùng một hình sẽ đúng cho chừng cất tri thức và cho chừng cất bộ lấy mẫu diffusion — chỉ cần đổi nhãn ba hộp.

E. Giới hạn và trường hợp hỏng

Giới hạn nền tảng của cả mục này là ta đang tin vào một hàm chấm điểm mà chính ta không kiểm soát. Mạng đã huấn luyện có vô số hướng trong không gian ảnh mà nó nhạy còn mất người thì không — tối ưu trên pixel đi thẳng vào những hướng đó, vì gradient bao giờ cũng chỉ về hướng dốc nhất. Đó là lý do trực quan hoá lớp không có phạt chính quy hoá cho ra nhiều, và là lý do mẫu đối kháng tồn tại ở mọi mạng.

Hai chế độ hỏng cụ thể đáng nhớ:

- **Kết quả phụ thuộc mạnh vào khởi tạo và bộ tối ưu.** Bài toán không lỗi trên hàng trăm nghìn biến; khởi tạo từ ảnh nội dung, từ nhiễu, hay từ ảnh trắng cho ba kết quả khác nhau. Đây là tính chất của bài toán chứ không phải lỗi cài đặt — báo cáo kết quả mà không nói khởi tạo là báo cáo thiếu.
- **Chuyển phong cách mất tính nhất quán theo thời gian.** Áp cho từng khung hình video thì ảnh nhấp nháy, vì hai khung hình gần nhau rơi vào hai cực tiểu địa phương khác nhau. Cách chữa là thêm ràng buộc thời gian dựa trên dòng quang — và ta lại quay về đúng chủ đề của [C/15-thi-giac-khong-thoi-gian](#).

CẠM BẤY

Đừng đọc một ảnh trực quan hoá lớp như bằng chứng “mạng hiểu khái niệm này”. Ảnh đó là nghiệm của một bài tối ưu *có phạt chính quy hoá do bạn chọn*, và phần lớn cấu trúc đẹp mắt trong đó đến từ chính phần phạt (biến thiên toàn phần, jitter, làm mượt) chứ không từ mạng. Bằng chứng: bỏ phạt đi thì ra nhiễu, dù điểm số của lớp còn cao hơn. Muốn dùng công cụ này để chẩn đoán, hãy luôn báo cáo phần phạt cùng với ảnh — xem thêm [E/25-do-tin-cay-va-van-hanh](#) về các phép kiểm tra tính đúng đắn của phương pháp giải thích [160].

CẦN NHỚ

Lan truyền ngược không phân biệt tham số với dữ liệu; chọn nút nào làm biến tối ưu là quyền của bạn. Đóng băng mạng và tối ưu trên pixel cho ra năm kỹ thuật khác nhau chỉ bằng cách đổi hàm mục tiêu — và bất cứ khi nào một vòng tối ưu lúc chạy quá đắt, hãy hỏi liệu có huấn luyện được một mạng làm thay nó hay không. Đó là mẫu hình dời chi phí sang lúc huấn luyện, gặp lại ở distillation. [T1]

F. Kết nối

Ý tưởng của mục này là bản sao chính xác của điều đã gặp ở [C/16-thi-giac-3d/03-nerf-3dgs-va-hop-nhat](#): đặt thứ ta muốn vào hàm mất mát rồi hạ gradient trên chính đối tượng cần tạo ra — ở đó đối tượng là một cảnh 3D, ở đây là một ảnh. Ma trận Gram dùng chung nguyên lý hàm đối xứng với PointNet ở [C/16-thi-giac-3d/02-bieu-dien-3d-va-depth](#). Mẫu đối kháng và trực quan hoá được nói kỹ ở [E/25-do-tin-cay-va-van-hanh](#); mẫu hình dời chi phí sang lúc huấn luyện quay lại ở [D/20-toi-uu-va-nen-mo-hinh](#) [23] và ở phần chứng cất bộ lấy mẫu của [C/17-mo-hinh-sinh/03-diffusion](#). Perceptual loss cũng chính là cầu nối sang [C/17-mo-hinh-sinh/04-low-level-vision](#).

❏ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao cùng một đoạn code cho ra được cả một tác phẩm nghệ thuật lẫn một mẫu đối kháng — điều gì thực sự khác nhau giữa hai trường hợp?
2. Ma trận Gram bất biến với vị trí; hai thất bại nào của chuyển phong cách suy ra trực tiếp từ tính chất đó?
3. AdaIN chỉ khớp trung bình và độ lệch chuẩn theo kênh mà vẫn ra phong cách. Điều đó gợi ý gì về lượng thông tin thực sự cần thiết trong ma trận Gram?
4. Bạn thấy một ảnh trực quan hoá lớp rất đẹp trong một bài báo. Ba câu hỏi bạn phải hỏi trước khi tin nó là gì?

5. Mẫu hình “dời chi phí sang lúc huấn luyện” được mua bằng cái gì, và trong trường hợp cụ thể nào thì cái giá đó là không chấp nhận được?

17.3 Diffusion và các bước làm cho nó khả thi (Diffusion Models)

PHỤ THUỘC C/17-mo-hinh-sinh/01-bon-ho-mo-hinh-sinh (vì sao diffusion ổn định hơn GAN); B/09-thiet-ke-ham-mat-mat (hồi quy trên nhiễu là bình phương trung bình); C/13-kien-truc-backbone (U-Net và transformer); C/12-co-che-huan-luyen (huấn luyện quy mô lớn).

TÓM TẮT

Diffusion biến một bài toán sinh rất khó thành một chuỗi bài toán khử nhiễu rất dễ, và đó là toàn bộ lý do nó thắng: hàm mất mát của nó là bình phương trung bình trên nhiễu, ổn định như mọi bài hồi quy. Mục này đi từ nguyên lý (quá trình thuận cố định, quá trình ngược học được, liên hệ với score function) tới bốn bước biến nó thành sản phẩm dùng được: bộ lấy mẫu ít bước, dẫn hướng không cần bộ phân lớp, khuếch tán trong không gian ẩn, và cơ chế điều khiển đầu ra. Đọc xong bạn biết mỗi tham số trong một pipeline sinh ảnh đang mua gì bằng gì — số bước, hệ số dẫn hướng, độ mạnh ảnh sang ảnh — và biết *latent diffusion* phá hỏng cái gì khi nó tiết kiệm tính toán. Nếu chỉ nhớ một điều: mọi cơ chế điều khiển trong diffusion đều mua độ trung thành bằng *độ đa dạng*, nên một pipeline chỉnh tới mức đẹp nhất cũng là pipeline có phân phối hẹp nhất — điều nguy hiểm nếu bạn định dùng đầu ra làm dữ liệu huấn luyện.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình sinh (generative model)** — mô hình học phân phối của dữ liệu chứ không học một ảnh xạ đầu vào → nhãn; dùng nó là lấy mẫu ra dữ liệu mới, không phải dự đoán.
- **nhiều Gauss (Gaussian noise)** — nhiễu cộng lấy từ phân phối chuẩn; “pha nhiễu” nghĩa là cộng thêm một lượng nhiễu như vậy vào ảnh, làm ảnh mờ dần thành hạt.
- **khử nhiễu (denoising)** — việc ước lượng lại ảnh sạch (hoặc lượng nhiễu đã cộng vào) từ ảnh đã bị pha nhiễu. Diffusion chỉ làm đúng việc này, lặp lại nhiều lần.
- **bộ lấy mẫu (sampler)** — đoạn mã quyết định đi từ nhiễu về ảnh theo mấy bước và mỗi bước dài bao nhiêu. Nó tách rời khỏi mô hình: cùng một mô hình đổi sampler cho ra ảnh khác nhau.
- **không gian ẩn (latent space)** — không gian chiều thấp mà một autoencoder nén ảnh vào; một ảnh 512×512 thành một tensor 64×64 , nên mọi phép tính sau đó rẻ đi hàng chục lần.
- **dẫn hướng (guidance)** — kỹ thuật kéo mẫu sinh ra về phía thoả mãn điều kiện (câu mô tả, ảnh cạnh...) mạnh hơn mức tự nhiên, đổi lại phân phối hẹp hơn.
- **score function (score function)** — gradient của log mật độ xác suất theo dữ liệu, $\nabla_x \log p(x)$; nó là một trường vector chỉ về hướng “đi đâu thì hợp lý hơn” tại mỗi điểm trong không gian ảnh.
- **SNR (signal-to-noise ratio)** — tỉ số giữa phần tín hiệu gốc còn lại và phần nhiễu ở một bước; SNR cao nghĩa là ảnh còn gần như nguyên vẹn.

A. Đặt vấn đề

Sau mục 17.1 ta có một yêu cầu rõ ràng: cần chất lượng của GAN mà không có sự bất ổn của một bài toán trò chơi. Ý tưởng của diffusion đơn giản một cách bất ngờ — nếu bài toán “sinh một ảnh từ hư không” quá khó, hãy chế nó thành một nghìn bài toán “làm ảnh này bớt nhiễu đi một chút”, mỗi bài dễ tới mức một mạng hồi quy bình thường giải được. Cộng đủ nhiều bước dễ thì ra một bước khó.

Cái phải trả cho sự chẻ nhỏ đó là thời gian: sinh một ảnh cần chạy mạng hàng chục tới hàng nghìn lần. Toàn bộ lịch sử kỹ thuật của mảng này, từ 2020 tới nay, là chuỗi các nỗ lực trả bớt cái giá đó mà không mất chất lượng — và đó cũng là cấu trúc của mục này.

Học mục này thì làm được gì mà trước đó không làm được? Đọc một tệp cấu hình `diffusers` và biết mỗi tham số đang đánh đổi gì; chẩn đoán được ảnh ra xấu là do bộ lấy mẫu, do hệ số dẫn hướng, hay do bộ mã hoá ẩn; và biết khi nào một mô hình sinh ảnh *không* nên được dùng để tạo dữ liệu huấn luyện.

TRỰC GIÁC

Hãy nghĩ tới một bức ảnh bị nhoè dần trong một chuỗi bản sao chép: bản 1 gần như nguyên vẹn, bản 500 lem nhem, bản 1000 chỉ còn là hạt. Việc đoán bản 1000 từ hư không là bất khả. Nhưng việc đoán bản 499 khi đã có bản 500 thì gần như tầm thường — chỉ phải lùi một nấc. Diffusion là phép nhận ra rằng nếu bạn làm được cái tầm thường đó một nghìn lần, bạn đã làm được cái bất khả. Và điều tinh tế: mấy nấc *cuối* (nhiều nhiều) là nơi mô hình quyết định bức ảnh sẽ vẽ *cái gì*, còn mấy nấc *đầu* (nhiều ít) chỉ tô nốt chi tiết.

B. Quá trình thuận, quá trình ngược

Quá trình thuận không có gì để học: nó chỉ là công thức pha nhiễu Gauss dần dần, và nhờ tính chất cộng của Gauss, ta nhảy thẳng từ ảnh gốc tới bước bất kỳ mà không cần lặp:

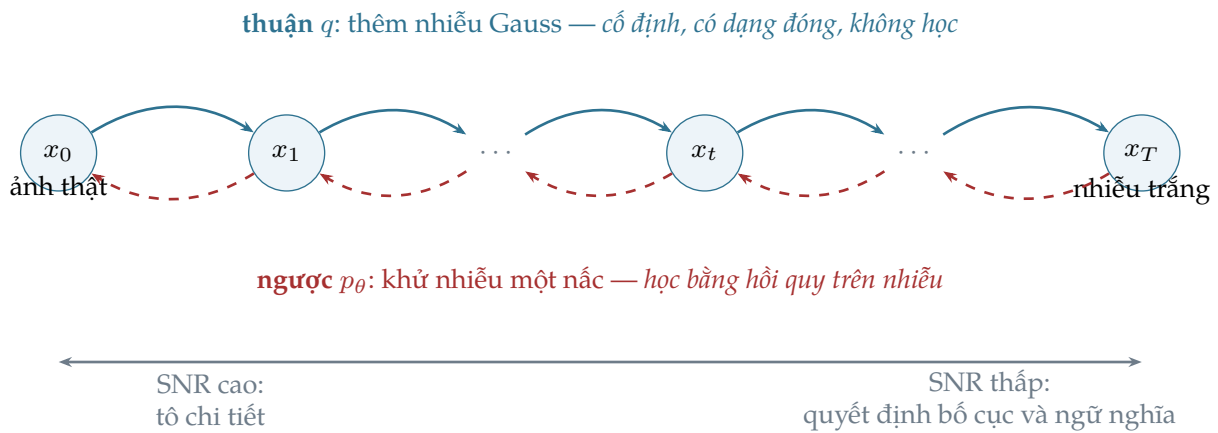
$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I), \quad \bar{\alpha}_t = \prod_{s \leq t} (1 - \beta_s).$$

Công thức này nói gì: ở bước t , ảnh còn lại đúng một phần $\sqrt{\bar{\alpha}_t}$ tín hiệu gốc, phần còn lại là nhiễu trắng — và vì có dạng đóng, ta lấy mẫu một t ngẫu nhiên rồi huấn luyện ngay, không phải chạy chuỗi. Đây là lý do kỹ thuật khiến diffusion huấn luyện được ở quy mô lớn.

Tính tay để thấy $T = 1000$ từ đâu ra. Với lịch β tuyến tính từ 10^{-4} tới 0,02, trung bình $\beta \approx 0,0101$, nên $\bar{\alpha}_T \approx \exp(-\sum_t \beta_t) = \exp(-10,1) \approx 4 \times 10^{-5}$, tức $\sqrt{\bar{\alpha}_T} \approx 0,006$. Nói cách khác, sau một nghìn bước thì chưa tới 1 % biên độ tín hiệu gốc còn sót lại — vừa đủ để coi x_T là nhiễu trắng thuần túy và bắt đầu quá trình ngược từ đó.

Quá trình ngược mới là phần học. Mạng nhận x_t và t , dự đoán *nhiều* đã thêm vào, và hàm mất mát là bình phương trung bình $\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$ [178]. Đây là điểm mấu chốt của cả họ mô hình: mục tiêu là một bài hồi quy tầm thường, không phải một trò chơi hai người, nên không có bất ổn, không có *mode collapse*, và tăng quy mô thì tốt lên đều đặn.

↪ **hệ quả của B/09** — ổn định của diffusion đến thẳng từ dạng của hàm mất mát — bình phương trung bình trên nhiễu — chứ không từ kiến trúc nào.



Hình 17.8. Hai chiều của cùng một chuỗi. Chiều thuận không có tham số nào, nên toàn bộ khả năng của mô hình dồn vào chiều ngược. Trục dưới giải thích một điều thực hành quan trọng: các bước ở phía nhiễu nhiều quyết định *nội dung* ảnh, nên cắt bớt bước ở phía đó thay đổi ảnh nhiều hơn hẳn cắt bớt ở phía nhiễu ít — đó là lý do các lịch lấy mẫu ít bước không chia đều khoảng cách.

C. Vì sao gọi là “khớp score function”, và vì sao cần nhiều bước

Có một cách nhìn thứ hai cho cùng một mô hình, và nó giải thích được nhiều thứ hơn. Với phân phối p_t của dữ liệu đã pha nhiễu ở mức t , **score function** là $\nabla_x \log p_t(x)$ — vector chỉ về hướng “đi đâu thì hợp lý hơn”. Có thể chứng minh rằng mạng dự đoán nhiễu ở trên chính là ước lượng của score function sai khác một hệ số: $\varepsilon_\theta(x_t, t) \approx -\sqrt{1 - \alpha_t} \nabla_x \log p_t(x_t)$ [179]. *Nói gọn thì:* mô hình học một trường vector phủ khắp không gian ảnh, và lấy mẫu là thả một điểm nhiễu vào trường đó rồi để nó trôi về vùng có mật độ cao.

Cách nhìn này trả lời ngay câu hỏi “vì sao cần nhiều bước”: trôi theo một trường vector là giải một phương trình vi phân, và mỗi bước rời rạc là một xấp xỉ tuyến tính chỉ đúng trong lân cận nhỏ. Bước dài thì sai số rời rạc hoá lớn, ảnh ra méo. Từ đó suy ra hai hướng cải thiện, và cả hai đều đã thành chuẩn:

- **Đổi bộ giải.** DDIM biến quá trình lấy mẫu ngẫu nhiên thành một phương trình vi phân thường tất định, cho phép nhảy bước dài hơn với cùng mô hình đã huấn luyện [182]. Không huấn luyện lại gì cả — đây thuần tuý là giải số tốt hơn.
- **Làm thẳng quỹ đạo.** *Flow matching* huấn luyện thẳng một trường vận tốc nổi nhiễu với dữ liệu theo đường càng gần đường thẳng càng tốt [183]; quỹ đạo thẳng thì bộ giải cần ít bước hơn. Đây là hướng thiết kế thống trị các mô hình sinh lớn hiện nay.

D. Điều khiển đầu ra: dẫn hướng, không gian ẩn, ràng buộc cấu trúc

I MẠCH 3 — bốn bước biến diffusion thành sản phẩm

1 Vấn đề

Giải pháp

mô hình sinh ra ảnh hợp lý nhưng không theo ý ta.

classifier-free guidance: trong lúc huấn luyện, thỉnh thoảng bỏ điều kiện đi để một mạng duy nhất học cả bản có điều kiện lẫn bản không; lúc lấy mẫu, ngoại suy $\tilde{\varepsilon} = \varepsilon_{\text{vô đk}} + w(\varepsilon_{\text{có đk}} - \varepsilon_{\text{vô đk}})$ [184]. *Công thức này nói gì:* lấy hiệu giữa “có điều kiện” và “không điều kiện” làm hướng của điều kiện, rồi đi quá lên w lần theo hướng đó.

Giá phải trả

$w > 1$ không phải là “làm đúng hơn” mà là *làm phân phối hẹp lại*: bám prompt tốt hơn, đa dạng kém đi, và ở w lớn thì màu bị cháy.

2 Vấn đề

chạy khuếch tán trên không gian pixel quá đắt.

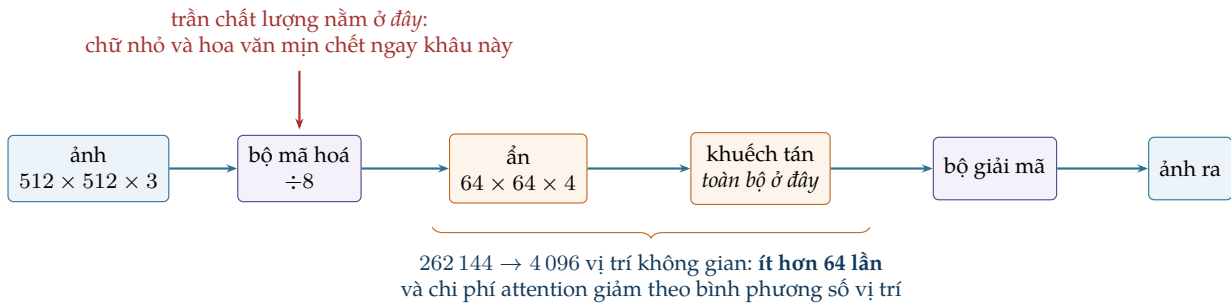
- Giải pháp** nén ảnh bằng một autoencoder rồi khuếch tán trong không gian ẩn [185]. Với hệ số nén 8, ảnh 512×512 còn 64×64 vị trí — ít hơn 64 lần, và vì chi phí *attention* tăng theo bình phương số vị trí, phần đó rẻ đi hàng nghìn lần.
- Hệ quả** đây chính là bước biến diffusion từ nghiên cứu thành sản phẩm chạy được trên GPU tiêu dùng.
- ↔ **ràng buộc bởi B/11** — lợi ích của không gian ẩn tính trực tiếp từ ngân sách bộ nhớ và chi phí attention theo số vị trí.
- Giá phải trả** chất lượng bị chặn trên bởi chất lượng autoencoder; chi tiết rất nhỏ — chữ nhỏ, mắt người ở xa, hoa văn mịn — bị phá ngay ở khâu nén và không mô hình khuếch tán nào cứu lại được.
- 3 Vấn đề** U-Net khó mở rộng quy mô một cách có quy luật.
- Giải pháp** thay bằng *transformer* thao tác trên các *token* của không gian ẩn — *diffusion transformer* [186].
- Hệ quả** quy luật mở rộng quy mô trở nên dự đoán được, giống như ở mô hình ngôn ngữ.
- 4 Vấn đề** prompt bằng chữ không đủ để ràng buộc cấu trúc hình học.
- Giải pháp** *ControlNet*: nhân bản phần mã hoá của mô hình gốc, cho nhánh sao chép nhận điều kiện cấu trúc (cạnh, tư thế người, bản đồ độ sâu), và nối nó vào mô hình gốc qua các tầng tích chập *khởi tạo bằng 0* [187]. *Chi tiết khởi tạo 0 mới là chỗ hay*: ở bước huấn luyện đầu tiên nhánh mới đóng góp đúng bằng không, nên mô hình gốc không bị phá; huấn luyện chỉ *thêm* khả năng chứ không làm hỏng prior đã có.
- Giá phải trả** càng nhiều ràng buộc thì càng ít đa dạng — cùng một quy luật với dẫn hướng ở bước 1.

Bảng 17.4. Ba cách điều khiển đầu ra. Cả ba mua độ trung thành bằng cùng một đồng tiền là độ đa dạng.

Cách	Cơ chế	Ràng buộc được gì	Mất gì
Dẫn hướng (CFG)	Ngoại suy giữa nhánh có và không điều kiện	Ngữ nghĩa, độ bám prompt	Đa dạng; w lớn gây cháy màu
ControlNet	Nhánh sao chép, nối bằng tầng khởi tạo 0	Cấu trúc hình học: cạnh, tư thế, độ sâu	Tự do bố cục; thêm chi phí tính toán
Ảnh sang ảnh / inpainting	Bắt đầu từ ảnh đã pha nhiễu tới mức $t < T$	Bố cục và màu của ảnh gốc	Mức thay đổi bị chặn bởi t đã chọn

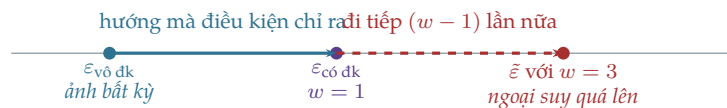
Riêng dòng cuối bảng đáng một câu giải thích, vì nó là hệ quả trực tiếp của Hình 17.8: tham số “độ mạnh” trong ảnh sang ảnh thực chất chỉ là *chọn bắt đầu từ nấc nào*. Bắt đầu ở nấc nhiễu ít thì ảnh gốc còn nguyên và mô hình chỉ tô lại bề mặt; bắt đầu ở nấc nhiễu nhiều thì bố cục cũng bị viết lại. Không có gì bí ẩn ở đây — chỉ là một chỉ số trên trục thời gian.

Hình 17.9 và Hình 17.10 vẽ hai bước quan trọng nhất trong mạch trên.



Hình 17.9. Latent diffusion, và chỗ nó lấy tiền về cũng như chỗ nó trả tiền ra. Toàn bộ phần đất — hàng chục tới hàng trăm lượt chạy mạng — được dời vào không gian ẩn nhỏ hơn 64 lần về số vị trí, và đó là lý do một mô hình sinh ảnh chạy được trên GPU tiêu dùng. Cái giá nằm ở mũi tên đỏ: mọi chi tiết mà bộ mã hoá không giữ được thì không mô hình khuếch tán nào ở phía sau dựng lại được. Vì vậy phép chẩn đoán đầu tiên khi ảnh mất chi tiết là mã hoá rồi giải mã ngay ảnh gốc, bỏ qua khuếch tán.

càng sang phải: bám prompt càng chặt, phân phối càng **hẹp**, màu càng dễ cháy



Hình 17.10. Dẫn hướng không cần bộ phân lớp, nhìn như một phép ngoại suy trên đường thẳng. Hiệu giữa dự đoán có điều kiện và dự đoán không điều kiện chính là “hướng của điều kiện”; hệ số w quyết định đi bao xa theo hướng đó. $w = 1$ là dừng đúng dự đoán có điều kiện; $w > 1$ là đi *quá* lên. Điều quan trọng cần đọc ra từ hình: không có chỗ nào trong phép này làm mô hình “đúng hơn” — nó chỉ đẩy mẫu về vùng điển hình và cắt bớt đuôi, nên ảnh riêng lẻ đẹp hơn còn tập ảnh thì nghèo đi.

E. Giới hạn và trường hợp hỏng

- **Chậm là tính chất, không phải lỗi.** Mọi cách tăng tốc — ít bước, chưng cất bộ lấy mẫu, quỹ đạo thẳng — đều đang xấp xỉ cùng một quỹ đạo, và ở số bước rất nhỏ thì chất lượng và đa dạng đều tụt. Chưng cất bộ lấy mẫu là đúng mẫu hình “dời chi phí sang lúc huấn luyện” của mục 17.2.

↔ cùng nguyên lý với D/20 — cùng phép đổi chi phí suy luận lấy chi phí huấn luyện đã gặp ở chưng cất mô hình.

- **Trần chất lượng nằm ở autoencoder.** Nếu ảnh ra mất chữ nhỏ hoặc chi tiết mịn, đừng đổ cho mô hình khuếch tán trước khi kiểm tra: mã hoá rồi giải mã ngay ảnh gốc, không chạy khuếch tán, và xem chi tiết đó có sống sót không. Phép thử này mất ba mươi giây và loại bỏ được một nửa số giả thuyết.
- **Đếm, chữ, và quan hệ không gian vẫn yếu.** Đây là hệ quả của việc điều kiện đi vào mô hình qua *cross-attention* trên biểu diễn văn bản, chứ không qua một cơ chế suy luận rời rạc nào.
- **Lệch giữa mô hình và bộ lấy mẫu.** Một mô hình huấn luyện với lịch nhiễu này, chạy bằng bộ lấy mẫu giả định lịch khác, cho ra ảnh xấu mà không có thông báo lỗi nào. Đây là lỗi cấu hình phổ biến nhất trong thực hành.

CẠM BẦY

Tăng hệ số dẫn hướng cho tới khi ảnh “đẹp nhất” là một cái bẫy có hậu quả xa. Dẫn hướng mạnh không làm mô hình đúng hơn — nó *cắt bớt đuôi phân phối*, giữ lại phần lõi điển hình nhất. Ảnh riêng lẻ đẹp hơn, nhưng tập ảnh sinh ra nghèo hơn hẳn tập ảnh thật. Nếu bạn định dùng đầu ra làm dữ liệu huấn luyện, đây chính là cơ chế gây thu hẹp

phân phối qua từng vòng lặp: mỗi thể hệ học từ đuôi đã bị cắt của thể hệ trước. Xem [C/17-mo-hinh-sinh/04-low-level-vision](#).

↔ **tiền đề** C/17/04 — muốn hiểu vì sao dữ liệu sinh có thể đầu độc mô hình sau, phải hiểu trước rằng mọi nóm đều khiến ở đây đều cắt đuôi phân phối.

CẦN NHỚ

Diffusion thắng không phải vì ý tưởng đẹp hơn GAN mà vì nó biến bài toán sinh thành một bài *hồi quy*, và hồi quy thì mở rộng quy mô được. Ba con số đáng nhớ khi chỉnh một pipeline: số bước quyết định sai số rời rạc hoá, hệ số dẫn hướng quyết định đánh đổi trung thành ↔ đa dạng, và hệ số nén của autoencoder quyết định trần chi tiết. Ba tham số đó độc lập nhau, và chẩn đoán sai lầm hay gặp nhất là đổ lỗi cho tham số thứ nhất khi thủ phạm là tham số thứ ba. [T2]

F. Kết nối

Mục này là phần chi tiết của họ thứ tư trong [C/17-mo-hinh-sinh/01-bon-ho-mo-hinh-sinh](#); “chưng cất bộ lấy mẫu” là bản sao của mẫu hình ở [C/17-mo-hinh-sinh/02-toi-uu-tren-pixel](#) và ở [D/20-toi-uu-va-nen-mo-hinh](#). Việc dùng đầu ra làm dữ liệu huấn luyện và hậu quả thu hẹp phân phối nằm ở [C/17-mo-hinh-sinh/04-low-level-vision](#); phần kiến trúc transformer thay U-Net nổi về [C/13-kien-truc-backbone/03-vit-va-inductive-bias](#); chi phí bộ nhớ và tính toán của attention theo số vị trí nằm ở [B/11-gioi-han-tinh-toan](#).

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao quá trình thuận *không có tham số nào* lại là điều kiện kỹ thuật khiến diffusion huấn luyện được ở quy mô lớn?
2. Cách nhìn “score function” giải thích được điều gì mà cách nhìn “khử nhiễu từng nấc” không giải thích được?
3. Latent diffusion tiết kiệm tính toán bằng cách nào, và bạn thiết kế phép thử ba mươi giây nào để biết chi tiết bị mất là do khâu nén hay do khâu khuếch tán?
4. Vì sao tăng hệ số dẫn hướng làm ảnh đẹp hơn nhưng làm tập ảnh nghèo hơn — và khi nào điều đó là nguy hiểm chứ không chỉ là đáng tiếc?
5. Tầng tích chập nổi của ControlNet được khởi tạo bằng 0. Bỏ chi tiết đó đi thì hỏng ở đâu, trong bước huấn luyện thứ mấy?
6. Theo Hình 17.8, nếu chỉ được giữ 10 trong 1000 bước, bạn đặt chúng ở đâu và vì sao?

17.4 Low-level vision và trade-off không thể tránh (Low-Level Vision)

PHỤ THUỘC [C/17-mo-hinh-sinh/01-bon-ho-mo-hinh-sinh](#) (vì sao loss trung bình cho ảnh mờ);
[A/02-vat-ly-tao-anh/01-anh-sang-va-nhieu](#) (nhiều cảm biến thật khác nhiều Gauss thế nào);
[B/07-chia-du-lieu-va-danh-gia](#) (chọn chỉ số là chọn kết luận).

TÓM TẮT

Khử nhiễu, siêu phân giải, khử nhoè và HDR đều là bài toán *ill-posed*: nhiều ảnh đầu ra khác nhau cùng khớp với đầu vào, nên vai trò thật của mô hình là *chọn một* chứ không phải *tìm ra đáp án*. Từ đó suy ra một kết quả có chứng minh: không thể đồng thời tối ưu sai lệch theo pixel và độ chân thực thị giác. Vậy nên mọi tranh cãi “phương pháp nào tốt hơn” thật ra là tranh cãi về việc chọn điểm nào trên đường đánh đổi. Đọc xong bạn đọc được một bảng kết quả khôi phục ảnh mà không bị con số dẫn dắt, và biết khi nào một mô hình “đẹp hơn” thực chất đang bịa. Nếu chỉ nhớ một điều: tối ưu PSNR cho ảnh mờ nhưng an toàn, còn tối ưu độ chân thực cho ảnh sắc nét nhưng có thể thêm chi tiết không hề có trong đầu vào. Trong ảnh y tế, giám định hay đo đạc, đó là lỗi đúng sai chứ không phải chuyện thẩm mỹ.

KIẾN THỨC CẦN CÓ TRƯỚC

- **low-level vision** — nhóm bài toán biến ảnh thành ảnh mà không cần hiểu nội dung: khử nhiễu, tăng độ phân giải, khử nhoè, ghép dải sáng động. Đối lập với high-level vision (phân loại, phát hiện) vốn biến ảnh thành nhân.
- **ill-posed (đặt không chỉnh)** — bài toán mà nghiệm không tồn tại duy nhất hoặc không phụ thuộc liên tục vào dữ liệu; ở đây luôn là trường hợp “nhiều nghiệm cùng hợp lệ”, vì phép làm hỏng ảnh đã xoá mất thông tin.
- **toán tử thuận (forward operator)** — mô hình toán của quá trình làm hỏng ảnh — lấy trung bình rồi lấy mẫu thưa (siêu phân giải), chụp với nhân nhoè (khử nhoè), cộng nhiễu (khử nhiễu). Biết nó là biết mình đang phải đảo ngược cái gì.
- **prior (prior)** — giả định về “ảnh thật trông thế nào” dùng để chọn một nghiệm trong vô số nghiệm hợp lệ; ngày nay prior thường nằm ẩn trong trọng số của một mạng đã học.
- **PSNR** — sai lệch theo pixel quy ra decibel, suy ra trực tiếp từ bình phương trung bình; đo “gần theo nghĩa số học”.
- **SSIM** — chỉ số so sánh độ sáng, tương phản và cấu trúc cục bộ giữa hai ảnh; gần thị giác hơn PSNR nhưng vẫn thuộc nhóm đo sai lệch theo cặp.
- **LPIPS** — khoảng cách giữa hai ảnh đo trong không gian đặc trưng của một mạng đã huấn luyện; bám thị giác người tốt hơn, nhưng cần có ảnh gốc để so cặp.
- **FID** — khoảng cách giữa *phân phối* ảnh sinh và *phân phối* ảnh thật. Nó khớp một Gauss trên đặc trưng của một mạng, cần hàng vạn mẫu, và không nói gì về từng ảnh riêng lẻ.

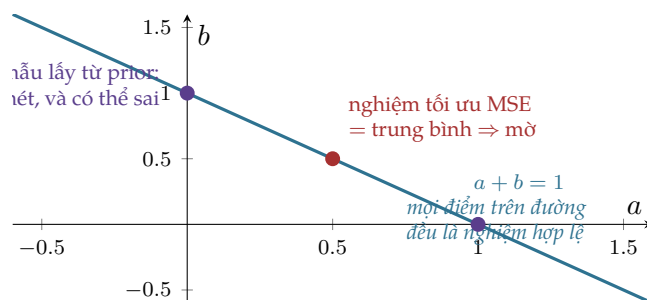
A. Đặt vấn đề

Mọi bài toán trong mục này có cùng một hình dạng: một toán tử thuận đã xoá bớt thông tin, và ta phải đi ngược. Điều quan trọng là thông tin đã mất thì *mất thật* — không thuật toán nào lấy lại được, chỉ có thể đoán.

Ví dụ nhỏ nhất, tính tay được. Lấy tín hiệu một chiều và giảm độ phân giải hai lần bằng cách lấy trung bình từng cặp: hai giá trị a, b cho ra một giá trị $m = (a + b)/2$. Bây giờ đi ngược: biết $m = 0,5$, hãy tìm a và b . Một phương trình, hai ẩn — nghiệm là cả một đường thẳng $\{(a, 1 - a)\}$. Cặp $(0,5; 0,5)$ hợp lệ, mà $(0; 1)$ cũng hợp lệ, và $(-3; 4)$ cũng vậy. Với siêu phân giải $\times 4$ trên ảnh, mỗi pixel thấp cấp ứng với 16 pixel cao cấp: một ràng buộc, mười sáu ẩn. *Mô hình không giải bài toán này; nó chọn một điểm trên không gian nghiệm, và cái quyết định điểm nào là prior.*

→ cùng nguyên lý với C/16/01 — thị giác 3D đối mặt đúng tình thế này: khi ràng buộc hình học không đủ, prior học được chọn nghiệm — và cũng mất luôn khả năng tự phát hiện sai.

Hình 17.11 vẽ đúng không gian nghiệm đó.



Hình 17.11. Bài toán ill-posed nhỏ nhất có thể vẽ được. Giảm độ phân giải hai lần bằng phép lấy trung bình biến (a, b) thành một số $m = 0,5$; đi ngược thì một phương trình phải xác định hai ẩn, nên nghiệm là cả một đường thẳng. Mô hình không “giải” bài toán này — nó *chọn một điểm* trên đường. Chọn trọng tâm thì được điểm số sai lệch tốt nhất và được ảnh mờ; chọn một điểm lấy từ phân phối ảnh thật thì được ảnh sắc nét và chấp nhận rủi ro chọn nhầm. Siêu phân giải $\times 4$ trên ảnh chỉ là hình này với một ràng buộc và mười sáu ẩn.

Điều đó dẫn thẳng tới câu hỏi trung tâm của cả mục: chọn điểm nào? Nếu chọn trung bình của mọi nghiệm hợp lệ thì được điểm số sai lệch tốt nhất, nhưng trung bình của nhiều ảnh sắc nét lệch nhau là một ảnh mờ — đúng cơ chế đã gặp ở mục 17.1.

↔ **hệ quả của C/17/01** — ảnh mờ ở đây và ảnh mờ của VAE là cùng một phép tính: nghiệm tối ưu của bình phương trung bình là kỳ vọng có điều kiện.

Nếu chọn một nghiệm cụ thể lấy từ phân phối ảnh thật thì được ảnh sắc nét trông rất thật, nhưng nó có thể khác hẳn ảnh gốc ở chi tiết. Đây không phải một lựa chọn kỹ thuật mà là một đánh đổi có chứng minh.

Học mục này thì làm được gì mà trước đó không làm được? Đọc một bảng so sánh phương pháp khôi phục ảnh và biết ngay vì sao phương pháp có PSNR cao nhất lại trông tệ nhất; chọn chỉ số đánh giá phù hợp với việc mình định làm; và quyết định có được phép dùng ảnh sinh làm dữ liệu huấn luyện hay không, dựa trên lý do chứ không dựa trên cảm giác.

TRỰC GIÁC

Khôi phục ảnh giống việc đọc một lá thư bị nhoè mực. Chỗ nhoè có thể là chữ “hoa” hoặc chữ “hòa” — nét mực không còn phân biệt được nữa. Bạn có hai cách. Cách thứ nhất: viết chồng cả hai khả năng lên nhau thành một vệt mờ; ai đọc cũng thấy chỗ đó không rõ, nên không ai bị lừa. Cách thứ hai: chọn dứt khoát một chữ và viết thật đẹp; lá thư đọc trôi chảy, trông như nguyên bản — và người đọc không có cách nào biết chữ đó là bạn đoán. Tối ưu PSNR là cách thứ nhất; tối ưu độ chân thực là cách thứ hai. Không có cách thứ ba.

B. Bốn bài toán, một hình dạng

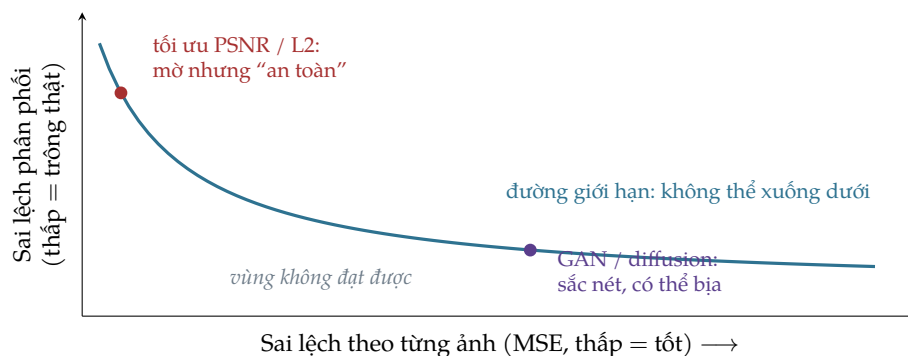
Bảng 17.5. Bốn bài toán low-level vision, đọc theo cái mà toán tử thuận đã xóa đi. Cột cuối cho biết prior phải bù vào chỗ nào — và do đó mô hình sẽ bịa ra loại nội dung gì khi nó sai.

Bài toán	Toán tử thuận	Cái bị xóa	Prior phải bù gì / hỏng khi
Khử nhiễu	Cộng nhiễu (thực tế phụ thuộc cường độ sáng, không phải Gauss đều)	Chi tiết biên độ nhỏ, lẫn vào nhiễu	Kết cấu tự nhiên; hỏng khi mô hình nhiễu lúc huấn luyện khác nhiễu thật
Siêu phân giải	Làm mờ rồi lấy mẫu thưa	Toàn bộ tần số cao trên ngưỡng Nyquist	Chi tiết tần số cao; hỏng ở chữ, khuôn mặt, biển số — những chỗ sai một chút là sai nghĩa
Khử nhoè	Chập với nhân nhoè (do chuyển động hoặc lệch nét)	Các tần số bị nhân nhoè triệt tiêu	Cấu trúc biên; hỏng khi nhân nhoè không đều trên ảnh hoặc không biết trước
HDR	Cắt ngưỡng trên và dưới của dải sáng	Thông tin ở vùng cháy sáng và vùng tối kịt	Nội dung vùng đã cháy; hỏng vì ở đó dữ liệu <i>bằng không</i> , không phải “nhiều”

Dòng cuối đáng chú ý vì nó là trường hợp cực đoan giúp nhìn rõ bản chất: ở vùng cháy sáng, cảm biến đã bão hoà nên mọi giá trị thật đều cho ra cùng một con số. Cái mô hình điền vào đó *hoàn toàn* là prior, không có một chút thông tin đo đạc nào. Ba bài toán còn lại chỉ là phiên bản nhẹ hơn của cùng tình huống.

C. Đánh đổi perceptual-distortion: một kết quả, không phải một kinh nghiệm

Đây là chỗ cần phân biệt rõ địa vị của các phát biểu. Trong mục 17.1, “thể lưỡng nan ba chiều” là một cách gọi tên quan sát thực nghiệm. Ở đây thì khác. Trước hết phải hình thức hoá hai đại lượng: *sai lệch* là kỳ vọng của một khoảng cách tính theo từng ảnh, còn *độ chân thực* là khoảng cách giữa phân phối ảnh khôi phục và phân phối ảnh thật. Với hai định nghĩa đó, người ta *chứng minh được* rằng hàm biểu diễn giới hạn giữa chúng là không tăng [104]. Nói bằng lời: muốn giảm sai lệch xuống dưới một mức nào đó thì buộc phải chấp nhận độ chân thực kém đi. Hình 17.12 vẽ tình thế đó.



Hình 17.12. Đánh đổi perceptual-distortion. Mọi phương pháp khôi phục ảnh đều nằm *trên* đường cong này; tiến bộ thuật toán đẩy đường cong xuống, còn việc chọn hàm mất mát chỉ là chọn *trượt dọc* theo nó. Hệ quả trực tiếp: không xếp hạng được các phương pháp bằng một con số duy nhất, và hai phương pháp ở hai đầu đường cong không so sánh được với nhau nếu chưa nói rõ mục đích sử dụng.

Hệ quả thực hành nghiêm túc hơn về ngoài của nó. Một mô hình siêu phân giải kiểu GAN hay diffusion [188] có thể dựng ra một khuôn mặt sắc nét, hợp lý và *sai người*, hoặc một biển số đọc

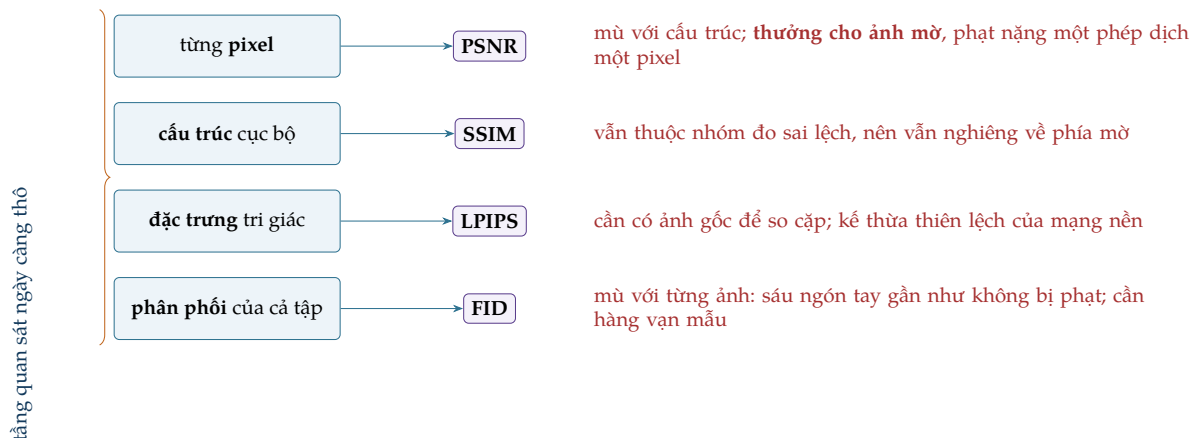
được và sai số. Trong ảnh giải trí, đó là tính năng. Trong ảnh y tế, giám định pháp y, đo đạc từ ảnh vệ tinh hay kiểm tra khuyết tật công nghiệp, đó là lỗi đúng sai. Nó còn nguy hiểm gấp bội vì đầu ra trông *đáng tin hơn* đầu vào.

D. Đánh giá: mỗi chỉ số nhạy với một thứ, và mù với thứ khác

Bảng 17.6. Bốn chỉ số hay dùng. Không chỉ số nào đủ một mình; chọn chỉ số là chọn trước loại lỗi mà bạn sẽ không nhìn thấy.

Chỉ số	Đo gì	Nhạy với	Mù với
PSNR	Bình phương trung bình theo pixel	Lệch sáng, lệch màu toàn cục, sai lệch biên độ lớn	Cấu trúc; nó <i>thường</i> cho ảnh mờ, và phạt nặng một phép dịch một pixel
SSIM	Tương quan cục bộ về sáng, tương phản, cấu trúc [189]	Mất cấu trúc cục bộ, nhoè	Sai lệch ngữ nghĩa; vẫn thuộc nhóm đo sai lệch nên vẫn nghiêng về ảnh mờ
LPIPS	Khoảng cách trong không gian đặc trưng học được [69]	Khác biệt mà mắt người thấy: kết cấu, nét	Cần ảnh gốc; kế thừa thiên lệch của mạng nền
FID	Khoảng cách giữa hai <i>phân phối</i> đặc trưng [70]	Lệch thống kê tổng thể, thu hẹp phân phối	Từng ảnh riêng lẻ; lỗi ngữ nghĩa (sáu ngón tay) gần như không bị phạt

Hình 17.13 xếp bốn chỉ số theo tầng mà chúng quan sát, kèm chỗ mù của từng cái.



Hình 17.13. Bốn chỉ số, bốn *tầng quan sát* khác nhau — và đó là lý do không chỉ số nào thay thế được chỉ số khác. Đi từ trên xuống, thước đo ngày càng ít quan tâm tới từng pixel và ngày càng quan tâm tới thống kê tổng thể; cột phải là cái mà mỗi tầng *không nhìn thấy*. Chọn chỉ số vì thế là chọn trước loại lỗi mà bạn sẽ để lọt, nên báo cáo tối thiểu phải có một chỉ số ở nhóm trên và một ở nhóm dưới.

Hai điểm thực hành ít được nói. Thứ nhất, FID cần cỡ hàng vạn mẫu mới ổn định. Giá trị của nó còn phụ thuộc mạnh vào chi tiết tiền xử lý: thư viện dùng để đổi kích thước ảnh, và chất lượng nén JPEG. Hệ quả là hai con số FID từ hai bài báo thường không so sánh trực tiếp được. Đó là lý do tồn tại các bản cài đặt chuẩn hoá như [GaParmar/clean-fid](#). Thứ hai, và quan trọng hơn: FID nhạy với việc *phân phối bị thu hẹp* — đúng thứ mà mắt người không phát hiện được khi nhìn từng ảnh. Đó là lý do nó vẫn là chỉ số chính dù mọi người đều biết nó có nhiều khuyết điểm.

E. Nhiều thật không phải nhiều Gauss

Một chế độ hỏng rất phổ biến, và hoàn toàn tránh được nếu biết trước. Gần như mọi mô hình khử nhiễu trong tài liệu được huấn luyện bằng cách cộng nhiều Gauss trắng vào ảnh sạch, vì làm vậy thì có dữ liệu vô hạn miễn phí. Nhưng nhiều của cảm biến thật là hỗn hợp của nhiều Poisson phụ thuộc cường độ sáng và nhiễu đọc. Bộ xử lý ảnh trong máy còn biến đổi nó một cách phi tuyến qua khử nhiễu, cân bằng trắng, gamma và nén. Sau chuỗi đó, nhiễu trở nên có tương quan không gian và phụ thuộc màu [29]. Mô hình huấn luyện trên nhiễu Gauss đạt điểm rất cao trên bộ kiểm thử tổng hợp và tụt hẳn trên ảnh chụp thật [30].

Đây là một ca sách giáo khoa của lệch phân phối ([B/08-dich-chuyen-phan-phoi](#)), và cách chữa cũng là cách chữa chuẩn: hoặc thu dữ liệu ghép cặp thật, hoặc mô phỏng đúng chuỗi vật lý bằng cách “gỡ ngược” quá trình xử lý ảnh về miền raw, thêm nhiễu đúng mô hình ở đó, rồi chạy xuôi trở lại. Chi tiết vật lý nằm ở [A/02-vat-ly-tao-anh](#).

→ **ràng buộc bởi A/02** — mô hình nhiễu dùng lúc huấn luyện phải khớp chuỗi vật lý thật của cảm biến, nếu không thì mọi con số đo được đều vô nghĩa.

→ **trường hợp riêng của B/08** — nhiễu Gauss tổng hợp đối lại nhiễu cảm biến thật là một ca sách giáo khoa của lệch phân phối.

F. Dùng ảnh sinh làm dữ liệu huấn luyện

Đây là vấn đề mới của thời đại này, và câu trả lời không phải “có” hay “không” mà là một điều kiện.

- **Có ích khi** bạn dùng mô hình sinh để bù vào các vùng *hiếm* mà bạn biết trước là hiếm: tư thế lạ, thời tiết hiếm, khuyết tật ít gặp. Điều kiện đi kèm là bạn giữ được một tập kiểm thử *thật* hoàn toàn tách biệt để đo. Ở đây ảnh sinh đóng vai một dạng tăng cường dữ liệu có kiểm soát, và tín hiệu đánh giá vẫn đến từ thế giới thật.
- **Tự đầu độc khi** vòng lặp được lặp lại: mô hình sinh dữ liệu, dữ liệu huấn luyện mô hình sau, mô hình sau lại sinh dữ liệu. Cơ chế thu hẹp rất cụ thể. Mọi thao tác làm ảnh “đẹp hơn” — dẫn hướng mạnh, cắt ngưỡng, chọn lọc thủ công — đều cắt bớt đuôi của phân phối. Mỗi vòng lặp vì thế co phương sai lại một chút, và các mode hiếm biến mất hẳn. Hiện tượng này đã được chứng minh bằng thực nghiệm trong nhiều công trình gần đây; mức độ nghiêm trọng phụ thuộc rất mạnh vào một điều kiện: mỗi vòng có giữ lại dữ liệu thật hay không. Điểm này đáng nói rõ, vì các phát biểu kiểu “mô hình sẽ tự sụp đổ” thường bỏ qua nó.

G. Giới hạn và trường hợp hỏng

Giới hạn lớn nhất của cả mảng này là ta không có cách đánh giá tốt. Đường cong ở Hình 17.12 nói rằng một con số không đủ về mặt nguyên tắc, chứ không phải vì ta chưa nghĩ ra chỉ số hay hơn. Hệ quả là mọi bảng xếp hạng khôi phục ảnh đều mang một giả định ngầm về mục đích sử dụng, và giả định đó hầu như không bao giờ được viết ra.

Ba chế độ hỏng còn lại đáng nhớ: mô hình khử nhiễu huấn luyện ở một mức nhiễu cố định hỏng ở mức khác; mô hình siêu phân giải huấn luyện với một toán tử làm mờ cố định (thường là bicubic) hỏng ngay khi ảnh thật bị làm hỏng theo cách khác; và mọi mô hình trong nhóm này đều hỏng *im lặng* — đầu ra luôn là một ảnh trông hợp lý, không bao giờ là một thông báo “tôi không biết”.

CẠM BÃY

Cạm bẫy phổ biến nhất là dùng PSNR để chọn mô hình rồi ngạc nhiên vì mô hình được chọn trông tệ. Đây không phải trực giác mà là hành vi đúng của chỉ số: PSNR suy ra từ bình

phương trung bình, và nghiệm tối ưu của bình phương trung bình là trung bình của mọi ảnh hợp lệ, tức là ảnh mờ. *Chọn chỉ số là chọn trước kết luận*. Cách làm tối thiểu chấp nhận được có hai phần. Thứ nhất, báo cáo một chỉ số sai lệch và một chỉ số chân thực, tức là một ở mỗi đầu đường cong. Thứ hai, với ứng dụng có rủi ro, thêm một phép kiểm tra rằng mô hình *không* thêm nội dung. Phép thử rẻ nhất: cho đầu ra chạy lại qua toán tử thuận rồi so với đầu vào. Chỗ nào lệch là chỗ mô hình đã bịa.

CẦN NHỚ

Mọi bài toán low-level vision đều ill-posed, nên mô hình không “khôi phục” mà *chọn một nghiệm*, và prior là thứ quyết định chọn cái nào. Từ đó suy ra đánh đổi perceptual-distortion, vốn là một kết quả có chứng minh chứ không phải kinh nghiệm: sắc nét hơn đồng nghĩa với bịa nhiều hơn. Câu hỏi đúng khi chọn mô hình không phải “cái nào tốt hơn” mà “sai kiểu nào thì tôi chịu được”. [T1]

H. Kết nối

Cơ chế “loss trung bình cho ảnh mờ” ở đây là đúng cơ chế của VAE trong [C/17-mô-hình-sinh/01-bon-ho-mô-hình-sinh](#); các mô hình khôi phục hiện đại nhất chính là mô hình diffusion có điều kiện của [C/17-mô-hình-sinh/03-diffusion](#), nên hệ số dẫn hướng ở đó cũng chính là nùm vặn trượt dọc đường cong ở đây. Vai trò của prior trong bài toán ill-posed là bản sao chính xác của vai trò prior trong [C/16-thi-giac-3d/01-bon-the-he-3d](#): cả hai lĩnh vực đều đòi ràng buộc cứng lấy tri thức học được, và đều mất khả năng tự phát hiện sai. Mô hình nhiễu thật nằm ở [A/02-vat-ly-tao-anh/01-anh-sang-va-nhieu](#); việc chọn chỉ số và giao thức đo nằm ở [B/07-chia-du-lieu-va-danh-gia](#); hậu quả vận hành của “hông im lặng” nằm ở [E/25-do-tin-cay-va-van-hanh](#).

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một mô hình tối ưu MSE cho ảnh mờ, và vì sao đó là *nghiệm đúng* chứ không phải mô hình kém?
2. Đánh đổi perceptual-distortion khác về địa vị thế nào so với “thể lưỡng nan ba chiều” ở mục 17.1, và vì sao sự khác biệt đó quan trọng khi đọc paper?
3. FID nhạy với thứ gì mà mắt người không để ý, và mù với thứ gì mà mắt người nhận ra ngay?
4. Một mô hình khử nhiễu đạt kết quả xuất sắc trên bộ kiểm thử tổng hợp và kém hẳn trên ảnh điện thoại; cơ chế nào giải thích điều đó và bạn sửa bằng cách nào?
5. Trong trường hợp nào thì việc dùng ảnh sinh làm dữ liệu huấn luyện là an toàn, và điều kiện nào bị vi phạm thì nó trở thành tự đầu độc?
6. Nếu phải giao một mô hình siêu phân giải cho một hệ thống đọc biển số, bạn chọn điểm nào trên đường cong ở Hình 17.12, và bạn thêm phép kiểm tra nào để bảo vệ mình?

17.5 Thực hành

Chương này có một đặc điểm hiếm: bài thực hành nhỏ nhất của nó — vẽ đường cong chất lượng theo số bước lấy mẫu — dạy được nhiều hơn cả chục trang lý thuyết, vì nó biến một đánh đổi trừu tượng thành một hình vẽ bạn tự tay tạo ra. Một lưu ý về bán rã trước khi vào danh sách: danh sách dưới đây chốt tại **tháng 9/2026**; tên các mô hình dẫn đầu sẽ cũ trong 6–12 tháng, trong

khi phần hạ tầng — [diffusers](#), [k-diffusion](#), [clean-fid](#) — thì bền hơn nhiều — hãy đầu tư kỹ năng vào nhóm sau.

Repo và tài nguyên. Hạ tầng: [huggingface/diffusers](#), [comfyanonymous/ComfyUI](#), [crowsonkb/k-diffusion](#). Ảnh: FLUX.2 (dẫn về ảnh chân thực; bản [klein](#) 4B chạy được trong khoảng 8 GB VRAM), Stable Diffusion 3.5 (hệ sinh thái tinh chỉnh sâu nhất), Qwen-Image (dựng chữ trong ảnh), Z-Image Turbo (nhanh). Video: Wan 2.2 (Apache 2.0, lựa chọn an toàn nhất về giấy phép, bản 5B từ 8 GB), LTX-2.3 (sinh âm thanh đồng bộ trong cùng lượt khuếch tán), HunyuanVideo 1.5. Học nguyên lý: [lucidrains/denoising-diffusion-pytorch](#) — đọc mã nguồn này một lần đáng giá hơn đọc ba bài báo. Đánh giá: [GaParmar/clean-fid](#), [richzhang/PerceptualSimilarity](#).

Câu hỏi kiểm tra hiểu. Vì sao mô hình tối ưu MSE cho ảnh mờ, và perceptual loss “sửa” điều đó bằng cách đánh đổi gì? FID nhạy với thứ gì mà mắt người không quan tâm, và ngược lại? Latent diffusion tiết kiệm tính toán bằng cách nào, và mất gì ở đúng chỗ đó?

Mini project (vài giờ đến hai ngày). Cài một DDPM tối giản khoảng 200 dòng, huấn luyện trên MNIST hoặc CIFAR-10. Cố định mô hình, chỉ đổi bộ lấy mẫu và số bước (10, 25, 50, 250), rồi vẽ FID theo số bước và theo thời gian tường. Đường cong đó là bài học đánh đổi rõ nhất của cả mảng diffusion — và nó cũng cho bạn thấy tận mắt rằng chất lượng bão hoà trước khi thời gian bão hoà.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Độ hợp lý	Xác suất mà mô hình gán cho dữ liệu đã quan sát. Chỉ họ autoregressive tính được nó chính xác; GAN không có nó, và đó là lý do GAN mất công cụ chẩn đoán tự nhiên.
•• ELBO	Cận dưới của log độ hợp lý, thứ mà VAE thực sự tối ưu. Phần tái tạo của nó là bình phương trung bình theo pixel, nên ảnh mờ là hệ quả trực tiếp của chính công thức này.
••• Mode collapse	Mô hình chỉ sinh ra một phần nhỏ của phân phối trong khi vẫn đạt điểm cao theo tiêu chí huấn luyện. Nhìn mẫu chọn lọc không phát hiện được, vì các mẫu vẫn đẹp — chúng chỉ giống nhau.
••• Score matching	Huấn luyện bằng cách khớp gradient của log mật độ, $\nabla_x \log p(x)$, thay vì khớp mật độ. Đây là nền toán của diffusion: mô hình học một trường vector, và lấy mẫu là thả một điểm nhiều vào trường đó rồi để nó trôi.
••• Vì sao mô hình tối ưu MSE luôn cho ảnh mờ?	Vì nghiệm cực tiểu của bình phương trung bình là kỳ vọng có điều kiện, tức trung bình của mọi ảnh hợp lệ. Các ảnh đó lệch nhau vài pixel ở biên, nên trung bình của chúng nhoè.
••• Vì sao diffusion huấn luyện ổn định hơn GAN?	Vì hàm mất mát của nó là bình phương trung bình trên nhiều, tức một bài hồi quy tầm thường. GAN đi tìm điểm cân bằng của một trò chơi hai người, nên không có “hội tụ” theo nghĩa thông thường.
•• Ma trận Gram	Tích trong giữa các kênh của một bản đồ đặc trưng, cộng dồn trên mọi vị trí không gian. Phép cộng dồn xoá hết thông tin vị trí, nên nó giữ được kết cấu và bảng màu nhưng không giữ được bố cục.
•• AdaIN	Chuẩn hoá đặc trưng của ảnh nội dung về trung bình–độ lệch chuẩn 0–1 theo từng kênh, rồi gán lại thống kê của ảnh phong cách. Cho phép một mạng duy nhất làm phong cách tùy ý, đổi lại chất lượng thấp hơn bản tối ưu.
•• Vì sao tối ưu trên pixel lại tìm ra mẫu đối kháng?	Vì gradient luôn chỉ về hướng dốc nhất của hàm chấm điểm, mà mạng có vô số hướng nó nhạy còn mắt người thì không. Tối ưu đi thẳng vào đúng những hướng bất đồng đó.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
••• Quá trình thuận	Phép pha nhiễu Gauss dần dần, không có tham số nào để học. Nhờ tính cộng của Gauss, ta nhảy thẳng từ ảnh gốc tới bước bất kỳ — đó là lý do kỹ thuật khiến diffusion huấn luyện được ở quy mô lớn.
•• Latent diffusion	Nén ảnh bằng một autoencoder rồi mới khuếch tán trong không gian ẩn. Với hệ số nén 8, ảnh 512×512 còn 64×64 vị trí — ít hơn 64 lần. Trần chất lượng vì thế nằm ở autoencoder.
•• Classifier-free guidance	Ngoại suy giữa dự đoán có điều kiện và không điều kiện: $\tilde{\epsilon} = \epsilon_{\text{vô dk}} + w(\epsilon_{\text{có dk}} - \epsilon_{\text{vô dk}})$. Hệ số lớn bám câu mô tả chặt hơn, nhưng cắt bớt đuôi phân phối nên đa dạng giảm.
•• ControlNet	Nhánh sao chép phần mã hoá của mô hình gốc, nhận điều kiện cấu trúc (cạnh, tư thế, độ sâu) và nối vào qua các tầng khởi tạo bằng 0. Chi tiết khởi tạo 0 mới là phần đáng nhớ: nó khiến huấn luyện chỉ thêm khả năng chứ không phá prior đã có.
•• Vì sao “độ mạnh” chỉ là một chỉ số thời gian?	Vì nó quyết định bắt đầu khử nhiễu từ nấc nào. Bắt đầu ở nấc nhiễu ít thì mô hình chỉ tô lại bề mặt; bắt đầu ở nấc nhiễu nhiều thì bố cục cũng bị viết lại.
••• Ill-posed	Bài toán có nhiều nghiệm cùng hợp lệ vì toán tử thuận đã xoá thông tin. Siêu phân giải $\times 4$ là một ràng buộc cho mười sáu lần; mô hình không giải mà <i>chọn</i> một nghiệm, và prior quyết định chọn cái nào.
••• Vì sao không thể vừa sắc nét vừa đúng từng pixel?	Đó là đánh đổi perceptual-distortion: giảm sai lệch theo từng ảnh xuống dưới một mức thì buộc phải chấp nhận độ chân thực kém đi. Đây là kết quả <i>có chứng minh</i> dưới một cách hình thức hoá cụ thể, không phải lời khuyên rút ra từ kinh nghiệm.
•• FID	Khoảng cách giữa phân phối ảnh sinh và phân phối ảnh thật, ước lượng bằng cách khớp Gauss trên đặc trưng của một mạng. Nhảy với việc phân phối bị thu hẹp, mù với lỗi ngữ nghĩa của từng ảnh, và cần hàng vạn mẫu.
• Bản GAN sắc nét hơn hẳn với mắt người nhưng PSNR thua bản mờ — vì sao?	Vì hai bên nằm ở hai đầu đường cong đánh đổi. Bản mờ là trung bình của mọi ảnh hợp lệ nên tối ưu sai lệch theo pixel; bản GAN chọn dứt khoát một nghiệm nên sắc nét mà lệch từng pixel. Muốn thấy khác biệt thì phải đo thêm bằng LPIPS hoặc FID.
• Ảnh sinh mất chữ nhỏ và hoa văn mịn — chẩn đoán đầu tiên?	Mã hoá rồi giải mã ngay ảnh gốc, bỏ qua khuếch tán. Nếu chi tiết đã chết ở đó thì thủ phạm là autoencoder, không phải mô hình khuếch tán hay số bước lấy mẫu.
• Khử nhiễu rất tốt trên bộ tổng hợp, kém hẳn trên ảnh điện thoại — nguyên nhân?	Nó được huấn luyện trên nhiễu Gauss trắng, còn nhiễu thật là Poisson phụ thuộc cường độ sáng cộng nhiễu đọc, rồi bị bộ xử lý ảnh làm cho có tương quan không gian. Cách chữa là mô phỏng đúng chuỗi vật lý ở miền raw.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
diffusers, k-diffusion	Tách rời mô hình khỏi bộ lấy mẫu — đúng phân tách của mục 17.3, và là lý do đối sampler không cần huấn luyện lại	Hạ tầng bền; cũng là nơi lỗi lệch lịch nhiễu hay xảy ra
Latent diffusion (dòng Stable Diffusion, FLUX)	Khuếch tán trong không gian ẩn để giảm số vị trí không gian; kiến trúc transformer thay U-Net	Trần chi tiết vẫn nằm ở autoencoder — kiểm tra bằng phép mã hoá rồi giải mã
ControlNet	Ràng buộc cấu trúc bằng nhánh sao chép nối qua tầng khởi tạo 0, để không phá prior đã học	Chi tiết khởi tạo 0 là phần đáng học nhất, không phải phần kiến trúc

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Classifier-free guidance	Ngoại suy giữa nhánh có và không điều kiện; có mặt trong gần như mọi mô hình sinh ảnh hiện nay	Cũng là cơ chế cắt đuôi phân phối được nói ở mục 17.4
AdaIN	Chuẩn hoá rồi gán lại thống kê theo kênh — vẫn được dùng lại trong nhiều mạng sinh và mạng điều kiện hoá	Ý tưởng sống lâu hơn ứng dụng style transfer ban đầu của nó
clean-fid, LPIPS	Đánh giá ở hai tầng khác nhau (phân phối và theo cặp), đúng như bảng chỉ số của mục 17.4	Hạ tầng; tồn tại chính vì FID nhạy với chi tiết tiền xử lý

MINI PROJECT SLAM · Khử nhiễu trước frontend: “đẹp hơn” có nghĩa là “bám tốt hơn” không? · 1–2 ngày

Mục tiêu	nói thẳng: đây là liên hệ <i>gián tiếp</i> . Chương này về mô hình sinh, còn chỗ nó thật sự chạm vào visual SLAM là khâu tiền xử lý ảnh thiếu sáng đặt trước frontend. Mục tiêu là kiểm chứng bằng số một giả định mà hầu như ai cũng mặc nhiên tin: ảnh sau khử nhiễu trông sạch hơn thì frontend bám đặc trưng tốt hơn.
Dữ liệu	một chuỗi EuRoC thiếu sáng và nhiễu nhoè (V2_03 hoặc MH_04), cộng ba bản sao được làm tối và thêm nhiễu có kiểm soát để tạo một thang bốn mức nhiễu. Giữ nguyên chân trị (<i>ground truth</i>) của chuỗi gốc cho cả bốn mức.
Các bước	(1) chạy ORB-SLAM3 trên từng mức nhiễu <i>chưa</i> khử để lấy đường cơ sở: ATE, và số đặc trưng bám được trung bình mỗi khung hình; (2) chèn một bộ khử nhiễu nhẹ trước frontend — BM3D hoặc một CNN khử nhiễu nhỏ chạy đủ nhanh; (3) chạy lại với cấu hình y hệt, mỗi cấu hình ít nhất năm lần vì harness không tắt định; (4) với mỗi mức nhiễu ghi ba đại lượng: PSNR (hoặc LPIPS) của ảnh sau khử nhiễu, số đặc trưng bám được, và ATE trung bình cùng độ lệch chuẩn.
Xong khi	bạn có một bảng “mức nhiễu × {PSNR, số đặc trưng bám được, ATE}” và trả lời được bằng số: mức nhiễu nào thì khử nhiễu cải thiện ATE vượt độ lệch chuẩn của harness, mức nào thì không, và có tồn tại một mức mà PSNR <i>tăng</i> trong khi số đặc trưng bám được <i>giảm</i> hay không. Nếu có, hãy chỉ đúng chỗ đó — đó là đánh đổi perceptual-distortion hiện ra ở dạng cụ thể nhất.
Nối với	C/17/04 cho cơ chế: làm mượt để tối ưu sai lệch theo pixel cũng chính là xoá đi các góc mà detector cần; A/02 cho mô hình nhiễu cảm biến thật; E/24 và B/07 cho cách quyết định một chênh lệch có thật hay chỉ là nhiễu của harness. Dùng lại đúng harness lặp nhiều lần mà mini project chương 16 đã buộc bạn dựng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì nghiệm cực tiểu của bình phương trung bình là kỳ vọng có điều kiện — trung bình của *mọi* ảnh sắc nét khớp với đầu vào. Các ảnh đó lệch nhau vài pixel ở biên, nên trung bình của chúng là một ảnh mờ. Đây là nghiệm đúng của bài toán đã đặt ra, không phải dấu hiệu huấn luyện chưa đủ.
- A2. Vì cả hai chỉ là hai hướng đo khoảng cách giữa phân phối mô hình và phân phối dữ liệu. Hướng thuận phạt việc bỏ sót mode nên mô hình trải rộng phủ cả hai mode và đặt khối lượng vào vùng trống — ra ảnh mờ. Hướng ngược phạt việc đặt khối lượng ở nơi dữ liệu hiếm nên mô hình co về một mode — ra mode collapse. Một lựa chọn, hai triệu chứng.
- A3. Vì hàm mất mát của nó là bình phương trung bình trên nhiễu, tức một bài hồi quy tầm thường, trong khi GAN tối ưu điểm cân bằng của một trò chơi hai người. Hồi quy thì tăng quy mô là tốt lên đều đặn; trò chơi thì không có “hội tụ” theo nghĩa thông thường. Diffusion mua sự ổn định đó bằng số bước lấy mẫu.
- A4. Vì mọi nóm làm ảnh đẹp hơn — dẫn hướng mạnh, cắt ngưỡng, chọn lọc thủ công — đều hoạt động bằng cách cắt bớt đuôi phân phối. Từng ảnh đẹp hơn nhưng tập ảnh nghèo hơn, và nếu vòng lặp lặp lại thì mỗi thể hệ học từ đuôi đã bị cắt của thể hệ trước, phương sai co dần và các mode hiếm biến mất.

NÓI LẠI THẬT ĐƠN GIẢN

Tưởng tượng một cuộc thi vẽ, và ba giám khảo chấm cùng một bài. Người thứ nhất đặt bài thi chồng lên bản mẫu rồi đo xem mỗi nét lệch bao nhiêu milimét. Người thứ hai lùi ra xa ba mét, nheo mắt, và hỏi “nhìn tổng thể nó có ra dáng không”. Người thứ ba trộn bài đó vào một chồng năm trăm bài thật rồi xem có rút nó ra được không.

Ba cách chấm đó cho ba thí sinh thủ khoa hoàn toàn khác nhau, và đây mới là điều đáng nhớ: thí sinh sẽ luyện đúng theo cách mình bị chấm. Nếu bị chấm bằng thước, cách an toàn nhất là vẽ nhoè — một nét nhoè nằm giữa mọi khả năng thì không bị trừ nhiều ở bất cứ chỗ nào. Nếu bị chấm bằng mắt từ ba mét, cách thắng là vẽ mạnh tay và tự bịa thêm chi tiết cho sống động, kể cả những chi tiết không có trong bản mẫu.

Toàn bộ chương này nói rằng máy cũng vậy. Người ta không dạy máy “vẽ đẹp hơn”; người ta chỉ chọn một cách chấm, rồi máy tự tìm cách thắng cách chấm đó. Ảnh mờ không phải vì máy kém mà vì nó đang bị chấm bằng thước. Ảnh sắc nét nhưng bịa chi tiết không phải vì máy nói dối mà vì nó đang bị chấm bằng mắt.

Cái giá là bạn không thể chọn cả hai. Muốn không bao giờ bịa thì phải chấp nhận nhoè; muốn sắc nét thì phải chấp nhận rằng một phần những gì bạn thấy là do máy tự thêm vào. Trong ảnh trang trí, đó là ưu điểm. Trong ảnh y tế hay ảnh dùng làm bằng chứng, đó là sai lầm nghiêm trọng.

Chỗ không rút gọn được: câu “không thể chọn cả hai” ở trên. Nói bằng lời thường thì nó nghe như một lời khuyên rút ra từ kinh nghiệm, nhưng thực chất nó là một kết quả *chứng minh được* dưới một cách phát biểu chặt chẽ. Sự khác biệt đó rất quan trọng khi đọc một bài báo tuyên bố “chúng tôi vừa chính xác hơn vừa sắc nét hơn”: với một lời khuyên kinh nghiệm thì tuyên bố ấy có thể đúng, với một định lý thì nó chỉ có thể đúng nếu bài báo đang đo hai thứ khác nhau.

Một câu: Mọi mô hình sinh đều là một thí sinh luyện theo cách chấm điểm, nên đổi thước đo là đổi kiểu sai lầm, không phải đổi mức thông minh.

Kết nối

[C/18-pretraining-va-foundation](#) tiếp tục câu chuyện “tín hiệu giám sát đến từ đâu” mà chương này chạm vào khi nói về dữ liệu sinh; [D/20-toi-uu-va-nen-mo-hinh](#) trình bày đầy đủ mẫu hình dời chi phí sang lúc huấn luyện; [E/25-do-tin-cay-va-van-hanh](#) biến “hông im lặng” của các mô hình khôi phục thành thiết kế phòng thủ cụ thể; và [C/16-thi-giac-3d](#) là chương song song, nơi cùng nguyên lý “prior chọn nghiệm” xuất hiện trong bài toán hình học.

Tổng kết chương

Ta đã đi qua bốn tầng của một câu hỏi duy nhất — khi nhiều đầu ra cùng hợp lệ thì cái gì chọn? — và câu trả lời lặp đi lặp lại là: hàm mất mát chọn, chứ không phải kiến trúc chọn. Bốn họ mô hình sinh khác nhau vì bốn cách viết “giống dữ liệu thật” thành một con số, và kiểu thất bại của mỗi họ suy ra được từ chính con số đó. Tối ưu trên pixel cho thấy điều này ở dạng thuần khiết nhất: cùng một vòng lặp, đổi hàm mục tiêu là được một kỹ thuật khác hẳn, từ tác phẩm nghệ thuật tới mẫu đối kháng. Diffusion thắng không nhờ ý tưởng đẹp hơn mà nhờ biến bài toán sinh thành một bài hồi quy có thể mở rộng quy mô. Và low-level vision cho thấy khi hậu quả là thật, cùng một đánh đổi đó trở thành câu hỏi “tôi chịu được kiểu sai nào”.

Ba chỗ còn bỏ ngỏ, đáng theo dõi: thứ nhất, việc rút số bước lấy mẫu xuống một chữ số mà không mất đa dạng — flow matching và chưng cất bộ lấy mẫu đang tiến nhanh nhưng chưa có lời giải sạch; thứ hai, đánh giá mô hình sinh, nơi mọi chỉ số hiện có đều mù ở một chỗ quan trọng và chưa ai có phương án thay thế thuyết phục; thứ ba, hệ quả dài hạn của việc dữ liệu sinh tràn vào tập huấn luyện của thể hệ mô hình sau — cơ chế thu hẹp phân phối đã rõ, nhưng độ nghiêm trọng trong thực tế còn phụ thuộc nhiều điều kiện chưa được đo đủ.

I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Câu hỏi chung của cả chương là “khi nhiều đầu ra cùng hợp lệ thì cái gì quyết định chọn cái nào”. Với VAE, GAN, diffusion và một mạng khôi phục ảnh huấn luyện bằng MSE, mỗi cái trả lời bằng cơ chế nào, và hai cái nào trong bốn thực ra đang đưa cùng một câu trả lời?
2. Mọi cơ chế làm ảnh đẹp hơn đều mua độ trung thành bằng độ đa dạng — hệ số dẫn hướng, loss cảm nhận và ma trận Gram đều thuộc họ đó. Ba cái này thu hẹp phân phối ở ba chỗ khác nhau nào trong pipeline?
3. Đánh đổi perceptual–distortion ở mục 4 và thể lưỡng nan ba chiều ở mục 1 nghe rất giống nhau. Chúng khác nhau ở địa vị lý thuyết thế nào, và vì sao gộp nhầm hai thứ đó lại dẫn tới kết luận sai khi đọc paper?
4. Mẫu đối kháng ở mục 2 và ảnh có FID rất tốt nhưng sai chi tiết ở mục 4 đều là “tối ưu đúng thước đo, sai mục đích”. Điểm chung về cơ chế là gì, và cùng một cách phòng vệ có dùng được cho cả hai không?
5. Bạn dùng đầu ra của một mô hình sinh làm dữ liệu huấn luyện cho mô hình khác. Điều kiện nào phải giữ để việc đó an toàn, và triệu chứng đầu tiên khi điều kiện bị vi phạm hiện ra ở đâu?
6. Latent diffusion, AdaIN và mẫu hình “dời chi phí sang lúc huấn luyện” đều đổi chất lượng lấy tốc độ. Với ràng buộc thời gian thực trên thiết bị nhúng, cái nào bị loại đầu tiên và vì sao?
7. Cả bốn mục đều cảnh báo về việc tin vào thứ nhìn thấy được — ảnh mẫu đẹp, bản đồ attention, ảnh trực quan hoá lớp, ảnh khử nhiễu sắc nét. Có quy tắc chung nào rút ra được không, và nó buộc bạn đo thêm cái gì?

Pretraining, transfer và mô hình nền tảng (Pretraining, Transfer and Foundation Models)

PHỤ THUỘC B/06-chat-luong-du-lieu và B/07-chia-du-lieu-va-danh-gia (chi phí nhân và cách đánh giá không tự lừa mình); B/09-thiet-ke-ham-mat-mat (*contrastive loss*, *cross-entropy*); C/13-kien-truc-backbone (ViT và *inductive bias* — chương này giải thích vì sao ViT chỉ toả sáng khi có tiền huấn luyện quy mô lớn).

TÓM TẮT

Chương này trả lời một câu hỏi: nhân thì đắt, dữ liệu không nhân thì vô hạn — vậy tín hiệu giám sát miễn phí lấy từ đâu, và mua bao nhiêu thì đủ. Mục 18.1 đi qua bốn nguồn giám sát rẻ dần và nói mỗi nguồn hỏng ở đâu; mục 18.2 đưa ra mặt định lượng: *scaling laws*, bài học Chinchilla, nhiệm vụ kiểm tra, và LoRA. Đọc xong bạn chọn được *backbone*, chọn được chế độ thích nghi, và ước lượng được hoá đơn compute — thay vì hỏi người khác hoặc thử mù. Nếu chỉ nhớ một điều: mô hình nền tảng đã biến bạn từ người huấn luyện thành người tích hợp, nên câu hỏi trung tâm đổi từ “mô hình này có đúng không” sang “tôi có biết nó giả định gì và hỏng ở đâu không”.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model)** — một hàm có tham số, ánh xạ đầu vào (ảnh) sang đầu ra (nhân, hộp, vector đặc trưng). “Huấn luyện” nghĩa là tìm bộ tham số làm hàm mất mát nhỏ nhất trên dữ liệu.
- **tiền huấn luyện (pretraining)** — huấn luyện một mô hình trên một tập dữ liệu lớn, tổng quát, *trước* khi biết bài toán cuối là gì. Mục đích là để mô hình học sẵn những đặc trưng chung của ảnh, thứ mà dữ liệu ít ỏi của bài toán cuối không đủ để học lại.
- **học chuyển giao (transfer learning)** — lấy mô hình đã tiền huấn luyện rồi dùng lại cho bài toán khác, hoặc bằng cách đóng băng nó, hoặc bằng cách huấn luyện tiếp trên dữ liệu của mình.
- **backbone** — phần thân của mạng chịu trách nhiệm biến ảnh thành vector đặc trưng; phần “đầu” (head) gắn sau nó mới sinh ra đầu ra cụ thể của từng bài toán. Tách hai phần ra là điều kiện để tái sử dụng.
- **học tự giám sát (self-supervised learning)** — huấn luyện mà không dùng nhãn người: đáp án của bài toán phụ được suy ra từ chính dữ liệu, chẳng hạn “hai lát cắt của cùng bức ảnh phải cho vector giống nhau”.
- **mô hình nền tảng (foundation model)** — một mô hình lớn được tiền huấn luyện trên dữ liệu quy mô rất lớn, đủ tổng quát để dùng lại cho nhiều bài toán khác nhau mà không huấn luyện lại từ đầu.
- **VLM (vision-language model)** — mô hình nhận cả ảnh lẫn văn bản làm đầu vào và sinh ra văn bản, thường bằng cách nối một bộ mã hoá ảnh vào một mô hình ngôn ngữ lớn.
- **ngân sách compute** — tổng số phép tính (FLOPs) bạn mua được cho một lần huấn luyện. Vì nó cố định, mọi quyết định về kích thước mô hình và lượng dữ liệu đều là quyết định phân bổ trong cùng một ngân sách.

CÂU HỎI MỞ ĐẦU

- Q1. Với 500 ảnh nội soi, *linear probe* trên một *backbone* khổng lồ hay tinh chỉnh toàn bộ một ResNet nhỏ sẽ thắng? Và vì sao câu trả lời đảo chiều ở 50 000 ảnh?
- Q2. Nếu chỉ cần dạy mô hình rằng hai lát cắt của cùng một bức ảnh phải cho vector giống nhau, vì sao nó không học ngay nghiệm rẻ nhất là xuất một hằng số cho mọi ảnh?
- Q3. Vì sao một dòng chú thích cầu thả trên internet lại là tín hiệu giám sát ngữ nghĩa mạnh hơn một nhãn ImageNet do người gán cẩn thận?
- Q4. Gopher lớn gấp bốn lần Chinchilla và tốn cùng số FLOPs để huấn luyện, nhưng thua — cả ngành đã tối ưu nhằm đạt lượng nào suốt mấy năm?
- Q5. Vì sao một mô hình chưa bao giờ nhìn thấy ảnh vệ tinh lại cho đặc trưng ảnh vệ tinh tốt hơn một mạng huấn luyện riêng trên ảnh vệ tinh?

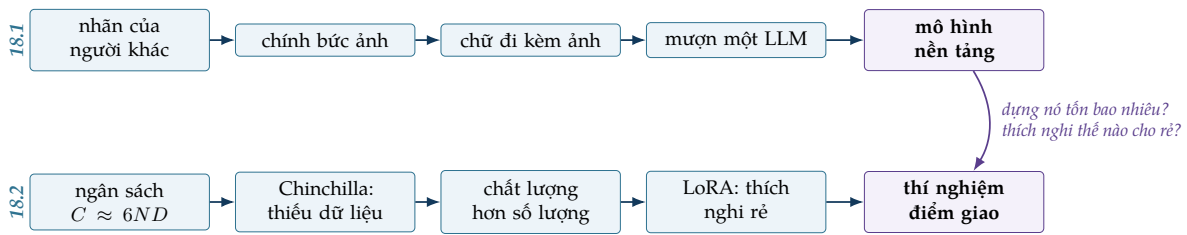
Câu hỏi dẫn dắt của cả chương chỉ có một dòng: *nhãn thì đắt, dữ liệu không nhãn thì vô hạn — tín hiệu giám sát miễn phí có thể lấy từ đâu?* Mọi kỹ thuật trong chương, từ học chuyển giao những năm 2014 tới VLM ngày nay, là một câu trả lời cho đúng câu hỏi đó, và các câu trả lời xếp thành một chuỗi ngày càng táo bạo: dùng nhãn của người khác, rồi dùng chính bức ảnh, rồi dùng dòng chữ đi kèm bức ảnh, rồi mượn luôn khả năng suy luận của một mô hình ngôn ngữ.

Chuỗi ấy là một mạch *định tính*: nó nói cho bạn biết có những nguồn giám sát nào và mỗi nguồn giả định gì. Nhưng nó không trả lời được câu hỏi mà bạn thực sự phải quyết định khi ngồi trước một cụm GPU có hạn: *tiêu ngân sách vào đâu, và bao nhiêu thì đủ?* Đó là một mạch khác, mang tính *định lượng* và kinh tế, với từ vựng riêng — *scaling laws*, tỉ lệ dữ liệu trên tham số, chi phí thích nghi. Hai mạch dùng chung đối tượng nhưng khác hẳn nhau về loại lập luận, và trộn chúng vào một mạch duy nhất là cách nhanh nhất để cả hai đều mờ đi. Vì vậy chương chia làm hai mục:

- **18.1 — Nguồn giám sát ngày càng rẻ.** Chuỗi bốn nguồn, dừng ở từng nguồn để hỏi nó chống lại nghiệm tầm thường bằng cách nào, nó ép vào biểu diễn những bất biến gì, và nó hỏng ở đâu.
- **18.2 — Kinh tế học của pretraining.** Lấy kết quả của mục trước làm cho sẵn rồi hỏi tiếp: với chừng này compute thì mô hình nên lớn cỡ nào, dữ liệu nên nhiều cỡ nào, và khi nào tinh chỉnh đáng giá hơn đóng băng.

Thứ tự đọc là bắt buộc theo chiều này: mục 18.2 nhắc tới *linear probe*, *fine-tune* và mô hình nền tảng như những khái niệm đã biết. Có một điểm nối giữa hai mục đáng nói trước: cả hai kết thúc ở cùng một kết luận về phương pháp luận. Mục 18.1 kết luận rằng bạn không thể biết một biểu diễn tự giám sát đã học gì nếu chỉ nhìn *benchmark* chung; mục 18.2 kết luận rằng bạn không thể biết chế độ thích nghi nào đúng nếu chỉ nghe lời khuyên chung. Cả hai đẩy bạn về cùng một hành động: đo trên chính dữ liệu của mình, với một tập đánh giá mà mô hình nền tảng chắc chắn chưa từng thấy.

Hình 18.1 vẽ chỗ nối giữa hai mục: mục 18.1 sản xuất ra một mô hình nền tảng, còn mục 18.2 hỏi cái mô hình ấy tốn bao nhiêu và thích nghi thế nào cho rẻ.



Hàng trên: giám sát rẻ dần, tín hiệu gián tiếp dần. Hàng dưới: cùng một ngân sách nhìn từ phía chi phí.

Hình 18.1. Bản đồ chương 18. Hai mục không nối tiếp nhau về thời gian mà nối nhau ở *đối tượng*: mục 18.1 kết thúc ở mô hình nền tảng, và mục 18.2 bắt đầu bằng câu hỏi mô hình đó tốn bao nhiêu. Mọi con số chung ở hàng dưới đều kết thúc ở cùng một chỗ — một thí nghiệm bạn phải tự chạy trên dữ liệu của mình.

Bản đồ chương 18 — Pretraining, transfer và mô hình nền tảng

Chương này trả lời một câu hỏi duy nhất — *tín hiệu giám sát miễn phí lấy từ đâu, và mua bao nhiêu thì đủ* — nhưng câu hỏi đó có hai nửa dùng hai loại lập luận khác nhau, nên chương chia làm hai mục và thứ tự đọc là bắt buộc.

18.1 — Nguồn giám sát ngày càng rẻ [T3] (cốt lõi). Chuỗi bốn nguồn giám sát xếp theo chi phí giảm dần: nhân của người khác (transfer learning và bảng bốn ô quyết định), chính bức ảnh (ba triết lý tự giám sát và ba cách chống sập khác nhau), văn bản đi kèm ảnh (CLIP và ngữ nghĩa mở), và cuối cùng là ghép thị giác vào một LLM (VLM). Đây là mục phải nắm chắc: nó là tiền đề trực tiếp của open-vocabulary detection ở chương 14 và của mọi quyết định chọn backbone về sau. Điểm cần mang theo là cột “hồng khi nào” của bảng ba họ tự giám sát, và ý thức rằng mỗi bất biến bạn ép vào augmentation là một loại thông tin bạn cố tình vứt đi.

18.2 — Kinh tế học của pretraining [T2][T3] (cốt lõi phần nguyên lý, tham khảo phần con số). Mặt định lượng: scaling laws cho phép ngoại suy thay vì thử mù; bài học Chinchilla cho thấy cả ngành đã thổi tham số mà quên dữ liệu; chất lượng corpus thắng số lượng; contamination phá hủy hiệu lực của mọi benchmark công khai; PEFT/LoRA đổi hoàn toàn kinh tế học của việc thích nghi; và quyết định cuối cùng luôn là thí nghiệm điểm giao trên dữ liệu của chính bạn. Nguyên lý ở đây thuộc tầng bền ([T2]); các hằng số cụ thể — tỉ lệ 20 token trên tham số, hạng LoRA hợp lý — thì không, và nên tra lại thay vì nhớ.

Vì sao chia như vậy. Mục 18.1 trả lời “có những gì và nó giả định gì”; mục 18.2 trả lời “tiêu bao nhiêu vào đâu”. Nếu gộp lại, phần định lượng sẽ nuốt mất phần định tính — người đọc nhớ con số mà quên cơ chế. Mục 18.2 dừng lại từ vừng của 18.1 (linear probe, fine-tune, mô hình nền tảng) như đã biết, nên đọc ngược sẽ vấp.

Phụ thuộc lên trên. [B/06-chat-luong-du-lieu](#) (vì sao nhân đắt và bản), [B/07-chia-du-lieu-va-danh-gia](#) (vì sao contamination là vấn đề chết người), [B/09-thiet-ke-ham-mat-mat](#) (contrastive loss), [C/13-kien-truc-backbone/03-vit-va-inductive-bias](#) (vì sao ViT cần pretraining lớn).

Toả xuống dưới. [C/14-bai-toan-cv-loi](#) kế thừa trực tiếp không gian nhúng CLIP; [D/20-toi-uu-va-nen-mo-hinh](#) và [D/21-inference-va-trien-khai](#) nhận lấy mô hình nền tảng như một ràng buộc phải nín; [E/24-thi-nghiem-va-ablation](#) dạy cách chạy thí nghiệm điểm giao mà không tự lừa mình.

18.1 Nguồn giám sát ngày càng rẻ (Increasingly Cheap Supervision)

PHỤ THUỘC B/06-chat-luong-du-lieu (nhân đất và bắn tới mức nào); B/09-thiet-ke-ham-mat-mat (*contrastive loss*, cross-entropy); C/13-kien-truc-backbone/03-vit-va-inductive-bias (vì sao ViT cần nhiều dữ liệu hơn CNN); C/14-bai-toan-cv-loi (*open-vocabulary detection* là hệ quả trực tiếp của mục này).

TÓM TẮT

Mục này xếp bốn nguồn giám sát theo chi phí giảm dần: nhãn của người khác, chính bức ảnh, dòng chữ đi kèm ảnh, và khả năng suy luận mượn từ một mô hình ngôn ngữ. Với mỗi nguồn, mục nói rõ nó chống nghiệm tầm thường bằng cơ chế nào và hỏng ở đâu. Đọc xong, trước một bài toán có vài trăm ảnh nhãn, bạn quyết được ngay ba việc: đóng băng hay tinh chỉnh, chọn backbone tự giám sát nào, và công thức augmentation nào sẽ phá hỏng đúng tín hiệu bạn cần. Đoán mò ba việc đó thì mỗi cái tốn vài ngày GPU mới biết là sai. Nếu chỉ nhớ một điều: mỗi bất biến bạn ép vào augmentation chính là một loại thông tin bạn cố tình vứt đi, nên hãy liệt kê các bất biến đó trước khi sao chép công thức của bất kỳ paper nào.

KIẾN THỨC CẦN CÓ TRƯỚC

- **nhãn (label) và tín hiệu giám sát** — nhãn là đáp án đúng gắn với một mẫu dữ liệu; tín hiệu giám sát là bất cứ thứ gì đóng vai trò đáp án để tính hàm mất mát. Mọi ý tưởng trong mục này là cách tạo tín hiệu giám sát mà không phải trả tiền cho nhãn.
- **đặc trưng (feature) và embedding** — đặc trưng là vector số mà mạng sinh ra để mô tả một bức ảnh; *embedding* là cách gọi khác khi ta quan tâm tới hình học của không gian đó — ảnh giống nhau thì vector gần nhau.
- **đóng băng (frozen) và linear probe** — đóng băng là giữ nguyên trọng số backbone, không cho gradient cập nhật. *Linear probe* là huấn luyện đúng một tầng tuyến tính trên đặc trưng đóng băng — phép thử rẻ nhất để xem đặc trưng có chứa thông tin cần thiết hay không.
- **fine-tune (tinh chỉnh)** — huấn luyện tiếp một mô hình đã có trọng số, thường với *learning rate* nhỏ hơn nhiều lần để không xoá mất thứ nó đã học.
- **augmentation** — biến đổi ngẫu nhiên áp lên ảnh lúc huấn luyện (cắt, lật, đổi màu, làm mờ) nhằm dạy mô hình rằng những biến đổi đó không làm đổi đáp án. Chính vì thế nó cũng xoá thông tin: bất biến nào bạn dạy thì thông tin đó mất.
- **contrastive (tương phản)** — kiểu học kéo các cặp “giống nhau” lại gần và đẩy các cặp “khác nhau” ra xa trong không gian đặc trưng. Cặp giống nhau gọi là *positive*, cặp khác nhau gọi là *negative*.
- **masked modeling** — che một phần đầu vào rồi bắt mô hình khôi phục phần bị che. Vì đáp án chính là dữ liệu gốc, không cần nhãn nào cả.
- **zero-shot** — dùng mô hình cho một lớp hoặc một tác vụ mà nó chưa từng được huấn luyện trên đó, chỉ bằng cách mô tả lớp ấy bằng ngôn ngữ.

A. Đặt vấn đề

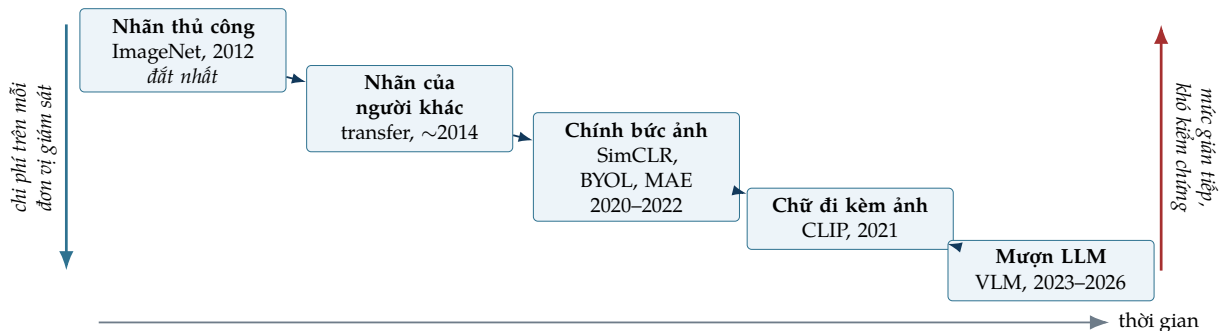
Nhãn đất, dữ liệu không nhãn thì gần như vô hạn. Một bộ ảnh nội soi có nhãn tổn thương cần bác sĩ ngồi vẽ *mask*; một bộ ảnh vệ tinh có nhãn loại đất cần chuyên gia đối chiếu thực địa. Trong khi đó, ổ cứng của cùng bệnh viện hay cùng cơ quan trắc địa chứa hàng triệu ảnh chưa ai chạm vào. Toàn bộ mục này là chuỗi câu trả lời ngày càng táo bạo cho một câu hỏi duy nhất: *tín hiệu giám sát miễn phí có thể lấy từ đâu?*

Câu hỏi nghe như một mẹo tiết kiệm chi phí, nhưng nó thực ra là câu hỏi về bản chất của việc học. Nhãn do người viết ra không phải thứ duy nhất mang thông tin về thế giới. Thông tin còn nằm ở ba chỗ khác. Hai bức ảnh chụp cùng một vật phải cho đặc trưng giống nhau. Phần bị che của một bức ảnh đoán được từ phần còn lại. Và dòng chú thích dưới một bức ảnh trên internet nói về đúng bức ảnh đó. Mỗi lần ngành khai thác được một nguồn như vậy, chi phí trên mỗi đơn

vị giám sát giảm một bậc. Cái giá cũng tăng một bậc: tín hiệu gián tiếp hơn, nhiều hơn, và khó truy ra mô hình đã học gì hơn.

Học xong mục này thì làm được gì mà trước đó không làm được? Bạn nhìn một bài toán có 300 ảnh nhân và quyết định ngay: đóng băng hay tinh chỉnh, chọn *backbone* tự giám sát nào, và công thức *augmentation* nào sẽ phá hỏng chính tín hiệu bạn cần. Đoán mò ba quyết định đó thì mỗi cái tốn vài ngày GPU mới lộ ra là sai.

Hình 18.2 đặt cả chuỗi lên một trục thời gian trước khi ta đi vào từng bước. Điều đáng nhìn không phải các mốc năm mà là *hai mũi tên ngược chiều nhau* ở hai bên: chi phí đi xuống trong khi mức gián tiếp đi lên. Toàn bộ chương là câu chuyện về việc đánh đổi giữa hai trục này.



Hình 18.2. Bốn nguồn giám sát trên một trục thời gian. Mỗi bước đi xuống một bậc chi phí và đi lên một bậc gián tiếp; không bước nào miễn phí thật sự, chúng chỉ dời hoá đơn từ “thuê người gán nhãn” sang “không biết mô hình đã học gì”.

TRỰC GIÁC

Nghĩ về nó như bài toán trắc địa. Để dựng một bản đồ, bạn không cần toạ độ tuyệt đối của từng cột mốc — một việc đắt như thuê máy đo GNSS cho mỗi điểm. Bạn chỉ cần đủ nhiều *ràng buộc tương đối* giữa các cột mốc (khoảng cách, góc), rồi giải hệ; hình dạng mạng lưới hiện ra chính xác, chỉ còn thiếu một phép biến đổi toàn cục — *gauge* — mà một vài điểm chuẩn rẻ tiền là ghim lại được.

Học tự giám sát làm đúng như vậy trong không gian đặc trưng. Mỗi cặp *augmentation* của cùng một ảnh là một ràng buộc “hai điểm này phải gần nhau”; mỗi cặp ảnh khác nhau là ràng buộc “hai điểm này phải xa nhau”. Giải cả hệ ra được một không gian đặc trưng có hình dạng đúng nhưng chưa gán nhãn — tức là xác định tới một *gauge*. Vài trăm ảnh có nhãn sau đó đóng vai trò các điểm chuẩn: *linear probe* chính là bước ghim *gauge*, và nó rẻ vì nó chỉ phải xác định một phép biến đổi chứ không phải toàn bộ hình dạng.

Mạch chính — bốn nguồn giám sát, theo thứ tự rẻ dần

- 1 Vấn đề** gán nhãn cho bài toán của mình quá đắt, nhưng người khác đã gán nhãn cho bài toán của họ.

Giải pháp **học chuyển giao** (*transfer learning*): lấy trọng số đã học trên ImageNet [82] làm điểm khởi đầu thay vì khởi tạo ngẫu nhiên.

Giả định các tầng đầu của mạng học những thứ chung cho mọi ảnh tự nhiên — cạnh, kết cấu, gradient màu — nên chúng không phụ thuộc vào việc nhãn đầu ra là gì.

Giá phải trả bạn thừa hưởng cả những thiên lệch của nguồn, kể cả những thiên lệch bạn không muốn.
- 2 Vấn đề** ngay cả nhãn của người khác cũng có giới hạn, và miền của bạn có thể chẳng ai gán nhãn.

Giải pháp	học tự giám sát (<i>self-supervised learning</i>): tạo bài toán phụ mà đáp án suy ra được từ chính bức ảnh.
Giả định	cấu trúc thống kê của ảnh đủ giàu để ép mô hình học biểu diễn có nghĩa khi ta buộc nó giải bài toán phụ đó.
Vấn đề mới	bài toán phụ có nghiệm tầm thường — mô hình xuất ra một hằng số là thoả mọi ràng buộc “giống nhau”. Chống lại sụp biểu diễn (<i>collapse</i> : mọi ảnh cho cùng một vector) này là toàn bộ nội dung kỹ thuật của mảng này.
3 Vấn đề	ảnh tự nó không nói cho ta biết ngữ nghĩa; nó chỉ nói về tương quan pixel.
Giải pháp	CLIP và tiền huấn luyện thị giác-ngôn ngữ (<i>vision-language</i>) [190]: dùng dòng chú thích đi kèm ảnh trên internet làm giám sát.
Hệ quả	ngôn ngữ trở thành nguồn giám sát ngữ nghĩa rẻ nhất từng có, và nó mang theo một tập nhãn <i>mở</i> thay vì 1000 lớp cố định.
Giá phải trả	dữ liệu web bẩn, thiên lệch, và không kiểm toán được.
4 Vấn đề	mô hình vẫn chỉ nhận dạng, chưa suy luận.
Giải pháp	mô hình thị giác-ngôn ngữ (<i>vision-language model, VLM</i>) — nối một <i>vision encoder</i> vào một LLM đã biết suy luận qua một tầng chiếu hoặc <i>cross-attention</i> [191], [192].
Giả định	biểu diễn thị giác có thể được “phiên dịch” thành thứ mà không gian <i>token</i> của LLM hiểu được, và phần suy luận không cần học lại.
Giá phải trả	đắt, khó kiểm soát, và ảo giác (<i>hallucination</i>) của phần ngôn ngữ lan sang phần thị giác.
Hệ quả	tích lũy của cả bốn bước: mô hình nền tảng (<i>foundation model</i>) trở thành một <i>thành phần dựng sẵn</i> , giống thư viện BLAS chứ không giống một mô hình. Nhiều khi dùng đặc trưng đã tiền huấn luyện tốt hơn hẳn tự train, <i>kể cả trong lĩnh vực hẹp</i> như ảnh y tế hay ảnh vệ tinh. Điều này trái trực giác, nhưng đã lặp lại đủ nhiều lần để tin [52].

B. Nguồn 1 — nhãn của người khác (Transfer Learning)

Học chuyển giao là kỹ thuật cũ nhất và vẫn là kỹ thuật đáng thử đầu tiên. Điều đáng nói không phải “nó hoạt động”, mà là *quyết định thực tế gói gọn trong một bảng bốn ô*. Học thuộc bảng này tiết kiệm rất nhiều thí nghiệm vô ích.

Bảng 18.1. Bốn ô quyết định của học chuyển giao. Trục dọc là khoảng cách miền, trục ngang là lượng dữ liệu có nhãn của bạn.

	Dữ liệu ít	Dữ liệu nhiều
Miền giống nguồn	<i>Linear probe</i> trên đặc trưng đóng băng	<i>Fine-tune</i> toàn bộ
Miền khác nguồn	<i>Fine-tune</i> vài tầng cuối, <i>augmentation</i> mạnh	<i>Fine-tune</i> toàn bộ; cân nhắc train từ đầu

Nói gọn thì logic bên dưới Bảng 18.1 chỉ có một câu: *số tham số bạn cho phép thay đổi phải tương xứng với lượng thông tin mới mà dữ liệu của bạn mang tới*. Đây chính là trục đánh đổi giả định–dữ liệu đã gặp ở B/04-nen-tang-hoc-tu-du-lieu, lần này xuất hiện dưới dạng “mở bao nhiêu tầng”. Ba hệ quả cụ thể:

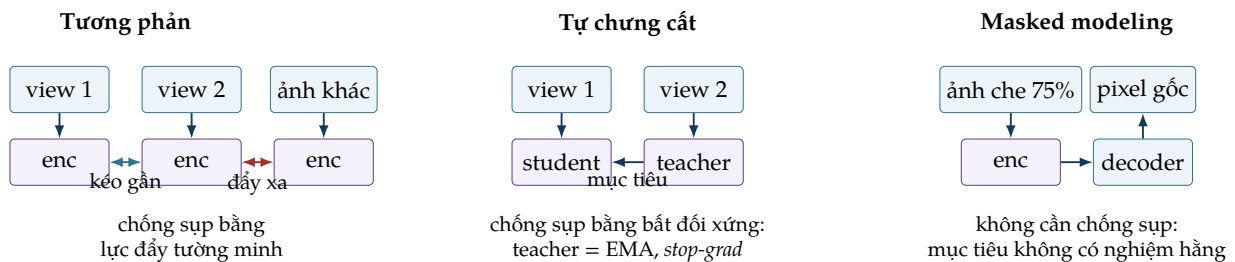
↪ **cùng nguyên lý với B/04** — đây chính là trục đánh đổi giả định–dữ liệu, lần này xuất hiện dưới dạng “mở bao nhiêu tầng thì vừa với lượng nhãn đang có.

- **Dữ liệu ít** $\Rightarrow$ **mở ít**. Mở toàn bộ mạng là mồi gọi *overfitting* và **quên thảm khốc** (*catastrophic forgetting*) — mạng quên đặc trưng tổng quát để chạy theo vài trăm mẫu.
- **Miền xa** $\Rightarrow$ **thay tầng cuối**. Các tầng cuối chứa thông tin chuyên biệt của nguồn nên phải thay; các tầng đầu vẫn dùng được vì cạnh vẫn là cạnh.
- **Cả hai điều kiện thuận** $\Rightarrow$ **tiền huấn luyện chỉ còn là khởi tạo tốt**. Khi bạn có nhiều dữ liệu và miền xa, lợi ích của tiền huấn luyện thu hẹp dần và đôi khi biến mất.

Một chi tiết dễ bị bỏ qua: chất lượng mô hình nguồn trên ImageNet có tương quan với chất lượng chuyển giao, nhưng tương quan đó *yếu đi rõ rệt* khi miền đích xa [193]. Nói cách khác, đừng chọn *backbone* chỉ vì nó đứng cao trên bảng ImageNet; với ảnh nội soi hay ảnh radar, thứ hạng đó gần như không mang thông tin.

C. Nguồn 2 – chính bức ảnh (Self-Supervised Learning)

Học tự giám sát tách thành ba triết lý khác nhau về cùng một câu hỏi: làm sao tạo tín hiệu từ hư không mà không rơi vào nghiệm tầm thường. Hình 18.3 cho thấy ba *cấu trúc đồ thị tính toán* khác nhau, và chính sự khác nhau về cấu trúc — chứ không phải về **hàm mất mát** (*loss function*) — mới là điều đáng nhớ.



Hình 18.3. Ba triết lý tự giám sát khác nhau ở *cấu trúc đồ thị*, không ở hàm mất mát. Tương phản chống sụp bằng lực đẩy từ *negative*; tự chưng cất chống sụp bằng bất đối xứng kiến trúc; *masked modeling* không cần chống sụp vì khôi phục pixel không có nghiệm hằng số.

Bảng 18.2. Ba họ tự giám sát. Cột cuối là cột đáng tiền nhất.

Triết lý	Cơ chế chống sụp	Giả định ngầm	Hỏng khi nào
Tương phản (<i>contrastive</i>) (SimCLR, MoCo)	Lực đẩy tương minh từ rất nhiều <i>negative</i>	Hai ảnh khác nhau trong <i>batch</i> thì thực sự khác lớp	Dữ liệu ít lớp hoặc lặp nhiều: <i>negative</i> hoá ra là <i>positive</i> ; <i>batch</i> nhỏ làm tín hiệu yếu
Tự chưng cất (<i>self-distillation</i>) (BYOL, DINO)	Bất đối xứng kiến trúc: teacher <i>EMA</i> (trung bình trượt, cập nhật chậm theo student), <i>stop-gradient</i> , <i>centering</i>	Điểm cố định của động lực học huấn luyện không phải hằng số	Cơ chế mong manh: đổi tốc độ <i>EMA</i> hay bỏ <i>predictor</i> là sụp; rất nhạy siêu tham số
Masked modeling (MAE)	Không cần: khôi phục pixel không có nghiệm tầm thường	Ngữ nghĩa nằm trong khả năng nội suy phần bị che	Đặc trưng thô, <i>linear probe</i> kém; phải <i>fine-tune</i> mới mạnh — tức đắt hơn ở phía sau

Bảng 18.2 nói rằng ba họ không cạnh tranh trên cùng một trục. Tương phản [194], [195] và tự chưng cất [51], [196] cho đặc trưng dùng được ngay với *linear probe*, hợp khi bạn định đóng băng *backbone*. Masked modeling [197] cho đặc trưng cần tinh chỉnh, hợp khi bạn có đủ dữ liệu đích và muốn tận dụng khả năng mở rộng của ViT.

Ví dụ nhỏ nhất: nhiệt độ quyết định gradient nằm ở đâu

Lấy một *batch* bốn ảnh. Đặc trưng đã chuẩn hoá, **độ tương đồng cosine** (*cosine similarity*) của cặp dương là 0,9 và của ba cặp âm đều là 0,1. Hàm InfoNCE [101] cho một mẫu là

$$L = -\log \frac{e^{s^+/\tau}}{e^{s^+/\tau} + \sum_j e^{s_j^-/\tau}}.$$

Đọc bằng lời: đây là cross-entropy của một bài toán phân loại mà “lớp đúng” là bạn đồng hành của chính mình, còn τ quyết định phân phối đó nhọn tới mức nào. Thay số cho ba giá trị **nhiệt độ** (*temperature*):

- $\tau = 1$: $L = -\log(2,460/5,775) = 0,853$
- $\tau = 0,5$: $L = -\log(6,050/9,714) = 0,474$
- $\tau = 0,1$: $L = -\log(8103,1/8111,2) = 0,0010$

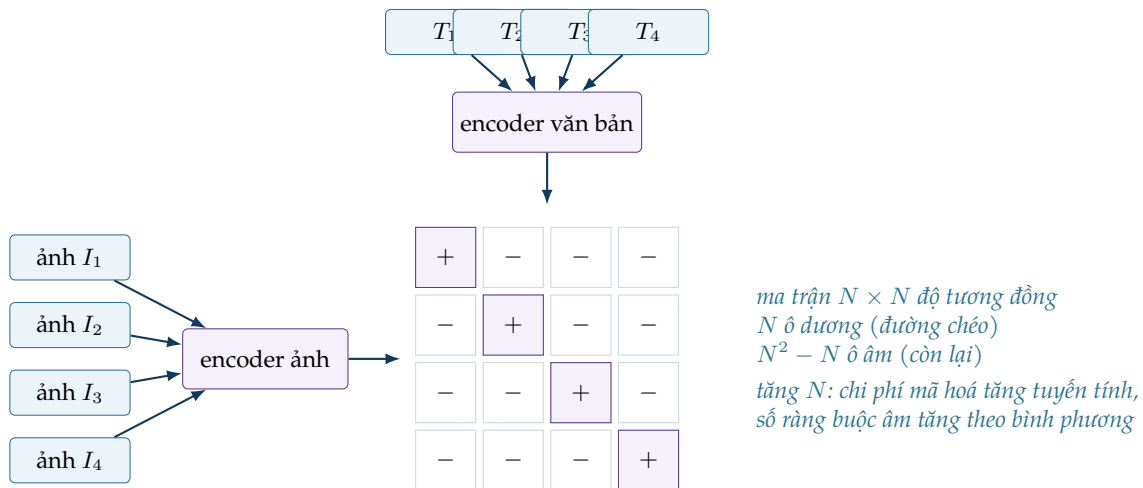
Ba con số nói rõ một điều mà công thức tổng quát giấu đi. Ở nhiệt độ thấp, một cặp đã tách được ở mức tầm thường (0,9 so với 0,1) đóng góp gần như *không* gradient; toàn bộ gradient dồn vào những *negative* có độ tương đồng gần với *positive*. Nhiệt độ vì thế không phải siêu tham số trang trí — nó là núm vặn quyết định hàm mất mát chú ý tới *negative* khó tới mức nào. Vặn quá thấp thì mô hình chỉ học từ vài cặp khó nhất và trở nên nhạy với nhiễu ngẫu nhiên; vặn quá cao thì mọi cặp đóng góp như nhau và biểu diễn không tách được các lớp gần nhau. Đây đúng là cơ chế đã gặp ở B/09-thiet-ke-ham-mat-mat dưới tên *hard negative mining*, chỉ khác là ở đây nó được điều khiển bằng một số thực thay vì bằng một bộ lọc.

↔ cùng nguyên lý với B/09 — nhiệt độ dồn gradient về các cặp khó chính là hard negative mining, chỉ khác là ở đây nó được điều khiển bằng một số thực thay vì bằng một bộ lọc.

D. Nguồn 3 — văn bản đi kèm ảnh (Vision-Language Pretraining)

CLIP giữ nguyên ý tưởng tương phản nhưng đổi định nghĩa cặp dương: thay vì hai *augmentation* của cùng một ảnh, cặp dương là (ảnh, dòng chú thích của chính nó), và cặp âm là mọi tổ hợp chéo còn lại trong *batch*. Với *batch* kích thước N , một lượt tính cho ma trận $N \times N$ độ tương đồng và $2N$ bài toán phân loại N lớp — đó là lý do CLIP thêm *batch* lớn tới mức bất thường.

Hình 18.4 cho thấy vì sao một *batch* lớn lại quan trọng đến thế: mỗi lượt chỉ có N ô dương trên đường chéo nhưng $N^2 - N$ ô âm, nên tăng N làm số ràng buộc âm tăng theo bình phương trong khi chi phí mã hoá chỉ tăng tuyến tính.



Hình 18.4. CLIP đối định nghĩa cặp dương: thay vì hai *augmentation* của cùng một ảnh, cặp dương là (ảnh, chú thích của chính nó). Một lượt cho N bài toán phân loại theo hàng và N bài theo cột trên cùng một ma trận — đó là toàn bộ lý do CLIP thêm *batch* lớn bất thường.

Nhận thức then chốt không nằm ở kiến trúc mà ở tính chất của tín hiệu:

- **Nhãn ImageNet** là một hàm từ ảnh vào một tập 1000 phần tử *cố định*.
- **Chú thích văn bản** là một hàm vào một không gian ngữ nghĩa *mở*.

Hệ quả trực tiếp là phân loại *zero-shot*: muốn thêm một lớp mới, bạn viết thêm một câu chứ không gán thêm nhãn. Hệ quả xa hơn là toàn bộ dòng **phát hiện từ vựng mở** (*open-vocabulary detection*) và phân vùng ở chương 14. Chúng chỉ ghép bộ đề xuất vùng với không gian *embedding* văn bản này, nên mọi điểm yếu của CLIP di truyền nguyên vẹn sang đó.

↪ **hệ quả của C/14** — phát hiện từ vựng mở là bộ đề xuất vùng cắm vào chính không gian embedding này, nên nó thừa hưởng nguyên vẹn điểm yếu của CLIP về quan hệ không gian và đếm.

Cái giá thì cụ thể và đã được đo nhiều lần. Mô hình học từ chú thích web hành xử gần với một túi từ hơn là một bộ phân tích cú pháp: nó phân biệt tốt “có con chó” với “có con mèo”, nhưng yếu ở quan hệ không gian (“con mèo bên trái con chó”), ở đếm số lượng, và ở phủ định. Kết quả *zero-shot* còn nhạy với cách viết *prompt* tới mức khó chịu. Chênh lệch giữa “a photo of a {}” và “{ }” có thể tới vài điểm phần trăm. Hãy đọc con số đó như dấu hiệu mô hình chưa ổn định, không phải như một mẹo tối ưu.

E. Nguồn 4 — ghép thị giác vào một LLM (Multimodal LLM)

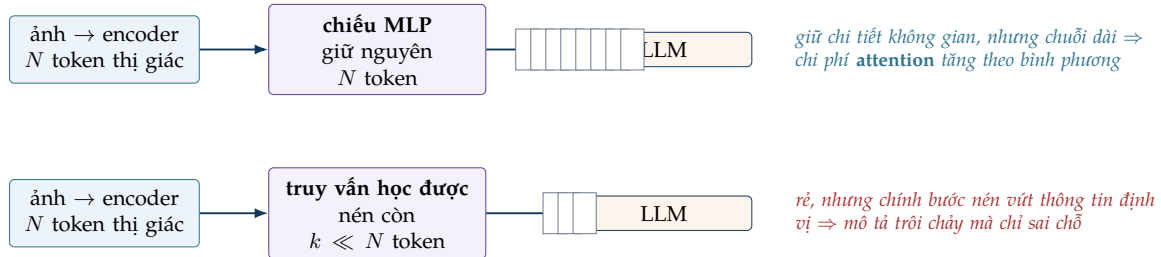
Bước tiếp theo hỏi: nếu đã có một mô hình ngôn ngữ biết suy luận, tại sao phải dạy lại phần suy luận cho thị giác? Kiến trúc VLM vì thế rất đơn giản về ý tưởng và gồm đúng ba khối:

1. **Vision encoder** — thường là CLIP hoặc SigLIP — biến ảnh thành một chuỗi *token* thị giác.
2. **Khối kết nối** — một MLP chiếu, hoặc một tập truy vấn học được kiểu Q-Former [191] — đưa chuỗi đó về cùng không gian với *token* văn bản.
3. **LLM** — xử lý chuỗi hỗn hợp hệt như xử lý văn bản thuần [192].

Trọng tâm thiết kế nằm ở khối thứ hai, và nó là một cuộc đánh đổi kinh điển. Chiếu MLP thẳng giữ nguyên số *token* thị giác nên giữ được chi tiết không gian, nhưng làm chuỗi dài ra và chi phí *attention* tăng theo bình phương. Nén xuống một số ít *token* truy vấn thì rẻ, nhưng chính bước nén đó vứt đi thông tin định vị — và đó là lý do nhiều VLM mô tả ảnh trôi chảy nhưng chỉ sai vị trí. Cái được là VQA, *grounding*, *captioning* và điều khiển giao diện bằng một mô hình duy

nhất; cái mất là chi phí, khả năng kiểm soát, và một dạng lỗi mới: mô hình bịa ra chi tiết không có trong ảnh với đúng giọng tự tin mà phần ngôn ngữ đã học được.

Hình 18.5 đặt hai biến thể của khối kết nối cạnh nhau, vì chính chỗ này quyết định VLM của bạn định vị được hay không.



Hình 18.5. Hai cách nối bộ mã hoá thị giác vào một LLM. Số ô nhỏ giữa khối kết nối và LLM là số **token** thị giác thực sự đi vào chuỗi; đó là đại lượng duy nhất quyết định cả chi phí lẫn khả năng định vị, nên đây là lựa chọn thiết kế quan trọng nhất của một VLM.

F. Kết quả tích lũy – mô hình nền tảng như một thành phần

Điều đáng ngạc nhiên nhất của cả mục không phải là mô hình nền tảng mạnh, mà là nó mạnh ở *những nơi lẽ ra nó không nên mạnh*. Lấy đặc trưng DINOv2 hoặc DINOv3 đóng băng rồi gắn thêm một đầu tuyến tính, bạn thường thắng một mạng train từ đầu ngay trên dữ liệu chuyên ngành [52]. Điều đó đúng với ảnh nội soi, ảnh vệ tinh và ảnh vi thể, ở mức vài trăm tới vài nghìn nhãn. Trực giác thông thường nói dữ liệu trong miền phải thắng dữ liệu ngoài miền; thực nghiệm nói vài trăm mẫu không đủ để học lại những thống kê ảnh cơ bản mà 140 triệu ảnh đã học hộ.

Điều này thay đổi vị trí của bạn trong hệ thống: bạn không còn là người huấn luyện mô hình mà là người *tích hợp* nó — giống hệt việc dùng một solver phi tuyến có sẵn thay vì tự viết Levenberg–Marquardt. Và cũng giống hệt như vậy, trách nhiệm chuyển từ “mô hình có đúng không” sang “tôi có biết mô hình này giả định gì và hỏng ở đâu không”.

G. Giới hạn và trường hợp hỏng

- **“Mở trọng số” không đồng nghĩa với “mở”.** Một số *backbone* tự giám sát mạnh nhất phát hành kèm giấy phép riêng hạn chế dùng thương mại, và dữ liệu huấn luyện thì hầu như không bao giờ được công bố. Với sản phẩm, đây là ràng buộc cứng cần kiểm tra *trước* khi chạy thí nghiệm.
- **Chuyển giao thất bại có hệ thống khi vật lý tạo ảnh khác nguồn** — ảnh nhiệt, radar, ảnh sự kiện, đa phổ. Ở đó thống kê tần số, phân bố nhiễu và cả số kênh đều khác, nên giả định “cạnh vẫn là cạnh” không còn đúng (xem [A/02-vat-ly-tao-anh/04-cam-bien-khac-va-domain-gap](#)).
- **Tự giám sát không miễn nhiễm với thiên lệch dữ liệu; nó chỉ làm thiên lệch khó nhìn hơn,** vì không còn bảng phân bố nhãn để kiểm tra. Nếu 90% ảnh không nhãn của bạn chụp ban ngày, biểu diễn học ra sẽ tốt ban ngày và không ai cảnh báo bạn.
- **Đánh giá bằng *linear probe* che giấu rất nhiều.** Một biểu diễn có thể tuyến tính phân tách tốt các lớp phổ biến mà vẫn vô dụng cho định vị chính xác — đúng loại thất bại quan trọng nhất nếu đầu ra của bạn là hộp, *mask* hay tương ứng điểm.

CẠM BẤY

Bất biến mà bạn ép vào *augmentation* chính là thông tin bạn cố tình vứt đi. *Color jitter* mạnh dạy mô hình rằng màu không quan trọng — một thảm hoạ nếu tác vụ đích là phát

hiện gỉ sét, phân loại nhuộm mô bệnh học, hay đọc đèn tín hiệu. *Random resized crop* dạy mô hình rằng tỉ lệ không quan trọng — một thảm họa nếu bạn cần ước lượng kích thước thật. Trước khi sao chép một công thức *augmentation* từ paper tự giám sát, hãy liệt kê các bất biến nó áp đặt và đối chiếu từng cái với thứ tác vụ của bạn cần *phân biệt*.

CẦN NHỚ

Mỗi bước trong chuỗi này đòi *nhân đất và chính xác lấy tín hiệu rẻ và gián tiếp*. Nó gần như luôn có lỗi, nhưng cái bạn mất là khả năng biết mô hình đã học gì — và bạn chỉ phát hiện ra điều đó ở đúng những trường hợp biên mà bạn quan tâm nhất. Hệ quả hành động: với mọi mô hình nền tảng, hãy tự thu một tập đánh giá nhỏ trên đúng phân phối triển khai trước khi tin bất kỳ con số nào. [T2]

H. Kết nối

Mục này là tiền đề trực tiếp của [C/14-bai-toan-cv-loi](#): phát hiện từ vụng mở chỉ là bộ đề xuất vùng cấm vào không gian *embedding* CLIP. Nó đối lập với [B/06-chat-luong-du-lieu](#). Chương kia lập luận rằng chất lượng nhãn quyết định trần hiệu năng; mục này lập luận rằng phần lớn tín hiệu lấy được mà không cần nhãn. Cả hai đều đúng, và ranh giới nằm ở chỗ bạn cần *ngữ nghĩa* hay cần *biểu diễn*. Về phía sau, [C/18/02-kinh-te-hoc-pretraining](#) trả lời câu hỏi định lượng: đầu tư compute vào đâu, và khi nào tinh chỉnh đáng giá hơn đóng băng.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao BYOL không sụp về nghiệm hằng số dù không có một *negative* nào, và bỏ thành phần nào thì nó sụp?
2. Với 500 ảnh nội soi, *linear probe* trên DINOv3 hay *fine-tune* toàn bộ ResNet ImageNet sẽ thắng, và lập luận đó dựa trên đại lượng nào — “số tham số được phép thay đổi so với lượng thông tin mới” nói gì ở đây?
3. Hạ nhiệt độ τ trong InfoNCE làm gradient dịch chuyển đi đâu, và tác dụng phụ nào xuất hiện khi tập dữ liệu có nhiều ảnh gần trùng nhau?
4. MAE cho *linear probe* kém hơn DINO nhưng tinh chỉnh lại tốt hơn. Điều đó nói gì về loại thông tin mà hai mục tiêu huấn luyện giữ lại?
5. Vì sao “mở trọng số” không đồng nghĩa với “mở”, và điều đó ảnh hưởng gì tới lựa chọn *backbone* cho một sản phẩm thương mại?
6. Tác vụ nào sẽ bị công thức *augmentation* chuẩn của SimCLR làm hỏng biểu diễn, và bất biến nào bị ép sai ở đó?

18.2 Kinh tế học của pretraining (The Economics of Pretraining)

PHỤ THUỘC [C/18/01-nguon-giam-sat](#) (mục ngay trước, bắt buộc); [B/11-gioi-han-tinh-toan](#) (compute, bảng thông, FLOPs); [B/07-chia-du-lieu-va-danh-gia](#) (vì sao nhiệm bản tập kiểm tra phá huỷ mọi kết luận); [E/24-thi-nghiem-va-ablation](#) (cách chạy thí nghiệm điểm giao cho từ tế).

TÓM TẮT

Mục này là mặt định lượng của việc tiền huấn luyện: có một ngân sách compute cố định thì chia cho tham số và dữ liệu theo tỉ lệ nào, và khi nào tính chính đáng giá hơn đồng bằng. Đọc xong bạn ước lượng được chi phí một lần tiền huấn luyện bằng một phép nhân trên giấy. Bạn đọc được một scaling law mà không bị nó lừa. Và bạn thiết kế được thí nghiệm điểm giao cho chính dữ liệu của mình, thay vì tin lời khuyến chung. Nếu chỉ nhớ một điều: compute, tham số và dữ liệu là ba mặt của cùng một ngân sách — bài học Chinchilla là ví dụ đắt nhất lịch sử ngành về việc tối ưu một mặt mà quên hai mặt kia.

KIẾN THỨC CẦN CÓ TRƯỚC

- **FLOPs và ngân sách compute** — FLOPs là số phép tính dấu phẩy động. Ngân sách compute C là tổng FLOPs bạn mua được cho một lần huấn luyện; nó cố định, nên chọn mô hình to hơn đồng nghĩa với chọn dữ liệu ít hơn.
- **token** — đơn vị nhỏ nhất của dữ liệu huấn luyện mà mô hình xử lý — một mẫu từ với văn bản, một mảnh ảnh (*patch*) với ViT. Lượng dữ liệu D luôn được đếm bằng token, không phải bằng số file.
- **hàm mất mát (loss)** — con số đo mức sai của mô hình trên dữ liệu; huấn luyện là quá trình giảm nó. Mọi “scaling law” trong mục này là công thức dự đoán con số đó sẽ giảm tới đâu.
- **scaling law** — quan hệ định lượng khớp từ thực nghiệm giữa loss, số tham số, lượng dữ liệu và compute. Giá trị của nó là cho phép *ngoại suy*: chạy vài mô hình nhỏ rồi dự đoán mô hình lớn trước khi tiêu tiền.
- **tập kiểm tra (test set) và tập validation** — tập validation dùng để chọn siêu tham số; tập test dùng đúng một lần để báo cáo. Trộn hai vai trò này là cách phổ biến nhất để tự lừa mình.
- **siêu tham số (hyperparameter)** — tham số bạn chọn chứ không phải mô hình học, ví dụ *learning rate* (độ dài mỗi bước cập nhật trọng số), kích thước *batch*, số *epoch*.
- **checkpoint** — ảnh chụp toàn bộ trọng số mô hình (và thường cả trạng thái optimizer) tại một thời điểm huấn luyện, lưu ra đĩa để chạy tiếp hoặc để triển khai.
- **seed** — số khởi tạo bộ sinh ngẫu nhiên. Đổi seed là đổi khởi tạo trọng số, thứ tự dữ liệu và augmentation — nên hai lần chạy khác seed cho hai kết quả khác nhau ngay cả khi mã hoàn toàn giống nhau.

A. Đặt vấn đề

Mục trước trả lời câu hỏi *lấy giám sát ở đâu*. Mục này trả lời câu hỏi đắt tiền hơn: *có một ngân sách compute cố định thì tiêu vào đâu*? Đây không phải câu hỏi triết học — nó là câu hỏi phân bổ nguồn lực, và trong nhiều năm cả ngành đã trả lời sai nó theo cùng một hướng.

Vấn đề khó ở chỗ ba đại lượng — số tham số N , lượng dữ liệu D , và compute C — ràng buộc nhau. Thêm tham số mà không thêm dữ liệu thì mô hình có thêm chỗ chứa mà không có gì để chứa. Thêm dữ liệu mà không thêm tham số thì mô hình bão hoà. Và cả hai đều bị chặn bởi tổng số phép tính bạn mua được. Nếu chỉ có trực giác, cách duy nhất tìm điểm cân bằng là thử; nhưng mỗi lần thử ở quy mô nghìn tỉ tham số tốn hàng triệu đô la, nên thử mù không chấp nhận được. Điều ngành cần là một *luật cho phép ngoại suy*: chạy vài mô hình nhỏ, khớp một đường cong, rồi dự đoán mô hình lớn trước khi tiêu tiền.

Học xong mục này thì làm được gì? Ba việc mà trước đó bạn chỉ làm được bằng cách hỏi người khác. Một, tính chi phí huấn luyện của một mô hình bất kỳ bằng một phép nhân trên giấy. Hai, phát hiện khi một bảng kết quả đang so hai mô hình có ngân sách khác nhau. Ba, thiết kế thí nghiệm điểm giao trả lời đúng câu hỏi bạn đang có.

TRỰC GIÁC

Hãy mượn ngôn ngữ của **hiệu chỉnh chùm tia** (*bundle adjustment*), với ý thức rằng đây là phép loại suy chứ không phải định lý. Trong BA, số ảnh là toạ độ điểm và tư thế camera; số phương trình là số quan sát. Nếu bạn thêm điểm 3D mà không thêm quan sát, hệ trở nên thiếu ràng buộc: nghiệm vẫn tồn tại, sai số tái chiếu vẫn nhỏ, nhưng cấu trúc thu được là ảo — nó khớp quan sát mà không tương ứng với thế giới.

Bài học Chinchilla nói rằng cả ngành đã chạy đúng tình huống đó suốt vài năm: mô hình được thổi phồng số tham số trong khi số “quan sát” — *token* dữ liệu — gần như đứng yên. Loss huấn luyện vẫn giảm, bảng kết quả vẫn đẹp, nên không ai thấy có gì sai. Chỗ mà phép loại suy gây cũng đáng nhớ. Mạng nơ-ron quá tham số hoá vẫn tổng quát được, nên “thiếu ràng buộc” ở đây không sinh nghiệm vô nghĩa như trong BA. Nó chỉ sinh ra *lãng phí*: cùng số FLOPs ấy lẽ ra mua được mô hình tốt hơn.

B. Scaling laws — từ thử mù sang dự đoán (Scaling Laws)

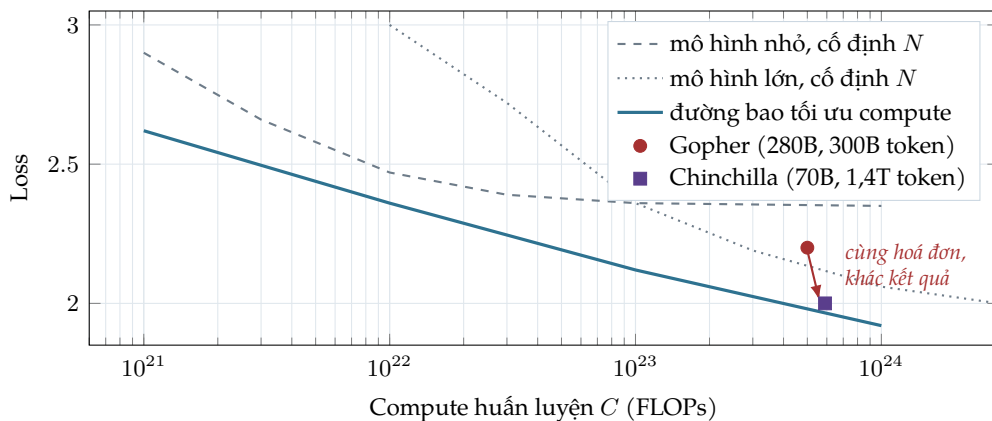
Quan sát thực nghiệm nền tảng là: khi tăng N , D và C , loss giảm theo quan hệ lũy thừa (*power law*) chứ không theo bậc thang [198]. Dạng thường dùng là

$$L(N, D) = L_{\infty} + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}.$$

Công thức này thực sự nói điều gì? Rằng loss của bạn là tổng của ba thứ độc lập. Thứ nhất là phần sàn không bao giờ giảm được, L_{∞} : entropy thật của dữ liệu, cộng mọi thứ mà kiến trúc này về nguyên tắc không biểu diễn nổi. Thứ hai là phần phạt vì mô hình quá nhỏ. Thứ ba là phần phạt vì dữ liệu quá ít. Muốn giảm loss, bạn phải biết mình đang bị phạt ở số hạng nào — và đó chính là điều mà đường cong cho bạn biết.

Điều làm công thức có giá trị thực dụng không phải hình dạng của nó, mà là việc nó *khớp được từ các mô hình nhỏ và ngoại suy đúng lên vài bậc độ lớn*. Đó là lần đầu tiên trong lịch sử ngành, một quyết định chi tiêu chín chữ số được đưa ra dựa trên đường cong khớp từ thí nghiệm rẻ tiền. Hiện tượng tương tự đã được xác nhận cho thị giác với ViT [199], dù các hằng số hoàn toàn khác.

Hình 18.6 cho thấy vì sao lập luận này không nhìn ra được nếu chỉ theo dõi một mô hình: mỗi kích thước mô hình có đường loss riêng, và điều đáng quan tâm là *đường bao* của cả họ chứ không phải bất kỳ đường nào trong đó.



Hình 18.6. Đường bao tối ưu compute và hai mô hình cùng ngân sách (*trục và hình dạng minh hoạ; chỉ vị trí tương đối của Gopher với Chinchilla là dựa trên kết quả đã công bố*). Một mô hình cố định kích thước sẽ rời khỏi đường bao khi ta đổ thêm dữ liệu vào nó: loss chững lại vì nó đã hết chỗ chứa. Gopher nằm phía trên đường bao ở đúng mức compute của Chinchilla — đó là hình ảnh của việc thừa tham số và thiếu dữ liệu.

↔ **đối lập với B/11** — chương kia đo giới hạn vật lý của phần cứng, mục này đo giới hạn kinh tế của việc dùng phần cứng đó — cùng một ngân sách nhìn từ hai phía.

Hai hệ quả cần đọc kỹ:

- **Lợi ích giảm dần nhưng không bao giờ dừng.** Luật lũy thừa không có ngưỡng mà thêm dữ liệu trở nên vô ích — chỉ có ngưỡng mà nó không còn đáng tiền. Câu hỏi đúng luôn là “đáng bao nhiêu”, không phải “còn tác dụng không”.
- **Scaling không sửa được lỗi bài toán.** Số hạng L_∞ nói rằng nếu nhân bản hoặc bài toán bị đặt sai, mọi đường cong đều chạm sàn ở một chỗ cao và không lượng compute nào cứu được.

C. Bài học Chinchilla (The Chinchilla Lesson)

Câu hỏi tiếp theo: với C cho trước, chọn N và D thế nào? Xấp xỉ chuẩn cho *transformer* là

$$C \approx 6ND \quad (\text{FLOPs cho toàn bộ quá trình huấn luyện}),$$

với hệ số 6 đến từ hai FLOPs cho mỗi phép nhân-cộng, nhân với ba lượt (một **lượt xuôi** (*forward pass*), hai **lượt ngược** (*backward pass*)). Diễn giải bằng lời: chi phí huấn luyện tỉ lệ thuận với tích của kích thước mô hình và lượng dữ liệu — nên với ngân sách cố định, hai đại lượng ấy nằm trên một đường hyperbol và bạn chỉ được chọn một điểm trên đó. Đáp số thực nghiệm là N và D nên tăng *cùng tốc độ*, mỗi cái tỉ lệ với $\sqrt{C}$ [200].

Ví dụ nhỏ nhất, dùng hai mô hình đã công bố.

- **Gopher:** $N = 280$ tỉ tham số, $D = 300$ tỉ *token* $\Rightarrow C = 6 \times 2,8 \times 10^{11} \times 3 \times 10^{11} \approx 5,0 \times 10^{23}$ FLOPs. Tỉ lệ $D/N = 1,07$.
- **Chinchilla:** $N = 70$ tỉ, $D = 1400$ tỉ *token* $\Rightarrow C = 6 \times 7 \times 10^{10} \times 1,4 \times 10^{12} \approx 5,9 \times 10^{23}$ FLOPs. Tỉ lệ $D/N = 20$.

Cùng bậc compute, tức cùng hoá đơn. Mô hình nhỏ hơn bốn lần nhưng ăn nhiều dữ liệu hơn gần năm lần thẳng rõ rệt trên hầu hết bài kiểm tra. Con số 20 *token* cho mỗi tham số chính là quy tắc ngón tay cái người ta nhớ.

Cơ chế của cái sai đáng học hơn con số 20. Số tham số là đại lượng dễ công bố, dễ so sánh và dễ gây ấn tượng; lượng *token* thì không ai khoe. *Khi một đại lượng dễ đo trở thành đại lượng được tối ưu, nó thôi là đại lượng đúng* — cùng một cơ chế đã làm hỏng nhiều bảng xếp hạng ở [E/24-thi-nghiem-va-ablation](#). Lưu ý thêm: con số 20 chỉ tối ưu cho *chi phí huấn luyện*. Mô hình phục vụ hàng tỉ lượt **suy luận** (*inference*) thì hoá đơn nằm ở phía chạy chứ không phải phía huấn luyện. Tối ưu tổng chi phí vòng đời vì thế đẩy bạn về mô hình *nhỏ hơn nữa*, huấn luyện lâu hơn nữa. Đó là lý do các mô hình mở gần đây thường vượt xa tỉ lệ 20.

D. Chất lượng hơn số lượng (Data Quality over Quantity)

Bước tiếp theo của cùng lập luận: nếu D quan trọng đến thế thì D là gì càng quan trọng hơn. Một dữ liệu bị lặp mười lần không mang thêm thông tin nào nhưng vẫn tốn đúng số FLOPs như dữ liệu mới. Với thị giác, hiệu ứng còn mạnh hơn vì ảnh web trùng lặp ở mức độ khủng khiếp: cùng một logo, cùng một ảnh *stock*, cùng một khung hình phim xuất hiện hàng nghìn lần.

Điều này gắn thẳng vào [B/06-chat-luong-du-lieu](#): các tập dữ liệu chuẩn của ngành có tỉ lệ nhân sai không nhỏ, đủ để đảo thứ hạng giữa các mô hình gần nhau [19]. Khi bạn tự tuyển *corpus*, một giờ viết bộ lọc trùng lặp thường có lãi cao hơn một giờ GPU.

E. Nhiễm bẩn tập kiểm tra — vấn đề do chính pretraining sinh ra

Tiền huấn luyện quy mô web tạo ra một vấn đề mà các phương pháp cũ không có: **nhiễm bẩn tập kiểm tra** (*test set contamination*). Nếu ảnh hoặc câu hỏi trong tập test đã nằm đâu đó trong *corpus* tiền huấn luyện, con số đánh giá không đo khả năng tổng quát hoá mà đo khả năng ghi nhớ. Cái làm nó nguy hiểm không phải bản thân hiện tượng — ai cũng biết là xấu — mà là việc nó *không kiểm chứng được*: *corpus* thường không công bố, và ngay cả khi công bố thì đối sánh hàng tỉ ảnh với tập test cũng không rẻ.

Có một dạng tinh vi hơn và phổ biến hơn: nhiễm bẩn qua cộng đồng. Một tập test công khai được cả ngành dùng mười năm sẽ bị việc chọn siêu tham số bào mòn dần, ngay cả khi không ai nhìn trực tiếp vào nó. Khi người ta dựng lại một tập test mới theo đúng quy trình cũ, hiệu năng mọi mô hình đều tụt, và tụt gần như song song [18]. Tính song song đó là tin tốt — thứ hạng tương đối còn dùng được — nhưng giá trị tuyệt đối thì không.

Kết luận thực dụng: với bất kỳ mô hình nền tảng nào, con số *zero-shot* trên *benchmark* công khai nên coi là chặn trên lạc quan. Bằng chứng duy nhất đáng tin là tập đánh giá do chính bạn thu, *sau* ngày mô hình được huấn luyện, trên đúng phân phối bạn sẽ triển khai.

↔ **tiền đề B/07** — quy trình chia dữ liệu sạch là điều kiện để con số đánh giá có nghĩa; nhiễm bẩn chỉ là rò rỉ dữ liệu ở quy mô web, nơi bạn không kiểm toán được nguồn.

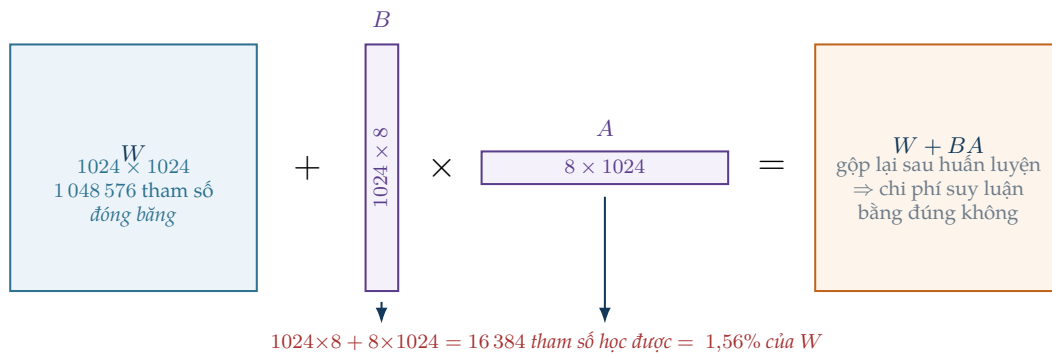
F. PEFT — đổi lại kinh tế học của việc thích nghi

Vấn đề rất cụ thể: tinh chỉnh toàn bộ một mô hình lớn nghĩa là lưu một bản sao toàn bộ trọng số cho mỗi khách hàng, mỗi tác vụ, mỗi phiên bản — và trạng thái *optimizer* của Adam còn tốn gấp đôi số đó. *PEFT* (tinh chỉnh chỉ một phần rất nhỏ tham số) giải bài toán này bằng cách đóng băng trọng số gốc và chỉ học một phần rất nhỏ tham số thêm vào:

- **Adapter** — chèn các khối nhỏ giữa các tầng, chỉ chúng được học [201].
- **LoRA** — phổ biến hơn: viết cập nhật dưới dạng tích hai ma trận hạng thấp, $\Delta W = BA$ với $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$ [3]. *Ý nghĩa*: ta giả định rằng “khoảng cách” giữa mô hình gốc và mô hình bạn cần nằm trong một không gian con rất thấp chiều, nên chỉ cần học tọa độ trong không gian con đó.
- **Prompt tuning** — không đụng trọng số nào, chỉ học một chuỗi *token* đầu vào. Rẻ nhất, cũng yếu nhất.

Con số làm rõ mọi thứ. Một ma trận trọng số 1024×1024 có 1 048 576 tham số. LoRA hạng $r = 8$ thay nó bằng $1024 \times 8 + 8 \times 1024 = 16\,384$ tham số, tức 1,56%. Nhân lên toàn mô hình: một *adapter* vài chục megabyte thay cho một *checkpoint* vài chục gigabyte — và vì ΔW cộng thẳng vào W được sau khi huấn luyện, chi phí suy luận bằng đúng không.

Hình 18.7 vẽ cùng phép tính đó dưới dạng diện tích, vì đó là cách nhanh nhất để thấy vì sao con số 1,56% không phải một thủ thuật mà là hệ quả hình học của việc chọn $r \ll d$.



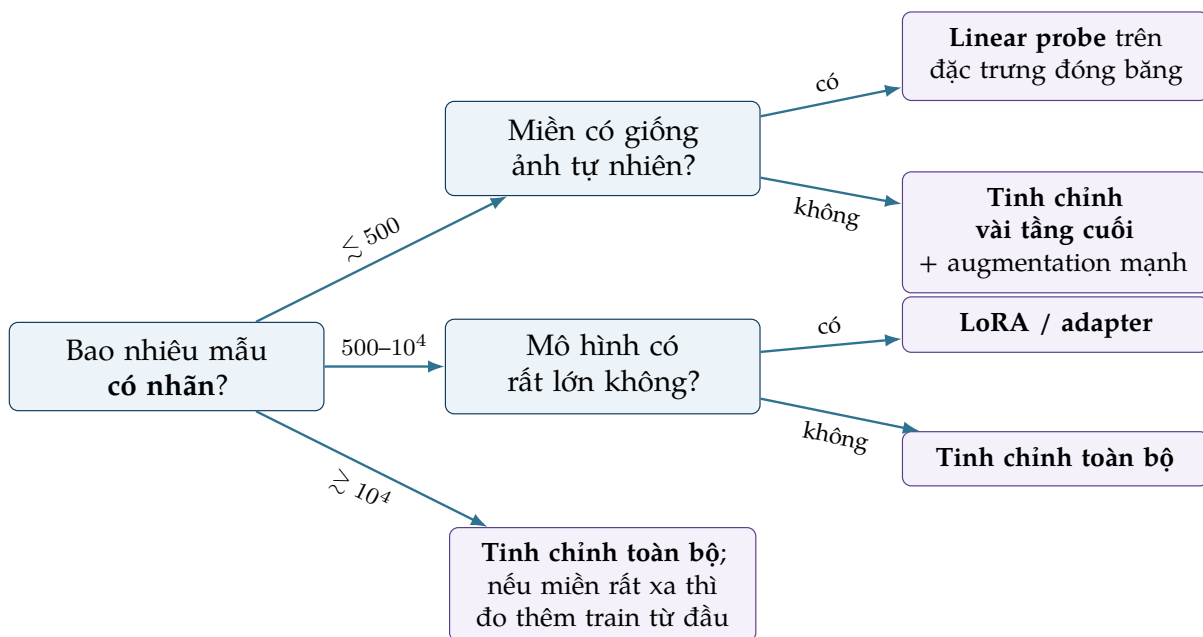
Hình 18.7. LoRA thay cập nhật đầy đủ ΔW bằng tích hai ma trận hạng thấp. Diện tích hai dải mỏng so với hình vuông lớn chính là tỉ lệ tham số phải học và phải lưu; và vì BA cộng thẳng vào W sau khi huấn luyện nên phần tiết kiệm này không phải trả lại ở lúc suy luận. Giả định ngầm là cập nhật cần thiết thật sự nằm trong một không gian con r chiều.

Giả định ngầm cần gọi tên: LoRA tin rằng *cập nhật cần thiết để thích nghi có hạng thấp*. Điều này hợp lý khi tác vụ đích là biến thể của thứ mô hình đã biết. Nó sai khi bạn đòi mô hình học một năng lực thực sự mới; khi đó hạng thấp thành cổ chai, và triệu chứng là loss huấn luyện dừng ở mức cao hơn hẳn tinh chỉnh toàn bộ bất kể học bao lâu. Phản ứng đúng lúc đó là tăng r hoặc mở thêm module, không phải tăng *learning rate*.

↔ cùng nguyên lý với $D/20$ — nén mô hình cũng giả định cái cần giữ nằm trong một không gian con nhỏ; LoRA là phiên bản của giả định đó ở phía thích nghi thay vì ở phía suy luận.

G. Quyết định cuối cùng vẫn là của bạn: thí nghiệm điểm giao

Mọi thứ ở trên là kiến thức chung. Hình 18.8 nén nó thành một cây quyết định dùng được trong ba mươi giây, và Bảng 18.3 nói rõ từng nhánh hỏng ở đâu. Cả hai chỉ thu hẹp không gian tìm kiếm — chúng không kết luận thay bạn.

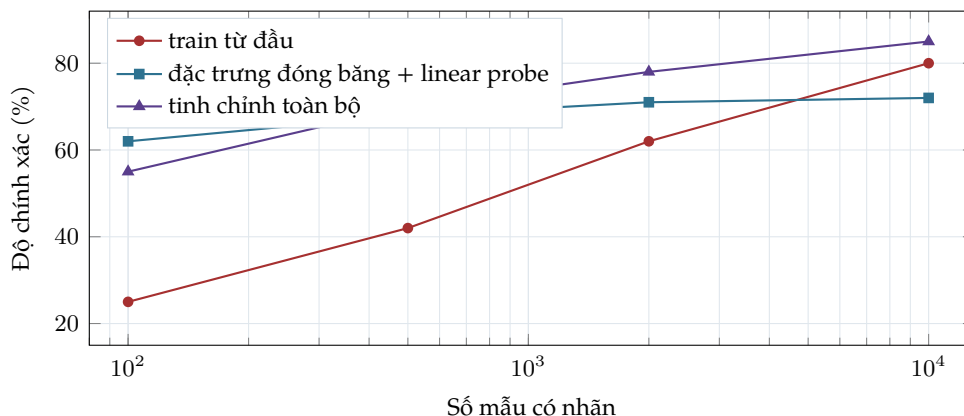


Hình 18.8. Cây quyết định chọn chế độ thích nghi. Các ngưỡng là điểm khởi đầu chứ không phải hằng số phổ quát — thí nghiệm điểm giao ở Hình 18.9 là cách xác định ngưỡng thật cho bài toán của bạn.

Bảng 18.3. Năm chế độ thích nghi, xếp theo số tham số được phép thay đổi.

Chế độ	Cơ chế	Giả định	Hỏng khi nào
Train từ đầu	Học mọi thứ từ dữ liệu của bạn	Dữ liệu đủ để học lại cả thống kê ảnh cơ bản	Dưới vài chục nghìn mẫu: gần như luôn thua
Linear probe	Chỉ học một siêu phẳng trên đặc trưng đóng băng	Thông tin cần thiết đã tuyến tính tách được	Tác vụ cần định vị chính xác, hoặc đặc trưng không có sẵn trong nguồn
Tinh chỉnh vài tầng cuối	Mở phần chuyên biệt của nguồn	Khác biệt miền nằm ở tầng cao	Khác biệt nằm ở tầng thấp (ảnh nhiệt, radar, đa phổ)
LoRA / adapter	Học cập nhật hạng thấp, gốc đóng băng	Cập nhật cần thiết có hạng thấp	Cần năng lực thực sự mới: loss chứng ở mức cao
Tinh chỉnh toàn bộ	Mở toàn bộ	Đủ dữ liệu để không quên đặc trưng gốc	Dữ liệu ít: overfit và quên thảm khốc

Thí nghiệm đáng làm nhất của cả chương là thí nghiệm điểm giao: chạy ba chế độ — train từ đầu, đóng băng, tinh chỉnh toàn bộ — ở các mức 100, 500, 2000, 10 000 mẫu, mỗi mức lặp ba *seed*, rồi vẽ ba đường cong trên một hình. Hình 18.9 minh hoạ hình dạng thường thấy, với cảnh báo rõ ràng rằng các con số là *minh hoạ*, không phải kết quả đo của một tập dữ liệu cụ thể.



Hình 18.9. Hình dạng điển hình của thí nghiệm điểm giao (*số liệu minh hoạ, không phải kết quả đo*). Đường đóng băng bảo hoà sớm nhưng thắng ở vùng dữ liệu rất ít; tinh chỉnh toàn bộ vượt lên sau điểm giao thứ nhất; train từ đầu chỉ cạnh tranh ở vùng dữ liệu lớn. Vị trí hai điểm giao khác nhau cho từng bài toán — đó chính là thứ bạn phải đo.

Hai điểm giao trên hình là hai con số đáng giá hơn mọi lời khuyên chung, vì chúng trả lời đúng câu hỏi bạn thực sự có: *với lượng nhãn tôi đang có, và với lượng nhãn tôi mua thêm được trong quý này, chế độ nào đúng?* Và vì trục hoành là log, chỉ bốn điểm đo đã đủ để nhìn ra hình dạng.

H. Giới hạn và trường hợp hỏng

- **Scaling law là quan hệ khớp thực nghiệm, không phải định luật.** Nó chỉ đúng *trong công thức huấn luyện đã dùng để khớp nó*: đổi kiến trúc, đổi lịch *learning rate*, đổi phân bố dữ liệu là các hằng số thay đổi và đường ngoại suy trượt. Đã có ít nhất một lần cả ngành ngoại suy sai vì công thức khớp có một chi tiết lịch học không nhất quán — chính là câu chuyện Chinchilla.
- **Scaling law mô tả loss, không mô tả năng lực.** Loss giảm mượt trong khi các năng lực rời rạc (đếm, suy luận nhiều bước) xuất hiện không mượt, nên đường cong đẹp không cho phép hứa hẹn gì về hành vi ở tác vụ đích.

- **LoRA dễ bị dùng để né việc chẩn đoán.** Khi kết quả kém, người ta tăng hạng thay vì hỏi liệu bài toán có thực sự là bài toán thích nghi hay không.
- **So sánh chỉ có nghĩa khi công sức tune ngang nhau.** So một *baseline* chưa tune với phương pháp đã tune là dạng thí nghiệm phổ biến nhất và vô giá trị nhất trong ngành [41].

CẠM BÃY

Ba lỗi hay đi cùng nhau khi chạy thí nghiệm điểm giao. **Một:** dùng cùng một tập *validation* để chọn siêu tham số cho cả ba chế độ rồi báo cáo trên chính tập đó — điểm giao dịch chuyển theo hướng có lợi cho chế độ được tune nhiều nhất. **Hai:** chạy một *seed* mỗi điểm; với 100 mẫu, độ lệch giữa các *seed* thường lớn hơn khoảng cách giữa hai chế độ, nên đường cong bạn vẽ là nhiễu. **Ba:** quên rằng *backbone* đã tiền huấn luyện có thể đã nhìn thấy dữ liệu đích của bạn — nếu ảnh của bạn từng ở trên internet, “*zero-shot*” không còn là *zero-shot*.

CẦN NHỚ

Compute, tham số và dữ liệu là ba mặt của cùng một ngân sách; tối ưu một mặt mà không nhìn hai mặt kia là cách sai tiền có hệ thống. Nhớ kèm hai công thức: $C \approx 6ND$ để ước lượng hoá đơn, và tỉ lệ $D/N \approx 20$ như điểm khởi đầu (không phải chân lý). Và mọi quy tắc chung chỉ là điểm khởi đầu — con số duy nhất ràng buộc quyết định của bạn là điểm giao đo trên chính dữ liệu của bạn. [T2]

I. Kết nối

Mục này là mặt định lượng của [C/18/01-nguồn-giam-sat](#): mục kia nói giám sát lấy ở đâu, mục này nói mua bao nhiêu thì đủ. Nó là tiền đề của [D/20-toi-uu-va-nen-mo-hinh](#), vì phần lớn kỹ thuật nén chỉ là cách mua lại compute suy luận bằng compute huấn luyện — cùng một logic ngân sách, chỉ khác cột chi. Nó phụ thuộc rất chặt vào [B/07-chia-du-lieu-va-danh-gia](#): không có quy trình chia dữ liệu sạch thì nhiệm bản biến mọi con số ở đây thành vô nghĩa. Và nó đối lập có ích với [B/11-gioi-han-tinh-toan](#): chương kia đo giới hạn vật lý của phần cứng, mục này đo giới hạn kinh tế của việc dùng phần cứng đó.

I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao Gopher và Chinchilla tốn xấp xỉ cùng số FLOPs, và điều gì trong cách ngành đo lường tiến bộ đã khiến sai lầm này sống sót nhiều năm?
2. Tỉ lệ 20 *token* trên tham số tối ưu cho chi phí gì? Nếu mô hình sẽ phục vụ một tỉ lượt suy luận thì tỉ lệ đúng dịch về phía nào, và vì sao?
3. Số hạng L_∞ nói gì về việc dùng compute để sửa một bài toán bị đặt sai?
4. LoRA hạng 8 trên ma trận 1024×1024 tiết kiệm bao nhiêu phần trăm tham số, và triệu chứng nào cho biết giả định hạng thấp đã bị vi phạm?
5. Vì sao khi dựng lại một tập test theo đúng quy trình cũ thì mọi mô hình đều tụt điểm, và vì sao việc chúng tụt gần như song song lại là tin tốt?
6. Ba cách phổ biến làm cho thí nghiệm điểm giao ra kết quả sai là gì, và sửa từng cái thế nào?

I 18.3 Thực hành

Phần thực hành của chương này có một đặc điểm khác các chương trước: bạn gần như không huấn luyện gì từ đầu. Công việc chủ yếu là chọn đúng thành phần dựng sẵn, đo xem nó hợp với dữ liệu của mình tới đâu, và biết dừng lại ở chế độ thích nghi rẻ nhất còn đủ tốt.

Repo và tài nguyên. *Backbone tự giám sát:* [facebookresearch/dinov3](#) (có cả bản cho ảnh vệ tinh; giấy phép riêng — dùng DINOv2 nếu bắt buộc Apache 2.0), [facebookresearch/mae](#), [lightly-ai/lightly](#) và [lightly-train](#) khi muốn tự tiền huấn luyện trên dữ liệu của chính mình. *Thị giác-ngôn ngữ:* [mlfoundations/open_clip](#), SigLIP. *VLM mở đánh thử tại thời điểm viết:* Qwen3-VL (mạnh toàn diện, OCR và video tốt), InternVL3 (giấy phép MIT, thoáng nhất nhóm dẫn đầu), Molmo2 của AI2 (mở cả dữ liệu huấn luyện — lựa chọn đúng khi cần tái lập), Pixtral (Apache 2.0). *Thích nghi:* [huggingface/peft](#). Danh sách tên mô hình này chốt tại tháng 9/2026 và sẽ cũ trong 6–12 tháng, trong khi các repo hạ tầng như [peft](#) hay [timm](#) bền hơn nhiều — hãy nhớ tên hạ tầng, tra lại tên mô hình.

Câu hỏi kiểm tra hiểu. Với 500 ảnh trong một lĩnh vực rất hẹp, chẳng hạn ảnh nội soi, thì *linear probe* trên DINOv3 hay tinh chỉnh toàn bộ một ResNet ImageNet sẽ thắng — và lập luận của bạn dựa trên đại lượng nào? Vì sao “mở trọng số” không đồng nghĩa với “mở”, và điều đó ràng buộc thế nào tới việc đưa mô hình vào sản phẩm? Nếu phải giải thích cho một đồng nghiệp trong ba câu vì sao BYOL không sụp dù không có *negative*, ba câu đó là gì?

Mini project (vài giờ đến hai ngày). Chạy thí nghiệm điểm giao trên một bài toán phân loại nhỏ của riêng bạn: ba chế độ train từ đầu, đóng băng và tinh chỉnh toàn bộ, ở bốn mức 100, 500, 2000 và 10 000 mẫu, mỗi ô ba *seed*. Vẽ ba đường cong trên một hình kèm dải phương sai giữa các *seed*. Sản phẩm cuối không phải cái hình, mà là hai con số: hai hoành độ điểm giao — ngưỡng quyết định cho chính bài toán của bạn.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Học chuyển giao	Lấy trọng số đã học trên một tập lớn làm điểm khởi đầu thay vì khởi tạo ngẫu nhiên. Đánh thử đầu tiên; hòng khi vật lý tạo ảnh của miền đích khác nguồn, ví dụ ảnh nhiệt hay radar.
••• Linear probe	Đóng băng backbone, chỉ huấn luyện một tầng tuyến tính trên đặc trưng. Phép thử rẻ nhất xem đặc trưng có chứa thông tin cần thiết; che giấu việc mô hình định vị kém.
•• Vì sao mở nhiều tham số lại hại khi ít dữ liệu?	Số tham số được phép thay đổi phải tương xứng với lượng thông tin mới. Vài trăm mẫu không trả nổi cho cả mạng, nên bạn nhận overfitting và quên thảm khốc.
••• Học tự giám sát	Tạo bài toán phụ mà đáp án suy ra từ chính bức ảnh, nên không cần nhãn người. Cái giá là bài toán phụ có nghiệm tầm thường phải chống.
••• Sụp biểu diễn	Mô hình xuất ra cùng một vector cho mọi ảnh, thoả mọi ràng buộc “giống nhau” mà không học gì. Chống nó là toàn bộ nội dung kỹ thuật của tự giám sát.
•• Vì sao contrastive cần rất nhiều negative?	Lực đẩy từ negative chính là cơ chế chống sụp duy nhất của nó. Batch nhỏ thì tín hiệu đẩy yếu; dữ liệu ít lớp thì negative hoá ra là positive.
•• Masked modeling	Che một phần ảnh rồi bắt mô hình khôi phục. Không cần chống sụp, nhưng đặc trưng thô nên linear probe kém — phải fine-tune mới mạnh.
•• Hạ nhiệt độ trong InfoNCE dồn gradient đi đâu?	Về những cặp âm có độ tương đồng gần với cặp dương. Vặn quá thấp thì chỉ học từ vài cặp khó nhất và rất nhạy với ảnh gần trùng nhau.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● CLIP	Tương phản giữa ảnh và chú thích của chính nó, trên ma trận $N \times N$ trong một batch. Cho tập nhãn mở và zero-shot; yếu ở quan hệ không gian, đếm và phủ định.
●● Vì sao VLM mô tả ảnh trôi chảy nhưng sai vị trí?	Vì khối kết nối nên nhiều token thị giác xuống ít token truy vấn. Bước nén đó rẻ nhưng vứt mất thông tin định vị.
●●● Mô hình nền tảng	Mô hình lớn tiền huấn luyện trên dữ liệu quy mô rất lớn, dùng lại được cho nhiều bài toán. Thường thắng cả mạng train từ đầu ngay trong lĩnh vực hẹp.
●● Vì sao chi phí huấn luyện xấp xỉ $6ND$?	Hai FLOPs cho mỗi phép nhân-cộng, nhân ba lượt: một lượt xuôi và hai lượt ngược. Hệ số 6 chỉ đúng cho transformer huấn luyện chuẩn.
●●● Scaling law	Quan hệ lũy thừa khớp từ thực nghiệm giữa loss, tham số, dữ liệu và compute. Cho phép ngoại suy từ thí nghiệm rẻ; trượt ngay khi bạn đổi kiến trúc hoặc lịch learning rate.
●●● Chinchilla	Kết quả cho thấy phần lớn mô hình lớn thiếu dữ liệu chứ không thiếu tham số; quy tắc ngón tay cái là 20 token cho mỗi tham số. Chỉ tối ưu cho chi phí huấn luyện.
●●● Contamination	Dữ liệu test lọt vào corpus tiền huấn luyện, nên điểm số đo trí nhớ chứ không đo tổng quát hoá. Không kiểm chứng được, vì corpus thường không công bố.
●●● LoRA	Học cập nhật hạng thấp $\Delta W = BA$, gốc đóng băng; hạng 8 trên ma trận 1024×1024 chỉ tốn 1,56% tham số. Cộng thẳng vào W sau huấn luyện nên suy luận không đắt thêm.
●● Loss huấn luyện chừng nào cao hơn hẳn full fine-tune dù học rất lâu?	Giả định hạng thấp của LoRA đã bị vi phạm: bạn đang đòi một năng lực mới chứ không phải thích nghi. Tăng r hoặc mở thêm module, đừng tăng learning rate.
●● Zero-shot đẹp trên benchmark nhưng tệ trên dữ liệu của bạn?	Nghi contamination trước tiên, rồi nghi khoảng cách miền. Cách duy nhất kết luận được là một tập đánh giá bạn tự thu sau ngày mô hình ra đời.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
DINOv2 / DINOv3	Tự chưng cất không cần negative, dùng đúng cơ chế bất đối xứng teacher-student ở mục 18.1; đặc trưng đồng bằng cho linear probe	Hạ tầng đặc trưng phổ biến nhất cho thị giác 2024–2026; DINOv3 có bản riêng cho ảnh vệ tinh
OpenCLIP / SigLIP	Tương phản chéo phương thức ảnh-văn bản, đúng ý tưởng “ngôn ngữ là giám sát ngữ nghĩa rẻ nhất”	Bộ mã hoá được dùng lại trong gần như mọi VLM mở hiện nay
LLaVA, Qwen-VL, InternVL	Khối kết nối nối bộ mã hoá thị giác vào LLM — chính cuộc đánh đổi số token và thông tin định vị ở mục 18.1	Kiến trúc ba khối này ổn định từ 2023 tới nay dù tên mô hình đổi liên tục
huggingface/peft	LoRA và adapter, tức giả định cập nhật thích nghi có hạng thấp	Hạ tầng; là cách mặc định để tinh chỉnh mô hình lớn từ 2023
Mô hình mở gần đây	Tỉ lệ dữ liệu trên tham số vượt xa 20, đúng như lập luận tối ưu <i>tổng chi phí vòng đời</i> chứ không chỉ chi phí huấn luyện	Được cho là động cơ chính; tôi không kiểm chứng được con số cụ thể của từng mô hình

MINI PROJECT SLAM · Điểm giao trên mảnh ảnh quanh đặc trưng EuRoC · 1–2 ngày

Mục tiêu	trả lời bằng số, trên chính dữ liệu của bạn, câu hỏi “bao nhiêu mẫu thì tinh chỉnh vượt đặc trưng đóng băng”. Bảng bốn ô chỉ thu hẹp không gian; con số này mới quyết định.
Dữ liệu	EuRoC MH_01 và V1_02 để dựng tập, V2_03 giữ riêng để đánh giá. Cắt mảnh 32×32 quanh các đặc trưng ORB do chính frontend của bạn phát hiện; nhấn “cùng điểm bản đồ / khác điểm” lấy miễn phí từ liên kết dữ liệu trong bản đồ sau khi chạy hệ đầy đủ, không cần ai gán tay.
Các bước	dựng tập ở bốn mức 100, 500, 2000 và 10 000 cặp. Chạy ba chế độ trên cùng bốn mức: một CNN nhỏ huấn luyện từ đầu; <i>linear probe</i> trên DINOv2 đóng băng; tinh chỉnh toàn bộ chính <i>backbone</i> đó. Ba <i>seed</i> mỗi ô, tổng 36 lần chạy ngắn. Đo độ chính xác ghép cặp trên chuỗi giữ riêng.
Xong khi	bạn vẽ được ba đường trên một hình kèm dải phương sai giữa các <i>seed</i> , và ghi ra được <i>hai con số</i> : hai hoành độ nơi thứ hạng đảo chiều. Nếu dải phương sai chồng lên nhau tại điểm giao, kết luận đúng là “chưa phân giải được” và bạn nói được cần bao nhiêu <i>seed</i> nữa.
Nối với	C/18 cho bảng bốn ô và hình điểm giao; B/07 cho lý do phải giữ riêng cả một chuỗi chứ không chỉ giữ riêng vài khung hình. Tập mảnh ảnh dựng ở đây được dùng lại nguyên vẹn ở mini project chương 19 và 20.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Với 500 ảnh, *linear probe* trên đặc trưng nền tảng thường thắng, vì mở toàn bộ tham số của một ResNet với vài trăm mẫu là mời gọi *overfitting* và quên thảm khốc. Ở 50 000 mẫu chiều đảo lại: lúc đó dữ liệu đủ để *trả tiền* cho số tham số được mở, nên tinh chỉnh toàn bộ vượt lên — đó chính là điểm giao thứ nhất ở Hình 18.9.
- A2. Vì nghiệm hằng số bị chặn bằng *cấu trúc*, không bằng hàm mất mát: tương phản thêm lực đẩy từ negative, tự chưng cất dựng bất đối xứng teacher EMA cộng stop-gradient, còn masked modeling đặt mục tiêu khôi phục pixel vốn không có nghiệm hằng. Bỏ đúng thành phần chống sụp của mỗi họ là mô hình sụp thật.
- A3. Vì nhãn ImageNet là hàm vào một tập 1000 phần tử cố định, còn chú thích là hàm vào một không gian ngữ nghĩa mở. Chú thích nhiều hơn theo từng mẫu, nhưng nó phủ được vô hạn khái niệm và đi kèm quan hệ giữa các khái niệm — thứ mà một tập nhãn đóng không bao giờ có.
- A4. Số tham số. Đó là đại lượng dễ công bố và dễ gây ấn tượng, nên nó bị tối ưu thay cho đại lượng đúng; lượng token thì không ai khoe. Với cùng $C \approx 6ND$, Chinchilla chọn $D/N = 20$ thay vì 1,07 và thắng rõ rệt.
- A5. Vì vài trăm tới vài nghìn ảnh chuyên ngành không đủ để học lại những thống kê ảnh cơ bản mà 140 triệu ảnh đã học hộ. Đặc trưng nền tảng cung cấp phần chung, còn phần chuyên ngành chỉ cần một đầu tuyến tính — trừ khi vật lý tạo ảnh khác hẳn, lúc đó lập luận này sụp.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn mở một xưởng và cần một người phân loại ốc vít. Cách cũ là thuê một người chưa biết gì rồi ngồi cạnh, cầm từng con ốc lên và nói tên nó, hàng nghìn lần. Rất tốn công, và bạn phải trả tiền cho từng lần nói tên.

Mẹo của mười năm qua là: đừng tuyển người chưa biết gì. Hãy nhận một người đã nhìn hàng trăm triệu bức ảnh đủ loại trong đời — xe cộ, cây cối, mặt người, nhà cửa — dù chưa từng thấy con ốc nào của bạn. Người đó đã có sẵn con mắt: biết thế nào là một cạnh, một vết xước, hai vật giống nhau. Bạn chỉ cần dạy tên gọi, và việc đó xong trong nửa buổi thay vì ba tháng.

Cái giá không nằm ở công sức của bạn mà nằm ở chỗ khác. Con mắt ấy được rèn bằng hàng trăm triệu bức ảnh và một hoá đơn tiền điện khổng lồ; chỉ là người khác đã trả hộ, và bạn không biết họ đã cho nó xem những gì. Nên khi ảnh của bạn chụp bằng máy hồng ngoại chứ không phải máy ảnh thường, kinh nghiệm kia bỗng vô dụng và không ai báo trước cho bạn.

Nửa sau của chương là một bài học về tiền: cả ngành từng tin máy càng to càng giỏi, rồi phát hiện phần lớn máy to bị bỏ đói — cùng số tiền điện, một cái máy nhỏ hơn nhưng được xem nhiều ảnh hơn thì giỏi hơn hẳn.

Chỗ không rút gọn được: vì sao con mắt rèn trên ảnh đời thường lại dùng tốt cho ảnh nội soi hay ảnh vệ tinh thì tới nay chưa ai giải thích gọn mà không sai; mọi cách nói đơn giản đều bỏ quên phần ví dụ là ảnh radar, nơi nó gần như không chuyển được.

Một câu: *Đừng dạy máy nhìn từ đầu — hãy mượn một con mắt đã được rèn bằng dữ liệu và tiền điện của người khác, rồi chỉ dạy nó tên gọi trong nghề của bạn.*

Tổng kết chương

Chương này kể một chuyện động lịch sử. Giám sát từng là thứ đắt nhất trong hệ thống; ngành đã tìm ra bốn cách lấy nó gần như miễn phí. Cái giá mỗi lần là tín hiệu gián tiếp hơn và khó kiểm chứng hơn. Kết quả tích lũy là mô hình nền tảng. Nó đổi vai trò của bạn từ người huấn luyện sang người tích hợp. Câu hỏi trung tâm đổi theo: không còn là “mô hình này có đúng không”, mà là “tôi có biết nó giả định gì và hỏng ở đâu không”.

Mặt định lượng gọn hơn nhưng tốn kém hơn nếu làm sai: compute, tham số và dữ liệu là ba mặt của một ngân sách; bài học Chinchilla là ví dụ đắt tiền nhất trong lịch sử ngành về việc tối ưu một mặt mà quên hai mặt kia; và mọi con số chung, kể cả tỉ lệ 20 *token* trên tham số, chỉ là điểm khởi đầu cho phép đo của riêng bạn.

Hai chỗ chương này để ngỏ và được nhận tiếp ở nơi khác. Thứ nhất, làm sao *đánh giá* một mô hình nền tảng cho tử tế khi bạn không biết nó đã thấy gì — đó là [E/24-thi-nghiem-va-ablation](#) và [E/25-do-tin-cay-va-van-hanh](#). Thứ hai, một khi đã chọn được mô hình thì làm sao ép nó vào ngân sách phần cứng thật. Đó là toàn bộ Phần D, bắt đầu ở [D/20-toi-uu-va-nen-mo-hinh](#). Ở đó cùng một logic ngân sách xuất hiện lại, chỉ khác là cột chi đổi từ compute huấn luyện sang compute **suy luận** (*inference*).

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Mục 1 xếp các nguồn giám sát theo độ rẻ, mục 2 xếp chi phí theo compute và dữ liệu. Ghép hai trục lại: nguồn giám sát rẻ nhất có phải nguồn có tổng chi phí thấp nhất không, và khoản chi bị bỏ sót nằm ở đâu?
2. Augmentation của học đối lập ép một tập bất biến vào biểu diễn, còn định luật scaling nói cứ thêm dữ liệu là loss giảm. Với một tác vụ mà bất biến bị ép sai, thêm dữ liệu pretraining có cứu được không, và số hạng nào trong công thức scaling mô tả đúng chỗ không cứu được?
3. Bạn nhận một backbone mở trọng số và định dùng cho ảnh y tế. Ba câu hỏi phải trả lời trước — một về nguồn giám sát, một về nhiệm vụ dữ liệu, một về giấy phép — là gì, và câu nào bạn thật sự kiểm chứng được bằng thí nghiệm?
4. Vì sao thí nghiệm tìm điểm giao giữa linear probe và fine-tune lại dễ ra kết quả sai, và trong các cách làm sai thường gặp, cách nào không lộ ra nếu chỉ nhìn bảng kết quả cuối?
5. LoRA giả định ma trận cập nhật có hạng thấp, còn mục 1 lập luận theo lượng thông tin mới so với số tham số được phép thay đổi. Hai lập luận đó gặp nhau ở đâu, và triệu chứng nào cho biết hạng bạn chọn quá nhỏ so với lượng thông tin mới?
6. Chương nói bạn đã đổi vai từ người huấn luyện thành người tích hợp. Việc đổi vai đó làm loại lỗi nào khó phát hiện hơn hẳn so với thời tự huấn luyện, và bạn bù lại bằng phép đo nào?

KỸ THUẬT, TỐI ƯU VÀ TRIỂN KHAI

TÓM TẮT

Phần D là tầng công cụ: tự viết forward và backward bằng NumPy để hiểu, rồi PyTorch để làm việc, rồi nén mô hình, triển khai, và mức MLOps tối thiểu đủ để sáu tháng sau dựng lại được kết quả của mình. Đọc xong bạn đưa được một mô hình từ notebook ra thiết bị thật với ngân sách latency cho trước. Nếu chỉ nhớ một điều: đây là phần hết hạn nhanh nhất của cây, trừ một ngoại lệ — việc tự viết backward pass thuộc tầng bền vững và nên làm ngay sau chương 12, không đợi tới đây.

Phần này là tầng công cụ, và nó được đặt sau phần C có chủ ý: đây là phần hết hạn nhanh nhất của cây, nên đầu tư vào nó trước khi có nền của phần B và C là đầu tư vào thứ sẽ cũ. Nguyên tắc chi phối là học đủ để làm việc, không đầu tư sâu, và tra cứu khi cần thay vì ghi nhớ.

Có một ngoại lệ quan trọng, và nó nằm ngay ở chương đầu. Việc tự viết forward và backward pass bằng NumPy, tự chạy kiểm tra gradient bằng sai phân hữu hạn, trông giống một bài tập công cụ nhưng thực ra thuộc tầng [T2]–[T3]. Nó xây trực giác về đồ thị tính toán, về nơi gradient chết, và về cái giá bộ nhớ của activation — ba thứ không hết hạn và không thể học được bằng cách gọi một hàm có sẵn. Vì vậy chương 19 chia làm hai giai đoạn rõ rệt: giai đoạn tự viết để hiểu, rồi mới tới giai đoạn dùng framework để làm việc.

Ba chương còn lại đi theo trình tự của một hệ thống thật rồi khỏi máy nghiên cứu. Chương 20 là tối ưu và nén mô hình, với một thứ tự thử nghiệm thực dụng và lý do của thứ tự đó, vì phần lớn người mới bắt đầu từ kỹ thuật hào nhoáng nhất thay vì từ kỹ thuật rẻ nhất. Chương 21 là inference và triển khai, nơi những con số đo được trong notebook thường không sống sót, và nơi cách đo latency sai là một trong những lỗi phổ biến nhất. Chương 22 là tái lập và mức MLOps tối thiểu: đủ để sáu tháng sau còn dựng lại được đúng kết quả của mình, không hơn.

Bảng 18.6. Bản đồ Phần D: các chương, nội dung, và mức đầu tư đề xuất.

Ch.	Thư mục	Nội dung	Mức
19	19-numpy-toi-pytorch	Hai giai đoạn: tự viết forward và backward bằng NumPy; rồi PyTorch thành thạo	giai đoạn 1 cốt lõi , giai đoạn 2 công cụ
20	20-toi-uu-va-nen-mo-hin	Quantization, pruning, distillation, chọn độ phân giải; thứ tự thử nghiệm thực dụng	công cụ
21	21-inference-va-trien-k	Export, runtime, batching, đo latency đúng cách, ràng buộc thiết bị biên	công cụ
22	22-tai-lap-va-mlops	Seed và nguồn bất định, version hoá, theo dõi thí nghiệm, giám sát sau triển khai	cốt lõi ở mức tối thiểu

Giai đoạn 1 của chương 19 nên làm ngay sau chương 12, không đợi tới phần D. Ba chương còn lại đọc khi thật sự phải triển khai, vì chúng hết hạn nhanh nhất trong cây.

Từ NumPy tự viết tới PyTorch thành thạo (From Hand-Written NumPy to Fluent PyTorch)

PHỤ THUỘC A/01-toan-toi-thieu/02-vi-phan-va-toi-uu (quy tắc chuỗi); B/09-thiet-ke-ham-mat-mat (hinge loss, cross-entropy); B/11-gioi-han-tinh-toan (roofline — bắt buộc trước khi đọc phần tăng tốc của mục 19.2); C/12-co-che-huan-luyen (BatchNorm, dropout, optimizer).

TÓM TẮT

Chương này có hai giai đoạn đọc theo đúng thứ tự: tự cài **lan truyền ngược** (*backpropagation*) bằng NumPy để có trực giác về cơ chế, rồi học PyTorch theo mạch vấn đề → công cụ để có năng suất. Đọc xong bạn debug được những lỗi không có thông báo lỗi: gradient sai, trạng thái train/eval lẫn lộn, rò bộ nhớ vì logging. Bạn cũng tăng tốc được một vòng lặp huấn luyện dựa trên số đo, thay vì dựa trên mẹo nghe được. Nếu chỉ nhớ một điều: giai đoạn 1 là bài tập *trực giác* tăng [T2][T3], làm đúng một lần rồi bỏ. Giai đoạn 2 là bài tập *công cụ* tăng [T4], phải cập nhật liên tục. Lẫn lộn hai loại này là cách phổ biến nhất để phí thời gian.

KIẾN THỨC CẦN CÓ TRƯỚC

- **lượt xuôi** (*forward pass*) và **lượt ngược** (*backward pass*) — forward là lượt tính từ đầu vào ra loss; lượt ngược là lượt tính ngược để lấy đạo hàm của loss theo từng tham số. Mọi khung huấn luyện đều là vòng lặp hai lượt này.
- **gradient** — vector đạo hàm của loss theo tham số; nó chỉ hướng làm loss *tăng* nhanh nhất, nên ta đi ngược lại nó.
- **lan truyền ngược** (*backpropagation*) — thuật toán tính toàn bộ gradient bằng một lượt đi ngược qua đồ thị tính toán, tốn cùng bậc chi phí với một lượt xuôi — lý do duy nhất khiến huấn luyện mạng sâu khả thi.
- **learning rate** — độ dài mỗi bước cập nhật trọng số theo hướng ngược gradient. Quá lớn thì loss phát tán, quá nhỏ thì huấn luyện không bao giờ xong; đây là siêu tham số quan trọng nhất.
- **framework** — thư viện lo hộ bạn đồ thị tính toán, lan truyền ngược tự động, quản lý thiết bị và dữ liệu — PyTorch là ví dụ. Giá trị của chương này nằm ở chỗ biết framework đang giấu cái gì.
- **tăng bán rã** [T2] **so với** [T4] — [T2] là kiến thức nguyên lý, không hết hạn khi công cụ đổi; [T4] là kiến thức công cụ, sẽ cũ trong vài năm. Giai đoạn 1 của chương là [T2]–[T3], giai đoạn 2 là [T4].

CÂU HỎI MỞ ĐẦU

- Q1. Một mạng 10^9 tham số huấn luyện được trong vài ngày, trong khi vi phân thuận cần 10^9 lượt chạy — lượt ngược lấy khoản tiết kiệm chín bậc đó ở đâu ra?
- Q2. Số trọng số y hệt nhau, vậy vì sao huấn luyện tốn bộ nhớ gấp nhiều lần suy luận?
- Q3. Kiểm tra gradient của bạn qua sạch mọi tầng nhưng mạng vẫn không học — phép kiểm đó theo thiết kế đã bỏ sót cái gì?
- Q4. Bạn bật torch.compile và AMP, mọi thứ chạy đúng, nhưng thời gian một *epoch* không đổi một giây nào. Vì sao đó không phải là bug?

Framework che giấu rất nhiều thứ, và phần lớn thời gian đó là điều tốt. Nhưng có một tập nhỏ những thứ mà không tự tay làm một lần thì bạn sẽ không bao giờ debug nổi. Chúng không hiện ra dưới dạng thông báo lỗi, chỉ dưới dạng một con số hơi lệch hoặc một đường cong loss hơi chậm. Câu hỏi dẫn dắt của chương là: *tập nhỏ ấy gồm những gì, và làm tới đâu thì đủ?*

Chương chia làm hai mục vì hai giai đoạn có *bản chất khác nhau*, và nhằm lẫn giữa chúng là cái bẫy chính:

- **19.1 — Giai đoạn 1, tự viết bằng NumPy** [T2][T3]. Mục tiêu là trực giác về cơ chế, không phải kỹ năng công cụ. Bạn sẽ không dùng lại một dòng NumPy nào trong việc thật. Có tiêu chí dừng rõ ràng, và khi đạt thì *dừng hẳn*.
- **19.2 — Giai đoạn 2, PyTorch** [T4]. Mục tiêu là năng suất và khả năng chẩn đoán. Nội dung sẽ cũ khi API đổi, nhưng danh sách các nút thắt mà công cụ gỡ thì không cũ.

Thứ tự phụ thuộc chỉ đi một chiều. Mục 19.2 giới thiệu autograd, buffer và cặp `train()/eval()` như *tên gọi mới* của những thứ bạn vừa tự cài bằng tay. Đọc ngược thì chúng thành từ vựng phải học thuộc, thay vì thành khái niệm đã hiểu. Ngược lại, nếu bạn đã từng viết một solver phi tuyến với Jacobian giải tích và bước so sai phân, giai đoạn 1 sẽ nhanh hơn bạn nghĩ. Nó là cùng một kỹ năng, đặt vào miền mới.

Một sợi chỉ chạy suốt cả chương: hầu hết bug ở đây là bug về *trạng thái*, không phải về công thức. Ba câu hỏi lặp lại ở cả hai giai đoạn. Cái gì được nhớ giữa lượt xuôi và lượt ngược? Cái gì được lưu vào checkpoint? Cái gì đổi hành vi giữa `train` và `eval`? Ở giai đoạn 1 chúng hiện ra dưới dạng “tôi phải cache gì”; ở giai đoạn 2 chúng mang tên `parameter` so với `buffer`.

Bản đồ chương 19 — Từ NumPy tự viết tới PyTorch thành thạo

Chương này chia làm hai mục vì hai giai đoạn có *bản chất khác nhau* và có tuổi thọ khác nhau. Nhầm lẫn giữa chúng là cái bẫy chính của cả chương.

19.1 — Giai đoạn 1: tự viết bằng NumPy [T2][T3] (*cốt lõi; làm đúng một lần rồi bỏ*). Cài linear classifier với cả hinge lẫn softmax, viết lượt ngược giải tích cho sáu tầng, kiểm tra gradient số cho từng tầng trước khi ghép, rồi ghép thành mạng n tầng với SGD + momentum. Đây *không* phải bài tập công cụ — mục tiêu là trực giác về cơ chế, thứ không hết hạn khi framework đổi. Tiêu chí dừng rất rõ: giải thích được vì sao BatchNorm khiến lượt ngược tốn thêm bộ nhớ, và vì sao dropout phải scale lúc train. Đạt rồi thì chuyển sang 19.2 và không viết NumPy nữa.

19.2 — Giai đoạn 2: PyTorch thành thạo [T4] (*cốt lõi phần ánh xạ triệu chứng → công cụ; tham khảo phần cú pháp*). Chín nút thắt và công cụ gỡ từng cái: `stride` và `view/copy`, autograd, `parameter` so với `buffer`, `DataLoader`, tự viết vòng lặp một lần, AMP và DDP và `torch.compile`, profiler, tính tắt định, và cuối cùng là kernel tùy biến (biết là đủ). Phần bèn của mục này là cây quyết định tăng tốc; phần sẽ cũ là tên API.

Vì sao chia như vậy và thứ tự bắt buộc. 19.2 giới thiệu autograd, buffer, `train()/eval()` như tên gọi mới của thứ bạn vừa tự cài trong 19.1. Đọc ngược sẽ biến chúng thành từ vựng phải học thuộc thay vì khái niệm đã hiểu.

Sợi chỉ xuyên chương. Hầu hết bug ở đây là bug về *trạng thái*, không phải công thức: cái gì được nhớ giữa lượt xuôi và lượt ngược, cái gì được lưu vào checkpoint, cái gì đổi hành vi giữa hai chế độ.

Phụ thuộc lên trên. [A/01-toan-toi-thieu/02-vi-phan-va-toi-uu](#) (quy tắc chuỗi), [B/09-thiet-ke-ham-mat-mat](#), [B/11-gioi-han-tinh-toan](#) (bắt buộc trước phần tăng tốc), [C/12-co-che-huan-luyen](#).

Toả xuống dưới. [D/20-toi-uu-va-nen-mo-hinh](#) dùng lại đúng những đòn bẩy này nhưng cho **suy luận** (*inference*); [D/21-inference-va-trien-khai](#) dùng lại kỹ năng phân biệt “lệch do bug” với “lệch do số học”; [D/22-tai-lap-va-mlops](#) mở rộng mục về tính tất định thành một chương.

I 19.1 Giai đoạn 1: tự viết bằng NumPy (Building It Yourself in NumPy)

PHỤ THUỘC [A/01-toan-toi-thieu/02-vi-phan-va-toi-uu](#) (quy tắc chuỗi, Jacobian); [B/09-thiet-ke-ham-mat-mat](#) (*hinge loss* và *cross-entropy*); [C/12-co-che-huan-luyen/01-co-hoc-cua-gradient](#) và [02-lam-cho-training-chay-duoc](#) (BatchNorm, dropout — ở đây ta cài lại chúng bằng tay).

TÓM TẮT

Mục này là bài tập một lần: tự cài **lan truyền ngược** (*backpropagation*) bằng NumPy cho sáu tầng, kiểm tra từng tầng bằng **kiểm tra gradient số** (*numerical gradient check*), rồi ghép thành một mạng chạy được. Nó không phải bài tập công cụ mà là bài tập trực giác tầng [T2][T3]: đọc xong bạn giải thích được vì sao bộ nhớ huấn luyện lớn hơn bộ nhớ **suy luận** (*inference*) nhiều lần, vì sao quên `eval()` lại đổi kết quả, và bạn tự chẩn đoán được một gradient sai thay vì đoán mò. Nếu chỉ nhớ một điều: không ghép một tầng chưa qua kiểm tra gradient — vì kiểm từng tầng giữ không gian nghi vấn ở đúng một phần tử, còn ghép trước kiểm sau làm nó phình lên thành số tầng nhân số tương tác.

KIẾN THỨC CẦN CÓ TRƯỚC

- **hàm mất mát (loss function)** — hàm biến đầu ra của mô hình và đáp án đúng thành một số thực đo mức sai. Huấn luyện là cực tiểu hoá số đó.
- **gradient và quy tắc chuỗi** — gradient là vector đạo hàm riêng của loss theo từng tham số. Vì mạng là hàm hợp của nhiều tầng, đạo hàm được tính bằng quy tắc chuỗi — nhân dồn các đạo hàm cục bộ từ đầu ra ngược về đầu vào.
- **kích hoạt (activation)** — đầu ra trung gian của một tầng. Lượng bộ nhớ huấn luyện phần lớn là chỗ chứa các kích hoạt này, vì lượt ngược cần đọc lại chúng.
- **ReLU, dropout, BatchNorm** — ReLU là hàm phi tuyến $\max(0, x)$; dropout tắt ngẫu nhiên một phần kích hoạt lúc huấn luyện để chống *overfitting*; BatchNorm chuẩn hoá kích hoạt theo thống kê của *batch* để việc huấn luyện ổn định hơn. Cả ba đều có hành vi *khác nhau* giữa lúc train và lúc test.
- **batch** — nhóm mẫu được xử lý cùng lúc trong một bước cập nhật. Kích thước batch ảnh hưởng tới bộ nhớ, tốc độ, và — với BatchNorm — tới cả kết quả.
- **SGD và momentum** — *stochastic gradient descent* cập nhật trọng số theo gradient của một batch; *momentum* cộng dồn hướng đi của các bước trước để giảm dao động và vượt qua vùng phẳng.
- **float32 so với float64** — hai độ chính xác dấu phẩy động. float32 có epsilon máy $\approx 1,2 \times 10^{-7}$, float64 khoảng $2,2 \times 10^{-16}$; khác biệt này quyết định việc kiểm tra gradient số có nói thật hay không.

A. Đặt vấn đề

Framework che giấu rất nhiều thứ, và phần lớn thời gian đó là điều tốt — không ai muốn viết lại `conv2d` mỗi sáng. Nhưng có một tập nhỏ những thứ mà nếu không tự tay làm một lần thì bạn sẽ không bao giờ debug nổi, vì triệu chứng của chúng không bao giờ xuất hiện dưới dạng một thông báo lỗi. Mạng vẫn chạy, loss vẫn giảm, chỉ là giảm chậm hơn đáng ra, hoặc kết quả lúc `eval()` lệch khỏi lúc train một cách không giải thích được. Không có exception nào để bắt, không có *stack trace* nào để đọc.

Cần nói rõ ngay từ đầu về *loại* bài tập này, vì hiểu sai loại là cách phổ biến nhất để lãng phí một tháng:

- **Đây không phải bài tập tăng bốn** — không phải bài tập công cụ. Bạn sẽ không dùng lại một dòng NumPy nào trong công việc thật, và đó là chuyện bình thường.
- **Đây là bài tập tăng hai và ba** [T2][T3]: mục tiêu là *trực giác về cơ chế*, thứ không hết hạn khi framework đổi phiên bản, khi PyTorch bị thay bằng cái khác, hay khi kiến trúc thời thượng đổi tên.
- **Làm một lần, đạt tiêu chí dừng, rồi không quay lại.** Cái bấy quen thuộc của người học theo CS231n là yêu quá trình này và tiếp tục viết NumPy sau khi nó đã hết tác dụng — lúc đó nó thành một sở thích thú công chứ không còn là học.

Học xong thì làm được gì mà trước đó không làm được? Bạn nhìn một đường cong loss bất thường và khoanh vùng được ngay nó là lỗi gradient, lỗi trạng thái hay lỗi dữ liệu; bạn đọc thông báo hết bộ nhớ GPU và biết tầng nào đang giữ activation; và bạn viết được một `autograd.Function` tùy biến trong PyTorch mà không phải sao chép mẫu từ diễn đàn.

Người đọc đã viết C++ và CUDA có một lợi thế ở đây, và cũng có một cái bấy riêng. Lợi thế là bạn đã quen với việc một hệ thống số học có thể sai ở mức không nhìn thấy được. Cái bấy là bạn dễ bỏ qua bước kiểm tra gradient vì thấy nó buồn tẻ. Đúng như một người viết solver phi tuyến để bỏ bước so Jacobian giải tích với Jacobian sai phân, rồi mất ba ngày cho một dấu trừ.

TRỰC GIÁC

Bạn đã làm chính xác bài tập này rồi, chỉ là ở lĩnh vực khác. Khi thêm một loại *residual* mới vào một bài toán bình phương tối thiểu phi tuyến, bạn viết Jacobian giải tích để solver chạy nhanh, rồi *trước khi tin nó*, bạn so với Jacobian tính bằng sai phân hữu hạn trên vài điểm ngẫu nhiên. Nếu sai số tương đối nhỏ thì dùng; nếu không thì bạn biết chắc lỗi nằm trong đúng khối vừa viết, chứ không phải ở solver.

Lan truyền ngược là cùng một việc, chỉ khác quy mô: một mạng n tầng là một hàm hợp, **lượt ngược** (*backward pass*) là chế độ nghịch của quy tắc chuỗi — đúng cái mà cộng đồng tối ưu gọi là *adjoint method* — và kiểm tra gradient là bước so Jacobian. Giai đoạn 1 vì thế không dạy bạn khái niệm mới; nó chuyển một kỹ năng bạn đã có sang miền mới, ở quy mô nhỏ tới mức mọi con số còn nhìn được bằng mắt.

B. Bước 1 — linear classifier với hai hàm mất mát

Bắt đầu ở chỗ đơn giản nhất mà vẫn còn thú vị: một phân loại tuyến tính $s = Wx + b$, cài cả *hinge loss* kiểu SVM lẫn *softmax loss* (*softmax cross-entropy loss*), viết cả **lượt xuôi** (*forward pass*) lẫn lượt ngược cho từng cái. Điều bạn sẽ thấy tận mắt — chứ không phải đọc thấy — là hai hàm này có *hành vi gradient khác nhau về bản chất*, không chỉ khác về công thức.

Ví dụ nhỏ nhất. Một mẫu, ba lớp, điểm số $s = (2,0; 0,5; -1,0)$, nhãn đúng là lớp 0.

- **Hinge loss**, lẽ $\Delta = 1$:

$$L = \sum_{j \neq y} \max(0, s_j - s_y + \Delta) = \max(0, -0,5) + \max(0, -2,0) = 0.$$

Loss bằng không, và *gradient cũng bằng đúng không* — mọi phần tử của $\partial L / \partial s$ là 0. Mẫu này đã thoả lẽ; nó không còn nói gì với mô hình nữa.

- **Softmax loss**, cùng bộ điểm số: $e^{2,0} = 7,389$, $e^{0,5} = 1,649$, $e^{-1,0} = 0,368$, tổng 9,406, nên $p = (0,786; 0,175; 0,039)$ và $L = -\ln 0,786 = 0,241$. Gradient có dạng đẹp đến mức đáng nhớ,

$$\partial L / \partial s = p - e_y.$$

$$\partial L / \partial s = (-0,214; +0,175; +0,039).$$

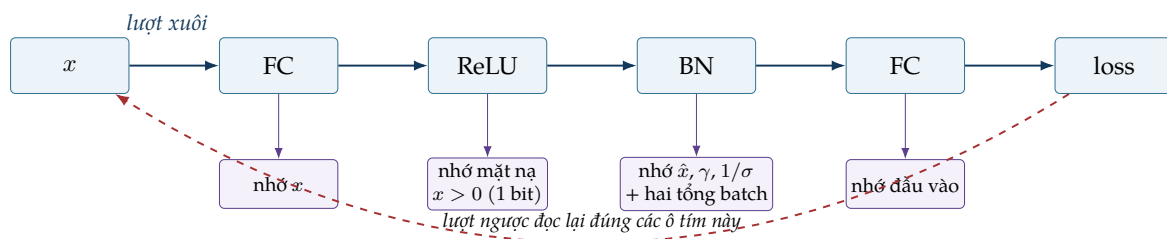
Ý nghĩa của công thức $p - e_y$: gradient chính là “sai lệch giữa niềm tin hiện tại và sự thật”. Không phần tử nào bằng không, nên softmax *luôn* còn đây — nó không có khái niệm “đủ tốt rồi”.

Khác biệt này giải thích rất nhiều hành vi thực nghiệm. Hinge tạo gradient thừa nên ổn định và ít bị chi phối bởi vài mẫu dễ, nhưng nó dừng học ngay khi lẽ được thoả, kể cả khi bạn còn muốn mô hình tự tin hơn. Softmax luôn có gradient nên tiếp tục tinh chỉnh, nhưng chính vì thế nó nhạy với nhãn sai — một mẫu bị gán nhầm sẽ bơm gradient vào suốt quá trình huấn luyện, không bao giờ tắt. Ai đã đọc [B/09-thiet-ke-ham-mat-mat](#) sẽ nhận ra đây là cùng trục lập luận đã dẫn tới *focal loss*.

↔ cùng nguyên lý với [B/09](#) — cùng trục lập luận đã dẫn tới focal loss — dời trọng số gradient khỏi những mẫu không còn dạy được gì.

C. Bước 2 — lượt ngược giải tích cho sáu tầng

Bước tiếp theo là viết lượt ngược giải tích cho *fully-connected*, ReLU, dropout, BatchNorm, *convolution* và *max-pooling*. Giá trị của bước này không nằm ở các công thức — chúng có sẵn ở mọi nơi — mà ở chỗ nó buộc bạn trả lời cho từng tầng một câu hỏi mà framework không bao giờ hỏi bạn: *backward của tầng này cần nhớ gì từ lượt xuôi?* Câu trả lời là toàn bộ lý do bộ nhớ huấn luyện lớn hơn bộ nhớ suy luận nhiều lần. Hình 19.1 vẽ đúng cơ chế đó.



Hình 19.1. Vì sao huấn luyện tốn bộ nhớ hơn suy luận: mỗi tầng phải *ghi lại* một phần trạng thái lúc lượt xuôi để lượt ngược đọc lại. Ô tím là thứ không giải phóng được cho tới khi lượt ngược chạy qua. ReLU rẻ (một bit mỗi phần tử), BatchNorm đắt nhất vì ngoài $\hat{x}$ nó còn ràng buộc cả *batch*.

Bảng 19.1. Sáu tầng, và thứ mà lượt ngược của mỗi tầng buộc phải giữ lại từ lượt xuôi.

Tầng	Backward cần nhớ	Chi phí	Chỗ hay sai
Fully-connected	Đầu vào x (cho $\partial L / \partial W$)	Bằng kích thước đầu vào	Quên chuyển vị; quên cộng dồn $\partial L / \partial b$ theo batch
ReLU	Chỉ mặt nạ $x > 0$ (1 bit mỗi phần tử)	Rất rẻ	Xử lý $x = 0$ tùy tiện — chính là điểm gây làm kiểm tra gradient báo động giả
Dropout	Mặt nạ đã dùng lúc lượt xuôi	Bằng kích thước đầu ra	Không dùng <i>inverted dropout</i> , hoặc sinh mặt nạ mới ở lượt ngược
BatchNorm	$\hat{x}, \gamma, 1/\sigma$ — và lượt ngược khớp cả <i>batch</i>	Đắt nhất trong sáu tầng	Coi lượt ngược là phép theo từng phần tử
Convolution	Ma trận <i>im2col</i> hoặc đầu vào để dựng lại nó	Rất đắt nếu vật chất hoá im2col	<i>col2im</i> phải cộng dồn vùng chồng lấn, không phải ghi đè
Max-pool	Chỉ số <i>argmax</i> của mỗi cửa sổ	Rẻ	Xử lý trường hợp bằng nhau khác giữa hai cài đặt

Ba dòng trong Bảng 19.1 đáng dừng lại.

BatchNorm – lượt ngược không phải phép theo từng phần tử

$$\frac{\partial L}{\partial x_i} = \frac{\gamma}{N\sigma} \left[N \frac{\partial L}{\partial y_i} - \sum_j \frac{\partial L}{\partial y_j} - \hat{x}_i \sum_j \frac{\partial L}{\partial y_j} \hat{x}_j \right].$$

Công thức này nói điều gì? Rằng gradient của mẫu i bị trừ đi hai đại lượng tính trên *toàn batch* — một cho trung bình, một cho phương sai. Nói cách khác: BatchNorm không cho phép một mẫu tự do đẩy gradient của nó; mọi mẫu trong batch cùng thoả hiệp. Ba hệ quả bạn sẽ gặp lại suốt sự nghiệp:

- **Bộ nhớ lượt ngược tăng** vì phải giữ cả $\hat{x}$ lẫn các thống kê, và không giải phóng được theo từng mẫu.
- **Batch nhỏ làm gradient nhiễu**, vì ước lượng σ nhiễu — lý do GroupNorm và LayerNorm tồn tại [110], [111].
- **Mô hình huấn luyện với BatchNorm là hàm của cả batch, không phải của từng mẫu** — lý do sâu xa khiến `train()` và `eval()` cho kết quả khác nhau.

Dropout – vì sao phải scale lúc train

Cài theo kiểu *inverted dropout*: lúc train, giữ lại phần tử với xác suất $1 - p$ rồi chia cho $1 - p$; lúc test không làm gì cả. Lý do là bảo toàn kỳ vọng — nếu $\tilde{y} = m \odot y / (1 - p)$ với $m \sim \text{Bernoulli}(1 - p)$ thì $\mathbb{E}[\tilde{y}] = y$, nên đường suy luận của các tầng sau không bị dịch khi ta bật tắt dropout. Cách còn lại (nhân $1 - p$ lúc test) tương đương về toán học nhưng dở về kỹ thuật, vì nó đặt một bước bắt buộc vào đường suy luận — và mọi đường inference đều có ngày bị viết lại, bị *export*, bị port sang C++, và bước ấy sẽ rơi mất. Một điểm trung thực cần ghi: *inverted dropout* chỉ khớp *mô-men bậc một*; phương sai lúc train vẫn lớn hơn lúc test, và đó là một phần lý do dropout tương tác xấu với BatchNorm [129].

Convolution – chỗ nào sinh ra bất định trên GPU

Cách cài dễ hiểu nhất là *im2col*: trải các cửa sổ thành cột rồi gọi một phép nhân ma trận, nên $\partial L / \partial W = \partial L / \partial Y \cdot X_{\text{col}}^\top$ và $\partial L / \partial X_{\text{col}} = W^\top \cdot \partial L / \partial Y$. Bước cuối — *col2im* — phải cộng dồn gradient vào những vị trí mà các cửa sổ chồng lên nhau. Ghi nhớ chi tiết này. Phép cộng dồn tán xạ đó chính là chỗ các cài đặt GPU dùng `atomicAdd`. Cộng số thực dấu phẩy động không có tính kết hợp, nên thứ tự luồng hoàn thành quyết định bit cuối. Đây là một trong những nguồn bất định chính của huấn luyện trên GPU, chủ đề của [D/22-tai-lap-va-mlops](#). Nó cùng một họ nguyên nhân với các lỗi bất định do thứ tự con trỏ mà bạn đã gặp trong SLAM.

↪ **tiền đề D/22** — cộng số thực không có tính kết hợp, nên mọi phép cộng dồn tán xạ trên GPU là một nguồn bất định — cùng họ nguyên nhân với thứ tự con trỏ trong một hệ thống C++ đa luồng.

D. Bước 3 – kiểm tra gradient, và kỷ luật đi kèm

Mọi công thức ở trên đều có thể sai, và sai một cách im lặng. Cách duy nhất phát hiện là so với sai phân trung tâm:

$$\frac{\partial f}{\partial x_k} \approx \frac{f(x + he_k) - f(x - he_k)}{2h},$$

Ta nhiễu *một* toạ độ về hai phía và xem loss thay đổi bao nhiêu. Rồi tính sai số tương đối $|g_{\text{số}} - g_{\text{giải tích}}| / \max(|g_{\text{số}}|, |g_{\text{giải tích}}|)$.

```
def grad_check(f, x, analytic, h=1e-5, n=20):
```



```
# f: hàm tra về scalar loss; analytic: gradient đã tính giai tích
idx = np.random.choice(x.size, size=n, replace=False)
for k in idx:
    old = x.flat[k]
    x.flat[k] = old + h; fp = f(x)
    x.flat[k] = old - h; fm = f(x)
    x.flat[k] = old
    num = (fp - fm) / (2 * h)
    ana = analytic.flat[k]
    rel = abs(num - ana) / max(1e-12, abs(num), abs(ana))
print(k, num, ana, rel)
```

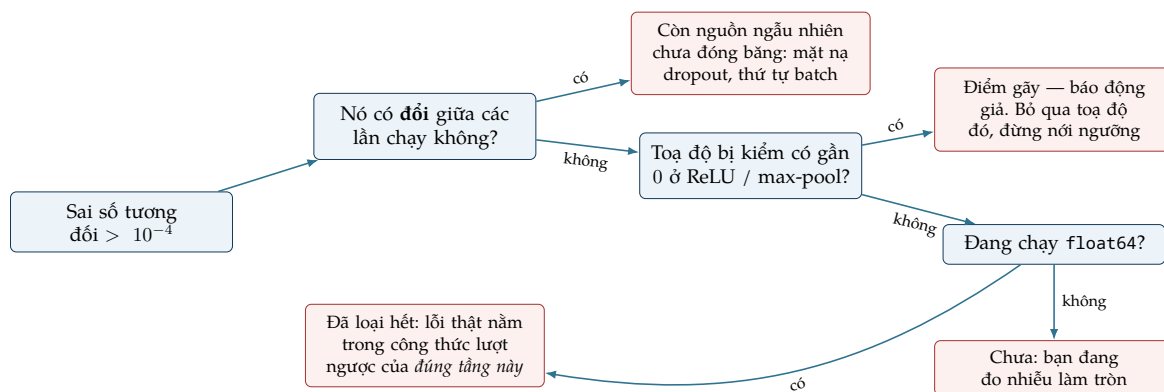
Ngưỡng diễn giải, theo *kinh nghiệm thực hành* chứ không phải kết quả lý thuyết:

- $< 10^{-7}$ — yên tâm.
- 10^{-7} đến 10^{-4} — chấp nhận được với mạng sâu có nhiều điểm gãy, đáng nghi với một tầng đơn giản.
- $> 10^{-4}$ — gần như chắc chắn có lỗi. $> 10^{-2}$ — chắc chắn có lỗi.

Ba chi tiết quyết định việc kiểm tra gradient nói thật hay nói dối:

1. **Phải dùng float64.** Sai phân trung tâm lấy hiệu hai số gần bằng nhau; với float32 (epsilon máy $\approx 1,2 \times 10^{-7}$) và $h = 10^{-5}$, bạn mất phần lớn chữ số có nghĩa vì mất chữ số có nghĩa khi trừ hai số gần bằng nhau — kết quả là bạn đang đo nhiễu làm tròn chứ không đo đạo hàm.
2. **Kiểm ở vài chục tọa độ ngẫu nhiên**, không phải toàn bộ tensor — vừa đủ tin cậy, vừa đủ nhanh để bạn thực sự chạy nó.
3. **Mọi nguồn ngẫu nhiên phải bị đóng băng giữa hai lần gọi f** ; nếu không, bạn đang so đạo hàm của hai hàm khác nhau.

Khi con số báo động, Hình 19.2 là thứ tự chẩn đoán tiết kiệm nhất.



Hình 19.2. Thứ tự chẩn đoán khi kiểm tra gradient báo động. Ba nhánh đầu là các nguyên nhân *giả*; chỉ khi loại hết chúng thì kết luận “công thức lượt ngược sai” mới có giá trị.

Kỷ luật đi kèm chỉ có một câu, và nó là phần đáng giá nhất của cả mục: *không ghép một tầng chưa qua kiểm tra*. Lý do thuần túy là số học tổ hợp. Ghép sáu tầng rồi mới phát hiện gradient sai thì không gian nghi vấn là sáu tầng cộng mọi tương tác giữa chúng; kiểm từng tầng ngay khi viết xong thì không gian nghi vấn luôn có đúng một phần tử. Đây chính xác là lập luận bạn đã dùng khi thêm từng loại ràng buộc vào một **đồ thị nhân tố** (*factor graph*) và kiểm tra sau mỗi lần thêm.

↪ **trường hợp riêng của A/01** — lan truyền ngược là chế độ nghịch của quy tắc chuỗi, nên kiểm tra gradient ở đây đúng là bước so Jacobian giải tích với Jacobian sai phân trong tối ưu phi tuyến.

E. Bước 4 — ghép thành mạng và tự viết vòng lặp

Bước cuối là ghép các tầng đã kiểm thành một mạng n tầng và tự viết vòng lặp huấn luyện với SGD kèm *momentum*. Ở đây bạn gặp một loại lỗi mới mà kiểm từng tầng không bắt được: lỗi về *trạng thái*.

- **Mặt nạ dropout** phải giống nhau giữa lượt xuôi và lượt ngược của cùng một bước, nhưng khác nhau giữa các bước.
- **BatchNorm** phải cập nhật thống kê chạy ở chế độ train và dùng thống kê ấy ở chế độ test.
- **Bộ đệm momentum** phải được cập phát một lần, không phải mỗi bước.

Ba lỗi này là ba phiên bản của cùng một câu hỏi: *cái gì là tham số, cái gì là trạng thái, cái gì là tạm*. Chính câu hỏi đó quay lại nguyên vẹn ở mục sau, dưới cái tên parameter so với buffer.

F. Tiêu chí dừng, giới hạn và trường hợp hỏng

Tiêu chí dừng nên phát biểu bằng năng lực giải thích, không bằng số bài đã làm. Bạn dừng được khi giải thích trôi chảy hai điều. Thứ nhất, vì sao BatchNorm khiến lượt ngược tốn thêm bộ nhớ: hai tổng trên toàn batch, nên không giải phóng được theo từng mẫu. Thứ hai, vì sao dropout phải scale lúc train chứ không lúc test: để bảo toàn kỳ vọng mà không đặt bước bắt buộc nào lên đường suy luận. Đạt rồi thì chuyển sang PyTorch và không viết NumPy nữa.

Giới hạn của cả giai đoạn cần nói thẳng:

- **Mọi trực giác về hiệu năng rút ra từ đây đều sai.** NumPy thuần chậm hơn nhiều bậc so với kernel thật, nên bạn chỉ chạy được ở quy mô đồ chơi; phần hiệu năng phải học lại ở [B/11-gioi-han-tinh-toan](#).
- **Không học được gì về ổn định số học ở độ chính xác thấp.** *Log-sum-exp*, *loss scaling*, tràn số fp16 chỉ xuất hiện khi bạn rời float64.
- **Kiểm tra gradient kiểm tính nhất quán, không kiểm tính đúng.** Một tầng cài sai công thức lượt xuôi nhưng có lượt ngược khớp với chính nó sẽ qua kiểm một cách hoàn hảo.

CẠM BÃY

Bốn cách làm cho kiểm tra gradient qua trong khi mã vẫn sai, xếp theo mức độ hay gặp. **Một:** quên cố định mặt nạ dropout, khiến $f(x + h)$ và $f(x - h)$ là hai hàm khác nhau — triệu chứng là sai số lớn và *nhảy loạn giữa các lần chạy*. **Hai:** để BatchNorm ở chế độ train trong lúc kiểm, khiến hàm bị kiểm phụ thuộc thống kê batch và thay đổi khi bạn nhiễu một toạ độ. **Ba:** kiểm ở đúng điểm gãy của ReLU hoặc max-pool, tức x rất gần 0. Ở đó sai phân hai phía đo qua một chỗ hàm không khả vi, nên báo động là giả. Cách xử lý đúng là bỏ qua toạ độ đó, không phải nới ngưỡng. **Bốn:** chạy ở float32 rồi kết luận rằng “sai số 10^{-3} là bình thường”; không, đó là dấu hiệu bạn đang đo nhiễu làm tròn.

CẦN NHỚ

Không ghép một tầng chưa qua kiểm tra gradient. Lý do không phải sự cẩn thận mà là tổ hợp: kiểm từng tầng giữ không gian nghi vấn ở đúng một phần tử; ghép trước rồi kiểm sau làm nó phình lên thành số tầng nhân với số tương tác. Và nhớ rằng kiểm tra gradient chỉ chứng minh lượt xuôi và lượt ngược *khớp nhau*, không chứng minh lượt xuôi đúng. [T2]

G. Kết nối

Mục này là tiền đề bắt buộc của [D/19/02-pytorch-thanh-thao](#). Ba khái niệm bạn vừa cài bằng tay — đồ thị tính toán, thứ phải nhớ giữa hai lượt, và tham số so với trạng thái — sẽ xuất hiện lại dưới dạng autograd, nn.Module và cặp `train()/eval()`. Bạn sẽ hiểu chúng như tên gọi mới của thứ đã biết. Nó là trường hợp riêng của [A/01-toan-toi-thieu/02-vi-phan-va-toi-uu](#): backpropagation chỉ là chế độ nghịch của quy tắc chuỗi áp cho hàm hợp [8]. Nó bổ sung cho [C/12-co-che-huan-luyen](#): chương kia nói BatchNorm và dropout *làm gì* cho việc huấn luyện, mục này nói chúng *tồn gì* và hỏng ở đâu. Và nó gieo trước một hạt cho [D/22-tai-lap-va-mlops](#): phép cộng dồn tán xạ trong *col2im* là nguồn bất định GPU mà bạn sẽ gặp lại ở đó.

I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao hinge loss cho gradient bằng đúng không ở một mẫu đã thoả lề, còn softmax thì không bao giờ? Hệ quả nào của điều đó xuất hiện khi tập dữ liệu có nhãn sai?
2. Hai tổng trên toàn batch trong lượt ngược của BatchNorm là gì, và chúng buộc ta giữ thêm gì trong bộ nhớ?
3. Vì sao *inverted dropout* tốt hơn về mặt kỹ thuật dù hai cách scale tương đương về toán học? Và nó *không* bảo toàn thứ gì?
4. Kiểm tra gradient cho sai số tương đối 3×10^{-3} và con số đó đổi mỗi lần chạy. Nguyên nhân khả dĩ nhất là gì, và nếu con số ổn định thì danh sách nghi vấn đổi thế nào?
5. Kiểm tra gradient qua hết mọi tầng nhưng mạng vẫn không học — hai loại lỗi nào mà kiểm tra gradient theo thiết kế không thể bắt được?
6. Chỗ nào trong lượt ngược của convolution là phép cộng dồn tán xạ, và vì sao chi tiết đó sẽ quay lại ám bạn khi chạy trên GPU?

I 19.2 Giai đoạn 2: PyTorch thành thạo (Becoming Fluent in PyTorch)

PHỤ THUỘC [D/19/01-tu-viet-bang-numpy](#) (bắt buộc — mọi khái niệm ở đây là tên gọi mới của thứ bạn đã cài bằng tay); [B/11-gioi-han-tinh-toan](#) (*roofline*, memory-bound so với compute-bound — không có nó thì mục về tăng tốc chỉ là danh sách mọ).

TÓM TẮT

Mục này đi theo mạch vấn đề → công cụ: mỗi tính năng của PyTorch được giới thiệu qua đúng cái bug hoặc cái nút thắt đã sinh ra nó, chứ không theo thứ tự tài liệu API. Đọc xong bạn đọc được một *profile* và biết nên bật *AMP*, sửa *DataLoader*, hay gọi `torch.compile` — thay vì bật cả ba rồi không biết cái nào có tác dụng. Nếu chỉ nhớ một điều: mọi công cụ tăng tốc ở đây chỉ giúp khi bạn đang nghẽn ở đúng chỗ nó gỡ, nên thứ tự bắt buộc luôn là *đo trước, sửa sau*.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tensor** — mảng nhiều chiều kèm kiểu dữ liệu (dtype) và thiết bị (device, CPU hay GPU). Trong bộ nhớ nó chỉ là một dải liên tục cộng với bộ *stride* nói mỗi chiều nhảy bao nhiêu phần tử.
- **đồ thị tính toán (computation graph)** — bản ghi các phép toán đã thực hiện ở lượt xuôi, để lượt ngược biết đi ngược theo đường nào. PyTorch dựng nó *động*, tức mỗi lần lượt xuôi là một đồ thị mới.
- **epoch, batch, DataLoader** — một *epoch* là một lượt đi hết dữ liệu; *batch* là nhóm mẫu xử lý cùng lúc; *DataLoader* là bộ máy đọc đĩa, giải mã, augment và gom mẫu thành batch — thường chạy trên CPU và thường là chỗ nghẽn.

- **độ chính xác hỗn hợp (mixed precision)** — tính phần lớn phép toán ở 16 bit trong khi giữ trọng số và một số phép cộng dồn ở 32 bit. Mục đích là giảm số byte phải nạp qua bộ nhớ, không phải giảm số phép tính.
- **memory-bound so với compute-bound** — một kernel memory-bound bị chặn bởi băng thông bộ nhớ (làm ít phép tính trên mỗi byte nạp về), compute-bound thì bị chặn bởi năng lực tính toán. Hầu hết mạng thị giác là memory-bound — lý do AMP có tác dụng.
- **profiler** — công cụ đo thời gian thực sự của từng kernel và từng giai đoạn, thay vì đoán. Trong PyTorch là `torch.profiler`; ở tầng dưới là `Nsight`.
- **checkpoint và state_dict** — `state_dict` là từ điển chứa toàn bộ trọng số và *buffer* của mô hình; lưu nó ra đĩa được gọi là checkpoint. Nó *không* chứa mã nguồn, nên vẫn cần đúng phiên bản code để nạp lại.
- **tính tất định (determinism)** — tính chất chạy lại cùng đầu vào cho ra cùng đầu ra tới từng bit. Trên GPU nó không mặc định, và ép nó thì phải trả giá bằng tốc độ.

A. Đặt vấn đề

Sau giai đoạn 1 bạn biết cơ chế nhưng chưa có công cụ. PyTorch cung cấp công cụ, nhưng nó cung cấp *rất nhiều* công cụ, và tài liệu chính thức tổ chức theo API chứ không theo vấn đề. Hệ quả là kiểu hiểu biết phổ biến nhất: biết gọi `amp.autocast` mà không biết nó giúp gì; bật `torch.compile` rồi thấy không nhanh hơn và kết luận rằng “compile không ăn thua”.

Mục này đảo lại thứ tự trình bày. Mỗi công cụ xuất hiện sau đúng cái vấn đề đã sinh ra nó, vì trong nghề, thứ bạn thực sự cần nhớ không phải tên hàm — tra được trong ba mươi giây — mà là *ánh xạ từ triệu chứng sang công cụ*. Cần nói rõ đây là nội dung tầng bốn [T4]: tên API sẽ đổi, và điều đó bình thường; cái không đổi là danh sách các nút thắt.

↪ **tiền đề D/19/01** — `autograd`, `buffer` và cặp `train/eval` chỉ là tên gọi mới của đồ thị tính toán, trạng thái và công tắc chế độ mà bạn đã tự cài bằng tay.

Học xong thì làm được gì? Bạn cầm một vòng lặp huấn luyện chậm gấp ba lần đáng ra, chạy *profiler*, và chỉ đúng một dòng cần sửa — đây chính là kỹ năng bạn đã có với `nsys` và `perf` ở phía C++/CUDA, lần này áp cho một *runtime* có thêm một tầng trừu tượng ở giữa.

TRỰC GIÁC

Autograd là một cuốn băng ghi âm chỉ dùng được một lần. Mỗi phép toán trên *tensor* có `requires_grad=True` ghi thêm một mục vào băng; `backward()` tua ngược băng, tính gradient, rồi *xoá* băng đi. Từ hình ảnh đó suy ra ba điều mà người ta thường phải học bằng cách vấp: gọi `backward()` hai lần thì lỗi vì băng đã bị xoá (trừ khi bạn xin `retain_graph`); giữ một *tensor* còn dính băng trong một danh sách log nghĩa là giữ luôn toàn bộ đoạn băng phía sau nó, nên bộ nhớ phình dần suốt *epoch*; và `detach()` chỉ đơn giản là cắt băng ở đúng chỗ đó.

B. Mạch vấn đề → công cụ

I Chín nút thắt, theo thứ tự bạn sẽ gặp

1 Vấn đề

bug im lặng về bộ nhớ và tính đúng đắn.

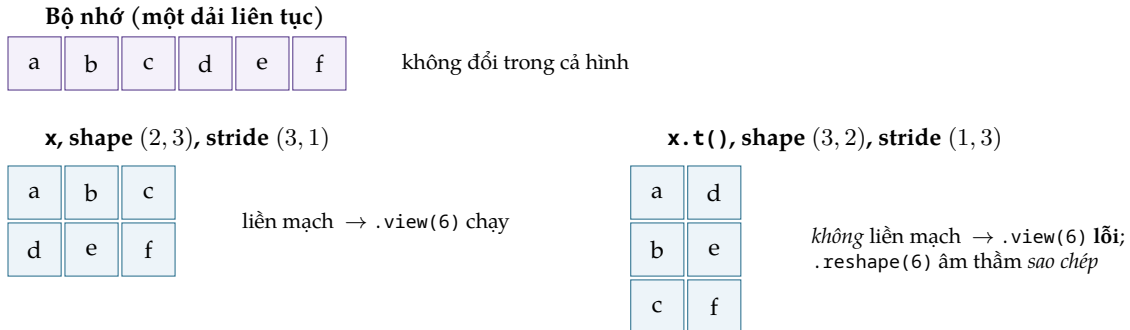
Giải pháp

hiểu *tensor* ở mức bố cục bộ nhớ: *dtype*, *device*, *stride* (mỗi bước theo một chiều nhảy bao nhiêu phần tử trong bộ nhớ), và khác biệt *view* với *copy*.

- Hệ quả `.view()` thất bại trên tensor không liên mạch, còn `.reshape()` âm thầm sao chép — một `.contiguous()` đặt sai chỗ có thể nhân đôi bộ nhớ mà không báo gì.
- 2 Vấn đề** không biết đồ thị được dựng và giải phóng lúc nào.
- Giải pháp** *autograd*: `backward`, `no_grad`, `detach`, và `autograd.Function` tùy biến khi bạn có công thức gradient riêng.
- Vấn đề mới** bug kinh điển: giữ tensor còn đồ thị trong một list để logging, làm rò bộ nhớ suốt epoch.
- 3 Vấn đề** trạng thái mô hình không chỉ là trọng số.
- Giải pháp** `nn.Module`: phân biệt *parameter* với *buffer*, hiểu `state_dict`, dùng *hook*, và cặp `train()/eval()`.
- Giả định** bạn nhớ rằng thống kê chạy của BatchNorm là *buffer* chứ không phải tham số — nó được lưu, được chuyển device, nhưng không được optimizer đụng tới.
- 4 Vấn đề** GPU đói dữ liệu.
- Giải pháp** `Dataset/DataLoader/sampler`, `num_workers`, `pin_memory`, *prefetch*, `collate_fn`; và một tầng *augmentation* (*alumentations*, `transforms v2`, hoặc DALI khi CPU không kịp).
- Hệ quả đây là nút thắt phổ biến nhất và cũng bị bỏ qua nhiều nhất — xem [B/11-gioi-han-tinh-toan](#).
- 5 Vấn đề** framework giấu quá nhiều.
- Giải pháp** tự viết vòng lặp huấn luyện ít nhất một lần trước khi dùng Lightning hay HF Trainer.
- Giá phải trả** vài giờ.
- Hệ quả sau đó dùng framework là hợp lý, vì bạn đã biết nó đang làm gì và biết chỗ để chọc vào khi nó làm sai.
- 6 Vấn đề** bộ nhớ và tốc độ.
- Giải pháp** *AMP* (tính phần lớn phép toán ở 16 bit, giữ trọng số ở 32 bit) với GradScaler; *DDP* (chia batch cho nhiều GPU, mỗi GPU giữ một bản sao mô hình) qua `torchrun`, hoặc FSDP khi một GPU không chứa nổi mô hình [122]; và `torch.compile` để hợp nhất kernel và giảm chi phí điều phối.
- 7 Vấn đề** đo trước khi tối ưu.
- Giải pháp** `torch.profiler`, `Nsight`, `trace viewer`.
- Giả định** bạn tin số đo hơn tin trực giác — nguyên tắc đã có ở [B/11-gioi-han-tinh-toan](#).
- 8 Vấn đề** kết quả không lặp lại.
- Giải pháp** *seed*, `torch.use_deterministic_algorithms`, tắt `cudnn.benchmark` — kèm cái giá về tốc độ.
- Giá phải trả** đáng kể, và *vẫn không* đảm bảo tái lập giữa hai máy khác GPU. Toàn bộ chủ đề này thuộc [D/22-tai-lap-va-mlops](#).
- Hệ quả biết là đủ: viết kernel riêng bằng CUDA hoặc Triton. Cần khi bạn có một mẫu truy cập bộ nhớ đặc thù mà không kernel nào khớp — hiếm, và thường có cách rẻ hơn.

C. Stride, view và copy — bố cục bộ nhớ là thứ thật

Đây là chỗ người từ C++ có lợi thế lớn nhất, vì *stride* chính là thứ bạn vẫn dùng khi viết kernel: một tensor là một con trỏ, một *shape*, và một bộ *stride* nói mỗi bước theo mỗi chiều nhảy bao nhiêu phần tử. Hình 19.3 cho thấy vì sao chuyển vị không tốn gì và vì sao `.view()` sau đó lại thất bại.



Hình 19.3. Chuyển vị chỉ đổi *stride*, không chạm vào bộ nhớ — nên nó gần như miễn phí. Nhưng `.view()` đòi hỏi các phần tử nằm đúng thứ tự trong bộ nhớ, nên nó thất bại; `.reshape()` thì không thất bại mà lặng lẽ cấp phát một bản sao. Chính sự “lặng lẽ” đó là nguồn của các lần bùng nổ bộ nhớ không giải thích được.

Quy tắc thực hành rút ra: dùng `.view()` khi bạn *muốn* chương trình báo lỗi nếu bố cục không như bạn nghĩ, và dùng `.reshape()` chỉ khi bạn đã chấp nhận cái giá sao chép. Đây là cùng một triết lý với `at()` so với `operator[]` trong C++: chọn phiên bản ồn ào ở chỗ bạn chưa chắc.

D. Parameter, buffer và cặp train/eval

Ở giai đoạn 1 bạn đã phải tự trả lời “cái gì là tham số, cái gì là trạng thái”. PyTorch đặt tên cho câu trả lời đó:

- **Parameter** — được optimizer cập nhật, có gradient, nằm trong `state_dict`. Ví dụ: γ, β của BatchNorm.
- **Buffer** — *không* có gradient và optimizer không đụng tới, nhưng vẫn nằm trong `state_dict` và vẫn theo mô hình khi bạn gọi `.to(device)`. Ví dụ: `running_mean`, `running_var` của BatchNorm.
- **Biến tạm** — không lưu, không chuyển; nếu bạn cần nó tồn tại qua một lần lưu checkpoint thì bạn đã chọn sai loại.

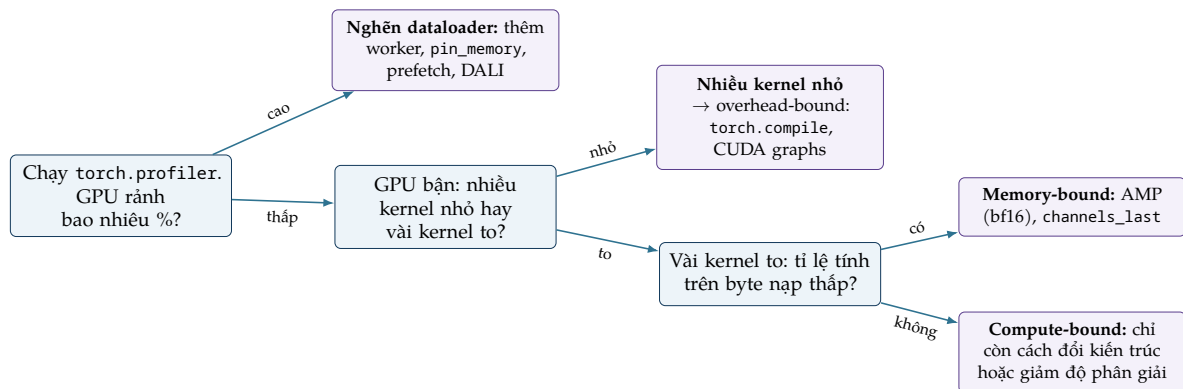
Từ đó suy ra vì sao quên `model.eval()` là lỗi kinh điển và im lặng: ở chế độ train, BatchNorm dùng thông kê của *batch hiện tại* thay vì buffer, và dropout bật lại. Kết quả là đầu ra phụ thuộc vào việc bạn tình cờ đưa mẫu nào vào cùng batch — một mẫu đánh giá có thể ra hai kết quả khác nhau chỉ vì bạn đổi thứ tự dữ liệu. Không có exception nào; chỉ có một con số hơi lệch.

→ **hệ quả của C/12** — BatchNorm được đưa vào để ổn định huấn luyện; chính cơ chế ấy làm mô hình phụ thuộc cả batch, nên nó kéo theo hai chế độ hành vi khác nhau.

E. Tăng tốc: đo trước, và biết mỗi đòn bẫy gỡ nút thắt nào

Đây là phần bị lạm dụng nhiều nhất, nên bắt đầu bằng nguyên tắc: *mỗi đòn bẫy chỉ giúp khi bạn đang nghẽn ở đúng chỗ nó gỡ*. Hình 19.4 là cây quyết định rút gọn, và Bảng 19.2 là phần chi tiết.

↪ **ràng buộc bởi B/11** — cây quyết định tăng tốc chỉ là mô hình roofline diễn đạt bằng tên API: nhánh nào đúng phụ thuộc bạn đang bị chặn bởi băng thông, năng lực tính, hay chi phí điều phối.



Hình 19.4. Cây quyết định tăng tốc một vòng lặp huấn luyện. Nhánh đầu tiên — GPU đang rảnh — là nhánh đúng trong phần lớn trường hợp thực tế, và cũng là nhánh mà mọi mẹo về `torch.compile` đều vô dụng.

Bảng 19.2. Năm đòn bẩy tăng tốc, và nút thắt mà mỗi cái gỡ.

Đòn bẩy	Cơ chế	Gỡ nút thắt nào	Hỏng / vô dụng khi nào
Sửa DataLoader	Nhiều <i>worker</i> , <i>pin_memory</i> , <i>prefetch</i> , chuyển augmentation lên GPU	GPU đói dữ liệu	Quá nhiều worker làm cạn RAM và tăng thời gian khởi động; augmentation trên GPU tranh chấp với training
AMP (fp16/bf16)	Nửa số byte phải nạp và ghi	Memory-bound	Compute-bound thuần; fp16 tràn số khi không có GradScaler
<code>channels_last</code>	Đổi bố cục sang NHWC cho <i>Tensor Core</i>	Conv memory-bound	Mô hình có op không hỗ trợ NHWC → thêm phép hoán vị, chậm hơn
<code>torch.compile</code>	Hợp nhất kernel, giảm chi phí điều phối Python	Overhead-bound	Shape động gây biên dịch lại liên tục; <i>graph break</i> do code Python động
DDP / FSDP	Chia batch (DDP) hoặc chia trọng số (FSDP) qua nhiều GPU	Không đủ thời gian hoặc bộ nhớ	Băng thông liên kết kém → <i>all-reduce</i> nuốt hết lợi ích; batch hiệu dụng đổi nên phải chỉnh <i>learning rate</i> [127]

Ví dụ nhỏ nhất cho AMP. Một tensor kích hoạt cỡ $64 \times 256 \times 56 \times 56$ có 51,4 triệu phần tử. Ở fp32 nó chiếm 205 MB; ở bf16 chiếm 103 MB. Trên một GPU có băng thông 900 GB/s, chỉ riêng việc đọc nó một lần tốn $228 \mu\text{s}$ so với $114 \mu\text{s}$. *Ý nghĩa:* nếu tăng đó chỉ làm vài phép tính trên mỗi byte nạp về, AMP cắt gần một nửa thời gian; nếu nó làm rất nhiều phép tính trên mỗi byte, AMP gần như không đổi gì. Đó chính là trục *arithmetic intensity* của **B/11-gioi-han-tinh-toan**, xuất hiện lại dưới dạng một lựa chọn hai dòng code.

fp16 hay bf16? Hai lựa chọn đối lập rõ ràng nên đáng đặt cạnh nhau: fp16 có 10 bit phần định trị và dải mũ hẹp, nên chính xác hơn ở vùng giá trị bình thường nhưng tràn hoặc triệt tiêu về không khi gradient nhỏ — vì thế nó *bắt buộc* cần GradScaler để nhân loss lên trước khi lượt ngược [13]. bf16 có cùng dải mũ với fp32 nhưng chỉ 7 bit định trị: kém chính xác hơn, đổi lại gần như không bao giờ tràn và không cần GradScaler. Trên phần cứng hỗ trợ bf16, mặc định hợp lý là bf16; fp16 chỉ còn lý do tồn tại trên phần cứng cũ.

```

scaler = torch.amp.GradScaler() # chỉ cần cho fp16
model = model.to(memory_format=torch.channels_last)
for x, y in loader:
    x = x.cuda(non_blocking=True).to(memory_format=torch.channels_last)
  
```

```
opt.zero_grad(set_to_none=True) # set_to_none re hon zero_
with torch.amp.autocast("cuda", dtype=torch.bfloat16):
    loss = criterion(model(x), y.cuda(non_blocking=True))
    scaler.scale(loss).backward()
    scaler.step(opt); scaler.update()
```

F. Giới hạn và trường hợp hỏng

- **torch.compile không phải nút bấm thần kỳ.** Nó giúp khi bạn đang overhead-bound; nó không giúp gì cho mô hình nghẽn ở dataloader, và nó có thể *chậm hơn* nếu shape đầu vào thay đổi liên tục khiến nó biên dịch lại. Với dữ liệu ảnh có kích thước thay đổi hoặc số hộp phát hiện thay đổi, đây là tình huống thường gặp chứ không phải ngoại lệ.
- **Profiler cũng nói dối nếu bạn đo sai.** Lệnh CUDA là bất đồng bộ, nên đo bằng đồng hồ Python quanh một lời gọi chỉ đo thời gian *xếp hàng*, không đo thời gian chạy. Phải đồng bộ hoá hoặc dùng CUDA event — một cái bẫy bạn đã gặp ở phía C++, và nó không biến mất khi lên Python.
- **DDP đổi ngữ nghĩa của batch.** Batch hiệu dụng là batch mỗi GPU nhân số GPU, nên mọi siêu tham số phụ thuộc batch (learning rate, lịch warm-up, thống kê BatchNorm) đều đổi theo. Một so sánh “1 GPU với 8 GPU” mà không chỉnh learning rate là một so sánh vô nghĩa.
- **Tắt định có giá.** Bật use_deterministic_algorithms buộc PyTorch chọn kernel chậm hơn hoặc báo lỗi ở những op không có bản tắt định — và nó vẫn không đảm bảo hai máy khác GPU cho cùng kết quả.

CẠM BẦY

Rò bộ nhớ vì logging là bug tốn thời gian nhất trong danh sách này, vì triệu chứng xuất hiện sau vài trăm bước chứ không phải ngay lập tức. Viết `losses.append(loss)` thì bạn đang giữ luôn toàn bộ đồ thị tính toán dẫn tới loss, kể cả mọi activation trung gian; sau một epoch, GPU hết bộ nhớ và *stack trace* chỉ vào một dòng hoàn toàn vô can. Cách đúng là `losses.append(loss.detach())` hoặc `loss.item()`. Cùng một họ lỗi: giữ output của mô hình trong một *dict* để debug, hoặc trả tensor còn đồ thị ra khỏi một hàm được gọi trong vòng lặp.

CẦN NHỚ

Ảnh xạ triệu chứng → công cụ mới là thứ đáng nhớ, không phải tên API. GPU rảnh thì sửa dataloader; nhiều kernel nhỏ thì `torch.compile`; ít tính trên mỗi byte thì AMP và `channels_last`; không đủ bộ nhớ thì FSDP hoặc *gradient checkpointing*. Và trước tất cả: chạy profiler, vì tối ưu thứ chưa đo là cách chắc chắn nhất để mất một tuần mà không nhanh hơn. [T2]

G. Kết nối

Mục này là bản dịch của [D/19/01-tu-viet-bang-numpy](#) sang ngôn ngữ công cụ: *autograd* là cuốn băng bạn đã tự cuộn bằng tay, *buffer* là “trạng thái” bạn đã tự quản lý, *eval()* là cái công tắc bạn đã tự viết. Nó là ứng dụng trực tiếp của [B/11-gioi-han-tinh-toan](#): cây quyết định ở Hình 19.4 chỉ là mô hình *roofline* được diễn đạt bằng tên API. Nó là tiền đề của [D/20-toi-uu-va-nen-mo-hinh](#) (nơi cùng những đòn bẫy này được áp cho **suy luận** (*inference*) thay vì huấn luyện) và của [D/22-tai-lap-va-mlops](#) (nơi mục về *seed* và tính tắt định được mở rộng thành một chương).

❗ **Câu hỏi tự kiểm** (trả lời không nhìn bài)

1. Khi nào `.view()` thất bại mà `.reshape()` thì không, và điều đó liên quan gì tới *stride*? Vì sao thất bại đôi khi lại là hành vi bạn muốn?
2. Vì sao `torch.compile` không giúp gì cho mô hình bị nghẽn ở dataloader, và làm sao biết mình đang ở tình huống đó trong ba mươi giây?
3. Thống kê chạy của BatchNorm là *parameter* hay *buffer*, và câu trả lời đó giải thích thế nào cho việc quên `eval()` lại làm kết quả phụ thuộc thứ tự dữ liệu?
4. Vì sao fp16 cần GradScaler mà bf16 thì không? Cái giá của bf16 là gì?
5. Bạn đo một lời gọi mô hình bằng `time.time()` và thấy nó tốn 0,2 ms. Vì sao con số đó gần như chắc chắn sai, và phải đo thế nào?
6. Chuyển từ 1 GPU sang 8 GPU bằng DDP, mọi thứ chạy nhưng độ chính xác tụt — hai nguyên nhân khả dĩ nhất là gì, biết rằng cả hai đều không phải bug trong DDP?

19.3 Thực hành

Hai mini project dưới đây được thiết kế để *đóng* giai đoạn 1 và *mở* giai đoạn 2. Cái thứ nhất chứng minh bạn đã hiểu đúng cơ chế; cái thứ hai chứng minh bạn tăng tốc được dựa trên số đo.

Repo và tài nguyên. Để học: bài tập CS231n (làm hết assignment 1 và 2), [karpathy/micrograd](#) (autograd trong hai trăm dòng), [karpathy/nn-zero-to-hero](#), [pytorch/examples](#). Để dùng thật: [huggingface/pytorch-image-models](#) (timm), [Lightning-AI/pytorch-lightning](#), [facebookresearch/hydra](#), [NVIDIA/DALI](#), [albumentations](#). Để hiểu framework từ bên trong: [tinygrad/tinygrad](#) — đủ nhỏ để đọc hết. Danh sách này chốt tại tháng 9/2026; các repo hạ tầng ở đây (timm, DALI, Hydra) bền hơn nhiều so với tên mô hình dẫn đầu bảng, nhưng API của chính PyTorch cũng đổi, nên hãy nhớ vấn đề và tra lại cú pháp.

Câu hỏi kiểm tra hiểu. Khi nào `.view()` thất bại mà `.reshape()` thì không, và điều đó liên quan gì tới *stride*? Vì sao `torch.compile` không giúp gì cho mô hình bị nghẽn ở dataloader? *Hook* nào cần dùng để lấy activation ở giữa mạng mà không sửa forward? Và câu hỏi đóng giai đoạn 1: vì sao BatchNorm khiến lượt ngược tốn thêm bộ nhớ, còn dropout phải scale lúc train chứ không lúc test?

Mini project (vài giờ đến hai ngày). Viết lại *chính xác* một mạng NumPy của bạn bằng PyTorch và kiểm tra hai bên khớp tới 10^{-6} ở cả lượt xuôi lẫn gradient. Phần đánh giá không phải lúc nó khớp mà lúc nó không khớp: với mỗi chỗ lệch, phân loại xem đó là *bug* (lệch có hệ thống, không giảm khi bạn tăng độ chính xác) hay *số học* (lệch cỡ epsilon máy, đôi khi bạn đổi thứ tự phép cộng). Kỹ năng phân loại này là thứ bạn sẽ dùng lại mỗi lần export mô hình ở chương 21.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Lượt xuôi, lượt ngược	Lượt xuôi tính từ đầu vào ra loss; lượt ngược đi ngược để lấy đạo hàm theo từng tham số. Mọi vòng lặp huấn luyện là hai lượt này lặp lại.
●●● Lan truyền ngược	Thuật toán vi phân cho mọi đồ thị khả vi, tốn cùng bậc chi phí với một lượt xuôi. Đừng nhầm nó (thuật toán) với lượt ngược (một lần chạy).
●● Hinge so softmax	Hinge cho gradient bằng đúng không khi lề đã thoả, nên ổn định. Softmax không bao giờ tắt, nên nó tính chính tiếp nhưng nhạy với nhân sai.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Gradient check	So gradient giải tích với sai phân trung tâm ở vài chục tọa độ ngẫu nhiên. Nó chỉ chứng minh xuôi và ngược khớp nhau, không chứng minh lượt xuôi đúng.
●●● Bộ nhớ kích hoạt	Chỗ chứa đầu ra trung gian mà lượt ngược phải đọc lại. Đây là lý do huấn luyện tốn bộ nhớ hơn suy luận nhiều lần, và là thứ chặn batch size.
●● Vì sao lượt ngược BatchNorm không theo từng phần tử?	Nó chứa hai tổng trên toàn batch, một cho trung bình và một cho phương sai. Gradient của một mẫu vì thế phụ thuộc mọi mẫu khác trong batch.
●● Inverted dropout	Giữ phần tử với xác suất $1 - p$ rồi chia cho $1 - p$ lúc train; lúc test không làm gì. Bảo toàn kỳ vọng mà không đặt bước bắt buộc nào lên đường suy luận.
●● Vì sao col2im sinh ra bất định trên GPU?	Nó cộng dồn gradient vào các vị trí của sổ chồng lán, và cài đặt GPU dùng atomicAdd. Cộng số thực không kết hợp, nên thứ tự luồng quyết định bit cuối.
●● Vì sao gradient check phải chạy float64?	Sai phân trung tâm lấy hiệu hai số gần bằng nhau. Ở float32, epsilon máy khoảng $1,2 \times 10^{-7}$ nên bạn đo nhiều làm tròn chứ không đo đạo hàm.
●●● Autograd	Cuốn băng ghi lại các phép toán lúc lượt xuôi, tua ngược khi gọi backward() rồi xoá. Giữ một tensor còn dính băng nghĩa là giữ luôn cả đoạn băng phía sau nó.
●● Stride	Mỗi bước theo một chiều nhảy bao nhiêu phần tử trong bộ nhớ. Chuyển vị chỉ đổi stride nên miễn phí, nhưng làm tensor không còn liền mạch.
●● view so với reshape	view đòi tensor liền mạch và báo lỗi nếu không; reshape lạng lẽ sao chép. Dùng view ở chỗ bạn muốn chương trình ồn ào.
●●● Parameter so với buffer	Parameter được optimizer cập nhật; buffer thì không, nhưng vẫn nằm trong state_dict và vẫn đổi thiết bị theo mô hình. Thống kê chạy của BatchNorm là buffer.
●● Vì sao quên eval() làm kết quả đổi theo batch?	Ở chế độ train, BatchNorm dùng thống kê của batch hiện tại thay vì buffer, và dropout bật lại. Một mẫu vì thế cho hai kết quả khác nhau tùy bạn xếp nó cùng ai.
●●● AMP	Tính phần lớn phép toán ở 16 bit, giữ trọng số ở 32 bit. Nó giảm số byte phải nạp chứ không giảm số phép tính, nên vô ích với kernel compute-bound.
●● torch.compile	Hợp nhất kernel và cắt chi phí điều phối Python. Chỉ giúp khi bạn overhead-bound; shape thay đổi liên tục làm nó biên dịch lại và chậm hơn.
●● Sai số gradient check khoảng 3×10^{-3} và đổi mỗi lần chạy?	Còn một nguồn ngẫu nhiên chưa đóng băng: mặt nạ dropout, hoặc BatchNorm đang ở chế độ train. Con số ổn định thì mới nên nghi công thức.
●● Bật AMP và torch.compile mà epoch không nhanh hơn?	Bạn đang ngهن ở dataloader, không phải ở GPU. Chạy profiler xem GPU rảnh bao nhiêu phần trăm trước khi đùng vào mô hình.
●● Bộ nhớ GPU phình dần suốt epoch rồi hết?	Bạn đang giữ tensor còn đồ thị trong một danh sách log. Dùng loss.detach() hoặc loss.item().

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
PyTorch, JAX	Toàn bộ autograd là hiện thực của vi phân chế độ nghịch trên đồ thị động — đúng thứ bạn tự cuộn bằng tay ở giai đoạn 1	Hạ tầng, ổn định từ 2017 tới nay

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
FlashAttention	Đánh đổi tính lại lượt xuôi để giảm bộ nhớ kích hoạt, tức chính <i>gradient checkpointing</i> áp cho attention	Đã thành mặc định trong gần như mọi thư viện transformer
<code>timm</code>	Vòng lặp huấn luyện chuẩn với AMP, <code>channels_last</code> và <code>torch.compile</code> — đúng bốn đòn bẩy của Bảng 19.2	Hạ tầng thị giác; công thức trong repo đáng đọc hơn tài liệu
FSDP và ZeRO	Chia trọng số và trạng thái optimizer thay vì nhân bản chúng, khi một GPU không chứa nổi mô hình	Ý tưởng từ 2020, vẫn là nền của mọi huấn luyện quy mô lớn
<code>tinygrad</code> , <code>micrograd</code>	Chính bài tập giai đoạn 1, viết đủ nhỏ để đọc hết trong một buổi	Giá trị sư phạm, không phải công cụ sản xuất

MINI PROJECT SLAM · Tự cài lại tầng mô tả của frontend bằng NumPy · 1–2 ngày

Mục tiêu	làm đúng một lần cái việc mà framework luôn làm hộ, trên đúng khối bạn đang dùng thật: viết cả lượt xuôi lẫn lượt ngược cho một mạng mô tả đặc trưng, và không tin nó cho tới khi kiểm tra gradient số cho phép.
Dữ liệu	chính tập mảnh ảnh 32×32 dựng ở mini project chương 18, cùng nhãn cặp lấy từ liên kết dữ liệu của bản đồ.
Các bước	viết lượt xuôi cho một khối gồm <i>convolution</i> 3×3 , BatchNorm, ReLU, max-pool và một tầng <i>fully-connected</i> rồi chuẩn hoá L2 ra vector mô tả 32 chiều. Viết lượt ngược giải tích cho từng tầng. Chạy kiểm tra gradient ở float64 trên 20 tọa độ ngẫu nhiên của <i>mỗi</i> tham số, với mọi nguồn ngẫu nhiên đóng băng và BatchNorm đặt ở chế độ cố định. Chỉ khi mọi tầng qua ngưỡng mới ghép lại và huấn luyện bằng một hàm mất mát tương phản trên các cặp có nhãn.
Xong khi	sai số tương đối của kiểm tra gradient dưới 10^{-6} cho mọi tham số của mọi tầng. Tọa độ nằm sát điểm gãy của ReLU hoặc max-pool được phép bỏ qua, nhưng phải ghi rõ đã bỏ bao nhiêu và vì sao. Sau đó bản NumPy khớp bản PyTorch tới 10^{-6} ở cả lượt xuôi lẫn <i>gradient</i> .
Nối với	D/19 cho bảng “backward cần nhớ gì” và cây chẩn đoán; A/01 cho quy tắc chuỗi. Mô hình mô tả sinh ra ở đây chính là thứ chương 20 đem đi lượng tử hoá.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì vi phân chế độ nghịch trả về đạo hàm theo *mọi* tham số trong một lượt, còn vi phân thuận chỉ trả về theo một hướng mỗi lượt. Chi phí một lượt ngược cùng bậc với một lượt xuôi, nên tổng chi phí không phụ thuộc số tham số — đó là toàn bộ khoản tiết kiệm.
- A2. Vì backward phải đọc lại trạng thái của lượt xuôi: đầu vào cho tầng *fully-connected*, mặt nạ cho ReLU và dropout, $\hat{x}$ cùng hai tổng batch cho BatchNorm. Suy luận không cần lượt ngược nên giải phóng được ngay sau mỗi tầng; huấn luyện thì không.
- A3. Nó chỉ kiểm tính *nhất quán* giữa lượt xuôi và lượt ngược, không kiểm tính đúng của lượt xuôi. Một tầng cài sai công thức xuôi nhưng có ngược khớp với chính nó sẽ qua hoàn hảo; ngoài ra nó không bắt được lỗi trạng thái, lỗi nhân và lỗi ở các chỗ nối.
- A4. Vì mỗi đòn bẩy chỉ gỡ đúng nút thắt của nó. Nếu GPU đang đói dữ liệu thì thời gian nằm ở dataloader, và cả AMP lẫn `torch.compile` đều không chạm tới chỗ đó. Cây quyết định ở Hình 19.4 bắt đầu bằng “GPU rảnh bao nhiêu phần trăm” chính vì lý do này.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng một dây chuyền làm bánh gồm mười khâu, mỗi khâu có vài cái nút chỉnh. Bánh ra lò bị mặn. Câu hỏi là: nút nào ở khâu nào đã làm nó mặn, và mặn thêm bao nhiêu?

Cách này thì là vận thử từng nướm rồi chạy lại cả mẻ bánh. Có một nghìn cái nướm thì phải chạy một nghìn mẻ. Không ai làm nổi.

Mẹo là đi ngược dây chuyền đúng một lần. Đứng ở cuối, bạn biết bánh mất hơn mong muốn bao nhiêu. Lùi về khâu thứ mười và hỏi: nếu khâu này đưa ra thứ mất hơn một chút, bánh cuối mất thêm bao nhiêu? Có câu trả lời rồi thì lùi tiếp về khâu chín, và cứ thế. Một lượt đi ngược chia được trách nhiệm cho cả nghìn cái nướm, tốn xấp xỉ bằng một lượt làm bánh bình thường.

Cái giá rất cụ thể: muốn hỏi ngược thì mỗi khâu phải giữ lại mẫu bán thành phẩm của lượt xuôi. Xưởng vì thế cần kho chứa gấp nhiều lần, và đó đúng là lý do máy tính hết bộ nhớ khi huấn luyện chứ không hết khi chạy thật. Cái giá thứ hai: công thức chia trách nhiệm ở mỗi khâu do bạn tự viết, và nếu sai một dấu thì cả dây chuyền vẫn chạy êm, bánh vẫn ra, chỉ là không bao giờ ngon lên. Vì vậy phải thử tay từng khâu trước khi ghép.

Một câu: *Chạy xuôi một lần và giữ lại dấu vết, đi ngược một lần để chia trách nhiệm — rẻ hơn thử từng nướm hàng nghìn lần, và cái giá đúng bằng chỗ chứa dấu vết đó.*

Tổng kết chương

Chương này đi từ chỗ tự tay cuộn một cuốn băng autograd trong NumPy tới chỗ đọc được một trace của `torch.profiler` và biết chính xác dòng nào cần sửa. Sợi chỉ xuyên suốt là *trạng thái*: cái gì được nhớ, cái gì được lưu, cái gì đổi hành vi giữa hai chế độ — và phần lớn bug im lặng trong deep learning là bug ở đúng ba câu hỏi đó.

Điều còn bỏ ngỏ là toàn bộ phía **suy luận** (*inference*). Ở đây ta chỉ quan tâm huấn luyện: bộ nhớ hoạt động, thông lượng, tính đúng đắn của gradient. Nhưng ràng buộc thật của một hệ thống trên thiết bị nằm ở phía kia: độ trễ đuôi, năng lượng, và tập toán tử mà runtime hỗ trợ. Cùng những đòn bẫy trong Bảng 19.2 sẽ xuất hiện lại ở [D/20-toi-uu-va-nen-mo-hinh](#), nhưng với một hàm mục tiêu khác hẳn. Điều thứ hai còn bỏ ngỏ là tính tái lập: mục 19.2 chỉ nêu tên `use_deterministic_algorithms` và nói nó không đủ; vì sao nó không đủ, và mục tiêu đúng là gì, thuộc về [D/22-tai-lap-va-mlops](#).

Câu hỏi tự kiểm (trả lời không nhìn bài)

- Chương chia hai giai đoạn theo tầng bán rã `[T2][T3]` và `[T4]`. Hãy nêu một hiểu biết từ giai đoạn 1 mà giai đoạn 2 *không* dạy được, và một sai lầm ở giai đoạn 2 mà giai đoạn 1 không giúp bạn tránh?
- Cùng một triệu chứng “mạng không học” có thể do đạo hàm sai, do lẫn lộn trạng thái train và eval, hoặc do dữ liệu. Hai giai đoạn cho bạn những công cụ phát hiện khác nhau nào, và thứ tự dùng chúng nên là gì?
- BatchNorm xuất hiện ở cả hai mục: một lần là hai tổng trên batch trong lượt ngược, một lần là buffer chứ không phải parameter. Hai góc nhìn đó nói cùng điều gì về BatchNorm, và điều đó giải thích thế nào cho hành vi khi lưu rồi nạp lại checkpoint?
- Phép cộng dồn tán xạ trong lượt ngược của convolution ở giai đoạn 1 quay lại ám bạn ở giai đoạn 2 dưới dạng nào, và nó liên quan gì tới việc hai lần chạy cho kết quả không trùng nhau?
- Vì sao “huấn luyện tốn bộ nhớ gấp nhiều lần suy luận” là kết luận rút ra được từ giai đoạn 1 nhưng chỉ đo được ở giai đoạn 2, và bạn đo thế nào để tách phần kích hoạt khỏi phần trạng thái optimizer?
- Bạn bật `torch.compile` và AMP mà thời gian một epoch không đổi. Trước khi kết luận công cụ vô dụng, bạn đo ba đại lượng nào, và mỗi đại lượng loại bỏ giả thuyết nào?
- Một mẹo tối ưu nghe được trên diễn đàn có tuổi thọ ngắn hơn hẳn hiểu biết ở giai đoạn 1. Bạn dùng tiêu chí nào để quyết định một mẹo có đáng đưa vào code nền của mình hay không?

Tối ưu và nén mô hình (Model Optimization and Compression)

PHỤ THUỘC B/11-gioi-han-tinh-toan (bắt buộc — *roofline*, memory-bound so với compute-bound; không có nó thì chương này chỉ là danh sách mẹo); B/09-thiet-ke-ham-mat-mat (phân phối mềm dùng trong chưng cất); C/18/02-kinh-te-hoc-pretraining (cùng logic ngân sách, đổi cột chi); D/19-numpy-toi-pytorch (AMP, channels_last, profiler).

TÓM TẮT

Mô hình tốt nhất của bạn không vừa ràng buộc triển khai. Chương này trình bày năm đòn bẩy để ép nó vừa — giảm độ phân giải, đổi kiến trúc, lượng tử hoá, chưng cất, tia — và lập luận rằng *thứ tự* dùng chúng quan trọng hơn kỹ thuật của từng cái. Đọc xong bạn lập được đường Pareto độ chính xác–độ trễ cho bài toán của mình và chọn một điểm có lý do, thay vì thử ngẫu nhiên rồi giữ cái nào trông ổn. Nếu chỉ nhớ một điều: FLOPs là đại diện tồi cho độ trễ, nên mỗi kỹ thuật nén phải kèm phép đo trên *đúng phân cứng đích*, và sai lệch giữa FLOPs và thời gian thật đi theo hướng khó đoán.

KIẾN THỨC CẦN CÓ TRƯỚC

- **độ trễ (latency) và thông lượng (throughput)** — độ trễ là thời gian một yêu cầu đi từ vào tới ra; thông lượng là số yêu cầu xử lý được mỗi giây. Gộp batch làm thông lượng tăng và độ trễ tăng — hai đại lượng này thường xung đột.
- **FLOPs** — số phép tính dấu phẩy động của một **lượt xuôi** (*forward pass*). Nó dễ tính trên giấy nhưng chỉ tương quan lỏng lẻo với thời gian thật, vì phần lớn mạng bị chặn bởi băng thông bộ nhớ chứ không bởi năng lực tính toán.
- **lượng tử hoá (quantization)** — biểu diễn trọng số và kích hoạt bằng số nguyên ít bit (thường INT8) thay vì dấu phẩy động 32 bit, để giảm số byte phải nạp và tăng thông lượng; cái giá là sai số làm tròn tích lũy qua các tầng.
- **calibration set** — một tập ảnh nhỏ (thường vài trăm) chạy qua mô hình để đo dải giá trị thực tế của kích hoạt, từ đó chọn hệ số tỉ lệ cho lượng tử hoá. Nó phải *đại diện* cho dữ liệu triển khai, nếu không dải đo được sẽ sai.
- **tia (pruning) và sparsity** — đặt một phần trọng số về không. *Sparsity* là tỉ lệ trọng số bằng không. Tia *không cấu trúc* rải các số không khắp nơi; tia *có cấu trúc* bỏ nguyên kênh hoặc nguyên khối.
- **chưng cất tri thức (knowledge distillation)** — huấn luyện một mô hình nhỏ (student) bắt chước đầu ra của một mô hình lớn (teacher), thay vì chỉ học từ nhãn cứng. Phân phối xác suất mềm của teacher mang nhiều thông tin hơn một nhãn đúng/sai.
- **độ phân giải đầu vào** — kích thước ảnh đưa vào mạng, ví dụ 640×640 . Chi phí tính toán của phần lớn mạng thị giác tỉ lệ với số điểm ảnh, tức tỉ lệ với *bình phương* cạnh.
- **phần cứng đích** — đúng con chip và đúng cấu hình mà mô hình sẽ chạy khi triển khai — Jetson Orin ở chế độ 30 W không phải cùng một cỗ máy với chính nó ở chế độ 15 W, và cả hai đều không phải là chiếc RTX trên bàn bạn.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn tĩa bỏ 90% trọng số, mô hình giữ nguyên độ chính xác, và nó không chạy nhanh hơn một mili-giây nào — phần cứng đã từ chối món quà đó ở chỗ nào?
- Q2. Vì sao kỹ thuật thô nhất trong cả chương — hạ độ phân giải đầu vào — lại được xếp đứng trước mọi kỹ thuật nén tinh vi?
- Q3. Một tầng có đúng *một* kênh trọng số bất thường; vì sao chừng đó đủ để đẩy phần còn lại của tầng xuống dưới ba bit hiệu dụng sau khi lượng tử hoá?
- Q4. Mô hình A ít FLOPs hơn mô hình B nhưng chạy chậm hơn trên cùng một Jetson — FLOPs đã nói dối theo bao nhiêu cách khác nhau?

20.1 Đặt vấn đề

Câu hỏi dẫn dắt của chương rất cụ thể: *mô hình tốt nhất của bạn không vừa ràng buộc triển khai — làm gì tiếp?* Bạn có năm đòn bẫy, chúng không tương đương nhau, và thứ tự dùng chúng quan trọng hơn kỹ thuật của từng cái.

Điều làm chương này khác một danh sách mẹo là chỗ nó xuất phát từ một sự thật vật lý đã có ở [B/11-gioi-han-tinh-toan](#): phần lớn mạng thị giác không bị chặn bởi năng lực tính toán mà bởi *băng thông bộ nhớ*. Từ sự thật đó suy ra gần như mọi thứ còn lại. Lượng tử hoá có tác dụng vì nó giảm số byte phải nạp chứ không phải vì nó giảm số phép nhân. Tĩa không cấu trúc thất bại vì nó giảm số phép nhân mà không giảm số byte phải nạp. Hợp nhất kernel có tác dụng vì nó cắt bớt các lượt đi về bộ nhớ. Ai chưa nắm trực này sẽ học chương này như học một danh sách; ai nắm rồi thì tự suy ra được phần lớn nội dung.

→ *ràng buộc bởi B/11* — phần lớn mạng thị giác bị chặn bởi băng thông bộ nhớ chứ không bởi năng lực tính, và chính sự thật đó quyết định kỹ thuật nén nào có tác dụng.

Học xong chương này thì làm được gì mà trước đó không làm được? Bạn cầm một ngân sách mili-giây và một mô hình quá chậm. Trong một buổi bạn vạch được đường Pareto độ chính xác–độ trễ, chọn một điểm, và giải thích được vì sao chọn nó. Cách cũ là thử ba kỹ thuật ngẫu nhiên rồi giữ lại cái nào trông ổn.

TRỰC GIÁC

Lượng tử hoá *theo tensor* giống việc chọn một khẩu độ và một tốc độ màn trập duy nhất cho cả khung hình có một bóng đèn sáng chói trong đó. Bóng đèn ép bạn phải khép sáng lại, và mọi vùng còn lại chìm xuống sát sàn nhiều — không phải vì máy ảnh kém, mà vì bạn chỉ có một dải động và đã tiêu nó cho một điểm.

Lượng tử hoá *theo kênh* là chụp mỗi vùng bằng một thiết lập riêng rồi ghép lại. Cùng số bit, nhưng dải động được phân bổ theo nhu cầu thật của từng vùng. Đây không phải ẩn dụ lỏng lẻo. Cả hai trường hợp đều là bài toán ánh xạ một dải giá trị liên tục vào một số mức rời rạc cố định. Và ở cả hai, *một giá trị ngoại lai duy nhất cũng đủ phá hỏng độ phân giải của tất cả những giá trị còn lại*.

20.2 Thứ tự thử nghiệm, và lý do của thứ tự đó

A. Bảng thứ tự

Bảng 20.1 là kết luận thực dụng của cả chương, đặt lên trước để bạn dùng được ngay; phần còn lại giải thích từng dòng.

Bảng 20.1. Năm đòn bẩy nén, theo thứ tự nên thử. Cột “vì sao trước” mới là nội dung; cột “cái giá” là thứ bạn phải chấp nhận.

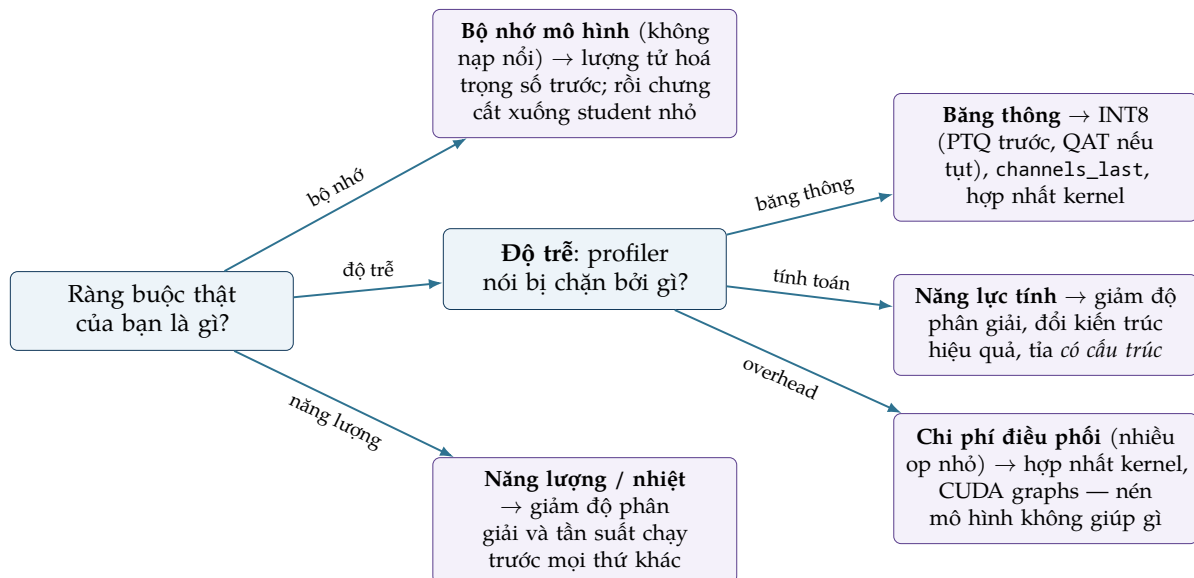
#	Đòn bẩy	Vì sao đứng trước	Cái giá
1	Giảm độ phân giải hoặc độ dài đầu vào	Rẻ nhất, hiệu quả gần như bậc hai theo cạnh ảnh, đảo ngược được ngay trong một dòng config	Mất vật nhỏ và chi tiết mảnh — đúng thứ bạn có thể không đo được trên tập test hiện có
2	Đổi sang kiến trúc hiệu quả	Tối ưu từ thiết kế luôn tốt hơn nén sau; cùng độ chính xác nhưng ít byte hơn ngay từ đầu	Phải huấn luyện lại từ đầu
3	Lượng tử hoá	Tỉ lệ lợi ích trên công sức cao nhất trong nhóm nén; gỡ đúng nút thắt bằng thông	Mất độ chính xác nhỏ, và phụ thuộc mạnh vào calibration đúng
4	Chưng cất	Giữ được nhiều chất lượng nhất trên mỗi đơn vị kích thước	Cần teacher và một chu kỳ huấn luyện đầy đủ
5	Tỉa	Thường <i>không</i> cho tốc độ thật trên phần cứng phổ thông	Công sức lớn, lợi ích không chắc

B. Vì sao thứ tự này chứ không phải thứ tự khác

Ba nguyên tắc sắp xếp, và cả ba đều là nguyên tắc kỹ thuật chứ không phải sở thích:

- **Đảo ngược được thì thử trước.** Giảm độ phân giải là một dòng config; nếu hỏng, bạn biết trong nửa giờ. Chưng cất là một tuần; nếu hỏng, bạn mất một tuần. Xếp theo chi phí của việc *sai* chứ không theo chi phí của việc *đúng*.
- **Thiết kế thắng nén.** Một mạng được thiết kế để có ít byte đi qua luôn tốt hơn một mạng to bị ép nhỏ lại, vì nén sau chỉ bỏ bớt thứ đã có chứ không đổi được cấu trúc luồng dữ liệu.
- **Ưu tiên đòn bẩy gỡ đúng nút thắt.** Nếu bạn memory-bound thì lượng tử hoá gỡ đúng chỗ; tỉa không cấu trúc thì không, dù nó nghe hứa hẹn hơn.

Hình 20.1 chuyển ba nguyên tắc đó thành một cây quyết định theo *ràng buộc* bạn đang bị chặn, vì trong thực tế bạn hiếm khi bị chặn bởi cả ba thứ cùng lúc.



Hình 20.1. Chọn đòn bẩy theo ràng buộc đang chặn bạn, không theo danh sách. Nhánh dưới cùng bên phải là nhánh hay bị bỏ sót nhất: khi thời gian bị nuốt bởi chi phí điều phối kernel, mọi kỹ thuật *nén* đều vô ích và việc phải làm là hợp nhất.

Ví dụ nhỏ nhất: đòn bẩy số 1 rẻ tới mức nào

Chi phí tính của phần lớn mạng tích chập tỉ lệ với số điểm ảnh. Hạ độ phân giải từ 640×640 xuống 512×512 cho tỉ lệ $(512/640)^2 = 0,64$. Tức là cắt 36% chi phí bằng một dòng config, không huấn luyện lại, và hoàn tác được tức thì. Không kỹ thuật nào khác trong chương có tỉ lệ lợi ích trên công sức gần bằng. Cái giá cũng cụ thể như vậy: một vật cao 20 điểm ảnh ở 640 chỉ còn 16 điểm ảnh ở 512, và nếu tập test của bạn không có vật nhỏ thì bạn sẽ *không đo được* thiệt hại — một trường hợp điển hình của việc đánh giá che giấu chế độ hỏng, đúng chủ đề của [B/07-chia-du-lieu-va-danh-gia](#).

↪ **ràng buộc bởi B/07** — thiệt hại của việc giảm độ phân giải tập trung ở nhóm vật nhỏ, nên nó vô hình nếu tập đánh giá không được phân tách theo nhóm.

20.3 Mạch vấn đề → giải pháp

Sáu nút thắt của việc đưa một mô hình vào ngân sách

1 Vấn đề

Giải pháp

Giả định

Giá phải trả

fp32 lãng phí bằng thông.

lượng tử hoá: giảm số bit thì giảm luôn lượng byte phải nạp, và trên phần cứng có đơn vị số nguyên chuyên dụng thì giảm cả thời gian tính.

phân bố giá trị đủ “gọn” để một hệ số tỉ lệ mô tả được.

sai số làm tròn tích lũy, và độ nhảy rất cao với việc chọn hệ số tỉ lệ.

2 Vấn đề

Giải pháp

Vấn đề mới

nhiều trọng số vô dụng.

tĩa.

phần cứng phổ thông không tận dụng được các số không rải rác, nên tiết kiệm trên giấy không thành tốc độ thật. Chỉ tia có cấu trúc, hoặc mẫu 2:4 trên phần cứng hỗ trợ, mới cho lợi ích đo được.

3 Vấn đề

mô hình nhỏ huấn luyện từ đầu thì yếu.

- Giải pháp** chưng cất: dùng phân phối mềm của teacher làm mục tiêu, giàu thông tin hơn nhãn cứng nhiều lần [23].
- Giá phải trả** cần teacher và một chu kỳ huấn luyện đầy đủ.
- 4 Vấn đề** nén sau luôn thua thiết kế tốt.
- Giải pháp** kiến trúc hiệu quả — tích chập tách theo chiều sâu, khối *inverted residual*, và tái tham số hoá kiểu RepVGG.
- Hệ quả** đây lại là mẫu hình “đẩy chi phí sang training-time” đã gặp ở chương 17 và sẽ gặp lại ở chương 21.
- 5 Vấn đề** chọn kiến trúc bằng tay là thủ công và thiên lệch.
- Giải pháp** NAS — tự động tìm kiến trúc.
- Giá phải trả** chi phí tìm kiếm thường lớn hơn lợi ích, và kết quả gắn chặt với phần cứng dùng để tìm, nên hết giá trị khi bạn đổi chip.
- 6 Vấn đề** cùng FLOPs nhưng khác tốc độ.
- Giải pháp** hợp nhất kernel để cắt số lượt đi về bộ nhớ, và bố cục bộ nhớ đúng (`channels_last`, căn lề, tận dụng cache).
- Hệ quả** đây là [B/11-gioi-han-tinh-toan](#) áp dụng trực tiếp, và là lý do hai mô hình cùng FLOPs có thể chênh nhau hai lần về thời gian thật.

A. Lượng tử hoá — chi tiết quyết định là hệ số tỉ lệ

Lượng tử hoá đối xứng INT8 ánh xạ một số thực w thành số nguyên $q = \text{round}(w/s)$ với $s = \max |w|/127$, và khôi phục bằng $\hat{w} = s q$. Công thức này nói điều gì? Rằng bạn có đúng 255 mức để phủ toàn bộ dải giá trị, và $\max |w|$ — giá trị lớn nhất, tức giá trị ngoại lai nếu có — quyết định độ mịn của tất cả các mức còn lại.

Ví dụ nhỏ nhất tính tay được. Một tầng có trọng số nằm trong $[-0,1; 0,1]$ ở mọi kênh, trừ một kênh duy nhất có trọng số lên tới 2,0.

- **Per-tensor:** $s = 2,0/127 = 0,01575$. Một trọng số 0,1 được ánh xạ thành $\text{round}(0,1/0,01575) = 6$. Vậy toàn bộ dải $[0; 0,1]$ chỉ còn sáu mức — tức chưa tới 3 bit hiệu dụng cho phần lớn mạng.
- **Per-channel:** mỗi kênh có s riêng, nên kênh bình thường dùng $s = 0,1/127$ và trọng số 0,1 được ánh xạ thành 127 — đủ 8 bit.

Cùng số bit, khác nhau hơn một bậc độ lớn về độ phân giải hiệu dụng. Đó là lý do per-channel gần như luôn là lựa chọn đúng cho trọng số của conv và linear, và là lý do một tầng duy nhất có phân bố lệch cũng đủ làm hỏng cả mô hình sau khi lượng tử hoá [202], [203].

Bảng 20.2. Ba chế độ lượng tử hoá thường gặp.

Chế độ	Cơ chế	Chi phí	Hỏng khi nào
Weight-only	Chỉ nén trọng số, kích hoạt vẫn fp16	Gần bằng không	Không giúp gì nếu nút thắt là kích hoạt chứ không phải trọng số
PTQ (post-training)	Chạy calibration set để đo dải kích hoạt, rồi chốt hệ số tỉ lệ	Vài phút	Calibration set không đại diện; mô hình có kích hoạt ngoại lai mạnh
QAT (quantization-aware)	Mô phỏng làm tròn ngay trong lúc huấn luyện để mạng học cách chịu đựng	Một chu kỳ huấn luyện	Không có dữ liệu huấn luyện; hoặc bạn dùng nó khi PTQ vốn đã đủ tốt

Kinh nghiệm thực hành đáng tin: thử PTQ trước, đo, và chỉ chuyển sang QAT khi PTQ tụt quá ngưỡng chấp nhận. Và nguyên nhân số một của việc tụt bất thường không phải thuật toán mà là *calibration set chọn sai*. Vài trăm ảnh lấy toàn từ một điều kiện chiếu sáng cho dài kích hoạt hẹp hơn thực tế. Mọi giá trị nằm ngoài dải đó bị cắt cụt khi triển khai.

B. Tỉa — vì sao 90% sparsity không nhanh hơn

Đây là chỗ trực giác đánh lừa mạnh nhất, nên đáng nói thẳng là bức tranh phức tạp chứ không đơn giản hoá được. Tỉa không cấu trúc đạt được sparsity rất cao mà gần như không mất độ chính xác [204], [205], nhưng *không* nhanh hơn trên GPU thông thường, vì ba lý do cộng lại: các số không rải rác không cho phép bỏ qua việc nạp dữ liệu (bạn vẫn phải đọc chỉ số để biết chỗ nào là không); kernel dense đã được tối ưu nhiều năm còn kernel thưa thì chưa; và lưu chỉ số của các phần tử khác không tự nó đã tốn bằng thông.

Bảng 20.3. Ba mức cấu trúc của tỉa. Chỉ hai dòng dưới cho tốc độ thật.

Kiểu	Cơ chế	Tốc độ thật	Hỏng / hạn chế
Không cấu trúc	Đặt từng trọng số nhỏ về không	Gần như không đổi trên GPU phổ thông	Chỉ có ích khi bạn cần giảm <i>dung lượng lưu trữ</i> , không phải thời gian
2:4 (bán cấu trúc)	Trong mỗi nhóm 4 trọng số, đúng 2 bằng không	Có, nhưng chỉ trên phần cứng có đơn vị sparse tensor	Ràng buộc cứng làm mất nhiều độ chính xác hơn tỉa tự do cùng tỉ lệ
Có cấu trúc	Bỏ nguyên kênh, nguyên đầu attention, nguyên khối	Có, trên mọi phần cứng — vì mạng thật sự nhỏ đi	Mất nhiều độ chính xác hơn; cần fine-tune lại sau khi tỉa

C. Chưng cất — vì sao phân phối mềm đáng giá

Nhãn cứng nói “đây là chó”. Phân phối mềm của teacher nói “0,7 chó, 0,2 sói, 0,08 cáo, 0,02 mèo” — và ba con số cuối chính là thông tin về *cấu trúc lớp* mà nhãn cứng vứt đi. Student học được rằng sói gần chó hơn mèo, một quan hệ mà nó sẽ phải tự phát hiện lại nếu chỉ có nhãn cứng. Đó là lý do một student nhỏ được chưng cất thường thắng chính student đó huấn luyện từ nhãn cứng, dù cả hai có cùng số tham số.

↔ cùng nguyên lý với B/09 — phân phối mềm mang thông tin về cấu trúc lớp mà nhãn cứng vứt đi — cùng lý do khiến làm mượt nhãn có tác dụng.

Ba biến thể, theo mức mạnh tăng dần cho thị giác: chưng cất *logit* (đơn giản nhất), chưng cất đặc trưng trung gian (thường mạnh hơn cho thị giác vì nó ép cả biểu diễn chứ không chỉ đầu ra), và chưng cất *attention* cho *transformer* [116]. Xu hướng tại thời điểm viết là chưng cất một mô hình nền tảng lớn xuống student cỡ 20–40M cho thiết bị biên; đây là cách phổ biến nhất để đưa các mô hình lớp SAM hoặc DINO lên thiết bị — ghi nhận này là quan sát thực hành, không phải một kết quả đo mà tôi kiểm chứng được.

D. Kiến trúc hiệu quả, tái tham số hoá, và NAS

Nén sau luôn thua thiết kế tốt, vì thiết kế đôi được cấu trúc luồng dữ liệu còn nén thì không. Dòng MobileNet/ShuffleNet/EfficientNet [117], [118] là các cách khác nhau để giảm số byte đi qua trên mỗi đơn vị sức chứa.

Đáng chú ý riêng là **tái tham số hoá** (*re-parameterization*) kiểu RepVGG: huấn luyện với một khối nhiều nhánh (để tối ưu), rồi *gộp toán học* các nhánh thành một tích chập đơn khi triển khai. Vì phép gộp là chính xác chứ không xấp xỉ, bạn được cả hai: động lực học huấn luyện tốt và đồ thị suy luận

gọn. Đây đúng là mẫu hình “đẩy độ khó sang lúc huấn luyện” của **B/03-giai-phau-he-thong**, ở dạng thuần khiết nhất.

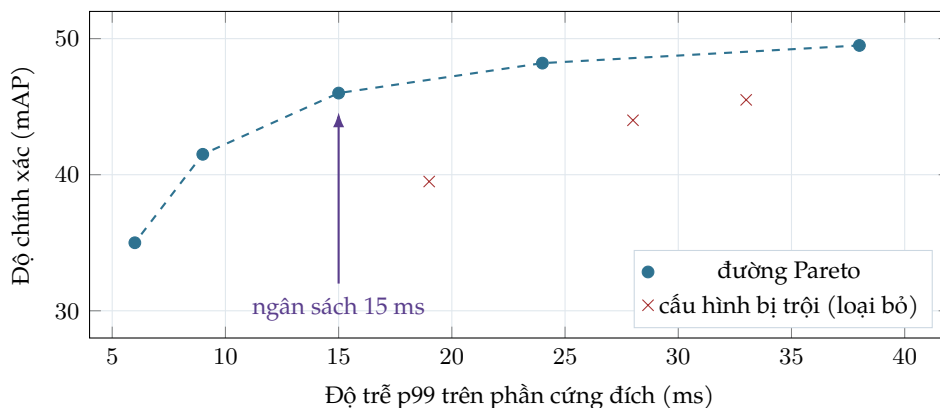
→ cùng nguyên lý với **B/03** — độ khó không biến mất mà bị đẩy đi chỗ khác — ở đây là đẩy từ lúc suy luận sang lúc huấn luyện, dạng thuần khiết nhất của nguyên lý đó.

Về NAS thì phát biểu trung thực nhất là: nó hoạt động, nhưng phần lớn dự án không nên dùng. Chi phí tìm kiếm thường lớn hơn lợi ích. Điểm quan trọng hơn: kết quả gắn chặt với phần cứng dùng để tìm. Một kiến trúc NAS tối ưu cho GPU máy chủ có thể không tối ưu chút nào trên NPU của điện thoại.

E. Nguyên tắc bao trùm: đo trên phần cứng đích

Mỗi kỹ thuật trong chương phải kèm phép đo độ trễ thật trên đúng phần cứng sẽ chạy. FLOPs là đại diện tồi, và điều tệ hơn là nó sai *theo hướng khó đoán*: một mô hình ít FLOPs hơn có thể chậm hơn vì nó có nhiều op nhỏ hơn, vì bố cục bộ nhớ của nó không hợp với kernel sẵn có, hoặc vì nó dùng một toán tử mà runtime phải chạy trên CPU.

Sản phẩm cuối của một chu kỳ nén không phải một mô hình mà là một *đường Pareto* như Hình 20.2: tập các điểm mà không điểm nào bị điểm khác trội hơn ở cả hai trục. Chỉ khi có đường này bạn mới chọn được một cách có lý do. Câu hỏi lúc đó không còn là “mô hình nào tốt nhất”, mà là “đổi bao nhiêu độ chính xác lấy bao nhiêu mili-giây”. Câu hỏi thứ hai có câu trả lời.



Hình 20.2. Sản phẩm của một chu kỳ nén (*số liệu minh họa*). Các dấu chéo là những cấu hình bị trội — vừa chậm hơn vừa kém hơn một cấu hình khác — và chúng bị loại bỏ ngay, không cần thảo luận. Quyết định chỉ còn là chọn một điểm trên đường, và ngân sách mili-giây quyết định điểm nào.

20.4 Thực hành

Repo và tài nguyên. Lượng tử hoá và sparsity: [pytorch/ao](#) (torchao), [NVIDIA/TensorRT-Model-Optimizer](#), [openvinotoolkit/nncf](#), [intel/neural-compressor](#), [bitsandbytes](#). Tỉa: [VainF/Torch-Pruning](#). Chưng cất cho thị giác: công thức có sẵn trong [timm](#); ngoài ra nên đọc các bài chưng cất gần đây trong chính lĩnh vực của bạn, vì chi tiết phụ thuộc bài toán nhiều hơn người ta tưởng. Danh sách này chốt tại tháng 9/2026 và tên các bộ công cụ dẫn đầu sẽ đổi trong 6–12 tháng; các repo hạ tầng (torchao, TensorRT, OpenVINO) bền hơn hẳn tên của thuật toán thời thượng.

Câu hỏi kiểm tra hiểu. Vì sao tỉa không cấu trúc đạt sparsity 90% mà không nhanh hơn trên GPU? Một calibration set 100 ảnh chọn sai thì hỏng theo cơ chế nào, và triệu chứng nhìn thấy được là gì? Khi nào chưng cất thắng lượng tử hoá, và ngược lại?

Mini project (vài giờ đến hai ngày). Lấy một classifier và áp lần lượt bốn bước: giảm độ phân giải 20%, đổi sang backbone hiệu quả, PTQ INT8, rồi QAT. Sau *mỗi* bước đo cả độ chính xác lẫn độ trễ thật trên phần cứng đích. Vẽ đường Pareto và chọn một điểm, kèm một đoạn giải thích vì sao chọn nó. Phần đáng giá nhất của bài này là phát hiện ra bước nào không cho lợi ích như bảng hứa hẹn — và tìm ra lý do.

20.5 Giới hạn và trường hợp hỏng

- **Mọi con số trong chương phụ thuộc phần cứng, và phụ thuộc mạnh.** Một kết quả đo trên GPU máy chủ không dự báo được gì về NPU điện thoại hay về DLA của Jetson. Chuyển chip thì phải đo lại từ đầu, không phải điều chỉnh.
- **Độ chính xác trung bình che giấu thiệt hại tập trung.** Lượng tử hoá và giảm độ phân giải thường lấy đi hiệu năng ở đúng những nhóm hiếm — vật nhỏ, lớp ít mẫu, điều kiện chiếu sáng bất thường. Nếu bạn chỉ theo dõi một con số tổng, bạn sẽ không thấy.
- **Nén nhiều bước không cộng tuyến tính.** Lượng tử hoá một mô hình đã tĩa thường tệ hơn dự đoán, vì tĩa làm phân bố trọng số lệch hơn và do đó khó lượng tử hoá hơn. Thứ tự áp dụng cũng là một siêu tham số.
- **Chưng cất thừa hưởng cả lỗi của teacher.** Student học cả những chỗ teacher sai một cách tự tin, và vì nó nhỏ hơn nên nó không có đủ năng lực để tự sửa.
- **Benchmark ngắn nói dối trên thiết bị bị giới hạn nhiệt.** Đây là chủ đề của chương 21, nhưng nó ảnh hưởng ngược lại mọi phép đo trong chương này.

CẠM BÃY

FLOPs không phải độ trễ, và tin vào FLOPs là lỗi tốn thời gian nhất trong chương. Ba tình huống nó sai theo hướng ngược nhau: một mô hình ít FLOPs với hàng trăm op nhỏ có thể chậm hơn một mô hình nhiều FLOPs với vài op lớn, vì chi phí điều phối trội hơn chi phí tính; một tích chập tách theo chiều sâu có FLOPs rất thấp nhưng tỉ lệ tính trên byte cũng rất thấp, nên nó memory-bound và không nhanh như bảng hứa; và một toán tử mà runtime không hỗ trợ sẽ bị đẩy về CPU, nuốt trọn mọi lợi ích mà chẳng xuất hiện trong bất kỳ phép đếm FLOPs nào.

CẦN NHỚ

Thứ tự đúng là: giảm độ phân giải → đổi kiến trúc → lượng tử hoá → chưng cất → tĩa; và mỗi bước phải kèm một phép đo độ trễ thật trên phần cứng đích, ở chế độ nguồn và nhiệt mà bạn sẽ thực sự chạy. Sản phẩm cuối không phải một mô hình mà là một đường Pareto — chỉ khi có nó bạn mới đang *chọn* thay vì *đoán*. [T2]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Độ phân giải	Kích thước ảnh đưa vào mạng; chi phí tỉ lệ với số điểm ảnh. Hạ từ 640 xuống 512 cắt 36% chi phí bằng một dòng config, đổi lại mất vật nhỏ.
••• Memory-bound so với compute-bound	Memory-bound là bị chặn bởi băng thông, compute-bound là bị chặn bởi năng lực tính. Phần lớn mạng thị giác memory-bound, và đó là lý do lượng tử hoá có tác dụng.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Lượng tử hoá	Biểu diễn trọng số và kích hoạt bằng số nguyên ít bit, thường INT8. Giảm số byte phải nạp; cái giá là sai số làm tròn tích lũy qua các tầng.
●●● Vì sao calibration set sai làm tụt độ chính xác?	Nó quyết định dải giá trị mà hệ số tỉ lệ phủ. Lấy toàn một điều kiện chiếu sáng thì dải hẹp hơn thực tế, nên mọi giá trị ngoài dải bị cắt cụt khi triển khai.
●● PTQ	Lượng tử hoá sau huấn luyện, chỉ cần calibration set và vài phút. Luôn thử trước; chỉ chuyển sang QAT khi nó tụt quá ngưỡng bạn đặt trước.
●● QAT	Mô phỏng làm tròn ngay trong lúc huấn luyện để mạng học cách chịu đựng. Giữ độ chính xác tốt hơn ở bit thấp; cái giá là một chu kỳ huấn luyện đầy đủ.
●●● Per-tensor so với per-channel	Per-tensor dùng một hệ số tỉ lệ cho cả tầng, per-channel dùng một hệ số cho mỗi kênh. Per-channel gần như luôn đúng cho trọng số conv và linear.
●● Vì sao một kênh ngoại lai đủ phá per-tensor?	Hệ số tỉ lệ đặt theo $\max w $. Với ngoại lai 2,0, một trọng số 0,1 chỉ còn sáu mức — chưa tới ba bit hiệu dụng cho phần lớn tầng.
●●● Vì sao tia không cấu trúc không cho tốc độ thật?	Số không rải rác không giảm số byte phải nạp; bạn còn phải nạp thêm chỉ số để biết chỗ nào là không. Ngoài ra kernel dense đã được tối ưu nhiều năm còn kernel thưa thì chưa.
●● 2:4 sparsity	Trong mỗi nhóm bốn trọng số, đúng hai bằng không. Cho tốc độ thật, nhưng chỉ trên phần cứng có đơn vị sparse tensor.
●●● Chưng cất	Huấn luyện student nhỏ bắt chước phân phối mềm của teacher thay vì học nhãn cứng. Giữ được nhiều chất lượng nhất trên mỗi MB; student thừa hưởng cả chỗ teacher sai.
●● Vì sao phân phối mềm giàu thông tin hơn nhãn cứng?	Nó mang quan hệ giữa các lớp: sỏi gần chó hơn mèo. Nhãn cứng vứt đúng phần thông tin đó đi.
●● Tái tham số hoá	Huấn luyện với khối nhiều nhánh rồi gộp chính xác thành một tích chập đơn khi triển khai. Được cả động lực học tốt lẫn đồ thị suy luận gọn.
● NAS	Tự động tìm kiến trúc. Phần lớn dự án không nên dùng: chi phí tìm thường lớn hơn lợi ích, và kết quả gắn chặt với phần cứng dùng để tìm.
●●● Đường Pareto	Tập các cấu hình mà không cấu hình nào trội hơn ở cả độ chính xác lẫn độ trễ. Đây mới là sản phẩm của một chu kỳ nén, không phải một mô hình đơn lẻ.
●● Tia 90% mà độ trễ không đổi?	Bạn đã tia không cấu trúc. Số không rải rác không giảm số byte phải nạp; chỉ tia cả kênh hoặc mẫu 2:4 mới cho tốc độ.
●● Mô hình ít FLOPs hơn nhưng chậm hơn trên Jetson?	Ba nghi phạm: quá nhiều op nhỏ nên chi phí điều phối trội; tích chập tách theo chiều sâu memory-bound; hoặc một op không được runtime hỗ trợ nên rơi về CPU.
●● Lượng tử hoá xong, độ chính xác trung bình không đổi mà hệ thống vẫn tụt đi?	Thiệt hại tập trung ở nhóm hiếm: vật nhỏ, lớp ít mẫu, điều kiện chiếu sáng bất thường. Phải đánh giá theo nhóm mới nhìn thấy.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
TensorRT, pytorch/ao	Lượng tử hoá per-channel, hợp nhất kernel, chọn bố cục bộ nhớ — gần như toàn bộ mạch của chương	Hạ tầng; API đổi nhưng ý tưởng bên dưới ổn định
Sparsity 2:4 trên Tensor Core	Tia bán cấu trúc: ràng buộc mẫu đủ đều để phần cứng khai thác được	Ví dụ rõ nhất cho luận điểm “tốc độ đến từ cấu trúc, không từ tỉ lệ số không”

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Dòng MobileNet, EfficientNet, RepVGG	Kiến trúc hiệu quả và tái tham số hoá — đẩy chi phí sang lúc huấn luyện	RepVGG là minh hoạ sạch nhất; ý tưởng gộp nhánh nay xuất hiện lại trong nhiều detector thời gian thực
Chưng cất mô hình nền tảng xuống student nhỏ	Dùng teacher đóng băng làm nguồn phân phối mềm để đưa mô hình lớp SAM hoặc DINO lên thiết bị	Quan sát thực hành phổ biến 2025–2026; tôi không kiểm chứng được con số cụ thể
bitsandbytes , weight-only INT4	Chỉ nén trọng số khi nút thất là bằng thông đọc trọng số chứ không phải kích hoạt	Là chuẩn mực khi phục vụ LLM trên GPU đơn

MINI PROJECT SLAM · INT8 hoá tầng mô tả và đo đủ ba trục · 1 ngày

Mục tiêu	biến “lượng tử hoá thì nhanh hơn” thành ba con số cụ thể trên phần cứng của bạn. Rồi tự quyết định cuộc đánh đổi có đáng ở ngân sách mili-giây của bạn hay không.
Dữ liệu	mô hình mô tả từ mini project chương 19 (hoặc một mô hình mô tả có sẵn nếu bạn bỏ qua bước đó). Calibration set lấy từ một chuỗi <i>khác</i> với chuỗi đánh giá, để không tự lừa mình.
Các bước	đo baseline fp32 trên đủ ba trục: độ chính xác ghép cặp, độ trễ p50 và p99 trên Orin ở đúng chế độ nvmode1 sẽ triển khai, và dung lượng tệp trọng số. Rồi lần lượt PTQ per-tensor, PTQ per-channel, và QAT nếu PTQ tụt quá ngưỡng bạn đặt trước. Sau <i>mỗi</i> bước đo lại đủ ba trục, không suy đoán từ bước trước.
Xong khi	một bảng bốn dòng — fp32, PTQ per-tensor, PTQ per-channel, QAT — nhân ba cột: độ chính xác ghép cặp, p99, dung lượng. Kèm một câu kết luận <i>có số</i> : “đổi x điểm ghép cặp lấy y ms và z MB, đáng hoặc không đáng ở ngân sách của tôi”. Nếu per-tensor tụt mạnh hơn per-channel, chỉ ra được kênh ngoại lai nào gây ra điều đó.
Nối với	D/20 cho bảng thứ tự đòn bẩy và ví dụ per-tensor so với per-channel; D/21 cho kỷ luật đo (warm-up, phân vị, tải nhiệt) mà mọi con số ở đây phải tuân theo.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì phần cứng không tiết kiệm được gì từ các số không rải rác: bạn vẫn phải nạp dữ liệu và nạp thêm chỉ số để biết chỗ nào là không, kernel dense đã được tối ưu nhiều năm còn kernel thưa thì chưa, và bản thân chỉ số cũng ăn bằng thông. Tốc độ chỉ đến khi cấu trúc đủ đều — 2:4 hoặc tía cả kênh.
- A2. Vì nó đảo ngược được trong một dòng config và cho lợi ích gần như bậc hai theo cạnh ảnh: 640 → 512 cắt 36% chi phí ngay. Nguyên tắc xếp thứ tự là chi phí của việc *sai*, không phải độ tinh vi: sai ở đây tốn nửa giờ, sai ở chưng cất tốn một tuần.
- A3. Vì một hệ số tỉ lệ per-tensor được đặt bởi $\max |w|$. Với ngoại lai 2,0 thì $s = 2,0/127$, nên một trọng số 0,1 chỉ được ánh xạ thành mức 6: toàn bộ dải bình thường còn sáu mức, chưa tới ba bit. Per-channel cho mỗi kênh một s riêng và trả lại đủ 8 bit.
- A4. Ít nhất ba cách. Một: nhiều op nhỏ làm chi phí điều phối trội hơn chi phí tính. Hai: tích chập tách theo chiều sâu có FLOPs thấp nhưng tỉ lệ tính trên byte cũng thấp, nên nó memory-bound. Ba: một toán tử runtime không hỗ trợ bị đẩy về CPU, nuốt trọn lợi ích mà không xuất hiện trong bất kỳ phép đếm FLOPs nào.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn phải nhét một tủ sách vào một chiếc vali có hạn cân. Đó là toàn bộ chương này: mô hình tốt nhất của bạn không vừa cái máy mà nó phải chạy trên. Có năm cách giảm cân, và thứ tự quan trọng hơn kỹ thuật của từng cách. Rẻ nhất là bỏ bớt những cuốn bạn không đọc — tương đương với việc đưa ảnh nhỏ hơn vào máy; làm trong một phút, hồi hận thì làm lại ngay. Kế đó là mua bản bìa mềm thay bìa cứng: cùng nội dung, ít giấy hơn. Rồi mới

đến in chữ nhỏ lại một chút, tức ghi mỗi con số bằng ít chữ số hơn. Cách này rất hiệu quả. Nhưng chỉ cần một trang có cỡ chữ khổng lồ là cả cuốn phải in nhỏ theo, và mọi trang khác thành khó đọc. Sau nữa là nhờ người đọc hộ rồi viết bản tóm tắt. Cuối cùng, cách nghe hay nhất mà thường vô ích nhất: xé bớt chữ trên trang. Bạn xoá được chín phần mười chữ, nhưng vẫn phải mang đủ số tờ giấy, nên vali chẳng nhẹ đi.

Cái giá thì luôn có, và nó không rơi đều: thứ mất đi trước tiên là những cuốn hiếm, những chi tiết nhỏ, những trường hợp bạn ít gặp nhưng lại quan tâm nhất. Và con số duy nhất đáng tin là cái cân ở sân bay, không phải con số bạn tự ước ở nhà.

Một câu: Nén mô hình là xếp vali: làm theo thứ tự rẻ tiền trước, và chỉ tin cái cân ở sân bay chứ đừng tin con số bạn ước ở nhà.

20.6 Kết nối

Chương này là ứng dụng trực tiếp của [B/11-gioi-han-tinh-toan](#): mọi kỹ thuật ở đây chỉ là cách di chuyển trên biểu đồ *roofline*, và biết mình đang ở nhánh nào của biểu đồ đó quyết định kỹ thuật nào có tác dụng. Nó song song với [C/18/02-kinh-te-hoc-pretraining](#): cùng một logic ngân sách, chỉ khác là cột chi đổi từ compute huấn luyện sang compute **suy luận** (*inference*) — và tái tham số hoá cùng chứng cất chính là hai cách chuyển tiền giữa hai cột đó. Nó là tiền đề bắt buộc của [D/21-inference-va-trien-khai](#), nơi mô hình đã nén được đem đi *export* và đo thật, và nơi bạn sẽ phát hiện rằng phần lớn thời gian không nằm ở mô hình. Cuối cùng nó phụ thuộc [B/07-chia-du-lieu-va-danh-gia](#): không có tập đánh giá phân tách theo nhóm hiếm thì bạn không nhìn thấy cái giá thật của việc nén.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao tia không cấu trúc cho sparsity 90% mà không nhanh hơn trên GPU phổ thông, và ba lý do đó lý do nào là lý do phần cứng thật sự?
2. Trong ví dụ per-tensor với một kênh ngoại lai, phần lớn trọng số còn lại bao nhiêu bit hiệu dụng, và per-channel sửa điều đó bằng cách nào?
3. Vì sao giảm độ phân giải đứng đầu danh sách dù nó là kỹ thuật “thô” nhất, và cái giá của nó thường bị bỏ sót ở đâu trong quy trình đánh giá?
4. Khi nào chưng cất thăng lượng tử hoá, và khi nào ngược lại — lập luận qua ràng buộc nào đang chặn bạn?
5. Một mô hình có ít FLOPs hơn nhưng chạy chậm hơn trên Jetson: ba nguyên nhân khả dĩ nhất là gì?
6. Vì sao lượng tử hoá một mô hình đã tia thường tệ hơn dự đoán, và điều đó nói gì về việc coi các kỹ thuật nén là độc lập với nhau?

Inference và triển khai (Inference and Deployment)

PHỤ THUỘC D/20-toi-uu-va-nen-mo-hinh (mô hình đã nén là đầu vào của chương này); B/11-gioi-han-tinh-toan (bảng thông, chỉ phí điều phối); B/08-dich-chuyen-phan-phoi (vì sao chỉ phân phối thật mới lộ ra lỗi thật); D/19/02-pytorch-thanh-thao (kỹ năng phân biệt “lệch do bug” với “lệch do số học”).

TÓM TẮT

Mô hình chạy tốt trong notebook và hỏng trong production. Chương này liệt kê từng chỗ nó hỏng, theo đúng trình tự bạn sẽ gặp: export, sai lệch số học sau export, thời gian nằm ở chỗ bạn không ngờ, xung đột giữa thông lượng và độ trễ đuôi, giới hạn nhiệt của thiết bị biên, và cuối cùng là rủi ro của việc thay mô hình. Đọc xong bạn thiết kế được một quy trình triển khai có kiểm chứng ở từng bước thay vì một chuỗi thao tác hy vọng. Nếu chỉ nhớ một điều: mô hình không có một con số độ trễ — *hệ thống ở một trạng thái* mới có, nên mọi phép đo phải nêu rõ trạng thái nguồn, nhiệt và tải kèm theo, và phải báo cáo p99 chứ không phải trung bình.

KIẾN THỨC CẦN CÓ TRƯỚC

- **export** — quá trình chuyển mô hình từ mã Python/PyTorch sang một biểu diễn đồ thị tĩnh mà một *runtime* khác nạp được — ONNX là định dạng trung gian phổ biến nhất.
- **opset và plugin** — *opset* là phiên bản tập toán tử mà ONNX định nghĩa; toán tử của bạn không nằm trong đó thì phải viết *plugin* cho runtime, hoặc chấp nhận nó bị thay bằng một tổ hợp xấp xỉ.
- **engine** — kết quả biên dịch của TensorRT cho một tổ hợp cụ thể gồm phiên bản thư viện, kiến trúc GPU và cấu hình shape. Đồi bất kỳ thứ nào trong ba thứ đó là phải dựng lại.
- **dynamic shape và optimization profile** — shape đầu vào thay đổi lúc chạy (batch, độ phân giải). TensorRT đòi bạn khai báo trước dải shape (*optimization profile*) để biên dịch kernel phù hợp; không khai báo thì shape bị đóng băng lúc trace.
- **độ trễ p50 / p99** — phân vị của phân phối độ trễ — p99 là ngưỡng mà 99% yêu cầu nằm dưới. Trung bình che giấu đuôi, và đuôi mới là thứ người dùng cảm nhận.
- **batching động (dynamic batching)** — gom các yêu cầu đến gần nhau về thời gian thành một batch để tăng thông lượng, kèm một trần thời gian chờ để không phá ngân sách độ trễ.
- **KV cache** — bộ nhớ đệm lưu khoá và giá trị *attention* của các token đã sinh, để mỗi token mới không phải tính lại toàn bộ chuỗi. Đổi bộ nhớ lấy tốc độ.
- **shadow deployment** — chạy mô hình mới song song với mô hình cũ trên lưu lượng thật nhưng *không* dùng kết quả của nó để phục vụ; chỉ để so sánh và phát hiện lỗi trước khi chuyển đổi.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao con số độ trễ đo trong notebook gần như luôn sai khi lên thiết bị thật, và nó sai theo hướng nào?
- Q2. Bạn tối ưu lượt xuôi nhanh hơn 30% nhưng hệ thống chỉ nhanh hơn 7% — phần còn lại đã trốn ở đâu?
- Q3. Gộp *batch* làm thông lượng tăng gấp ba; vì sao nó vẫn có thể khiến hệ thống của bạn vi phạm ngân sách?

- Q4.** Cùng một engine, cùng một chip, cùng một đầu vào — vì sao phút thứ tám cho con số khác hẳn phút đầu tiên?
- Q5.** Sau khi export, đầu ra lệch 10^{-2} : làm sao biết đó là một bug hay chỉ là số học?

21.1 Đặt vấn đề

Chương này viết cho người đã triển khai thật, nên nó không giải thích lại độ trễ là gì. Câu hỏi dẫn dắt hẹp hơn và khó hơn: *mô hình chạy tốt trong notebook thì hỏng ở đâu khi ra production, và theo thứ tự nào?*

Điểm khiến việc triển khai deep learning khác với việc triển khai một thư viện C++ thông thường không phải độ phức tạp mà là *số lớp dịch*. Giữa mô hình bạn huấn luyện và thứ chạy trên thiết bị có ít nhất ba lần dịch — đồ thị Python sang ONNX, ONNX sang engine của runtime, engine sang kernel cụ thể — và mỗi lần dịch là một cơ hội để ngữ nghĩa thay đổi im lặng. Cộng thêm việc phần lớn phép đo hiệu năng trên GPU dễ đo sai theo những cách đã quen thuộc với người viết CUDA, và bạn có một hệ thống mà mọi con số đều phải được nghi ngờ cho tới khi có bằng chứng.

Học xong chương này thì làm được gì mà trước đó không làm được? Bạn đặt được một quy trình triển khai trong đó *mỗi bước có một phép kiểm chứng riêng*: so khớp từng tầng sau export, đo từng khối trong pipeline, đo p99 dưới tải nhiệt liên tục. Khi có sự cố, bạn khoanh vùng trong một giờ thay vì bởi cả chuỗi.

TRỰC GIÁC

Một mô hình không có độ trễ. Một hệ thống ở một trạng thái mới có độ trễ. Cùng một engine TensorRT trên cùng một Orin cho các con số khác nhau khi: chưa chạy lần nào (kernel autotune và cấp phát chưa xong), vừa chạy được ba mươi giây (mát, xung nhịp cao nhất), chạy liên tục tám phút (đã tụt xung vì nhiệt), hoặc chạy khi tiến trình SLAM bên cạnh đang ăn băng thông bộ nhớ chung.

Đây chính là bài học bạn đã học khi làm benchmark cho SLAM: một lần chạy không phải một phép đo, và hai lần chạy khác nhau không phải một sự bất thường mà là dữ liệu về phương sai. Sự khác biệt duy nhất là ở đây phương sai có thêm một nguồn vật lý — nhiệt độ — vốn có tính *trôi* chứ không phải *ngẫu nhiên*, nên nó không tự triệt tiêu khi bạn lấy trung bình nhiều lần lặp ngắn.

21.2 Mạch vấn đề → giải pháp, theo trình tự triển khai thật

Tám nút thắt, đúng thứ tự bạn sẽ gặp

1 Vấn đề

Giải pháp

Giá phải trả

PyTorch eager quá chậm và quá nặng để triển khai.

export sang một runtime chuyên dụng.

mất tính linh hoạt của Python và nhận về một tạo tác gắn chặt với phiên bản.

2 Vấn đề

Giải pháp

export xong thì đầu ra lệch.

so khớp từng tầng chứ không chỉ kết quả cuối, và kiểm tra bốn nguyên nhân theo đúng thứ tự.

- Giả định bạn có một mẫu đầu vào cố định để so hai phía.
- 3 Vấn đề** **lượt xuôi** (*forward pass*) không phải chỗ tốn thời gian.
Giải pháp đo từng khối — giải mã, tiền xử lý, lượt xuôi, hậu xử lý — trước khi tối ưu bất cứ thứ gì.
 Hệ quả trong hệ thống video, việc đưa giải mã và tiền xử lý lên GPU thường cho lợi ích lớn hơn mọi thứ bạn làm với mô hình.
- 4 Vấn đề** thông lượng và độ trễ xung đột.
Giải pháp batching động kèm trần thời gian chờ.
Giá phải trả p99 xấu đi khi hàng đợi dài, nên trần chờ là tham số thiết kế chứ không phải chi tiết cài đặt.
- 5 Vấn đề** mô hình sinh và VLM có cấu trúc chi phí khác hẳn.
Giải pháp KV cache, giảm số bước sampler, chưng cất sampler, giải mã suy đoán.
 Hệ quả lại là mẫu hình “đẩy chi phí sang training-time” của chương 17 và 20.
- 6 Vấn đề** vận hành.
Giải pháp health check, autoscaling, warm-up bắt buộc, và báo cáo cả p50 lẫn p99.
 Giả định bạn chấp nhận rằng lần chạy đầu luôn chậm và không cố giấu nó bằng cách bỏ qua mẫu đầu.
- 7 Vấn đề** thiết bị biên có giới hạn năng lượng và nhiệt.
Giải pháp đo trong mười phút liên tục ở đúng chế độ nguồn sẽ triển khai, không đo một trăm lần lặp.
 Hệ quả benchmark ngắn nói dối một cách có hệ thống, luôn theo hướng lạc quan.
- 8 Vấn đề** thay mô hình mới là một canh bạc.
Giải pháp shadow deployment rồi A/B test.
 Giả định chỉ phân phối thật mới lộ ra các lỗi mà tập test không có (*B/08-dịch-chuyen-phan-phoi*).

21.3 Export và chọn runtime

A. Bón họ runtime

Bảng 21.1. Bón họ runtime và cái mà mỗi cái đánh đổi.

Runtime	Tối ưu cho	Đánh đổi	Hỏng khi nào
ONNX Runtime	Trung lập, chạy được nhiều nơi	Không nhanh nhất ở bất kỳ đâu	Khi bạn thực sự cần ép sát ngân sách trên một chip cụ thể
TensorRT	GPU và DLA của NVIDIA	Khoá nhà cung cấp; build lâu; engine gắn với phiên bản và kiến trúc	Toán tử lạ phải viết plugin; nâng cấp JetPack là phải dựng lại toàn bộ
OpenVINO	CPU và iGPU của Intel	Hẹp về phần cứng	Ngoài hệ sinh thái Intel
ExecuTorch / CoreML / TFLite	Di động và nhúng	Tập toán tử hạn chế hơn hẳn	Mô hình dùng op động hoặc điều khiển luồng phức tạp

Với một hệ thống thời gian thực trên Orin, hai chi tiết quyết định nhiều hơn bảng trên. Thứ nhất, engine TensorRT phải được *đựng trên chính thiết bị đích* hoặc trên một môi trường trùng khít về phiên bản, vì nó nhúng cả lựa chọn kernel lẫn giả định về kiến trúc GPU. Thứ hai, Orin có DLA bên cạnh GPU; đẩy một phần đồ thị xuống DLA giải phóng GPU cho phần khác của hệ thống, nhưng DLA hỗ trợ ít toán tử hơn nên phần không chạy được sẽ rơi ngược về GPU và tạo ra các lần chuyển đổi tốn kém giữa hai bộ máy.

B. Sai lệch số học sau export — bốn nguyên nhân theo thứ tự

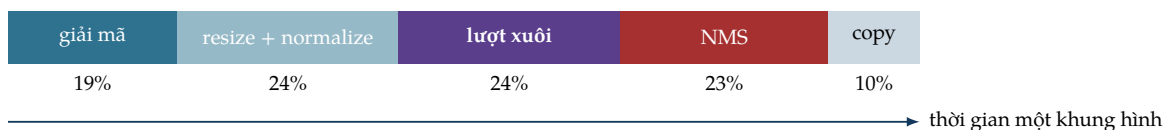
Đây là bước mà kỹ thuật quan trọng hơn kỹ thuật: *so khớp đầu ra từng tầng sau mỗi lần export, không chỉ so kết quả cuối*. Lý do giống hết lý do kiểm tra gradient từng tầng ở chương 19 — giữ không gian nghi vấn ở một phần tử. Khi đã thấy chỗ lệch, kiểm theo thứ tự này:

↪ **cùng nguyên lý với D/19/01** — so khớp từng tầng sau export là đúng một lập luận với kiểm tra gradient từng tầng: giữ không gian nghi vấn ở một phần tử thay vì ở tích các tương tác.

1. **Dynamic shape bị cố định lúc trace.** Triệu chứng đặc trưng: đúng với đúng một kích thước đầu vào, sai hoặc lỗi với mọi kích thước khác. Đây là nguyên nhân phổ biến nhất và cũng rẻ nhất để loại trừ.
2. **Toán tử không được hỗ trợ bị thay bằng bản xấp xỉ.** Triệu chứng: lệch tập trung ở một tầng cụ thể và không giảm khi bạn tăng độ chính xác. Kiểm bằng cách đọc log export — nó thường có cảnh báo mà không ai đọc.
3. **Khác biệt thứ tự phép toán ở fp16.** Triệu chứng: lệch nhỏ, phân tán đều, cỡ epsilon của fp16 và tích lũy dần theo độ sâu. Đây là *số học*, không phải bug — và phân biệt được hai thứ này là kỹ năng bạn đã luyện ở mini project của chương 19.
4. **Tiền xử lý khác nhau giữa hai phía.** Thứ tự kênh BGR so với RGB, giá trị mean/std, cách resize (song tuyến tính có căn góc hay không), và việc chia cho 255 hay không. Triệu chứng: lệch lớn và có hệ thống ngay từ tầng đầu tiên. Nghe tầm thường, nhưng đây là nguyên nhân tốn nhiều giờ người nhất trong danh sách, vì nó nằm ngoài mô hình nên không ai nghĩ tới nó khi đang debug mô hình [43].

21.4 Chỗ thời gian thực sự nằm

Kỳ vọng mặc định là lượt xuôi chiếm phần lớn thời gian. Trong một hệ thống thị giác thật, kỳ vọng đó thường sai. Hình 21.1 minh họa một phân rã điển hình cho một bộ phát hiện chạy trên luồng video — các con số là minh họa cho hình dạng, không phải một phép đo cụ thể.



Hình 21.1. Phân rã thời gian điển hình của một bộ phát hiện trên luồng video (*hình dạng minh họa, không phải số đo*). Phần được viết paper — lượt xuôi — thường chiếm khoảng một phần tư. Tối ưu 30% của một phần tư cho lợi ích tổng 7%; đưa giải mã và tiền xử lý lên GPU cho lợi ích lớn hơn nhiều, và đó là lý do các bộ công cụ như DALI hay DeepStream tồn tại.

Hai chỗ đáng chú ý riêng. **Triệt phi cực đại (non-maximum suppression, NMS)** là hậu xử lý tuần tự và phụ thuộc dữ liệu. Chi phí của nó tăng theo số hộp *trước* ngưỡng. Nên một ngưỡng tin cậy đặt thấp làm p99 xấu đi nhiều hơn p50, đúng ở những khung hình đông đúc mà bạn quan tâm nhất [148]. Sao chép giữa CPU và GPU thường bị bỏ qua hoàn toàn trong phép đo; trên Orin,

bộ nhớ hợp nhất cho phép tránh phần lớn các bản sao này, nhưng chỉ khi bạn cấp phát đúng loại bộ nhớ ngay từ đầu — một quyết định phải làm ở lúc thiết kế chứ không sửa được về sau.

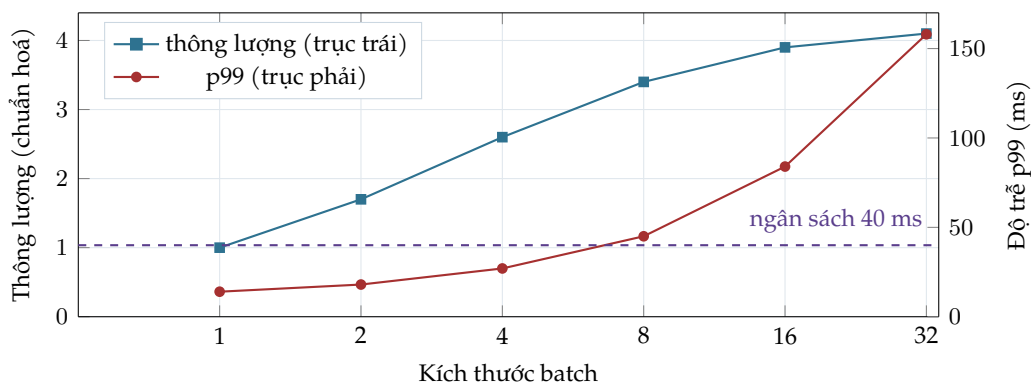
→ **ràng buộc bởi B/11** — trên Orin, GPU và CPU chia chung băng thông bộ nhớ, nên một tiến trình khác đang chạy đủ để đổi con số độ trễ mà không đổi một dòng mã nào.

21.5 Batching động và ngân sách độ trễ

Gộp batch làm thông lượng tăng và độ trễ tăng. Cách đúng để nghĩ về nó là coi trần thời gian chờ là một tham số thiết kế, rồi tính xem tốc độ đến của yêu cầu có đủ để lấp batch trong trần đó không.

Ví dụ nhỏ nhất tính tay được. Ngân sách p99 là 40 ms. Forward cho batch 8 tốn 20 ms, nên còn 20 ms cho hàng đợi. Muốn lấp đầy batch 8 trong 20 ms thì tốc độ đến phải ít nhất $8/0,02 = 400$ yêu cầu mỗi giây. Nếu thực tế chỉ có 100 yêu cầu mỗi giây, trong 20 ms bạn chỉ gom được 2 yêu cầu — nên bạn đang trả cái giá về độ trễ của batching mà chỉ nhận được lợi ích của batch 2.

Kết luận thiết kế: batching động chỉ có lãi khi tốc độ đến vượt ngưỡng $B/T_{\text{chờ}}$. Dưới ngưỡng đó, cấu hình đúng là batch nhỏ hoặc không batch, và mọi nỗ lực chỉnh tham số batching là vô ích. Hình 21.2 cho thấy hình dạng của cuộc đánh đổi.



Hình 21.2. Thông lượng bão hoà trong khi p99 tiếp tục tăng gần tuyến tính theo kích thước batch (số liệu minh hoạ). Sau batch 8 bạn gần như không mua thêm được thông lượng nhưng vẫn trả đủ giá về độ trễ — và đúng ở đó ngân sách bị phá.

21.6 Mô hình sinh và VLM — cấu trúc chi phí khác hẳn

Với mô hình sinh tự hồi quy, chi phí không nằm ở một lượt xuôi mà ở n lượt, mỗi lượt sinh một *token*. KV cache biến bài toán từ “tính lại toàn bộ chuỗi mỗi bước” thành “chỉ tính token mới”: đổi bộ nhớ lấy tốc độ. Nhưng lượng bộ nhớ ấy tăng tuyến tính theo độ dài chuỗi và theo số yêu cầu đồng thời. Nó vì thế trở thành ràng buộc chính của việc phục vụ. Với mô hình khuếch tán, đòn bẩy tương ứng là giảm số bước sampler và chứng cất sampler; với LLM là giải mã suy đoán. Cả ba đều là cùng một mẫu hình: chuyển công việc từ lúc chạy sang lúc huấn luyện.

Điểm cần nhớ cho một hệ thống robotics: đưa một VLM vào vòng điều khiển thời gian thực là một quyết định kiến trúc, không phải một tối ưu. Độ trễ của nó phụ thuộc độ dài đầu ra, tức phụ thuộc *nội dung*, nên nó không có chặn trên tự nhiên — và mọi thành phần thời gian thực nổi vào nó phải được thiết kế với một cơ chế hết giờ và một hành vi dự phòng.

21.7 Vận hành và cách đo cho đúng

Ba nguyên tắc, nêu ngắn vì bạn đã biết chúng ở phía C++:

- **Warm-up là bắt buộc, không phải mẹo.** Lần chạy đầu gán cấp phát bộ nhớ, autotune kernel của cuDNN, và nạp trọng số vào cache. Bỏ vài chục lần lặp đầu là chuẩn mực; không bỏ thì bạn đang báo cáo một phân phối trộn hai chế độ.
- **Đo phân vị, không đo trung bình.** Trung bình che giấu đúng thứ làm người dùng khó chịu và làm hệ thống điều khiển mất ổn định. p99 là con số ràng buộc thiết kế; p50 chỉ để biết hệ thống có đang lành mạnh không.
- **Đồng bộ hoá trước khi bấm giờ.** Lệnh CUDA là bất đồng bộ; một phép đo bằng đồng hồ tường quanh lời gọi chỉ đo thời gian xếp hàng.

21.8 Thiết bị biên: nơi benchmark ngắn nói dối

Trên Jetson và các NPU di động, hai ràng buộc vật lý chi phối mọi thứ: ngân sách năng lượng và tản nhiệt. Hệ quả trực tiếp là *một trăm lần lặp không phải một phép đo*. Chip chạy ở xung nhịp cao khi mát, rồi tụt xung khi nhiệt độ chạm ngưỡng; nếu vòng lặp benchmark của bạn kết thúc trước lúc đó, bạn đo được một con số mà hệ thống sẽ không bao giờ duy trì được.

Quy trình đo tối thiểu cho một thiết bị biên gồm bốn điều kiện, và cả bốn đều phải ghi vào báo cáo: chạy *liên tục ít nhất mười phút*; ở *đúng chế độ nguồn* sẽ triển khai (trên Orin là `nvpmodel`, và cần biết rằng `jetson_clocks` không tự giữ qua lần khởi động lại); *cùng với các tiến trình khác* sẽ chạy song song, vì chúng chia sẻ băng thông bộ nhớ; và ghi lại nhiệt độ cùng xung nhịp theo thời gian chứ không chỉ ghi độ trễ. Nếu đường độ trễ theo thời gian có một bậc thang, bạn vừa tìm thấy ngưỡng nhiệt — và con số đúng để báo cáo là con số *sau* bậc thang đó.

21.9 Thay mô hình an toàn

Thay mô hình mới là canh bạc vì tập test của bạn, dù tốt tới đâu, cũng không phải phân phối thật ([B/08-dich-chuyen-phan-phoi](#)). Quy trình giảm rủi ro có hai bước và thứ tự không đảo được: **shadow deployment** — chạy song song trên lưu lượng thật, ghi lại kết quả nhưng không dùng chúng — rồi **A/B test** trên một phần lưu lượng. Shadow bắt được các lỗi kỹ thuật (tiền xử lý sai, shape sai, độ trễ vượt ngân sách, phân phối đầu ra lệch) mà không gây thiệt hại; A/B mới đo được tác động thật lên chỉ số nghiệp vụ. Bỏ qua bước shadow để đi thẳng vào A/B là cách phổ biến để phát hiện một lỗi tiền xử lý bằng chính người dùng.

↔ **tiền đề B/08** — tập test là một mẫu cố định của quá khứ; chỉ lưu lượng thật mới bộc lộ các vùng phân phối đã trôi kể từ lúc thu dữ liệu.

21.10 Thực hành

Repo và tài nguyên. *Runtime:* [microsoft/onnxruntime](#), [NVIDIA/TensorRT](#), [openvinotoolkit/openvino](#), [pytorch/executorch](#). *Phục vụ:* [triton-inference-server/server](#), [Lightning-AI/LitServe](#). *Pipeline video:* [NVIDIA/DALI](#) và DeepStream. *Phục vụ mô hình sinh*

và VLM: [vllm-project/vllm](https://github.com/vllm-project/vllm), [sgl-project/sglang](https://github.com/sgl-project/sglang). Danh sách chốt tại tháng 9/2026; các runtime hạ tầng ở đây bền hơn nhiều so với tên các bộ phục vụ LLM, vốn thay đổi từng quý.

Câu hỏi kiểm tra hiểu. Sau khi export ONNX, đầu ra lệch 10^{-2} so với PyTorch — bốn nguyên nhân theo thứ tự cần kiểm là gì, và triệu chứng phân biệt chúng? Vì sao p99 mới là con số ràng buộc thiết kế chứ không phải trung bình? Batching động cải thiện thông lượng nhưng làm hỏng gì, và ngưỡng tốc độ đến tối thiểu để nó có lãi được tính thế nào?

Mini project (vài giờ đến hai ngày). Export một bộ phát hiện sang ONNX rồi sang TensorRT. Ở mỗi bước, kiểm tra khớp đầu ra *từng tầng* và đo p50 cùng p99. Lập bảng thời gian theo từng khối: giải mã, tiền xử lý, lượt xuôi, NMS, sao chép. Kết quả đáng giá nhất của bài này thường là phát hiện rằng khối chiếm nhiều thời gian nhất không phải khối bạn định tối ưu.

21.11 Giới hạn và trường hợp hỏng

- **Engine gắn chặt với phiên bản.** Nâng JetPack, đổi phiên bản TensorRT, hay đổi biến thể GPU là phải dựng lại và *kiểm chứng lại* — không phải chỉ dựng lại. Đây là chi phí vận hành thường xuyên chứ không phải chi phí một lần.
- **So khớp từng tầng không bắt được lỗi phụ thuộc dữ liệu.** Một mẫu đầu vào cố định không kích hoạt nhánh điều khiển hiếm; NMS với số hộp bất thường, hoặc một shape ngoài optimization profile, vẫn có thể hỏng sau khi mọi phép so đã qua.
- **p99 đo trên lưu lượng tổng hợp không dự báo p99 thật.** Lưu lượng thật có cụm và có tương quan; một bộ sinh yêu cầu đều đặn cho phân phối hàng đợi lạc quan hơn nhiều so với thực tế.
- **Shadow deployment không phát hiện được vấn đề về vòng phản hồi.** Nếu mô hình mới thay đổi hành vi hệ thống và hành vi đó thay đổi phân phối đầu vào, chỉ A/B mới thấy — shadow theo định nghĩa không tác động gì nên không tạo ra vòng lặp đó.

CẠM BÃY

Ba cách đo sai làm bạn tin vào một con số không tồn tại. **Một:** bấm giờ bằng đồng hồ tường quanh một lời gọi CUDA mà không đồng bộ hoá — bạn đo thời gian *xếp hàng lệnh*, thường nhỏ hơn thời gian chạy thật một bậc. **Hai:** không warm-up, rồi lấy trung bình gồm cả lần chạy đầu vốn gánh autotune và cấp phát; con số ra vừa sai vừa có phương sai lớn giả tạo. **Ba:** benchmark một trăm lần lặp trên thiết bị biên đang mát, rồi báo cáo con số đó như hiệu năng bền vững. Hệ thống sẽ chạy chậm hơn ngay khi vào ca làm việc thật. Và không ai tái hiện được vấn đề, vì mọi người đều benchmark theo cách cũ.

CẦN NHỚ

Một phép đo độ trễ chỉ có nghĩa khi kèm trạng thái: chế độ nguồn, nhiệt độ đã bão hoà chưa, có tiến trình nào chạy cùng không, đã warm-up chưa, và đó là phân vị nào. Thiếu bất kỳ mục nào trong danh sách đó thì con số không tái hiện được và không dùng để ra quyết định được. Và nguyên tắc bao trùm cả chương: sau mỗi lần dịch — Python sang ONNX, ONNX sang engine — phải có một phép kiểm chứng riêng, vì mỗi lần dịch là một cơ hội để ngữ nghĩa đổi im lặng. [T2]

■ **Active recall** (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Export	Chuyển mô hình từ mã Python sang một đồ thị tĩnh mà runtime khác nạp được; ONNX là định dạng trung gian phổ biến nhất. Mỗi lần dịch là một cơ hội đổi ngữ nghĩa im lặng.
●● Opset và plugin	Opset là phiên bản tập toán tử ONNX; plugin là phần bạn phải tự viết khi runtime không có op đó. Không viết thì op bị thay bằng một xấp xỉ.
●●● Engine TensorRT	Kết quả biên dịch cho một tổ hợp cụ thể gồm phiên bản thư viện, kiến trúc GPU và dải shape. Đối bất kỳ thứ nào là phải dựng lại và kiểm chứng lại.
●● Optimization profile	Khai báo trước dải shape động để TensorRT sinh kernel phù hợp. Không khai báo thì shape bị đóng băng lúc trace.
●● Vì sao phải so khớp đầu ra từng tầng sau export?	Để giữ không gian nghi vấn ở một tầng thay vì cả chuỗi ba lần dịch. Cùng lập luận với kiểm tra gradient từng tầng ở chương 19.
●●● Lệnh sau export	Theo thứ tự kiểm: shape động bị cố định lúc trace; op không hỗ trợ bị thay bằng xấp xỉ; khác thứ tự phép toán ở fp16; và tiền xử lý lệch giữa hai phía.
●●● Triệt phi cực đại	Hậu xử lý tuần tự, phụ thuộc dữ liệu, chi phí tăng theo số hộp trước ngưỡng. Ngưỡng tin cậy thấp làm p99 xấu đi nhiều hơn p50.
●●● Vì sao phải warm-up trước khi đo?	Lần chạy đầu gánh cấp phát bộ nhớ, autotune kernel và nạp trọng số vào cache. Không bỏ chúng đi thì bạn báo cáo một phân phối trộn hai chế độ.
●●● Vì sao p99 mới là con số ràng buộc thiết kế?	Vì trung bình che đậy cái đuôi mà người dùng cảm nhận và mà hệ điều khiển phải chịu. p50 chỉ để biết hệ thống có đang lành mạnh không.
●● Batching động	Gom yêu cầu đến gần nhau thành một batch, kèm trần thời gian chờ. Chỉ có lỗi khi tốc độ đến vượt $B/T_{\text{chờ}}$.
●● KV cache	Lưu khoá và giá trị attention của các token đã sinh để không tính lại chuỗi mỗi bước. Bộ nhớ tăng tuyến tính theo độ dài chuỗi và số yêu cầu đồng thời.
●● DLA trên Orin	Bộ máy suy luận riêng bên cạnh GPU, dùng để giải phóng GPU cho phần khác của hệ thống. Hỗ trợ ít op hơn, nên phần không chạy được rơi ngược về GPU.
●● Vì sao bấm giờ quanh một lời gọi CUDA cho số sai?	Lệnh CUDA bất đồng bộ, nên đồng hồ tường chỉ đo thời gian xếp hàng. Phải đồng bộ hoá hoặc dùng CUDA event.
● Shadow deployment	Chạy mô hình mới song song trên lưu lượng thật nhưng không dùng kết quả. Bắt lỗi kỹ thuật mà không gây thiệt hại; không bắt được vấn đề vòng phản hồi.
●● Thông lượng tăng gấp ba mà p99 vượt ngân sách?	Batch quá lớn. Thông lượng bão hoà trong khi p99 vẫn tăng gần tuyến tính, nên sau một điểm bạn trả giá mà không mua thêm được gì.
●● Đầu ra đúng với một kích thước ảnh, sai với mọi kích thước khác?	Shape động đã bị cố định lúc trace, hoặc thiếu optimization profile. Đây là nguyên nhân phổ biến nhất và rẻ nhất để loại trừ.
●● Độ trễ ổn định tám phút rồi nhảy một bậc?	Chíp chạm ngưỡng nhiệt và tụt xung. Con số đúng để báo cáo là con số sau bậc thang đó.
●● Tối ưu lướt xuôi nhanh hơn 30% mà hệ thống chỉ nhanh hơn 7%?	Lướt xuôi chỉ chiếm khoảng một phần tư thời gian. Phần còn lại là giải mã, tiền xử lý, triệt phi cực đại và sao chép giữa CPU với GPU.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ONNX Runtime, TensorRT, OpenVINO	Toàn bộ mạch export → engine → kernel, kể cả optimization profile cho shape động	Hạ tầng; ONNX là định dạng trung gian bền nhất trong nhóm
Triton Inference Server	Batching động kèm trần thời gian chờ — đúng cuộc đánh đổi thông lượng với p99 ở mức batching	Hạ tầng phục vụ; cơ chế batching của nó là hiện thực sạch của bất đẳng thức $B/T_{\text{chờ}}$
vllm, sglang	KV cache và lập lịch yêu cầu cho mô hình sinh, tức đổi bộ nhớ lấy tốc độ ở quy mô nhiều người dùng	Nhóm công cụ đổi nhanh nhất trong danh sách; ý tưởng KV cache thì không đổi
DALI, DeepStream	Đưa giải mã và tiền xử lý lên GPU vì chúng, chứ không phải lượt xuôi, mới chiếm phần lớn thời gian	Đúng kết luận của Hình 21.1, đóng gói thành sản phẩm
DLA trên Jetson Orin	Đẩy một phần đồ thị sang bộ máy khác để giải phóng GPU cho phần còn lại của hệ thống	Chỉ có lái khi toàn bộ nhánh chạy được trên DLA; op không hỗ trợ rơi ngược về GPU và tạo chuyển đổi tốn kém

MINI PROJECT SLAM • Phân rã độ trễ của một pipeline SLAM có thành phần học • 1–2 ngày

Mục tiêu	tìm ra khối nào thật sự ăn thời gian trong hệ của bạn. Rồi đo nó theo cách mà con số còn đúng sau mười phút chạy liên tục, không chỉ trong một trăm lần lặp.
Dữ liệu	hệ SLAM của bạn chạy trên Orin với một chuỗi EuRoC phát lại ở tốc độ thật, có cảm tăng mô tả INT8 từ mini project chương 20 vào frontend.
Các bước	bọc năm khối bằng CUDA event — giải mã và hiệu chỉnh méo, phát hiện đặc trưng, mô tả bằng mạng, ghép cặp cùng RANSAC, và tối ưu — chứ không bấm giờ bằng đồng hồ tường. Bỏ 200 khung hình đầu làm warm-up. Chạy liên tục mười phút ở đúng chế độ nvmodel sẽ triển khai, với các tiến trình thật chạy song song. Ghi lại nhiệt độ và xung nhịp theo thời gian cùng với độ trễ.
Xong khi	bạn chỉ ra được khối chiếm nhiều thời gian nhất, kèm phần trăm của nó. Bạn nói được p99 cao hơn trung bình bao nhiêu phần trăm. Và trên đồ thị độ trễ theo thời gian, bạn chỉ được bậc thang do tụt xung vì nhiệt — hoặc chứng minh bằng đường nhiệt độ rằng nó chưa xuất hiện trong mười phút.
Nối với	D/21 cho hình phân rã thời gian và cuộc đánh đổi thông lượng với p99; B/11 cho lý do bằng thông chung trên Orin làm con số đôi khi có tiến trình khác. Bộ đo dựng ở đây chính là thứ mini project chương 22 đem đi kiểm tính tái lập.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì notebook đo một hệ thống ở trạng thái tốt nhất: đã warm-up hoặc chưa, mát, không có tiến trình nào tranh băng thông, và thường bấm giờ bằng đồng hồ tường quanh một lệnh CUDA bất đồng bộ. Cả bốn sai lệch đều đi cùng một hướng — lạc quan.
- A2. Vào những khối bạn không đo: giải mã, resize và chuẩn hoá, triết phi cực đại, và các bản sao giữa CPU với GPU. Nếu lượt xuôi chỉ chiếm một phần tư thời gian thì 30% của một phần tư đúng bằng 7,5% tổng — số học chứ không phải nghịch lý.
- A3. Vì thông lượng bão hoà trong khi p99 tiếp tục tăng gần tuyến tính theo kích thước batch. Sau một điểm nào đó bạn gần như không mua thêm được thông lượng nhưng vẫn trả đủ giá độ trễ, và nếu tốc độ đến nhỏ hơn $B/T_{\text{chờ}}$ thì bạn còn không lấp nổi batch trong trần chờ.
- A4. Vì phút thứ tám là một trạng thái khác: chip đã chạm ngưỡng nhiệt và tụt xung. Đường độ trễ theo thời gian có một bậc thang, và con số đúng để báo cáo là con số *sau* bậc thang đó.
- A5. Bằng triệu chứng. Lệch do số học thì nhỏ, phân tán đều, cỡ epsilon của fp16 và tích lũy dần theo độ sâu. Lệch do bug thì tập trung ở một tầng và không giảm khi bạn tăng độ chính xác — hoặc lớn và có hệ thống ngay từ tầng đầu, dấu hiệu kinh điển của tiền xử lý lệch.

NÓI LẠI THẬT ĐƠN GIẢN

Một nhà hàng bị chê phục vụ chậm. Chủ quán liền thuê đầu bếp giỏi hơn, nấu nhanh hơn ba mươi phần trăm — và khách vẫn chê. Lý do: nấu chỉ chiếm một phần tư thời gian một món. Phần còn lại là nhặt rau, rửa, thái, xếp đĩa và bưng ra. Muốn nhanh thì phải bấm giờ từng khâu trước đã, chứ không phải cải thiện đúng cái khâu ai cũng nói tới.

Chuyện thứ hai là cách báo cáo. Nói “trung bình hai mươi phút một món” nghe rất ổn, nhưng khách nhớ cái lần họ chờ năm mươi phút. Con số ràng buộc thiết kế là cái lần xui xẻo đó, không phải trung bình.

Chuyện thứ ba là trạng thái của bếp. Bếp lúc mới mở khác hẳn bếp đã chạy tám phút liên tục: nóng lên thì mọi thứ chậm lại. Đo mười lượt lúc bếp còn nguội rồi công bố con số ấy là tự lừa mình.

Chuyện thứ tư là gộp đơn. Nấu một lượt cho tám bàn thì tổng số món phục vụ được nhiều hơn hẳn, nhưng bàn đến trước phải ngồi đợi cho đủ tám. Nếu quán chỉ có một trăm khách một giờ thì chờ cho đủ tám bàn là quá lâu, và bạn đang trả cái giá của gộp đơn mà không nhận được lợi ích của nó.

Cái giá chung của cả chương: mọi con số bạn đo chỉ đúng kèm điều kiện — bếp nóng hay nguội, đông hay vắng, đã chạy thử chưa.

Một câu: Đừng hỏi mô hình chạy nhanh bao nhiêu; hãy hỏi cả hệ thống, ở đúng trạng thái thật, phục vụ được người khách xui xẻo thứ chín mươi chín trong một trăm.

21.12 Kết nối

Chương này nhận đầu vào từ [D/20-toi-uu-va-nen-mo-hinh](#) và trả lại cho nó một bài học ngược: mọi phép đo dùng để vẽ đường Pareto ở chương 20 chỉ hợp lệ nếu được đo theo đúng kỷ luật của chương này, nên hai chương thực chất là một vòng lặp chứ không phải một chuỗi. Nó là ứng dụng của [B/11-gioi-han-tinh-toan](#) ở phía **suy luận** (*inference*), và nó dựa vào [B/08-dich-chuyen-phan-phoi](#) cho toàn bộ lập luận về shadow deployment. Về phía sau, nó là tiền đề của [D/22-tai-lap-va-mlops](#): một khi mô hình đã chạy trong production, câu hỏi đổi từ “nó có nhanh không” sang “sáu tháng nữa tôi có dựng lại được đúng nó không, và tôi có biết nó đang xuống cấp không”. Và nó nối thẳng sang [E/25-do-tin-cay-va-van-hanh](#) cho phần hành vi khi mô hình gặp đầu vào ngoài phân phối.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Sau khi export ONNX, đầu ra lệch 10^{-2} — bốn nguyên nhân theo thứ tự cần kiểm là gì, và triệu chứng nào phân biệt “lệch do số học” với “lệch do toán tử bị thay thế”?
2. Vì sao p99 là con số ràng buộc thiết kế còn trung bình thì không, và điều đó đổi cách bạn chọn ngưỡng tin cậy của NMS như thế nào?
3. Với ngân sách p99 là 40 ms và lượt xuôi batch 8 tốn 20 ms, tốc độ đến tối thiểu để batching động có lãi là bao nhiêu, và dưới ngưỡng đó cấu hình đúng là gì?
4. Vì sao một benchmark một trăm lần lặp trên Jetson nói dối, và phép đo tối thiểu để không bị nó lừa gồm những điều kiện nào?
5. Vì sao engine TensorRT phải dựng lại sau khi nâng JetPack, và vì sao “dựng lại” là chưa đủ?
6. Shadow deployment bắt được loại lỗi nào mà tập test không bắt được, và loại lỗi nào nó vẫn không bắt được?

Tái lập và MLOps tối thiểu (Reproducibility and Minimal MLOps)

PHỤ THUỘC B/06-chat-luong-du-lieu và B/07-chia-du-lieu-va-danh-gia (dữ liệu là thứ thay đổi nhiều nhất); B/08-dich-chuyen-phan-phoi (vì sao cần giám sát sau triển khai); D/19/02-pytorch-thanh-thao (seed, chế độ tắt định); D/21-inference-va-trien-khai (mô hình đã ở production); E/24-thi-nghiem-va-ablation (một thay đổi = một config diff).

TÓM TẮT

Sáu tháng sau, bạn cần biết chính xác cái gì đã tạo ra con số này — mọi công cụ trong chương tồn tại vì trí nhớ con người không làm được việc đó. Chương đi qua config tập trung, theo dõi thí nghiệm, quản lý phiên bản dữ liệu, đóng gói artifact đầy đủ, các nguồn bất định trên GPU, CI cho mô hình, và giám sát drift sau triển khai. Đọc xong bạn dựng lại được một kết quả cũ chỉ từ một config, và — quan trọng hơn — biết khi nào việc dựng lại *từng chữ số* là mục tiêu sai. Nếu chỉ nhớ một điều: mục tiêu đúng không phải tái lập từng bit mà là tái lập *phân phối* kết quả, nên một con số không kèm phương sai không phải bằng chứng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **seed** — số khởi tạo mọi bộ sinh ngẫu nhiên trong một lần chạy — khởi tạo trọng số, thứ tự trộn dữ liệu, augmentation, dropout. Cùng seed không đủ để cùng kết quả, nhưng khác seed thì chắc chắn khác kết quả.
- **config** — tệp mô tả toàn bộ siêu tham số và lựa chọn của một lần chạy, tách khỏi mã nguồn. Nếu hai lần chạy khác nhau mà không diff được config thì bạn không ablate được.
- **experiment tracking** — hệ thống ghi lại mỗi lần chạy kèm commit mã, config, phiên bản dữ liệu và các chỉ số theo thời gian, để sau này truy ngược được từ một con số về nguồn gốc của nó.
- **data versioning** — quản lý phiên bản cho dữ liệu giống như git quản lý mã: mỗi trạng thái của tập dữ liệu có một mã băm, và một lần chạy trở tới đúng mã băm đó.
- **artifact** — gói đầy đủ cần để tái hiện hoặc triển khai một mô hình: trọng số, config, phiên bản thư viện, phiên bản dữ liệu, và mã tiền xử lý. Chỉ có checkpoint là chưa đủ.
- **tính tắt định (determinism)** — chạy lại cùng đầu vào cho ra cùng đầu ra tới từng bit. Trên GPU nó không mặc định, và ép nó phải trả giá bằng tốc độ.
- **CI cho mô hình** — một tập kiểm tra rẽ chạy tự động mỗi commit — overfit một batch, kiểm shape, kiểm export thành công, kiểm chỉ số không tụt quá ngưỡng trên một tập nhỏ cố định.
- **drift** — sự trôi dạt của phân phối dữ liệu thật so với phân phối lúc huấn luyện. Nó làm mô hình xuống cấp mà không có lỗi nào được ném ra.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn cố định *seed*, chạy lại đúng mã trên đúng máy, và vẫn nhận hai con số khác nhau — ai đã đổi thứ tự phép cộng sau lưng bạn?
- Q2. Vì sao ép mọi thứ tắt định lại là một mục tiêu *sai*, dù nó nghe đúng hết thứ một kỹ sư nên làm?

- Q3.** Cấu hình mới hơn cấu hình cũ 0,3 mAP. Vì sao con số đó có thể hoàn toàn vô nghĩa, và cần bao nhiêu lần chạy để nó có nghĩa?
- Q4.** Sáu tháng sau bạn có checkpoint, có mã, có config — và vẫn không dựng lại được kết quả cũ. Thứ còn thiếu thường là gì?

22.1 Đặt vấn đề

Câu hỏi dẫn dắt rất cụ thể và rất quen: *sáu tháng sau, bạn cần biết chính xác cái gì đã tạo ra con số này*. Không phải “đại khái cấu hình nào”, mà là chính xác: commit nào, config nào, phiên bản dữ liệu nào, thư viện phiên bản mấy, chạy trên GPU gì. Mọi công cụ trong chương tồn tại vì trí nhớ con người không làm được việc đó, và vì thư mục `runs/exp_final_v2_real_final` không phải một hệ thống.

Nhưng chương này còn một nửa thứ hai, và nửa ấy mới là phần khó. Ngay cả khi bạn đã ghi lại mọi thứ và cố định seed, kết quả *vẫn* không lặp lại chính xác. Người đã debug bắt định trong một hệ thống C++ đã luồng biết ngay tại sao: khi thứ tự các phép toán không được xác định, kết quả số học cũng không được xác định. Trong deep learning, số nguồn gây ra điều đó nhiều hơn hẳn, và phần lớn nằm dưới tầng trừu tượng mà bạn không nhìn thấy.

Học xong chương này thì làm được gì mà trước đó không làm được? Bạn phân biệt được ba tình huống mà người ta hay gộp làm một: *có bug, có bất định nhưng vẫn đúng, và có thay đổi thật nhưng bị nhiễu che*. Ba tình huống ấy đòi ba hành động hoàn toàn khác nhau, và nhằm chúng là cách phổ biến nhất để hoặc đuổi theo một con ma, hoặc công bố một cải tiến không tồn tại.

TRỰC GIÁC

Coi mỗi con số bạn báo cáo là giá trị của một hàm băm với rất nhiều đối số: mã nguồn, config, dữ liệu, phiên bản thư viện, kiến trúc GPU, seed, và cả những thứ bạn không kiểm soát như thứ tự hoàn thành của các luồng. Kỹ thuật tái lập không phải là “làm cho hàm băm trở nên tắt định” — điều đó thường bất khả thi và luôn đắt. Nó là hai việc: *ghim càng nhiều đối số càng tốt, và đo phương sai gây ra bởi những đối số còn lại*.

Bạn đã sống qua đúng bài học này. Khi một benchmark SLAM cho kết quả khác nhau giữa các lần chạy, cách chữa không phải là ép mọi thứ tắt định rồi đi tiếp. Cách chữa là tách làm hai chế độ. Chế độ ghim chặt — một luồng, thứ tự cố định — *chỉ* dùng để cô lập bug. Chế độ chạy thật lặp nhiều lần dùng để báo cáo, kèm khoảng biến thiên. Chương này là cùng một kiến trúc suy nghĩ, áp cho một hệ thống có nhiều nguồn bất định hơn.

22.2 Ghim các đối số kiểm soát được

A. Bốn lớp phải ghim

Bốn nút thắt của việc truy ngược một con số

1 Vấn đề

siêu tham số rải rác trong code.

Giải pháp

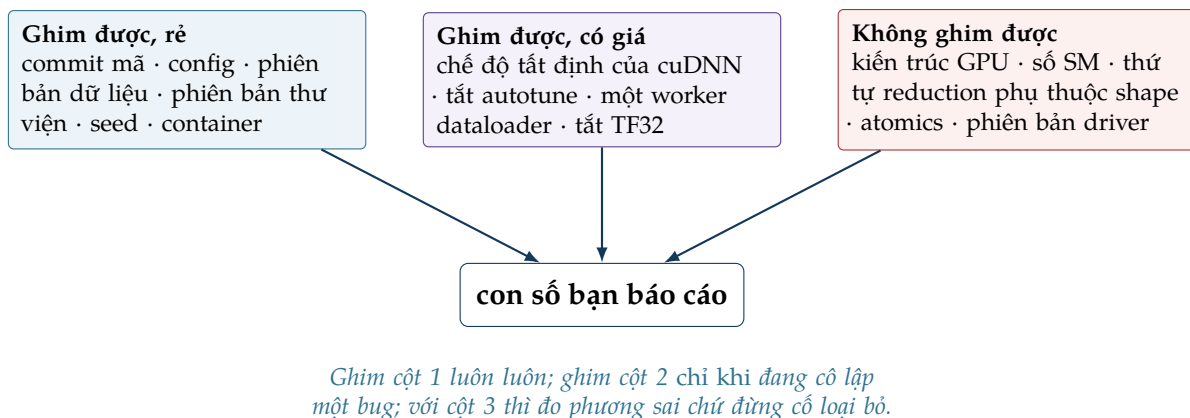
config tập trung (Hydra, hoặc YAML thuần cũng được).

Giả định

bạn tuân thủ nguyên tắc *một thay đổi = một config diff*.

Hệ quả	nếu không diff được thì không ablate được, và toàn bộ E/24-thi-nghiem-va-ablation sụp đổ.
2 Vấn đề	không biết lần chạy nào ứng với mã nào.
Giải pháp	experiment tracking gắn mỗi lần chạy với commit, config và phiên bản dữ liệu — tự động, không dựa vào kỷ luật của con người lúc hai giờ sáng.
3 Vấn đề	vấn đề bị bỏ quên nhiều nhất: dữ liệu thay đổi nhiều hơn mã.
Giải pháp	data versioning.
Giả định	mọi so sánh giữa hai thời điểm chỉ có nghĩa khi bạn biết dữ liệu có đổi hay không.
Vấn đề mới	không có nó thì một cải tiến 2 điểm có thể chỉ là kết quả của việc ai đó dọn lại nhân tuần trước.
4 Vấn đề	checkpoint không đủ để chạy lại.
Giải pháp	lưu artifact đầy đủ: trọng số, config, phiên bản thư viện, phiên bản dữ liệu, và mã tiền xử lý.
Hệ quả	đúng cái tiền xử lý mà chương 21 đã chỉ ra là nguyên nhân tổn giờ người nhất khi export.

Hình 22.1 liệt kê toàn bộ đối số của hàm bấm ấy và chia chúng theo mức độ bạn kiểm soát được. Cách đọc hình: cột trái ghim được bằng công cụ, cột giữa ghim được nhưng phải trả giá, cột phải không ghim được và chỉ có thể đo phương sai.



Hình 22.1. “Chữ ký” của một con số. Sai lầm thường gặp là dồn công sức vào cột 2 — ép tắt định bằng mọi giá — trong khi cột 1 còn chưa ghim đủ và cột 3 thì vốn không thể ghim.

22.3 Các nguồn bất định — và vì sao chế độ tắt định không đủ

Đây là phần cốt lõi của chương và là phần nổi thẳng vào kinh nghiệm bạn đã có. Bảng 22.1 liệt kê các nguồn theo cơ chế, vì hiểu cơ chế thì bạn tự suy ra được nguồn mới khi gặp thư viện lạ.

Bảng 22.1. Các nguồn bất định trong huấn luyện GPU, theo cơ chế.

Nguồn	Cơ chế	Ghim thế nào	Còn lại gì sau khi ghim
Autotune của cuDNN	Chọn kernel bằng cách <i>bấm giờ</i> lúc chạy; máy đang bận thì chọn kernel khác, và kernel khác cộng theo thứ tự khác	<code>cuda.nv.benchmark = False</code>	Chậm hơn ở các shape cố định
Atomics dấu phẩy động	<code>atomicAdd</code> cộng theo thứ tự luồng hoàn thành; cộng số thực không kết hợp nên tổng khác nhau ở bit cuối	<code>use_deterministic_algorithms(True)</code>	Một số op không có bản tắt định → ném lỗi thay vì chạy
Thứ tự reduction	Chia khối phụ thuộc shape và số SM của GPU	Không ghim được	Đổi GPU là đổi kết quả, kể cả khi mọi thứ khác giống hệt
TF32 và fp16	Giảm số bit phân định trị của phép nhân ma trận; kết quả đúng nhưng khác	Tắt TF32, dùng fp32	Chậm hơn nhiều
DataLoader nhiều worker	Mỗi worker có luồng ngẫu nhiên riêng; số worker đổi là chuỗi augmentation đổi	Cố định số worker và seed từng worker	Không còn tự do chỉnh hiệu năng
Thứ tự duyệt tập hợp	Duyệt một set các đối tượng Python, hay <code>glob</code> không <code>sort()</code> , cho thứ tự phụ thuộc địa chỉ bộ nhớ hoặc hệ tệp	Luôn <code>sorted()</code> , luôn khoá theo id ổn định	Không mất gì — đây là lỗi thuần tuý, nên sửa

Dòng cuối đáng dừng lại vì nó là *cùng một lỗi* mà bạn đã gặp ở phía SLAM: một tập hợp được sắp theo địa chỉ con trỏ thì thứ tự duyệt phụ thuộc vào cách bộ cấp phát tình cờ xếp bộ nhớ trong lần chạy đó. Trong C++ nó là `std::set<T*>`; trong Python nó là một set các đối tượng không có `__hash__` tùy biến, hay một `os.listdir` chưa sắp. Điểm chung, và cũng là điểm đáng nhớ nhất: *đây không phải bất định số học mà là một bug*, nên nó không thuộc cột “không ghim được” — nó phải được sửa, và sửa xong thì không mất gì cả.

Từ đó rút ra phát biểu trung thực về giới hạn: tái lập chính xác tới từng bit giữa hai máy khác GPU thường *không khả thi*, vì thứ tự reduction phụ thuộc số SM và cách chia khối. Chế độ tắt định của PyTorch giải quyết được các nguồn trong cùng một máy, không giải quyết được nguồn giữa các máy. Mục tiêu đúng vì thế không phải tái lập từng chữ số mà là tái lập *phân phối* kết quả.

→ **hệ quả của D/19/02** — chế độ tắt định của PyTorch chỉ chặn được các nguồn trong cùng một máy, nên nó mua một đảm bảo hẹp hơn nhiều so với cái tên gợi ra.

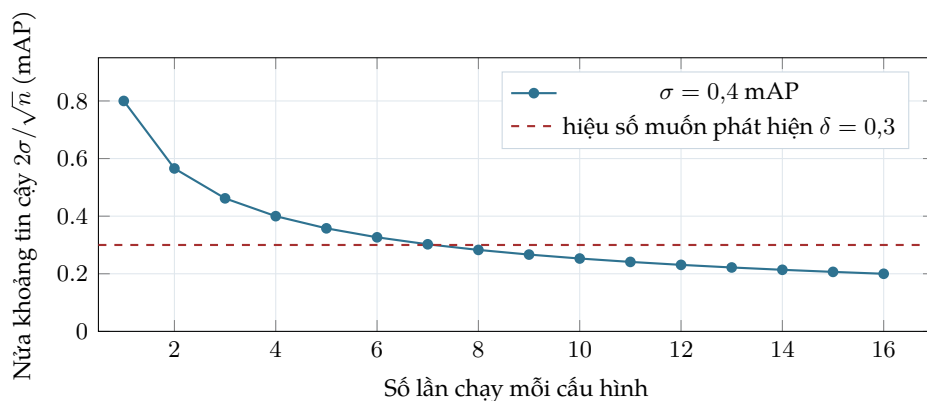
A. Ví dụ nhỏ nhất: bao nhiêu lần chạy thì đủ

Giả sử độ lệch chuẩn giữa các seed của chỉ số bạn quan tâm là $\sigma = 0,4$ điểm mAP — một giá trị không hiếm với dữ liệu vừa. Bạn thấy cấu hình mới hơn cấu hình cũ 0,3 điểm trong một lần chạy mỗi bên. Con số đó nói gì? Gần như không gì cả, vì nó nhỏ hơn cả độ lệch chuẩn của nhiễu.

Chạy n lần mỗi bên thì sai số chuẩn của trung bình là $\sigma/\sqrt{n}$. Muốn khoảng ± 2 sai số chuẩn hẹp hơn hiệu số δ mà bạn muốn phát hiện, cần

$$\frac{2\sigma}{\sqrt{n}} < \delta \iff n > \left(\frac{2\sigma}{\delta}\right)^2.$$

Ý nghĩa: độ nhạy của thí nghiệm tăng theo *căn* số lần chạy, nên muốn phát hiện hiệu số nhỏ đi một nửa thì phải chạy nhiều gấp bốn. Thay số: với $\sigma = 0,4$ và $\delta = 0,3$, ta cần $n > (0,8/0,3)^2 = 7,1$, tức tối thiểu tám lần chạy mỗi cấu hình. Nếu không đủ ngân sách cho tám lần chạy, kết luận đúng không phải “chạy một lần rồi tin”. Kết luận đúng là “hiệu số 0,3 nằm ngoài khả năng phân giải của tôi”. Đó là một kết luận hợp lệ và đáng viết ra. Hình 22.2 cho thấy vì sao.



Hình 22.2. Độ nhảy của một so sánh tăng theo căn bậc hai số lần chạy. Hai đường cắt nhau ở $n \approx 7$: dưới ngưỡng đó, mọi kết luận về hiệu số 0,3 mAP đều là đọc nhiều.

22.4 CI cho mô hình và giám sát sau triển khai

A. Bốn kiểm tra rẻ chạy mỗi commit

Mã học máy có một đặc điểm khó chịu: nó hiếm khi hỏng bằng cách ném lỗi. Vì vậy CI cho mô hình không thể là “chạy unit test” mà phải là một tập kiểm tra nhắm vào các chế độ hỏng im lặng, và cả bốn đều phải rẻ để không ai muốn tắt chúng:

↔ **tiền đề E/24** — không có config diff được và không có phương sai đo được thì mọi ablation chỉ là kể chuyện; CI là cách ép hai điều kiện đó thành thói quen.

1. **Overfit một batch.** Cho mô hình học thuộc đúng một batch nhỏ; loss phải xuống gần không trong vài chục bước. Nếu không, có lỗi ở gradient, ở nhãn, hoặc ở chỗ nối — và bạn biết ngay trong một phút thay vì sau một đêm huấn luyện.
2. **Kiểm shape và dtype** ở mọi ranh giới khối, đặc biệt sau khi đổi độ phân giải hoặc số lớp.
3. **Kiểm export thành công** sang định dạng triển khai, và so khớp đầu ra với một mẫu cố định. Điều này biến cả chương 21 thành một bài kiểm tra tự động thay vì một sự kiện đau đớn mỗi quý.
4. **Kiểm chỉ số không tụt quá ngưỡng** trên một tập nhỏ cố định. Ngưỡng phải đặt dựa trên phương sai giữa các seed đã đo, nếu không bạn sẽ có một bài kiểm tra hỏng ngẫu nhiên — và một bài kiểm tra hỏng ngẫu nhiên sẽ bị vô hiệu hoá trong hai tuần.

B. Sau triển khai: drift và vòng lặp dữ liệu

Mô hình xuống cấp sau khi triển khai mà không có lỗi nào được ném ra, vì thế giới đổi còn trọng số thì không. Hai việc tối thiểu. Thứ nhất là **giám sát drift**: theo dõi phân phối đầu vào và phân phối đầu ra, chứ không chỉ độ chính xác. Lý do là chân trị thường đến muộn hoặc không bao giờ đến. Thứ hai là **thu thập lỗi làm dữ liệu mới**, tức biến các trường hợp mô hình sai hoặc không chắc thành hàng đợi gán nhãn.

Bước thứ hai là chỗ Phần D nói ngược về Phần B và đóng vòng của cả tài liệu. Dữ liệu sinh ra mô hình; mô hình chạy trong thực tế; thực tế sinh ra dữ liệu mới. Chất lượng của vòng lặp ấy, chứ không phải kiến trúc, mới quyết định hệ thống tốt lên hay xấu đi theo thời gian.

↔ **hệ quả của B/08** — drift là dịch chuyển phân phối quan sát từ phía vận hành; thu thập lỗi làm dữ liệu mới là cách đóng vòng lại về khâu dữ liệu.

22.5 Thực hành

Repo và tài nguyên. *Config:* [facebookresearch/hydra](#). *Theo dõi thí nghiệm:* [wandb/wandb](#), [mlflow/mlflow](#), [aimhubio/aim](#). *Phiên bản dữ liệu:* [iterative/dvc](#). *Nổi đánh giá với dữ liệu:* [voxel51/fiftyone](#). *Drift:* [evidentlyai/evidently](#). Danh sách chốt tại tháng 9/2026; nhóm công cụ này thay đổi chậm hơn hầu hết phần còn lại của tài liệu, nhưng các nền tảng theo dõi thí nghiệm dạng dịch vụ thì đến và đi khá nhanh — hãy chọn thứ xuất được dữ liệu ra định dạng mở.

Câu hỏi kiểm tra hiểu. Bạn có một checkpoint từ sáu tháng trước và số đo không tái lập được — liệt kê theo thứ tự sáu thứ có thể đã đổi. Vì sao bật chế độ tắt định không đảm bảo tái lập giữa hai máy khác GPU? Với $\sigma = 0,4$ mAP, hiệu số nhỏ nhất mà bạn có thể phát hiện với năm lần chạy mỗi bên là bao nhiêu?

Mini project (vài giờ đến hai ngày). Chuyển một dự án hiện có sang Hydra cộng một hệ theo dõi thí nghiệm, rồi chạy lại một thí nghiệm cũ *chỉ từ config* và kiểm tra khớp số. Phần đáng giá nhất không phải lúc nó khớp mà lúc nó lệch: mỗi chỗ lệch chỉ ra đúng một đối số của hàm băm mà bạn tưởng đã ghim nhưng chưa. Ghi lại danh sách đó — nó là bản kiểm kê nợ kỹ thuật của chính dự án bạn.

22.6 Giới hạn và trường hợp hỏng

- **Tắt định không phải chính xác.** Một pipeline hoàn toàn tắt định vẫn có thể sai hoàn toàn; tái lập chỉ đảm bảo bạn sai một cách nhất quán. Đây là lỗi phân loại phổ biến nhất trong cả chương.
- **Chế độ tắt định che giấu chính phương sai bạn cần biết.** Nếu bạn chỉ chạy tắt định một seed, bạn không bao giờ đo được độ nhạy của kết quả với khởi tạo — và một phương pháp chỉ hoạt động với một seed là một phương pháp không hoạt động [48].
- **Ghi lại mọi thứ không đồng nghĩa với truy ngược được.** Nếu artifact thiếu mã tiền xử lý hoặc thiếu phiên bản thư viện, bạn có một kho lưu trữ đầy đủ về mặt hình thức mà vô dụng về mặt thực hành.
- **Giám sát drift phát hiện thay đổi, không phát hiện thiệt hại.** Phân phối đầu vào có thể trôi mà hiệu năng không đổi, hoặc ngược lại — đứng yên mà hiệu năng vẫn tụt vì quan hệ giữa đầu vào và nhãn đã đổi. Cảnh báo drift là tín hiệu để đi xem, không phải kết luận.
- **Chi phí vận hành là chi phí thật.** Toàn bộ hạ tầng trong chương cần người bảo trì; một hệ MLOps bị bỏ mặc còn nguy hiểm hơn không có, vì nó tạo cảm giác an toàn sai.

CẠM BẮY

Hai hiểu lầm về `use_deterministic_algorithms(True)`. **Một:** nó không đảm bảo tái lập giữa hai máy khác GPU — thứ tự reduction phụ thuộc số SM và cách chia khối, cả hai đều là thuộc tính phần cứng. Kết quả là bạn trả giá tốc độ để mua một sự đảm bảo hẹp hơn bạn tưởng. **Hai:** một lần chạy tình cờ tắt định không chứng minh được gì. Nếu mô hình của bạn hôm nay không đi qua nhánh nào dùng atomics, hai lần chạy sẽ khớp; thêm một tầng dùng `scatter_add` vào tuần sau là nó hết khớp, và bạn sẽ đi tìm bug ở đúng chỗ không có bug.

CẦN NHỚ

Một con số không kèm phương sai không phải bằng chứng. Quy trình tối thiểu: ghim mọi thứ ghim được rẻ (commit, config, dữ liệu, thư viện, container), *chỉ* bật chế độ tắt định khi đang cô lập một bug, và báo cáo trung bình kèm khoảng biến thiên trên nhiều seed. Và luôn tách ba tình huống: có bug, có bất định mà vẫn đúng, có thay đổi thật bị nhiễu che — ba tình huống ấy đòi ba hành động khác nhau. [T2]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Seed	Số khởi tạo mọi bộ sinh ngẫu nhiên: khởi tạo trọng số, thứ tự trộn dữ liệu, augmentation, dropout. Cùng seed chưa đủ để cùng kết quả; khác seed thì chắc chắn khác.
●●● Config	Tập mô tả toàn bộ siêu tham số của một lần chạy, tách khỏi mã. Nguyên tắc: một thay đổi bằng một config diff; không diff được thì không ablate được.
●● Experiment tracking	Ghi tự động mỗi lần chạy kèm commit, config và phiên bản dữ liệu. Phải tự động, vì nó tồn tại chính để bù cho trí nhớ con người lúc hai giờ sáng.
●●● Data versioning	Quản lý phiên bản cho tập dữ liệu như git cho mã. Đây là mục hay bị bỏ quên nhất, và thiếu nó thì mọi so sánh giữa hai thời điểm đều mơ hồ.
●●● Artifact	Gói đủ để tái hiện: trọng số, config, phiên bản thư viện, phiên bản dữ liệu, và mã tiền xử lý. Thiếu một mục là cả gói mất giá trị.
●●● cudnn.benchmark	Chế độ autotune: chọn kernel bằng cách bấm giờ lúc chạy. Máy đang bận thì chọn kernel khác, và kernel khác cộng theo thứ tự khác.
●●● atomicAdd	Cộng dồn song song theo thứ tự luồng hoàn thành. Cộng số thực không có tính kết hợp, nên kết quả lệch ở bit cuối giữa hai lần chạy.
●● TF32	Chế độ nhân ma trộn giảm số bit phần định trị trên GPU Ampere trở lên, bật mặc định. Nhanh hơn nhưng cho số khác fp32 thuần.
●● Vì sao chế độ tắt định không đủ khi đổi máy?	Thứ tự reduction phụ thuộc cách chia khối và số SM, tức phụ thuộc phần cứng. <code>use_deterministic_algorithms</code> chặn được nguồn trong cùng một máy, không chặn được nguồn giữa hai máy.
●● Vì sao cùng seed, cùng máy vẫn ra hai số khác nhau?	Ba nguồn chính: autotune của cuDNN, atomics dấu phẩy động, và thứ tự reduction phụ thuộc cách chia khối. Không nguồn nào nằm trong mã của bạn.
●●● Vì sao thứ tự duyệt tập hợp là bug, không phải nhiễu?	Vì nó phụ thuộc địa chỉ bộ nhớ mà bộ cấp phát tình cờ trả về, chứ không phụ thuộc phần cứng. Sửa bằng <code>sorted()</code> và khoá theo id ổn định; sửa xong không mất gì cả.
●●● Vì sao tái lập phân phối mới là mục tiêu đúng?	Vì ép trùng từng bit xuyên máy thường bất khả thi và luôn đắt. Chạy nhiều lần rồi báo cáo khoảng dao động cho bạn đúng thứ bạn cần: biết hiệu số nào là thật.
●●● $n > (2\sigma/\delta)^2$	Số lần chạy cần để phân giải một hiệu số δ khi độ lệch giữa seed là σ . Với $\sigma = 0,4$ và $\delta = 0,3$ thì cần tám lần mỗi bên.
●● CI cho mô hình	Bốn bài kiểm tra mỗi commit: overfit một batch, kiểm shape, kiểm export và so mẫu cố định, kiểm chỉ số không tụt quá ngưỡng.
●● Drift	Phân phối dữ liệu thật trôi dần khỏi phân phối lúc huấn luyện, làm mô hình xuống cấp mà không ném lỗi nào. Giám sát cả đầu vào lẫn đầu ra, vì chân trị thường đến muộn.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Checkpoint sáu tháng tuổi không tái hiện được số cũ?	Thường thiếu mã tiền xử lý và phiên bản thư viện, kể đó là phiên bản dữ liệu. Checkpoint chỉ chứa trọng số.
●● Bài kiểm CI lúc đỏ lúc xanh mà không ai đổi gì?	Ngưỡng đặt không dựa trên phương sai giữa các seed đã đo. Một bài kiểm hòng ngẫu nhiên sẽ bị vô hiệu hoá trong hai tuần.
●● Drift báo động nhưng độ chính xác không đổi?	Phân phối đầu vào đổi mà quan hệ đầu vào–nhân chưa đổi. Cảnh báo drift là tín hiệu để đi xem, không phải kết luận.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
iterative/dvc	Quản lý phiên bản dữ liệu bằng mã băm, để một lần chạy trở tới đúng trạng thái tập dữ liệu	Hạ tầng; nhóm công cụ ổn định nhất trong chương
hydra + mlflow / wandb / aim	Config tập trung cộng truy ngược từ một con số về commit, config và phiên bản dữ liệu	Hạ tầng; nên chọn thứ xuất được dữ liệu ra định dạng mở vì các dịch vụ đến và đi khá nhanh
torch.use_deterministic	Hiện thực chính xác của ranh giới “ghim được có giá” so với “không ghim được” ở Hình 22.1	Tài liệu của nó liệt kê thẳng các op không có bản tắt định — đáng đọc như một bảng kiểm
evidentlyai/evidently	Giám sát drift phân phối đầu vào và đầu ra khi chân trị đến muộn hoặc không đến	Công cụ; ý tưởng bền hơn tên công cụ
voxel51/fiftyone	Nổi đánh giá với dữ liệu để biến trường hợp sai thành hàng đợi gắn nhãn — bước khép vòng	Chỗ mà chương 22 gặp lại chương 6 và 7 trong một giao diện

MINI PROJECT SLAM · Ép một lần chạy SLAM có thành phần học tái lập tới từng bit · 1–2 ngày

Mục tiêu	đạt được tái lập bit-for-bit, hoặc chứng minh được vì sao không thể. Hai kết quả có giá trị ngang nhau, miễn là bạn nêu tên từng nguồn bất định chứ không nói “GPU vốn thể”.
Dữ liệu	chính pipeline của mini project chương 21, một chuỗi EuRoC, cấu hình mặc định, không đổi gì khác.
Các bước	chạy mười lần, ghi ATE mỗi lần, tính σ ban đầu. Rồi ghim từng nguồn một và sau mỗi lần ghim chạy lại năm lần để xem σ còn lại bao nhiêu: seed toàn cục; <code>sorted()</code> cho mọi lần duyệt tập hợp, <code>glob</code> và <code>listdir</code> ; một <i>worker</i> dataloader; <code> cudnn.benchmark = False; use_deterministic_algorithms(True); tắt TF32</code> . Ghi cả cái giá về thời gian của từng bước.
Xong khi	mười lần chạy cho ATE giống nhau tới chữ số cuối cùng bạn in ra. Hoặc một bảng ba cột: nguồn bất định, đã cố định được chưa, và giá phải trả hay lý do không cố định được. Lý do phải cụ thể: op nào không có bản tắt định, chỗ nào dùng <code>atomicAdd</code> , chỗ nào phụ thuộc số SM. Kèm một câu dùng được ngay: “chênh lệch ATE nhỏ hơn X cm giữa hai cấu hình là nhiều, không phải cải tiến”.
Nối với	D/22 cho hình chữ ký của một con số và bảng nguồn bất định; E/24 cho cách đọc con số σ đó khi so hai cấu hình; E/26 nếu bạn muốn xếp các mini project này vào một lộ trình học theo mục tiêu.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Chủ yếu là ba thứ: autotune của cuDNN chọn kernel bằng cách bấm giờ nên máy bạn thì chọn khác; atomicAdd cộng theo thứ tự luồng hoàn thành mà cộng số thực không có tính kết hợp; và thứ tự reduction phụ thuộc cách chia khối. Không thứ nào trong ba thứ đó nằm trong mã của bạn.
- A2. Vì tất định không phải chính xác — một pipeline hoàn toàn tất định vẫn có thể sai một cách nhất quán — và vì chạy tất định một seed che mất chính phương sai bạn cần biết. Một phương pháp chỉ hoạt động với một seed là một phương pháp không hoạt động.
- A3. Vì 0,3 nhỏ hơn cả độ lệch chuẩn giữa các seed, vốn thường quanh 0,4 mAP với dữ liệu vừa. Muốn $2\sigma/\sqrt{n} < \delta$ thì cần $n > (0,8/0,3)^2 \approx 7,1$, tức tối thiểu tám lần chạy mỗi bên; không đủ ngân sách thì kết luận đúng là “hiệu số này nằm ngoài khả năng phân giải của tôi”.
- A4. Thường là mã tiền xử lý và phiên bản thư viện, kể đó là phiên bản tập dữ liệu. Checkpoint chỉ chứa trọng số; nếu cách resize hay giá trị chuẩn hoá đã đổi thì mô hình cũ chạy trên dữ liệu mới là một hệ thống khác, dù mọi tệp đều còn nguyên.

NÓI LẠI THẬT ĐƠN GIẢN

Bà bạn có một món ăn ngon. Bà đọc cho bạn công thức, bạn ghi lại, và sáu tháng sau nấu ra một món khác hẳn. Không phải bạn làm sai — chỉ là công thức không ghi loại bột nào, lò của ai, đun mấy phút ở nấc nào, và hôm ấy trời ẩm ra sao.

Đây đúng là tình cảnh của mọi người làm mô hình. Con số đẹp bạn đo hôm nay là kết quả của rất nhiều thứ, phần lớn không nằm trong tệp mã nguồn: phiên bản thư viện, thứ tự dữ liệu, loại máy, và cả những chuyện xảy ra bên trong máy mà bạn không điều khiển được. Máy tính cộng số lẻ theo thứ tự khác nhau thì ra kết quả lệch nhau ở chữ số cuối, và thứ tự đó lại phụ thuộc vào việc lúc ấy máy đang bận thế nào.

Mẹo gồm hai nửa. Nửa thứ nhất là ghi lại mọi thứ ghi được, và ghi tự động chứ đừng trông vào trí nhớ lúc hai giờ sáng: đúng bản mã nào, đúng bộ dữ liệu nào, đúng phiên bản thư viện nào. Nửa thứ hai quan trọng hơn và hay bị bỏ qua: chấp nhận rằng phần còn lại không ghim được, nên hãy nấu năm lần và biết món ăn dao động trong khoảng nào. Khi ai đó bảo cách mới ngon hơn, bạn hỏi được ngay: hơn nhiều hơn khoảng dao động đó không?

Cái giá là công sức: toàn bộ bộ máy ghi chép này cần người duy trì, và một hệ thống ghi chép bị bỏ mặc còn nguy hiểm hơn không có, vì nó tạo cảm giác an toàn giả.

Một câu: Một con số không kèm khoảng dao động không phải bằng chứng — hãy ghim mọi thứ ghim được rẻ, rồi đo phần còn lại thay vì giả vờ nó không tồn tại.

22.7 Kết nối

Chương này là điều kiện cần của [E/24-thi-nghiem-va-ablation](#): không có config diff được và không có phương sai đo được thì mọi ablation chỉ là kể chuyện. Nó nối ngược về [B/06-chat-luong-du-lieu](#) và [B/08-dich-chuyen-phan-phoi](#) qua vòng lặp thu thập lỗi, và chính vòng lặp đó là chỗ Phần D khép lại với Phần B. Nó mở rộng mục về tính tất định trong [D/19/02-pytorch-thanh-thao](#) từ một đoạn thành một khung suy nghĩ. Và nó là nền cho [E/25-do-tin-cay-va-van-hanh](#): giám sát drift chỉ là một trường hợp riêng của câu hỏi lớn hơn — làm sao biết mô hình đang ở ngoài vùng nó đáng tin.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Bạn có một checkpoint sáu tháng tuổi và số đo không tái lập được — sáu thứ có thể đã đổi, xếp theo xác suất, là những gì?
2. Vì sao bật chế độ tất định không đảm bảo tái lập giữa hai máy khác GPU, và cơ chế phần cứng nào chịu trách nhiệm?

3. Thứ tự duyệt một set các đối tượng thuộc cột “không ghim được” hay cột “phải sửa”, và vì sao phân loại đó quan trọng?
4. Với $\sigma = 0,4$ mAP, cần bao nhiêu lần chạy mỗi cấu hình để phát hiện tin cậy một hiệu số 0,2 mAP, và con số đó nói gì về chi phí của việc theo đuổi các cải tiến nhỏ?
5. Vì sao “overfit một batch” là bài kiểm tra CI đáng giá nhất trong bốn bài, và nó bắt được loại lỗi nào mà kiểm shape không bắt được?
6. Giám sát drift báo động nhưng độ chính xác không đổi — điều đó có nghĩa gì, và ngược lại thì sao?

ĐÁNH GIÁ, ĐỘ TIN CẬY VÀ DỰ ÁN

TÓM TẮT

Phần E trả lời câu hỏi khó chịu nhất: làm sao biết mình không đang tự lừa mình. Bốn chương gồm đọc paper, thiết kế thí nghiệm và ablation, độ tin cậy và vận hành, và dự án xuyên suốt qua ba vòng. Đọc xong bạn phân biệt được một cải thiện thật với một cải thiện nằm trong khoảng nhiễu, và biết một so sánh không công bằng trông như thế nào. Nếu chỉ nhớ một điều: cơ chế tự lừa trong học máy rất cụ thể và xảy ra với người trung thực đang vội, nên nó phải được chặn bằng quy trình chứ không bằng thiện chí.

Bốn phần trước dạy cách xây. Phần này trả lời câu hỏi khó chịu hơn và quan trọng hơn: **làm sao biết mình không đang tự lừa mình?** Đó không phải một câu hỏi về đạo đức mà là một câu hỏi kỹ thuật, bởi vì cơ chế tự lừa trong học máy rất cụ thể và đã được mô tả kỹ: một baseline chưa được tune tử tế, một test set đã bị nhìn quá nhiều lần, một cải thiện nằm gọn trong khoảng nhiễu giữa các lần chạy, một ablation chỉ chạy một hạt giống. Không cái nào trong số đó cần ác ý; tất cả đều xảy ra với người trung thực đang vội.

Chương 23 là kỹ năng đọc: đọc paper theo ba lượt, đọc phần giới hạn trước phần kết quả, và ghi chú theo cách buộc phải nêu được chỗ mình không tin. Chương 24 là chương nặng nhất của phần và là nơi phần lớn giá trị nằm: thiết kế so sánh cho công bằng, kỹ thuật ablation, lượng thống kê tối thiểu để không kết luận sai, phân tích lỗi có hệ thống, và cách đọc ablation của người khác. Chương 25 chuyển từ đúng sang *đáng tin*: trực quan hoá và những gì nó nói dối, hiệu chỉnh xác suất, ước lượng bất định, rồi vận hành sau khi mô hình đã ra thế giới. Chương 26 đóng vòng bằng dự án xuyên suốt qua ba vòng lặp, và chứa luôn thứ tự học đề xuất cho cả cây.

Một điểm đáng nói trước khi vào: kết luận âm — một ý tưởng đã thử và không có tác dụng — vẫn là kết quả, và là kết quả tồi kém nhất khi bị mất. Ghi nó lại đầy đủ ngang với kết quả dương là một trong những thói quen phân biệt người làm nghiên cứu tử tế với người sưu tầm số đẹp.

Bảng 22.4. Bản đồ Phần E: các chương, nội dung, và mức đầu tư đề xuất.

Ch.	Thư mục	Nội dung	Mức
23	23-doc-paper	Đọc ba lượt; đọc giới hạn trước kết quả; ghi chú phải nêu chỗ mình không tin	cốt lõi
24	24-thi-nghiem-va-ablati	Thiết kế so sánh, ablation, thống kê tối thiểu, phân tích lỗi, báo cáo	cốt lõi
25	25-do-tin-cay-va-van-ha	Trực quan hoá và giới hạn của nó, hiệu chỉnh, bất định, vận hành	cốt lõi
26	26-lo-trinh-hoc	Ba vòng dự án, gợi ý đề tài, và thứ tự học đề xuất cho cả cây	cốt lõi

Chương 23 và 24 nên đọc song song với phần C chứ không đợi tới cuối, vì chúng thay đổi cách bạn đọc mọi paper trong phần C. Chương 26 chứa thứ tự học đề xuất cho toàn cây.

Đọc paper và theo dõi một lĩnh vực (Reading Papers, Following a Field)

PHỤ THUỘC B/03-giai-phau-he-thong (ba tầng bài toán → nguyên lý → cài đặt); B/07-chia-du-lieu-va-danh-gia (giao thức đánh giá — cần để đọc được phần dữ liệu của paper). Đọc song song với E/24-thi-nghiem-va-ablation: chương này dạy đọc bằng chứng, chương 24 dạy tạo ra bằng chứng, và hai kỹ năng đó là hai mặt của cùng một tấm giấy.

TÓM TẮT

Chương này là về cách phân bổ sự chú ý khi số paper vượt xa thời gian đọc, và về cách tách đóng góp thật khỏi cách kể chuyện. Đọc xong bạn định vị được một paper lạ trong bốn mươi phút. Bạn phát biểu được đóng góp thật nằm ở đâu, bằng chứng nào chưa đủ, và thí nghiệm nào sẽ khiến bạn đổi ý. Chừng đó đủ để quyết định có bỏ hai tuần tái hiện nó hay không. Hai thao tác đối nhiều nhất là đọc limitations và phần dữ liệu *trước* bảng kết quả, và ghi lại mục “chỗ tôi không tin” kèm thí nghiệm sẽ thuyết phục được mình. Nếu chỉ nhớ một điều: paper là văn bản thuyết phục chứ không phải biên bản thí nghiệm, nên câu hỏi trung tâm luôn là *bỏ đóng góp chính đi nhưng giữ nguyên công thức huấn luyện thì còn lại bao nhiêu*.

KIẾN THỨC CẦN CÓ TRƯỚC

- **paper** — một bài báo khoa học, thường 8–10 trang, có cấu trúc cố định: abstract, mở đầu, phương pháp, thí nghiệm, kết luận. “Preprint” là bản tự đăng (thường trên arXiv) chưa qua bình duyệt.
- **baseline** — phương pháp đối chứng mà kết quả mới phải vượt qua. Một baseline chưa được tinh chỉnh tử tế làm mọi so sánh sau đó vô nghĩa, nên chất lượng baseline quan trọng ngang chất lượng phương pháp mới.
- **ablation** — thí nghiệm bỏ đi hoặc thay thế đúng một thành phần của hệ thống rồi đo lại, để biết thành phần đó đóng góp bao nhiêu. Đây là bằng chứng chính cho câu “vì sao nó hoạt động”.
- **công thức huấn luyện (recipe)** — toàn bộ các lựa chọn quanh việc huấn luyện mà không phải kiến trúc: số epoch, learning rate và lịch của nó, augmentation, cách khởi tạo, EMA trọng số. Đối recipe một mình đã đủ tạo ra khác biệt lớn về kết quả.
- **seed** — số khởi tạo cho bộ sinh số ngẫu nhiên. Đổi seed thì khởi tạo trọng số và thứ tự dữ liệu đổi theo, nên cùng một cấu hình chạy hai seed vẫn ra hai con số khác nhau.
- **tập validation và tập test** — validation là tập dùng để chọn cấu hình; test là tập chỉ được dùng một lần để báo cáo. Trộn hai vai này là dạng rò rỉ phổ biến nhất trong các bảng kết quả.
- **mAP** — *mean average precision* — metric tổng hợp thông dụng cho phát hiện vật thể, gộp cả độ chính xác lẫn độ bao phủ qua nhiều ngưỡng. Ở đây bạn chỉ cần biết nó là “điểm số càng cao càng tốt”, đơn vị thường tính theo điểm phần trăm.
- **benchmark** — một tập dữ liệu cộng giao thức đánh giá cố định (ví dụ COCO, ADE20K) để nhiều nhóm so được với nhau. Con số chỉ có nghĩa khi kèm tên benchmark và phiên bản giao thức.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao đọc phần giới hạn của một paper *trước* bảng kết quả lại tiết kiệm thời gian hơn, dù lượng chữ bạn nạp vào là y hệt?

- Q2. Hai paper cùng báo cáo +1,0 điểm trên cùng benchmark; một cái đáng tin, một cái không. Chi tiết nào trong phần thiết lập tách được chúng, và vì sao nó không nằm trong bảng kết quả?
- Q3. Nếu phần lớn “tiền bộ” của một mảng biến mất khi baseline được tinh chỉnh bằng cùng ngân sách, thì suốt cả thập kỷ các paper đó đã thật sự đo cái gì?
- Q4. Ghi chú tóm tắt một paper gần như vô dụng sau sáu tháng. Mục nào trong ghi chú thì không, và vì sao đúng mục đó lại làm kỹ năng đọc của bạn lên tay?

23.1 Đặt vấn đề

Một mảng hẹp của thị giác máy tính hiện xuất bản nhiều hơn số paper mà một người đọc kỹ được trong cả năm. Bài toán vì thế không phải là “đọc thế nào cho hiểu” — điều đó bạn làm được với bất kỳ paper đơn lẻ nào nếu có đủ thời gian — mà là “phân bổ sự chú ý thế nào để không phá sản”. Phần lớn người tự học giải bài toán này sai theo cùng một cách: đọc tuần tự từ đầu đến cuối mọi paper thấy thú vị, đọc chậm dần, rồi bỏ cuộc và chuyển sang chỉ đọc abstract trên mạng xã hội. Cả hai cực đều không cho ra hiểu biết dùng được.

Có một lý do sâu hơn khiến việc đọc phải có kỹ thuật riêng. Paper là **một văn bản thuyết phục, không phải một biên bản thí nghiệm**. Tác giả đã biết kết quả trước khi viết câu đầu tiên của phần mở đầu, nên toàn bộ trật tự trình bày được tối ưu ngược từ bảng kết quả trở lên: đóng góp được đặt tên sao cho nghe như một nguyên lý, các lựa chọn không hiệu quả biến mất, và những lần chạy hỏng không xuất hiện ở đâu cả. Điều này không đòi hỏi ai phải gian dối; nó là hệ quả tự nhiên của việc một cộng đồng thưởng cho câu chuyện gọn gàng. Do đó câu hỏi trung tâm khi đọc không phải “họ làm gì” mà là **bằng chứng có tương xứng với tuyên bố không** — và cùng với nó là hai câu hỏi nữa, hỏi trước: bài toán thật sự là gì (đầu vào, đầu ra, tín hiệu giám sát), và họ giả định gì mà không nói ra.

Học xong chương này bạn làm được gì mà trước đó không làm được? Bạn đọc một paper lạ trong bốn mươi phút và phát biểu được ba câu: đóng góp thật nằm ở đâu, bằng chứng nào chưa đủ để kết luận, và thí nghiệm nào sẽ khiến bạn đổi ý. Đó là điều kiện đủ để ra quyết định đắt nhất mà một kỹ sư tự học phải ra thường xuyên — có nên bỏ hai tuần tái hiện một phương pháp hay không.

TRỰC GIÁC

Đọc một paper giống nhận một *pull request* mà tác giả vừa viết code, vừa viết test, vừa được quyền chọn công khai lần chạy CI nào. Không ai phải nói dối để bạn hiểu sai: chỉ cần những lần chạy đỏ không nằm trong diff là đủ. Việc của người đọc, giống việc của người review, là đoán ra *hình dạng của những lần chạy không được đưa vào* — và biết cần yêu cầu thêm test nào thì mới đủ tin.

23.2 Ba lượt đọc và thứ tự bên trong một lượt (Three-Pass Reading)

A. Ba lượt và điểm dừng của mỗi lượt

Khung ba lượt của Keshav là điểm khởi đầu tốt vì nó cho phép bỏ dở một cách có kỷ luật [44]. Lượt một mất năm đến mười phút — tiêu đề, abstract, hình, kết luận — và kết thúc bằng đúng một quyết định nhị phân: rút hay đọc tiếp. Lượt hai mất khoảng một giờ, đọc phương pháp và thiết lập thí nghiệm nhưng bỏ mọi chứng minh, kết thúc bằng khả năng tóm tắt paper cho người khác kèm những chỗ mình chưa nắm. Lượt ba là đọc để tái dựng: tự đặt mình vào vị trí tác giả

và dừng lại từng bước, mỗi chỗ mình chọn khác đi là một chỗ đáng ghi. Chỉ vài paper mỗi năm xứng đáng với lượt ba.

Bảng 23.1. Ba lượt đọc: mỗi lượt có một sản phẩm khác nhau, và điều kiện dừng của lượt trước quyết định có bước vào lượt sau hay không.

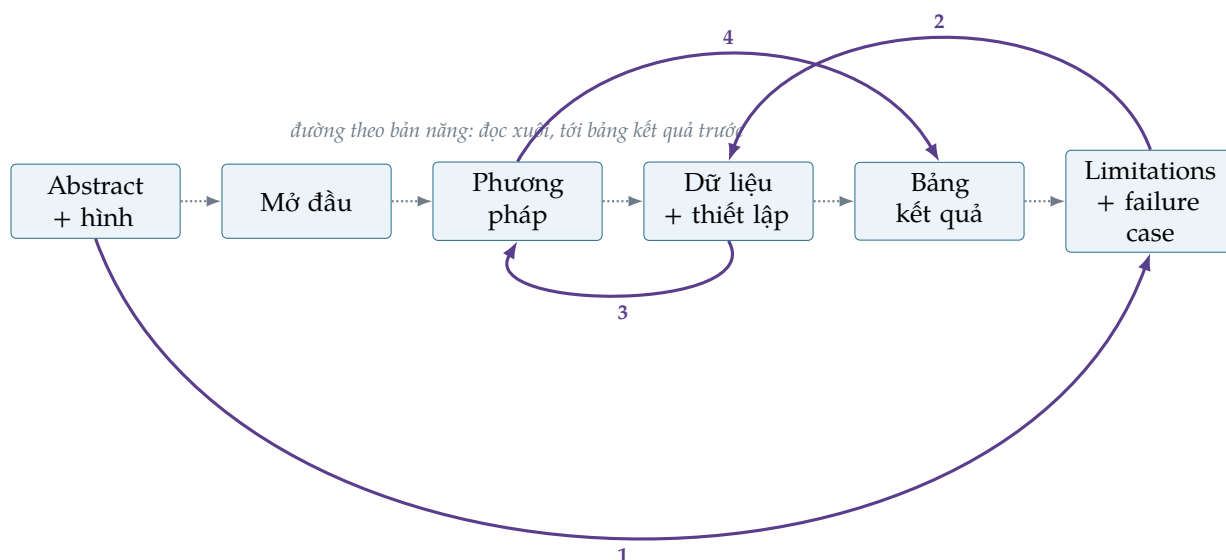
Lượt	Chi phí	Sản phẩm	Dừng lại nếu
1 — định vị	5–10 phút	Một quyết định nhị phân: vớt hay đọc tiếp	Bài toán không phải bài toán của bạn, hoặc giả định gốc đã sai với bối cảnh của bạn
2 — hiểu	≈ 1 giờ	Tóm tắt được cho người khác, kèm danh sách chỗ chưa nắm	Bảng chứng không tương xứng với tuyên bố; ghi lại lý do rồi dừng
3 — tái dựng	Nửa ngày trở lên	Dựng lại được thiết kế và biết mình sẽ chọn khác ở đâu	Chỉ dành cho paper bạn định cài lại hoặc xây tiếp lên

B. Thứ tự đọc bên trong một lượt

Điều mà khung này không nói, và là chỗ dễ sai nhất trong thị giác máy tính, là **thứ tự đọc bên trong một lượt**. Bản năng dẫn ta tới bảng kết quả trước, vì đó là chỗ có con số. Nên làm ngược lại, theo đúng trình tự sau. Trước hết đọc phần *limitations* và các trường hợp hỏng — thường nằm cuối, thường ngắn, thường là phần được đầu tư ít công sức nhất, và chính vì thế nó rõ ràng nhiều sự thật nhất. Nó cho bạn biết tác giả *tự biết* mình yếu ở đâu, và nếu bạn đọc nó trước thì mọi con số sau đó được đọc trong đúng bối cảnh của chúng. Nếu một paper không có phần này, đó là tín hiệu cảnh báo chứ không phải dấu hiệu hệ thống hoàn hảo. Tiếp theo mới là phần dữ liệu: dữ liệu nào, chia thế nào, nhân từ đâu, có trùng lặp giữa train và test không. Cách chia dữ liệu quyết định con số có nghĩa hay không, và một bảng kết quả đọc trước phần dữ liệu là một bảng kết quả chưa được diễn giải. Bảng kết quả đọc sau cùng, và khi đó nó chỉ còn là xác nhận hay bác bỏ những gì bạn đã dự đoán từ hai phần trước.

↔ **tiền đề B/07** — một con số chỉ mang nghĩa bên trong giao thức chia dữ liệu đã sinh ra nó; đọc bảng trước phần dữ liệu là đọc một đại lượng chưa được định nghĩa

Hình 23.1 đặt cạnh nhau hai đường đi qua cùng một paper: đường theo bản năng và đường nên đi.



Hình 23.1. Cùng một paper, hai đường đi. Đường chấm mờ là thứ tự xuất bản; đường đậm đánh số là thứ tự nên đọc: limitations trước, rồi phần dữ liệu, rồi phương pháp, và bảng kết quả sau cùng. Thứ thay đổi không phải lượng thông tin nhận được mà là *bối cảnh* mà mỗi con số được đọc trong đó.

C. Đọc code cùng paper

Cuối cùng, trong thị giác máy tính, đọc paper mà không mở code là đọc một nửa. Các chi tiết thật sự quyết định con số hầu như luôn nằm trong repo, và hầu như không bao giờ nằm đủ trong bài viết. Danh sách quen thuộc gồm **gán mẫu** (*label assignment*) cho các **anchor**, tập **tăng cường dữ liệu** (*data augmentation*), lịch **learning rate**, ngưỡng **triệt phi cực đại** (*non-maximum suppression, NMS*), và việc có dùng trung bình trượt của trọng số (*EMA*) hay không. Bằng chứng rõ nhất cho điều này đến từ chính một paper detection: khi tách bạch từng khác biệt giữa dòng anchor-based và anchor-free, phần lớn khoảng cách hoá ra nằm ở **định nghĩa mẫu dương/âm** chứ không ở kiến trúc đầu ra [46]. Một chi tiết vốn bị coi là tiểu tiết cài đặt lại chính là đóng góp.

↔ **trường hợp riêng của C/14** — gán mẫu trong detector là ví dụ điển hình nhất của loại chi tiết quyết định kết quả mà chỉ code mới nói ra

I 23.3 Ghi chú và mục “chỗ tôi không tin” (Paper Notes)

Ghi chú tóm tắt lại nội dung paper gần như vô dụng: sáu tháng sau bạn sẽ đọc lại chính abstract nhanh hơn là đọc ghi chú của mình. Ghi chú có giá trị là ghi chú chứa những thứ *không có* trong paper. Một khuôn năm mục vừa đủ và đáng giữ nguyên qua nhiều năm:

1. Bài toán trong ba dòng: đầu vào, đầu ra, tín hiệu giám sát. Nếu viết không nổi ba dòng này, bạn chưa đọc xong.
2. Ý tưởng lõi trong một câu, không dùng tên riêng của phương pháp.
3. Bằng chứng mạnh nhất trong bài — cụ thể là thí nghiệm nào, không phải con số nào.
4. **Chỗ tôi không tin** — và vì sao.
5. Thí nghiệm nào sẽ khiến tôi đổi ý.

Mục bốn là mục quan trọng nhất và cũng là mục duy nhất tạo ra một vòng phản hồi. Đọc mà không ghi lại sự hoài nghi thì sự hoài nghi tan biến, và bạn không bao giờ biết trực giác của mình đúng hay sai. Ghi lại thì sáu tháng sau, khi paper cải tiến hoặc paper phản biện xuất hiện, bạn đối chiếu được: lần này tôi nghi đúng chỗ, lần kia tôi nghi sai vì tôi không biết một ràng buộc thực tế. Đó là cách duy nhất để khả năng đọc paper của một người thật sự lên tay. Vòng này vận hành đúng như “dự đoán trước, đối chiếu sau” của thí nghiệm ở chương 24: cùng một cơ chế học, chỉ khác đối tượng.

Mục năm tồn tại để buộc sự hoài nghi phải cụ thể. “Tôi thấy nghi ngờ” là một cảm giác; “tôi sẽ tin nếu họ chạy phương pháp này với *baseline* được tune bằng cùng ngân sách trên ba *seed*” là một mệnh đề kiểm chứng được. Không viết nổi mục năm thì mục bốn chỉ là định kiến.

23.4 Đóng góp thật và cách kể chuyện (Contribution vs. Narrative)

Phần lớn cải tiến được công bố trong thị giác máy tính là tổ hợp của ba nguồn: một ý tưởng mới, một **công thức huấn luyện** (*training recipe*) tốt hơn, và nhiều dữ liệu hơn. Cách kể chuyện chuẩn của paper quy toàn bộ chênh lệch cho nguồn thứ nhất, vì hai nguồn còn lại không kể thành câu chuyện được. Đây không phải phỏng đoán mà là điều đã được đo nhiều lần. Dòng ConvNeXt lấy một ResNet rồi đưa dần từng thành phần của công thức huấn luyện và thiết kế hiện đại vào. Phần lớn khoảng cách so với Swin biến mất trước khi chạm tới bất cứ thứ gì mang bản chất *attention* [121]. Trong metric learning, một đánh giá lại có kiểm soát cho thấy đường cong tiến bộ gần như phẳng suốt nhiều năm khi mọi phương án được **tinh chỉnh siêu tham số** (*hyperparameter tuning*) bằng cùng ngân sách và đo bằng cùng giao thức [42]. Trong hệ khuyến nghị, kết quả tương tự: phần lớn phương pháp neural được công bố không vượt nổi baseline cổ điển được tune tử tế [41]. Trong học tăng cường, phương sai giữa các seed lớn tới mức nhiều so sánh đã công bố không phân biệt được với nhiễu [48].

Bài học rút ra không phải là hoài nghi mọi thứ, mà là một phép thử duy nhất áp cho mọi paper: **nếu bỏ đóng góp chính đi nhưng giữ nguyên toàn bộ công thức huấn luyện, con số còn lại bao nhiêu — và paper có cho bạn biết điều đó không?** Paper trả lời được câu này (bằng một *ablation* có kiểm soát) đáng tin hơn hẳn paper không trả lời, độc lập với việc con số cuối cùng đẹp tới đâu. Cũng nên phân biệt: một công thức huấn luyện tốt hơn *là* một đóng góp thật, chỉ là một đóng góp khác loại — nó chuyển giao được sang mọi kiến trúc, trong khi một module mới thường không. Vấn đề chỉ nằm ở việc gọi tên sai.

↔ **cùng nguyên lý với C/12** — cơ chế huấn luyện giải thích phần lớn chênh lệch giữa hai hệ thống — cùng một quan sát, ở đây nhìn từ phía người đọc paper

23.5 Nhận ra một so sánh không công bằng (Unfair Comparison)

Phần lớn so sánh lệch không đến từ ý đồ xấu. Nó đến từ bất đối xứng công sức: tác giả dành sáu tháng cho phương pháp của mình và một buổi chiều cho baseline. Cơ chế toán học phía sau đơn giản và đã được mô tả rõ trong bối cảnh tìm siêu tham số: hiệu năng tốt nhất quan sát được là một hàm tăng theo số lần thử, nên hai phương án được thử với ngân sách khác nhau thì không so được, kể cả khi mọi thứ khác giống hệt [206]. Bảng 23.2 liệt kê các dấu hiệu thường gặp cùng cách kiểm tra nhanh và — quan trọng không kém — trường hợp mà dấu hiệu đó vô hại.

Bảng 23.2. Các dạng so sánh không công bằng hay gặp trong paper thị giác máy tính. Cột cuối tồn tại để bạn không biến sự hoài nghi thành phần xạ bác bỏ mọi thứ.

Dấu hiệu	Cơ chế gây lệch	Kiểm tra nhanh	Khi nào vô hại
Số baseline chép từ paper cũ	Khác thể hệ recipe: epoch, augmentation, độ phân giải	Đối chiếu số epoch và tập augmentation của cả hai	Baseline được chạy lại trong cùng codebase và ghi rõ điều đó
So ở cùng số epoch	Phương pháp mới đắt hơn mỗi bước nên được nhiều tính toán hơn	Tìm chi phí mỗi bước hoặc tổng GPU-giờ	Chi phí bước gần bằng nhau và có báo cáo
Không ghi số seed	Chênh lệch có thể nằm dưới sàn nhiễu	Tìm dấu $\pm$ và số lần chạy trong chú thích bảng	Benchmark có phương sai rất nhỏ và điều đó được chứng minh
Ablation chạy trên tập test	Cấu hình được chọn trên chính tập dùng để báo cáo	Đọc chú thích bảng ablation, không đọc bảng chính	Ghi rõ chọn trên val, báo cáo trên test một lần
Ngân sách tune bất đối xứng	Tối ưu nhiều lần cho mình, ít lần cho đối thủ	Tìm số cấu hình đã thử cho mỗi phương án	Cả hai dùng chung không gian tìm kiếm và cùng số lần thử

CẠM BÃY

Cạm bẫy tinh vi nhất không nằm ở bảng kết quả mà ở việc *đọc bảng trước phần dữ liệu*. Khi đã nhìn thấy con số, bạn sẽ vô thức đi tìm lý do để tin nó, và phần dữ liệu đọc sau đó trở thành thủ tục. Đảo thứ tự lại thì cùng một bảng đó bỗng đọc ra khác hẳn — đây là hiệu ứng neo, không phải vấn đề kiến thức, nên biết về nó không đủ để tránh; phải đổi thứ tự thao tác.

23.6 Theo dõi một lĩnh vực mà không bị ngập (Field Tracking)

Đọc theo dòng thời gian là cách chắc chắn để bị ngập, vì luồng paper mới không có cấu trúc. Cách bền hơn là **đọc theo cụm**: một paper gốc, các paper cải tiến trực tiếp lên nó, và ít nhất một paper phản biện hoặc đánh giá lại. Ba loại bổ khuyết cho nhau — paper gốc cho ý tưởng ở dạng thuần khiết nhất, paper cải tiến cho biết chỗ nào của ý tưởng đó thật sự chịu lực, paper phản biện cho biết phần nào của con số đến từ chỗ khác. Thiếu loại thứ ba là lý do phổ biến nhất khiến một người nắm rất nhiều tên phương pháp mà không có phán đoán nào.

Sản phẩm nên có của việc theo dõi là một **cây phả hệ** (*genealogy of a field*), không phải một danh sách. Mỗi node đúng một dòng, và dòng đó phải có dạng “giải quyết vấn đề X mà node cha không giải được”. Ràng buộc một dòng ấy chính là bộ lọc: nếu bạn không viết nổi câu đó cho một paper, thì hoặc bạn chưa hiểu nó, hoặc nó chưa cần thiết với bạn — và cả hai kết luận đều hữu ích. Mười lăm node cho một mảng hẹp thường đủ để bạn định vị bất kỳ paper mới nào trong vài phút, vì paper mới gần như luôn là con của một node đã có.

Cuối cùng, nên duy trì *hai* danh sách paper nền chứ không phải một:

- **Nhánh nguyên lý mô hình** — kiến trúc, cơ chế huấn luyện, các dòng phương pháp. Đây là nhánh ai cũng có, và cũng là nhánh bán rã nhanh nhất: một nửa số tên trong đó sẽ hết quan trọng sau vài năm.
- **Nhánh dữ liệu, đo lường và đánh giá** — cách xây benchmark, lỗi nhãn, dịch chuyển phân phối, giao thức so sánh. Nhánh này gần như luôn bị bỏ qua, bán rã chậm hơn nhiều, và quyết định phần lớn khả năng phán đoán. Với một kỹ sư đã quen đo đạc hệ thống thật, đây là chỗ lợi thế sẵn có lớn nhất — bạn đã có bản năng đúng, chỉ thiếu tên gọi và tài liệu tham chiếu.

↔ **hệ quả của E/24** — mọi câu hỏi bạn dùng để nghi ngờ bảng của người khác đều bắt nguồn từ cách bằng chứng được sinh ra ở chương 24

23.7 Giới hạn và trường hợp hỏng

Quy trình trên giả định ba điều: paper có phần ablation chạy nghiêm túc, có code công khai, và lĩnh vực đã đủ chín để có paper phản biện. Cả ba giả định đều sai trong hai năm đầu của một hướng mới, và đó chính là lúc bạn cần phán đoán nhất. Trong giai đoạn ấy, thứ thay thế được là đọc chéo giữa các nhóm độc lập: nếu ba nhóm không liên quan báo cáo cùng một hiện tượng với ba thiết lập khác nhau, hiện tượng đó đáng tin hơn một paper duy nhất có bảng đẹp.

Chế độ hỏng thứ hai nằm ở phía người đọc: hoài nghi có chi phí. Áp dụng đủ mạnh, bạn sẽ không tin gì cả và không quyết định được gì — trạng thái này tẻ nhạt với cả tin. Sự hoài nghi hữu ích luôn kèm một mệnh đề cụ thể có thể kiểm chứng, tức là mục nằm trong khuôn ghi chú ở trên; sự hoài nghi không kèm mệnh đề đó chỉ là sự trì hoãn được nguy trang. Chế độ hỏng thứ ba tinh vi hơn: một cây phả hệ đẹp tạo cảm giác đã hiểu lĩnh vực, trong khi thứ bạn có mới chỉ là bản đồ tên gọi. Phép thử là thử dự đoán kết quả của một thí nghiệm chưa đọc — nếu không dự đoán nổi thì bản đồ vẫn còn rỗng.

CẦN NHỚ

Nếu chỉ mang theo một câu: *bỏ đóng góp chính đi nhưng giữ nguyên công thức huấn luyện thì còn lại bao nhiêu?* Câu hỏi đó tách đóng góp thật khỏi cách kể chuyện nhanh hơn mọi thứ khác, và nó áp dụng cho cả paper của người khác lẫn công việc của chính bạn. [T2]

23.8 Thực hành

Công cụ và tài nguyên. [alphaXiv](#) và [arxiv-sanity](#) để lọc luồng paper mới; Semantic Scholar và Connected Papers để dựng cây trích dẫn; [huggingface.co/papers](#) cho thảo luận hằng ngày. Các bảng xếp hạng kiểu [paperswithcode](#) nay đã phân tán, nên bám **benchmark** chính thức của từng mảng (COCO, ADE20K, RF100-VL, WILDS, VBench) thay vì một bảng tổng. Danh sách công cụ này chốt tại tháng 9/2026 và sẽ cũ trong sáu đến mười hai tháng; riêng thói quen dựng cây phả hệ và đọc theo cụm thì không.

Câu hỏi kiểm tra hiểu. Trong paper bạn vừa đọc: nếu bỏ đóng góp chính nhưng giữ nguyên recipe huấn luyện thì còn lại bao nhiêu, và paper có cho bạn biết điều đó không? Baseline của họ được tune bằng bao nhiêu ngân sách so với phương pháp mới? Phần dữ liệu có cho phép bạn tự dựng lại cách chia train/val/test không?

Mini project. Dựng cây phả hệ mười lăm node cho một mảng hẹp bạn quan tâm, mỗi node đúng một dòng theo mẫu “giải quyết vấn đề X mà node cha không giải được”. Node nào viết không nổi thì bỏ ra khỏi cây và ghi lại vì sao.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Ba lượt đọc	Khung đọc có ba mức đầu tư: 5–10 phút để quyết vớt hay đọc tiếp, khoảng một giờ để tóm tắt được cho người khác, và nửa ngày trở lên để dựng lại thiết kế. Lượt ba chỉ dành cho paper bạn định cài lại.
●●● Limitations	Phần cuối bài, nơi tác giả tự nêu chỗ yếu. Nó được đầu tư ít công nhất nên rò rỉ nhiều sự thật nhất; đọc nó trước sẽ đặt bối cảnh cho mọi con số phía sau.
●● Vì sao đọc limitations trước bảng kết quả?	Vì thứ tự đọc quyết định cái neo. Đọc bảng trước thì bạn dành phần còn lại để đi tìm lý do tin nó; đọc limitations trước thì cùng những con số ấy được đọc trong bối cảnh tác giả đã tự thừa nhận.
●● Gán mẫu (label assignment)	Quy tắc quyết định anchor hay điểm nào là mẫu dương, mẫu nào là âm, khi huấn luyện detector. Nó hầu như chỉ có trong code, và có lần nó chính là toàn bộ khoảng cách giữa hai dòng phương pháp.
●● Vì sao phải mở code cùng paper?	Vì trong thị giác máy tính, các chi tiết quyết định con số nằm trong repo chứ không trong bài viết: gán mẫu, augmentation, lịch learning rate, ngưỡng NMS, có dùng EMA hay không.
●●● Công thức huấn luyện	Toàn bộ lựa chọn quanh việc huấn luyện mà không phải kiến trúc: số epoch, lịch learning rate, augmentation, khởi tạo, EMA. Đối riêng nó đã đủ tạo ra phần lớn chênh lệch giữa hai hệ thống.
●●● “Chỗ tôi không tin”	Mục thứ tư trong ghi chú năm mục, ghi lại sự hoài nghi kèm thí nghiệm sẽ khiến bạn đổi ý. Nó là thứ duy nhất kiểm chứng được về sau, nên là vòng phản hồi duy nhất trong việc đọc paper.
●●● Ngân sách tinh chỉnh	Số cấu hình đã thử cho mỗi phương án. Kết quả tốt nhất quan sát được là cực đại của một mẫu, nên hai phương án thử với ngân sách khác nhau thì không so được, kể cả khi mọi thứ khác giống hệt.
●● Vì sao “baseline yếu” là lỗi phổ biến nhất?	Vì nó rẻ: tác giả dành sáu tháng cho phương pháp của mình và một buổi chiều cho baseline. Bất đối xứng công sức đó tạo ra chênh lệch mà không cần ai gian dối.
●● Ablation trên tập test	Chọn cấu hình trên chính tập dùng để báo cáo, tức là một dạng rò rỉ. Tìm dấu hiệu này ở chú thích bảng ablation, không phải ở bảng chính.
●●● Paper báo cải tiến 1,0 mAP nhưng không ghi số seed — đọc thế nào?	Chưa kết luận được gì: chênh lệch có thể nằm dưới sàn nhiễu giữa các seed. Tìm dấu ± và số lần chạy; không có thì hạ mức tin cậy chứ đừng suy đoán bù.
●● Paper không có phần limitations — suy ra gì?	Đó là tín hiệu cảnh báo, không phải dấu hiệu hệ thống hoàn hảo. Mọi hệ thống đều trả giá ở đâu đó; không tìm ra nghĩa là bạn chưa hiểu nó, hoặc tác giả không muốn bạn hiểu.
●●● Bạn tái hiện một repo và thấp hơn số công bố 2 điểm — kiểm tra gì trước?	Theo thứ tự: phiên bản dữ liệu và tiền xử lý, rồi công thức huấn luyện (epoch, augmentation, độ phân giải), rồi phần cứng và cách tính metric. Nghi phương pháp là bước cuối, không phải bước đầu.
●● Vì sao công thức huấn luyện vẫn là đóng góp thật?	Vì nó chuyển giao được sang mọi kiến trúc, trong khi một module mới thường không. Nó chỉ là đóng góp <i>khác loại</i> ; vẫn đề nằm ở việc gọi tên sai, không ở giá trị.
●● Đọc theo cụm	Đọc một paper gốc, các paper cải tiến trực tiếp lên nó, và ít nhất một paper phản biện. Thiếu loại thứ ba là lý do phổ biến nhất khiến một người nhớ rất nhiều tên mà không có phán đoán nào.
● Cây phả hệ	Danh sách node, mỗi node đúng một dòng: “giải quyết vấn đề X mà node cha không giải được”. Ràng buộc một dòng ấy chính là bộ lọc; viết không nổi thì node đó chưa cần với bạn.
● Hai danh sách nền	Một nhánh về nguyên lý mô hình, một nhánh về dữ liệu và đánh giá. Nhánh thứ hai bán rã chậm hơn nhiều, hay bị bỏ qua, và là lợi thế sẵn có của người làm hệ thống.

● ● ● phải thuộc ● ● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Quy trình bình duyệt CVPR / NeurIPS	Biểu mẫu review yêu cầu đúng bộ câu hỏi của chương: điểm mạnh, điểm yếu, thí nghiệm còn thiếu, câu hỏi cho tác giả	Đây là nơi kỹ năng này được hành nghề hằng ngày; mẫu review công khai
Reproducibility checklist (NeurIPS, ICLR)	Ép khai báo số seed, độ lệch chuẩn, ngân sách tính toán — chính bốn dấu hiệu ở bảng so sánh lệch	Thành thông lệ từ khoảng 2019, nay gần như bắt buộc
Semantic Scholar, Connected Papers	Dùng cây trích dẫn tự động, tức cây phá hệ ở dạng công cụ	Hạ tầng; bền hơn mọi tên mô hình
Dòng ConvNeXt [121]	Cả paper là một ablation recipe-so-với-kiến trúc có kiểm soát, đúng phép thử “bỏ đóng góp chính, giữ recipe”	Ví dụ mẫu mực để đọc trước khi tự viết ablation
Đánh giá lại metric learning [42]	Hiện thực hoá phép thử “cùng ngân sách tính chính, cùng giao thức” trên cả một mảng	Kết quả đã công bố, không phải phỏng đoán

MINI PROJECT SLAM · Đọc ba lượt một paper frontend học được, và viết ghi chú có mục “chỗ tôi không tin” · nửa ngày

Mục tiêu	biến quy trình ba lượt của chương thành một thói quen đo được. Đối tượng là đúng loại paper bạn phải quyết định có tái hiện hay không: một paper về frontend học được cho visual SLAM, tức đặc trưng học được, so khớp học được, hoặc odometry học được.
Dữ liệu	một paper bạn chưa đọc, kèm repo của nó nếu có. Chọn paper báo cáo kết quả trên EuRoC hoặc TUM-VI để bạn đối chiếu được với số bạn đã tự đo.
Các bước	lượt một năm phút (abstract, hình, kết luận) rồi quyết vớt hay đọc tiếp; lượt hai theo đúng thứ tự limitations → dữ liệu → phương pháp → bảng kết quả; mở repo tìm ba chi tiết không có trong bài viết (ngưỡng so khớp, augmentation, lịch huấn luyện); viết ghi chú năm mục theo khuôn ở mục “Ghi chú”.
Xong khi	bạn nêu được một chi tiết cụ thể trong phần thí nghiệm mà nếu đổi đi thì kết luận của paper có thể sụp. Ví dụ: chuỗi EuRoC nào bị loại khỏi bảng, hay baseline chạy ở độ phân giải nào. Và bạn viết được mục “thí nghiệm sẽ khiến tôi đổi ý” dưới dạng một cấu hình chạy được.
Nối với	E/24 cho cách kiểm tra chính chi tiết đó bằng số; B/07 cho phần giao thức đánh giá mà bạn dùng để đọc phân dữ liệu.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- Vì thứ tự đọc quyết định cái neo. Đọc limitations trước thì mọi con số sau đó được diễn giải trong bối cảnh tác giả tự thừa nhận; đọc bảng trước thì bạn dành phần còn lại để đi tìm lý do tin nó. Cùng lượng thông tin, khác kết luận.
- Số seed kèm độ lệch chuẩn, ngân sách tính chính của từng phương án, và việc ablation chạy trên val hay test — xem Bảng 23.2. Chúng không nằm trong bảng kết quả vì bảng kết quả được thiết kế để thuyết phục, còn ba chi tiết đó chỉ làm yếu đi câu chuyện.
- Phần lớn là ngân sách tính chính và công thức huấn luyện, tức cỡ mẫu của một cuộc tìm kiếm, chứ không phải ý tưởng. Điều đó không có nghĩa recipe vô giá trị — nó là đóng góp thật, chỉ khác loại và chuyển giao được rộng hơn một module mới.
- Mục “chỗ tôi không tin” cộng mục “thí nghiệm sẽ khiến tôi đổi ý”. Chúng kiểm chứng được về sau, nên tạo ra vòng phản hồi duy nhất trong việc đọc paper: sáu tháng sau bạn biết mình đã nghi đúng chỗ hay nghi sai vì thiếu một ràng buộc thực tế.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn đang tìm mua nhà, và mỗi ngày có ba mươi tin rao mới. Mỗi tin đều do người bán viết: ảnh chụp góc rộng lúc trời nắng, phòng nào chật thì không có ảnh, và không tin nào nhắc tới cái cống trước cửa. Không ai nói dối cả; họ chỉ được quyền chọn tấm ảnh nào đăng lên. Khó khăn của bạn không phải là đọc không hiểu — tin nào bạn cũng hiểu — mà là không đủ thời gian đi xem hết, nên phải đoán từ trang giấy xem căn nào đáng đi.

Mẹo gồm ba việc. Một, đảo thứ tự đọc: xem phần người bán tự nhận là điểm yếu trước, rồi tới giấy tờ và diện tích thật, và để giá cùng ảnh đẹp xuống sau cùng. Hai, hỏi xem họ đã sửa sang bao nhiêu trước khi chụp, vì một căn vừa sơn lại toàn bộ và một căn để nguyên thì không đặt cạnh nhau được. Ba, ghi lại đúng một dòng “chỗ tôi thấy nghi” cho mỗi tin, để sáu tháng sau quay lại bạn biết lần nào mình nghi đúng và lần nào nghi sai.

Cái giá phải trả: đọc một tin lâu hơn hẳn, và nếu quen nghi ngờ quá tay thì bạn không bao giờ dám đi xem căn nào. Nghi ngờ chỉ có ích khi kèm một câu hỏi cụ thể bạn định hỏi người bán.

Một câu: Người viết được quyền chọn ảnh nào đăng lên, nên việc của bạn là đoán ra những tấm không được đăng trông như thế nào.

23.9 Kết nối

Chương này là mặt đọc của cùng một kỹ năng mà [E/24-thi-nghiem-va-ablation](#) dạy ở mặt viết: mọi tiêu chí bạn dùng để nghi ngờ ablation của người khác đều là tiêu chí bạn phải tự thoả khi làm ablation của mình, nên hai chương nên đọc gần nhau. [B/07-chia-du-lieu-va-danh-gia](#) là tiền đề: không đọc được phần dữ liệu thì không đọc được bảng kết quả. [B/03-giai-phau-he-thong](#) cho bạn khung ba tầng để phân loại một đóng góp là ý tưởng hay chi tiết cài đặt. [D/22-tai-lap-va-mlops](#) là mặt đối lập hữu ích: nó cho thấy vì sao ngay cả tác giả trung thực cũng không tái lập được số của chính mình sau sáu tháng, nên khoảng cách giữa paper và thực tế không phải lúc nào cũng là dấu hiệu của sự thiếu trung thực.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao đọc phần limitations trước bảng kết quả lại đổi được cách bạn diễn giải cùng một con số, dù thông tin nhận được là như nhau?
2. Một paper báo cáo cải tiến 1,0 mAP nhưng không ghi số seed: ba cách giải thích khác nhau cho con số đó là gì, và với mỗi cách thì chi tiết nào trong paper sẽ giúp bạn loại trừ nó?
3. Vì sao “công thức huấn luyện tốt hơn” là một đóng góp thật nhưng lại là loại đóng góp khác với “một module mới”? Sự khác nhau nằm ở đâu về mặt khả năng chuyển giao?
4. Mục “chỗ tôi không tin” vô dụng nếu thiếu mục nào đi kèm, và vì sao?
5. Khi nào việc đọc theo cụm hỏng, và bạn thay bằng gì trong giai đoạn đó?
6. Cây phả hệ có thể tạo ảo giác hiểu biết. Phép thử nào phát hiện được ảo giác đó?

Thiết kế thí nghiệm, ablation và phân tích lỗi (Experiment Design, Ablation, Error Analysis)

PHỤ THUỘC B/07-*chia-du-lieu-va-danh-gia* (chia dữ liệu, chọn metric — chương này giả định bạn đã có một giao thức đánh giá đúng); D/22-*tai-lap-va-mlops* (config, tracking, nguồn phi tắt định); E/23-*doc-paper* là mặt đọc của cùng kỹ năng này.

TÓM TẮT

Chương này là về cách *sinh ra bằng chứng*: thiết kế một so sánh có nghĩa, cắt bỏ thành phần đúng cách, biết bao nhiêu lần chạy là đủ, và đọc lỗi để quyết định đầu tư tiếp vào đâu. Đọc xong bạn tính được cỡ mẫu cần thiết từ sà nhiều, chọn được giữa leave-one-out và oracle experiment cho đúng câu hỏi mình đang hỏi, và viết được một bảng ablation mà người hoài nghi nhất cũng dùng được. Với nền SLAM sẵn có, phần bạn còn thiếu không phải kỹ thuật lặp lại nhiều lần mà là ba thứ đặc thù deep learning: phi tắt định không sửa được, bất đối xứng ngân sách tune, và rò rỉ tập test tích lũy qua nhiều tháng. Nếu chỉ nhớ một điều: đo sà nhiều *trước*, nói về cải tiến *sau* — và một kết luận âm ghi kèm điều kiện thí nghiệm vẫn là một kết quả.

KIẾN THỨC CẦN CÓ TRƯỚC

- **ablation** — thí nghiệm bỏ đi hoặc thay thế đúng *một* thành phần của hệ thống rồi đo lại, để biết thành phần đó đóng góp bao nhiêu.
- **baseline** — phương pháp đối chứng mà kết quả mới phải vượt qua; baseline chưa được tinh chỉnh từ tế làm mọi so sánh sau đó vô nghĩa.
- **seed và phương sai giữa các lần chạy** — seed là số khởi tạo bộ sinh ngẫu nhiên; chạy lại cùng cấu hình với seed khác cho kết quả khác, và độ tán mát của các kết quả đó là phương sai giữa các lần chạy — phần bạn đã quen từ harness SLAM, ở đây chỉ đổi nguồn gây ra nó.
- **độ lệch chuẩn σ** — thước đo độ tán mát của một tập số quanh trung bình của chúng. Trong chương này σ luôn là độ lệch chuẩn *giữa các seed* của cùng một cấu hình, tức là “sà nhiều”.
- **khoảng tin cậy** — một khoảng số kèm mức tin cậy (thường 95%), tính từ dữ liệu, sao cho nếu lặp lại toàn bộ quy trình nhiều lần thì khoảng ấy chứa giá trị thật trong đúng tỉ lệ đó. Nó trả lời “tôi biết con số này chính xác tới đâu”, không trả lời “giả thuyết của tôi đúng hay sai”.
- **p-value** — xác suất quan sát được chênh lệch ít nhất bằng chênh lệch đã thấy, *nếu* giả thuyết “không có khác biệt” là đúng. Nó *không* phải xác suất giả thuyết của bạn đúng, và một p-value nhỏ trên tập test rất lớn có thể ứng với một chênh lệch nhỏ tới mức vô nghĩa về mặt triển khai.
- **cỡ hiệu ứng (effect size)** — độ lớn của chênh lệch, đo bằng đơn vị của bài toán (0,8 mAP) hoặc chuẩn hoá theo σ . Đây mới là đại lượng quyết định “có đáng triển khai không”.
- **oracle experiment** — thay một khối trong hệ thống bằng đáp án đúng, tức **chân trị** (*ground truth*), để đo *trần* đóng góp của khối đó. Nếu khối hoàn hảo cũng chỉ nâng hệ thống 2% thì mọi công sức tối ưu nó đều bị chặn trên bởi 2%.
- **mAP** — metric tổng hợp thông dụng cho phát hiện vật thể; ở đây chỉ cần hiểu là “điểm số càng cao càng tốt”, tính theo điểm phần trăm.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn thử hai mươi cấu hình rồi lấy cái tốt nhất và thấy nó hơn baseline 1,1 điểm. Vì sao con số đó có thể hoàn toàn là nhiễu — và bạn tính ra điều đó bằng phép tính nào, trong ba mươi giây?
- Q2. Quy tắc “chạy ba lần” mà bạn đã dùng cho harness SLAM là đúng, nhưng ba là bao nhiêu phần trăm của cái cần thiết? Với sàn nhiễu 0,6 điểm, ba lần chạy bỏ sót bao nhiêu phần số cải tiến có thật?
- Q3. Bạn bỏ một module và mAP giảm 1,5. Kể ra bốn cách giải thích khác ngoài “module đó quan trọng” — và cách nào trong số đó luôn dẫn kết luận về cùng một hướng?
- Q4. Nếu một detector hoàn hảo cũng chỉ nâng hệ thống tổng thể 2%, bạn còn lý do gì để tối ưu detector — và làm sao biết được con số 2% ấy *trước* khi bỏ ba tháng?
- Q5. Vì sao một kết luận NO-GO không ghi điều kiện thí nghiệm lại có hại hơn là không ghi gì cả?

24.1 Đặt vấn đề

Chương 7 nói về *giao thức đánh giá*: chia dữ liệu thế nào, đo bằng metric gì. Chương này nói về một việc khác hẳn — *sinh ra bằng chứng*: làm thế nào để một so sánh có nghĩa, và làm thế nào để biết nên đầu tư công sức tiếp theo vào đâu. Đây là kỹ năng phân biệt người làm nghiên cứu với người chạy script, và nó gần như không được dạy ở đâu cả; hầu hết người làm nghề học nó bằng cách tự trả giá.

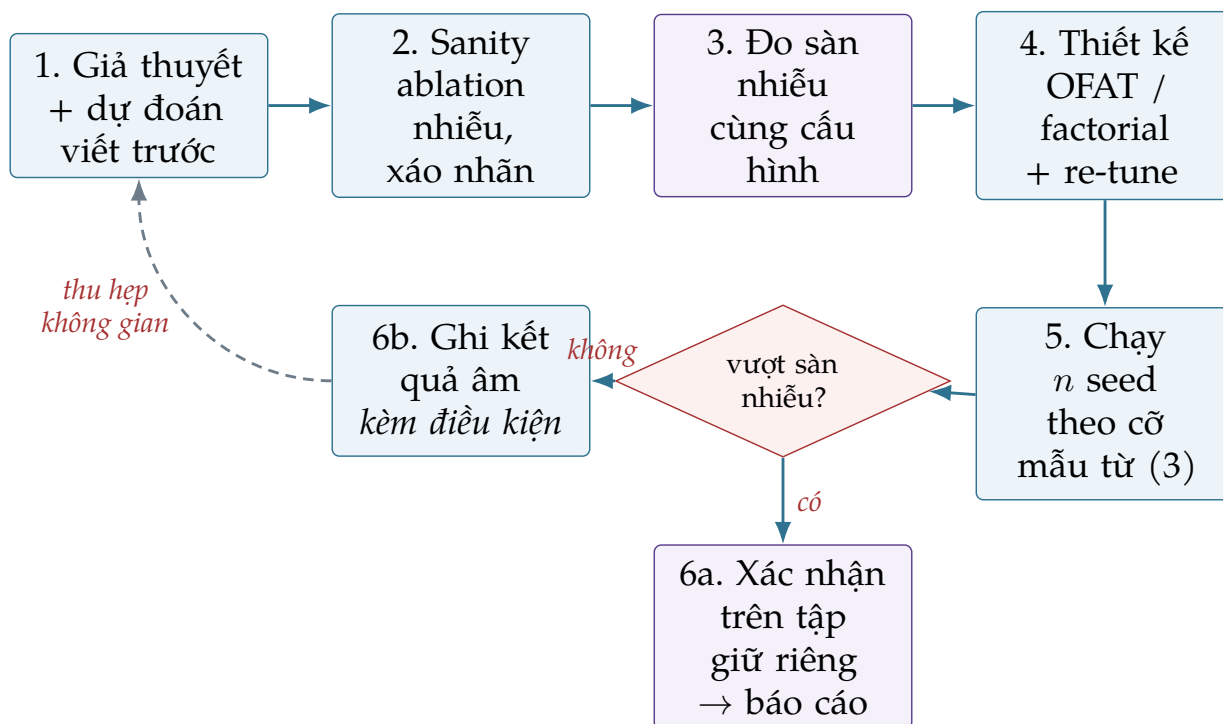
Bạn đã trả một phần cái giá đó rồi, ở phía SLAM. Khi dựng một harness chạy lặp trên EuRoC, bạn phát hiện chính hệ thống của mình là phi tất định: hệ thống có hai nguồn phi tất định cùng lúc.

- **Thứ tự duyệt container** — các container đánh chỉ mục bằng con trỏ (*pointer-ordered containers*) đổi thứ tự theo cách cấp phát bộ nhớ.
- **Tranh chấp luồng** (*race condition*) — giữa luồng tracking và luồng local mapping, khiến một cờ trạng thái đọc ra khác nhau giữa các lần chạy.

Hệ quả giống nhau ở cả hai nguồn: chênh lệch một inlier lan thành một quyết định keyframe khác, rồi thành một quỹ đạo khác. Từ đó bạn đặt quy tắc: mọi cấu hình chạy ít nhất ba lần với ba *seed* khác nhau. Bạn cũng đã tự viết những loạt ablation nhiều cấu hình và rút ra vài kết luận NO-GO, tức là đã biết bằng kinh nghiệm rằng phần lớn ý tưởng nghe hay đều chết, và rằng biết chúng chết là một kết quả.

Chương này vì thế không giảng lại nhóm C từ đầu. Nó hệ thống hoá những gì bạn đã biết thành một khung có tên gọi, để bạn áp được sang bài toán mới mà không phải phát hiện lại, và chỉ ra ba chỗ mà môi trường deep learning *khác*. Thứ nhất, nguồn phi tất định ở đây không sửa được — không có bug nào để tìm — nên mục tiêu phải chuyển từ “tái lập từng chữ số” sang “tái lập phân phối kết quả”. Thứ hai, không gian siêu tham số lớn hơn nhiều, nên bất đối xứng ngân sách tune giữa phương pháp mới và baseline trở thành nguồn kết luận sai lớn nhất, lớn hơn cả nhiễu seed. Thứ ba, tập test bị dùng lại hàng trăm lần trong nhiều tháng. Nó còn có thể đã nằm sẵn trong dữ liệu pretraining của mô hình nền bạn dùng. Đó là hai dạng rò rỉ mà một pipeline SLAM chạy trên EuRoC cũng mắc, chỉ là ít khi được gọi tên.

Hình 24.1 tóm cả chương thành một quy trình. Đọc nó trước rồi mới đọc năm nhóm bên dưới sẽ nhanh hơn nhiều, vì mỗi nhóm chỉ là một trạm trên đúng đường đi này.



Hình 24.1. Từ giả thuyết tới kết luận. Ba chỗ hay bị bỏ qua nhất được tô đậm hoặc rẽ nhánh. Bước 3: đo sản nhiều *trước* khi so sánh. Bước 5: tính cỡ mẫu từ chính sản nhiều đó. Bước 6b: coi nhánh “không vượt sản” là một kết quả phải ghi lại, không phải một lần chạy bỏ đi; mũi tên đứt là chỗ nó trả giá trị về cho vòng sau.

TRỰC GIÁC

Bỏ một module ra khỏi hệ thống rồi giữ nguyên learning rate cũ giống tháo giảm chấn khỏi hệ treo rồi vẫn chạy thử ở đúng tốc độ và đúng áp suất lốp cũ. Con số bạn đo được không phải “xe không có giảm chấn” mà là “xe không có giảm chấn *cộng* với một bộ tham số vận hành đã trở nên sai cho nó”. Sai lệch này luôn cùng chiều — nó thổi phồng tầm quan trọng của thành phần bị bỏ — nên nó không tự triệt tiêu khi bạn lặp lại nhiều lần.

24.2 Nhóm A — thiết kế so sánh (Comparison Design)

A. Ba loại câu hỏi, ba thiết kế

Lỗi phổ biến nhất trong thiết kế thí nghiệm không phải lỗi thống kê mà là lỗi phân loại: dùng một thiết kế để trả lời câu hỏi mà nó không trả lời được. Có đúng ba loại câu hỏi và chúng cần ba thiết kế khác nhau. “Cái này có hoạt động không” là so sánh với baseline, và thiết kế của nó đòi hỏi kiểm soát chặt mọi biến gây nhiễu cùng nhiều seed. “Vì sao nó hoạt động” là ablation, và thiết kế của nó đòi hỏi mỗi lần chỉ đổi một thứ, kèm re-tune. “Nó hoạt động tới đâu” là sensitivity và scaling. Thiết kế của nó cần nhiều điểm dọc một trục: kích thước dữ liệu, kích thước mô hình, hoặc mức nhiễu. Bù lại nó chấp nhận ít seed mỗi điểm, vì cái ta đọc là *xu hướng* chứ không phải một hiệu số. Lấy một bảng ablation ra để chứng minh phương pháp hoạt động, hoặc lấy một so sánh baseline ra để giải thích cơ chế, là hai cách trộn lẫn hay gặp nhất và cả hai đều cho kết luận không đứng vững.

B. Ba ràng buộc phải thoả trước khi con số có nghĩa

Trên nền đó, ba ràng buộc phải thoả trước khi bất kỳ con số nào có nghĩa. **Baseline mạnh là nghĩa vụ, không phải phép lịch sự:** so với baseline bị tune kém là cách tự lừa mình phổ biến nhất, và nó phổ biến vì rẻ. **Kiểm soát biến gây nhiễu:** cùng dữ liệu, cùng augmentation, cùng lịch huấn luyện, cùng số seed. Nếu phương pháp mới đắt hơn 1,4 lần mỗi bước mà vẫn chạy cùng số epoch, nó đã được tặng thêm 40% tính toán. Riêng lượng tính toán thêm đó đủ tạo khác biệt đo được với các công thức huấn luyện hiện đại [127]. **Ngân sách tune công bằng:** cơ chế ở đây thuần xác suất — giá trị tốt nhất quan sát được là cực đại của một mẫu, và kỳ vọng của cực đại tăng theo cỡ mẫu [206], nên một cải tiến 1% rút ra từ 50 lần thử so với baseline được thử 5 lần không phải là cải tiến mà là hiệu ứng của cỡ mẫu.

→ hệ quả của E/23 — đây chính là dấu hiệu “ngân sách tính chính bất đối xứng” mà bạn dùng để đọc bảng kết quả của người khác

C. Một-biến-một-lần hay giai thừa

Còn một lựa chọn thiết kế nữa hay bị bỏ qua: một-biến-một-lần (OFAT) hay giai thừa (factorial). OFAT rẻ và trực giác, nhưng mù hoàn toàn với tương tác — nếu hai thành phần cộng hưởng hoặc triệt tiêu nhau, OFAT không thể phát hiện. Với ba thành phần, một thiết kế giai thừa đầy đủ có $2^3 = 8$ cấu hình, nhân ba seed là 24 lần chạy; OFAT chỉ cần cấu hình gốc cộng ba lần bỏ, tức 4 cấu hình và 12 lần chạy. Gấp đôi chi phí để đổi lấy toàn bộ tương tác bậc hai thường là món hời. Khi số thành phần lớn hơn, thiết kế giai thừa rút gọn cho phép trả ít hơn nhiều so với 2^k mà vẫn tách được các tương tác đáng kể.

24.3 Nhóm B — kỹ thuật ablation (Ablation Techniques)

A. Hai chiều bỏ thành phần, hai câu hỏi khác nhau

Có hai chiều bỏ thành phần, và chúng trả lời hai câu hỏi khác nhau. *leave-one-out* bỏ từng thành phần khỏi hệ thống đầy đủ; *add-one-in* thêm từng thành phần vào baseline trần. Khi các thành phần độc lập, hai chiều cho cùng con số. Khi có tương tác, chúng tách ra, và *chính khoảng cách giữa chúng là thông tin*. Ví dụ nhỏ nhất: giả sử thêm riêng A được +1,0, thêm riêng B được +1,0, nhưng A và B cùng lúc chỉ được +1,2. Khi đó *leave-one-out* báo “bỏ A mất 0,2” còn *add-one-in* báo “thêm A được 1,0”. Cả hai đều đúng; chúng chỉ trả lời hai câu khác nhau — “A còn đáng giữ trong hệ thống này không” và “A có tự nó tạo ra giá trị không”. Chỉ báo cáo một chiều rồi phát biểu như thể đó là “đóng góp của A” là chỗ mập mờ hay gập nhất trong các bảng ablation đã công bố.

B. Ba loại đối tượng, và luật re-tune

Ablation có ba loại theo đối tượng bị bỏ: kiến trúc, dữ liệu, và công thức huấn luyện. Hai loại sau gần như luôn bị bỏ qua, dù chúng gần như luôn giải thích phần lớn chênh lệch. Đây chính là mặt trong của kết luận ở chương 23 về đóng góp thật so với recipe: nếu bạn không ablate recipe, bạn không có cách nào biết đóng góp của mình có thật không.

Ràng buộc quan trọng nhất của cả nhóm là **ablation phải re-tune**, vì lý do ở hộp trực giác: bỏ một thành phần có tác dụng ổn định hoá thì learning rate tối ưu của phần còn lại thường nhỏ đi, nên giữ nguyên learning rate cũ vừa làm hệ thống tệ hơn vừa thổi phồng tầm quan trọng của thành phần vừa bỏ. Thoả hiệp thực dụng là re-tune *ít nhất* learning rate trên một lưới thô ba điểm và ghi rõ rằng chỉ có learning rate được re-tune.

C. Sanity ablation chạy trước

Trước mọi ablation tinh vi, hãy chạy nhóm **sanity ablation**: thay đầu vào bằng nhiễu, xáo trộn nhãn, cắt bỏ một modality, thay một module bằng hằng số. Nếu điểm không giảm thì hoặc có rò rỉ, hoặc thành phần đó vô dụng, và cả hai kết luận đều dừng cuộc điều tra lại tại chỗ. Nhóm này rẻ và có sức phá hoại giả thuyết cao nhất, nên chạy trước là đúng thứ tự kinh tế. Một lưu ý về diễn giải: mạng đủ lớn có thể học thuộc cả một tập nhãn ngẫu nhiên, nên “train loss vẫn giảm khi xáo nhãn” không phải dấu hiệu rò rỉ — chỉ có *điểm trên tập đánh giá* không giảm mới là [54].

D. Oracle experiment và ablation theo quy mô

Công cụ mạnh nhất để quyết định đầu tư lại là loại ablation ngược: **oracle** hay **skyline**, tức thay một khối bằng chân trị và đo trần đóng góp của khối đó. Nếu một detector hoàn hảo chỉ nâng hệ thống tổng thể lên 2%, thì mọi công sức tối ưu detector đều bị chặn trên bởi 2% và bạn nên đi chỗ khác. Bạn đã dùng đúng công cụ này ở phía SLAM khi thay tầng phát hiện vòng lặp bằng một oracle lấy từ chân trị để tách “không phát hiện được vòng lặp” khỏi “phát hiện được nhưng không dùng được” — và kết luận thu được đã định hướng lại toàn bộ hướng làm việc, đó là điều một ablation thông thường không làm được. Cuối cùng, mọi kết luận ablation đều gắn với quy mô: kết luận rút ra trên 1% dữ liệu thường đảo ngược ở 100%, vì các thành phần có tác dụng chống quá khớp mất ý nghĩa khi dữ liệu đủ nhiều. Luôn ghi ablation chạy ở quy mô nào.

→ **trường hợp riêng của B/03** — oracle experiment là cách trả lời bằng số cho câu hỏi “độ khó đã bị đẩy đi đâu”: nó xoá hẳn độ khó của một khối rồi đo phần còn lại

Bảng 24.1. Năm kiểu ablation, mỗi kiểu trả lời một câu hỏi khác nhau. Cột cuối là cột đáng giá nhất: nó cho biết khi nào con số thu được không có nghĩa như ta tưởng.

Kiểu	Trả lời câu hỏi	Giả định	Hỏng khi nào
Leave-one-out	Thành phần này còn đáng giữ không	Phần còn lại vẫn ở điểm vận hành tốt	Không re-tune; hoặc thành phần bị bỏ kéo theo thay đổi ngầm
Add-one-in	Thành phần này tự nó có giá trị không	Baseline trần vẫn huấn luyện ổn định	Có tương tác mạnh: con số không cộng lại thành hệ thống đầy đủ
Sanity	Có rò rỉ, hay thành phần vô dụng	Metric đánh giá nhạy với thứ bị phá	Đọc nhầm train loss thay vì điểm đánh giá
Oracle / skyline	Trần đóng góp của khối này là bao nhiêu	Chân trị cho khối đó tồn tại và đúng định dạng	Oracle làm đổi phân phối đầu vào của khối sau, cho trần lạc quan giả
Theo quy mô	Kết luận có bền theo lượng dữ liệu không	Xu hướng đơn điệu giữa hai đầu	Chỉ đo hai điểm; giao nhau nằm giữa mà không thấy

24.4 Nhóm C — thống kê tối thiểu (Minimal Statistics)

A. Sàn nhiễu trước, cỡ mẫu sau

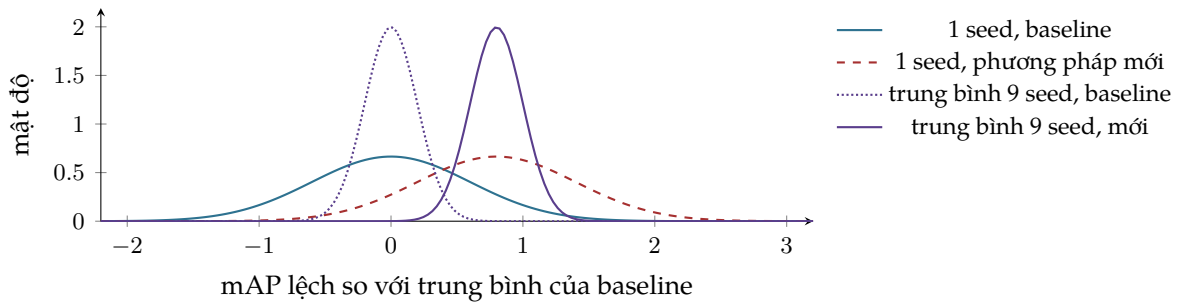
Quy tắc ba lần chạy của bạn đúng về nguyên tắc nhưng ba là *sàn*, không phải mục tiêu; số lần chạy cần thiết phụ thuộc vào tỉ số giữa hiệu ứng muốn phát hiện và độ lệch chuẩn giữa các seed. Thứ tự đúng vì thế là **đo sàn nhiễu (noise floor) trước, nói về cải tiến sau**: chạy *cùng một cấu*

hình nhiều lần chỉ đổi seed, lấy độ lệch chuẩn, và chỉ từ đó mới biết chênh lệch bao nhiêu là đáng nói. Một tổ chức không biết sản phẩm của mình sẽ dành nhiều tháng để đuổi theo nhiều.

Ví dụ tính tay được, dùng đúng những con số hay gặp trong detection. Giả sử độ lệch chuẩn giữa các seed là $\sigma = 0,6$ mAP và bạn muốn phát hiện một cải tiến thật $\Delta = 0,8$ mAP. Với kiểm định hai mẫu, xấp xỉ chuẩn cho cỡ mẫu mỗi nhóm

$$n \approx \frac{2(z_{1-\alpha/2} + z_{1-\beta})^2 \sigma^2}{\Delta^2} = \frac{2(1,96 + 0,84)^2 (0,6)^2}{(0,8)^2} \approx 8,8,$$

tức khoảng chín seed mỗi cấu hình, mười tám lần chạy cho một so sánh. Bây giờ đi ngược lại, với ba seed mỗi bên. Sai số chuẩn của hiệu số khi đó là $0,6\sqrt{2/3} \approx 0,49$. Hiệu số thật 0,8 chỉ nằm cách ngưỡng 1,96 đúng 0,33 độ lệch. Xác suất phát hiện được rơi vào khoảng 0,35 theo xấp xỉ chuẩn, và còn thấp hơn nếu tính bằng phân phối t với bậc tự do nhỏ. Nói cách khác, ở ba seed bạn bỏ sót khoảng hai phần ba số cải tiến có thật. Hình 24.2 cho thấy vì sao: hai phân phối một-lần-chạy chồng lên nhau gần hết, và chỉ khi lấy trung bình nhiều seed thì chúng mới tách ra.



Hình 24.2. Với $\sigma = 0,6$ mAP giữa các seed, một cải tiến thật 0,8 mAP nằm gần như hoàn toàn trong vùng chồng lấn của hai phân phối một-lần-chạy. Lấy trung bình chín seed thu hẹp độ lệch xuống $\sigma/3 = 0,2$ và tách hai phân phối ra. Đây là lý do hình học của công thức cỡ mẫu ở trên.

B. Hai cách rẻ hơn là tăng số seed

Có hai cách rẻ hơn là tăng số seed. Thứ nhất, **so sánh theo cặp** (*paired comparison*): hai mô hình được đánh giá trên đúng cùng một tập test, nên phần phương sai do độ khó của từng mẫu là chung và biến mất khi lấy hiệu số. Bootstrap theo cặp làm thế này: lấy mẫu lại các ảnh, rồi tính hiệu số metric trên mỗi lần lấy mẫu. Cách đó cho khoảng tin cậy chặt hơn hẳn so với việc so hai con số tổng, và không tốn thêm một lần huấn luyện nào. Thứ hai, báo cáo **cỡ hiệu ứng** (*effect size*) chứ không chỉ p-value: “có ý nghĩa thống kê” và “đáng để triển khai” là hai mệnh đề khác nhau, và với tập test đủ lớn thì mọi khác biệt đều có ý nghĩa thống kê. Khi phải so nhiều phương pháp trên nhiều tập dữ liệu thì các kiểm định phi tham số theo hạng là lựa chọn an toàn hơn kiểm định t [17].

C. So sánh bội và rò rỉ tập test

Vấn đề còn lại là **so sánh bội** (*multiple comparisons*), và nó nghiêm trọng hơn vẻ ngoài. Nếu bạn thử 20 biến thể đều vô dụng như nhau và báo cáo cái tốt nhất, thì cái tốt nhất đó là cực đại của 20 mẫu độc lập từ cùng phân phối nhiễu. Kỳ vọng của cực đại 20 mẫu chuẩn tắc vào khoảng 1,9; với $\sigma = 0,6$ mAP, đó là một “cải tiến” khoảng 1,1 mAP sinh ra hoàn toàn từ nhiễu. Con số này lớn hơn phần lớn cải tiến được công bố trong các bảng detection, và nó giải thích vì sao thứ tự thao tác — chọn trên val, báo cáo trên test đúng một lần — không phải hình thức. Cùng cơ chế đó vận hành chậm hơn nhưng chắc chắn hơn khi bạn dùng lại một tập test suốt nhiều tháng: mỗi quyết định thiết kế dựa trên điểm test là một lần rò rỉ nhỏ. Bằng chứng thực nghiệm ở đây tinh tế: khi dựng lại một tập test mới cho ImageNet và CIFAR-10 theo đúng quy trình gốc, mọi mô hình đều tụt điểm đáng kể, nhưng *thứ hạng giữa chúng gần như được bảo toàn* [18]. Diễn giải đúng là khoảng

cách tuyệt đối không đáng tin trong khi so sánh tương đối thì bền hơn ta lo — chứ không phải “rò rỉ thích ứng không tồn tại”.

CẠM BẮY

Hai dạng ô nhiễm đặc thù của deep learning mà kỷ luật SLAM không dạy bạn. *Một*, mô hình nền được pretrain trên dữ liệu web quy mô lớn có thể đã nhìn thấy chính ảnh trong tập test của bạn; khi đó “zero-shot” là một tuyên bố không kiểm chứng được nếu không dò trùng lặp. *Hai*, chính tập test có lỗi nhãn ở tỉ lệ không nhỏ trong các benchmark phổ biến, và ở vùng chênh lệch dưới một điểm thì thứ hạng có thể đảo khi sửa nhãn [19]. Cả hai đều không hiện ra dưới dạng phương sai giữa seed, nên chạy thêm seed không phát hiện được chúng.

D. Phi tất định trong deep learning

Cuối cùng là phần bạn đã sống qua nhưng ở dạng khác: nguồn phi tất định. Các nguồn chính trong deep learning:

- **Chọn thuật toán** — *autotune* của *cuDNN* chọn kernel tích chập khác nhau giữa các lần chạy tùy theo đo thời gian tức thời.
- **Cộng không có thứ tự** — các kernel dùng `atomicAdd` ở **lan truyền ngược** (*backpropagation*) cộng dồn theo thứ tự do lịch chạy quyết định; phép cộng dấu phẩy động không kết hợp, nên tổng đổi ở chữ số cuối. Đây đúng là cơ chế đã sinh ra chênh lệch một inlier trong hệ SLAM của bạn, chỉ khác là ở đây nó nằm trong thư viện chứ không nằm trong code bạn sửa được.
- **Độ chính xác rút gọn** — *TF32* và các đường *mixed precision* trên tensor core cho kết quả khác FP32 và khác nhau giữa các thế hệ GPU.
- **Thứ tự dữ liệu** — dataloader nhiều *worker* trả *batch* theo thứ tự hoàn thành, và mỗi worker có dòng số ngẫu nhiên riêng cho *augmentation*.

Bật chế độ tất định xử lý được vài nguồn trong số đó, phải trả bằng tốc độ, và *vẫn* không đảm bảo hai máy khác GPU cho cùng kết quả. Kết luận thực dụng — giống hệt kết luận bạn đã rút ra cho harness của mình, chỉ khác là ở đây không có bug nào để sửa — là mục tiêu đúng phải chuyển từ tái lập từng chữ số sang tái lập phân phối: cùng seed, cùng máy thì khớp; khác máy thì trung bình và độ lệch chuẩn phải nằm trong khoảng tin cậy của nhau. Phần hạ tầng để làm được điều đó nằm ở *D/22-tai-lap-va-mlops*.

→ cùng nguyên lý với *D/22* — cùng lý do phải gắn mỗi lần chạy với một commit, một config và một phiên bản dữ liệu: cái không ghi lại được thì không so lại được

24.5 Nhóm D — phân tích lỗi (Error Analysis)

A. Một trăm lỗi gán tay

Đến đây bạn đã biết một cải tiến có thật hay không. Câu hỏi tiếp theo đắt hơn nhiều: nên đầu tư vào đâu. Không có công cụ tự động nào thay thế được việc **phân loại một trăm lỗi bằng tay**. Quy trình là lấy ngẫu nhiên một trăm mẫu bị sai, xem từng mẫu, dựng dần một taxonomy lỗi từ chính chúng, đếm tần suất mỗi loại, rồi ước lượng trần cải thiện nếu loại đó biến mất hoàn toàn. Cỡ mẫu một trăm không tùy tiện: với một loại chiếm 20%, sai số chuẩn của tỉ lệ là $\sqrt{0,2 \cdot 0,8/100} = 0,04$, tức là đủ để xếp hạng các loại lỗi lớn nhưng không đủ để phân biệt hai loại chênh nhau vài điểm

phần trăm. Điều đó là đủ, vì mục đích ở đây là chọn ra một hoặc hai loại đáng theo đuổi, chứ không phải xếp hạng chính xác.

B. Phân rã tự động và phân tích theo lát cắt

Phân rã tự động bổ sung chứ không thay thế bước thủ công. Với detection, các công cụ tách lỗi thành nhầm lớp, lệch vị trí, trùng lặp, bỏ sót và nền bị nhận nhầm; cách tính đúng đắn là đo phần mAP lấy lại được nếu sửa riêng từng loại, thay vì chỉ đếm số lượng [66], [85]. Giá trị của cách phân rã này là mỗi loại lỗi dẫn tới một cách sửa *khác nhau*:

- **Lệch vị trí** — hàm mất mát hồi quy hộp và độ phân giải đặc trưng.
- **Nhầm lớp** — dữ liệu và ranh giới định nghĩa lớp.
- **Bỏ sót** — quy tắc gán mẫu dương và ngưỡng điểm.
- **Trùng lặp** — hậu xử lý.

Bên cạnh đó là **phân tích theo lát cắt** (*slice analysis*): chia tập đánh giá theo các tiêu chí có nghĩa — kích thước vật thể, ánh sáng, camera, nhóm người — và đọc metric trên từng lát. Đây là nơi con số tổng nói dối nhiều nhất, vì một mô hình có thể giữ nguyên điểm tổng trong khi tụt sâu ở một lát chiếm 5% dữ liệu nhưng lại là 100% các trường hợp quan trọng. Với hệ thống robot, các lát tự nhiên là chuyển động nhanh, ánh mờ, bề mặt ít texture — và bạn đã biết rằng một con số trung bình trên toàn EuRoC che giấu hoàn toàn việc một chuỗi cụ thể sập.

C. Bốn ô hành động

Bước cuối khép vòng: mỗi loại lỗi phải được gán vào đúng một trong bốn ô hành động.

- **Sửa bằng dữ liệu** — lỗi tập trung ở một vùng phân phối thiếu mẫu, hoặc ở ranh giới định nghĩa lớp mà chính người gán nhãn cũng bất đồng.
- **Sửa bằng hàm mất mát (*loss function*)** — lỗi có dạng lệch hệ thống mà hàm mất mát hiện tại không phạt, ví dụ lệch vị trí hộp hoặc mất cân bằng lớp.
- **Sửa bằng kiến trúc** — lỗi do thiếu thông tin ở đầu vào của tầng quyết định: độ phân giải đặc trưng, trường tiếp nhận, phạm vi ngưỡng cảnh.
- **Sửa bằng ràng buộc hệ thống** — lọc theo thời gian, luật hậu kiểm, hoặc từ chối trả lời. Đây là ô mà một kỹ sư robot quen nhất, và cũng là ô rẻ nhất. Nó chỉ giấu lỗi đi chứ không sửa mô hình. Nhưng với các lỗi hiếm mà hậu quả lớn, giấu đúng cách lại chính là đáp án; xem [E/25-do-tin-cay-va-van-hanh](#).

Việc gán này phải làm *trước* khi viết một dòng code, vì nó là chỗ duy nhất trong quy trình mà bạn còn ra quyết định bằng suy nghĩ thay vì bằng quán tính. Kinh nghiệm thực hành lặp lại ở nhiều nhóm — không phải một kết quả đo được — là ô thứ nhất thắng nhiều hơn người ta tưởng và ô thứ ba thắng ít hơn người ta tưởng.

I 24.6 Nhóm E — báo cáo và đọc ablation của người khác (Reporting)

↔ **tiền đề B/06** — taxonomy lỗi dựng từ một trăm mẫu gán tay chính là đầu vào của vòng lỗi → dữ liệu ở chương chất lượng dữ liệu

A. Bảng ablation đạt chuẩn

Một bảng ablation đạt chuẩn phải mang theo bốn thứ mà phần lớn bảng đã công bố không có:

- **Số seed và độ lệch chuẩn** — thiếu hai con số này thì mỗi dòng chỉ là một mẫu duy nhất.
- **Ngân sách tính chỉnh của từng dòng** — mỗi dòng đã được thử bao nhiêu cấu hình.
- **Tập nào** — nói rõ ablation chạy trên validation chứ không phải trên test.
- **Thay đổi ngầm kéo theo** — bỏ một thành phần thường đổi luôn số tham số, thời gian huấn luyện và learning rate tối ưu; phải ghi rõ cái nào đã được re-tune.

Thiếu bốn thứ này thì bảng vẫn đọc được như một bảng, nhưng nó không còn là bằng chứng.

B. Ghi cả kết quả âm

Điểm thứ hai quan trọng hơn và ít được làm nhất: **ghi lại cả kết quả âm**. Một ý tưởng đã thử và không hiệu quả có giá trị gần bằng một ý tưởng hiệu quả, vì nó thu hẹp không gian tìm kiếm cho chính bạn sáu tháng sau và cho người tiếp theo. Bạn đã có sẵn thói quen này ở phía SLAM — vài kết luận NO-GO được ghi lại kèm điều kiện thí nghiệm — và nên giữ nguyên nó khi chuyển sang deep learning, với một điều chỉnh: kết quả âm chỉ có giá trị khi nó *ghi kèm phạm vi hiệu lực*. “Ý tưởng X không hiệu quả” là một câu mê tín; “ý tưởng X không cho cải tiến vượt sà nhiều $\pm 0,6$ mAP trên tập này, ở quy mô này, với ngân sách tune này, qua ba seed” là một kết quả. Mệnh đề thứ hai cho bạn biết điều kiện nào thay đổi thì đáng thử lại. Trong deep learning điều kiện thay đổi liên tục, nên một NO-GO không ghi điều kiện sẽ chặn bạn khỏi đúng cơ hội về sau.

C. Đọc ablation của người khác

Điểm thứ ba là mặt đọc, chính là chương 23 áp vào một bảng cụ thể: ablation chạy trên tập nào, bao nhiêu seed, baseline có được tune không, và thành phần bị ablate có kéo theo thay đổi ngầm nào không. Bốn câu hỏi đó loại được phần lớn bảng ablation rỗng, nhanh hơn mọi cách đọc khác.

CẦN NHỚ

Một kết luận âm được ghi lại *kèm điều kiện thí nghiệm* là một kết quả, không phải một thất bại. Nó là thứ duy nhất trong công việc thí nghiệm không bị mất giá theo thời gian, miễn là bạn ghi đủ điều kiện để biết khi nào nên thử lại. [T2]

24.7 Giới hạn và trường hợp hỏng

Kỷ luật trên có một chi phí thẳng thắn: chín seed nhân một thiết kế giai thừa là ngân sách rất ít nhóm trả nổi. Thoả hiệp thực dụng là phân tầng — sàng nhanh bằng một seed để loại ứng viên vô vọng, rồi chỉ chạy đầy đủ cho cấu hình sống sót. Nhưng vòng sàng đã tiêu một phần “ngân sách tin cậy”: cấu hình sống sót được chọn bởi nhiều cũng nhiều như bởi chất lượng, nên con số của vòng đầy đủ vẫn thiên lệch lên trên nếu dùng lại cùng tập dữ liệu. Cách sửa rẻ nhất là tách tập sàng khỏi tập xác nhận.

Chế độ hỏng thứ hai mang tính khái niệm. Ablation giả định các thành phần *tách rời được*, tức bỏ một thành phần không làm hệ thống chuyển sang chế độ vận hành khác. Giả định này đúng với phần lớn mạng feed-forward và sai với hệ có vòng phản hồi: trong một pipeline SLAM, chất lượng bản đồ chỉ phối việc chọn keyframe và ngược lại, nên bỏ một thành phần trong vòng đó cho ra một hệ thống khác chứ không phải “hệ thống cũ trừ thành phần”. Khi ấy “đóng góp của

thành phần” không còn là đại lượng xác định, và câu hỏi đúng đổi thành: hệ chuyển sang chế độ nào, và chế độ đó tốt hay xấu hơn ở đâu.

Chế độ hỏng thứ ba là chính sự tự tin do thống kê mang lại: một khoảng tin cậy hẹp chỉ nói rằng bạn đã đo chính xác *trên phân phối của tập đánh giá*. Nếu tập đó không đại diện cho môi trường triển khai, mọi kỷ luật ở chương này chỉ giúp bạn sai một cách rất chính xác — địa hạt của *B/08-dich-chuyen-phan-phoi*.

24.8 Thực hành

Repo và công cụ. [dbolya/tide](#) để phân rã lỗi detection; [voxel51/fiftyone](#) để cắt lát, xem lỗi và so hai mô hình trên cùng mẫu; [wandb/wandb](#) sweeps và [optuna](#) để quản lý nhiều lần chạy có kỷ luật; `scipy.stats` cộng một hàm bootstrap tự viết cho kiểm định theo cặp; [facebookresearch/hydra](#) multirun để một thay đổi luôn tương ứng một config diff. Nên đọc thêm phần ablation của những paper có kỷ luật thí nghiệm tốt — dòng ConvNeXt, dòng “bag of tricks”, các paper về công thức huấn luyện — vì chúng dạy phương pháp rõ hơn hẳn các paper kiến trúc. Danh sách công cụ này chốt tại tháng 9/2026 và sẽ cũ trong sáu đến mười hai tháng; các nguyên tắc trong chương thì không.

Câu hỏi kiểm tra hiểu. Bạn bỏ một module và mAP giảm 1,5 — nêu bốn cách giải thích khác ngoài “module đó quan trọng”. Nếu phương sai giữa seed là $\pm 0,6$ mAP, cần bao nhiêu lần chạy để tin một cải tiến 0,8 mAP, và vì sao so sánh theo cặp lại rẻ hơn cách đó? Oracle experiment nào bạn nên chạy đầu tiên cho hệ thống hiện tại của mình, và nó chặn trên cái gì?

Mini project. Lấy một repo có sẵn, *viết trước* dự đoán cho năm ablation kèm con số bạn nghĩ sẽ thấy, chạy mỗi cái ba seed, rồi đối chiếu: chỗ nào đúng, chỗ nào sai, trực giác nào cần sửa. Cột “dự đoán” là phần quan trọng nhất, vì nó là thứ duy nhất biến kết quả thành thông tin về mô hình tư duy của bạn.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
•• Vì sao ba loại câu hỏi thí nghiệm cần ba thiết kế?	Vì chúng hỏi những thứ khác nhau. “Có hoạt động không” cần kiểm soát chặt và nhiều seed; “vì sao hoạt động” cần ablation có re-tune; “hoạt động tới đâu” cần nhiều điểm dọc một trục, ít seed mỗi điểm.
••• Baseline	Phương pháp đối chứng mà kết quả mới phải vượt qua. Một baseline chưa được tinh chỉnh tử tế làm vô nghĩa mọi con số phía sau, và đó là cách tự lừa mình phổ biến nhất vì nó rẻ.
•• Thiết kế giai thừa	Chạy mọi tổ hợp bật/tắt của các thành phần thay vì đổi từng cái một. Với ba thành phần là 8 cấu hình so với 4, tức gấp đôi chi phí, đổi lại bạn thấy được toàn bộ tương tác bậc hai.
••• Leave-one-out	Bỏ từng thành phần khỏi hệ thống đầy đủ rồi đo lại. Nó trả lời “thành phần này còn đáng giữ không”, và chỉ đúng khi phần còn lại vẫn ở điểm vận hành tốt.
•• Add-one-in	Thêm từng thành phần vào baseline trần. Nó trả lời “thành phần này tự nó có giá trị không”; khi có tương tác, con số của nó tách khỏi leave-one-out, và khoảng cách ấy chính là lượng tương tác.
••• Vì sao ablation phải re-tune?	Vì bỏ một thành phần ổn định hoá thì learning rate tối ưu của phần còn lại nhỏ đi. Giữ nguyên learning rate cũ là đo “thành phần đó cộng một điểm vận hành đã sai”, và sai lệch luôn thổi phồng.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Sanity ablation	Nhóm phép thử thô: thay đầu vào bằng nhiễu, xáo trộn nhãn, thay module bằng hằng số, cắt bỏ một modality. Rẻ nhất mà phá được nhiều giả thuyết nhất, nên chạy trước mọi ablation tinh vi.
●●● Xáo trộn nhãn mà điểm đánh giá không giảm — nghĩa là gì?	Hoặc có rò rỉ dữ liệu, hoặc thành phần đang xét vô dụng. Phân biệt với hiện tượng bình thường: train loss vẫn giảm khi xáo nhãn là chuyện thường, chỉ có điểm trên tập đánh giá mới là bằng chứng.
●●● Oracle experiment	Thay một khối bằng chân trị rồi đo hệ thống tổng thể, để biết trần đóng góp của khối đó. Nếu detector hoàn hảo chỉ nâng hệ thống 2%, mọi công sức tối ưu detector đều bị chặn trên bởi 2%.
●●● Sàn nhiễu	Độ lệch chuẩn giữa các seed của <i>cùng một</i> cấu hình. Nó là đơn vị đo của mọi so sánh: chênh lệch nhỏ hơn nó thì chưa nói được gì, dù bạn lặp lại bao nhiêu lần.
●●● Cỡ mẫu tối thiểu	Số seed cần cho mỗi cấu hình, tính từ sàn nhiễu và độ lớn cải tiến muốn phát hiện: $n \approx 2(1,96 + 0,84)^2 \sigma^2 / \Delta^2$. Với $\sigma = 0,6$ và $\Delta = 0,8$ là khoảng chín seed mỗi bên.
●● So sánh theo cặp	Đánh giá hai mô hình trên đúng cùng tập test rồi lấy hiệu số trên từng mẫu. Phần phương sai do độ khó từng ảnh là chung nên biến mất, và bootstrap theo cặp cho khoảng tin cậy chặt hơn mà không tốn thêm lần huấn luyện nào.
●●● So sánh bội	Thử nhiều biến thể rồi báo cáo cái tốt nhất. Kỳ vọng của cực đại hai mươi lần rút đã cỡ 1,9 độ lệch chuẩn, nên với $\sigma = 0,6$ mAP thì “cải tiến” 1,1 điểm có thể là nhiễu thuần.
●●● Cải thiện 1,1 mAP sau khi thử hai mươi cấu hình — nghĩa là gì?	Nhiều khả năng là nhiễu. Cách phân biệt: chạy lại đúng cấu hình thắng đó với vài seed mới trên một tập giữ riêng; nếu cải tiến bốc hơi thì bạn vừa đo cỡ mẫu của cuộc tìm kiếm.
●● Rò rỉ tập test	Tập test mất dần tư cách trọng tài vì bị dùng lại hàng trăm lần, hoặc vì nó nằm sẵn trong dữ liệu pretraining. Nó không hiện ra dưới dạng phương sai giữa seed, nên chạy thêm seed không phát hiện được.
●● Vì sao chế độ tắt định vẫn không tái lập được?	Vì nguồn phi tắt định nằm trong thư viện và phần cứng: cuDNN chọn kernel khác nhau, atomics cộng không theo thứ tự, TF32 khác giữa các đời GPU. Mục tiêu đúng là tái lập phân phối, không phải từng chữ số.
●●● Hai máy khác GPU cho ATE lệch 3 cm trên cùng cấu hình — bug hay không?	So với sàn nhiễu đã đo trước đó. Nếu 3 cm nằm trong khoảng dao động giữa các seed thì đó là phi tắt định của thư viện, không phải bug. Nếu vượt hẳn, hãy nghi khác phiên bản thư viện, khác dữ liệu, hoặc một cấu hình chưa được ghi.
●● Vì sao công cụ tự động không thay được gán tay?	Vì công cụ chỉ đếm được các loại lỗi đã có tên sẵn. Gán tay là chỗ taxonomy mới sinh ra; sau đó công cụ mới đếm và ước lượng trần cải thiện cho từng loại.
●● Bốn ô hành động	Mỗi loại lỗi phải được gán vào đúng một ô: sửa bằng dữ liệu, bằng hàm mất mát, bằng kiến trúc, hoặc bằng ràng buộc hệ thống. Gán trước khi viết code, vì đó là chỗ duy nhất bạn còn quyết định bằng suy nghĩ.
●●● Kết quả âm	Ghi lại thứ đã thử và không hiệu quả, kèm tập nào, quy mô nào, mấy seed, sàn nhiễu bao nhiêu. Thiếu điều kiện thì nó thành mê tín và sẽ chặn bạn khỏi đúng cơ hội sáu tháng sau.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
dbolya/tide [66]	Phân rã lỗi detection theo loại, mỗi loại kèm phần mAP lấy lại được nếu sửa riêng nó — chính ý tưởng trần cải thiện theo loại lỗi	Kế thừa phân tích chẩn đoán của Hoiem [85]
voxel51/fiftyone	Cắt lát tập đánh giá và so hai mô hình trên cùng mẫu — hiện thực của phân tích theo lát cắt và so sánh theo cặp	Hạ tầng; bền hơn tên mô hình
Dòng ConvNeXt [121]	Cả paper là một chuỗi ablation recipe-so-với-kiến trúc có kiểm soát	Nên đọc phần ablation hơn phần kiến trúc
Reproducibility checklist (NeurIPS, ICLR)	Ép khai báo số seed, độ lệch chuẩn và ngân sách tính toán — đúng bốn yêu cầu của một bảng ablation đạt chuẩn	Thông lệ từ khoảng 2019
Benchmark harness EuRoC của chính bạn	Quy tắc ba lần chạy, chế độ lockstep để cô lập phi tắt định, các kết luận NO-GO có ghi điều kiện — nhóm C và nhóm E ở dạng thực địa	Ví dụ nội bộ; số liệu nằm trong ghi chú dự án

MINI PROJECT SLAM · Đo sần nhiễu của chính harness EuRoC của bạn · 1–2 ngày

Mục tiêu	biến câu “một lần chạy không nói lên điều gì” thành một con số cụ thể cho hệ SLAM của bạn. Từ con số đó suy ra số lần chạy tối thiểu cho mọi so sánh về sau.
Dữ liệu	hai chuỗi EuRoC có độ khó khác nhau (ví dụ một chuỗi dễ và V2_03), cấu hình mặc định, không đổi bất cứ tham số nào giữa các lần chạy.
Các bước	chạy <i>cùng một</i> cấu hình mười lần trên mỗi chuỗi; với mỗi lần ghi ATE bằng evo cùng tỉ lệ khung hình bám được; lập bảng mười dòng, tính trung bình và độ lệch chuẩn σ riêng cho từng chuỗi; rồi áp công thức cỡ mẫu $n \approx 2(z_{1-\alpha/2} + z_{1-\beta})^2 \sigma^2 / \Delta^2$ cho hai mức Δ bạn quan tâm, chẳng hạn 1 cm và 3 cm.
Xong khi	bạn phát biểu được, kèm bảng số chống lưng, câu “trên chuỗi này, chênh lệch ATE nhỏ hơn X cm giữa hai cấu hình là nhiễu”, và nói được số lần chạy tối thiểu cần cho mỗi Δ . Kiểm tra chéo: σ của chuỗi khó phải lớn hơn hẳn σ của chuỗi dễ; nếu không, harness của bạn còn một nguồn phi tắt định chưa lộ.
Nối với	B/07 cho giao thức đánh giá và cách chọn chuỗi; D/22 cho việc gắn mỗi lần chạy với một commit và một config; E/25 sẽ dùng lại chính bảng số này khi đặt ngưỡng theo chi phí.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- Vì con số bạn báo cáo là *cực đại* của hai mươi mẫu chứ không phải một mẫu, và kỳ vọng của cực đại tăng theo cỡ mẫu. Với 20 mẫu chuẩn tắc, kỳ vọng cực đại khoảng 1,9; nhân với $\sigma = 0,6$ ra $\approx 1,1$ điểm sinh ra hoàn toàn từ nhiễu.
- Với $\sigma = 0,6$ và cải tiến thật 0,8, cần khoảng chín seed mỗi bên. Ở ba seed, sai số chuẩn của hiệu số là $0,6\sqrt{2/3} \approx 0,49$, nên xác suất phát hiện chỉ khoảng 0,35 — bỏ sót chừng hai phần ba số cải tiến có thật. Ba là sần để phát hiện *sự bất định*, không phải mức đủ để *kết luận*.
- Bốn cách: chênh lệch nằm trong sần nhiễu; learning rate không được re-tune sau khi bỏ module; việc bỏ module kéo theo một thay đổi ngầm (số tham số, thời gian huấn luyện, đường dữ liệu); và module chỉ quan trọng ở *quy mô dữ liệu này*. Cách thứ hai luôn đẩy kết luận về cùng một hướng — thổi phồng — nên nó không tự triệt tiêu khi lặp lại nhiều lần.
- Không còn lý do nào: mọi công sức tối ưu detector bị chặn trên bởi 2%. Bạn biết con số ấy trước bằng một oracle experiment — thay detector bằng chân trị rồi đo hệ thống tổng thể — và đó là thí nghiệm nên chạy *đầu tiên* chứ không phải cuối cùng.
- Vì “X không hiệu quả” không cho biết điều kiện nào thay đổi thì đáng thử lại. Trong deep learning điều kiện đổi liên tục: quy mô dữ liệu, mô hình nền, ngân sách tính chính. Một NO-GO không ghi điều kiện sẽ chặn bạn khỏi đúng cơ hội về sau, trong khi không ghi gì thì ít nhất bạn còn thử lại.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn mua một cái cân trong nhà tắm. Bước lên ba lần trong cùng một phút, nó hiện ba số khác nhau, lệch nhau chừng ba lạng. Bây giờ bạn ăn kiêng một tuần rồi cân lại: nhẹ hơn hai lạng. Bạn có giảm cân thật không? Không ai trả lời được, vì hai lạng còn nhỏ hơn mức cái cân tự dao động. Câu hỏi phải hỏi trước là: cái cân này dao động bao nhiêu? Cân mười lần trong cùng một buổi sáng, ghi cả mười số lại, rồi mới biết chênh lệch bao nhiêu mới đáng nói.

Từ đó ra hai điều. Thứ nhất, muốn phát hiện một thay đổi nhỏ hơn mức dao động, bạn phải cân nhiều lần hơn — và số lần ấy tính ra được chứ không phải chọn theo thói quen. Thứ hai, nếu bạn thử hai mươi thực đơn rồi chỉ kể lại cái cho số đẹp nhất, thì phần lớn cái “đẹp nhất” ấy là do cái cân chứ không do thực đơn.

Cái giá phải trả: cân mười lần thì tốn gấp mười lần công. Và bạn phải chịu khó ghi lại cả những tuần không giảm được lạng nào, kèm điều kiện đầy đủ — ăn gì, cân lúc nào, cân nào — vì chính những đồng đó ngăn bạn thử lại đúng thực đơn ấy vào tháng sau.

Chỗ không rút gọn được: phần tính toán. Rút gọn cả chương thành “chạy nhiều lần cho chắc” nghe rất hợp lý nhưng làm mất đúng chỗ có giá trị: số lần chạy không phải một thói quen mà là kết quả của một phép tính, suy ra từ mức dao động bạn đã đo và từ độ lớn cải thiện bạn muốn phát hiện. Ba lần là thói quen; chín lần là một phép tính — và khác biệt giữa hai con số đó là bạn nói được mình đang chấp nhận bỏ sót bao nhiêu.

Một câu: Trước khi hỏi câu hình nào tốt hơn, hãy hỏi cái cân của bạn dao động bao nhiêu.

24.9 Kết nối

[B/07-chia-du-lieu-va-danh-gia](#) là tiền đề trực tiếp: cách chia dữ liệu sai thì mọi thống kê ở đây chỉ đo rất chính xác một đại lượng vô nghĩa. Phần đọc *ý nghĩa* của từng con số — precision, recall, F1, đường ROC và PR, hiệu chỉnh, và bảng chẩn đoán ngược từ triệu chứng về nguyên nhân — cũng nằm ở [B/07-chia-du-lieu-va-danh-gia](#). [E/23-doc-paper](#) là mặt đối xứng: tiêu chí bạn dùng để nghi ngờ ablation của người khác chính là tiêu chí bạn phải tự thoả. [D/22-tai-lap-va-mlops](#) là hạ tầng khiến kỷ luật này khả thi — một thay đổi bằng một config diff, mỗi lần chạy gắn với một commit và một phiên bản dữ liệu. [B/08-dich-chuyen-phan-phoi](#) là giới hạn ngoài: nó cho thấy khi nào một kết luận đúng trên tập đánh giá vẫn không chuyển được sang thực tế. Cuối cùng, [E/26-lo-trinh-hoc](#) cho biết chương này nằm ở đâu trong từng lộ trình theo mục tiêu — nó là chương trung tâm của mọi lộ trình có chữ “đánh giá” trong mục tiêu.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao sai lệch do không re-tune sau khi bỏ một thành phần lại luôn cùng một chiều, và vì sao chạy thêm seed không sửa được sai lệch đó?
2. Leave-one-out và add-one-in cho hai con số khác nhau cho cùng một thành phần. Điều đó nói lên gì về hệ thống, và bạn báo cáo con số nào?
3. Với $\sigma = 0,6$ và 20 biến thể vô dụng như nhau, “cải tiến” tốt nhất bạn kỳ vọng thấy là bao nhiêu? Con số đó so với các cải tiến thường được công bố thì thế nào?
4. Bật chế độ tắt định của framework rồi mà hai máy vẫn cho kết quả khác nhau: ba nguyên nhân là gì, và vì sao mục tiêu “tái lập từng chữ số” là mục tiêu sai?
5. Một oracle experiment cho trần lạc quan giả trong trường hợp nào — cụ thể, chuyện gì xảy ra với phân phối đầu vào của khối đứng ngay sau oracle?
6. Vì sao một kết luận NO-GO không ghi điều kiện thí nghiệm lại có hại hơn là không ghi gì?
7. Ablation giả định điều gì về cấu trúc hệ thống, và giả định đó hỏng thế nào với một hệ có vòng phản hồi?

Trực quan hoá, độ tin cậy và vận hành (Visualization, Reliability, Operations)

PHỤ THUỘC C/12-co-che-huan-luyen/01-co-hoc-cua-gradient (mọi công cụ ở nhóm A chỉ là gradient theo đầu vào thay vì theo trọng số); B/08-dich-chuyen-phan-phoi (bất định và drift là hai mặt của cùng một hiện tượng); E/24-thi-nghiem-va-ablation (kỷ luật đo đạc — chương này thêm câu hỏi “khi nào được phép tin một con số đơn lẻ”).

TÓM TẮT

Chương này trả lời câu hỏi vận hành: *khi nào tôi được phép tin đầu ra của mô hình, và làm gì khi không tin được*. Đọc xong bạn biết vì sao một bản đồ nhiệt đẹp gần như không chứng minh điều gì và phải kiểm chứng bằng can thiệp nào; bạn hiệu chỉnh được xác suất đầu ra và đặt được ngưỡng từ chối trả lời bằng chi phí lỗi thật chứ bằng cảm tính; và bạn thiết kế được đường giám sát khi không có nhãn ở môi trường triển khai. Nếu chỉ nhớ một điều: một ước lượng bất định chỉ có giá trị khi nó nổi vào một *hành động khác* — từ chối trả lời, chuyển cho người, hoặc hạ mức tự chủ; bất định không đối hành vi là bất định trang trí.

KIẾN THỨC CẦN CÓ TRƯỚC

- **logit và softmax** — logit là vector điểm số thô mà mạng xuất ra trước bước chuẩn hoá; softmax biến nó thành các số dương cộng lại bằng 1, trông như xác suất. “Trông như” là chỗ mấu chốt: tổng bằng 1 không đảm bảo con số đó phản ánh tần suất đúng thật.
- **độ tin cậy (confidence)** — giá trị softmax lớn nhất của một dự đoán, tức là mức mà mô hình “tự nhận” mình chắc chắn.
- **hiệu chỉnh (calibration)** — tính chất mong muốn rằng trong nhóm các lần mô hình nói “chắc 90%”, nó đúng khoảng 90% số lần. Đây là tính chất của *một tập* dự đoán, không phải của một dự đoán đơn lẻ, và nó tách rời với độ chính xác: một mô hình có thể chính xác cao nhưng hiệu chỉnh tệ, và ngược lại.
- **ECE** — viết tắt của *expected calibration error*. Nó chia các dự đoán thành từng khoảng theo độ tin cậy, rồi lấy trung bình khoảng cách giữa độ tin cậy và độ chính xác thật trong mỗi khoảng.
- **tập hiệu chuẩn (calibration set)** — một tập dữ liệu có nhãn, tách riêng khỏi train và test, dùng để học tham số hiệu chỉnh hoặc ngưỡng. Dùng lại tập test làm tập hiệu chuẩn là một dạng rò rỉ.
- **ensemble** — tập hợp k mô hình huấn luyện độc lập, dự đoán bằng cách lấy trung bình. Mức bất đồng giữa các thành viên là một ước lượng bất định rẻ về mặt khái niệm nhưng đắt về mặt tính toán.
- **conformal prediction** — họ phương pháp trả về một *tập* nhãn thay vì một nhãn. Tập đó chứa nhãn đúng với xác suất đặt trước, nhờ một ngưỡng lấy từ phân vị của tập hiệu chuẩn.
- **saliency map** — bản đồ nhiệt gán cho mỗi vùng ảnh một điểm “mức ảnh hưởng tới đầu ra”, thường tính từ gradient của điểm số theo pixel. Nó là một giả thuyết về nơi mô hình nhìn, không phải bằng chứng.
- **drift** — hiện tượng phân phối dữ liệu ở môi trường triển khai trôi khỏi phân phối lúc huấn luyện. Phân biệt hai loại: *dịch chuyển đầu vào* (ảnh đối, quan hệ ảnh-nhãn giữ nguyên) và *dịch chuyển khái niệm* (ảnh trông như cũ nhưng nhãn đúng đã đổi nghĩa).

CÂU HỎI MỞ ĐẦU

- Q1. Grad-CAM chỉ đúng vào con vật trong ảnh. Vì sao điều đó vẫn chưa chứng minh được rằng mô hình nhìn vào con vật — và thí nghiệm một dòng nào bác bỏ được nó?
- Q2. Một mô hình đạt 94% chính xác trong khi độ tin cậy trung bình là 0,99. Con số nào sai, và sửa nó có làm mô hình chính xác hơn không?
- Q3. Nếu bỏ sót đất gấp một trăm lần báo động giả thì ngưỡng tối ưu là 0,01. Vì sao chính cái ngưỡng “đúng” ấy lại có thể khiến hệ thống không triển khai được?
- Q4. Một phương pháp hứa “tập dự đoán chứa nhãn đúng 90% số lần”. Vì sao lời hứa đó vẫn tương thích với việc sai 40% ở đúng lớp bạn quan tâm nhất?

25.1 Đặt vấn đề

Một mô hình phân loại luôn trả về một câu trả lời. Đó là thiết kế, không phải lỗi: hàm *softmax* chuẩn hoá cho tổng bằng một, nên kể cả với một ảnh không thuộc bất kỳ lớp nào nó vẫn cho ra một lớp cùng một con số trông giống xác suất. Với một hệ thống chạy trong phòng thí nghiệm, điều này vô hại vì mọi ảnh đều đến từ tập test. Với một hệ thống chạy thật, nó là chế độ hỏng nguy hiểm nhất, bởi hệ thống sai mà không phát tín hiệu nào cho biết nó đang sai.

Chỗ nổi với nền sẵn có của bạn nằm ngay đây. Một pipeline SLAM không hỏi “đúng bao nhiêu phần trăm”. Nó hỏi “khi nào tôi biết mình đang trôi”. Cả kiến trúc phòng vệ của nó tồn tại để trả lời câu ấy: số inlier, ngưỡng chi-square trên phần dư, cờ mất bám, chế độ định vị lại. Deep learning cho thị giác mang văn hoá ngược lại: tối ưu một con số tổng trên một tập test cố định, hầu như không có cơ chế tự báo sai. *Học xong chương này bạn làm được ba việc mà trước đó không làm được*: đọc một bản đồ nhiệt mà không rút ra kết luận sai; biến điểm số đầu ra thành xác suất dùng được cho quyết định có chi phí bất đối xứng; và viết được tài liệu chế độ hỏng cho một hệ thống thật.

TRỰC GIÁC

Một mạng chưa hiệu chỉnh giống một đồng hồ đo áp suất mà kim đã bị lệch cỡ: nó vẫn chuyển động đúng chiều khi áp suất thay đổi, nên vẫn dùng để *so sánh* được, nhưng con số đọc trên mặt đồng hồ không dùng để *quyết định* được. Hiệu chỉnh là chỉnh lại cỡ; nó không làm cảm biến nhạy hơn, chỉ làm con số trở nên có nghĩa. Và như mọi lần chỉnh cỡ, nó chỉ đúng trong dải điều kiện mà bạn đã chỉnh — ra ngoài dải đó, kim lại lệch.

25.2 Nhìn vào bên trong mô hình (Looking Inside the Model)

Cả nhóm công cụ này chia sẻ đúng một kỹ thuật: lấy *gradient* theo *đầu vào* thay vì theo trọng số. Khi huấn luyện, ta hỏi “đổi trọng số thế nào để mất mát giảm”; khi trực quan hoá, ta hỏi “đổi pixel nào thì điểm số của lớp này đổi nhiều nhất”. Cùng một phép tính, khác biến được coi là tự do — và nhận ra điều đó thì cả nhóm A trở thành một ý tưởng duy nhất thay vì một danh sách kỹ thuật rời rạc.

A. Bốn tầng công cụ

- **Nhìn thẳng trọng số** — filter tầng đầu của một CNN hiện ra dưới dạng bộ lọc biên và đốm màu. Chỉ tầng đầu đọc được bằng mắt; từ tầng hai trở đi, trọng số không còn sống trong không gian ảnh.

- **Chiều *embedding*** — *t-SNE* [6] và *UMAP* [7] đưa đặc trưng nhiều chiều xuống hai chiều để nhìn cấu trúc cụm.
- **Tối ưu trên pixel** — từ một ảnh nhiễu, đi lên theo gradient của điểm số một lớp, để xem mô hình “muốn thấy” gì ở lớp đó.
- **Quy gán** — *saliency map*, *guided backpropagation*, và *Grad-CAM* [207] trả lời câu “mô hình đang nhìn vào đâu” bằng cách gán điểm cho từng vùng ảnh.

B. Vì sao một bản đồ nhiệt đẹp gần như không chứng minh gì

Đây là chỗ cần nói thẳng là phức tạp chứ không đơn giản hoá. Kết quả đã được công bố và có tính quyết định: một số phương pháp quy gán vẫn cho ra bản đồ trông rất hợp lý ngay cả khi trọng số của mô hình bị ngẫu nhiên hoá, hoặc khi mô hình được huấn luyện trên nhãn xáo trộn [160]. Nếu bản đồ gần như không đổi khi mô hình đã bị phá hỏng, thì bản đồ đó phản ánh cấu trúc của *ảnh đầu vào* và của chính thuật toán quy gán, chứ không phản ánh cái mô hình đã học. Đó là lý do phép thử tính tảo phải chạy trước khi tin bất kỳ bản đồ nào: so bản đồ của mô hình đã huấn luyện với bản đồ của mô hình khởi tạo ngẫu nhiên, và nếu hai bản đồ giống nhau thì phương pháp đó không nói gì về mô hình của bạn.

Cùng lỗi diễn giải xảy ra với phép chiếu hai chiều: trong một bản đồ t-SNE, khoảng cách giữa các cụm và kích thước cụm gần như không mang thông tin, và cấu trúc nhìn thấy phụ thuộc mạnh vào tham số *perplexity*. Kết luận đúng cho cả nhóm: các công cụ này dùng để *đặt giả thuyết*, không đủ để *kết luận*.

C. Che một vùng rồi đo lại

Cách biến một giả thuyết thành bằng chứng là can thiệp thật rồi đo lại. Đó đúng là một ablation như ở chương 24, chỉ khác chỗ bạn cắt bỏ trên *đầu vào* chứ không trên kiến trúc. Nếu bản đồ nhiệt nói mô hình dựa vào một vùng, hãy che vùng đó đi và đo điểm số tụt bao nhiêu; hãy che một vùng ngẫu nhiên cùng diện tích để có nhóm đối chứng; và hãy làm trên đủ mẫu để so được với sàn nhiễu. Đây là chuyển từ quan sát tương quan sang thí nghiệm can thiệp, và nó là ranh giới giữa một hình đẹp trong slide và một kết luận dùng được.

↪ **trường hợp riêng của E/24** — đây đúng là một ablation, chỉ khác là ablate trên đầu vào thay vì trên kiến trúc — nên nó cũng cần nhóm đối chứng và cũng phải vượt sàn nhiễu

Bảng 25.1. Bốn công cụ nhìn vào bên trong mô hình. Cột cuối là cột quyết định: nó cho biết kết luận nào *không* được phép rút ra.

Công cụ	Cơ chế	Nói được gì	Hỏng khi nào
Filter tầng đầu	Nhìn thẳng trọng số	Mô hình có học được bộ lọc biên/màu hợp lý không	Chỉ đọc được ở tầng đầu; im lặng về mọi thứ sâu hơn
t-SNE / UMAP	Bảo toàn lân cận cục bộ	Có tồn tại cấu trúc cụm trong không gian đặc trưng không	Khoảng cách và kích thước cụm vô nghĩa; đổi perplexity đổi câu chuyện
Grad-CAM	Gradient theo đặc trưng, gộp theo kênh	Một giả thuyết về vùng ảnh có ảnh hưởng	Vẫn đẹp với trọng số ngẫu nhiên [160]; độ phân giải thô
Che vùng + đo lại	Can thiệp thật rồi đo	Bằng chứng nhân quả trong phạm vi mô hình đó	Che tạo ảnh ngoài phân phối; cần nhóm đối chứng che ngẫu nhiên

I 25.3 Bất định và hiệu chỉnh (Uncertainty and Calibration)

A. Hai loại bất định, và vì sao phải phân biệt

Loại thứ nhất, tiếng Anh gọi là *aleatoric*, là phần mơ hồ nằm trong chính dữ liệu: hai chuyên gia nhìn cùng một ảnh X-quang và bất đồng, hay một vật thể bị che quá nửa. Thu thêm dữ liệu cùng loại không làm nó giảm. Loại thứ hai, tiếng Anh gọi là *epistemic*, là phần do mô hình chưa từng thấy vùng đầu vào này; nó *giảm* khi có thêm dữ liệu đúng chỗ. Phân biệt hai loại là chuyện ngân sách, không phải chuyện học thuật. Nếu bất định ở một nhóm mẫu thuộc loại thứ nhất thì gán nhãn thêm mười nghìn ảnh không đổi được gì. Tiền nên đi chỗ khác: đổi cảm biến, siết lại định nghĩa lớp, hoặc chấp nhận nó và thiết kế đường từ chối trả lời.

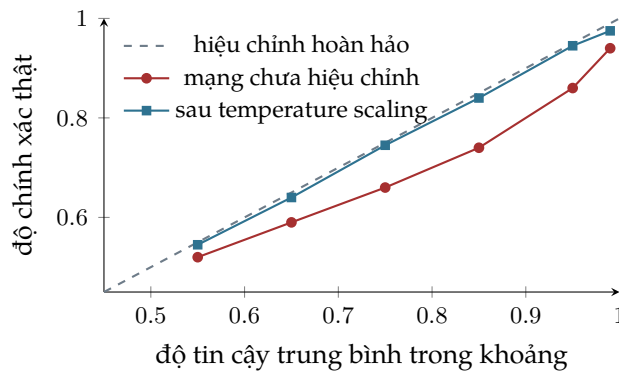
↔ **tiền đề B/06** — mức đồng thuận giữa người gán nhãn là thước đo trực tiếp của bất định ngẫu nhiên: nó chặn trên mọi mô hình học từ tập nhãn đó

B. Hiệu chỉnh: mạng hiện đại tự tin quá mức

Kết quả đã được đo và lặp lại nhiều lần: các mạng sâu hiện đại cho xác suất cao hơn hẳn độ chính xác thật, và hiện tượng này nặng hơn ở mạng lớn hơn [24]. Cách đo là *ECE*, viết tắt của *expected calibration error*: chia các dự đoán thành các khoảng theo độ tin cậy, rồi lấy trung bình có trọng số của $|\text{độ chính xác} - \text{độ tin cậy}|$ trên từng khoảng,

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)|.$$

Nói bằng lời: trong nhóm những lần mô hình nói “tôi chắc 90%”, nó có đúng khoảng 90% số lần không — ECE đo khoảng cách trung bình giữa lời nói và sự thật. Ví dụ tính tay: một mô hình đạt 94% chính xác với độ tin cậy trung bình 0,99 thì riêng khoảng tin cậy cao nhất đã đóng góp $|0,99 - 0,94| = 0,05$; mô hình này “nói quá” năm điểm phần trăm. Hình 25.1 vẽ chính tình huống đó.



Hình 25.1. Biểu đồ độ tin cậy (*reliability diagram*). Đường chấm là chuẩn lý tưởng; đường đỏ nằm dưới nó có nghĩa mô hình nói chắc hơn thực tế ở mọi mức. Temperature scaling kéo đường đỏ về sát đường chuẩn mà *không* đổi thứ hạng giữa các lớp — nên độ chính xác giữ nguyên, chỉ con số xác suất trở nên đúng được. Các điểm ở đây là minh hoạ theo đúng hình dạng đã báo cáo trong tài liệu [24], không phải số đo của một mô hình cụ thể.

C. Bốn cách lấy bất định, và giá của mỗi cách

Bảng 25.2. Bốn công cụ ước lượng bất định. Không cái nào chiếm ưu thế tuyệt đối; chọn theo ngân sách suy luận và theo việc bạn cần bảo đảm loại nào.

Phương pháp	Cơ chế	Chi phí	Hỏng khi nào
Temperature scaling	Chia logit cho một hằng số học trên tập validation	Gần như bằng không	Chỉ sửa hiệu chỉnh tổng thể, không phát hiện mẫu lạ; hỏng khi phân phối dịch chuyển [96]
MC dropout	Lấy mẫu nhiều lần với dropout bật lúc suy luận [208]	k lần forward	Ước lượng phụ thuộc chỗ đặt dropout; thường hẹp hơn bất định thật
Ensemble	Huấn luyện k mô hình độc lập rồi lấy trung bình [209]	k lần huấn luyện và suy luận	Đắt nhất; các thành viên vẫn chia sẻ thiên lệch của dữ liệu
Conformal prediction	Dùng phân vị của điểm bất phù hợp trên tập hiệu chuẩn [25]	Một tập hiệu chuẩn	Bảo đảm là <i>biên</i> , không theo từng lớp; cần tính khả hoán, hỏng dưới dịch chuyển

Trong bốn cách trên, conformal prediction đáng dừng lại vì nó là cách duy nhất cho một *bảo đảm* chứ không chỉ một ước lượng. Ý tưởng rút gọn tới mức đơn giản nhất gồm ba bước. Lấy một tập hiệu chuẩn chưa từng dùng. Với mỗi mẫu, tính một điểm “bất phù hợp”, chẳng hạn $1 - \hat{p}_y$ với $\hat{p}_y$ là xác suất mô hình gán cho nhãn đúng. Rồi lấy phân vị của tập điểm đó làm ngưỡng. Với $n = 1000$ mẫu hiệu chuẩn và mức sai cho phép $\alpha = 0,1$, ngưỡng là điểm lớn thứ $\lceil (n+1)(1-\alpha) \rceil = \lceil 900,9 \rceil = 901$. Khi dự đoán, ta trả về *tập* mọi nhãn có điểm bất phù hợp dưới ngưỡng đó, và tập ấy chứa nhãn đúng với xác suất ít nhất 90%. Điều cần hiểu cho đúng là bảo đảm này mang tính biên: nó đúng trên trung bình toàn bộ phân phối, nên hoàn toàn có thể phủ 99% ở các lớp dễ và 60% ở một lớp khó. Và nó đòi hỏi tính khả hoán giữa tập hiệu chuẩn và dữ liệu triển khai — một giả định mà dữ liệu chuỗi thời gian từ robot vi phạm gần như ngay lập tức, vì các khung hình liên tiếp tương quan mạnh với nhau.

D. Hai hiện tượng nổi vào đây

- **Chưa xác định đủ (*underspecification*)** — nhiều mô hình có cùng điểm test nhưng hành xử rất khác ngoài đời, vì tập test không ràng buộc đủ để chọn ra một trong số chúng. Đây chính là mặt thực nghiệm của điều bạn đã thấy ở chương 24: điểm số bằng nhau không có nghĩa hai hệ thống tương đương.
- **Mẫu đối kháng và ảnh đánh lừa** — vẫn là gradient theo pixel, nhưng đi ngược chiều để phá. Sự tồn tại của chúng nói một điều rất cụ thể: độ tin cậy cao tại một điểm không suy ra độ tin cậy ở lân cận điểm đó. Vì thế phép đo độ bền dưới nhiễu chuẩn hoá [37] phải nằm trong bộ đo mặc định, không phải một thí nghiệm thêm thắt.

CẠM BÃY

Hai cách hiểu sai hay gặp nhất. *Một*, ECE thấp không có nghĩa từng dự đoán đáng tin: một mô hình luôn trả về 0,5 trên một bài toán hai lớp cân bằng có ECE bằng không trong khi không phân biệt được gì cả — hiệu chỉnh là tính chất của *tập* dự đoán, không phải của một dự đoán. *Hai*, ECE phụ thuộc cách chia khoảng: đổi số khoảng thì con số đổi, nên chỉ so ECE giữa hai mô hình khi dùng cùng cách chia. Cả hai đều dẫn tới cùng một kết luận thực hành: đọc biểu đồ độ tin cậy, đừng chỉ đọc một con số ECE.

25.4 Vận hành (Operations)

A. Ngưỡng quyết định đến từ chi phí, không từ cảm tính

Đây là chỗ mọi thứ ở trên biến thành hành động. Với một quyết định nhị phân, ngưỡng tối ưu theo chi phí kỳ vọng là

$$p^* = \frac{c_{FP}}{c_{FP} + c_{FN}},$$

tức là bạn nên báo động khi xác suất hậu nghiệm vượt tỉ lệ giữa chi phí báo động giả và tổng hai chi phí. Câu diễn giải: ngưỡng không phải một siêu tham số để dò, nó là *tỉ số chi phí viết dưới dạng xác suất*.

→ cùng nguyên lý với B/09 — trọng số lớp trong hàm mất mát và ngưỡng quyết định là hai chỗ khác nhau cùng mã hoá một tỉ số chi phí; đặt ở tầng nào là một lựa chọn thiết kế

Ví dụ tính tay: nếu bỏ sót đất gấp một trăm lần báo động giả thì $p^* = 1/101 \approx 0,01$ — ngưỡng thấp đến mức phản trực giác. Nhưng hãy đi tiếp một bước nữa, vì đây là chỗ con số tổng nói dối. Giả sử tỉ lệ dương thật là 1% và ở ngưỡng đó mô hình bắt được 95% ca dương với tỉ lệ báo động giả 10%. Trên mười nghìn mẫu: 100 ca dương thật, bắt được 95; 9900 ca âm, báo nhầm 990. Tức là trong 1085 lần báo động chỉ có 95 lần đúng, độ chính xác của cảnh báo dưới 9%. Ngưỡng ấy *vẫn* là ngưỡng đúng theo chi phí đã khai báo, nhưng nó chỉ khả thi nếu hệ thống có năng lực xử lý hơn một nghìn cảnh báo trên mười nghìn mẫu. Nếu không có năng lực đó thì bài toán thật không phải bài toán ngưỡng mà là bài toán chi phí đã khai báo sai, hoặc bài toán cần một tầng lọc thứ hai.

B. Từ chối trả lời và con người trong vòng lặp

Hệ quả trực tiếp là mọi hệ thống nghiêm túc cần một **đường từ chối trả lời**, và thiết kế nó là phần quan trọng nhất mà ít người học. Ba câu hỏi phải trả lời trước khi triển khai:

1. **Khi nào từ chối?** Một ngưỡng trên xác suất đã hiệu chỉnh, hoặc kích thước tập conformal vượt một giá trị, hoặc một tín hiệu ngoài phân phối [95]. Ngưỡng này phải chọn trên tập validation và phải được báo cáo kèm tỉ lệ từ chối, vì một hệ thống từ chối 40% số ca thì độ chính xác trên phần còn lại là con số vô nghĩa nếu đọc tách rời.
2. **Từ chối rồi thì ai làm tiếp?** Người, một mô hình đắt hơn, hay một hành vi an toàn mặc định. Nếu không có ai làm tiếp thì việc từ chối chỉ chuyển lỗi thành sự im lặng — tệ hơn, vì im lặng không được đếm vào bất kỳ metric nào.
3. **Hành vi an toàn là gì?** Với robot, câu trả lời thường là hạ mức tự chủ: giảm tốc, dừng, hoặc chuyển sang một bộ ước lượng bảo thủ hơn. Bạn đã có sẵn khuôn mẫu này từ SLAM — mất bám thì chuyển sang định vị lại chứ không tiếp tục tích lũy sai số — và nó chuyển sang phía deep learning gần như nguyên vẹn.

→ cùng nguyên lý với C/16 — hạ mức tự chủ khi mất bám trong SLAM và từ chối trả lời khi bất định cao là cùng một mẫu thiết kế: đổi hành vi thay vì đổi ước lượng

C. Giám sát khi không có nhãn

Ở môi trường triển khai bạn gần như không bao giờ có nhãn ngay, nên không đo được độ chính xác. Thứ đo được là các tín hiệu thay thế, chia làm hai nhóm theo giá. Nhóm rẻ tính ngay từ một lần chạy: phân phối độ tin cậy đầu ra, thống kê đặc trưng ở một tầng cố định so với tập tham chiếu, và tỉ lệ từ chối trả lời. Nhóm đắt hơn nhưng nhạy hơn cần thêm hạ tầng: mức bất đồng giữa hai mô hình chạy song song, và các tín hiệu hạ nguồn như người dùng sửa lại kết quả hay robot phải dừng khẩn. Các tín hiệu này phát hiện được *dịch chuyển đầu vào* nhưng gần như mù

với *dịch chuyển khái niệm*. Lý do nằm ở chỗ: khi ảnh trông y hệt mà nhãn đúng đã đổi nghĩa, phân phối đầu vào không nhúc nhích, nên mọi tín hiệu không nhãn đều im lặng. Cách duy nhất là gán nhãn một mẫu nhỏ định kỳ; kinh nghiệm thực hành là ngân sách gán nhãn định kỳ luôn rẻ hơn cái giá phát hiện muộn, dù không có con số phổ quát nào cho “rẻ hơn bao nhiêu”. Vòng khép kín từ lỗi thu được ở production quay về tập dữ liệu chính là chỗ phần E nổi ngược về [B/06-chat-luong-du-lieu](#).

D. Ràng buộc pháp lý là ràng buộc kỹ thuật

Với ảnh mặt người, biển số hay dữ liệu y tế, các ràng buộc về quyền riêng tư quyết định kiến trúc chứ không chỉ quyết định thủ tục giấy tờ: dữ liệu lưu ở đâu, giữ bao lâu, có ra khỏi thiết bị không, có phải xoá theo yêu cầu không. Gắn chúng vào sau khi thiết kế xong thường buộc phải viết lại đúng hai phần đắt nhất — thu thập dữ liệu và giám sát. Kèm theo là nghĩa vụ tài liệu: ghi rõ mô hình được huấn luyện trên dữ liệu nào và dự định dùng trong phạm vi nào [63].

I 25.5 Giới hạn và trường hợp hỏng

Giới hạn lớn nhất của cả chương: mọi thứ ở đây được hiệu chỉnh và ngưỡng hoá trên một tập hiệu chuẩn, nên chúng chỉ đúng chừng nào dữ liệu triển khai còn giống tập đó. Dưới dịch chuyển phân phối, chất lượng hiệu chỉnh xuống cấp và các phương pháp ước lượng bất định xuống cấp không đồng đều — ensemble thường bền hơn temperature scaling, nhưng không cái nào giữ được bảo đảm ban đầu [96]. Nói cách khác, các công cụ ở chương này quản trị được *bất định đã biết dạng*, không quản trị được cái chưa từng thấy.

Giới hạn thứ hai nằm ở phía diễn giải. Công cụ trực quan hoá không có tiêu chuẩn đúng/sai khách quan, nên rất khó chứng minh một bản đồ nhiệt là sai. Vì thế chúng phải luôn đi kèm can thiệp và đo lại; một bản đồ đứng một mình không được phép là bằng chứng. Giới hạn thứ ba mang tính tổ chức: đường từ chối trả lời chỉ hoạt động nếu thật sự có người và quy trình đón phân bị từ chối. Cách phát hiện sớm là tính trước số ca đẩy sang người mỗi ngày — đúng phép nhân ở mục A — rồi hỏi bộ phận đó xem con số ấy có khả thi không.

CẦN NHỚ

Một ước lượng bất định chỉ có giá trị khi nó nổi vào một *hành động khác*: từ chối trả lời, chuyển cho người, hoặc hạ mức tự chủ. Nếu hệ thống làm y hệt nhau dù bất định cao hay thấp thì bạn đã tổn công tính một con số trang trí. [T2]

I 25.6 Thực hành

Repo và SOTA. [pytorch/captum](#) (bộ quy gán đầy đủ, có sẵn cả phép kiểm tra tính tảo), [jacobgil/pytorch-grad-cam](#), [lutzroeder/netron](#) để đọc đồ thị mô hình, TensorBoard embedding projector, [ENSTA-U2IS-AI/torch-uncertainty](#), [gpleiss/temperature_scaling](#), [Trusted-AI/adversarial-robustness-toolbox](#) và [bethgelab/foolbox](#), [evidentlyai/evidently](#) cho giám sát drift. Danh sách này chốt tại tháng 9/2026 và sẽ cũ trong sáu đến mười hai tháng; các thư viện hạ tầng bền hơn nhiều so với tên mô hình dẫn đầu bảng.

Câu hỏi kiểm tra hiểu. Mô hình đạt 94% chính xác với độ tin cậy trung bình 0,99 — điều gì sai, đo bằng chỉ số nào, và sửa bằng cách rẻ nhất nào? Grad-CAM chỉ đúng vào vật thể: điều đó chứng minh được gì và *không* chứng minh được gì? Bất định ngẫu nhiên cao ở một nhóm mẫu

— thu thêm dữ liệu có giúp không, và nếu không thì tiêu tiền vào đâu? Vì sao một tập dữ liệu conformal phủ 90% vẫn có thể phủ chỉ 60% ở lớp bạn quan tâm nhất?

Mini project. Chạy phép kiểm tra tính tảo của Adebayo và cộng sự trên một phương pháp quy gán: so bản đồ của mô hình đã huấn luyện với bản đồ của mô hình khởi tạo ngẫu nhiên trên cùng ảnh. Nếu hai bản đồ giống nhau thì ghi lại kết luận đó như một kết quả âm theo đúng chuẩn ở chương 24.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●● Saliency map	Bản đồ nhiệt gán cho mỗi vùng ảnh một điểm “mức ảnh hưởng tới đầu ra”, tính từ gradient của điểm số theo pixel. Nó dùng để <i>đặt giả thuyết</i> ; nó không đủ để kết luận.
●●● Vì sao bản đồ nhiệt đẹp không chứng minh gì?	Vì nhiều phương pháp quy gán vẫn cho bản đồ trông hợp lý ngay cả khi trọng số bị ngẫu nhiên hoá. Khi đó bản đồ phản ánh cấu trúc của ảnh và của chính thuật toán, không phản ánh cái mô hình đã học.
●●● Che vùng, đo lại	Phép kiểm chứng thật: che vùng mà bản đồ chỉ vào, đo điểm số tụt bao nhiêu, và so với nhóm đối chứng che một vùng ngẫu nhiên cùng diện tích. Đây là chỗ biến quan sát thành bằng chứng.
● t-SNE	Phép chiếu đặc trưng nhiều chiều xuống hai chiều, bảo toàn lân cận cục bộ. Khoảng cách giữa các cụm và kích thước cụm gần như không mang thông tin, và hình vẽ đổi theo tham số perplexity.
●●● Hiệu chỉnh (calibration)	Tính chất rằng trong nhóm những lần mô hình nói “chắc 90%”, nó đúng khoảng 90% số lần. Đây là tính chất của <i>một tập</i> dữ liệu đoán, và nó tách rời hoàn toàn với độ chính xác.
●● ECE	Chia dự đoán thành các khoảng theo độ tin cậy, rồi lấy trung bình có trọng số của khoảng cách giữa độ tin cậy và độ chính xác thật. Nó phụ thuộc cách chia khoảng, nên chỉ so được giữa hai mô hình khi dùng cùng cách chia.
●●● Accuracy 94% mà độ tin cậy trung bình 0,99 — sai ở đâu?	Mô hình tự tin quá mức chừng năm điểm phần trăm. Đo bằng ECE và đọc biểu đồ độ tin cậy; sửa rõ nhất bằng temperature scaling, và độ chính xác sẽ không đổi.
●● Temperature scaling	Chia logit cho một hằng số học trên tập hiệu chuẩn. Nó không đổi thứ hạng giữa các lớp nên độ chính xác giữ nguyên; nó cũng không phát hiện mẫu lạ và mất hiệu lực khi phân phối dịch chuyển.
●● Vì sao thêm dữ liệu chỉ giảm được một loại bất định?	Vì chỉ loại do mô hình chưa từng thấy vùng đầu vào đó mới giảm khi có thêm mẫu ở đúng chỗ. Phần mơ hồ nằm trong chính dữ liệu thì không: hai chuyên gia vẫn bất đồng trên cùng tấm ảnh.
●● Ensemble	Huấn luyện k mô hình độc lập rồi lấy trung bình; mức bất đồng giữa chúng là ước lượng bất định. Đắt gấp k lần cả khi huấn luyện lẫn suy luận, nhưng bền nhất dưới dịch chuyển phân phối.
●● Conformal prediction	Trả về một <i>tập</i> nhãn kèm bảo đảm phủ, lấy ngưỡng từ phân vị của tập hiệu chuẩn. Với 1000 mẫu và mức sai 0,1, ngưỡng là điểm bất phù hợp lớn thứ 901.
●●● Tập conformal phủ 90% nhưng lớp bạn cần chỉ phủ 60% — vì sao?	Vì bảo đảm là <i>biên</i> : nó đúng trên trung bình toàn phân phối, nên có thể phủ dư ở lớp dễ và thiếu ở lớp khó. Muốn bảo đảm theo lớp thì phải hiệu chuẩn riêng từng lớp và trả giá bằng cỡ tập hiệu chuẩn.
●●● Ngưỡng theo chi phí	Ngưỡng báo dương bằng chi phí báo động giả chia cho tổng hai chi phí. Nó là tỉ số chi phí viết dưới dạng xác suất, nên không phải một siêu tham số để dò: bỏ sót đắt gấp trăm lần thì ngưỡng khoảng 0,01.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Vì sao ngưỡng đúng vẫn không triển khai được?	Vì tỉ lệ dương thật thấp làm số báo động giả áp đảo. Với 1% dương thật, độ nhạy 95% và tỉ lệ báo động giả 10%, mười nghìn mẫu sinh 1085 cảnh báo mà chỉ 95 cái đúng.
●●● Từ chối trả lời	Nhánh xử lý riêng cho các ca mô hình không đủ chắc, kèm ngưỡng chọn trên tập validation. Nó chỉ hoạt động khi có người hoặc một hành vi an toàn đón phần bị từ chối; không thì nó chỉ đổi lỗi thành im lặng.
●● Drift	Phân phối dữ liệu ở môi trường triển khai trôi khỏi phân phối huấn luyện. Chia hai loại: dịch chuyển đầu vào, khi ảnh đổi; và dịch chuyển khái niệm, khi ảnh trông y hệt nhưng nhãn đúng đã đổi nghĩa.
●●● Độ chính xác ở production tụt nhưng mọi tín hiệu giám sát vẫn bình thường — nghĩ gì?	Hai khả năng: dịch chuyển khái niệm, hoặc một thay đổi ở khâu tiền xử lý mà tín hiệu đang theo dõi không chạm tới. Phân biệt bằng cách gán nhãn một mẫu nhỏ và dựng lại đường dữ liệu từ cảm biến tới tensor đầu vào.
●● Vì sao giám sát không nhãn mù với dịch chuyển khái niệm?	Vì mọi tín hiệu thay thế đều đọc từ đầu vào hoặc từ đầu ra của mô hình. Khi ảnh không đổi mà chỉ nhãn đúng đổi nghĩa, phân phối đầu vào không nhúc nhích, nên phải gán nhãn một mẫu nhỏ định kỳ.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
pytorch/captum	Cài đặt gần như toàn bộ họ quy gán, và quan trọng hơn, cả các phép kiểm tra tinh tế đi kèm	Hạ tầng; là nơi kết luận của Adebayo [160] được đóng gói thành công cụ
Temperature scaling [24]	Bước hậu xử lý một tham số, nay là mặc định trong nhiều hệ thống có ngưỡng quyết định	Rẻ tới mức gần như không có lý do bỏ qua
Deep ensembles [209]	Ước lượng bất định bằng mức bất đồng giữa các mô hình độc lập	Vẫn là chuẩn so sánh mạnh nhất dưới dịch chuyển phân phối [96]
Conformal prediction [25]	Bảo đảm phủ dựa trên phân vị của tập hiệu chuẩn, dùng cho các hệ có ngưỡng và có đường từ chối	Đang lan sang y tế và dự báo; mảng còn mở rộng nhanh
Model cards [63]	Ghi rõ phạm vi sử dụng dự kiến và dữ liệu huấn luyện — phần tài liệu của mục vận hành	Nay là thông lệ ở các nhà phát hành mô hình lớn
evidentlyai/evidently	Giám sát drift bằng các tín hiệu thay thế khi không có nhãn ở production	Hạ tầng; nổi trực tiếp vào vòng lỗi → dữ liệu

MINI PROJECT SLAM · Hiệu chỉnh bộ phân loại đóng vòng và đặt ngưỡng theo chi phí · 1–2 ngày

Mục tiêu	biến điểm số thô của tăng nhận diện lại vị trí (<i>loop closure</i>) thành một xác suất dùng được. Sau đó đặt ngưỡng chấp nhận vòng lặp bằng chi phí lỗi, không bằng cảm tính. Chi phí ở đây lệch hẳn về một phía: một vòng lặp sai kéo cả bản đồ về sai vị trí và hỏng vĩnh viễn, còn một vòng lặp bỏ lỡ chỉ để lại phần trôi vốn đã có.
Dữ liệu	các ứng viên vòng lặp sinh ra khi chạy hệ của bạn trên một chuỗi EuRoC, gán nhãn đúng/sai bằng chân trị tư thế (hai khung hình cách nhau dưới một ngưỡng khoảng cách và góc thì là vòng lặp thật). Vài nghìn ứng viên là đủ; tách một nửa làm tập hiệu chuẩn.
Các bước	vẽ biểu đồ độ tin cậy cho điểm số hiện tại trên tập hiệu chuẩn; học một tham số nhiệt độ rồi vẽ lại để thấy đường cong về sát đường chuẩn; khai báo tỉ số chi phí giữa một

- vòng lặp sai và một vòng lặp bỏ lỡ, tính $p^* = c_{FP} / (c_{FP} + c_{FN})$; đếm số cảnh báo giả ở ngưỡng đó.
- Xong khi** bạn có hai biểu đồ độ tin cậy, một trước và một sau khi hiệu chỉnh. Bạn có một ngưỡng suy ra từ chi phí chứ không chọn tay. Và bạn viết được câu này: “ở ngưỡng đó, trên mười nghìn ứng viên có k vòng lặp sai lọt qua và m vòng lặp thật bị bỏ”, kèm nhận xét hệ thống có chịu nổi k hay không.
- Nối với** E/24 cho việc kiểm tra rằng cải thiện sau hiệu chỉnh vượt sản nhiều bạn đã đo ở mini project chương trước; B/09 cho quan hệ giữa ngưỡng quyết định và trọng số lớp trong hàm mất mát.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì bản đồ có thể gần như không đổi khi trọng số bị ngẫu nhiên hoá, tức là nó phản ánh cấu trúc ảnh và thuật toán quy gán chứ không phản ánh cái mô hình đã học [160]. Thí nghiệm bác bỏ: vẽ cùng bản đồ cho một mô hình khởi tạo ngẫu nhiên và so hai hình.
- A2. Con số sai là 0,99: mô hình nói chắc hơn thực tế khoảng năm điểm phần trăm. Sửa bằng temperature scaling thì *không* làm mô hình chính xác hơn — nó không đổi thứ hạng giữa các lớp — nhưng làm con số xác suất trở nên dùng được để ra quyết định.
- A3. Vì ngưỡng thấp cộng với tỉ lệ dương thật thấp sinh ra rất nhiều báo động giả: với 1% dương thật, độ nhạy 95% và tỉ lệ báo động giả 10%, trên mười nghìn mẫu có 1085 cảnh báo mà chỉ 95 đúng. Ngưỡng vẫn đúng theo chi phí đã khai báo; cái thiếu là năng lực xử lý ngàn ấy cảnh báo.
- A4. Vì bảo đảm của conformal prediction là bảo đảm *biên*: nó đúng trên trung bình toàn bộ phân phối, nên hoàn toàn có thể phủ 99% ở các lớp dễ và 60% ở một lớp khó. Muốn bảo đảm theo từng lớp thì phải dùng biến thể hiệu chuẩn riêng cho từng lớp, và trả giá bằng cỡ tập hiệu chuẩn.

NÓI LẠI THẬT ĐƠN GIẢN

Trong bếp nhà bạn có một cái báo khói, có núm vặn độ nhạy và một dây đèn từ một tới mười cho biết nó “chắc” tới đâu. Cả chương này nói về đúng ba chuyện xảy ra với cái máy ấy.

Chuyện thứ nhất: dây đèn nói dối. Chưa ai từng ngồi đếm xem trong một trăm lần nó bật tới đèn số chín thì thật sự có cháy bao nhiêu lần. Muốn con số ấy dùng được để quyết định, phải đếm rồi chỉnh lại thang đèn cho khớp. Chỉnh xong, cái máy không nhạy hơn chút nào; chỉ có con số trên đèn trở nên đọc được.

Chuyện thứ hai: vặn núm độ nhạy tới đâu không phải chuyện sở thích. Nếu bỏ sót một đám cháy tệ hơn nhiều so với việc nó kêu oan lúc bạn rán trứng, thì vặn rất nhạy là đúng — nhưng phải đếm trước xem mỗi tuần nó sẽ kêu oan bao nhiêu lần, và nhà bạn chịu nổi bấy nhiêu lần không. Máy kêu mà không ai chạy tới thì vặn kiểu gì cũng vô nghĩa.

Chuyện thứ ba: cái đèn chỉ về phía bếp không chứng minh khói đến từ bếp. Muốn biết, đóng cửa bếp lại rồi xem nó còn kêu không.

Cái giá phải trả: mọi thứ bạn chỉnh chỉ đúng cho căn nhà và mùa đã chỉnh; đổi bếp hay sang mùa hanh khô là phải làm lại.

Một câu: Một con số “chắc chín phần mười” chỉ đáng tin khi đã có người ngồi đếm, và chỉ đáng tính khi có ai đó làm khác đi lúc nó báo “không chắc”.

25.7 Kết nối

C/12-co-che-huan-luyen là tiền đề kỹ thuật: cả nhóm trực quan hoá chỉ là gradient đổi biến. B/07-chia-du-lieu-va-danh-gia là nơi đọc ý nghĩa của các con số đi trước hiệu chỉnh — bốn ô của ma trận nhầm lẫn, biểu đồ độ tin cậy, ECE và phân rã điểm Brier. B/08-dich-chuyen-phan-phoi là giới hạn ngoài: hiệu chỉnh và ngưỡng đều mất hiệu lực khi phân phối dịch chuyển, và giám sát drift là cách phát hiện lúc đó. E/24-thi-nghiem-va-ablation

cung cấp kỷ luật đo: cải thiện về hiệu chỉnh cũng phải vượt sà nhiều như mọi cải thiện khác. [B/06-chat-luong-du-lieu](#) là nơi vòng lặp khép lại, khi lỗi thu ở production quay về thành dữ liệu mới. [D/21-inference-va-trien-khai](#) là mặt song song: nó lo hệ thống có *chạy kịp* không, chương này lo nó có *đáng tin* không.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một bản đồ Grad-CAM đẹp trên mô hình khởi tạo ngẫu nhiên lại đủ để bác bỏ giá trị của phương pháp đó, thay vì chỉ làm ta nghi ngờ?
2. Nêu một mô hình có ECE bằng không nhưng hoàn toàn vô dụng. Điều đó nói gì về việc dùng ECE làm metric duy nhất?
3. Temperature scaling không đổi độ chính xác. Vì sao, và vì sao đó vừa là ưu điểm vừa là giới hạn của nó?
4. Chi phí bỏ sót gấp một trăm lần chi phí báo động giả, tỉ lệ dương thật là 1%. Ngưỡng tối ưu là bao nhiêu, và vì sao con số đó có thể khiến hệ thống không triển khai được?
5. Conformal prediction cho bảo đảm gì và *không* cho bảo đảm gì? Giả định nào của nó bị dữ liệu robot vi phạm?
6. Bạn giám sát một mô hình đã triển khai mà không có nhãn. Loại dịch chuyển nào bạn phát hiện được, loại nào không, và bạn bù bằng cách gì?
7. Vì sao một đường từ chối trả lời không có người đón phía sau lại tệ hơn là không có đường từ chối nào?

Lộ trình học theo mục tiêu (Learning Paths by Goal)

PHỤ THUỘC không bắt buộc. Đây là chương điều hướng, nên đọc *trước* khi bắt đầu và đọc lại mỗi khi đổi mục tiêu.

TÓM TẮT

Hai mươi lăm chương trước không phải một hàng đợi phải đi hết theo thứ tự. Chương này cắt chúng thành sáu lộ trình theo mục tiêu, mỗi lộ trình nói rõ đọc chương nào, theo thứ tự nào, chương nào bỏ được và bỏ thì mất gì. Đọc xong bạn biết mình cần đọc bao nhiêu phần trăm của cây để đạt mục tiêu đang có, thay vì đọc tuần tự rồi bỏ dở ở giữa. Nếu chỉ nhớ một điều: mục tiêu quyết định lộ trình, và lộ trình ngắn nhất tới một mục tiêu cụ thể thường chỉ chiếm một phần ba số chương.

KIẾN THỨC CẦN CÓ TRƯỚC

- **lộ trình (learning path)** — một thứ tự đọc có chủ đích, chọn ra tập chương tối thiểu đủ cho một mục tiêu; khác với mục lục ở chỗ nó *loại bỏ* chương không cần.
- **tầng bán rã [T1]–[T4]** — nhân ở tiêu đề mỗi chương cho biết kiến thức đó sống được bao lâu: [T1] là toán và vật lý, [T4] là công cụ. Khi thời gian eo hẹp, cắt [T4] trước.
- **chương cốt lõi** — chương mà nếu bỏ thì các chương sau biến thành danh sách quy tắc phải học thuộc; trong bảng dưới đánh dấu “bắt buộc”.
- **mini project** — bài tập vài giờ tới hai ngày ở cuối mỗi chương, trong cây này đều gắn với visual SLAM để chúng nối thành một chuỗi liên tục.

CÂU HỎI MỞ ĐẦU

- Q1. Nếu chỉ cần fine-tune một mô hình có sẵn cho một tác vụ hẹp, bao nhiêu phần trăm của cây này là thừa?
- Q2. Vì sao lộ trình “đưa lên thiết bị biên” lại bắt đầu từ một chương ở Phần B chứ không từ Phần D?
- Q3. Vì sao lộ trình dài nhất không phải lộ trình học hết mọi chương?
- Q4. Nếu mục tiêu là áp dụng vào SLAM production, chương nào bạn có thể đọc lướt vì đã biết, và chương nào tuyệt đối không được lướt?

I 26.1 Đặt vấn đề

Một cây tri thức bốn trăm trang có một chế độ hỏng rất phổ biến: người đọc bắt đầu từ chương một, đi được đến chương bảy hoặc tám, rồi dừng, và phần dừng lại thường là phần họ cần nhất. Nguyên nhân không phải thiếu kỷ luật mà là sai thứ tự. Thứ tự tuyến tính tối ưu cho việc trình bày, không tối ưu cho việc *dùng được sớm*, và một người không thấy kiến thức của mình biến thành thứ chạy được thì sẽ mất động lực trước khi tới chương có giá trị nhất với họ.

Chương này sửa điều đó bằng cách thừa nhận rằng không có một thứ tự đúng. Có sáu mục tiêu thường gặp, và mỗi mục tiêu có một tập chương tối thiểu cùng một thứ tự riêng. Nguyên tắc chung là mọi lộ trình đều phải chạm ba chương nền, còn lại thì cắt theo mục tiêu.

TRỰC GIÁC

Cây này giống một thành phố chứ không giống một con đường. Mục lục là danh sách tên phố; chương này là bản đồ tuyến xe buýt. Cùng một thành phố, người đi làm và người đi du lịch lên hai tuyến khác nhau, và cả hai đều đúng. Sai lầm duy nhất là đi bộ qua từng con phố theo thứ tự bảng chữ cái.

26.2 Ba chương nền, mọi lộ trình đều phải chạm

Trước khi chia nhánh, có ba chương mà mọi lộ trình đều đi qua, vì bỏ chúng thì mọi chương sau mất chỗ bám.

- **B/03** — cách đọc một hệ thống lạ. Không có nó thì bạn đọc repo theo tên module thay vì theo giả định, và mọi chương Phần C trở thành danh mục kiến trúc.
- **B/04** — nền của việc học từ dữ liệu, và trực đánh đổi giữa giả định và dữ liệu. Đây là trực quay lại nhiều nhất trong cả cây.
- **C/12** — cơ chế huấn luyện. Không hiểu vì sao gradient chết thì mọi mô hình là hộp đen, kể cả khi bạn chỉ fine-tune.

26.3 Sáu lộ trình

A. Fine-tune một mô hình có sẵn cho tác vụ hẹp

Mục tiêu: có kết quả dùng được nhanh nhất, chấp nhận chưa hiểu hết bên trong. Đây là lộ trình ngắn nhất, khoảng một phần ba cây.

Bảng 26.1. Lộ trình A — fine-tune. Bảy chương, ước lượng ba tới bốn tuần kể cả mini project.

#	Chương	Vì sao cần	Bỏ thì mất gì
1	B/05	Định nghĩa nhãn và đơn vị phân tích; sai ở đây thì không sửa được ở bước sau	Mô hình học đúng thứ bạn không muốn
2	B/07	Chia dữ liệu và chọn metric; chống rò rỉ	Số đẹp mà vô nghĩa
3	B/03	Đọc được repo và mô hình bạn định dùng	Không biết nó giả định gì
4	C/18	Quyết định scratch, frozen hay fine-tune theo lượng dữ liệu	Chọn sai chế độ, tốn gấp mấy lần
5	C/12	Đủ để chẩn đoán khi loss không giảm	Bé tắc không biết hỏi gì
6	B/09	Loss phải khớp điều bạn thật sự muốn	Tối ưu nhằm mục tiêu
7	E/24	Biết cải thiện là thật hay là nhiều	Tự lừa mình

Bỏ được trong lộ trình này: toàn bộ Phần A trừ khi ảnh đầu vào có vấn đề vật lý, cả **C/16** và **C/17**, và phần lớn Phần D cho tới khi thật sự phải triển khai.

B. Hiểu cơ chế và tự huấn luyện từ đầu

Mục tiêu: không còn hộp đen nào. Lộ trình này nặng nhất về Phần C.

Thứ tự: [B/04](#) → [C/12](#) → [D/19](#) (giai đoạn tự viết bằng NumPy, làm ngay sau chương 12 chứ không đợi tới Phần D) → [C/13](#) → [B/09](#) → [B/10](#) → [C/14](#) → [E/24](#). Phần A đọc theo nhu cầu, mỗi khi một mục ở Phần C cần tới. Ước lượng tám tới mười tuần.

C. Đưa mô hình lên thiết bị biên với ngân sách độ trễ

Lộ trình này bắt đầu từ Phần B chứ không từ Phần D, và đó là điểm dễ gây bất ngờ nhất. Lý do là mọi quyết định nén và tăng tốc chỉ có nghĩa sau khi bạn biết hệ đang nghẽn ở đâu, mà điều đó thuộc [B/11](#).

Thứ tự: [B/11](#) → [B/10](#) → [D/20](#) → [D/21](#) → [C/13](#) (chỉ phần backbone nhẹ) → [D/22](#). Ước lượng ba tới bốn tuần. Bỏ được: [C/16](#), [C/17](#), phần lớn Phần A.

D. Áp dụng vào SLAM production

Đây là lộ trình dài nhất trong sáu lộ trình, nhưng không phải lộ trình đọc hết mọi chương. Nó cắt bỏ mô hình sinh và phần lớn kinh tế học pretraining, đổi lại đi rất sâu vào hình học, chuỗi thời gian và độ tin cậy. Đây cũng là lộ trình duy nhất đi hết Phần F, phần áp dụng học sâu vào chính bài toán SLAM.

Bảng 26.2. Lộ trình D — SLAM production. Năm giai đoạn, ước lượng mười tám tới hai mươi bốn tuần.

Giai đoạn	Chương, theo thứ tự	Điều bạn có được ở cuối giai đoạn
1. Nền	A/02 → B/03 → B/04	Biết ánh vào hệ của bạn đã qua những gì, và biết đọc một repo học sâu như đọc một hệ thống
2. Cơ chế	C/12 → D/19 → C/13 → B/09	Tự huấn luyện và chẩn đoán được một mô hình; hiểu backbone đủ để chọn
3. Miền của bạn	C/15 → C/16 → B/08 → B/11	Nối được flow, tracking, hình học và ngân sách tính toán vào hệ SLAM sẵn có
4. Đưa ra thật	D/20 → D/21 → E/24 → E/25 → D/22	Chạy trên thiết bị trong ngân sách, và chứng minh được cải thiện là thật
5. Áp dụng	F/27 → chọn theo nỗi đau: F/28 , F/29 , F/30 , F/34 → F/31	Quyết định được có nên thay một khối cổ điển bằng mạng hay không, và bảo vệ được quyết định đó bằng số đo

Hai lưu ý riêng cho lộ trình này. Thứ nhất, *đọc lướt được* là [A/01](#) và phần hình học của [A/02](#) nếu bạn đã quen SE(3) và hiệu chỉnh chùm tia, cùng phần ATE và RPE của [B/07](#). Thứ hai, *tuyệt đối không lướt* là [B/05](#) và [B/06](#): định nghĩa nhãn và chất lượng nhãn là chỗ người có nền hình học hay chủ quan nhất, vì trong SLAM cổ điển không có khái niệm nhãn do người gán. Thứ ba, giai đoạn 5 không đọc tuần tự: sau [F/27](#), chọn chương theo nỗi đau đang gặp, và bỏ hẳn [F/32](#) cùng [F/33](#) nếu bài toán của bạn chỉ là quỹ đạo chứ không phải bản đồ dày hay ngữ nghĩa.

E. Đọc paper và làm nghiên cứu

Thứ tự: [E/23](#) → [B/03](#) → [E/24](#) → [B/04](#) → một chương Phần C thuộc lĩnh vực bạn chọn → [E/25](#). Ước lượng bốn tuần, và lộ trình này nên chạy *song song* với mọi lộ trình khác chứ không nối tiếp, vì nó đổi cách bạn đọc mọi chương còn lại.

F. Học toàn bộ

Thứ tự đầy đủ là $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$ theo số chương, với ba ngoại lệ đáng làm khác đi: đọc [A/02](#) kỹ ngay từ đầu nhưng [A/01](#) thì đọc lướt rồi quay lại theo nhu cầu; làm giai đoạn NumPy của [D/19](#) ngay sau [C/12](#); và đọc [E/23](#) cùng [E/24](#) song song với Phần C thay vì để tới cuối. Ước lượng sáu tới chín tháng với nhịp mười giờ mỗi tuần.

I 26.4 Chuỗi mini project chạy xuyên cây

Mỗi chương kết thúc bằng một mini project gắn với visual SLAM, và chúng được thiết kế để nối thành một chuỗi liên tục thay vì hai mươi lăm bài tập rời. Chuỗi đó thay cho một dự án lớn duy nhất ở cuối, vì một dự án lớn có hai nhược điểm: nó dồn toàn bộ rủi ro vào một lần, và nó chỉ cho phản hồi sau nhiều tuần, tức vi phạm đúng điều kiện phản hồi nhanh mà [C/12](#) và [E/24](#) nói là bắt buộc.

Cách dùng chuỗi này: làm mini project của mọi chương trên lộ trình bạn chọn, bỏ qua mini project của chương bạn đã cắt. Mỗi bài đều có dòng *Xong khi* nêu một tiêu chí đo được, nên bạn biết chắc lúc nào được đi tiếp thay vì tự hỏi đã đủ chưa.

CẠM BẮY

Đừng đọc hết một lộ trình rồi mới bắt đầu làm. Chuỗi mini project chỉ có tác dụng khi mỗi bài được làm ngay sau chương của nó, lúc kiến thức còn tươi và lúc sai lầm còn rẻ. Đọc trước rồi làm sau biến chuỗi thành một dự án lớn duy nhất, tức đúng thứ chương này đang cố tránh.

I 26.5 Giới hạn và trường hợp hỏng

Sáu lộ trình này là phán đoán về thứ tự phụ thuộc, không phải kết quả đo được; một người có nền khác bạn sẽ cần thứ tự khác. Ước lượng thời gian giả định nhịp mười giờ mỗi tuần và đã có nền kỹ sư phần mềm, nên nó sẽ sai đáng kể với người mới hoàn toàn. Và điều quan trọng nhất: một lộ trình đọc xong không tạo ra năng lực. Chỉ chuỗi mini project mới tạo, còn lộ trình chỉ quyết định bạn làm những bài nào.

CẦN NHỚ

Mục tiêu quyết định lộ trình. Chọn một trong sáu, cắt thẳng tay những chương không nằm trên đó, và làm mini project của từng chương ngay sau khi đọc nó. Ba chương không bao giờ được cắt là [B/03](#), [B/04](#) và [C/12](#).

I Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Ba chương không bao giờ được cắt	B/03 đọc hệ thống lại, B/04 nền học từ dữ liệu, C/12 cơ chế huấn luyện. Bỏ một trong ba thì mọi chương sau mất chỗ bám.
●●● Lộ trình	Một thứ tự đọc chọn theo mục tiêu, kèm danh sách chương được phép cắt. Chọn lộ trình là chọn thứ mình <i>không</i> học.
●●● Chuỗi mini project	Bài tập cuối mỗi chương, nối liên tục thay cho một dự án lớn duy nhất ở cuối khoá.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
•• Vì sao chuỗi thay cho một dự án lớn?	Dự án lớn dồn hết rủi ro vào một lần và chỉ cho phản hồi sau nhiều tuần, tức vi phạm điều kiện phản hồi nhanh mà C/12 và E/24 đòi hỏi.
•• Vì sao lộ trình biên bắt đầu từ Phần B chứ không Phần D?	Vì mọi kỹ thuật nén và tăng tốc chỉ có nghĩa sau khi biết hệ nghẽn ở đâu, mà chẩn đoán nghẽn nằm ở B/11 .
•• Vì sao lộ trình nghiên cứu chạy song song chứ không nối tiếp?	Vì nó đổi cách bạn đọc mọi chương khác. Để tới cuối thì bạn đã đọc xong bằng thói quen cũ.
•• Ba ngoại lệ của thứ tự A đến F	Đọc A/02 kỹ ngay nhưng A/01 lướt trước; làm giai đoạn NumPy của D/19 ngay sau C/12 ; đọc E/23 và E/24 song song với Phần C.
•• Giai đoạn 5 của lộ trình SLAM	Phần F, và nó không đọc tuần tự: sau F/27 thì chọn chương theo nỗi đau đang gặp.
• Hai chương của Phần F bỏ được nếu chỉ cần quỹ đạo	F/32 bản đồ neural và F/33 ngữ nghĩa. Chúng phục vụ bản đồ dày và nhiệm vụ, không phục vụ độ chính xác quỹ đạo.
• Dòng Xong khi	Tiêu chí đo được của mỗi mini project, để biết lúc nào được đi tiếp thay vì tự hỏi đã đủ chưa.
•• Đọc hết một lộ trình rồi mới làm — hỏng ở đâu?	Nó biến chuỗi mini project trở lại thành một dự án lớn duy nhất, đúng thứ chương này cố tránh.
•• Ước lượng thời gian ở đây giá định gì?	Nhịp mười giờ mỗi tuần và đã có nền kỹ sư phần mềm. Sai đáng kể với người mới hoàn toàn.

••• phải thuộc •• nên nắm • biết là có

MINI PROJECT SLAM · Viết lộ trình của chính bạn thành lịch chạy được · 1–2 ngày, rồi chạy tuần đầu

Mục tiêu	Biến sáu lộ trình trừu tượng thành một lịch có ngày tháng, để tuần sau bạn mở ra là biết đọc chương nào và làm bài nào, không phải quyết định lại.
Dữ liệu	Không cần dữ liệu. Cần danh sách chương của cây, quỹ thời gian thật mỗi tuần của bạn, và một câu phát biểu mục tiêu trong một dòng.
Các bước	Chọn một trong sáu lộ trình theo mục tiêu đó. Liệt kê các chương của nó và <i>gạch tên</i> những chương bị cắt — viết ra danh sách bị cắt quan trọng ngang danh sách phải đọc. Chia theo tuần với nhịp thật của bạn chứ không nhịp lý tưởng. Với mỗi tuần, ghi kèm mini project của chương đó và dòng <i>Xong khi</i> của nó. Cuối cùng đặt một mốc rà lại sau bốn tuần.
Xong khi	Bạn có một bảng gồm tuần, chương, mini project, tiêu chí xong; và bạn đã hoàn thành mini project của tuần đầu tiên đúng theo tiêu chí đó.
Nối với	E/24 cho cách đo tiến độ mà không tự lừa mình; B/03 là chương đầu tiên của mọi lộ trình.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Khoảng hai phần ba. Lộ trình A chỉ cần bảy chương; toàn bộ Phần A, hai chương 3D và mô hình sinh, cùng phần lớn Phần D đều bỏ được cho tới khi thật sự triển khai.
- A2. Vì mọi kỹ thuật nén và tăng tốc chỉ có nghĩa sau khi biết hệ nghẽn ở đâu, mà chẩn đoán nghẽn thuộc [B/11](#). Bắt đầu từ Phần D thì bạn chọn kỹ thuật trước khi biết bệnh.
- A3. Vì lộ trình dài nhất là lộ trình SLAM production, và nó vẫn cắt bỏ mô hình sinh cùng phần lớn kinh tế học pretraining. Học hết mọi chương là lộ trình *rộng* nhất, không phải *sâu* nhất theo một mục tiêu.
- A4. Lướt được phần toán ở [A/01](#), phần hình học nhiều view ở [A/02](#), và phần ATE cùng RPE ở [B/07](#). Không được lướt [B/05](#) và [B/06](#), vì nhân do người gán là khái niệm không tồn tại trong SLAM cổ điển.

NÓI LẠI THẬT ĐƠN GIẢN

Hình dung bạn có bản đồ một thành phố và mười giờ rảnh mỗi tuần. Bản đồ vẽ mọi con đường, nhưng bạn không đi hết được, và cũng không cần. Việc đầu tiên không phải chọn đường nào để đi mà là nói ra bạn muốn đến đâu; khi đã nói được điều đó thì phần lớn con đường tự loại mình. Chương này làm đúng ba việc. Nó hỏi bạn muốn đến đâu, và ép câu trả lời gói trong một dòng. Nó đưa sáu con đường đã có người đi, kèm danh sách những khu phố bạn được quyền không ghé. Và nó nhắc rằng đi bộ mới tính là đi: mỗi chương có một bài tập nhỏ, làm ngay sau khi đọc, vì kiến thức còn tươi và sai lầm còn rẻ.

Một câu: Chọn lộ trình thực chất là chọn thứ mình sẽ không học, và đó mới là quyết định khó.

Chỗ không rút gọn được: Vì sao ba chương nền lại đúng là ba chương đó, chứ không phải ba chương khác — đây là phán đoán về thứ tự phụ thuộc, chưa ai đo được.

26.6 Kết nối

Chương này là bản đồ của toàn cây, nên nó nối tới mọi chương. Hai liên kết đáng nhớ nhất: **E/24** giải thích vì sao mọi lộ trình đều kết thúc bằng chương thực nghiệm, và **B/03** là chương duy nhất xuất hiện trong cả sáu lộ trình.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Mục tiêu của bạn, viết trong một dòng, dẫn tới lộ trình nào trong sáu lộ trình?
2. Ba chương nào không bao giờ được cắt, và vì sao cắt một trong ba lại làm hỏng mọi chương sau?
3. Lộ trình đưa mô hình lên thiết bị biên bắt đầu từ Phần B chứ không Phần D — lý do là gì?
4. Vì sao chuỗi mini project thay cho một dự án lớn duy nhất ở cuối, và hai nhược điểm của dự án lớn là gì?
5. Giai đoạn 5 của lộ trình SLAM không đọc tuần tự; bạn chọn chương của Phần F theo tiêu chí nào?
6. Nếu bạn đọc hết một lộ trình trước rồi mới làm bài tập, điều gì hỏng?

HỌC SÂU CHO VISUAL SLAM



TÓM TẮT

Phần F đặt toàn bộ năm phần trước vào một câu hỏi duy nhất: có nên thay một khối trong hệ SLAM cổ điển bằng một mạng, và nếu có thì khối nào. Tám chương đi từ khung quyết định, qua frontend học được, đóng vòng, độ sâu và dòng quang, backend khả vi, bản đồ neural, ngữ nghĩa, tới IMU và triển khai. Mỗi phương pháp được hỏi đúng bốn câu: nó chữa thất bại cụ thể nào, cơ chế nào làm được, có đáng cái giá không, và nếu không dùng thì học được gì từ thiết kế của nó. Nếu chỉ nhớ một điều: phần lớn công trình trong phần này chưa đủ chín để thay thế một hệ cổ điển đã tinh chỉnh kỹ, nhưng vài ý tưởng trong đó chữa đúng những chỗ mà hình học thuần túy không chữa nổi.

Năm phần trước dạy học sâu như một môn học. Phần này dùng nó như một hộp công cụ, cho một hệ thống cụ thể đã tồn tại và đang đau ở những chỗ cụ thể.

Cách đặt vấn đề vì thế đảo ngược so với phần lớn tài liệu về chủ đề này. Tài liệu thường bắt đầu từ phương pháp rồi đi tìm chỗ dùng. Ở đây ta bắt đầu từ bốn thất bại có thật của một hệ stereo-inertial đã tinh chỉnh kỹ — **chuyển động nhanh, ảnh mờ do chuyển động, đóng vòng không nổ, và đặt trọng số cho IMU** — rồi hỏi ngược lại xem có công trình nào thật sự chạm tới chúng.

Câu trả lời trung thực là: có, nhưng ít hơn nhiều so với ấn tượng mà số lượng paper tạo ra. Rất nhiều kết quả ấn tượng trong phần này đến từ chuỗi ngắn, cảnh trong nhà, và máy để bàn có GPU rời. Một hệ chạy trên Jetson Orin, ngoài trời, thời gian thực, với ngân sách vài chục mili-giây mỗi khung, sống trong một thế giới khác. Chương 27 dựng khung để phân biệt hai thế giới đó trước khi đọc bất cứ chương nào sau.

Một lưu ý về giọng của phần này. Nó phê phán nhiều hơn năm phần trước, và điều đó là cố ý. Với một người chưa có hệ thống, học sâu là con đường nhanh nhất để có một hệ chạy được. Với một người đã có một hệ chạy tốt, phần lớn phương pháp trong đây là một cuộc đánh đổi đắt: đổi tính chẵn đoán được, tính lặp lại và ngân sách tính toán lấy một cái thiện chỉ hiện ra trên một số cảnh. Biết trước điều đó không làm ta bỏ lỡ cơ hội; nó làm ta tiêu đúng ba tháng vào đúng một ý tưởng thay vì rải mỏng vào tám ý tưởng.

Bảng 26.4. Bản đồ Phần F: các chương, nội dung, và nỗi đau SLAM mà chương đó chạm tới.

Ch.	Thư mục	Nội dung	Chạm nỗi đau
27	27-dat-hoc-sau-vao-da	Năm trụ, khung bốn cổng để quyết định có nên thay một khối cổ điển bằng mạng	cả bốn (bộ lọc)
28	28-dac-trung-va-ghep-	Đặc trưng và ghép cặp học được; vì sao ORB gây và cái gì thay được	chuyển động nhanh, ảnh mờ
29	29-dinh-vi-lai-va-don	Nhận dạng lại địa điểm, định vị lại, và bốn tầng của đóng vòng	đóng vòng
30	30-do-sau-dong-quang-	Độ sâu đơn ảnh, dòng quang, và mô hình nền hình học	chuyển động nhanh, tỉ lệ
31	31-backend-kha-vi-va-	Tối ưu khả vi và hệ đầu-cuối; khả vi mua được gì, mất gì	backend
32	32-ban-do-neural	NeRF và 3D Gaussian Splatting làm bản đồ; đẹp ảnh không bằng đúng pose	biểu diễn bản đồ
33	33-ngu-nghia-va-do-th	Ngữ nghĩa, vật thể động, đồ thị cảnh 3D	cảnh động
34	34-imu-bat-dinh-va-tr	Trọng số IMU, bất định học được, ảnh mờ, và đưa lên Orin	trọng số IMU, ảnh mờ

Đọc chương 27 trước, luôn luôn: nó là bộ lọc quyết định bảy chương còn lại chương nào đáng đọc kỹ với bạn. Bảy chương sau đọc lập với nhau, chọn theo nỗi đau đang gặp chứ không đọc tuần tự.

Đặt học sâu vào đâu trong một hệ SLAM (Where Learning Fits in a SLAM System)

PHỤ THUỘC B/03-giai-phau-he-thong (đọc một hệ thống bằng luồng dữ liệu và ranh giới); E/24-thi-nghiem-va-ablation (sản nhiễu và oracle experiment — chương này dùng lại cả hai); B/11-gioi-han-tinh-toan cùng D/21-inference-va-trien-khai cho công ngân sách; B/08-dich-chuyen-phan-phoi cho công thứ tư.

TÓM TẮT

Chương này không dạy một phương pháp nào. Nó dựng bộ lọc để bạn quyết định phương pháp nào đáng đọc tiếp. Bạn sẽ có bốn công cụ: năm chỗ có thể cắm mạng vào một hệ SLAM cổ điển, một khung bốn cổng để loại sớm, một bản đồ từ bốn nỗi đau sang nhóm phương pháp, và một quy trình đọc paper trong ba mươi phút. Đọc xong bạn phát biểu được câu “chưa đáng” kèm con số thay vì kèm cảm giác, và biết đầu tư ba tháng tới vào đúng một chỗ. Nếu chỉ nhớ một điều: đo trần đóng góp của một khối trước, rồi mới đi tìm mạng để thay khối đó — và ưu tiên những chỗ mà khi hỏng thì hệ thống hỏng ồn ào.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình học sâu** — một hàm có rất nhiều tham số, được đi tìm bằng cách cho xem nhiều ví dụ có đáp án. Trong chương này nó luôn xuất hiện như một *khối* nhận ảnh và trả về một đại lượng: điểm đặc trưng, độ sâu, mặt nạ, hoặc một pose.
- **suy luận (inference)** — một lần chạy mô hình đã huấn luyện trên một đầu vào mới. SLAM chạy suy luận trên từng khung hình, mỗi lần đúng một ảnh, nên nó bị chặn bởi *độ trễ* chứ không phải bởi *thông lượng*.
- **batch** — số mẫu được đưa qua mạng cùng lúc. GPU đạt hiệu suất cao khi batch lớn; SLAM thời gian thực buộc batch bằng 1, nên phần lớn con số FPS trong paper không áp dụng được cho bạn.
- **phân phối huấn luyện** — tập các tình huống mà mô hình đã được xem trong lúc huấn luyện. “Ra ngoài phân phối” nghĩa là gặp tình huống khác hẳn — đêm tối, mưa, hành lang không vãn, ảnh mờ — và đó là chỗ mọi bảo đảm về độ chính xác biến mất.
- **ma trận thông tin** — nghịch đảo của ma trận hiệp phương sai; trong một đồ thị nhân tử nó chính là *trọng số* của một phép đo. Đặt nó quá lớn nghĩa là bảo bộ tối ưu “hãy tin phép đo này”, và đó là cách hỏng âm thầm nhất của một hệ visual-inertial.
- **APE và RPE** — APE là sai số tuyệt đối của quỹ đạo sau khi căn chỉnh với chân trị; RPE là sai số tương đối trên các đoạn ngắn. APE nhạy với trôi tích lũy và với vòng đóng; RPE nhạy với chất lượng odometry cục bộ.
- **tỉ lệ hoàn thành chuỗi** — phần trăm khung hình mà hệ thống còn bám được. Nó phải luôn được báo cùng APE, vì một hệ mất bám giữa chừng chỉ còn được chấm trên đoạn dễ và vì thế có APE rất đẹp.
- **trần đóng góp (oracle ceiling)** — mức cải thiện tối đa mà một khối có thể mang lại, đo bằng cách thay khối đó bằng chân trị rồi chạy cả hệ. Nếu một bộ ghép cặp hoàn hảo chỉ giảm APE được một lượng c , thì mọi bộ ghép cặp học được cũng bị chặn trên bởi đúng c . Giá trị của c là thứ bạn phải tự đo trên hệ của mình; không mượn được của ai và không đọc được từ bảng của ai.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn thay DBoW bằng một mô hình nhận dạng địa điểm hiện đại. Recall@1 tăng vọt, nhưng số vòng đồng thành công không đổi một cái nào. Chuyện gì đã xảy ra, và lẽ ra bạn phải đo gì trước?
- Q2. Một paper báo APE thấp hơn ORB-SLAM3 nhiều lần. Ba câu hỏi nào phải hỏi trước khi tin, và câu nào loại được nhiều paper nhất chỉ trong ba phút?
- Q3. Vì sao một phép đo sai mà “tự tin” lại nguy hiểm hơn hẳn việc không có phép đo nào — trong khi với một bộ phân lớp ảnh thì hai chuyện đó gần như tương đương?
- Q4. Một paper báo mạng chạy rất nhanh trên một GPU để bàn. Vì sao con số đó gần như không nói gì về việc nó có chạy nổi 20 Hz trên Jetson Orin hay không?
- Q5. Bạn có ba tháng và bốn nỗi đau. Phép đo nào cho bạn quyền tiêu cả ba tháng vào đúng một nỗi đau, mà không cần đọc thêm một paper nào?

I 27.1 Đặt vấn đề

Bạn đang bảo trì một nhánh ORB-SLAM3 stereo-inertial [40]. Nó chạy EuRoC [210], TUM-VI [211] và OpenLORIS, triển khai trên Jetson Orin, và đã được tinh chỉnh nhiều tháng. Trên phần lớn chuỗi EuRoC nó chạy trơn chuỗi và cho một quỹ đạo mà bạn đã đo đi đo lại nhiều lần, nên bạn biết rất rõ mức sai số của nó — biết bằng số đo của chính mình chứ không bằng bảng của ai. Mỗi tuần lại có một paper nói rằng một mạng nào đó làm tốt hơn.

Câu hỏi thật không phải “học sâu có tốt hơn hình học không”. Câu hỏi đó vô nghĩa vì không có phép đo nào trả lời được nó. Câu hỏi thật gồm bốn phần, và phải hỏi theo đúng thứ tự: khối nào trong hệ của tôi đang thật sự làm hỏng việc, trên chuỗi nào, với ngân sách bao nhiêu mili-giây, và khi nó hỏng thì tôi có biết không.

Có một sự bất đối xứng khiến việc này khó hơn vẻ ngoài. Paper được viết để cho thấy nơi phương pháp *thắng*. Bạn cần biết nơi nó *thua*. Không phải vì tác giả không trung thực, mà vì cấu trúc khuyến khích của việc công bố không thưởng cho một bảng liệt kê các cảnh mà phương pháp gây. Hệ quả là bạn phải tự dựng phần thiếu đó.

Còn một bất đối xứng nữa, nặng hơn. Baseline trong phần lớn paper là ORB-SLAM3 chạy cấu hình mặc định. Baseline của bạn là ORB-SLAM3 đã được chính bạn tinh chỉnh cho cảm biến của mình. Khoảng cách giữa hai thứ đó không nhỏ. Một phần đáng kể của “cải thiện nhiều lần” trong bảng có thể chỉ là khoảng cách tinh chỉnh, chứ không phải đóng góp của phương pháp.

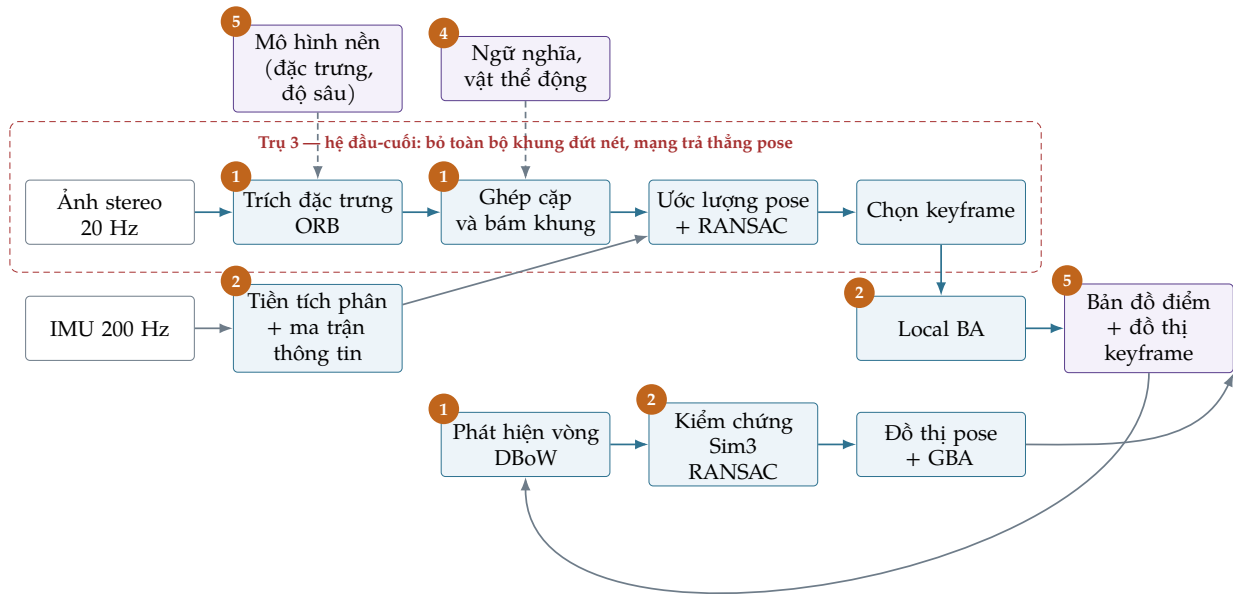
↪ **tiền đề E/24** — đây chính là ràng buộc “baseline mạnh là nghĩa vụ” của chương 24, lần này nhìn từ phía người đọc bằng chứ không phải người làm bảng

Chương này vì thế là một bộ lọc, không phải một khảo sát. Nó có bốn phần và bốn phần đó dùng được độc lập. Phần đầu nói *có thể* cắm mạng vào đầu, tức năm trụ. Phần hai nói *có nên* cắm không, tức khung bốn cổng. Phần ba nói bốn nỗi đau cụ thể của bạn sang nhóm phương pháp nào hứa chữa và chương nào nói về nó. Phần bốn nói *cắm sâu tới mức nào*, tức ba mức can thiệp. Phần cuối là kỹ năng thao tác: đọc một paper trong ba mươi phút và biết loại nó lúc nào.

Một lời cảnh báo về giọng. Chương này sẽ kết luận “chưa đáng” nhiều lần. Đó không phải thái độ bảo thủ mà là hệ quả trực tiếp của việc bạn đã có một hệ thống chạy tốt. Với người chưa có hệ thống nào, một hệ đầu-cuối học được là con đường nhanh nhất để có thứ chạy được. Với người đã có, phần lớn phương pháp trong Phần F là một cuộc đổi chác đất: đổi tính chẩn đoán được, tính lặp lại và ngân sách tính toán, lấy một cải thiện chỉ xuất hiện trên vài cảnh.

I 27.2 Năm trụ: học sâu cắm được vào đầu (The Five Pillars)

Trước khi hỏi “có nên”, phải biết “ở đâu”. Một hệ SLAM cổ điển không phải một khối. Nó là một chuỗi khối nối nhau bằng những giao diện rất rõ ràng, và chính các giao diện đó mới là chỗ một mạng có thể chen vào. Hình 27.1 vẽ hệ thống đó cùng năm chỗ cắm.



Hình 27.1. Một hệ stereo-inertial cổ điển và năm chỗ có thể cắm mạng vào. Trụ 1 đổi cách *sinh ra* phép đo (đặc trưng, ghép cặp, lấy ứng viên vòng). Trụ 2 đổi cách *cân* phép đo trong bộ tối ưu. Trụ 3 xóa cả khung đứt nét đồ và thay bằng một mạng trả pose. Trụ 4 đổi *phép đo nào được phép vào*. Trụ 5 đổi *bản đồ là cái gì*, hoặc cấp một tiên nghiệm học sẵn cho các khối khác. Đi từ 1 tới 3 là đi từ ngoài vào trong: càng vào sâu, càng ít đại lượng trung gian mà bạn còn kiểm tra được bằng một kỳ vọng vật lý.

A. Trụ 1 — frontend nhận thức

Trụ này thay cách *sinh ra phép đo*. ORB là một bộ dò góc cộng một chuỗi bit so sánh cường độ điểm ảnh. Thay nó bằng một mạng nghĩa là: ảnh vào, danh sách điểm ảnh cùng vectơ mô tả ra. Ghép cặp cũng vậy, và tầng lấy ứng viên vòng cũng vậy. SuperPoint [212] là ví dụ điển hình cho bộ dò và mô tả; SuperGlue [213] và LightGlue [214] cho bước ghép cặp; LoFTR [215] bỏ hẳn bước dò điểm.

Điều quan trọng về mặt kiến trúc là giao diện không đổi. Backend vẫn nhận đúng thứ nó vẫn nhận: các tương ứng hai chiều kèm một ước lượng độ bất định. Vì thế RANSAC, hiệu chỉnh chùm tia, đồ thị nhân tử, tất cả giữ nguyên. Đây là trụ có rủi ro thấp nhất và cũng là trụ đã chín nhất, và chương F/28 dành trọn cho nó.

B. Trụ 2 — backend tối ưu

Trụ này không đổi phép đo, nó đổi *trọng số* của phép đo. Trong một đồ thị nhân tử, mỗi cạnh có một ma trận thông tin. Với IMU, ma trận đó đến từ mô hình nhiễu của cảm biến qua bước tiền tích phân [216]. Với điểm ảnh, nó thường là một hằng số nhân với hệ số tăng kim tự tháp. Cả hai đều là giả định, và cả hai đều sai ở mức nào đó.

Học sâu vào đây theo ba đường. Thứ nhất, học độ bất định của từng phép đo thay vì dùng hằng số. Thứ hai, học mô hình nhiễu của IMU, gồm cả trôi bias và nhiễu phụ thuộc chuyển động. Thứ ba, làm cho cả bộ tối ưu khả vi để huấn luyện xuyên qua nó. Hai đường đầu nằm ở chương F/34,

đường thứ ba ở chương F/31. Đây là trụ ít được khai thác nhất và, theo đánh giá của chương này, là trụ có tỉ số lợi ích trên rủi ro tốt nhất cho một hệ đã chạy.

C. Trụ 3 — hệ đầu-cuối

Trụ này xoá cả chuỗi. Ảnh vào, pose ra, không có bước nào ở giữa mà bạn đặt tên được bằng một khái niệm hình học. DROID-SLAM [217] là mốc quan trọng nhất của dòng này; nó vẫn giữ một lõi tối ưu nhưng toàn bộ phần sinh tương ứng là học được. Các hệ mới hơn đi xa hơn nữa, đưa cả bước dựng hình vào một mô hình nền duy nhất.

Cái được là tính bền: những hệ này thường không mất bám ở chỗ mà hệ đặc trưng thừa mất bám. Cái mất là toàn bộ bề mặt chẩn đoán. Không còn số inlier để nhìn, không còn hiệp phương sai để hỏi, không còn cách nào trả lời câu “vì sao khung 1842 sai”. Với một hệ sản phẩm, đó là cái giá rất khó trả. Chương F/31 nói kỹ.

D. Trụ 4 — hiểu cảnh

Trụ này không ước lượng pose và không sinh phép đo. Nó quyết định *phép đo nào được phép vào*. Ví dụ điển hình là loại bỏ điểm nằm trên vật thể chuyển động: DynaSLAM [218] dùng mặt nạ ngữ nghĩa để vớt các điểm rơi trên người và xe trước khi chúng vào bộ tối ưu. Ở mức cao hơn, đồ thị cảnh 3D như Kimera [219] dựng một tầng ngữ nghĩa nằm *cạnh* bộ ước lượng hình học chứ không thay nó.

Đặc điểm cấu trúc của trụ này rất đáng chú ý: nó chỉ *trừ đi* chứ không cộng thêm. Nếu mạng chết, hệ thống trở về đúng hành vi cổ điển. Đó là lý do nó thuộc mức can thiệp thấp nhất, dù mô hình bên trong có thể rất lớn. Chương F/33 nói kỹ.

E. Trụ 5 — mô hình nền và biểu diễn neural

Trụ này gồm hai thứ hay bị gộp làm một, và tách chúng ra là việc đầu tiên phải làm.

- **Mô hình nền hình học** — một mạng lớn huấn luyện trên rất nhiều dữ liệu, trả về độ sâu, đặc trưng, hoặc trực tiếp một đám mây điểm đã căn chỉnh. DUST3R [34] là mốc; MAST3R-SLAM [220] là một nỗ lực đưa dòng đó vào SLAM thời gian thực. Chúng cấp *tiền nghiệm* cho các khối khác.
- **Biểu diễn bản đồ neural** — NeRF [28] và 3D Gaussian Splatting [175] đổi câu trả lời cho câu hỏi “bản đồ là cái gì”. Thay vì một đám mây điểm thưa, bản đồ thành một trường liên tục hoặc một tập splat. MonoGS [221] là ví dụ đưa nó vào một vòng SLAM.

Hai thứ đó phục vụ hai mục đích khác nhau. Mô hình nền nhắm vào độ bền của phép đo. Biểu diễn neural nhắm vào chất lượng bản đồ dựng lại, mà chất lượng đó *không* đồng nghĩa với độ chính xác quỹ đạo. Trộn hai mục tiêu là nguồn hiểu nhầm lớn nhất khi đọc mẩu này, và chương F/32 mở đầu bằng đúng việc tách chúng.

Bảng 27.1. Năm trụ đọc theo cùng bốn tiêu chí. Cột cuối là cột đáng giá nhất: nó cho biết khi nào lợi ích hứa hẹn của trụ đó biến mất.

Trụ	Cơ chế	Giả định	Hỏng khi nào
1. Frontend nhận thức	Mạng sinh điểm, mô tả, tương ứng, hoặc ứng viên vòng; giao diện với backend không đổi	Cảnh triển khai nằm trong phân phối huấn luyện; có đủ ngân sách độ trễ	Cảnh lạ về kết cấu hoặc ánh sáng; mạng vẫn trả đủ điểm nhưng mô tả không còn phân biệt được
2. Backend tối ưu	Học trọng số, hiệp phương sai, hoặc mô hình nhiễu IMU; cấu trúc đồ thị nhân tử giữ nguyên	Độ bất định thật sự phụ thuộc dữ liệu quan sát được, không chỉ phụ thuộc cảm biến	Trọng số học được quá tự tin; bộ tối ưu tin nhầm một cạnh sai và không có tín hiệu nào báo
3. Hệ đầu-cuối	Mạng nhận ảnh và trả pose hoặc trả trực tiếp quỹ đạo; xoá các bước hình học tường minh	Chuyển động và cảnh khi triển khai giống lúc huấn luyện; tài nguyên GPU dồi dào	Chuỗi dài hơn nhiều so với lúc huấn luyện; bộ nhớ tăng theo số khung; không còn đại lượng trung gian để chẩn đoán
4. Hiểu cảnh	Mạng quyết định phép đo nào bị loại, hoặc dựng một tầng ngữ nghĩa cảnh bản đồ hình học	Vật thể động nhận dạng được bằng loại của nó; loại bỏ nhầm ít hơn giữ lại nhầm	Vật động không thuộc lớp đã học (rèm cửa, bóng, phản chiếu); cảnh mà phần lớn điểm nằm trên vật động
5. Mô hình nền, bản đồ neural	Tiên nghiệm học sẵn cho độ sâu và đặc trưng, hoặc đối hẩn cấu trúc bản đồ	Tiên nghiệm đủ đúng để không kéo lệch hình học; đủ bộ nhớ giữ biểu diễn	Sai tỉ lệ tích lũy; chuỗi dài ngoài trời; ảnh dựng lại đẹp trong khi quỹ đạo tệ đi

F. Vì sao thứ tự năm trụ lại chính là thứ tự lịch sử

Năm trụ không phải một danh sách phẳng. Chúng xếp theo độ sâu của can thiệp, và độ sâu đó gần như trùng với thứ tự mà cộng đồng đã đi qua. Mạch dưới đây giải thích vì sao.

I Mạch 1 — vì sao học sâu vào SLAM từ frontend trước

- Vấn đề** Hệ cổ điển đo cảnh bằng đặc trưng do người thiết kế. ORB rẻ, bất biến với xoay trong mặt phẳng ảnh, và chạy được trên CPU. Nhưng nó giả định ảnh sắc nét, có vân, và độ sáng ổn định.

Giải pháp Thay bộ dò và bộ mô tả bằng một mạng, huấn luyện trên các cặp ảnh đã biết tương ứng.

Giả định Cảnh triển khai giống cảnh huấn luyện, và có GPU rảnh để chạy suy luận mỗi khung.

Giá phải trả thêm độ trễ, thêm bộ nhớ, thêm một phụ thuộc phần cứng.
- Vấn đề** Đầu ra vẫn là điểm và mô tả, nên backend không phải sửa dòng nào. Đó chính là điều làm bước này hấp dẫn: rủi ro tích hợp thấp.

Giải pháp Giữ nguyên toàn bộ phần hình học, chỉ đổi nguồn phép đo.

Vấn đề mới Bạn mất một tín hiệu cũ rất quý. ORB không tìm được góc nào thì trả về danh sách rỗng, và hệ thống biết ngay là đang mù. Một mạng gần như luôn trả đủ số điểm bạn yêu cầu, kể cả trên tường trắng.
- Vấn đề** Nếu phép đo đã học được, vì sao còn cân chúng bằng hằng số do người đặt? Ma trận thông tin của cảnh ảnh trong phần lớn hệ thống là một hằng số nhân hệ số tăng.

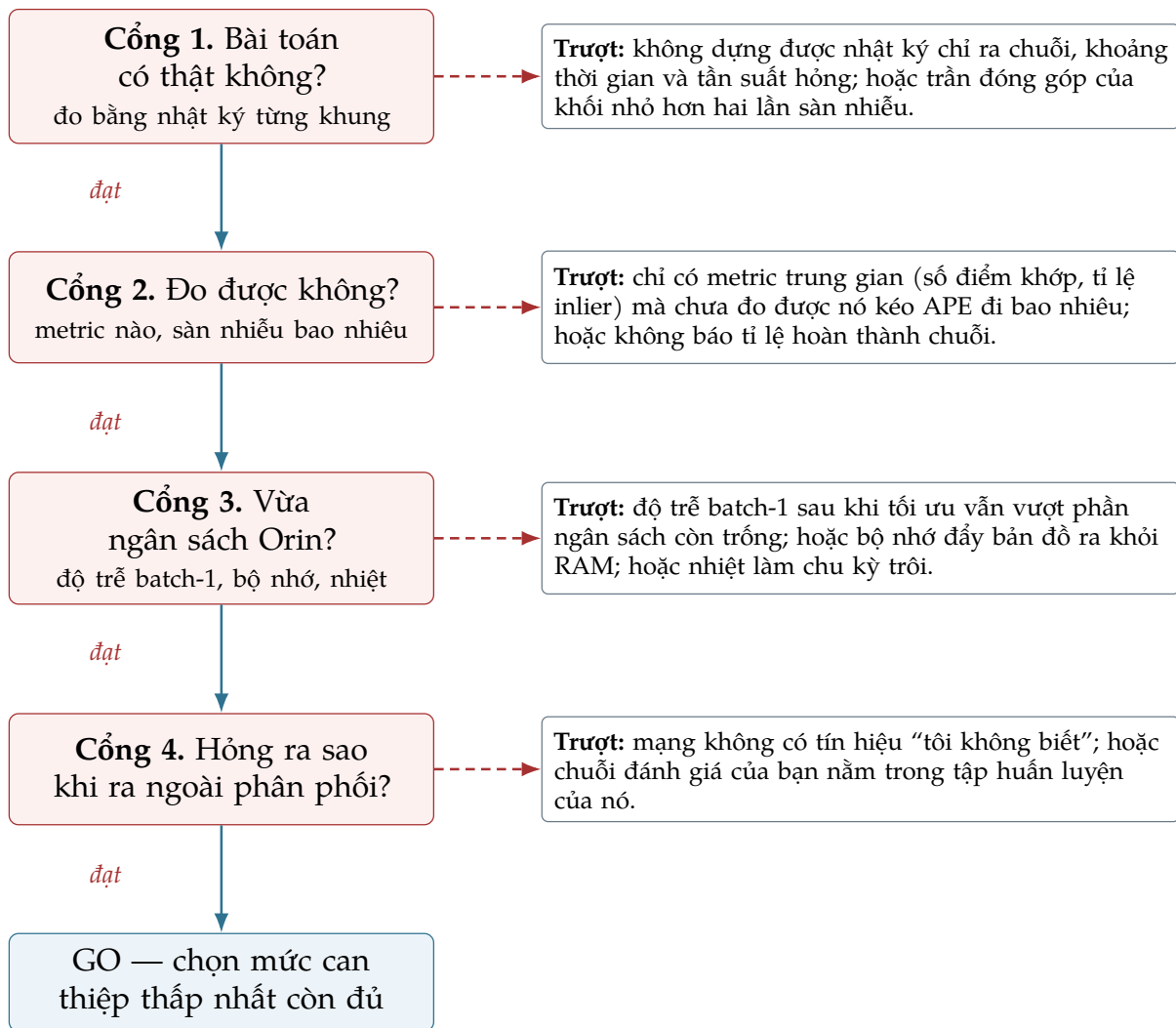
Giải pháp Học độ bất định của từng phép đo, hoặc học mô hình nhiễu IMU.

Giả định	Có chân trị đủ tốt để dạy mạng biết khi nào phép đo của chính nó sai.
Hệ quả	Đây là trụ 2. Nó khó hơn trụ 1 vì nhãn khó kiểm, nhưng nó nằm hoàn toàn bên trong bộ ước lượng cổ điển, nên không phá vỡ gì.
4 Vấn đề	Cả hai bước trên vẫn bị chặn bởi thiết kế của hệ cổ điển: đặc trưng thừa, tương ứng rời rạc, keyframe rời rạc.
Giải pháp	Làm cho cả chuỗi khả vi rồi huấn luyện đầu-cuối.
Giá phải trả	mất bề mặt chẩn đoán, mất khả năng quay lại, và mất luôn nhiều năm tính chính đã nằm trong các ngưỡng của hệ cũ.
Vấn đề mới	Khi hệ đầu-cuối sai, bạn không còn đại lượng trung gian nào để hỏi. Cách sửa duy nhất là huấn luyện lại, và đó là vòng lặp tính bằng tuần chứ không bằng phút.

Mạch này cho một quy tắc đọc rất hữu dụng. Khi gặp một phương pháp mới, hãy hỏi ngay: nó đang ở trụ nào? Câu trả lời cho biết bạn phải xoá cái gì trong hệ hiện tại để nhận nó, và đó thường là thông tin quyết định, chứ không phải con số APE trong bảng.

27.3 Khung bốn cổng (The Four-Gate Decision Framework)

Biết một phương pháp nằm ở trụ nào chưa đủ để quyết định. Cần một quy trình loại bỏ, và quy trình đó phải rẻ. Bốn cổng dưới đây được xếp theo chi phí tăng dần: cổng 1 tốn vài ngày đo trên hệ của bạn, cổng 4 tốn nhiều nhất và vì thế đứng cuối. Hình 27.2 là sơ đồ đầy đủ, kèm lối ra ở mỗi cổng.



Hình 27.2. Bốn cổng xếp theo chi phí tăng dần, mỗi cổng có tiêu chí trượt cụ thể. Phần lớn ứng viên chết ở cổng 1 và cổng 3, và cả hai cổng đó đều trả lời được bằng phép đo trên hệ của chính bạn, không cần chạy phương pháp mới lần nào.

A. Cổng 1 – bài toán có thật không, và đo bằng gì

Đây là cổng loại được nhiều ứng viên nhất, và cũng là cổng hay bị bỏ qua nhất. Lý do bỏ qua rất người: một paper mô tả rất sống động một thất bại, bạn nhận ra hệ mình cũng có thất bại đó, và thế là “vấn đề đã được xác nhận”. Nhưng nhận ra không phải là đo.

Cổng này đòi hai thứ. Thứ nhất là **nhật ký từng khung**: chuỗi nào, khoảng thời gian nào, triệu chứng gì, tần suất bao nhiêu. Nếu bạn nói “hệ hay mất bám khi quay nhanh” mà không nói được nó xảy ra ở bao nhiêu khung trên tổng số bao nhiêu, bạn chưa qua cổng. Thứ hai là **trần đóng góp**: thay khối bị nghi bằng chân trị rồi đo lại cả hệ.

Bước thứ hai là bước quyết định, và nó rõ ràng hơn về ngoài. Muốn biết trần của một bộ ghép cặp tốt hơn, không cần bộ ghép cặp nào. Dùng pose chân trị để chiếu ngược và loại mọi tương ứng sai, rồi chạy lại. Con số thu được là mức tốt nhất mà *bất kỳ* bộ ghép cặp nào cũng không vượt qua được. Gọi nó là c : mọi paper hứa hẹn về ghép cặp đều bị chặn trên bởi đúng c , bất kể bảng của họ đẹp tới đâu. Và c chỉ có nghĩa khi nó là số bạn tự đo trên chuỗi của mình.

→ **trường hợp riêng của E/24** — oracle experiment ở chương 24, áp cho một hệ SLAM: nó biến câu hỏi “nên đầu tư vào đâu” thành một phép đo chạy trong một ngày

Có một điều kiện trượt thứ ba, ít trực giác hơn nhưng rất hay xảy ra: thất bại giải thích được bằng một lỗi cấu hình. Trước khi kết luận rằng frontend yếu, hãy kiểm tra ngưỡng, số tầng kim tự tháp, chính sách chọn keyframe, và thời gian “vừa mới định vị lại”. Sửa những thứ đó không tốn gì cả, và chúng giải thích một phần lớn các thất bại nghe có vẻ cần học sâu.

B. Cổng 2 – đo được không, metric nào, sần nhiều bao nhiêu

Giả sử vấn đề có thật. Câu tiếp theo: bạn sẽ nhận ra cải thiện bằng cách nào? Cổng này có ba yêu cầu.

- **Metric đầu cuối, không phải metric trung gian.** Số điểm khớp, tỉ lệ inlier, độ chính xác ghép cặp đều là đại lượng trung gian. Chúng có thể tăng mà APE không nhúc nhích, vì bộ tối ưu vốn đã loại được phần lớn phép đo xấu. Nếu bạn định dùng metric trung gian thì phải đo trước mỗi liên hệ của nó với APE trên chính hệ của bạn.
- **Tỉ lệ hoàn thành chuỗi báo cùng APE.** Đây là cái bẫy đặc thù của SLAM. Một hệ mất bám giữa chừng chỉ được chấm trên đoạn nó còn sống, mà đoạn đầu của một chuỗi hầu như luôn là phần dễ. APE của nó vì thế đẹp một cách vô nghĩa, và càng mất bám sớm thì trông càng đẹp.
- **Sần nhiều đo trước.** Nhánh của bạn phi tất định: thứ tự duyệt các container đánh chỉ mục bằng con trỏ và tranh chấp giữa luồng bám và luồng dựng bản đồ đều làm kết quả đổi giữa các lần chạy. Vì thế mọi so sánh phải chạy nhiều lần, và “nhiều” là một con số tính ra được từ độ lệch chuẩn giữa các lần chạy.

Tiêu chí trượt của cổng này rất gọn: nếu mức cải thiện mà phương pháp hứa nhỏ hơn sần nhiều bạn đã đo, thì bạn không có cách nào xác nhận nó, và việc tích hợp là một canh bạc chứ không phải một quyết định kỹ thuật.

Ví dụ tính tay, chạy trong đầu được. Mọi con số dưới đây là số *giả định* để thấy hình dạng của công thức, không phải số đo của một hệ nào. Giả sử bạn đo sần nhiều và được $\sigma = 0,6$ cm APE giữa các lần chạy. Bạn muốn phát hiện một cải thiện thật $\Delta = 1,0$ cm. Công thức cỡ mẫu quen thuộc cho số lần chạy mỗi cấu hình:

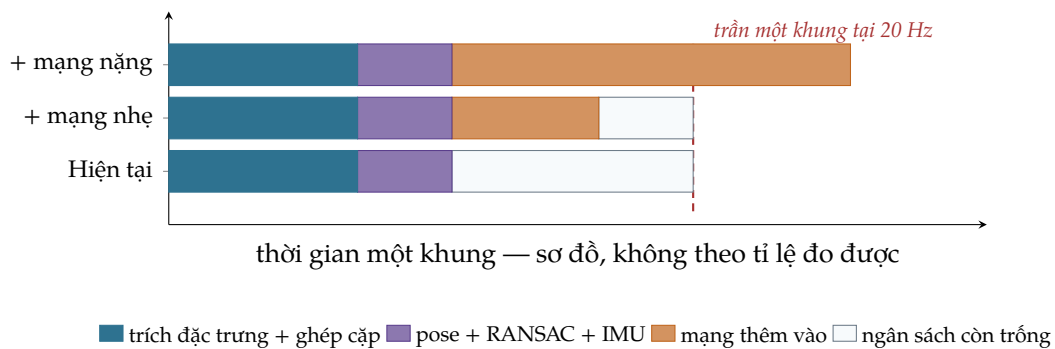
$$n \approx \frac{2(z_{1-\alpha/2} + z_{1-\beta})^2 \sigma^2}{\Delta^2} = \frac{2(1,96 + 0,84)^2 (0,6)^2}{(1,0)^2} \approx 5,6.$$

Tức khoảng sáu lần chạy mỗi bên, mười hai lần cho một so sánh. Bây giờ thử $\Delta = 0,4$ cm: mẫu số nhỏ đi hơn sáu lần, nên n vọt lên khoảng 35 mỗi bên. Hình dạng đó nói thẳng một điều rất hữu ích, và nó đúng bất kể σ của bạn bằng bao nhiêu: số lần chạy tăng theo *bình phương* tỉ số giữa sần nhiều và mức cải thiện cần phát hiện. Nghĩa là có một ranh giới cứng — cải thiện nhỏ hơn sần nhiều chừng một nửa thì nằm ngoài tầm ngân sách thí nghiệm của bạn, và một paper hứa hẹn ở mức đó đã trượt cổng 2 trước cả khi bạn tải mã nguồn về. Ranh giới cứng đó nằm ở đâu thì bạn thay σ đo được của mình vào công thức trên là ra.

C. Cổng 3 – có vừa ngân sách Jetson Orin không

Cổng này là số học, và chính vì là số học nên nó loại được rất nhanh. Bắt đầu từ tốc độ khung hình. Camera EuRoC chạy 20 Hz, tức 50 mili-giây mỗi khung [210]. Đó là toàn bộ ngân sách của luồng bám, và ngân sách đó đã có người ở: trích đặc trưng trên hai ảnh, ghép cặp, RANSAC, tối ưu pose. Phần còn trống là phần bạn được tiêu.

Hình 27.3 vẽ *hình dạng* của phép tính đó và cố ý không mang một con số nào. Nó chỉ phát biểu ba điều: ngân sách là một cái hộp có kích thước cố định, cái hộp đó đã có người ở, và một mạng thêm vào thì hoặc lọt vào phần trống hoặc đẩy cả luồng bám vượt trần. Việc đầu tiên khi qua cổng này là điền số đo thật của chính bạn vào từng dải.



Hình 27.3. Hình dạng của phép tính ngân sách ở cổng 3. Trục không có vạch số và các dải không phải số đo của bất kỳ máy nào — chúng chỉ nói rằng cái hộp có kích thước cố định và đã có người ở. Ba điều đáng mang đi là: một mạng đủ nhẹ thì lọt vào phần còn trống; một mạng nặng thì đẩy luồng bám vượt trần và hệ bắt đầu bỏ khung; và phần còn trống là đại lượng bạn phải *đo* trên Orin, ở đúng chế độ điện và sau khi máy đã nóng, trước khi nói bất cứ điều gì về ứng viên.

Bốn cái bẫy làm phép tính này sai lệch, và cả bốn đều đẩy về hướng lạc quan.

- **Số FPS trong paper là số thông lượng.** Nó đo với batch lớn, ảnh đã nằm sẵn trên GPU, và thường không tính khâu tiền xử lý. SLAM chạy batch bằng 1 và phải trả cả chi phí chuyển dữ liệu. Chênh lệch giữa hai chế độ này không phải chuyện xê dịch vài phần trăm: nó lớn tới mức một kết luận rút ra từ FPS trong bảng hoàn toàn có thể đảo chiều khi bạn tự đo.
- **GPU của bạn có thể đã bận.** Nếu bạn đã chuyển trích đặc trưng sang GPU, thì mạng mới không chạy trên một GPU rảnh mà tranh chấp với chính phần đó.
- **Bộ nhớ dùng chung.** Trên Orin, CPU và GPU chia nhau cùng vùng nhớ. Một mô hình chiếm vài gigabyte không chỉ tốn bộ nhớ của nó, nó lấy đi chỗ của bản đồ.
- **Nhiệt.** Số đo trong ba mươi giây đầu không phải số đo lúc vận hành. Chu kỳ trôi khi máy nóng, và một hệ vừa đủ 50 ms lúc nguội sẽ bỏ khung sau mười phút.

Có một phép sàng nhanh giúp loại ứng viên *trước* khi cài đặt bất cứ thứ gì, và nó chỉ cần hai đại lượng. Tử số: số phép nhân cộng mà mạng cần cho một ảnh — đọc từ bảng của paper, hoặc đếm bằng công cụ. Mẫu số: thông lượng mà máy của bạn *thật sự* đạt được ở batch bằng 1, ở độ chính xác bạn định dùng, trên đúng dạng phép toán đó. Chia tử cho mẫu ra một khoảng thời gian, rồi nhân đôi nếu là ảnh stereo.

Toàn bộ chỗ dễ sai nằm ở mẫu số, và nó sai theo một hướng cố định là hướng lạc quan. Hiệu năng đỉnh ghi trên tờ thông số là con số đạt được ở batch lớn với ma trận lớn; ở batch bằng 1, phần lớn thời gian trôi vào việc chờ dữ liệu chứ không vào việc nhân, và với các phép toán ma trận nhỏ thì thứ chặn là băng thông bộ nhớ chứ không phải số nhân. Vì thế mẫu số phải là một con số bạn đã *đo* bằng một mạng nhỏ đã biết trên chính máy đó, không phải một con số bạn tra được.

Giá trị của phép chia này không nằm ở kết quả mà ở bậc độ lớn của nó, và nó cho ba phán quyết chứ không cho một con số. Nếu kết quả lớn hơn phần ngân sách còn trống nhiều lần thì trượt, khỏi cần cài đặt. Nếu nó cùng bậc với phần còn trống thì chưa kết luận được gì, và đáng bỏ một ngày ra đo thật. Nếu nó nhỏ hơn hẳn thì phép chia này đã hết việc, và rủi ro chuyển sang ba cái bẫy còn lại ở trên: tranh chấp GPU, bộ nhớ dùng chung, và nhiệt.

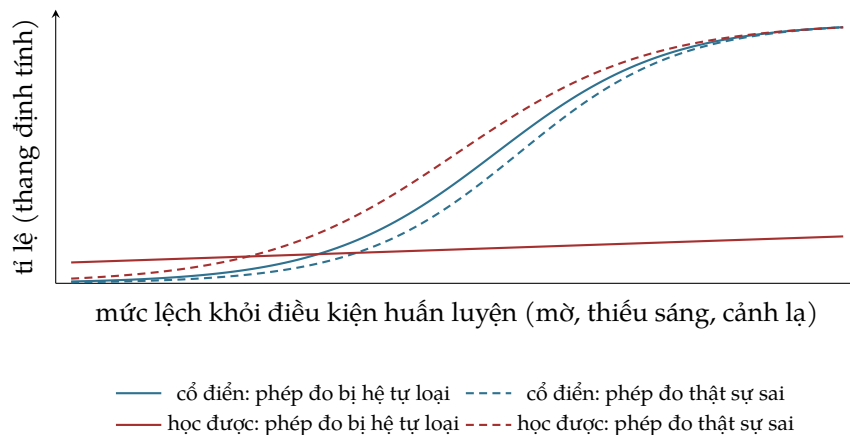
↪ **ràng buộc bởi B/11** — đây chính là chỗ “bộ nhớ và băng thông, không phải FLOPs, mới là thứ chặn” của chương 11, xuất hiện dưới dạng một ngân sách mili-giây cứng

D. Cổng 4 – hỏng ra sao khi ra ngoài phân phối

Ba cổng đầu nói về lợi ích và chi phí. Cổng này nói về rủi ro, và với một hệ SLAM nó quan trọng hơn cả ba cổng kia cộng lại.

Lý do nằm ở cấu trúc, không nằm ở độ chính xác. SLAM là một vòng phản hồi. Phép đo đi vào bản đồ, bản đồ quyết định phép đo tiếp theo được ghép với cái gì. Một phép đo sai không chỉ làm sai một khung; nó ở lại trong bản đồ. Với một vòng đóng sai, hậu quả gần như không sửa được: cả một đoạn quỹ đạo bị kéo lệch vĩnh viễn.

Bây giờ so sánh hai kiểu hỏng. Khi ORB gặp một bức tường trắng, nó không tìm được góc nào. Số điểm về gần 0, RANSAC thất bại, hệ thống tuyên bố mất bám và chuyển sang định vị lại. Thất bại đó *ồn ào*: nó tự báo cáo. Khi một mạng gặp bức tường trắng, nó vẫn trả về đúng số điểm bạn yêu cầu, kèm điểm tin cậy trông bình thường. Thất bại đó *im lặng*, và im lặng là chỗ chết người trong một vòng phản hồi.



Hình 27.4. Hai kiểu hỏng, vẽ theo dạng định tính chứ không phải theo số đo. Với khối cổ điển, hai đường gần như trùng nhau: cái gì sai thì cũng bị chính hệ thống loại, nên bạn nhìn số inlier là biết. Với khối học được, hai đường tách ra: tỉ lệ sai tăng trong khi hệ thống vẫn nhận vào bình thường. Khoảng cách giữa hai đường đồ chính là lượng phép đo sai chảy thẳng vào bản đồ mà không ai chặn.

TRỰC GIÁC

Hãy nghĩ về bộ tối ưu như một quan toà và các phép đo như nhân chứng. Ma trận thông tin là mức tin cậy toà dành cho từng nhân chứng, và RANSAC là phần đối chất. Một bộ dò đặc trưng cổ điển là nhân chứng thật thà kiểu cũ: hỏi tới chỗ không thấy thì nói thẳng “tôi không biết”. Một mạng chưa được hiệu chỉnh là nhân chứng ăn nói lưu loát, không bao giờ nói “tôi không biết”, và trả lời tự tin cả về những việc chưa từng chứng kiến. Vấn đề không phải nhân chứng ấy nói sai nhiều hơn. Vấn đề là toà mất khả năng biết lúc nào nên giảm mức tin cậy, vì tín hiệu tự tin đã không còn mang thông tin.

Từ đó ra hai tiêu chí trượt của cổng 4. Thứ nhất: nếu mạng không có một tín hiệu “tôi không biết” dùng được trong lúc chạy, thì nó chỉ nên được cấm ở nơi mà đầu ra của nó *trừ đi* chứ không cộng vào, hoặc phải có một tầng kiểm chứng hình học đứng sau. Thứ hai: nếu chuỗi bạn dùng để đánh giá nằm trong tập huấn luyện của mô hình, thì con số bạn đo được không nói gì về hành vi khi triển khai. Điều này xảy ra thường xuyên hơn người ta tưởng, vì EuRoC và TUM-VI là các tập rất phổ biến.

→ **cùng nguyên lý với B/08** — đây là dịch chuyển phân phối nhìn từ phía hệ thống: cái đặt không phải độ chính xác giảm, mà là mất tín hiệu báo rằng độ chính xác đã giảm

Bảng 27.2. Bốn cổng, phép đo quyết định của từng cổng, và tiêu chí trượt. Cột thứ ba là chỗ nên đầu tư công sức: nó nói bạn phải chạy thí nghiệm nào, chứ không phải đọc thêm paper nào.

Cổng	Câu hỏi	Phép đo quyết định	Tiêu chí trượt
1	Bài toán có thật không	Nhật ký từng khung kèm tần suất; oracle thay khối bằng chân trị để lấy trần đóng góp	Không dựng được nhật ký; trần nhỏ hơn hai lần sàn nhiều; thất bại giải thích được bằng một tham số cấu hình
2	Đo được không	Sàn nhiều từ nhiều lần chạy cùng cấu hình; APE kèm tỉ lệ hoàn thành; kiểm mối liên hệ giữa metric trung gian và APE	Cải thiện hứa hẹn nhỏ hơn sàn nhiều; chỉ có metric trung gian; không báo tỉ lệ hoàn thành
3	Vừa ngân sách Orin không	Hồ sơ thời gian luồng bám trên Orin; độ trễ batch-1 của mạng sau khi tối ưu, đo lúc máy đã nóng	Vượt phần ngân sách còn trống; bộ nhớ lẫn bản đồ; chu kỳ trôi theo nhiệt
4	Hông ra sao khi ra ngoài phân phối	Chạy trên cảnh cố ý khác: thiếu sáng, mờ, hành lang không vắn; đo tỉ lệ phép đo sai lọt qua tầng kiểm chứng	Không có tín hiệu “tôi không biết”; chuỗi đánh giá nằm trong tập huấn luyện; hông im lặng ở chỗ cộng thêm phép đo

E. Chạy thử khung: một ứng viên đi qua bốn cổng

Khung chỉ có nghĩa khi chạy được trên một ví dụ. Lấy ứng viên phổ biến nhất: thay ORB bằng một bộ dò và mô tả học được, kèm một bộ ghép cặp học được. Về vị trí, nó là trụ 1 ở mức can thiệp 2. Dưới đây cố ý không có một con số nào, vì con số phải là của bạn. Thứ đáng học ở ví dụ này là *thứ tự câu hỏi* và *dạng kết luận* mà mỗi cổng cho phép phát biểu.

Cổng 1. Bạn dựng nhật ký và thấy sai số không rải đều mà dồn vào các đoạn xoay nhanh. Bạn chạy oracle ghép cặp: dùng pose chân trị loại mọi tương ứng sai rồi chạy lại. Kết luận có đúng một dạng, và dạng đó là một tỉ số: trần đóng góp lớn hơn sàn nhiều *bao nhiêu lần*. Lớn hơn nhiều lần thì qua cổng. Xấp xỉ bằng thì dừng lại ngay ở đây, và bạn vừa tiết kiệm toàn bộ phần còn lại.

Cổng 2. Bạn báo APE kèm tỉ lệ hoàn thành, và bạn đã có sàn nhiều từ nhiều lần chạy cùng một cấu hình. Bạn cũng ghi chú rằng không được kết luận bằng số điểm khớp: đó là metric trung gian, và cái trần đo được ở cổng 1 mới là thứ đáng so. Qua cổng.

Cổng 3. Đây là chỗ ứng viên gặp khó, và lý do là cấu trúc chứ không phải một con số. Bộ dò chạy trên hai ảnh mỗi khung, bộ ghép cặp chạy thêm một lần nữa mỗi khung, và cả ba khoản đó nằm trong luồng bám, tức là cùng chen vào một ngân sách vốn đã có người ở. Với một mô hình cỡ trung, kết luận thường là trượt — nhưng trượt *có điều kiện*, và điều kiện đó rất cụ thể.

Ở đây có một lối thoát mà phép tính thô che mất. Không phải mọi chỗ dùng ghép cặp đều chạy ở nhịp camera. Tầng ghép cặp trong đường đóng vòng chỉ chạy khi có ứng viên, tức vài lần mỗi phút, nên ngân sách của nó rộng hơn ngân sách một khung của luồng bám nhiều bậc. Vậy cùng một mô hình trượt cổng 3 ở luồng bám lại lọt thoải mái ở đường đóng vòng. Đó là kết luận đáng giá nhất của cả ví dụ này, và nó chỉ hiện ra khi bạn tính ngân sách *theo từng đường*, không tính theo cả hệ.

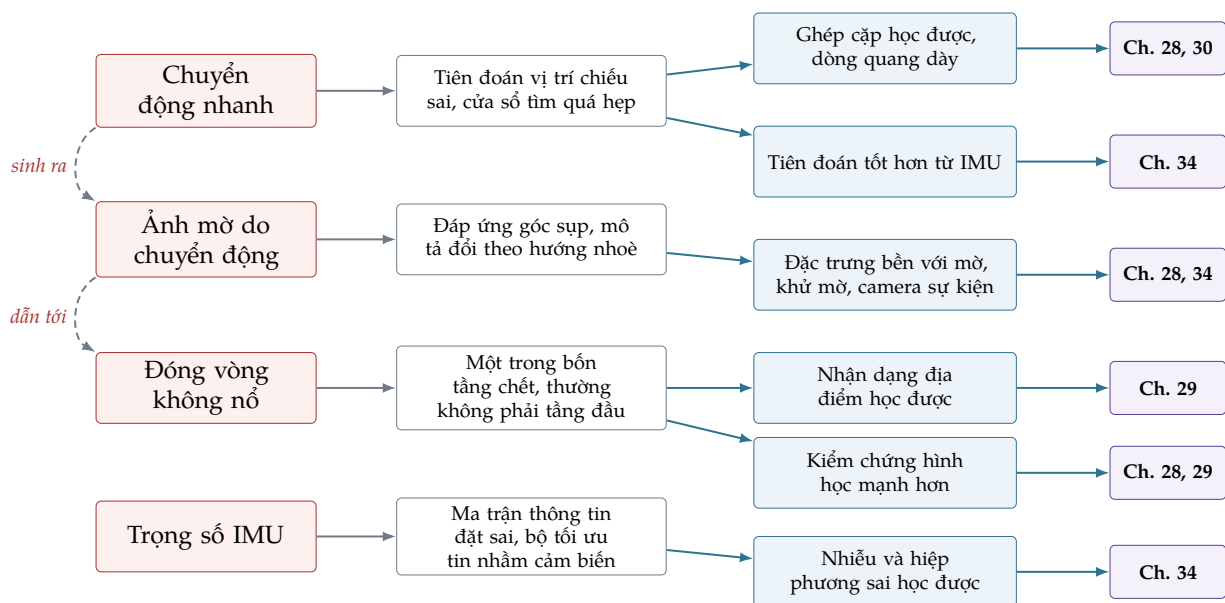
Cổng 4. Bộ dò học được không có tín hiệu “tôi không biết” dùng được. Nhưng ở đường đóng vòng, đằng sau nó vẫn còn tầng kiểm chứng Sim3 bằng RANSAC, tức vẫn còn một trọng tài hình học. Rủi ro vì thế bị chặn: một tương ứng sai muốn vào bản đồ phải qua được kiểm chứng hình học. Điều kiện kèm theo là bạn không được nới ngưỡng inlier để bù cho việc mô hình mới trả nhiều tương ứng hơn.

Phán quyết. Không phải một chữ GO hay NO-GO, mà là một câu có điều kiện: chưa đưa vào luồng bám vì cổng 3, nhưng đáng thử ở đường đóng vòng vì ở đó cổng 3 rộng và cổng 4 đã có người canh. Chi phí thử: cỡ một tuần. Điều kiện xem lại phán quyết đầu tiên: khi độ trễ của luồng bám giảm đủ để mở ra một chỗ trống thật, hoặc khi có một mô hình nhẹ hơn ở cùng chất lượng.

27.4 Bản đồ bốn nỗi đau (Mapping Four Pains to Method Families)

Đây là mục trung tâm của chương. Bốn cổng cho bạn cách loại; mục này cho bạn cách *chọn*. Nó trả lời một câu duy nhất: với mỗi nỗi đau cụ thể, nhóm phương pháp nào hứa chữa, chương nào nói về nó, và điều kiện gì phải đúng thì mới đáng thử.

Trước khi vào bảng, một lời cảnh báo về cấu trúc. Bốn nỗi đau không độc lập. Chuyển động nhanh sinh ra ảnh mờ; ảnh mờ làm mất bám; mất bám làm ta cần định vị lại và cần vòng đóng. Vì thế quy gán một con số APE cho đúng một nỗi đau là việc khó, và cách duy nhất làm được là quy gán theo từng khung. Hình 27.5 vẽ bản đồ ba tầng: nỗi đau, cơ chế hỏng, nhóm phương pháp.



Hình 27.5. Từ nỗi đau sang cơ chế hỏng sang nhóm phương pháp. Cột giữa là cột phải đo trước: nó nói *cái gì* hỏng, và chỉ khi biết điều đó thì cột phải mới có nghĩa. Hai mũi tên đứt bên trái nhắc rằng bốn nỗi đau nối nhau, nên một cải thiện ở nỗi đau đầu có thể làm ba nỗi đau còn lại nhẹ đi mà không cần đụng tới chúng.

A. Chuyển động nhanh

Cơ chế hỏng ở đây không phải “ảnh xấu”. Nó là một chuỗi rất cụ thể. Bộ bám chiếu các điểm bản đồ vào khung mới bằng một tiên đoán pose, rồi tìm mô tả khớp trong một cửa sổ quanh vị trí chiếu. Khi camera xoay nhanh, tiên đoán lệch nhiều hơn bán kính cửa sổ. Số tương ứng tụt, RANSAC còn ít điểm, và một loạt quyết định phía sau đổi theo.

Điều đó cho ra hai hướng chữa hoàn toàn khác nhau, và chúng nằm ở hai trụ khác nhau. Hướng thứ nhất là làm cho việc ghép cặp không cần tiên đoán tốt: ghép cặp học được kiểu SuperGlue hoặc LightGlue dùng ngữ cảnh toàn ảnh, và dòng quang dày kiểu RAFT [167] không cần cửa sổ nào cả. Hướng thứ hai là làm cho tiên đoán tốt hơn, bằng chính IMU mà bạn đã có.

Nhận định thẳng thắn của chương này: hướng thứ hai nên thử trước, vì nó rẻ hơn nhiều bậc. IMU 200 Hz cho một ước lượng xoay rất tốt trên khoảng thời gian 50 mili-giây. Nếu tiên đoán của bạn còn dùng mô hình vận tốc không đổi ở những đoạn xoay nhanh, thì bạn đang bỏ không một cảm biến đã có sẵn. Chỉ khi hướng đó đã cạn mới đáng trả giá độ trễ cho một bộ ghép cặp học được.

B. Ảnh mờ do chuyển động

Ảnh mờ tấn công đúng giả định nền của ORB: rằng một góc là một góc. Khi ảnh nhoè theo một hướng, đáp ứng góc sụp xuống, và cái còn lại giống một cạnh hơn một góc. Tệ hơn, mô tả đổi theo *hướng* nhoè, nên hai ảnh mờ theo hai hướng khác nhau của cùng một cảnh cho hai mô tả khác nhau.

Có bốn nhóm chữa, và chúng khác nhau cả về trụ lẫn về giá. Đặc trưng học được bền hơn với mờ ở mức nào đó, và đó là mảng của chương F/28. Khử mờ trước khi trích đặc trưng là một tầng cộng thêm, tốn đúng ngân sách mà bạn đang thiếu. Hiệu chỉnh chùm tia có ý thức về mờ mô hình hoá chính quá trình phơi sáng, đắt nhưng đúng về mặt vật lý. Camera sự kiện thì bỏ hẳn khái niệm phơi sáng. Ba nhóm sau nằm ở chương F/34.

Điều kiện đáng thử ở đây nghiêm ngặt hơn ta tưởng. Trước tiên phải đo: trên chuỗi của bạn, bao nhiêu phần trăm khung thật sự mờ tới mức làm hỏng việc, và bao nhiêu phần APE quy được về những khung đó. Nếu chỉ một phần rất nhỏ số khung bị mờ nặng và hệ thống vượt qua chúng nhờ IMU, thì cả bốn nhóm trên đều đang chữa một vết thương không chảy máu.

C. Đóng vòng — ví dụ mẫu của cổng 1

Đóng vòng là nỗi đau minh hoạ rõ nhất vì sao khung bốn cổng lại cần thiết. Nó có bốn tầng nối tiếp, và một vòng chỉ đóng khi qua được cả bốn:

1. **Lấy ứng viên** — tìm các keyframe cũ có thể là cùng một chỗ. Cổ điển dùng túi từ trên mô tả nhị phân.
2. **Ghép đặc trưng** — tìm tương ứng giữa khung hiện tại và ứng viên.
3. **Kiểm chứng hình học** — ước lượng Sim3 bằng RANSAC và đòi đủ số inlier.
4. **Tối ưu** — hiệu chỉnh đồ thị pose rồi hiệu chỉnh chùm tia toàn cục.

Bây giờ là chỗ mấu chốt. Phần lớn công trình học sâu về đóng vòng cải thiện tầng 1: NetVLAD [222] và các hậu duệ của nó, hoặc một pipeline phân cấp kiểu HF-Net [223]. Nếu tầng chết trên hệ của bạn không phải tầng 1, thì mọi cải thiện đó là công cốc, và bạn sẽ thấy đúng hiện tượng ở câu hỏi mở đầu: recall tăng mạnh, số vòng đóng không đổi.

I MẠCH 2 — chẩn đoán đóng vòng trước khi chọn phương pháp

- | | |
|---------------------|--|
| 1 Vấn đề | Trên chuỗi dài, vòng gần như không đóng. Triệu chứng dễ thấy nhất là quỹ đạo trôi mà không bao giờ được kéo về. |
| Giải pháp | Đếm số ứng viên sống sót sau <i>từng</i> tầng trong bốn tầng, trên vài chục lần đi qua cùng một chỗ. |
| Giả định | Bốn tầng nối tiếp và độc lập đủ để quy trách nhiệm cho một tầng. |
| Giá phải trả | một buổi thêm bộ đếm và một buổi chạy lại. |
| 2 Vấn đề | Bộ đếm đặt sai chỗ thì số ra vô nghĩa. Nếu bộ đếm nằm trước câu lệnh điều kiện, nó đếm cả những ứng viên chưa từng được xét. |
| Giải pháp | Đặt bộ đếm ngay sau mỗi điều kiện, và ghi cả lý do bị loại. |
| Hệ quả | Đây không phải chi tiết vụn vặt. Một lần đo nội bộ trên chính nhánh này bạn đầu quy sai thủ phạm chỉ vì bộ đếm nằm trước điều kiện, và kết luận đảo ngược sau khi sửa. |
| 3 Vấn đề | Giả sử phép đếm đúng cho thấy phần lớn ứng viên chết ở tầng 3, tức Sim3 RANSAC không hội tụ. |

- Giải pháp** Khi đó tầng đáng đầu tư là tầng 2, vì RANSAC chết do quá ít tương ứng đúng. Ghép cặp học được ở chương F/28 chứa đúng chỗ đó.
- Vấn đề mới** Nói ngưỡng inlier để RANSAC dễ hội tụ hơn là cá mồi lớn nhất và cũng là sai lầm đắt nhất, vì nó mua thêm vòng đóng bằng cách mua thêm vòng *sai*.
- 4 Vấn đề** Một vòng sai đắt hơn một vòng bị bỏ lỡ rất nhiều. Bỏ lỡ chỉ để lại độ trôi; đóng sai kéo lệch cả một đoạn bản đồ và gần như không cứu được.
- Giải pháp** Chọn điểm vận hành lệch hẳn về phía precision, và giữ tầng kiểm chứng hình học kể cả khi tầng lấy ứng viên đã học được.
- Giả định** Chi phí của hai loại lỗi bất đối xứng, và bạn ước lượng được mức bất đối xứng đó.

CẠM BẮY

Cạm bẫy hay gặp nhất khi đọc mẩu này là tin vào một metric của tầng 1 để quyết định cho cả bốn tầng. Recall@1 cao nghĩa là “ứng viên đúng có mặt trong danh sách”. Nó không nói gì về việc ứng viên ấy có đủ tương ứng để Sim3 hội tụ hay không. Hai đại lượng đó chỉ trùng nhau khi tầng 2 và tầng 3 dư sức, mà đó chính là điều bạn đang nghi ngờ. Cách phân biệt rõ nhất: cấp thẳng ứng viên đúng từ chân trị vị trí, rồi đếm xem bao nhiêu ứng viên hoàn hảo đó vẫn không đóng được vòng. Nếu con số đó lớn, tầng 1 không phải thủ phạm.

D. Trọng số cho IMU

Nỗi đau thứ tư ít được nói tới nhất, và theo đánh giá của chương này, nó là chỗ có tỉ số lợi ích trên rủi ro tốt nhất. Lý do rất cấu trúc: mọi thứ ở đây diễn ra *bên trong* bộ ước lượng cổ điển, nên không có khối nào bị xóa đi.

Cơ chế hỏng như sau. Trong đồ thị nhân tử, cạnh IMU mang một ma trận thông tin suy ra từ mật độ phổ nhiễu của con quay và gia tốc kế, lan truyền qua bước tiền tích phân [216]. Các con số đó đến từ bảng thông số cảm biến hoặc từ một lần hiệu chỉnh tĩnh. Chúng không mô tả nhiễu thật lúc thiết bị đang rung, đang nóng, hoặc đang chịu gia tốc lớn. Đặt trọng số quá cao thì bộ tối ưu bám theo IMU và bỏ qua ảnh; quá thấp thì IMU thành vô dụng và bạn mất tỉ lệ tuyệt đối cùng khả năng vượt qua vài khung mù.

Nhóm phương pháp hứa chữa gồm: học mô hình chuyển động quán tính trực tiếp từ chuỗi IMU, khử nhiễu con quay bằng mạng, và học hiệp phương sai theo từng phép đo thay vì dùng hằng số. TLIO [224] là mốc dễ đọc nhất của nhóm đầu. Tất cả nằm ở chương F/34.

Điều kiện đáng thử: bạn phải chỉ ra được rằng trọng số hiện tại *đang* sai. Phép thử rõ nhất là một oracle trên chính siêu tham số đó. Quét hệ số nhân của ma trận thông tin IMU trên một lưới thô, lấy kết quả tốt nhất cho từng chuỗi, và so với giá trị mặc định. Khoảng cách đó là trần đóng góp của mọi phương pháp học trọng số. Nếu nó nhỏ, cả nhóm bị loại ở cổng 1 và bạn tiết kiệm được nhiều tháng.

Bảng 27.3. Bản đồ bốn nỗi đau. Cột “điều kiện để đáng thử” là cột quyết định: nó là phép đo bạn phải chạy trước, và trong phần lớn trường hợp nó sẽ tự loại bớt ba trong bốn dòng.

Nỗi đau	Cơ chế hỏng và cách đo	Nhóm hứa chữa	Chương	Điều kiện để đáng thử
Chuyển động nhanh	Tiên đoán chiều lệch quá bán kính cửa sổ tìm. Đo: vận tốc góc so với số tương ứng còn lại, theo từng khung	Ghép cặp học được, dòng quang đây; hoặc tiên đoán tốt hơn bằng IMU	28, 30, 34	Đã dùng hết thông tin xoay của IMU cho tiên đoán mà số tương ứng vẫn tụt ở đúng các đoạn xoay nhanh
Ảnh mờ	Đáp ứng góc sụp, mô tả đổi theo hướng nhỏ. Đo: chỉ số nhỏ theo khung, đổi chiều với khung bị mất bám	Đặc trưng bền với mờ; khử mờ; hiệu chỉnh chùm tia có ý thức về mờ; camera sự kiện	28, 34	Tỉ lệ khung mờ nặng đủ lớn, và quy được một phần APE về đúng các khung đó
Đóng vòng	Một trong bốn tầng chết. Đo: đếm ứng viên sống sót sau từng tầng, bộ đếm đặt sau điều kiện	Nhận dạng địa điểm học được (tầng 1); ghép cặp học được (tầng 2); kiểm chứng mạnh hơn (tầng 3)	28, 29	Phép đếm chỉ đúng tầng mà nhóm đó chữa. Cập ứng viên từ chân trị để loại trừ tầng 1 trước
Trọng số IMU	Ma trận thông tin đến từ bảng thông số, không mô tả nhiều lúc rung và lúc nóng. Đo: quét hệ số nhân trên lưới thô	Học nhiều và bias IMU; hiệp phương sai theo từng phép đo; khử nhiễu con quay	34	Khoảng cách giữa hệ số tốt nhất theo chuỗi và giá trị mặc định lớn hơn hẳn sản nhiễu

E. Thứ tự ưu tiên đề xuất, và lập luận đằng sau nó

Nếu buộc phải xếp thứ tự khi chưa có bảng trần đóng góp của riêng bạn, chương này đề xuất thứ tự sau, kèm lý do. Đây là phán đoán chứ không phải kết quả đo, nên hãy coi nó là điểm khởi hành và thay bằng số của bạn ngay khi có.

1. **Trọng số IMU trước.** Nó nằm hoàn toàn bên trong bộ ước lượng cổ điển, nên không phải xoá khỏi nào. Phép thử oracle của nó rẻ nhất trong bốn nỗi đau: chỉ là quét một siêu tham số. Và nếu trần lớn, cách chữa đầu tiên thậm chí chưa cần mạng nào.
2. **Đóng vòng thứ hai.** Chi phí một vòng sai rất cao, nhưng phép chẩn đoán bốn tầng lại rẻ và cho kết luận sắc. Nhiều khả năng nó chỉ ra rằng tầng cần chữa không phải tầng mà phần lớn paper chữa, và riêng kết luận đó đã tiết kiệm hàng tháng.
3. **Chuyển động nhanh thứ ba.** Hướng rẻ là dùng tốt hơn IMU đã có, và hướng đó nên cạn trước khi trả giá độ trễ cho ghép cặp học được.
4. **Ảnh mờ cuối cùng.** Không phải vì nó không quan trọng, mà vì nó khó quy trách nhiệm nhất và cách chữa đắt nhất. Nó cũng chông lên chuyển động nhanh, nên một phần lợi ích của nó sẽ tự đến khi bạn xử lý mục thứ ba.

Thứ tự này có một điểm chung đáng chú ý: nó xếp theo *chi phí của phép chẩn đoán*, không xếp theo mức đau. Lý do là ở giai đoạn này bạn đang mua thông tin chứ chưa mua cải thiện, và thông tin rẻ nhất nên mua trước.

27.5 Ba mức can thiệp (Three Levels of Intervention)

Giả sử một ứng viên đã qua bốn cổng. Còn một quyết định nữa, và nó quyết định phần lớn chi phí thật: cắm sâu tới mức nào. Có ba mức, và chúng khác nhau không phải ở độ khó viết code mà ở *thứ bạn phải từ bỏ*.

A. Mức 1 — bổ trợ

Mạng chỉ cấp thêm một tín hiệu; đường cổ điển vẫn chạy và vẫn quyết định. Ba ví dụ điển hình: một mặt nạ vật thể động chỉ dùng để *loại* phép đo; một hiệp phương sai học được chỉ dùng để *cân lại* phép đo đã có; một tiên đoán học được chỉ dùng để *thu hẹp* cửa sổ tìm.

Tính chất định nghĩa của mức này là: nếu mạng chết hoặc trả rác, hệ thống suy biến về đúng baseline cổ điển. Điều đó cho bạn hai thứ vô giá. Thứ nhất là một công tắc: bật và tắt để so sánh, trên cùng một lần chạy nếu muốn. Thứ hai là một trần thiết hại: điều tệ nhất có thể xảy ra là mất một ít phép đo tốt.

B. Mức 2 — thay một khối

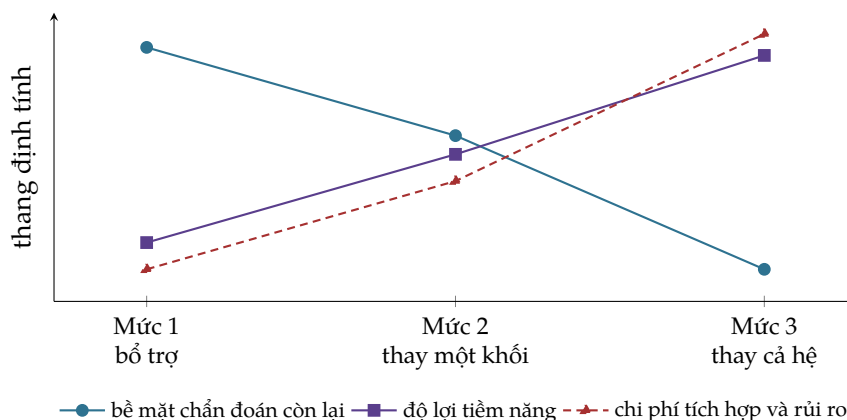
Mạng thay hẳn một khối cổ điển, nhưng giữ nguyên giao diện của khối đó. Trích đặc trưng học được thay ORB; nhận dạng địa điểm học được thay DBoW; độ sâu học được thay stereo. Backend không đổi một dòng.

Cái phải từ bỏ ở đây thường bị đánh giá thấp. Bạn không chỉ bỏ đoạn code cũ. Bạn bỏ luôn toàn bộ tinh chỉnh đã gắn với nó, bỏ tập các chế độ hỏng mà bạn đã thuộc lòng, và bỏ khả năng chạy song song hai đường để đối chứng lúc gỡ lỗi. Bề mặt chẩn đoán vẫn còn — vẫn có điểm, vẫn có tương ứng, vẫn có inlier — nhưng mỗi triệu chứng bây giờ có hai chỗ trú, và tách chúng ra cần một oracle chứ không cần một câu lệnh in.

C. Mức 3 — thay cả hệ

Mạng thay cả chuỗi ước lượng. Đây là trụ 3, và nó không phải một cuộc tích hợp mà là một cuộc chuyển hệ. Bạn được một hệ thống mới với các chế độ hỏng mới, và mất những thứ mà một hệ trưởng thành mang theo: quản lý bản đồ tăng dần, nhiều bản đồ, định vị lại, khả năng hỏi hiệp phương sai, và nhiều năm tinh chỉnh ngưỡng.

Nói thế không có nghĩa mức 3 luôn sai. Nó là lựa chọn đúng trong hai tình huống. Thứ nhất, khi bạn đang bắt đầu từ con số không trên một miền mà hệ đặc trưng thừa vốn đã không chạy nổi. Thứ hai, khi bạn cần bản đồ dày cho một mục đích khác và độ chính xác quỹ đạo chỉ là yêu cầu phụ.



Hình 27.6. Ba mức can thiệp trên một thang định tính, không phải thang đo. Điều đáng chú ý không phải ba đường mà là *khoảng cách* giữa chúng: độ lợi tăng gần tuyến tính, còn chi phí và rủi ro tăng nhanh hơn, trong khi bề mặt chẩn đoán — số đại lượng trung gian bạn còn kiểm tra được bằng một kỳ vọng vật lý — sụt gần như đứng ở mức 3.

D. Quy tắc chọn mức, và vì sao nó là quy tắc một chiều

Quy tắc rất gọn: **không lên mức tiếp theo cho tới khi đã đo được rằng mức hiện tại bị chặn dưới mức bạn cần**. Nó nghe hiển nhiên nhưng gần như không ai làm, vì mức cao hơn luôn hấp dẫn hơn về mặt trí tuệ.

Có một lý do sâu hơn để đi từ dưới lên: **tính đảo ngược**. Mức 1 gỡ ra bằng một cờ cầu hình. Mức 2 gỡ ra trong vài ngày nếu bạn còn giữ đường cũ. Mức 3 thì không gỡ ra được, vì sau sáu tháng thì mọi thứ quanh nó đã được xây theo nó. Chi phí thật của mức 3 không phải chi phí tích hợp mà là chi phí của một quyết định không thu hồi được.

Bảng 27.4. Ba mức can thiệp. Cột “cách quay lại” là cột hay bị bỏ qua nhất khi lên kế hoạch, và nó là cột định giá rủi ro thật của quyết định.

Mức	Cơ chế	Phải bỏ đi cái gì	Cách quay lại	Hỏng ra sao
1. Bỏ trợ	Mạng cấp thêm tín hiệu; đường cổ điển vẫn quyết định	Không bỏ gì; chỉ thêm một tầng lọc hoặc một trọng số	Một cờ cầu hình	Mất một ít phép đo tốt; hệ suy biến về baseline
2. Thay một khối	Mạng thay khối cổ điển, giữ nguyên giao diện với backend	Đoạn code cũ, phần tinh chỉnh gắn với nó, và tập chế độ hỏng đã quen	Vài ngày, nếu còn giữ đường cũ chạy song song	Triệu chứng có hai chỗ trú; cần oracle để tách; hỏng im lặng nếu mạng không biết nói “không biết”
3. Thay cả hệ	Mạng thay cả chuỗi ước lượng, không còn bước hình học tường minh	Quản lý bản đồ, nhiều bản đồ, định vị lại, hiệp phương sai, nhiều năm tinh chỉnh	Gần như không; sau vài tháng mọi thứ quanh nó đã xây theo nó	Không còn đại lượng trung gian để hỏi; sửa lỗi nghĩa là huấn luyện lại

CẦN NHỚ

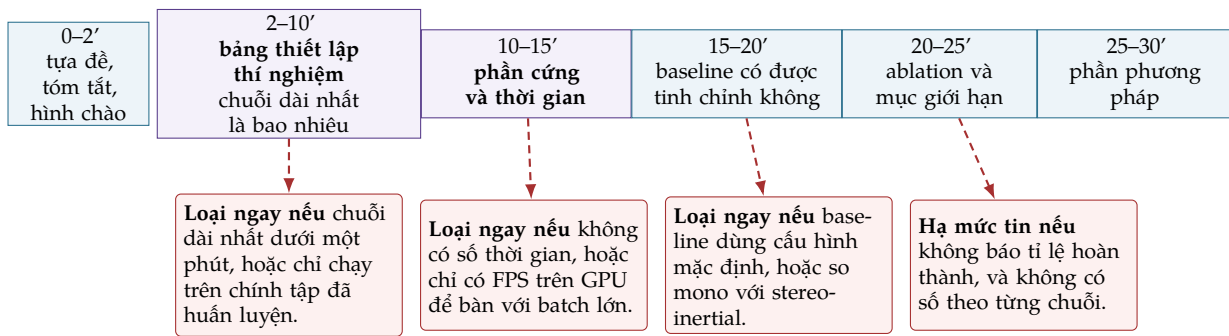
Hai câu quyết định phần lớn các lựa chọn trong Phần F. Thứ nhất: đo trần đóng góp của một khối *trước*, rồi mới đi tìm phương pháp thay khối đó — nếu một khối hoàn hảo cũng chỉ cải thiện được vài phần trăm thì cả nhánh nghiên cứu đó đã bị chặn trên. Thứ hai: trong một vòng phản hồi, một phép đo sai mà im lặng đắt hơn nhiều một phép đo thiếu mà lên tiếng, nên hãy ưu tiên các can thiệp chỉ *trừ đi* phép đo hoặc chỉ *cân lại* chúng. [T1]

I 27.6 Đọc một paper SLAM học sâu trong ba mươi phút (Reading a Deep SLAM Paper in 30 Minutes)

Bốn công và ba mức là khung để quyết định. Mục này là kỹ năng thao tác hằng ngày: mở một paper, và trong ba mươi phút biết nên bỏ hay nên đọc kỹ. Chương E/23-doc-paper đã dạy đọc paper nói chung theo lối ba lượt [44]. Ở đây thứ tự bị đảo lại có chủ ý, vì với mảng này thông tin quyết định nằm ở phần thí nghiệm chứ không ở phần phương pháp.

A. Thứ tự đọc, và vì sao nó ngược

Lý do đảo thứ tự rất đơn giản. Phần phương pháp cho bạn biết tác giả *định* làm gì. Phần thí nghiệm cho bạn biết điều đó đã được kiểm chứng tới đâu, và với mảng SLAM học sâu thì khoảng cách giữa hai thứ ấy thường rất lớn. Một phương pháp thanh lịch chỉ được thử trên chuỗi trong nhà dài một phút không nói gì về hệ của bạn.



Hình 27.7. Thứ tự đọc ba mươi phút, đảo ngược so với thứ tự trình bày của paper. Hai ô tím là hai ô đắt nhất về mặt thông tin và rẻ nhất về mặt thời gian, nên chúng đứng trước. Bốn ô đỏ là các cổng loại: chạm vào một trong ba ô đầu thì dừng đọc, khỏi mở phần phương pháp.

B. Sáu chặng, và câu hỏi của từng chặng

- **0–2 phút, định vị.** Phương pháp này ở trụ nào trong năm trụ, và nó đòi mức can thiệp nào? Chỉ hai câu đó thôi đã đủ để biết bạn phải bỏ cái gì nếu nhận nó.
- **2–10 phút, bảng thiết lập thí nghiệm.** Tập nào, chuỗi nào, và quan trọng nhất: *chuỗi dài nhất họ chạy dài bao nhiêu*. Rất nhiều kết quả ấn tượng của mảng bản đồ neural đến từ các cảnh tổng hợp trong nhà, mỗi cảnh cỡ hai nghìn khung. Một chuỗi EuRoC ngoài đời dài vài phút, và một lần chạy sản phẩm dài hàng chục phút. Cũng ở chặng này: dữ liệu huấn luyện của họ có chứa tập đánh giá của bạn không.
- **10–15 phút, phần cứng và thời gian.** GPU nào, độ trễ hay thông lượng, batch bao nhiêu, và phép đo có tính cả tiền xử lý lẫn chuyển dữ liệu không. Nếu không có mục nào nói về thời gian, hãy coi như phương pháp chưa được thiết kế cho thời gian thực.
- **15–20 phút, baseline.** Họ so với ORB-SLAM3 phiên bản nào, cấu hình nào, có bật đóng vòng không, đơn mắt hay stereo-inertial. So một hệ stereo-inertial học được với một baseline đơn mắt là phép so sánh phổ biến và vô nghĩa.
- **20–25 phút, ablation và mục giới hạn.** Mục giới hạn viết thật là dấu hiệu chất lượng mạnh nhất mà bạn có. Nếu paper có một đoạn nói thẳng phương pháp gây ở đâu, hãy tăng mức tin cậy cho toàn bộ phần còn lại.
- **25–30 phút, phương pháp.** Và chỉ đọc với đúng một câu hỏi trong đầu: *ý tưởng thiết kế nào còn giá trị kể cả khi tôi không dùng hệ này*. Đó thường là thứ duy nhất bạn mang về được.

C. Ba dấu hiệu bên ngoài bài báo

Ba việc kiểm tra sau tốn dưới năm phút và thường quyết định nhiều hơn cả bài báo. Thứ nhất, có mã nguồn chạy được không, và lần cập nhật gần nhất là bao giờ. Thứ hai, mục theo dõi lỗi của kho mã: nếu nhiều người báo cùng một vấn đề tái lập mà không có trả lời, đó là dữ liệu. Thứ ba, có ai độc lập với nhóm tác giả chạy lại được chưa. Trong mảng SLAM, khoảng cách giữa “chạy được trên máy tác giả” và “chạy được trên máy bạn” đặc biệt rộng, vì phụ thuộc vào phiên bản thư viện, dữ liệu và hiệu chỉnh cảm biến.

D. Ba câu hỏi bạn phải tự trả lời sau ba mươi phút

Kết thúc một buổi đọc bằng ba câu, viết ra giấy, không giữ trong đầu. Một: phương pháp này chạm nổi đâu nào trong bốn nổi đầu, và nó chứa *tầng* nào của nổi đầu đó. Hai: nếu nhận nó, tôi phải xoá cái gì khỏi hệ hiện tại. Ba: bằng chứng cụ thể nào sẽ làm tôi đổi kết luận, và bao giờ tôi nên xem lại. Câu thứ ba biến một kết luận “chưa đáng” thành một kết luận có hạn dùng, thay vì thành một định kiến.

E. Bốn câu hỏi mang sang bảy chương sau

Cả Phần F được viết theo đúng bốn câu hỏi dưới đây, và bạn nên đọc mọi phương pháp bằng chúng. Chúng là bản rút gọn của cả chương này thành một khuôn dùng lại được.

1. **Nó giải quyết việc gì?** Phát biểu bằng một thất bại cụ thể của hệ cổ điển, có chuỗi và có tần suất. “Học đặc trưng tốt hơn” không phải câu trả lời; “giữ được tương ứng ở các đoạn xoay nhanh, nơi hệ hiện tại tụt xuống dưới ngưỡng inlier” thì có.
2. **Cơ chế nào làm được điều đó?** Một đoạn, nói đúng thiết kế cốt lõi. Nếu bạn không tóm được cơ chế trong một đoạn thì hoặc paper viết chưa rõ, hoặc bạn chưa hiểu, và cả hai trường hợp đều nên dừng lại.
3. **Có đáng làm không?** Đây là bốn cổng: chi phí độ trễ trên Orin, lượng dữ liệu cần, cách nó hỏng khi ra ngoài phân phối, và — câu hay bị quên nhất — *cái gì trong hệ cổ điển phải bị bỏ đi để nhận nó*.
4. **Nếu đi sâu thì học được gì từ thiết kế của nó?** Bài học còn giá trị kể cả khi bạn không dùng phương pháp. Đây thường là thứ duy nhất bạn mang về được, và cũng là lý do đọc tiếp bảy chương sau dù phần lớn phán quyết là “chưa đáng”.

Câu thứ ba là câu mà chương này phục vụ. Câu thứ tư là câu giữ cho việc đọc không biến thành một chuỗi từ chối, và nó là lý do bảy chương sau vẫn đi vào cơ chế chứ không dừng ở kết luận.

27.7 Giới hạn và trường hợp hỏng

Khung trong chương này có giới hạn riêng, và một chương bộ lọc mà không tự soi thì chính nó thành cái bẫy.

Nó lệch về phía từ chối. Bốn cổng tối ưu cho việc không phí công, chứ không tối ưu cho việc không bỏ lỡ. Một khung loại hết mọi thứ cũng là một khung hỏng, chỉ là hỏng theo cách khó thấy. Thuốc giải là giữ một danh sách theo dõi ngắn: mỗi ứng viên bị loại kèm đúng một dòng ghi bằng chứng nào sẽ lật kết luận, rồi chạy lại khung mỗi năm một lần.

Trần đóng góp giả định các khối tách rời được. Thay một khối bằng chân trị làm đổi phân phối đầu vào của khối đứng ngay sau. Trong một hệ có vòng phản hồi giữa bản đồ và quyết định keyframe, “đóng góp của khối X” không còn là một đại lượng xác định. Đây đúng là giới hạn đã nêu ở chương E/24, và ở SLAM nó nặng hơn vì vòng phản hồi chặt hơn.

Phép tính ngân sách giả định pipeline nối tiếp và bị chặn bởi độ trễ. Nếu CPU đang là nút thắt còn GPU rảnh, một mạng có thể gần như miễn phí về thời gian tường. Ngược lại, nếu bạn đã đẩy trích đặc trưng lên GPU thì phép cộng đơn giản ở hình 27.3 sẽ cho kết luận sai theo hướng lạc quan.

Bốn nỗi đau không độc lập. Quy một con số APE về đúng một nỗi đau đòi hỏi quy gán theo từng khung, mà việc đó chưa có phương pháp gọn. Trong thực tế bạn sẽ phải sống với những kết luận kiểu “phần lớn sai số tập trung ở một đoạn vài chục giây, nơi có cả xoay nhanh lẫn ảnh mờ”, và chấp nhận rằng tách hai nguyên nhân đó cần một thí nghiệm riêng.

Mọi phán quyết trong Phần F đều có hạn dùng. Chúng chốt tại tháng 9/2026 và nói về *trạng thái bằng chứng*, không nói về giới hạn của cái khả dĩ. Mô hình nền hình học đang chuyển động nhanh, và một kết luận “chưa đáng” của hôm nay hoàn toàn có thể sai trong mười hai tháng.

Khung này được hiệu chỉnh cho người đã có hệ thống. Nếu bạn đang bắt đầu từ đầu, trên một cảm biến hoặc một miền mà hệ đặc trưng thừa vốn không chạy nổi, thì thứ tự ưu tiên đảo lại: một hệ đầu-cuối có thể là con đường nhanh nhất tới thứ chạy được, và tính chẩn đoán được là món xa xỉ bạn mua sau.

27.8 Thực hành

Repo và tài nguyên. Dựng lại nền so sánh trước khi thử bất cứ thứ gì: [UZ-SLAMLab/ORB_SLAM3](#) làm baseline, [MichaelGrupp/evo](#) cho APE và RPE, và một harness chạy lập có ghi seed. Để thử nhanh phía học được mà không phải viết lại pipeline: [cvv/Hierarchical-Localization](#) cho tuyến đặc trưng và nhận dạng địa điểm, [cvv/glue-factory](#) cho ghép cặp, [kornia](#) cho các phép hình học khả vi. Phía triển khai: TensorRT và các công cụ đo trên Jetson. Bản đồ hướng của mảng này thì xem kho [changh95/visual-slam-roadmap](#). Danh sách tên mô hình dẫn đầu chốt tại tháng 9/2026 và sẽ cũ trong sáu tới mười hai tháng; hạ tầng như COLMAP, evo, TensorRT thì bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Với hệ hiện tại của bạn, oracle nào nên chạy *đầu tiên*, và nó chặn trên cái gì? Nếu $\text{recall}@1$ của tầng lấy ứng viên tăng mạnh mà số vòng đóng không đổi, ba cách giải thích là gì và phép thử nào tách được chúng? Một mạng báo một tốc độ khung hình rất cao trên GPU để bàn thì con số đó nói được gì — và không nói được gì — về Orin ở batch bằng 1, và bạn sẽ đo phần nó không nói bằng cách nào? Một can thiệp mức 1 và một can thiệp mức 2 cùng hứa cải thiện như nhau: chọn cái nào và vì sao?

Mini project. Hai khối dự án ngay dưới đây là phần thực hành chính của chương; chạy chúng theo thứ tự, vì khối thứ hai dùng lại số đo của khối thứ nhất.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Trụ 1 — frontend nhận thức	Mạng sinh ra phép đo: điểm, mô tả, tương ứng, ứng viên đóng vòng. Giao diện với backend không đổi, nên đây là trụ rủi ro thấp nhất và chín nhất.
••• Trụ 2 — backend tối ưu	Mạng không đổi phép đo mà đổi <i>trọng số</i> của nó: hiệp phương sai học được, mô hình nhiễu IMU, hoặc bộ tối ưu khả vi. Nằm hoàn toàn bên trong bộ ước lượng cổ điển.
••• Trụ 3 — hệ đầu-cuối	Ảnh vào, pose ra, không còn bước hình học tường minh. Được tính bền, mất toàn bộ đại lượng trung gian để chẩn đoán.
•• Trụ 4 — hiểu cảnh	Mạng quyết định phép đo nào được phép vào, thường bằng cách loại điểm trên vật thể động. Nó chỉ trừ đi, nên mạng chết thì hệ về đúng hành vi cổ điển.
•• Trụ 5 — mô hình nền và bản đồ neural	Hai thứ khác nhau bị gộp tên: tiên nghiệm học sẵn cho độ sâu và đặc trưng, và việc đổi bản đồ thành trường liên tục hay tập splat. Cái đầu nhắm độ bền phép đo, cái sau nhắm chất lượng ảnh dựng lại.
••• Trần đóng góp	Mức cải thiện tối đa của một khối, đo bằng cách thay khối đó bằng chân trị rồi chạy cả hệ. Nếu trần là c thì mọi phương pháp cho khối đó đều bị chặn trên bởi đúng c , bất kể bảng của họ đẹp tới đâu — và c phải do bạn đo, không do ai báo.
••• Sàn nhiễu	Độ lệch chuẩn giữa các lần chạy cùng một cấu hình. Nó là đơn vị đo của mọi so sánh; cải thiện nhỏ hơn nó thì không xác nhận được, nên ứng viên đó trượt cổng 2.
••• Tỷ lệ hoàn thành chuỗi	Phần trăm khung hệ thống còn bám được. Phải báo cùng APE, vì một hệ mất bám giữa chừng chỉ được chấm trên phần dễ và vì thế có APE đẹp giả.
•• Batch bằng 1	Chế độ chạy bất buộc của SLAM thời gian thực: mỗi lần đúng một khung. GPU chỉ đạt hiệu suất cao ở batch lớn, nên FPS trong paper thường lệch rất xa thực tế của bạn, xa tới mức không dùng để kết luận được.
••• Hông im lặng	Khối vẫn trả về đầu ra trông bình thường, kèm điểm tin cậy bình thường, trong khi đầu ra đó sai. Nguy hiểm vì SLAM là vòng phản hồi: phép đo sai ở lại trong bản đồ.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Bề mặt chẩn đoán	Số đại lượng trung gian mà bạn còn kiểm tra được bằng một kỳ vọng vật lý: số inlier, sai số chiều ngược, hiệp phương sai. Nó sụt gần như dựng đứng khi lên mức can thiệp 3.
●●● Bốn tầng của đóng vòng	Lấy ứng viên, ghép đặc trưng, kiểm chứng hình học bằng Sim3 RANSAC, rồi tối ưu. Một vòng chỉ đóng khi qua đủ bốn, nên trước khi chọn phương pháp phải biết tầng nào đang chết.
●● Ba mức can thiệp	Bổ trợ (mạng chỉ cấp thêm tín hiệu), thay một khối (giữ nguyên giao diện), thay cả hệ. Chúng khác nhau chủ yếu ở thứ phải từ bỏ và ở khả năng quay lại.
●●● Vì sao đọc bảng thí nghiệm trước phần phương pháp?	Vì phần phương pháp nói tác giả <i>định</i> làm gì, còn bảng thí nghiệm nói điều đó đã được kiểm chứng tới đâu. Với mảng này khoảng cách giữa hai thứ rất lớn, nên đọc ngược tiết kiệm phần lớn thời gian.
●●● Vì sao không suy ngân sách Orin từ FPS trong paper?	Vì FPS là số thông lượng, đo ở batch lớn với dữ liệu sẵn trên GPU. SLAM chạy batch bằng 1, trả thêm chi phí chuyển dữ liệu, chia bộ nhớ với bản đồ, và chậm dần khi máy nóng.
●●● Vì sao vòng đóng sai đắt hơn vòng bỏ lỡ?	Bỏ lỡ chỉ để lại độ trôi, còn sửa được sau. Đóng sai kéo lệch cả một đoạn bản đồ và gần như không cứu được, nên điểm vận hành phải lệch hẳn về phía precision.
●● Vì sao đi từ mức 1 lên, không nhảy thẳng mức 3?	Vì tính đảo ngược. Mức 1 gỡ bằng một cờ, mức 2 gỡ trong vài ngày, mức 3 thì sau vài tháng mọi thứ quanh nó đã xây theo nó. Chi phí thật của mức 3 là chi phí của một quyết định không thu hồi được.
●●● Recall@1 tăng mạnh mà số vòng đóng không đổi — nguyên nhân?	Tầng chết không phải tầng lấy ứng viên. Thường là Sim3 RANSAC không hội tụ vì quá ít tương ứng đúng. Phân biệt: cấp ứng viên đúng từ chân trị vị trí rồi đếm xem bao nhiêu ứng viên hoàn hảo vẫn không đóng được vòng.
●●● APE đẹp bất thường trên chuỗi khó — nghĩa là gì?	Nghi ngay mắt bắm giữa chừng: hệ chỉ được chằm trên đoạn dễ. Phân biệt bằng tỉ lệ hoàn thành chuỗi; nếu nó dưới 100% thì con số APE kia không so sánh được với ai.
●● Mạng löt cồng 3 lúc thử nhưng bỏ khung sau mười phút — nguyên nhân?	Nhiệt và chế độ điện. Chu kỳ trôi khi máy nóng, nên số đo trong ba mươi giây đầu không phải số đo lúc vận hành. Phân biệt: đo lại phân vị 95 sau mười phút chạy liên tục ở đúng chế độ điện.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
hloc [223]	Trụ 1 ở mức 2: thay cả tầng lấy ứng viên lẫn tầng ghép cặp bằng khối học được, giữ nguyên phần kiểm chứng hình học	Bảng chứng rằng mức 2 đã chín cho định vị ngoại tuyến; chưa phải cho vòng bám thời gian thực
DynaSLAM [218] và các nhánh cùng ý tưởng	Trụ 4 ở mức 1: mặt nạ ngữ nghĩa chỉ <i>loại</i> phép đo, không thêm gì vào bộ tối ưu	Ví dụ sạch nhất của can thiệp đảo ngược được bằng một cờ
Kimera [219]	Trụ 4: tầng ngữ nghĩa dựng <i>cạnh</i> bộ ước lượng hình học chứ không thay nó	Kiến trúc đáng học kể cả khi không dùng phần ngữ nghĩa
DROID-SLAM [217]	Trụ 3 ở mức 3: xoá chuỗi đặc trưng thừa, giữ lại một lõi tối ưu khả vi	Mốc tham chiếu của mức 3; ngân sách GPU vượt xa Orin theo cấu hình gốc
MASt3R-SLAM [220]	Trụ 5: mô hình nền hình học làm nguồn tương ứng và độ sâu cho một vòng SLAM	Rất mới; ở đây chỉ nhắc ý tưởng, không dẫn con số
TLIO [224]	Trụ 2: học mô hình quán tính và độ bất định của nó thay vì lấy từ bảng thông số cảm biến	Nhóm gần nhất với nỗi đau trọng số IMU

MINI PROJECT SLAM · Bảng trần đóng góp bốn khối — đo trước khi đọc thêm paper nào · 2 ngày

Mục tiêu	biến câu hỏi “nên đầu tư vào đâu” thành một bảng bốn dòng có số, để ba tháng tới đi vào đúng một chỗ.
Dữ liệu	ba chuỗi EuRoC với độ khó khác nhau, trong đó phải có một chuỗi bạn biết là hay hỏng; cấu hình mặc định của chính nhánh bạn.
Các bước	trước hết đo sàn nhiều: chạy cùng cấu hình mười lần mỗi chuỗi, ghi APE cùng tỉ lệ hoàn thành, lấy độ lệch chuẩn. Rồi chạy bốn biến thể oracle. Một: dùng pose chân trị chiều ngược để loại mọi tương ứng sai, đo trần của ghép cặp. Hai: cấp ứng viên đóng vòng từ khoảng cách vị trí chân trị, đo trần của tầng lấy ứng viên. Ba: quét hệ số nhân của ma trận thông tin IMU trên lưới thô rồi lấy giá trị tốt nhất mỗi chuỗi, đo trần của việc học trọng số. Bốn: bỏ mọi khung có chỉ số nhỏ vượt ngưỡng, đo phần APE quy được về ảnh mờ.
Xong khi	bạn có một bảng bốn dòng, mỗi dòng ghi mức cải thiện trần kèm sai số, và bạn khoanh được đúng những dòng có trần lớn hơn hai lần sàn nhiều. Kiểm tra chéo: tổng bốn trần phải lớn hơn khoảng cách giữa hệ của bạn và một hệ hoàn hảo; nếu nhỏ hơn thì có một nguồn sai số thứ năm bạn chưa gọi tên.
Nối với	E/24 cho công thức cỡ mẫu và cho khái niệm oracle; bảng này là đầu vào bắt buộc của khối dự án tiếp theo và của cả bảy chương sau.

MINI PROJECT SLAM · Ngân sách Orin, đo trước khi viết dòng code tích hợp nào · 1 ngày

Mục tiêu	biến cổng 3 từ một phỏng đoán thành một con số, và biết trước một ứng viên có lọt hay không.
Dữ liệu	chính Jetson Orin của bạn, ở đúng chế độ điện dùng khi vận hành; một chuỗi EuRoC chạy được ít nhất mười phút liên tục.
Các bước	lập hồ sơ thời gian luồng bám theo từng khối: trích đặc trưng, ghép cặp, RANSAC, tối ưu pose, tiền tích phân IMU. Ghi cả trung vị lẫn phân vị 95, vì phân vị mới là thứ làm bỏ khung. Tính phần ngân sách còn trống so với trần 50 mili-giây tại 20 Hz. Rồi lấy một mạng ứng viên, chuyển sang TensorRT ở độ chính xác nửa, và đo độ trễ ở batch bằng 1 trên đúng máy đó. Đo hai lần: lúc máy nguội và sau mười phút chạy liên tục.
Xong khi	bạn phát biểu được một câu có ba con số: “luồng bám đang tiêu X mili-giây ở phân vị 95, còn trống Y , mạng tốn Z ở batch 1 sau khi máy nóng — do đó GO hoặc NO-GO”. Kiểm tra chéo: chênh lệch giữa số lúc nguội và lúc nóng phải khác không; nếu bằng nhau thì bạn chưa đo đúng chế độ điện.
Nối với	D/21-inference-va-trien-khai cho phần chuyển đổi và lượng tử hoá; B/11-gioi-han-tinh-toan cho lý do vì sao batch bằng 1 lại xa peak tới vậy; chương F/34 cho phần đo lại cho trung thực trên phần cứng nhúng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Bạn đã cải thiện tầng không hỏng. Vòng đóng có bốn tầng nối tiếp, và nhận dạng địa điểm chỉ chạm tầng đầu. Lẽ ra phải đếm số ứng viên sống sót sau *từng* tầng trước, với bộ đếm đặt sau câu lệnh điều kiện chứ không phải trước.
- A2. Ba câu: chuỗi dài nhất họ chạy dài bao nhiêu, đo trên phần cứng nào ở batch bao nhiêu, và baseline có được tinh chỉnh không. Câu thứ nhất loại được nhiều paper nhất, vì nó trả lời được trong ba phút bằng cách nhìn đúng một bảng.
- A3. Vì SLAM là vòng phản hồi còn phân lớp ảnh thì không. Một dự đoán sai của bộ phân lớp làm sai đúng một mẫu. Một phép đo sai trong SLAM đi vào bản đồ, rồi bản đồ quyết định phép đo tiếp theo; với một vòng đóng sai thì cả đoạn quỹ đạo lệch vĩnh viễn.
- A4. Vì đó là số thông lượng ở batch lớn, còn bạn chạy batch bằng 1 và bị chặn bởi độ trễ. Thêm vào đó: chi phí chuyển dữ liệu, GPU có thể đã bận với phần trích đặc trưng, bộ nhớ dùng chung với bản đồ, và chu kỳ trôi khi máy nóng. Cách duy nhất là đo trên chính Orin của bạn.

- A5. Bảng trần đóng góp. Thay lần lượt từng khối bằng chân trị rồi đo cả hệ; dòng nào có trần lớn hơn hai lần sàn nhiễu thì đáng, còn lại thì không. Bảng đó chạy trong hai ngày và nó quyết định ba tháng.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn có một bếp ăn chạy tốt nhiều năm. Bạn biết từng vị trí, biết ai chậm ở đâu, biết khi nào món ra trễ thì phải nhìn chỗ nào. Một người bán hàng mang tới cái máy thái rau rất nhanh, đẹp hơn tay người, và có clip chứng minh.

Trước khi mua, bạn hỏi bốn câu. Một: món ra trễ có thật sự vì khâu thái rau không, hay vì cái bếp lò? Cách biết duy nhất là bấm giờ từng khâu trong một tuần. Hai: nếu mua, tôi nhận ra cái thiện bằng gì, khi mà mỗi tối bếp lại đông khác nhau? Ba: cái máy có vừa mặt bàn và vừa cầu dao không, lúc mọi thứ khác cũng đang bật? Bốn: khi gặp một loại rau nó chưa từng thấy, nó dừng lại hay vẫn đẩy ra một mớ trông giống rau đã thái?

Câu thứ tư là câu đắt nhất. Một người thái rau sẽ dừng và hỏi. Cái máy thì không biết dừng, và thứ nó đẩy ra sẽ đi thẳng vào nồi. Trong một cái bếp mà món này nấu tiếp từ món kia, một sai lầm im lặng lan ra khắp bữa ăn.

Rồi còn chuyện lắp sâu tới đâu. Đặt cái máy cạnh bàn cũ thì hôm nào hỏng bạn rút phích. Bỏ hẳn bàn cũ thì phải chờ mua bàn mới. Đập cả bếp đi xây lại quanh cái máy thì không quay về được nữa. Cái giá không nằm ở tiền mua máy. Cái giá là mỗi cái máy lắp thêm là một chỗ bạn không còn nhìn vào bên trong được, khi món ra sai mà không hiểu vì sao.

Chỗ không rút gọn được: chỗ bốn nỗi đau chồng lên nhau. Nói “chứa ảnh mờ thì sai số giảm” nghe gọn nhưng sai, vì đoạn ảnh mờ cũng là đoạn xoay nhanh, và xoay nhanh mới là thứ làm mất bám. Chia phần trách nhiệm giữa hai nguyên nhân cùng xảy ra một lúc thì hiện chưa có cách làm gọn, và mọi cách nói đơn giản về nó đều đang giấu bước đó đi.

Một câu: Trước khi hỏi mạng nào tốt hơn, hãy đo xem khối nào trong hệ của bạn đang thật sự làm hỏng việc, và bạn còn bao nhiêu mili-giây để trả cho nó.

27.9 Kết nối

[B/03-giai-phau-he-thong](#) là tiền đề trực tiếp: năm trụ chỉ là câu hỏi “độ khó đã bị đẩy đi đâu” đặt cho một hệ SLAM, và ranh giới giữa các khối chính là chỗ một mạng chen vào được. [E/24-thi-nghiem-va-ablation](#) cấp hai công cụ mà cổng 1 và cổng 2 dùng lại nguyên vẹn, là oracle experiment và sàn nhiễu. [B/07-chia-du-lieu-va-danh-gia](#) là nơi đọc ý nghĩa của các con số, đặc biệt phần precision và recall cho tầng lấy ứng viên đóng vòng. [B/08-dich-chuyen-phan-phoi](#) là mặt lý thuyết của cổng 4: nó giải thích vì sao độ chính xác tụt khi ra ngoài phân phối, còn chương này quan tâm chuyện khác — vì sao bạn không biết là nó đã tụt. [B/11-gioi-han-tinh-toan](#) và [D/21-inference-va-trien-khai](#) là hai chương phải mở khi làm cổng 3.

Về phía trước, bảy chương còn lại của Phần F đều dùng khung này. [F/28](#) và [F/30](#) là trụ 1. [F/29](#) là nỗi đau đóng vòng, và nó bắt đầu đúng bằng phép đếm bốn tầng của mạch 2. [F/31](#) là trụ 2 và trụ 3. [F/32](#) là trụ 5, đọc với cảnh giác rằng chất lượng ảnh dựng lại không phải độ chính xác quỹ đạo. [F/33](#) là trụ 4. [F/34](#) gom hai nỗi đau còn lại, trọng số IMU và ảnh mờ, cộng phần đưa lên phần cứng. Cuối cùng, [E/26-lo-trinh-hoc](#) cho biết nên đọc bảy chương đó theo thứ tự nào tùy mục tiêu của bạn.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao trần đóng góp phải đo *trước* khi đọc paper, chứ không phải sau khi đã chọn được phương pháp ứng viên?

2. Một can thiệp mức 1 và một can thiệp mức 2 hứa cùng mức cải thiện: bạn chọn cái nào, và lập luận của bạn dựa vào đại lượng nào chứ không dựa vào độ chính xác?
3. Điều gì làm một phép đo sai mà im lặng đắt hơn hẳn trong SLAM so với trong một bài toán phân lớp ảnh, và cơ chế nào của hệ thống gây ra sự khác biệt đó?
4. Bốn tầng của đóng vòng có thể chết ở tầng nào, và với mỗi tầng thì nhóm phương pháp nào chữa đúng chỗ — vì sao chọn nhầm tầng lại cho ra hiện tượng “metric tăng mà kết quả không đổi”?
5. Phép tính ngân sách ở cổng 3 hỏng trong trường hợp nào, và sai lệch của nó đi theo hướng lạc quan hay bi quan?
6. Nếu bốn nỗi đau không độc lập thì việc quy một phần APE về đúng một nỗi đau còn có nghĩa không, và bạn sẽ phát biểu kết luận theo dạng nào để vẫn dùng được?
7. Khung bốn cổng lệch về phía từ chối: cơ chế nào trong quy trình làm việc của bạn ngăn nó biến thành một định kiến lâu dài?

Đặc trưng và ghép cặp học được (Learned Features and Matching)

PHỤ THUỘC F/27-dat-hoc-sau-vao-dau (khung bốn cổng — chương này là bài tập lớn đầu tiên của khung đó); C/16 cho *attention* (SuperGlue và LoFTR đều là *transformer*); E/24-thi-nghiem-va-ablation cho *oracle* experiment, công cụ quyết định của chương này.

TÓM TẮT

Chương này trả lời một câu hỏi rất hẹp: có nên thay ORB bằng một bộ đặc trưng học được không. Câu trả lời cần ba thứ. Thứ nhất là chẩn đoán: ORB gây ở đúng bốn chỗ, và cả bốn đọc được từ cơ chế của FAST và BRIEF. Thứ hai là bản đồ phương pháp: SuperPoint, R2D2, DISK, XFeat, ALIKED cho đặc trưng; SuperGlue và LightGlue cho ghép cặp; LoFTR và RoMa bỏ hẳn bộ phát hiện. Thứ ba là cổng đánh giá, và đây là phần quan trọng nhất. Đọc xong bạn nói được bằng cơ chế vì sao ảnh mờ giết bộ *phát hiện* còn đối độ sáng giết bộ *mô tả*, viết được công thức chi phí bậc hai của SuperGlue theo số điểm, và biết chính xác những gì trong ORB-SLAM3 phải bỏ đi để nhận một bộ đặc trưng mới. Nếu chỉ nhớ một điều: tính lặp lại, độ chính xác ghép và tỉ lệ inlier đều có thể tăng mạnh mà APE không nhúc nhích, nên hãy chạy *oracle* ghép cặp *trước* khi tích hợp bất cứ mạng nào.

KIẾN THỨC CẦN CÓ TRƯỚC

- **FAST** — bộ phát hiện góc của ORB. Nó xét 16 điểm ảnh trên một vòng tròn bán kính 3 quanh điểm p , và nhận p là góc nếu có ít nhất 9 điểm liên tiếp cùng sáng hơn $I_p + t$ hoặc cùng tối hơn $I_p - t$, với t là ngưỡng tương phản [225].
- **BRIEF** — bộ mô tả của ORB. Nó lấy 256 cặp điểm cố định trong một ô ảnh nhỏ quanh góc, mỗi cặp trả về một bit “điểm A sáng hơn điểm B hay không” [226]. ORB thêm bước xoay bộ cặp theo hướng của góc.
- **khoảng cách Hamming** — số bit khác nhau giữa hai chuỗi nhị phân. Với ORB thì hai bộ mô tả 256 bit gần nhau nghĩa là hai ô ảnh trông giống nhau.
- **tính lặp lại (repeatability)** — tỉ lệ điểm được bộ phát hiện tìm thấy ở cùng một vị trí trên vật thật trong hai ảnh khác nhau. Đây là thước đo của riêng bộ phát hiện, chưa dính gì tới bộ mô tả.
- **độ chính xác ghép trung bình (MMA)** — trong các cặp được ghép, tỉ lệ cặp có sai số vị trí dưới một ngưỡng điểm ảnh cho trước. Đây là thước đo chung của cả bộ mô tả lẫn bộ ghép cặp.
- **tỉ lệ inlier** — tỉ lệ cặp ghép sống sót sau khi RANSAC ước lượng mô hình hình học. Nó đo mức sạch của tập ghép cặp đưa vào tối ưu.
- **APE** — sai số vị trí tuyệt đối của quỹ đạo so với chân trị, sau khi căn chỉnh hai quỹ đạo. Đây là con số cuối cùng mà một hệ SLAM bị đánh giá, và là thứ duy nhất trong bốn thước đo trên thật sự quan trọng.
- **vận tải tối ưu (optimal transport)** — bài toán tìm cách chuyển một phân phối khối lượng sang một phân phối khác với tổng chi phí nhỏ nhất. Ghép hai tập điểm ảnh với ràng buộc mỗi điểm dùng một lần chính là một bài toán như vậy.

CÂU HỎI MỞ ĐẦU

- Q1. Ảnh mờ giết FAST qua đúng một đại lượng. Đại lượng đó là gì, những tham số nào của hệ bạn quyết định ngưỡng, và bạn đổi ngưỡng đó sang tốc độ quay bằng phép chia nào?
- Q2. BRIEF chỉ so sánh “A sáng hơn B hay không”, nên nó bất biến với mọi phép đổi độ sáng đơn điệu. Vậy vì sao đổi độ sáng vẫn giết nó trong thực tế?
- Q3. SuperGlue tốn chi phí tỉ lệ N^2 theo số điểm. Hệ SLAM của bạn đã có một dự đoán chuyển động từ IMU. Vì sao dự đoán đó biến N^2 thành gần như tuyến tính, và vì sao không paper nào tận dụng nó?
- Q4. Bạn thay ORB bằng SuperPoint và thấy số cặp ghép đúng tăng mạnh, tỉ lệ inlier cũng tăng theo. Vì sao APE vẫn có thể không đổi một milimét nào?
- Q5. Nếu bỏ bộ phát hiện đi thì bức tường trắng có điểm ghép. Vậy vì sao không hệ SLAM thời gian thực nào bỏ bộ phát hiện?

28.1 Đặt vấn đề

Frontend là khối bị đề nghị thay bằng mạng nhiều hơn mọi khối khác trong SLAM. Lý do dễ hiểu. Nó đứng ngoài cùng, nó chạm trực tiếp vào điểm ảnh, và nó là chỗ mà bạn *nhìn thấy* hệ thống chết. Khi camera quay nhanh và tracking mất, thứ hiện ra trên màn hình gỡ lỗi là số cặp ghép tụt từ vài trăm xuống một nhúm. Kết luận trực giác là frontend yếu.

Kết luận đó đúng một nửa. Frontend đúng là yếu ở bốn chỗ cụ thể, và chương này chỉ ra chính xác bốn chỗ đó bằng cơ chế chứ không bằng cảm giác. Nhưng nửa còn lại quan trọng hơn: frontend hiếm khi là ràng buộc chặt nhất của độ chính xác quỹ đạo. Số cặp ghép chỉ tụt xuống mức nguy hiểm trong những khoảng rất ngắn của một chuỗi dài nhiều phút, và tỉ lệ giữa hai khoảng thời gian đó là con số đầu tiên bạn phải tự đo. Trong phần còn lại, ORB đã cho nhiều cặp ghép hơn mức tối ưu hoá cần.

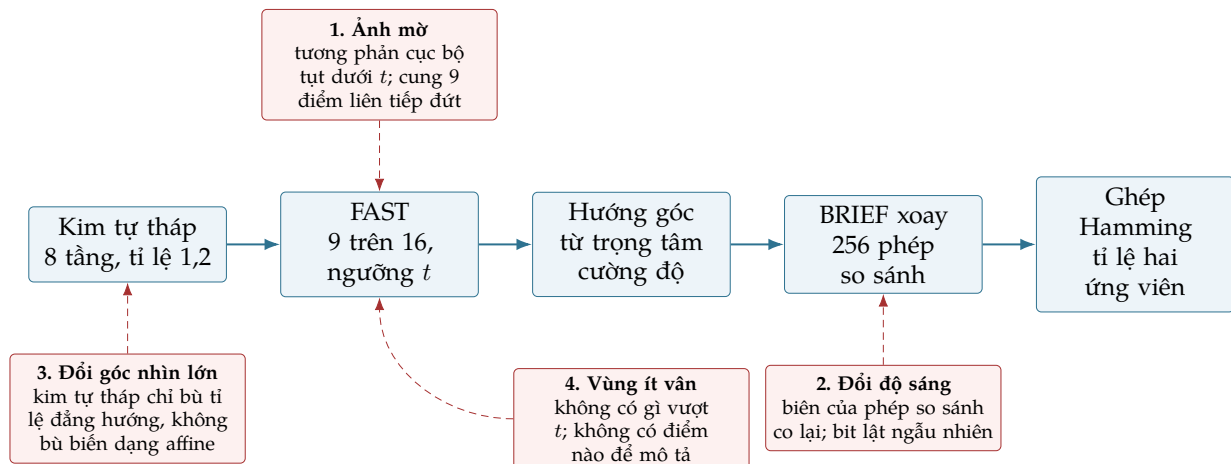
Vì thế câu hỏi đúng không phải “bộ đặc trưng nào tốt hơn”. Câu hỏi đúng là: *trong bao nhiêu phần trăm thời gian của chuỗi, frontend là thứ chặn?* Và trong khoảng thời gian đó, một frontend hoàn hảo thì cứu được bao nhiêu? Chương 24 đã cho công cụ trả lời câu này, đó là *oracle* experiment. Chương này chỉ áp nó vào một khối cụ thể.

Cách trình bày vì thế đi theo thứ tự chẩn đoán rồi mới tới thuốc. Mục 2 mổ xẻ bốn chế độ hỏng của ORB bằng chính công thức của FAST và BRIEF, kèm hai phép tính tay ra con số. Mục 3 và 4 là bản đồ phương pháp, tách làm hai vì đặc trưng và ghép cặp là hai bài toán khác nhau và tiến hoá tách nhau. Mục 5 nói về nhánh bỏ hẳn bộ phát hiện. Mục 6 là cổng đánh giá và là mục nên đọc trước nếu bạn chỉ có mười lăm phút.

→ **tiền đề** F/27 — bốn cổng của chương 27 — bài toán có thật, đo được, vừa ngân sách, hỏng có kiểm soát — là khung mà cả chương này đang điền vào

28.2 Bốn chế độ hỏng của ORB, đọc bằng cơ chế (Four Failure Modes)

ORB là hai thứ ghép lại: một bộ phát hiện góc FAST và một bộ mô tả nhị phân BRIEF có xoay [227]. Bốn chế độ hỏng phân bố không đều giữa hai thứ đó, và biết cái nào hỏng ở đâu quyết định bạn nên thay cái gì. Hình 28.1 đặt cả bốn lên đúng vị trí của chúng trong đường ống.



Hình 28.1. Bốn chế độ hỏng của ORB gắn vào đúng khối mà chúng phá. Hai chế độ đầu tấn công hai khối khác nhau, nên chúng đòi hai thuộc tính khác nhau: mờ giết bộ *phát hiện*, đổi sáng giết bộ *mô tả*. Chế độ 3 là giới hạn của bất biến kim tự tháp. Chế độ 4 không phải lỗi cài đặt mà là hệ quả tất yếu của việc bắt bộ phát hiện quyết định sớm và cục bộ.

A. Ảnh mờ do chuyển động giết bộ phát hiện, và đại lượng quyết định là gì

FAST hỏi một câu hoàn toàn cục bộ: quanh điểm này, có chín điểm ảnh liên tiếp trên vòng tròn bán kính 3 nào lệch hơn ngưỡng t không. Mờ do chuyển động là một phép lấy trung bình dọc theo hướng chuyển động. Trung bình làm giảm biên độ của mọi biến thiên có bước sóng ngắn hơn cửa sổ trung bình.

Mô hình một chiều đủ để thấy hết cơ chế. Gọi cấu trúc ảnh quanh góc có bề rộng w điểm ảnh và biên độ tương phản C mức xám. Làm mờ bằng một cửa sổ dài d điểm ảnh giữ lại xấp xỉ

$$C_{\text{hiệu dụng}} \approx C \cdot \min\left(1, \frac{w}{d}\right).$$

Góc còn được phát hiện chừng nào $C_{\text{hiệu dụng}} > t$. Cho hai vế bằng nhau thì được độ dài vết mờ tới hạn:

$$d^* \approx \frac{wC}{t}.$$

Công thức này nói ba điều, và cả ba dùng được ngay mà không cần một con số nào. Thứ nhất, dung sai mờ tỉ lệ *ngược* với ngưỡng t : hạ ngưỡng thì mua thêm dung sai theo tỉ lệ, chứ không phải bù trừ chút ít. Thứ hai, dung sai tỉ lệ *thuận* với bề rộng cấu trúc, nên cấu trúc thô sống lâu hơn cấu trúc mịn — và đó là lý do khi mờ thì tập điểm còn sót lại toàn góc thô, những góc định vị kém, làm sai số tái chiếu tăng ngay cả khi số cặp ghép chưa tụt. Thứ ba, dung sai tỉ lệ thuận với tương phản, nên cùng một chuyển động sẽ giết bộ phát hiện sớm hơn hẳn trong cảnh thiếu sáng.

Đổi d^* sang đơn vị vật lý cũng chỉ là một phép chia. Một chuyển động quay làm ảnh trượt d điểm ảnh trong thời gian phơi sáng Δt ứng với tốc độ góc

$$\omega \approx \frac{d/f}{\Delta t},$$

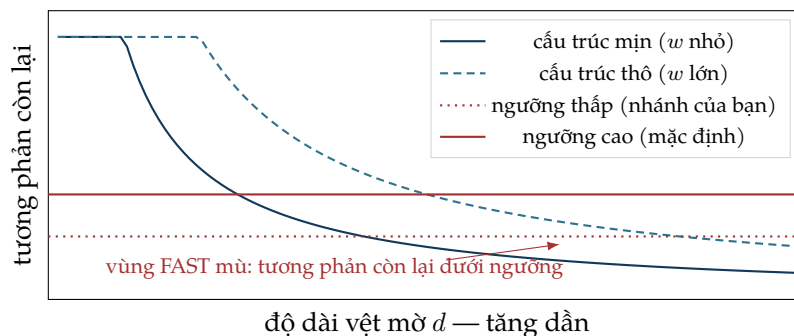
với f là tiêu cự tính bằng điểm ảnh. Ghép hai công thức lại thì ngưỡng tốc độ góc của *hệ bạn* tỉ lệ với $wC/(tf\Delta t)$.

Chương này cố ý dừng ở đó và không đưa một con số. Lý do rất đơn giản: w , C và Δt thật của chuỗi bạn chạy thì mình chưa đo, và một con số chưa đo thì không đáng nằm trong tài liệu học — dán nhãn “ước lượng” cũng không cứu được, vì con số vẫn sẽ được nhớ và được trích lại. Bốn đại lượng ở vế phải đều nằm trong tầm tay bạn: ngưỡng FAST nằm trong cấu hình, tiêu cự nằm

trong file hiệu chỉnh, thời gian phơi sáng nằm trong nhật ký camera, còn w và C thì đo được từ chính ảnh. Thay chúng vào là bạn có ngưỡng của riêng mình trong vòng mười phút, và có nó bằng số đo chứ không bằng lời của ai.

Cái đáng mang đi là một dự đoán định tính, và nó đủ sắc để kiểm chứng được: ngưỡng ấy nằm trong tầm tốc độ quay của một người cầm camera đi bộ bình thường, chứ không phải một tình huống hiểm — nói cách khác, các đợt xoay tay bình thường đã đủ đẩy FAST vào vùng mù, và các chuỗi khó của EuRoC thì vào vùng đó liên tục. Dự án thứ hai ở cuối chương là cách kiểm chứng dự đoán đó trên dữ liệu của chính bạn, và nó chạy trong nửa ngày.

Hình 28.2 vẽ quan hệ đó dưới dạng thuần hình dạng, không một vạch số nào. Điều đáng chú ý nhất trong hình không phải đường cong mà là độ dốc: quanh vùng ngưỡng, giảm t một chút mua thêm được khá nhiều dung sai mờ. Đó chính là lý do hạ iniThFAST từ 20 xuống 12 có tác dụng thật, và cũng là lý do nó có giới hạn — hạ tiếp thì nhiễu bắt đầu qua cửa.



Hình 28.2. Tương phản còn lại sau khi làm mờ, theo mô hình trung bình một chiều. Trục cố ý không có vạch số: hình này nói về *hình dạng*, không về một mức mờ cụ thể nào. Ba điều đọc được từ hình: giao điểm với đường ngưỡng là chỗ FAST ngừng thấy góc; hạ ngưỡng đẩy giao điểm sang phải khá nhiều, nên nó có tác dụng thật; và cấu trúc thô sống lâu hơn cấu trúc mịn, nên khi mờ thì các góc còn sót lại đều là góc thô — chúng định vị kém hơn, và sai số tái chiếu tăng ngay cả khi số cặp ghép chưa tụt. Muốn có con số thì thay w, C, t của chính bạn vào $d^* = wC/t$.

Có một hệ quả tinh tế mà mô hình trên chưa nói hết. FAST không đòi biên độ lớn, nó đòi *chín điểm liên tiếp*. Mờ do chuyển động là bất đẳng hướng: nó xóa tương phản theo hướng chuyển động và giữ nguyên theo hướng vuông góc. Nên với một góc, cung liên tiếp bị cắt đứt ở hai đầu trước khi biên độ trung bình chạm ngưỡng. Nói cách khác, FAST chết sớm hơn mô hình một chiều dự đoán, và chết theo cách phụ thuộc vào hướng của cạnh so với hướng quay. Đó là lý do khi quay ngang thì các cạnh dọc biến mất trước, và tập điểm còn lại bị lệch về một hướng — một tập điểm lệch hướng làm ma trận thông tin của bài toán tư thế bị điều kiện xấu, chứ không chỉ làm ít điểm đi.

B. Đổi độ sáng giết bộ mô tả, qua biên chứ không qua giá trị

Đây là chỗ trực giác thông thường sai. BRIEF không lưu giá trị cường độ; nó chỉ lưu 256 kết quả so sánh dạng $I(a) > I(b)$. Một phép biến đổi độ sáng *đơn điệu tăng* và *toàn cục* — chỉnh gamma, đổi thời gian phơi sáng — không đảo được dấu của bất kỳ phép so sánh nào. Về lý thuyết, BRIEF bất biến hoàn toàn với nhóm biến đổi đó.

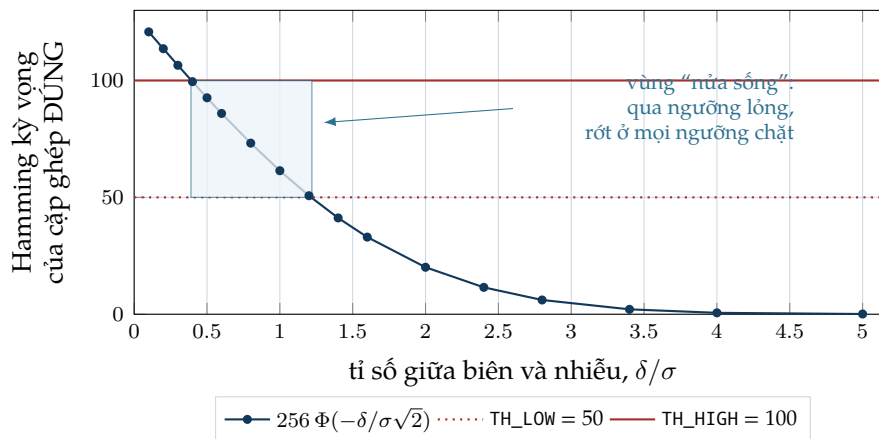
Thứ giết nó là biên. Gọi δ là chênh lệch cường độ thật của một cặp thử, và σ là độ lệch chuẩn của nhiễu trên mỗi mẫu đã làm trơn. Bit lật khi nhiễu đủ lớn để đảo dấu, với xác suất

$$p_{\text{lật}} = \Phi\left(-\frac{\delta}{\sigma\sqrt{2}}\right).$$

Khoảng cách Hamming kỳ vọng giữa hai bộ mô tả của *cùng một* điểm 3D là $256 p_{\text{lật}}$. Đây là đại lượng phải so với hai hằng số mà bạn đã có trong mã: TH_LOW = 50 và TH_HIGH = 100.

Đọc công thức đó theo tỉ số δ/σ thì mọi giá trị tuyệt đối biến mất và cái còn lại mới là thứ dùng được. Khi biên lớn hơn nhiều vài lần, xác suất lật gần như bằng không và khoảng cách Hamming của cặp đúng chỉ vài bit — ghép cặp thoải mái. Khi biên tụt về cùng bậc với nhiễu, Φ bước vào vùng dốc nhất của nó, nên Hamming kỳ vọng nhảy vọt chỉ sau một mức nén tương phản rất nhỏ.

Điều quan trọng không phải nó nhảy bao nhiêu, mà nhảy *vào đâu*. Hai hằng số trong mã chia trực Hamming thành ba vùng, và vùng giữa — trên TH_LOW, dưới TH_HIGH — ứng với một dải δ/σ rất hẹp. Trong dải đó, cặp ghép vẫn qua được mọi hàm dùng ngưỡng lỏng nhưng bị loại ở mọi hàm dùng ngưỡng chặt, tức là tam giác hoá điểm mới và ghép theo túi từ. Hình 28.3 cho thấy toàn bộ đường cong đó; vì trực hoành là một tỉ số không thứ nguyên, hình đúng với mọi mức nhiễu và mọi mức sáng, và bạn chỉ cần biết σ của cảm biến mình để đọc ra ngưỡng δ tương ứng.



Hình 28.3. Vì sao đổi độ sáng giết BRIEF dù BRIEF bất biến với đổi sáng đơn điệu. Đại lượng quyết định là *tỉ số* giữa biên δ và nhiễu σ , không phải giá trị tuyệt đối của độ sáng. Đường cong là hàm $256 \Phi(-\delta/\sigma\sqrt{2})$, tức là toán chứ không phải số đo; hai đường ngang là hai hằng số có thật trong mã nguồn. Điều đáng mang đi: dải δ/σ nằm giữa hai hằng số ấy rất hẹp, nên chỉ cần tương phản nén đi một chút là cặp ghép rơi từ “qua mọi cửa” xuống “chỉ qua cửa lỏng”. Hệ quả chẩn đoán: khi vào vùng thiếu sáng, thứ tụt trước là số điểm bản đồ mới được tam giác hoá, chứ không phải số cặp bám được.

Điều này giải thích một triệu chứng quen thuộc mà trực giác “đổi sáng làm hỏng mô tả” không giải thích được. Khi robot đi vào hành lang tối, tracking vẫn bám nhưng bản đồ ngừng lớn. Nguyên nhân nằm ở đúng khoảng cách giữa hai ngưỡng trong hình.

TRỰC GIÁC

Coi mỗi bit của BRIEF như một đồng xu có thiên lệch. Thiên lệch đến từ tương phản cục bộ: tương phản càng cao, đồng xu càng chắc chắn rơi về một mặt, và chuỗi 256 mặt đó là vân tay của điểm ảnh. Làm mờ ảnh hay nén tương phản không đổi *mặt* nào được ưa; nó chỉ làm mọi đồng xu công bằng dần. Một vân tay gồm toàn đồng xu công bằng thì khớp với tất cả mọi thứ, tức là không khớp với gì cả. Đó là lý do bộ mô tả không “sai” khi ảnh mờ — nó trở nên vô nghĩa, và hai chuyện này cần hai cách chữa khác nhau.

C. Đổi góc nhìn lớn vượt quá bất biến có sẵn

ORB có đúng hai loại bất biến. Bất biến xoay trong mặt phẳng, nhờ xoay bộ cặp thử theo hướng trọng tâm cường độ. Và bất biến tỉ lệ *đẳng hướng*, nhờ kim tự tháp tám tầng hệ số 1,2. Dải tỉ lệ mà kim tự tháp phủ là $1,2^7 \approx 3,6$ lần.

Đổi góc nhìn ngoài mặt phẳng không nằm trong hai loại đó. Nhìn một mặt phẳng nghiêng một góc θ thì ảnh của nó bị nén theo *một* hướng với hệ số $\cos \theta$, còn hướng vuông góc giữ nguyên. Đó là một biến dạng affine bất đẳng hướng. Kim tự tháp co đều cả hai hướng, nên nó không bù được.

Ở $\theta = 45^\circ$ hệ số nén là 0,71; ở 60° là 0,5. Văn liệu về vùng bất biến affine đã đo điều này từ lâu: điểm số ghép của các bộ mô tả không affine sụp rất nhanh khi góc nhìn ra khỏi vùng gần trực diện, trong khi các vùng bất biến affine giữ được lâu hơn hẳn [228]. Ngưỡng cụ thể phụ thuộc bộ mô tả và bộ dữ liệu, nên nó phải đọc từ chính bài đó chứ không mượn lại ở đây.

Với SLAM cầm tay đi liên tục thì chế độ hồng này hiếm, vì hai khung hình liên tiếp cách nhau vài chục mili-giây. Nó chỉ nổ ở đúng một chỗ: **đóng vòng**. Quay lại một hành lang theo hướng ngược lại là đổi góc nhìn tối đa. Đây là lý do đóng vòng và bám khung hình đòi hai loại đặc trưng khác nhau, và là lý do một hệ dùng chung một bộ mô tả cho cả hai việc luôn phải thỏa hiệp.

↪ **tiền đề F/29** — chế độ hồng thứ ba là nguyên nhân gốc của phần lớn vòng bị bỏ lỡ; chương 29 mổ xẻ bốn tầng của đóng vòng để xem tầng nào thật sự giết vòng

D. Vùng ít vân: không phải lỗi, mà là hệ quả của thiết kế

Chế độ thứ tư khác hẳn ba chế độ trên. Ba chế độ đầu là chuyện ngưỡng và bất biến, có thể chỉnh. Chế độ thứ tư là hệ quả logic của việc có một bộ phát hiện.

Bộ phát hiện phải ra một quyết định **cứng**, **sớm** và **cục bộ**: điểm ảnh này có đáng giữ không, quyết bằng thông tin trong một cửa sổ bán kính 3. Trên một bức tường trắng, không cửa sổ nào chứa đủ thông tin để trả lời. Hạ ngưỡng xuống 4 thì cái được chọn là nhiều cảm biến, và nhiều không lặp lại giữa hai khung hình. Không có ngưỡng nào đúng, vì thông tin cần thiết không nằm trong cửa sổ đó — nó nằm ở mép tường cách xa hai trăm điểm ảnh.

Đây là điểm cần nhớ khi đọc mục 5. Bộ phát hiện không phải một thủ thuật kỹ thuật; nó là cách duy nhất để hoãn một quyết định mà thông tin cục bộ không đủ để quyết. Cái giá của việc hoãn thì được nói ở đúng mục đó.

E. Bốn chế độ cộng dồn thế nào trong một giây thật

Bốn chế độ ở trên không xảy ra riêng lẻ. Trong một đợt quay nhanh, chúng nối thành một chuỗi, và biết chuỗi đó giúp bạn đọc nhật ký gỡ lỗi nhanh hơn nhiều.

Bước một, tốc độ góc vượt ngưỡng suy ra từ d^* ở mục A, nên tương phản cục bộ tụt và FAST mất phần lớn góc nhỏ. Bước hai, các góc còn sót lại đều là cấu trúc thô, nên chúng định vị kém hơn và sai số tái chiếu của chúng lớn hơn. Bước ba, các góc còn lại phân bố lệch theo hướng vuông góc với chuyển động, nên ma trận thông tin tư thế xấu đi theo đúng một hướng — không phải xấu đều. Bước bốn, số cặp ghép rơi xuống dưới cổng “dưới 15 cặp thì coi như mất bám”, và hệ chuyển sang chế độ dùng IMU dự đoán.

Điểm đáng chú ý là bước năm. Khi hệ quay lại chế độ bám, nó phải ghép khung hình mới với các điểm bản đồ được tạo *trước* đợt mờ. Các điểm đó có bộ mô tả sắc nét, còn khung hình mới vẫn còn hơi mờ, nên khoảng cách Hamming của cặp đúng nằm trong vùng “nửa sống” của Hình 28.3. Kết quả là hệ bám lại chậm hơn mức đáng lẽ có, và trong khoảng chờ đó IMU tích lũy trôi. Phần lớn sai số quỹ đạo do một đợt quay nhanh gây ra không nằm trong lúc quay, mà nằm trong khoảng *sau* khi quay xong.

Chuỗi này có một hệ quả thực hành ngay. Nếu bạn định can thiệp một chỗ duy nhất, hãy can thiệp ở bước năm chứ không ở bước một — nới ngưỡng ghép cặp *tạm thời* sau một đợt mờ rẻ hơn nhiều so với làm bộ phát hiện chịu mờ giỏi hơn. Đây là suy luận từ cơ chế; mình chưa đo nó, và nó xứng đáng là một thí nghiệm ablation.

Bảng 28.1. Bốn chế độ hỏng, khối bị phá, và thứ thật sự chữa được nó. Cột cuối là cột dùng được nhất: nó cho biết đối bộ đặc trưng có phải là thuốc đúng bệnh không.

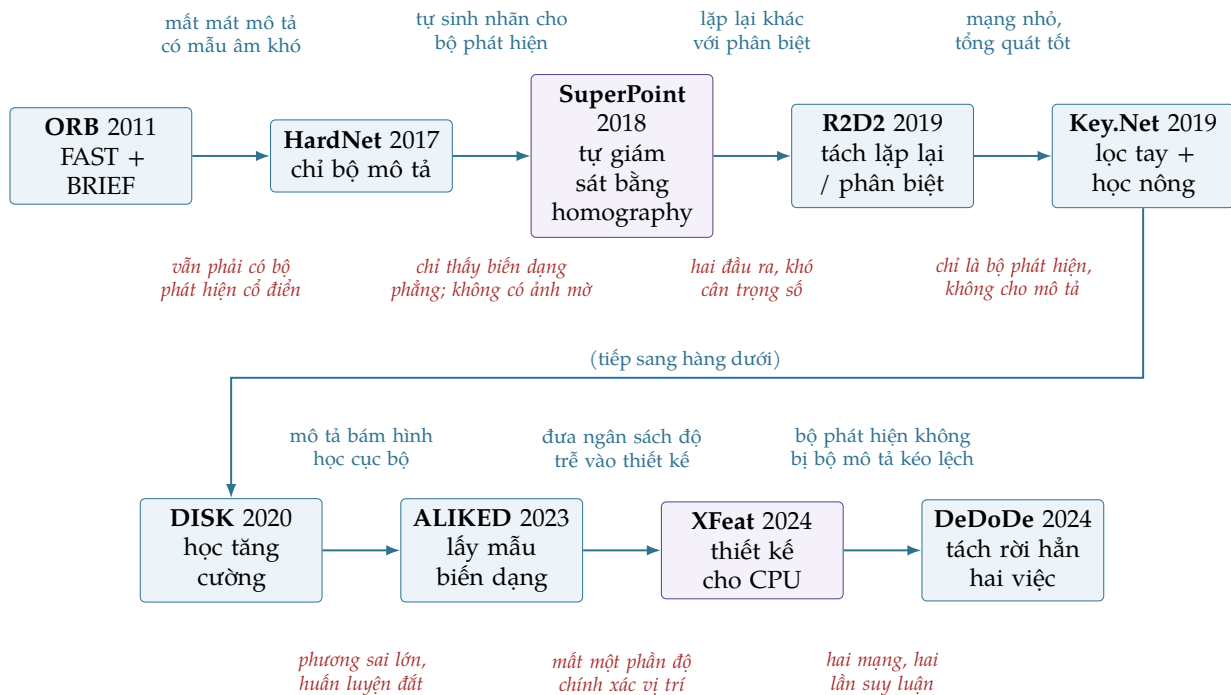
Chế độ	Khối bị phá	Cơ chế	Thuốc đúng bệnh
Ảnh mờ do chuyển động	Bộ phát hiện (FAST)	Trung bình dọc hướng chuyển động kéo tương phản xuống dưới ngưỡng, và cắt đứt cung 9 điểm	Bộ phát hiện học được, hoặc bỏ hẳn bộ phát hiện; giảm phơi sáng nếu đủ sáng; camera sự kiện
Đổi độ sáng	Bộ mô tả (BRIEF)	Biên δ của các phép so sánh co lại, bit lật ngẫu nhiên, Hamming của cặp đúng trôi qua ngưỡng	Bộ mô tả thực học bằng mất mát tương phản (HardNet, SuperPoint)
Đổi góc nhìn lớn	Bộ mô tả và kim tự tháp	Biến dạng affine bất đẳng hướng nằm ngoài bất biến tỉ lệ đẳng hướng	Bộ mô tả học được trên dữ liệu có biến dạng; hoặc bộ ghép cặp có ngữ cảnh toàn cục
Vùng ít vân	Bản thân ý tưởng có bộ phát hiện	Quyết định cứng, sớm, cục bộ trên vùng không có thông tin cục bộ	Ghép cặp dày không cần bộ phát hiện; hoặc chấp nhận và dựa vào IMU

Bảng 28.1 đã trả lời trước một phần câu hỏi của cả chương. Chỉ có hai trong bốn chế độ được chữa bởi việc “đổi ORB sang SuperPoint”. Chế độ thứ nhất được chữa một phần, và chế độ thứ tư thì không — nó cần một loại phương pháp khác hẳn.

28.3 Bộ phát hiện và mô tả học được (Learned Detectors and Descriptors)

Học một bộ mô tả là chuyện dễ hiểu: có nhãn, vì hai ô ảnh của cùng một điểm 3D là một cặp dương. Học một bộ *phát hiện* thì không. Không tồn tại chân trị cho câu “điểm ảnh này có đáng là điểm đặc trưng không”. Toàn bộ lịch sử của dòng này là lịch sử của các cách lách quanh chỗ thiếu nhãn đó, và mỗi cách lách để lại một dấu vết riêng trong cách phương pháp hỏng.

Hình 28.4 xếp cả dòng lên một trục. Mỗi mũi tên ghi nút thắt được gỡ và vấn đề mới sinh ra. Đọc hình trước rồi đọc phần chữ sẽ nhanh hơn hẳn.



Hình 28.4. Dòng đặc trưng học được, đọc theo nút thất chứ không theo năm. Nhấn phía trên mỗi mũi tên là thứ được gỡ; nhấn phía dưới in nghiêng đỏ là vấn đề mới sinh ra. Hai ô tô tím là hai bước ngoặt về *cách nghĩ*: SuperPoint giải quyết chỗ thiếu nhân, XFeat là công trình đầu tiên trong dòng đặt ngân sách độ trễ làm ràng buộc thiết kế thay vì làm ghi chú cuối bài.

A. HardNet: hàm mất mát phải mô phỏng đúng quy tắc quyết định lúc chạy

HardNet chỉ làm nửa dễ, tức bộ mô tả, nhưng nó là công trình có bài học thiết kế chuyển được sang việc khác nhiều nhất [229]. Cơ chế của nó gói gọn trong một câu. Lấy một *batch* gồm n cặp ô ảnh khớp nhau, tính ma trận khoảng cách $n \times n$, rồi với mỗi mô neo lấy *mẫu âm gần nhất trong batch* làm mẫu âm, và tối đa hoá biên giữa cặp dương và mẫu âm đó.

Vì sao chọn mẫu âm khó nhất chứ không lấy ngẫu nhiên? Vì lúc chạy thật, đối thủ của một bộ mô tả không phải một ô ảnh ngẫu nhiên. Đối thủ của nó là láng giềng gần nhất trong toàn bộ ảnh kia. Mẫu âm ngẫu nhiên gần như luôn ở rất xa, nên *gradient* tắt và mạng học được rất ít. Sâu hơn nữa: quy tắc quyết định lúc chạy là *phép thử tỉ lệ* giữa ứng viên tốt nhất và ứng viên nhì. Lấy mẫu âm khó nhất trong batch chính là cách xấp xỉ ứng viên nhì đó ngay trong hàm mất mát.

Bài học chuyển được: **hàm mất mát nên mô phỏng quy tắc quyết định sẽ dùng lúc chạy, không phải một đại lượng dễ tối ưu hơn**. Bạn dùng được bài học này ngay cả khi không dùng mạng nào. Nếu frontend của bạn quyết bằng tỉ lệ giữa hai ứng viên tốt nhất, thì mọi phép đo chất lượng đặc trưng cũng phải đo theo tỉ lệ đó, chứ không đo khoảng cách tuyệt đối.

↔ cùng nguyên lý với B/09 — đây là dạng cụ thể nhất của nguyên tắc “hàm mất mát mã hoá điều bạn thật sự quan tâm” ở chương thiết kế hàm mất mát

B. SuperPoint: chế ra nhân từ một phép biến đổi bạn nghịch đảo được

SuperPoint giải chỗ thiếu nhân bằng ba bước [212]. Bước một, huấn luyện một bộ phát hiện thô trên ảnh tổng hợp gồm các hình khối đơn giản, nơi vị trí góc biết trước theo dựng hình. Bước hai gọi là làm giàu bằng homography: lấy ảnh thật, áp hàng trăm phép homography ngẫu nhiên, chạy bộ phát hiện thô trên từng ảnh đã biến đổi, rồi *ánh xạ ngược* các điểm về ảnh gốc và cộng dồn. Điểm nào sống sót qua nhiều phép biến đổi thì được ghi nhận là điểm giả định. Bước ba, huấn luyện chung một mạng có hai đầu ra — một đầu cho bản đồ điểm, một đầu cho bộ mô tả

256 chiều — với nhân của bộ mô tả lấy từ chính homography đã dùng, vì homography cho biết chính xác điểm nào ứng với điểm nào.

Ý tưởng cốt lõi rất gọn: **khi nhãn không tồn tại, hãy chế nó ra từ một phép biến đổi mà bạn nghịch đảo được chính xác**. Đây là cùng một mạch tư duy với tự giám sát bằng bất biến ở phần C, chỉ khác là phép biến đổi ở đây có ý nghĩa hình học.

Cái giá nằm ở chính chỗ đó, và với người làm SLAM thì đây là điều quan trọng nhất phải biết về SuperPoint. Homography là biến đổi của một *mặt phẳng*. Phân phối huấn luyện vì thế không chứa thị sai do độ sâu, không chứa che khuất, và — điểm chí mạng — **không chứa ảnh mờ do chuyển động** trừ khi bạn tự thêm vào. Nói cách khác, phương pháp được ca ngợi nhiều nhất trong họ này không hề được huấn luyện cho nỗi đau số một của bạn. Nếu định dùng nó cho chuyển động nhanh, việc đầu tiên phải làm là tăng cường dữ liệu bằng mờ có hướng, và đo lại. Đây là một suy luận từ thiết kế của phương pháp, không phải một kết quả đã công bố.

C. R2D2: lặp lại và phân biệt là hai chuyện khác nhau

R2D2 chỉ ra một lỗ hổng khái niệm mà cả ORB lẫn SuperPoint đều mắc [230]. Một điểm có thể rất *lặp lại* mà hoàn toàn vô dụng. Ví dụ chuẩn là góc của một viên gạch trên bức tường gạch. Bộ phát hiện tìm thấy nó ở đúng chỗ trong mọi khung hình, nên tính lặp lại hoàn hảo. Nhưng nó giống hết năm trăm góc gạch khác, nên ghép cặp luôn sai.

Giải pháp là hai đầu ra riêng. Đầu thứ nhất dự đoán tính lặp lại, huấn luyện bằng cách ép bản đồ điểm của ảnh gốc và bản đồ điểm của ảnh đã biến đổi có các đỉnh trùng nhau. Đầu thứ hai dự đoán *độ tin cậy*, tức là ước lượng xem bộ mô tả tại điểm ảnh đó có ghép đúng được không. Điểm được chọn là điểm cao ở cả hai bản đồ.

Bài học ở đây dùng được ngay mà không cần mạng. Bộ chọn điểm của ORB-SLAM3 tối đa hoá đúng một thứ là điểm số góc Harris, tức là một xấp xỉ của tính lặp lại. Nó không có khái niệm nào về tính phân biệt. Đó là nguyên nhân cơ chế của một lỗi rất đắt: vòng đóng sai trên cấu trúc lặp như hàng cột, cửa sổ, hay gạch lát. Nếu bạn muốn một cái tiền rẻ, hãy thêm một điểm số phân biệt cục bộ vào bước chọn điểm — chẳng hạn khoảng cách Hamming tới láng giềng gần nhất *trong cùng ảnh* — và loại các điểm quá giống hàng xóm của chính mình.

D. Key.Net, DISK và DeDoDe: ba cách khác để lách chỗ thiếu nhãn

Ba công trình còn lại tấn công cùng vấn đề bằng ba đường khác nhau, và mỗi đường dạy một điều.

- **Key.Net — dùng lại phần đã giải xong.** Nó đưa các bộ lọc đạo hàm bậc nhất viết tay ở nhiều tỉ lệ vào làm đầu vào, rồi chỉ học một mạng rất nông để tổ hợp chúng [231]. Kết quả là một mô hình rất nhỏ và tổng quát hoá tốt sang ảnh khác kiểu. Bài học: phần nào của bài toán đã có lời giải viết tay tốt thì đừng bắt mạng học lại; hãy để mạng học phần còn lại.
- **DISK — tối ưu đúng mục tiêu rời rạc.** Chọn điểm và chấp nhận một cặp ghép đều là quyết định rời rạc, nên không lấy *gradient* qua được. DISK coi chúng là hành động lấy mẫu từ một phân phối, thưởng +1 cho cặp ghép đúng và phạt -1 cho cặp sai, rồi dùng *gradient* chính sách [232]. Bài học: khi mục tiêu thật là rời rạc và nằm ở cuối đường ống, học tăng cường là lối thoát thay cho việc bịa ra một hàm thay thế khả vi. Cái giá là phương sai *gradient* lớn và huấn luyện đắt hơn hẳn.
- **DeDoDe — tách rời để không kéo lệch nhau.** Huấn luyện chung bộ phát hiện với bộ mô tả có một tác dụng phụ ngầm: bộ phát hiện dần chỉ chọn những điểm mà bộ mô tả *hiện tại* tình cờ mô tả tốt. Hai chế độ hồng quần vào nhau, không tách ra chẩn đoán được. DeDoDe huấn luyện bộ phát hiện riêng, bằng tín hiệu “điểm này có được theo dõi bền qua một tái dựng ba

chiều lớn không”, rồi huấn luyện bộ mô tả riêng [233]. Bài học chính là bài học chẩn đoán, và nó rất hợp với kỷ luật ablation của chương 24.

E. XFeat và ALIKED: hai công trình nói cùng ngôn ngữ với bạn

Hai công trình cuối đáng chú ý riêng, vì chúng là hai công trình duy nhất trong danh sách đặt ràng buộc của bạn vào ngay từ đầu.

XFeat được thiết kế cho CPU và thiết bị biên [234]. Ba lựa chọn thiết kế đáng học. Thứ nhất, mạng rất nông và giảm mẫu sớm, làm việc chủ yếu ở một phần tư độ phân giải. Thứ hai, số kênh ở các tầng đầu bị giữ rất nhỏ, vì các tầng đầu chạy trên độ phân giải cao nên chúng chiếm phần lớn chi phí. Thứ ba, độ chính xác vị trí bị mất do giảm mẫu được lấy lại bằng một mạng tinh chỉnh nhỏ chạy *sau* khi đã ghép cặp, tức là chỉ chạy trên vài trăm điểm thay vì trên toàn ảnh. Đây là mẫu thiết kế đáng nhớ nhất của cả chương: **làm phân thô trên toàn ảnh ở độ phân giải thấp, làm phân tinh chỉ ở nơi cần, sau khi đã biết nơi nào cần**. Cùng mẫu này xuất hiện lại ở LoFTR trong mục 5.

ALIKED tấn công thẳng chế độ hồng thứ ba [235]. Bộ mô tả cổ điển lấy mẫu trên một ô vuông cố định quanh điểm. Khi mặt phẳng nghiêng, ô vuông đó trên ảnh ứng với một hình bình hành trên vật thật, nên nội dung lấy được đã méo trước khi mô tả. ALIKED học các vị trí lấy mẫu *biến dạng* được: mạng dự đoán một tập nhỏ các độ dời, và bộ mô tả lấy mẫu tại các vị trí đã dời đó. Hình dạng vùng hỗ trợ vì thể thích nghi theo hình học cục bộ. Chi phí thấp vì số vị trí lấy mẫu ít.

F. Hai trật tự: phát hiện rồi mô tả, hay mô tả rồi phát hiện

Có một lựa chọn kiến trúc chạy ngầm dưới cả bảy phương pháp trên, và nó đáng được gọi tên riêng. Nó quyết định phương pháp hồng thể nào khi ảnh mờ.

Trật tự cổ điển là **phát hiện rồi mô tả**. FAST chọn điểm trước, BRIEF mô tả sau. SIFT cũng theo đúng trật tự này, chỉ khác ở chỗ nó dùng cực trị của hàm sai khác Gauss thay cho phép thử vòng tròn [236]. LIFT là bản học được đầu tiên của đúng trật tự đó: ba mạng nối tiếp cho phát hiện, ước lượng hướng, và mô tả [237]. Ưu điểm là chi phí thấp, vì bộ mô tả chỉ chạy trên vài trăm ô ảnh. Nhược điểm nằm ở chỗ nếu bộ phát hiện đã bỏ sót một điểm thì bộ mô tả không có cơ hội nào cứu.

Trật tự ngược là **mô tả rồi phát hiện**, do D2-Net đề xuất [90]. Mạng tính một bản đồ đặc trưng dày cho toàn ảnh trước. Sau đó điểm được chọn ngay trên chính bản đồ đặc trưng đó, tại các vị trí vừa là cực đại theo không gian vừa là cực đại theo kênh. Nghĩa là quyết định “đây là điểm đáng giữ” được đưa ra bằng biểu diễn đã qua nhiều tầng tích chập, chứ không bằng cường độ thô.

Khác biệt này quan trọng với bạn vì một lý do rất cụ thể. Ở trật tự cổ điển, ảnh mờ giết bộ phát hiện ở tầng cường độ thô, nơi không có gì cứu được. Ở trật tự ngược, quyết định dựa trên đặc trưng có trường tiếp nhận rộng vài chục điểm ảnh, nên cấu trúc lớn vẫn còn tín hiệu ngay cả khi cấu trúc nhỏ đã bị trung bình mất. Cái giá là chi phí: bản đồ đặc trưng dày phải tính cho toàn ảnh, và độ chính xác vị trí kém hơn vì cực đại được lấy trên lưới đã giảm mẫu. Đây chính là đánh đổi mà XFeat gỡ bằng bước tinh chỉnh sau ghép cặp, và là đánh đổi mà LoFTR đẩy tới cực đoạn ở mục 5.

G. Cần bao nhiêu dữ liệu, và bạn có nó không

Cổng thứ ba của chương 27 hỏi “cần bao nhiêu dữ liệu”. Với đặc trưng học được, câu trả lời dễ chịu hơn hầu hết mọi chỗ khác trong học sâu, và lý do đáng hiểu.

Nhân ở đây không đến từ con người. Nó đến từ hình học. Muốn biết điểm ảnh nào trong ảnh i ứng với điểm ảnh nào trong ảnh j , bạn chỉ cần tư thế tương đối và độ sâu. Vì thế mọi tập dữ liệu huấn luyện của dòng này đều là đầu ra của một hệ thống tái dựng: ảnh thu thập từ internet chạy qua tái dựng cấu trúc từ chuyển động cho cảnh ngoài trời, hoặc ảnh RGB-D quét trong nhà. Không có ai ngồi gán nhãn.

Ba mức đầu tư, theo thứ tự rẻ dần lên đắt dần.

- **Không dữ liệu của bạn.** Dùng trọng số công bố sẵn. Rủi ro là ra ngoài phân phối huấn luyện, và mục “Giới hạn” cuối chương nói rõ kiểu hỏng.
- **Chỉ ảnh của bạn, không cần tư thế.** Đây là chỗ tự giám sát bằng homography có giá trị thực dụng lớn nhất, và nó ít được nói tới. Bạn áp phép biến đổi lên chính ảnh của mình rồi huấn luyện, không cần chân trị nào cả. Nếu miền của bạn khác về *kết cấu* — ảnh hầm mỏ, ảnh ban đêm — thì mức này đã giải quyết phần lớn. Nếu miền của bạn khác về *chuyển động*, hãy thêm mờ có hướng vào tập biến đổi; đó là chỗ SuperPoint để trống.
- **Ảnh của bạn kèm tư thế.** Chạy COLMAP trên các chuỗi đã thu, lấy tư thế và độ sâu thưa, rồi sinh cặp ghép chân trị. Tốn vài ngày tới vài tuần công sức, chủ yếu là công sức lọc các tái dựng hỏng. Đổi lại bạn có một tập huấn luyện đúng miền và đúng chế độ chuyển động của mình.

Nói cách khác, đây là một trong rất ít can thiệp học sâu vào SLAM mà con đường dữ liệu rõ ràng từ đầu tới cuối và không phụ thuộc vào người gán nhãn. Đó là một lý lẽ ủng hộ thật, và nó nên được cân nhắc song song với các lý lẽ phản đối ở mục 6.

Bảng 28.2. Bảy phương pháp, so theo cùng bốn tiêu chí. Cột “hỏng khi nào” là cột quyết định: nó cho biết phương pháp mang theo giả định gì từ dữ liệu huấn luyện của nó. Cột chi phí là xếp hạng tương đối theo số phép nhân cộng và độ sâu mạng, không phải số đo trên Orin.

Tên	Cơ chế	Nguồn giám sát	Hỏng khi nào	Chi phí
HardNet	Chỉ bộ mô tả; triplet với mẫu âm khó nhất trong batch	Cặp ô ảnh khớp từ tái dựng ba chiều	Vẫn cần bộ phát hiện cổ điển, nên thừa hưởng nguyên chế độ hỏng khi ảnh mờ	Rất thấp
SuperPoint	Một mạng, hai đầu ra: bản đồ điểm và mô tả 256 chiều	Tự giám sát bằng homography ngẫu nhiên	Phân phối huấn luyện chỉ có biến dạng phẳng; không có ảnh mờ, không có thị sai	Trung bình
R2D2	Tách hai bản đồ: tính lặp lại và độ tin cậy	Cặp ảnh có dòng quang đã biết	Hai mục tiêu phải cân trọng số bằng tay; nặng hơn SuperPoint	Trung bình cao
Key.Net	Lọc đạo hàm viết tay nhiều tỉ lệ, mạng nông tổ hợp	Mất mát kiểu tính lặp lại trên cặp ảnh	Chỉ là bộ phát hiện; không cho bộ mô tả	Rất thấp
DISK	Chọn điểm và ghép cặp là hành động; thưởng theo cặp ghép đúng	Học tăng cường trên ảnh có tư thế đã biết	Huấn luyện đắt, phương sai lớn, nhạy với cách đặt phần thưởng	Trung bình
XFeat	Mạng rất nông ở một phần tư độ phân giải, tinh chỉnh sau khi ghép	Cặp ảnh tổng hợp và thật	Vị trí kém chính xác hơn ở tầng thô; đổi độ chính xác lấy tốc độ	Thấp
ALIKED	Vị trí lấy mẫu biến dạng được cho vùng hỗ trợ của bộ mô tả	Cặp ảnh có tư thế đã biết	Biến dạng học được vẫn giả định bề mặt trơn cục bộ	Thấp

Mạch 1 — vì sao dòng này tiến theo đúng thứ tự đó

- 1 Vấn đề** Không có chân trị cho câu “điểm nào đáng là điểm đặc trưng”. Mọi cố gắng học bộ phát hiện đều dừng ở đây suốt nhiều năm, trong khi bộ mô tả thì đã học được từ 2016.
- Giải pháp** SuperPoint chế nhạo từ homography nghịch đảo được. Điểm nào sống sót qua hàng trăm phép biến đổi thì được coi là điểm tốt.
- Giả định** Ảnh là của một mặt phẳng, hoặc ít nhất biến dạng giữa hai ảnh xấp xỉ được bằng homography.
- Giá phải trả** Phân phối huấn luyện mất sạch thị sai, che khuất và ảnh mờ. Mạng không bao giờ thấy nỗi đau thật của SLAM.
- Vấn đề mới** Điểm được chọn tối đa hoá tính lặp lại, mà tính lặp lại cao không có nghĩa là ghép đúng được.
- 2 Vấn đề** Cấu trúc lặp cho điểm cực kỳ lặp lại nhưng không phân biệt được, nên ghép cặp sai một cách có hệ thống.
- Giải pháp** R2D2 dự đoán hai bản đồ tách rời, chọn điểm cao ở cả hai.
- Giá phải trả** Hai hàm mất mát phải cân trọng số bằng tay, và trọng số đó không chuyển được sang tập dữ liệu khác.
- Vấn đề mới** Bộ phát hiện và bộ mô tả vẫn huấn luyện chung, nên bộ phát hiện bị kéo về phía sở thích của bộ mô tả.
- 3 Vấn đề** Mọi hàm mất mát ở trên đều là hàm thay thế. Thứ ta thật sự muốn — số cặp ghép đúng sau bước lọc — là một đại lượng rời rạc.
- Giải pháp** DISK tối ưu thẳng đại lượng rời rạc đó bằng *gradient* chính sách; DeDoDe tách hẳn hai việc để chúng không kéo lệch nhau.
- Giá phải trả** Huấn luyện đắt hơn nhiều và nhạy với cách đặt phần thưởng; DeDoDe phải chạy hai mạng.
- Hệ quả** Tới đây chất lượng đã bão hoà trên các bảng xếp hạng ghép ảnh, và trực cạnh tranh chuyển sang *chi phí*. Đó chính là chỗ XFeat và ALIKED bước vào, và cũng là chỗ người làm SLAM nên bắt đầu đọc.

28.4 Ghép cặp học được và chi phí bậc hai (Learned Matching)

Ghép cặp trong ORB-SLAM3 là một chuỗi các quyết định *độc lập và cục bộ*. Với mỗi bộ mô tả, tìm láng giềng gần nhất theo Hamming. Loại nếu tỉ lệ giữa ứng viên nhất và nhì lớn hơn mfNNratio . Giữ nếu hai chiều cùng chọn nhau. Cuối cùng, bỏ phiếu bằng biểu đồ chênh lệch góc với HISTO_LENGTH bằng 30, và giữ ba ô cao nhất.

Đọc lại danh sách đó theo con mắt khác thì nó là một xấp xỉ thô của hai ràng buộc toàn cục. Phép thử tỉ lệ là xấp xỉ cục bộ của ràng buộc *mỗi điểm chỉ được dùng một lần*. Biểu đồ góc là xấp xỉ rất thô của ràng buộc *mọi cặp ghép phải nhất quán với một phép biến đổi chung*. SuperGlue là câu trả lời cho câu hỏi: nếu ta thay hai xấp xỉ thô đó bằng hai thứ thật thì được gì.

A. SuperGlue: đưa ngữ cảnh và ràng buộc một-một vào trong mô hình

SuperGlue nhận vào hai tập điểm kèm bộ mô tả và trả về ma trận gán [213]. Bốn bước, mỗi bước gỡ một thứ.

- Nhúng vị trí.** Mỗi điểm thành một vector, ghép từ bộ mô tả và từ một phép nhúng của toạ độ cùng điểm số tin cậy. Từ lúc này, “ở đâu” và “trông thế nào” nằm chung một biểu diễn.

2. **Chín tầng *attention* xen kẽ.** Tầng tự chú ý cho mỗi điểm nhìn mọi điểm khác trong *cùng* ảnh; tầng chú ý chéo cho nó nhìn mọi điểm trong ảnh kia. Sau chín tầng, biểu diễn của một điểm không còn là mô tả cục bộ nữa mà đã mang thông tin của cả hai ảnh.
3. **Ma trận điểm số kèm thùng rác.** Tích vô hướng giữa hai tập vectơ cho ma trận $N \times N$, thêm một hàng và một cột “thùng rác” để chứa các điểm không có cặp.
4. **Sinkhorn.** Chuẩn hoá xen kẽ theo hàng và theo cột nhiều vòng, tức là giải xấp xỉ bài toán vận tải tối ưu có chính quy hoá entropy [238]. Kết quả là ma trận gần như một-một, và tính một-một do *mô hình* bảo đảm chứ không do một phép lọc sau đó.

Vì sao cơ chế này thắng phép thử tỉ lệ? Vì phép thử tỉ lệ chỉ nhìn được hai ứng viên của riêng một điểm. Trên một bức tường gạch, cả hai ứng viên đều giống nhau, nên phép thử loại luôn cặp ghép — kể cả khi cặp đúng nằm trong đó. Chú ý chéo thì cho điểm đó biết rằng các hàng xóm của nó đều ghép về một vùng nhất quán, và thông tin đó đủ để chọn đúng viên gạch. Nói gọn: **SuperGlue đổi một quyết định cục bộ lấy một quyết định toàn cục, và trả bằng chi phí bậc hai.**

B. Chi phí bậc hai, và vì sao nó chi phối mọi quyết định khác

Chi phí của *attention* tỉ lệ với bình phương số điểm. Kế toán thô nhưng đủ dùng: mỗi phép chú ý trên N điểm với số chiều d tốn khoảng $2N^2d$ phép nhân cộng, gồm ma trận điểm số và tổng có trọng số. SuperGlue có chín tầng, mỗi tầng áp cho cả hai ảnh, nên khoảng 18 phép chú ý:

$$\text{MAC} \approx 18 \cdot 2N^2d = 36 N^2d.$$

Công thức đó là toàn bộ thứ bạn cần, và nó nói hai điều. Thứ nhất, chi phí tăng theo N^2 : nhân đôi số điểm là gấp bốn chi phí, nên số điểm là đòn bẩy mạnh nhất mà bạn có, mạnh hơn mọi thủ thuật cài đặt. Thứ hai, hệ số đứng trước N^2 không nhỏ — chín tầng, mỗi tầng áp cho cả hai ảnh, số chiều d cỡ vài trăm — nên ngay ở số điểm mà ORB-SLAM3 đang dùng, đây đã là một khối lượng tính toán khác hẳn *bậc* so với một phép tìm láng giềng gần nhất trên chuỗi bit. Khác bậc, không phải khác vài phần trăm.

Đổi từ phép nhân cộng sang mili-giây thì chương này dừng lại, và dừng có lý do. Muốn chia thì phải có mẫu số, tức thông lượng thật của Orin ở batch bằng 1 trên đúng dạng phép toán này; mà *attention* với ma trận nhỏ thường bị chặn bởi băng thông bộ nhớ chứ không bởi số nhân, nên mẫu số ấy cách hiệu năng đỉnh rất xa và không tra được ở đâu cả. Một con số mili-giây suy ra từ một mẫu số đoán mò là con số vô giá trị — tệ hơn cả không có, vì nó sẽ được nhớ và được trích lại — nên ở đây không có con số nào. Cách đo mẫu số đó nằm ở mục 6.

Thứ chuyển được sang mọi phần cứng là *độ dốc*: mọi thứ ở đây tăng theo N^2 , nên quyết định về số điểm quan trọng hơn mọi thứ khác, và nó vẫn đúng nguyên khi bạn đổi máy.

C. LightGlue: ba cách rút ngắn cùng một chong tâng

LightGlue giữ nguyên ý tưởng của SuperGlue và tấn công riêng phần chi phí [214]. Ba cơ chế, và cả ba đều dùng lại được ở nơi khác.

- **Thoát sớm thích nghi.** Sau mỗi tầng, một bộ phân lớp nhỏ ước lượng xem lời giải đã đủ chắc chưa. Cặp ảnh dễ — và trong SLAM thì gần như mọi cặp khung hình liên tiếp đều dễ — dừng sau vài tầng thay vì chạy hết chín tầng. Ý tưởng gốc: *độ sâu tính toán nên tỉ lệ với độ khó của mẫu, không phải là hằng số.*
- **Cắt điểm.** Điểm nào được dự đoán là không thể ghép thì bị loại giữa chừng, nên các tầng sau chạy trên N nhỏ hơn. Vì chi phí là bậc hai, bỏ một nửa số điểm làm phần còn lại rẻ đi bốn lần. Đây là chỗ tiết kiệm lớn nhất.

- **Bỏ Sinkhorn.** Thay vì hàng chục vòng chuẩn hoá, LightGlue dự đoán riêng một điểm số “có ghép được không” cho từng điểm, rồi kết hợp nó với ma trận tương đồng trong một bước. Phần gán không còn là vòng lặp nữa.

Bài báo báo cáo tăng tốc vài lần so với SuperGlue ở cùng mức chính xác, và mức tăng lớn hơn trên cặp ảnh dễ. Mình không dẫn con số mili-giây cụ thể ở đây, vì con số đó gần với phần cứng máy để bàn trong bài và không chuyển sang Orin được.

D. Ba thứ lấy được từ SuperGlue mà không cần chạy SuperGlue

Trước khi bàn chi phí, đáng dừng lại ở một câu hỏi rõ hơn nhiều: trong ý tưởng của SuperGlue, phần nào dùng được ngay trong ORBmatcher hiện tại, không cần mạng nào?

- **Gán một-một thật thay cho phép thử tỉ lệ.** Bạn đã có ma trận chi phí Hamming giữa các ứng viên. Giải bài toán gán trên ma trận đó — bằng thuật toán Hungary, hoặc bằng vài vòng chuẩn hoá kiểu Sinkhorn — cho ra một lời giải toàn cục thay vì một chuỗi quyết định độc lập. Chi phí là bậc ba theo số ứng viên trong cửa sổ tìm kiếm, mà số đó rất nhỏ, nên trên thực tế nó gần như miễn phí. Đây là phần rẻ nhất và mình cho là đáng thử trước.
- **Bỏ phiếu nhất quán mịn hơn.** Biểu đồ chênh lệch góc với 30 ô là một phép bỏ phiếu một chiều rất thô. Bỏ phiếu trên một không gian hai chiều — chênh lệch góc cộng với độ dời trung bình — bắt được nhiều cặp sai hơn với cùng chi phí. Đây đúng là thứ mà tầng *attention* của SuperGlue học được, chỉ ở dạng viết tay.
- **Điểm số “có ghép được không” riêng cho từng điểm.** LightGlue dự đoán nó bằng mạng. Bạn xấp xỉ được nó bằng một đại lượng có sẵn: khoảng cách Hamming tới láng giềng gần nhất *trong cùng ảnh*. Điểm nào có láng giềng cùng ảnh rất gần thì nằm trên cấu trúc lặp, và nên bị hạ trọng số. Đây cũng chính là ý tưởng “tính phân biệt” của R2D2, áp bằng tay.

Ba việc này không thay thế được SuperGlue. Nhưng chúng cho bạn một *baseline* mạnh hơn trước khi so, và mục 6 sẽ giải thích vì sao có một *baseline* mạnh lại quan trọng tới vậy ở đây.

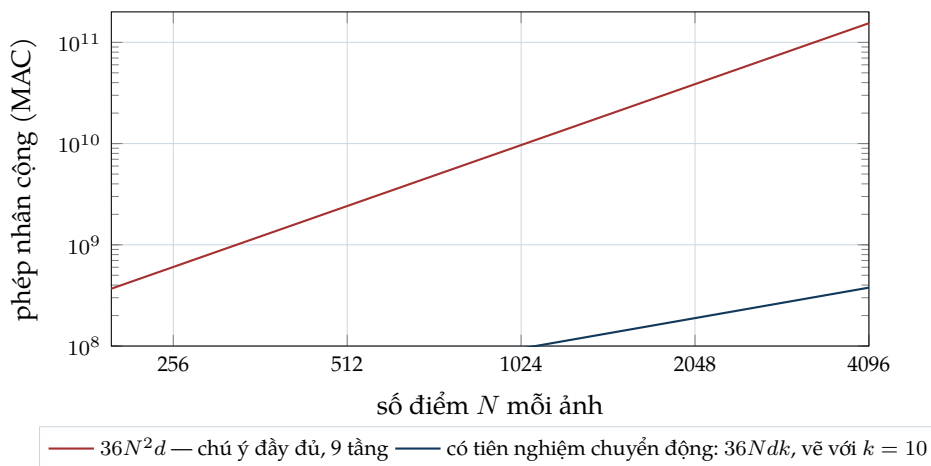
E. Sáu cách né chi phí bậc hai, xếp theo mức hữu ích cho SLAM

Đây là mục đáng giá nhất của cả phần ghép cặp, vì nó là chỗ nền SLAM của bạn có lợi thế thật so với người viết các bài báo trên.

1. **Giảm N .** Rẻ nhất và hiệu quả nhất, vì tiết kiệm là bậc hai: giảm một nửa số điểm thì chi phí còn một phần tư. Hãy đo APE theo số điểm trước khi làm bất cứ việc gì khác — rất có thể hệ của bạn đã bão hoà ở một số điểm thấp hơn hẳn cấu hình mặc định, và nếu đúng thế thì bạn vừa lấy được phần lớn khoản tiết kiệm mà không đụng tới một mô hình nào.
2. **Chỉ chạy ở nơi thật sự cần.** Giữ ORB cho bám khung hình ở 30 Hz, và chỉ gọi bộ ghép cặp học được cho ứng viên đóng vòng và định vị lại, ở cỡ 1 Hz. Ngân sách giảm khoảng ba mươi lần, và nó nhắm đúng chế độ hồng thứ ba — chế độ mà đối góc nhìn lớn thật sự xảy ra. Theo mình đây là phương án có tỉ lệ lợi ích trên rủi ro cao nhất trong cả chương.
3. **Dùng dự đoán chuyển động để giới hạn cặp ứng viên.** Hệ của bạn đã có mô hình vận tốc không đổi và có IMU. Chúng cho một cửa sổ tìm kiếm hẹp quanh vị trí chiếu. Khi đó mỗi điểm chỉ còn k ứng viên trong cửa sổ thay vì cả N điểm của ảnh kia, nên chi phí thành $N \cdot k$ chứ không phải N^2 , tức là gần như tuyến tính — và k là một đại lượng bạn đo được ngay từ nhật ký của ORBmatcher. Về mặt *attention*, đây chính là chú ý thừa theo khối.
4. **Cắt điểm và thoát sớm.** Đúng hai cơ chế của LightGlue; dùng được ngay vì đã có mã nguồn mở.

5. **Chú ý tuyến tính.** Thay hàm mũ trong *softmax* bằng một ánh xạ đặc trưng để đổi thứ tự phép nhân ma trận, cho chi phí $O(N)$ [239]. LoFTR dùng cách này. Cái giá là sức biểu diễn giảm, và với N nhỏ thì nó còn chậm hơn vì hằng số lớn.
6. **Ghép thô rồi tinh.** Ghép ở độ phân giải thấp trước rồi chỉ tinh chỉnh quanh các cặp đã thắng. Đây là mẫu thiết kế đã gặp ở XFeat và sẽ gặp lại ở LoFTR.

Điểm số ba đáng được nói thẳng, vì nó là một khoảng trống thật trong văn liệu. Gần như mọi bài báo ghép cặp đều đánh giá trên cặp ảnh hai khung nhìn rộng, lấy từ tái dựng ba chiều, *không có tiên nghiệm chuyển động nào*. Trong bối cảnh đó, giải bài toán gán toàn cục với chi phí N^2 là hợp lý. Trong bối cảnh của bạn, giữa hai khung hình cách nhau 33 mili-giây và có IMU nối chúng, phần lớn chi phí N^2 đó là tiền trả cho một bài toán bạn không có. Hình 28.5 vẽ khoảng cách giữa hai chế độ đó.



Hình 28.5. Chi phí ghép cặp theo số điểm. Hai đường là hai công thức được vẽ ra chứ không phải hai phép đo: đỏ là chú ý đầy đủ, xanh là cùng mô hình nhưng chỉ xét k ứng viên nằm trong cửa sổ do IMU dự đoán. Khoảng cách giữa hai đường đúng bằng N/k , nên nó rộng ra khi bạn xin nhiều điểm hơn và hẹp lại khi tiên nghiệm của bạn kém đi — đó là toàn bộ nội dung của hình. Trục tung đếm phép nhân cộng chứ không đếm mili-giây; muốn đổi sang thời gian thì phải có thông lượng đo được của máy bạn ở batch bằng 1, và mục 6 nói cách lấy nó.

CẠM BẮY

Bảng xếp hạng ghép cặp đo trên cặp ảnh hai khung nhìn rộng, chọn ra từ tái dựng ba chiều, thường cách nhau nhiều mét và nhiều chục độ [240]. Frontend của bạn ghép hai khung hình cách nhau 33 mili-giây, với một tiên nghiệm tư thế tốt tới vài điểm ảnh. Hai bài toán này khác nhau tới mức thứ hạng trên bảng kia gần như không dự đoán được gì cho bạn. Chuyển kết luận thẳng từ bảng xếp hạng sang hệ của mình là sai lầm đắt nhất trong cả chương — không phải vì các con số sai, mà vì chúng trả lời một câu hỏi khác. Cách kiểm tra rẻ: đo phân bố *góc giữa hai tia quan sát* của các cặp ghép trong chuỗi của bạn, rồi so với phân bố đó trong tập kiểm thử của bài báo.

28.5 Bỏ hẳn bộ phát hiện (Detector-free Matching)

Mục 2 đã kết luận rằng vùng ít vân không phải một lỗi cài đặt. Nó là hệ quả logic của việc bắt một cửa sổ bán kính 3 quyết định một câu hỏi mà thông tin trả lời nằm cách đó hai trăm điểm ảnh. Có đúng một cách thoát: đừng quyết sớm.

A. LoFTR: ghép dày ở tầng thô, rồi tinh chỉnh

LoFTR bỏ hẳn bước chọn điểm [215]. Nó chạy hai giai đoạn.

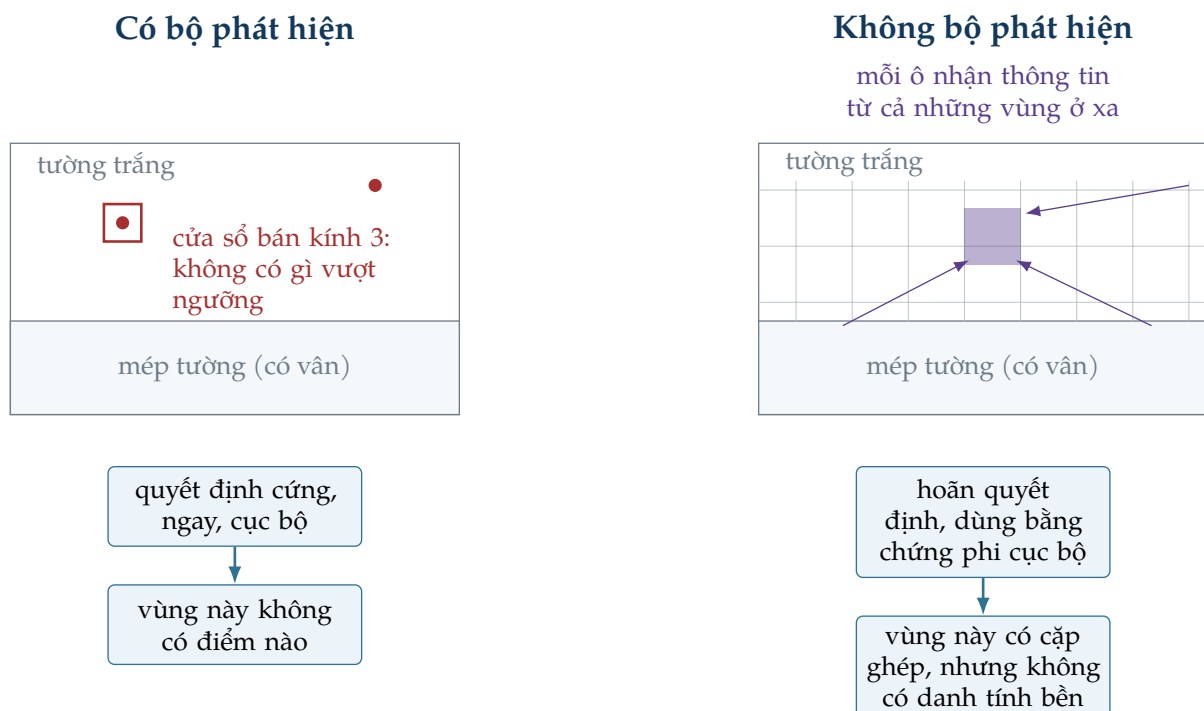
Giai đoạn thô làm việc trên bản đồ đặc trưng ở một phần tám độ phân giải. Với ảnh 640 nhân 480, đó là lưới 80 nhân 60, tức 4800 ô. Mỗi ô thành một *token*, rồi một chồng tầng tự chú ý và chú ý chéo cho mỗi ô nhìn thấy toàn bộ hai ảnh. Vì 4800 ô mà chú ý đầy đủ thì ma trận điểm số có hơn hai chục triệu ô, LoFTR bắt buộc phải dùng chú ý tuyến tính. Sau đó một tầng ghép khả vi — *softmax* hai chiều hoặc vận tải tối ưu — cho ra tập cặp ghép ở mức lưới thô.

Giai đoạn tinh chạy trên từng cặp đã thắng. Nó cắt một cửa sổ nhỏ quanh mỗi cặp trên bản đồ đặc trưng ở một phần hai độ phân giải, tương quan hai cửa sổ, rồi lấy kỳ vọng trên bản đồ tương quan để có vị trí dưới mức điểm ảnh. Đây đúng là mẫu thiết kế đã gặp ở XFeat: thô trên toàn ảnh ở độ phân giải thấp, tinh chỉ ở nơi đã biết là cần.

B. Vì sao bỏ bộ phát hiện lại cứu được vùng ít vân

Câu trả lời hay bị nói sai. Bộ phát hiện *không* làm bức tường trắng có vân. Nó đối thứ khác: *đổi loại bằng chứng* được phép dùng để quyết định một cặp ghép.

Bộ phát hiện dùng bằng chứng cục bộ và quyết ngay. Trên tường trắng thì bằng chứng cục bộ bằng không, nên quyết định nào cũng sai. Chồng tầng chú ý của LoFTR làm cho biểu diễn của một ô ở giữa bức tường mang thông tin từ mép tường, từ khung cửa, từ bố cục cả căn phòng. Ô đó vẫn “không có vân”, nhưng cặp ghép của nó được quyết bằng bằng chứng *phi cục bộ*. Hình 28.6 vẽ đúng khác biệt đó.



Hình 28.6. Vì sao bỏ bộ phát hiện cứu được vùng ít vân. Bên trái, cửa sổ bán kính 3 ở giữa bức tường không chứa thông tin nào, nên mọi ngưỡng đều sai. Bên phải, chồng tầng chú ý đưa thông tin từ mép tường và từ bố cục phòng vào chính ô đó, nên cặp ghép được quyết bằng bằng chứng phi cục bộ. Cái mất nằm ở ô cuối bên phải: cặp ghép có, nhưng điểm đó không mang một danh tính có thể tìm lại ở khung hình sau — và danh tính mới là thứ SLAM cần để dựng bản đồ.

C. Bốn cái giá, và cái thứ tư mới là cái giết

- **Chi phí sần theo diện tích ảnh, không theo số điểm.** Bạn không giảm được chi phí bằng cách xin ít điểm hơn. Sần được đặt bởi độ phân giải, và nó cao.
- **Cặp ghép neo vào lưới.** Bước tinh chỉnh lấy lại độ chính xác vị trí, nhưng không lấy lại được tính lặp lại của *vị trí trên vật thật* qua nhiều khung hình.
- **Chỉ hai ảnh một lúc.** LoFTR ghép ảnh với ảnh. SLAM cần một điểm bản đồ được quan sát bởi nhiều khung khoá, tức là một vệt theo dõi. Nối các cặp ghép hai-ảnh thành vệt thì tích lũy sai lệch và đứt ngay khi bỏ qua một khung.
- **Không có bộ mô tả để lưu.** Đây là cái giá quyết định. Bám khung hình là bài toán một-nhiều: ghép khung mới với hàng vạn điểm bản đồ. Có bộ mô tả thì đó là một phép tìm láng giềng gần nhất. Không có bộ mô tả thì bạn phải chạy mạng *một lần cho mỗi cặp ảnh ứng viên*. Với N khung khoá là N lần chạy mạng. Và túi từ nhị phân [241] — nền của cả định vị lại lẫn lấy ứng viên vòng — không còn dùng được, vì nó cần bộ mô tả.

D. RoMa và nhánh ghép dày mạnh nhất

RoMa đẩy ý tưởng đi xa hơn: thay vì một tập cặp ghép, nó hồi quy một trường biến dạng dày cho toàn ảnh, kèm một bản đồ độ chắc chắn theo từng điểm ảnh [242]. Đặc trưng thô lấy từ một mô hình nền thị giác đông lạnh, đặc trưng tinh lấy từ một mạng tích chập chuyên dụng. Điểm mạnh nhất của nó là các cặp ảnh cực khó: đổi góc nhìn rất lớn, đổi mùa, đổi ánh sáng. Đây là loại cặp ảnh mà mọi phương pháp thừa đều bỏ cuộc.

Cái giá tương ứng. Xương sống là một *transformer* lớn, nên chi phí ở bậc hoàn toàn khác so với XFeat hay LightGlue. Với ngân sách một khung hình trên Orin thì nhánh này nằm ngoài tầm, và mình không thấy lý do kỹ thuật nào để nghĩ khác. Có một bản kế nhiệm là RoMa v2, công bố năm 2025, được giới thiệu là nhanh hơn và dày hơn; mình chưa đọc kỹ nên chỉ nhắc tên và không dẫn bất kỳ con số nào. Ở nhánh nhẹ hơn, Efficient LoFTR kéo chi phí của ghép bản dày về gần mức của bộ ghép thưa bằng cách gộp phép chú ý và làm gọn giai đoạn thô [243].

E. Nếu vẫn muốn dùng ghép dày trong SLAM thì phải thêm gì

Giả sử bạn quyết định dùng LoFTR ở tầng đóng vòng. Có ba mảnh còn thiếu, và biết trước ba mảnh đó giúp bạn ước lượng đúng khối lượng công việc.

- **Một tầng lấy ứng viên.** Ghép dày chỉ trả lời câu “hai ảnh này khớp tới đâu”. Nó không trả lời câu “ảnh nào trong một vạn khung khoá là ứng viên”. Bạn vẫn cần bộ mô tả toàn ảnh cho bước đó, và đó là nội dung chương 29.
- **Một cách gán danh tính.** Sau khi có cặp ghép giữa khung mới và khung khoá, bạn phải nối chúng vào các điểm bản đồ đã tồn tại. Cách rẻ là chiếu ngược: điểm ảnh được ghép nằm gần điểm đặc trưng ORB nào của khung khoá thì nhận danh tính của điểm bản đồ đó. Cách này hoạt động, và nó cho thấy hai họ bổ sung nhau chứ không loại trừ nhau.
- **Một mô hình bất định.** Cặp ghép của LoFTR không có tầng kim tự tháp, nên không có `mvInvLevelSigma2`. Bạn phải lấy trọng số từ chỗ khác — điểm số tin cậy của tầng ghép, hoặc độ nhọn của bản đồ tương quan ở bước tinh chỉnh. Đây lại là đúng vấn đề của mục 6E, xuất hiện lần thứ hai, và sự lặp lại đó là dấu hiệu nó là vấn đề trung tâm chứ không phải chi tiết.

Bảng 28.3. Ba họ ghép cặp, so theo cùng tiêu chí, kèm cột quan trọng nhất với bạn: nên đặt nó ở đâu trong hệ SLAM. Cột chi phí là bậc tương đối theo số phép nhân cộng, không phải số đo.

Họ	Cơ chế	Mạnh khi nào	Hỏng khi nào	Đặt ở đâu
Thừa cổ điển (ORB)	Lắng giềng gần nhất theo Hamming, phép thử tỉ lệ, biểu đồ góc	Khung nhìn hẹp, có tiên nghiệm tốt, cần độ trễ thấp và đoán trước được	Ảnh mờ, đối sáng mạnh, đối góc nhìn lớn, vùng ít vân	Bám khung hình ở 30 Hz
Thừa học được (SuperPoint + LightGlue)	Đặc trưng học được cộng gán toàn cục bằng <i>attention</i>	Khung nhìn rộng, cấu trúc lặp, đối sáng; vẫn cho bộ mô tả lưu lại được	Ra ngoài phân phối huấn luyện; chi phí bậc hai theo số điểm	Kiểm chứng ứng viên đóng vòng, định vị lại
Dây không bộ phát hiện (LoFTR, RoMa)	Ghép mọi ô lưới, không chọn điểm; thô rời tinh	Vùng ít vân, đối góc nhìn cực lớn, cặp ảnh khó nhất	Không có bộ mô tả để lưu; chỉ hai ảnh một lúc; chi phí theo diện tích ảnh	Ghép ngoại tuyến, tái dựng, hoặc kiểm chứng vòng khi có ngân sách

Kết luận của mục này trùng với kết luận của mục 4, và sự trùng đó đáng chú ý. Cả hai nhánh — ghép cặp học được và ghép dây không bộ phát hiện — đều quá đắt cho mỗi khung hình, và đều mạnh nhất đúng ở chỗ không có tiên nghiệm chuyển động. Chỉ có một chỗ trong hệ SLAM thoả cả hai điều kiện đó: **kiểm chứng ứng viên đóng vòng và định vị lại**. Đó là nơi nên đặt chúng.

→ **hệ quả của F/29** — chương 29 nhận đúng kết luận này làm điểm xuất phát và đo xem tầng nào của đóng vòng thật sự là nút thắt

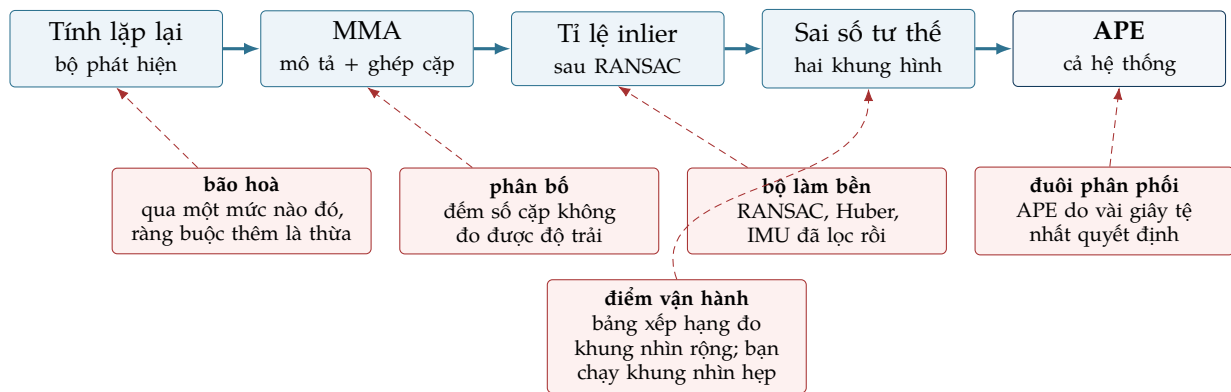
28.6 Đánh giá cho hệ thời gian thực (Evaluation for a Real-time System)

Đây là mục quan trọng nhất của chương. Ba mục trước là bản đồ; mục này là cái cân.

A. Bốn thước đo, mỗi cái cô lập một khối

- **Tính lặp lại** — tỉ lệ điểm được tìm thấy ở cùng vị trí trên vật thật trong hai ảnh. Nó cô lập riêng *bộ phát hiện*.
- **Độ chính xác ghép trung bình (MMA)** — trong các cặp được ghép, tỉ lệ cặp sai dưới ngưỡng vài điểm ảnh. Nó đo *bộ mô tả cộng bộ ghép cặp*, thường trên HPatches [244].
- **Tỉ lệ inlier sau RANSAC** — mức sạch của tập đưa vào tối ưu. Nó đo *toàn bộ frontend cộng mô hình hình học*.
- **APE** — con số duy nhất mà hệ của bạn bị đánh giá.

Điểm mấu chốt của cả chương nằm ở đây: **ba thước đo đầu có thể tăng mạnh mà APE không đổi một milimét**. Đây không phải nghịch lý, và nó có đúng năm cơ chế. Hình 28.7 vẽ chỗ tín hiệu bị hấp thụ ở từng nấc.



Hình 28.7. Thang đo từ frontend tới APE, và năm chỗ tín hiệu bị hấp thụ trên đường đi. Mỗi ô đỏ là một cơ chế độc lập; chúng cộng dồn, nên một cải thiện lớn ở nấc trái có thể tới nấc phải mà không còn gì. Hệ quả thực hành: đừng báo cáo cải tiến ở ba nấc trái rồi ngụ ý nấc phải. Hãy đo nấc phải, hoặc đo trần của nó bằng *oracle*.

B. Năm cơ chế hấp thụ, nói rõ từng cái

1. **Bảo hoà.** Tư thế có sáu bậc tự do, và mỗi cặp ghép cho hai phương trình. Vài chục cặp trải đều đã thừa sức xác định sáu ẩn đó; vượt qua mức bảo hoà — mức mà bạn đo được bằng cách quét số điểm và nhìn APE — mỗi cặp ghép thêm chỉ làm ma trận thông tin dày lên theo những hướng vốn đã chắc. Ràng buộc chặt chuyển sang chỗ khác: đường cơ sở, phân bố độ sâu, và trọng số IMU.
2. **Phân bố quan trọng hơn số lượng.** Rất nhiều cặp ghép dồn vào một góc ảnh cho ma trận thông tin điều kiện xấu hơn hẳn một số cặp ít hơn nhiều nhưng trải đều khắp khung hình. Không thước đo nào ở trên đo độ trải. Nếu muốn một chỉ số tốt hơn, hãy đo trị riêng nhỏ nhất của ma trận thông tin tư thế, không phải đếm cặp ghép.
3. **Bộ làm bền đã ăn phần lớn lợi ích.** Hệ của bạn có RANSAC, mô hình vận tốc không đổi, hàm mất mát Huber trong tối ưu, và ràng buộc IMU. Bốn thứ đó đã loại phần lớn cặp ghép xấu. Làm sạch đầu vào chỉ khiến chúng đỡ vất vả hơn. Đầu vào tốt hơn chỉ tạo khác biệt ở đúng nơi bộ làm bền đã *thất bại*, mà nơi đó chiếm rất ít thời lượng.
4. **Đuôi phân phối quyết định APE.** APE bị đặt bởi vài giây tệ nhất của chuỗi. Ba thước đo kia là trung bình trên toàn tập kiểm thử. Một cải thiện trung bình lớn hoàn toàn tương thích với việc vài giây tệ nhất không đổi gì.
5. **Sai điểm vận hành.** Bảng xếp hạng đo cặp ảnh khung nhìn rộng, còn bạn ghép hai khung hình cách nhau 33 mili-giây. Ở khung nhìn hẹp có tiên nghiệm tốt, ORB đã gần tối ưu, nên khoảng trống để cải thiện vốn đã nhỏ.

C. Thí nghiệm quyết định: *oracle* ghép cặp

Từ năm cơ chế trên suy ra một quy trình rất gọn, và nó nên chạy *trước* khi tích hợp bất cứ mạng nào.

Thay đầu ra của bộ ghép cặp bằng cặp ghép dựng từ chân trị — chiều điểm bản đồ bằng tư thế chân trị và độ sâu chân trị, rồi lấy cặp ghép trong bán kính nhỏ. Chạy lại toàn hệ. APE thu được là **trần** mà mọi cải tiến frontend bị chặn trên. Nếu trần đó chỉ tốt hơn APE hiện tại vài phần trăm, bạn vừa tiết kiệm được ba tháng. Nếu nó tốt hơn hẳn, bạn vừa biết đầu tư vào đâu và biết trước lợi ích tối đa.

Chạy thêm hai biến thể sẽ tách được câu hỏi ra rõ hơn nữa. Biến thể một: chỉ bật *oracle* trong các đoạn ảnh mờ, để đo riêng phần đóng góp của chế độ hồng thứ nhất. Biến thể hai: giữ nguyên cặp ghép của ORB nhưng thay *trọng số* của chúng bằng trọng số lý tưởng, để tách “ghép sai” khỏi

“ghép đúng nhưng cân sai”. Nếu biến thể hai cải thiện nhiều hơn biến thể một, thì bài toán của bạn nằm ở chương 34 chứ không nằm ở chương này.

→ **trường hợp riêng của E/24** — đây đúng là *oracle* experiment của chương 24, áp cho một khối cụ thể; giá trị của nó vẫn là chặn trên công sức trước khi bỏ công sức ra

D. Ngân sách trên Jetson Orin

Ở 30 khung hình mỗi giây, ngân sách là 33 mili-giây một khung, và đó là ngân sách cho *tất cả*: rút trích, bấm, quản lý bản đồ cục bộ, và phần dành sẵn cho biến động. Cách kế toán thì gọn: liệt kê từng khoản đang tiêu, đo từng khoản ở phân vị 95 chứ không ở trung vị vì phân vị mới là thứ làm bỏ khung, cộng lại, rồi lấy trần trừ đi. Chương này không đưa một bảng số mẫu, vì một bảng số chưa đo trên máy của bạn chỉ tạo cảm giác an tâm sai và rồi sẽ bị trích lại như thể nó là số đo. Nhưng ba kết luận về *cấu trúc* của bảng đó thì đúng trước cả khi bạn đo, và chúng mới là phần đáng nhớ.

- **Phần dự phòng cho biến động là khoản bị ăn mất trước tiên.** Nó không có tên trong bất kỳ hồ sơ thời gian nào, nên nó bị tiêu mà không ai ghi sổ. Khi nó hết, triệu chứng không phải chậm đều mà là bỏ khung từng đợt — và bỏ khung từng đợt trông giống hết một lỗi thuật toán, nên nó ăn mất rất nhiều thời gian gỡ lỗi.
- **Một bộ đặc trưng nhẹ chỉ vừa nếu nó thay chứ không thêm.** Nếu bạn vẫn chạy ORB song song để đối chứng, hãy tính đó là hai khoản chứ không phải một, và đừng dùng lần đo có chạy song song ấy để kết luận về ngân sách lúc vận hành.
- **Một bộ ghép cặp đầy đủ chạy mỗi khung thì gần như chắc chắn vượt trần.** Đây là hệ quả của chi phí bậc hai ở mục 4B chứ không của một con số cụ thể nào, nên nó không đôi khi bạn đổi phần cứng; cái đổi chỉ là số điểm mà tại đó nó bắt đầu vượt. Lối thoát duy nhất là hạ tần suất chạy, và đó chính là lỗi hai ở mục 6F.

Ba điều nữa mà kế toán trên chưa thể hiện. Thứ nhất, mạng phải qua TensorRT với kích thước đầu vào cố định, và bước chuyển đổi này thường lộ ra các phép toán không được hỗ trợ. Thứ hai, GPU trên Orin đã bị dùng cho những việc khác, nên độ trễ đo lúc chạy một mình không phải độ trễ lúc chạy trong hệ thật. Thứ ba, độ trễ của mạng gần như hằng số, còn độ trễ của ORB thay đổi theo nội dung ảnh — đổi sang mạng làm hệ dễ dự đoán hơn, và đó là một lợi ích thật, ít khi được kể tới.

E. Tích hợp vào ORB-SLAM3 thì phải bỏ những gì

Đây là danh sách mà mọi bài báo đều bỏ qua, và là chỗ chi phí thật nằm. Thay bộ đặc trưng không phải thay một hàm; nó gỡ ra bốn thứ đã ăn sâu vào toàn hệ thống.

- **Từ điển túi từ.** DBow2 được huấn luyện trên bộ mô tả nhị phân ORB. Bộ mô tả số thực làm nó vô dụng ngay lập tức. Mất túi từ là mất cả lấy ứng viên đóng vòng lẫn định vị lại sau khi mất bấm. Bạn phải hoặc huấn luyện lại một từ điển mới, hoặc chuyển sang bộ mô tả toàn ảnh — và đó là nội dung của chương 29.
- **Ngưỡng Hamming đã tinh chỉnh nhiều năm.** TH_HIGH bằng 100 và TH_LOW bằng 50 không còn nghĩa gì với khoảng cách L2. Mỗi chỗ gọi phải suy lại ngưỡng, và trong ORB-SLAM3 có hàng chục chỗ gọi như vậy, mỗi chỗ dùng một *mfNNratio* riêng.
- **Bất biến tỉ lệ theo kim tự tháp — và đây là chỗ nguy hiểm nhất.** Tầng kim tự tháp của mỗi điểm không chỉ dùng cho việc phát hiện. Nó làm ba việc nữa. Nó đặt bán kính tìm kiếm khi chiếu điểm bản đồ xuống ảnh. Nó đặt kiểm tra nhất quán tỉ lệ khi quyết định một điểm bản đồ có quan sát được không. Và quan trọng nhất, nó đặt *ma trận thông tin của mọi cạnh tái chiếu* trong tối ưu, qua *mvInvLevelSigma2[octave]*. Một bộ phát hiện học được không trả về

tầng kim tự tháp. Nếu bạn điền một hằng số vào đó, bạn vừa đặt trọng số bằng nhau cho mọi phương trình trong bài toán tối ưu, một cách âm thầm. Thay frontend hoá ra là thay cả backend.

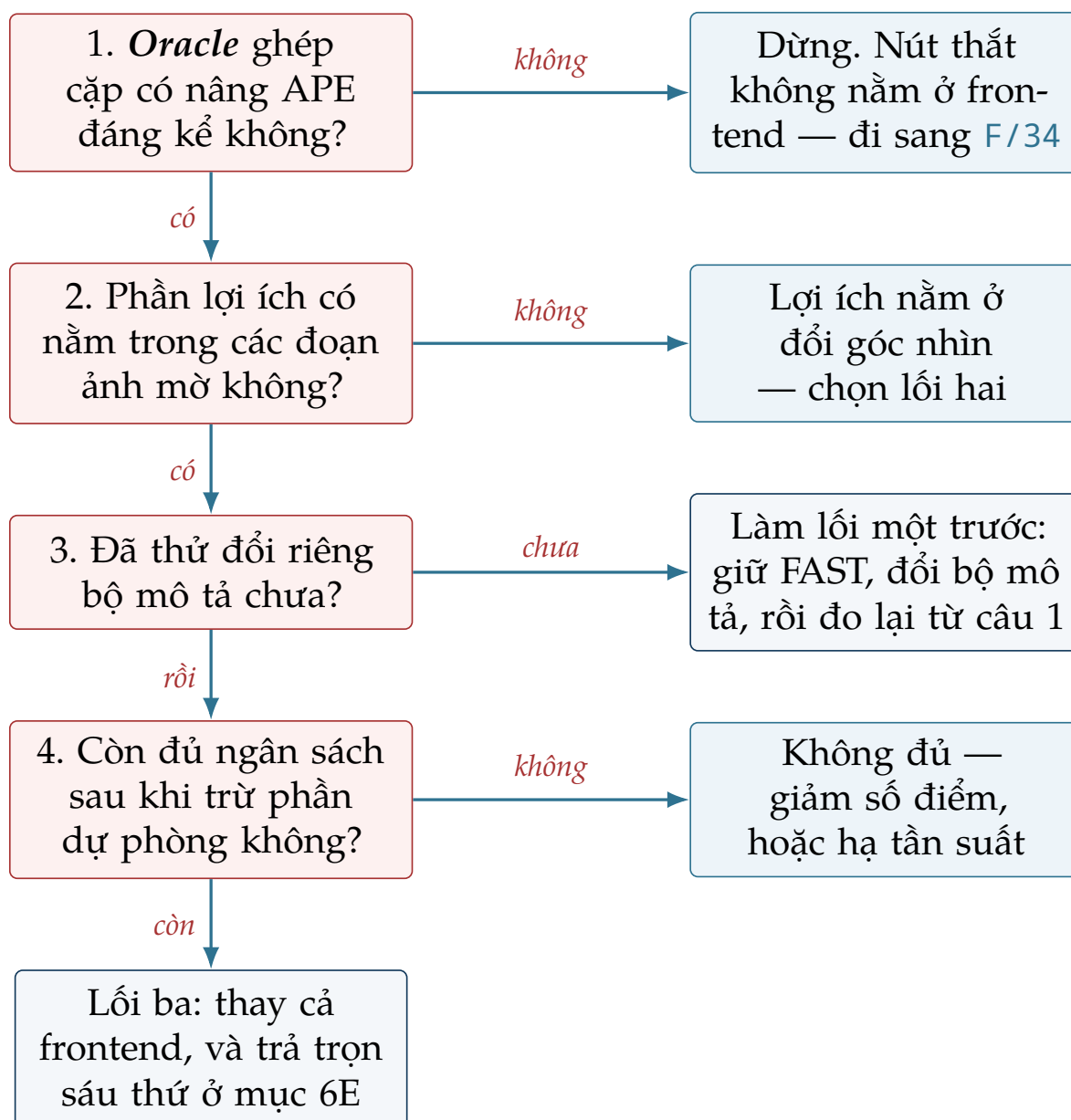
- **Bộ mô tả đại diện của điểm bản đồ.** MapPoint chọn bộ mô tả đại diện bằng cách lấy cái có khoảng cách Hamming trung vị nhỏ nhất tới các quan sát khác. Với vector số thực thì khái niệm đó phải định nghĩa lại, và trung bình không tương đương trung vị.
- **Phân bố đều theo cây tám nhánh.** DistributeOctTree ép các điểm trải đều trên ảnh. Cơ chế triệt phi cực đại của một mạng có hành vi khác hẳn. Nhớ lại cơ chế hấp thụ số hai: độ trải là thứ quyết định điều kiện của bài toán, nên đây không phải chi tiết nhỏ.
- **Các hằng số cống.** Số đặc trưng 1200, iniThFAST 12, minThFAST 4, và các cổng dạng “dưới 15 cặp thì coi như mất tám”. Tất cả đều được chỉnh cho phân phối số cặp ghép của ORB. Bộ đặc trưng mới cho phân phối khác, nên các cổng đó phải chỉnh lại từ đầu.

F. Ba lối đi giữ được phần lớn hệ thống

Danh sách trên nghe như một lời từ chối. Nó không phải. Nó chỉ nói rằng “thay ORB” là lựa chọn đắt nhất trong ba lựa chọn, và hai lựa chọn kia ít khi được nhắc tới.

- **Lối một — giữ nguyên bộ phát hiện, chỉ đổi bộ mô tả.** Vẫn chạy FAST trên kim tự tháp, vẫn có tầng, vẫn có phân bố đều theo cây tám nhánh, nhưng tính bộ mô tả bằng HardNet hoặc bằng phần mô tả của SuperPoint trên các ô ảnh đã cắt. Lối này giữ nguyên toàn bộ ba chức năng của tầng kim tự tháp, nên ma trận thông tin của backend không đổi. Nó chứa chế độ hồng số hai và một phần số ba, và không chứa số một. Chi phí thêm rất thấp, vì mạng chỉ chạy trên vài trăm ô ảnh nhỏ chứ không trên toàn ảnh. Theo mình đây là lối có tỉ lệ lợi ích trên rủi ro tốt nhất nếu bạn bắt buộc phải đổi một thứ ở mỗi khung hình.
- **Lối hai — giữ nguyên ORB, chỉ thêm một bộ ghép cặp học được ở tầng đóng vòng.** Không đụng gì tới đường tám khung hình. Bộ ghép cặp chạy ở cỡ 1 Hz, chỉ trên các cặp ứng viên đã qua bước lấy ứng viên. Nó nhắm đúng chế độ hồng số ba, và đó là chế độ mà tám khung hình không bao giờ gặp. Rủi ro thấp vì nếu nó hỏng thì bạn chỉ mất một vòng, không mất tám.
- **Lối ba — thay cả frontend.** Chỉ nên chọn khi *oracle* cho thấy trần rất cao và lối một đã được thử và không đủ. Khi đó bạn phải trả trọn danh sách sáu thứ ở mục trên, và phải tự đặt lại một mô hình bất định thay cho tầng kim tự tháp.

Hình 28.8 gói cả chương thành một cây quyết định. Nó chỉ có bốn câu hỏi, và ba trong bốn câu trả lời được bằng hai ngày làm việc chứ không bằng ba tháng.



Hình 28.8. Cây quyết định cho câu hỏi “có nên thay frontend không”. Ba trong bốn câu hỏi trả lời được bằng hai ngày đo đạc, và câu đắt nhất — câu số bốn — chỉ được hỏi sau khi ba câu kia đã cho phép. Chú ý hai nhánh sớm: nếu *oracle* cho trần thấp, hoặc nếu phần lợi ích không nằm trong các đoạn ảnh mờ, thì cả chương này không áp dụng cho hệ của bạn.

G. Dựng một cái cân của riêng bạn

Vấn đề với HPatches và các bảng xếp hạng là chúng đo sai điểm vận hành. Cách sửa không phải bỏ đo, mà là dựng một cái cân từ chính dữ liệu của bạn. Việc này tốn nửa ngày và dùng được mãi.

Nguyên liệu là một chuỗi EuRoC có chân trị tư thế và có ảnh stereo. Từ tư thế chân trị và độ sâu stereo, bạn dựng được cặp ghép đúng cho bất kỳ cặp khung hình nào: chiếu điểm từ khung i sang khung j , và chấp nhận nếu sai lệch dưới hai điểm ảnh. Đó là chân trị ghép cặp, không cần gán tay.

Điều quan trọng là *cách chọn cặp khung hình*, vì đó chính là chỗ các bảng xếp hạng sai. Hãy chia các cặp thành ba nhóm theo đúng ba chế độ vận hành của hệ bạn.

- **Nhóm tám** — cặp khung hình liên tiếp, cách nhau 33 mili-giây. Đây là tuyệt đại đa số các lần ghép cặp trong một lần chạy, nên nó phải là nhóm có trọng số lớn nhất khi bạn đọc bảng.
- **Nhóm mờ** — cặp khung hình liên tiếp nhưng lấy trong các đoạn có tốc độ góc cao nhất chuỗi, lọc từ IMU; ngưỡng thì lấy từ chính phân bố tốc độ góc của bạn, chẳng hạn phân vị trên cùng, chứ không lấy từ một con số cho sẵn.
- **Nhóm vòng** — cặp khung hình cách nhau trong thời gian nhưng gần nhau trong không gian, lấy từ chân trị. Đây là chế độ vận hành của đóng vòng.

Rồi đo cùng ba con số cho từng nhóm: số cặp ghép đúng, tỉ lệ inlier, và *độ trái* của các cặp ghép trên ảnh. Độ trái đo bằng trị riêng nhỏ nhất của ma trận hiệp phương sai toạ độ các điểm được ghép, hoặc đơn giản hơn là số ô còn có điểm khi chia ảnh thành lưới bốn nhân bốn.

Bảng ba nhóm nhân ba con số này nói được điều mà không bảng xếp hạng nào nói: phương pháp nào tốt hơn ở *nhóm nào*. Dự đoán của mình, và đây là dự đoán chứ không phải kết quả đo: các phương pháp học được sẽ hơn rất ít ở nhóm tám, hơn rõ ở nhóm mờ, và hơn nhiều ở nhóm vòng. Nếu bảng của bạn ra đúng hình dạng đó thì cây quyết định ở Hình 28.8 đưa bạn thẳng tới lối hai.

H. Điền bốn cổng của chương 27 cho ba họ

Gộp tất cả lại, Bảng 28.4 là câu trả lời của chương này cho khung bốn cổng. Đây là bảng nên chụp lại và dán lên tường.

Bảng 28.4. Bốn cổng của chương 27, điền cho ba họ phương pháp. Cột cuối là kết luận của mình tính tới tháng 9/2026, và nó có điều kiện: nó giả định bạn *đã* có một hệ stereo-inertial tinh chỉnh kỹ chạy trên Orin.

Họ	Bài toán có thật?	Đo được?	Vừa ngân sách Orin?	Kết luận
Bộ mô tả học được trên điểm ORB	Có: chế độ hồng đo đổi sáng, đo được bằng biên δ	Có: nhóm mờ và nhóm vòng của mục 6G	Nhiều khả năng vừa: mạng chỉ chạy trên vài trăm ô ảnh nhỏ	Đáng thử trước tiên
Đặc trưng học được thay cả FAST	Có, nhưng chỉ chứa hai trong bốn chế độ hồng	Có, nhưng phải đo lại toàn bộ chuỗi vì tăng kim tự tháp mất	Cần đo; XFeat và ALIKED là hai ứng viên duy nhất đáng thử	Chỉ khi <i>oracle</i> cho trần cao
Bộ ghép cặp học được mỗi khung	Bài toán gán toàn cục không phải nút thất lúc tám	Có, nhưng cải thiện dễ bị năm cơ chế hấp thụ	Gần như chắc chắn không, do chi phí bậc hai	Chưa đáng
Bộ ghép cặp học được ở tầng vòng	Có: đây đúng là chế độ hồng thứ ba	Có: đo bằng recall vòng, xem chương 29	Có, vì chỉ chạy ở cỡ 1 Hz trên vài cặp ứng viên	Đáng, và là phương án tốt nhất
Ghép dày không bộ phát hiện	Có: vùng ít vân là chế độ hồng thật	Khó, vì phải dựng thêm tầng danh tính mới đo được trong hệ	Không cho mỗi khung; có thể cho tầng vòng nếu tối ưu kỹ	Chưa đáng, trừ khi cảnh của bạn rất ít vân

Một lưu ý về cách đọc bảng này. Nó không nói các phương pháp ở hàng dưới kém hơn. Nó nói chúng được thiết kế cho một điểm vận hành khác, và ở điểm vận hành đó chúng thắng rõ. Nhầm hai câu này là nhầm phổ biến nhất khi đọc văn liệu ghép cặp.

CẦN NHỚ

Trước khi tích hợp bất cứ mạng nào, hãy chạy *oracle* ghép cặp: thay đầu ra của bộ ghép cặp bằng cặp ghép dựng từ chân trị, rồi đo APE. Con số đó là *trần* của mọi công sức frontend. Nếu trần chỉ hơn hiện tại vài phần trăm thì câu trả lời cho cả chương này là “chưa đáng”, và bạn biết điều đó sau hai ngày thay vì sau ba tháng. Nếu vẫn muốn đi tiếp, hãy đi theo

con đường rẻ nhất: giữ ORB cho bám khung hình ở 30 Hz, và chỉ đặt bộ ghép cặp học được ở tầng kiểm chứng đóng vòng, chạy ở cỡ 1 Hz.

28.7 Giới hạn và trường hợp hỏng

Giới hạn lớn nhất của chương này là giới hạn của bằng chứng. Gần như mọi con số so sánh đặc trưng đều đo trên cặp ảnh hai khung nhìn rộng, không có tiên nghiệm chuyển động, không có ràng buộc thời gian thực. Chưa có một nghiên cứu ablation nào đủ sạch trả lời câu “thay ORB bằng X thì APE trên EuRoC đổi bao nhiêu, với cùng ngân sách tính toán”. Vài công trình có báo cáo, nhưng chúng thường đổi nhiều thứ cùng lúc và không cố định ngân sách. Nên mọi kết luận trong chương này về APE là *suy luận từ cơ chế*, không phải kết quả đã đo, và mình muốn nói rõ điều đó.

Chế độ hỏng thứ hai là hỏng khi ra ngoài phân phối huấn luyện. Bộ đặc trưng học được mang theo giả định của dữ liệu huấn luyện nó. Nếu cảnh của bạn là hầm mỏ, ban đêm, hoặc ảnh hồng ngoại, thì mọi thứ hạng trên bảng xếp hạng ngoài trời ban ngày đều không chuyển sang được. Tệ hơn, kiểu hỏng cũng đổi: ORB hỏng bằng cách không cho điểm nào, còn mạng hỏng bằng cách cho ra các điểm tự tin nhưng sai. Kiểu hỏng thứ hai độc hơn nhiều với một đường ống có RANSAC, vì các cặp sai *nhất quán* với nhau thì RANSAC không loại được.

Chế độ hỏng thứ ba mang tính hệ thống. Chương này giả định frontend tách rời được khỏi phần còn lại, để có thể thay riêng nó. Mục 6E cho thấy giả định đó sai ở một chỗ cụ thể: tầng kim tự tháp là cầu nối giữa frontend và ma trận thông tin của backend. Khi một khối bị nối với khối khác qua một kênh không có tên trong tài liệu như vậy, ablation cho ra con số không có nghĩa như ta tưởng — đúng cảnh báo cuối chương 24.

Cuối cùng là giới hạn về tính chẩn đoán được. Khi ORB không tìm ra điểm, bạn mở ảnh lên và thấy ngay vì sao. Khi một mạng cho ra tập điểm kỳ lạ ở một cảnh cụ thể, bạn không có cách nào đi ngược từ triệu chứng về nguyên nhân trong vài giờ. Với một hệ đang chạy sản phẩm, mất khả năng chẩn đoán là một chi phí thật, và nó không xuất hiện trong bất kỳ bảng so sánh nào.

Giới hạn thứ năm là phạm vi mà chương này cố ý bỏ qua. Frontend của ORB-SLAM3 không chỉ ghép giữa hai khung hình liên tiếp. Nó còn ghép giữa hai ảnh stereo bằng ràng buộc đường epipolar, với tinh chỉnh dưới mức điểm ảnh dọc đường quét, để lấy độ sâu. Nó còn ghép giữa khung hình và bản đồ cục bộ theo phép chiếu. Ba bài toán ghép đó có ràng buộc khác nhau và điểm vận hành khác nhau. Một bộ đặc trưng học được phải làm tốt cả ba, nhưng mọi bảng xếp hạng chỉ đo bài toán thứ nhất. Đây là một khoảng trống bằng chứng nữa, và nó cùng loại với khoảng trống đã nói ở hộp cam bẫy.

Cuối cùng là một giới hạn về chính chương này. Nó viết theo giọng hoài nghi, và giọng đó phù hợp với một người *đã có* một hệ thống chạy tốt. Với một người đang xây từ đầu, cân nhắc đảo ngược: một đường ống dựa trên SuperPoint cộng LightGlue cho kết quả chấp nhận được nhanh hơn nhiều so với việc tinh chỉnh ORB trong hai năm. Kết luận “chưa đáng” của chương này là kết luận có điều kiện, và điều kiện đó là hệ thống hiện có của bạn.

28.8 Thực hành

Repo và tài nguyên. [kornia](#) có sẵn HardNet, DISK, LoFTR và LightGlue dưới một giao diện chung, nên nó là chỗ rẻ nhất để so nhiều phương pháp mà không dựng lại từng repo. [magicleap/SuperGluePretrainedNetwork](#) và [cvgl/LightGlue](#) cho hai bộ ghép cặp;

`cvg/Hierarchical-Localization` cho một đường ống hoàn chỉnh từ đặc trưng tới định vị; `verlab/accelerated_features` cho XFeat; `zju3dv/LoFTR` cho nhánh không bộ phát hiện. Về hạ tầng dài hạn thì COLMAP và `opencv` bền hơn hẳn mọi tên mô hình ở trên. Danh sách này chốt tại tháng 9/2026 và sẽ cũ trong sáu tới mười hai tháng; riêng bốn chế độ hồng ở mục 2 thì không cũ.

Câu hỏi kiểm tra hiểu. Viết công thức của độ dài vệt mờ tối hạn theo w, C, t , rồi nói ngưỡng tốc độ góc đổi theo hướng nào và theo tỉ lệ nào khi bạn hạ ngưỡng FAST xuống một nửa, và khi cảnh tối đi làm tương phản giảm một nửa? Nếu bạn giảm số điểm còn một phần ba thì chi phí của một bộ ghép cặp kiểu SuperGlue giảm mấy lần? Trong sáu thứ phải bỏ ở mục 6E, thứ nào chạm vào backend chứ không chỉ chạm vào frontend?

Mini project trong khối *duan bên dưới*. Hai dự án ở cuối chương nối tiếp nhau: một cái đo trần, một cái đo ngưỡng hồng. Chạy cái đo trần trước.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● FAST	Bộ phát hiện góc của ORB. Nó nhận một điểm là góc nếu có 9 điểm ảnh liên tiếp trên vòng tròn bán kính 3 cùng lệch quá ngưỡng t . Nó hồng khi ảnh mờ, vì trung bình theo hướng chuyển động kéo tương phản xuống dưới t và cắt đứt cung liên tiếp.
●●● BRIEF	Bộ mô tả nhị phân 256 bit, mỗi bit là kết quả một phép so sánh “điểm A sáng hơn điểm B”. Nó bất biến với đổi độ sáng đơn điệu toàn cục, nhưng hồng khi biên của các phép so sánh co lại và bit bắt đầu lật ngẫu nhiên.
●●● Biên δ của một phép so sánh	Chênh lệch cường độ thật của một cặp thử trong BRIEF. Xác suất lật bit là $\Phi(-\delta/\sigma\sqrt{2})$, nên chính δ chứ không phải độ sáng tuyệt đối quyết định bộ mô tả còn dùng được hay không.
●● MMA	Trong các cặp được ghép, tỉ lệ cặp có sai số vị trí dưới một ngưỡng điểm ảnh. Nó đo bộ mô tả cộng bộ ghép cặp, và nó là thước đo phổ biến nhất trên HPatches.
●● HardNet	Bộ mô tả học bằng mất mát triplet với mẫu âm khó nhất trong batch. Chọn mẫu âm khó nhất là cách xấp xỉ đúng đối thủ lúc chạy, tức ứng viên nhì trong phép thử tỉ lệ.
●●● Làm giàu bằng homography	Cách SuperPoint chế nhãn cho bộ phát hiện: áp hàng trăm homography ngẫu nhiên, phát hiện, ánh xạ ngược, rồi giữ các điểm sống sót. Giới hạn của nó là phân phối huấn luyện chỉ có biến dạng phẳng, không có ảnh mờ và không có thị sai.
●●● SuperPoint	Một mạng, hai đầu ra: bản đồ điểm và bộ mô tả 256 chiều, huấn luyện tự giám sát bằng homography. Nó là mặc định của cả dòng, nhưng chưa từng thấy ảnh mờ trong huấn luyện trừ khi bạn tự thêm vào.
●● R2D2	Tách tính lặp lại khỏi tính phân biệt thành hai bản đồ riêng. Lý do: góc của một viên gạch rất lặp lại nhưng ghép luôn sai, vì nó giống năm trăm viên khác.
● DISK	Coi chọn điểm và chấp nhận cặp ghép là hành động lấy mẫu, thường cặp đúng và phạt cặp sai, rồi học bằng <i>gradient</i> chính sách. Nó tối ưu thẳng mục tiêu rời rạc thay vì một hàm thay thế khả vi. DeDoDe đi hướng khác: tách hẳn việc huấn luyện bộ phát hiện khỏi bộ mô tả, để bộ phát hiện không bị kéo về phía sở thích của bộ mô tả.
●● XFeat	Đặc trưng thiết kế cho CPU và biên: mạng nông, chạy ở một phần tư độ phân giải, và tinh chỉnh vị trí bằng một mạng nhỏ chạy sau khi đã ghép cặp. Đây là công trình duy nhất trong dòng lấy ngân sách độ trễ làm ràng buộc thiết kế.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
• ALIKED	Học các vị trí lấy mẫu biến dạng được cho vùng hỗ trợ của bộ mô tả, thay cho ô vuông cố định. Nó nhắm thẳng chế độ hồng do đối góc nhìn, vì ô vuông cố định méo khi mặt phẳng nghiêng.
••• SuperGlue	Bộ ghép cặp dùng chín tầng chú ý xen kẽ trong và giữa hai ảnh, rồi giải bài toán gán bằng Sinkhorn có thùng rác. Nó đổi quyết định cục bộ lấy quyết định toàn cục, và trả bằng chi phí bậc hai theo số điểm.
••• LightGlue	Cùng ý tưởng SuperGlue nhưng rút ngắn ba chỗ: thoát sớm khi cặp ảnh dễ, cắt bỏ điểm không ghép được giữa chừng, và bỏ vòng lặp Sinkhorn. Cắt điểm là chỗ tiết kiệm lớn nhất vì chi phí là bậc hai.
••• LoFTR	Ghép không cần bộ phát hiện: ghép dày ở một phần tám độ phân giải bằng chú ý tuyến tính, rồi tinh chỉnh dưới mức điểm ảnh trong cửa sổ nhỏ. Nó cứu vùng ít vân, nhưng chỉ ghép được hai ảnh một lúc và không để lại bộ mô tả nào để lưu.
••• Oracle ghép cặp	Thay đầu ra bộ ghép cặp bằng cặp ghép dựng từ chân trị rồi đo APE. Kết quả là trần của mọi cải tiến frontend, và nó tốn hai ngày thay vì ba tháng.
••• Vì sao chi phí SuperGlue là bậc hai?	Vì mỗi điểm phải nhìn mọi điểm khác, nên ma trận điểm số có N^2 ô. Kế toán thô là $36N^2d$ phép nhân cộng cho chín tầng, tức nhân đôi số điểm thì chi phí gấp bốn.
••• Vì sao bỏ bộ phát hiện cứu được vùng ít vân?	Vì nó không làm tường có vân, mà đổi loại bằng chứng được dùng. Chồng tầng chú ý đưa thông tin từ mép tường vào ô ở giữa tường, nên cặp ghép được quyết bằng bằng chứng phi cục bộ thay vì cục bộ.
••• Vì sao thay bộ phát hiện lại là thay cả backend?	Vì tầng kim tự tháp không chỉ dùng để phát hiện. Nó đặt bán kính tìm kiếm, kiểm tra nhất quán tỉ lệ, và đặt ma trận thông tin của mọi cạnh tái chiếu qua <code>mvInvLevelSigma2</code> . Bộ phát hiện học được không trả về nó, nên thay frontend là thay luôn trọng số của backend.
••• Tỉ lệ inlier tăng mạnh mà APE không đổi — nghĩa là gì?	Nghĩa là frontend không phải ràng buộc chặt. Năm cơ chế hấp thụ: bão hoà sau vài chục cặp trái đều, phân bố quan trọng hơn số lượng, bộ làm bền đã lọc rồi, APE do đuôi phân phối quyết định, và điểm vận hành khác với bảng xếp hạng.
••• Vào hành lang tối, tracking vẫn bám nhưng bản đồ ngừng lớn — nghĩa là gì?	Khoảng cách Hamming của cặp ghép đúng đã trôi vào khoảng giữa <code>TH_LOW</code> và <code>TH_HIGH</code> . Cặp ghép còn qua được ngưỡng lỏng của bám khung hình, nhưng rớt ở ngưỡng chặt của tam giác hoá điểm mới.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
kornia	Hiện thực chung của HardNet, DISK, LoFTR, LightGlue dưới một giao diện — chỗ so sánh rẻ nhất	Hạ tầng; bền hơn tên mô hình
hloc	SuperPoint cộng SuperGlue hoặc LightGlue trong một đường ống định vị hoàn chỉnh	Đây là nơi dòng này thật sự được dùng trong sản phẩm
COLMAP với đặc trưng học được	Ghép cặp khung nhìn rộng để tái dựng ba chiều — chỗ ghép cặp học được thắng rõ ràng nhất	Không có ràng buộc thời gian thực, nên đúng điểm vận hành
Image Matching Challenge [240]	Bảng xếp hạng quyết định thứ hạng của cả dòng này	Các giải pháp thắng thường là tổ hợp nhiều mô hình, xa mức thời gian thực

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Chính ORBmatcher của bạn	Phép thử tỉ lệ và biểu đồ nhất quán góc là hai bản thô của đúng hai ràng buộc mà SuperGlue đưa vào trong mô hình	Ví dụ nội bộ; đọc lại mã theo góc nhìn này rất đáng

MINI PROJECT SLAM · Đo trần của frontend bằng *oracle* ghép cặp · 1–2 ngày

Mục tiêu	trả lời dứt điểm câu “đổi bộ đặc trưng thì APE tốt lên nhiều nhất là bao nhiêu”, trước khi tích hợp bất cứ mạng nào.
Dữ liệu	hai chuỗi EuRoC, một dễ và một khó có chuyển động nhanh (chẳng hạn V2_03), cùng chân trị tư thế.
Các bước	dựng một chế độ <i>oracle</i> trong ORBmatcher: với mỗi điểm bản đồ, chiếu nó bằng tư thế chân trị và nhận cặp ghép nếu điểm đặc trưng gần nhất nằm trong bán kính hai điểm ảnh, bỏ qua hoàn toàn khoảng cách mô tả. Chạy ba lần mỗi cấu hình để có sần nhiễu. So ba cấu hình: ORB thường, <i>oracle</i> toàn chuỗi, và <i>oracle</i> chỉ bật trong các đoạn có tốc độ góc cao nhất chuỗi — ngưỡng thì lấy từ dự án thứ hai bên dưới, tức là từ số đo của chính bạn.
Xong khi	bạn có một bảng ba dòng kèm độ lệch chuẩn, và phát biểu được “trần của mọi cải tiến frontend trên chuỗi này là X cm”. Kiểm tra chéo: hiệu giữa dòng hai và dòng ba cho biết bao nhiêu phần của trần đó nằm riêng trong các đoạn chuyển động nhanh.
Nối với	E/24 cho cách đọc con số và cách tính sần nhiễu; F/29 sẽ dùng lại đúng chế độ <i>oracle</i> này ở tầng đóng vòng.

MINI PROJECT SLAM · Tìm ngưỡng mờ giết FAST trên chính chuỗi của bạn · nửa ngày

Mục tiêu	biến công thức $d^* = wC/t$ ở mục 2A thành một con số của bạn, đo trên dữ liệu thật, rồi biến nó thành một cờ chắn đoán chạy trực tuyến.
Dữ liệu	một chuỗi EuRoC có IMU, cộng nhật ký số điểm FAST phát hiện được mỗi khung hình.
Các bước	ghi hai chuỗi số theo thời gian: chuẩn của tốc độ góc từ IMU, và số điểm đặc trưng rút được. Vẽ số điểm theo tốc độ góc dưới dạng biểu đồ tán xạ. Lập lại với iniThFAST bằng 20 và bằng 12 để thấy đường cong dịch chuyển. Song song đó, tự tính dự đoán của mô hình: đo w và C trên vài ô ảnh điển hình của chuỗi, lấy tiêu cự từ file hiệu chỉnh và thời gian phơi sáng từ nhật ký camera, rồi ráp vào $\omega \approx (wC/t)/(f \Delta t)$.
Xong khi	bạn đọc ra được từ biểu đồ một ngưỡng tốc độ góc mà quá đó số điểm sụp, và bạn có một con số dự đoán từ mô hình để so với nó. Hai con số đều là của bạn, không mượn của ai. Kiểm tra chéo: hạ ngưỡng FAST một nửa thì ngưỡng tốc độ góc đo được phải dịch theo đúng hướng mà công thức nói, và dịch cỡ đúng tỉ lệ; nếu không thì mô hình một chiều đang thiếu một cơ chế, và việc tìm ra cơ chế đó đáng giá hơn cả con số.
Nối với	nối thẳng vào dự án trên: các đoạn vượt ngưỡng chính là các đoạn cần bật <i>oracle</i> riêng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Đại lượng bị mờ tấn công là tương phản cục bộ còn lại. Mô hình trung bình một chiều cho $C_{\text{hiệu dụng}} \approx C \min(1, w/d)$, nên FAST mờ khi vệt mờ đạt $d^* \approx wC/t$. Đổi sang tốc độ góc bằng $\omega \approx (d^*/f)/\Delta t$, với f là tiêu cự tính theo điểm ảnh và Δt là thời gian phơi sáng. Ba tham số quyết định là ngưỡng FAST, tiêu cự và thời gian phơi sáng; thay số của chính bạn vào thì ra ngưỡng của bạn. Ở đây không đưa con số vì nó chưa được đo trên hệ của bạn.
- A2. Vì thứ giết BRIEF không phải giá trị mà là biên, và chính xác hơn nữa là tỉ số giữa biên và nhiễu. Xác suất lật một bit là $\Phi(-\delta/\sigma\sqrt{2})$, nên Hamming kỳ vọng của cặp đúng là $256 \Phi(-\delta/\sigma\sqrt{2})$. Khi δ/σ tụt về cùng bậc với 1, hàm Φ vào vùng dốc nhất và Hamming nhảy vọt — đủ để vượt TH_LOW trong khi vẫn dưới TH_HIGH, tức là qua cửa lỏng nhưng rớt ở mọi cửa chặt.

- A3. Vì tiên nghiệm biến bài toán gán toàn cục thành bài toán gán trong một cửa sổ hẹp: mỗi điểm chỉ còn một nhóm ứng viên, nên chi phí từ N^2 xuống $N \cdot k$ với k là số ứng viên trong cửa sổ, gần như tuyến tính. Không bài báo nào tận dụng vì bảng xếp hạng của họ đo cặp ảnh hai khung nhìn rộng, nơi không tồn tại tiên nghiệm nào để tận dụng.
- A4. Vì năm cơ chế hấp thụ nằm giữa frontend và APE: bảo hoà sau vài chục cặp trải đều, phân bố quan trọng hơn số lượng, các bộ làm bền đã lọc phần lớn rồi, APE do vài giây tệ nhất quyết định trong khi ba thước đo kia là trung bình, và điểm vận hành của bảng xếp hạng khác hẳn của bạn.
- A5. Vì bỏ bộ phát hiện thì mất bộ mô tả, mà mất bộ mô tả thì mất danh tính của điểm. SLAM cần ghép một khung mới với hàng vạn điểm bản đồ, tức một-nhiều; ghép dày chỉ làm được ảnh với ảnh. Hệ quả kèm theo: túi từ nhị phân không dùng được nữa, nên mất luôn định vị lại và lấy ứng viên đóng vòng.

NÓI LẠI THẬT ĐƠN GIẢN

Hai người muốn gặp nhau ở một góc phố nhưng chỉ nói chuyện được qua điện thoại. Cách cũ: mỗi người tự ghi lại hai trăm năm mươi sáu điều nhỏ về chỗ mình đứng, kiểu “viên gạch bên trái sẫm hơn viên bên phải”, rồi hai bên đọc danh sách và so xem có khớp không. Cách này chạy rất nhanh và rất tốt, miễn là trời sáng và hai người đứng yên. Nếu có sương mù, hoặc nếu ai đó vừa chạy vừa nhìn, thì mọi điều nhỏ đều trở nên nửa vời, và hai danh sách toàn điều nửa vời thì khớp với mọi góc phố trong thành phố.

Cách thứ hai: đừng ghi riêng, hãy nói chuyện với nhau. “Tôi thấy một mái hiên đỏ bên trái, bạn có thấy không?” Nhờ trao đổi mà từng chi tiết mơ hồ được đặt vào ngữ cảnh, nên vẫn phân biệt được viên gạch này với viên gạch kia. Cái giá là cuộc gọi tốn thời gian, và thời gian tăng theo bình phương số chi tiết mỗi bên đem ra bàn.

Cách thứ ba: bỏ hẳn việc chọn ra vài chi tiết, so nguyên cả khung cảnh. Cách này ăn được cả bức tường trắng trơn. Nhưng nó không để lại một cuốn sổ nào, nên ngày mai muốn tìm lại đúng góc phố ấy thì phải gọi điện lại từ đầu, với từng người một.

Chỗ không rút gọn được: chuyện cái thước ở đầu ống lại quyết định trọng số ở cuối ống. Trong hệ thật, việc một điểm được nhìn thấy ở mức phóng to nào cũng chính là thứ quyết định người ta tin điểm đó tới đâu khi ghép mọi thứ lại. Đổi cách chọn điểm mà quên điều này thì bạn vô tình tuyên bố mọi quan sát đáng tin như nhau, và không có cách nào nói điều đó bằng ngôn ngữ đời thường mà không mất đúng phần quan trọng.

Một câu: *Đặc trưng học được chứa được hai trong bốn chỗ ORB gây, nhưng chỗ nó chứa lại hiếm khi là chỗ quỹ đạo của bạn đang hỏng — nên hãy đo trần trước, rồi mới tiêu tiền.*

28.9 Kết nối

[F/27-dat-hoc-sau-vao-dau](#) là tiền đề trực tiếp: chương này là lần đầu tiên khung bốn cổng được điền đầy đủ cho một khối cụ thể, và kết quả điền ra là “chưa đáng cho mỗi khung hình, đáng cho tầng đóng vòng”. [F/29-dinh-vi-lai-va-dong-vong](#) nhận đúng kết luận đó làm điểm xuất phát, và trả lời câu hỏi mà chương này để lại: nếu đặt bộ ghép cặp học được ở tầng đóng vòng thì tầng nào của đóng vòng thật sự được lợi. [F/30-do-sau-dong-quang-va-mo-hinh-nen](#) là nhánh đối lập: thay vì ghép cặp thừa tốt hơn, nó bỏ ghép cặp và dùng dòng quang dày — cùng nỗi đau chuyển động nhanh, cách tiếp cận ngược lại. [F/34-imu-bat-dinh-va-trien-khai](#) nhận phần chương này cố ý để lại: nếu *oracle* cho thấy vấn đề nằm ở *trọng số* chứ không ở *cặp ghép*, thì lời giải nằm ở đó. [E/24-thi-nghiem-va-ablation](#) là công cụ của cả chương, vì *oracle* experiment là thứ duy nhất biến câu hỏi “có đáng không” thành một con số. [C/16](#) giải thích cơ chế *attention* mà SuperGlue và LoFTR đều dựa vào.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao ảnh mờ và đối độ sáng cần hai loại thuộc khác nhau, dù triệu chứng bề ngoài đều là “ít cặp ghép”?
2. Mô hình một chiều dự đoán một ngưỡng vết mờ $d^* = wC/t$, nhưng thực tế FAST chết sớm hơn ngưỡng đó — cơ chế nào giải thích phần chênh, và nó ảnh hưởng thế nào tới điều kiện của bài toán tư thế?
3. SuperPoint được huấn luyện bằng homography ngẫu nhiên. Điều đó khiến nó *không* học được ba hiện tượng nào, và hiện tượng nào trong ba cái đó là nỗi đau số một của bạn?
4. Nếu bạn chỉ được giữ một trong hai — ngữ cảnh toàn cục của SuperGlue hay ràng buộc một-một của Sinkhorn — thì giữ cái nào, và trên loại cảnh nào lựa chọn đó đảo ngược?
5. Tính lặp lại, MMA và tỉ lệ inlier cùng tăng mạnh mà APE không đổi: cơ chế nào trong năm cơ chế hấp thụ bạn kiểm chứng được rẻ nhất, và bằng phép đo gì?
6. Vì sao việc bộ phát hiện học được không trả về tăng kim tự tháp lại là một thay đổi ở *backend* chứ không chỉ ở *frontend*?
7. Nếu *oracle* ghép cặp cho thấy trần chỉ tốt hơn hiện tại vài phần trăm, bạn kết luận gì, và bạn ghi lại kết luận đó kèm những điều kiện nào để nó còn dùng được sau sáu tháng?

Nhận dạng lại địa điểm và đóng vòng (Place Recognition and Loop Closure)

PHỤ THUỘC F/27-dat-hoc-sau-vao-dau (khung bốn cổng để quyết định thay hay không thay); F/28-dac-trung-va-ghep-cap (đặc trưng và ghép cặp học được — chương này dùng lại chúng ở tầng kiểm chứng); B/07-chia-du-lieu-va-danh-gia (giao thức đánh giá, recall và precision).

TÓM TẮT

Chương này mô phỏng vòng thành bốn tầng, rồi dạy cách đo xem tầng nào đang giết vòng. Đọc xong bạn đặt được biển đếm đúng chỗ, dựng được nhãn đúng-sai từ chân trị, và phân biệt được “thiếu mẫu” với “trần inlier”. Bạn cũng biết khi nào một bộ mô tả toàn ảnh học được thật sự hơn túi từ, và khi nào nó chỉ dời nút thắt sang chỗ khác. Nếu chỉ nhớ một điều: người ta hay đổ lỗi cho tầng lấy ứng viên, nhưng trên một hệ stereo-inertial thì tầng kiểm chứng hình học — chỗ RANSAC ước lượng Sim3 — thường là bộ lọc mạnh hơn nhiều, và chương này dạy bạn cách tự đo xem điều đó có đúng với hệ của mình không thay vì tin vào một con số của người khác.

KIẾN THỨC CẦN CÓ TRƯỚC

- **đóng vòng (loop closure)** — việc nhận ra rằng robot đang ở lại một nơi đã đi qua, rồi thêm một ràng buộc nối hai thời điểm đó trong đồ thị tư thế. Đây là cơ chế duy nhất xoá được sai số tích lũy; không có nó thì mọi hệ đo bằng chuyển động tương đối đều trôi vô hạn.
- **ứng viên (candidate)** — một khung hình khoả cũ mà tầng truy hồi cho rằng có thể là cùng một nơi với khung hình hiện tại. Ứng viên chỉ là một đề xuất, chưa phải một vòng.
- **túi từ (bag of words)** — cách mô tả một ảnh bằng biểu đồ tần suất các “từ thị giác”, tức các cụm đặc trưng cục bộ đã được gom sẵn trong một từ điển. Hai ảnh giống nhau thì biểu đồ giống nhau, nên so ảnh trở thành so hai vector thưa.
- **mô tả toàn ảnh (global descriptor)** — một vector duy nhất, thường vài trăm tới vài nghìn chiều, tóm cả bức ảnh. Truy hồi địa điểm chính là tìm láng giềng gần nhất của vector này trong cơ sở dữ liệu.
- **Sim3** — phép biến đổi gồm xoay, tịnh tiến và một hệ số tỉ lệ chung, tức bảy bậc tự do. Bản đồ đơn ảnh có tỉ lệ trôi theo thời gian, nên hai đầu của một vòng thường lệch cả tỉ lệ chứ không chỉ lệch tư thế.
- **RANSAC và inlier** — RANSAC rút ngẫu nhiên một tập nhỏ nhất các khớp, dựng một giả thuyết từ đó, rồi đếm xem bao nhiêu khớp còn lại phù hợp với giả thuyết ấy. Các khớp phù hợp gọi là inlier; giả thuyết nhiều inlier nhất được giữ lại.
- **perceptual aliasing** — hiện tượng hai nơi khác nhau trông giống hệt nhau — hành lang lặp, dây cửa giống nhau, hàng cây đều. Đây là nguồn chính của vòng sai, và nó không giảm khi ta tăng độ phân giải ảnh.
- **recall và precision** — recall là tỉ lệ vòng có thật mà hệ phát hiện được; precision là tỉ lệ vòng được chấp nhận mà thật sự đúng. Hai số này đánh đổi nhau qua một ngưỡng, nên nói một số mà giấu số kia là vô nghĩa.

CÂU HỎI MỞ ĐẦU

- Q1. Hệ của bạn duyệt hàng chục nghìn ứng viên mà gần như không đóng được vòng nào. Bạn đặt bao nhiêu biến đếm, ở đúng những chỗ nào, để biết tầng nào đang giết vòng — và vì sao đặt sai một dòng làm bạn đổ tội nhầm cho suốt nhiều tháng?
- Q2. Nếu tầng kiểm chứng loại 99% ứng viên, đó là bằng chứng nó hỏng hay bằng chứng nó đang làm đúng việc? Bạn phân biệt hai khả năng đó bằng phép đo nào?
- Q3. Lý thuyết RANSAC nói rằng giảm tập mẫu tối thiểu từ ba điểm xuống hai điểm sẽ nâng xác suất hội tụ lên rất nhiều. Vì sao lời hứa đó vẫn có thể không đề ra thêm một vòng nào, và bạn kiểm giả định nào của nó trước khi bỏ công cài đặt?
- Q4. NetVLAD hơn túi từ trên mọi bảng xếp hạng nhận dạng địa điểm. Vì sao thay DBoW2 bằng nó vẫn có thể không đóng thêm được một vòng nào trên hệ của bạn?
- Q5. Vì sao điểm vận hành đúng của một bộ phát hiện vòng lại nằm lệch hẳn về phía precision, tới mức bạn sẵn sàng bỏ lỡ chín vòng để tránh một vòng sai?

29.1 Đặt vấn đề

Một hệ stereo-inertial đã tinh chỉnh kỹ vẫn trôi. Trôi là hệ quả tất yếu của việc cộng dồn các phép đo tương đối. Đóng vòng là cơ chế duy nhất xoá được phần trôi đã tích lũy, vì nó là ràng buộc duy nhất nối hai thời điểm cách xa nhau. Vì vậy khi vòng không nổ, sai số cuối chuỗi không có gì kéo về.

Câu hỏi mà hầu hết mọi người đặt ở điểm này là câu hỏi sai. Họ hỏi: “bộ nhận dạng địa điểm của tôi yếu, có nên thay túi từ bằng một mạng học được không?”. Câu hỏi ấy giả định trước rằng nút thắt nằm ở tầng truy hồi. Giả định đó thường sai, và nó sai theo một cách rất tốn kém: bạn bỏ ba tháng thay một tầng vốn không phải thủ phạm.

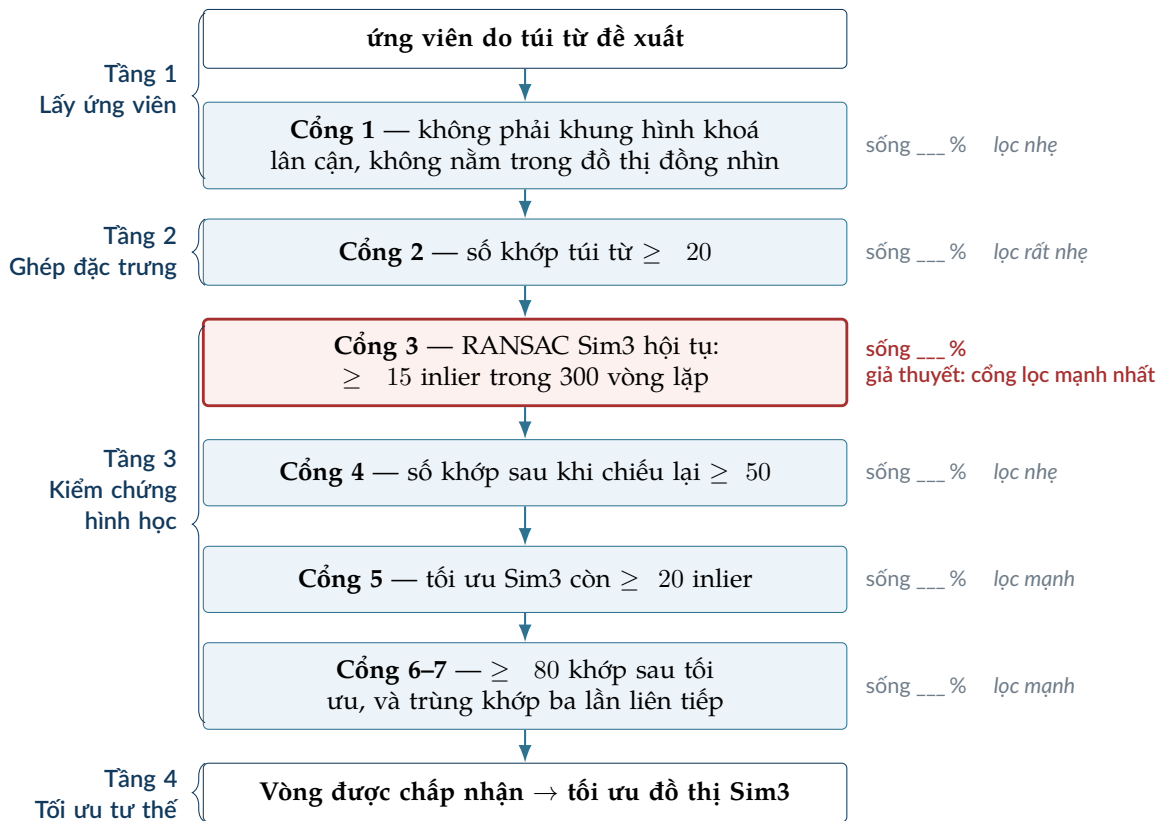
Đóng vòng không phải một khối. Nó là một dây chuyền bốn tầng, mỗi tầng là một bộ lọc riêng. Tầng lấy ứng viên đề xuất các khung hình khoá cũ có thể trùng nơi. Tầng ghép đặc trưng dựng các cặp điểm tương ứng giữa hai khung hình. Tầng kiểm chứng hình học hỏi xem tập cặp đó có giải thích được bằng một phép biến đổi cứng duy nhất không. Tầng tối ưu nhận ràng buộc đã được chấp nhận và phân bổ lại sai số cho cả bản đồ.

Mỗi tầng loại bỏ một phần ứng viên. Tổng của bốn tầng là một cái phễu rất hẹp. Nếu bạn chỉ nhìn đầu ra của phễu, bạn thấy đúng một con số: số vòng đóng được. Con số đó không cho biết tầng nào hẹp. Giá trị của chương này là biến câu “vòng không nổ” thành bốn con số riêng, mỗi con số cho một tầng, đo được trong một buổi chiều.

Sau khi có bốn con số đó, chương mới trả lời câu hỏi ban đầu. Bộ mô tả toàn ảnh học được giải quyết việc gì mà túi từ không giải quyết được, và giải quyết ở tầng nào. Định vị lại bằng mạng nơ-ron đứng ở đâu trong bốn tầng ấy. Ghép cặp học được đưa vào tầng kiểm chứng thì đổi gì. Và cuối cùng, làm sao đo cho đúng, khi mà một vòng sai đắt hơn nhiều lần một vòng bị bỏ lỡ.

Phần F bám bốn nỗi đau của một hệ đã chạy thật: chuyển động nhanh, ảnh mờ do chuyển động, đóng vòng, và đặt trọng số cho IMU. Chương này chạm thẳng vào nỗi đau thứ ba, và chỉ chạm gián tiếp vào hai nỗi đau đầu. Chuyển động nhanh và ảnh mờ xuất hiện ở đây dưới dạng hệ quả: chúng làm chất lượng cặp tương ứng tụt, và chính chất lượng cặp là thứ quyết định tầng ba. Trọng số IMU không thuộc chương này, trừ một chỗ: phương trọng lực từ IMU được dùng để rút bậc tự do trong bài toán kiểm chứng.

Hình 29.1 là bản đồ của cả chương. Nó vẽ bốn tầng cùng chuỗi cổng cụ thể của một hệ kiểu ORB-SLAM3, và chỗ để bạn tự điền tỉ lệ sống sót đo được trên hệ của mình. Đọc nó trước, rồi mỗi mục sau chỉ là một trạm trên đúng cái phễu đó.



Hình 29.1. Bốn tầng của đóng vòng và chuỗi cổng cụ thể thực hiện chúng; các ngưỡng ghi trong ô là hằng số trong mã nguồn của một hệ kiểu ORB-SLAM3, không phải kết quả đo. Các ô “sống ___ %” là chỗ trống bạn điền sau khi chạy phép đo ở mục C: mỗi tỉ lệ là *có điều kiện*, tức phần trăm ứng viên tới được cổng đó mà vượt qua nó. Nhân định tính bên phải là giả thuyết làm việc, không phải số đo: cổng kiểm chứng hình học thường là chỗ lọc mạnh nhất trong bốn tầng, và trên nhiều hệ thống nó cũng là nơi phần lớn ứng viên đúng bị loại. Chương này không nói tỉ lệ ấy bằng bao nhiêu trên hệ của bạn — nó dạy bạn cách tự lấy con số đó, và mục D dạy tiếp cách biết tỉ lệ ấy là loại đúng hay loại sai.

TRỰC GIÁC

Hình dung một quy trình tuyển dụng bốn vòng. Cuối quý bạn thấy phần lớn ứng viên bị loại ở vòng phỏng vấn kỹ thuật. Phản xạ đầu tiên là sửa vòng phỏng vấn: nói thang điểm, đổi người hỏi. Nhưng nếu bạn mở lại hồ sơ những người bị loại, gần như tất cả họ thật sự không đạt. Vòng phỏng vấn đang làm đúng việc của nó. Vài người giỏi bị loại oan thì bị loại vì hồ sơ gửi lên quá sơ sài, nên người phỏng vấn không có gì để hỏi. Chỗ phải sửa là khâu chuẩn bị hồ sơ, ở phía trước. Tầng kiểm chứng hình học trong đóng vòng đúng đúng vị trí vòng phỏng vấn ấy.

29.2 Bốn tầng và cách đo tầng nào đang giết vòng (Anatomy and Instrumentation)

A. Mỗi tầng làm gì, và giả định ngầm của nó

Tầng một, lấy ứng viên, so một mô tả toàn ảnh với cơ sở dữ liệu và trả về danh sách ngắn. Giả định ngầm là hai ảnh cùng một nơi thì gần nhau trong không gian mô tả. Giả định này gây khi ánh sáng đổi, khi góc nhìn lệch lớn, và khi cảnh có cấu trúc lặp.

Tầng hai, ghép đặc trưng, dựng các cặp điểm tương ứng giữa khung hình hiện tại và ứng viên. Trong ORB-SLAM3, việc này được dẫn hướng bằng chính cây từ điển: chỉ so các đặc trưng rơi

vào cùng một nút từ vệt [40]. Giả định ngầm là hai đặc trưng của cùng một điểm ba chiều sẽ rơi vào cùng một nút. Đây là một giả định mạnh, và nó là chỗ số lượng cặp bị chặn trên.

Tầng ba, kiểm chứng hình học, hỏi một câu duy nhất: có tồn tại một phép Sim3 giải thích được đủ nhiều cặp không. Đây là câu hỏi có đáp án đúng hoặc sai, nên nó là tầng duy nhất trong bốn tầng thật sự *chứng minh* được điều gì. Giả định ngầm là tỉ lệ inlier đủ cao để RANSAC bắt trúng một mẫu sạch.

Tầng bốn, tối ưu, đưa ràng buộc mới vào đồ thị và phân bổ lại sai số. Giả định ngầm ở đây là ràng buộc mới đúng. Nếu nó sai, tầng bốn không phát hiện được; nó chỉ trung thành lan cái sai ra khắp bản đồ.

B. Hai cổng rẻ nhất: ràng buộc thời gian và đồng nhìn

Trước khi bàn tới bất cứ mô-đun học được nào, cần thấy hai cổng rẻ nhất trong cả chuỗi. Cả hai không tốn một phép nhân nào, và cả hai mua precision bằng cách trả recall.

Cổng thứ nhất loại các khung hình khoá đang nằm trong vùng đồng nhìn hiện tại. Không có nó, hệ sẽ “đóng vòng” với chính chỗ nó vừa đi qua ba giây trước. Ràng buộc kiểu đó đúng nhưng vô ích, vì nó không mang thông tin mới cho đồ thị. Trong bảng sống sót, đây chính là cổng một.

Cổng thứ hai là ràng buộc trùng khớp nhiều lần liên tiếp. ORB-SLAM3 chỉ chấp nhận một vòng khi ba khung hình khoá liên tiếp cùng chỉ về một nhóm đồng nhìn. Cơ chế đằng sau rất gọn: perceptual aliasing thường không bền theo thời gian. Hai hành lang giống nhau cho điểm cao ở một khung hình, nhưng hiếm khi giữ được điểm cao qua ba khung hình liên tiếp với hình học nhất quán.

Giá phải trả thì cụ thể và đo được. Bạn mất ba khung hình khoá đầu của mỗi lần đi lại. Với một lần đi lại rất ngắn, chẳng hạn robot chỉ lướt qua chỗ cũ trong hai giây, bạn mất luôn cả vòng. Đây là ví dụ sạch nhất trong toàn chương về việc đổi recall lấy precision, và nó xảy ra trước khi bất kỳ mạng nào được gọi.

Có một biến thể đáng biết. Thay vì chấm điểm từng khung hình khoá riêng lẻ, ta gom các khung hình khoá cùng nhóm đồng nhìn lại rồi cộng điểm của cả nhóm [245]. Cách này giảm nhiễu của điểm số truy hồi, vì một nơi đúng thường làm nhiều khung hình khoá lân cận cùng ăn điểm. Đó là một dạng kiểm chứng không gian rất rẻ, làm ngay tại tầng một.

C. Đặt biến đếm ở đâu, và cái bẫy dịch nhãn

Cách đo rất rẻ. Với mỗi ứng viên, tăng một biến đếm ngay trước mỗi cổng, rồi ghi ra một dòng nhật ký cho mỗi khung hình khoá. Sau vài chục lần chạy bạn có bảng sống sót từng tầng. Toàn bộ công việc là vài chục dòng mã và không ảnh hưởng thời gian thực.

Nhưng có một cái bẫy, và nó đã làm hỏng nhiều tuần phân tích trong chính dự án này.

CẠM BÃY

Nếu bạn đặt lệnh tăng biến đếm *ngay trước* câu lệnh `if` của cổng, thì cột mang tên cổng đó thật ra đang đếm “số ca đã vượt cổng *trước* nó”. Mọi nhãn cột bị dịch đi một bậc. Hậu quả rất cụ thể và rất khó thấy: cái cổng bị đổ tội luôn là cổng đứng *sau* thủ phạm thật, và vì bảng vẫn nhất quán với chính nó nên không có triệu chứng nào để bạn nghi ngờ. Trong chính dự án này, một bảng đặt sai kiểu đó đã dẫn phân tích đi lạc suốt nhiều tuần trước khi được đếm lại từ nhật ký thô. Cách phòng: đặt lệnh tăng biến đếm *bên trong* khối `if`, hoặc viết ngay trong tên biến rằng nó đếm “đã tới cổng nào” chứ không phải “đã qua cổng nào”. Bài học tổng quát: một cột nhật ký không tự nói ra ngữ nghĩa của nó, nên ngữ nghĩa phải nằm trong tên.

D. Nhãn đúng-sai từ chân trị: phép đo quan trọng thứ hai

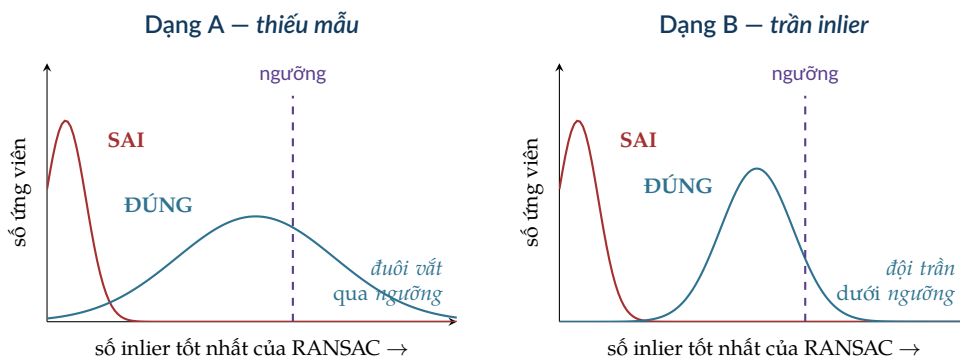
Bảng sống sót nói tăng nào loại nhiều nhất. Nó không nói tăng đó loại *đúng* hay *sai*. Đây là phân biệt quan trọng nhất của cả chương, và nó cần một phép đo thứ hai.

Phép đo ấy là gắn nhãn cho từng ứng viên bằng chân trị. Với mỗi cặp khung hình khoá được đề xuất, tra tư thế thật của cả hai, tính khoảng cách giữa chúng, rồi gọi ứng viên là “đúng” nếu khoảng cách dưới một ngưỡng và “sai” nếu vượt xa ngưỡng đó. Ngưỡng ấy là một lựa chọn thiết kế chứ không phải sự thật: trong nhà, vài mét là mức hợp lý; ngoài trời thì phải rộng hơn nhiều. Khi đã có nhãn, bảng sống sót tách làm hai bảng, và hai bảng đó kể hai câu chuyện khác nhau.

Việc phải làm sau đó là vẽ, cho riêng từng nhóm nhãn, phân bố của số *inlier* tốt nhất mà RANSAC tìm được. Đây là biểu đồ quan trọng nhất của cả chương, vì hình dạng của nó trả lời thẳng câu hỏi “cổng này loại đúng hay loại sai”. Có hai câu hỏi đọc được từ nó, và cả hai đều đọc bằng mắt.

1. **Các ứng viên sai có bao giờ tới gần ngưỡng không?** Nếu phân bố của chúng nằm sát số không và tắt hẳn từ lâu trước ngưỡng, thì mọi ca chết trong nhóm đó là *loại đúng*, và bạn không có lý do gì để đụng vào cổng. Một tỉ lệ sống sót rất thấp khi ấy là bằng chứng cổng đang làm đúng việc, không phải bằng chứng nó hỏng.
2. **Giữa hai phân bố có một khoảng trống không, và ngưỡng nằm ở đâu so với khoảng trống ấy?** Nếu có một khoảng mà chỉ ứng viên *đúng* chết trong đó và không ứng viên sai nào chạm tới, thì đúng chỗ đó — và chỉ chỗ đó — là dư địa mà việc nói ngưỡng lấy lại được. Đếm số ứng viên nằm trong khoảng trống ấy là bạn có ngay trần lợi ích của mọi ý tưởng “nói ngưỡng”, trước khi cài đặt gì.

Hình 29.2 vẽ hai hình dạng mà biểu đồ ấy có thể mang, và chúng dẫn tới hai kết luận trái ngược nhau. Điểm cốt lõi: một tỉ lệ sống sót thấp ở cổng ba *không* tự nó chứng minh điều gì. Cùng một tỉ lệ ấy có thể là dấu hiệu của một bộ kiểm chứng khoẻ mạnh hoặc của một tăng ghép cặp đôi cặp đúng, và chỉ hình dạng phân bố mới phân biệt được.



Hình 29.2. Hai hình dạng mà biểu đồ số inlier tốt nhất có thể mang, vẽ theo *cơ chế* chứ không theo số đo — các trục cố ý không có vạch số, vì con số là thứ bạn phải tự lấy trên hệ của mình. Ở cả hai hình, nhóm ứng viên SAI nằm sát số không và tắt trước ngưỡng, nghĩa là mọi ca chết trong nhóm ấy là loại đúng và cổng đang làm việc của nó. Khác biệt nằm ở nhóm ĐÚNG. Dạng A: phân bố trải rộng và vượt qua ngưỡng, tức inlier vốn có đủ mà chỉ thiếu lượt rút trúng — đây là *thiếu mẫu*, và tăng số vòng lặp hay rút mẫu có ưu tiên sẽ có lãi. Dạng B: phân bố đội trần ngay dưới ngưỡng và không có đuôi, tức số cặp đúng thật sự tồn tại đã ít hơn ngưỡng — đây là *trần inlier*, và không cách rút mẫu nào chạm tới được. Nhìn nhầm dạng B thành dạng A là cách tốn nhiều tháng nhất để không sửa được gì.

E. Phân biệt “thiếu mẫu” với “trần inlier”

Khi RANSAC không hội tụ, có đúng hai nguyên nhân, và chúng đòi hai cách chữa trái ngược nhau.

- **Thiếu mẫu (sampling failure)** — inlier có đủ, nhưng số vòng lặp không đủ để rút trúng một tập mẫu tối thiểu toàn inlier. Cách chữa: tăng số vòng lặp, giảm cỡ tập mẫu tối thiểu, hoặc rút mẫu có ưu tiên.
- **Trần inlier (inlier ceiling)** — số cặp đúng thật sự tồn tại đã ít hơn ngưỡng. Không cách rút mẫu nào cứu được. Cách chữa duy nhất là tạo ra nhiều cặp đúng hơn, tức sửa tăng hai.

Hai nguyên nhân này phân biệt được bằng một biểu đồ duy nhất, chính là Hình 29.2. Nếu số inlier tốt nhất của các ứng viên đúng phân bố cao nhưng thưa thớt qua ngưỡng, đó là thiếu mẫu. Nếu nó đội trần ngay dưới ngưỡng, đó là trần inlier.

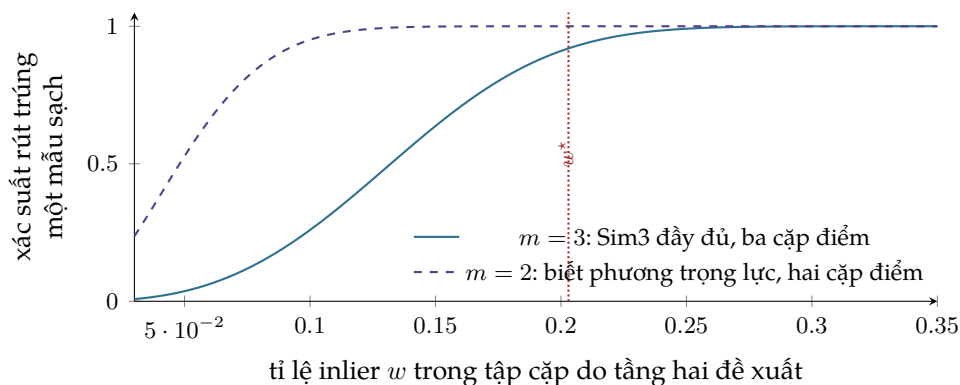
Có một con số thứ hai phải ghi kèm để đọc biểu đồ ấy cho đúng, và nó là mẫu số: *số cặp mà tăng hai đề xuất*. Số inlier tốt nhất chỉ có nghĩa khi so với mẫu số đó, vì tỉ lệ inlier w mới là đại lượng đi vào công thức bên dưới. Mười lăm inlier trên hai trăm cặp và mười lăm inlier trên bốn mươi cặp là hai tình huống hoàn toàn khác nhau. Ghi cả tử lẫn mẫu vào cùng một dòng nhật ký là việc rẻ nhất bạn làm được, và không ghi nó là lý do phổ biến nhất khiến bảng của bạn không trả lời được câu hỏi nào.

Lý thuyết RANSAC làm rõ tại sao phân biệt này quan trọng. Xác suất rút được ít nhất một tập mẫu sạch trong N vòng lặp là

$$p = 1 - (1 - w^m)^N,$$

với w là tỉ lệ inlier và m là cỡ tập mẫu tối thiểu [246]. Với Sim3 tổng quát thì $m = 3$, vì ba cặp điểm ba chiều đủ để giải bài toán định hướng tuyệt đối có tỉ lệ [247], [248]. Đây là một đẳng thức, nên thay số vào là ra ngay, và việc thay số cho thấy quan hệ dốc tới mức nào. Giả sử $N = 300$ vòng lặp: ở $w = 0,20$ ta được $p \approx 0,91$, còn ở $w = 0,10$ chỉ còn $p \approx 0,26$. Hai giá trị w này là giả định để thấy hình dạng quan hệ, không phải số đo của hệ nào; điều đáng nhớ là chia đôi tỉ lệ inlier đủ để biến một cổng gần như luôn mở thành một cổng gần như luôn đóng.

Nhưng nó chỉ là nút thắt nếu nguyên nhân là thiếu mẫu. Hình 29.3 vẽ hai đường cong cho $m = 3$ và $m = 2$. Khoảng cách giữa chúng ở vùng w thấp là rất lớn, nên lý thuyết hứa hẹn một cải thiện mạnh nếu ta giảm được cỡ tập mẫu. Mục 29.5 mổ ba giả định ẩn của lời hứa đó, và nói cách kiểm từng giả định trước khi bỏ công cài đặt.



Hình 29.3. Đồ thị của đúng công thức $1 - (1 - w^m)^N$ với $N = 300$; đây là toán, không phải số đo. Giảm cỡ tập mẫu tối thiểu từ ba xuống hai đẩy cả đường cong sang trái rất mạnh ở vùng tỉ lệ inlier thấp — đó là toàn bộ sức hấp dẫn của ý tưởng dùng trọng lực để rút bậc tự do. Đường chấm đứng là w^* , tỉ lệ inlier tối thiểu để chạm ngưỡng, tính bằng ngưỡng inlier chia cho số cặp mà tăng hai đề xuất; bạn tự tính nó cho hệ mình từ hai cột nhật ký ở Bảng 29.4. Bên trái w^* thì hai đường cong này vô nghĩa, vì ở đó số inlier *thật* đã dưới ngưỡng và không cách rút mẫu nào chạm tới được.

F. Ba con số nên có trước khi đọc bất cứ paper nào

Trước khi cân nhắc thay một tăng, hãy có ba con số cho hệ của chính bạn.

1. **Bảng sống sót từng cổng**, với biến đếm đặt đúng chỗ. Nó cho biết tầng nào hẹp.
2. **Bảng sống sót tách theo nhãn chân trị**. Nó cho biết tầng hẹp đó loại đúng hay loại sai.
3. **Số ứng viên đúng tồn tại**. Nếu một chuỗi không có lần đi lại nào thì việc nó không đóng vòng là kết quả đúng, không phải lỗi.

Con số thứ ba hay bị bỏ sót nhất, và nó rẻ đến mức không có lý do gì để bỏ. Với mỗi chuỗi, quét mọi cặp khung hình khoá bằng *chân trị* rồi đếm xem có bao nhiêu cặp thoả điều kiện “cùng nơi” mà không phải khung hình khoá lân cận về thời gian. Nếu con số đó bằng không thì chuỗi ấy đơn giản là không có lần đi lại nào, và việc hệ không đóng vòng trên nó là kết quả *đúng*, không phải lỗi. Nhiều chuỗi phổ biến trong nhà là đường đi một chiều không quay lại, nên khả năng bạn đang chữa một hệ vốn đang đúng cao hơn nhiều so với cảm giác. Không đo con số này trước là cách chắc chắn nhất để mất nhiều tuần.

↔ **cùng nguyên lý với E/24** — đây chính là thí nghiệm oracle: thay tầng lấy ứng viên bằng chân trị để đo trần đóng góp của nó, trước khi tối ưu nó

29.3 Toàn ảnh học được: từ túi từ tới SALAD và AnyLoc (Learned Global Descriptors)

A. DBoW2 làm được gì, và nó gãy ở đâu

Túi từ nhị phân của DBoW2 là một thiết kế rất kinh tế [245]. Nó dùng lại đúng các mô tả ORB đã được trích cho việc bám vết, nên chi phí trích đặc trưng bằng không. Từ điển được huấn luyện ngoại tuyến một lần, dạng cây phân cấp với khoảng một triệu từ. Truy vấn là tra chỉ mục ngược trên vectơ thừa, nên rẻ.

Về chi phí, điều đáng nói không phải một con số mà là một tính chất: chi phí biên của tăng một trong một hệ kiểu ORB-SLAM3 gần như bằng không, vì mọi thứ nó cần — các mô tả ORB — đã được trích sẵn cho việc bám vết. Trước khi tin điều đó trên hệ của mình, hãy đo hai thứ: độ trễ của tầng phát hiện ở *phân vị* 99, không phải trung vị, và độ dài hàng đợi khung hình khoá. Nếu hàng đợi luôn rỗng thì tầng một chưa phải chỗ đáng động tới, và mọi lập luận về việc thay nó phải dựa trên chất lượng chứ không dựa trên tốc độ.

DBoW2 gãy ở bốn chỗ, và cả bốn đều bắt nguồn từ cùng một điểm: nó thừa hưởng giới hạn của chính mô tả ORB.

- **Đổi ánh sáng mạnh** — ngày với đêm, trong nhà bật tắt đèn. Mô tả nhị phân dựa trên so sánh cường độ điểm ảnh, nên đảo tương phản làm chúng đổi hẳn.
- **Lệch góc nhìn lớn** — đi lại cùng hành lang theo chiều ngược, hoặc lệch ngang vài mét. Đây là chế độ hỏng thường gặp nhất trong nhà.
- **Cấu trúc lặp** — perceptual aliasing. Túi từ bỏ hoàn toàn thông tin bố cục không gian, nên hai hành lang giống nhau cho cùng biểu đồ tần suất.
- **Nền ít vân** — tường trắng, sàn phẳng. Không có đặc trưng thì không có từ.

Hai chế độ đầu là chỗ mà mô tả học được thật sự thắng. Chế độ thứ ba thì không: một mô tả toàn ảnh tốt hơn vẫn không phân biệt được hai nơi thật sự giống nhau. Đó là việc của tầng ba và của ràng buộc thời gian.

↔ **tiền đề F/28** — túi từ kế thừa mọi điểm gãy của mô tả cục bộ mà nó gom lại, nên chương 28 giải thích trước nguyên nhân gốc của ba trong bốn chế độ hỏng ở đây

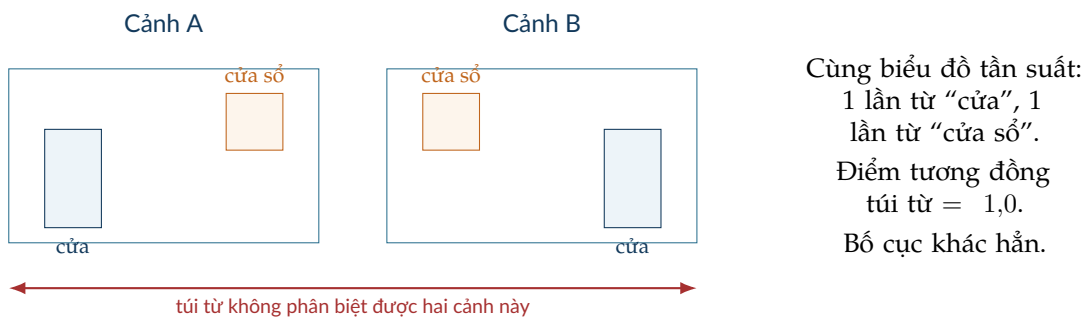
B. Vì sao túi từ mất bố cục, và điều đó có nghĩa gì

Trong bốn chế độ hỏng ở trên, chế độ cấu trúc lặp đáng nhìn kỹ, vì nó không phải lỗi của mô tả ORB mà là lỗi của phép gộp. Túi từ đếm xem mỗi từ thị giác xuất hiện bao nhiêu lần, rồi vứt bỏ hoàn toàn thông tin về vị trí của chúng. Hai bức ảnh chứa cùng các thành phần nhưng sắp xếp khác nhau cho cùng một biểu đồ tần suất.

Hình 29.4 vẽ trường hợp nhỏ nhất. Hai cảnh khác nhau, cùng số cửa và cùng số cửa sổ, chỉ đổi chỗ trái phải. Điểm tương đồng của túi từ giữa chúng bằng đúng một, tức là chúng không phân biệt được.

Điều này giải thích một quan sát thực tế hay gặp: mọi cải tiến ở tầng mô tả cục bộ đều không chứa được perceptual aliasing. Muốn chứa thì phải đưa lại thông tin không gian vào, và có ba cách. Cách rẻ nhất là ràng buộc thời gian ở mục trước. Cách thứ hai là kiểm chứng hình học ở tầng ba, vốn chính là kiểm tra bố cục một cách nghiêm ngặt. Cách thứ ba là xếp hạng lại bằng mảnh đặc trưng có vị trí, tức hướng của Patch-NetVLAD.

Đây cũng là lý do một mô tả toàn ảnh học được không tự nó xoá được perceptual aliasing. Nó gộp tốt hơn, nó bất biến với ánh sáng tốt hơn, nhưng nó vẫn là một vectơ duy nhất cho cả ảnh. Hai nơi thật sự giống nhau vẫn cho hai vectơ gần nhau, và đúng như vậy mới là hành vi đúng của một bộ mô tả.



Hình 29.4. Trường hợp nhỏ nhất cho thấy túi từ mất bố cục không gian. Hai cảnh chứa đúng cùng các thành phần, chỉ đổi vị trí trái phải, nên biểu đồ tần suất từ thị giác của chúng trùng nhau hoàn toàn. Đây là lý do mọi cải tiến ở tầng mô tả cục bộ đều không chứa được perceptual aliasing, và cũng là lý do một mô tả toàn ảnh học được cũng không chứa được.

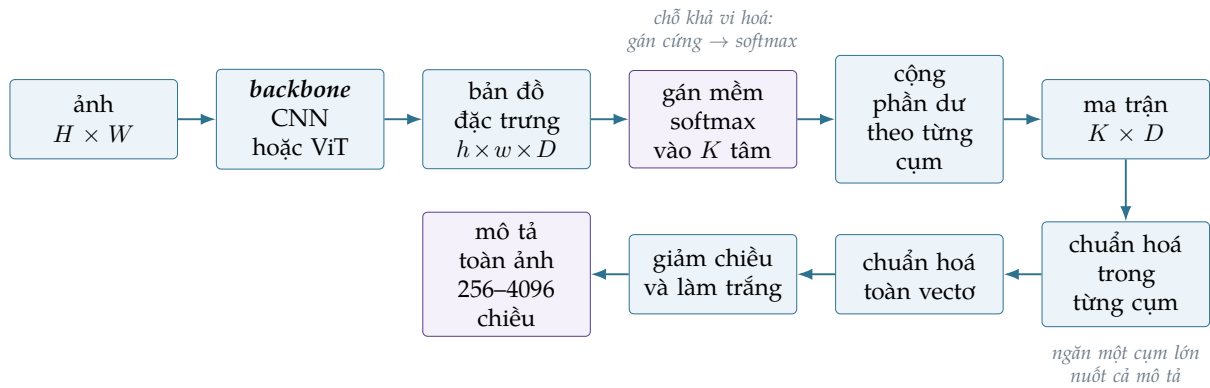
C. NetVLAD: biến phép gộp thành thứ học được

Ý tưởng của VLAD có trước học sâu. Thay vì đếm số đặc trưng rơi vào mỗi cụm, VLAD cộng *phần dư* giữa đặc trưng và tâm cụm. Kết quả là một ma trận $K \times D$, giữ được nhiều thông tin hơn hẳn một biểu đồ tần suất.

Vấn đề là phép gán đặc trưng vào cụm là gán cứng, nên không khả vi, nên không huấn luyện được đầu-cuối. NetVLAD thay gán cứng bằng gán mềm dạng softmax [249]. Chỉ một thay đổi đó làm cả khối gộp trở nên khả vi, và từ đó cả mạng trích đặc trưng lẫn các tâm cụm đều học được cùng lúc.

Điểm thứ hai quan trọng không kém là tín hiệu huấn luyện. NetVLAD học từ ảnh phổ chụp cùng một chỗ ở các thời điểm và góc nhìn khác nhau, với hàm mất mát dạng xếp hạng bộ ba. Nói cách khác, mạng được dạy trực tiếp cái bất biến mà ta cần: cùng nơi thì gần nhau, dù ánh sáng và góc nhìn đổi. Túi từ không có cách nào học điều đó, vì nó không có tham số nào để học.

Hình 29.5 vẽ đường đi của dữ liệu. Chỗ đáng chú ý là hai bước chuẩn hoá cuối: chuẩn hoá trong từng cụm trước, rồi chuẩn hoá toàn vectơ. Bước đầu ngăn một cụm chiếm ưu thế chỉ vì nó có nhiều đặc trưng, tức là ngăn một mảng tường lớn nuốt cả mô tả.



Hình 29.5. Đường đi của dữ liệu trong NetVLAD. Toàn bộ khác biệt so với VLAD cổ điển nằm ở một khối: phép gán đặc trưng vào cụm được làm mềm bằng softmax, nên khả vi, nên cả tâm cụm lẫn mạng trích đặc trưng đều học được. Hai bước chuẩn hoá cuối không phải trang trí; bỏ bước chuẩn hoá trong cụm thì một vùng ảnh đồng nhất lớn sẽ chi phối mô tả.

D. Dòng thời gian: mỗi bước gỡ nút thắt nào

Sau NetVLAD, dòng nghiên cứu này đi theo một mạch rất rõ. Điều đáng chú ý là nút thắt dịch chuyển liên tục, và không phải lúc nào nó cũng nằm ở kiến trúc.

I Mạch 1 – từ túi từ tới đặc trưng nền

- 1 Vấn đề** Túi từ mất hoàn toàn tính bất biến với ánh sáng và góc nhìn lớn, vì nó thừa hưởng mô tả thủ công.

Giải pháp Học phép gộp bằng cách làm nó khả vi, rồi huấn luyện trên ảnh cùng nơi khác thời điểm. Đó là NetVLAD.

Giá phải trả Một lượt xuôi qua mạng cho mỗi khung hình khoá, và một mô tả vài nghìn chiều thay cho vectơ thưa.

Vấn đề mới Mô tả toàn ảnh vẫn không có thông tin bố cục, nên vẫn nhảm ở cấu trúc lặp.
- 2 Vấn đề** Truy hồi toàn ảnh xếp hạng sai khi hai nơi khác nhau có cùng thống kê đặc trưng.

Giải pháp Xếp hạng lại danh sách ngắn bằng chính các mảnh đặc trưng trích từ NetVLAD, có tính tới vị trí không gian. Đó là Patch-NetVLAD [250].

Hệ quả Ranh giới giữa tầng một và tầng ba bắt đầu mờ đi: một phần kiểm chứng bị kéo lên tầng truy hồi.

Giá phải trả Chi phí xếp hạng lại tỉ lệ với số ứng viên, nên ngân sách thời gian thực bị ràng buộc lại.
- 3 Vấn đề** Chạy một mạng cho truy hồi và một mạng khác cho đặc trưng cục bộ là trả hai lần cho cùng một bức ảnh.

Giải pháp Một mạng, một *backbone* chung, ba đầu ra: mô tả toàn ảnh, điểm đặc trưng, mô tả cục bộ. Đó là HF-Net [251].

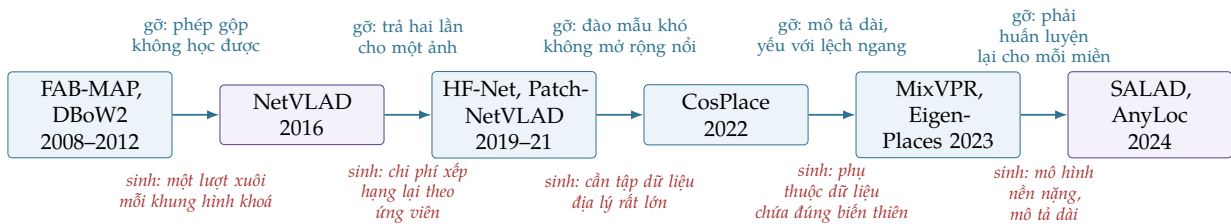
Giả định Ba nhiệm vụ đó dùng chung được biểu diễn ở tầng giữa.

Hệ quả Đây là bài học thiết kế đáng giá nhất của cả mục này với một hệ có ngân sách độ trễ chặt.
- 4 Vấn đề** Huấn luyện bằng xếp hạng bộ ba đòi đào mẫu khó, rất tốn và rất khó ổn định ở quy mô lớn.

Giải pháp Đổi bài toán thành phân lớp: chia thế giới thành các ô theo vị trí và hướng, mỗi ô là một lớp. Đó là CosPlace [252].

Hệ quả	Nút thắt hoá ra nằm ở quy trình dữ liệu và hàm mục tiêu, không nằm ở kiến trúc.
5 Vấn đề	Khối gộp NetVLAD nặng và mô tả đầu ra rất dài.
Giải pháp	Thay gộp theo cụm bằng một chồng khối trộn đặc trưng đơn giản, cho mô tả gọn hơn. Đó là MixVPR [253].
Vấn đề mới	Mô tả gọn hơn nhưng độ bền với lệch góc nhìn ngang chưa được giải quyết riêng.
6 Vấn đề	Lệch góc nhìn ngang, kiểu đi lại cùng con đường ở làn khác, vẫn phá truy hồi.
Giải pháp	Dựng các lớp huấn luyện sao cho mỗi lớp gồm nhiều hướng nhìn khác nhau của cùng một nơi. Đó là EigenPlaces [254].
Giả định	Bất biến góc nhìn học được nếu và chỉ nếu dữ liệu huấn luyện chứa đúng loại biến thiên đó.
7 Vấn đề	Gán mềm của NetVLAD vẫn ép mọi đặc trưng phải thuộc về một cụm nào đó, kể cả đặc trưng vô nghĩa.
Giải pháp	Xem việc gán đặc trưng vào cụm như một bài toán vận tải tối ưu, có thêm một cụm “thùng rác” cho đặc trưng vô ích. Đó là SALAD [255].
Giả định	Đặc trưng đầu vào đã đủ tốt, vì SALAD đặt trên nền DINOv2 [52].
8 Vấn đề	Mọi phương pháp trên đều cần huấn luyện lại khi ra khỏi phân phối: dưới nước, trên không, trong nhà ban đêm.
Giải pháp	Bỏ hẳn việc huấn luyện chuyên biệt. Lấy đặc trưng trung gian của một mô hình nền rồi gộp bằng VLAD cổ điển với từ điển dựng theo miền. Đó là AnyLoc [256].
Giá phải trả	Mô tả dài hơn và một lượt xuôi qua mô hình nền, tức đắt hơn về tính toán.
Hệ quả	Với một robot phải chạy ở miền chưa từng thấy, đây thường là lựa chọn an toàn hơn một mô hình đã tinh chỉnh chặt.

Hình 29.6 tóm mạch trên thành một trục. Điều đáng rút ra không phải thứ tự các tên, mà là việc nút thắt nhảy qua lại giữa ba chỗ: phép gộp, dữ liệu huấn luyện, và đặc trưng đầu vào.



Hình 29.6. Dòng thời gian của mô tả toàn ảnh. Hàng trên ghi nút thắt mà mỗi bước gỡ được; hàng dưới ghi vấn đề mới mà nó tạo ra. Nút thắt nhảy giữa ba chỗ: phép gộp, dữ liệu huấn luyện, và chất lượng đặc trưng đầu vào. Hai ô tô đậm là hai bước đổi hẳn cách nghĩ, không chỉ đổi con số.

E. So sánh có cột “hông khi nào”

Bảng 29.1 đặt các họ cạnh nhau. Cột cuối là cột đáng đọc nhất, vì nó cho biết bạn sẽ mất gì khi đổi.

Bảng 29.1. Các họ mô tả toàn ảnh, so theo cơ chế, giả định và chế độ hỏng. Cột chi phí cố ý ghi định tính: nó nói *loại* chi phí bạn phải trả cho mỗi khung hình khoá, không nói bao nhiêu mili-giây. Con số mili-giây chỉ có nghĩa khi đo trên chính phần cứng đích với hệ đang chạy đủ tải, và đó là việc của bạn chứ không phải của một bảng.

Họ	Cơ chế	Mạnh khi nào	Hỏng khi nào	Chi phí
DBoW2	Biểu đồ tần suất từ thị giác nhị phân, chỉ mục ngược, tf-idf	Cùng điều kiện chụp, cùng hướng đi, bản đồ vừa phải	Đổi ánh sáng mạnh, lệch góc nhìn lớn, cấu trúc lặp, nền ít vân	chi phí biên gần như bằng 0: dừng lại mô tả ORB
NetVLAD	Gộp phần dư quanh K tâm, gán mềm khả vi	Đổi thời điểm và ánh sáng, ngoài trời đô thị	Miền khác hẳn dữ liệu huấn luyện; vẫn nhằm ở cấu trúc lặp	một lượt xuôi mạng cỡ vừa mỗi khung hình khoá
Patch-NetVLAD	Xếp hạng lại danh sách ngắn bằng mảnh đặc trưng có vị trí	Cần precision cao và danh sách ngắn nhỏ	Chi phí tăng tuyến tính theo số ứng viên; không hợp ngân sách chặt	cao, theo ứng viên
HF-Net	Một <i>backbone</i> , ba đầu: toàn ảnh, điểm, mô tả cục bộ	Khi bạn cũng cần đặc trưng cục bộ học được	Ba nhiệm vụ tranh nhau sức chứa nếu <i>backbone</i> quá nhỏ	một lượt chung
CosPlace, MixVPR, EigenPlaces	Đổi hàm mục tiêu và cách dựng lớp huấn luyện	Bản đồ rất lớn; lệch góc nhìn ngang	Vẫn là mô hình huấn luyện theo miền; ra ngoài phân phối thì tụt	cùng bậc với NetVLAD
SALAD	Gán đặc trưng vào cụm bằng vận tải tối ưu, có thùng rác	Nền DINOv2 sẵn; cần mô tả gọn và mạnh	Phụ thuộc chất lượng mô hình nền bên dưới	một lượt xuôi mô hình nền: nặng hơn hẳn
AnyLoc	Đặc trưng mô hình nền, không huấn luyện lại, gộp VLAD	Miền lạ: dưới nước, trên không, trong nhà ban đêm	Khi có nhiều dữ liệu đúng miền, mô hình tinh chỉnh vẫn hơn	như SALAD, cộng mô tả dài hơn

F. Tính tay ba khoản chi phí, và khoản duy nhất đáng kể

Lập luận “túi từ nhẹ hơn” được nhắc rất nhiều nhưng hiếm khi được tính ra số. Có ba khoản chi phí, và chỉ một khoản thật sự quan trọng.

Bộ nhớ mô tả. Đây là một phép nhân chia thuận tuý, và nó cho một kết luận đi ngược trực giác. Giả sử mỗi khung hình khoá giữ khoảng một nghìn đặc trưng — cấu hình mặc định quen thuộc. Vectơ túi từ thừa của nó có tối đa một nghìn cặp gồm mã từ và trọng số; ở tám byte một cặp, đó là cỡ tám kilobyte. Một mô tả toàn ảnh 4096 chiều dạng số thực 32 bit là mười sáu kilobyte, tức lớn hơn. Nhưng nó là vectơ *dày* có độ dài cố định, nên giảm chiều xuống 256 rồi lượng tử hoá về 8 bit là hợp lệ, và khi đó nó chỉ còn vài trăm byte. Vectơ thừa của túi từ không chịu được phép nén tương tự, vì mỗi mục của nó là một cặp chỉ số–trọng số cần thiết cho việc tra chỉ mục ngược. Kết luận về *hình dạng* quan hệ: khi bản đồ lớn dần, mô tả học được là thứ nén được, còn túi từ thì không — và tỉ số nghiêng về phía học được, chứ không phải ngược lại như người ta hay nghĩ.

Chi phí truy vấn. Chỉ mục ngược chỉ duyệt các khung hình khoá có chung ít nhất một từ với truy vấn. Nó rẻ khi từ vựng phân biệt tốt và dặt dằn khi bản đồ lớn. Ở phía kia, tìm láng giềng gần nhất là một phép nhân ma trận với vectơ, tức số phép tính tỉ lệ với số khung hình khoá nhân số chiều — một đại lượng nhỏ so với mọi thứ khác trong hệ, và có thư viện vectơ hoá sẵn. Với cả hai bên, nếu bạn nghi ngờ thì hãy đo; nhưng đây hiếm khi là chỗ nút thắt nằm.

Chi phí trích mô tả. Đây là khoản duy nhất đáng kể, và chênh lệch là tuyệt đối chứ không phải theo tỉ lệ. Túi từ trả bằng *không*, vì nó dùng lại đúng các mô tả ORB đã được trích cho việc bám vết. Một mô hình học được trả trọn một lượt xuôi qua mạng cho mỗi khung hình khoá, và lượt xuôi đó là một khoản mới hoàn toàn trong ngân sách. Bao nhiêu mili-giây thì chỉ có phép đo trên phần cứng đích mới nói được, và phải đo khi hệ đang chạy đủ tải chứ không đo mô-đun đứng một mình.

Vậy câu “DBoW2 nhẹ hơn” là đúng, nhưng đúng vì một lý do khác với lý do người ta hay nêu. Nó nhẹ vì chi phí biên của nó bằng không, không phải vì nó tốn ít bộ nhớ. Phân biệt này quan trọng khi bạn cân nhắc: nếu hệ của bạn đã chạy một mạng khác trên khung hình khoá, chi phí biên của việc thêm một đầu ra toàn ảnh có thể gần bằng không, và toàn bộ lập luận đảo chiều. Đó chính là ý tưởng của HF-Net.

G. Thử một bộ truy hồi mà không đụng vào mã của hệ

Có một cách rẻ để trả lời câu “mô tả này có hơn không” trước khi viết một dòng mã tích hợp. Ý tưởng là tách hoàn toàn việc đánh giá tầng một ra khỏi hệ SLAM, và làm nó ngoại tuyến trên chính dữ liệu của bạn.

Quy trình gồm bốn bước.

1. Chạy hệ như bình thường, xuất ra ảnh của mọi khung hình khoá cùng tư thế ước lượng của chúng.
2. Dựng nhãn chân trị cho từng cặp khung hình khoá: đúng nếu khoảng cách thật dưới ngưỡng và hai hướng nhìn không ngược nhau.
3. Với mỗi bộ mô tả cần so, tính vectơ cho mọi khung hình khoá rồi lấy danh sách láng giềng gần nhất.
4. Vẽ recall theo số ứng viên cho từng bộ mô tả, trên cùng một tập khung hình khoá.

Bước bốn cho một đường cong so sánh trực tiếp, trên đúng dữ liệu của bạn, mà không cần tích hợp gì. Nếu hai đường cong gần trùng nhau, câu trả lời đã có và bạn tiết kiệm được vài tuần.

Hai lưu ý làm phép so này trung thực. Thứ nhất, phải loại các cặp khung hình khoá gần nhau về thời gian, nếu không mọi bộ mô tả đều đạt recall gần một và đường cong vô nghĩa. Thứ hai, tập khung hình khoá của bạn được sinh ra bởi chính chính sách chọn khung hình khoá hiện tại, nên nó không phải mẫu ngẫu nhiên của cảnh. Kết luận rút ra chỉ có hiệu lực cho chính chính sách đó.

H. Có đáng thay DBoW2 trên hệ của bạn không

Đây là chỗ phải trả lời thẳng. Câu trả lời phụ thuộc hoàn toàn vào ba con số ở mục trước, chứ không phụ thuộc bảng xếp hạng nào.

Nếu tầng một của bạn đang để lọt phần lớn ứng viên đúng xuống tầng sau, thì thay nó không mua được gì. Đây là điều bảng sống sót trả lời trong một buổi chiều: nếu tỉ lệ sống sót ở cổng một và cổng hai đều cao, tầng một không phải nút thắt, và một bộ truy hồi tốt hơn chỉ đưa thêm ứng viên vào một cái phễu vốn đang nghẽn ở chỗ khác. Trên một hệ chạy chuỗi ngắn cùng điều kiện ánh sáng, đây là kết cục thường gặp — nhưng bạn phải tự đo chứ đừng tin câu này.

Có bốn tình huống mà mô tả học được thật sự thắng, và nên kiểm xem hệ của bạn có rơi vào tình huống nào không.

- **Đi lại sau nhiều giờ hoặc nhiều ngày** — ánh sáng đối hản. Đây là điểm mạnh rõ nhất và cũng là thứ EuRoC không có.
- **Đi lại theo chiều ngược hoặc lệch ngang** — mô tả nhị phân gây sớm hơn nhiều so với mô tả học được.

- **Bản đồ rất lớn hoặc nhiều phiên** — khi số khung hình khoá lên hàng trăm nghìn, phân biệt của tf-idf bão hoà.
- **Định vị lại nguội trong bản đồ đã dựng sẵn** — không có tiên nghiệm chuyển động nào để dựa vào.

Về chi phí, có một tình tiết giảm nhẹ đáng biết: mô tả toàn ảnh chỉ cần tính trên *khung hình khoá*, không phải trên mọi khung hình, nên tần suất gọi thấp hơn tần số ảnh cả một bậc. Ngân sách vì thế thường khả thi, nhưng con số cụ thể phải đo trên phần cứng đích. Cái phải bỏ đi thì rõ ràng hơn và không cần đo: bạn mất khả năng dùng lại mô tả ORB miễn phí, mất tính suy luận được của từ điển, và thêm một phụ thuộc thời gian chạy vào một trình biên dịch suy luận.

CẦN NHỚ

Chỉ thay tầng lấy ứng viên khi bạn đã đo được rằng ứng viên đúng đang chết ở *tầng đó*. Trên một hệ stereo-inertial chạy chuỗi ngắn cùng điều kiện ánh sáng, hãy vào cuộc với giả thuyết ngược lại — rằng tầng đó không phải thủ phạm — rồi bắt bảng sống sót bác bỏ giả thuyết ấy. [T2]

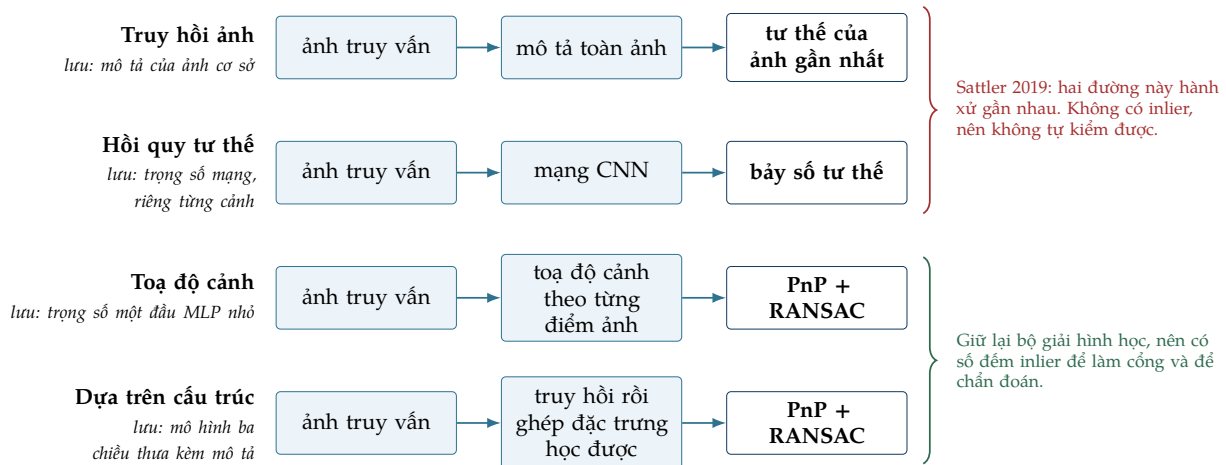
29.4 Định vị lại: hồi quy tư thế, hồi quy tọa độ cảnh và hloc (Relocalization)

A. Bốn họ, phân biệt bằng “cái gì được lưu”

Định vị lại là bài toán anh em của đóng vòng: cho một ảnh và một bản đồ, tìm tư thế camera. Khác biệt là đóng vòng có tiên nghiệm chuyển động, còn định vị lại nguội thì không. Có bốn họ giải bài toán này, và cách phân biệt gọn nhất là hỏi *cái gì được lưu trong bản đồ*.

- **Truy hồi ảnh** — lưu mô tả toàn ảnh của ảnh cơ sở. Trả về tư thế của ảnh gần nhất, hoặc nội suy vài ảnh gần nhất. Không cần hình học.
- **Hồi quy tư thế tuyệt đối** — lưu trọng số một mạng. Mạng nhận ảnh và trả thẳng ra bảy số. Đó là PoseNet [257].
- **Hồi quy tọa độ cảnh** — lưu trọng số một mạng, nhưng mạng trả về tọa độ ba chiều của *từng điểm ảnh* trong hệ tọa độ cảnh. Tư thế được giải sau đó bằng PnP và RANSAC. Đó là dòng DSAC và ACE.
- **Dựa trên cấu trúc** — lưu một mô hình ba chiều thưa kèm mô tả. Ghép hai chiều với ba chiều rồi giải PnP. Đó là hloc và cũng chính là cách ORB-SLAM3 định vị lại.

Hình 29.7 vẽ bốn đường đi này cạnh nhau. Trục ngang là “bao nhiêu hình học còn được giữ ở cuối đường ống”.



Hình 29.7. Bốn họ định vị lại, phân biệt bằng thứ được lưu trong bản đồ và bằng lượng hình học còn lại ở cuối đường ống. Hai họ dưới giữ một bộ giải hình học có kiểm chứng nên có inlier để chẩn đoán; hai họ trên không có. Đó là luận điểm của Sattler và cộng sự ở dạng sơ đồ: hồi quy tư thế hành xử gần với truy hồi ảnh hơn là với định vị dựa trên cấu trúc.

B. PoseNet và phản bác của Sattler 2019

PoseNet đưa ra một ý tưởng rất gọn: huấn luyện một mạng tích chập hồi quy thẳng từ ảnh sang tư thế [257]. Bản đồ trở thành trọng số mạng, kích thước cố định, truy vấn có thời gian cố định. Nghe rất hợp với robot.

Phản bác đến vào năm 2019 và nó rất sắc [258]. Nhóm tác giả dựng một đường cơ sở chỉ dùng truy hồi ảnh cộng nội suy tư thế trên tập huấn luyện. Đường cơ sở đơn giản đó ngang bằng hoặc tốt hơn phần lớn phương pháp hồi quy tư thế. Họ còn chỉ ra rằng sai số của hồi quy tư thế tăng vọt khi tư thế truy vấn ra ngoài phân phối tư thế huấn luyện.

Cơ chế đằng sau rất dễ hiểu khi đã nói ra. Mạng không học hình học chiều; nó học một ánh xạ từ ảnh vào đa tạp các tư thế đã thấy khi huấn luyện. Vì vậy nó nội suy tốt và ngoại suy rất tệ. Về mặt hành vi, nó là một bộ truy hồi có nội suy, chứ không phải một bộ định vị.

Với kỹ sư SLAM, hệ quả thực dụng còn quan trọng hơn kết quả số. Hồi quy tư thế không trả về inlier. Không có inlier thì không có cách nào biết một tư thế trả về là đáng tin hay rác. Trong một hệ mà một ràng buộc sai làm hỏng cả bản đồ, mất khả năng tự kiểm ấy là cái giá quá đắt.

↔ cùng nguyên lý với B/08 — đây đúng là dịch chuyển phân phối ở dạng hình học: tư thế truy vấn ngoài bao lồi của tư thế huấn luyện là một mẫu ngoài phân phối

C. Định vị lại nguội trong hệ của bạn, và ba chỗ chen được mô-đun học được

Mọi so sánh phải so với đường mà hệ của bạn đang chạy, nên cần nhắc lại đường đó trước. ORB-SLAM3 khi mất bám vết sẽ truy vấn túi từ để lấy ứng viên, ghép đặc trưng theo cây từ vừng, chạy PnP trong RANSAC để có tư thế thô, rồi tìm thêm khớp bằng cách chiếu bản đồ và tối ưu lại [40]. Đó vẫn là bốn tầng cũ, chỉ khác ở chỗ bài toán là sáu bậc chứ không phải bảy.

Có ba chỗ chen được một mô-đun học được vào đường đó, và rủi ro của chúng rất khác nhau.

- **Chỉ thay tầng một** — thêm một mô tả toàn ảnh học được, chạy song song với túi từ, rồi hợp nhất hai danh sách ứng viên. Rủi ro thấp nhất, vì tầng ba vẫn kiểm chứng mọi thứ. Nếu mô-đun mới đề xuất rác, rác bị loại.
- **Thay tầng hai cho riêng cặp ứng viên** — chạy bộ ghép cặp học được thay cho ghép theo cây từ vừng. Rủi ro trung bình, vì nó đổi phân bố inlier nên buộc phải đặt lại ngưỡng ở tầng ba.

- **Thay cả tầng ba** — dùng một mô-đun trả thẳng tư thế. Rủi ro cao nhất, vì bạn mất số đếm inlier, tức mất luôn cái công.

Thứ tự rủi ro này cũng là thứ tự nên thử. Nó cho phép bạn dừng ngay khi đo được rằng bước tiếp theo không mua thêm gì.

D. Từ DSAC tới ACE: giữ lại bộ giải hình học

Hồi quy toạ độ cảnh chọn một chỗ đứng khác. Mạng không dự đoán tư thế; nó dự đoán, cho mỗi điểm ảnh, toạ độ ba chiều tương ứng trong hệ toạ độ cảnh. Tư thế được giải sau bằng PnP trong vòng RANSAC. Bộ giải hình học được giữ nguyên.

DSAC giải quyết trở ngại kỹ thuật của cách làm này: RANSAC không khả vi, nên không lan được *gradient* về mạng [259]. Giải pháp là thay bước chọn giả thuyết tốt nhất bằng một phép lấy mẫu theo xác suất, rồi lấy kỳ vọng. Nhờ vậy toàn bộ đường ống huấn luyện được đầu-cuối.

DSAC++ bỏ được yêu cầu phải có mô hình ba chiều khi huấn luyện, bằng cách khởi tạo qua sai số chiều lại [260]. DSAC* hợp nhất các biến thể ảnh màu và ảnh có độ sâu, đồng thời rút ngắn huấn luyện [261].

ACE là bước có ý nghĩa thực dụng nhất của dòng này [262]. Nó tách mạng thành hai phần: một *backbone* trích đặc trưng, huấn luyện sẵn và không phụ thuộc cảnh, cộng một đầu MLP nhỏ riêng cho từng cảnh. Chỉ đầu MLP được huấn luyện lại, trên các *batch* rất lớn gồm đặc trưng lấy ngẫu nhiên khắp tập ảnh. Kết quả là bản đồ của một cảnh thu về đúng trọng số của một đầu MLP nhỏ, và thời gian dựng nó ngắn hơn hẳn một quy trình dựng lại cấu trúc từ chuyển động.

ACE Zero đi thêm một bước: học toạ độ cảnh từ tập ảnh *không có tư thế*, tức là thay luôn vai trò của một quy trình dựng lại cấu trúc từ chuyển động [263]. Dòng này còn tiếp tục với các biến thể mang tên ACE-G và ACE-SLAM, hướng tới tổng quát hoá sang cảnh mới và tới việc nhúng thẳng vào vòng SLAM. Tôi chưa đối chiếu chi tiết hai công trình đó, nên chỉ nêu tên và ý tưởng, không trích dẫn và không mô tả kết quả số của chúng.

E. hloc: chính cái phễu của bạn, thay từng tầng bằng mô-đun học được

hloc là quy trình định vị đang được dùng như chuẩn tham chiếu [251]. Nó gồm bốn bước: truy hồi bằng mô tả toàn ảnh, trích đặc trưng cục bộ học được, ghép cặp học được, rồi PnP trong RANSAC trên một mô hình ba chiều dựng bằng COLMAP [33].

Điều cần thấy là hloc *chính là* cái phễu bốn tầng ở Hình 29.1, với ba tầng đầu được thay bằng mô-đun học được và tầng cuối giữ nguyên bộ giải hình học. Đây là kiến trúc an toàn, và nó là bài học thiết kế lớn nhất của cả chương: học phần tìm tương ứng, giữ nguyên phần ước lượng.

Lý do rất cụ thể. Bộ ước lượng hình học cho bạn ba thứ mà một mạng hồi quy không cho: một số đếm inlier để làm công, một hiệp phương sai để đặt trọng số trong đồ thị nhân tử, và khả năng chẩn đoán khi hỏng. Bỏ cả ba thứ đó để đổi lấy vài phần trăm recall là một cuộc đổi chác tồi với hệ đang chạy thật.

Về chi phí, hloc không được thiết kế cho thời gian thực và cũng không nên bị ép vào đó. Điều cần thấy là *cấu trúc* của chi phí chứ không phải một con số: một truy vấn đầy đủ gồm một lượt trích mô tả toàn ảnh, một lượt trích đặc trưng cục bộ, rồi *một lượt ghép cặp cho mỗi ảnh trong danh sách ngắn*. Khoản cuối là khoản nhân lên: danh sách ngắn mười ảnh nghĩa là mười lần chạy bộ ghép cặp cho một truy vấn. Vì thế hai núm vặn duy nhất có ý nghĩa là độ dài danh sách ngắn và tần suất gọi. Cách dùng thực tế trong một hệ SLAM là cắt danh sách ngắn xuống hai hoặc ba, và chỉ chạy khi đã mất tám vết hoặc khi có một ứng viên vòng, chứ không chạy mỗi khung hình.

F. Gộp bản đồ: cùng chuỗi cổng, người tiêu thụ khác

Có một chi tiết kỹ thuật quyết định việc mô-đun định vị lại nào cắm được vào hệ của bạn. ORB-SLAM3 dùng đúng một chuỗi cổng cho hai việc khác nhau: đóng vòng trong cùng một bản đồ, và gộp hai bản đồ rời [40]. Khác biệt nằm ở đầu ra chứ không ở đầu vào.

Khi gộp hai bản đồ, hai bản đồ có hai gốc tọa độ độc lập và, với hệ đơn ảnh, hai tỉ lệ độc lập. Vì vậy phép biến đổi cần tìm bắt buộc là Sim3 bảy bậc, không phải một tư thế tuyệt đối. Với hệ stereo hoặc stereo-inertial, tỉ lệ trong mỗi bản đồ đã quan sát được nên hệ số tỉ lệ gần một, nhưng nó vẫn phải được ước lượng chứ không được giả định.

Hệ quả cho các phương pháp học được rất thẳng. Một mô-đun chỉ trả về tư thế tuyệt đối trong một hệ tọa độ cố định thì không dùng được cho việc gộp bản đồ, vì nó không nói gì về quan hệ giữa hai bản đồ. Đây là một lý do kỹ thuật, không phải lý do độ chính xác, và nó thường bị bỏ qua khi người ta so bảng số của các phương pháp định vị lại.

G. Cổng đánh giá cho hệ của bạn

Bảng 29.2 tóm điều kiện dùng được. Kết luận ngắn: hồi quy tọa độ cảnh có chỗ đứng thật, nhưng không phải trong vòng SLAM trực tuyến.

Bảng 29.2. Bốn họ định vị lại, đọc từ góc nhìn một hệ SLAM đang chạy. Cột cuối nói thẳng khi nào không nên dùng.

Họ	Bản đồ là gì	Mạnh khi nào	Hỏng khi nào
Truy hồi ảnh	Mô tả toàn ảnh của ảnh cơ sở	Cần ứng viên nhanh; chấp nhận sai số tư thế lớn	Không cho tư thế chính xác, không có inlier để kiểm chứng
Hồi quy tư thế	Trọng số mạng, riêng từng cảnh	Cảnh nhỏ, tư thế truy vấn nằm trong phân phối huấn luyện	Ngoại suy ngoài quỹ đạo huấn luyện; không có cách tự kiểm
Tọa độ cảnh	Trọng số một đầu MLP nhỏ	Toà nhà đã biết, dựng bản đồ một lần, cần bản đồ rất gọn	Bản đồ đang mở rộng trực tuyến; cần Sim3 để gộp hai bản đồ
Dựa trên cấu trúc	Mô hình ba chiều thừa kèm mô tả	Cần tư thế chính xác kèm inlier; cần gộp bản đồ	Bản đồ rất lớn thì tốn bộ nhớ; ghép cặp học được thì tốn tính toán

Ba lý do khiến hồi quy tọa độ cảnh chưa vào được vòng SLAM trực tuyến. Thứ nhất, nó cần huấn luyện lại cho mỗi cảnh, trong khi bản đồ SLAM lớn dần theo thời gian. Thứ hai, nó trả về tư thế trong một hệ tọa độ cố định, còn đóng vòng cần một phép Sim3 giữa hai đoạn bản đồ. Thứ ba, nó giả định cảnh tĩnh và đã được quan sát đủ khi huấn luyện.

Ngược lại, có một tình huống triển khai mà nó rất hợp: một toà nhà cố định, quét một lần, rồi robot chạy trong đó nhiều tháng. Khi đó việc huấn luyện một đầu MLP cho mỗi khu là chi phí trả một lần, và một bản đồ gọn hơn hẳn mô hình ba chiều thừa là lợi thế thật.

29.5 Kiểm chứng: hình học so với học được (Verification)

A. Vì sao RANSAC Sim3 là chỗ hẹp nhất

Ở tầng ba, hệ phải tìm phép biến đổi bảy bậc tự do nối hai đoạn bản đồ. Bảy bậc vì tỉ lệ của bản đồ đơn ảnh trôi theo thời gian, nên hai đầu vòng nói chung khác tỉ lệ [264]. Nghiệm đóng cho ba cặp điểm đã có từ lâu [247], [248], nên tập mẫu tối thiểu là ba.

Ba yếu tố cùng đẩy tăng này vào thế khó.

- **Số cặp đầu vào ít** — tăng hai chỉ đề xuất vài chục cặp, vì việc ghép bị giới hạn trong cùng nút từ vừng. Đây là mẫu số của w , và bạn phải ghi nó lại chứ không được đoán.
- **Tỉ lệ inlier thấp** — lệch góc nhìn lớn làm phần lớn cặp sai. Ngưỡng inlier chia cho mẫu số ấy cho ra w^* , tỉ lệ inlier tối thiểu bắt buộc phải có; với vài chục cặp và ngưỡng vài chục phần trăm của chúng, w^* rơi vào vùng mà đường cong ở Hình 29.3 đang rất dốc.
- **Bậc ba trong công thức** — xác suất một mẫu sạch là w^3 , nên tụt rất nhanh khi w giảm.

Nhìn ba yếu tố này, hướng chữa hiển nhiên là giảm m . Với camera stereo, tỉ lệ đã quan sát được, nên về lý thuyết bài toán rút xuống sáu bậc. Nếu thêm phương trọng lực từ IMU, hai bậc xoay nữa bị ghim, chỉ còn góc phương vị và tịnh tiến, và tập mẫu tối thiểu rút xuống hai. Hình 29.3 cho thấy lời hứa: ở vùng w thấp, khoảng cách giữa hai đường cong rất lớn.

B. Ví dụ tính tay: vì sao ba cặp điểm là số nhỏ nhất

Lấy ba cặp điểm cụ thể. Trong bản đồ thứ nhất, ba điểm là $p_1 = (0, 0, 0)$, $p_2 = (1, 0, 0)$, $p_3 = (0, 1, 0)$. Trong bản đồ thứ hai, chúng lần lượt là $q_1 = (1, 1, 0)$, $q_2 = (1, 3, 0)$, $q_3 = (-1, 1, 0)$.

Bước một, tìm tỉ lệ. Khoảng cách $|p_2 - p_1| = 1$ còn $|q_2 - q_1| = 2$, nên $s = 2$. Kiểm chéo bằng cặp còn lại: $|p_3 - p_1| = 1$ và $|q_3 - q_1| = 2$, cũng cho $s = 2$.

Bước hai, tìm phép xoay. Sau khi chia cho tỉ lệ, vectơ $(1, 0, 0)$ đi thành $(0, 1, 0)$, và vectơ $(0, 1, 0)$ đi thành $(-1, 0, 0)$. Đó là phép xoay 90 độ quanh trục z .

Bước ba, tìm tịnh tiến. Tâm của hai bộ điểm là $\bar{p} = (\frac{1}{3}, \frac{1}{3}, 0)$ và $\bar{q} = (\frac{1}{3}, \frac{5}{3}, 0)$. Khi đó $t = \bar{q} - sR\bar{p} = (1, 1, 0)$. Kiểm lại với điểm thứ hai: $sRp_2 + t = 2(0, 1, 0) + (1, 1, 0) = (1, 3, 0) = q_2$. Đúng.

Bây giờ tới phần đáng giá. Vì sao tập mẫu tối thiểu là ba chứ không phải hai? Mỗi cặp điểm cho ba ràng buộc vô hướng. Sim3 có bảy bậc tự do. Hai cặp cho sáu ràng buộc, thiếu một. Ba cặp cho chín, dư hai. Vì vậy ba là số nhỏ nhất, và không thể nhỏ hơn nếu không có thông tin ngoài.

Câu cuối chính là chỗ ý tưởng dùng trọng lực xuất hiện. Phương trọng lực từ IMU ghim hai bậc xoay, và camera stereo ghim bậc tỉ lệ. Còn lại bốn bậc, cần sáu ràng buộc, nên hai cặp điểm là đủ. Về mặt tổ hợp, đó là một cải thiện rất lớn: xác suất rút trúng đi từ w^3 lên w^2 .

Bài học tổng quát nằm ở đây, và nó vượt ra ngoài đóng vòng. Mỗi thông tin ngoài mà bạn đưa vào một bộ ước lượng đều rút bậc tự do, và mỗi bậc rút được đều làm xác suất rút mẫu sạch tăng theo hàm mũ. Đó là lý do lập luận này nghe rất thuyết phục. Mục sau kể chuyện gì xảy ra khi đem nó đi đo.

C. Ba giả định ẩn của lời hứa đó, và cách đem nó đi đo

Lập luận ở mục trước rất chặt về mặt toán, nên nó dễ khiến người ta bỏ vài tuần cài đặt trước khi kiểm bất cứ điều gì. Đừng làm thế. Công thức $1 - (1 - w^m)^N$ đứng trên ba giả định ẩn, và mỗi giả định vỡ theo một kiểu riêng. Ba giả định ấy đều kiểm được *trước* khi viết dòng mã nào, bằng đúng cái nhậ ký ở Bảng 29.4.

I. Mạch 2 — ba giả định ẩn của việc rút tập mẫu tối thiểu

1 Vấn đề

Giả định

Hệ quả

Giả định thứ nhất: inlier có đủ, chỉ thiếu lượt rút trúng.

Toàn bộ công thức nói về xác suất *rút*. Nó im lặng hoàn toàn về việc có tồn tại đủ cặp đúng hay không.

Nếu biểu đồ ở Hình 29.2 mang Dạng B, tức đội trần dưới ngưỡng, thì giả định này sai và hạ m không mua được gì. Phép kiểm: so số inlier tốt nhất của các ứng

viên đúng với w^* nhân số cặp đề xuất. Nếu phần lớn nằm dưới, dừng lại ở đây — nút thất nằm ở tầng hai.

2 Vấn đề

Giả định

Giả định thứ hai: hai bậc xoay bị ghim bằng trọng lực là *ghim đúng*.

Phương trọng lực ước lượng được từ IMU có sai số, và sai số nghiêng còn dư tại điểm gặp lại không nhất thiết nhỏ.

Hệ quả

Một giả thuyết chỉ có góc phương vị sẽ ép phép biến đổi về thẳng đứng. Nếu phần nghiêng còn dư đáng kể, ràng buộc sinh ra sai một cách có hệ thống, và tầng bốn sẽ trung thành bơm cái sai đó ra khắp bản đồ. Phép kiểm: với mỗi vòng đóng được bằng nghiệm bảy bậc, ghi lại phần nghiêng của phép Sim3 tìm được; nếu phân bố của nó không tập trung quanh không, việc ghim hai bậc là sai mô hình.

3 Vấn đề

Giả định

Giả định thứ ba: thêm vòng thì bản đồ chỉ tốt lên.

Mỗi vòng được chấp nhận kéo theo một lần hiệu chỉnh bản đồ, và mỗi lần hiệu chỉnh là một cửa sổ mà luồng dựng bản đồ cục bộ bị chặn.

Hệ quả

Nới một cổng để nhận thêm vòng vì thế đổi cả hai đầu: recall tăng, nhưng số cửa sổ rủi ro cũng tăng. Phép kiểm: đếm số lần mất bám vết và số lần hệ sập, không chỉ đếm số vòng.

Cách đo cả ba giả định trong một thí nghiệm duy nhất thì rẻ, và nó không đòi bạn cài đặt bộ giải hai điểm. Chạy hệ như bình thường với bộ giải ba điểm, nhưng *tăng mạnh số vòng lặp RANSAC* — chẳng hạn nhân lên mười lần — rồi so số vòng đóng được. Nếu nguyên nhân thật là thiếu mẫu thì tăng N đã phải mua được một phần lợi ích, vì N và m đi vào cùng một công thức; nếu tăng N không đổi gì thì hạ m cũng sẽ không đổi gì, và bạn vừa tiết kiệm được vài tuần bằng một dòng cấu hình. Đây là thí nghiệm phải chạy *trước*, và nó đúng tinh thần của thí nghiệm oracle ở E/24: đo trần đóng góp của một hướng trước khi đầu tư vào nó.

Cần ghi kèm điều kiện hiệu lực cho toàn bộ lập luận này. Nó được viết cho một hệ stereo-inertial chạy chuỗi ngắn trong nhà, nơi tỉ lệ trôi tỉ lệ nhỏ và tỉ lệ inlier không quá thấp. Trên hệ đơn ảnh ngoài trời, tỉ lệ inlier thấp hơn nhiều, giả định thứ nhất có nhiều khả năng đúng hơn, và cân nhắc có thể đảo chiều.

D. Cổng thứ năm: cái cổng đứng ngay sau RANSAC, và vì sao nó bị bỏ quên

Cổng RANSAC chiếm hết sự chú ý vì nó là cổng lọc mạnh nhất. Nhưng cổng đứng ngay sau nó — tối ưu Sim3 rồi đếm lại inlier — thường cũng loại một phần đáng kể những gì đã vượt được RANSAC, và gần như không ai đo nó. Đây là chỗ dễ có bất ngờ nhất trong cả chuỗi, đúng vì không ai nhìn.

Có ba nguyên nhân khả dĩ, và chúng đòi ba cách xử lý khác nhau.

- **Nghiệm thô quá thô** — Sim3 dựng từ ba cặp điểm nằm sát ngưỡng chỉ chính xác cục bộ. Khi dùng nó để chiếu lại và tìm thêm khớp với bán kính hẹp, phép tìm hụt.
- **Tối ưu rơi vào cực tiểu gần đó** — với ít inlier, hàm mục tiêu nông và có nhiều cực tiểu địa phương gần nhau.
- **Giả thuyết vốn dĩ sai** — một ứng viên sai tình cờ vượt được ngưỡng inlier vẫn có thể bị loại đúng ở đây. Trong trường hợp này, cổng năm đang làm hàng phòng thủ thứ hai.

Tỉ lệ giữa ba nguyên nhân này là một câu hỏi thực nghiệm, và cách trả lời nó giống hệt cách đã dùng cho cổng ba: gắn nhãn chân trị cho các ứng viên chết ở cổng năm rồi tách bảng làm hai. Nếu phần lớn là ứng viên sai, cổng năm là bạn và không nên dựng vào. Nếu phần lớn là ứng viên

đúng, thì hai nguyên nhân đầu là chỗ đáng đầu tư, và cách chữa rẻ nhất là nối bán kính chiếu lại rồi tối ưu lại một lần nữa.

Nguyên tắc chung rút ra được: mỗi khi một cổng loại nhiều, câu hỏi tiếp theo luôn giống nhau, và nó là câu hỏi về nhân chứ không phải về ngưỡng.

E. Năm hướng chữa, xếp theo chỗ chúng tác động

Bảng 29.3 liệt kê các hướng đã biết. Hai hướng đầu không dùng học sâu chút nào.

Bảng 29.3. Năm hướng chữa cổng kiểm chứng, xếp theo tầng mà chúng thật sự tác động. Cột cuối không ghi kết quả của ai cả — nó ghi *điều kiện* để hướng đó có lãi, và phép đo bạn phải chạy để biết điều kiện ấy có thoả trên hệ mình hay không.

Hướng	Tác động ở tầng	Cơ chế	Bằng chứng
Rút mẫu tốt hơn: PROSAC, LO-RANSAC, MAGSAC++	Tầng ba, phần rút mẫu	Xếp thứ tự mẫu theo điểm khớp; tối ưu cục bộ trên tập inlier; bỏ ngưỡng cứng [265]–[267]	Rẻ, không cần huấn luyện. Chỉ giúp nếu biểu đồ inlier mang Dạng A
Giảm bậc tự do bằng trọng lực và stereo	Tầng ba, phần rút mẫu	Ghim tỉ lệ bằng stereo, ghim nghiêng bằng IMU, còn hai cặp điểm	Cùng điều kiện với dòng trên, cộng hai giả định nữa. Kiểm bằng phép thử tăng N ở Mạch 2 trước khi cài
Nối cổng đếm, để tầng sau quyết	Tầng ba, phần ngưỡng	Cho giả thuyết tốt nhất đi tiếp dù dưới ngưỡng, dùng chuỗi cổng sẵn có để lọc	Trần lợi ích bằng đúng số ứng viên đúng nằm trong khoảng trống giữa hai phân bố. Đếm nó trước, rồi mới quyết
Ghép cặp học được cho riêng cặp ứng viên	Tầng hai	SuperGlue hoặc LightGlue trên cặp ảnh, tăng cả số cặp lẫn tỉ lệ inlier [268], [269]	Hướng duy nhất tấn công thẳng trần inlier. Chi phí trả một lần cho mỗi ứng viên
Ghép cặp dày không cần điểm đặc trưng	Tầng hai	LoFTR hoặc RoMa cho tương ứng dày ở đường cơ sở rộng [270], [271]	Mạnh nhất khi lệch góc nhìn lớn; cũng đắt nhất, và chi phí trả theo từng cặp ứng viên

Còn một hướng thứ sáu, rẻ và không ai nói tới, nằm ngay trong tầng hai. Việc ghép cặp hiện bị giới hạn trong cùng nút từ vừng ở một mức cây cố định. Nối mức cây đó, hoặc ghép vết cận có kiểm tra tỉ số khoảng cách chỉ cho riêng cặp ứng viên, sẽ tăng số cặp đề xuất. Chi phí trả đúng một lần cho mỗi ứng viên, không phải cho mỗi khung hình. Đây là suy luận từ cơ chế trần inlier, chưa phải một kết quả đã kiểm chứng — nhưng nó rẻ tới mức đáng thử trước mọi hướng học sâu trong bảng.

F. Kiểm chứng học được: chấm điểm một cặp thay vì giải hình học

Có một hướng khác hẳn ở tầng ba: thay vì giải một phép biến đổi, huấn luyện một mạng chấm điểm “hai ảnh này có phải cùng một nơi không”. Hướng này xuất hiện dưới nhiều tên, thường là xếp hạng lại danh sách ngắn. Patch-NetVLAD thuộc nhóm này, và mọi bộ ghép cặp học được cũng có thể dùng theo kiểu này bằng cách lấy số cặp tin cậy làm điểm.

Điểm mạnh thì rõ. Một mạng có thể học các dấu hiệu mà hình học không dùng được: ngữ cảnh toàn ảnh, kết cấu bề mặt, bố cục thô của cảnh. Nó cũng không bị chặn bởi trần inlier, vì nó không cần một tập cặp nhất quán để đưa ra điểm số.

Nhưng có một khác biệt mà không lập luận nào xoá được. Bộ giải hình học trả về một *con số kiểm chứng được*: số inlier, tức số phép đo độc lập cùng ủng hộ một giả thuyết. Một mạng trả về một điểm số đã hiệu chỉnh kém và không có ý nghĩa vật lý. Với một hệ mà một vòng sai bẻ cả bản đồ, khác biệt đó quyết định.

Vì vậy cách dùng đúng là dùng nó làm *bộ lọc trước*, không phải làm bộ thay thế. Cụ thể: cho mạng chấm điểm và sắp xếp lại danh sách ứng viên, rồi chỉ chạy hình học cho vài ứng viên đầu. Bạn tiết kiệm được phần lớn chi phí ở tầng ba mà không mất cái công.

Có một trường hợp mà kiểm chứng học được thật sự cần thiết: khi tỉ lệ inlier thấp tới mức hình học không bao giờ hội tụ, chẳng hạn đi lại sau sáu tháng ở cảnh ngoài trời đổi mùa. Ở đó hình học không có gì để làm việc, nên một điểm số học được vẫn hơn không có gì. Đây không phải chế độ vận hành của một hệ chạy chuỗi ngắn trong nhà.

G. Đưa ghép cặp học được vào tầng kiểm chứng thì đổi gì

SuperGlue coi ghép cặp là bài toán gán tối ưu giữa hai tập điểm, giải bằng một mạng đồ thị có *attention* chéo giữa hai ảnh [268]. LightGlue giữ cùng ý tưởng nhưng cho phép dừng sớm theo độ khó của cặp, nên chi phí thích nghi [269].

Ba thay đổi xảy ra khi đưa chúng vào tầng hai của vòng lặp.

- **Số cặp tăng và tỉ lệ inlier tăng** — đây là tác động mong muốn, và nó đánh đúng vào trần inlier.
- **Chi phí trả theo ứng viên, không theo khung hình** — khác hẳn khi dùng chúng cho bám vết. Đây là tình tiết giảm nhẹ quan trọng nhất: số lần gọi bằng số ứng viên mỗi khung hình khoá, tức nhỏ hơn tần số ảnh nhiều bậc. Con số mili-giây thì phải đo trên phần cứng đích, nhưng *cấu trúc* chi phí đã đủ để nói rằng hướng này khả thi hơn hẳn việc chạy cùng mô hình đó cho bám vết.
- **Điểm vận hành dịch chuyển** — nhiều cặp hơn cũng nghĩa là ứng viên sai có nhiều cơ hội hơn để tạo ra một Sim3 trông hợp lý. Precision có thể tụt.

Điểm thứ ba là chỗ hay bị quên. Khoảng trống giữa hai phân bố trong Hình 29.2 là một tính chất của *bộ ghép cặp hiện tại*, không phải một hằng số của bài toán. Đối bộ ghép cặp là đổi cả mẫu số lẫn phân bố, nên phải đo lại khoảng trống đó và đặt lại ngưỡng theo phân bố mới. Nếu không, bạn nhận thêm recall kèm một tỉ lệ vòng sai chưa được đo — và mục sau giải thích vì sao đó là cuộc đổi chác tồi nhất có thể.

I 29.6 Đo cho đúng: recall, precision và cái giá của một vòng sai (Measuring Loop Closure)

A. Định nghĩa trước, con số sau

Không có con số recall nào có nghĩa nếu chưa nói ba thứ. Thứ nhất, ngưỡng khoảng cách để gọi hai khung hình là “cùng nơi”. Ngoài trời người ta hay dùng 25 mét; trong nhà con số đó vô nghĩa, phải dùng vài mét. Thứ hai, mẫu số: đếm trên tổng số truy vấn, hay chỉ trên các truy vấn thật sự có lần đi lại. Thứ ba, đơn vị: một khung hình, hay một lần đi lại gồm nhiều khung hình.

Ba định nghĩa này có thể làm cùng một hệ hiện ra với recall rất khác nhau. Vì vậy khi đọc một bảng của người khác, hãy tìm ba định nghĩa đó trước khi đọc con số [272].

Một ví dụ tính tay cho thấy độ nhạy. Giả sử một hệ đóng đúng 40 vòng. Với ngưỡng bốn mét, có 100 lần đi lại được coi là “có thật”, nên recall là 0,40. Nới ngưỡng lên mười mét, số lần đi lại “có thật” tăng lên 150, trong khi số vòng đóng được chỉ tăng lên 45 vì các cặp mới thêm đều lệch

góc nhìn lớn. Recall khi đó là 0,30. Cùng một hệ, cùng một lần chạy, hai con số khác nhau một phần tư chỉ vì đổi định nghĩa. Đây cũng là lý do không so được recall giữa hai bài báo dùng hai ngưỡng khác nhau.

B. Đường cong theo số ứng viên, và giới hạn của nó

Chỉ số chuẩn của tầng một là recall theo số ứng viên: xác suất có ít nhất một ứng viên đúng trong N ứng viên đầu. Đường cong này tăng đơn điệu theo N , nên nó luôn đẹp lên khi ta lấy nhiều ứng viên hơn.

Đó cũng chính là giới hạn của nó. Nó là chỉ số của *một tầng*, không phải của hệ thống. Tầng N làm recall tầng một tăng, đồng thời làm khối lượng việc của tầng ba tăng tuyến tính và làm số ứng viên sai lọt vào tầng ba tăng theo. Một hệ có recall tầng một tuyệt vời vẫn có thể đóng số vòng bằng không.

Vì vậy phải báo cáo hai đường cong cạnh nhau: recall theo số ứng viên ở tầng một, và precision-recall của *toàn bộ chuỗi cổng*. Cặp đường cong này là thứ duy nhất trả lời được câu hỏi “nới ngưỡng thì được gì và mất gì”.

C. Dựng đường precision-recall cho cả chuỗi cổng

Một chuỗi sáu cổng không có một ngưỡng duy nhất để quét, nên việc dựng đường cong cần kỹ luật. Quy trình đúng gồm ba bước.

1. Chọn *một* ngưỡng làm trục quét, thường là ngưỡng inlier ở cổng ba, rồi giữ mọi ngưỡng khác cố định.
2. Với mỗi giá trị, chạy lại chuỗi cổng *trên nhật ký đã ghi*, không chạy lại hệ SLAM.
3. Với mỗi giá trị, đếm vòng được chấp nhận, tra nhãn chân trị, và ghi ra một cặp precision-recall.

Bước hai là bước hay bị làm sai nhất, và làm sai nó khiến cả đường cong vô nghĩa. Nếu bạn chạy lại toàn bộ hệ với ngưỡng mới, quỹ đạo sẽ khác, nên tập khung hình khoá khác, nên tập ứng viên khác. Hai điểm trên đường cong khi ấy không còn so được với nhau, vì chúng đến từ hai tập dữ liệu khác nhau. Đây đúng là quy tắc mỗi lần chỉ đổi một thứ, áp vào một chuỗi cổng.

Muốn quét lại trên nhật ký thì nhật ký phải đủ giàu. Bảng 29.4 liệt kê các trường tối thiểu và nói mỗi trường chẩn đoán được gì. Sáu trường này đủ để tái dựng mọi quyết định của chuỗi cổng một cách ngoại tuyến.

Bảng 29.4. Các trường tối thiểu cần ghi cho mỗi ứng viên. Cột cuối là lý do ghi: mỗi trường trả lời một câu hỏi chẩn đoán mà các trường khác không trả lời được.

Trường	Nội dung	Chẩn đoán được gì
Mã hai khung hình khoá	Định danh truy vấn và ứng viên	Tra chân trị để gắn nhãn đúng-sai
Điểm truy hồi	Điểm số của tầng một	Dựng đường recall theo số ứng viên
Số khớp túi từ	Số cặp mà tầng hai đề xuất	Mẫu số của tỉ lệ inlier; phát hiện trần inlier
Số inlier tốt nhất của RANSAC	Giả thuyết tốt nhất sau 300 vòng lặp	Phân biệt thiếu mẫu với trần inlier
Số khớp sau khi chiếu lại	Kết quả tìm thêm khớp có dẫn hướng	Cho biết nghiệm thô có đủ chính xác không
Số inlier sau tối ưu Sim3	Kết quả cổng năm	Tách ba nguyên nhân chết ở cổng năm

Chỉ số nên báo cáo là recall tại mức precision 100 %. Nó thô, nhưng nó đúng bản chất bài toán: nếu tôi không được phép sai lần nào thì tôi bắt được bao nhiêu phần số vòng có thật. Kèm theo nó phải là một cảnh báo về cỡ mẫu. Với vài chục vòng đúng trong toàn bộ thí nghiệm, đúng một vòng sai đã kéo precision từ 100 % xuống khoảng 97 %. Vì vậy luôn báo cáo số vòng tuyệt đối bên cạnh tỉ lệ phần trăm.

D. Bất đối xứng chi phí: vì sao điểm vận hành lệch hẳn về precision

Đây là điểm mấu chốt của cả mục. Một vòng bị bỏ lỡ và một vòng sai không đối xứng nhau, và khoảng cách giữa chúng rất lớn.

Một vòng bỏ lỡ để lại đúng phần trôi đã tích lũy. Phần đó có giới hạn, nó không lan ra chỗ khác, và một vòng sau vẫn có thể sửa nó.

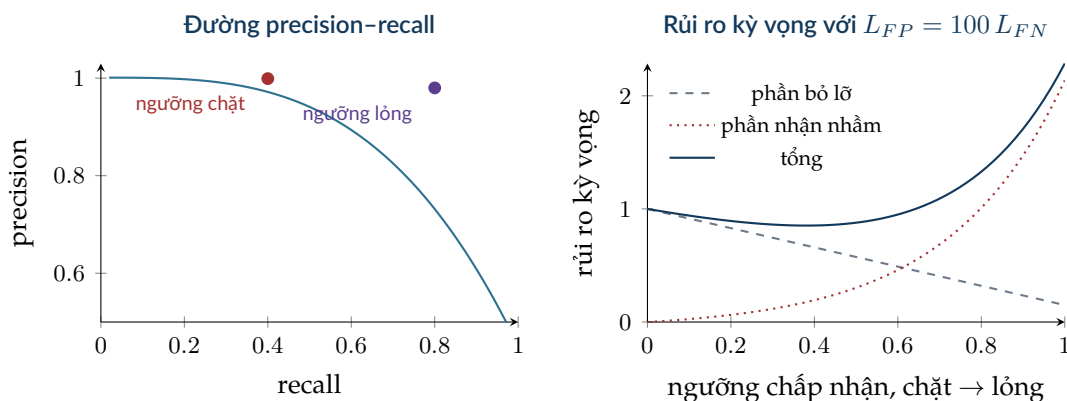
Một vòng sai thì khác hẳn. Tầng bốn nhận nó như một sự thật và phân bố lại sai số cho toàn bộ đồ thị. Bản đồ bị bẻ, và các khung hình khoá cách xa hàng trăm mét cũng bị dời. Hư hỏng không tự khỏi, và việc gỡ nó ra đòi cơ chế phục hồi bản đồ mà phần lớn hệ không có.

Viết thành rủi ro kỳ vọng thì rất gọn. Gọi p_{FP} là xác suất nhận nhầm và p_{FN} là xác suất bỏ lỡ, với L là thiệt hại tương ứng:

$$R = p_{FP} L_{FP} + p_{FN} L_{FN}.$$

Câu diễn giải: bạn không tối thiểu hoá số lần sai, bạn tối thiểu hoá *thiệt hại*. Khi L_{FP} lớn hơn L_{FN} chừng một bậc, điểm tối ưu dịch rất xa về phía precision. Đó là lý do cộng đồng nhận dạng địa điểm báo cáo “recall ở mức precision 100 %” thay vì recall trung bình.

Ví dụ tính tay được. Giả sử $L_{FP} = 100$ và $L_{FN} = 1$, tính bằng đơn vị tùy ý. Ngưỡng lỏng cho $p_{FN} = 0,2$ và $p_{FP} = 0,02$, nên $R = 2,0 + 0,2 = 2,2$. Ngưỡng chặt cho $p_{FN} = 0,6$ và $p_{FP} = 0,001$, nên $R = 0,1 + 0,6 = 0,7$. Ngưỡng chặt thắng, dù nó bỏ lỡ ba phần năm số vòng. Hình 29.8 vẽ cả hai đường.



Hình 29.8. Trái: đường precision–recall minh hoạ của một chuỗi cổng đóng vòng, kèm hai điểm vận hành. Phải: rủi ro kỳ vọng khi một vòng sai đắt gấp một trăm lần một vòng bỏ lỡ. Cực tiểu của đường tổng nằm lệch hẳn về phía ngưỡng chặt, dù ở đó recall chỉ khoảng bốn phần mười. Cả hai đồ thị là minh hoạ dạng hàm, không phải số đo.

E. Đo ở tầng hệ thống, và đọc đuôi phân phối

Recall và precision là chỉ số của bộ phát hiện. Chúng không nói bản đồ tốt lên hay xấu đi. Chỉ số hệ thống là sai số quỹ đạo, đo bằng *evo*, chạy nhiều lần.

Điểm kỹ thuật quan trọng: đọc *đuôi phân phối*, không đọc trung vị. Một vòng sai xảy ra hiếm, nên nó gần như không dời trung vị, nhưng mỗi lần xảy ra thì nó phá hỏng nguyên một lần chạy. Dấu hiệu đặc trưng vì thế rất dễ nhận: trung vị nhích lên một chút trong khi phân vị 95 nhảy lên nhiều

lần. Nếu chỉ nhìn trung vị, thay đổi trông như một khoản lỗ nhỏ đáng chấp nhận; nhìn đuôi thì nó là hồng. Đây là lý do một thay đổi ở tầng ba *bắt buộc* phải báo cáo cả hai con số.

Cách chọn số lần chạy và cách phân biệt cải tiến thật với nhiễu nằm ở [B/07-chia-du-lieu-va-danh-gia](#) và [E/24-thi-nghiem-va-ablation](#). Ở đây chỉ cần một quy tắc: mọi so sánh về đóng vòng phải báo cáo cả trung vị lẫn phân vị 95, trên ít nhất mười lần chạy.

Còn một cạm bẫy về chân trị. Trên TUM-VI, chân trị chỉ phủ phòng có hệ bất chuyển động, nên các vòng đóng giữa hành lang không thể kiểm bằng chân trị. Ở những chuỗi như vậy, bằng chứng duy nhất là sai số quỹ đạo cuối cùng, không phải nhân đúng-sai của từng vòng.

F. Đo chi phí cho đúng: đuôi độ trễ và thời gian bị chặn

Còn một loại số phải đo, và nó không phải recall. Đóng vòng là mô-đun duy nhất trong hệ có hai chế độ chi phí rất khác nhau. Chi phí phát hiện chạy trên mọi khung hình khoá; chi phí hiệu chỉnh chỉ chạy khi có vòng.

Chi phí phát hiện phải đọc ở đuôi, không ở trung vị, và lý do là cấu trúc chứ không phải một con số cụ thể: số ứng viên mỗi khung hình khoá dao động rất mạnh, nên phân vị 99 của tầng này thường cách trung vị vài lần. Lập ngân sách theo trung vị nghĩa là trượt hạn đúng ở những chỗ hệ đang bận nhất — tức là đúng những chỗ bạn cần nó nhất.

Chi phí hiệu chỉnh thuộc loại khác hẳn, vì nó *chặn* luồng dựng bản đồ cục bộ. Nó chạy hiếm, nên nó gần như không xuất hiện trong bất kỳ thống kê trung bình nào; nhưng khi chạy thì nó dài hơn một chu kỳ khung hình rất nhiều, và trong suốt khoảng đó hệ không tạo được điểm bản đồ mới. Nếu robot đang di chuyển nhanh ngay lúc ấy, cửa sổ đó là một rủi ro mất bám vết thật. Con số phải đo là thời gian *bị chặn* của luồng dựng bản đồ, không phải thời gian chạy của hàm hiệu chỉnh.

Ba quy tắc đo rút ra được, và cả ba đều dễ bỏ sót.

1. Đo trên phần cứng đích khi hệ đang chạy đủ tải, không đo mô-đun đứng một mình.
2. Báo cáo phân vị 99 bên cạnh trung vị, cho cả phát hiện lẫn hiệu chỉnh.
3. Ghi lại thời gian mà các luồng khác bị chặn, vì đó mới là chi phí hệ thống thật của một vòng.

G. Ba dấu hiệu của một vòng sai mà bạn chưa phát hiện

Vòng sai hiếm khi tự khai. Nó không sinh ra lỗi, không làm sập hệ, và trên phần lớn chuỗi nó không dờn trung vị. Vì vậy cần biết ba dấu hiệu gián tiếp, và cả ba đều rẻ để ghi lại.

Dấu hiệu thứ nhất: bước nhảy tư thế tại thời điểm hiệu chỉnh. Sau khi đóng vòng, tư thế của khung hình khoá gần nhất bị dờn. Lượng dờn đó có ý nghĩa: nó chính là phần trôi mà vòng vừa xoá. Nếu nó lớn hơn nhiều so với trôi hợp lý của quãng đường đã đi, vòng đó đáng nghi. Ghi lại chuẩn của lượng dờn cho mỗi lần hiệu chỉnh là một dòng mã.

Dấu hiệu thứ hai: chi phí còn dư sau tối ưu đồ thị tăng vọt. Một ràng buộc sai mâu thuẫn với phần còn lại của đồ thị, nên nó để lại phần dư lớn ở chính nó hoặc ở các cạnh lân cận. Ghi tổng chi phí trước và sau mỗi lần tối ưu, rồi so với các lần trước, cho một cảnh báo sớm.

Dấu hiệu thứ ba: quỹ đạo có đoạn gấp khúc không giải thích được bằng chuyển động. Robot không dịch chuyển tức thời. Một đoạn quỹ đạo gãy góc tại đúng thời điểm hiệu chỉnh là bằng chứng hình học trực tiếp.

Ba dấu hiệu này đều là dấu hiệu *sau khi* vòng đã được nhận. Chúng không thay được công ở tầng ba, nhưng chúng biến một lỗi im lặng thành một lỗi có triệu chứng. Với một hệ chạy nhiều giờ

ngoài hiện trường, đó là khác biệt giữa việc phát hiện vấn đề trong ngày hôm đó và phát hiện nó sau ba tuần.

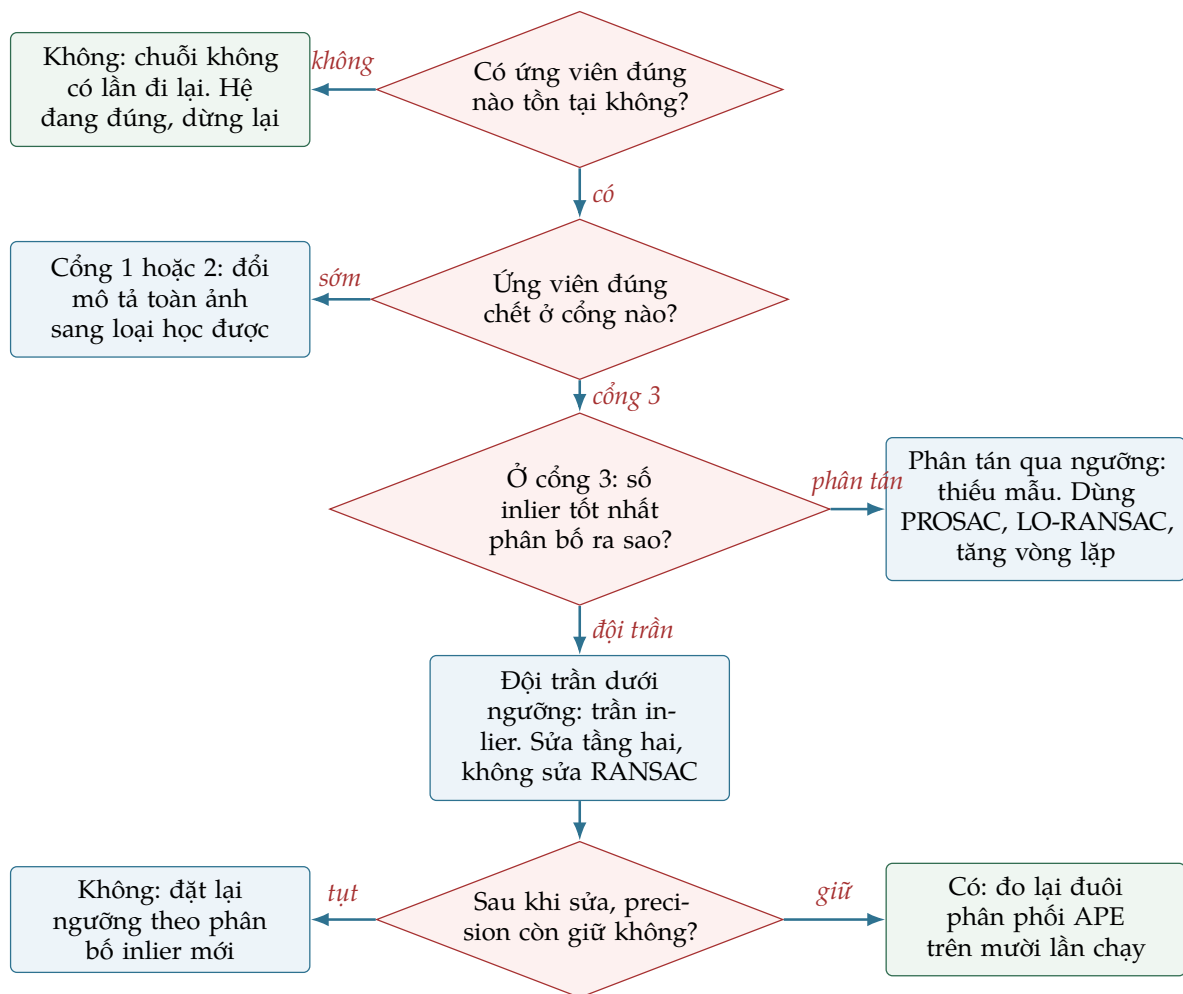
H. Hàng phòng thủ thứ hai: để bộ tối ưu tự chối vòng xấu

Vì precision không bao giờ đạt tuyệt đối, nên có một dòng nghiên cứu đặt lớp bảo vệ ở tầng bốn. Ý tưởng chung là để bộ tối ưu tự hạ trọng số hoặc tự tắt một ràng buộc trông không nhất quán với phần còn lại của đồ thị.

Ba cách kinh điển. Ràng buộc có công tắc gắn thêm một biến ẩn cho mỗi vòng, và biến đó được tối ưu cùng tư thế [273]. Co giãn hiệp phương sai động cho cùng hiệu ứng bằng một biểu thức đóng, rẻ hơn [274]. Mô hình hỗn hợp cực đại thay mỗi ràng buộc bằng một hỗn hợp Gauss, trong đó một thành phần rất rộng đóng vai trò “ràng buộc này có thể sai” [275]. Ngoài ra còn hướng kiểm tra tính nhất quán của cả nhóm vòng theo thời gian rồi loại cả nhóm nếu chúng mâu thuẫn nhau [276].

Điểm cần nói thẳng: các cơ chế này giảm thiệt hại, chúng không thay được một cổng kiểm chứng tốt. Chúng cũng có chi phí riêng, vì một ràng buộc đúng nằm ở vùng đuôi cũng có thể bị tắt nhầm. Với hệ đang chạy thời gian thực, hướng rẻ nhất vẫn là đặt ngưỡng chặt ở tầng ba và chấp nhận bỏ lỡ.

Hình 29.9 gói toàn bộ chương thành một cây quyết định chẩn đoán.



Hình 29.9. Cây chẩn đoán đóng vòng. Điểm quan trọng là thứ tự: hai câu hỏi đầu là phép đo, không phải lựa chọn kỹ thuật, nên bạn không được phép đi tiếp khi chưa có số. Giả thuyết làm việc — và chỉ là giả thuyết — là trên một hệ stereo-inertial chạy chuỗi ngắn cùng điều kiện ánh sáng, nhánh hay gặp nhất đi thẳng xuống ô “trần inlier”, tức phải sửa ở tầng ghép đặc trưng chứ không ở tầng RANSAC. Nếu hệ của bạn không đi vào nhánh đó, cái sai nằm ở giả thuyết chứ không ở cây.

29.7 Giới hạn và trường hợp hỏng

Giới hạn lớn nhất của cả chương phải nói thẳng ngay: chương này *không chứa số đo*. Nó không nói với bạn rằng cổng ba giết bao nhiêu phần trăm ứng viên trên hệ nào, vì một con số như thế chỉ có nghĩa kèm theo một hệ cụ thể, một nhóm tập dữ liệu cụ thể và một chính sách chọn khung hình khoá cụ thể — và khi thiếu ba thứ đó, nó gây hại nhiều hơn giúp. Thứ chương này cho bạn là bộ khung: bốn tầng, các cổng, phép đo đặt ở đâu, và cách đọc kết quả. Mọi phát biểu định tính trong chương — “cổng ba thường là chỗ lọc mạnh nhất”, “trần inlier hay gặp hơn thiếu mẫu trên hệ stereo-inertial chuỗi ngắn” — là giả thuyết làm việc, hợp lý về cơ chế nhưng chưa được chương này chứng minh. Hãy coi chúng là chỗ để bắt đầu tìm, không phải kết luận để trích dẫn.

Kèm theo đó là một giới hạn về phạm vi. Toàn bộ lập luận được viết cho một hệ stereo-inertial chạy chuỗi ngắn, trong nhà hoặc bán trong nhà, cùng điều kiện ánh sáng. Ba đặc điểm ấy quyết định kết luận.

Nếu đổi sang đơn ảnh, tỉ lệ trôi tỉ lệ sẽ lớn hơn, Sim3 thật sự cần bảy bậc, và tỉ lệ inlier thấp hơn. Nếu đổi sang ngoài trời qua nhiều buổi trong ngày, tầng một sẽ trở thành nút thắt thật, và kết luận “đừng thay DBow2” đảo ngược. Nếu đổi sang bản đồ hàng trăm nghìn khung hình khoá,

tf-idf bão hoà và truy hồi học được thắng rõ. Nói cách khác, kết luận trung tâm của chương là một kết luận *có điều kiện*, và điều kiện phải đi kèm mỗi lần nó được nhắc lại.

Chế độ hỏng thứ hai là chính phép đo. Bảng sống sót đo được các ứng viên *đã được đề xuất*. Nó mù hoàn toàn với các lần đi lại mà tăng một không đề xuất gì. Muốn thấy phần mù đó, phải dựng danh sách ứng viên bằng chân trị chứ không bằng bộ truy hồi, tức là chạy một thí nghiệm oracle. Không có bước đó, bạn có thể kết luận rằng tăng một tốt trong khi thật ra nó đang im lặng bỏ sót.

Chế độ hỏng thứ ba nằm ở chân trị. Ngưỡng khoảng cách để gọi hai khung hình là cùng nơi là một lựa chọn, không phải sự thật. Hai khung hình cách nhau ba mét nhưng quay lưng vào nhau thì không hề nhìn chung một cảnh. Một nhãn chỉ dựa trên khoảng cách sẽ gọi chúng là ứng viên đúng, và tăng ba loại chúng là hoàn toàn hợp lý. Nhãn tốt hơn phải tính cả phần chồng lấn trường nhìn, và việc bỏ qua điều này làm recall trông thấp hơn thực tế.

Chế độ hỏng thứ tư mang tính hệ thống. Khi vòng được chấp nhận, việc hiệu chỉnh bản đồ chặn luồng dựng bản đồ cục bộ trong một khoảng dài hơn hẳn một chu kỳ khung hình, và trên phần cứng nhúng thì khoảng đó còn dài hơn. Nghĩa là mỗi vòng đóng thành công cũng là một cửa sổ rủi ro cho việc mất bám vết và cho tranh chấp luồng. Một hệ đóng nhiều vòng hơn chưa chắc là một hệ tốt hơn, và đó là lý do chỉ số cuối cùng phải là sai số quỹ đạo chứ không phải số vòng.

Cuối cùng là một giới hạn về niềm tin. Toàn bộ chương giả định rằng bạn tin vào phép đo của chính mình. Cái bẫy biến đếm ở mục trước cho thấy niềm tin đó có thể sai suốt nhiều tuần mà không có triệu chứng nào. Cách duy nhất để bảo vệ là kiểm chéo mỗi bảng sống sót bằng một đường độc lập, chẳng hạn đếm lại từ nhật ký thô thay vì tin biến đếm.

29.8 Thực hành

Repo và tài nguyên. [cvgl/Hierarchical-Localization](#) là quy trình định vị nên đọc trước tiên, vì nó chính là cái phễu bốn tầng ở dạng mã nguồn. [dorian3d/DBow2](#) để đọc lại phần bạn đang dùng. [QVPR/Patch-NetVLAD](#) và [AnyLoc/AnyLoc](#) cho hai đầu của phổ mô tả toàn ảnh. [gmberton/CosPlace](#) và [amaralibey/MixVPR](#) nếu bạn muốn thử huấn luyện lại. [cvgl/LightGlue](#) cho ghép cặp có chi phí thích nghi, và [zju3dv/LoFTR](#) cho ghép cặp dày. [nianticlabs/ace](#) nếu tình huống triển khai của bạn là toà nhà cố định. [colmap/colmap](#) và [MichaelGrupp/evo](#) là hai công cụ hạ tầng bền nhất trong danh sách. Danh sách tên mô hình này chốt tại tháng 9/2026 và sẽ cũ trong sáu tới mười hai tháng; các repo hạ tầng như COLMAP và evo thì bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Bạn đặt biến đếm ở đâu để biết tầng nào đang giết vòng, và vì sao đặt trước câu lệnh `if` lại làm sai nhãn cột? Nhìn một biểu đồ số inlier tốt nhất, bạn phân biệt thiếu mẫu với trần inlier bằng dấu hiệu nào? Vì sao hồi quy tư thế trực tiếp lại nguy hiểm hơn hồi quy toạ độ cảnh trong một hệ SLAM? Nếu một vòng sai đất gấp một trăm lần một vòng bỏ lỡ, ngưỡng tối ưu nằm ở đâu trên đường precision–recall?

Mini project. Xem khối dự án ở cuối chương: dựng bảng sống sót bốn tầng cho chính hệ của bạn, kèm nhãn đúng–sai từ chân trị.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Bốn tầng của đóng vòng	Lấy ứng viên, ghép đặc trưng, kiểm chứng hình học, tối ưu tư thế. Mỗi tầng là một bộ lọc riêng, nên câu hỏi đúng luôn là “tầng nào đang hẹp” chứ không phải “hệ có đóng vòng không”.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Bảng sống sót từng cổng	Số ứng viên vượt qua mỗi cổng, tính theo phần trăm của số ứng viên tới cổng đó. Nó là phép đo rõ nhất cho biết tầng nào giết vòng, và dụng nó chỉ tồn vài chục dòng mã.
●●● Biến đếm đặt trước câu lệnh <code>if</code> — hậu quả?	Cột mang tên cổng đó thật ra đếm số ca đã vượt cổng trước nó, nên mọi nhân cột dịch một bậc và cổng bị đổ tội luôn là cổng đứng <i>sau</i> thủ phạm thật. Bảng vẫn nhất quán với chính nó nên không có triệu chứng nào. Cách phòng: tăng biến đếm bên trong khối <code>if</code> .
●●● Trần inlier	Số cặp đúng thật sự tồn tại đã ít hơn ngưỡng, nên không cách rút mẫu nào cứu được. Dấu hiệu nhận biết: số inlier tốt nhất của các ứng viên đúng đội trần ngay dưới ngưỡng.
●● Thiếu mẫu	Inlier có đủ nhưng số vòng lặp không đủ để rút trúng một tập mẫu toàn inlier. Dấu hiệu: số inlier tốt nhất phân tán qua ngưỡng. Chữa bằng PROSAC, LO-RANSAC, hoặc tăng số vòng lặp.
●● Công thức hội tụ của RANSAC	$p = 1 - (1 - w^m)^N$, với w là tỉ lệ inlier, m là cỡ tập mẫu tối thiểu, N là số vòng lặp. Thay thử với $m = 3$, $N = 300$: $w = 0,2$ cho $p \approx 0,91$, còn $w = 0,1$ chỉ cho $0,26$ — chia đôi w đủ để biến một cổng gần như luôn mở thành một cổng gần như luôn đóng.
●●● Ba giả định ẩn khi rút tập mẫu từ ba xuống hai	Một: inlier có đủ, chỉ thiếu lượt rút trúng — sai nếu biểu đồ mang Dạng B. Hai: trọng lực ghim hai bậc xoay <i>đúng</i> — sai nếu phần nghiêng còn dư đáng kể. Ba: thêm vòng thì bản đồ chỉ tốt lên — sai vì mỗi lần hiệu chỉnh là một cửa sổ rủi ro. Phép kiểm rõ nhất: tăng N lên mười lần trước đã.
●● Nhãn đúng–sai từ chân trị	Gắn nhãn cho từng ứng viên bằng khoảng cách tư thế thật, với một ngưỡng do bạn chọn — vài mét trong nhà, rộng hơn nhiều ngoài trời. Nó biến bảng sống sót thành hai bảng, và chỉ khi đó mới biết tầng hẹp đang loại đúng hay loại sai.
●●● Túi từ DBoW2	Biểu đồ tần suất các từ thị giác nhị phân, dùng lại chính mô tả ORB đã trích cho bám vết nên gần như miễn phí. Gây khi đổi ánh sáng mạnh, lệch góc nhìn lớn, cấu trúc lặp, hoặc nền ít vân.
●●● NetVLAD	Gộp phần dư quanh K tâm cụm, với phép gán làm mềm bằng softmax nên khả vi. Nhờ đó cả tâm cụm lẫn mạng trích đặc trưng đều học được từ ảnh cùng nơi khác thời điểm.
●● Patch-NetVLAD	Xếp hạng lại danh sách ngắn bằng các mảnh đặc trưng có tính tới vị trí không gian. Nó kéo một phần việc kiểm chứng lên tầng truy hồi, và chi phí tăng theo số ứng viên.
●● HF-Net	Một <i>backbone</i> chung, ba đầu ra: mô tả toàn ảnh, điểm đặc trưng, mô tả cục bộ. Bài học thiết kế là trả một lần cho một bức ảnh thay vì trả hai lần cho hai mạng.
●● CosPlace	Đổi huấn luyện nhận dạng địa điểm từ xếp hạng bộ ba sang phân lớp trên các ô địa lý. Nó cho thấy nút thắt khi ấy nằm ở quy trình dữ liệu chứ không ở kiến trúc.
●● SALAD	Xem việc gán đặc trưng vào cụm như bài toán vận tải tối ưu, kèm một cụm thùng rác cho đặc trưng vô ích, đặt trên nền DINOv2.
●● AnyLoc	Lấy đặc trưng trung gian của mô hình nền rồi gộp bằng VLAD, không huấn luyện lại. Mạnh nhất khi ra ngoài phân phối; đắt hơn về tính toán và mô tả dài hơn.
●●● Phản bác của Sattler 2019	Hồi quy tư thế trực tiếp hành xử gần với truy hồi ảnh có nội suy hơn là với định vị dựa trên cấu trúc. Nó nội suy tốt trong phân phối tư thế huấn luyện và ngoại suy rất tệ ngoài đó.
●● Hồi quy toạ độ cảnh	Mạng dự đoán toạ độ ba chiều cho từng điểm ảnh, rồi PnP trong RANSAC giải tư thế. Khác hồi quy tư thế ở chỗ nó giữ lại bộ giải hình học, nên vẫn có inlier để làm cổng.
●● ACE	Tách <i>backbone</i> không phụ thuộc cảnh khỏi một đầu MLP nhỏ riêng cảnh; bản đồ của một cảnh thu về đúng trọng số của đầu MLP đó. Không hợp vòng SLAM trực tuyến vì bản đồ ở đó lớn dần liên tục.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● hloc	Truy hồi, trích đặc trưng học được, ghép cặp học được, rồi PnP và RANSAC trên mô hình ba chiều. Chính là cái phễu bốn tầng với ba tầng đầu học được và tầng cuối giữ nguyên hình học.
●●● Vì sao điểm vận hành lệch về precision?	Vì một vòng sai bẽ cả bản đồ và không tự khởi, còn một vòng bỏ lỡ chỉ để lại phần trôi có giới hạn. Khi thiệt hại lệch một bậc, cực tiểu rủi ro kỳ vọng dịch hẳn về ngưỡng chặt.
●● APE trung vị gần như không đổi nhưng phân vị 95 nhảy lên nhiều lần — nghĩa là gì?	Dấu hiệu điển hình của vòng sai: nó hiếm nên không dời trung vị, nhưng mỗi lần xảy ra thì phá một lần chạy. Luôn báo cáo cả trung vị lẫn đuôi trên ít nhất mười lần chạy.
●● Ràng buộc có công tắc	Gắn thêm một biến ẩn cho mỗi vòng để bộ tối ưu tự tắt ràng buộc không nhất quán. Nó giảm thiệt hại nhưng không thay được một công kiểm chứng tốt, và có thể tắt nhầm ràng buộc đúng.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [40]	Chuỗi bốn tầng ở Hình 29.1 là mô tả trực tiếp mô-đun đóng vòng và gộp bản đồ của nó	Hạ tầng tham chiếu; mã nguồn mở
hloc [251]	Thay ba tầng đầu bằng mô-đun học được, giữ nguyên bộ giải hình học ở tầng cuối	Quy trình chuẩn cho định vị trực quan
LightGlue [269]	Ghép cặp học được với chi phí thích nghi theo độ khó cặp ảnh — đúng chỗ trần inlier	Dùng rộng rãi, có bản ONNX
AnyLoc [256]	Đặc trưng mô hình nền cộng gộp VLAD, không huấn luyện lại	Nhắm bài toán ngoài phân phối
ACE [262]	Tách <i>backbone</i> không phụ thuộc cảnh khỏi đầu MLP riêng cảnh, nên dựng bản đồ là huấn luyện một đầu nhỏ	Hợp với triển khai một toà nhà cố định
Đo trên nhánh của bạn	Bảng sống sót từng cổng, nhân chân trị, phép thử tăng N trước khi hạ m	Chương không đưa số; nó đưa chỗ đặt biến đếm

MINI PROJECT SLAM · Bảng sống sót bốn tầng cho bộ đóng vòng của bạn · 1–2 ngày

Mục tiêu	biến câu “vòng không nổ” thành bốn con số, rồi tách bốn con số ấy theo nhãn đúng–sai để biết tầng hẹp đang loại đúng hay loại sai.
Dữ liệu	bốn chuỗi có lần đi lại thật và một chuỗi đi thẳng không quay lại, làm đối chứng âm. Cấu hình mặc định, không đổi tham số nào.
Các bước	thêm một biến đếm <i>bên trong</i> mỗi khối if của chuỗi cổng, không đặt trước nó; ghi mỗi ứng viên thành một dòng nhật ký gồm mã khung hình khoá, số khớp túi từ, số inlier tốt nhất của RANSAC, và kết cục; chạy mười lần mỗi chuỗi; tra chân trị cho từng cặp khung hình khoá rồi gắn nhãn đúng nếu khoảng cách dưới bốn mét; lập bảng sống sót hai lần, một lần chung và một lần tách theo nhãn; cuối cùng vẽ biểu đồ số inlier tốt nhất cho hai nhóm nhãn.
Xong khi	bạn nói được câu “trên hệ của tôi, $X\%$ số ca chết nằm ở cổng Y , và trong đó $Z\%$ là loại đúng” kèm bảng số. Kiểm tra chéo bắt buộc: chuỗi đi thẳng phải cho gần như không có ứng viên đúng nào; nếu nó vẫn cho nhiều, nhãn chân trị của bạn sai. Kiểm tra thứ hai: đếm lại một cổng từ nhật ký thô và đối chiếu với biến đếm.
Nối với	E/24 cho thiết kế thí nghiệm và số lần chạy; F/28 là nơi bạn sẽ sửa nếu kết quả chỉ vào trần inlier; B/07 cho cách định nghĩa recall và precision trước khi báo cáo.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Đặt một biến đếm *bên trong* mỗi khối *if* của chuỗi cổng, cộng một dòng nhật ký cho mỗi ứng viên. Đặt trước câu lệnh *if* thì cột đếm “đã tới cổng” chứ không phải “đã qua cổng”, nên mọi nhân dịch một bậc và thủ phạm bị đổ nhầm.
- A2. Bằng nhãn chân trị. Tách bảng sống sót làm hai theo nhãn, rồi nhìn phân bố số inlier tốt nhất của riêng nhóm SAI: nếu nó tắt hẳn từ lâu trước ngưỡng thì mọi ca chết trong nhóm ấy là loại đúng, và một tỉ lệ sống sót thấp là bằng chứng cổng khỏe mạnh chứ không phải cổng hỏng.
- A3. Nếu nguyên nhân thật là trần inlier chứ không phải thiếu mẫu thì không cách rút mẫu nào chạm tới ngưỡng, vì số cặp nhất quán hình học vốn đã ít hơn ngưỡng. Dấu hiệu là Dạng B ở Hình 29.2, và phép kiểm rẻ nhất là tăng số vòng lặp lên mười lần: nếu tăng N không đổi gì thì hạ m cũng sẽ không đổi gì.
- A4. Vì nếu bảng sống sót cho thấy hai cổng đầu để lọt gần hết ứng viên thì tăng lấy ứng viên không phải nút thắt. Một bộ truy hồi tốt hơn khi ấy chỉ đưa thêm ứng viên vào một cái phiếu đang nghẽn ở tầng ba, nên số vòng đóng được không đổi dù recall theo số ứng viên tăng rõ rệt.
- A5. Vì thiệt hại lệch. Một vòng sai bẻ toàn bộ đồ thị và không tự khỏi; một vòng bỏ lỡ chỉ để lại phần trôi có giới hạn. Với $L_{FP} = 100 L_{FN}$, ngưỡng chặt cho rủi ro 0,7 còn ngưỡng lỏng cho 2,2.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn đi bộ trong một thành phố lạ và muốn biết mình đã đi qua chỗ này chưa. Bạn làm bốn việc, theo thứ tự. Đầu tiên bạn liếc nhanh và thấy vài góc phố quen quen. Sau đó bạn nhìn kỹ hơn, tìm những chi tiết cụ thể: cái biển hiệu, vết nứt trên tường, chậu cây ở góc. Tiếp theo bạn thử xem những chi tiết ấy có nằm đúng vị trí tương đối với nhau như trong trí nhớ không. Nếu có, bạn kết luận đã đi qua đây, và bạn sửa lại toàn bộ bản đồ trong đầu.

Bây giờ giả sử bạn không bao giờ nhận ra chỗ cũ. Phản xạ thông thường là đổ lỗi cho cái liếc đầu tiên, rồi cố nhìn kỹ hơn ngay từ bước một. Nhưng nếu bạn ghi lại xem mình dừng ở bước nào, bạn sẽ thấy điều khác. Bạn nhận ra góc phố rất tốt. Bạn dừng ở bước thứ ba, vì bạn chỉ nhớ được ba chi tiết, mà ba chi tiết thì không đủ để chắc chắn. Vấn đề nằm ở bước thứ hai: bạn ghi nhớ quá ít chi tiết.

Cái giá của việc dễ tính hơn thì rất nặng. Nếu bạn nhận nhầm một hành lang giống hệt hành lang khác, bạn không chỉ sai ở chỗ đó. Bạn sẽ kéo lệch toàn bộ bản đồ trong đầu, kể cả những đoạn bạn nhớ hoàn toàn chính xác. Một lần nhận nhầm phá nhiều hơn mười lần không nhận ra. Vì vậy quy tắc đúng là: thà đi qua chỗ cũ mà không biết, còn hơn tin nhầm một chỗ mới là chỗ cũ.

Chỗ không rút gọn được: lý do vì sao “ba chi tiết là không đủ” lại là một con số cụ thể chứ không phải cảm tính. Nó đến từ một phép tính xác suất về việc rút ngẫu nhiên bao nhiêu lần thì trúng một bộ chi tiết đều đúng. Mọi cách nói đơn giản hơn đều làm mất chỗ quan trọng nhất: bạn phải phân biệt “rút chưa đủ nhiều lần” với “vốn dĩ không có đủ chi tiết đúng để mà rút”.

Một câu: Đóng vòng là bốn cái sàng nối tiếp, và việc đầu tiên phải làm không phải là đổi cái sàng nào, mà là đo xem cái nào đang giữ lại thứ bạn cần.

29.9 Kết nối

[F/27-dat-hoc-sau-vao-dau](#) là tiền đề trực tiếp: khung bốn cổng ở đó chính là thứ được áp dụng nguyên vẹn trong mục quyết định có thay DBow2 hay không. [F/28-dac-trung-va-ghep-cap](#) là chương anh em và là chỗ mà giả thuyết trung tâm của chương này chỉ tới: nếu biểu đồ inlier của bạn mang Dạng B thì việc phải sửa nằm ở đó. [B/07-chia-du-lieu-va-danh-gia](#) cho định nghĩa recall và precision cùng cách chia dữ liệu để những con số ấy có nghĩa. [E/24-thi-nghiem-va-ablation](#) cho cách quyết định bao nhiêu lần chạy là đủ, và cho khái niệm thí nghiệm oracle mà mục đo dùng lại nguyên vẹn. [E/25-do-tin-cay-va-van-hanh](#) là nơi lập luận rủi ro kỳ vọng được trình bày tổng quát, còn ở đây nó chỉ là một trường hợp riêng với thiệt hại rất lệch. [F/34-imu-bat-dinh-va-trien-khai](#) nối tiếp về hai chuyện: phương trọng lực từ

IMU đã được dùng ở đây để rút bậc tự do, và ngân sách độ trễ trên Orin quyết định mọi lựa chọn trong bảng so sánh.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một tỉ lệ sống sót rất thấp ở tầng kiểm chứng không tự nó chứng minh rằng tầng đó hỏng, và bạn cần thêm phép đo nào?
2. Nếu biểu đồ số inlier tốt nhất của các ứng viên đúng đội trần ngay dưới ngưỡng, ba hướng chữa nào bị loại ngay lập tức, và vì sao?
3. Lý thuyết RANSAC hứa hẹn một bước nhảy lớn về xác suất hội tụ khi rút tập mẫu từ ba xuống hai. Ba giả định ẩn của lời hứa đó là gì, và với mỗi giả định, phép đo rẻ nhất để biết nó có đúng trên hệ của bạn là gì?
4. Bạn thay DBoW2 bằng một mô tả học được và recall theo số ứng viên tăng rõ rệt, nhưng số vòng đóng được không đổi. Chuyện gì đang xảy ra, và bạn kiểm bằng cách nào?
5. Vì sao hồi quy tư thế trực tiếp lại nguy hiểm hơn hồi quy tọa độ cảnh khi đưa vào một hệ SLAM đang chạy, ngoài chuyện sai số lớn hơn?
6. Đưa một bộ ghép cặp học được vào tầng hai làm tăng số cặp. Vì sao việc đó buộc bạn phải đo lại ngưỡng ở tầng ba, thay vì giữ nguyên?
7. Trên một chuỗi mà chân trị chỉ phủ một phần đường đi, bạn dựa vào bằng chứng nào để nói một vòng vừa đóng là đúng hay sai?
8. Vì sao một hệ đóng nhiều vòng hơn chưa chắc là một hệ tốt hơn, kể cả khi mọi vòng đều đúng?

Độ sâu, dòng quang và mô hình nền hình học (Depth, Flow and Geometric Foundation Models)

PHỤ THUỘC F/27-dat-hoc-sau-vao-dau (khung bốn cổng để quyết định thay hay không thay); F/28-dac-trung-va-ghep-cap (ghép cặp thừa hồng ở đâu); C/16/02-bieu-dien-3d-va-depth (tính không xác định được của tỉ lệ, bốn nguồn độ sâu); C/15/03-flow-tracking-va-rang-buoc (bài toán khẩu độ, Lucas-Kanade, kim tự tháp).

TÓM TẮT

Chương này hỏi một câu duy nhất: ba họ mô hình đang rất nóng — độ sâu đơn ảnh, dòng quang dày, và mô hình nền hình học — thêm được *thông tin* gì cho một hệ stereo-inertial vốn đã có tỉ lệ mét thật. Đọc xong bạn biết chính xác chỗ nào chúng thêm thông tin thật, chỗ nào chúng chỉ thêm phỏng đoán, và bạn tính được bằng số khi nào một tiên nghiệm dày sẽ đè bẹp phép đo thừa của bạn trong bài toán bình phương tối thiểu. Bạn cũng đọc được ba thiết kế đáng học nhất trong nhóm này: vòng lặp tra cứu của RAFT, phép chuẩn hoá theo tiêu cự của Metric3D, và cách DUST3R đổi biểu diễn đầu ra để xoá luôn bước ước lượng tư thế. Nếu chỉ nhớ một điều: những mô hình này không cho bạn phép đo mới, chúng cho bạn phỏng đoán dày — và chúng không bao giờ nói “tôi không biết”.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tỉ lệ (scale)** — ba nghĩa khác nhau trong chương này — (a) *tỉ lệ mét* của một bản đồ SLAM: hệ số nhân biến toạ độ bản đồ thành mét, thứ mà stereo và gia tốc kế cố định được; (b) *tỉ lệ* trong “bất biến tỉ lệ và dịch”: một hệ số nhân tự do trên đầu ra của mạng độ sâu, khác nhau ở mỗi khung hình; (c) *đa tỉ lệ* của kim tự tháp ảnh: mức thu nhỏ. Ba nghĩa không liên quan gì nhau, và trộn chúng là nguồn hiểu nhầm lớn nhất của cả chủ đề này.
- **độ chênh lệch (disparity)** — với một cặp ảnh stereo đã nắn thẳng hàng, đó là hiệu toạ độ ngang của cùng một điểm giữa ảnh trái và ảnh phải, tính bằng pixel. Nó tỉ lệ nghịch với khoảng cách: $d = fb/Z$, trong đó f là tiêu cự tính bằng pixel, b là **đường cơ sở** (stereo baseline) tính bằng mét.
- **độ sâu nghịch đảo (inverse depth)** — đại lượng $1/Z$. Mạng độ sâu hiện đại hầu hết dự đoán đại lượng này chứ không dự đoán Z , vì nó bị chặn (điểm ở vô cực cho giá trị 0) và vì sai số của nó gần với sai số pixel hơn.
- **bản đồ điểm (pointmap)** — một tensor kích thước $H \times W \times 3$: mỗi pixel của ảnh được gán thẳng một điểm ba chiều (X, Y, Z) trong một hệ toạ độ nào đó. Khác với bản đồ độ sâu ở chỗ nó đã mang sẵn hệ toạ độ, nên không cần biết tiêu cự mới đọc ra được hình học.
- **khối chi phí (cost volume)** — bảng ghi mức giống nhau giữa mỗi vị trí ở ảnh này và mỗi vị trí ứng viên ở ảnh kia. Nó là dạng dữ liệu chung của mọi bài toán tương ứng: stereo, dòng quang, ghép cặp.
- **làm cong (warping)** — lấy ảnh hoặc bản đồ đặc trưng thứ hai rồi dịch từng pixel theo một trường chuyển động ước lượng, để nó nằm chồng lên ảnh thứ nhất. Sau khi làm cong, phần sai lệch còn lại chính là phần chưa ước lượng đúng.
- **ma trận thông tin** — nghịch đảo của ma trận hiệp phương sai. Trong một bài toán bình phương tối thiểu có trọng số, nó chính là trọng số: một phép đo có ma trận thông tin lớn kéo nghiệm về phía nó mạnh hơn. Cộng thêm phép đo là cộng thêm thông tin, và tổng thông tin quyết định nghiệm.

- **sai lệch có hệ thống (bias) so với nhiễu ngẫu nhiên** — nhiễu ngẫu nhiên triệt tiêu khi cộng nhiều phép đo; sai lệch có hệ thống thì không, vì mọi phép đo lệch cùng một hướng. Trong chương này, sai số của mạng độ sâu gần với loại thứ hai hơn — đó là lý do nó nguy hiểm.

CÂU HỎI MỞ ĐẦU

- Q1. Hệ của bạn là stereo-inertial, đã có tỉ lệ mét thật từ đường cơ sở và gia tốc kế. Vậy một mạng độ sâu đơn ảnh — vốn không có đơn vị nào cả — còn thêm được *thông tin* gì mà bạn chưa có?
- Q2. RAFT bỏ kim tự tháp ảnh nhưng vẫn dùng một kim tự tháp. Nó dựng kim tự tháp trên cái gì, và vì sao chính chỗ đặt kim tự tháp quyết định một vật nhỏ bay nhanh có sống sót hay không?
- Q3. DUST3R nhận hai ảnh chưa hiệu chỉnh và trả về hình học ba chiều mà không hề chạy bước ước lượng tư thế. Tư thế biến đi đâu, hay chính xác hơn, nó nằm ở chỗ nào trong đầu ra?
- Q4. Bạn nhét 5000 pixel độ sâu học được vào cùng bài toán tối ưu với 20 điểm stereo đo thật. Vì sao mẹo “đặt trọng số nhỏ cho độ sâu học được” không cứu được bạn, và phép tính nào cho thấy điều đó trong ba mươi giây?
- Q5. Vì sao một mô hình độ sâu dựa trên khuếch tán lại phá hỏng đúng cái harness chạy lặp mười lần mà bạn đã mất nhiều công để làm cho tắt định?

30.1 Đặt vấn đề

Đọc nhanh. Nếu chỉ có mười phút: đọc Hình 30.1 và mục *Cổng đánh giá* ở cuối. Kết luận của cả chương là ba họ mô hình này chưa nên thay khối nào trong *frontend* của bạn, nhưng hai ý tưởng thiết kế trong đó thì đáng mang đi ngay: tách phần tính một lần khỏi phần lặp, và mô hình hoá sai số tương quan bằng một biến phụ thay vì hạ trọng số.

Chương 28 thay bộ phát hiện và bộ ghép cặp trong *frontend*. Chương này đi xa hơn một bước: nó hỏi xem có nên thay chính *phép đo* bằng một phỏng đoán học được hay không. Ba họ mô hình đang đứng xếp hàng ngoài cửa hệ thống của bạn.

- **Độ sâu từ một ảnh** — đưa vào một khung hình, trả về một giá trị độ sâu cho mọi pixel. Không cần cặp stereo, không cần thị sai, không cần chuyển động.
- **Dòng quang dày** — đưa vào hai khung hình, trả về một vector dịch chuyển cho mọi pixel. Không cần bộ phát hiện điểm đặc trưng.
- **Mô hình nền hình học** — đưa vào một nhóm ảnh chưa hiệu chỉnh, trả về thẳng hình học ba chiều và tư thế camera. Không cần cả pipeline hình học cổ điển.

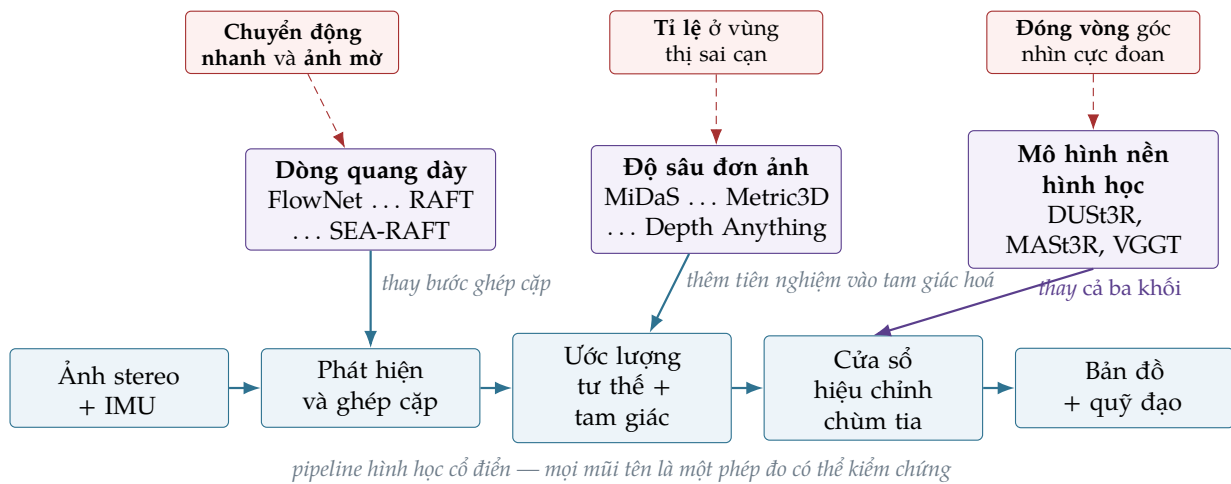
Ba lời chào hàng đó nghe rất khác nhau nhưng có chung một cấu trúc: chúng *đổi thưa lấy dày*. Hệ của bạn hiện sống bằng vài trăm phép đo mỗi khung hình, mỗi phép đo kèm một bài kiểm tra để loại điểm sai. Ba họ trên cho bạn hàng trăm nghìn giá trị mỗi khung hình, không kèm bài kiểm tra nào.

Và đây là chỗ phải nói thẳng ngay từ đầu, vì nó quyết định cách đọc cả chương. Bạn đang chạy một hệ *stereo-inertial*. Đường cơ sở stereo cho bạn tỉ lệ mét từ hình học; gia tốc kế cho bạn tỉ lệ mét từ vật lý. Phần lớn giá trị mà các bài báo độ sâu đơn ảnh quảng cáo — “lấy được hình học ba chiều từ một ảnh” — bạn đã có rồi, và có bằng phép đo chứ không bằng phỏng đoán. Vì vậy câu hỏi đúng không phải “mô hình nào chính xác nhất” mà là câu hỏi hẹp hơn nhiều: *ở đúng những pixel nào phép đo của tôi cạn kiệt, và ở đó một phỏng đoán học được có tốt hơn không có gì hay không*.

Học xong chương này bạn làm được gì mà trước đó không làm được? Bạn khoanh được bằng số vùng ảnh mà stereo của bạn đã hết thông tin. Bạn biết chi phí của từng họ mô hình *tỉ lệ với cái gì*, nên bạn loại được phương án sai cả bậc trước khi cài đặt, và bạn biết phép đo nào biến quan

hệ tỉ lệ ấy thành một con số dùng được cho phần cứng của mình. Và bạn tính được ngưỡng mà một tiên nghiệm dày bắt đầu lấn át phép đo thừa trong bộ tối ưu.

Hình 30.1 đặt ba họ mô hình cạnh nhau theo đúng chỗ chúng cắm vào một hệ SLAM, và theo nỗi đau mà mỗi họ hứa chữa.



Hình 30.1. Ba họ mô hình của chương này, đặt theo đúng khối mà chúng thay thế trong một hệ SLAM cổ điển, và theo nỗi đau mà mỗi họ hứa chữa (mũi tên đứt màu đỏ). Điểm cần chú ý là mức độ xâm lấn tăng dần từ trái sang phải: dòng quang chỉ thay một bước, độ sâu đơn ảnh chèn thêm một tiên nghiệm vào bước tam giác hoá, còn mô hình nền hình học nuốt luôn cả ba khối và trả về hình học đã dựng sẵn. Mức xâm lấn càng cao thì càng khó chẩn đoán khi hỏng, vì càng ít ranh giới để đặt điểm dừng.

TRỰC GIÁC

Hãy tìm một người bạn trong sân vận động đông người. Cách kim tự tháp ảnh: bạn lùi ra thật xa cho cả sân lọt vào tầm mắt, thấy một đám đông ở khán đài B, rồi tiến lại gần dần. Nếu bạn ấy đứng lẻ ở góc sân thì ở tầm nhìn xa bạn ấy chỉ là một chấm mờ, bạn đã đi nhầm về khán đài B, và mọi bước tiến lại gần chỉ tinh chỉnh quanh chỗ sai. Cách của RAFT khác hẳn: trước tiên lập một bảng ghi mức giống nhau giữa *mọi* cặp vị trí, rồi mỗi vòng lặp chỉ đọc vài ô quanh phỏng đoán hiện tại. Khi phỏng đoán còn xa đích, bạn đọc ở mức thô của *bảng* — nhưng cái bị làm thô là bảng, không phải sân, nên người đứng lẻ ở góc sân không hề biến mất. Toàn bộ chương này quay quanh đúng phân biệt đó: làm thô cái gì, và cái gì thì tuyệt đối không được làm thô.

30.2 Độ sâu từ một ảnh và vấn đề tỉ lệ (Monocular Depth and the Scale Problem)

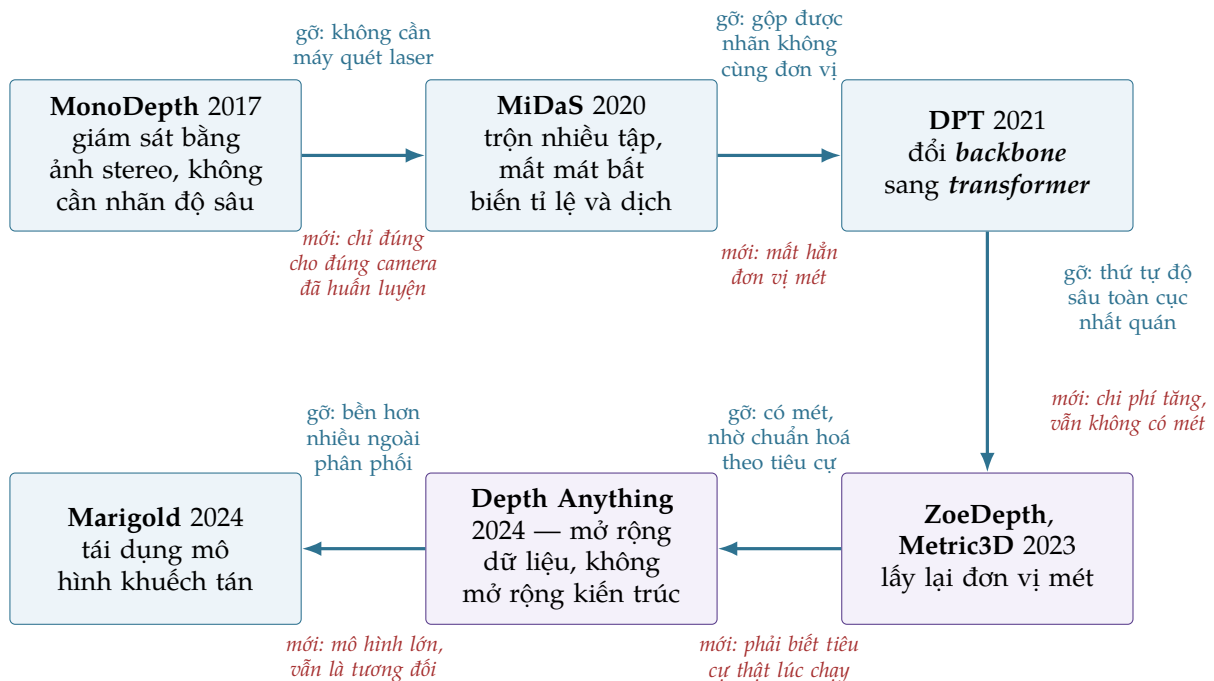
A. Bài toán gốc, và vì sao nó không phải bài toán khó mà là bài toán không giải được

Chương 16 đã chứng minh điều này bằng một hình vẽ, nên ở đây chỉ cần nhắc lại kết luận. Nhân toàn bộ cảnh với hệ số k và nhân luôn quãng đường camera với k thì mọi ảnh thu được giống hệt. Vì thế không lượng dữ liệu nào chữa được, chỉ **tiên nghiệm** (*prior*) mới chữa được.

↔ **tiền đề C/16/02** — tính không xác định được của tỉ lệ đã được dựng đầy đủ ở đó; chương này chỉ dùng lại kết luận và đi tiếp vào hệ quả kỹ thuật.

Điều đó đặt ra ngay ranh giới đọc cả mục này: mọi con số độ sâu mà một mạng đơn ảnh trả về đều là *suy đoán có học*, không phải phép đo. Câu hỏi kỹ thuật thật sự là suy đoán ấy dựa vào tiên nghiệm gì, và tiên nghiệm ấy vỡ ở đâu.

Sáu thể hệ trong dòng này có thể đọc như sáu lần trả lời cho cùng một câu: *lấy giám sát ở đâu ra*. Hình 30.2 xếp chúng trên một trục, mỗi mũi tên ghi nút thắt được gỡ và vấn đề mới sinh ra.



Hình 30.2. Sáu bước của dòng độ sâu đơn ảnh, đọc theo câu hỏi “lấy giám sát ở đâu”. Đọc hàng trên từ trái sang phải rồi vòng xuống hàng dưới đi ngược lại, theo chiều mũi tên. Nhãn xanh là nút thắt được gỡ, nhãn đỏ nghiêng là vấn đề mới sinh ra. Chú ý hai ô tô tím: chúng là hai cách trả lời khác nhau cho cùng câu hỏi đơn vị mét — ZoeDepth gắn thêm một đầu ra riêng cho từng miền, còn Metric3D chuẩn hoá đầu vào để mọi ảnh trông như chụp bằng một camera duy nhất.

B. Từ MonoDepth tới Depth Anything, đọc theo nguồn giám sát

MonoDepth [277] là bước đầu tiên đáng nhớ vì nó xoá bỏ nhu cầu có nhãn độ sâu. Mạng nhận ảnh trái và dự đoán một trường độ chênh lệch; ta dùng trường đó để làm cong ảnh phải về vị trí ảnh trái; sai lệch quang trắc giữa hai ảnh chính là **hàm mất mát** (*loss function*). Thêm một số hạng buộc độ chênh lệch dự đoán từ ảnh trái và từ ảnh phải phải nhất quán với nhau. Vì lúc huấn luyện đường cơ sở đã biết, đầu ra thực chất có đơn vị mét — nhưng chỉ đúng cho đúng cấu hình camera đó. Monodepth2 sau này sửa hai chỗ hỏng cụ thể: dùng cực tiểu thay vì trung bình của sai lệch quang trắc qua các khung nguồn để chịu được vùng bị che, và tự động che các pixel không đối giữa hai khung để loại vật thể chuyển động cùng tốc độ với camera [278].

Chỗ hỏng thứ hai đáng dừng lại một nhịp, vì nó là ví dụ đẹp về việc một hàm mất mát tự tạo ra một nghiệm sai. Xét một chiếc xe chạy phía trước, cùng vận tốc với camera. Giữa hai khung hình liên tiếp, mọi pixel của chiếc xe đó *không dịch chuyển*. Bây giờ hỏi: giá trị độ sâu nào làm cho phép làm cong không dịch chuyển pixel? Đáp án là độ sâu vô cực. Vậy mạng có thể đạt sai lệch quang trắc bằng đúng không trên cả chiếc xe, chỉ bằng cách khai báo nó ở vô cùng xa. Triệu chứng đặc trưng là những cái lỗ đen ngòm hình chiếc xe trong bản đồ độ sâu. Không có gì sai trong quá trình tối ưu; hàm mất mát đúng là được cực tiểu hoá. Cái sai nằm ở giả định cảnh tĩnh, và cách chữa của Monodepth2 — che các pixel không đối giữa hai khung — là ép giả định đó thành một điều kiện lọc chứ không phải một hy vọng.

MiDaS [31] giải một bài toán khác hẳn: làm sao huấn luyện trên mười tập dữ liệu mà nhãn của chúng không cùng đơn vị. Có tập là quét laser ra mét, có tập là ảnh 3D từ phim với đường cơ sở không ai biết, có tập là kết quả dựng lại từ ảnh nên chỉ đúng sai khác một hệ số. Lời giải là bỏ luôn tham vọng về đơn vị. Trước khi tính sai lệch, ta căn dự đoán về nhãn bằng một phép biến

đối bậc nhất tối ưu: tìm s và t làm cực tiểu $\sum_i (s \hat{d}_i + t - d_i)^2$, rồi mới đo phần dư. Mạng vì thế chỉ phải học *hình dạng* của trường độ sâu nghịch đảo, không phải giá trị tuyệt đối của nó. Kết quả là khả năng khái quát vượt trội, và một cái giá rất cụ thể mà mục sau sẽ khai thác kỹ.

DPT [279] giữ nguyên công thức dữ liệu và hàm mất mát của MiDaS, chỉ đổi *backbone* sang *transformer* với một bộ giải mã dày. Lý do nó ăn thua nằm ở **trường tiếp nhận** (*receptive field*): trong một mạng tích chập, tầng có tầm nhìn toàn ảnh cũng là tầng có độ phân giải thấp nhất, nên thông tin toàn cục và chi tiết biên không bao giờ gặp nhau ở cùng một tầng. **Attention** có tầm nhìn toàn ảnh ngay từ tầng đầu, nên mô hình giữ được thứ tự độ sâu nhất quán trên toàn khung hình mà không phải hy sinh biên. Với người làm SLAM, đây là điểm đáng chú ý riêng: *thứ tự độ sâu toàn cục* chính là thứ mà mọi bộ ước lượng cục bộ đều thiếu.

ZoeDepth [280] và **Metric3D** [281] đi lấy lại đơn vị mét bằng hai đường khác nhau. ZoeDepth giữ mạng độ sâu tương đối làm phần thân, rồi gắn thêm một đầu ra dự đoán các khoảng độ sâu có đơn vị, tinh chỉnh riêng cho từng miền, cộng một bộ định tuyến chọn đầu ra theo ảnh. Nó dùng được khi ảnh thử nghiệm rơi vào đúng một trong các miền đã huấn luyện. Metric3D làm điều thú vị hơn nhiều, và mục sau dành riêng cho nó.

Depth Anything [282] là một bài học về dữ liệu chứ không phải về kiến trúc. Một mạng thầy được huấn luyện trên phần dữ liệu có nhãn, rồi gán nhãn giả cho hàng chục triệu ảnh không nhãn; mạng trò học trên nhãn giả đó nhưng dưới nhiều loạn mạnh, cộng một số hạng buộc đặc trưng của nó bám theo một mô hình nền thị giác đã đóng băng. Ý tưởng cốt lõi: chỉ chép lại nhãn giả thì trò không giỏi hơn thầy, nên phải làm bài toán *khó hơn* cho trò. Bản v2 [283] đổi một chi tiết có sức nặng bất ngờ: thầy được huấn luyện hoàn toàn trên ảnh tổng hợp. Lý do rất rõ ràng khi nói ra — nhãn độ sâu từ cảm biến thật luôn có lỗi, có nhiễu, và sai bét ở vật trong suốt hoặc phản chiếu, còn nhãn tổng hợp thì hoàn hảo tuyệt đối. Khoảng cách miền giữa ảnh tổng hợp và ảnh thật sau đó được lấp bằng chính bước gán nhãn giả trên ảnh thật. Có một bản v3 xuất hiện trong giai đoạn 2025–2026, theo mô tả công khai thì nó mở rộng cùng công thức sang dự đoán hình học từ số lượng khung hình bất kỳ; tôi chưa tự kiểm chứng chi tiết, nên chỉ nhắc tên và ý tưởng.

Marigold [284] tái dụng một mô hình khuếch tán sinh ảnh đã huấn luyện sẵn: giữ nguyên bộ mã hoá tiềm ẩn, tinh chỉnh phần khử nhiễu để nó sinh ra *bản đồ độ sâu* với điều kiện là ảnh đầu vào. Điều đáng kinh ngạc là nó chỉ cần một lượng ảnh tổng hợp rất nhỏ và một ngân sách huấn luyện rất khiêm tốn so với việc huấn luyện từ đầu, vì tiên nghiệm về cấu trúc cảnh đã nằm sẵn trong mô hình sinh. Biên rất sắc, khái quát rất tốt. Nhưng với hệ của bạn thì hai tính chất của nó là án tử: suy luận cần nhiều bước khử nhiễu, và *nó ngẫu nhiên* — cùng một ảnh, hai lần chạy cho hai bản đồ độ sâu khác nhau, nên người ta phải chạy nhiều lần rồi lấy trung bình. Ta sẽ quay lại chỗ này ở mục nói về chẩn đoán.

Cuối cùng, **Align3R** là một hướng đáng theo dõi vì nó tấn công đúng điểm yếu chí mạng của cả họ này: sự không nhất quán giữa các khung hình. Ý tưởng là kết hợp một mô hình độ sâu đơn ảnh với một mô hình bản đồ điểm hai ảnh, rồi tối ưu chung để tìm tỉ lệ và độ dịch của từng khung sao cho các khung khớp nhau trong không gian ba chiều. Đây là công trình rất mới, tôi không nắm chắc chi tiết nên chỉ nêu ý tưởng và không mô tả kết quả.

Bảng 30.1. Sáu họ độ sâu đơn ảnh. Cột giá trị nhất là cột cuối: mỗi họ mượn tiên nghiệm từ một nguồn khác nhau, nên nó hỏng đúng khi nguồn đó cạn. Cột đơn vị cho biết đầu ra có đọc thẳng ra mét được hay không.

Họ	Đơn vị	Cơ chế và giả định	Hỏng khi nào
MonoDepth	mét, riêng cho một camera	Giám sát bằng sai lệch quang trắc giữa hai ảnh stereo; giả định cảnh tĩnh và độ sáng không đổi	Đổi camera hoặc đổi đường cơ sở; vật chuyển động cùng tốc độ camera; vùng bị che ở biên
MiDaS, DPT	không có	Mất mát bất biến tỉ lệ và dịch trên độ sâu nghịch đảo, trộn nhiều tập dữ liệu	Mọi chỗ cần đơn vị; và tỉ lệ với độ dịch <i>khác nhau ở mỗi khung hình</i>
ZoeDepth	mét, theo miền	Thân tương đối cộng một đầu ra có đơn vị, tinh chỉnh riêng từng miền, có bộ định tuyến	Ảnh rơi ra ngoài mọi miền đã huấn luyện; bộ định tuyến chọn nhầm miền
Metric3D	mét	Chuẩn hoá ảnh về một camera chuẩn theo tiêu cự, dự đoán, rồi chuyển ngược lại	Tiêu cự khai báo sai; ống kính méo mạnh hoặc mất cá, nơi mô hình lỗ kim không còn đúng
Depth Anything	tương đối (có bản có đơn vị)	Mở rộng dữ liệu bằng nhãn giả dưới nhiễu loạn mạnh; v2 dùng thầy huấn luyện trên ảnh tổng hợp	Cảnh xa mọi thứ trong tập ảnh không nhãn; mô hình lớn, chi phí cao
Marigold	không có	Tái dụng mô hình khuếch tán sinh ảnh làm bộ dự đoán độ sâu	Cần nhiều bước khử nhiễu; đầu ra <i>ngẫu nhiên</i> giữa các lần chạy

C. Bất biến tỉ lệ và dịch: hai ẩn tự do, và cái bẫy nằm ở ẩn thứ hai

Đây là chỗ phải viết đủ chứ không được rút gọn, vì rút gọn là chỗ mọi người vấp. Một mạng thuộc họ MiDaS trả về một đại lượng $\hat{d}$ liên hệ với độ sâu thật theo

$$\hat{d} \approx \frac{s}{Z} + t,$$

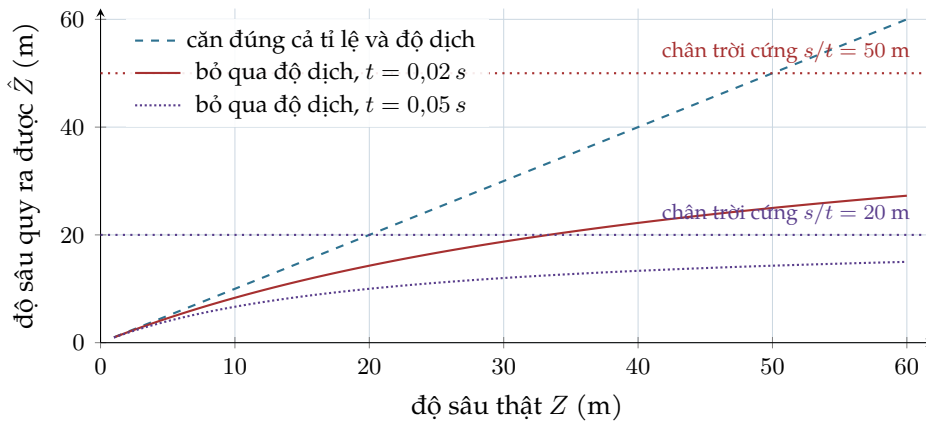
tức nó bất biến với một phép biến đổi bậc nhất trên *độ sâu nghịch đảo*. Nói bằng lời: mạng biết chắc điểm nào gần hơn điểm nào và gần hơn bao nhiêu lần, nhưng nó không biết gốc và không biết đơn vị.

Cách dùng ngây thơ nhất là chia một hằng số: lấy trung vị của $\hat{d}$ rồi co giãn cho khớp với trung vị độ sâu bạn đã biết. Làm thế là ngầm đặt $t = 0$. Hậu quả tính được ngay:

$$\hat{Z} = \frac{s}{\hat{d}} = \frac{s}{s/Z + t} = \frac{Z}{1 + tZ/s}.$$

Khi $Z \rightarrow \infty$, giá trị này không tiến ra vô cực mà bão hoà ở s/t . Mô hình của bạn vừa mọc ra một *chân trời cứng*: mọi thứ xa hơn s/t mét đều được báo về đúng khoảng đó.

Ví dụ nhỏ tính tay được. Giả sử sau khi căn tỉ lệ, độ dịch còn sót lại tương đương $t = 0,02 s$ trên thang độ sâu nghịch đảo, tức chân trời $s/t = 50$ m. Điểm ở 10 m được báo về $10/(1 + 0,2) = 8,3$ m, sai 17%. Điểm ở 20 m được báo về $20/(1,4) = 14,3$ m, sai 29%. Điểm ở 40 m được báo về 22 m, sai 45%. Sai số không ngẫu nhiên mà tăng đơn điệu theo khoảng cách — đúng dạng sai lệch có hệ thống, tức dạng nguy hiểm nhất cho một bộ tối ưu. Hình 30.3 vẽ chính hiện tượng đó.



Hình 30.3. Cái giá của việc bỏ quên ẩn thứ hai. Đầu ra của một mạng bất biến tỉ lệ và dịch cần *hai* tham số để quy về mét; cần bằng một hằng số nhân duy nhất tương đương đặt độ dịch bằng không, và điều đó tạo ra một chân trời cứng ở s/t mét. Đường liền đỏ và đường chấm tím là hai mức độ dịch còn sót khác nhau. Đây là hình minh hoạ công thức $\hat{Z} = Z/(1 + tZ/s)$, không phải số đo từ một mô hình cụ thể. Điểm cần nhớ: sai số tăng đơn điệu theo khoảng cách, nên nó là sai lệch có hệ thống chứ không phải nhiễu.

CẠM BẦY

Hai lỗi hay đi cùng nhau khi cắm một mô hình họ MiDaS vào pipeline. Thứ nhất là quên rằng đầu ra nằm trên thang *độ sâu nghịch đảo*, nên đem trung vị độ sâu ra co giãn thẳng là căn sai không gian, và sai số sinh ra sẽ tập trung hết vào vùng xa. Thứ hai là bỏ quên ẩn độ dịch, tạo ra chân trời cứng vừa nói ở trên. Nhưng lỗi thứ ba mới là lỗi giết hệ thống: s và t là của riêng từng khung hình. Bạn căn đúng cho khung k không có nghĩa là khung $k + 1$ dừng lại được cập số đó. Một bức tường đứng yên sẽ “thở” phập phồng qua các khung, và bộ tối ưu của bạn — vốn tin rằng cảnh cứng — sẽ hấp thụ nhịp thở đó vào tư thế và vào vận tốc.

D. Metric3D: xoá một tính không xác định được bằng cách chuẩn hoá đầu vào

Mục này đáng đọc kỹ ngay cả khi bạn không định dùng Metric3D, vì thiết kế của nó là một bài học dùng lại được.

Bắt đầu từ nguồn gốc của tính không xác định được, lần này ở dạng khác với tính không xác định được của tỉ lệ. Với một camera lỗ kim, tọa độ ảnh của một điểm là $u = fX/Z$. Nhân đồng thời f và Z với cùng hệ số k thì u không đổi. Nghĩa là: cùng một bức ảnh có thể là một phòng nhỏ chụp bằng ống kính góc rộng, hoặc một nhà kho lớn chụp bằng ống kính tele. Một mạng chỉ nhìn pixel không có cách nào phân biệt hai trường hợp đó. Đây không phải khiếm khuyết của mạng; đó là thiếu thông tin thật.

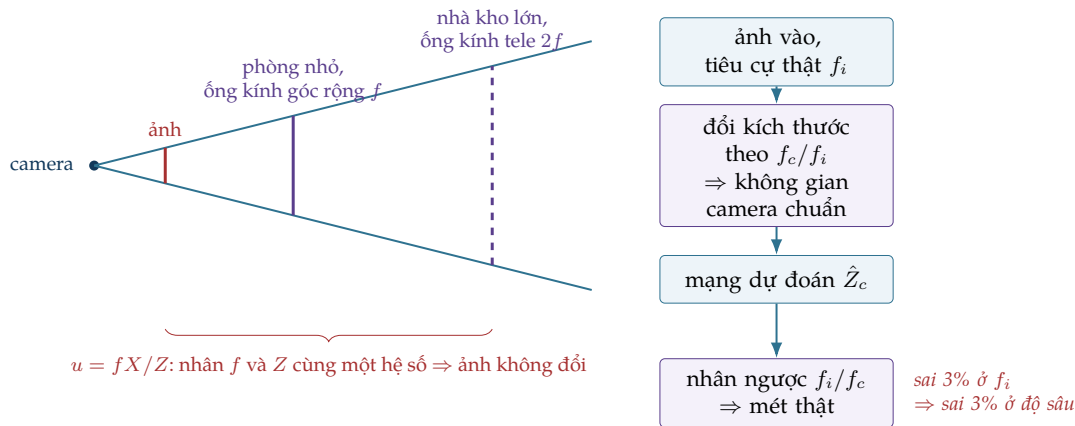
Điều đó giải thích một hiện tượng vốn khó hiểu khi huấn luyện: gộp nhiều tập dữ liệu chụp bằng nhiều camera khác nhau, có nhãn mét đầy đủ, mà mô hình vẫn học ra một thứ nhoè nhoẹt. Lý do là cùng một hình ảnh trong hai tập lại đi kèm hai nhãn mét khác nhau, chỉ vì hai camera có tiêu cự khác nhau. Mạng bị ép học một hàm không phải hàm.

Lời giải của Metric3D là gỡ ẩn số đó ra khỏi bài toán học, đưa nó về bước tiền xử lý. Chọn một tiêu cự chuẩn f_c . Với mỗi ảnh huấn luyện có tiêu cự thật f_i , hoặc thay đổi kích thước ảnh theo tỉ số f_c/f_i để tiêu cự hiệu dụng thành f_c , hoặc giữ ảnh và nhân nhãn độ sâu với f_c/f_i . Sau bước này, toàn bộ dữ liệu huấn luyện hành xử như thể chụp bằng một camera duy nhất, và ánh xạ từ ảnh sang mét trở lại là một hàm. Lúc chạy thật, ta đưa ảnh vào không gian chuẩn, dự đoán, rồi nhân ngược lại theo tiêu cự thật của camera mình.

Ý nghĩa bằng lời: đây chính là cố định gauge. Bạn đã làm đúng việc này mỗi lần cố định khung khoá đầu tiên trong một bài toán hiệu chỉnh chùm tia. Có một bậc tự do mà dữ liệu không nói gì về nó. Thay vì hy vọng bộ tối ưu tự xử lý, ta ghim nó lại bằng tay.

→ cùng nguyên lý với F/34 — đặt trọng số IMU cũng là bài toán “một đại lượng dữ liệu không nói gì tới”, và cách chữa cũng là ghim nó bằng một quy ước rõ ràng thay vì để bộ tối ưu đoán.

Cái giá thì rất thẳng thắn và rất dễ kiểm tra: đầu ra tuyến tính theo tiêu cự bạn khai báo. Nếu f_x hiệu chỉnh của bạn lệch 3%, độ sâu lệch đúng 3%, đều khắp, không có triệu chứng nào khác. Và trên ống kính mắt cá của một số cấu hình thực địa, “tiêu cự lỗ kim” thậm chí không phải một đại lượng xác định. Hình 30.4 vẽ lại toàn bộ lập luận.



Hình 30.4. Trái: vì sao độ sâu mét từ một ảnh là bài toán không giải được nếu không biết tiêu cự — hai cảnh khác nhau chụp bằng hai ống kính khác nhau cho đúng một ảnh. Phải: cách Metric3D gỡ nút. Ấn số bị đẩy ra khỏi bước học và bị ghim bằng một phép chuẩn hoá ở hai đầu, đúng kiểu cố định gauge. Hệ quả trực tiếp là độ chính xác của độ sâu bị chặn trên bởi độ chính xác của phép hiệu chỉnh tiêu cự.

E. Với một hệ stereo-inertial thì độ sâu học được thêm được gì

Bây giờ mới tới câu hỏi thật, và cách trả lời nó là bằng số chứ không bằng ý kiến.

Stereo cho độ sâu qua $Z = fb/d$, nên độ bất định của nó là $\Delta Z = Z^2 \Delta d / (fb)$. Đó là quan hệ bậc hai: sai số tăng theo bình phương khoảng cách. Lấy đúng con số của EuRoC làm ví dụ, với $f \approx 458$ pixel và $b \approx 0,11$ m, ta có $fb \approx 50,5$ đơn vị mét-pixel. Giả sử bộ ghép cặp của bạn đạt độ chính xác độ chênh lệch cỡ 0,2 pixel.

- Ở $Z = 5$ m: $\Delta Z = 25 \times 0,2/50,5 \approx 0,10$ m, tức 2%.
- Ở $Z = 15$ m: $\Delta Z = 225 \times 0,2/50,5 \approx 0,89$ m, tức 6%.
- Ở $Z = 30$ m: $\Delta Z = 900 \times 0,2/50,5 \approx 3,6$ m, tức 12%.

Ở khoảng 50 m, độ chênh lệch tụt xuống dưới một pixel và stereo của bạn về bản chất đã thành đơn mắt.

Điểm cần thấy không phải một con số mà là *hai hình dạng đường cong khác nhau*. Sai số stereo tăng theo Z^2 ; sai số tương đối của một mô hình độ sâu đơn ảnh, chạy trong miền nó đã học, về cơ bản *không* tăng theo khoảng cách, vì nó không dựa vào thị sai chút nào. Một đường cong tăng bậc hai và một đường cong gần như phẳng thì bắt buộc cắt nhau ở đâu đó. Xa hơn điểm cắt ấy, phỏng đoán học được đã tốt hơn phép đo của bạn; gần hơn, phép đo thắng.

Chương này cố ý *không* nói điểm cắt đó nằm ở bao nhiêu mét, vì một con số như thế đòi hai thứ mà không tài liệu nào cho bạn được: độ chính xác độ chênh lệch thật của bộ ghép cặp của bạn, và sai số thật của mô hình trên đúng loại cảnh của bạn. Cả hai đều đo được, và đo chúng chính là

nội dung mini project của chương. Nếu điểm cắt rơi ra ngoài dải khoảng cách mà cảnh của bạn có, thì cả hướng này đóng lại và bạn tiết kiệm được nhiều tuần.

Ba chỗ còn lại mà phép đo của bạn thật sự cạn:

- **Vùng ít vân** — tường trắng, hành lang, mặt sàn nhẵn. Không có vân thì không có ghép cặp, nên không có độ chênh lệch, nên không có độ sâu ở bất kỳ khoảng cách nào.
- **Khởi tạo khi thị sai chưa đủ** — lúc đứng yên hoặc lúc chỉ xoay tại chỗ. Với hệ stereo thì đường cơ sở cứu bạn, nên chỗ này ít quan trọng hơn nhiều so với hệ đơn mắt. Đây là lý do phải nói thẳng: rất nhiều lợi ích được quảng cáo trong các bài báo độ sâu học được là lợi ích của hệ đơn mắt, và chúng biến mất khi bạn có stereo.
- **Lắp lỗ cho bản đồ dày** — nếu bạn cần một bản đồ chiếm dụng để điều hướng chứ không phải để định vị. Đây là cách dùng an toàn nhất, vì nó nằm *hạ nguồn* của đồ thị tư thế và không phản hồi ngược vào nó.

F. Ba cách lấy tỉ lệ mét, và cách nào hợp với bạn

Nếu quyết định dùng một mô hình tương đối, bạn phải chọn một trong ba cách đưa nó về mét. Chúng không tương đương, và cách thứ ba là cách duy nhất bạn nên cân nhắc.

- **Căn theo chính phép đo stereo của bạn.** Ở mỗi khung khoá, lấy các điểm bản đồ đã có độ sâu đo được, khớp một phép biến đổi bậc nhất giữa dự đoán và phép đo trên tập điểm đó, rồi áp phép biến đổi ấy cho toàn ảnh. Ưu điểm: dùng đúng dữ liệu bạn tin. Nhược điểm nặng: cặp tham số được ước lượng từ vùng *gần*, nơi bạn có phép đo, rồi được ngoại suy sang vùng *xa*, nơi bạn cần nó. Ngoại suy một phép biến đổi bậc nhất ra ngoài miền khớp là đúng công thức để sai có hệ thống.
- **Căn theo IMU.** Dùng tích phân trước của gia tốc kế để cố định tỉ lệ trên một đoạn quỹ đạo, rồi buộc độ sâu theo đó. Vấn đề: gia tốc kế cố định tỉ lệ của *chuyển động*, không phải của cấu trúc, và nó chỉ làm được điều đó khi có gia tốc đủ kích thích. Trong một đoạn đi thẳng đều, nó không nói gì cả.
- **Dùng mô hình vốn đã ra mét.** Metric3D và các đầu ra có đơn vị của ZoeDepth hoặc Depth Anything không cần bước căn nào, nên không có chỗ nào để ngoại suy sai. Đổi lại bạn nhận toàn bộ rủi ro lệch miền và rủi ro sai tiêu cự.

Nhận xét thực dụng: hai cách đầu *tạo ra* một tham số mới ước lượng ở mỗi khung, tức tạo ra đúng cái nhíp thử mà mục sau nói là nguy hiểm. Cách thứ ba không tạo thêm tham số nào, nên nếu bạn buộc phải dùng độ sâu học được trong bộ tối ưu, hãy dùng mô hình ra mét và kiểm tra kỹ tiêu cự. Nếu chỉ dùng ngoài bộ tối ưu, cách nào cũng được.

G. Đo độ sâu học được cho đúng mục đích của bạn

Bảng xếp hạng của các *benchmark* độ sâu dùng một bộ thước quen thuộc: sai số tương đối tuyệt đối trung bình trên mọi pixel, và tỉ lệ pixel có sai số dưới một ngưỡng. Cả hai đều là trung bình trên toàn ảnh, và cả hai đều *giấu đi* đúng hai thứ quyết định với bạn.

Thứ nhất, chúng trộn lẫn mọi khoảng cách. Nhưng bạn không cần độ sâu ở mọi khoảng cách; bạn chỉ cần nó ở dải mà stereo đã cạn. Một mô hình đứng đầu bảng nhờ chính xác ở ba mét đầu có thể hoàn toàn vô dụng ở ba mươi mét. Phép đo đúng là vẽ sai số tương đối *theo khoảng cách*, rồi đọc riêng phần đuôi xa.

Thứ hai, và nặng hơn nhiều, chúng không nói gì về *cấu trúc* của sai số. Sai số trung bình 6% có thể là 6% nhiễu độc lập từng pixel, hoặc là 6% lệch chung cho cả mảng tường. Với bộ tối ưu của bạn, hai trường hợp đó cách nhau ba bậc độ lớn về mức nguy hiểm, và mục sau sẽ tính ra con số ấy. Không bảng xếp hạng nào phân biệt chúng.

Cách đo đúng thì đơn giản và bạn tự làm được. Với mỗi khung hình có chân trị độ sâu, tách sai số làm hai phần:

- **Phần chung của khung** — một cặp tỉ lệ và độ dịch tối ưu cho riêng khung đó, tìm bằng bình phương tối thiểu. Đây là phần mà một biến phụ trong bộ tối ưu có thể hấp thụ, nên nó *không* nguy hiểm nếu bạn mô hình hoá nó.
- **Phần dư sau khi trừ đi phần chung** — đây mới là thông tin thật mà mô hình đóng góp, và nó là thứ duy nhất đáng đem so giữa hai mô hình.

Rồi làm thêm một việc nữa: vẽ *phần chung* theo thời gian trên cả chuỗi. Nếu nó dao động ở tần số khung hình thì bạn đã nhìn thấy trước, bằng biểu đồ, đúng cái nhịp thở sẽ làm hỏng ước lượng vận tốc của mình — và bạn thấy nó trước khi viết một dòng mã tích hợp nào.

→ cùng nguyên lý với E/24 — đây là phân rã lỗi theo loại, áp cho một mô-đun mới: tách phần sửa được bằng một tham số khỏi phần thật sự là sai số.

30.3 Dòng quang dày: từ FlowNet tới SEA-RAFT (Dense Optical Flow)

A. Vì sao ghép cặp thưa chết mà dòng quang sống

Chương 15 đã dựng bài toán khâu độ và Lucas–Kanade, nên ở đây ta đi thẳng vào chỗ liên quan tới nỗi đau của bạn: hai khung hình liên tiếp trong một pha xoay nhanh, ảnh bị mờ, và bộ bám đặc trưng mất dấu.

Cơ chế của cái chết đó đáng viết ra rõ ràng, vì nó giải thích luôn vì sao dòng quang lại sống sót. **Ảnh mờ do chuyển động** (*motion blur*) là một phép tích chập với một nhân dạng đoạn thẳng, dài

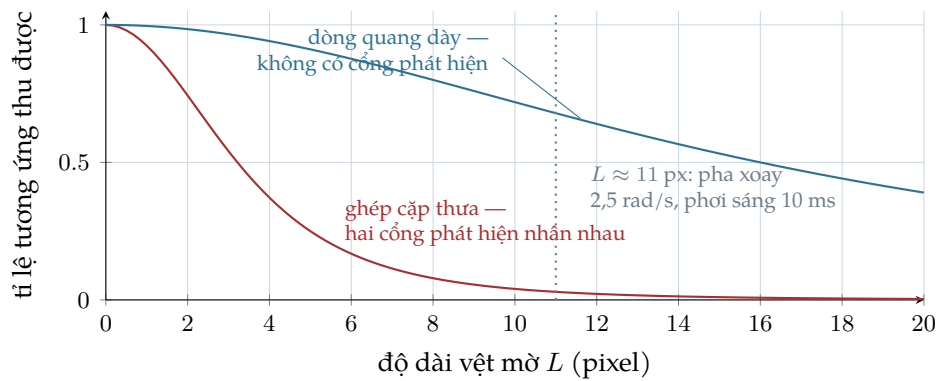
$$L \approx f \omega t_{\text{phơi sáng}} \quad \text{pixel},$$

với ω là tốc độ góc. Lấy số của EuRoC: $f \approx 458$ pixel, một pha xoay nhanh cỡ $\omega = 2,5$ rad/s, thời gian phơi sáng 10 ms trong nhà thiếu sáng. Ra $L \approx 11$ pixel. Đó là một vệt dài hơn cả bán kính của một mảnh mô tả ORB.

Bây giờ tới phần quan trọng. Bộ phát hiện góc đo hai giá trị riêng của ma trận mô men bậc hai của gradient trong một cửa sổ. Làm mờ theo một hướng là lọc thông thấp theo hướng đó: đỉnh gradient tụt đi khoảng L lần và trải ra trên L pixel, nên tổng bình phương gradient theo hướng ấy giảm cỡ L lần. Với $L \approx 11$, giá trị riêng nhỏ nhất mất khoảng một bậc độ lớn. Ngưỡng phát hiện là ngưỡng cứng, nên điểm đơn giản là *không được phát hiện nữa*. Đây là lập luận bậc độ lớn từ cơ chế, không phải số đo.

Và ghép cặp thưa là một chuỗi cổng nhân với nhau. Điểm phải được phát hiện ở khung này và ở khung kia, mô tả phải nằm trong ngưỡng, phải qua kiểm tra tỉ số, rồi phải sống sót qua RANSAC. Nếu tính lặp lại của bộ phát hiện tụt từ 0,8 xuống 0,3 thì xác suất một điểm sống qua cả hai khung tụt từ 0,64 xuống 0,09 — giảm bảy lần chỉ vì một cổng. Đó là lý do số điểm khớp sụp đổ chứ không giảm từ từ.

Dòng quang dày không có cổng nào cả. Nó không phát hiện gì hết; nó hỏi “pixel này đi đâu” cho mọi pixel, và nó trả lời bằng cách tổng hợp bằng chứng trên một vùng lớn. Làm mờ phá tần số cao nhưng giữ nguyên cấu trúc tần số thấp, mà tương quan ở mức thô lại đúng là thứ đọc cấu trúc tần số thấp. Thêm nữa, tính trơn học được của mạng lan nghiệm từ vùng có vân sang vùng mờ bên cạnh. Hình 30.5 vẽ hai đường cong tương ứng với hai cơ chế đó.



Hình 30.5. Vì sao ghép cặp thưa sụp đổ còn dòng quang chỉ xuống dốc. Hai đường này là *hình dạng của cơ chế*, không phải số đo: đường đỏ là tích của hai xác suất phát hiện nên nó bình phương mọi suy giảm, còn đường xanh không có cổng phát hiện nào để nhân. Kết luận dùng được: khi vật mờ vượt cỡ bán kính mảnh mô tả, khoảng cách giữa hai đường mở ra nhanh, và đó chính là dải mà một bộ dòng quang mới có lại.

B. Bốn thể hệ dòng quang học được

I Mạch 1 — dòng quang học được, từ FlowNet tới SEA-RAFT

- 1 Vấn đề** Phương pháp biến phân cổ điển chính xác nhưng chậm, và mỗi cảnh phải chỉnh lại tham số bằng tay.

Giải pháp FlowNet đưa cả bài toán vào một mạng tích chập huấn luyện đầu-cuối [165]. Có hai biến thể đáng nhớ vì chúng thể hiện hai triết lý: bản đơn giản xếp chồng hai ảnh thành sáu kênh rồi để mạng tự xoay xở, còn bản có tương quan cài sẵn một tầng tương quan giữa hai luồng đặc trưng.

Giả định cảnh trong dữ liệu tổng hợp dùng để huấn luyện đủ giống cảnh thật.

Vấn đề mới bản có tương quan thẳng rõ, cho thấy để mạng “tự học hình học” là lãng phí; nhưng độ chính xác vẫn thua các phương pháp cổ điển tốt.
- 2 Vấn đề** Một mạng đơn không đủ chính xác, và chuyển động nhỏ bị bỏ sót.

Giải pháp FlowNet 2.0 xếp chồng nhiều FlowNet nối tiếp, mỗi tầng *làm cong* ảnh thứ hai theo nghiệm của tầng trước rồi chỉ ước lượng phần dư; thêm một nhánh nhỏ chuyên cho chuyển động nhỏ và một khối hợp nhất ở cuối [285].

Hệ quả bài học lớn nhất của công trình này không phải kiến trúc mà là *lịch trình dữ liệu*: huấn luyện trên tập đơn giản trước rồi mới sang tập phức tạp cho kết quả khác hẳn so với trộn lẫn.

Giá phải trả mô hình rất lớn, và cấu trúc xếp chồng khiến mỗi tầng chỉ tinh chỉnh quanh nghiệm của tầng trước.
- 3 Vấn đề** Mạng quá to, và ba nguyên lý cổ điển đang bị học lại một cách lãng phí.

Giải pháp PWC-Net viết thẳng ba nguyên lý đó thành ba khối kiến trúc [166]: kim tự tháp đặc trưng học được, làm cong đặc trưng của ảnh hai theo nghiệm của mức thô hơn, và khối chi phí cục bộ với bán kính tìm kiếm nhỏ ở mỗi mức.

Hệ quả nhỏ hơn FlowNet 2.0 hơn một bậc mà chính xác hơn. Đây là ví dụ sạch nhất trong cả chương về giá trị của thiên lệch quy nạp đúng.

Giá phải trả thừa hưởng nguyên vẹn chế độ hồng của chiến lược thô tới mịn: vật nhỏ bay nhanh biến mất ở mức thô, và khối chi phí cục bộ không đủ tầm để sửa lại.

- 4 Vấn đề** Sai ở mức thô là sai vĩnh viễn, vì mọi mức mịn hơn chỉ tinh chỉnh chứ không tìm lại.
- Giải pháp** RAFT tách hẳn hai việc [167]: tính *một lần* khối tương quan giữa mọi cặp vị trí, rồi lặp lại một phép tra cứu cục bộ quanh phỏng đoán hiện tại, với phép cập nhật do một mạng hồi tiếp đảm nhiệm.
- Giả định** khối tương quan toàn cặp vừa bộ nhớ.
- Hệ quả** chuyển động lớn không còn cần thu nhỏ ảnh, nên vật nhỏ bay nhanh không biến mất. Số vòng lặp trở thành một núm vận độ trễ chỉnh được lúc chạy.
- Giá phải trả** bộ nhớ của khối tương quan, và một mô hình mà bạn không ablate được từng phần.
- 5 Vấn đề** Phép tra cứu của RAFT chỉ nhìn cục bộ, nên vùng bị che và vùng không vãn phải chờ thông tin lan qua nhiều vòng lặp.
- Giải pháp** FlowFormer mã hoá chính khối chi phí bằng một *transformer*, để thông tin đi khắp khối trong một lượt thay vì bò dần qua các vòng lặp [286].
- Giá phải trả** bộ nhớ và độ trễ tăng mạnh; đây là hướng cho máy để bàn, không cho hệ nhúng.
- 6 Vấn đề** RAFT cần nhiều vòng lặp vì nó khởi động từ dòng bằng không, và nó không nói được nó tin vào pixel nào.
- Giải pháp** SEA-RAFT đổi ba thứ [287]: hồi quy thẳng một nghiệm khởi đầu thay vì bắt đầu từ không, đổi hàm mất mát sang hỗn hợp hai phân phối Laplace, và đổi dữ liệu tiền huấn luyện.
- Hệ quả** ít vòng lặp hơn nhiều nên nhanh hơn hẳn; và hàm mất mát hỗn hợp cho ra một *độ bất định theo từng pixel* dùng được làm trọng số.
- Giả định** hai thành phần Laplace đủ để mô tả tính đa phương thức của dòng ở biên vật thể.

RAFT-3D là một nhánh đáng biết vì nó đổi đầu ra chứ không đổi thuật toán [288]. Thay vì một vector dịch chuyển hai chiều cho mỗi pixel, nó dự đoán một phép biến đổi cứng ba chiều cho mỗi pixel, rồi ép các pixel có biểu diễn giống nhau phải chia sẻ cùng một chuyển động cứng. Hệ quả trực tiếp và rất hữu ích: các mảng cứng của cảnh tự tách ra thành cụm. Nói cách khác, bài toán phân tách vật thể động rơi ra như một sản phẩm phụ của biểu diễn, không cần một bộ phân vùng ngữ nghĩa riêng.

↔ **đối lập với F/33** — đó là hai cách khác nhau để loại vật động: RAFT-3D dùng tính không nhất quán của chuyển động, còn mặt nạ ngữ nghĩa dùng tên lớp của vật.

C. Vòng lặp tra cứu của RAFT, giải thích kỹ

Đây là thiết kế đáng học nhất trong cả mục, và nó đáng học ngay cả khi bạn không bao giờ chạy RAFT.

Ba thành phần, theo đúng thứ tự dữ liệu chảy qua:

- Mã hoá đặc trưng.** Cả hai ảnh được đưa qua cùng một bộ mã hoá xuống độ phân giải $1/8$. Một bộ mã hoá thứ hai, chỉ chạy trên ảnh một, cho ra đặc trưng ngữ cảnh nuôi vào phép cập nhật.
- Khối tương quan toàn cặp.** Với mọi vị trí i trên lưới ảnh một và mọi vị trí j trên lưới ảnh hai, tính tích vô hướng của hai vector đặc trưng. Với ảnh EuRoC 752×480 , lưới $1/8$ là $94 \times 60 = 5640$ vị trí, nên khối có 5640^2 phần tử — *bình phương số ô lưới*, và đó là con số đáng nhớ vì nó nói rằng độ phân giải lưới là núm vận chi phí mạnh nhất. Rồi *gộp trung bình theo hai chiều cuối* với các hệ số 1, 2, 4, 8 để được bốn mức. Đây là câu quan trọng nhất của cả mục: cái bị làm thô là

hai chiều của ảnh hai trong khối tương quan, không phải lưới của ảnh một. Lưới ảnh một giữ nguyên độ phân giải ở mọi mức, nên một vật nhỏ ở ảnh một không bao giờ biến mất.

3. **Vòng lặp tra cứu và cập nhật.** Trường dòng khởi đầu bằng không. Ở mỗi vòng, với mỗi vị trí x của ảnh một, ta tính vị trí ứng viên $x' = x + f_k(x)$ rồi đọc các giá trị tương quan trong một lân cận bán kính 4 quanh x' , ở cả bốn mức. Mỗi vòng vì thế lấy ra $4 \times (2 \cdot 4 + 1)^2 = 324$ con số cho mỗi pixel. Ba thứ — các con số tra cứu được, dòng hiện tại, và đặc trưng ngữ cảnh — được đưa vào một khối hồi tiếp tích chập, khối này trả về Δf , và ta cộng vào. Lặp lại, dùng chung trọng số ở mọi vòng.

Ý nghĩa của thiết kế này bằng lời: nó tách bạch “chỗ nào giống chỗ nào” — tính một lần, đắt, nhưng không phụ thuộc nghiệm — khỏi “tôi đang đoán ở đâu” — rẻ, lặp lại, phụ thuộc nghiệm. Phép tra cứu ở mức thô thứ tư với bán kính 4 phủ được $\pm 4 \times 8 = \pm 32$ ô của lưới 1/8, tức ± 256 pixel trong ảnh gốc. Chuyển động lớn được xử lý mà không hề thu nhỏ ảnh một lần nào.

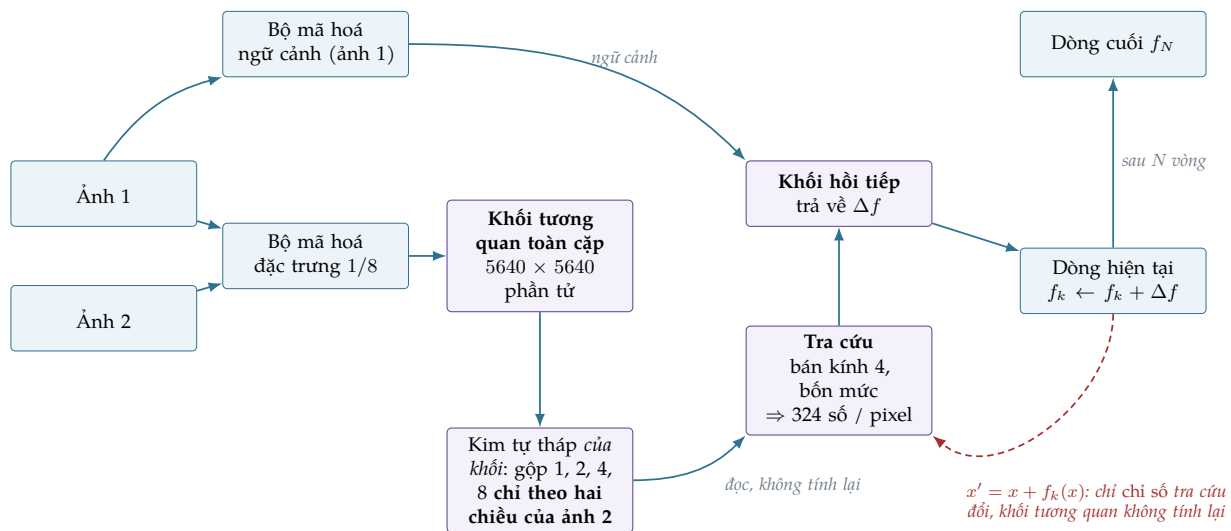
Bảng 30.2. Độ phủ của một phép tra cứu bán kính 4 ở bốn mức của kim tự tháp khối tương quan, quy về pixel của ảnh gốc. Cột cuối cho thấy vì sao không cần thu nhỏ ảnh: mức thô nhất đã phủ quá đủ cho mọi chuyển động thực tế giữa hai khung hình liên tiếp, trong khi lưới của ảnh một vẫn giữ nguyên độ phân giải ở cả bốn mức.

Mức	Hệ số gộp	Ô lưới 1/8	Pixel ảnh gốc	Vai trò
0	1	± 4	± 32	Tinh chỉnh dưới pixel ở các vòng cuối
1	2	± 8	± 64	Chuyển động vừa
2	4	± 16	± 128	Chuyển động nhanh
3	8	± 32	± 256	Xoay mạnh, mất khung, tìm lại từ xa

Với người đã viết bộ tối ưu phi tuyến, khối hồi tiếp kia chính là một *bước tối ưu học được*: nó nhận phần dư quanh nghiệm hiện tại và trả về một bước đi, đúng vai trò của một bước Gauss–Newton, chỉ khác là hướng đi được học chứ không được suy ra từ Jacobi. Và như mọi bộ giải lặp, số vòng lặp là ngân sách bạn tự đặt lúc chạy — huấn luyện với 12 vòng, chạy với 32 vòng nếu rảnh, hoặc 6 vòng nếu gấp.

↔ **trường hợp riêng của C/15/03** — cùng một trực giác về bộ giải lặp có ngân sách, chỉ khác là ở đó số vòng lặp do bạn đặt còn ở đây nó là siêu tham số lúc suy luận.

Hình 30.6 là sơ đồ của đúng vòng lặp đó.



Hình 30.6. Vòng lặp tra cứu của RAFT. Phần bên trái chạy đúng một lần và không phụ thuộc nghiệm; phần bên phải chạy lặp và rẻ, chỉ đổi *chỉ số* tra cứu chứ không tính lại tương quan. Chi tiết quyết định nằm ở ô kim tự tháp: việc gộp trung bình chỉ áp lên hai chiều của ảnh hai, nên lưới của ảnh một giữ nguyên độ phân giải ở mọi mức. Đó là lý do RAFT xử lý được chuyển động lớn mà không mắc bẫy “vật nhỏ biến mất ở mức thô” của chiến lược thô tới mịn. Số vòng lặp là siêu tham số lúc suy luận, nên nó là núm vặn độ trễ chỉnh được ngay khi chạy.

E. Ba cách cắm dòng quang vào hệ của bạn, xếp theo mức xâm lấn

Trước khi tính giá, cần thấy rằng “dùng dòng quang” không phải một lựa chọn mà là ba lựa chọn rất khác nhau. Xếp theo mức xâm lấn tăng dần:

1. **Dùng làm bộ dự đoán, giữ nguyên ghép cặp thừa.** Chạy dòng quang, lấy vector dịch chuyển tại vị trí các điểm bản đồ đang bám, rồi dùng nó để đặt tâm cửa sổ tìm kiếm cho bộ ghép cặp thừa vốn có. Mọi bài kiểm tra cũ vẫn chạy: kiểm tra tỉ số, kiểm tra sai số chiều lại, RANSAC. Bạn chỉ đổi *chỗ nhìn*, không đổi *cái tin*.
2. **Dùng làm nguồn tương ứng, bỏ bộ phát hiện.** Lấy thẳng trường dòng làm tập tương ứng, lấy mẫu thừa ra vài trăm điểm theo một tiêu chí nào đó, rồi đưa vào bộ giải. Bạn mất bài kiểm tra tỉ số vì không còn hai ứng viên để so, và bạn phải tự dựng lại một cơ chế loại điểm sai.
3. **Dùng làm phép đo dày, đưa thẳng vào bộ tối ưu.** Đây là con đường của các hệ đầu-cuối ở chương 31. Bạn được nhiều ràng buộc nhất và mất nhiều khả năng chẩn đoán nhất.

Với một hệ đã tinh chỉnh kỹ, lựa chọn thứ nhất gần như luôn là lựa chọn đúng để thử trước, vì lý do rất thực dụng: nó *có thể tắt được*. Nếu dòng quang cho dự đoán tồi, cửa sổ tìm kiếm đặt sai chỗ, số điểm khớp giảm, và bạn thấy ngay bằng đúng các bộ đếm đang có. Không có gì âm thầm cả. Lựa chọn thứ hai và thứ ba đều xoá mất tín hiệu “không còn điểm nội biên nào” — tín hiệu duy nhất mà logic phục hồi của bạn đang dựa vào.

Có một chi tiết dễ bỏ sót ở lựa chọn thứ nhất và nó quyết định lời lỗ. Hệ của bạn đã có một bộ dự đoán rồi: mô hình chuyển động hằng vận tốc, hoặc tốt hơn nữa là tích phân trước của IMU. Trong pha xoay nhanh, IMU chính là thứ dự đoán tốt nhất, vì con quay hồi chuyển đo trực tiếp cái gây ra dịch chuyển ảnh. Vậy dòng quang chỉ có lãi ở đúng những khung hình mà dự đoán từ IMU cũng hỏng: khi có tịnh tiến nhanh ở cự ly gần, khi mất đồng bộ thời gian, hoặc khi độ lệch con quay đang trôi. Đó là một tập khung hình hẹp hơn nhiều so với “mọi khung bị mờ”, và đo được nó chính là nội dung mini project thứ hai của chương.

↔ **đối lập với F/34** — IMU và dòng quang là hai bộ dự đoán cạnh tranh cho cùng một cửa sổ tìm kiếm; chọn cái nào là câu hỏi về việc cái nào hỏng trước.

F. Cái giá, nói bằng số

Bốn khoản, xếp theo mức độ dễ bị bỏ quên.

Bộ nhớ và độ trễ. Điều phải nắm là *cấu trúc* của chi phí, vì nó khác hẳn cấu trúc chi phí của một mạng thông thường. Khối tương quan toàn cặp có số phần tử bằng bình phương số ô lưới, và nó phải nằm nguyên trong bộ nhớ suốt các vòng lặp tra cứu. Hai hệ quả đi kèm. Thứ nhất, chi phí bộ nhớ tăng theo *bình phương* độ phân giải lưới, nên hạ độ phân giải là núm vặn mạnh nhất bạn có. Thứ hai, sau khi khối đã dựng xong thì mỗi vòng lặp chỉ là đọc bộ nhớ, nên bài toán chuyển từ bị chặn bởi phép tính sang bị chặn bởi băng thông. Con số mili-giây và megabyte cụ thể thì phải đo trên chính phần cứng đích, sau khi đã biên dịch bằng trình suy luận của nó — và mini project thứ hai của chương bắt bạn đo đúng con số ấy thay vì ước lượng nó.

↪ **ràng buộc bởi B/11** — bài toán ở đây bị chặn bởi bộ nhớ và băng thông chứ không phải bởi số phép tính, đúng như phân tích roofline ở chương giới hạn tính toán.

Không có tập điểm nội biên. Đây là khoản đắt nhất mà ít ai nói. Pipeline thừa của bạn trả về một tập điểm khớp *kèm nhãn đúng hay sai*, và khi mọi thứ hỏng thì nó trả về con số không — một tín hiệu rõ ràng để kích hoạt logic phục hồi. Dòng quang luôn trả lời. Ở vùng bị che, ở vùng ra khỏi khung, ở mặt gương, nó vẫn cho ra một vector trông rất thuyết phục. Hệ cổ điển hỏng âm ỉ; hệ học được hỏng lạng lẽ.

Nó chỉ thay bước ghép cặp. Dòng quang cho tương ứng hai chiều. Bạn vẫn cần độ sâu và vẫn cần một bộ giải để ra tư thế. Nó không rút ngắn pipeline, nó chỉ đổi một khối.

Chẩn đoán và tính lặp lại. Bạn vừa đổi một khối có thể ablate từng phần lấy một khối chỉ ablate được nguyên cục. Bù lại, đây đúng là chỗ SEA-RAFT có giá trị đặc biệt với bạn: hàm mất mát hỗn hợp Laplace cho ra một độ bất định theo từng pixel, và độ bất định đó chuyển thẳng thành trọng số trong bài toán bình phương tối thiểu. Bài học thiết kế rút ra được, dùng lại được ở chỗ khác: *muốn có trọng số dùng được thì phải huấn luyện cho nó*. Một hàm mất mát đơn phương thức như L_1 không có cách nào diễn đạt “pixel này nằm trên biên che khuất nên câu trả lời có hai đáp án”.

↪ **tiền đề F/34** — đó chính là cầu nối sang bài toán hiệp phương sai học được: trọng số trong đồ thị nhân tử phải đến từ một mô hình có khả năng biểu diễn tính đa phương thức.

30.4 Mô hình nền hình học: DUST3R, MAST3R và VGGT (Geometric Foundation Models)

A. Ý tưởng: đối biểu diễn đầu ra để xoá cả một chuỗi bước

Đây là hướng mới nhất và đáng chú ý nhất trong cả chương, và lý do đáng chú ý không nằm ở độ chính xác mà ở chỗ nó *xoá bỏ* cái gì.

Pipeline hình học cổ điển là một chuỗi dài: phát hiện điểm, mô tả, ghép cặp, ước lượng ma trận cơ bản, phân rã ra tư thế, tam giác hoá, rồi hiệu chỉnh chùm tia. Mỗi bước có giả định riêng, và mỗi bước hỏng theo kiểu riêng. Ước lượng tư thế đặc biệt mong manh: nó cần đủ điểm khớp đúng, nó suy biến khi các điểm gần đồng phẳng hoặc khi chuyển động thuần xoay, và nó luôn cần biết tiêu cự.

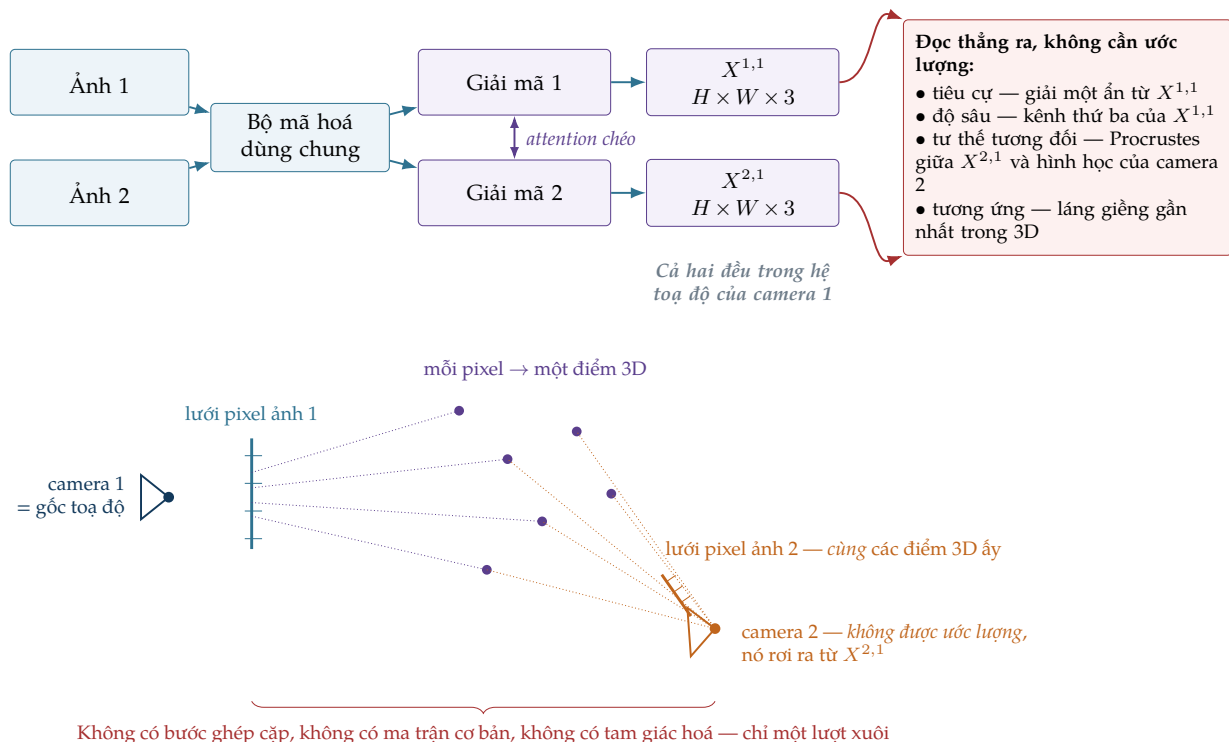
DUST3R [34] đặt câu hỏi ngược: nếu đầu ra của mạng không phải độ sâu mà là *bản đồ điểm* thì cần bao nhiêu bước trong chuỗi trên. Cụ thể, đưa vào hai ảnh chưa hiệu chỉnh, mạng trả về hai tensor $H \times W \times 3$: mỗi pixel của ảnh một được gán một điểm ba chiều, và mỗi pixel của ảnh hai cũng được gán một điểm ba chiều — nhưng *cả hai đều nằm trong hệ toạ độ của camera một*.

Câu “cả hai đều nằm trong hệ toạ độ của camera một” chính là toàn bộ mẹo. Tư thế tương đối không biến mất; nó bị *gói vào* đầu ra. Giả sử tôi biết mỗi pixel của ảnh hai ứng với điểm ba chiều nào trong hệ của camera một. Khi đó tư thế của camera hai chỉ là phép biến đổi cứng khớp bản

đồ điểm ấy với hình học của chính nó. Đó là một bài toán Procrustes hoặc PnP, có nghiệm dạng đóng, không cần RANSAC trên ma trận cơ bản. Tương tự, tiêu cự đọc ra được bằng cách giải một ẩn từ bản đồ điểm của ảnh một; độ sâu là kênh thứ ba; và tương ứng giữa hai ảnh là láng giềng gần nhất trong không gian ba chiều.

Kiến trúc thì đơn giản đến mức đáng ngạc nhiên: một bộ mã hoá *transformer* dùng chung cho hai ảnh, rồi hai bộ giải mã *liên tục nhìn sang nhau* bằng *attention chéo*, rồi hai đầu ra. Hàm mất mát là hồi quy trong không gian ba chiều, sau khi chuẩn hoá cả dự đoán lẫn nhãn về cùng một thang, và có trọng số theo một độ tin cậy do chính mạng dự đoán. Chuẩn hoá về cùng thang là chỗ tỉ lệ tuyệt đối bị bỏ đi, nên đầu ra gốc của DUST3R không có đơn vị mét.

Hình 30.7 vẽ cả cơ chế lẫn ý nghĩa hình học.



Hình 30.7. Bản đồ điểm của DUST3R: cơ chế ở trên, ý nghĩa hình học ở dưới. Mạng gán thẳng cho mỗi pixel của *cả hai* ảnh một điểm ba chiều trong *cùng* hệ toạ độ của camera một. Vì đã ở chung một hệ, mọi đại lượng mà pipeline cổ điển phải ước lượng qua nhiều bước đều trở thành phép đọc: tiêu cự, độ sâu, tư thế tương đối và tương ứng. Bài học thiết kế: chọn đúng biểu diễn đầu ra xoá được nhiều bước hơn là chọn đúng kiến trúc. Cái không xoá được là tỉ lệ tuyệt đối — hàm mất mát chuẩn hoá thang trước khi so, nên đầu ra không có đơn vị mét.

B. MAST3R, MAST3R-SLAM, VGGT, VGGT-SLAM

I MẠCH 2 — từ hai ảnh tới một hệ SLAM

- | | |
|-------------------------|---|
| <p>1 Vấn đề</p> | DUST3R cho hình học tốt nhưng tương ứng của nó chỉ là láng giềng gần nhất trong không gian ba chiều, nên không đủ chính xác để làm bộ ghép cặp. |
| <p>Giải pháp</p> | MASt3R gắn thêm một đầu ra mô tả đặc trưng dày lên chính kiến trúc đó, huấn luyện bằng một hàm mất mát ghép cặp, cộng một thủ tục ghép tương hỗ nhanh để tránh so sánh mọi cặp pixel [289]. |
| <p>Hệ quả</p> | nó trở thành một bộ ghép cặp <i>được neo trong ba chiều</i> , nên nó chịu được thay đổi góc nhìn cực đoan — đúng trường hợp mà mọi bộ mô tả hai chiều đều chết. |

- Giá phải trả** vẫn là một mô hình lớn chạy trên *từng cặp* ảnh, tức chậm hơn một bộ ghép cặp thừa nhiều bậc, và chi phí nhân lên theo số cặp.
- 2 Vấn đề** Có một bộ ghép cặp mạnh vẫn chưa phải một hệ SLAM: thiếu quản lý khung khoá, thiếu tối ưu toàn cục, thiếu đóng vòng.
- Giải pháp** MAST3R-SLAM dựng một hệ SLAM dày, chạy trực tuyến, đặt thẳng trên tiên nghiệm hai ảnh đó, và chạy được cả khi camera chưa hiệu chỉnh [220].
- Hệ quả** điểm thiết kế đáng học: nó tối ưu *trong không gian bản đồ điểm* chứ không trong không gian sai số chiếu lại, vì bản đồ điểm mới là thứ mạng cho ra.
- Giả định** phần cứng có GPU rời đủ mạnh. Đây là công trình rất mới; tôi nêu cơ chế mà không nêu con số hiệu năng.
- 3 Vấn đề** Suy luận theo từng cặp có chi phí bậc hai theo số ảnh, và phải có một bước căn chỉnh toàn cục riêng để ghép các cặp lại.
- Giải pháp** VGGT bỏ hẳn cấu trúc theo cặp: một *transformer* lớn nhận cả nhóm ảnh cùng lúc và trả về trong một lượt xuôi cả nội tham số, ngoại tham số, bản đồ độ sâu, bản đồ điểm và vết điểm ba chiều [290].
- Giải pháp** cơ chế cốt lõi là *xen kẽ hai loại attention*: một lượt attention trong phạm vi từng ảnh, rồi một lượt attention trên toàn bộ token của mọi ảnh, lặp lại nhiều lần. Lượt trong ảnh giữ chi tiết, lượt toàn cục làm cho các ảnh biết về nhau.
- Hệ quả** không còn bước căn chỉnh toàn cục nào cả; hình học ra thẳng, trong hệ toạ độ của camera đầu tiên.
- Giá phải trả** attention toàn cục trên mọi token của mọi khung là chỗ bộ nhớ nổ, và nó đặt một trần cứng lên số khung hình — mục sau tính con số đó.
- 4 Vấn đề** Nếu ghép các mảnh bản đồ do VGGT sinh ra bằng một phép biến đổi tương tự thì kết quả sai, chứ không chỉ là chưa tối ưu.
- Giải pháp** VGGT-SLAM nhận ra rằng với camera chưa hiệu chỉnh và cảnh gần phẳng, hai mảnh bản đồ liên hệ với nhau bằng một phép biến đổi xạ ảnh tổng quát chứ không phải phép tương tự; nên nó tối ưu trên nhóm $SL(4)$ mười lăm bậc tự do [291].
- Hệ quả** đây là bài học thiết kế đắt giá nhất của cả mục: *lớp bất định của đầu ra frontend quyết định đa tạp mà backend phải tối ưu trên đó*. Chọn nhầm đa tạp là sai mô hình, không phải chậm hội tụ.
- Giả định** cảnh và cấu hình camera thật sự rơi vào lớp bất định xạ ảnh đó. Công trình rất mới, tôi nêu ý tưởng chứ không nêu kết quả.

C. Hai kiểu trần số khung, và vì sao cả hai đều là trần

Đây là phần quyết định các phương pháp này cắm vào đâu được trong một hệ nhúng. Điểm mấu chốt không phải một con số megabyte — con số ấy đổi theo bản cài đặt và theo phần cứng, nên nêu nó ra ở đây là gây hiểu nhầm — mà là *dạng* của quan hệ giữa chi phí và số khung hình. Hai nhánh có hai dạng khác nhau, và chúng dẫn tới hai kiểu trần khác nhau.

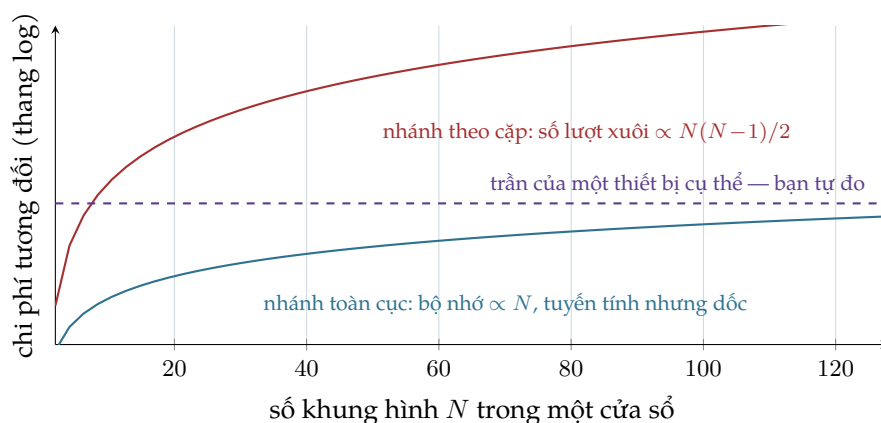
Nhánh theo cặp (DUST3R, MAST3R). Với N ảnh, đồ thị liên kết đầy đủ có $N(N - 1)/2$ cặp, và *mỗi cặp là một lượt xuôi riêng*. Đây là quan hệ bậc hai theo số khung, và nó là thứ chặn bạn chứ không phải bộ nhớ: một cửa sổ vài chục khung đã kéo theo hàng trăm lượt xuôi cho một lần cập nhật. Bộ nhớ thì tăng tuyến tính và hiền hơn nhiều, vì mỗi khung chỉ giữ một bản đồ điểm cỡ ba kênh trên mỗi pixel.

Nhánh toàn cục (VGGT). Ở đây chi phí đi theo số *token*, và số token là đại lượng đáng đếm vì nó suy ra được từ kiến trúc: một ảnh 518×518 chia thành mảnh 14×14 cho $37 \times 37 = 1369$ token

mỗi khung, nên N khung cho $1369N$ token. Attention *toàn cục* chạy trên toàn bộ chỗ token ấy, và mọi token phải giữ khoá cùng giá trị ở mọi tầng của thân *transformer*. Quan hệ vì thế tuyến tính theo N — nhưng với một hệ số nhân rất lớn, vì hệ số ấy là số tầng nhân chiều ẩn. Tuyến tính với hệ số lớn vẫn chậm trần rất sớm.

Cách đọc đúng hai đoạn trên: đừng nhớ megabyte, hãy nhớ *số mũ*. Nhánh theo cặp là bậc hai theo số khung; nhánh toàn cục là bậc một nhưng dốc. Muốn biết trần cụ thể của mình nằm ở đâu thì chạy một lượt xuôi với số khung tăng dần trên đúng phần cứng đích rồi đọc bộ nhớ đỉnh — một buổi chiều, và kết quả đúng cho máy của bạn thay vì đúng cho máy của người viết bài báo.

Hình 30.8 vẽ lại hai dạng ấy. Kết luận kiến trúc rút ra được ngay và không phụ thuộc con số: cả hai nhánh đều có một trần cứng theo số khung, nên mọi hệ SLAM dựng trên nhóm này đều buộc phải chia bản đồ thành các mảnh nhỏ rồi ghép lại — và bước ghép ấy chính là chỗ VGGT-SLAM phát hiện ra rằng phép biến đổi tương tự là sai mô hình.



Hình 30.8. Hai kiểu trần số khung, vẽ trên thang log. Trục đứng cố ý không có vạch số: hình này nói về *số mũ*, không về megabyte, vì con số tuyệt đối đổi theo bản cài đặt và theo phần cứng còn số mũ thì không. Nhánh theo cặp bị chặn bởi số lượt xuôi tăng bậc hai — qua vài chục khung thì số cặp đã lên hàng nghìn, và đó là thứ chặn bạn trước cả bộ nhớ. Nhánh attention toàn cục tăng tuyến tính, nhưng hệ số nhân là số tầng nhân chiều ẩn, nên nó vẫn chạm trần sớm. Đường ngang là trần của một thiết bị cụ thể; vị trí thật của nó là thứ bạn đo bằng cách tăng dần số khung trên đúng phần cứng đích rồi đọc bộ nhớ đỉnh. Hệ quả kiến trúc không phụ thuộc con số: mọi hệ SLAM dựng trên nhóm này đều buộc phải chia bản đồ thành mảnh nhỏ rồi ghép lại.

Bảng 30.3. Năm công trình trong nhánh mô hình nền hình học. Cột cuối là cột quyết định với một hệ nhúng thời gian thực, và nó ghi *loại* ràng buộc chứ không ghi con số: mọi nhận định ở đây lấy từ mô tả phương pháp, không từ số đo mà tôi tự chạy, nên chúng nói được cái gì chặn bạn nhưng không nói được chặn ở mức nào.

Công trình	Cơ chế cốt lõi	Thêm được gì	Hỏng khi nào / cái giá
DUST3R	Hồi quy bản đồ điểm cho cặp ảnh, cả hai trong hệ của camera một	Bỏ hẳn ghép cặp, ma trận cơ bản và tam giác hoá	Không có đơn vị mét; nhiều ảnh thì chi phí theo cặp tăng bậc hai
MASt3R	Thêm đầu ra mô tả dày và ghép tương hỗ nhanh lên DUST3R	Ghép cặp neo trong ba chiều, chịu được góc nhìn cực đoan	Một lượt xuôi mô hình lớn cho <i>mỗi cặp</i> ; chi phí nhân theo số cặp
MASt3R-SLAM	Hệ SLAM dày trực tuyến, tối ưu trong không gian bản đồ điểm	Chạy được cả khi camera chưa hiệu chỉnh	Cần GPU rời mạnh; rất mới, ít bằng chứng ngoài phòng thí nghiệm
VGGT	Một <i>transformer</i> nhận nhiều ảnh, xen kẽ attention trong ảnh và toàn cục	Bỏ luôn bước căn chỉnh toàn cục; ra thẳng tư thế, độ sâu, bản đồ điểm	Bộ nhớ attention toàn cục đặt trần cứng lên số khung
VGGT-SLAM	Ghép các mảnh bản đồ của VGGT trên nhóm xạ ảnh SL(4)	Chỉ ra rằng phép biến đổi tương tự là <i>sai mô hình</i> khi camera chưa hiệu chỉnh	Chỉ đúng khi lớp bất định thật sự là xạ ảnh; rất mới

D. Chỗ nhánh này chạm vào nỗi đau đóng vòng

Có một chỗ mà nhánh này đáng để bạn theo dõi nghiêm túc, và nó không phải chỗ được quảng cáo nhiều nhất.

Chương 29 tách đóng vòng thành bốn tầng và chỉ ra rằng tầng kiểm chứng hình học mới là chỗ hay giết vòng nhất, đặc biệt khi hai lần đi qua cùng một chỗ theo hai hướng ngược nhau. Đó chính xác là trường hợp mà một bộ mô tả hai chiều không cứu được: cùng một góc tường nhìn từ hai phía cho hai mảnh ảnh không có gì chung. MASt3R ghép cặp trong không gian ba chiều, nên nó không phụ thuộc vào việc hai mảnh ảnh trông giống nhau; nó phụ thuộc vào việc hai bản đồ điểm mô tả cùng một hình học.

→ **hệ quả của F/29** — đây là một lối thoát khá dễ cho đúng cái nút thắt mà chương 29 đo được ở tầng kiểm chứng hình học.

Cái giá thì rõ, và nó là lập luận về *tần suất* chứ không về mili-giây: đóng vòng chạy không thường xuyên và không nằm trên đường tới hạn thời gian thực, nên một mô hình đắt gấp nhiều lần ngân sách một khung hình vẫn dùng được ở đó, trong khi cùng mô hình ấy đặt vào *frontend* thì không. Nói cách khác, nếu có một chỗ trong hệ của bạn mà nhánh mô hình nền hình học đáng thử trước tiên, thì đó là tầng kiểm chứng ứng viên vòng, không phải *frontend*.

E. Tối ưu trong không gian bản đồ điểm: được gì và mất gì

MASt3R-SLAM tối ưu trong không gian bản đồ điểm chứ không trong không gian sai số chiều lại. Chi tiết ấy nghe như một lựa chọn cài đặt, nhưng nó đổi bản chất bài toán, và người đã viết bộ hiệu chỉnh chùm tia sẽ thấy ngay vì sao.

Sai số chiều lại có một tính chất mà ta hay quên vì nó quá quen: nó là sai số *trong không gian đo*. Nhiều của bộ phát hiện điểm gần với nhiều đẳng hướng vài phần mười pixel, nên bình phương tối thiểu trên sai số chiều lại xấp xỉ đúng ước lượng hợp lý cực đại. Chuyển sang đo sai lệch trong không gian ba chiều phá vỡ điều đó. Một sai lệch nửa pixel ở một điểm cách ba mét tương ứng

vài milimet trong không gian; cùng nửa pixel ấy ở ba mươi mét tương ứng vài chục centimet. Đo bằng mét là ngầm gán cho điểm xa một trọng số lớn hơn hàng trăm lần so với điều mà phép đo thật sự đáng được hưởng.

Vậy tại sao vẫn làm? Vì mạng không cho bạn phép đo trong không gian ảnh; nó cho bạn bản đồ điểm. Ép nó quay về không gian ảnh đòi hỏi biết nội tham số và đòi hỏi tin vào đó, mà một trong những điểm bán hàng của cả nhánh này lại là không cần hiệu chỉnh. Đây là một đánh đổi thật, không phải một sơ suất — nhưng nó giải thích một hiện tượng bạn sẽ thấy khi đọc kết quả của nhánh này: chất lượng dựng lại nhìn rất đẹp trong khi độ chính xác quỹ đạo thì chưa bằng một hệ cổ điển đã tinh chỉnh kỹ.

F. Lỗi hổng lớn nhất của nhánh này: không có chỗ cho IMU

Có một điều mà không bảng kết quả nào nói ra, và với bạn nó có thể là điều quan trọng nhất trong cả mục: *cả nhánh này thuần thị giác*. DUST3R, MAST3R và VGGT đều nhận vào ảnh và chỉ ảnh.

Hệ quả không phải là “thiếu một tính năng”. Nó là một mâu thuẫn ở gốc. Nhánh này thừa hưởng trọn vẹn tính không xác định được của tỉ lệ — đầu ra của chúng không có đơn vị mét, và mục trước đã chỉ ra rằng không lượng dữ liệu nào chứa được điều đó. Đồng thời chúng không có chỗ nào để nhận cảm biến duy nhất chứa được nó. Một hệ SLAM cổ điển chỉ cần thêm một gia tốc kế là mất hẳn bậc tự do tỉ lệ; một mô hình nền hình học không có cơ chế tương đương.

Điều đó dẫn tới ba việc phải làm nếu bạn muốn ghép nhánh này vào hệ của mình.

- **Tỉ lệ phải được cố định bên ngoài mạng.** Cách sạch nhất là để mạng lo hình dạng, còn tỉ lệ do phần cổ điển của bạn quyết, giống như phân công giữa tiên nghiệm và phép đo ở mục trước.
- **Không có bộ dự đoán chuyển động.** Mạng xử lý một nhóm ảnh như một tập, không như một chuỗi có động học. Nó không biết rằng khung sau phải gần khung trước, trong khi tích phân trước của IMU biết điều đó rất chắc.
- **Không có mốc thời gian.** Không có chỗ khai báo độ trễ giữa ảnh và IMU, và không có chỗ khai báo màn trập cuộn. Với chuyển động nhanh, cả hai thứ đó đều là nguồn sai số bậc nhất.

Nói cách khác, ba lỗi hổng ấy đúng là ba thứ mà một hệ thị giác-quán tính đã giải xong từ lâu. Đây là lý do mạnh nhất để đọc nhánh này như một *nguồn ý tưởng* và như một công cụ cho các tác vụ ngoài đường tới hạn, chứ không như một ứng viên thay thế.

↪ **tiền đề F/34** — mọi thứ chương 34 nói về trọng số IMU đều giả định có một chỗ trong bài toán để đặt phép đo IMU vào; nhánh này chưa có chỗ đó.

G. Đọc bằng chứng của nhánh này cho đúng

Ba điều mà các bảng kết quả trong nhánh này thường không nói ra, và cả ba đều quan trọng với bạn.

- **Chất lượng ảnh dựng lại không phải độ chính xác quỹ đạo.** Hai đại lượng này bị trình bày cạnh nhau đến mức người đọc tự động gộp chúng. Một bản đồ dày đẹp mắt vẫn có thể đi kèm một quỹ đạo trôi, vì phần lớn sai số quỹ đạo là sai số tần số thấp mà mắt không thấy trên ảnh dựng lại.
- **Phép căn chỉnh trước khi đo che mất đúng thứ bạn quan tâm.** Với hệ đơn mắt, quỹ đạo phải được căn chỉnh cả tỉ lệ trước khi tính sai số, và phép căn chỉnh đó hấp thụ trọn vẹn phần trôi tỉ lệ. Hệ stereo-inertial của bạn không được hưởng ưu ái ấy, nên hai con số không so trực tiếp được với nhau.

- **Điều kiện chạy khác hẳn điều kiện của bạn.** Phần lớn kết quả đến từ chuỗi ngắn, cảnh trong nhà, và máy để bàn có GPU rời. Ba khác biệt đó cộng lại đủ để đảo ngược một kết luận, và không cái nào trong ba xuất hiện dưới dạng một cột trong bảng.

Cách đọc thực dụng: khi gặp một hệ trong nhánh này, hãy tìm ba con số trước khi tìm bảng chính — độ dài chuỗi tính bằng khung hình, phần cứng, và quỹ đạo được căn chỉnh theo phép biến đổi nào. Nếu ba con số ấy không có trong bài, phần còn lại chưa dùng để quyết định được.

→ cùng nguyên lý với E/23 — đây là cùng danh sách câu hỏi bạn dùng để đọc mọi bảng kết quả, chỉ đổi ba mục cho đúng đặc thù của nhánh này.

30.5 Cổng đánh giá: khi nào nó cứu, khi nào nó phá (The Evaluation Gate)

A. Bốn chỗ nó thật sự cứu

Gộp lại từ ba mục trên, danh sách các chỗ mà một tiên nghiệm học được thêm *thông tin* chứ không chỉ thêm số:

- **Vùng thị sai đã cạn.** Xa hơn khoảng cách mà độ chênh lệch tụt xuống dưới sàn nhiều, stereo của bạn không còn nói gì. Ở đó một phỏng đoán có sai số vừa phải vẫn tốt hơn một phép đo mà sai số đã lớn hơn nhiều lần — và điểm cắt giữa hai đường cong ấy là thứ bạn tự đo, không phải thứ chép từ bảng của ai.
- **Vùng không có vân.** Không có ghép cặp thì không có phép đo ở bất kỳ khoảng cách nào. Đây là chỗ duy nhất mà tiên nghiệm không cạnh tranh với cái gì cả.
- **Chuyển động nhanh và ảnh mờ.** Dòng quang dày cho tương ứng ở đúng dải mà bộ phát hiện góc đã tắt. Đây là chỗ có bằng chứng cơ chế mạnh nhất trong cả chương.
- **Lắp lỗ cho bản đồ dày, ở hạ nguồn.** Nếu sản phẩm cuối là bản đồ chiếm dụng để điều hướng chứ không phải quỹ đạo, thì tiên nghiệm dày là đúng công cụ — miễn là nó không quay ngược vào đồ thị tư thế.

Có một quan sát về chính tập dữ liệu của bạn đáng ghi ra, vì nó thu hẹp cổng thứ nhất đi nhiều. Các chuỗi phòng Vicon của EuRoC diễn ra trong một không gian cỡ tám mét, nên gần như không có điểm nào rơi vào dải thị sai cạn. Các chuỗi nhà máy thì rộng cỡ vài chục mét, nên ở đó cổng mới thật sự hé mở. Nghĩa là: nếu bạn đo lợi ích của độ sâu học được trên các chuỗi phòng Vicon rồi kết luận “không có tác dụng”, kết luận ấy đúng nhưng nó chỉ đúng cho những chuỗi đó. Ngược lại, một kết quả tốt trên chuỗi nhà máy cũng chưa chuyển được sang cảnh ngoài trời rộng hơn nữa. Đây là lý do mini project của chương yêu cầu hai chuỗi có quy mô không gian khác nhau, chứ không phải hai chuỗi có độ khó khác nhau.

Còn khởi tạo đơn ảnh, thứ được nhắc nhiều nhất trong tài liệu, gần như không liên quan tới bạn. Hệ stereo có đường cơ sở nên nó khởi tạo được ngay cả khi đứng yên. Phải nói thẳng điều này vì nó là nguồn lãng phí thời gian phổ biến: rất nhiều lợi ích được quảng cáo là lợi ích *của hệ đơn mắt*, và chúng biến mất khi bạn có hai camera.

B. Bốn chỗ nó phá — và phép tính chứng minh chỗ thứ nhất

Thứ nhất: nó đưa sai lệch có hệ thống vào bộ tối ưu, và đặt trọng số nhỏ không cứu được.

Đây là chỗ đáng tính bằng số nhất trong cả chương, vì kết quả đi ngược trực giác. Xét việc ước lượng một đại lượng vô hướng, chẳng hạn khoảng cách tới một mảng tường. Bạn có hai nguồn.

- 20 phép đo stereo thật, mỗi phép đo độ lệch chuẩn $\sigma_m = 0,10$ m. Thông tin đóng góp: $20/0,10^2 = 2000$.
- 5000 pixel độ sâu học được, bạn khai báo độ lệch chuẩn $\sigma_p = 0,30$ m — đã cẩn thận đặt gấp ba lần phép đo thật. Thông tin đóng góp: $5000/0,30^2 \approx 55600$.

Tiên nghiệm áp đảo phép đo gấp gần ba mươi lần. Giả sử mạng lệch đều 0,25 m — cả mảng tường bị đẩy ra xa cùng một khoảng. Nghiệm bình phương tối thiểu có trọng số dịch đi

$$\frac{55600}{55600 + 2000} \times 0,25 \approx 0,241 \text{ m.}$$

Bạn vừa dịch nghiệm 24 cm bằng một nguồn mà bạn tưởng đã hạ trọng số.

Bây giờ thử hạ trọng số thật mạnh, khai báo $\sigma_p = 3,0$ m, tức gấp mười lần. Thông tin còn $5000/9 \approx 556$, và độ dịch còn $556/2556 \times 0,25 \approx 5,4$ cm. Muốn tiên nghiệm thật sự không đáng kể, bạn cần $5000/\sigma_p^2 \ll 2000$, tức $\sigma_p \gg 1,6$ m — một mức bất định mà ở đó tiên nghiệm chẳng còn nói gì nữa.

Vì sao lại thế? Vì bạn đang đếm sai số mẫu độc lập. Nếu 5000 pixel đó có sai số *độc lập* thì trung bình của chúng có sai số chuẩn $0,30/\sqrt{5000} \approx 0,004$ m, tức chính xác hơn phép đo stereo cả bậc độ lớn — và điều đó rõ ràng vô lý. Sự thật là sai số của chúng *tương quan gần như hoàn toàn*: cả mảng tường lệch cùng một hướng vì mạng hiểu sai cả mảng tường. Số mẫu độc lập hiệu dụng gần với 1 hơn là 5000. Một ma trận thông tin chéo giả định độc lập, nên nó thổi phồng lượng thông tin lên cỡ ba bậc độ lớn.

Suy ra cách chữa đúng, và nó không phải hạ trọng số: hãy **mô hình hoá phần tương quan bằng một biến phụ**. Thêm vào bài toán một cặp tham số tỉ lệ và độ dịch cho mỗi khung hình, để thành phần sai số chung bị hấp thụ vào tham số đó thay vì đẩy vào tư thế. Đây đúng là cách bạn đã xử lý độ lệch của con quay hồi chuyển — không hạ trọng số phép đo con quay, mà thêm một biến độ lệch vào trạng thái.

↪ **cùng nguyên lý với F/34** — thêm biến phụ để hấp thụ thành phần sai số chung chính là cách xử lý độ lệch cảm biến trong đồ thị nhân tử; ở đây đối tượng đối từ con quay sang mạng độ sâu.

Và đó cũng chính là bài toán mà Align3R đang tấn công ở phía mô hình.

Thứ hai: nó phá tính nhất quán đa khung. Với họ MiDaS, cặp tỉ lệ và độ dịch là của riêng từng khung hình. Một bức tường đứng yên vì thế thở phập phồng qua các khung. Bộ tối ưu của bạn tin rằng cảnh cứng, nên nó không có chỗ nào để nhét nhịp thở đó, ngoài tư thế và vận tốc. Chuỗi hậu quả rất cụ thể: tỉ lệ dao động ở tần số khung hình, vận tốc thị giác nhiễu theo, và bộ ước lượng độ lệch gia tốc kè — vốn là thứ hấp thụ mọi thứ không giải thích được giữa thị giác và IMU — bắt đầu trôi. Nói cách khác, một tiên nghiệm độ sâu đặt sai chỗ hiện ra dưới dạng một bệnh của IMU. Đó là kiểu triệu chứng tốn nhiều ngày nhất để lần ra.

Thứ ba: nó làm hỏng chẩn đoán. Ba dạng, tăng dần độ khó chịu.

- **Mất ranh giới ablation.** Một mạng là một khối. Bạn không tắt được nửa nó để xem nửa kia đóng góp gì.
- **Hỏng lặng lẽ.** Không có tín hiệu nào tương đương với “không còn điểm nội biên nào”. Mạng luôn trả về một bản đồ độ sâu trông hợp lý, kể cả trên mặt gương, kể cả khi ống kính bị bẩn.
- **Mất tính lặp lại.** Một mô hình dựa trên khuếch tán như Marigold cho hai kết quả khác nhau ở hai lần chạy trên cùng một ảnh, vì nó bắt đầu từ nhiễu ngẫu nhiên. Giả sử bạn đã bỏ công làm cho hệ của mình tắt định: ghim thứ tự duyệt container, xoá tranh chấp luồng. Cắm một khối như thế vào là tự tay dựng lại đúng cái sản phẩm bạn vừa đẹp. Và lần này không có bug nào để sửa.

Thứ tư: nó lệch miền một cách âm thầm. Metric3D cho độ sâu tuyến tính theo tiêu cự bạn khai báo, nên một sai số hiệu chỉnh 3% thành sai số độ sâu 3% đều khắp. Các mô hình huấn luyện

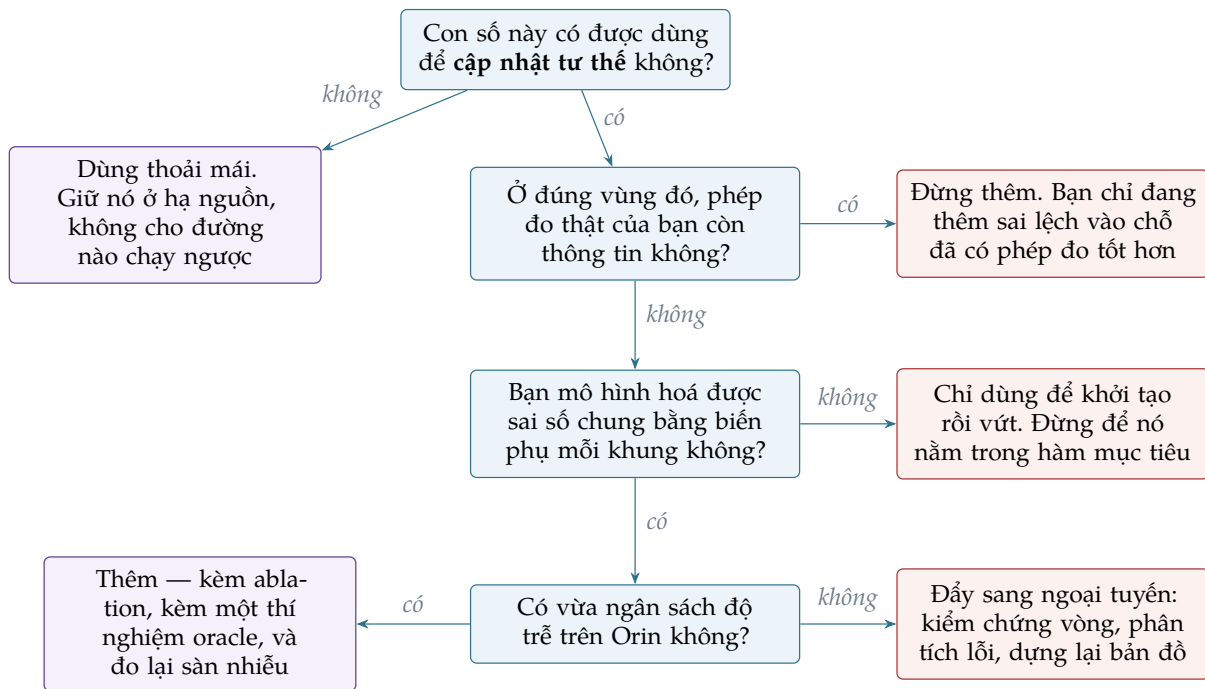
chủ yếu trên cảnh đường phố và cảnh trong nhà thông thường sẽ gặp gì ở một hành lang tối của TUM-VI, ở một nhà máy có kết cấu kim loại lặp lại, hay qua một ống kính mắt cá? Không ai biết trước, và triệu chứng duy nhất là quỹ đạo hơi tệ đi.

Bảng 30.4. Bốn chỗ cứu và bốn chỗ phá, đặt cạnh nhau. Cột cuối là điều bạn phải làm được *trước* khi tích hợp; nếu chưa làm được thì công chưa mở.

	Tình huống	Cơ chế	Phép kiểm tra trước khi tích hợp
Cứu	Thị sai đã cạn	ΔZ tăng theo Z^2 còn sai số tương đối của mạng thì không	Vẽ biểu đồ tỉ lệ điểm bản đồ theo khoảng cách; đo phần nằm ngoài ngưỡng một pixel độ chênh lệch
Cứu	Vùng không có vân	Không có ghép cặp thì không có phép đo để cạnh tranh	Đếm số khung hình có số điểm khớp dưới ngưỡng phục hồi
Cứu	Chuyển động nhanh, ảnh mờ	Ghép cặp thừa là chuỗi cổng nhân nhau, dòng quang không có cổng nào	Vẽ tỉ lệ điểm tìm lại được theo tốc độ góc lấy từ IMU
Cứu	Bản đồ dày ở hạ nguồn	Không phản hồi vào đồ thị tư thế nên không đưa sai lệch vào	Kiểm tra bằng mã rằng không có đường dữ liệu nào chạy ngược
Phá	Sai lệch có hệ thống	Sai số tương quan bị ma trận thông tin chéo đếm thành hàng nghìn phiếu độc lập	Tính tỉ số thông tin giữa hai nguồn trước khi viết dòng code nào
Phá	Mất nhất quán đa khung	Tỉ lệ và độ dịch riêng từng khung bị hấp thụ vào tư thế và vận tốc	Vẽ tỉ lệ ước lượng theo thời gian; nếu nó dao động ở tần số khung hình thì hỏng
Phá	Mất chẩn đoán	Không ablate được từng phần; hỏng không phát tín hiệu; đầu ra có thể ngẫu nhiên	Chạy cùng cấu hình mười lần và so sánh nhiều trước và sau khi thêm
Phá	Lệch miền	Đầu ra tuyến tính theo tiêu cự khai báo; miền huấn luyện không phủ cảnh của bạn	Chạy trên chuỗi khó nhất của bạn và so với chân trị độ sâu, không chỉ so quỹ đạo

C. Cây quyết định

Gộp tất cả lại thành một thứ dùng được trong ba mươi giây. Hình 30.9 là cây quyết định cho câu hỏi “có nên cấm tiên nghiệm học được này vào không”.



Hình 30.9. Cây quyết định cho việc cấm một tiên nghiệm dày học được vào hệ SLAM. Câu hỏi đầu tiên không phải “mô hình nào tốt nhất” mà “con số này có chạy ngược vào tư thế không”: mọi rắc rối trong chương này chỉ xuất hiện khi câu trả lời là có. Hai nhánh giữa lần lượt lọc theo thông tin (đừng cạnh tranh với phép đo tốt hơn) và theo khả năng mô hình hoá sai số tương quan. Nhánh cuối là ngân sách, và lời đáp “không” ở đó không phải lời từ chối — nó chỉ đổi chỗ dùng sang các tác vụ ngoại tuyến.

D. Bốn bài học thiết kế mang đi được, kể cả khi bạn không dùng mô hình nào

Phần này trả lời câu hỏi cuối trong bốn câu mà mọi phương pháp trong Phần F phải trả lời: nếu không dùng nguyên phương pháp thì học được gì từ thiết kế của nó. Bốn bài học dưới đây dùng lại được ngay trong mã C++ của bạn.

- **Tách phần tính một lần khỏi phần lặp** — từ *RAFT*. Bất cứ khi nào một vòng lặp tối ưu tính lại cùng một đại lượng ở mọi vòng, hãy hỏi xem đại lượng ấy có phụ thuộc nghiệm hiện tại không. Nếu không, đưa nó ra ngoài và biến vòng lặp thành phép tra cứu. Hệ quả kèm theo cũng quý: số vòng lặp trở thành núm vận độ trễ chính được lúc chạy.
- **Làm thô đúng trục** — cũng từ *RAFT*. Mọi sơ đồ đa tỉ lệ đều phải chọn làm thô cái gì. Làm thô sai trục thì bạn mất đúng thứ mình đang tìm, và mọi mức mịn hơn chỉ tinh chỉnh quanh chỗ sai. Câu hỏi cần đặt mỗi lần thiết kế một kim tự tháp: *trục nào ở đây không được phép mất độ phân giải*.
- **Ẩn số mà dữ liệu không nói tới thì phải ghim, không phải học** — từ *Metric3D*. Đưa nó ra khỏi phần học, chuẩn hoá ở đầu vào, khôi phục ở đầu ra. Đây là cổ định gauge, và bạn đã dùng nó mỗi lần cố định khung khoá đầu.
- **Muốn có trọng số dùng được thì phải huấn luyện cho nó** — từ *SEA-RAFT*. Một hàm mất mát đơn phương thức không thể diễn đạt “câu trả lời ở đây có hai đáp án”. Nếu bạn cần một hiệp phương sai để đưa vào đồ thị nhân tử thì phải chọn hàm mất mát cho phép biểu diễn nó, ngay từ lúc huấn luyện.
- **Lớp bất định của đầu ra quyết định đa tạp của bộ tối ưu** — từ *VGGT-SLAM*. Trước khi viết bộ ghép mảnh bản đồ, hãy hỏi hai mảnh ấy thật sự liên hệ với nhau bằng nhóm biến đổi nào. Chọn nhầm nhóm là sai mô hình, và sai mô hình không hiện ra dưới dạng hội tụ chậm — nó hiện ra dưới dạng một nghiệm sai mà bộ tối ưu rất tự tin.

Bài học thứ sáu không thuộc riêng công trình nào mà thuộc cả chương, và nó là bài học đắt nhất: *đổi biểu diễn đầu ra xoá được nhiều bước hơn là đổi kiến trúc*. DUS3R không giỏi hơn pipeline cổ điển ở bất kỳ bước riêng lẻ nào; nó chỉ chọn một đầu ra làm cho phần lớn các bước trở nên không cần thiết.

Bảng 30.5. Ba họ, đọc theo *cấu trúc* chi phí thay vì theo con số. Bảng này cố ý không có cột mili-giây hay cột megabyte: một con số như thế chỉ đúng cho một bản cài đặt, một trình biên dịch suy luận và một mức tải cụ thể, nên chép nó vào tài liệu học là dạy sai. Cột giữa nói chi phí *tỉ lệ với cái gì* — đó mới là thứ không đổi — và cột phải nói phép đo bạn phải chạy để biến nó thành một con số dùng được.

Họ	Chi phí tỉ lệ với cái gì	Chỗ dùng được	Phép đo phải chạy
Độ sâu đơn ảnh, bản nhỏ	Một lượt xuôi mỗi lần gọi; gọi được ở tần số thấp hơn tần số khung hình	Có thể vừa ngân sách nếu bạn chấp nhận cập nhật thưa hơn ảnh	Biên dịch bằng trình suy luận của thiết bị, đo độ trễ khi hệ đang chạy đủ tải
Độ sâu khuếch tán	Số bước khử nhiễu nhân một lượt xuôi, và phải chạy nhiều lần rồi lấy trung bình	Chỉ ngoại tuyến; thêm nữa nó không tắt định	Chạy cùng ảnh nhiều lần và đo độ tàn mạn của đầu ra
Dòng quang dày	Bình phương độ phân giải lưới cho khối tương quan, cộng số vòng lặp	Ngang cả ngân sách <i>frontend</i> hiện tại, nên là quyết định thay thế chứ không phải thêm vào	Đo bộ nhớ đỉnh và độ trễ ở đúng độ phân giải bạn định dùng
Mô hình nền hình học	Bậc hai theo số khung (nhánh theo cặp) hoặc tuyến tính dốc theo số token (nhánh toàn cục)	Chỉ dùng ngoài đường tới hạn: kiểm chứng vòng, dựng lại bản đồ, phân tích lỗi	Tăng dần số khung cho tới khi hết bộ nhớ; đó là trần thật của bạn

CẦN NHỚ

Ba họ mô hình trong chương này không cho bạn phép đo mới; chúng cho bạn phỏng đoán dày. Với một hệ đã có tỉ lệ mét thật, giá trị của chúng chỉ nằm ở đúng những chỗ phép đo thật đã cạn — thị sai dưới sàn nhiều, vùng không có vân, khung bị mờ. Ở mọi chỗ khác, chúng chỉ thêm sai lệch có hệ thống. Và cái giá cố định, không thương lượng được, là: *chúng không bao giờ nói “tôi không biết”*. Hệ cổ điển của bạn hồng âm ỉ bằng cách trả về không điểm nội biên; một mạng hồng lặng lẽ bằng cách trả về một câu trả lời trông rất hợp lý.

30.6 Giới hạn và trường hợp hồng

Mục này nói về giới hạn của chính chương, không chỉ của các phương pháp trong chương.

Chương này cố ý không đưa con số ngân sách nào. Không có mili-giây trên Orin, không có megabyte, không có FLOP. Lý do không phải là lười: một ước lượng tự tính từ đếm token hay đếm phép nhân cộng có sai số cả bậc độ lớn so với thực tế, vì nó bỏ qua hiệu suất sử dụng, trình biên dịch suy luận, và các thủ thuật tiết kiệm trong bản cài đặt thật — nhưng khi đã in ra giấy thì nó *trông* như một phép đo và bị dùng như một phép đo. Thứ chương này đưa thay vào đó là *dạng* của quan hệ: chi phí tỉ lệ với cái gì, và tăng theo số mũ nào. Dạng ấy không đổi theo phần cứng, nên nó dùng được lâu; con số thì bạn phải tự đo, và Bảng 30.5 nói rõ mỗi họ đo bằng cách nào.

Lập luận về ảnh mờ là lập luận cơ chế, không phải đường cong đo được. Chuyện giá trị riêng của ma trận mô men bậc hai giảm cỡ L lần là đúng về bậc, nhưng bộ phát hiện thật có ngưỡng phi tuyến và có bước triệt phi cực đại, nên đường cong thật có thể dốc hơn hoặc thoải hơn nhiều. Đó là lý do mini project thứ hai của chương đi đo chính đường cong này.

Phép tính về sai lệch có hệ thống đã đơn giản hoá bài toán xuống một chiều. Trong bài toán tư thế đầy đủ, hướng mà sai lệch đẩy nghiệm phụ thuộc vào hình học của cảnh, và một phần của nó có thể bị chiếu triệt tiêu. Hãy coi con số 24 cm là chặn trên trên một trục, không phải dự báo cho ATE.

Ba giả định chung của cả ba họ, và chúng vỡ ở đâu.

- **Màn trập toàn cục.** Mọi mô hình trong chương này giả định một khung hình ứng với một thời điểm. Camera màn trập cuộn dưới chuyển động nhanh vì phạm vi điều đó ngay ở mức pixel, và một bản đồ điểm sẽ pha trộn hình học của nhiều thời điểm mà không có chỗ nào để khai báo điều đó.
- **Cảnh cứng.** DUST3R, VGGT và phần lớn dữ liệu huấn luyện của chúng là cảnh tĩnh. Người đi qua trước ống kính không chỉ làm sai vùng đó; nó làm hỏng cả phép căn chỉnh vì bản đồ điểm được hồi quy chung.
- **Gần lỗ kim.** Metric3D chuẩn hoá theo một tiêu cự, và DUST3R suy nội tham số từ bản đồ điểm với giả định điểm chính nằm giữa ảnh. Ống kính mắt cá hoặc méo mạnh phá cả hai, và triệu chứng là sai số biến thiên theo bán kính ảnh chứ không đều.

Ngân sách GPU là ràng buộc cứng nằm ngoài chương. Nếu GPU của Orin đã bận cho một khối khác — một bộ phát hiện vật thể, một bộ khử nhiễu — thì mọi lựa chọn trong chương này đóng lại, bất kể chúng chính xác tới đâu. Cổng ngân sách phải mở trước cổng độ chính xác.

Các công trình 2025–2026 được mô tả theo phương pháp, không theo kết quả tôi tự kiểm chứng. MAST3R-SLAM, VGGT, VGGT-SLAM và bản thứ ba của Depth Anything đều rất mới. Cơ chế mà tôi trình bày lấy từ mô tả phương pháp và tôi tin là đúng; các nhận định về hiệu năng thì tôi cố ý không đưa ra. Trần bộ nhớ và trần số khung cũng là thứ sẽ dịch chuyển nhanh nhất khi có bản cài đặt tốt hơn.

30.7 Thực hành

Repo và tài nguyên. Độ sâu đơn ảnh: [is1-org/MiDaS](https://github.com/is1-org/MiDaS) là bản tham chiếu để đọc hàm mất mát bất biến tỉ lệ và dịch — hãy mở thẳng phần cài đặt phép căn bậc nhất, nó ngắn và nói rõ hơn cả bài báo. [DepthAnything/Depth-Anything-V2](https://github.com/DepthAnything/Depth-Anything-V2) cho bản dùng được nhất hiện nay, [YvanYin/Metric3D](https://github.com/YvanYin/Metric3D) cho phần chuẩn hoá theo tiêu cự, [prs-eth/Marigold](https://github.com/prs-eth/Marigold) nếu muốn thấy cách một mô hình khuếch tán được tái dụng. Dòng quang: [princeton-vl/RAFT](https://github.com/princeton-vl/RAFT) rồi [princeton-vl/SEA-RAFT](https://github.com/princeton-vl/SEA-RAFT); đọc RAFT trước vì bản gốc gọn hơn, và hàm tra cứu là chừng bốn mươi dòng đáng đọc từng dòng. Mô hình nền hình học: [naver/dust3r](https://github.com/naver/dust3r), [naver/mast3r](https://github.com/naver/mast3r), [facebookresearch/vgggt](https://github.com/facebookresearch/vgggt), và [rmurai0610/MASt3R-SLAM](https://github.com/rmurai0610/MASt3R-SLAM) nếu muốn xem một hệ SLAM hoàn chỉnh dựng trên chúng. Đối chiếu với hệ cổ điển thì vẫn là [COLMAP](https://github.com/colmap/colmap) và [evo](https://github.com/ethz-asl/evoflow). Danh sách tên mô hình này chốt tại tháng 9/2026 và sẽ cũ trong sáu tới mười hai tháng; hạ tầng như COLMAP, evo và trình biên dịch suy luận thì bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Với cấu hình stereo của bạn, ở khoảng cách nào thì độ bất định độ sâu vượt 10% — và bạn tính ra nó bằng công thức nào? Nếu một mô hình họ MiDaS được căn bằng đúng một hằng số nhân, chân trời cứng của nó nằm ở đâu và vì sao nó tồn tại? Vì sao việc gộp trung bình khối tương quan chỉ theo hai chiều của ảnh thứ hai lại là chi tiết quyết định của RAFT? Nếu DUST3R không ước lượng tư thế thì tư thế nằm ở chỗ nào trong đầu ra của nó, và bạn đọc nó ra bằng phép toán gì?

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●● MonoDepth	Mạng đoán độ chênh lệch từ một ảnh, huấn luyện bằng sai lệch quang trắc sau khi làm cong ảnh stereo còn lại. Vì đường cơ sở lúc huấn luyện đã biết nên đầu ra có đơn vị mét — nhưng chỉ đúng cho đúng cấu hình camera đó.
●●● Độ sâu bất biến tỉ lệ và dịch	Đầu ra chỉ xác định sai khác một phép biến đổi bậc nhất trên độ sâu nghịch đảo: $d \approx s/Z + t$. Mạng biết thứ tự và tỉ lệ tương đối, không biết đơn vị và không biết gốc. Hai ẩn đó khác nhau ở mỗi khung hình.
●●● MiDaS	Trộn nhiều tập dữ liệu có nhãn không cùng đơn vị, bằng cách căn dự đoán về nhãn qua một phép biến đổi bậc nhất tối ưu trước khi tính sai lệch. Đối lấy khả năng khái quát rất tốt, cái giá là mất hẳn đơn vị mét.
●●● Metric3D	Lấy lại mét bằng cách chuẩn hoá mọi ảnh về một camera chuẩn theo tiêu cự, dự đoán, rồi nhân ngược lại. Đây là cố định gauge: một ẩn số dữ liệu không nói gì tới thì bị ghim ở bước tiền xử lý. Hệ quả: sai 3% ở tiêu cự thành sai 3% ở độ sâu.
●● Depth Anything V2	Thầy huấn luyện hoàn toàn trên ảnh tổng hợp để nhận hoàn hảo, rồi gán nhãn giả cho hàng chục triệu ảnh thật để lấp khoảng cách miền. Bài học: mở rộng dữ liệu, không mở rộng kiến trúc.
●●● Độ sâu quy ra mét bão hoà ở một khoảng cách cố định — nguyên nhân?	Bạn căn bằng một hằng số nhân duy nhất, tức ngầm đặt độ dịch bằng không. Khi đó $\hat{Z} = Z/(1 + tZ/s)$ và nó bão hoà ở s/t . Phân biệt với lệch miền: chân trời cứng cho sai số tăng đơn điệu theo khoảng cách, lệch miền thì không có dạng đó.
●●● Chân trời stereo	Khoảng cách mà độ chênh lệch tụt xuống dưới sàn nhiễu. Từ $Z = fb/d$ suy ra $\Delta Z = Z^2 \Delta d/(fb)$: sai số tăng theo bình phương khoảng cách. Với $fb \approx 50$ của EuRoC, độ chênh lệch một pixel đã ứng với 50 m.
●●● Khối tương quan toàn cặp	Tích vô hướng đặc trưng giữa mọi vị trí của ảnh một và mọi vị trí của ảnh hai, tính một lần. Với lưới 1/8 của ảnh EuRoC là 5640^2 phần tử — chi phí bộ nhớ tăng theo bình phương độ phân giải lưới, nên hạ độ phân giải là núm vặn mạnh nhất.
●●● RAFT	Tách “chỗ nào giống chỗ nào” — tính một lần — khỏi “tôi đang đoán ở đâu” — tra cứu lặp quanh phỏng đoán hiện tại, cập nhật bằng một khối hồi tiếp. Số vòng lặp là núm vặn độ trễ chỉnh được lúc chạy.
●● Vì sao RAFT không cần thu nhỏ ảnh?	Vì kim tự tháp của nó dựng trên khối tương quan, và chỉ gộp hai chiều thuộc ảnh thứ hai. Lưới của ảnh một giữ nguyên độ phân giải ở mọi mức, nên vật nhỏ bay nhanh không biến mất — đúng chỗ chiến lược thô tới mịn thất bại.
●● SEA-RAFT	Bản RAFT gọn hơn: hồi quy thẳng nghiệm khởi đầu nên cần ít vòng lặp hơn, và đổi hàm mất mát sang hỗn hợp hai phân phối Laplace. Nhờ hàm mất mát đó nó xuất ra được độ bất định theo từng pixel, dùng thẳng làm trọng số.
●●● Vì sao ghép cặp thưa sụp đổ dưới ảnh mờ còn dòng quang chỉ xuống dốc?	Vì ghép cặp thưa là chuỗi cổng nhân nhau: phải phát hiện được ở cả hai khung, rồi mới tới mô tả và kiểm tra tỉ số. Tính lặp lại tụt từ 0,8 xuống 0,3 thì xác suất sống qua cả hai khung tụt từ 0,64 xuống 0,09. Dòng quang không có cổng phát hiện nào.
●●● Bản đồ điểm (pointmap)	Tensor $H \times W \times 3$ gán thẳng cho mỗi pixel một điểm ba chiều trong một hệ toạ độ cho trước. Khác bản đồ độ sâu ở chỗ nó đã mang sẵn hệ toạ độ, nên đọc ra hình học mà không cần biết tiêu cự trước.
●●● DUST3R	Nhận hai ảnh chưa hiệu chỉnh, trả về hai bản đồ điểm cùng nằm trong hệ toạ độ của camera một. Tư thế không biến mất mà bị gói vào đầu ra, nên nó đọc ra bằng Procrustes chứ không cần ma trận cơ bản. Đầu ra không có đơn vị mét.
●● MAST3R	DUST3R cộng một đầu ra mô tả đặc trưng dày và một thủ tục ghép tương hỗ nhanh. Kết quả là bộ ghép cặp neo trong ba chiều, nên nó chịu được thay đổi góc nhìn cực đoan — chính chỗ mà mọi bộ mô tả hai chiều chết.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
•• VGGT	Một <i>transformer</i> nhận nhiều ảnh cùng lúc, xen kẽ attention trong từng ảnh và attention trên toàn bộ token của mọi ảnh, rồi trả thẳng ra tư thế, độ sâu và bản đồ điểm. Không còn bước căn chỉnh toàn cục; đổi lại bộ nhớ attention toàn cục đặt trần cứng lên số khung.
• VGGT-SLAM	Ghép các mảnh bản đồ của VGGT trên nhóm xạ ảnh SL(4) thay vì phép biến đổi tương tự, vì với camera chưa hiệu chỉnh thì lớp bất định là xạ ảnh. Bài học: lớp bất định của frontend quyết định đa tạp mà backend phải tối ưu trên đó.
••• Vì sao hạ trọng số không cấu được một tiên nghiệm dày?	Vì sai số của nó tương quan gần như hoàn toàn, còn ma trận thông tin chéo lại đếm 5000 pixel thành 5000 phiếu độc lập. Với 20 phép đo $\sigma = 0,1$ và 5000 tiên nghiệm $\sigma = 0,3$, tiên nghiệm áp đảo gần ba mươi lần. Cách chữa đúng là thêm biến phụ mỗi khung để hấp thụ sai số chung.
•• Bản đồ dày thờ phập phồng và độ lệch gia tốc kể trôi — nghĩa là gì?	Bạn đang nhét độ sâu bất biến tỉ lệ và dịch vào một bộ tối ưu tin rằng cảnh cứng. Tỉ lệ riêng từng khung bị hấp thụ vào tư thế và vận tốc, và độ lệch gia tốc kể — vốn hấp thụ mọi thứ không giải thích được — lãnh phần còn lại.
•• Cùng một ảnh, hai lần chạy cho hai bản đồ độ sâu khác nhau — nguyên nhân?	Mô hình dựa trên khuếch tán như Marigold khởi đầu từ nhiễu ngẫu nhiên, nên nó vốn ngẫu nhiên; người ta phải chạy nhiều lần rồi lấy trung bình. Phân biệt với phi tắt định của thư viện: cố định hạt giống mà vẫn khác thì là thư viện, cố định hạt giống mà giống nhau thì là bản chất mô hình.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
DROID-SLAM [217]	Dùng nguyên vòng lặp tra cứu trên khối tương quan của RAFT làm <i>frontend</i> , rồi nối vào một tầng hiệu chỉnh chùm tia khả vi	Chi tiết ở F/31; đây là bằng chứng rõ nhất rằng ý tưởng của RAFT sống ngoài bài toán dòng quang
MASt3R-SLAM [220]	Đặt thẳng một hệ SLAM dày lên tiên nghiệm bản đồ điểm hai ảnh, và tối ưu trong không gian bản đồ điểm	Công trình 2025; nêu cơ chế, không nêu số
Depth-Anything-V2 [283]	Công thức thầy tổng hợp cộng nhân giả trên ảnh thật — mở rộng dữ liệu thay vì mở rộng kiến trúc	Được dùng rộng rãi làm nguồn độ sâu trong các pipeline dựng lại và điều hướng
Marigold [284], diffusers	Tái dụng mô hình khuếch tán sinh ảnh làm bộ ước lượng độ sâu, huấn luyện trên rất ít dữ liệu	Hạ tầng <i>diffusers</i> bền hơn nhiều so với tên mô hình
SEA-RAFT [287]	Hàm mất mát hỗn hợp Laplace để mô hình được tính đa phương thức và xuất ra độ bất định theo pixel	Đây là cầu nối thẳng sang bài toán hiệp phương sai học được ở F/34
VGGT [290]	Xen kẽ attention trong ảnh và attention toàn cục để bỏ hẳn bước căn chỉnh toàn cục	Công trình 2025; trần số khung là hệ quả cấu trúc của attention toàn cục, mức cụ thể phải tự do

MINI PROJECT SLAM • Đo trần đóng góp của độ sâu học được bằng một thí nghiệm oracle • 1–2 ngày

Mục tiêu trả lời câu “độ sâu học được có thêm được gì cho hệ stereo-inertial của tôi không” *trước* khi cài bất kỳ mạng nào. Nếu một mô hình hoàn hảo cũng chỉ nâng APE dưới sàn nhiễu, cả hướng này đóng lại và bạn tiết kiệm được nhiều tuần.

Dữ liệu	hai chuỗi EuRoC có độ khó khác nhau, chẳng hạn MH_01 và V2_03. Chân trị tư thế đi kèm; nếu bản tải của bạn có bản quét laser phòng Vicon thì dùng nó làm chân trị hình học, nếu không thì dùng bản đồ dày do chính hệ dựng sau khi tối ưu toàn cục làm chân trị xấp xỉ và ghi rõ hạn chế đó.
Các bước	(1) chạy hệ hiện tại, ghi độ sâu của mọi điểm bản đồ tại thời điểm tam giác hoá, vẽ biểu đồ phân bố và đếm tỉ lệ điểm có độ chênh lệch dưới 1 pixel và dưới 2 pixel — đó là phần “thị sai đã cặn”; (2) dựng bản đồ độ sâu oracle cho từng khung khoá từ chân trị; (3) thay độ sâu của riêng nhóm thị sai cặn bằng giá trị oracle, giữ nguyên mọi thứ khác, chạy năm lần mỗi cấu hình; (4) so chênh lệch APE với sàn nhiễu bạn đã đo ở chương 24.
Xong khi	bạn phát biểu được một câu có số chống lưng: “trên chuỗi này, một mô hình độ sâu hoàn hảo nâng APE nhiều nhất X cm, và X lớn hơn hay nhỏ hơn sàn nhiễu”. Kiểm tra chéo: tỉ lệ điểm thị sai cặn phải lớn hơn hẳn ở chuỗi ngoài trời hoặc chuỗi có cảnh xa; nếu hai chuỗi cho cùng tỉ lệ thì khả năng cao bạn đang đếm sai.
Nối với	E/24 cho thiết kế thí nghiệm oracle và cách đọc sàn nhiễu; F/27 cho khung bốn cổng; kết quả của dự án này quyết định có nên đọc kỹ F/32 hay không.

MINI PROJECT SLAM · Dòng quang dày có cứu được các khung mất bám không · 1–2 ngày

Mục tiêu	biến câu “dòng quang chịu được ảnh mờ tốt hơn” thành một ngưỡng tốc độ góc cụ thể cho hệ của bạn, kèm một con số độ trễ đo trên Orin chứ không ước lượng.
Dữ liệu	các đoạn mất bám hoặc số điểm khớp thấp trong log chạy của bạn trên V2_03 và MH_05; chân trị tư thế của EuRoC; chuỗi IMU để lấy tốc độ góc.
Các bước	(1) trích danh sách các cặp khung hình có số điểm khớp dưới ngưỡng phục hồi; (2) với mỗi cặp, chạy princeton-vl/SEA-RAFT ngoại tuyến, lấy trường dòng, chiếu các điểm bản đồ của khung trước theo trường đó và đếm bao nhiêu điểm rơi đúng chỗ theo chân trị tư thế; (3) gán cho mỗi cặp một tốc độ góc trung bình từ IMU và một chỉ số độ mờ, chẳng hạn năng lượng gradient đã chuẩn hoá; (4) vẽ tỉ lệ điểm tìm lại được theo tốc độ góc cho cả hai nhánh, thưa và dày; (5) biên dịch mạng bằng trình suy luận của Jetson ở đúng độ phân giải EuRoC và đo độ trễ.
Xong khi	bạn có hai đường cong trên cùng một trục, một ngưỡng tốc độ góc mà trên đó nhánh dày mới có lãi, và một con số độ trễ đo được. Từ ba thứ đó bạn phát biểu được: “dòng quang dày đáng bật ở $X\%$ số khung hình của chuỗi này, và nó tốn Y ms mỗi lần bật”.
Nối với	F/28 cho phía đặc trưng học được của cùng bài toán; F/34 cho việc biến độ bất định theo pixel của SEA-RAFT thành trọng số thật; dự án trước cung cấp sàn nhiễu để đọc kết quả.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Chỉ ở đúng những chỗ phép đo của bạn đã cặn. Với $fb \approx 50$ của EuRoC và giả định độ chính xác độ chênh lệch 0,2 pixel, sai số stereo tăng theo Z^2 và vượt mười phần trăm ở vài chục mét, trong khi sai số tương đối của mạng không tăng theo khoảng cách — nên hai đường cong bắt buộc cắt nhau, và điểm cắt là thứ bạn phải tự đo. Cộng thêm vùng không có vân, nơi bạn không có phép đo nào để cạnh tranh. Ngoài hai chỗ đó, nó chỉ thêm sai lệch có hệ thống.
- A2. Nó dựng kim tự tháp trên *khối tương quan*, và chỉ gộp trung bình hai chiều thuộc ảnh thứ hai. Lưới của ảnh một giữ nguyên độ phân giải ở mọi mức, nên vật nhỏ không bao giờ bị thu nhỏ tới mức biến mất. Chiến lược thô tới mịn cổ điển thu nhỏ chính bức ảnh, nên nó đánh mất vật nhỏ trước khi kịp ước lượng.
- A3. Tư thế nằm trong chính đầu ra. Cả hai bản đồ điểm đều được dự đoán trong hệ toạ độ của camera một. Vì thế phép biến đổi cứng khớp bản đồ điểm của ảnh hai với hình học của camera hai chính là tư thế tương đối, và nó có nghiệm dạng đóng. Bước ước lượng bị đối thành bước đọc.
- A4. Vì bạn đang chống lại một tỉ số đếm: 5000 so với 20 là lợi thế 250 lần, nên muốn triệt tiêu nó phải nhân phương sai lên 250 lần, tức nhân độ lệch chuẩn lên gần 16 lần. Với $\sigma_m = 0,1$ thì bạn

phải khai báo $\sigma_p \gg 1,6$ m, mức mà tiên nghiệm không còn nói gì. Phép tính ba mươi giây là so hai đại lượng N/σ^2 .

- A5. Vì nó khởi đầu từ nhiễu ngẫu nhiên, nên cùng một ảnh cho hai đầu ra khác nhau. Bạn vừa dựng lại một sàn nhiễu ngay giữa pipeline, và lần này không có bug nào để sửa — khác hẳn với thứ tự duyệt container hay tranh chấp luồng. Một mô hình hồi quy thì tất định theo trọng số, nên nó không mang thêm nguồn phi tất định nào.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn cần biết mọi vật trong phòng cách bạn bao xa. Bạn có hai con mắt cách nhau mười một phân, nên với thứ ở gần bạn đo được khá chuẩn: hai mắt nhìn lệch nhau, độ lệch càng lớn thì vật càng gần. Nhưng với thứ ở cuối hành lang, độ lệch giữa hai mắt gần như bằng không, và bạn hết cách. Bây giờ có một người bạn chưa từng cầm thước bao giờ, nhưng đã nhìn hàng triệu căn phòng trong đời. Hỏi anh ta ghế kia xa bao nhiêu, anh ta nói ngay, và anh ta nói cho cả nghìn điểm trong phòng cùng lúc. Anh ta luôn đúng về chuyện cái nào gần hơn cái nào. Anh ta gần như luôn sai về chuyện bao nhiêu mét, và cái sai ấy đổi mỗi khi bạn đưa cho anh ta một bức ảnh khác.

Chuyện gì xảy ra nếu bạn ghi lời anh ta vào cùng cuốn sổ với các phép đo của mình rồi lấy trung bình? Bạn đo hai mươi lần, anh ta đoán năm nghìn lần, nên trung bình sẽ nghiêng hẳn về phía anh ta. Tệ hơn: khi anh ta sai, anh ta sai cả năm nghìn lần theo cùng một hướng, nên việc có nhiều số không giúp gì cả. Và điều nguy hiểm nhất là anh ta không bao giờ nói “chỗ này tôi không nhìn rõ”. Anh ta luôn có câu trả lời, kể cả khi nhìn vào một tấm gương.

Cách dùng anh ta cho đúng vì thế rất hẹp: chỉ hỏi ở cuối hành lang, chỗ thước của bạn đã hết tầm; và ghi lời anh ta vào một trang riêng, đừng trộn vào trang có số đo.

Chỗ không rút gọn được: chuyện “cái sai đổi mỗi khi đưa một bức ảnh khác”. Mọi cách nói gọn hơn đều làm sai lệch, vì đây không phải chuyện anh ta đoán lúc trúng lúc trượt. Cái đổi là *đơn vị* anh ta đang dùng, và nó đổi âm thầm giữa hai bức ảnh chụp cách nhau một phần ba mươi giây. Một bức tường đứng yên vì thế lúc gần lúc xa, và hệ thống của bạn — vốn tin rằng tường không di chuyển — buộc phải kết luận rằng chính bạn mới là người đang nhúc nhích.

Một câu: Chúng cho bạn hàng nghìn phỏng đoán tự tin ở chỗ bạn chỉ có vài chục phép đo thật, nên hãy chỉ hỏi chúng ở đúng chỗ phép đo của bạn đã hết.

30.8 Kết nối

[F/27-dat-hoc-sau-vao-dau](#) là bộ lọc mà chương này áp dụng: bốn cổng — bài toán có thật, đo được, vừa ngân sách, hòng có kiểm soát — xuất hiện lại ở đây dưới dạng cây quyết định. [F/28-dac-trung-va-ghep-cap](#) là mặt đối lập trực tiếp: nó giữ cấu trúc thừa và thay chất lượng của từng điểm, còn chương này bỏ luôn cấu trúc thừa. Đọc hai chương cạnh nhau là cách nhanh nhất để thấy đánh đổi giữa “ít phép đo nhưng kiểm chứng được” và “nhiều phỏng đoán nhưng không kiểm chứng được”. [F/29-dinh-vi-lai-va-dong-vong](#) nhận từ chương này một lối thoát cụ thể cho tầng kiểm chứng hình học: ghép cặp neo trong ba chiều không cần hai mảnh ảnh trông giống nhau. [F/31-backend-kha-vi-va-dau-cuoi](#) là bước tiếp theo tự nhiên, vì DROID-SLAM chính là vòng lặp tra cứu của RAFT nối vào một tầng hiệu chỉnh chùm tia khả vi. [F/32-ban-do-neural](#) thụ đầu ra của chương này: một bản đồ dày cần một nguồn độ sâu, và chất lượng nguồn đó quyết định chất lượng bản đồ. [F/34-imu-bat-dinh-va-trien-khai](#) là nơi kết luận quan trọng nhất của chương được xử lý đàng hoàng: muốn dùng một tiên nghiệm dày thì phải có trọng số đúng, và trọng số đúng phải được huấn luyện chứ không được đoán. Về phía nền: [C/16/02-bieu-dien-3d-va-depth](#) là tiền đề hình học, [C/15/03-flow-tracking-va-rang-buoc](#) là tiền đề về dòng quang, [B/11-gioi-han-tinh-toan](#) cho khung phân tích bộ nhớ và băng thông, và [E/24-thi-nghiem-va-ablation](#) cho thí nghiệm oracle mà cả hai mini project của chương đều dựa vào.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao việc nhân toàn bộ cảnh và toàn bộ quãng đường camera với cùng một hệ số lại khiến độ sâu đơn ảnh trở thành bài toán *không giải được* chứ không phải bài toán khó, và điều đó ràng buộc gì lên mọi mô hình trong mục 2?
2. Metric3D ghim ẩn số tiêu cự ở bước tiền xử lý thay vì để mạng học. Chỗ nào trong công việc SLAM của bạn dùng đúng thủ thuật đó, và cái giá tương ứng ở cả hai chỗ là gì?
3. Nếu bạn chỉ được giữ *một* trong hai thành phần của RAFT — khối tương quan toàn cặp, hoặc vòng lặp tra cứu bằng khối hồi tiếp — thì bỏ cái nào ít hại hơn cho bài toán chuyển động nhanh, và vì sao?
4. Dòng quang dày luôn trả lời, kể cả ở vùng bị che. Vì sao tính chất đó lại nguy hiểm hơn cho một hệ SLAM so với việc bộ ghép cặp thừa trả về không điểm nội biên?
5. DUS3R xoá bước ước lượng tư thế nhưng không xoá được tính không xác định được của tỉ lệ. Vì sao một cái xoá được còn cái kia thì không?
6. Bạn hạ trọng số của một tiên nghiệm độ sâu xuống mười lần mà nghiệm vẫn dịch mấy centimet. Điều đó nói gì về cấu trúc sai số của tiên nghiệm, và cách chữa đúng là gì thay vì hạ tiếp?
7. Trần số khung của VGGT đến từ bộ nhớ attention toàn cục, còn trần của DUS3R đến từ số cặp. Hai trần đó dẫn tới hai chiến lược chia mảnh bản đồ khác nhau như thế nào?

Backend khả vi và hệ VO/SLAM đầu-cuối (Differentiable Backends and End-to-End Systems)

PHỤ THUỘC F/27-dat-hoc-sau-vao-dau (khung bốn cổng để quyết định có nên thay một khối); A/01-toan-toi-thieu/02-vi-phan-va-toi-uu (Gauss-Newton, Levenberg-Marquardt, quy tắc chuỗi); C/15-thi-giac-khong-thoi-gian/03-flow-tracking-va-rang-buoc (dòng quang và toán tử cập nhật hồi quy); F/28-dac-trung-va-ghep-cap là nửa frontend của cùng câu chuyện.

TÓM TẮT

Bạn đã thạo Gauss-Newton, Levenberg-Marquardt và đồ thị nhân tử. Chương này không dạy lại chúng. Nó nói về đúng một thứ mà chữ “khả vi” thêm vào: cho *gradient* chạy *xuyên qua* lời giải của bài toán tối ưu, để những con số bạn đang đặt bằng tay được huấn luyện theo lỗi quỹ đạo cuối cùng. Đọc xong bạn viết được đạo hàm của một nghiệm arg min bằng định lý hàm ẩn, viết ra được công thức chi phí bộ nhớ của lượt ngược qua một lớp tối ưu, và đọc DROID-SLAM hay CodeSLAM mà biết ngay chỗ nào là hình học cũ, chỗ nào là phần học được. Nếu chỉ nhớ một điều: khả vi *không* làm bộ giải tốt lên, nó chỉ biến các nút vặn thủ công thành tham số học được — và cái giá là bộ nhớ lượt ngược, độ ổn định số ở đúng những khung hình khó, cùng gần hết khả năng chẩn đoán khi hệ hỏng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **bài toán bình phương tối thiểu phi tuyến** — tìm x làm nhỏ nhất tổng bình phương của các phần dư $r_i(x)$. Trong SLAM, x là tư thế và điểm bản đồ, còn r_i là sai lệch chiếu lại của một quan sát.
- **ma trận thông tin (information matrix)** — nghịch đảo hiệp phương sai của một phép đo, và cũng là trọng số của cạnh đó trong bài toán bình phương tối thiểu. Đặt lớn nghĩa là “tôi tin phép đo này”.
- **Hessian H** — ma trận hệ phương trình chuẩn của Gauss-Newton, $H = J^T \Sigma^{-1} J$. Nghịch đảo của nó xấp xỉ hiệp phương sai của nghiệm, tức thứ bạn dùng khi biên hoá một khung hình.
- **tự do đo (gauge freedom)** — các hướng mà bài toán không xác định được. Dịch cả quỹ đạo đi một khoảng thì lỗi không đổi. Theo những hướng đó H suy biến, nên phải neo gốc toạ độ hoặc thêm damping.
- **damping λ** — số cộng vào đường chéo của H trong Levenberg-Marquardt. Nó vừa làm hệ khả nghịch, vừa rút ngắn bước đi khi mô hình tuyến tính hoá không đáng tin.
- **đạo hàm tự động (autodiff)** — kỹ thuật để máy tính đạo hàm của một chương trình bằng cách áp quy tắc chuỗi lên từng phép toán. **lan truyền ngược (backpropagation)** là autodiff chạy theo chiều ngược.
- **lượt xuôi và lượt ngược** — lượt xuôi là một lần chạy mạng để ra kết quả; lượt ngược là một lần đi ngược để tính đạo hàm. Lượt ngược cần các giá trị trung gian của lượt xuôi, nên phải giữ chúng trong bộ nhớ.
- **tự giám sát (self-supervised)** — huấn luyện không cần nhãn do người tạo. Tín hiệu dạy đến từ chính dữ liệu, ví dụ “dừng lại ảnh này từ ảnh kia thì phải khớp”.

CÂU HỎI MỞ ĐẦU

- Q1. Backend của bạn đã tối ưu tốt rồi. Vậy chữ “khả vi” còn thêm được gì, và nếu nó không làm Gauss-Newton hội tụ nhanh hơn thì nó bán cái gì?
- Q2. Đạo hàm của một nghiệm arg min theo tham số đầu vào tính được mà không cần lưu bước lặp nào. Vì sao lại có chuyện đó, và điều kiện nào phải đúng để nó không sai?
- Q3. Quyết định inlier hay outlier là một phép so sánh với ngưỡng, nên đạo hàm của nó bằng 0 gần như ở mọi nơi. Vậy một hệ đầu-cuối học cách loại điểm sai bằng cách nào?
- Q4. DROID-SLAM chính xác hơn hẳn nhiều hệ cổ điển trên *benchmark* công khai. Vì sao nó vẫn không phải lựa chọn cho một robot chạy Jetson Orin, và cái gì chặn nó?
- Q5. Nén cả bản đồ độ sâu vào 32 con số rồi tối ưu chung với tư thế nghe rất gọn. Ý tưởng đó ra đời năm 2018 và tới nay vẫn chưa thắng. Nó hỏng ở chỗ nào?

31.1 Đặt vấn đề

Backend của bạn không hỏng. Nó tuyến tính hoá, nó dùng phần bù Schur, nó chạy nhân Huber, nó biên hoá khung hình cũ. Đưa cho nó các phần dư đúng và các trọng số đúng thì nó trả về nghiệm tốt nhất theo nghĩa bình phương tối thiểu. Vấn đề nằm ở chỗ khác: **ai quyết định các trọng số đó.**

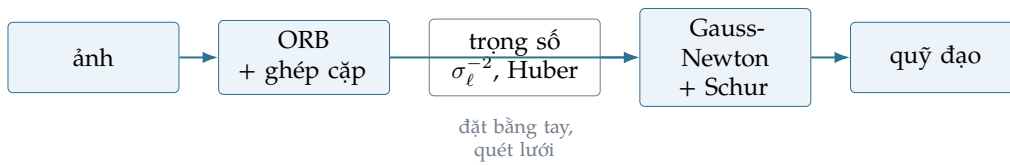
Hãy liệt kê thật cụ thể, trong một hệ kiểu ORB-SLAM3 [40]:

- **Trọng số cạnh chiếu lại** — ma trận thông tin là $\sigma_\ell^{-2} I$, với σ_ℓ suy từ hệ số tỉ lệ của kim tự tháp ORB. Nó chỉ phụ thuộc vào *tăng* mà điểm được phát hiện. Nó không biết mảnh ảnh đó có bị mờ không, có nằm trên vệt loá không, có nằm trên một hoa văn lặp không.
- **Bề rộng nhân robust** — ngưỡng χ^2 5,991 cho cạnh hai bậc tự do và 7,815 cho cạnh ba bậc. Đây là phân vị 95% của một giả định phân phối chuẩn, và giả định ấy hiếm khi được kiểm lại trên dữ liệu thật.
- **Nhiều IMU** — lấy từ tờ thông số cảm biến, cộng thêm một bước ngẫu nhiên cho độ trôi. Cùng một con số cho lúc đứng yên và lúc quay đầu 200 độ mỗi giây.
- **Các ngưỡng rời rạc** — tỉ lệ khoảng cách mô tả, ngưỡng inlier của RANSAC, chính sách chèn khung then chốt.

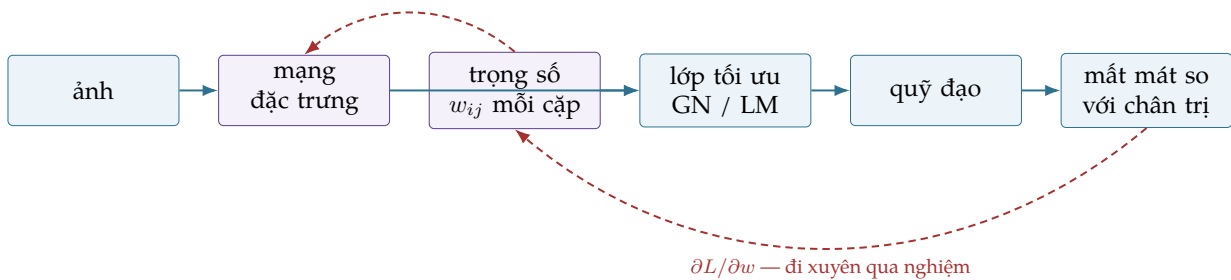
Mọi con số trên đều trả lời cùng một câu hỏi: *tôi nên tin phép đo này bao nhiêu*. Và mọi con số trên đều được đặt một lần, toàn cục, từ một tờ thông số hoặc một bài báo. Trong khi đó bạn có một đại lượng thật sự quan tâm ở cuối đường ống, là sai số quỹ đạo tuyệt đối. Giữa các nút vận và đại lượng đó **không có đạo hàm nào**. Nên bạn đóng khoảng trống ấy bằng cách duy nhất còn lại: quét lưới, chạy lặp nhiều lần, đọc bảng. Đó chính là lý do tồn tại của mọi *benchmark* harness.

Chữ **khả vi** (*differentiable*) trong chương này là lời hứa rằng đạo hàm ấy tồn tại được. Nếu toàn bộ đường đi từ điểm ảnh tới quỹ đạo là một đồ thị phép tính khả vi, thì đạo hàm của APE theo trọng số cạnh là một đại lượng tính được. Khi đó các trọng số trở thành tham số học được. Hình 31.1 vẽ đúng sự khác biệt: mũi tên xuôi giống nhau ở hai phía, mũi tên ngược chỉ tồn tại ở phía dưới.

Hệ cổ điển



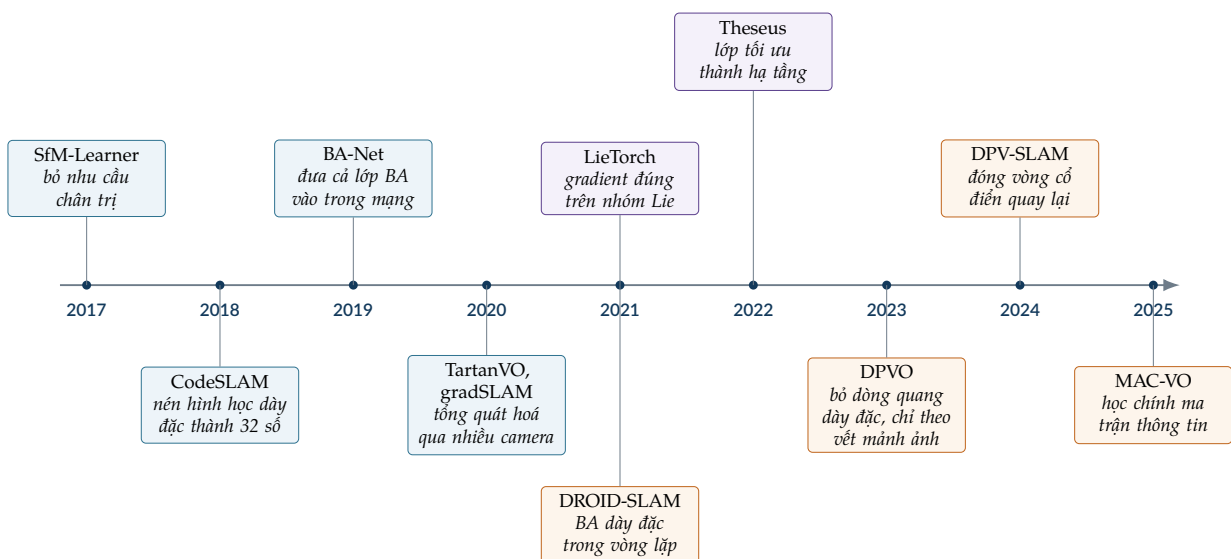
Hệ khả vi



Hình 31.1. Cái mà chữ “khả vi” thêm vào. Chiều xuôi, vẽ bằng mũi tên liền, giống hệt nhau ở hai hệ: đặc trưng, trọng số, bộ giải, quỹ đạo. Khác biệt duy nhất là mũi tên đứt màu đỏ. Nó đi ngược *xuyên qua* lời giải của bài toán tối ưu, nhờ đó các trọng số w_{ij} và mạng sinh ra chúng được huấn luyện theo lỗi quỹ đạo cuối cùng, thay vì được đặt bằng tay từ tờ thông số cảm biến.

Nói gọn giá trị của chương: học xong bạn phân biệt được ba nhóm hay bị gộp làm một. Nhóm thứ nhất là *lớp tối ưu khả vi*, một mảnh hạ tầng có thể lấy riêng. Nhóm thứ hai là *hệ đầu-cuối* kiểu DROID-SLAM, một kiến trúc trọn gói và đắt. Nhóm thứ ba là *VO tự giám sát*, vốn là một cách huấn luyện chứ không phải một hệ SLAM. Ba thứ đó có chi phí và rủi ro rất khác nhau. Trộn chúng lại là cách nhanh nhất để đánh giá sai cả ba.

Hình 31.2 xếp các công trình của chương lên một trục thời gian, và ghi cạnh mỗi mốc *nút thắt mà nó gỡ*. Đọc hình đó trước sẽ nhanh hơn đọc tuần tự, vì cả chương chỉ là một chuỗi lời đáp cho những nút thắt liên tiếp nhau.



Hình 31.2. Dòng thời gian của chương, mỗi mốc ghi kèm nút thắt mà nó gỡ. Hộp tím là hạ tầng, tức thứ dùng lại được ở mọi hệ. Hộp cam là các hệ đầu-cuối, và ba mốc cuối cho thấy hướng đi chung của chúng: bớt dây đặc, mượn lại đóng vòng cổ điển, rồi chuyển mục tiêu học từ đặc trưng sang ma trận thông tin — tức là càng ngày càng gần với một backend cổ điển được cấp trọng số tốt hơn.

31.2 Đạo hàm xuyên qua một nghiệm tối ưu (Differentiating Through an Argmin)

A. Bài toán phát biểu chính xác

Gọi θ là mọi thứ mạng sinh ra: trọng số cạnh, mục tiêu dòng quang, hệ số damping. Gọi

$$x^*(\theta) = \arg \min_x E(x, \theta)$$

là nghiệm mà bộ giải trả về, với x là tư thế và hình học. Ở cuối có một hàm mất mát $L(x^*(\theta))$, thường là sai lệch so với quỹ đạo chân trị. Ta cần $\partial L / \partial \theta$.

Chỗ khó là x^* không có công thức đóng. Nó là kết quả của một vòng lặp. Có đúng hai cách lấy đạo hàm qua nó, và hai cách đánh đổi ngược nhau.

B. Cách thứ nhất: trải phẳng vòng lặp

Cách thô là coi K bước lặp như một mạng K tầng rồi để autodiff làm việc:

$$x_{k+1} = x_k - (H_k + \lambda I)^{-1} g_k.$$

Mỗi bước là một biểu thức khả vi, nên cả vòng lặp cũng vậy. Cách này gọi là **trải phẳng** (*unrolling*).

Ưu điểm của nó là tính trung thực. Đạo hàm thu được là đạo hàm của *thuật toán thật sự đã chạy*, kể cả khi thuật toán đó mới chạy năm bước và chưa hội tụ. Nhược điểm là bộ nhớ. Lướt ngược cần mọi giá trị trung gian của lượt xuôi. Bạn phải giữ cả K bộ Jacobian, K bản phân tích Cholesky, và mọi bản đồ đặc trưng đã sinh ra chúng.

Chi phí ấy nên nhớ dưới dạng công thức, vì công thức mới là thứ dùng lại được. Gọi K là số bước lặp, N là số khung trong cửa sổ, C là số kênh của bản đồ đặc trưng, và HW là số vị trí ở độ phân giải đang chạy. Bộ nhớ lướt ngược tỉ lệ với tích $K N C H W$ — tuyến tính theo *cả bốn* thừa số cùng lúc, và một bước lặp hồi quy còn giữ vài tensor cùng cỡ nên hằng số nhân phía trước cũng không nhỏ. Đó là lý do các hệ trải phẳng thật đều huấn luyện với cửa sổ ngắn và độ phân giải thấp: N và HW là hai thừa số duy nhất người dùng cắt được mà không phải đổi kiến trúc. Nếu bạn muốn một con số gigabyte cho hệ của mình thì phải tự đo; công thức chỉ nói hình dạng, không nói độ lớn.

C. Cách thứ hai: định lý hàm ẩn

Cách thứ hai không lưu bước lặp nào. Nó dùng đúng một sự kiện: tại nghiệm, **gradient** (*gradient*) theo x bằng không.

$$g(x^*, \theta) = \left. \frac{\partial E}{\partial x} \right|_{x^*} = 0.$$

Đây là một hệ phương trình ràng buộc x^* với θ . Lấy vi phân toàn phần hai vế theo θ :

$$\underbrace{\frac{\partial^2 E}{\partial x^2}}_H \frac{\partial x^*}{\partial \theta} + \underbrace{\frac{\partial^2 E}{\partial x \partial \theta}}_B = 0 \quad \implies \quad \frac{\partial x^*}{\partial \theta} = -H^{-1} B.$$

Đây là **định lý hàm ẩn** (*implicit function theorem*) áp cho điều kiện tối ưu. Nó nói rằng nghiệm trượt đi theo hướng nào khi ta đẩy tham số, và câu trả lời chỉ phụ thuộc hai thứ: độ cong H tại nghiệm, và mức độ tham số đẩy vào gradient qua B .

Với hàm mất mát cuối, ta không cần cả ma trận $\partial x^* / \partial \theta$. Chỉ cần một tích vectơ:

$$\frac{\partial L}{\partial \theta} = - \left(\frac{\partial L}{\partial x} \right)^\top H^{-1} B = -v^\top B, \quad \text{với } H v = \frac{\partial L}{\partial x}.$$

Đọc dòng cuối cho kỹ, vì đó là điểm mấu chốt của cả chương. **Lượt ngược chỉ tốn thêm đúng một lần giải hệ, với chính ma trận H mà lượt xuôi đã phân tích rồi.** Với cửa sổ 8 khung, hệ camera rút gọn chỉ là 48×48 , tức nhỏ hơn nhiều bậc so với chính lượt xuôi vừa chạy, nên lượt ngược không thêm khoản chi phí nào đáng kể. Bộ nhớ cho vòng lặp bằng không, vì không bước lặp nào được lưu.

Còn một cách đọc nữa mà kỹ sư SLAM sẽ thấy quen ngay. H^{-1} chính là hiệp phương sai của nghiệm, đúng đại lượng bạn đã tính khi biên hoá một khung hình. Cổ máy cần cho lượt ngược đã nằm sẵn trong mã nguồn của bạn. Ý tưởng lấy đạo hàm qua một bài toán tối ưu không sinh ra từ thị giác; nó đến từ dòng lớp tối ưu khả vi trong học máy [292], [293], và cùng một thủ thuật cũng là thứ làm cho mô hình cân bằng sâu chạy được [294].

TRỰC GIÁC

Hãy hình dung nghiệm là một hòn bi nằm dưới đáy một cái bát. Bạn không quan tâm nó lăn xuống đáy bằng đường nào. Bạn chỉ quan tâm một việc: nghiêng cái bát đi một chút thì hòn bi dịch bao nhiêu. Trả lời câu đó chỉ cần biết *độ cong của cái bát tại đáy* và *lực nghiêng đáy vào đâu*. Đó đúng là H^{-1} và B . Cách trải phẳng thì ngược lại: nó quay phim toàn bộ đường lăn rồi tua ngược. Tốn băng — nhưng nếu hòn bi chưa lăn tới đáy thì đoạn phim vẫn dùng được, còn cái bát thì không.

D. Ví dụ tính tay được

Lấy bài toán nhỏ nhất còn giữ nguyên bản chất: một ẩn, hai phép đo, một trọng số.

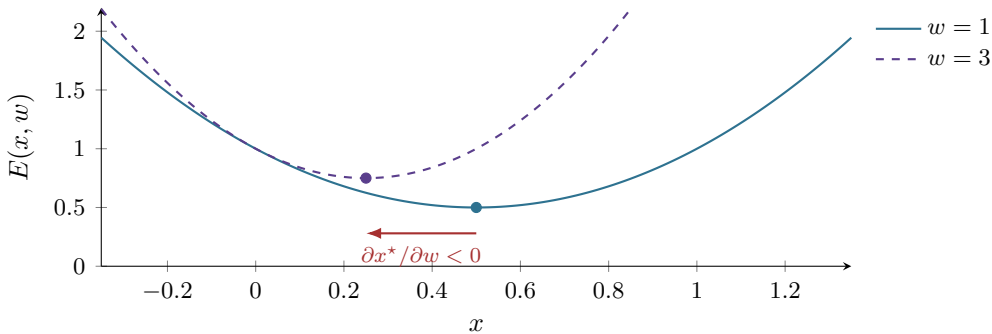
$$E(x, w) = w(x - a)^2 + (x - b)^2.$$

Nghiệm có công thức đóng, $x^* = (wa + b)/(w + 1)$. Bây giờ tính lại bằng định lý hàm ẩn. Ta có $H = 2(w + 1)$ và $B = \partial^2 E / \partial x \partial w = 2(x - a)$, nên

$$\frac{\partial x^*}{\partial w} = -\frac{2(x^* - a)}{2(w + 1)} = -\frac{x^* - a}{w + 1}.$$

Thay $a = 0, b = 1, w = 1$. Khi đó $x^* = 0,5$ và đạo hàm bằng $-0,25$. Kiểm tra trực tiếp: $x^* = 1/(w + 1)$, đạo hàm là $-1/(w + 1)^2 = -1/4$. Khớp.

Con số $-0,25$ nói điều gì bằng lời thường? Tăng trọng số của phép đo a lên 0,1 thì nghiệm dịch về phía a khoảng 0,025. Nếu hàm mất mát cuối muốn nghiệm nhỏ đi, **gradient** sẽ bảo mạng “hãy tin phép đo a hơn”. Đó chính xác là thứ một hệ đầu-cuối học, chỉ khác là với hàng chục nghìn phép đo cùng lúc. Hình 31.3 vẽ lại cùng ý đó bằng hình học.



Hình 31.3. Định lý hàm ẩn nhìn bằng mắt, cho $E = wx^2 + (x - 1)^2$. Tăng trọng số w của phép đo tại $x = 0$ làm đường cong dốc hơn về phía trái, nên đáy trượt từ $x^* = 0,5$ sang $x^* = 0,25$. Đạo hàm $-H^{-1}B$ chính là độ dốc của đường trượt ấy. Nó chỉ cần độ cong tại đáy, chứ không cần biết bộ giải đã đi đường nào tới đó.

E. Ba điều kiện, và cả ba đều bị vi phạm trong SLAM

Định lý hàm ẩn đẹp nhưng có điều kiện. Nó cần x^* là điểm dừng thật, cần H khả nghịch, và cần E đủ trơn quanh nghiệm. Trong SLAM cả ba đều lung lay.

- **H suy biến theo cấu tạo.** Bài toán đơn ảnh có bảy hướng tự do đo. Ngay cả stereo-inertial vẫn còn vài hướng nếu chưa neo gốc. Theo những hướng đó H không khả nghịch. Cách chữa duy nhất là neo gốc hoặc cộng damping, và *giá trị damping đổi thì đạo hàm cũng đổi*. Bạn đang huấn luyện theo một **gradient** phụ thuộc vào một lựa chọn số học, không phải vào vật lý.
- **Bộ giải bị cắt ngắn.** Năm bước LM từ một khởi tạo tồi không cho một điểm dừng. Khi đó định lý hàm ẩn trả về đạo hàm của một bài toán mà bạn *chưa* giải. Sai lệch này hiếm khi được đo, và nó lớn nhất đúng ở những khung khó.
- **Có những quyết định không khả vi.** Đây là điều sâu nhất trong chương. So sánh χ^2 với ngưỡng, chọn inlier trong RANSAC, quyết định chèn khung then chốt — tất cả đều là hàm bậc thang. Đạo hàm của chúng bằng 0 hầu khắp nơi và không tồn tại tại chỗ nhảy. **Gradient** không nhìn thấy chúng.

Gạch đầu dòng cuối giải thích được toàn bộ thiết kế của các hệ trong chương này. Vì **không thể học một quyết định cứng**, mọi hệ đầu-cuối đều thay quyết định cứng bằng một *trọng số mềm liên tục*. DROID-SLAM không xuất ra mặt nạ inlier; nó xuất ra một con số tin cậy cho từng điểm ảnh. MAC-VO không xuất ra nhãn “điểm này tốt”; nó xuất ra một hiệp phương sai. Đó không phải một lựa chọn thẩm mỹ mà là hệ quả bắt buộc của việc muốn có đạo hàm.

→ cùng nguyên lý với B/09-thiet-ke-ham-mat-mat — lại là quy tắc “muốn học được thì phải làm cho trơn”, cùng lý do focal loss thay cho việc chọn cứng mẫu khó

F. Vì sao gradient nổ đúng ở khung khó — một phép tính EuRoC

Điều kiện “ H khả nghịch” nghe như một chi tiết kỹ thuật. Thực ra nó là chế độ hỏng nặng nhất trong huấn luyện, và nó tính ra được bằng số liệu bạn đã quen.

Xét một điểm bản đồ quan sát bằng stereo. Bỏ qua các hướng khác, giữ hai bậc tự do: dịch ngang và độ sâu. Với tiêu cự f và đường cơ sở b , độ bất định của hai hướng là

$$\sigma_x = \frac{Z}{f} \sigma_u, \quad \sigma_Z = \frac{Z^2}{fb} \sigma_d,$$

với σ_u là sai số định vị điểm ảnh và σ_d là sai số disparity. Lấy $\sigma_d = \sigma_u$ cho gọn, tỉ số rút lại còn một biểu thức rất sạch:

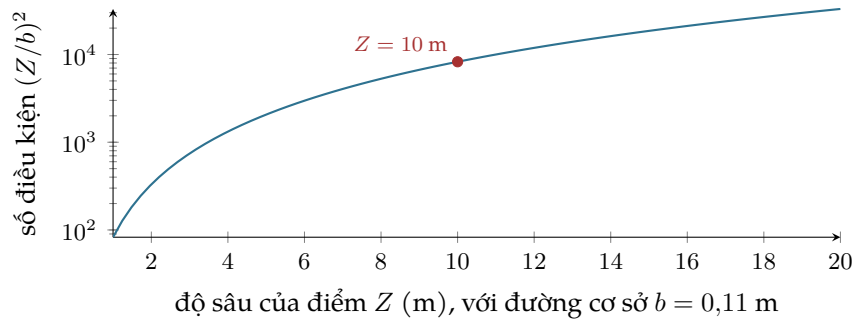
$$\frac{\sigma_Z}{\sigma_x} = \frac{Z}{b}.$$

Bây giờ thay số vào công thức — đây là phép thay số, không phải phép đo trên máy nào. Lấy thông số công khai của EuRoC, $b = 0,11$ m và $f \approx 460$ px, rồi *giả sử* $\sigma_u = 0,5$ px và một điểm ở $Z = 10$ m. Khi đó $\sigma_Z/\sigma_x = Z/b$ cho tỉ số cỡ chín chục lần: bất định theo hướng độ sâu lớn hơn bất định theo hướng ngang gần hai bậc độ lớn.

Ma trận thông tin là nghịch đảo bình phương của các độ lệch đó, nên khối 2×2 của H có số điều kiện khoảng $(Z/b)^2 \approx 8300$. Đây là con số quyết định. Trong công thức $v = H^{-1} \partial L / \partial x$, thành phần **gradient** theo hướng độ sâu bị nhân lên khoảng tám nghìn lần so với thành phần theo hướng ngang. Nói bằng lời: **ước lượng càng ít ràng buộc một hướng, thì đạo hàm theo hướng đó càng lớn**, vì một thay đổi nhỏ ở trọng số làm nghiệm trượt rất xa theo hướng yếu.

Hệ quả cho huấn luyện thì cụ thể và khó chịu. Các khung thị sai thấp, chuyển động gần như quay thuần túy, và ảnh mờ đều làm Z/b hiệu dụng lớn lên. Đó lại đúng là những khung bạn muốn mạng học từ chúng. Nên **gradient** từ mẫu quý nhất cũng là **gradient** nhiều nhất và dễ nổ nhất.

Mọi cách chữa thông dụng — cắt chuẩn *gradient*, tăng damping, bỏ qua mẫu có H xấu — đều làm giảm ảnh hưởng của đúng nhóm mẫu ấy. Hình 31.4 cho thấy số điều kiện tăng theo bình phương độ sâu.



Hình 31.4. Số điều kiện của khối hai bậc tự do (ngang, sâu) cho một điểm stereo trên cấu hình EuRoC. Đây là đường của công thức $(Z/b)^2$, không phải một tập điểm đo. Nó chính là hệ số khuếch đại mà H^{-1} áp lên *gradient* theo hướng độ sâu trong lượt ngược. Điều đáng nhớ là hình dạng: hệ số tăng theo bình phương độ sâu, nên tăng độ sâu gấp đôi thì nó tăng gấp bốn. Đây là lý do định lượng khiến huấn luyện qua một lớp tối ưu mất ổn định trên đúng những khung ít thị sai.

G. Khối nào trong hệ của bạn học được, khối nào không

Gộp hai kết luận vừa rồi — *gradient* chỉ chảy qua thứ liên tục, và nó nổ ở chỗ H xấu — ta có một bản kiểm kê rất cụ thể cho một hệ kiểu ORB-SLAM3. Bảng 31.1 là bảng đáng dán lên tường trước khi bắt đầu bất cứ thí nghiệm nào trong chương này.

Bảng 31.1. Kiểm kê các khối của một hệ stereo-inertial cổ điển theo tiêu chí “*gradient* có chảy qua được không”. Hai dòng in đậm là hai chỗ rẻ nhất để bắt đầu, vì chúng đã liên tục sẵn và không đòi thay kiến trúc.

Khối	Đầu ra	Liên tục?	Học được thể nào
Trích đặc trưng ORB	vị trí + mô tả nhị phân	không	Phải thay bằng bản đồ nhiệt và mô tả số thực; đó là chương 28
Ghép cặp + kiểm tra tỉ lệ	tập cặp khớp	không	Thay bằng ma trận gán mềm rồi mới lấy <i>gradient</i> qua nó
RANSAC và cổng χ^2	mặt nạ inlier	không	Thay hẳn bằng trọng số liên tục; không có cách nào khác
Ma trận thông tin cạnh	số thực dương	có	Học trực tiếp; đây là điểm vào rẻ nhất của cả chương
Bề rộng nhân robust	số thực dương	có	Học được, nhưng <i>gradient</i> yếu vì nhân robust bão hoà ở phần đuôi
Hiệu chỉnh chùm tia	nghiệm	có	Đi xuyên qua bằng định lý hàm ẩn, với điều kiện đã neo gốc
Chính sách chèn khung then chốt	quyết định đúng/sai	không	Không; phải giữ luật viết tay hoặc học bằng cách khác hẳn
Lấy ứng viên đóng vòng	xếp hạng + ngưỡng	một nửa	Học được phần <i>embedding</i> , không học được phần ngưỡng; chương 29
Kiểm chứng Sim3 + RANSAC	quyết định đúng/sai	không	Không
Tối ưu đồ thị tư thế	nghiệm	có	Đi xuyên qua được, nhưng đồ thị lớn nên lượt ngược đắt
Khởi tạo IMU	nghiệm + cổng chấp nhận	một nửa	Chỉ phần nghiệm; cổng chấp nhận vẫn rời rạc

Bảng này trả lời luôn câu hỏi “đầu-cuối tới mức nào”. Sáu trên mười một khối không cho *gradient* đi qua, và trong sáu khối đó có cả đóng vòng lẫn chèn khung then chốt — hai thứ chi phối sai số dài hạn của bạn. Nên không có hệ nào trong chương thật sự đầu-cuối theo nghĩa đen. Chúng chỉ đầu-cuối trên phần vốn đã trơn.

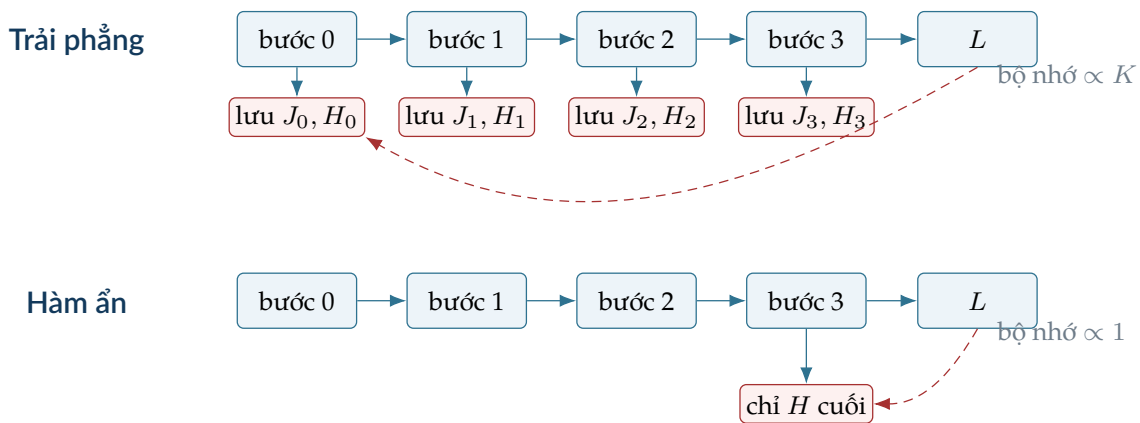
H. Ba cái giá, nói thẳng

Bảng 31.2 tổng kết cái được và cái mất. Cột cuối là cột đáng đọc nhất.

Bảng 31.2. Khả vi mua được gì và trả bằng gì. Cột “hông khi nào” là cột quyết định: nó cho biết trong hoàn cảnh nào một hệ khả vi trở nên tệ hơn hệ cổ điển, chứ không chỉ đắt hơn.

Chiều	Được gì	Trả bằng gì	Hông khi nào
Trọng số đo lường	Trọng số theo từng cặp khớp, biết mảnh ảnh này mờ hay sắc	Phải huấn luyện; cần chân trị quỹ đạo	Ra ngoài phân phối huấn luyện: trọng số vẫn tự tin, chỉ là sai
Bộ nhớ lượt ngược	—	Trải phẳng: tỉ lệ với tích của số bước, số khung, số kênh và số vị trí. Hàm ẩn: không phụ thuộc số bước	Trải phẳng buộc cửa sổ huấn luyện ngắn, nên không học được hành vi dài hạn
Ổn định số	—	Phải damping, cắt <i>gradient</i> , giới hạn số bước	H gần suy biến: thị sai thấp, quay thuần túy, ảnh mờ — đúng các khung cần học nhất
Chẩn đoán	—	Mất phần lớn	Quỹ đạo lệch mà không có χ^2 , không có số inlier, không có cạnh nào để đổ lỗi
Tính lặp lại	—	Phi tắt định của thư viện lọt vào trong bộ ước lượng	Hai lần chạy cho hai quỹ đạo; hết phân biệt được lỗi thuật toán với nhiễu

Chiều thứ tư đáng nói rõ hơn, vì với một người bảo trì hệ thống nó thường là chiều đắt nhất. Trong backend cổ điển, khi quỹ đạo lệch bạn có một chuỗi câu hỏi rất cụ thể để hỏi. Chi bình phương của cạnh nào lớn bất thường? Bao nhiêu cạnh bị nhân robust ép xuống? Hiệp phương sai biên của khung nào nở ra? Mỗi câu đều có một con số trả lời, và con số đó trở vào một dòng mã. Khi trọng số do một mạng sinh ra, câu hỏi “vì sao nó ra tư thế này” không còn câu trả lời ở mức đồ thị. Bạn vẫn in được phân dư. Bạn không truy được trọng số về nguyên nhân. Và điều tệ nhất là: với bộ giải, một trọng số sai trông y hệt một trọng số đúng.



Hình 31.5. Hai lối đi ngược qua một bộ giải lặp. Trái phẳng giữ mọi đại lượng trung gian, nên bộ nhớ tăng tuyến tính theo số bước lặp K ; đổi lại đạo hàm đúng với thuật toán đã chạy, kể cả khi nó chưa hội tụ. Định lý hàm ẩn chỉ cần ma trận H ở bước cuối, nên bộ nhớ không phụ thuộc K ; đổi lại đạo hàm chỉ đúng nếu bước cuối thật sự là một điểm dừng.

31.3 Hạ tầng khả vi: LieTorch, Theseus và BA-Net (Differentiable Infrastructure)

A. LieTorch: gradient sống trong không gian tiếp tuyến

Có một trục trặc rất cụ thể khi ghép autodiff với hình học. PyTorch chỉ biết tensor. Một phép biến đổi $SE(3)$ lưu dưới dạng ma trận 4×4 là 16 số; lưu dưới dạng quaternion cộng tịnh tiến là 7 số. Nhưng nhóm chỉ có 6 bậc tự do. Nếu để autodiff chạy trên biểu diễn nhúng, *gradient* có 16 hoặc 7 thành phần, và phần lớn trong đó trở ra ngoài đa tạp. Người ta chữa bằng cách chuẩn hoá lại quaternion sau mỗi bước, hoặc phạt độ lệch trực giao. Cả hai đều là vá vúi, và cả hai đều làm *gradient* sai một lượng không kiểm soát được.

LieTorch [295] sửa từ gốc. Nó lưu và lan truyền *gradient* trong không gian tiếp tuyến, tức đại số Lie. Một phép biến đổi vẫn được lưu bằng 7 số, nhưng đạo hàm của nó là một vectơ 6 chiều, và quy tắc chuỗi được viết lại bằng các Jacobian trái và phải của nhóm. Đó đúng là những Jacobian bạn đã gặp ở Barfoot [26]. Hệ quả thực dụng: không còn chuẩn hoá giữa đồ thị, số chiều *gradient* đúng bằng số bậc tự do, và bước cập nhật là một phép exp chứ không phải một phép cộng.

Bài học thiết kế lấy được ngay cả khi không dùng thư viện: **ghép một mạng vào một khối hình học thì hãy kiểm tra ngay số chiều của gradient**. Nếu nó lớn hơn số bậc tự do thì có một phần *gradient* đang chảy vào các hướng vô nghĩa. Phần đó sẽ hiện ra dưới dạng huấn luyện chậm hoặc không ổn định.

B. Theseus: bắt bạn phải chọn kiểu lượt ngược

Theseus [296] đóng gói cả một bộ giải phi tuyến thành một tầng PyTorch. Bạn khai báo biến và hàm chi phí, nó chạy Gauss-Newton hoặc Levenberg-Marquardt với bộ giải thừa, và nó khả vi. Điều đáng học không nằm ở sự tiện lợi mà ở chỗ nó *bắt người dùng khai báo kiểu lượt ngược*: trái phẳng đầy đủ, trái phẳng cắt ngắn, hay hàm ẩn. Thư viện không chọn hộ, vì không lựa chọn nào đúng cho mọi bài toán.

Đó là một tín hiệu trung thực đáng quý. Nó nói thẳng rằng đạo hàm qua một bộ giải là một *xấp xỉ có thiên lệch*, và bạn phải chọn nhận thiên lệch nào. Trước Theseus, gradSLAM [297] đã viết lại cả một đường ống SLAM dày đặc dưới dạng khả vi, chủ yếu để chứng minh rằng việc đó làm được.

Khác biệt giữa hai kiểu lượt ngược gọn tới mức viết ra được trong mười lăm dòng giả mã.


```
# Kiểu 1: trái phằng. Autodiff tu lo, nhưng giu moi buoc trong bo nho.
x = x0
for k in range(K):
    H, g = build_normal_equations(x, theta) # do thi phép tính lon dan
    x = x - solve(H + lam*I, g)
L = loss(x); L.backward() # bo nho ~ K

# Kiểu 2: ham an. Chay bo giai KHONG ghi do thi, roi noi lai bang mot lop tu viet.
with torch.no_grad():
    x = newton_solve(x0, theta, K) # bo nho ~ 1
H, B = hessian_and_cross(x, theta) # chi tinh tai nghiem cuoi
v = solve(H, grad_of_loss_wrt_x(x)) # dung lai phan tich cua luot xuoi
grad_theta = -(B.t() @ v) # dinh ly ham an
```

Dòng `torch.no_grad()` chính là chỗ đánh đổi. Nó xoá đồ thị phép tính của vòng lặp, nên bộ nhớ tụt về hằng số. Đổi lại, mọi thứ đã xảy ra bên trong vòng lặp trở nên vô hình với **gradient**: bao nhiêu bước đã chạy, damping đã đổi thế nào, có bước nào bị từ chối không. Bảng 31.3 nói rõ ba lựa chọn và chỗ hỏng của từng cái.

Bảng 31.3. Ba kiểu lướt ngược qua một bộ giải lặp. Không kiểu nào đúng cho mọi bài toán, nên thư viện bắt bạn chọn — và lựa chọn đó phải được ghi lại cùng kết quả, giống như số seed.

Kiểu	Bộ nhớ	Đạo hàm là của cái gì	Hỏng khi nào
Trái phằng đầy đủ	Tỉ lệ với số bước K	Đúng thuật toán đã chạy	K lớn hoặc bài toán lớn: hết bộ nhớ, và gradient nhân dồn qua K tầng nên tắt hoặc nổ
Trái phằng cắt ngắn	Tỉ lệ với vài bước cuối	Xấp xỉ, thiên về hành vi cuối vòng lặp	Khởi tạo tồi: phần quan trọng của quỹ đạo tồi ưu nằm ở các bước đầu, và bạn vừa cắt bỏ nó
Hàm ẩn	Hằng số	Nghiem của bài toán <i>giả sử đã hội tụ</i>	Bộ giải bị cắt ngắn, hoặc H suy biến vì chưa neo gốc toạ độ

Một quy tắc thực dụng rút ra từ bảng trên. Nếu bộ giải của bạn hội tụ tốt và bạn neo gốc tử tế thì dùng hàm ẩn, vì nó rẻ hơn hẳn. Nếu bộ giải bị ngân sách thời gian ép dừng sớm — trường hợp thường gặp trong hệ thời gian thực — thì trái phằng cắt ngắn trung thực hơn, dù đắt. Và dù chọn cái nào, hãy ghi lựa chọn ấy vào cấu hình thí nghiệm. Nó ảnh hưởng tới kết quả ngang một siêu tham số.

C. BA-Net: học hệ số damping thay vì tự chỉnh

BA-Net [298] là công trình đầu tiên đặt nguyên một lớp **hiệu chỉnh chùm tia** (*bundle adjustment*) vào trong mạng và huấn luyện xuyên qua nó. Nó có ba ý tưởng, và giá trị của ba ý rất khác nhau.

- **Sai lệch trên đặc trưng thay vì trên độ sáng.** Phương pháp trực tiếp cổ điển tối ưu sai khác độ sáng, nên nó thừa hưởng giả định độ sáng không đổi và có lòng chảo hội tụ rất hẹp. BA-Net tối ưu trên các bản đồ đặc trưng học được. Bề mặt lỗi trơn hơn, lòng chảo rộng hơn, và kết quả bất biến với đổi sáng. Đây là ý tưởng bền nhất trong ba ý.
- **Damping học được.** Levenberg-Marquardt cổ điển chỉnh λ bằng luật kinh nghiệm: bước tốt thì chia mười, bước xấu thì nhân mười. BA-Net thay luật đó bằng một mạng nhỏ, đọc phần dư hiện tại rồi xuất ra một vectơ λ cho từng tham số. Vì λ vào bước cập nhật một cách trơn, cả bước LM vẫn khả vi, nên chính sách damping được huấn luyện theo lỗi tư thế cuối.
- **Độ sâu theo cơ sở.** Thay vì coi mỗi điểm ảnh là một ẩn, một mạng sinh ra khoảng một trăm bản đồ độ sâu cơ sở, và ẩn thật sự là các hệ số tổ hợp. Ý này là tổ tiên trực tiếp của CodeSLAM ở mục 6. Số phận của nó cũng báo trước số phận của CodeSLAM: cơ sở độ sâu là một tiên

nghiệm học được về hình dạng cảnh, nên nó tốt đúng trong phân phối huấn luyện và bịa ra bề mặt ở ngoài đó.

Có đáng không? Ý thứ nhất đáng, và quan trọng hơn là nó tách rời được: bạn có thể dùng đặc trưng học được làm mục tiêu căn chỉnh cho một phương pháp trực tiếp mà không cần bất cứ thứ gì khác của BA-Net. Dòng GN-Net đi đúng hướng này, huấn luyện đặc trưng sao cho bề mặt Gauss-Newton quanh nghiệm đúng là một lòng chảo sạch.

Ý thứ hai thì phải nghi ngờ. Một chính sách damping học được vẫn là *chính tay*, chỉ khác ở chỗ hằng số bây giờ được học từ một tập dữ liệu thay vì được gõ vào file cấu hình. Câu hỏi kiểm chứng đúng đắn không phải “nó có tốt hơn trên tập này không”. Câu hỏi đúng là “nó có còn tốt hơn trên một chuỗi có động lực khác hẳn không”. Mình chưa thấy bằng chứng mạnh cho về sau.

CẠM BẤY

Đừng nhầm *bộ giải khả vi* với *bộ giải tốt hơn*. Một lớp Gauss-Newton khả vi hội tụ đúng bằng tốc độ của Gauss-Newton thường, vì ở lượt xuôi nó *chính là* Gauss-Newton thường. Toàn bộ giá trị nằm ở lượt ngược, và lượt ngược chỉ có ý nghĩa nếu bạn thật sự huấn luyện một thứ gì đó bằng nó. Cấm một lớp tối ưu khả vi vào hệ thống rồi không huấn luyện gì cả thì bạn vừa đổi một bộ giải C++ đã tối ưu lấy một bộ giải Python chậm hơn, mà không nhận lại gì.

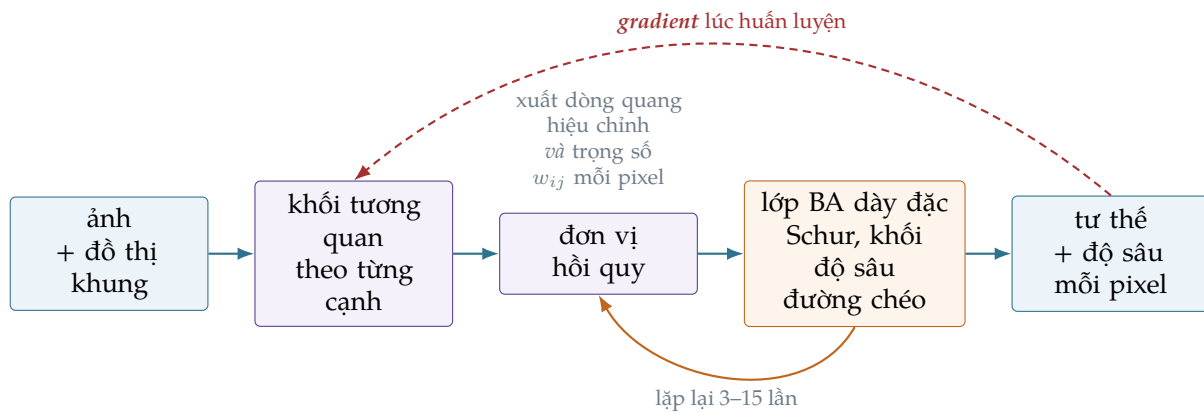
31.4 DROID-SLAM và cuộc thừa hoá sau nó (DROID-SLAM and the Sparsification Line)

A. Cơ chế của DROID-SLAM

DROID-SLAM [217] là hệ đầu-cuối thuyết phục nhất tính tới nay, và cấu tạo của nó đáng vẽ ra. Nó ghép hai thứ đã có: toán tử cập nhật hồi quy của RAFT [167] và một lớp hiệu chỉnh chùm tia dày đặc.

Vòng lặp chạy như sau. Hệ giữ một *đồ thị khung*, với nút là khung then chốt và cạnh là các cặp có chồng lẫn thị giác. Với mỗi cạnh, một khối tương quan được dựng từ hai bản đồ đặc trưng. Một đơn vị hồi quy đọc khối tương quan cùng dòng quang hiện tại rồi xuất ra hai thứ: một mục tiêu dòng quang đã hiệu chỉnh cho từng điểm ảnh, và một trọng số tin cậy cho từng điểm ảnh. Lớp hiệu chỉnh chùm tia dày đặc nhận hai thứ đó làm phép đo, rồi giải bài toán bình phương tối thiểu theo tư thế và theo nghịch đảo độ sâu từng điểm ảnh. Vì khối độ sâu là ma trận đường chéo, phần bù Schur cho ra một hệ camera rút gọn rất nhỏ. Nghiệm mới được đưa ngược lại cho đơn vị hồi quy, và vòng lặp tiếp tục.

Huấn luyện chạy trên TartanAir [299] với chân trị tư thế. *Gradient* đi xuyên qua lớp hiệu chỉnh chùm tia, về tới tận mạng đặc trưng.



Hình 31.6. Vòng lặp của DROID-SLAM. Điểm cốt lõi không phải mạng hồi quy, cũng không phải bộ giải, mà là *vòng phân hồi màu cam* giữa hai cái đó: mạng nhìn thấy nghiệm mà bộ giải vừa trả về rồi sửa phép đo cho lần sau. Mùi tên đứt là đường *gradient* lúc huấn luyện. Nhờ nó, mạng học cách xuất ra dòng quang và trọng số mà bộ giải dùng được, chứ không phải dòng quang trông đẹp mắt.

B. Vì sao một hệ 144 nghìn ẩn vẫn giải được

Với cửa sổ 30 khung then chốt ở $1/8$ độ phân giải, lớp hiệu chỉnh chùm tia này có khoảng 144 nghìn ẩn hình học. Con số đó nghe như một hệ không giải nổi trong thời gian thực. Lý do nó vẫn giải được là một tính chất cấu trúc bạn đã quen, chỉ khác quy mô.

Trong hiệu chỉnh chùm tia cổ điển, mỗi điểm bản đồ đóng góp một khối 3×3 trên đường chéo của khối cấu trúc, nên khử chúng bằng phân bù Schur là rẻ. Ở đây còn dễ hơn. Mỗi ẩn là *một* nghịch đảo độ sâu của *một* điểm ảnh, và nó không nối trực tiếp với điểm ảnh nào khác. Khối cấu trúc vì thế là một ma trận đường chéo thật sự, không phải khối đường chéo. Nghịch đảo nó là 144 nghìn phép chia.

Sau khi khử, hệ còn lại chỉ là hệ camera rút gọn cỡ $6N \times 6N$. Với $N = 30$ khung then chốt thì đó là ma trận 180×180 . Phân tích Cholesky một ma trận cỡ đó là chuyện vặt so với phần còn lại của vòng lặp. Nói cách khác, **phần đắt của DROID-SLAM không nằm ở bộ giải**. Nó nằm ở việc dựng khối tương quan rồi chạy mạng hồi quy để sinh ra 144 nghìn phép đo cùng 144 nghìn trọng số, lặp lại tới mười lăm lần cho mỗi khung.

Kết luận thực dụng rút ra khá rõ: nếu định thử một ý tưởng theo hướng này thì đừng tối ưu bộ giải. Ba chỗ cắt được là độ phân giải, số bước lặp, và số cạnh trong đồ thị khung. Cả ba đều là tham số cấu hình chứ không phải kiến trúc, nên thử rất nhanh.

C. Vì sao nó chính xác

Bốn cơ chế cộng lại, và chỉ một trong bốn liên quan tới chữ “khả vi”.

- **Dày đặc là dư thừa.** Mỗi điểm ảnh bỏ một lá phiếu, nên vài nghìn phiếu sai không lay chuyển được nghiệm. Một hệ thừa chỉ có vài trăm cặp khớp thì có.
- **Trọng số theo từng điểm ảnh.** Mạng hạ trọng số vùng trời, vật thể chuyển động, bề mặt phản chiếu. Kiểm tra tỉ lệ khoảng cách mô tả không làm được việc đó, vì nó không nhìn ngữ cảnh.
- **Sửa lặp lại.** Đơn vị hồi quy thấy nghiệm hiện tại rồi sửa phép đo, nên hệ tự gỡ được khởi tạo tồi. Một đường ống một chiều không có cơ chế này.
- **Hình học vẫn được ép.** Đầu ra đi qua một bộ giải thật, nên nó nhất quán đa khung. Nó không trôi theo kiểu một mạng hồi quy tư thế.

Cơ chế thứ ba mới là chỗ tính khả vi trả tiền. Không có *gradient* xuyên qua bộ giải thì mạng sẽ được huấn luyện để dự đoán dòng quang khớp với chân trị dòng quang. Có *gradient*, nó được huấn luyện để dự đoán dòng quang làm cho *tư thế* đúng. Hai mục tiêu đó khác nhau, và khác biệt nằm đúng ở các điểm ảnh có hình học xấu.

D. Vì sao nó đắt

Đếm ẩn số trước. Ảnh 640×480 ở $1/8$ độ phân giải cho 4 800 ẩn nghịch đảo độ sâu cho mỗi khung then chốt. Với 30 khung trong cửa sổ là 144 000 ẩn hình học, so với 6 ẩn tư thế mỗi khung. Bài toán vẫn giải được vì khối độ sâu là đường chéo, nhưng mọi tensor đều mang kích thước đó.

Đếm bộ nhớ sau, và đếm bằng công thức chứ không quy ra một con số. Một khối tương quan toàn phần giữa hai khung giữ một mục cho *mỗi cặp* vị trí, tức bình phương số vị trí ở độ phân giải đang chạy — và đó mới là một cạnh, một mức. Đồ thị khung có nhiều cạnh cho mỗi nút, nên tổng bộ nhớ tỉ lệ với số cạnh nhân bình phương số vị trí. Chữ “bình phương” đó giải thích vì sao bộ nhớ tăng rất nhanh theo độ phân giải, vì sao nó tăng theo độ dài chuỗi, và vì sao các công trình sau phải thêm cơ chế quản lý bộ nhớ.

Còn phần cứng. Các tác giả công bố hệ trên GPU để bàn cao cấp, không phải trên máy nhúng. Một Jetson Orin có ngân sách tính toán và băng thông bộ nhớ thấp hơn hẳn một GPU để bàn cùng thế hệ, và khoảng cách ấy thuộc loại bậc độ lớn chứ không phải vài chục phần trăm. Đây là hướng, không phải số đo: mình không có phép đo nào cho phát biểu này, và muốn một con số thì phải tự chạy. Thêm nữa, DROID-SLAM không dùng IMU, tức là bỏ đi tài sản mạnh nhất của một hệ stereo-inertial.

E. Một bảng tính ngân sách Orin, làm trong mười phút

Trước khi tải bất cứ mã nguồn nào về, có một việc rẻ hơn nhiều: đếm phép tính trên giấy. Nó không cho bạn con số đúng, nhưng nó cho bạn biết bạn đang hụt *hai lần* hay *hai mươi lần*. Hai kết luận đó dẫn tới hai quyết định hoàn toàn khác nhau.

Giả sử mục tiêu là vài chục khung mỗi giây, và frontend cổ điển của bạn đã ăn một phần đáng kể ngân sách mỗi khung. Bảng 31.4 liệt kê các khoản mục, cách tính từng khoản, và mức nặng nhẹ tương đối giữa chúng.

Bảng 31.4. Các khoản mục ngân sách của một hệ đầu-cuối kiểu DROID trên phần cứng nhúng. Cột giữa là công thức đếm; cột cuối là *xếp hạng định tính* giữa các khoản, kèm lý do của thứ hạng đó. Bảng này cố tình không có con số: nó dùng để biết nên cắt ở đâu, còn độ lớn thật thì chỉ phép đo trên chính thiết bị của bạn mới nói được.

Khoản mục	Cách tính	Thứ hạng, và vì sao
Mạng trích đặc trưng	Đếm phép nhân cộng của backbone ở độ phân giải đầu vào thật, không phải ở 224×224	Nặng. Chi phí tỉ lệ thẳng với số điểm ảnh, nên đo ở độ phân giải trong bài rồi suy ra độ phân giải thật là cách hệt nhiều lần
Khối tương quan	$(H/8)(W/8)$ nhân $(H/8)(W/8)$ cho mỗi cạnh, nhân số byte	Nặng nhất về bộ nhớ. Đây là khoản duy nhất tăng theo <i>bình phương</i> số vị trí, lại còn nhân với số cạnh của đồ thị khung
Vòng lặp hồi quy	Nhân chi phí một bước với số bước, thường 3 tới 15	Khoản đội chi phí lớn nhất, vì nó nhân thẳng mọi khoản phía trên lên vài lần tới hơn chục lần
Giải hệ BA	Hệ camera rút gọn $6N \times 6N$, với N là số khung trong cửa sổ	Nhẹ nhất, và hay bị đổ oan. Nó nhỏ hơn phần sinh phép đo nhiều bậc, nên tối ưu nó gần như không đổi được gì
Chuyển dữ liệu	Tổng byte đi qua bộ nhớ chia cho băng thông thật của thiết bị	Thường mới là nút cổ chai thật, và chỉ lộ ra khi bạn đếm byte thay vì đếm phép tính

Cách dùng bảng quan trọng ngang bản thân bảng. Nhân các khoản theo đúng công thức ở cột giữa, chia cho thông lượng *đo được* của thiết bị — không phải con số đỉnh trên tờ thông số, vốn luôn cao hơn nhiều — rồi so với ngân sách mỗi khung. Thứ bạn cần lấy ra từ phép tính đó không phải một con số mili-giây, mà đúng một bit thông tin: bạn đang hệt *một chút* hay hệt *một bậc độ lớn*. Hệt một chút thì đi đo thật, vì phép tính trên giấy không đủ chính xác để kết luận. Hệt một bậc thì đừng đo vội, vì kết luận đã rõ rồi: phải cắt hẳn một thừa số trước đã. Và đúng ba chỗ cắt được là: hạ độ phân giải, giảm số bước lặp, hoặc giảm mật độ. Mục kế tiếp là câu chuyện của lựa chọn thứ ba.

↔ cùng nguyên lý với B/11-gioi-han-tinh-toan — đây chính là bài toán roofline của chương 11 áp cho một hệ SLAM: đếm phép tính và đếm byte trước, đo sau

F. Thừa hoá: DPVO, DPV-SLAM và MAC-VO

Nếu chỗ đất là mật độ, thì câu hỏi tự nhiên là mật độ có cần thiết không.

I Mạch — từ dày đặc về thưa

1 Vấn đề

DROID-SLAM chính xác nhưng cần phần cứng GPU để bàn cao cấp và một lượng bộ nhớ vượt xa máy nhúng, nên không đưa lên robot được.

Giải pháp

DPVO [300] bỏ trường dòng quang dày đặc. Thay vào đó nó theo vết một tập nhỏ các *mảnh ảnh* vuông, mỗi khung khoảng một trăm mảnh, mỗi mảnh mang đúng một ẩn nghịch đảo độ sâu. Vẫn cùng toán tử hồi quy và cùng lớp hiệu chỉnh chùm tia, chỉ khác là chạy trên đồ thị mảnh.

Giả định

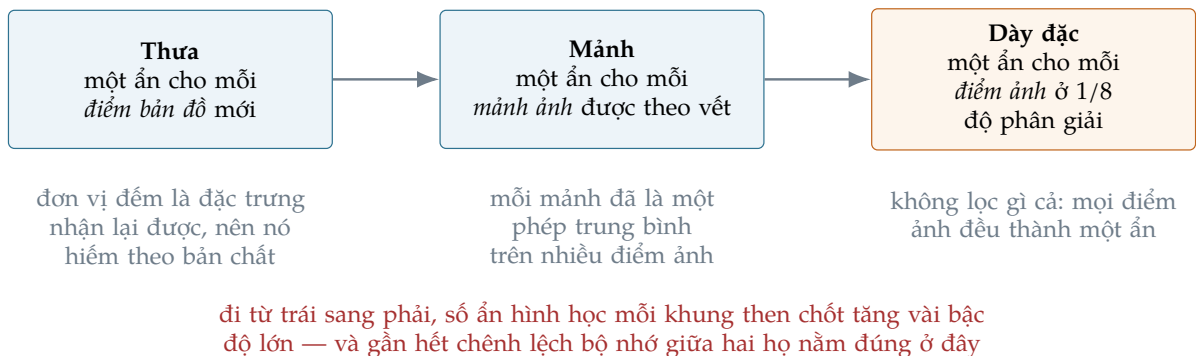
Ích lợi thống kê của việc dày đặc bão hoà rất sớm, vì bản thân một mảnh đã là một phép trung bình trên nhiều điểm ảnh.

Hệ quả

Số ẩn hình học mỗi khung giảm vài bậc độ lớn, và theo báo cáo của chính các tác giả thì hệ nhẹ đi đủ nhiều để chạy trên một GPU để bàn duy nhất thay vì nhiều card. Mình không kiểm chứng được các con số tốc độ và bộ nhớ họ đưa ra, nên ở đây chỉ giữ lại hướng của thay đổi.

- Giá phải trả** Mất khả năng tự nhiên cho ra bản đồ dày đặc, và mất một phần sức chống chịu khi cảnh có rất ít vân.
- 2 Vấn đề** DPVO là odometry: không có bản đồ toàn cục, nên nó trôi và không bao giờ đóng vòng được.
- Giải pháp** DPV-SLAM [301] thêm hai cơ chế đóng vòng. Một cơ chế dựa trên chính đồ thị mảnh cho các vòng gần, và một cơ chế truy hồi ảnh kiểu cổ điển cho các vòng xa. Sau đó chạy tối ưu đồ thị tư thế toàn cục.
- Hệ quả** Thành một hệ SLAM đầy đủ, vẫn gần thời gian thực trên một GPU.
- Giả định** Bài học còn quan trọng hơn kết quả: *chưa ai học được cách thay thế đóng vòng và tối ưu đồ thị tư thế*. Ngay cả một hệ đầu-cuối cũng phải mượn lại đúng hai khối cổ điển đó.
- 3 Vấn đề** Trọng số vô hướng cho mỗi cặp khớp vẫn quá thô. Bất định của một cặp khớp stereo có hình dạng rất dẹt: dài theo tia nhìn và ngắn theo phương ngang.
- Giải pháp** MAC-VO [302] dự đoán cả một hiệp phương sai cho từng cặp khớp. Quan trọng là hiệp phương sai ấy *có đơn vị mét*, tức nó tính tới việc sai số độ sâu nở ra khi disparity nhỏ. Con số đó được dùng hai lần: để chọn điểm nào đáng giữ, và để làm ma trận thông tin trong bài toán tư thế.
- Hệ quả** Đây là cầu nối trực tiếp sang [F/34-imu-bat-dinh-va-trien-khai](#). Thứ được học không còn là đặc trưng hay dòng quang, mà chính là *ma trận thông tin* — đúng đại lượng bạn đang đặt bằng tay.
- Giá phải trả** Vẫn cần huấn luyện có chân trị, và bất định học được kế thừa mọi giới hạn phân phối ở mục 5. Chi tiết số của công trình này mình chưa tự kiểm chứng, nên chỉ nêu ý tưởng.

Hình 31.7 đặt ba mức mật độ cạnh nhau để thấy khoảng cách thật sự lớn tới đâu.



Hình 31.7. Ba mức mật độ, đọc theo *đơn vị đếm* chứ không theo con số. Điều đáng nhớ không phải mỗi họ giữ bao nhiêu ẩn, mà là mỗi họ lấy gì làm đơn vị: một hệ thưa đếm theo điểm bản đồ nhận lại được, DPVO đếm theo mảnh ảnh, DROID đếm theo điểm ảnh. Vì mỗi bậc trong ba bậc đó cách nhau vài bậc độ lớn về số lượng, thứ tự này quyết định gần như toàn bộ chênh lệch bộ nhớ giữa hai họ — nhiều hơn hẳn mọi khác biệt về cài đặt.

G. Đánh giá theo bốn câu hỏi

Nó giải quyết việc gì. Đóng góp thật của dòng này là trọng số theo từng cặp khớp và vòng sửa lặp lại. Hai thứ đó chạm đúng vào chuyển động nhanh và ảnh mờ. Đó là các trường hợp mà kiểm tra tỉ lệ khoảng cách mô tả cho ra hoặc quá ít cặp, hoặc quá nhiều cặp sai.

Có đáng làm không. Với một hệ stereo-inertial đã tinh chỉnh chạy trên Orin thì chưa. Ba lý do, theo thứ tự nặng dần. Ngân sách tính toán chênh một bậc độ lớn. Toàn bộ dòng này bỏ IMU, nên

bạn sẽ đánh đổi thứ đang cứu bạn ở những khung mờ để lấy thứ hứa cứu bạn ở những khung mờ. Và tính lặp lại sụt — một hệ bạn không chẩn đoán được là một hệ bạn không bảo trì được.

Học gì từ thiết kế. Ba bài học tách rời được, xếp theo mức dễ áp dụng. Dùng trọng số liên tục thay cho quyết định inlier nhị phân. Đưa nghiệm hiện tại quay lại cho tầng sinh phép đo, thay vì chạy một chiều. Và giữ đồ thị khung như một cấu trúc dữ liệu tường minh, thay vì để nó ẩn trong danh sách kề của bản đồ.

→ [tiền đề F/30-do-sau-dong-quang-va-mo-hinh-nen](#) — chất lượng của cả họ này bị chặn trên bởi chất lượng dòng quang, tức bởi đúng dòng RAFT mà chương 30 bàn

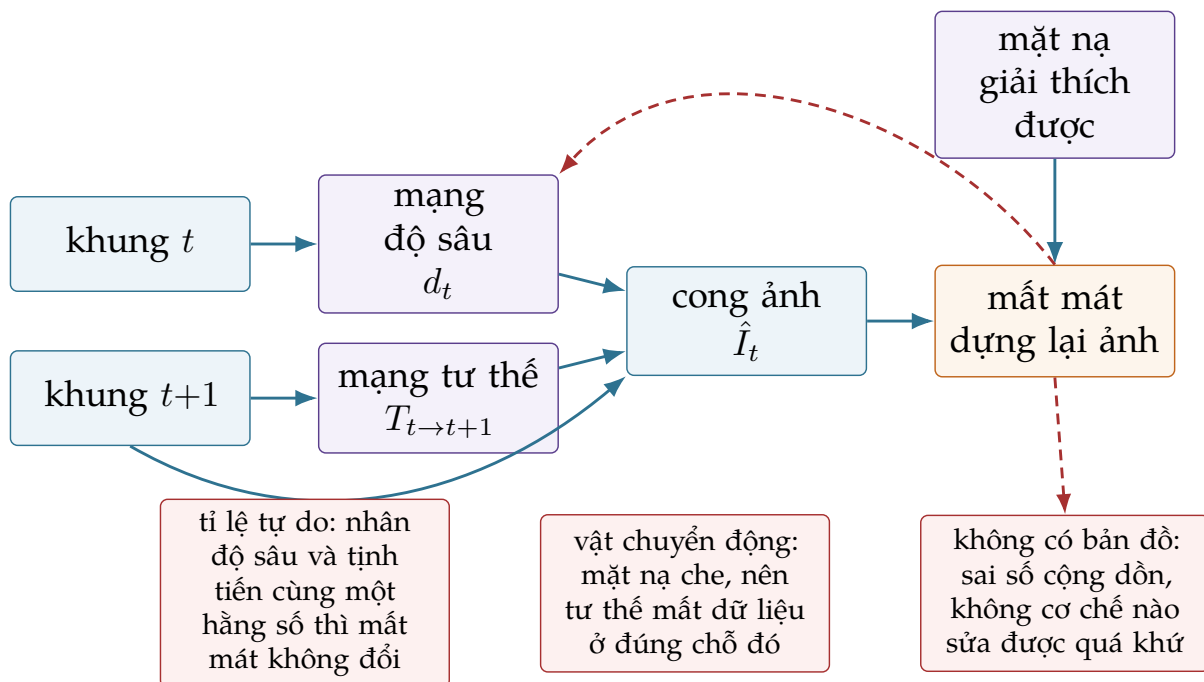
31.5 VO tự giám sát và giới hạn thật của nó (Self-Supervised VO)

A. Bài toán và cơ chế của SfM-Learner

Chân trị tư thế đắt. Cần một hệ đo động ngoài, hoặc một buổi chạy COLMAP dài. Câu hỏi của SfM-Learner [303] là: có thể huấn luyện chỉ bằng video thô không?

Cơ chế gọn và đẹp. Hai mạng cùng học. Mạng độ sâu nhận một khung và xuất bản đồ độ sâu. Mạng tư thế nhận một cặp khung và xuất phép biến đổi giữa chúng. Có độ sâu, có tư thế, có nội tham số camera thì *dựng lại* được khung đích từ khung nguồn bằng phép cong ảnh. **Hàm mất mát** (*loss function*) là chênh lệch giữa ảnh dựng lại và ảnh thật. Không cần nhãn nào.

Chi tiết cấu cả ý tưởng là *mặt nạ giải thích được*. Đó là một đầu ra thứ ba, một con số mỗi điểm ảnh, cho phép mạng nói “điểm này tôi không giải thích được, đừng phạt tôi”. Nó xử lý vật thể chuyển động, vùng bị che, vùng ra ngoài khung. Nhưng nó có một nghiệm suy biến hiển nhiên: đặt mặt nạ bằng 0 ở mọi nơi thì mất mát bằng 0. Nên phải cộng một số hạng chính quy kéo mặt nạ về 1. Điểm này đáng ghi nhớ, vì nó là mẫu chung của mọi thiết kế tự giám sát: **cho mô hình quyền bỏ qua dữ liệu thì phải bắt nó trả tiền cho quyền đó**.



Hình 31.8. Vòng huấn luyện tự giám sát kiểu SfM-Learner, và ba chỗ hỏng gắn liền với nó. Tín hiệu dạy duy nhất là “dựng lại khung t từ khung $t+1$ thì phải khớp”. Ba hộp đỏ không phải lỗi cài đặt mà là hệ quả trực tiếp của chính hàm mất mát đó: mặt nạ mua sự bền vững bằng cách vớt dữ liệu, tỉ lệ không quan sát được từ ảnh đơn, và không có trạng thái nào để một lần quan sát lại có thể sửa quá khứ.

B. Bốn giới hạn, nói thẳng

Đây là mục một kỹ sư SLAM cần đọc kỹ nhất, vì các bảng kết quả trong dòng này rất dễ đọc nhầm.

- **Mơ hồ tỉ lệ.** Với ảnh đơn, nhân độ sâu với s và nhân tịnh tiến với s cho ra đúng cùng một ảnh dựng lại. Mắt mắt không phân biệt được hai trường hợp. Nên đầu ra chỉ đúng tới một hằng số chưa biết, và hằng số đó còn trôi giữa các cảnh. Nhiều bảng KITTI trong dòng này căn tỉ lệ với chân trị *theo từng chuỗi* trước khi tính lỗi. Một robot không làm được việc đó, nên con số ấy không phải con số odometry dùng được. UnDeepVO [304] chữa bằng cách huấn luyện trên cặp stereo — đường cơ sở đã biết bom tỉ lệ mét vào — rồi chạy đơn ảnh lúc suy luận.
- **Không có bản đồ, nên không có đóng vòng.** Mạng tư thế là phép ước lượng chết theo từng cặp khung. Sai số cộng dồn và không gì kéo nó về được. Toàn bộ độ chính xác của hệ thống của bạn đến từ việc quan sát lại cùng một điểm bản đồ qua nhiều khung then chốt. Ở đây cơ chế đó không tồn tại.
- **Không có bảo đảm, và hỏng êm.** Không hiệp phương sai, không phần dư để kiểm, không cổng χ^2 . Khi sai, nó sai một cách mượt mà và tự tin. So sánh với hệ của bạn: khi sắp mất bám, số inlier tụt trước, và bạn thấy được.
- **Phụ thuộc phân phối huấn luyện.** DeepVO [305] hồi quy tư thế trực tiếp và rất tốt trên KITTI. Một phần lý do là nó học luôn *tiên nghiệm chuyển động của một chiếc ô tô*: đi thẳng, tốc độ trong một dải hẹp, góc lệch nhỏ. Đó là một cơ chế thật chứ không phải lời che. Đưa mô hình sang camera cầm tay hoặc drone thì tiên nghiệm ấy sai, và nó sai theo hướng có hệ thống.

C. Mơ hồ tỉ lệ, nhìn bằng một phép tính

Giới hạn thứ nhất đáng được làm cho cụ thể, vì nó là chỗ nhiều người đọc bảng kết quả bị lừa.

Phép cong ảnh dùng đúng một biểu thức. Điểm ảnh u ở khung nguồn được đưa về khung đích qua độ sâu $d(u)$ và phép biến đổi (R, t) . Bây giờ thay $d \rightarrow s d$ và $t \rightarrow s t$ với cùng một hằng số $s > 0$. Toạ độ chiếu ra không đổi, vì s rút gọn giữa tử và mẫu. Ảnh dựng lại giống hệt, nên mắt mắt giống hệt. Kết luận: hàm mắt mắt *không chứa thông tin* nào về s .

Sự kiện nhỏ này dẫn tới một hệ quả lớn khi đọc bảng. Giả sử — đây là một phép tính giả định, không trích từ bài nào — một hệ báo lỗi tịnh tiến vài phần trăm chiều dài quỹ đạo, sau khi đã căn tỉ lệ theo từng chuỗi. Nếu bạn không được căn tỉ lệ, và tỉ lệ ước lượng lệch mười phần trăm, thì riêng sai lệch tỉ lệ đã sinh ra lỗi bằng mười phần trăm chiều dài quỹ đạo — lớn hơn hẳn con số được báo cáo, và lớn dần theo quãng đường đi. Con số trong bảng khi đó nói về *hình dạng* quỹ đạo, không nói về *vị trí*. Với một robot cần biết mình đang ở đâu, hai đại lượng đó không thay thế nhau được.

Cách kiểm nhanh khi đọc một bài báo trong dòng này chỉ có hai câu hỏi. Tỉ lệ được lấy từ đâu: từ stereo, từ IMU, hay từ chân trị? Và nếu từ chân trị thì căn một lần cho cả chuỗi hay căn lại theo từng đoạn? Căn theo từng đoạn là dấu hiệu tỉ lệ đang trôi, và độ trôi đó chính là thứ bị giấu đi.

D. TartanVO và cách chữa đúng chỗ

TartanVO [306] là câu trả lời có kỷ luật cho gạch đầu dòng cuối. Hai thủ thuật của nó đáng học kể cả khi bạn không dùng mô hình.

Thứ nhất, **bỏ tỉ lệ ra khỏi hàm mất mát**. Ép mạng học tỉ lệ tuyệt đối từ ảnh đơn thì nó buộc phải học tỉ lệ đặc thù của tập dữ liệu. Chuẩn hoá vectơ tịnh tiến trong hàm mất mát thì mạng chỉ còn phải học hướng, và hướng thì tổng quát hoá được.

Thứ hai, **đưa mô hình camera vào đầu vào**. Nối thêm vài kênh mã hoá nội tham số đã chuẩn hoá vào tensor đầu vào. Nhờ đó một mô hình duy nhất phục vụ nhiều camera có tiêu cự và độ méo khác nhau, thay vì phải huấn luyện lại cho từng camera.

Kết quả là một mô hình chạy qua nhiều tập dữ liệu mà không tinh chỉnh lại. Độ chính xác khiêm tốn, nhưng tính tổng quát là thật. Bài học chung: muốn tổng quát hoá thì lấy đại lượng đặc thù tập dữ liệu ra khỏi hàm mất mát, và đưa thông tin thiết bị vào đầu vào.

E. Nhánh giữ lại bộ giải cổ điển

Bốn công trình còn lại trong mục này đáng nhắc vì chúng chọn một kiến trúc khác hẳn. Với người đọc này, đó mới là kiến trúc đáng dùng.

- **DeepTAM** [307] tách tracking và mapping đúng như một hệ cổ điển rồi học từng phần, thay vì trộn cả hai vào một mạng.
- **DeepV2D** [308] luân phiên một khối độ sâu và một khối chuyển động, trong đó khối chuyển động là một bước Gauss-Newton khả vi. Nó là cầu nối giữa mục này và mục 4.
- **MonoRec** [309] học độ sâu đa khung có xử lý vật chuyển động, nhưng *lấy tư thế từ một hệ SLAM cổ điển*. Nó không cố thay bộ ước lượng tư thế.
- **TANDEM** [310] đi xa nhất theo hướng đó. Tracking bằng phương pháp trực tiếp cổ điển, còn phần học được chỉ lo dựng bản đồ dày đặc, rồi hợp nhất vào một trường TSDF.

Bốn công trình trên vẽ ra một mẫu thiết kế rõ ràng: **giữ bộ ước lượng tư thế cổ điển, học phần dày đặc**. Với một hệ stereo-inertial đã chạy tốt, đây là mẫu duy nhất trong cả mục 5 mà mình cho là đáng thử, vì nó không đặt cược thứ đang hoạt động.

31.6 Biểu diễn ẩn: nén hình học vào một mã ngắn (Latent Scene Representations)

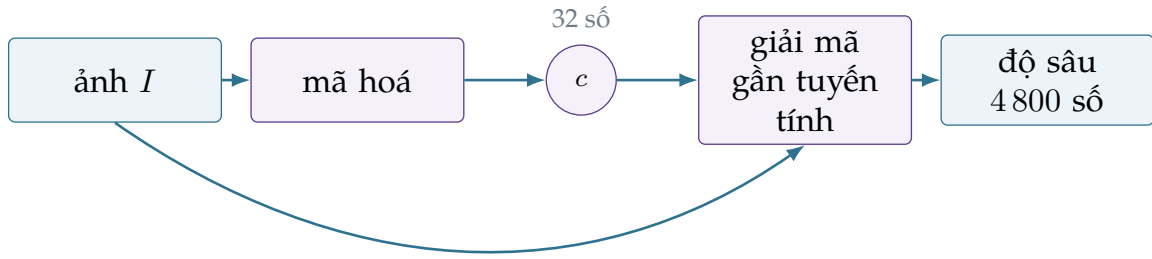
A. Vấn đề mà mã ngắn định giải

Bản đồ độ sâu dày đặc của một khung có cỡ 10^5 ẩn. Đưa từng ấy ẩn vào một bài toán bình phương tối thiểu thì hoặc phải chính quy hoá bằng luật kinh nghiệm, hoặc phải có GPU. Nhưng hình học của một căn phòng thật không có 10^5 bậc tự do. Tường phẳng, sàn phẳng, vật thể có hình dạng quen thuộc. Hình học ấy nằm gần một đa tạp số chiều thấp.

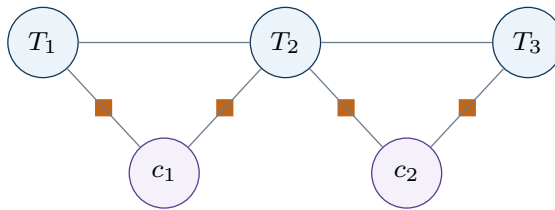
CodeSLAM [311] rút ra kết luận trực tiếp. Học một bộ mã hoá đưa ảnh về một *mã ngắn* c cỡ vài chục chiều, cùng một bộ giải mã $D(c, I)$ trả về bản đồ độ sâu. Rồi đặt c vào đồ thị nhân tử *ngay cạnh các nút tư thế*, và tối ưu chung.

Chi tiết kỹ thuật làm ý tưởng chạy được ít khi được nhắc, nhưng nó là mấu chốt. Bộ giải mã được thiết kế để *gần tuyến tính theo c* khi đã cố định ảnh. Nhờ đó $\partial D / \partial c$ tính một lần cho mỗi khung then chốt rồi dùng lại cho mọi bước Gauss-Newton. Không có tính chất đó thì mỗi lần tuyến tính hoá tốn một lượt ngược qua mạng, và bài toán hết khả thi thời gian thực.

Bộ mã hoá và bộ giải mã



Đồ thị nhân tử



mỗi nút mã thay cho 4 800 ẩn độ sâu, nhưng nó nối tới *mọi* tư thể cùng quan sát khung đó

Hình 31.9. Ý tưởng của CodeSLAM. Bộ giải mã được thiết kế gần tuyến tính theo mã c , nên $\partial D/\partial c$ là một ma trận cố định, tính một lần cho mỗi khung then chốt. Nhờ đó mã trở thành một biến bình thường trong đồ thị nhân tử, đứng ngang hàng với tư thể, và hình học dày đặc được tối ưu chung với quỹ đạo bằng đúng bộ giải bạn đã có.

B. Đếm ẩn và đếm chi phí tuyến tính hoá

Ba con số cho thấy vì sao mã ngăn hấp dẫn tới vậy, và cũng cho thấy chỗ nó âm thầm đắt.

Số ẩn. Một bản đồ độ sâu ở 1/8 của ảnh 640×480 có 4 800 ẩn. Một mã 32 chiều thay thế cả từng ấy. Tỉ số nén là 150 lần. Khối cấu trúc trong hệ phương trình chuẩn co từ 4800×4800 đường chéo xuống 32×32 đặc, và biên hoá một khối 32×32 là chuyện vặt.

Chi phí Jacobian. Đây mới là chỗ ẩn chi phí. Ma trận $\partial D/\partial c$ có kích thước 4800×32 , tức khoảng 154 nghìn số. Nếu bộ giải mã *không* gần tuyến tính theo c thì ma trận đó phải tính lại ở mỗi lần tuyến tính hoá, và mỗi lần tính là 32 lượt ngược qua bộ giải mã. Với năm bước Gauss-Newton là 160 lượt ngược cho mỗi khung then chốt. Không hệ thời gian thực nào chịu nổi.

Vì sao gần tuyến tính lại cứu được. Nếu $D(c, I) \approx D_0(I) + J(I)c$ thì J chỉ phụ thuộc ảnh, nên tính một lần khi khung then chốt được tạo rồi dùng lại mãi. Chi phí mỗi bước Gauss-Newton trở về đúng một phép nhân ma trận, y như một cạnh chiếu lại thông thường. Toàn bộ tính khả thi của CodeSLAM nằm ở dòng vừa rồi. Và giá của nó là điều kiện gần tuyến tính chỉ đúng trong một lân cận nhỏ, tức lý do thứ hai ở mục C bên dưới.

C. Dòng công trình

SceneCode [312] mở rộng mã sang cả nhãn ngữ nghĩa, để hình học và ngữ nghĩa cùng được tinh chỉnh. DeepFactors [313] đưa mã vào một đồ thị nhân tử thật với ba loại nhân tử — quang trắc, chiếu lại, và hình học thưa — rồi chạy được thời gian thực. NodeSLAM [314] đổi phạm vi: mỗi *vật thể* có một mã hình dạng riêng, và tối ưu chạy trên tư thể cộng hình dạng của từng vật. CodeMapping [315] thực dụng nhất: giữ nguyên ORB-SLAM3 làm xương sống thưa, rồi chạy một tầng bản đồ dày đặc bằng mã ở bên cạnh, có điều kiện trên chính các điểm thưa đó.

D. Vì sao hướng này chưa thẳng

Câu hỏi này giá trị hơn danh sách trên. Ý tưởng đẹp, cài đặt chạy được, người làm giỏi. Vậy tại sao tám năm sau nó vẫn không nằm trong hệ sản xuất nào? Bốn lý do, xếp theo mức nền tảng.

Thứ nhất, sai số của tiên nghiệm có tương quan chứ không thưa. Toàn bộ phòng thủ của ước lượng robust dựa trên một giả định: điểm ngoại lai thì ít và độc lập với nhau. Một bộ giải mã huấn luyện trên ảnh RGB-D trong nhà không sinh ra sai số như vậy. Nó sinh ra những bề mặt trơn, hợp lý, sai một cách có hệ thống, và *nhất quán giữa các khung* vì cùng một bộ giải mã tạo ra chúng. Với một cổng χ^2 , thứ đó trông y hệt một bức tường thật. Đây là lý do sâu nhất, và nó áp cho mọi tiên nghiệm học được chứ không riêng mã hình học.

Để thấy điều đó, hãy so hai kiểu lỗi trên cùng một cổng χ^2 , bằng một ví dụ *giả định* — không phải số đo trên hệ nào. Một cặp khớp sai lệch cỡ vài chục điểm ảnh cho χ^2 vọt lên hàng trăm, nên cổng 5,991 loại nó ngay ở khung đầu tiên. Bây giờ giả sử bộ giải mã dựng một bề mặt lệch ΔZ vài chục centimét ở khoảng cách vài mét. Đưa độ lệch ấy qua công thức chiếu $\Delta u = f b \Delta Z / Z^2$, với f và b của một cấu hình stereo cỡ EuRoC, phần dư thu được nhỏ hơn một điểm ảnh — vì Z^2 ở mẫu số nuốt gần hết độ lệch. Nó lọt qua mọi cổng. Tệ hơn, nó lọt qua ở *mọi* khung cùng một kiểu, nên các khung củng cố lẫn nhau và bộ ước lượng càng lúc càng tự tin vào một bức tường không có thật.

Thứ hai, bán kính hiệu lực của phép tuyến tính hoá rất nhỏ. Bộ giải mã chỉ gần tuyến tính *quanh mã mà bộ mã hoá vừa xuất ra*. Một hiệu chỉnh lớn sẽ đi ra ngoài vùng đó, và $\partial D / \partial c$ không còn đúng. Nói cách khác, tối ưu chỉ sửa được những sai lệch nhỏ — đúng trường hợp mà dự đoán ban đầu vốn đã tốt.

Thứ ba, độ phủ. Ba mươi hai chiều đủ cho phân phối huấn luyện và không đủ cho thứ khác. Ngoài trời, ban đêm, một sàn nhà máy đầy kim loại phản chiếu — bộ giải mã không có cơ sở nào để dựng bề mặt ở đó, nhưng nó vẫn dựng ra một bề mặt.

Thứ tư, phần thưởng đã dờn chỗ. Năm 2018, đây gần như là con đường duy nhất để có bản đồ dày đặc tối ưu được. Tới 2023, 3D Gaussian Splatting [175] cho một bản đồ dày đặc, tinh chỉnh tăng dần được, chất lượng ảnh cao hơn hẳn, mà không cần bất cứ tiên nghiệm học được nào về hình dạng. Đó là nội dung của [F/32-ban-do-neural](#). Thêm nữa, xét riêng độ chính xác quỹ đạo, dòng mã ngắn chưa bao giờ vượt được SLAM đặc trưng thưa. Bản đồ mới là sản phẩm, còn tư thế chỉ là sản phẩm phụ.

E. Cái còn lại

Trước khi rời mục này, đáng chốt lại một bài kiểm tra hai câu, dùng được cho *mọi* tiên nghiệm học được mà bạn định cắm vào backend — mã hình học, độ sâu đơn ảnh, hay pháp tuyến bề mặt.

Câu thứ nhất: sai số của nó có độc lập giữa các khung không? Nếu cùng một mạng sinh ra dự đoán cho mọi khung thì câu trả lời gần như luôn là không. Khi đó nhân robust và cổng χ^2 mất tác dụng, và bạn phải hạ trọng số của cả nhóm phép đo ấy bằng tay, chứ không thể trông vào bộ ước lượng tự loại.

Câu thứ hai: sai số của nó có biểu hiện thành phần dư lớn không? Đây là câu hỏi định lượng, và mục trước đã cho cách tính: đổi độ lệch hình học ra phần dư điểm ảnh bằng công thức chiếu. Nếu một sai lệch nguy hiểm chỉ tạo ra phần dư dưới một điểm ảnh thì không cổng nào bắt được nó, và tiên nghiệm ấy phải được cấp một hiệp phương sai lớn ngay từ đầu.

Hai câu đó không cần thí nghiệm nào. Chúng trả lời được trên giấy trong mười phút, và chúng loại đi phần lớn ý tưởng nghe hay.

Dù vậy có một mảnh đáng giữ, và nó không phải cái mã. Đó là ý tưởng đặt một biến ẩn số chiều thấp vào đồ thị nhân tử rời ước lượng luôn hiệp phương sai của nó. Dạng khoẻ mạnh nhất của ý tưởng ấy là NodeSLAM. Một mã hình dạng cho *một loại vật thể cụ thể* là tiên nghiệm được xác

định rõ hơn nhiều so với một mã cho “một căn phòng”. Tiên nghiệm càng hẹp thì càng ít cơ hội bịa ra bề mặt.

CẦN NHỚ

Trong một bộ ước lượng bình phương tối thiểu, thứ nguy hiểm không phải là tiên nghiệm học được *sai*, mà là nó sai *một cách trơn tru và nhất quán giữa các khung*. Nhân robust, cổng χ^2 và RANSAC đều được thiết kế cho ngoại lai thừa và độc lập. Không cái nào bắt được một bề mặt bịa ra mà mọi khung đều đồng ý. Trước khi cắm bất cứ đầu ra học được nào vào backend, hãy hỏi đúng một câu: *sai số của nó có độc lập giữa các khung không*. [T1]

31.7 Giới hạn và trường hợp hỏng

A. Giới hạn của chính ý tưởng khả vi

Chế độ hỏng nền tảng nhất đã nêu ở mục 2 và đáng nhắc lại thành một câu: ***gradient chỉ chảy qua những thứ vốn đã liên tục***. Trong một hệ SLAM đầy đủ, các quyết định đắt giá nhất lại rời rạc. Chèn hay không chèn khung then chốt. Chấp nhận hay từ chối một ứng viên đóng vòng. Gộp hay không gộp hai bản đồ. Không cái nào có đạo hàm. Nên cụm từ “đầu-cuối” luôn hẹp hơn nghĩa đen của nó. Phần được học chính là phần vốn đã trơn, còn xương sống rời rạc vẫn do luật viết tay quyết định — và đó thường là phần gây ra các lần hỏng nặng nhất.

Chế độ hỏng thứ hai nằm ở ổn định số. H^{-1} nở ra khi H gần suy biến, tức khi thị sai thấp, khi camera quay gần như thuần túy, khi ảnh mờ. Đó chính xác là những khung bạn muốn mạng học từ chúng. Nên ***gradient*** lớn nhất và nhiều nhất lại đến từ các mẫu quý nhất. Mọi cách chữa — tăng damping, cắt ***gradient***, giảm số bước lặp — đều làm thiên lệch đạo hàm theo hướng làm nhẹ đúng các mẫu đó.

Chế độ hỏng thứ ba là hàm mất mát. ***Gradient*** chỉ tốt bằng đại lượng bạn đặt ở cuối. Huấn luyện trên các đoạn video ngắn với lỗi tư thế theo cặp khung thì chỉ tối ưu tính nhất quán ngắn hạn. Nó không tối ưu độ trôi sau ba trăm mét, cũng không tối ưu xác suất đóng vòng đúng. Hai thứ đó lại chiếm phần lớn sai số thật của bạn. Đây là biến thể của bài học đo lường ở [E/24-thi-nghiem-va-ablation](#): tối ưu cái bạn đo được thay vì cái bạn quan tâm là một cái bẫy, không phải một chiến lược.

B. Giới hạn của các hệ cụ thể

Ba giới hạn dùng chung cho cả bốn họ trong chương. *Dữ liệu huấn luyện* gần như luôn là TartanAir, KITTI hoặc ScanNet, nên phân phối chuyển động và phân phối cảnh đều hẹp hơn hiện trường. *Vấn đề IMU*: dòng đầu-cuối hầu như không dùng IMU, nên nó vứt đi thứ duy nhất còn tin được khi ảnh mờ. *Ngân sách nhúng*: các con số ẩn tượng đều đo trên GPU để bàn, và khoảng cách tới Orin là bậc độ lớn.

Còn một giới hạn ít được nói mà bạn sẽ gặp ngay ngày đầu, đó là ***tính lặp lại***. Bạn đã tốn công truy các nguồn phi tắt định trong hệ hiện tại — thứ tự duyệt container, tranh chấp luồng — để mỗi cấu hình chạy ba lần cho ra số dùng được. Đưa một mạng vào giữa bộ ước lượng là đưa thêm phi tắt định của thư viện GPU vào đúng chỗ đó, và lần này không có bug nào để sửa. Sàn nhiều của harness sẽ dâng lên, và mọi kết luận cũ phải đo lại.

Bảng 31.5. Bốn họ trong chương, đọc theo cùng bốn câu hỏi. Cột cuối là cột quyết định với một hệ stereo-inertial đã tinh chỉnh kỹ, chạy trên phần cứng nhúng.

Họ	Cơ chế cốt lõi	Hỏng khi nào	Lấy riêng được phần nào
Lớp tối ưu khả vi	Đạo hàm qua nghiệm arg min, bằng hàm ẩn hoặc trải phẳng	H suy biến; bộ giải bị cắt ngắn; hoặc không có gì để huấn luyện	Gần như toàn bộ: nó là hạ tầng, không phải một hệ SLAM
Đầu-cuối dày đặc	Dòng quang dày, trọng số học được, BA trong vòng lặp	Ngân sách nhúng; không dùng IMU; bộ nhớ tăng theo độ dài chuỗi	Trọng số liên tục thay quyết định inlier; vòng phản hồi nghiệm về phép đo
Đầu-cuối thưa	Theo vết mảnh ảnh; hiệp phương sai theo từng cặp khớp	Cảnh rất ít vân; vẫn cần GPU; vẫn phụ thuộc phân phối	Hiệp phương sai học được có đơn vị mét — mảnh đáng lấy nhất chương
Tự giám sát và mã ẩn	Mất mát dựng lại ảnh; nén hình học vào mã ẩn	Mơ hồ tỉ lệ; không đóng vòng; tiên nghiệm bịa bề mặt nhất quán	Mẫu “giữ bộ giải cổ điển, học phần dày đặc” kiểu TANDEM

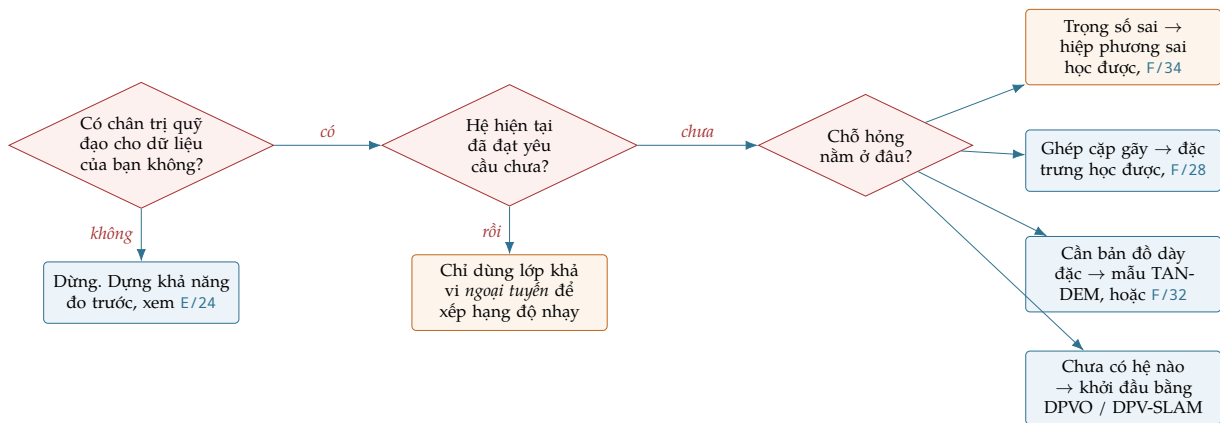
C. Kết luận thẳng thắn

Với một hệ stereo-inertial đã tinh chỉnh kỹ, chạy thời gian thực trên Orin, có harness đo được và có kỷ luật ablation: **hệ đầu-cuối hiện chưa phải lựa chọn thay thế**. Không phải vì ý tưởng sai, mà vì bảng cân đối không có lợi. Bạn sẽ đổi tính chẩn đoán, tính lặp lại và một bậc độ lớn ngân sách tính toán, để lấy một cải thiện chỉ hiện ra trên một phần các cảnh, và mất luôn IMU trên đường đi.

Nhưng kết luận đó không kéo theo “cả chương này vô dụng”. Có hai mảnh tách rời được, và cả hai đều nhỏ hơn nhiều so với việc thay cả hệ.

- **Lớp bất định học được.** Thay ma trận thông tin hằng số $\sigma_\ell^{-2}I$ bằng một hiệp phương sai dự đoán theo từng cặp khớp. Nó cắm vào đúng chỗ backend hiện có đang đọc trọng số, không đụng tới bộ giải, và tắt được bằng một cờ. Đây là mảnh đáng thử trước tiên, và [F/34-imu-bat-dinh-va-trien-khai](#) là chỗ bàn nó cho cả camera lẫn IMU.
- **Lớp tối ưu khả vi như một công cụ chẩn đoán ngoại tuyến.** Bạn không cần chạy nó lúc suy luận. Bạn có thể dựng lại bài toán backend của mình trong một lớp khả vi, chạy ngoại tuyến trên dữ liệu đã có chân trị, rồi lấy *gradient* của APE theo các nút vận để biết nút nào thật sự quan trọng. Đó là một công cụ *phân tích độ nhạy*, rẻ hơn quét lưới rất nhiều, và nó không thêm rủi ro nào vào hệ đang chạy.

Hình 31.10 gói cả kết luận đó thành một cây quyết định bốn câu hỏi. Nó cố tình bắt đầu bằng câu hỏi về dữ liệu chứ không phải về phương pháp, vì thiếu chân trị thì mọi nhánh còn lại đều vô nghĩa.



Hình 31.10. Cây quyết định cho cả chương. Câu hỏi đầu tiên cố tình không phải về phương pháp mà về dữ liệu: không có chân trị thì không có *gradient* nào để lấy, nên mọi nhánh phía sau đều vô nghĩa. Hai ô màu cam là hai kết luận “đáng làm” duy nhất cho một hệ stereo-inertial đã tinh chỉnh kỹ; ba ô còn lại chỉ dùng cho các tình huống khác hẳn.

31.8 Thực hành

Repo và tài nguyên. [princeton-vl/lietorch](#) cho phép toán nhóm Lie có *gradient*. [facebookresearch/theseus](#) cho lớp tối ưu phi tuyến khả vi; nên đọc phần tài liệu về các kiểu lượt ngược trước khi đọc mã. [princeton-vl/DRUID-SLAM](#) và [princeton-vl/DPVO](#) để so hai đầu của trục mật độ trên cùng một chuỗi. [nianticlabs/monodepth2](#) là cài đặt sạch nhất của dòng tự giám sát, và là chỗ tốt nhất để tự tay thấy hiện tượng mơ hồ tỉ lệ. [castacks/tartanvo](#) cho ý tưởng tầng nội tham số. *evo* để mọi con số bạn đo có cùng định nghĩa với phần còn lại của dự án. Danh sách này chốt tại tháng 9/2026 và sẽ cũ trong sáu đến mười hai tháng; riêng LieTorch, Theseus và *evo* thuộc loại hạ tầng nên bền hơn hẳn các tên hệ thống đang dẫn đầu bảng.

Câu hỏi kiểm tra hiểu. Viết ra công thức $\partial L / \partial \theta$ qua một lớp arg min mà không nhìn lại, rồi nói rõ mỗi ký hiệu ứng với đại lượng nào trong backend của bạn. Nếu bộ giải chỉ chạy ba bước LM thì đạo hàm theo định lý hàm ẩn sai theo hướng nào, và bạn kiểm tra điều đó bằng thí nghiệm gì? Trong một hệ đầu-cuối, vì sao không có bước nào tương đương với “loại cạnh có χ^2 vượt ngưỡng”?

Bài tập phụ. Lấy một chuỗi EuRoC, chạy DPVO ở chế độ mặc định, rồi vẽ quỹ đạo của nó cạnh quỹ đạo hệ của bạn sau khi căn tỉ lệ. Mục đích không phải so hơn thua. Mục đích là thấy hình dạng lỗi của hai họ khác nhau ra sao: một bên lệch dần và trơn, một bên nhảy tại các khung khó.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
● ● ● Lớp tối ưu khả vi	Một bộ giải Gauss-Newton hoặc LM mà <i>gradient</i> đi ngược xuyên qua được. Lượt xuôi giống hệt bộ giải thường; toàn bộ giá trị nằm ở lượt ngược, và nó chỉ có nghĩa nếu bạn thật sự huấn luyện thứ gì đó bằng nó.
● ● ● Định lý hàm ẩn cho arg min	Tại nghiệm, gradient theo x bằng không; vì phân điều kiện đó cho $\partial x^* / \partial \theta = -H^{-1} B$. Nhờ nó, lượt ngược chỉ tốn thêm một lần giải hệ với chính H đã phân tích ở lượt xuôi.
● ● ● Trái phải (unrolling)	Coi K bước lặp như một mạng K tầng rồi để autodiff chạy. Đạo hàm đúng với thuật toán đã chạy, kể cả khi chưa hội tụ, nhưng bộ nhớ tăng tuyến tính theo K .

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Khi nào định lý hàm ẩn cho đạo hàm sai?	Khi bước cuối không phải điểm dừng thật, tức bộ giải bị cắt ngắn. Hoặc khi H suy biến vì tự do đo chưa được neo. Cả hai đều thường xuyên xảy ra trong SLAM.
●●● Vì sao hệ đầu-cuối không xuất mặt nạ inlier?	Vì so sánh với ngưỡng là hàm bậc thang, đạo hàm bằng 0 hầu khắp nơi. Muốn học được thì phải thay quyết định cứng bằng một trọng số liên tục.
●● LieTorch	Thư viện lan truyền <i>gradient</i> trong không gian tiếp tuyến của nhóm Lie. Nó làm số chiều <i>gradient</i> bằng đúng số bậc tự do, sáu cho $SE(3)$, nên bỏ được việc chuẩn hoá quaternion giữa đồ thị.
●● Theseus	Lớp tối ưu phi tuyến khả vi trong PyTorch, có bộ giải thừa. Điểm đáng học là nó bắt người dùng chọn kiểu lượt ngược, tức thừa nhận thẳng rằng đạo hàm qua bộ giải là một xấp xỉ có thiên lệch.
●● BA-Net	Đặt cả một lớp hiệu chỉnh chùm tia vào mạng. Ba ý: sai lệch trên đặc trưng thay vì trên độ sáng, damping học được, và độ sâu theo cơ sở. Ý đầu bền nhất vì căn chỉnh trên đặc trưng học được có lòng chảo hội tụ rộng hơn và bất biến với đổi sáng.
●●● DROID-SLAM	Toán tử hồi quy kiểu RAFT sinh dòng quang dày và trọng số theo từng điểm ảnh; một lớp BA dày đặc giải tư thế và độ sâu; nghiệm quay lại làm đầu vào cho vòng sau, nhờ đó hệ tự gỡ được khởi tạo tồi. Chính xác, nhưng cần GPU để bàn và không dùng IMU.
●●● DPVO	Thưa hoá DROID bằng cách theo vết khoảng một trăm mảnh ảnh mỗi khung, mỗi mảnh một ẩn độ sâu. Số ẩn hình học giảm vài bậc độ lớn, đủ để hệ chạy trên một GPU để bàn duy nhất thay vì nhiều card.
●● DPV-SLAM	DPVO cộng thêm đóng vòng và tối ưu đồ thị tư thế. Bài học lớn hơn kết quả: đóng vòng và tối ưu đồ thị tư thế vẫn là hai khối cổ điển mà chưa ai học thay được.
●●● MAC-VO	Dự đoán hiệp phương sai có đơn vị mét cho từng cặp khớp stereo, dùng để chọn điểm và để làm ma trận thông tin. Đây là mảnh ghép nhất với thứ một hệ cổ điển mượn được ngay.
●●● SfM-Learner	Huấn luyện mạng độ sâu và mạng tư thế chỉ bằng video thô, tín hiệu dạy là mất mát dựng lại ảnh. Cần một mặt nạ giải thích được, và mặt nạ đó phải bị chính quy hoá vì nghiệm suy biến là tất hết.
●●● Mơ hồ tỉ lệ với ảnh đơn	Nhân độ sâu và tịnh tiến với cùng một hằng số cho ra đúng cùng ảnh dựng lại, nên mất mát không phân biệt được. Hệ quá: nhiều bảng kết quả căn tỉ lệ theo từng chuỗi, việc mà robot không làm được.
●● DeepVO tốt trên KITTI mà kém chỗ khác — nguyên nhân?	Nó học luôn tiên nghiệm chuyển động của ô tô: đi thẳng, tốc độ trong dải hẹp, góc lệch nhỏ. Trên camera cầm tay hoặc drone, tiên nghiệm ấy sai theo hướng có hệ thống.
●● TartanVO	Chuẩn hoá tịnh tiến để bỏ tỉ lệ ra khỏi hàm mất mát, và nổi nội tham số camera vào đầu vào. Nhờ hai thủ thuật đó, một mô hình chạy được qua nhiều tập dữ liệu mà không tinh chỉnh lại.
●● TANDEM	Giữ tracking trực tiếp cổ điển, chỉ học phần dựng bản đồ dày đặc rồi hợp nhất TSDF. Đây là mẫu thiết kế đáng dùng nhất trong dòng tự giám sát cho một hệ đã chạy tốt.
●●● CodeSLAM	Nén bản đồ độ sâu vào một mã cỡ vài chục chiều, với bộ giải mã gần tuyến tính theo mã, để đưa mã vào đồ thị nhân tử cạnh tư thế. Tính gần tuyến tính là điều kiện sống còn, nếu không thì mỗi lần tuyến tính hoá tốn một lượt qua mạng.
●●● Vì sao dòng mã ngắn chưa thắng?	Sai số của tiên nghiệm trơn và nhất quán giữa các khung nên cổng χ^2 không bắt được. Thêm nữa bán kính hiệu lực của phép tuyến tính hoá nhỏ, độ phủ hẹp, và 3DGS đã lấy mất phần thường “bản đồ dày đặc”.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
● ● ● Ngoại lai thừa và độc lập	Giả định nền của mọi ước lượng robust. Tiên nghiệm học được vi phạm nó, vì cùng một bộ giải mã sinh ra sai số ở mọi khung nên chúng đồng ý với nhau và trông như hình học thật.

● ● ● phải thuộc ● ● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
LieTorch [295]	Lan truyền <i>gradient</i> trong không gian tiếp tuyến của nhóm Lie — hạ tầng cho gần như mọi hệ trong chương	Hạ tầng; bền hơn tên hệ thống
Theseus [296]	Lớp Gauss-Newton và LM khả vi, bắt người dùng khai báo kiểu lượt ngược: trải phẳng, cắt ngắn, hay hàm ẩn	Hạ tầng; dùng được ngoại tuyến để phân tích độ nhạy
DROID-SLAM [217] và các nhánh dẫn xuất	BA dày đặc trong vòng lặp, cộng trọng số học được theo từng điểm ảnh	Vẫn là mốc chính xác tham chiếu cho nhiều bảng SLAM học sâu
DPVO và DPV-SLAM [300], [301]	Thừa hoá bằng mảnh ảnh; và việc phải mượn lại đóng vòng cổ điển	Ví dụ rõ nhất rằng đóng vòng chưa được học thay
MAC-VO [302]	Hiệp phương sai học được theo từng cặp khớp, dùng làm ma trận thông tin	Sau 2024; mình nêu ý tưởng, chưa tự kiểm chứng số liệu
Dòng monodepth2 và các hệ độ sâu tự giám sát	Mất mát dừng lại ảnh và mặt nạ giải thích được của SfM-Learner [303]	Ý tưởng còn sống mạnh ở phía độ sâu hơn là ở phía tư thế

MINI PROJECT SLAM · Đo độ nhạy của APE theo trọng số cạnh, bằng đạo hàm thay vì quét lưới · 1–2 ngày

Mục tiêu	biến câu “trọng số cạnh đặt bằng tay” thành một bảng số nói rõ nút vặn nào thật sự lay chuyển quỹ đạo. Đây là phiên bản rẻ nhất và không rủi ro của toàn bộ ý tưởng khả vi: bạn chỉ dùng <i>gradient</i> để <i>chẩn đoán</i> , không đưa mạng nào vào hệ đang chạy.
Dữ liệu	một chuỗi EuRoC có chân trị. Lấy đúng một cửa sổ 8–10 khung then chốt từ log của hệ hiện tại, kèm toàn bộ quan sát và khởi tạo của cửa sổ đó.
Các bước	dựng lại đúng bài toán bình phương tối thiểu của cửa sổ ấy trong một lớp tối ưu khả vi; kiểm tra nghiệm khớp với nghiệm của backend C++ tới cỡ 10^{-4} trước khi làm bất cứ việc gì khác; đặt L là sai lệch tư thế so với chân trị; lấy $\partial L / \partial \theta$ cho ba nhóm tham số là trọng số cạnh chiếu lại theo tầng kim tự tháp, bề rộng nhân robust, và trọng số cạnh IMU; cuối cùng đối chiếu với đạo hàm số bằng sai phân hữu hạn trên vài thành phần.
Xong khi	bạn có một bảng xếp hạng độ nhạy, và kiểm chứng được nó bằng một thí nghiệm độc lập. Đổi tay đúng nút vặn đúng đầu bảng một lượng nhỏ, rồi xác nhận APE đổi theo đúng dấu mà <i>gradient</i> dự đoán, trên ít nhất ba lần chạy để vượt sần nhiễu. Nếu dấu không khớp thì gần như chắc chắn cửa sổ của bạn chưa hội tụ tới điểm dừng. Đó cũng là một kết quả, và nó nói rằng định lý hàm ẩn chưa dùng được ở đây.
Nối với	E/24-thi-nghiem-va-ablation cho sần nhiễu và cỡ mẫu; F/34-imu-bat-dinh-va-trien-khai sẽ dùng lại chính bảng độ nhạy này để quyết định có đáng học hiệp phương sai hay không; A/01-toan-toi-thieu/02-vi-phan-va-toi-uu cho phần kiểm tra đạo hàm bằng sai phân hữu hạn.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Nó không làm Gauss-Newton hội tụ nhanh hơn, vì lượt xuôi y hết. Nó bán một thứ khác: đạo hàm của lỗi quỹ đạo theo các nút vận mà bạn đang đặt bằng tay, tức là biến việc quét lưới thành việc huấn luyện.
- A2. Vì tại nghiệm, gradient theo x bằng không, và điều kiện đó ràng buộc nghiệm với tham số. Vì phân nó ra $-H^{-1}B$, chỉ cần H ở bước cuối. Điều kiện là H phải khả nghịch và bước cuối phải là điểm dừng thật; trong SLAM cả hai đều hay bị vi phạm, vì tự do đo và vì bộ giải bị cắt ngắn.
- A3. Không học được, nên các hệ đầu-cuối không cố học. Chúng thay quyết định cứng bằng một trọng số liên tục cho từng cặp khớp: mạng hạ trọng số điểm xấu thay vì loại nó. Đây là hệ quả bắt buộc của việc muốn có *gradient*, không phải một lựa chọn thẩm mỹ.
- A4. Thứ chặn nó là ngân sách tính toán và bộ nhớ. Nó giữ một ẩn độ sâu cho *mỗi* điểm ảnh ở 1/8 độ phân giải, cộng các khối tương quan có kích thước bằng bình phương số vị trí cho mỗi cạnh của đồ thị khung. Hệ được công bố trên GPU để bàn cao cấp, còn máy nhúng thấp hơn hẳn về cả tính toán lẫn băng thông — khoảng cách thuộc loại bậc độ lớn chứ không phải vài chục phần trăm. Thêm nữa nó không dùng IMU.
- A5. Hồng ở chỗ sai số của bộ giải mã trơn và nhất quán giữa các khung, nên nó vượt qua mọi cổng robust vốn giả định ngoại lai thừa và độc lập. Cộng thêm ba điều: phép tuyến tính hoá chỉ đúng quanh mã ban đầu, độ phủ hẹp ngoài phân phối huấn luyện, và 3DGS đã cho bản đồ dày đặc tốt hơn mà không cần tiên nghiệm hình dạng.

NÓI LẠI THẬT ĐƠN GIẢN

Hình dung bạn có một hội đồng gồm hai trăm người cùng đoán một con số. Bạn không lấy trung bình tròn. Bạn lấy trung bình có trọng số: ai bạn tin hơn thì tiếng nói nặng hơn. Từ trước tới nay bạn phát trọng số theo một quy tắc cứng, kiểu “ai ngồi gần bảng thì tin hơn”. Quy tắc đó không quan tâm hôm nay ai quên kính, ai ngồi ngược sáng, ai vừa ngủ dậy.

Bây giờ giả sử cuối buổi bạn biết đáp án đúng. Câu hỏi tự nhiên là: nếu cho người thứ 47 nói to hơn một chút, kết quả chung sẽ dịch về phía đáp án hay ra xa nó? Trả lời được câu đó cho cả hai trăm người thì bạn có một cách phát trọng số tốt hơn mọi quy tắc cứng. Điều bất ngờ là bạn không cần diễn lại cả buổi họp để trả lời. Chỉ cần biết kết quả chung nhảy thể nào quanh điểm cân bằng của nó, rồi làm một phép tính.

Cái giá thì có ba khoản. Muốn học kiểu đó, bạn phải có nhiều buổi họp đã biết đáp án. Phép tính trên trở nên rất nhạy đúng vào những buổi mà hội đồng chia rẽ nhất, mà đó lại là những buổi bạn cần học nhất. Và khi trọng số do một cỗ máy phát ra, hôm nào kết quả sai bạn sẽ không còn chỉ được vào ai đã nói sai. Bạn chỉ biết rằng nó sai.

Chỗ không rút gọn được: chỗ vấp thật nằm ở các quyết định bật-tắt. Nhiều thứ trong một hệ SLAM không phải chuyện tin nhiều hay tin ít, mà là chuyện có mời người đó vào phòng họp hay không: chèn khung then chốt hay không, nhận một vòng lặp hay không. Với những quyết định đó, câu hỏi “nói to hơn một chút thì sao” không có nghĩa, vì không có “một chút” nào cả. Mọi cách nói đơn giản mà mình biết đều làm mờ mất ranh giới này, trong khi chính nó mới là ranh giới thật giữa phần học được và phần không học được.

Một câu: Khả vi không làm bộ giải khôn hơn; nó chỉ cho phép hỏi “tin phép đo này thêm một chút thì quỹ đạo tốt lên hay xấu đi”, và bạn trả tiền cho câu hỏi đó bằng bộ nhớ, độ ổn định và khả năng chẩn đoán.

31.9 Kết nối

[F/27-dat-hoc-sau-vao-dau](#) là bộ lọc mà chương này vừa được đưa qua: bốn cổng của nó cho ra kết luận “chưa đáng” với cả hệ, và “đáng thử” với hai mảnh nhỏ. [F/28-dac-trung-va-ghep-cap](#) là nửa còn lại của cùng một câu chuyện — ở đó phần học được nằm trước bộ giải, ở đây nó nằm bên trong. [F/30-do-sau-dong-quang-va-mo-hinh-nen](#) là tiền đề vật liệu: dòng quang và độ sâu học được chính là đầu vào mà mọi hệ trong chương này tiêu thụ. [F/32-ban-do-neural](#) là nơi cuộc thi về biểu diễn bản đồ dày đặc chuyển sang, và là lý do dòng mã ngăn mất động lực.

[F/34-imu-bat-dinh-va-trien-khai](#) là chỗ mảnh đáng lấy nhất của chương, tức hiệp phương sai học được, được bàn cho đủ, cùng với ngân sách độ trễ trên Orin.

Ngoài phần F thì có bốn mối nối. [A/01/02](#) là nền của định lý hàm ẩn và của cả hai lối đi ngược. [C/15/03](#) giải thích toán tử cập nhật hồi quy mà DROID-SLAM mượn lại. [B/11-gioi-han-tinh-toan](#) cho khung để biến các con số ẩn số và tensor ở mục 4 thành một ngân sách bộ nhớ thật. [E/24](#) là kỷ luật bắt buộc nếu bạn định thử một trong hai mảnh tách rời: đo sần nhiều mới trước, nói về cải tiến sau.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao đạo hàm theo định lý hàm ẩn chỉ tốn thêm một lần thế ngược, trong khi trải phẳng tốn bộ nhớ tỉ lệ với số bước lặp?
2. Bộ giải của bạn dừng sau ba bước LM vì ngân sách thời gian. Đạo hàm thu được khi đó là đạo hàm của cái gì, và bạn thiết kế thí nghiệm nào để phát hiện sai lệch ấy?
3. Nếu quyết định inlier có đạo hàm bằng 0 hầu khắp nơi, thì cơ chế nào trong DROID-SLAM đang đóng vai trò loại điểm sai, và nó khác gì về hành vi khi gặp một cảnh ngoài phân phối huấn luyện?
4. Vì sao *gradient* lại lớn và nhiều nhất đúng ở các khung thị sai thấp hoặc quay thuần túy, và vì sao mọi cách chữa đều làm thiên lệch việc học ở chính các khung đó?
5. Một hệ VO tự giám sát báo lỗi quỹ đạo rất nhỏ trên KITTI. Bạn phải kiểm đúng một điều trước khi tin con số đó — điều gì, và vì sao nó quyết định con số ấy có dùng được cho robot hay không?
6. Vì sao một bề mặt do bộ giải mã bịa ra lại nguy hiểm hơn một cặp khớp sai, xét theo giả định mà nhân robust dựa vào?
7. Nếu chỉ được lấy đúng một mảnh từ chương này về hệ hiện tại, bạn lấy mảnh nào, và bạn đo cái gì để biết nó có tác dụng vượt sần nhiều?

Bản đồ neural: NeRF và 3D Gaussian Splatting (Neural Map Representations)

PHỤ THUỘC C/16/03-nerf-3dgs-va-hop-nhat (cơ chế dựng ảnh thể tích, rasterize Gaussian, và vì sao PSNR không chứng minh hình học — chương này giả định bạn đã nắm phần đó); F/27 (khung bốn cổng để quyết định thay một khối cổ điển); E/24 (thiết kế ablation, oracle experiment); F/31 nếu bạn muốn hiểu bộ tám vết mà nhiều hệ trong chương này mượn lại.

TÓM TẮT

Chương này hỏi một câu duy nhất: có nên thay bản đồ điểm thưa của hệ SLAM hiện tại bằng một bản đồ neural. Bạn sẽ đọc được hai dòng công trình — NeRF và 3D Gaussian Splatting — theo đúng nút thắt mà mỗi mốc gỡ, chứ không theo thứ tự thời gian. Phần nặng nhất là phần đánh giá, vì đây là chủ đề mà chất lượng ảnh và độ chính xác quỹ đạo bị trộn lẫn nhiều nhất. Đọc xong bạn tách được hai trục đó, chỉ ra được điều kiện thí nghiệm ẩn sau một bảng kết quả đẹp, và lập được ngân sách bộ nhớ cùng băng thông cho Jetson Orin dưới dạng công thức, trước khi thử. Nếu chỉ nhớ một điều: một bản đồ đẹp ảnh không phải một bản đồ đúng hình học, và cho tới nay chưa có bằng chứng nào nói rằng bản đồ neural giúp *quỹ đạo* của một hệ stereo-inertial đã tinh chỉnh kỹ tốt lên.

KIẾN THỨC CẦN CÓ TRƯỚC

- **bản đồ điểm thưa** — bản đồ của ORB-SLAM3 và họ hàng: vài chục nghìn tới vài trăm nghìn điểm 3D, mỗi điểm kèm một bộ mô tả để nhận lại. Nó đủ để trả lời “tôi đang ở đâu” và không trả lời được “chỗ kia có trống không”.
- **bản đồ dày, TSDF, lưới chiếm dụng** — ba cách nói về cùng một ý: bản đồ gán một giá trị cho *mọi* vị trí trong không gian, không chỉ ở vài điểm đặc trưng. TSDF lưu ở mỗi ô một khoảng cách có dấu tới bề mặt gần nhất — âm ở trong vật, dương ở ngoài, bằng không đúng trên mặt — nên bề mặt có định nghĩa sạch. Lưới chiếm dụng lưu xác suất mỗi ô bị chiếm, và đó là thứ bộ lập kế hoạch đường đi thật sự cần.
- **dựng ảnh ở góc nhìn mới** — cho một số ảnh đã chụp, tạo ra ảnh của cùng cảnh đó từ một vị trí camera chưa từng chụp. Đây là bài toán mà NeRF và 3DGS sinh ra để giải.
- **PSNR, SSIM, LPIPS** — ba thước đo “ảnh dựng ra giống ảnh thật tới đâu”. PSNR đo sai lệch bình phương theo từng điểm ảnh, SSIM đo tương đồng cấu trúc cục bộ, LPIPS đo khoảng cách trong không gian đặc trưng của một mạng đã huấn luyện. Cả ba đều nói về *ảnh*, không nói gì về hình học.
- **ATE và RPE** — hai thước đo quỹ đạo. ATE là sai số tuyệt đối của cả quỹ đạo sau khi căn chỉnh với chân trị; RPE là sai số của các chuyển vị tương đối trong một cửa sổ thời gian. Đây là hai con số quyết định một hệ SLAM có dùng được không.
- **bám vết theo mô hình** — cách ước lượng tư thế bằng cách dựng ảnh từ bản đồ hiện có rồi tối ưu tư thế cho ảnh dựng khớp ảnh thật, thay vì ghép đặc trưng giữa hai khung hình. Nó chính xác khi bản đồ đúng và trôi theo bản đồ khi bản đồ sai.
- **quên thăm khốc** — hiện tượng một mạng học dữ liệu mới rồi mất khả năng tái tạo dữ liệu cũ, vì cùng bộ trọng số phải gánh cả hai. Trong bản đồ neural, nó hiện ra thành “đi sang phòng bên rồi quay lại thì phòng cũ mờ đi”.

- **Replica, TUM RGB-D, ScanNet** — ba tập dữ liệu mà gần như mọi bài trong chương này báo cáo trên đó. Replica là cảnh tổng hợp có độ sâu hoàn hảo; TUM RGB-D và ScanNet là cảnh trong nhà quay bằng cảm biến thật, chuỗi ngắn, tốc độ đi bộ.

CÂU HỎI MỞ ĐẦU

- Q1. Bản đồ của ORB-SLAM3 chỉ có vài trăm nghìn điểm rời rạc, mà hệ vẫn định vị chính xác tới centimét. Vậy chính xác thì một bản đồ dày cho bạn thêm *điều gì* mà bạn đang thiếu?
- Q2. Nếu bạn thật sự cần bản đồ dày, vì sao lại là bản đồ neural chứ không phải TSDF — thứ đã chạy thời gian thực từ 2011 và cho bề mặt có định nghĩa sạch?
- Q3. Một hệ dựng ảnh đẹp tới mức khó phân biệt với ảnh thật. Vì sao điều đó vẫn không nói gì về việc quỹ đạo của nó tốt hay tệ hơn hệ bạn đang có?
- Q4. 3DGS nhanh hơn NeRF nhiều bậc độ lớn. Khoản chênh đó đến từ một thay đổi thuật toán duy nhất — nó là gì, và vì sao nó lại đúng lúc mở ra khả năng sửa bản đồ sau khi đóng vòng?
- Q5. Một triệu Gaussian nghe không nhiều. Nhưng khi đưa lên Orin và chạy vài chục vòng tối ưu mỗi khung, cái gì cản trước — dung lượng bộ nhớ hay băng thông bộ nhớ — và vì sao câu trả lời đó làm mọi mẹo tăng tốc suy luận trở nên vô dụng?

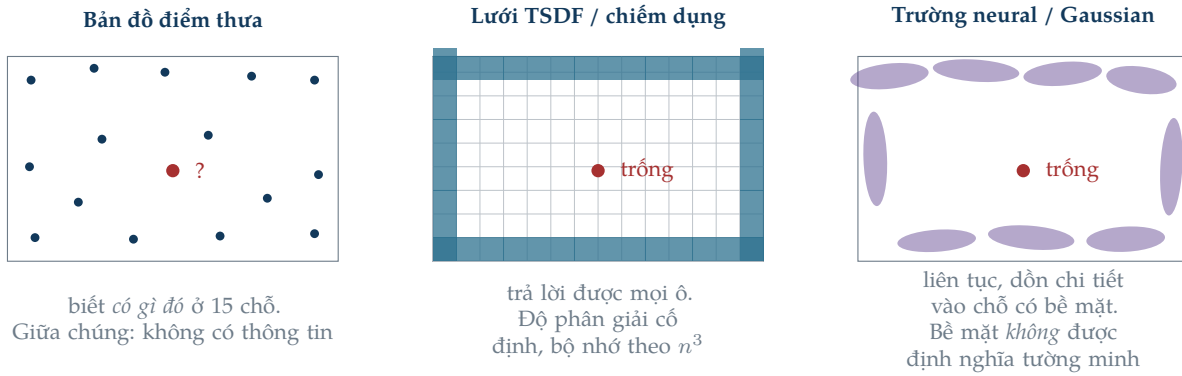
32.1 Đặt vấn đề: bản đồ thưa đủ cho việc gì

Hệ SLAM bạn đang bảo trì dựng một **bản đồ điểm thưa** (*sparse point map*). Mỗi điểm là một vị trí 3D cộng một bộ mô tả nhị phân. Với một tầng văn phòng, con số thường rơi vào bậc 10^5 điểm. Bản đồ đó làm rất tốt đúng một việc: trả lời câu “tôi đang ở đâu”. Nó làm tốt vì nó chỉ cần các điểm *nhận lại được*, và những điểm nhận lại được thì hiếm — góc, biên, đốm vân.

Nhưng có ba câu hỏi hạ nguồn mà chính bản đồ ấy không trả lời nổi. Thứ nhất: khoảng không giữa hai điểm là tường hay là lối đi? Bản đồ thưa không nói. Thứ hai: bề mặt này hình dạng ra sao, để robot đặt tay hoặc để người kiểm tra vết nứt? Bản đồ thưa không nói. Thứ ba: cảnh nhìn từ chỗ camera chưa từng đi qua thì trông thế nào, để người điều khiển từ xa xem trước, hoặc để sinh dữ liệu huấn luyện? Bản đồ thưa cũng không nói.

Trước khi đi tiếp, một điều chỉnh cần thiết cho đúng hệ của bạn. Một cặp stereo đã cho độ sâu dày ở *mỗi* khung hình. Vậy thứ bạn thiếu không phải độ sâu, mà là một bản đồ dày *bền vững*: một chỗ để hợp nhất hàng nghìn quan sát độ sâu nhiều thành một mô hình duy nhất, nhất quán, và tra cứu được ở mọi vị trí. Phân biệt này quan trọng vì nó thu hẹp câu hỏi rất nhiều. Nếu bộ lập kế hoạch của bạn chỉ cần nhìn một mét phía trước, độ sâu stereo tức thời đã đủ và bạn không cần chương này. Nhu cầu bản đồ dày chỉ thành thật khi robot phải nhớ chỗ nó đã đi qua.

Hình 32.1 đặt ba biểu diễn cạnh nhau và hỏi chúng cùng một câu. Đây là hình đáng nhìn kỹ nhất của phần đặt vấn đề, vì nó cho thấy khác biệt không nằm ở “chi tiết hơn” mà nằm ở *loại câu hỏi trả lời được*.



Hình 32.1. Cùng một hành lang, ba biểu diễn, và cùng một câu hỏi hỏi vào điểm đỏ ở giữa: chỗ này trống hay đặc? Bản đồ thưa không có câu trả lời — không phải vì thiếu chi tiết mà vì nó chưa bao giờ mô hình hoá khoảng không. Lưới trả lời được mọi chỗ nhưng trả bằng bộ nhớ tăng theo lũy thừa ba của độ phân giải. Biểu diễn neural trả lời được mọi chỗ và dồn tham số vào nơi có bề mặt, đổi lại bề mặt chỉ tồn tại ngầm dưới dạng một vùng mật độ cao.

TRỰC GIÁC

Bản đồ thưa giống một mạng lưới mốc trắc địa đóng trên nền đất. Đứng giữa công trường, bạn nắm ba cái mốc là biết chính xác mình ở đâu, tới từng centimét. Nhưng không ai đưa bản vẽ mốc trắc địa cho người lái xe cầu, vì nó không nói cái cột nào đang chắn đường. Bản đồ dày là bản vẽ mặt bằng: kém chính xác hơn ở từng điểm, nhưng nói được chỗ nào đi lọt. Hai thứ trả lời hai câu hỏi khác nhau, và một hệ SLAM tốt cho việc định vị hoàn toàn có thể vô dụng cho việc điều hướng.

A. Đối thủ thật của bản đồ neural không phải bản đồ thưa

Đây là chỗ phần lớn bài báo trong chương này lạng lế bỏ qua, và cũng là cổng đầu tiên bạn nên dựng. Nhu cầu **bản đồ dày** (*dense map*) có từ lâu và đã có lời giải cổ điển. KinectFusion hợp nhất độ sâu vào một lưới **trường khoảng cách có dấu** (*truncated signed distance field*), viết tắt TSDF, theo thời gian thực từ 2011 [316]. OctoMap cho **lưới chiếm dụng** (*occupancy grid*) dạng cây tám nhánh, tiết kiệm bộ nhớ, có mô hình xác suất rõ ràng, và đã nằm trong hệ điều hướng của rất nhiều robot [317]. Cả hai đều chạy được trên phần cứng nhúng, đều cho bề mặt hoặc thể tích trống có định nghĩa sạch, và đều đã được kiểm chứng hàng chục năm.

Vậy câu hỏi đúng không phải “thưa hay neural”. Nó là: bản đồ neural cho thêm gì so với TSDF, và đổi lại mất gì. Câu trả lời trung thực gồm ba khoản được và ba khoản mất, và cả sáu khoản đều nên nằm sẵn trong đầu bạn trước khi đọc bất kỳ bảng kết quả nào.

- **Được — nội suy chỗ chưa quan sát.** Lưới TSDF để trống ô nào chưa có tia đi qua. Trường neural, do là một hàm liên tục có thiên lệch về hàm trơn, tự lấp các khe nhỏ. Với cảnh quét thưa, khác biệt này rất dễ thấy.
- **Được — dựng được ảnh.** TSDF cho hình học nhưng không cho màu phụ thuộc góc nhìn. Muốn ảnh dựng ra giống ảnh thật, bạn cần một mô hình bức xạ, và đó chính là thứ neural rendering mang lại.
- **Được — tham số dồn theo nội dung.** Lưới đều tiêu bộ nhớ cho cả không khí. Trường neural và tập Gaussian tiêu tham số ở nơi có bề mặt.
- **Mất — bề mặt không tường minh.** Trong TSDF, bề mặt là tập mức 0. Trong trường mật độ hoặc tập Gaussian, không có định nghĩa nào tương đương, nên mọi thứ hạ nguồn cần bề mặt đều phải thêm một bước dựng lại.

- **Mất — không có mô hình xác suất chiếm dụng.** OctoMap nói “ô này bị chiếm với xác suất 0,8”. Mật độ của NeRF không phải xác suất chiếm dụng, và độ đục của một Gaussian càng không. Bộ lập kế hoạch nào cần biên an toàn sẽ không có thứ để dựa vào.
- **Mất — chi phí là vòng lặp tối ưu, không phải một lượt tích hợp.** TSDF cập nhật bằng một phép trộn có trọng số, chi phí cố định mỗi khung. Bản đồ neural cập nhật bằng vài chục bước hạ gradient, và chi phí đó không giảm được bằng lượng tử hoá hay TensorRT.

Khoản mất cuối cùng là khoản đắt nhất trên phần cứng nhúng, và mục 32.5 sẽ quy nó thành con số. Trước đó, hai mục tiếp theo đọc hai dòng công trình theo đúng mạch nút thắt.

32.2 Dòng NeRF: một MLP làm bản đồ, rồi mọi thứ sinh ra từ chỗ nó hỏng

Cơ chế dựng ảnh thể tích đã nằm ở [C/16/03-nerf-3dgs-va-hop-nhat](#), nên mục này không nhắc lại. Điều đáng học ở đây là chuyện khác: khi mang cơ chế đó vào một hệ SLAM chạy trực tuyến, những giả định nào vỡ, và mỗi công trình tiếp theo vá đúng chỗ nào.

A. iMAP — ý tưởng cực đoan nhất, và nó giải quyết việc gì

iMAP làm đúng một việc táo bạo: dùng *một* perceptron nhiều lớp làm toàn bộ bản đồ của một hệ SLAM RGB-D chạy trực tuyến [318]. Không có lưới, không có point cloud, không có TSDF. Cảnh là trọng số của một mạng nhỏ. **Bám vết theo mô hình** (*frame-to-model tracking*) là tối ưu tư thế khung hình mới sao cho ảnh và độ sâu dựng ra từ mạng khớp với quan sát. Dựng bản đồ là tiếp tục hạ gradient trên trọng số mạng, dùng các tia lấy mẫu từ một tập **khung hình chốt** (*keyframe*).

Nó giải quyết việc gì cụ thể? Một hệ TSDF phải chọn trước độ phân giải ô lưới và phải trả bộ nhớ cho toàn bộ thể tích. iMAP không phải chọn gì cả: sức chứa nằm ở số trọng số, và mạng tự phân bổ chi tiết. Với một căn phòng, tác giả báo cáo mô hình chỉ vài trăm nghìn tham số — nhỏ hơn nhiều bậc so với lưới tương đương. Thêm một điểm thiết kế đáng học: iMAP chọn khung hình chốt theo *lượng thông tin mới*, đo bằng sai số dựng ảnh của khung hình đó dưới bản đồ hiện tại. Đây là một tiêu chí có cơ sở, khác hẳn các luật chốt khung hình bằng ngưỡng số điểm khớp mà hệ cổ điển vẫn dùng.

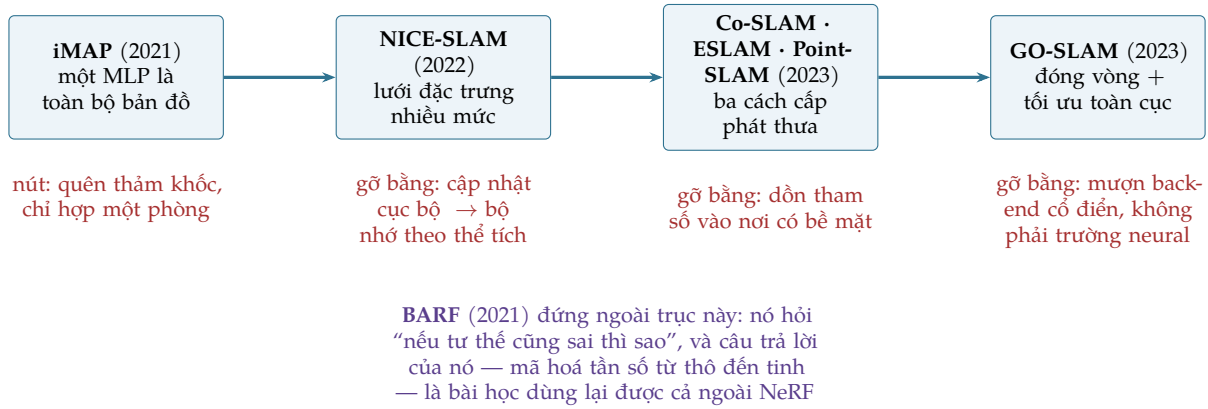
Cụm “vài trăm nghìn tham số” đáng dừng lại một nhịp, vì nó là lý lẽ mạnh nhất của cả dòng này. Điều đáng nhớ không phải một tỉ số cụ thể mà là *hai đại lượng đó lớn lên theo hai kiểu khác nhau*. Sức chứa của một MLP bốn tầng ẩn rộng 256 là một hằng số: nó không đổi khi căn phòng to ra. Bộ nhớ của một lưới thể tích thì tăng theo lũy thừa ba của độ phân giải và tỉ lệ với thể tích, lại còn trả tiền cho cả không khí lẫn tường. Nên ở độ phân giải vài centimet cho một căn phòng, biểu diễn ẩn gọn hơn hẳn — và càng gọn hơn khi bạn đòi độ phân giải mịn hơn. Tỉ số gọn hơn bao nhiêu lần thì phụ thuộc độ phân giải và kích thước phòng, nên nó không phải một hằng số trích dẫn được.

Cái giá thì lộ ra ngay. Vì toàn bộ cảnh nằm trong một bộ trọng số dùng chung, mỗi bước cập nhật ở phòng B đều làm đổi hàm ở phòng A. Đó là **quên thảm khốc** (*catastrophic forgetting*), và cách chữa duy nhất trong khuôn khổ ấy là liên tục phát lại các tia từ khung hình cũ — tức là trả chi phí tăng theo kích thước cảnh cho một biểu diễn vốn được quảng cáo là gọn. Các tác giả công bố hệ trên một GPU để bàn đời cao, với khâu dựng bản đồ chạy chậm hơn hẳn khâu bám vết. Chính chênh lệch đó, chứ không phải con số tốc độ, mới là điều đáng nhớ: khâu bản đồ là khâu bị ngân sách ép, và mọi công trình sau đều tấn công vào đúng nó. Phạm vi thí nghiệm là một căn phòng.

Mạch 1 — dòng NeRF làm bản đồ SLAM

- 1 Vấn đề** Bản đồ dày cổ điển phải chọn trước độ phân giải và trữ bộ nhớ cho cả không khí.
- Giải pháp** iMAP đặt toàn bộ cảnh vào trọng số một MLP, tối ưu bằng chính sai số ảnh và độ sâu [318].
- Giả định** cảnh cỡ một căn phòng, có độ sâu RGB-D, camera đi chậm, và một GPU để bàn luôn sẵn.
- Giá phải trả** mỗi khung hình cần vài chục bước hạ gradient thay vì một lượt tích hợp.
- Vấn đề mới** trọng số dùng chung cho cả cảnh, nên học chỗ mới làm hỏng chỗ cũ.
- 2 Vấn đề** Quên thảm họa, và chi phí phát lại tăng theo kích thước cảnh.
- Giải pháp** NICE-SLAM thay MLP đơn bằng lưới đặc trưng nhiều mức, mỗi bước cập nhật chỉ chạm các ô lưới quanh tia [319].
- Giả định** cảnh có biên giới hạn trước để cấp phát lưới; vẫn cần độ sâu.
- Giá phải trả** bộ nhớ quay lại phụ thuộc thể tích cảnh, đúng thứ iMAP muốn tránh.
- Vấn đề mới** lưới đều lại tiêu tham số cho không khí, và tra nhiều mức thì chậm.
- 3 Vấn đề** Lưới đều tốn bộ nhớ; MLP đơn quên; cần một chỗ đứng giữa.
- Giải pháp** ba lời giải song song — Co-SLAM ghép mã hoá toạ độ với bảng băm thưa [320]; ESLAM thay lưới ba chiều bằng ba mặt phẳng đặc trưng và dùng TSDF thay mật độ [321]; Point-SLAM neo đặc trưng lên chính các điểm đã quan sát, dày thưa theo độ chi tiết ảnh [322].
- Giả định** bề mặt chiếm phần rất nhỏ thể tích, nên cấu trúc thưa là đúng bản chất bài toán.
- Giá phải trả** thêm siêu tham số cấp phát, và mỗi lời giải mạnh trên một loại cảnh khác nhau.
- Vấn đề mới** cả ba vẫn không có tính nhất quán toàn cục: không đóng vòng, không hiệu chỉnh chùm tia toàn phần.
- 4 Vấn đề** Đi một vòng lớn rồi quay lại thì bản đồ lệch, và không có cơ chế nào kéo nó về.
- Giải pháp** GO-SLAM ghép một bộ bám vết dày kiểu DROID với bản đồ neural, thêm đóng vòng và tối ưu toàn cục, rồi biến dạng bản đồ theo tư thế đã sửa [323].
- Giả định** bản đồ neo được vào các khung hình chốt, nên sửa tư thế thì kéo được bản đồ theo.
- Giá phải trả** chi phí tăng thêm một bộ bám vết nặng, và toàn hệ cần bộ nhớ GPU lớn.
- Hệ quả** đây là mốc đầu tiên trong dòng này trả lời được nỗi đau đóng vòng — và nó trả lời bằng cách mượn lại kiến trúc của SLAM cổ điển, không phải bằng bản thân trường neural.

Hình 32.2 vẽ lại chính mạch đó trên một trục, để bạn nhìn thấy mỗi mốc gỡ nút nào.



Hình 32.2. Dòng NeRF làm bản đồ SLAM, đọc theo nút thất chứ không theo năm. Mỗi mũi tên là một câu trả lời cho đúng một chỗ hỏng của mốc trước, và mốc cuối đáng chú ý nhất vì nó gỡ nút bằng cách gọi lại kiến trúc SLAM cổ điển. Nếu bài toán của bạn không có nút thất tương ứng, mốc đó không mang lại gì cho bạn.

B. NICE-SLAM và ý nghĩa của tính cục bộ

NICE-SLAM chữa quên thăm khốc bằng một thay đổi nghe rất kỹ thuật nhưng thực ra là thay đổi khái niệm [319]. Thay vì để cảnh nằm trong trọng số, nó để cảnh nằm trong **lưới đặc trưng nhiều mức** (*multi-level feature grid*): mỗi ô lưới giữ một vectơ đặc trưng học được, còn các bộ giải mã nhỏ thì cố định. Muốn biết mật độ tại một điểm, ta nội suy đặc trưng từ các ô lân cận rồi cho qua bộ giải mã.

Hệ quả quan trọng nằm ở *giá đỡ của gradient*. Với MLP đơn, một tia ở phòng B sinh gradient chảy vào mọi trọng số, nên nó đổi hàm ở khắp nơi. Với lưới đặc trưng, tia đó chỉ chạm vài ô quanh nó. Cập nhật trở thành cục bộ, và quên thăm khốc biến mất gần hết. Hình 32.3 vẽ đúng khác biệt này.



Hình 32.3. Vì sao đổi từ trọng số sang lưới lại chữa được quên thăm khốc. Câu trả lời nằm ở giá đỡ của gradient: bên trái, một quan sát mới chạm tới toàn bộ tham số, nên nó bắt buộc phải đổi cả những vùng cảnh không liên quan; bên phải, quan sát chỉ chạm các ô lân cận. Đây là cùng một nguyên lý với việc chia bản đồ thành nhiều bản đồ con trong SLAM cổ điển: giới hạn phạm vi ảnh hưởng của một quan sát chính là cách mua lấy tính ổn định.

Cái giá thì đúng bằng thứ iMAP muốn tránh: bộ nhớ quay lại phụ thuộc thể tích cảnh, và bạn phải biết trước biên của cảnh để cấp phát. Ba công trình năm 2023 đi tìm chỗ đứng giữa hai cực đó, mỗi cái theo một hướng khác. Co-SLAM ghép một mã hoá toạ độ trơn với một bảng băm thưa, để có cả tính lặp khe của hàm liên tục lẫn tốc độ của bảng tra [320]. ESLAM bỏ lưới ba chiều, thay bằng ba mặt phẳng đặc trưng vuông góc nhau, nên bộ nhớ tăng theo bình phương chứ không theo lũy thừa ba của độ phân giải; nó cũng đổi mật độ lấy TSDF, tức là kéo bề mặt về lại chỗ có định nghĩa sạch [321]. Point-SLAM đi xa nhất theo hướng “chỉ cấp phát nơi có vật”:

đặc trưng neo trên chính đám mây điểm đã quan sát, và mật độ điểm được điều chỉnh theo độ biến thiên của ảnh, tức là dồn sức chứa vào chỗ nhiều chi tiết [322].

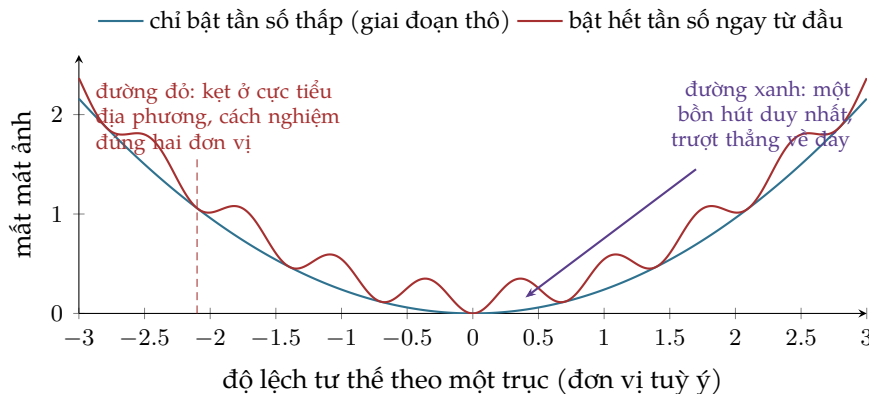
Ba lời giải này đáng nhớ không phải vì tên của chúng mà vì chúng là ba câu trả lời cho cùng một câu hỏi: *đặt tham số ở đâu*. Đó cũng đúng là câu hỏi bạn sẽ phải trả lời nếu tự dựng một bản đồ dày cho hệ của mình.

C. BARF: khi tư thế cũng là ẩn số, và bài học từ thô đến tinh

Cả nhóm trên giả định tư thế camera đã có, hoặc ít nhất có một ước lượng ban đầu tốt. BARF hỏi câu ngược lại: nếu tối ưu tư thế *cùng lúc* với trường thì sao [324]. Về mặt hình thức, đây chính là **hiệu chỉnh chùm tia** (*bundle adjustment*), chỉ khác là điểm 3D được thay bằng một trường liên tục và phần dư được đo trên chính điểm ảnh.

Phát hiện quan trọng của BARF không phải kết quả mà là *nguyên nhân của thất bại*. **Mã hoá vị trí** (*positional encoding*) bằng chuỗi sin và cos nhiều tần số là thứ cho NeRF độ sắc nét. Nhưng chính nó phá hỏng gradient theo tư thế. Lý do là đạo hàm của một thành phần tần số k có biên độ tỉ lệ với k . Các thành phần tần số cao vì thế tạo ra một mặt phẳng mất mát lớn chồm với vô số cực tiểu địa phương cách nhau rất gần. Hạ gradient trên tư thế rơi vào cái hố gần nhất và đứng lại đó.

Lời giải của BARF là che dần: bật các tần số từ thấp lên cao theo tiến trình tối ưu. Giai đoạn đầu, mặt phẳng mất mát trơn và có một bồn hút rộng, nên tư thế đi đúng hướng. Giai đoạn sau, tần số cao mở ra và chi tiết trở lại. Hình 32.4 vẽ minh hoạ mặt cắt một chiều của hai mặt phẳng mất mát đó.



Hình 32.4. Mặt cắt một chiều của mặt phẳng mất mát theo tư thế, hình minh hoạ chứ không phải số đo. Đường đỏ là điều xảy ra khi mọi tần số của mã hoá vị trí đều bật: biên độ đạo hàm của thành phần tần số k tỉ lệ với k , nên các gợn tần số cao sinh ra hàng loạt cực tiểu địa phương gần nhau. Đường xanh là cùng bài toán khi chỉ bật tần số thấp: một bồn hút duy nhất, đủ rộng để hạ gradient tìm về đáy. BARF bật dần từ đường xanh sang đường đỏ trong quá trình tối ưu.

Bài học ở đây quan trọng hơn bản thân BARF, và nó nói bằng đúng ngôn ngữ bạn đã quen. Mọi phương pháp căn chỉnh theo cường độ ảnh đều cần một lịch làm trơn. Các hệ trực tiếp cổ điển gọi nó là kim tự tháp ảnh: căn chỉnh ở mức thô trước, rồi mới xuống mức tinh. BARF phát hiện lại đúng nguyên lý đó trong không gian tần số của mã hoá vị trí. Nếu một ngày bạn ghép một hàm mất mát quang trắc vào hệ của mình — dù là để tinh chỉnh tư thế, để căn chỉnh với bản đồ, hay để hiệu chỉnh trực tuyến — thì câu hỏi đầu tiên phải hỏi là: lịch làm trơn của tôi ở đâu.

D. Bốn nhánh còn lại, mỗi nhánh một câu hỏi khác

Bốn công trình còn lại trong danh sách không nằm trên cùng một trục với nhóm trên, vì mỗi cái hỏi một câu riêng.

- **NeRF-SLAM — bỏ độ sâu cảm biến đi thì sao?** Nó ghép một bộ bám vết dày đơn ảnh với một bản đồ kiểu Instant-NGP, và điểm thiết kế đáng học là dùng *độ bất định* của độ sâu ước lượng làm trọng số cho hàm mất mát dựng bản đồ [325]. Đây chính là tư duy ma trận thông tin của đồ thị nhân tử, mang sang phía bản đồ. Nếu bạn chỉ đọc một ý từ cả nhóm này, hãy đọc ý đó.
- **NICER-SLAM — chỉ có RGB thì lấy hình học ở đâu?** Nó thay độ sâu cảm biến bằng các tín hiệu học được: độ sâu đơn ảnh, pháp tuyến bề mặt, và ràng buộc căn chỉnh giữa các khung hình [326]. Nói cách khác, khi mất một ràng buộc vật lý thì phải mượn một tiên nghiệm học được để thay — và mọi giới hạn của tiên nghiệm đó, đặc biệt là vấn đề tỉ lệ, đi theo vào hệ. Chỗ này nổi thẳng sang F/30.
- **vMAP — vì sao cả cảnh phải nằm chung một trường?** Nó cấp cho mỗi vật thể một mạng nhỏ riêng, rồi hợp thành cảnh lúc dựng ảnh [327]. Cái được rất đáng chú ý: vật thể quan sát được một phần vẫn ra hình dạng kín, vì mỗi mạng chỉ phải học một vật chứ không phải cả căn phòng. Đây là bước bắc cầu sang bản đồ mức đối tượng ở F/33.
- **GO-SLAM — ai kéo bản đồ về khi đóng vòng?** Nó thêm phát hiện vòng lặp và tối ưu toàn cục, rồi biến dạng bản đồ neural theo tư thế đã sửa [323]. Đáng chú ý là cách nó làm được: bản đồ được neo vào các khung hình chốt, nên sửa tư thế thì kéo được bản đồ theo. Bài học chung sẽ trở lại ở mục sau: *biểu diễn nào neo được vào tư thế thì sửa được sau khi đóng vòng*.

E. Giải phẫu hàm mất mát, để đọc bài nhanh hơn

Mọi hệ trong dòng này tối ưu một tổng của vài số hạng, và bốn số hạng dưới đây gần như luôn có mặt. Nhận ra chúng giúp bạn đọc một bài mới trong mười phút thay vì một giờ, vì phần khác biệt giữa các bài thường chỉ là trọng số và một số hạng phụ.

- **Số hạng màu** — sai lệch giữa ảnh dựng và ảnh thật, trên các tia lấy mẫu. Đây là số hạng duy nhất bắt buộc, và cũng là số hạng *không* ràng buộc hình học, đúng như phép tính ở mục sau cho thấy.
- **Số hạng độ sâu** — sai lệch giữa độ sâu dựng ra và độ sâu cảm biến. Đây mới là số hạng thực sự dựng hình học, và nó biến mất trong các hệ chỉ có RGB. Bỏ nó đi thì phải mượn tiên nghiệm học được để thay.
- **Số hạng không gian trống** — ép mật độ về không dọc phần tia trước điểm chạm. Nghe phụ nhưng rất quan trọng: nếu thiếu, mô hình thoải mái để lại vật chất giả lơ lửng, vì không có gì phạt nó.
- **Số hạng chính quy hoá** — ép bề mặt trơn, ép Gaussian dẹt và bám mặt, ép pháp tuyến nhất quán. Đây là chỗ người ta *trả lại* phần ràng buộc hình học đã cố tình bỏ đi lúc đầu.

Một quy tắc đọc rút ra: xem tỉ lệ giữa số hạng độ sâu và số hạng màu. Tỉ lệ nghiêng về màu thì bài đó đang tối ưu cho ảnh đẹp; nghiêng về độ sâu thì đang tối ưu cho hình học đúng. Rất ít bài nói thẳng điều này, nhưng bảng siêu tham số luôn nói.

Bảng 32.1. Dòng NeRF làm bản đồ SLAM. Cột cuối là cột đáng đọc nhất: nó cho biết bạn sẽ gặp gì khi mang hệ đó ra khỏi điều kiện mà nó được công bố.

Hệ	Cơ chế cốt lõi	Giả định	Hỏng khi nào
iMAP	Một MLP là toàn bộ bản đồ; bám vết và dựng bản đồ đều bằng hạ gradient	RGB-D, một phòng, GPU để bàn	Cảnh lớn dần: quên thăm khốc, chi phí phát lại tăng theo diện tích
NICE-SLAM	Lưới đặc trưng nhiều mức, bộ giải mã cố định, cập nhật cục bộ	Biên cảnh biết trước để cập phát lưới	Cảnh lớn hoặc mở: bộ nhớ theo thể tích; không có đóng vòng
Co-SLAM	Mã hoá toạ độ trơn ghép bảng băm thưa	Bề mặt trơn từng mảnh	Va chạm bám ở vùng chi tiết dày
ESLAM	Ba mặt phẳng đặc trưng thay lưới 3D; TSDF thay mật độ	Cảnh không quá nhiều lớp chồng theo mỗi trục	Cảnh phức tạp làm ba mặt phẳng chồng lẫn thông tin
Point-SLAM	Đặc trưng neo trên điểm đã quan sát, dày theo chi tiết ảnh	Độ sâu đủ tốt để đặt điểm đúng chỗ	Độ sâu nhiễu đặt điểm sai; vùng chưa quan sát để trống
NeRF-SLAM	Bám vết dày đơn ảnh + bản đồ bám, trọng số theo bất định độ sâu	Bộ bám vết đủ tốt và chạy được thời gian thực	Bộ bám vết trôi thì bản đồ nhận sai; nặng GPU
NICER-SLAM	Chỉ RGB, mượn độ sâu và pháp tuyến học được làm ràng buộc	Tiên nghiệm đơn ảnh đúng cho loại cảnh này	Ra ngoài phân phối huấn luyện; tỉ lệ trôi
vMAP	Mỗi vật một mạng nhỏ, hợp lúc dựng ảnh	Có bộ tách vật thể đủ ổn định	Tách nhầm vật; cảnh không phân được thành vật
GO-SLAM	Bám vết dày + đóng vòng + tối ưu toàn cục, bản đồ biến dạng theo	Bản đồ neo được vào khung hình chốt	Ngân sách GPU lớn; độ trễ khi biến dạng bản đồ lớn

32.3 Dòng 3D Gaussian Splatting: vì sao rasteri hoá đổi cuộc chơi

Từ 2023 trở đi gần như cả lĩnh vực chuyển sang 3D Gaussian splatting. Chuyện này thường được kể như “nhanh hơn nên tốt hơn”, và cách kể đó bỏ mất phần đáng học. Khoản chênh tốc độ đến từ một thay đổi thuật toán duy nhất, và chính thay đổi ấy kéo theo ba hệ quả rất hợp với SLAM.

A. Khác biệt cơ chế, nói gọn trong một đoạn

NeRF phải *đi tìm* vật chất: với mỗi điểm ảnh, nó lấy hàng chục mẫu dọc tia và chạy mạng ở từng mẫu, vì nó không biết trước mặt nằm ở đâu. 3DGS đã lưu sẵn vật chất thành các nguyên thể tường minh, nên nó chỉ cần *rasterize*: chiếu chúng xuống mặt ảnh, sắp theo độ sâu rồi trộn — đúng việc mà phần cứng đồ hoạ sinh ra để làm [175]. Chi phí mỗi điểm ảnh vì thế đổi từ “số mẫu dọc tia” thành “số nguyên thể phủ lên điểm ảnh đó”, và con số thứ hai nhỏ hơn hẳn. Cơ chế đầy đủ nằm ở [C/16/03-nerf-3dgs-va-hop-nhat](#); ở đây ta chỉ cần ba hệ quả của nó.

Chênh lệch nằm ở đâu thì viết ra được mà không cần một con số nào. Chi phí một khung hình của NeRF là (số điểm ảnh) $\times$ (số mẫu dọc mỗi tia) $\times$ (chi phí *một lần chạy mạng*). Của 3DGS là (số điểm ảnh) $\times$ (số nguyên thể phủ lên điểm ảnh đó) $\times$ (chi phí *một phép trộn*). Hai công thức có chung thừa số đầu và khác hẳn ở hai thừa số còn lại: số nguyên thể phủ lên một điểm ảnh nhỏ hơn số mẫu dọc một tia, và một phép trộn số học rẻ hơn một lần chạy mạng rất nhiều. Hai thừa số cùng nhỏ đi, nhân với nhau, là lý do khoảng cách lên tới nhiều bậc độ lớn — và đó là chênh lệch về *bản chất thuật toán*, không phải về cài đặt khéo hay vụng.

- **Cập nhật bản đồ trở thành sửa chữa cục bộ.** Thêm quan sát mới nghĩa là thêm vài nghìn Gaussian ở vùng vừa thấy và tinh chỉnh các Gaussian lân cận. Không có tham số dùng chung toàn cục nào bị chạm. Quên thăm hoạ, nút thắt lớn nhất của iMAP, biến mất theo kiến trúc chứ không nhờ mẹo huấn luyện.
- **Bản đồ dịch chuyển được sau khi đóng vòng.** Mỗi Gaussian có một vị trí tường minh. Gắn nó vào khung hình chốt đã sinh ra nó, rồi khi tối ưu toàn cục sửa tư thế khung hình đó, ta dịch chuyển cứng cả cụm Gaussian theo. Với một MLP đơn, phép tương đương không tồn tại: không có cách nào “dịch một phần của trọng số”.
- **Đạo hàm theo tư thế có dạng giải tích.** Phép chiếu Gaussian là một chuỗi phép biến đổi trơn, nên gradient của sai số ảnh theo sáu bậc tự do của camera viết ra được, không cần đi qua mạng. Đây là điều làm bám vết theo mô hình chạy được ở tốc độ dùng được [328].

Hệ quả thứ hai đáng dừng lại một nhịp, vì nó chạm đúng một trong bốn nỗi đau của bạn. Trong hệ hiện tại, đóng vòng sửa tư thế rồi kéo các điểm bản đồ theo, và toàn bộ việc đó rẻ vì bản đồ là một tập điểm rời rạc gắn với khung hình chốt. Một bản đồ Gaussian giữ nguyên tính chất đó. Một bản đồ MLP thì không. Nếu bạn phải chọn một biểu diễn dày cho một hệ có đóng vòng, tiêu chí “sửa được sau khi đóng vòng” quan trọng hơn hẳn tiêu chí PSNR.

I Mạch 2 — dòng Gaussian làm bản đồ SLAM

- Vấn đề** Dòng NeRF chậm ngay cả trên GPU để bản — chậm tới mức khâu dựng bản đồ không theo kịp khâu bám vết — và lại quên chỗ cũ khi đi sang chỗ mới.

Giải pháp thay trường ảnh bằng tập Gaussian tường minh, dựng ảnh bằng raster hoá thay vì lấy mẫu dọc tia [175].

Giả định bề mặt xấp xỉ được bằng các hạt mờ dị hướng, và số hạt phủ lên mỗi điểm ảnh không quá lớn.

Giá phải trả dung lượng bản đồ lớn hơn hẳn một MLP, và từ đây bộ nhớ chứ không phải tốc độ mới là ràng buộc quyết định.

Hệ quả cập nhật thành cục bộ, nên quên thăm khốc biến mất theo kiến trúc.
- Vấn đề** Bám vết cần đạo hàm của sai số ảnh theo tư thế, và đi qua một mạng thì vừa chậm vừa ồn.

Giải pháp MonoGS viết thẳng Jacobian giải tích của phép chiếu Gaussian theo tư thế trên nhóm Lie [328].

Giả định chuyển động giữa hai khung hình đủ nhỏ để nghiệm nằm trong bồn hút quang trắc.

Giá phải trả vẫn là bám vết theo mô hình, nên lỗi bản đồ chảy thẳng vào quỹ đạo.

Vấn đề mới chuyển động nhanh và ảnh mờ đẩy nghiệm ra ngoài bồn hút, và không có gì kéo nó về.
- Vấn đề** Vòng lặp dựng ảnh nằm trên đường ước lượng tư thế nên vừa chậm vừa rủi ro.

Giải pháp GS-ICP tách hẳn hai việc: tư thế do ICP tổng quát trên tâm Gaussian, dựng ảnh chỉ là sản phẩm [329]. Photo-SLAM đi xa hơn, giữ nguyên đặc trưng và hiệu chỉnh chùm tia cổ điển [330].

Giả định đã có một nguồn tư thế đủ tốt — và với bạn thì điều này đúng.

Giá phải trả bản đồ không còn đóng góp gì cho quỹ đạo; nó chỉ là bên tiêu thụ.

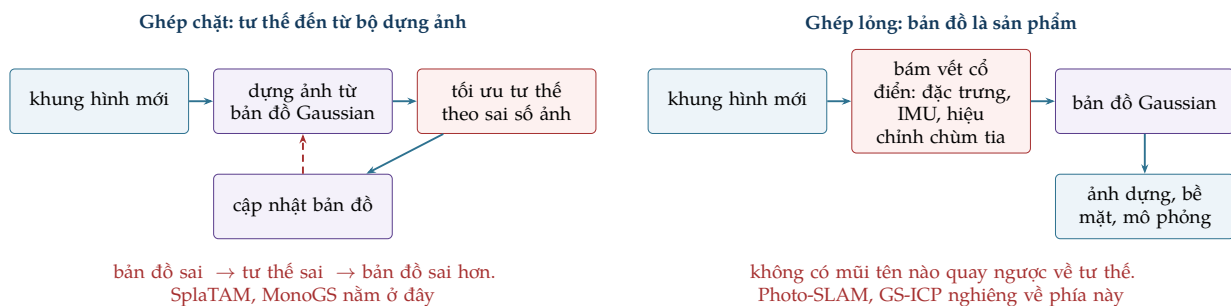
Hệ quả đây chính là mẫu ghép lỏng, và là cách vào rẻ nhất cho một hệ đang chạy sản phẩm.
- Vấn đề** Một triệu Gaussian không vừa ngân sách bộ nhớ và băng thông của máy nhúng.

Giải pháp	RTG-SLAM ép mỗi Gaussian hoặc đục hẳn hoặc trong hẳn, để một bề mặt cần ít hạt hơn nhiều [331].
Giả định	bề mặt trong cảnh phần lớn là đục, không bán trong suốt.
Giả phải trả	mất khả năng biểu diễn khói, kính, và vật phản chiếu.
Vấn đề mới	số hạt vẫn tăng theo diện tích đã đi qua, nên vấn đề chỉ được đẩy lùi chứ chưa được giải.

B. Sáu hệ, và điểm thiết kế đáng học của mỗi hệ

- **SplaTAM** — hệ RGB-D thuần Gaussian: bám vết bằng cách dựng ảnh từ bản đồ rồi tối ưu tư thế, dựng bản đồ bằng cách thêm Gaussian ở vùng chưa được phủ [332]. Điểm thiết kế đáng học là **mặt nạ hiển thị** (*visibility mask*): hệ chỉ tính mất mát ở các điểm ảnh mà bản đồ hiện có thật sự phủ tới. Nói theo ngôn ngữ của bạn, đó là một cổng loại bỏ phần dư không có cơ sở — cùng vai trò với việc bỏ các điểm không quan sát được trước khi tính phần dư trong hiệu chỉnh chùm tia.
- **MonoGS** — đưa 3DGS về đơn ảnh, và viết ra đạo hàm giải tích của sai số ảnh theo tư thế camera trong nhóm Lie [328]. Đây là bài trong nhóm gần với gu của một người làm tối ưu trên đa tạp nhất; nếu bạn định đọc mã nguồn một hệ 3DGS SLAM, hãy đọc hệ này.
- **GS-ICP SLAM** — tách hẳn bám vết khỏi dựng ảnh: tư thế do một biến thể ICP tổng quát tính trên chính tâm của các Gaussian, còn dựng ảnh chỉ là sản phẩm [329]. Đây là bằng chứng trực tiếp cho luận điểm trung tâm của chương: khi bỏ vòng lặp dựng ảnh ra khỏi đường ước lượng tư thế, hệ nhanh hơn hẳn, và chất lượng quỹ đạo không xấu đi.
- **Photo-SLAM** — giữ nguyên xương sống đặc trưng và hiệu chỉnh chùm tia cổ điển cho tư thế, gắn thêm một bản đồ Gaussian chỉ để dựng ảnh [330]. Đây là mẫu kiến trúc mà tôi cho là đáng bắt chước nhất cho một hệ đang chạy sản phẩm, vì nó không đặt tư thế vào tay bộ dựng ảnh.
- **RTG-SLAM** — tấn công thẳng vào số lượng Gaussian: ép mỗi Gaussian hoặc đục hẳn hoặc trong hẳn, để một bề mặt cần ít hạt hơn nhiều [331]. Nó cho thấy số Gaussian là một biến thiết kế chứ không phải hằng số của bài toán — điều rất quan trọng khi bạn có ngân sách bộ nhớ cố định.
- **OpenGS-SLAM** — dòng ghép ngữ nghĩa mở vào bản đồ Gaussian, để truy vấn bản đồ bằng ngôn ngữ. Tôi *không* chắc về chi tiết và kết quả của công trình này, nên chỉ nhắc tên và ý tưởng; phần ngữ nghĩa mở được bàn kỹ ở F/33.

Sáu hệ trên chia thành hai kiểu ghép nối, và phân biệt được hai kiểu đó quan trọng hơn nhớ tên từng hệ. Hình 32.5 vẽ cả hai.



Hình 32.5. Hai kiểu ghép nối, và đây là phân biệt quan trọng nhất của cả chương. Bên trái, tư thế được ước lượng bằng cách so ảnh dựng với ảnh thật, nên mọi lỗi bản đồ chày thẳng vào quỹ đạo; mũi tên đứt là vòng phản hồi đó. Bên phải, tư thế vẫn do đường cổ điển tính, còn bản đồ neural chỉ tiêu thụ tư thế; nó không thể làm quỹ đạo xấu đi, và cũng không thể làm quỹ đạo tốt lên. Chọn kiểu nào là quyết định kiến trúc, không phải chi tiết cài đặt.

Bảng 32.2. Dòng 3D Gaussian splatting làm bản đồ SLAM. Cột “tư thế đến từ đâu” quyết định hệ đó có thể làm hồng quỹ đạo của bạn hay không.

Hệ	Tư thế đến từ đâu	Điểm thiết kế đáng học	Hồng khi nào
SplaTAM	Dựng ảnh khác biệt được, RGB-D	Mặt nạ hiển thị: chỉ tính mất mát ở điểm ảnh bản đồ thật sự phủ	Độ sâu thiếu hoặc nhiều; chuyển động lớn giữa hai khung hình
MonoGS	Dựng ảnh, đơn ảnh, đạo hàm giải tích trên nhóm Lie	Viết thẳng Jacobian theo tư thế thay vì vi phân tự động	Tỉ lệ đơn ảnh; cảnh ít vân; chuyển động nhanh
GS-ICP SLAM	ICP tổng quát trên tâm Gaussian	Tách bám vết khỏi dựng ảnh, và hệ nhanh lên rõ rệt	Hình học nghèo nàn làm ICP trượt; vẫn cần độ sâu
Photo-SLAM	Đặc trưng + hiệu chỉnh chùm tia cổ điển	Bản đồ neural là bên tiêu thụ tư thế, không phải bên tạo ra nó	Bản đồ ảnh xấu ở nơi bám vết thừa khung hình chốt
RTG-SLAM	Bám vết dày, RGB-D	Ép Gaussian đục hoặc trong để giảm mạnh số hạt	Bề mặt bán trong suốt và vật phản chiếu
OpenGS-SLAM	<i>chưa kiểm chứng</i>	Ghép ngữ nghĩa mở vào bản đồ Gaussian	<i>chưa kiểm chứng</i>

C. Chỗ nổi rẻ nhất vào hệ bạn đang có

Nếu bạn muốn thử dòng này mà không đung vào đường ước lượng tư thế, có một chỗ nổi rất tự nhiên, và ít bài báo nói ra vì với họ nó hiển nhiên.

Bản 3DGS gốc khởi tạo các Gaussian từ đám mây điểm thừa của một pipeline dựng cảnh cổ điển. Hệ của bạn *đã* sinh ra đúng loại đám mây đó, mỗi khung hình chốt, kèm tư thế đã được hiệu chỉnh chùm tia. Nghĩa là bạn có sẵn đầu vào tốt nhất mà một hệ 3DGS có thể mơ tới: điểm 3D đã lọc ngoại lai, tư thế đã tối ưu toàn cục, và quan hệ đồng quan sát giữa các khung hình chốt.

Từ đó ra một kiến trúc ghép lỏng rất gọn, và nó không thêm rủi ro nào cho quỹ đạo:

- **Gieo hạt.** Mỗi điểm bản đồ thành một Gaussian, tỉ lệ khởi tạo theo khoảng cách tới các điểm lân cận.
- **Neo vào khung hình chốt.** Mỗi Gaussian ghi nhớ khung hình chốt đã sinh ra nó. Khi đóng vòng sửa tư thế khung hình đó, cả cụm Gaussian dịch theo bằng một phép biến đổi cứng.
- **Chia bản đồ con.** Dùng chính đồ thị đồng quan sát để cắt cảnh thành các bản đồ con, rồi chỉ tối ưu bản đồ con đang hoạt động. Đây là cách duy nhất giữ ngân sách bộ nhớ hữu hạn khi robot đi mãi.
- **Chạy lịch nhip.** Khâu Gaussian chạy ở tần số thấp, trên một luồng riêng, và được phép rút khung hình. Nó là bên tiêu thụ, nên rút khung chỉ làm ảnh xấu đi chứ không làm hồng quỹ đạo.

Đây không phải một phương pháp mới; nó là mẫu Photo-SLAM viết lại bằng các khối bạn đã có. Giá trị của nó nằm ở chỗ rủi ro gần bằng không: nếu bỏ hắt khâu Gaussian đi, hệ quay về đúng trạng thái hiện tại, không mất gì.

D. Ba chỗ dòng này vẫn chưa chạm tới

Đọc xong bảng trên dễ có cảm giác vấn đề đã xong. Chưa. Ba chỗ vẫn để ngỏ, và cả ba đều nằm đúng chỗ hệ của bạn đau.

- **Chuyển động nhanh.** Bám vết bằng dựng ảnh là một bài toán tối ưu quang trắc, nên nó có bồn hút hữu hạn. Chuyển động lớn giữa hai khung hình đẩy nghiệm ra ngoài bồn, và cách

chứa duy nhất trong khuôn khổ đó là một dự đoán ban đầu tốt — tức là quay lại cần IMU hoặc cần ghép đặc trưng. Đây chính là bài học của BARF nói lại lần nữa.

- **Ảnh mờ do chuyển động.** Một khung hình mờ không phải quan sát nhiễu của cảnh tĩnh. Nó là tích phân của cảnh dọc một quãng chuyển động. Hàm mất mát quang trắc không có chỗ nào biểu diễn điều đó, nên tối ưu sẽ giải thích vết mờ bằng cách nhét thêm hình học sai. Các dòng khử mờ có ý thức về chuyển động, như BAD-NeRF và BAD-Gaussians, đưa mô hình phơi sáng vào bộ dựng ảnh; chúng được bàn ở F/34.
- **Phơi sáng và cân bằng trắng tự động.** Cùng một cơ chế: mô hình buộc phải giải thích thay đổi độ sáng bằng hình học. Các hệ trong chương này thường chạy trên chuỗi có phơi sáng cố định, và điều đó ít khi được ghi rõ.

E. Điều cả hai dòng đều thiếu: một bài toán tối ưu chung

Có một khoảng trống chung cho toàn bộ chương, và nó là khoảng trống mà một người quen đồ thị nhân tử sẽ thấy ngay.

Hiệu chỉnh chùm tia tối ưu tư thế và cấu trúc *cùng lúc*, trong một bài toán bình phương tối thiểu duy nhất. Nó cho ba thứ đi kèm: nghiệm hội tụ bậc hai gần cực tiểu, một **ma trận thông tin** (*information matrix*) để biết mình chắc chắn tới đâu, và một chỗ tự nhiên để cấm thêm nhân tử — tiền tích phân IMU, ràng buộc đóng vòng, tiên nghiệm hiệu chỉnh.

Gần như mọi hệ trong chương này làm khác hẳn. Chúng *luân phiên*: cố định bản đồ rồi tối ưu tư thế vài bước, cố định tư thế rồi tối ưu bản đồ vài bước. Đây là **hạ theo khối toạ độ** (*block coordinate descent*), và nó kém hơn ở đúng ba chỗ vừa kể.

- **Hội tụ chậm và phụ thuộc lịch trình.** Số bước mỗi phía trở thành siêu tham số, và nó tương tác với ràng buộc thời gian thực. Chạy trên máy chậm hơn nghĩa là chạy ít bước hơn, tức là một hệ *khác*. Đây là lý do sâu xa vì sao kết quả trên Orin không suy ra được từ kết quả trên máy để bàn.
- **Không có ma trận thông tin.** Không có hiệp phương sai của tư thế, nên không có gì để cấm vào một bộ lọc, không có gì để so bằng kiểm định chi bình phương, và không có cách nào phát hiện hệ đang không chắc chắn.
- **Không có chỗ cho IMU.** Đây là chỗ thiếu nghiêm trọng nhất với hệ của bạn. Trong khuôn khổ luân phiên, không có chỗ tự nhiên nào để đặt một nhân tử tiền tích phân IMU với trọng số đúng. Vài công trình gần đây có ghép IMU vào, thường bằng cách dùng nó làm dự đoán ban đầu; tôi *không* nắm chắc kết quả của chúng và không mô tả số liệu.

Nói ngắn gọn: dòng này mạnh về biểu diễn và yếu về ước lượng. Hệ của bạn thì ngược lại. Đó chính là lý do mẫu ghép lỏng không phải một thoả hiệp tạm bợ, mà là cách ghép đúng thể mạnh của hai bên.

I 32.4 Hai trục đánh giá, và vì sao chúng bị trộn lẫn

Đây là mục quan trọng nhất của chương. Nếu bạn chỉ đọc một mục, đọc mục này.

A. Hai câu hỏi hoàn toàn khác nhau

Một hệ trong chương này báo cáo hai nhóm con số. Nhóm thứ nhất là chất lượng ảnh dựng lại: PSNR, SSIM, LPIPS [69], [189]. Nhóm thứ hai là độ chính xác quỹ đạo: **sai số quỹ đạo tuyệt đối** (*absolute trajectory error*), viết tắt ATE, đôi khi kèm RPE. Hai nhóm này trả lời hai câu hỏi khác nhau và có thể tách rời hoàn toàn.

Lý do tách rời đã được nêu ở [C/16/03-nerf-3dgs-va-hop-nhat](#) và đáng nhắc lại một câu: hàm mất mát chỉ ràng buộc ảnh dựng ra, không ràng buộc hình học. Màu phụ thuộc góc nhìn cho mô hình thêm một đường thoát: nó có thể “vẽ” thứ đáng lẽ phải “dựng”. Vì thế PSNR cao chứng minh mô hình khớp được các ảnh đã thấy, không chứng minh bản đồ đúng, và càng không chứng minh tư thế đúng.

Chiều ngược lại cũng đúng và ít được nói tới. Một hệ có ATE rất tốt vẫn có thể cho ảnh dựng xấu, đơn giản vì nó dành ít tài nguyên cho bản đồ. Photo-SLAM là ví dụ có chủ ý theo hướng ngược lại: quỹ đạo do đường cổ điển bảo đảm, còn ảnh đẹp là phần thêm.

Hình 32.6 bày cả hai trục lên một mặt phẳng. Đây là hình bạn nên vẽ lại cho chính hệ của mình.



Hình 32.6. Hai trục đánh giá trên một mặt phẳng; các nhãn là góc phân tư khái niệm, không phải vị trí đo được của hệ cụ thể nào. Bảng kết quả chỉ có cột PSNR không phân biệt được góc trên bên phải với góc dưới bên phải, và đó chính là chỗ người đọc bị dẫn sai. Quy tắc thực dụng: trước khi tin một hệ, hãy hỏi nó nằm ở đâu theo trục dọc, đo bằng đúng giao thức mà bạn dùng cho hệ hiện tại.

Bảng 32.3. Ba nhóm thước đo cho cùng một hệ. Cột cuối là cột hay bị bỏ qua nhất, và nó là lý do một hệ có thể đứng đầu bảng ở cột đầu mà vẫn không dùng được cho robot.

Nhóm	Đo cái gì	Trả lời được câu hỏi nào	Không chứng minh được gì
PSNR, SSIM, LPIPS	Ảnh dựng gần ảnh thật tới đâu	Bản đồ có dùng được để xem, để mô phỏng cảm biến không	Không nói gì về hình học, và không nói gì về tư thế
ATE, RPE	Quỹ đạo lệch chân trị bao nhiêu	Hệ có định vị được không — câu hỏi quyết định của SLAM	Không nói bản đồ có dày, có kín, có dùng để tránh vật cản được không
Sai số hình học: độ sâu L1, độ chính xác và độ phủ, F-score với lưới chân trị	Bề mặt dựng ra lệch bề mặt thật bao nhiêu	Bản đồ có dùng để lập kế hoạch, để đo đạc được không	Không nói ảnh có đẹp không, và cần chân trị hình học nên hiếm khi có ngoài đời

B. Ví dụ tính tay: vì sao một sai lệch hình học lớn chỉ tốn một điểm ảnh

Câu “PSNR không chứng minh hình học” nghe như một lời cảnh báo chung chung. Nó không phải vậy. Nó là một bất đẳng thức số học, và tính ra được trong ba dòng.

Trước khi tính, nói rõ một điều: **ví dụ dưới đây là một bài toán giả định hoàn toàn**. Mọi con số trong đó đều do mình đặt ra để thay vào công thức, không con số nào đến từ một phép đo trên thiết bị, cũng không con số nào trích từ một bài báo. Thứ đáng mang đi là *hình dạng* của quan hệ — Δu tỉ lệ thuận với khoảng dịch camera và tỉ lệ nghịch với bình phương độ sâu — chứ không phải con số ở vế phải.

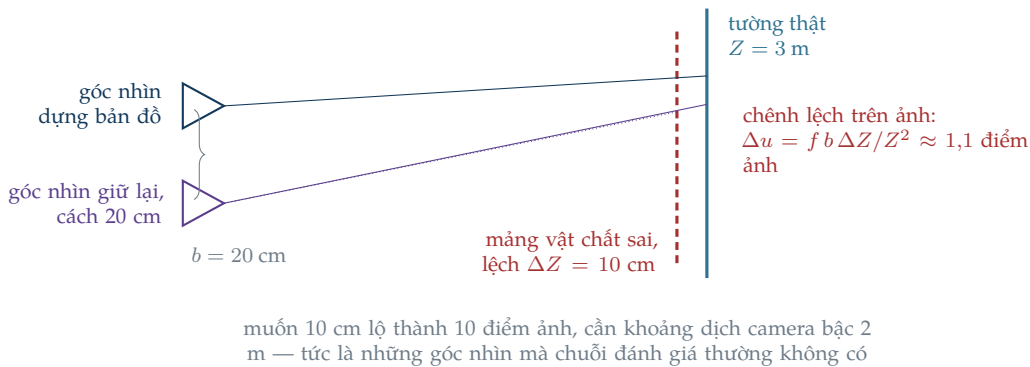
Xét một bức tường phẳng cách camera $Z = 3$ m. Giả sử bản đồ đặt sai một mảng vật chất, lệch $\Delta Z = 10$ cm về phía trước tường. Với robot, sai số này rất nghiêm trọng: nó là chênh lệch giữa dừng đúng và va vào tường. Bây giờ hỏi xem nó lộ ra bao nhiêu trong ảnh.

Một điểm ở độ sâu Z , nhìn từ một camera dịch ngang một đoạn b , có vị trí ảnh dịch đi một lượng tỉ lệ với fb/Z . Sai một lượng ΔZ làm vị trí ảnh lệch đi

$$\Delta u \approx \frac{fb\Delta Z}{Z^2} = \frac{500 \times 0,2 \times 0,1}{3^2} \approx 1,1 \text{ điểm ảnh},$$

với tiêu cự $f = 500$ điểm ảnh và khoảng dịch camera $b = 20$ cm. *Một điểm ảnh*. Trên một ảnh mà mô hình còn được phép chỉnh màu theo góc nhìn, một điểm ảnh chìm hoàn toàn trong nhiễu cảm biến và trong sai số nội suy. PSNR gần như không đổi. Bản đồ thì sai 10 cm.

Hình 32.7 vẽ đúng hình học của phép tính đó.



Hình 32.7. Vì sao sai số hình học trốn được khỏi PSNR; các con số trên hình là *giả định* của ví dụ, không phải số đo. Với những giá trị giả định đó, sai 10 cm ở tầm 3 m chỉ dời điểm ảnh khoảng một điểm ảnh khi góc nhìn kiểm tra chỉ cách góc nhìn dựng bản đồ 20 cm. Độ nhạy tỉ lệ thuận với khoảng dịch camera và tỉ lệ nghịch với bình phương độ sâu. Hệ quả thực hành: chỉ khi tập góc nhìn giữ lại *cách xa* quỹ đạo chụp thì chất lượng ảnh mới bắt đầu nói được điều gì về hình học — và đó đúng là loại tập đánh giá mà các bài trong chương này hiếm khi dựng.

Ba hệ quả rút ra ngay, và cả ba đều dùng được khi bạn đọc bảng của người khác.

- **Độ nhạy tỉ lệ nghịch với bình phương độ sâu.** Vật xa hoàn toàn không bị ràng buộc bởi ảnh. Đây là lý do các bản đồ neural dựng ngoài trời hay có “sương mù” vật chất giả ở nền xa.
- **Độ nhạy tỉ lệ thuận với khoảng dịch camera.** Một tập góc nhìn giữ lại nằm sát quỹ đạo chụp gần như không kiểm tra được hình học. Muốn kiểm tra thật, phải giữ lại những góc nhìn *lệch hẳn* khỏi quỹ đạo.
- **Muốn ràng buộc hình học thì phải đo hình học.** Không có cách nào lách: nếu đầu ra sẽ được dùng để tránh vật cản, thước đo phải là sai số bề mặt so với một quét chân trị, không phải PSNR.

C. Bảy câu hỏi phải hỏi trước khi tin một bảng kết quả

Phần lớn kết quả trong chương này được công bố trong một điều kiện rất hẹp, và điều kiện đó thường nằm rải rác trong phần thí nghiệm chứ không nằm ở bảng. Bảy câu hỏi dưới đây lọc được gần hết.

1. **Ảnh được đo trên góc nhìn nào?** Rất nhiều bảng báo cáo PSNR trên chính các khung hình đầu vào hoặc các khung hình chốt. Con số đó đo *độ khớp*, không đo khả năng tổng quát sang góc nhìn mới. Hai đại lượng này khác nhau rất xa với một mô hình có hàng triệu tham số cho một cảnh.
2. **Tư thế lúc dựng bản đồ là tư thế ước lượng hay chân trị?** Một số thí nghiệm chạy ở chế độ “chỉ dựng bản đồ” với tư thế chân trị. Kết quả đó hợp lệ, nhưng nó nói về bản đồ chứ không nói về hệ SLAM.
3. **Độ sâu đến từ đâu?** Replica cho độ sâu tổng hợp hoàn hảo [333]. Cảm biến thật cho độ sâu có nhiễu, có lỗ, và có giới hạn tầm. Một hệ dựa nặng vào độ sâu sẽ tụt hạng rất nhiều giữa hai điều kiện này.
4. **Chuỗi dài bao nhiêu và có vòng không?** Phần lớn chuỗi trong Replica và TUM RGB-D chỉ dài vài chục giây tới vài phút, đi bộ trong một phòng [71], [334]. Trôi tích lũy và đóng vòng gần như không được kiểm tra.
5. **ATE được căn chỉnh thế nào?** Với hệ đơn ảnh, ATE thường tính sau khi căn chỉnh có cả tỉ lệ. Phép căn chỉnh đó *hấp thụ* sai lệch tỉ lệ, tức là giấu đúng chế độ hỏng nguy hiểm nhất của hệ đơn ảnh. Nếu bạn so với hệ stereo-inertial của mình, hai con số không cùng đơn vị nghĩa.
6. **Phần cứng nào và tốc độ nào?** Con số fps trong bài gần như luôn là GPU để bàn. Và “thời gian thực” trong nhiều bài nghĩa là xử lý kịp tốc độ phát lại đã giảm, không phải kịp tốc độ cảm biến.
7. **Baseline cổ điển có được tinh chỉnh tử tế không?** Đây là câu hỏi của E/24 áp vào đây. Một bảng so bản đồ neural với ORB-SLAM3 chạy cấu hình mặc định trên chuỗi mà cấu hình đó không hợp là một so sánh không có nghĩa.

CẠM BẦY

Cái bẫy hay gặp nhất không phải số liệu sai mà là *ghép hai kết luận đúng thành một kết luận sai*. Bài báo nói “PSNR của chúng tôi cao nhất bảng” — đúng. Bài báo nói “ATE của chúng tôi tốt hơn NICE-SLAM” — cũng đúng. Người đọc ghép lại thành “hệ này tốt hơn hệ tôi đang chạy” — sai, vì cả hai hệ được so đều thuộc dòng neural, và cả hai đều có thể tệ hơn một hệ đặc trưng cổ điển đã tinh chỉnh trên cùng chuỗi. Cách kiểm tra rẻ nhất là tìm trong bài xem có dòng nào so với một hệ cổ điển *trên cùng giao thức căn chỉnh* hay không. Nếu không có, bạn chưa biết gì về vị trí của nó theo trục dọc trong Hình 32.6.

Bảng 32.4. Điều kiện ẩn sau bốn tập dữ liệu hay gặp. Bảng này giải thích vì sao thứ hạng trên hai dòng đầu gần như không dự đoán được hành vi trên dòng cuối — tập dữ liệu của chính bạn.

Tập	Độ sâu	Độ dài, vòng	Điều kiện để bị bỏ qua
Replica	Tổng hợp, hoàn hảo, không lỗ	Ngắn, quỹ đạo dựng sẵn	Không nhiều ảnh, không mờ, phơi sáng cố định; đây là điều kiện dễ nhất có thể
TUM RGB-D	Cảm biến kết cấu ánh sáng, có nhiều và lỗ	Vài chục giây, ít vòng lớn	Trong nhà, đi bộ, một phòng; chuyển động vẫn khá êm
ScanNet	Cảm biến, quét tay	Dài hơn, có quay lại chỗ cũ	Ảnh mờ và phơi sáng tự động xuất hiện thật; chân trị tự thể cũng từ một hệ dựng lại
EuRoC, chuỗi của bạn	Stereo, không có độ sâu trực tiếp	Vài phút, chuyển động mạnh	Bay trong không gian rộng, chuyển động nhanh, ảnh mờ, thay đổi sáng — gần như mọi giả định của chương đều bị vi phạm

D. Ba thí nghiệm bạn phải tự chạy

Đọc bảng của người khác chỉ để loại bớt ứng viên. Muốn biết một hệ có giúp được mình không, có ba thí nghiệm, và chúng đi theo đúng thứ tự kinh tế: rẻ trước, đắt sau.

- **Thí nghiệm oracle tư thế.** Cấp tư thế chân trị cho hệ và chỉ để nó dựng bản đồ. Con số thu được là *trần* chất lượng bản đồ khi khâu tư thế hoàn hảo. Nếu trần đó đã không đủ cho việc hạ nguồn của bạn, mọi công sức về sau đều bị chặn trên. Đây đúng là công cụ ở [E/24](#), áp vào đây.
- **Thí nghiệm tách ghép nối.** Chạy cùng hệ ở hai chế độ: tư thế do bộ dựng ảnh tính, và tư thế do đường cổ điển của bạn tính. Hiệu số ATE giữa hai chế độ trả lời thẳng câu hỏi “bản đồ neural có làm quỹ đạo tốt lên không”. Kinh nghiệm đọc các bảng đã công bố cho thấy câu trả lời thường là không; nhưng đó là kết luận rút từ bảng của người khác, không phải phép đo của tôi.
- **Thí nghiệm ngoài phân phối.** Chạy trên chính chuỗi khó của bạn: chuyển động nhanh, ảnh mờ, phơi sáng tự động, ngoài trời. Đây là chỗ khoảng cách giữa bảng công bố và thực tế lộ ra nhiều nhất, và cũng là chỗ rẻ nhất để phát hiện sớm.

Và một nhắc nhở về kỷ luật đo, vì nó đã cản bạn ở chỗ khác: mọi con số trên đây phải chạy nhiều lần. Các hệ này có thêm hai nguồn tản mát mà hệ cổ điển không có — khởi tạo ngẫu nhiên của bản đồ và số bước tối ưu thực sự chạy được dưới ràng buộc thời gian thực. Sàn nhiều vì thế thường rộng hơn bạn quen.

E. Dựng tập góc nhìn giữ lại cho đúng

Hai mục trên gộp lại thành một yêu cầu rất cụ thể, và nó đáng viết riêng vì hầu như không ai làm đúng.

Cách làm mặc định là giữ lại mỗi khung hình thứ tám của chính chuỗi đầu vào. Cách này rẻ và gần như vô dụng. Lý do đã tính ở mục B: khung hình thứ tám chỉ cách khung hình lân cận vài centimét, mà $\Delta u \approx fb\Delta Z/Z^2$ tỉ lệ thuận với khoảng dịch b . Thu b từ hàng chục centimét xuống vài centimét thì độ nhạy tụt theo đúng tỉ lệ ấy, và với các giá trị giả định của mục B thì sai lệch hình học lộ ra ở mức nhỏ hơn một phần mười điểm ảnh — tức là chìm hẳn dưới nhiễu. Bạn đang đo khả năng nội suy giữa hai ảnh liền kề, không đo bản đồ.

Ba cách dựng tập giữ lại có nghĩa, xếp theo công sức tăng dần:

- **Giữ lại theo đoạn, không theo khung hình.** Cắt bỏ nguyên một đoạn quỹ đạo dài vài giây khỏi dữ liệu dựng bản đồ, rồi đánh giá trên đoạn đó. Khoảng dịch camera khi ấy tính bằng mét chứ không bằng centimét.
- **Quay hai lượt.** Đi qua cùng một chỗ hai lần theo hai đường khác nhau, dựng bản đồ bằng lượt một, đánh giá bằng lượt hai. Đây là cách rẻ nhất cho dữ liệu tự thu, và nó cũng cho bạn luôn một phép thử đóng vòng.
- **Đo thẳng hình học.** Nếu có một quét chân trị — máy quét laser, hoặc một mô hình đã dựng kỹ ngoại tuyến — thì bỏ hẳn ảnh đi và đo sai số bề mặt. Đây là câu trả lời đúng khi đầu ra sẽ được dùng để tránh vật cản.

Nếu bạn chỉ đọc bảng của người khác, câu hỏi rút gọn thành một dòng: tập giữ lại của họ cách quỹ đạo chụp bao xa? Bài nào không trả lời được câu đó thì cột PSNR của nó chỉ nói về nội suy.

32.5 Ngân sách trên Jetson Orin, và câu hỏi có đáng cho robot chưa

Mọi thứ ở trên đều giả định có một GPU để bàn. Mục này hỏi chi phí ấy có *hình dạng* gì trên phần cứng bạn thật sự triển khai. Mục này cố tình **không đưa ra con số nào**: mình không có phép đo trên Orin cho bất kỳ hệ nào trong chương, và một ước lượng tự tính trên giấy rất dễ bị đọc lại thành một số đo. Cái đáng học ở đây không phải độ lớn, mà là *thừa số nào nhân với thừa số nào* — vì chính cấu trúc đó nói cho bạn biết nên cắt ở chỗ nào, và nó đúng bất kể con số cụ thể là bao nhiêu.

A. Bộ nhớ: đếm bằng công thức, không quy ra con số

Một Gaussian lưu 3 số vị trí, 3 số tỉ lệ, 4 số quaternion, 1 số độ đục và 48 hệ số màu bậc ba — gần sáu chục số thực cho *một* hạt. Đã là bội số lớn so với một điểm bản đồ thưa, vốn chỉ cần ba số tọa độ cộng một bộ mô tả ngắn. Và một cảnh cỡ căn phòng dựng cho tử tế cần bậc một triệu hạt như thế.

Nhưng bộ nhớ tham số chưa phải khoản phải trả. Trong SLAM, bản đồ *đang được tối ưu* chứ không nằm yên. Mỗi tham số cần thêm một ô gradient và hai ô trạng thái của bộ tối ưu kiểu Adam, nên bộ nhớ thực tế là bốn lần bộ nhớ tham số:

$$\text{bộ nhớ bản đồ} \propto \underbrace{4}_{\text{tham số, gradient, hai ô Adam}} \times \underbrace{59}_{\text{số thực mỗi hạt}} \times \underbrace{N}_{\text{số Gaussian}}.$$

Thừa số duy nhất bạn điều khiển được là N ; hai thừa số kia là hằng số của biểu diễn. Đó là lý do mọi công trình muốn đưa 3DGS lên máy nhúng đều tấn công vào N chứ không vào chỗ nào khác.

Hướng của kết luận thì rõ, và nó không cần một con số nào: **bản đồ Gaussian tốn bộ nhớ hơn đám mây điểm thưa vài bậc độ lớn cho cùng một căn phòng, và bộ nhớ mới là ràng buộc quyết định của cả dòng này chứ không phải tốc độ dựng ảnh**. Muốn biết bao nhiêu megabyte trên cảnh của bạn thì phải đo trên cảnh của bạn: số hạt mà một cảnh cần phụ thuộc vào chính cảnh đó, nên không có hằng số nào chuyển thẳng được từ bài báo sang.

Dung lượng nghe thì vẫn có vẻ vừa một Orin. Nhưng bộ nhớ trên Orin là bộ nhớ *dùng chung* giữa CPU và GPU, và nó đang phải chứa cả hệ SLAM, cả các mạng nhận thức khác, cả ROS. Quan trọng hơn, dung lượng không phải ràng buộc cản trước.

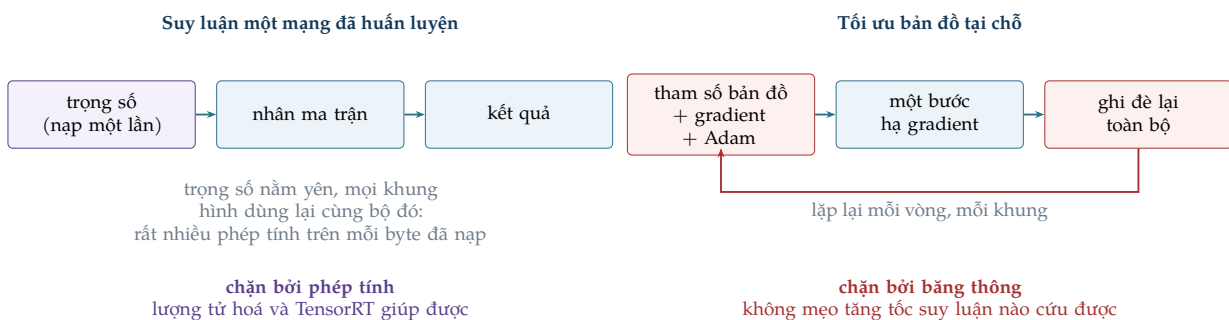
B. Bảng thông mới là ràng buộc cần trước

Mỗi vòng tối ưu phải đọc *toàn bộ* tham số cùng gradient và trạng thái bộ tối ưu, rồi ghi lại chúng. Viết lưu lượng ấy thành công thức là đủ thấy vấn đề:

$$\text{lưu lượng mỗi giây} \propto \underbrace{N}_{\text{số hạt}} \times \underbrace{4}_{\text{tham số+gradient+Adam}} \times \underbrace{2}_{\text{đọc rồi ghi}} \times \underbrace{n_{\text{vòng}}}_{\text{vòng tối ưu mỗi khung}} \times \underbrace{f}_{\text{khung mỗi giây}}.$$

Năm thừa số, và bốn trong năm là hằng số của kiến trúc chứ không phải của cài đặt. Kết luận định tính rút ra: với một bản đồ cỡ căn phòng, riêng việc *di chuyển* tham số qua lại đã ăn phần lớn băng thông bộ nhớ của một máy nhúng, trước khi thực hiện một phép nhân nào. Đây là lý do cơ học — không phải lý do kỹ thuật cài đặt — khiến một hệ 3DGS SLAM đầy đủ chưa chạy được ở tốc độ cảm biến trên phần cứng nhúng. Mình không có phép đo để nói nó hút bao nhiêu lần; công thức chỉ nói nó hút theo hướng nào, và cắt thừa số nào thì đỡ. Ba hướng thoát đều tồn tại và đều phải trả giá: giảm số Gaussian như RTG-SLAM, giảm số vòng tối ưu mỗi khung, hoặc chạy khâu bản đồ ở tần số thấp hơn hẳn khâu bám vết.

Hình 32.8 nói vì sao khoản này khác hẳn mọi khoản học sâu khác bạn từng đưa lên Orin.



Hình 32.8. Vì sao khoản chi phí ở chương này không giống bất kỳ khoản học sâu nào khác trong Phần F. Bên trái là thứ bạn đã quen triển khai: trọng số nạp một lần rồi nằm yên, nên mỗi byte đã nạp được dùng cho rất nhiều phép tính và nút thắt nằm ở phép tính. Bên phải là thứ đang chạy trong một hệ SLAM neural: chính tham số bản đồ là biến đang bị sửa, nên mỗi vòng phải đọc rồi ghi lại toàn bộ, và số phép tính trên mỗi byte rất thấp. Mũi tên đỏ quay ngược là toàn bộ khác biệt — nó biến bài toán từ “chặn bởi phép tính” thành “chặn bởi băng thông”, và đó là lý do lượng tử hoá cùng TensorRT gần như không giúp gì.

Còn một ràng buộc nữa mà bảng thông số không nói: nhiệt và điện. Orin chạy ở nhiều mức công suất, và mức thấp cắt cả xung nhịp GPU lẫn băng thông. Một khối chiếm gần trọn băng thông trong nhiều phút sẽ kéo cả hệ vào vùng giảm xung, nên các khối khác — phát hiện vật thể, lập kế hoạch — chậm theo. Đây là dạng ghép nối không xuất hiện trong bất kỳ phép đo đơn lẻ nào, và chỉ lộ ra khi chạy cả hệ trong thời gian dài.

Với dòng NeRF, chỗ nghẽn dời từ băng thông sang phép tính, và công thức đổi theo. Chi phí một vòng tỉ lệ với (số tia lấy mẫu) $\times$ (số mẫu đọc mỗi tia) $\times$ (chi phí một lần chạy MLP), rồi vẫn phải nhân tiếp với số vòng mỗi khung và số khung mỗi giây. Ba thừa số đầu đều là hằng số kiến trúc, và tích của chúng lớn vì mỗi tia phải dò chứ không *chiếu* — đúng cơ chế đã nói ở mục 3. Hướng kết luận giống hệt dòng Gaussian: một khối duy nhất ăn phần lớn ngân sách của máy, trong khi máy còn phải chạy cả phần nhận thức lẫn phần điều khiển. Lần nữa, đây là hình dạng của bài toán chứ không phải một phép đo; con số thật chỉ có nếu bạn chạy trên chính thiết bị của mình.

Một phản xạ không dùng được ở đây. Khi thấy một mô hình quá chậm trên Orin, phản xạ quen thuộc là lượng tử hoá và biên dịch bằng TensorRT. Ở đây phản xạ đó gần như không giúp gì, và lý do đáng nhớ: cái đang chạy không phải *suy luận* mà là một *vòng lặp tối ưu* trên chính tham số bản đồ. Lượng tử hoá làm hỏng gradient; đồ thị tính toán thay đổi liên tục vì Gaussian được thêm và bị cắt; và phần lớn thời gian nằm ở lưu lượng bộ nhớ chứ không ở phép nhân. Đây là chỗ

khác biệt căn bản giữa chương này và mọi chương khác của Phần F, nơi ta triển khai một mạng đã huấn luyện xong.

C. Vì sao gần như mọi kết quả đều là trong nhà và chuỗi ngắn

Ba ràng buộc trên giải thích luôn một điều mà người đọc thường coi là ngẫu nhiên: vì sao gần như toàn bộ kết quả trong chương này đến từ cảnh trong nhà, chuỗi vài chục giây tới vài phút, có độ sâu RGB-D, chạy trên GPU để bàn.

- **Trong nhà** vì cảnh có biên, nên lưới hoặc tập Gaussian cấp phát được. Ngoài trời không có biên, và bộ nhớ tăng theo diện tích đã đi qua.
- **Chuỗi ngắn** vì bản đồ chỉ lớn thêm chứ không bao giờ nhỏ đi, và vì chuỗi ngắn thì trôi tích lũy chưa kịp thành vấn đề.
- **RGB-D** vì độ sâu cho hình học gần như miễn phí. Bỏ độ sâu đi thì phải mượn tiên nghiệm học được, và kéo theo mọi giới hạn của tiên nghiệm đó.
- **GPU để bàn** vì ở đó cả dung lượng lẫn băng thông bộ nhớ đều dư ra rất nhiều so với một máy nhúng, và lại không phải chia cho phần còn lại của robot.
- **Camera đi chậm và đều** vì bám vết quang trắc có bồn hút hữu hạn, và vì ảnh mờ phá thẳng vào hàm mất mát.

Không điều nào ở trên là lỗi của các tác giả. Đó là những điều kiện cần để bài toán giải được ở trạng thái hiện tại của lĩnh vực. Nhưng chúng *là* khoảng cách giữa bảng kết quả và chiếc robot của bạn, và đó là khoảng cách bạn phải tự trả.

D. Điền số vào bốn cổng

Khung bốn cổng ở F/27 có ích nhất khi nó được điền số thật thay vì cảm giác. Bảng 32.5 điền cho chương này, với ba nhu cầu hạ nguồn khác nhau. Chú ý là cùng một công nghệ cho ba kết luận khác nhau, và khác biệt nằm ở nhu cầu chứ không ở công nghệ.

Bảng 32.5. Bốn cổng của F/27 điền cho bản đồ neural, tách theo nhu cầu hạ nguồn. Cùng một công nghệ, ba kết luận khác nhau — và đó là lý do câu hỏi “bản đồ neural có đáng không” không có câu trả lời chung.

Cổng	Để định vị	Để tránh vật cản	Để dựng lại và xem
Bài toán có thật không	Không: hệ hiện tại đã định vị tốt	Có, nhưng TSDF và lưới chiếm dụng đã giải	Có, và không có lời giải cổ điển nào thay được
Đo được không	Có: ATE, cùng giao thức căn chỉnh	Có, nhưng cần chân trị hình học, thứ khó có ngoài đời	Có: PSNR, SSIM, LPIPS trên góc nhìn giữ lại thật sự
Vừa ngân sách Orin không	Không: riêng lưu lượng đọc-ghi tham số mỗi vòng đã ăn phần lớn băng thông	Không ở tốc độ cảm biến; có nếu chạy khâu bản đồ ở tần số thấp	Có, nếu chạy ngoại tuyến trên máy trạm
Hồng ra sao	Im lặng, qua vòng phản hồi dương, không có chỉ báo	Vật chất giả lơ lửng bị coi là vật cản, hoặc ngược lại	Ảnh xấu đi, thấy ngay bằng mắt — chế độ hồng lạnh nhất
Kết luận	Chưa đáng	Dùng TSDF trước	Đáng, và nên làm ngoại tuyến

Hàng cuối là chỗ đáng nhớ. Cùng một đồng mã nguồn, cùng một bài báo, mà kết luận đảo hẳn theo việc bạn định dùng đầu ra vào việc gì. Ai hỏi “bản đồ neural có đáng không” mà không nói rõ dùng để làm gì thì đang hỏi một câu không có đáp án.

E. Kết luận thẳng thắn: có đáng chưa

Trả lời tách theo mục đích, vì gộp chung là chỗ mọi tranh luận đi vào ngõ cụt.

- **Thay bản đồ định vị bằng bản đồ neural: chưa đáng.** Không có bằng chứng nào cho thấy nó cải thiện ATE của một hệ stereo-inertial đã tinh chỉnh kỹ, trên chuỗi thật, ở ngân sách của Orin. Đổi lại bạn mất tính chắc chắn đoán được, mất tính lặp lại, và tiêu gần hết ngân sách tính toán. Đây là kết luận đọc từ điều kiện thí nghiệm của các bài đã công bố, không phải từ một phép đo của tôi — nhưng nó là kết luận mà chính các điều kiện đó bắt buộc.
- **Bản đồ neural như một sản phẩm phụ, chạy ở tần số thấp: đáng thử, nếu có nhu cầu thật.** Nếu bài toán hạ nguồn cần ảnh — xem lại từ xa, mô phỏng cảm biến, sinh dữ liệu huấn luyện, kiểm tra công trình — thì mẫu ghép lỏng của Photo-SLAM là cách vào rẻ và không rủi ro. Tư thế vẫn do đường cổ điển bảo đảm, nên xấu nhất là bạn mất một luồng GPU.
- **Bản đồ dày để tránh vật cản: chọn TSDF hoặc lưới chiếm dụng trước.** Chúng rẻ hơn nhiều bậc, có mô hình xác suất, có bề mặt định nghĩa sạch, và đã được kiểm chứng lâu năm. Chỉ chuyển sang biểu diễn neural khi bạn chỉ ra được một việc cụ thể mà chúng không làm nổi.
- **Dựng lại chất lượng cao ngoại tuyến: đáng, và đây là chỗ dòng này thật sự thắng.** Bỏ ràng buộc thời gian thực đi thì mọi phép tính ở trên đối hân. Ghi bag trên robot, chạy dựng bản đồ trên máy trạm, đó là cách dùng đúng sức của công nghệ này hôm nay.

F. Bốn bài học thiết kế đáng giữ kể cả khi không dùng

Đây là phần trả lời câu hỏi thứ tư của khung đọc: nếu không lấy nguyên phương pháp thì lấy được gì.

- **Chọn biểu diễn theo tiêu chí “sửa được sau khi đóng vòng”.** Đây là tiêu chí phân loại mạnh nhất trong cả chương, và nó tách dòng Gaussian khỏi dòng MLP đơn rõ hơn mọi bảng PSNR. Nó cũng đúng cho mọi thứ khác bạn gắn vào bản đồ: bản đồ ngữ nghĩa, bản đồ chiếm dụng, bộ nhớ đóng vòng.
- **Mọi hàm mất mát quang trắc cần một lịch làm trơn.** Bài học của BARF, và nó là bản sao của kim tự tháp ảnh trong các hệ trực tiếp cổ điển. Nếu bạn thêm một số hạng quang trắc vào hệ mình mà không có lịch làm trơn, nó sẽ mắc kẹt và bạn sẽ đổ lỗi nhầm cho khởi tạo.
- **Chốt khung hình theo lượng thông tin mới, đo được.** iMAP đo bằng sai số dựng ảnh dưới bản đồ hiện tại. Ý tưởng đó chuyển được sang hệ cổ điển: thay ngưỡng “số điểm khớp dưới 90%” bằng một đại lượng có nghĩa thống kê. Bạn đã biết chính sách chốt khung hình là chỗ nhạy cảm bậc nhất của hệ mình.
- **Trọng số của phần dư phải đến từ bất định, không đến từ hằng số.** NeRF-SLAM cân hàm mất mát dựng bản đồ bằng bất định của độ sâu. Đây đúng là tư duy ma trận thông tin, và nó là sợi dây nối chương này với F/34.

CẦN NHỚ

Hai câu hỏi phải luôn tách rời: *bản đồ này trông có đẹp không* và *quỹ đạo này có đúng không*. Chúng được tối ưu bởi hai hàm mất mát khác nhau, đo bằng hai bộ thước đo khác nhau, và một hệ hoàn toàn có thể thắng ở câu thứ nhất trong khi thua ở câu thứ hai. Quy tắc kiến trúc rút ra: đừng để bộ dựng ảnh là nguồn của tư thế, trừ khi bạn đã tự đo và thấy nó tốt hơn đường cổ điển của mình trên chính chuỗi khó của mình. [T3]

I 32.6 Giới hạn và trường hợp hổng

Giới hạn nặng nhất đã nói ở trên và đáng nhắc lại theo cách khác: các hệ này tối ưu *riêng cho từng cảnh*. Chi phí vì thế là chi phí lặp lại, không phải chi phí một lần như huấn luyện một mạng rồi triển khai. Mọi kỹ thuật tăng tốc suy luận mà bạn quen đều không áp được. Các dòng mô hình tiến thẳng, dự đoán thẳng tập Gaussian từ vài ảnh, đang xoá dần ranh giới này; tại thời điểm viết, chất lượng của chúng vẫn dưới bản tối ưu riêng từng cảnh, và đây là quan sát xu hướng chứ chưa phải kết quả đã ổn định.

Chế độ hỏng thứ hai là hỏng *im lặng* và nó đáng sợ nhất. Khi tư thế sai vài phần mười độ, tối ưu không báo lỗi. Nó bù bằng cách sinh ra vật chất giả lơ lửng, và ảnh dựng ra vẫn đẹp. Với một hệ ghép chặt, vật chất giả đó lại quay lại làm sai tư thế của khung hình sau. Bạn có một vòng phản hồi dương mà không có chỉ báo nào bật lên. Ở hệ cổ điển, cùng tình huống ấy làm số điểm khớp tụt và làm phần dư tăng — tức là bạn *nhìn thấy* nó. Mất tính chẩn đoán được là khoản mất ít được nhắc nhất và đắt nhất khi triển khai.

Chế độ hỏng thứ ba nằm ở giả định về quá trình tạo ảnh. Cảnh phải tĩnh, phơi sáng phải nhất quán, ảnh không được mờ. Ba giả định này bị vi phạm liên tục trên một robot thật, và chúng bị vi phạm *cùng lúc* đúng vào những đoạn khó: robot quay nhanh trong hành lang thiếu sáng, camera tự tăng thời gian phơi, ảnh mờ đi, người đi qua khung hình. Mỗi vi phạm đều được mô hình giải thích bằng cách thêm hình học sai, vì đó là biến duy nhất nó có.

Cuối cùng, một giới hạn về kiến thức chứ không về công nghệ: tôi không có phép đo riêng nào cho các hệ trong chương này trên phần cứng Orin, và cũng không kiểm chứng được các con số hiệu năng mà các bài báo đưa ra. Vì thế mục 32.5 cổ tình chỉ nói về *cấu trúc* của chi phí — thừa số nào nhân với thừa số nào — chứ không đưa ra con số nào. Cấu trúc đó đủ để nói “khoản này bị chặn bởi băng thông chứ không bởi phép tính” và “thừa số duy nhất cắt được là số hạt”. Nó không đủ để nói hệ nào nhanh hơn hệ nào, cũng không đủ để nói hệt bao nhiêu lần. Nếu bạn cần một con số, chỉ có một cách lấy nó, là chạy trên chính thiết bị của mình.

32.7 Thực hành

Repo và tài nguyên. Mã nguồn của các hệ trong chương đều mở: [cvg/nice-slam](#), [HengyiWang/Co-SLAM](#), [idiap/ESLAM](#), [eriksandstroem/Point-SLAM](#), [youmi-zym/GO-SLAM](#) cho dòng NeRF; [spla-tam/SplaTAM](#), [muskie82/MonoGS](#), [Lab-of-AI-and-Robotics/GS_ICP_SLAM](#), [HuajianUP/Photo-SLAM](#) cho dòng Gaussian. Bản 3DGS gốc là [graphdeco-inria/gaussian-splatting](#). Để đo, dùng lại chính *evo* mà bạn đã quen cho ATE, và một script nhỏ tính PSNR cùng LPIPS trên tập góc nhìn giữ lại. Tập dữ liệu vào cửa nhanh nhất là TUM RGB-D vì nó nhỏ và mọi hệ đều báo cáo trên đó. Danh sách tên hệ ở đây chốt tại tháng 9/2026 và sẽ cũ trong sáu tới mười hai tháng; các hạ tầng bền hơn nhiều là bộ rasterize gốc, *evo*, và *opencv*.

Câu hỏi kiểm tra hiểu. Vì sao lưới đặc trưng chứa được quên thăm khốc trong khi tăng số tham số của một MLP đơn thì không? Một bài báo cáo PSNR đứng đầu bảng trên Replica và ATE nhỏ hơn mọi hệ neural khác được so: hai kết quả đó nói gì và không nói gì về việc hệ chạy được trên chuỗi EuRoC của bạn? Nếu chỉ được chạy một thí nghiệm để quyết định có theo đuổi hướng này không, bạn chạy thí nghiệm nào, và nó chặn trên cái gì?

Mini project. Xem khối *Mini project SLAM* bên dưới; nó chính là thí nghiệm rẻ nhất trả lời được câu hỏi trung tâm của chương.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Bản đồ điểm thưa	Vài chục tới vài trăm nghìn điểm 3D kèm bộ mô tả. Đủ để trả lời “tôi ở đâu”, không trả lời được “chỗ kia có trống không” vì nó chưa bao giờ mô hình hoá khoảng không giữa các điểm.
••• TSDF	Lưới lưu khoảng cách có dấu tới bề mặt gần nhất. Bề mặt là tập mức 0 nên có định nghĩa sạch, và đây là đối thủ thật sự của bản đồ neural chứ không phải bản đồ thưa.
•• iMAP	Hệ SLAM RGB-D dùng một MLP làm toàn bộ bản đồ. Gọn tới mức một phòng chỉ cần vài trăm nghìn tham số, nhưng trọng số dùng chung nên học chỗ mới làm mờ chỗ cũ.
••• Quên thảm hoạ	Học dữ liệu mới làm mất khả năng tái tạo dữ liệu cũ, vì cùng bộ tham số gánh cả hai. Triệu chứng trong bản đồ neural: sang phòng bên rồi quay lại thì phòng cũ mờ đi.
••• Vì sao lưới đặc trưng chưa được quên thảm khốc?	Vì nó thu hẹp giá đỡ của gradient. Một tia chỉ chạm vài ô lưới quanh nó thay vì chạm mọi trọng số, nên cập nhật trở thành cục bộ. Cái giá là bộ nhớ quay lại phụ thuộc thể tích cảnh.
•• Ba cách cấp phát thưa	Co-SLAM ghép mã hoá toạ độ với bảng băm; ESLAM dùng ba mặt phẳng đặc trưng nên bộ nhớ theo bình phương chứ không theo lũy thừa ba; Point-SLAM neo đặc trưng lên chính điểm đã quan sát.
••• BARF	Tối ưu tư thế cùng lúc với trường bức xạ. Phát hiện chính: mã hoá vị trí tần số cao làm mặt phẳng mất mát lờm chờm, nên phải bật tần số từ thô tới tinh.
••• Vì sao tần số cao phá gradient theo tư thế?	Vì đạo hàm của thành phần tần số k có biên độ tỉ lệ với k . Các gọn tần số cao sinh ra vô số cực tiểu địa phương gần nhau, và hạ gradient rơi vào cái hố gần nhất.
•• GO-SLAM	Hệ đầu tiên trong dòng NeRF có đóng vòng và tối ưu toàn cục thật sự. Nó làm được vì bản đồ neo vào khung hình chốt, nên sửa tư thế thì kéo bản đồ theo.
• vMAP	Mỗi vật thể một mạng nhỏ riêng, hợp lại lúc dựng ảnh. Vật quan sát được một phần vẫn ra hình dạng kín, vì mỗi mạng chỉ phải học một vật.
••• Vì sao 3DGS nhanh hơn NeRF về bản chất?	NeRF phải đi tìm vật chất bằng cách lấy hàng chục mẫu dọc mỗi tia. 3DGS đã lưu vật chất thành nguyên thể tường minh nên chỉ chiếu, sắp theo độ sâu rồi trộn. Chi phí mỗi điểm ảnh đổi từ số mẫu sang số nguyên thể phủ lên nó.
••• Ba hệ quả của việc chuyển sang nguyên thể tường minh	Cập nhật bản đồ thành sửa chữa cục bộ; bản đồ dịch chuyển được sau khi đóng vòng; đạo hàm theo tư thế có dạng giải tích.
••• Ghép chặt và ghép lỏng	Ghép chặt lấy tư thế từ chính bộ dựng ảnh, nên lỗi bản đồ chảy vào quỹ đạo. Ghép lỏng để tư thế cho đường cổ điển và coi bản đồ là sản phẩm; nó không thể làm quỹ đạo xấu đi.
•• Photo-SLAM	Giữ xương sống đặc trưng và hiệu chỉnh chùm tia cho tư thế, gắn thêm bản đồ Gaussian chỉ để dựng ảnh. Đây là mẫu ghép lỏng điển hình và là cách vào rẻ nhất cho một hệ đang chạy sản phẩm.
••• PSNR cao nhưng ATE tệ — giải thích thế nào?	Hàm mất mát chỉ ràng buộc ảnh dựng, không ràng buộc hình học, và màu phụ thuộc góc nhìn cho mô hình chỗ để “vẽ” thay vì “dựng”. Hai đại lượng đó tách rời được hoàn toàn.
••• PSNR đo trên khung hình đầu vào	Dấu hiệu cảnh báo quan trọng nhất khi đọc bảng: con số đó đo độ khớp chứ không đo khả năng tổng quát sang góc nhìn mới. Phải tìm xem bài có tập góc nhìn giữ lại không.
••• ATE có căn chỉnh tỉ lệ	Với hệ đơn ảnh, ATE thường tính sau khi căn chỉnh cả tỉ lệ. Phép căn chỉnh đó hấp thụ sai lệch tỉ lệ, tức là giấu đúng chế độ hỏng nguy hiểm nhất, nên không so trực tiếp được với hệ stereo-inertial.
•• Thí nghiệm oracle tư thế	Cấp tư thế chân trị và chỉ để hệ dựng bản đồ, để lấy trần chất lượng bản đồ khi khâu tư thế hoàn hảo. Trần đó không đủ thì mọi công sức về sau bị chặn trên.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Ngân sách của một bản đồ Gaussian	Gần sáu chục số thực cho mỗi hạt, nhân bốn khi đang tối ưu vì còn gradient và hai ô Adam. Lưu lượng mỗi giây tỉ lệ với số hạt nhân số vòng tối ưu mỗi khung nhân số khung mỗi giây — nên bằng thông cần trước dung lượng, và thừa số duy nhất bạn điều khiển được là số hạt.
●● Vì sao TensorRT không cứu được?	Vì cái chạy trên Orin không phải suy luận mà là vòng lặp tối ưu trên chính tham số bản đồ. Lượng tử hoá phá gradient, đồ thị đổi liên tục, và nút thất nằm ở băng thông chứ không ở phép nhân.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
gaussian-splatting	Bộ rasterize Gaussian khác biệt được — hạ tầng mà toàn bộ dòng SLAM Gaussian trong chương dựng lên trên đó	Hạ tầng; bền hơn tên từng hệ SLAM
Photo-SLAM [330]	Mẫu ghép lỏng: tư thế do đặc trưng và hiệu chỉnh chùm tia cổ điển, bản đồ Gaussian chỉ tiêu thụ tư thế	Kiến trúc đáng bất chước nhất cho hệ đang chạy sản phẩm
GO-SLAM [323]	Neo bản đồ neural vào khung hình chốt để biến dạng được sau khi đóng vòng	Bằng chứng cho tiêu chí “sửa được sau khi đóng vòng”
NeRF-SLAM [325]	Dùng bất định của độ sâu làm trọng số cho hàm mất mát dựng bản đồ	Tư duy ma trận thông tin mang sang phía bản đồ
Công cụ số hoá công trình và quét nội thất	Dựng lại chất lượng cao <i>ngoại tuyến</i> từ video quay tay — đúng chỗ dòng này thắng rõ	Được cho là đã dùng trong sản phẩm thương mại; tôi không kiểm chứng chi tiết

MINI PROJECT SLAM · Đặt một hệ bản đồ neural lên đúng hai trục đánh giá · 1–2 ngày

Mục tiêu	trả lời bằng số cho câu hỏi trung tâm của chương: một hệ SLAM Gaussian đứng ở góc phần tư nào của Hình 32.6 khi đo bằng <i>giao thức của bạn</i> , chứ không phải giao thức của bài báo.
Dữ liệu	một chuỗi TUM RGB-D để hệ chạy được, cộng đúng một chuỗi EuRoC khó mà bạn đã có sẵn nhiều từ dự án chương 24 — ví dụ V2_03. Baseline là hệ ORB-SLAM3 của chính bạn ở cấu hình mặc định đã tinh chỉnh.
Các bước	cài một hệ ghép chặt và chạy nó ba lần trên mỗi chuỗi; ghi ATE bằng <i>evo</i> với <i>cùng một</i> phép căn chỉnh cho cả hai hệ, và ghi rõ có căn chỉnh tỉ lệ hay không; ghi PSNR trên các khung hình giữ lại, không phải trên khung hình đầu vào; chạy thêm chế độ oracle với tư thế chân trị để lấy trần chất lượng bản đồ; đo đỉnh bộ nhớ GPU và thời gian mỗi khung hình, trên máy để bàn trước, rồi trên Orin nếu nó chạy nổi.
Xong khi	bạn vẽ được một điểm cho mỗi hệ trên mặt phẳng (chất lượng ảnh, ATE) kèm thanh sai số ba lần chạy, và phát biểu được một trong hai câu: “hệ neural nằm cùng góc với ORB-SLAM3 theo trục dọc” hoặc “nó tệ hơn X cm, tức là ngoài sàn nhiều đã đo”. Kiểm tra chéo bắt buộc: nếu bỏ căn chỉnh tỉ lệ mà ATE của hệ đơn ảnh nhảy vọt, bạn vừa nhìn thấy chỗ mà bảng công bố đã giấu.
Nối với	E/24 cho sàn nhiều và cho thí nghiệm oracle; F/27 cho khung bốn cổng để kết luận đi tiếp hay dừng; F/34 nếu chuỗi khó của bạn hỏng vì ảnh mờ chứ không vì bản đồ.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Bản đồ dày cho bạn ba thứ mà bản đồ thưa không có: biết chỗ nào trống để tránh vật cản, biết hình dạng bề mặt để thao tác hoặc kiểm tra, và dựng được ảnh ở góc nhìn mới. Cả ba đều là câu hỏi về *khoảng không giữa các điểm*, thứ mà bản đồ thưa chưa bao giờ mô hình hoá.
- A2. Không có lý do mặc định nào cả — và đó chính là cổng đầu tiên. Bản đồ neural chỉ hơn TSDF ở ba chỗ: nội suy vùng chưa quan sát, dựng được ảnh có màu phụ thuộc góc nhìn, và dồn tham số theo nội dung. Nếu nhu cầu của bạn không nằm trong ba chỗ đó thì TSDF hoặc lưới chiếm dụng vẫn là câu trả lời đúng.
- A3. Vì hàm mất mát chỉ ràng buộc ảnh dựng ra. Hai cấu hình hình học rất khác nhau có thể sinh ra cùng một bộ ảnh, nhất là khi màu được phép phụ thuộc hướng nhìn. Muốn biết quỹ đạo, phải đo ATE bằng đúngng giao thức căn chỉnh mà bạn dùng cho hệ hiện tại.
- A4. Thay đổi duy nhất là bỏ việc đi dò dọc tia, thay bằng chiếu các nguyên thể tường minh rồi trộn. Nó mở ra đóng vòng vì nguyên thể có vị trí tường minh: gắn chúng vào khung hình chốt thì sửa tư thế sẽ kéo được cả cụm theo, còn với một MLP đơn thì không có phép nào tương đương.
- A5. Bảng thông cần trước dung lượng, và lý do nằm ở công thức chứ không ở một con số: mỗi vòng tối ưu đọc rồi ghi lại *toàn bộ* tham số cùng gradient và hai ô trạng thái Adam, và lưu lượng đó còn phải nhân với số vòng mỗi khung rồi nhân với số khung mỗi giây. Với một bản đồ cỡ căn phòng, riêng việc di chuyển tham số đã ăn phần lớn băng thông của một máy nhúng trước khi tính một phép nhân nào. Muốn biết hệt bao nhiêu lần thì phải đo trên chính thiết bị của mình.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy nghĩ về hai cách ghi lại một căn phòng. Cách thứ nhất là đóng vài chục cái đinh lên tường ở những chỗ dễ nhận ra, rồi ghi lại khoảng cách giữa chúng. Cách thứ hai là thuê một họa sĩ vẽ căn phòng đó thật giống, giống tới mức nhìn bức tranh cứ tưởng đang nhìn phòng thật. Cái đinh giúp bạn biết mình đang đứng ở đâu, chính xác tới từng centimét. Nhưng nhìn bản đồ đinh thì không biết cái ghế có chắn lối đi hay không. Bức tranh thì ngược lại: nhìn rất rõ căn phòng, mà đo tường từ bức tranh lại sai, vì họa sĩ chỉ cần vẽ cho *trông* giống chứ không cần vẽ cho *đo* đúng. Anh ta được phép vẽ một cái bóng thay cho một cái hốc tường, và người xem không phát hiện ra. Toàn bộ chương này là chuyện có nên đổi đinh lấy tranh hay không. Câu trả lời hôm nay là: đừng đổi, hãy giữ đinh để định vị và treo thêm bức tranh nếu bạn thật sự cần nhìn. Vì bức tranh rất đắt — vẽ một căn phòng ngốn gần hết sức của cái máy tính nhỏ gắn trên robot — và vì độ giống của bức tranh không hề chứng minh nó đo đúng.

Chỗ không rút gọn được: chỗ bức tranh *có thể* đo đúng. Khi được vẽ từ rất nhiều góc nhìn phủ kín, họa sĩ hết chỗ để gian lận, và bức tranh dần trở thành phép đo thật. Không có ranh giới gọn nào nói bao nhiêu góc nhìn là đủ, và mọi cách nói đơn giản về ranh giới đó đều bỏ sót một phần ví dụ.

Một câu: *Đinh để biết mình ở đâu, tranh để biết chỗ đó trông thế nào — và tranh đẹp không có nghĩa là tường được đo đúng.*

32.8 Kết nối

[C/16/03-nerf-3dgs-va-hop-nhat](#) là tiền đề trực tiếp: cơ chế dựng ảnh thể tích, phép rasterize Gaussian, và lập luận vì sao PSNR không chứng minh hình học đều nằm ở đó, còn chương này chỉ hỏi chuyện dùng chúng làm bản đồ. [F/27](#) là bộ lọc: khung bốn cổng ở đó chính là thứ dẫn tới kết luận “chưa đáng” của mục [32.5](#), và bạn nên chạy lại khung đó với số liệu của riêng mình. [F/31](#) giải thích các bộ bám vết dày mà NeRF-SLAM và GO-SLAM mượn lại, nên đọc nó trước nếu bạn muốn hiểu vì sao hai hệ đó chính xác hơn phần còn lại của dòng. [F/30](#) là nguồn của mọi tiên nghiệm độ sâu và pháp tuyến mà NICER-SLAM dùng để thay cảm biến, kèm theo vấn đề tỉ lệ đi cùng. [F/33](#) tiếp nhận ý tưởng bản đồ mức đối tượng của vMAP và đẩy nó lên đồ thị cảnh. [F/34](#) nhận lại hai chỗ chương này bỏ ngỏ: ảnh mờ do chuyển động, và việc lấy trọng số phần dư

từ bất định thay vì từ hằng số. Còn E/24 là bộ công cụ để chạy ba thí nghiệm ở mục 32.4 một cách có kỷ luật — đặc biệt là thí nghiệm oracle, thứ trả lời được nhiều nhất với chi phí ít nhất.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao khoản chênh tốc độ giữa NeRF và 3DGS lại kéo theo khả năng sửa bản đồ sau khi đóng vòng, trong khi hai chuyện đó nghe không liên quan gì tới nhau?
2. Một hệ báo PSNR cao nhất bảng và ATE tốt hơn mọi hệ neural khác. Vì sao điều đó vẫn chưa nói gì về việc nó có thay được hệ của bạn không, và bạn tìm dòng nào trong bài để biết?
3. Giả sử bạn buộc phải dùng bản đồ neural cho tránh vật cản. Đại lượng nào của TSDF hoặc lưới chiếm dụng bị mất, và bạn dựng lại nó bằng cách nào?
4. Bài học từ thô đến tinh của BARF tương đương với kỹ thuật cổ điển nào, và điều gì xảy ra nếu bạn thêm một số hạng quang trắc vào hệ mình mà quên nó?
5. Vì sao lượng tử hoá và TensorRT gần như vô dụng ở chương này, trong khi chúng là công cụ chính ở mọi chương khác của Phần F?
6. Một hệ ghép chặt bị lệch tư thế nhẹ ở khung hình thứ 200. Chuỗi sự kiện nào dẫn tới hỏng, và vì sao không có chỉ báo nào bật lên như ở hệ cổ điển?
7. Trong bốn bài học thiết kế ở cuối chương, bài nào áp được vào hệ của bạn ngay tuần này mà không cần thêm một dòng học sâu nào?

Ngữ nghĩa, vật thể động và đồ thị cảnh (Semantics, Dynamics and Scene Graphs)

PHỤ THUỘC C/14-bai-toan-cv-loi (detection và segmentation là gì — chương này chỉ nói về việc *dùng* chúng); F/27-dat-hoc-sau-vao-dau (khung bốn cổng); F/30-do-sau-dong-quang-va-mo-hinh-nen (dòng quang và độ sâu, hai nguồn bằng chứng hình học cho “đang động”); B/08-dich-chuyen-phan-phoi (mặt nạ hồng đúng lúc bạn cần nó nhất).

TÓM TẮT

SLAM cổ điển giả định thế giới đứng yên. Giả định đó không nằm trong dòng code nào, nhưng mọi hàm giá đều dựa vào nó. Chương này chỉ ra chỗ giả định gây, rồi hỏi ngữ nghĩa mua lại được bao nhiêu và với giá nào. Đọc xong bạn tách được ba việc mà tài liệu hay trộn làm một: vật *có thể* động, vật *đang* động, và vật *đã* động rồi lại đứng yên. Bạn cũng có một bảng xếp hạng định tính về sức nặng của các bộ nhận thức, kèm phép đo mà bạn phải tự chạy trên máy mình, để biết mô hình nào có cửa trong vòng lặp và mô hình nào chỉ dùng được ngoại tuyến. Nếu chỉ nhớ một điều: mặt nạ ngữ nghĩa là một điều chỉnh *trọng số*, không phải một lệnh *xoá* — xoá quá tay làm hỏng điều kiện của bài toán tối ưu, và hỏng điều kiện là kiểu hỏng im lặng, không có cờ ngoại lai nào báo.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mặt nạ và phân đoạn thực thể** — mặt nạ là một ảnh nhị phân cùng kích thước ảnh gốc, đánh dấu từng điểm ảnh thuộc hay không thuộc một vật; ở đây nó nói “đừng tin các phép đo rơi vào vùng này”. Phân đoạn thực thể là việc tách *từng* vật riêng lẻ thành một mặt nạ riêng kèm nhãn lớp, khác với phân đoạn ngữ nghĩa vốn gộp hai người đứng cạnh nhau làm một.
- **vật thể động** — trong chương này luôn phải nói rõ nghĩa nào trong ba nghĩa: thuộc lớp *có khả năng* tự di chuyển, *đang* thật sự di chuyển ở khung hình này, hay *đã* đổi chỗ giữa hai lần robot đi qua.
- **điểm bản đồ và phần dư tái chiếu** — điểm bản đồ là một điểm 3D mà hệ thống tin là cố định. Phần dư tái chiếu là khoảng cách pixel giữa chỗ ta chiếu nó xuống ảnh và chỗ ta thật sự quan sát nó. Nếu điểm không hề cố định, phần dư đó *không* còn chỉ nói về sai số tư thế camera.
- **ma trận thông tin** — nghịch đảo của hiệp phương sai, tức là trọng số của một ràng buộc trong bài toán tối ưu. Đặt trọng số bằng không chính là *xoá* ràng buộc; hạ trọng số là giữ nó lại nhưng bớt tin.
- **điều kiện của bài toán** — mức độ “đủ mọi hướng” của tập ràng buộc. Nếu mọi điểm còn lại nằm dồn về một phía ảnh, tư thế camera vẫn giải được nhưng có một hướng gần như không bị ràng buộc. Sai số theo hướng đó lớn mà *không ai báo*.
- **tập nhãn đóng và mở** — tập đóng nghĩa là mô hình chỉ biết đúng những lớp có trong tập huấn luyện, thường là 80 lớp của COCO. Tập mở nghĩa là mô hình nhận một chuỗi chữ bất kỳ làm tên lớp lúc chạy, qua một *prompt* — một gợi ý dạng điểm bầm, khung, hoặc câu chữ.
- **bản đồ đặc** — cách biểu diễn bề mặt bằng lưới ô lập phương hoặc lưới tam giác, thay vì bằng một đám mây điểm rời rạc. Nó tồn bộ nhớ hơn *nhiều bậc* so với bản đồ điểm thưa mà ORB-SLAM3 dựng.
- **độ trễ đuôi** — phân vị cao của phân bố độ trễ, ví dụ p99. Với vòng lặp thời gian thực, chính *đuôi* quyết định có rớt khung hay không, chứ không phải trung bình.

CÂU HỎI MỞ ĐẦU

- Q1. Một người ngồi yên trên ghế chiếm một mảng lớn liên khối của khung hình. Mặt nạ “person” của bạn xoá đúng vùng đó, và quỹ đạo tệ đi. Vì sao xoá đúng vật lại làm hỏng kết quả?
- Q2. RANSAC đã là một bộ loại ngoại lai rất tốt và bạn đã dùng nó nhiều năm. Vậy chính xác thì mặt nạ ngữ nghĩa bắt được loại ngoại lai nào mà RANSAC không bắt được?
- Q3. Vì sao mạng phân đoạn của bạn hỏng nặng nhất đúng ở những khung hình mà bạn cần nó nhất, và điều đó có phải trùng hợp không?
- Q4. Ba hệ thống cùng báo “xử lý được cảnh động”: một cái nhìn nhãn lớp, một cái nhìn phần dư hình học, một cái nhìn lịch sử bản đồ. Chúng giải ba bài toán khác nhau ở chỗ nào?
- Q5. Nếu đầu ra của hệ thống bạn chỉ là một quỹ đạo và một con số APE, thì tầng ngôn ngữ mở kiểu “tìm cho tôi cái cốc” giúp được gì — và câu trả lời trung thực nói lên điều gì về việc nên đầu tư vào đâu?

33.1 Đặt vấn đề

ORB-SLAM3 không có khái niệm “vật thể”. Mô hình thế giới của nó là một đám mây điểm 3D, mỗi điểm được giả định nằm cố định trong một hệ quy chiếu cố định [40]. Giả định đó không được viết ra ở đâu cả. Nhưng mọi hàm giá trong hệ đều dựa vào nó.

Hãy nhìn kỹ một phần dư tái chiếu. Nó là hiệu giữa chỗ ta chiếu một điểm bản đồ xuống ảnh và chỗ ta thật sự đo được điểm đó. Ta đọc phần dư ấy như sai số tư thế camera. Cách đọc đó chỉ đúng khi điểm không tự di chuyển. Nếu điểm có di chuyển, phần dư trộn hai ẩn số vào một con số. Bộ tối ưu không biết điều đó, nên nó gỡ rối theo cách duy nhất nó biết: dịch camera. Một vật động vì thế không tạo ra ngoại lai ngẫu nhiên. Nó tạo ra một sai lệch *có hướng*, và sai lệch có hướng là thứ nguy hiểm nhất với một bộ ước lượng bình phương tối thiểu.

A. Vì sao OpenLORIS khó hơn EuRoC

Bạn đã có bằng chứng cho điều này trong chính bảng kết quả của mình. EuRoC quay trong một sảnh máy bay và một phòng thí nghiệm, gần như không có người trong khung hình. OpenLORIS quay ở văn phòng, nhà ở, siêu thị và hành lang, từ độ cao của một robot dịch vụ, lặp lại nhiều lần trong nhiều tuần [335]. Chênh lệch độ khó giữa hai tập không nằm ở chuyển động của camera. Nó nằm ở nội dung cảnh, và nội dung đó phá vỡ ba giả định cùng lúc.

- **Có người đi lại** — giả định thế giới tĩnh gãy trong lúc đang chạy, ở quy mô vài giây.
- **Cảnh đổi giữa các lần đi qua** — ghế bị đẩy, hộp bị chuyển, cửa mở rồi đóng. Bản đồ cũ vẫn còn điểm ở chỗ không còn gì cả.
- **Bề mặt trắng và sàn bóng** — số đặc trưng còn lại vốn đã ít, nên bất kỳ việc xoá bớt nào cũng đắt hơn hẳn so với trên EuRoC.

Điểm thứ ba quan trọng hơn về ngoài của nó, và ta sẽ quay lại. Trên một cảnh giàu văn, xoá một phần đáng kể số điểm gần như không mất gì. Trên một hành lang trắng, đó có thể là toàn bộ khoảng cách giữa bám được và mất bám.

B. Ngữ nghĩa là kênh đưa tri thức “cái gì có thể di chuyển” vào hệ

Cần phát biểu cho hẹp lại, vì cụm “hiểu cảnh” quá rộng để làm việc được. Với một hệ đo quỹ đạo, ngữ nghĩa làm đúng một việc: nó đưa vào hệ một mẫu tri thức tiên nghiệm mà hình học không thể tự suy ra được từ hai khung hình liên tiếp — *loại vật nào trên đời này có khả năng tự di*

chuyển. Hình học chỉ nhìn thấy điểm ảnh và phần dư. Nó không biết một mảng điểm ảnh là cái ghế hay là bức tường. Ngữ nghĩa biết, vì nó đã xem hàng triệu ảnh có nhãn.

Đó là tất cả những gì tầng ngữ nghĩa cho không. Mọi thứ khác — vật này *có* đang động không, nó động lúc nào, nó sẽ còn ở đó lần sau không — vẫn phải do hình học hoặc do bộ nhớ của bản đồ trả lời.

Còn một việc thứ hai, lớn hơn nhiều nhưng phục vụ người khác. Ngữ nghĩa cho bản đồ một *từ vựng*. Một bản đồ có từ vựng dùng được bởi một bộ lập kế hoạch hoặc một con người, chứ không chỉ bởi một script tính APE. Hai việc này thường bị viết chung trong cùng một paper, và trộn chúng là nguồn nhầm lẫn lớn nhất khi đọc mảng này. Nửa sau của chương tách chúng ra.

Học chương này thì làm được gì mà trước đó không làm được? Bạn đọc một paper “semantic dynamic SLAM” và nói ngay được nó đang giải bài nào trong ba bài, nó nặng cỡ nào so với ngân sách bạn còn trống, và bảng kết quả của nó có chuyển sang robot của bạn được không. Trong phần lớn trường hợp câu trả lời là không, và biết lý do sẽ tiết kiệm cho bạn vài tháng.

TRỰC GIÁC

Một mặt nạ ngữ nghĩa là danh sách nhãn chứng mà bạn quyết định không tin, dựa trên nghề nghiệp của họ chứ không dựa trên lời khai. Loại nhãn chứng theo nghề rất rẻ và thường đúng, vì có những nghề hay nói dối thật. Nhưng giả sử tất cả người làm nghề đó tình cờ đứng ở nửa trái căn phòng. Sau khi loại họ, mọi lời khai còn lại đều đáng tin tuyệt đối — và tất cả đều nói về nửa phải. Bạn không còn cách nào dựng lại chuyện xảy ra ở nửa trái. Đó chính xác là chuyện xảy ra với điều kiện của bài toán tối ưu khi mặt nạ xoá một mảng lớn liên khối của ảnh: phần dư còn lại nhỏ và đẹp, còn tư thế thì có một hướng không ai ràng buộc.

C. Chương này chạm nỗi đau nào

Trung thực về phạm vi trước khi đi tiếp. Trong bốn nỗi đau của Phần F, chương này chạm thẳng vào **cảnh động**, một nỗi đau mà EuRoC không hề đo và vì thế dễ bị bỏ quên. Nó chạm *gián tiếp* vào **đóng vòng**, theo hai đường: một vòng sai có thể sinh ra vì hai khung hình giống nhau ở chỗ có cùng một chiếc xe đẩy chứ không phải cùng một hành lang; và đồ thị cảnh phân tầng đề xuất một cách tìm ứng viên vòng khác hẳn cách dùng bộ mô tả ảnh toàn cục.

Nó gần như không chạm vào **chuyển động nhanh**, **ảnh mờ** và **trọng số IMU**. Tệ hơn thế: chuyển động nhanh và ảnh mờ làm cho chính tầng ngữ nghĩa hỏng, nên chương này *thừa hưởng* hai nỗi đau đó chứ không chữa chúng. Đó là một trong những kết luận đáng nhớ nhất ở đây, và mục cuối sẽ nói kỹ.

33.2 Công cụ nhận thức chạy được trong vòng lặp (Perception Front-Ends in the Loop)

Chương 14 đã dạy các mô hình này là gì: một giai đoạn khác hai giai đoạn ở đâu, vì sao **triệt phi cực đại** (*non-maximum suppression*) là heuristic cuối cùng, DETR bỏ nó bằng **ghép cặp Hungary** (*Hungarian matching*) thế nào, và phân đoạn hợp nhất được ba bài toán ra sao. Mục này không nhắc lại cơ chế. Nó hỏi một câu khác hẳn, câu mà một kỹ sư hệ thống thời gian thực phải hỏi trước tiên: *cái nào bỏ vừa vào ngân sách của tôi, và bỏ vào chỗ nào*.

A. Ngân sách thật trên Orin, trước khi bàn tới mô hình nào

Bắt đầu từ ràng buộc chứ đừng bắt đầu từ bảng mAP. Ngân sách của một vòng theo vết là chu kỳ khung hình, không hơn. Trong ngân sách đó, một hệ stereo-inertial đã tiêu phần lớn cho trích xuất đặc trưng, ghép cặp và tối ưu tư thế. Phần còn trống cho một mạng nhận thức chỉ là phần thừa ra sau ba việc ấy, và nó hẹp hơn nhiều so với trực giác. Bao nhiêu thì *chỉ máy của bạn trả lời được*, và phép đo thì rẻ: chạy hệ SLAM một mình trên phần cứng thật, ghi phân vị cao của chu kỳ luồng theo vết, rồi lấy chu kỳ khung trừ đi con số đó. Phần dư ra mới là ngân sách thật, và nó thường nhỏ tới mức bất ngờ.

Ba sự thật về phần cứng quyết định thiết kế, và cả ba đều hay bị bỏ qua khi đọc bảng FPS trong paper.

- **GPU là tài nguyên dùng chung.** Trên Jetson Orin không có GPU rời thứ hai. Nếu bạn đã chuyển trích xuất ORB lên GPU, mạng nhận thức và bộ trích xuất tranh nhau cùng các nhân tính toán và cùng băng thông bộ nhớ. “Chạy song song” trên giấy trở thành chạy xen kẽ trên máy, và độ trễ của cả hai đều tăng.
- **Bộ tăng tốc học sâu chuyên dụng là đòn bẩy song song thật.** Dòng AGX Orin có hai khối tăng tốc học sâu tách rời khỏi GPU. Đưa mạng nhận thức xuống đó là cách duy nhất để nó thật sự chạy song song với luồng theo vết. Cái giá là tập phép toán được hỗ trợ hẹp hơn nhiều, nên phần lớn mạng phải sửa mới chạy nổi, và một số tầng vẫn rơi ngược về GPU.
- **Đuôi mới giết bạn, không phải trung bình.** Một mạng có trung bình nằm gọn trong ngân sách nhưng có phân vị 99 vượt hẳn ngân sách sẽ làm rớt khung đúng lúc cảnh phức tạp. Với các detector còn dùng triết phi cực đại, thời gian hậu xử lý tỉ lệ với số khung sống sót, nên đuôi độ trễ dày lên đúng ở cảnh đông người — tức là đúng lúc bạn cần mặt nạ.

Từ ba điều đó suy ra một kết luận kiến trúc, và nó quan trọng hơn việc chọn mô hình nào. **Tầng nhận thức không nên nằm trong đường tới hạn của luồng theo vết.** Cấu hình thực dụng là chạy mạng trên một luồng riêng, ở tần số thấp hơn, chỉ trên khung hình chính, rồi truyền mặt nạ sang các khung xen giữa bằng một cơ chế rẻ. Hình 33.1 vẽ cả hai cách nối, và cho thấy vì sao cách đồng bộ hỏng.



Hình 33.1. Hai cách nối tầng nhận thức vào vòng lặp. Cách đồng bộ cộng thẳng độ trễ mạng vào ngân sách mỗi khung, và đuôi độ trễ dày lên đúng ở cảnh đông người. Cách bất đồng bộ giữ nhịp cho luồng theo vết nhưng đổi lấy một mặt nạ luôn lạc hậu và khung — sai lệch đó tỉ lệ với tốc độ của vật, nên nó lớn nhất đúng ở trường hợp mặt nạ đáng giá nhất.

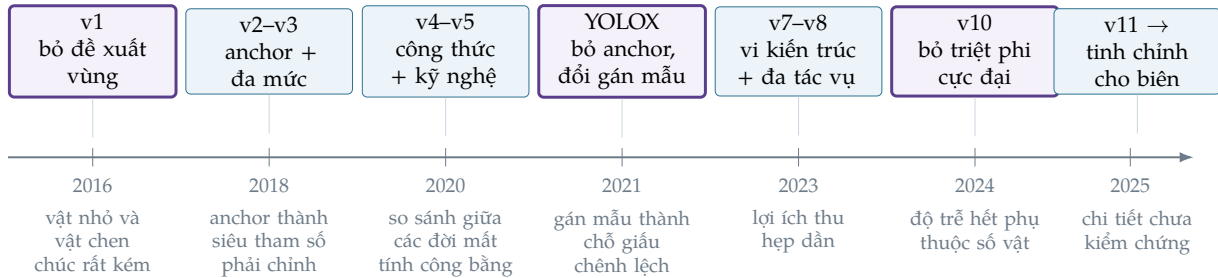
B. Dòng YOLO: cái gì thật sự đổi qua các đời

Mười một phiên bản trong tám năm tạo ấn tượng về một chuỗi đột phá liên tiếp. Đọc kỹ thì không phải vậy. Phần lớn chênh lệch giữa các đời đến từ ba nguồn ít được viết trong tiêu đề: quy tắc gán mẫu, công thức huấn luyện, và kỹ nghệ triển khai. Chỉ có vài bước thật sự đổi cấu trúc bài toán. Với một kỹ sư SLAM, chỉ hai bước trong số đó đáng quan tâm.

- **v1 (2016)** — bỏ hẳn giai đoạn đề xuất vùng, hồi quy thẳng trên một lưới [144]. Đây là lần đổi phát biểu thật sự đầu tiên, và nó mua lấy tốc độ bằng cách hy sinh vật nhỏ và vật chen chúc.
- **v2, v3 (2017–2018)** — *anchor* lấy từ phân cụm kích thước hộp, rồi dự đoán trên ba mức độ phân giải. Bước thứ hai chứa đúng điểm yếu vật nhỏ của v1.
- **v4 (2020)** — một tập hợp thủ thuật huấn luyện và một cái *neck* tốt hơn, gói lại thành “bag of freebies” [336]. Đóng góp thật nằm ở khảo sát có hệ thống xem thủ thuật nào cộng dồn được, chứ không ở một ý tưởng mới.
- **v5 (2020)** — không có bài báo. Cái đổi là kỹ nghệ: đóng gói, xuất mô hình, huấn luyện lại dễ. Với người triển khai, đây là phiên bản đổi nhiều nhất trong đời thực, và đó là một nhận xét về ngành chứ không phải về thuật toán.
- **YOLOX (2021) và v8 (2023)** — bỏ *anchor*, tách đầu phân loại khỏi đầu hồi quy, và đổi quy tắc gán mẫu [337]. Đây là bước đổi cấu trúc thứ hai, và Chương 14 đã cho thấy vì sao gán mẫu mới là chỗ giấu phần lớn chênh lệch.
- **v6, v7, v9, v11 (2022–2024)** — vi kiến trúc *backbone* và *neck*: khôi tái tham số hoá cho thân thiện với lượng tử hoá, luồng thông tin phụ lúc huấn luyện, khối chú ý nhẹ [338]. Có lợi, nhưng lợi ích thu hẹp dần qua từng đời và không đổi cách bạn nối nó vào hệ.
- **v10 (2024)** — huấn luyện với hai phép gán nhất quán để bỏ hẳn triết phi cực đại lúc suy luận [339]. Đây là bước đáng quan tâm nhất cho hệ thời gian thực, vì nó cắt đúng cái phần hậu xử lý có thời gian chạy phụ thuộc nội dung cảnh.

Dòng này vẫn tiếp tục sau v11; tới 2025–2026 đã có các bản đi tiếp theo hướng bỏ triệt phi cực đại và tối ưu cho biên. Tôi không kiểm chứng được chi tiết của chúng nên không mô tả số liệu ở đây. Hình 33.2 xếp cả dòng trên một trục và ghi rõ mỗi bước gỡ nút thắt nào.

Khung viền tím: ba bước thật sự đổi cấu trúc bài toán. Các bước còn lại đổi công thức, vi kiến trúc hoặc kỹ nghệ.



Hình 33.2. Dòng YOLO đọc theo nút thắt chứ không theo số phiên bản. Dòng dưới mỗi mốc ghi vấn đề còn lại sau bước đó. Chỉ ba mốc đổi cấu trúc: bỏ đề xuất vùng, bỏ anchor kèm đổi gán mẫu, và bỏ triệt phi cực đại. Với hệ thời gian thực, mốc cuối đáng giá nhất vì nó cắt phần hậu xử lý có thời gian chạy phụ thuộc nội dung cảnh.

C. Họ DETR: vì sao “không triệt phi cực đại” là một tính chất hệ thống

DETR đổi phát biểu thành dự đoán một tập hợp, ghép với chân trị bằng ghép cặp Hungary một-một, nên mỗi vật chỉ có đúng một dự đoán được thưởng và mô hình tự học cách không trùng lặp [47]. Chương 14 đã giải thích cơ chế đó. Điều Chương 14 chưa nói là vì sao một kỹ sư thời gian thực nên quan tâm.

Lý do không phải độ chính xác. Lý do là *hình dạng phân bố độ trễ*. Một detector có triệt phi cực đại tiêu thời gian tỉ lệ với số khung sống sót sau ngưỡng điểm. Số đó nhảy vọt ở cảnh đông người. Nói cách khác, đuôi độ trễ của bạn dày lên đúng ở cảnh mà bạn bật tầng ngữ nghĩa lên để xử lý. Một mô hình không triệt phi cực đại có thời gian chạy gần như hằng số, và với một bộ lập lịch thời gian thực thì hằng số quý hơn nhanh.

↪ **tiền đề C/14/02** – cơ chế ghép cặp một-một nằm ở Chương 14; ở đây ta chỉ dùng hệ quả của nó về phân bố độ trễ

RT-DETR đẩy ý tưởng đó vào vùng thời gian thực [340]. Cơ chế là một bộ mã hoá lai: nó tách việc trộn thông tin trong cùng một mức độ phân giải khỏi việc trộn giữa các mức. Cách chọn truy vấn khởi đầu cũng tốt hơn bản gốc. Nó còn một tính chất hiếm và rất hợp với robot: có thể đổi tốc độ bằng cách bớt tầng giải mã lúc chạy, *không cần huấn luyện lại*. Với một hệ phải hạ chất lượng khi nóng máy hoặc khi tải cao, đó là một nút vặn có thật. Dòng RF-DETR năm 2025 đi tiếp bằng cách gắn một *backbone* tự giám sát mạnh vào khung DETR; các con số họ công bố tôi chưa kiểm chứng độc lập nên không dẫn lại ở đây.

D. SAM và SAM 2: phân đoạn bất kỳ, và chi phí thật của nó

SAM đổi câu hỏi từ “vật này là lớp gì” sang “vùng nào là một vật”, và trả lời theo prompt: một điểm bấm, một khung, hoặc một mặt nạ thô [62]. SAM 2 mở rộng sang video bằng một bộ nhớ chạy dọc thời gian, nên mặt nạ theo được vật qua các khung thay vì phải phân đoạn lại từ đầu [341].

Cấu trúc chi phí của SAM là bài học thiết kế đáng lấy đi kể cả khi bạn không dùng SAM. Bộ mã hoá ảnh rất nặng và chạy *một lần mỗi ảnh*. Bộ giải mã mặt nạ rất nhẹ và chạy *một lần mỗi prompt*. Vì thế hỏi thêm mười vùng nữa trên cùng một ảnh gần như miễn phí, trong khi chuyển sang ảnh mới thì trả lại toàn bộ giá. Đây đúng là mẫu “đặt phần đắt ở chỗ được khấu hao” mà bạn đã dùng khi tách trích xuất đặc trưng khỏi ghép cặp.

Nhưng có hai giới hạn phải nói thẳng, và cả hai đều chí mạng cho việc lọc vật động.

- **SAM không cho bạn nhãn.** Nó trả về các vùng, không trả về chữ “person”. Muốn biết vùng nào là vật có thể động, bạn vẫn cần một bộ phân loại hoặc một mô hình ngôn ngữ-thị giác chạy thêm. Chi phí thật vì thế là chi phí SAM cộng chi phí gán nhãn.
- **Chi phí bộ mã hoá không vừa ngân sách khung hình.** Bản gốc dùng một *transformer* thị giác cỡ lớn chạy toàn độ phân giải, tức dạng phép tính nặng nhất trong tất cả các công cụ mục này bàn tới. Các biến thể nhẹ của SAM 2 nhẹ hơn hẳn, nhưng vẫn không cùng hạng với một detector nhỏ. Tôi không nêu con số vì tôi không tự đo được nó trên phần cứng của bạn; phép đo đúng nằm ở dự án thứ hai cuối chương. Kết luận thực dụng thì không phụ thuộc con số: đây là công cụ ngoại tuyến, hoặc công cụ chạy *thừa* trên khung hình chính.

Tới 2026 dòng này đã có bản nhận prompt ở mức khái niệm và chạy trên video; tôi không kiểm chứng được chi tiết nên chỉ ghi nhận sự tồn tại của nó.

E. Grounding DINO và Open-YOLO 3D: khi tập nhãn phải mở

Grounding DINO nhận một câu chữ làm điều kiện và phát hiện vật theo mô tả, nhờ trộn đặc trưng ảnh với đặc trưng chữ ở nhiều tầng [342]. Nó gỡ đúng nút thắt tập nhãn đóng: nhà kho của bạn có xe đẩy hàng, pallet và tấm chắn, và COCO không có lớp nào cho chúng.

Cái giá là kích thước. Một bộ mã hoá chữ cộng một *backbone* nặng đặt nó vào cùng hạng nặng với SAM. Vai trò đúng của nó vì thế không phải trong vòng lặp. Nó là **bộ gán nhãn ngoại tuyến**: cho nó chạy chậm trên dữ liệu thu được, lấy nhãn ra, rồi chùng cất xuống một detector nhỏ chạy được thời gian thực. Mẫu “mô hình lớn dạy mô hình nhỏ” này là cách duy nhất tôi biết để có tập nhãn mở mà vẫn giữ được ngân sách.

Open-YOLO 3D là một ví dụ tốt của cùng tinh thần tiết kiệm, ở một tầng khác [343]. Bài toán là phân đoạn thực thể 3D với tập nhãn mở. Cách làm thông thường là chạy một mô hình thị giác-ngôn ngữ nặng trên từng khung rồi nâng kết quả lên 3D, và nó rất chậm. Open-YOLO 3D thay mặt nạ 2D bằng *hộp* 2D từ một detector mở nhẹ hơn, rồi dùng các hộp đó để bỏ phiếu nhãn cho các đề xuất 3D. Bài học thiết kế nằm ở chỗ này, và nó áp được sang chỗ khác: hỏi xem bạn thật sự cần độ chi tiết nào, rồi mua đúng độ chi tiết đó. Nếu việc chỉ cần biết “đám điểm này thuộc vật gì”, một cái hộp là đủ và mặt nạ từng điểm ảnh là lãng phí.

Bảng 33.1. Sáu công cụ nhận thức, đọc theo con mắt ngân sách. Cột sức nặng là một *thứ hạng định tính*, xếp theo hai thứ nhìn thấy được mà không cần chạy: kích thước mô hình, và dạng phép tính (một lượt trên ảnh nhỏ là nhẹ; một *transformer* thị giác chạy toàn độ phân giải là nặng). Thứ hạng này *không thay được phép đo*: sức nặng thật chỉ hiện ra khi bạn chạy mô hình cùng lúc với hệ SLAM trên chính máy của mình, theo quy trình ở dự án thứ hai cuối chương. Cột cuối là cột quyết định.

Công cụ	Cho bạn cái gì	Sức nặng (thứ hạng, không phải số đo)	Song song với theo vết?	Hóng khi nào
YOLO nhỏ (n/s, 640)	Hộp + nhãn, tập đóng	Nhẹ — mạng nhỏ, một lượt, ảnh nhỏ	Được, kể cả trên bộ tăng tốc chuyên dụng	Vật nhỏ, vật bị che, lớp ngoài COCO; đuôi độ trễ nếu còn triết phi cực đại
YOLO trung bình, RT-DETR nhẹ	Hộp + nhãn, độ trễ ổn định hơn	Trung bình — cùng dạng phép tính, mô hình lớn hơn	Chỉ ở tần số thấp hơn nhịp khung	Tranh chấp GPU với trích xuất đặc trưng; vẫn tập đóng
Phân đoạn thực thể nhẹ	Mặt nạ + nhãn	Trung bình — thêm một đầu ra dày đặc theo từng điểm ảnh	Khung hình chính thời	Biên mặt nạ lệch vài điểm ảnh — đúng chỗ góc ORB hay nằm
SAM	Vùng, <i>không</i> nhãn	Nặng — <i>transformer</i> thị giác cỡ lớn cho mỗi ảnh	Không	Không nói được vùng nào là vật động; phải cộng thêm bộ gán nhãn
SAM 2 (bản nhẹ)	Vùng + theo vết qua khung	Trung bình tới nặng, tùy biến thể	Chỉ trên khung hình chính, chạy thưa	Trôi mặt nạ khi vật bị che lâu; vẫn không có nhãn
Grounding DINO	Hộp theo mô tả chữ, tập mở	Nặng — cộng thêm cả một bộ mã hoá chữ	Không	Quá chậm cho vòng lặp; dùng làm bộ gán nhãn ngoại tuyến

CẠM BẤY

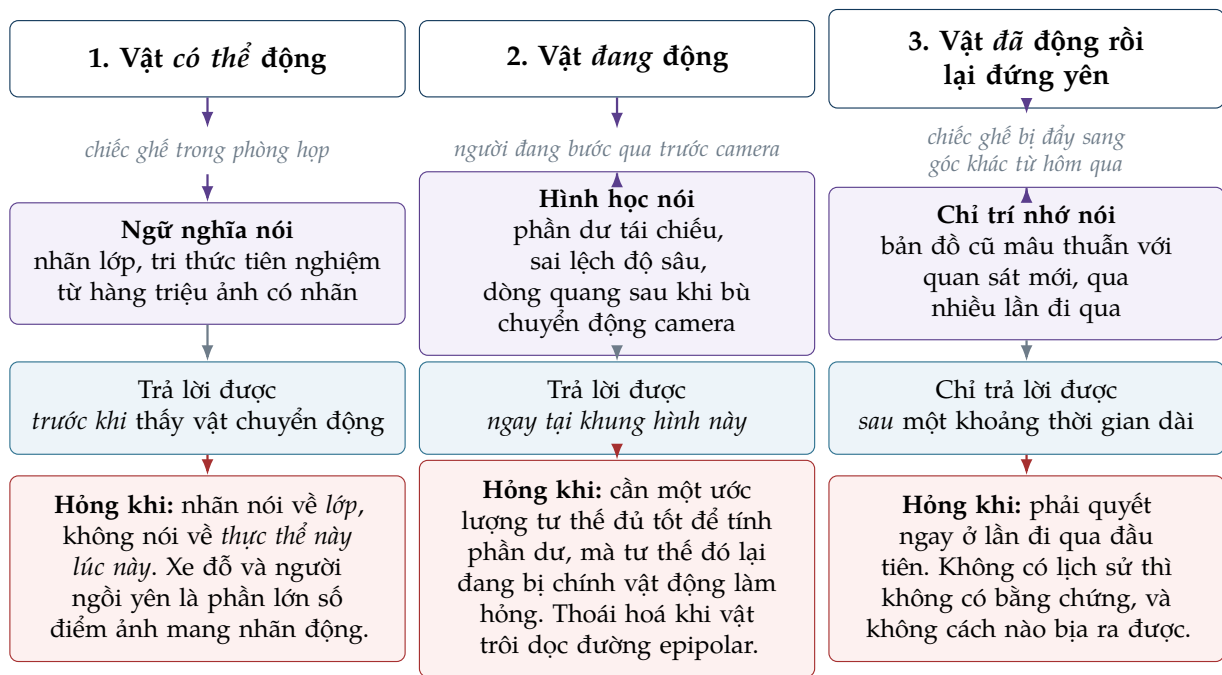
Bảng FPS trong paper gần như luôn đo trên máy để bàn có GPU rời, ảnh đơn lẻ, kích thước *batch* lớn, và *không có* tiến trình nào khác chạy cùng. Ba điều kiện đó đều sai với bạn. Chuyển sang Orin, *batch* bằng một, và có một hệ SLAM đang chiếm GPU, con số tụt đi rất xa — và không bảng nào cho bạn biết trước tụt bao nhiêu. Cách kiểm nhanh và trung thực: chạy mạng *cùng lúc* với hệ SLAM thật, đo p99 của cả hai, rồi mới nói tới trung bình.

33.3 Ba việc khác nhau khi loại vật thể động (Three Different Jobs)

Đây là mục cốt lõi của chương. Tài liệu về mảng này gộp ba việc rất khác nhau vào cùng một cụm “xử lý cảnh động”, và chính việc gộp đó khiến người đọc không thấy được vì sao một phương pháp hoạt động trên tập này mà chết trên tập kia. Tách ra thì mọi thứ sáng ra ngay.

A. Ba câu hỏi, ba nguồn bằng chứng

Ba việc là ba câu hỏi khác nhau về cùng một mảng điểm ảnh, và mỗi câu chỉ có đúng một nguồn bằng chứng trả lời được. Hình 33.3 là bản đồ của cả mục, và cũng là hình đáng nhớ nhất của chương.



Ba cột độc lập: không cột nào suy ra được cột nào. Một hệ chỉ có cột 1 xoá nhầm xe đỗ. Một hệ chỉ có cột 2 không khởi động được khi vật động chiếm phần lớn khung hình. Một hệ chỉ có cột 3 vô dụng ở lần chạy đầu tiên.

Hình 33.3. Ba việc khác nhau bị gộp dưới cùng một cái tên “xử lý cảnh động”. Mỗi cột có một nguồn bằng chứng riêng, một thời điểm mà nó trả lời được, và một kiểu hông riêng. Đây là bản đồ để đọc mọi paper trong mảng này: hãy hỏi paper đó đứng ở cột nào, và nó giả định hai cột kia đã được ai lo.

B. Cột 1 – vật có thể động, và ngữ nghĩa là nguồn duy nhất

Cột này là chỗ ngữ nghĩa thật sự không thể thay thế. Không có phép đo hình học nào trên hai khung hình liên tiếp nói được rằng cái ghế *có khả năng* bị đẩy, còn bức tường thì không. Tri thức đó đến từ nơi khác: từ việc mô hình đã xem hàng triệu ảnh trong đó ghế xuất hiện ở đủ chỗ còn tường thì không.

Giá trị của tri thức tiên nghiệm ấy là nó có *trước* khi có bằng chứng. Ở khung hình đầu tiên của chuỗi, bạn chưa có tư thế nào để tính phần dư, chưa có lịch sử nào để so. Nhưng bạn đã biết ngay rằng nửa dưới khung hình là một người. Điều đó cho phép hạ trọng số ngay lập tức, và giai đoạn khởi tạo là giai đoạn mong manh nhất của cả hệ.

Giới hạn cũng nằm ngay trong định nghĩa. Nhãn là một mệnh đề về *lớp*, không phải về *thực thể này ở khung hình này*. Trong phần lớn cảnh thật, đa số điểm ảnh mang nhãn “có thể động” lại đang đứng yên hoàn toàn: xe đỗ ven đường, người ngồi làm việc, ghế kê quanh bàn. Chúng là cấu trúc tốt, ổn định, giàu văn, và thường nằm ở khoảng cách vừa đẹp cho tam giác hoá. Xoá chúng là xoá dữ liệu tốt.

C. Cột 2 – vật đang động, và cái vòng luẩn quẩn của nó

Cột này thuộc về hình học, và bằng chứng thì có nhiều dạng. Bốn dạng hay dùng nhất, xếp theo độ mạnh của tín hiệu.

1. **Sai lệch độ sâu.** Chiếu một điểm bản đồ vào khung hiện tại, so độ sâu dự đoán với độ sâu đo được từ stereo hoặc RGB-D. Lênh quá ngưỡng nghĩa là vật đã dịch dọc trục nhìn. Đây là kiểm tra mạnh nhất khi có độ sâu tin cậy, và là kiểm tra mà DynaSLAM dùng [218].

2. **Khoảng cách tới đường epipolar.** Với hai khung và một tư thế tương đối, một điểm tĩnh phải nằm trên đường epipolar của nó. Lệch khỏi đường đó là bằng chứng chuyển động. Rẻ, không cần độ sâu, nhưng mù với chuyển động *dọc theo* đường epipolar.
3. **Dòng quang sau khi bù chuyển động camera.** Dự đoán trường dòng quang mà một cảnh tĩnh sẽ tạo ra dưới chuyển động camera đã ước lượng, rồi trừ khỏi dòng quang đo được. Phần dư khác không là chuyển động thật của vật. Đây là tín hiệu dày đặc nhất, và là chỗ [F/30-do-sau-dong-quang-va-mo-hinh-nen](#) nổi vào mục này.
4. **Tính nhất quán của một cụm điểm.** Nếu một nhóm điểm cùng vi phạm ràng buộc theo cùng một hướng, đó là một vật, không phải nhiễu. Đây là tín hiệu phân biệt vật động thật với ngoại lai ngẫu nhiên, và cũng là tín hiệu đắt nhất để tính.

Cả bốn đều mắc chung một vòng luẩn quẩn, và nói thẳng ra nó là điều trung thực nhất có thể nói về cột này. Muốn tính phần dư, bạn cần tư thế camera. Nhưng tư thế camera đang được ước lượng từ chính tập điểm chứa vật động. Khi vật động chiếm phần nhỏ khung hình, RANSAC gỡ được vòng này: nghiệm đồng thuận bám theo nền tĩnh, phần dư của vật động vọt lên, và bạn phát hiện được chúng. Khi vật động chiếm phần lớn khung hình, nghiệm đồng thuận bám theo *vật động*, phần dư của nền tĩnh vọt lên, và bạn phát hiện ngược hoàn toàn.

CẦN NHỚ

Đây là câu trả lời cho câu hỏi “RANSAC đã tốt rồi, mặt nạ thêm được gì”. RANSAC hỏng khi ngoại lai *đồng và nhất quán*, vì khi đó chúng thắng phiếu đồng thuận. Một đám đồng đi cùng chiều, một xe tải chạy song song, một bức tường người trong thang máy — đó chính xác là các trường hợp đó. Ngữ nghĩa hữu ích ở đúng chỗ này, và chỉ ở đây: nó phá thế bế tắc bằng một lá phiếu đến từ ngoài hình học. Ngoài vùng đó, nó chủ yếu chỉ lấy đi dữ liệu tốt. [T3]

Thoái hoá thứ hai đáng nhớ vì nó im lặng: vật di chuyển *dọc theo* tia nhìn hoặc dọc theo đường epipolar không tạo ra vi phạm nào cả. Một người đi thẳng ra xa camera trông y hệt một người đứng yên ở xa hơn một chút. Không kiểm tra hai khung nào bắt được trường hợp này. Chỉ có độ sâu đo trực tiếp, hoặc nhãn ngữ nghĩa, mới nói được.

D. Cột 3 – vật đã động rồi lại đứng yên, và chỉ trí nhớ nói được

Cột này bị bỏ quên nhiều nhất, dù với một robot dịch vụ nó lại là cột tồn kém nhất. Chiếc ghế hôm qua ở giữa phòng, hôm nay ở góc phòng. Ở cả hai lần đi qua, nó đứng yên tuyệt đối. Không phép kiểm tra hình học nào trong một lần chạy phát hiện được gì, vì trong lần chạy đó nó đứng là tĩnh. Nhãn ngữ nghĩa thì báo động ở cả hai lần, kể cả khi ghế không hề bị đụng tới.

Bằng chứng duy nhất là mâu thuẫn giữa bản đồ cũ và quan sát mới. Điểm bản đồ ở vị trí cũ liên tục không được quan sát lại dù nằm trong tầm nhìn và không bị che. Đó là tín hiệu “chỗ này không còn gì”. Nó đòi hỏi ba thứ mà một hệ SLAM một-lần-chạy không có: bản đồ sống qua nhiều phiên, cơ chế đếm số lần quan sát được và số lần đáng lẽ phải quan sát được, và một chính sách xoá điểm dựa trên tỉ số đó.

Đây chính là chỗ khái niệm “bản đồ theo thời gian” bước vào, và mục về đồ thị cảnh sẽ quay lại với Khronos. Điều đáng ghi nhớ ngay bây giờ: đây là bài toán *bảo trì bản đồ*, không phải bài toán nhận thức. Ném một mạng phân đoạn vào nó không giải quyết được gì.

E. DynaSLAM và họ hàng: các thiết kế đã có

Với ba cột trong tay, đọc các hệ thống có sẵn trở nên nhanh. Mạch dưới đây đi theo thứ tự lịch sử, và mỗi bước gỡ đúng một nút thắt của bước trước.

I Mạch — từ mặt nạ tới bản đồ bốn chiều

- 1 Vấn đề** ORB-SLAM2 chạy trên chuỗi có người đi ngang thì quỹ đạo lệch hẳn, dù mọi thứ khác đều đúng. Nguyên nhân là các điểm trên người vẫn được đưa vào tối ưu như điểm tĩnh.

Giải pháp DynaSLAM đặt một mạng phân đoạn thực thể trước bộ theo vết, xoá mọi đặc trưng rơi vào mặt nạ của các lớp có thể động, rồi chạy thêm một kiểm tra hình học đa khung cho phần còn lại [218].

Giả định cột 1 và cột 2 bổ sung cho nhau, và cái nào bắt được thì bắt.

Giá phải trả một lượt mạng nặng cho mỗi khung, và mất toàn bộ đặc trưng trên các vật đứng yên thuộc lớp động.
- 2 Vấn đề** kiểm tra hình học của DynaSLAM cần độ sâu, nên bản đơn ảnh chỉ còn lại mặt nạ.

Giải pháp DS-SLAM ghép một mạng phân đoạn nhẹ với một kiểm tra nhất quán chuyển động dựa trên dòng quang và ràng buộc epipolar, chạy trên hai luồng song song [344].

Hệ quả đây là bản thiết kế đầu tiên coi tầng ngữ nghĩa là một luồng riêng thay vì một bước trong đường tới hạn.

Giá phải trả mặt nạ trở một khung, và kiểm tra epipolar mù với chuyển động dọc đường epipolar.
- 3 Vấn đề** xoá vật động là vứt bỏ thông tin; vật động cũng là một vật có tư thế đáng ước lượng.

Giải pháp nhóm hợp nhất theo vật — MaskFusion và họ hàng — dựng một mô hình 3D riêng cho từng vật và theo vết tư thế của nó song song với tư thế camera [345]. DynaSLAM II đưa hẳn tư thế vật vào bài toán hiệu chỉnh chùm tia, nên vật động trở thành ràng buộc chứ không còn là nhiễu [346].

Giả định mỗi vật là một khối cứng.

Giá phải trả giả định khối cứng sai với con người, tức là với loại vật động phổ biến nhất; chi phí tính toán tăng theo số vật.
- 4 Vấn đề** mọi thứ ở trên chỉ sống trong một lần chạy, nên cột 3 vẫn bỏ ngỏ.

Giải pháp Khronos phát biểu lại bài toán theo trục thời gian: thay vì phân loại tĩnh hay động, hệ ước lượng *khoảng thời gian tồn tại* của từng vật, nên xử lý được cả chuyển động ngắn hạn lẫn thay đổi dài hạn trong cùng một khung [347].

Hệ quả bản đồ trả lời được câu hỏi “thế giới trông thế nào lúc t ” thay vì chỉ “thế giới trông thế nào”.

Giá phải trả cần một stack bản đồ đặc và cần lưu lịch sử, hai thứ mà một hệ điểm thưa không có.

F. Vì sao mặt nạ ngữ nghĩa một mình là không đủ

Gom lại thành sáu lý do độc lập. Mỗi lý do là một chế độ hỏng riêng, và một hệ thật thường gặp vài lý do cùng lúc.

- **Nhân là mệnh đề về lớp, không phải về thực thể.** Đây là lý do gốc, và năm lý do sau chỉ là hệ quả ở các tầng khác nhau. Xe đỗ, người ngồi, ghế kê sát tường đều mang nhãn động mà không hề động.

- **Vật động ngoài tập nhãn.** Xe đẩy hàng, cánh cửa, rèm cửa, thùng hàng được bê đi, con chó. Tám mươi lớp của COCO không phủ được một nhà kho hay một bệnh viện. Tập nhãn mở giải được vấn đề này về nguyên tắc, nhưng với chi phí đã nêu ở mục trước.
- **Chuyển động gián tiếp.** Một vật tĩnh bị vật động làm đổi vẻ ngoài: bóng đổ chạy trên tường, ánh phản chiếu trên sàn bóng và trên kính, một cái ghế bị người đẩy. Điểm ảnh nằm trên bề mặt tĩnh, nên không mặt nạ nào phủ chúng, nhưng phép đo thì sai.
- **Biên mặt nạ lệch đúng chỗ đau nhất.** Mạng phân đoạn sai vài điểm ảnh ở biên. Mà góc ORB thì thích nằm đúng ở biên, vì đó là chỗ gradient mạnh nhất. Bạn buộc phải chọn: nở mặt nạ ra và xoá thêm cấu trúc tĩnh, hay giữ nguyên và để lọt đúng những điểm tệ nhất — các điểm nằm trên đường viền vật động.
- **Mặt nạ luôn cũ.** Nó được tính trên khung k và đến nơi ở khung $k + 2$ hoặc muộn hơn. Sai lệch vị trí tỉ lệ với tốc độ vật, nên nó lớn nhất đúng ở trường hợp mặt nạ đáng giá nhất. Truyền tiếp bằng dòng quang hoặc theo vết hợp chỉ giảm chứ không xoá được sai lệch này.
- **Mạng hồng đúng lúc bạn cần nó.** Ảnh mờ do chuyển động, thiếu sáng, phơi sáng nhảy — đó vừa là những khung hình mà hình học yếu nhất, vừa là những khung hình nằm ngoài phân phối huấn luyện của mạng. Hai kiểu hồng *tương quan dương* với nhau. Một hệ dự phòng chỉ có giá trị khi nó hồng độc lập với hệ chính, và ở đây thì không.

Lý do cuối đáng được nhấn mạnh vì nó là kết luận thiết kế, không phải một chi tiết kỹ thuật. Bạn không thể coi tầng ngữ nghĩa là lưới an toàn cho các khung hình khó. Nó là một tầng hỗ trợ chỉ hoạt động tốt ở đúng những khung hình mà hình học vốn đã ổn.

↪ cùng nguyên lý với B/08 — đây là dịch chuyển phân phối ở dạng độc nhất: phân phối trượt đi đúng theo hướng làm cả hai thành phần của hệ cùng yếu đi

G. Hai phép tính nhỏ cho thấy vùng an toàn hẹp tới đâu

Trước khi bàn tới cái giá của việc xoá, hãy làm hai phép tính bằng tay. Cả hai chỉ dùng số liệu bạn đã có, và cả hai đều cho ra kết quả hẹp hơn trực giác.

Phép tính thứ nhất: một người đứng gần chiếm bao nhiêu phần khung hình. Đây là một phép tính *giả định*, làm để thấy hình dạng của quan hệ chứ không phải để lấy con số. Giả sử một camera kiểu EuRoC, tiêu cự khoảng 460 điểm ảnh, ảnh 752×480 , tức 360 960 điểm ảnh. Một người rộng chừng 0,5 m và cao 1,7 m. Ở khoảng cách 3 m, bề rộng chiếu xuống ảnh là $460 \times 0,5/3 \approx 77$ điểm ảnh và bề cao là $460 \times 1,7/3 \approx 261$. Diện tích chừng 20 000 điểm ảnh, tức *khoảng 6%* khung hình. Vô hại.

Bây giờ cũng người đó ở khoảng cách 1 m. Bề rộng thành $460 \times 0,5 \approx 230$ điểm ảnh, bề cao thành 782 nên bị cắt còn 480. Diện tích là $230 \times 480 = 110\,400$, tức *khoảng 31%* khung hình. Cùng một người, chỉ đổi khoảng cách ba lần, mà tỉ lệ nhảy năm lần. Điều đáng mang đi không phải hai con số mà là quan hệ: diện tích đi theo nghịch đảo bình phương khoảng cách, nên phân bố diện tích vật động có đuôi rất dày. Hệ quả thực dụng là bạn phải đo ở một phân vị cao chứ không ở trung bình — trung bình của cả chuỗi có thể rất nhỏ trong khi đúng vài chục khung hình quyết định lại rất lớn, và chính vài chục khung đó làm hồng quỹ đạo.

Phép tính thứ hai: RANSAC chịu được tới đâu. Số vòng lặp cần để lấy được một mẫu sạch với xác suất p là

$$N = \frac{\log(1 - p)}{\log(1 - (1 - \varepsilon)^s)},$$

với ε là tỉ lệ ngoại lai và s là số điểm tối thiểu mỗi mẫu. Lấy $p = 0,99$ và $s = 4$, rồi thay vài giá trị:

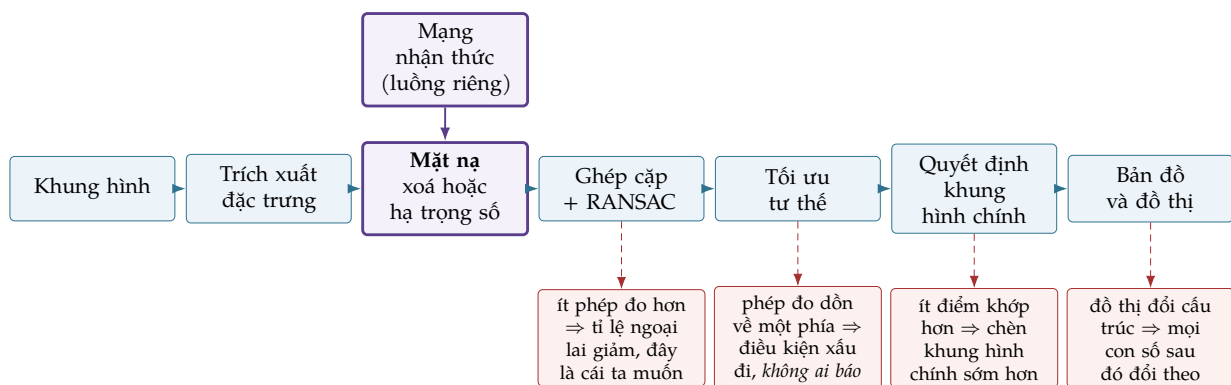
- $\varepsilon = 0,20 \rightarrow (0,8)^4 = 0,41 \rightarrow N \approx 9$ vòng. Rẻ như không.
- $\varepsilon = 0,50 \rightarrow (0,5)^4 = 0,0625 \rightarrow N \approx 72$ vòng. Vẫn chạy được.

- $\varepsilon = 0,70 \rightarrow (0,3)^4 = 0,0081 \rightarrow N \approx 566$ vòng. Bắt đầu đau.
- $\varepsilon = 0,85 \rightarrow (0,15)^4 = 5,1 \times 10^{-4} \rightarrow N \approx 9\,000$ vòng. Hết cửa.

Chi phí bùng nổ theo hàm mũ của s , nhưng đó chưa phải phần đáng sợ nhất. Công thức trên giả định ngoại lai là *ngẫu nhiên*, nên chúng không bao giờ đồng thuận với nhau. Vật động thì không ngẫu nhiên: cả đám điểm trên một người cùng vi phạm theo cùng một hướng. Khi tỉ lệ đó vượt một nửa, tập điểm động trở thành *nhóm đa số nhất quán*, và RANSAC chọn đúng chúng làm nghiệm. Lúc đó nó không chỉ chậm; nó cho ra câu trả lời sai một cách tự tin.

Ghép hai phép tính lại thì ra vùng làm việc thật của tầng ngữ nghĩa, và nó hẹp. Khi vật động chỉ chiếm một phần nhỏ khung hình, RANSAC gánh được và mặt nạ gần như không thêm gì. Khi vật động chiếm quá nửa số phép đo và cùng vi phạm theo một hướng, chúng thành nhóm đa số nhất quán, RANSAC lật ngược, và mặt nạ là thứ duy nhất cứu được — ranh giới “quá nửa” này suy từ chính cơ chế bỏ phiếu, không phải từ một ngưỡng chính tay. Giữa hai đầu là một vùng xám, và chỗ nó bắt đầu với chỗ nó kết thúc thì phụ thuộc cảnh, phụ thuộc mật độ đặc trưng và phụ thuộc chính hệ của bạn. Đó là thứ phải *đo*, không phải thứ đọc ra từ một con số in sẵn, và thí nghiệm oracle ở mục I là cách đo nó.

Hình 33.4 cho thấy mặt nạ cắm vào chỗ nào trong đường ống, và quan trọng hơn, nó đổi những gì ở phía sau. Chú ý rằng nó không chỉ đổi tập phép đo: nó đổi luôn quyết định chèn khung hình chính, và quyết định đó lan ra toàn bộ đồ thị.



Mặt nạ can thiệp ở một chỗ, nhưng hậu quả lan ra bốn chỗ. Chỉ hậu quả đầu tiên là thứ bạn định làm. Ba hậu quả sau là tác dụng phụ, và chúng là lý do một so sánh “có mặt nạ với không mặt nạ” hiếm khi cô lập được đúng biến bạn muốn đo.

Hình 33.4. Mặt nạ cắm vào đâu và nó đổi những gì. Điểm can thiệp chỉ có một, ngay sau trích xuất đặc trưng, nhưng hậu quả lan qua bốn khối phía sau. Hậu quả thứ hai là kiểu hồng im lặng, và hậu quả thứ ba giải thích vì sao mọi so sánh trong mảng này đều cần khóa hoặc báo cáo chính sách khung hình chính.

H. Vì sao xóa quá tay còn tệ hơn không xóa

Đây là điểm mà tôi cho là quan trọng nhất của cả chương, và cũng là điểm ít được viết ra nhất trong các paper về mảng này. Trực giác thông thường nói rằng xóa thừa thì cùng lắm là lãng phí. Trực giác đó sai, vì nó coi các phép đo là những mẫu bằng chứng độc lập và có thể thay thế cho nhau. Chúng không phải vậy.

Bộ tối ưu không cần *nhiều* ràng buộc. Nó cần ràng buộc *trải đủ mọi hướng*. Một mặt nạ xóa đi một mảng lớn liền khối của ảnh — và mặt nạ ngữ nghĩa luôn liền khối, vì vật thể thì liền khối. Các điểm còn lại vì thế dồn về phần còn lại của khung hình. Ma trận thông tin của bài toán tư thế mất đi một hướng, và tư thế có một chiều gần như không bị ràng buộc.

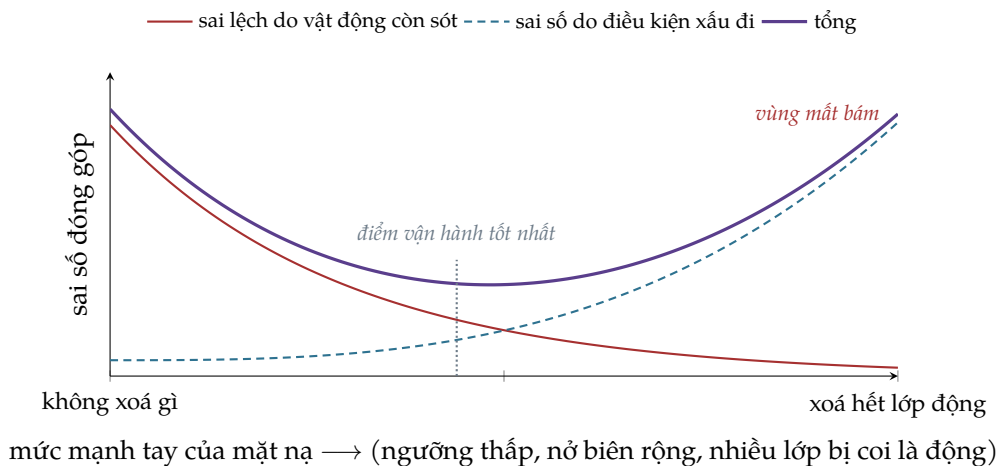
Kiểu hồng này im lặng, và đó là chỗ nguy hiểm. Không có cờ ngoại lai nào bật lên. Phần dư còn lại nhỏ và đẹp, vì mọi phép đo còn lại đều là phép đo tốt. Hiệp phương sai mà hệ báo ra thì lạc

quan, vì nó tính từ chính tập điểm đã bị lọc. Sai số chỉ hiện ra sau đó vài giây, dưới dạng trôi, và lúc đó bạn sẽ đi tìm nguyên nhân ở chỗ khác.

Ba cái giá cụ thể, xếp theo mức độ hay gặp:

- **Mất bám hần.** Hệ tuyên bố mất bám khi số điểm bản đồ theo được tụt dưới ngưỡng. Trên một hành lang trắng, xoá một người đứng gần camera có thể lấy đi phần lớn số đặc trưng còn lại. Cái giá không phải vài xăng-ti-mét sai số; nó là một lần định vị lại, và định vị lại có thể chọn nhầm chỗ.
- **Đổi chính sách khung hình chính.** Ít điểm khớp hơn làm điều kiện chèn khung hình chính bật sớm hơn. Cấu trúc đồ thị đổi, mật độ khung hình chính đổi, và mọi con số sau đó đổi theo. Khi bạn so hai cấu hình mà một bên có mặt nạ, bạn thường đang so hai đồ thị khác nhau chứ không phải hai bộ phép đo khác nhau.
- **Điều kiện xấu đi mà không ai báo.** Trường hợp đã mô tả ở trên. Nó là kiểu hỏng khó chẩn đoán nhất vì mọi chỉ số nội bộ đều trông khoẻ.

Hình 33.5 vẽ hai cái giá đối nghịch nhau theo mức độ mạnh tay của mặt nạ. Đây là hình minh hoạ dạng hàm, không phải số đo: hình dạng của hai đường thì tôi tin, còn vị trí cực tiểu thì phụ thuộc cảnh và phải tự đo trên dữ liệu của bạn.



Hình 33.5. Hai cái giá đối nghịch. Xoá ít thì vật động còn sót lại và tạo sai lệch có hướng. Xoá nhiều thì các điểm còn lại dồn về một phía và điều kiện của bài toán tư thế xấu đi; đường này dốc lên nhanh vì nó phi tuyến, và ở cuối nó chuyển thành mất bám hần. Đường tổng có cực tiểu ở giữa, và cực tiểu đó *phụ thuộc cảnh*: hành lang trắng đẩy nó sang trái, quảng trường đông người đẩy nó sang phải. Hình minh hoạ dạng hàm, không phải số đo.

Từ hình đó rút ra hai kết luận thực dụng. Thứ nhất, **hãy hạ trọng số thay vì xoá**. Một mặt nạ nhị phân là một quyết định cứng đặt ở chỗ tồi nhất có thể: đặt ngay đầu đường ống, trước khi bất kỳ bằng chứng nào được thu thập. Nhân độ tin cậy của mặt nạ vào ma trận thông tin của phép đo thì mềm hơn nhiều. Điểm nghi ngờ vẫn đóng góp vào điều kiện của bài toán, nhưng đóng góp ít. Nếu nó thật sự là ngoại lai, hàm mất mát bền vững sẽ đập nó ở vòng lặp sau, khi đã có bằng chứng thật.

↪ **cùng nguyên lý với F/34** — đây đúng là luận điểm của chương 34: mọi quyết định “tin hay không tin” trong đồ thị nhân tử cuối cùng đều là một con số trong ma trận thông tin, và biến nó thành nhị phân là vứt đi thông tin

Kết luận thứ hai ngắn hơn. **Điểm vận hành phải phụ thuộc vào số điểm còn lại**, chứ không phải là một hằng số chỉnh một lần. Quy tắc mà tôi thấy hợp lý — và nói rõ đây là suy luận thiết kế chứ chưa phải kết quả đo — là để mặt nạ mạnh tay khi số đặc trưng còn lại vẫn dư dả, rồi tự động

nơi lỏng khi số đó tụt về gần ngưỡng mất bám. Hệ nên chấp nhận vài phép đo bản hơn là chấp nhận mất bám, vì mất bám đắt hơn nhiều bậc.

I. Đo cho đúng, nếu không bạn sẽ tin nhầm

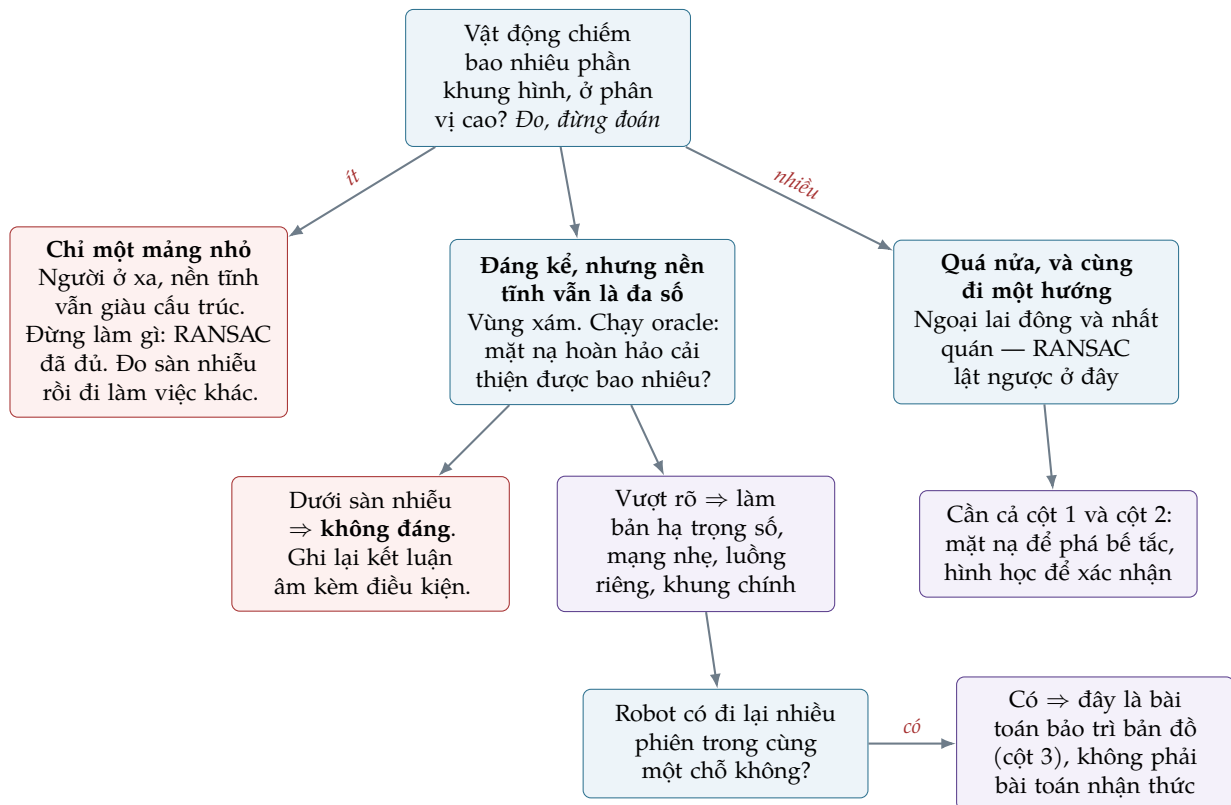
Mảng này có một vấn đề đánh giá nghiêm trọng, và biết trước nó sẽ giúp bạn đọc mọi bảng kết quả trong mảng với đúng liều hoài nghi.

Các con số “cải thiện rất lớn” trong mảng này phần lớn được báo trên vài chuỗi RGB-D trong nhà, nơi có một hoặc hai người chiếm phần rất lớn khung hình, và đối chứng là một hệ hoàn toàn không xử lý cảnh động [71]. Kể cả khi các con số đó thật, chúng gần như vô nghĩa với robot của bạn, vì chúng đo một chế độ hồng thảm khốc trong một cấu hình cực đoan. Robot dịch vụ đi hành lang gặp người chiếm một phần nhỏ khung hình, và ở chế độ đó thì RANSAC vốn đã xử lý ổn.

Bốn kỷ luật đo, và cả bốn đều là thứ bạn đã dùng ở phía SLAM, chỉ cần bê nguyên sang.

1. **Báo tỉ lệ hoàn thành cùng với APE, luôn luôn.** Một hệ mất bám giữa chừng có thể có APE rất đẹp trên đoạn nó còn sống. Với mặt nạ ngữ nghĩa, đây không phải rủi ro lý thuyết mà là chế độ hồng chính. Xem [B/07-chia-du-lieu-va-danh-gia](#) cho cách đọc cặp số này.
2. **Đo sần nhiều trước.** Chạy cùng một cấu hình nhiều lần và lấy độ lệch chuẩn, rồi mới nói về cải tiến [206]. Bạn đã biết hệ của mình là phi tất định; thêm một mạng vào không làm nó tất định hơn.
3. **Chạy một thí nghiệm oracle trước khi làm gì cả.** Gán mặt nạ bằng tay, hoặc bằng một mô hình rất nặng chạy ngoại tuyến, trên vài trăm khung hình. Đó là mặt nạ hoàn hảo. Mặt nạ hoàn hảo cho bạn cái *trần*: mọi công sức làm mặt nạ chạy thời gian thực đều bị chặn trên bởi đúng con số đó. Nếu trần ấy không vượt nổi sần nhiều bạn đã đo, hãy dừng và đi làm việc khác. Đây là công cụ có giá trị cao nhất trong cả mục.
4. **Cô lập yếu tố gây nhiễu về khung hình chính.** So sánh có mặt nạ với không mặt nạ thường đồng thời đổi mật độ khung hình chính. Muốn biết mặt nạ đóng góp gì, hãy khoá chính sách khung hình chính lại, hoặc báo cáo số khung hình chính kèm theo.

Hình 33.6 gói cả mục thành một cây quyết định, và câu trả lời phổ biến nhất trên cây đó là “chưa cần”.



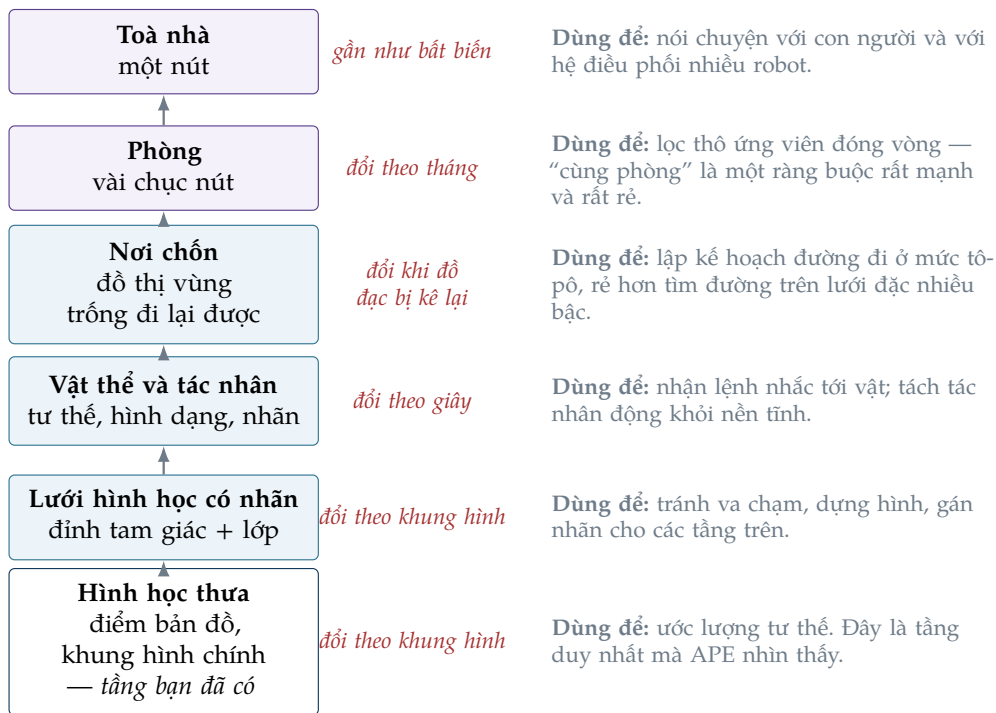
Hình 33.6. Cây quyết định cho tầng ngữ nghĩa. Các nhánh được mô tả bằng *tình huống* chứ không bằng ngưỡng số, vì ngưỡng đúng phụ thuộc mật độ đặc trưng của cảnh và phải tự do. Nhánh trái cùng là nhánh hay đúng nhất và ít được chọn nhất. Chú ý rằng nhánh phải cùng dẫn tới một bài toán khác hẳn — bảo trì bản đồ qua nhiều phiên — mà không mạng nhận thức nào giải được.

33.4 Đồ thị cảnh 3D (3D Scene Graphs)

Tới đây ta đổi câu hỏi. Nửa đầu chương hỏi ngữ nghĩa giúp *ước lượng tư thế* được gì. Nửa sau hỏi ngữ nghĩa cho *bản đồ* thêm được gì, và người tiêu thụ bản đồ ở đây không còn là script tính APE. Điểm xuất phát là một quan sát về biểu diễn. Bản đồ của ORB-SLAM3 là một danh sách phẳng gồm điểm 3D và khung hình chính. Nó hoàn hảo cho việc nó sinh ra để làm: cung cấp ràng buộc cho bộ tối ưu. Nhưng nó không trả lời được bất kỳ câu hỏi nào khác. “Tôi đang ở phòng nào” không có câu trả lời, vì không có khái niệm phòng. “Cái ghế nào gần tôi nhất” cũng vậy. Đồ thị cảnh 3D là câu trả lời cho câu hỏi: nếu ta xếp thêm vài tầng trừu tượng lên trên đám mây điểm đó thì được gì.

A. Cấu trúc tầng, và mỗi tầng mua được gì

Hình 33.7 vẽ cấu trúc mà dòng Kimera và Hydra dùng, cùng tốc độ thay đổi và người tiêu thụ của từng tầng [348], [349]. Cột bên phải là cột đáng đọc nhất: nó cho thấy các tầng trên không phải trang trí, chúng có người dùng thật.



Khi một vòng đóng lại, toàn bộ các tầng phải biến dạng theo, không chỉ đồ thị tư thể. Đó là bài toán kỹ thuật khó nhất của Hydra, và là lý do một đồ thị cảnh không phải chỉ là “bản đồ cộng nhãn”.

Hình 33.7. Sáu tầng của một đồ thị cảnh 3D, kèm tốc độ thay đổi và người tiêu thụ. Tốc độ thay đổi giảm dần khi lên cao, và đó chính là giá trị của cấu trúc tầng: các tầng trên ổn định nên dùng được làm mỏ neo, trong khi tầng dưới thì đổi liên tục. Tầng dưới cùng là thứ hệ SLAM của bạn đã có; năm tầng trên là thứ phải trả thêm để có.

B. Kimera: thư viện, và bài học về ranh giới mô-đun

Ý tưởng gắn nhãn ngữ nghĩa lên một bản đồ đặc không mới; SemanticFusion đã làm điều đó từ 2017 bằng cách hợp nhất dự đoán từng khung vào một bản đồ bề mặt [350]. Cái Kimera thêm vào là tính mô-đun và tính thời gian thực. Kimera là một thư viện mã nguồn mở dựng bản đồ đo đạc kèm ngữ nghĩa theo thời gian thực, gồm bốn mô-đun tách rời: một bộ đo quán tính–thị giác, một bộ tối ưu đồ thị tư thể bền vững, một bộ dựng lưới cục bộ nhanh, và một bộ gắn nhãn ngữ nghĩa lên lưới toàn cục [219].

Với bạn, giá trị lớn nhất của Kimera không nằm ở thuật toán nào cả. Nó nằm ở chỗ bốn mô-đun ấy có giao diện rõ ràng và thay được từng cái. Bạn có thể giữ nguyên bộ đo quán tính–thị giác của mình và chỉ mượn phần dựng lưới có nhãn. Rất ít hệ thống trong Phần F cho phép mượn từng phần như vậy; phần lớn là một khối liền không tháo rời được. Đó là một bài học thiết kế đáng lấy đi kể cả khi bạn không dùng dòng code nào của Kimera.

Bộ tối ưu đồ thị tư thể bền vững cũng đáng chú ý riêng. Nó loại vòng sai bằng một cơ chế bền vững tăng dần thay vì bằng một ngưỡng cứng, và đó là một câu trả lời trực tiếp cho câu hỏi “một vòng sai đắt bao nhiêu” của F/29-dinh-vi-lai-va-dong-vong.

C. Hydra: dựng đồ thị theo thời gian thực, và biến dạng khi vòng đóng

Hydra là bước từ “đồ thị cảnh như một sản phẩm hậu xử lý” sang “đồ thị cảnh dựng tăng dần trong lúc chạy” [349]. Ba đóng góp, và cái thứ hai là cái khó nhất.

- **Dựng tăng dần.** Các tầng được cập nhật liên tục từ dữ liệu cảm biến, chứ không dựng lại từ đầu. Phân đoạn phòng chạy trên đồ thị nơi chôn theo lối hình học chứ không cần một mạng riêng.
- **Biến dạng toàn đồ thị khi đóng vòng.** Đây là chỗ khó. Một đồ thị tư thế chỉ cần chỉnh lại vài chục tư thế. Một đồ thị cảnh phải kéo theo lưới hàng trăm nghìn đỉnh, các vật thể, các nơi chôn và ranh giới phòng, và phải làm trong thời gian thực. Hydra dùng một đồ thị biến dạng nhúng để lan hiệu chỉnh xuống các tầng dưới. Nếu bạn từng viết phần cập nhật bản đồ sau một lần tối ưu toàn cục, bạn biết đây là loại việc dễ sinh bết tắc và dễ sinh trạng thái không nhất quán.
- **Phát hiện vòng từ chính cấu trúc phân tầng.** Thay vì so một bộ mô tả ảnh toàn cục, Hydra so từ trên xuống: cấu hình phòng, rồi chòm sao vật thể, rồi mới tới vẻ ngoài. Đây là ý đáng suy nghĩ nhất của cả mục đối với nỗi đau đóng vòng, vì nó có *chế độ hồng khác* với bộ mô tả vẻ ngoài. Bộ mô tả vẻ ngoài chết vì đổi ánh sáng và vì hành lang giống nhau; chòm sao vật thể thì không quan tâm tới ánh sáng, nhưng lại chết ở nơi trống trải không có vật nào.

Hydra-Multi mở rộng sang nhiều robot cùng dựng chung một đồ thị, với một nút tập trung lo việc gộp và lo các vòng liên robot [351]. Vấn đề kỹ thuật thật nằm ở gộp: hai robot gọi cùng một cái phòng bằng hai nút khác nhau, và phải quyết định khi nào chúng là một.

D. Clio: tác vụ quyết định mức chi tiết

Clio đặt một câu hỏi mà tôi cho là câu hỏi hay nhất trong cả mảng đồ thị cảnh: *cái gì được tính là một vật thể?* [352]

Câu trả lời thông thường lấy từ tập nhãn: một vật là một thứ có tên trong danh sách lớp. Clio nói câu trả lời đó sai về nguyên tắc, vì “vật thể” không phải thuộc tính của cảnh, nó là thuộc tính của *tác vụ*. Một chồng sách là một vật nếu nhiệm vụ là “dọn chồng sách này đi”. Nó là mười vật nếu nhiệm vụ là “tìm quyển bìa đỏ”. Không có mức chi tiết nào đúng khi chưa biết nhiệm vụ.

Cơ chế Clio dùng để hình thức hoá điều đó là nút thất thông tin: cho trước một danh sách nhiệm vụ viết bằng ngôn ngữ tự nhiên, gom các mảnh nguyên thuỷ thành các cụm sao cho cụm giữ được nhiều thông tin nhất về nhiệm vụ trong khi vẫn đơn giản nhất có thể. Bản đồ vì thế *co giãn theo nhiệm vụ*: cùng một cảnh cho ra hai đồ thị khác nhau với hai danh sách nhiệm vụ khác nhau.

Bài học thiết kế ở đây rộng hơn đồ thị cảnh rất nhiều, và nó nổi thẳng vào thói quen của bạn. Đó chính là nguyên tắc “chọn metric khớp chi phí thật” đặt vào tầng biểu diễn: mức chi tiết của bản đồ phải suy ra từ việc bản đồ sẽ được dùng để làm gì, chứ không suy ra từ việc cảm biến cho được bao nhiêu.

→ cùng nguyên lý với B/05 — đây là bước “phát biểu bài toán từ chi phí thật” của chương thiết kế bài toán, lần này áp cho biểu diễn bản đồ chứ không cho hàm mất mát

E. Khronos: thời gian là một trục, không phải một cái cờ

Khronos quay lại đúng cột 3 của mục trước, và nó là câu trả lời nghiêm túc nhất tôi biết cho cột đó [347]. Ý tưởng cốt lõi là bỏ hẳn phép phân loại nhị phân tĩnh hay động. Thay vào đó, mỗi vật được gán một *khoảng thời gian tồn tại*: nó xuất hiện ở đâu, từ lúc nào tới lúc nào.

Phát biểu đó gộp được hai hiện tượng vốn bị coi là khác loại. Một người đi ngang là một vật có khoảng tồn tại rất ngắn ở mỗi vị trí. Một chiếc ghế bị kê lại là một vật có khoảng tồn tại dài ở vị trí cũ, rồi một khoảng khác ở vị trí mới. Cùng một mô hình, cùng một cơ chế suy luận. Bản đồ kết quả trả lời được câu hỏi “thế giới trông thế nào lúc t ”, và với robot đi lại lâu dài thì đó mới là câu hỏi đúng.

Cái giá thì thẳng thắn. Bạn cần bản đồ đặc, cần lưu lịch sử, và cần đủ nhiều lần đi qua để có bằng chứng. Với một hệ điểm thưa chạy một lần trên một chuỗi, không có gì để lấy về ở đây cả.

F. ConceptGraphs và EFM3D: tập nhãn mở, và một cái thước

ConceptGraphs dựng đồ thị cảnh với tập nhãn mở bằng cách ghép ba mảnh có sẵn: một bộ phân đoạn không phân biệt lớp cho ra vùng, một mô hình thị giác-ngôn ngữ gán đặc trưng cho từng vùng, và một mô hình ngôn ngữ suy ra quan hệ giữa các vật để làm cảnh [353]. Không huấn luyện gì thêm.

Đây là kiến trúc rất hợp thời và cũng rất đắt. Mỗi khung hình phải qua ít nhất hai mô hình nền, và chất lượng đồ thị bị chặn trên bởi chất lượng của bước liên kết vùng qua các khung — đúng cái bước mà không mô hình nền nào lo hộ bạn. Với ngân sách Orin, đây là công cụ ngoại tuyến.

EFM3D không phải một hệ thống mà là một cái thước: một benchmark cho các mô hình nền 3D nhìn từ góc nhìn người đeo thiết bị, với hai tác vụ là phát hiện vật thể 3D và hồi quy bề mặt [354]. Tôi nhắc nó ở đây vì một lý do đã lặp lại nhiều lần trong Phần F: trong một mảng chưa chín, một cái thước tốt thường đẩy tiến bộ đi xa hơn một hệ thống mới. Mảng đồ thị cảnh vẫn đang thiếu thước đo chung, và đó là một phần lý do khó so sánh các công trình trong mảng.

G. Cái giá thật của việc nhận một đồ thị cảnh vào hệ của bạn

Cần nói rất rõ chỗ này, vì nó dễ bị đánh giá thấp. Nhận một đồ thị cảnh không phải là “thêm một mạng”. Nó là đổi loại bản đồ.

Mọi hệ trong mục này đều dựng trên một bản đồ *đặc*: lưới tam giác hoặc lưới voxel, cập nhật liên tục từ độ sâu. ORB-SLAM3 không có thứ đó và không có chỗ để cắm vào. Nhận đồ thị cảnh nghĩa là phải thêm một luồng dựng bản đồ đặc, và luồng đó tiêu bộ nhớ cùng băng thông ở bậc hoàn toàn khác với một đám mây điểm thưa. Trên Orin, đây thường là ràng buộc chặn chứ không phải chi phí mạng nhận thức.

Kèm theo đó là một khoản nợ kỹ thuật ít ai nhắc: mỗi lần tối ưu toàn cục, tất cả các tầng phải được kéo theo cho nhất quán. Bạn đã từng gặp bế tắc giữa luồng lập bản đồ cục bộ và luồng hiệu chỉnh toàn cục. Số tầng phải giữ đồng bộ giờ tăng từ một lên sáu.

33.5 Ngữ nghĩa mở và ngôn ngữ (Open-Vocabulary Semantics and Language)

Nhóm cuối cùng là nhóm ồn ào nhất, và với riêng bạn thì là nhóm ít liên quan nhất. Vì vậy mục này ngắn, và nó tập trung vào một câu hỏi duy nhất: *cái này giải bài toán của ai?*

A. Bốn công trình, một ý tưởng

Ý tưởng chung của cả nhóm là gán một véc-tơ đặc trưng vào từng phần của bản đồ 3D, sao cho véc-tơ đó nằm trong cùng không gian với đặc trưng của chữ. Khi đó truy vấn bằng chữ trở thành phép so cosin, và bạn không cần biết trước danh sách lớp.

- **ConceptFusion** hợp nhất đặc trưng đa phương thức mở vào bản đồ 3D, không cần huấn luyện thêm, và truy vấn được bằng chữ, bằng ảnh hoặc bằng một cú bấm [355]. Đóng góp kỹ thuật đáng chú ý là cách trộn đặc trưng cục bộ của từng vùng với đặc trưng toàn cục của cả ảnh, vì đặc trưng cục bộ đơn thuần mất ngữ cảnh.
- **LERF** nhúng đặc trưng ngôn ngữ vào một trường bức xạ ở nhiều mức tỉ lệ, rồi dựng ảnh chúng như dựng ảnh màu [356]. Kết quả là một trường “mức liên quan” trong không gian

3D. Giới hạn nằm ngay trong cách làm: nó cho vùng chứ không cho ranh giới thực thể, nó cần tối ưu riêng cho từng cảnh, và nó chậm.

- **OpenScene** chứng cất đặc trưng mở từ ảnh 2D xuống một mạng 3D, nên mỗi điểm 3D mang sẵn một đặc trưng đồng không gian với chữ [357]. Ưu điểm so với LERF là không phải tối ưu lại cho từng cảnh.
- **SpatialLM** (2025) đi theo hướng khác: nhận đám mây điểm và sinh ra mô tả cảnh có cấu trúc — tường, cửa, cửa sổ, hộp bao vật thể — ở dạng máy đọc được. Tôi chưa kiểm chứng chi tiết công trình này, nên chỉ ghi nhận ý tưởng: dùng một mô hình ngôn ngữ làm bộ giải mã *bổ cục*, không phải làm bộ trả lời câu hỏi.

B. Cái này giải bài toán của ai

Trả lời thẳng: không phải của bạn, nếu đầu ra của hệ thống bạn là một quỹ đạo.

Không công trình nào trong nhóm này làm APE tốt lên. Chúng không chạm vào bốn nỗi đau của Phần F. Chúng cộng thêm một mô hình nền vào ngân sách vốn đã chật, và cộng thêm một chế độ hỏng mới — một truy vấn chữ trả về sai vật, không có cách nào chẩn đoán và không có ngưỡng nào để chỉnh. Với một hệ đo quỹ đạo, đây là chi phí thuần.

Nhưng câu trả lời đối hẩn khi người tiêu thụ bản đồ đổi. Nếu robot của bạn phải nhận lệnh “ra chỗ máy in lấy tập tài liệu”, thì không có tầng nào trong hệ hiện tại của bạn hiểu được chữ “máy in”. Đám mây điểm không có từ vựng. Đây đúng là tầng còn thiếu, và không có cách nào vòng qua nó bằng hình học. Với robot có nhiệm vụ, nhóm này chuyển từ “chi phí thuần” thành “điều kiện cần”.

Có đúng một đường mà nhóm này có thể quay lại giúp phần hình học, và tôi nêu ra như một giả thuyết chưa được kiểm chứng chứ không phải một kết luận. Đó là kiểm chứng vòng: một chòm sao vật thể có nhãn là bất biến với ánh sáng và với góc nhìn theo cách mà bộ mô tả vẽ ngoài không có. Hydra đã đi được một đoạn theo hướng này bằng cấu trúc phân tầng [349]. Liệu ngữ nghĩa mở có làm tốt hơn ngữ nghĩa tập đóng ở đây không thì tôi chưa thấy bằng chứng nào, và đó là một ablation đáng làm.

Bảng 33.2. Bốn nhóm trong nửa sau chương, đọc theo người tiêu thụ. Cột “ảnh hưởng tới APE” là cột phân loại nhanh nhất: nếu nó bằng không thì công trình đó thuộc về một cuộc trò chuyện khác, dù nó xuất hiện trong cùng một hội nghị và cùng một phiên.

Nhóm	Người tiêu thụ thật	Ảnh hưởng tới APE	Điều kiện để nó đáng làm
Lọc vật động (cột 1 + 2)	Bộ tối ưu tư thế	Có, cả hai chiều	Vật động vượt vùng RANSAC xử lý được; oracle cho thấy trần cải thiện đủ lớn
Bản đồ theo thời gian (cột 3)	Bản đồ sống qua nhiều phiên	Gián tiếp, qua chất lượng định vị lại	Robot đi lại nhiều lần cùng một chỗ, và cảnh thật sự đổi giữa các lần
Đồ thị cảnh phân tầng	Bộ lập kế hoạch; và tầng lấy ứng viên vòng	Gián tiếp, chỉ qua đóng vòng	Đã có sẵn bản đồ đặc; nếu chưa có thì đây là dự án nhiều tháng
Ngữ nghĩa mở, ngôn ngữ	Con người ra lệnh	Bằng không	Nhiệm vụ được phát biểu bằng chữ; không có nhiệm vụ như vậy thì đây là chi phí thuần

33.6 Giới hạn và trường hợp hỏng

Giới hạn nặng nhất đã nêu ở giữa chương, nhưng nó đáng được nhắc lại như một kết luận độc lập: **tầng ngữ nghĩa hỏng tương quan với tầng hình học, chứ không độc lập**. Ảnh mờ, thiếu sáng, chuyển động nhanh, phơi sáng nhảy — đó vừa là những khung hình mà ghép cặp đặc

trung yếu nhất, vừa là những khung hình nằm xa nhất khỏi phân phối huấn luyện của mạng. Một tầng dự phòng chỉ đáng tiền khi nó hỏng độc lập với tầng chính. Ở đây thì không, nên đừng thiết kế hệ thống dựa trên giả định rằng ngữ nghĩa sẽ đỡ cho bạn ở các khung hình khó.

Chế độ hỏng thứ hai là giả định khối cứng. Các hệ theo vết vật thể đều mô hình mỗi vật như một khối cứng có một tư thế. Con người thì không phải khối cứng, và con người lại chính là vật động phổ biến nhất trong mọi tập dữ liệu trong nhà. Hệ quả là các phương pháp đẹp nhất về mặt lý thuyết trong dòng này lại yếu nhất đúng ở trường hợp thường gặp nhất.

Chế độ hỏng thứ ba nằm ở tập nhãn. Một mô hình huấn luyện trên COCO biết tám mươi lớp, và nhà kho của bạn có xe nâng, pallet, tấm chắn, xe đẩy. Tập nhãn mở giải được về nguyên tắc, nhưng chi phí của nó đặt nó ra ngoài vòng lặp, nên trong thực tế bạn quay về đường chưng cất: mô hình lớn gán nhãn ngoại tuyến, mô hình nhỏ chạy trực tuyến. Đường đó có một điểm gãy riêng — mô hình nhỏ kế thừa luôn mọi sai lệch của mô hình lớn, và bạn không còn chân trị nào để phát hiện.

Chế độ hỏng thứ tư là chế độ hỏng của *đánh giá*, và nó âm thầm nhất. Các bảng kết quả trong mảng này phần lớn đo trên chuỗi trong nhà, RGB-D, người chiếm phần rất lớn khung hình, đối chứng là hệ không xử lý gì. Con số cải thiện lớn ở đó không chuyển sang chế độ vận hành của bạn. Muốn biết nó có chuyển hay không, chỉ có một cách: chạy thí nghiệm oracle trên dữ liệu của chính bạn, trước khi viết dòng code nào.

Riêng với đồ thị cảnh, ba giới hạn thêm. Khái niệm “phòng” không tồn tại ngoài trời và rất mơ hồ trong không gian mở, nên tầng phòng mất giá trị đúng ở môi trường mà bạn hay chạy nhất. Chi phí bản đồ đặc là ràng buộc chặn thật trên Orin, không phải chi phí mạng. Và việc giữ sáu tầng nhất quán sau mỗi lần tối ưu toàn cục là một nguồn bế tắc và trạng thái hỏng mà bạn đã nếm ở quy mô nhỏ hơn nhiều.

Cuối cùng, một giới hạn về chính chương này. Phần lớn công trình tôi nhắc sau năm 2023 tôi chỉ đọc chứ chưa chạy. Vì thế chương này cố ý *không* nêu một con số hiệu năng nào: không mili-giây, không FPS, không tỉ lệ cải thiện. Vài phép tính bằng tay còn lại đều có dẫn “giả sử” và chỉ nói về hình dạng của một quan hệ hình học, không nói về hiệu năng của hệ nào. Chỗ nào trước đây cần một con số thì bây giờ là một thứ hạng định tính kèm phép đo bạn phải tự chạy. Thứ hạng dùng để loại phương án; chỉ con số bạn tự đo trên máy mình mới dùng được để lập kế hoạch.

33.7 Thực hành

Repo và tài nguyên. Bắt đầu từ phía nhận thức: [ultralytics/ultralytics](#) cho dòng YOLO và cho đường xuất sang TensorRT, [open-mmlab/mmdetection](#) khi cần so nhiều họ detector trong cùng một khung, [facebookresearch/segment-anything](#) và [facebookresearch/sam2](#) để thử phân đoạn theo prompt, [IDEA-Research/GroundingDINO](#) khi cần một bộ gán nhãn tập mở chạy ngoại tuyến. Phía SLAM động: [BertaBescos/DynaSLAM](#) là bản tham chiếu ngắn gọn nhất để đọc, vì nó cắm thẳng vào ORB-SLAM2 nên bạn thấy rõ đúng chỗ mặt nạ can thiệp. Phía đồ thị cảnh: [MIT-SPARK/Kimera-VIO](#), [MIT-SPARK/Hydra](#) và [MIT-SPARK/Khronos](#) là ba bậc thang của cùng một dòng, đọc theo thứ tự đó thì thấy rõ mỗi bậc thêm gì. Phía tập nhãn mở: [concept-graphs/concept-graphs](#). Cuối cùng vẫn là [evo](#) để mọi con số bạn báo ra so được với nhau. Danh sách này chốt tại tháng 9/2026; tên mô hình dẫn đầu sẽ cũ trong sáu tới mười hai tháng, còn hạ tầng như TensorRT, [evo](#) và các thư viện dựng lưới thì bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Trên chuỗi khó nhất của bạn, vật động chiếm bao nhiêu phần khung hình ở phân vị 95, và bạn đo con số đó bằng cách nào? Nếu mặt nạ của bạn xóa một phần lớn số đặc trưng, chính sách chèn khung hình chính có đổi không, và bạn kiểm điều đó ra sao? Một vật đi thẳng ra xa camera vi phạm ràng buộc hình học nào — và nếu câu trả lời là “không

cái nào”, thì cái gì phát hiện được nó? Đồ thị cảnh của Hydra cần thứ gì mà bản đồ điểm thưa của bạn không có, và thêm thứ đó tốn bao nhiêu bộ nhớ trên Orin?

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Mặt nạ ngữ nghĩa	Ảnh nhị phân đánh dấu các điểm ảnh thuộc lớp có thể di chuyển. Nó là tri thức tiên nghiệm về <i>lớp</i> , không phải phép đo về thực thể trước mặt, nên một mình nó không kết luận được vật đang động.
●●● Ba việc khác nhau khi loại vật động	Vật <i>có thể</i> động (ngữ nghĩa nói), vật <i>đang</i> động (hình học nói), vật <i>đã</i> động rồi đứng yên (chỉ trí nhớ của bản đồ nói). Không việc nào suy ra được việc nào, và mỗi việc có một kiểu hỏng riêng.
●● Vòng luẩn quẩn tư thế–vật động	Muốn tính phần dư để phát hiện vật động thì cần tư thế camera, mà tư thế lại đang ước lượng từ chính tập điểm chứa vật động. Khi vật động chiếm phần lớn khung hình, kết luận đảo ngược hoàn toàn.
●●● Điều kiện của bài toán	Mức độ trải đủ hướng của tập ràng buộc còn lại. Xóa một mảng liên khối làm điểm dồn về một phía và tư thế có một hướng gần như không bị ràng buộc; không cờ ngoại lai nào bật lên, nên đây là kiểu hỏng im lặng.
●●● Hạ trọng số thay vì xóa	Nhân độ tin cậy của mặt nạ vào ma trận thông tin của phép đo, thay cho một quyết định nhị phân đặt ở đầu đường ống. Điểm nghi ngờ vẫn góp vào điều kiện của bài toán, và hàm mất mát bền vững vẫn đập được nó khi có bằng chứng thật.
●●● Thí nghiệm oracle cho mặt nạ	Dựng mặt nạ hoàn hảo bằng tay hoặc bằng mô hình nặng chạy ngoại tuyến, rồi đo cải thiện. Nó chặn trên mọi công sức làm mặt nạ thời gian thực; nếu trần đó nằm dưới sàn nhiều thì dừng ngay.
●●● DynaSLAM	Ghép mặt nạ phân đoạn thực thể cho các lớp có thể động với một kiểm tra sai lệch độ sâu đa khung, rồi vá lại nền. Bản tham chiếu gọn nhất của việc dùng cả hai nguồn bằng chứng; giá phải trả là một lượt mạng nặng mỗi khung. Họ hàng gần: DS-SLAM chạy mạng trên luồng riêng, DynaSLAM II đưa tư thế vật vào hiệu chỉnh chùm tia.
●●● SAM	Phân đoạn theo prompt: điểm, khung hoặc mặt nạ thô. Bộ mã hoá ảnh nặng chạy một lần mỗi ảnh, bộ giải mã nhẹ chạy một lần mỗi prompt. Nó <i>không</i> trả về nhãn lớp, nên một mình nó không lọc được vật động.
●● Grounding DINO	Phát hiện vật theo mô tả bằng chữ, nên gỡ được ràng buộc tập nhãn đóng. Quá nặng cho vòng lặp trên Orin; vai trò đúng là bộ gán nhãn ngoại tuyến để chưng cất xuống một detector nhỏ. Open-YOLO 3D làm cùng tinh thần tiết kiệm ở tầng 3D bằng cách thay mặt nạ nặng bằng hộp.
●●● Đồ thị cảnh 3D	Bản đồ nhiều tầng: hình học, lưới có nhãn, vật thể và tác nhân, nơi chốn, phòng, toà nhà. Tốc độ thay đổi giảm dần khi lên cao, nên tầng trên dùng được làm mỏ neo. Kimera là thư viện mở dựng tầng lưới có nhãn, với các mô-đun thay được từng cái.
●● Hydra	Dựng đồ thị cảnh tăng dần theo thời gian thực. Việc khó nhất không phải nhận dạng mà là biến dạng cả sáu tầng cho nhất quán mỗi khi một vòng đóng lại. Nó cũng phát hiện vòng bằng cấu trúc phân tầng thay vì bằng vẽ ngoài, nên có chế độ hỏng khác.
●● Clio	Mức chi tiết của bản đồ được quyết định bởi <i>danh sách nhiệm vụ</i> , qua nút thắt thông tin. Một chồng sách là một vật hay mười vật tùy nhiệm vụ; “vật thể” không phải thuộc tính của cảnh.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Khronos	Bỏ phân loại tĩnh–động, thay bằng ước lượng khoảng thời gian tồn tại của từng vật. Nhờ đó chuyển động ngắn hạn và thay đổi dài hạn dùng chung một cơ chế, và bản đồ trả lời được “lúc t thế giới trông thế nào”.
●● Ngữ nghĩa mở	ConceptFusion, LERF và OpenScene gắn vào bản đồ 3D các đặc trưng nằm cùng không gian với chữ, nên truy vấn được bằng ngôn ngữ; ConceptGraphs ghép chúng thành đồ thị. Không cái nào làm APE tốt lên: chúng phục vụ người ra lệnh, không phục vụ bộ ước lượng tư thế.
●●● Vì sao RANSAC không đủ?	RANSAC hỏng khi ngoại lai đông và nhất quán, vì khi đó chúng thắng phiếu đồng thuận. Vật động tạo sai lệch <i>có hướng</i> chứ không phải nhiễu ngẫu nhiên, nên cả cụm cùng bỏ phiếu cho một nghiệm sai. Đó là vùng duy nhất mặt nạ thật sự đáng tiền.
●●● Vì sao mặt nạ ngữ nghĩa một mình không đủ?	Sáu lý do: nhãn nói về lớp chứ không về thực thể; có vật động ngoài tập nhãn; có chuyển động gián tiếp như bóng đổ và phản chiếu; biên mặt nạ lệch đúng chỗ góc ORB nằm; mặt nạ luôn cũ vài khung; và mạng hỏng đúng ở khung hình khó.
●● Vì sao bỏ triết phi cực đại lại quan trọng với hệ thời gian thực?	Vì thời gian hậu xử lý tỉ lệ với số khung sống sót, nên đuôi độ trễ dày lên đúng ở cảnh đông người. YOLOv10 bỏ nó bằng hai phép gán nhất quán, RT-DETR bỏ nó bằng dự đoán tập hợp; cái được là độ trễ hằng số, không phải mAP.
●●● Vì sao mạng hỏng đúng lúc cần nó nhất?	Ảnh mờ, thiếu sáng và chuyển động nhanh vừa làm hình học yếu, vừa nằm ngoài phân phối huấn luyện của mạng. Hai kiểu hỏng tương quan dương, nên tầng ngữ nghĩa không dùng được làm lưới an toàn.
●● APE đẹp lên nhưng chuỗi hay mất bám — nghĩa là gì?	Gần như chắc chắn là xoá quá tay: APE chỉ tính trên đoạn còn sống nên hệ mất bám sớm lại có APE đẹp. Luôn báo tỉ lệ hoàn thành cùng APE, và so cả hai với sản phẩm đo từ nhiều lần chạy.
●● Bật mặt nạ thì số khung hình chính giảm hẳn — nghĩa là gì?	Mặt nạ đã đổi luôn chính sách chèn khung hình chính, nên bạn đang so hai đồ thị khác nhau chứ không phải hai bộ phép đo. Cách phân biệt: khoá chính sách khung hình chính lại rồi chạy lại, hoặc báo cáo số khung hình chính kèm APE.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
BertaBescos/DynaSLAM [218]	Ghép cột 1 với cột 2: mặt nạ ngữ nghĩa cho “có thể động”, kiểm tra độ sâu đa khung cho “đang động”	Bản tham chiếu ngắn nhất để đọc; cắm thẳng vào ORB-SLAM2
MIT-SPARK/Hydra [349]	Dựng đồ thị cảnh phân tầng theo thời gian thực và biến dạng cả sáu tầng khi vòng đóng	Đang được bảo trì; là nền cho Clio và Khronos
Khronos [347]	Thay phân loại tĩnh–động bằng khoảng thời gian tồn tại của từng vật — lời giải cho cột 3	Cần bản đồ đặc và cần nhiều lần đi qua
Clio [352]	Mức chi tiết của bản đồ suy ra từ danh sách nhiệm vụ, qua nút thất thông tin	Bài học thiết kế rộng hơn phạm vi đồ thị cảnh
ultralytics	Dòng detector nhẹ chạy được trong vòng lặp; đường xuất sang TensorRT là phần dùng nhiều nhất	Hạ tầng; bền hơn tên phiên bản mô hình
Dòng SAM [62], [341]	Cấu trúc chi phí “mã hoá một lần, giải mã theo prompt” — mẫu đặt phần đất ở chỗ được khẩu hao	Chủ yếu dùng ngoại tuyến để gán nhãn dữ liệu

MINI PROJECT SLAM · Đo trần của việc lọc vật động trên dữ liệu của bạn · 1–2 ngày

Mục tiêu	trả lời câu hỏi “có đáng làm không” bằng một con số, <i>trước</i> khi bỏ công làm mặt nạ chạy thời gian thực. Đây là thí nghiệm oracle của chương 24, áp cho tầng ngữ nghĩa.
Dữ liệu	một chuỗi có người đi lại thật — OpenLORIS, hoặc một chuỗi bạn tự quay ở hành lang có người qua lại. Kèm một chuỗi EuRoC làm đối chứng âm, vì EuRoC gần như không có vật động và kết quả ở đó phải là “không đối gì”.
Các bước	trước hết đo phân bố diện tích vật động: chạy một mô hình phân đoạn nặng ngoại tuyến, ghi tỉ lệ điểm ảnh mang nhãn động cho từng khung, lấy phân vị 95. Sau đó dựng <i>mặt nạ oracle</i> bằng chính mô hình nặng đó chạy ngoại tuyến, nở biên vài điểm ảnh, lưu ra đĩa. Chạy ba cấu hình, mỗi cấu hình mười lần: không mặt nạ, mặt nạ oracle dạng xoá cứng, mặt nạ oracle dạng hạ trọng số. Với mỗi lần chạy ghi ba số chứ không phải một: APE, tỉ lệ hoàn thành, và số khung hình chính.
Xong khi	bạn có một bảng ba cấu hình nhân ba số, kèm độ lệch chuẩn qua mười lần chạy, và bạn phát biểu được một trong hai câu: “mặt nạ hoàn hảo cải thiện APE thêm X cm, vượt sàn nhiều, nên bản thời gian thực đáng làm”, hoặc “mặt nạ hoàn hảo không vượt sàn nhiều trên chuỗi này, nên dừng”. Kiểm tra chéo bắt buộc: trên EuRoC cả ba cấu hình phải trùng nhau trong sàn nhiều; nếu không, đường ống mặt nạ của bạn đang đổi một thứ khác ngoài mặt nạ.
Nối với	E/24 cho thí nghiệm oracle và cỡ mẫu; B/07 cho cách đọc cặp APE và tỉ lệ hoàn thành; kết quả của dự án này là điều kiện vào của dự án thứ hai bên dưới.

MINI PROJECT SLAM · Ngân sách thật của mạng nhận thức trên Orin · 1 ngày

Mục tiêu	biến con số FPS trong paper thành con số dùng được, bằng cách đo trong đúng điều kiện mà nó sẽ chạy: cùng máy, cùng lúc, cùng tranh chấp tài nguyên.
Dữ liệu	một chuỗi bất kỳ phát lại ở tốc độ thật trên Jetson Orin, cộng một detector nhẹ đã xuất sang TensorRT ở nửa độ chính xác.
Các bước	đo ba lần. Lần một: chỉ chạy mạng, ghi trung bình và p99 của độ trễ. Lần hai: chỉ chạy SLAM, ghi trung bình và p99 của thời gian mỗi khung, cùng tỉ lệ khung bị bỏ. Lần ba: chạy cả hai cùng lúc, ghi lại đúng bốn con số đó. Sau đó lặp lần ba với mạng đặt ở tần số một phần ba nhịp khung, chỉ trên khung hình chính.
Xong khi	bạn có một bảng bốn dòng cho thấy p99 của cả hai bên tăng bao nhiêu khi chạy chung, và bạn nói được tần số cao nhất mà mạng chạy được mà không đẩy p99 của luồng theo vết vượt ngân sách khung. Nếu p99 chạy chung gần bằng tổng hai p99 riêng, bạn vừa xác nhận rằng “song song” ở đây là chạy xen kẽ.
Nối với	D/21-inference-va-trien-khai cho phần xuất mô hình và lượng tử hoá; B/11 cho cách đọc con số theo mô hình roofline; F/34 cho phần đo lại trung thực trên phần cứng thật.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì bộ tối ưu không cần nhiều ràng buộc mà cần ràng buộc trái đủ hướng. Mặt nạ luôn liên khối, nên xoá nó làm các điểm còn lại dồn về một phía và tư thế có một hướng gần như không bị ràng buộc. Kiểu hỏng này im lặng: phần dư còn lại nhỏ và đẹp, hiệp phương sai báo ra thì lạc quan, và sai số chỉ hiện ra vài giây sau dưới dạng trôi.
- A2. Đúng một loại: ngoại lai *đồng và nhất quán*. RANSAC chọn nghiệm được nhiều phiếu nhất, nên khi vật động chiếm phần lớn khung hình và cùng chuyển động theo một hướng, chính chúng thắng phiếu và nền tĩnh bị coi là ngoại lai. Ngữ nghĩa phá thế bế tắc bằng một lá phiếu đến từ ngoài hình học. Ngoài vùng đó, nó chủ yếu lấy đi dữ liệu tốt.
- A3. Không trùng hợp, và đó là kết luận thiết kế quan trọng nhất của chương. Ảnh mờ, thiếu sáng và chuyển động nhanh vừa là các khung hình mà ghép cặp yếu nhất, vừa là các khung hình xa nhất khỏi phân phối huấn luyện của mạng. Một tầng dự phòng chỉ đáng tiền khi nó hỏng độc lập với tầng chính; ở đây hai kiểu hỏng tương quan dương.

- A4. Cái thứ nhất trả lời “lớp này có khả năng tự di chuyển không” và trả lời được ngay từ khung hình đầu, nhưng nó nói về lớp chứ không về thực thể. Cái thứ hai trả lời “vật này đang động ở khung hình này không” nhưng cần một tư thế đủ tốt, tức là mắc vòng luẩn quẩn. Cái thứ ba trả lời “chỗ này đã đổi so với lần trước chưa” và chỉ trả lời được sau nhiều lần đi qua; nó là bài toán bảo trì bản đồ, không phải bài toán nhận thức.
- A5. Nó không giúp gì cả, và câu trả lời trung thực đó chính là công cụ phân loại. Nếu người tiêu thụ bản đồ là một script tính APE, tầng ngôn ngữ là chi phí thuần cộng thêm một chế độ hồng không chẩn đoán được. Nếu người tiêu thụ là một bộ lập kế hoạch nhận lệnh bằng chữ, thì đám mây điểm không có từ vựng và đây đúng là tầng còn thiếu. Hãy hỏi “ai đọc bản đồ này” trước khi hỏi “phương pháp nào tốt hơn”.

NÓI LẠI THẬT ĐƠN GIẢN

Tưởng tượng bạn vẽ lại sơ đồ một siêu thị bằng cách đi vòng quanh và ghi nhớ đồ vật ở đâu. Cách này chạy tốt, cho tới khi bạn ghi nhớ nhầm một hộp ngũ cốc nằm trong xe đẩy của người khác. Hôm sau bạn quay lại, hộp ngũ cốc đã đi mất. Nhưng bạn thì tin nó vẫn ở chỗ cũ, nên bạn kết luận rằng *chính bạn* mới là người đang đứng sai chỗ. Toàn bộ sơ đồ trượt theo.

Mẹo là dạy cho máy biết loại đồ nào hay nằm trong xe đẩy, rồi bảo nó bớt tin những thứ đó. Mẹo này ăn tiền vì máy học được từ hàng triệu bức ảnh, còn bản thân phép đo thì không thể tự biết điều đó.

Cái giá thì có ba phần, và cả ba đều thật. Thứ nhất, máy chỉ biết *loại* đồ, không biết cái hộp trước mặt nó lúc này đang nằm trên kệ hay trong xe. Thứ hai, nếu nó gạch bỏ mọi thứ có thể nằm trong xe đẩy, nó có thể gạch nguyên một dãy kệ, và rồi cả một phía của sơ đồ không còn mốc nào để dựa. Thứ ba, nhìn ra đồ vật tốn thời gian, mà mỗi bước chân bạn chỉ có một khoảnh khắc rất ngắn.

Chỗ không rút gọn được: vì sao gạch bỏ dữ liệu *đúng* lại làm sơ đồ tệ đi. Mọi cách nói gọn đều thành “cần mốc rải đều”, nghe như một sự tiện lợi. Sự thật là một phát biểu về hình dạng của một ma trận, và tôi chưa tìm được cách nói ngắn nào vừa đúng vừa không cần tới ma trận.

Một câu: *Máy chỉ biết loại đồ nào hay bị xô dịch, chứ không biết cái trước mặt nó lúc này có xô dịch hay không — nên đừng bảo nó xoá, hãy bảo nó bớt tin.*

33.8 Kết nối

- **C/14-bai-toan-cv-loi — tiền đề.** Chương 14 dạy detection và segmentation là gì: gán mẫu, triệt phi cực đại, dự đoán tập hợp, phân loại mặt nạ. Chương này chỉ dùng chúng, và chỗ nổi là câu hỏi “mô hình nào bỏ vừa vào ngân sách khung hình”.
- **F/28-dac-trung-va-ghep-cap — cùng nguyên lý với.** Mặt nạ ngữ nghĩa và bộ mô tả học được can thiệp vào đúng cùng một chỗ: quyết định của frontend về việc tin phép đo nào. Cái thứ nhất đổi *trọng số*, cái thứ hai đổi *chất lượng*. Hai đòn bẩy độc lập, và nên được đo tách rời nhau.
- **F/29-dinh-vi-lai-va-dong-vong — hệ quả của.** Vòng sai vì hai chỗ chia nhau cùng một chiếc xe đẩy là một chế độ hồng của tầng lấy ứng viên. Đồ thị cảnh phân tầng đề xuất một tầng lấy ứng viên khác, có chế độ hồng khác — và “chế độ hồng khác” mới là thứ đáng giá khi ghép hai bộ phát hiện lại.
- **F/30-do-sau-dong-quang-va-mo-hinh-nen — tiền đề.** Cột 2 của Hình 33.3 chạy bằng dòng quang và độ sâu. Chất lượng phát hiện “đang động” bị chặn trên bởi chất lượng của hai thứ đó, nên hai chương phải đọc cùng nhau.
- **F/32-ban-do-neural — đối lập với.** Cả hai chương đều thay bản đồ điểm thừa bằng thứ khác. Bản đồ neural đi về phía *đặc và liên tục*, tối ưu cho việc dựng lại ảnh. Đồ thị cảnh đi về phía *thừa và có ký hiệu*, tối ưu cho việc suy luận. Hai hướng ngược nhau, và chúng phục vụ hai người tiêu thụ khác nhau.

- **F/34-imu-bat-dinh-va-trien-khai** — cùng nguyên lý với. Kết luận “hạ trọng số thay vì xoá” là một trường hợp riêng của luận điểm chính chương 34: mọi quyết định tin cậy trong đồ thị nhân tử cuối cùng đều là một con số trong ma trận thông tin. Chương 34 cũng cho một nguồn bằng chứng mà chương này không dùng tới: IMU biết camera đã chuyển động thế nào, nên nó giúp tách chuyển động của vật khỏi chuyển động của người quan sát.
- **B/08-dich-chuyen-phan-phoi** — trường hợp riêng của. Việc mạng hồng đúng ở khung hình khó là dịch chuyển phân phối ở dạng độc nhất, vì hướng dịch chuyển trùng với hướng làm hình học yếu đi.
- **E/24-thi-nghiem-va-ablation** — tiền đề. Thí nghiệm oracle và quy tắc đo sản nhiều trước đều lấy nguyên từ chương 24. Không có hai công cụ đó, mọi kết luận trong chương này đều không kiểm chứng được.
- **B/11-gioi-han-tinh-toan** — ràng buộc bởi. Bảng 33.1 chỉ là chương 11 áp cho một loại phần cứng cụ thể; điều đáng mang đi là thói quen bắt đầu từ ngân sách chứ không từ bảng độ chính xác.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một vật động tạo ra sai lệch *có hướng* chứ không phải nhiễu ngẫu nhiên, và điều đó thay đổi cách bạn chọn hàm mất mát bền vững ra sao?
2. Nếu mặt nạ của bạn nở thêm vài điểm ảnh ở biên, bạn được gì và mất gì — và bạn đo hai thứ đó bằng phép đo nào?
3. Một chuỗi cho APE tốt lên rõ rệt khi bật mặt nạ, nhưng số khung hình chính cũng giảm hẳn. Bạn kết luận được gì, và thí nghiệm nào tách được hai hiệu ứng đó?
4. Vì sao thí nghiệm oracle phải chạy *trước* khi bạn viết dòng code nào cho mặt nạ thời gian thực, và điều gì xảy ra nếu bạn đảo thứ tự?
5. Đồ thị cảnh phân tầng phát hiện vòng bằng cấu trúc thay vì bằng vẻ ngoài. Chế độ hồng của nó khác chế độ hồng của bộ mô tả ảnh toàn cục ở đâu, và vì sao sự khác biệt đó mới là lý do đáng ghép hai bộ lại?
6. Nếu bạn phải chọn giữa một detector nhanh hơn hai lần và một detector có độ trễ ổn định hơn hai lần, bạn chọn cái nào cho vòng lặp theo vết, và lý do nằm ở đại lượng nào?
7. Vì sao “bản đồ theo thời gian” không phải là một bài toán nhận thức, và việc ném thêm một mạng phân đoạn vào nó hồng ở bước nào?

IMU, bất định học được và đưa lên phần cứng (IMU, Learned Uncertainty and Deployment)

PHỤ THUỘC F/27-dat-hoc-sau-vao-dau (khung bốn cổng — chương này giả định bạn đã có bộ lọc đó); F/28-dac-trung-va-ghep-cap (ghép cặp học được, nguồn của bất định theo từng điểm khớp); D/21-inference-va-trien-khai và D/20-toi-uu-va-nen-mo-hinh (TensorRT, lượng tử hoá — chương này không nhắc lại); E/24-thi-nghiem-va-ablation (sản nhiều, cỡ mẫu).

TÓM TẮT

Chương này gom hai nỗi đau còn lại của một hệ stereo-inertial đã tinh chỉnh kỹ: đặt trọng số cho IMU, và ảnh mờ do chuyển động. Cả hai thật ra là một bệnh, nhìn từ hai phía. Hệ cổ điển ước lượng *trạng thái* rất tốt và ước lượng *độ tin cậy của chính nó* rất tệ. Đọc xong bạn kiểm được bằng số ma trận thông tin hiện tại của mình có trung thực không, biết mỗi công trình đóng góp cái gì thật, và biết vì sao khứ mờ trước khi trích đặc trưng thường không giúp. Nếu chỉ nhớ một điều: cái mà học sâu mang lại ở đây không phải một ước lượng tư thế tốt hơn, mà là một ước lượng *độ tin cậy* tốt hơn — và đó đúng là thứ hệ cổ điển đang thiếu.

KIẾN THỨC CẦN CÓ TRƯỚC

- **ma trận thông tin (information matrix)** — nghịch đảo của ma trận hiệp phương sai, ký hiệu $\Omega = \Sigma^{-1}$. Trong một đồ thị nhân tử, nó chính là *trọng số* của một ràng buộc: càng tin phép đo thì Ω càng lớn và ràng buộc kéo nghiệm càng mạnh.
- **độ lệch cảm biến (sensor bias)** — sai số cộng thêm, thay đổi chậm, của con quay hồi chuyển và gia tốc kế. Trong tài liệu này “độ lệch” luôn mang nghĩa đó. Nó khác hẳn hai nghĩa còn lại của *bias*: độ lệch hệ thống của một bộ ước lượng thống kê, và tham số cộng b trong một tầng tuyến tính.
- **tiền tích phân IMU (IMU preintegration)** — cách gộp hàng trăm mẫu IMU giữa hai khung hình thành một ràng buộc duy nhất về xoay, vận tốc và vị trí, sao cho khi tư thế đổi thì không phải tích phân lại từ đầu [216]. Nó đi kèm một ma trận hiệp phương sai, và ma trận đó chính là “trọng số IMU” mà cả chương bàn tới.
- **phương sai Allan (Allan variance)** — một cách đo đặc tính nhiễu của cảm biến quán tính: ghi dữ liệu lúc đứng yên trong nhiều giờ rồi vẽ độ lệch chuẩn của trung bình theo độ dài cửa sổ trung bình. Nó tách được nhiễu trắng, độ trôi của độ lệch, và bước đi ngẫu nhiên [358].
- **NEES** — viết tắt của *normalized estimation error squared*. Với phần dư r và hiệp phương sai Σ mà bạn khai báo, đại lượng $r^T \Sigma^{-1} r$ phải có trung bình đúng bằng số bậc tự do nếu Σ trung thực. Đây là phép thử chính của cả chương.
- **bất định thay đổi theo đầu vào** — tiếng Anh gọi là *heteroscedastic*. Nghĩa là mức nhiễu không phải một hằng số mà phụ thuộc chính đầu vào: một điểm ảnh trên vùng mờ có sai số lớn hơn hẳn một điểm trên vùng sắc nét, dù cùng một camera.
- **thời gian phơi sáng (exposure time)** — khoảng thời gian cảm biến ảnh gom ánh sáng cho một khung hình, ký hiệu τ . Ảnh ghi được là trung bình của cảnh trong suốt khoảng đó, nên mọi chuyển động trong τ đều biến thành vết mờ.

- **độ trễ đuôi (tail latency)** — phân vị cao của phân bố độ trễ, thường là p99. Nó khác trung bình ở chỗ nó mô tả những lần chậm nhất — và trong hệ thời gian thực thì chính những lần đó phá hạn chết.

CÂU HỎI MỞ ĐẦU

- Q1. Tin IMU quá mức và tin ảnh quá mức đều làm hỏng quỹ đạo. Vì sao hai kiểu hỏng đó nhìn hoàn toàn khác nhau trên đồ thị lỗi, và bạn tách chúng bằng phép đo nào?
- Q2. Con số nhiều con quay trên datasheet là một phép đo thật, do nhà sản xuất làm. Vậy vì sao đặt đúng con số đó vào đồ thị nhân tử lại gần như luôn sai?
- Q3. Nếu một mạng nơ-ron không cho tư thế tốt hơn hình học, nó còn cho được cái gì đáng để bạn trả một phần ngân sách khung hình trên Orin?
- Q4. Khử mờ bằng một mạng rồi mới trích ORB nghe hiển nhiên là đúng. Vì sao nó thường *không* làm tăng số điểm khớp đúng?
- Q5. Bạn đo riêng mạng mới và thấy nó chỉ chiếm một phần nhỏ chu kỳ khung hình. Vì sao phép đo đó vẫn chưa cho phép kết luận là nó vừa ngân sách?

34.1 Đặt vấn đề

Bảy chương trước của Phần F đi theo các khối của hệ thống: đặc trưng, đóng vòng, độ sâu, back-end, bản đồ, ngữ nghĩa. Chương này đi theo hai *triệu chứng* còn lại. Cả hai đều không nằm gọn trong một khối nào.

Triệu chứng thứ nhất: bạn không biết đặt trọng số IMU bằng bao nhiêu. Nói ra thì hệ rung khi ảnh xấu. Siết vào thì hệ trôi theo độ lệch con quay. Bạn đã thử vài giá trị, thấy mỗi giá trị thắng ở một chuỗi khác nhau, và không có lý do nguyên tắc nào để chọn. Triệu chứng thứ hai: những đoạn ảnh mờ làm mất bám, và mọi cách chữa ảnh nhĩ ra đều có mùi vá vú.

Hai triệu chứng này trông không liên quan. Chúng là một bệnh. Trong cả hai, hệ thống đang dùng một con số về độ tin cậy mà nó không hề đo. Trọng số IMU là hằng số lấy từ datasheet. Trọng số của một điểm ảnh là σ^2 nhân theo tăng kim tự tháp của điểm đó, tức một hàm của *độ phân giải*, không phải của *chất lượng*. Không con số nào trong hai con số ấy biết ảnh có mờ không, robot có rung không, hay camera vừa quay nhanh cỡ nào.

Học xong chương này thì làm được gì mà trước đó không làm được? Bạn đo được ma trận thông tin hiện tại của mình sai bao nhiêu lần. Phép thử có sẵn trong lý thuyết ước lượng và chạy được trong một buổi chiều. Từ con số đó bạn mới quyết định có cần một mạng hay không. Và phần lớn trường hợp, câu trả lời trung thực là chưa cần: cần sửa lại hằng số theo một mô hình. Đó là lý do chương này đặt phép đo trước phương pháp.

TRỰC GIÁC

Một đồ thị nhân tử là một mô hình lò xo. Mỗi ràng buộc là một lò xo nối hai nút, và ma trận thông tin Ω chính là *độ cứng* của lò xo đó. Nghiệm tối ưu là vị trí cân bằng của cả hệ lò xo.

Hình ảnh này giải thích ngay ba chuyện. Một, chỉ *tỉ lệ* độ cứng mới quan trọng: nhân đôi mọi lò xo thì cân bằng không đổi. Hai, một lò xo khai quá cứng sẽ kéo cả hệ về phía nó, kể cả khi nó gắn sai chỗ. Đó chính là một phép đo mờ được tin tưởng quá mức. Ba, độ cứng là một *ma trận* chứ không phải một số: một lò xo có thể rất cứng theo phương ngang và rất mềm theo phương dọc. Chúng ta sẽ thấy vết mờ tạo ra đúng loại lò xo dị hướng đó.

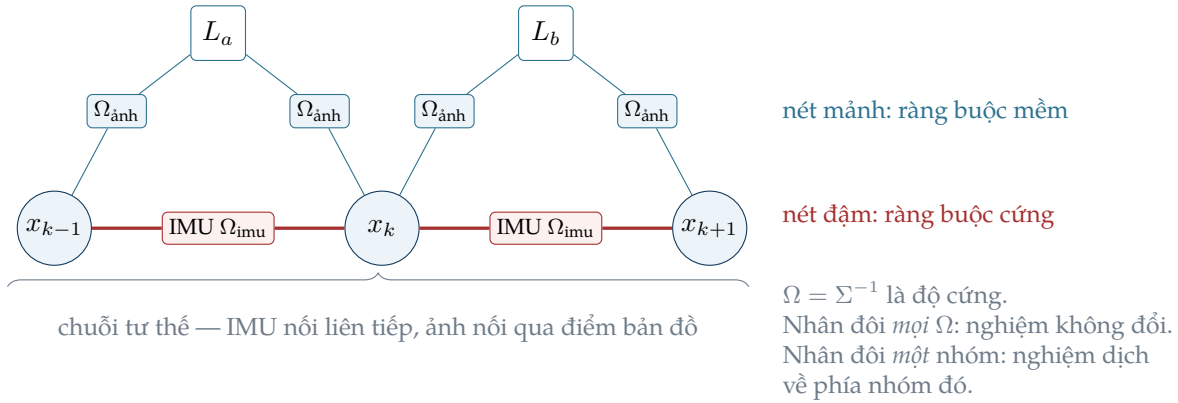
34.2 Trọng số chính là ma trận thông tin (Weights Are the Information Matrix)

A. Trọng số không phải siêu tham số, nó là đại lượng vật lý

Trong ước lượng bình phương tối thiểu phi tuyến, mỗi ràng buộc đóng góp một số hạng $r^T \Sigma^{-1} r$ vào hàm mục tiêu. Ma trận $\Omega = \Sigma^{-1}$ không phải một nôm vậ tùy ý. Nó là hiệp phương sai nghịch đảo của phép đo, và nó có nghĩa xác suất: bạn đang khai báo phép đo này sai cỡ bao nhiêu [26].

Đây là chỗ phân biệt quan trọng nhất của cả chương. Nó cũng khớp với nguyên tắc bạn đã tự đặt ra là từ chối các phương pháp kiểu tinh chỉnh. Đối Σ bằng tay cho tới khi APE đẹp là tinh chỉnh. Suy Σ từ một mô hình sai số rồi kiểm lại bằng số là ước lượng. Hai việc có thể cho cùng một con số, nhưng chúng khác nhau hoàn toàn về chỗ sẽ hỏng. Con số tinh chỉnh chỉ đúng trên tập bạn đã tinh chỉnh. Con số suy từ mô hình đúng ở mọi chỗ mô hình còn đúng, và bạn biết mô hình đó giả định gì.

Hình 34.1 cho thấy chỗ mỗi ma trận thông tin nằm trong đồ thị, và vì sao chỉ tỉ lệ giữa chúng mới có tác dụng.



Hình 34.1. Ma trận thông tin nằm ở đâu trong một đồ thị nhân tử stereo-inertial. Mỗi cạnh IMU mang Ω_{imu} , mỗi phép chiếu mang $\Omega_{\text{ảnh}}$. Vì nghiệm là cân bằng của cả hệ, chỉ *tỉ số* giữa hai nhóm độ cứng mới đổi kết quả. Hệ quả thực dụng: “siết trọng số IMU” và “nới trọng số ảnh” là cùng một thao tác, nên không sửa được cái này mà không nghĩ về cái kia.

B. Ví dụ nhỏ nhất tính tay được

Lấy bài toán một chiều. Hai phép đo cùng một đại lượng x . IMU nói $x = 1,0$ với $\sigma_{\text{imu}} = 0,1$. Ảnh nói $x = 0,0$ với $\sigma_{\text{ảnh}} = 1,0$. Nghiệm hợp nhất là trung bình có trọng số theo nghịch đảo phương sai:

$$\hat{x} = \frac{\Omega_{\text{imu}} \cdot 1,0 + \Omega_{\text{ảnh}} \cdot 0,0}{\Omega_{\text{imu}} + \Omega_{\text{ảnh}}} = \frac{100 \cdot 1 + 1 \cdot 0}{100 + 1} = 0,990.$$

Ảnh chỉ kéo được nghiệm đi 1%. Bây giờ giả sử σ_{imu} thật là 1,0 chứ không phải 0,1; datasheet đã lạc quan mười lần. Nghiệm đúng khi đó là 0,5. Sai số do khai nhầm trọng số là 0,49, tức gần bằng toàn bộ khoảng cách giữa hai phép đo.

Đây là con số đáng nhớ nhất của mục này. Sai một bậc về σ không làm nghiệm lệch một chút. Nó làm nghiệm bỏ gần hết một trong hai nguồn thông tin. Vì Ω đi với bình phương của σ , sai mười lần về độ lệch chuẩn là sai một trăm lần về trọng số.

C. Hai kiểu hỏng đối xứng

Đặt sai trọng số hỏng theo đúng hai kiểu, và chúng đối xứng nhau.

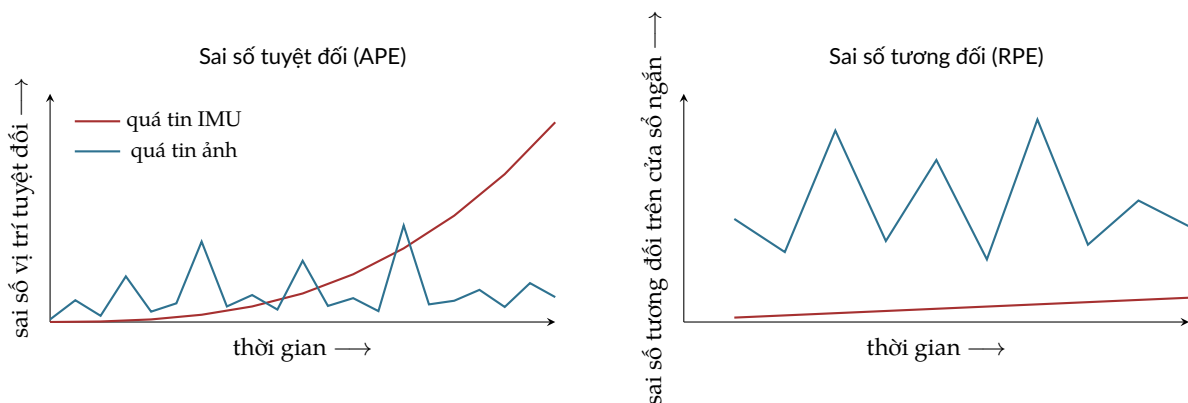
Quá tin IMU nghĩa là Σ_{imu} khai quá nhỏ. Bộ tối ưu không còn được phép sửa phép đo quán tính. Nó hấp thụ mọi mâu thuẫn vào chỗ khác: vào vị trí điểm bản đồ, vào tỉ lệ, vào độ lệch cảm biến. Cơ chế trôi thì thuần vật lý. Một độ lệch con quay ϵ tạo sai số góc ϵt . Sai số góc làm vectơ trọng lực bị nghiêng, nên một phần của g rò vào gia tốc ngang, cỡ $g\epsilon t$. Tích phân hai lần cho sai số vị trí

$$\Delta p(t) \approx \frac{1}{6} g \epsilon t^3.$$

Giả sử $\epsilon = 0,01$ rad/s và $t = 10$ s — một phép tính giả định để thấy *bậc* của công thức, không phải số đo của hệ nào: $\frac{1}{6} \cdot 9,81 \cdot 0,01 \cdot 1000 \approx 16$ m. Trong một hệ có ảnh, thị giác chặn phần lớn mức trôi đó. Nhưng *hình dạng* vẫn giữ nguyên: mượt, đơn điệu, tăng theo lũy thừa của thời gian.

Quá tin ảnh nghĩa là $\Sigma_{\text{ảnh}}$ khai quá nhỏ. Mỗi khung hình xấu có toàn quyền kéo nghiệm. Trên khung mờ, phần dư phép chiếu không phải nhiễu trắng mà là một *độ lệch có hệ thống*, vì đỉnh cực trị của điểm đặc trưng dịch dọc theo vệt mờ. Nghiệm giật theo. Rồi phần giật đó chảy ngược vào ước lượng độ lệch cảm biến qua nhân tử tiền tích phân, làm hỏng độ lệch, làm dự đoán khung sau tệ hơn. Đó là một vòng phản hồi dương. Nó là cơ chế đứng sau phần lớn các lần mất bám ở đoạn quay nhanh.

Hình 34.2 cho thấy hai chữ ký khác nhau. Quan trọng hơn, nó cho thấy phép đo tách được chúng.



Hình 34.2. Hai kiểu hỏng đối xứng khi đặt sai trọng số. Trục cố ý không có nhãn số: hình này nói về *hình dạng* của hai đường, không về độ lớn của sai số, và độ lớn thì bạn phải tự đo. Quá tin IMU cho một đường trôi mượt và đơn điệu: APE xấu dần trong khi RPE trên cửa sổ ngắn vẫn rất đẹp. Quá tin ảnh cho một đường răng cưa bị chặn: APE có thể trông chấp nhận được trong khi RPE ngắn hạn xấu ở mọi chỗ. Vì thế *tỉ số giữa RPE ngắn hạn và APE* tách được hai kiểu hỏng này, còn con số APE tổng thì không.

Kết luận chẩn đoán rút ra từ hình này đáng ghi lại. Báo APE một mình không phân biệt được hai bệnh ngược nhau. Báo thêm RPE ở vài độ dài cửa sổ thì phân biệt được ngay, và đó là lý do các hướng dẫn đánh giá quỹ đạo đòi cả hai [359].

D. Vì sao hằng số lấy từ datasheet gần như luôn sai

Datasheet không nói dối. Nó trả lời một câu hỏi khác với câu hỏi bạn đang hỏi. Có năm lý do, và chúng cộng dồn.

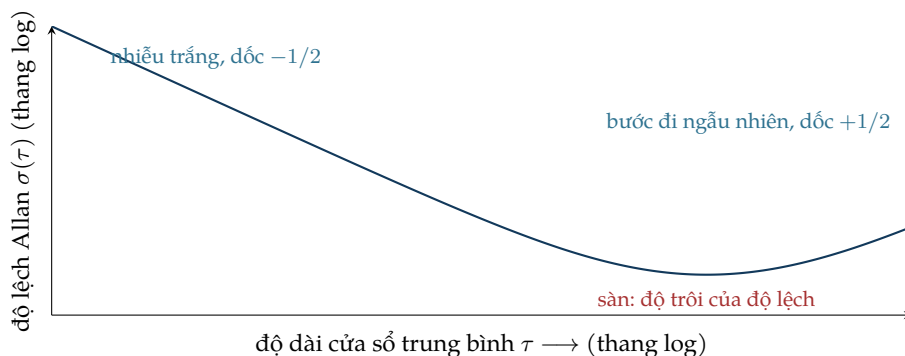
- **Con số datasheet là của con chip, không phải của thiết bị.** Nó đo trên đế thử nghiệm, nhiệt độ ổn định, không rung, và thường là giá trị điển hình chứ không phải chặn trên. Thiết bị của bạn có bo mạch, giá đỡ, động cơ và một vỏ máy.
- **Cái đồ thị nhân tử cần là một σ hiệu dụng.** Nó không phải nhiễu trắng của cảm biến. Nó phải nuốt hết mọi thứ mô hình không tả: sai lệch thời gian t_d giữa camera và IMU, sai số hiệu chỉnh ngoại, sai số hệ số tỉ lệ và lệch trục, sai số rời rạc hoá của chính phép tiền tích phân. Vì vậy giá trị đúng luôn *lớn hơn* datasheet, thường là vài lần. Đây là lý do các hệ thực dụng như VINS-Mono khuyến nghị nhân hệ số nhiễu lên so với giá trị đo được [360].

- **Nhiệt độ.** Độ lệch và hệ số tỉ lệ của con quay MEMS phụ thuộc nhiệt độ khá mạnh. Thiết bị nóng dần trong lúc chạy, nên độ lệch *trôi có hướng*. Mô hình bước đi ngẫu nhiên trong tiền tích phân giả định trôi không hướng, nên nó mô tả sai đúng cái đang xảy ra.
- **Rung.** Động cơ và cánh quạt bơm năng lượng vào dải tần cao. Mọi thành phần trên tần số Nyquist của bộ lấy mẫu bị gập xuống dải đo. Nó trông như nhiễu trắng nhưng không phải. Cùng một IMU trên bàn và trên drone đang bay có hai mức nhiễu hiệu dụng khác hẳn nhau.
- **Tốc độ chuyển động.** Ở tốc độ góc thấp, nhiễu trắng chi phối. Ở tốc độ góc cao, sai số hệ số tỉ lệ chi phối, và sai số đó tỉ lệ với ω chứ không phải hằng số. Nói cách khác, σ đúng cho lúc đứng yên và σ đúng cho lúc quay 200 độ mỗi giây không thể là cùng một số.

Điểm cuối giết chết ý tưởng “tìm một hằng số tốt”. Không tồn tại một hằng số đúng cho cả hai chế độ. Đó cũng là lý do vì sao khi bạn thử vài giá trị, mỗi giá trị lại thắng ở một chuỗi khác nhau: bạn đang tìm một hằng số cho một đại lượng không hằng.

CẠM BẤY

Phương sai Allan là công cụ đúng và bị dùng sai thường xuyên. Nó đo lúc cảm biến *đứng yên*, trong nhiều giờ, ở nhiệt độ ổn định. Đó chính xác là chế độ bạn không vận hành. Số Allan cho bạn một *sàn*: hệ thống không thể tốt hơn thế. Nó không cho bạn giá trị hiệu dụng lúc bay. Dùng thẳng số Allan làm Σ trong đồ thị nhân tử là khai quá tự tin một cách có hệ thống, và luôn theo cùng một hướng. Nên chạy thêm chuỗi không phát hiện ra nó.



Hình 34.3. Hình dạng của một đường Allan cho con quay MEMS. Trục không có nhãn số vì hình này nói về *ba đoạn dốc*, không về giá trị của một cảm biến cụ thể — giá trị thì bạn đo trên chính con quay của mình. Ba đoạn ứng với ba tham số mà tiền tích phân cần: mật độ nhiễu trắng ở dốc trái, độ trôi của độ lệch ở đáy, bước đi ngẫu nhiên của độ lệch ở dốc phải. Cả ba đo lúc *đứng yên*, nên chúng là sàn chứ không phải giá trị vận hành.

E. Phép thử biến câu hỏi thành con số

Trước khi nghĩ tới một mạng, hãy đo xem Σ hiện tại sai bao nhiêu. Phép thử là NEES, và nó rẻ.

Nếu Σ trung thực thì đại lượng $r^\top \Sigma^{-1} r$ có kỳ vọng đúng bằng số bậc tự do d của phần dư. Với phần dư phép chiếu hai chiều, $d = 2$. Cách làm: ghi lại mọi phần dư ở cuối một lần tối ưu, tính trung bình của $r^\top \Sigma^{-1} r$ theo từng loại nhân tử, rồi so với d .

Đọc kết quả thì gọn nhất là viết ra công thức chứ không thay số. Đặt

$$k = \frac{r^\top \Sigma^{-1} r}{d},$$

thì k chính là hệ số mà Σ bạn khai đang sai, tính theo *phương sai*; sai theo *độ lệch chuẩn* là $\sqrt{k}$. Ba trường hợp và chỉ ba: $k \approx 1$ là trung thực; $k > 1$ nghĩa là bạn tự tin hơn mức được phép đúng $\sqrt{k}$

lần; $k < 1$ nghĩa là bạn quá dè dặt và đang vứt đi thông tin. Không cần một con số cụ thể nào để phát biểu điều đó, và đó chính là lý do phép thử này chuyển được từ hệ này sang hệ khác. Với nhân tử tiền tích phân, d là 9 cho phần xoay, vận tốc và vị trí, hoặc 15 nếu tính cả hai độ lệch.

Ba lý do khiến phép thử này quan trọng hơn về ngoài của nó. Thứ nhất, nó là *phép đo*, không phải tính chỉnh: bạn không dò tìm giá trị làm APE đẹp, bạn kiểm một mệnh đề thống kê. Thứ hai, nó tách được theo nhóm: NEES của nhân tử ảnh, của con quay, của gia tốc kể là ba con số riêng, nên bạn biết chỗ nào sai. Thứ ba, nó chạy được trên chính chuỗi bạn đang thất bại, và không cần chân trị.

Một tình tế phải nói rõ. Sau khi tối ưu, phần dư đã bị co lại vì chính nó tham gia định ra nghiệm. Nên NEES tính trên phần dư sau tối ưu thiên về phía *nhỏ hơn* d . Cách sạch hơn là tính trên phần dư *dự đoán*: đánh giá nhân tử của khung mới bằng trạng thái ước lượng từ các khung trước, trước khi khung mới tham gia tối ưu. Nếu bạn chỉ làm được cách thứ nhất, hãy nhớ rằng con số bạn thấy đã là bản lạc quan. Một NEES lớn hơn d khi đó là bằng chứng còn mạnh hơn.

Để thấy một bảng NEES đọc như thế nào, Bảng 34.1 viết ra bốn tình huống theo hệ số k . Giá trị của bảng nằm ở chỗ bốn dòng dẫn tới bốn kết luận khác nhau, và không dòng nào cần một con số cụ thể mới đọc được.

Bảng 34.1. Bốn cách một bảng NEES đọc ra, viết theo hệ số $k = \overline{r^T \Sigma^{-1} r} / d$ chứ không theo con số cụ thể. Bảng cố ý không có số: con số phải đến từ chính lần chạy của bạn, còn cái chuyển được từ hệ này sang hệ khác là bốn dòng dưới đây. Không dòng nào đọc được nếu thiếu bậc tự do d .

Điều bạn thấy	Nghĩa là	Làm gì tiếp
$k \approx 1$ cho một nhóm nhân tử	Σ của nhóm đó trung thực	Không đụng tới. Ghi lại làm mốc để so khi đổi cấu hình
$k > 1$, và lớn dần theo tầng kim tự tháp	Hệ số trọng số theo tầng đang quá lạc quan ở tầng cao	Sửa <i>hình dạng</i> của hàm phụ thuộc tầng, đừng nhân một hằng số chung cho mọi tầng
$k > 1$ trên nhân tử con quay	Hoặc σ khai quá nhỏ, hoặc phần dư có một thành phần <i>hệ thống</i>	Loại trừ sai lệch thời gian t_d và sai số hiệu chỉnh ngoại trước; chỉ khi hết cả hai mới được kết luận là nhiễu
$k < 1$ trên nhân tử gia tốc kể	Quá dè dặt: bạn đang vứt đi thông tin gia tốc	Siết lại theo đúng hệ số $\sqrt{k}$; hệ quả quan sát được là tỉ lệ đơn ảnh hội tụ nhanh hơn

Chú ý dòng cuối. Quá dè dặt ít khi làm hệ sập, nên nó gần như không bao giờ bị phát hiện bằng cách nhìn quỹ đạo. Nó chỉ hiện ra khi bạn tính k cho từng nhóm. Đó là lý do phép thử đáng chạy cả khi hệ đang chạy tốt.

I Năm nấc của việc đặt trọng số IMU

1 Vấn đề

Không có số nào cả.

Giải pháp

lấy hằng số từ datasheet.

Giả định

cảm biến hoạt động như trong phòng thí nghiệm.

Giá phải trả

mọi sai lệch chưa mô hình hoá bị dồn vào chỗ khác, và luôn theo hướng quá tự tin.

2 Vấn đề

Datasheet không khớp thiết bị thật.

Giải pháp

đo phương sai Allan trên chính thiết bị, nhiều giờ, đứng yên.

Hệ quả

bạn có ba tham số đúng cho một chế độ.

Vấn đề mới

chế độ đó là chế độ đứng yên, không phải chế độ bay.

3 Vấn đề

Độ lệch trôi theo nhiệt và theo thời gian.

- Giải pháp** đưa độ lệch vào vectơ trạng thái và ước lượng nó trực tuyến, đúng như ORB-SLAM3 và VINS-Mono làm [40], [360].
- Giả định** trôi là bước đi ngẫu nhiên không hướng.
- Giá phải trả** bạn phải chọn một tham số bước đi ngẫu nhiên, và nó lại là một hằng số nữa.
- 4 Vấn đề** Nhiều hiệu dụng đổi theo chuyển động và theo rung.
- Giải pháp** luật thích nghi: nói Σ khi phát hiện tốc độ góc cao hoặc rung.
- Giả định** bạn viết đúng được hàm phụ thuộc đó bằng tay.
- Giá phải trả** đây là tinh chỉnh trá hình nếu hàm được chọn theo kết quả; kinh nghiệm thực hành trên EuRoC cho thấy một hằng số chặt, hiệu chỉnh đúng, thường không thua các luật thích nghi viết tay.
- 5 Vấn đề** Không viết được hàm phụ thuộc đó bằng tay.
- Giải pháp** học nó: một mạng nhận cửa sổ IMU thô và trả về cả phép đo lẫn hiệp phương sai của phép đo.
- Giả định** phân phối chuyển động lúc chạy giống lúc huấn luyện.
- Giá phải trả** mất tính chắn đoán, thêm một tạo tác phải bảo trì, và một chế độ hỏng mới là tự tin sai khi ra ngoài phân phối.

Bảng 34.2. Năm cách đặt trọng số IMU. Cột cuối là cột đáng giá nhất: nó nói khi nào con số bạn khai không còn nghĩa.

Cách	Cơ chế	Mạnh khi nào	Hỏng khi nào
Hằng số datasheet	Đọc mật độ nhiễu từ tài liệu chip	Bản dựng đầu tiên, cần một con số để chạy được	Gần như luôn quá tự tin; sai theo cùng một hướng nên lặp lại nhiều lần không lộ
Hiệu chỉnh Allan	Đo nhiều giờ lúc đứng yên, tách ba tham số	Thiết bị cố định, ít rung, nhiệt ổn định	Chế độ vận hành có rung hoặc quay nhanh; số Allan là sà n chứ không phải giá trị lúc chạy
Ước lượng độ lệch trực tuyến	Đưa độ lệch vào trạng thái, ràng buộc bằng bước đi ngẫu nhiên	Mọi hệ thị giác-quán tính hiện đại; bất được trôi chậm	Trôi có hướng theo nhiệt; giai đoạn khởi tạo khi chưa đủ quan sát để tách độ lệch khỏi trọng lực
Luật thích nghi viết tay	Nói Σ theo một chỉ báo như tốc độ góc hay mức rung	Khi bạn có mô hình vật lý thật cho phần phụ thuộc đó	Khi hàm được chọn theo kết quả: đó là tinh chỉnh và nó không chuyển sang tập khác
Hiệp phương sai học được	Mạng xuất cả phép đo lẫn Σ , huấn luyện bằng độ hợp lý	Có dữ liệu đúng nền tảng và đúng phân phối chuyển động	Ra ngoài phân phối: mạng vẫn khai Σ nhỏ, tức tự tin sai — chế độ hỏng tệ nhất cho một trọng số

↔ cùng nguyên lý với E/24 — NEES cũng là một phép “đo sà n trước, kết luận sau”: trước khi so hai cấu hình trọng số, hãy kiểm xem trọng số hiện tại có trung thực không

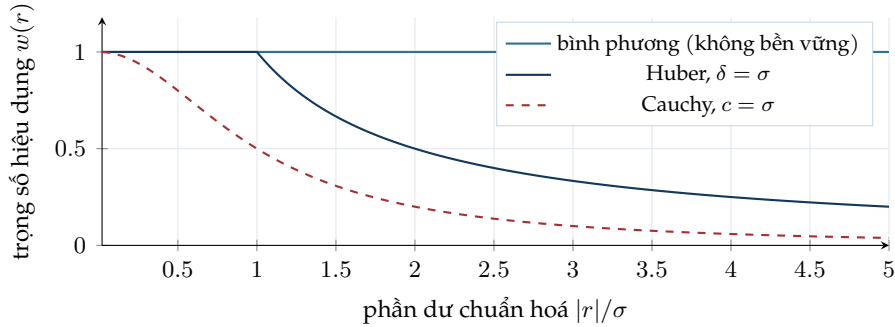
F. Hàm mất mát bền vững là một trọng số thích nghi trá hình

Còn một chỗ mà trọng số đã bị thay đổi trong hệ của bạn mà ít ai gọi tên như vậy: hàm mất mát bền vững. Huber và Cauchy được dùng ở khắp nơi trong SLAM để chống điểm ngoại lai, và cơ chế của chúng đúng là đổi trọng số.

Cách nhìn gọn nhất là bình phương tối thiểu có trọng số lặp lại. Một hàm mất mát bền vững $\rho(r)$ tương đương với bình phương tối thiểu thường, nhưng mỗi phần dư mang thêm một hệ số

$$w(r) = \frac{\rho'(r)}{r}.$$

Với chuẩn bình phương thì $w \equiv 1$. Với Huber ngưỡng δ , $w = 1$ khi $|r| \leq \delta$, và $w = \delta/|r|$ khi ra ngoài. Tính tay một trường hợp: phần dư bằng 3δ thì trọng số hiệu dụng chỉ còn một phần ba, tức ma trận thông tin của ràng buộc đó bị chia ba. Với Cauchy tham số c , $w = 1/(1 + (r/c)^2)$, nên cùng phần dư $3c$ chỉ còn một phần mười.



Hình 34.4. Hàm mất mát bền vững nhìn dưới dạng trọng số. Đường cong là $w(r) = \rho'(r)/r$, tức hệ số nhân vào ma trận thông tin của một ràng buộc khi phần dư của nó lớn. Đây là *công thức*, không phải số đo. Điểm cần thấy: một hàm bền vững cũng là cách đặt trọng số thích nghi, nhưng nó thích nghi theo *phần dư* chứ không theo *đầu vào*.

Hình 34.4 cho thấy điểm phân biệt quan trọng nhất của mục này. Hàm bền vững phản ứng *sau khi* đã thấy phần dư. Một hiệp phương sai học được, hay một Σ suy từ vệt mờ, dự đoán *trước khi* thấy phần dư. Hai thứ không thay thế nhau, và lý do rất cụ thể. Phần dư lớn có thể do phép đo tồi, mà cũng có thể do trạng thái hiện tại đang sai. Hàm bền vững không phân biệt được hai nguyên nhân đó; nó hạ trọng số trong cả hai trường hợp.

Hệ quả rơi thẳng ra là một cạm bẫy hay gặp. Đặt hàm bền vững lên cạnh IMU thường sai, và sai theo hướng nguy hiểm. Phần dư IMU lớn hiếm khi có nghĩa là phép đo quán tính bị ngoại lai. Nó thường có nghĩa là tư thế hoặc độ lệch đang sai, tức đúng lúc bạn cần ràng buộc IMU kéo mạnh nhất. Hạ trọng số ở đúng lúc đó là giấu triệu chứng đi. Với cạnh ảnh thì ngược lại: sai ghép là chuyện thường ngày, nên hàm bền vững ở đó là công cụ đúng.

Tóm lại, ba tầng nên tách bạch. Tầng một là Σ trung thực, suy từ mô hình và kiểm bằng NEES. Tầng hai là hàm bền vững, dành cho các loại ràng buộc thật sự có ngoại lai. Tầng ba là loại bỏ hẳn bằng kiểm chứng hình học. Trộn ba tầng — ví dụ nối Σ để bù cho việc thiếu bước loại bỏ — là cách phổ biến nhất để một hệ trở nên không chẩn đoán được.

34.3 Học nhiễu và bất định IMU (Learned Noise and Learned Uncertainty)

A. Ý trung tâm của cả chương

Đọc các công trình trong mục này rất dễ rút ra kết luận sai. Kết luận sai là: mạng nơ-ron ước lượng tư thế tốt hơn hình học. Nó không tốt hơn. Trên dữ liệu có ảnh tốt và hiệu chỉnh tốt, một bộ ước lượng hình học viết cẩn thận vẫn thắng, và thắng với chi phí thấp hơn nhiều.

Kết luận đúng là: mạng ước lượng *độ tin cậy* tốt hơn. Nó đọc được từ đầu vào những thứ mà một hằng số không đọc được. Ảnh này có mờ không, đoạn IMU này có rung không, điểm khớp này

nằm trên cạnh hay trên góc. Và ước lượng độ tin cậy chính là ô trống trong hệ cổ điển, vì hệ cổ điển lấy nó từ datasheet và từ tầng kim tự tháp.

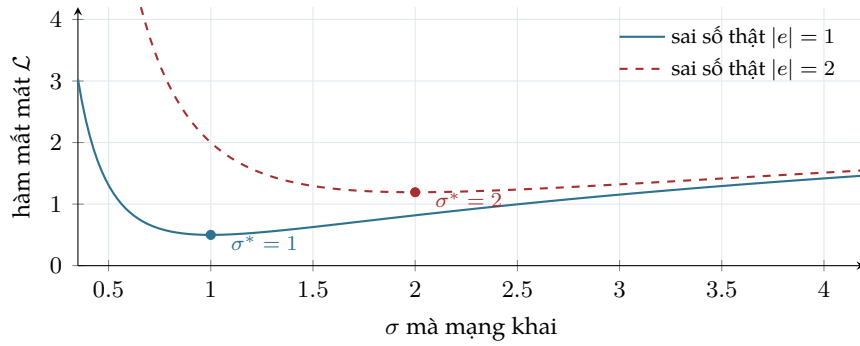
Cơ chế làm được điều đó gọn tới bất ngờ, và đáng suy dẫn ra vì nó nói rõ mạng thực sự học cái gì. Cho mạng xuất ra cả giá trị dự đoán μ lẫn phương sai σ^2 , rồi huấn luyện bằng độ hợp lý âm của phân phối chuẩn [361]:

$$\mathcal{L} = \frac{1}{2} \left(\frac{(y - \mu)^2}{\sigma^2} + \log \sigma^2 \right).$$

Số hạng thứ nhất phạt việc tự tin mà sai. Số hạng thứ hai phạt việc khai bất định lớn để né. Đặt $e = y - \mu$ rồi lấy đạo hàm theo σ^2 , cực tiểu nằm ở

$$\sigma^{2*} = e^2.$$

Nói bằng lời: đầu ra tối ưu của nhánh phương sai là *bình phương sai số kỳ vọng*, có điều kiện theo đầu vào. Một hiệp phương sai học được không phải phép màu. Nó là một bài hồi quy bình phương sai số lên chính đầu vào. Hình 34.5 cho thấy hình dạng của hàm mất mát đó và vì sao nó có đúng một cực tiểu.



Hình 34.5. Vì sao độ hợp lý âm ép mạng nói thật. Nhánh trái dốc đứng: khai σ quá nhỏ bị phạt bởi số hạng bình phương. Nhánh phải đi lên chậm: khai σ quá lớn bị phạt bởi $\log \sigma^2$. Cực tiểu nằm đúng tại $\sigma = |e|$, nên nhánh phương sai của mạng chỉ đang hồi quy sai số kỳ vọng theo đầu vào — không hơn, và cũng không kém.

Hai hệ quả rơi thẳng ra từ suy dẫn này, và cả hai đều là giới hạn.

- **Mạng chỉ biết loại sai số nó đã thấy.** Bất định học được theo cách này là bất định của dữ liệu, tức phần nhiễu vốn có trong đầu vào. Nó không phải bất định của mô hình. Gặp một đầu vào lạ, mạng vẫn nội suy và vẫn khai một σ nhỏ. Đó là tự tin sai, và với một trọng số thì đây là chế độ hổng tệ nhất có thể [362].
- **Muốn bất phần bất định của mô hình thì phải trả thêm.** Cách rẻ nhất là hợp nhiều mô hình rồi lấy độ tán mát giữa chúng làm bất định [209], hoặc lấy mẫu bằng dropout lúc suy luận [208]. Cả hai nhân chi phí suy luận lên vài lần, nên với một hệ thời gian thực trên Orin, chúng gần như luôn ngoài ngân sách.

B. Bốn công trình về phía IMU

Bốn công trình dưới đây được xếp theo mức độ can thiệp vào hệ của bạn, từ nhẹ tới nặng. Với mỗi cái, câu hỏi đúng không phải “nó chính xác bao nhiêu” mà là bốn câu của Phần F: nó chữa thất bại nào, cơ chế gì, có đáng không, và học được gì kể cả khi không dùng.

Khử nhiễu con quay bằng mạng tích chập giãn

Đây là can thiệp nhẹ nhất [363]. Thất bại được chữa: tư thế góc ước lượng vòng hở từ con quay MEMS trôi rất nhanh, vì độ lệch, sai số hệ số tỉ lệ và lệch trục cộng dồn.

Cơ chế: một mạng tích chập giãn nhận cửa sổ dữ liệu IMU thô và xuất ra một *lượng hiệu chỉnh* cộng vào phép đo con quay. Hàm mất mát không đặt trên từng mẫu. Nó đặt trên số gia xoay tích lũy qua nhiều độ dài cửa sổ, tính trên nhóm xoay.

Hai chi tiết thiết kế đáng lấy dù bạn không dùng phương pháp. Thứ nhất, mạng sửa *phép đo* chứ không sửa *trạng thái*, nên toàn bộ phần tiền tích phân phía sau không phải đưng tới. Thứ hai, hàm mất mát đặt trên đúng đại lượng mà đồ thị nhân tử tiêu thụ, chứ không phải trên đại lượng dễ đo nhất. Tích chập giãn cho trường tiếp nhận dài với ít phép tính, và điều đó quan trọng khi tần số mẫu là 200 Hz.

Chi phí: đây là mạng nhỏ nhất trong cả chương — một chồng tích chập một chiều trên một cửa sổ tín hiệu, không phải một mô hình ảnh — nên nó là ứng viên duy nhất trong mục này có cửa ngay trên luồng theo vết. Nhưng “có cửa” không phải “vừa”; đại lượng duy nhất đáng tin là mức thay đổi phân vị cao của chu kỳ luồng theo vết khi bạn bật nó lên trong hệ đầy đủ. Hồng khi nào: nó được hiệu chỉnh cho một đơn vị IMU cụ thể và một phân phối chuyển động cụ thể. Đổi thiết bị thì lượng hiệu chỉnh sai, và sai một cách im lặng.

TLIO

TLIO can thiệp ở mức vừa [224]. Thất bại được chữa: định vị chỉ bằng quán tính trên thiết bị đeo trôi trong vài giây, nên không dùng được khi mất ảnh.

Cơ chế: một mạng nhận cửa sổ IMU đã căn theo trọng lực và hồi quy một vectơ dịch chuyển ba chiều *cùng với hiệp phương sai của nó*. Cặp giá trị đó được đưa vào một bộ lọc như một phép đo giả.

Đây là bài học thiết kế quan trọng nhất của cả mục: *mạng không phải bộ ước lượng, bộ lọc mới là*. Mạng chỉ cung cấp một phép đo kèm mức tin cậy đã khai báo. Trạng thái — độ lệch, vận tốc, hướng trọng lực — vẫn do bộ lọc giữ. Vì thế nó cắm được vào một đồ thị nhân tử có sẵn dưới dạng một nhân tử mới, không đưng gì tới phần còn lại.

Có đáng không, với trường hợp của bạn: nó chữa một nỗi đau bạn không có, vì bạn luôn có stereo. Nhưng *khuôn* của nó là thứ đáng bắt chước nhất trong toàn chương.

RoNIN

RoNIN đi xa hơn về phía dữ liệu [364]. Nó hồi quy vận tốc trong một hệ quy chiếu không phụ thuộc hướng thiết bị, chỉ từ IMU, và đi kèm một bộ dữ liệu chuẩn để so sánh.

Điều đáng học không phải con số mà là nguồn gốc của độ chính xác. Nó đến từ một *tiên nghiệm chuyển động* rất mạnh: dáng đi của người có cấu trúc lặp lại, và mạng học chính cấu trúc đó. Đây vừa là lý do nó chạy được vừa là lý do nó không chuyển sang nền tảng khác. Một drone hay một xe không có tiên nghiệm tương đương. Nếu chuyển động của robot bạn không bị ràng buộc, đây là hướng sai.

IDOL

IDOL tách bài toán làm hai tầng [365]: học hướng trước, rồi học vị trí trong hệ quy chiếu đó. Bài học thiết kế khớp thẳng với phép tính $\frac{1}{6}get^3$ ở mục trước. Sai số xoay là thứ đầu độc sai số tịnh tiến, nên giải nó trước và giải riêng là hợp lý về mặt cấu trúc. Mức tin cậy của tôi về chi tiết hiện thực của công trình này chỉ ở mức vừa, nên tôi nêu ý tưởng và không mô tả kết quả số.

Ngoài bốn cái trên, có một mảnh bằng chứng đáng để ý cho người làm chuyển động mạnh. Hướng học đo quán tính cho đưa drone cho thấy khuôn “phép đo học được cộng hiệp phương sai, đưa vào bộ ước lượng cổ điển” vẫn hoạt động ở chế độ chuyển động rất hung hãn [366]. Điều kiện là phân phối chuyển động lúc huấn luyện khớp lúc chạy. Điều kiện đó không phải chi tiết nhỏ; nó là toàn bộ vấn đề.

C. MAC-VO: bất định theo từng điểm khớp

Phía ảnh có một ô trống lớn hơn phía IMU, và ít người nói ra. Trong ORB-SLAM3, trọng số của một phần dư phép chiếu là một hằng số nhân theo hệ số tỉ lệ của tầng kim tự tháp mà điểm đặc trưng được phát hiện [40]. Đó là một đại diện cho *độ phân giải*. Nó không biết gì về mờ, về vùng ít vân, về hoa văn lặp lại, về vật thể đang chuyển động, hay về việc điểm khớp thứ hai gần bằng điểm khớp thứ nhất tới mức nào.

MAC-VO lấp đúng ô đó [302]. Thất bại được chữa: mọi điểm khớp bị đối xử như nhau, nên một nhóm nhỏ điểm khớp tồi có thể kéo lệch cả tư thế.

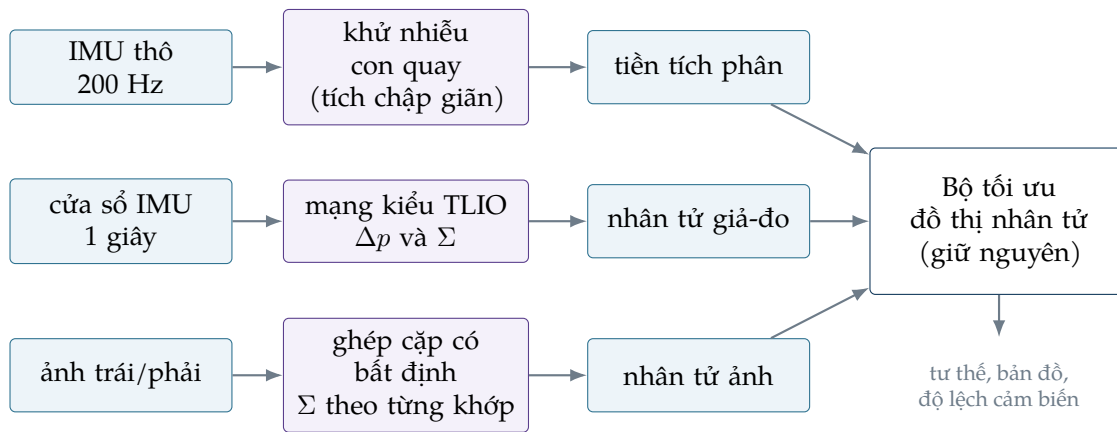
Cơ chế: bộ ghép cặp học được xuất ra bất định của chính nó cho từng điểm khớp. Bất định trong mặt phẳng ảnh được đưa qua hình học phép chiếu để thành một hiệp phương sai ba chiều đúng nghĩa, dẹt và dài dọc theo tia nhìn với điểm ở xa. Hiệp phương sai đó trực tiếp làm trọng số cho từng ràng buộc ảnh. Đây là ý “bất định phải mang đơn vị đo đúng”. Một sai số hai điểm ảnh tương ứng với sai số độ sâu rất khác nhau tùy điểm gần hay xa, và một trọng số vô hướng trong mặt phẳng ảnh không tài nào biểu diễn được điều đó.

Có đáng làm không, xét theo bốn cổng của F/27. Chi phí: đây là cả một frontend học được, không phải một mô-đun cắm thêm. Nó chiếm GPU, và GPU của bạn đang bận trích ORB và chạy hiệu chỉnh chùm tia cục bộ. Dữ liệu: nó cần dữ liệu có chân trị độ sâu hoặc mô phỏng. Hồng khi ra ngoài phân phối: giống mọi bất định học được, mạng vẫn khai bất định nhỏ trên cảnh lạ. Kết luận trung thực tính tới tháng 9/2026 là chưa nên thay frontend.

Nhưng bài học thiết kế thì rút được ngay và gần như miễn phí: **hãy làm cho ma trận thông tin của ảnh phụ thuộc vào một đại lượng đo được của từng điểm khớp**. Bạn không cần một mạng để làm điều đó. Ba đại diện rẻ, tính được ngay trong lúc trích đặc trưng, đều tốt hơn tầng kim tự tháp:

- **Khoảng cách tới điểm khớp tốt thứ hai.** Tỉ số giữa khoảng cách mô tả tốt nhất và tốt nhì đã là một đo lường mơ hồ, và nó có sẵn trong mọi bộ ghép cặp.
- **Năng lượng gradient cục bộ và tính dị hướng của nó.** Ma trận cấu trúc 2×2 quanh điểm đặc trưng cho ngay hai trị riêng. Tỉ số giữa chúng nói điểm đó bị ràng buộc theo một phương hay hai phương. Đây đúng là thứ dùng để dựng một Σ dị hướng.
- **Vệt mờ dự đoán từ con quay.** Mục sau tính con số này; nó cho cả hướng lẫn độ dài của trục dài trong ellipse bất định.

Và ba lựa chọn ấy kiểm chứng được bằng NEES, theo đúng cách ở mục trước. Chia điểm khớp thành các nhóm theo tỉ số khoảng cách mô tả rồi tính NEES riêng từng nhóm, bạn sẽ thấy ngay nhóm nào đang bị khai quá tự tin. Đó là bằng chứng, không phải tình hình.



■ mạng ■ khối cổ điển giữ nguyên

Cả ba chỗ cắm đều *cung cấp phép đo kèm Σ* , không thay bộ ước lượng. Đó là lý do chúng thử được từng cái một, và bỏ ra được khi không đáng.

Hình 34.6. Ba chỗ mà một mạng cắm được vào một hệ thị giác-quán tính cổ điển mà không đụng bộ tối ưu. Trên: sửa phép đo con quay trước khi tiền tích phân. Giữa: thêm một nhân tử giả-đo dịch chuyển kèm hiệp phương sai, theo khuôn TLIO. Dưới: thay trọng số vô hướng của ảnh bằng hiệp phương sai theo từng điểm khớp, theo khuôn MAC-VO. Điểm chung là bộ ước lượng vẫn là bộ ước lượng cổ điển, nên mỗi chỗ cắm là một ablation độc lập.

Bảng 34.3. Năm công trình, đọc theo bốn câu hỏi của Phần F. Cột cuối là phần giữ được giá trị kể cả khi bạn kết luận là chưa đáng dùng.

Công trình	Chứa thất bại nào	Có đáng với hệ của bạn không	Bài học thiết kế giữ lại
Khử nhiễu con quay	Tư thế góc vòng hờ trôi vì độ lệch và sai số tỉ lệ	Có thể: mạng rất nhỏ, không đụng backend; nhưng phải hiệu chỉnh lại cho từng đơn vị IMU	Sửa <i>phép đo</i> , không sửa trạng thái; đặt hàm mất mát trên đúng đại lượng backend tiêu thụ
TLIO	Định vị chỉ bằng quán tính trôi trong vài giây	Không trực tiếp: bạn luôn có stereo	Mạng cấp phép đo kèm Σ ; bộ lọc vẫn là bộ ước lượng và vẫn giữ mọi trạng thái
RoNIN	Không có tư thế khi thiếu ảnh, trên thiết bị đeo	Không: tiên nghiệm đáng đi không tồn tại cho drone hay xe	Độ chính xác đến từ tiên nghiệm chuyển động; đó cũng là chỗ nó không chuyển được
IDOL	Sai số hướng đầu đọc sai số vị trí	Không trực tiếp	Tách hướng khỏi vị trí; xử lý xoay trước vì lỗi xoay khuếch đại theo t^3
MAC-VO	Mọi điểm khớp bị đối xử như nhau dù chất lượng rất khác	Chưa, tính tới 9/2026: cả một frontend, tranh GPU với ORB và hiệu chỉnh chùm tia	Cho $\Omega_{\text{ảnh}}$ phụ thuộc đại lượng đo được của từng khớp, và cho nó dị hướng

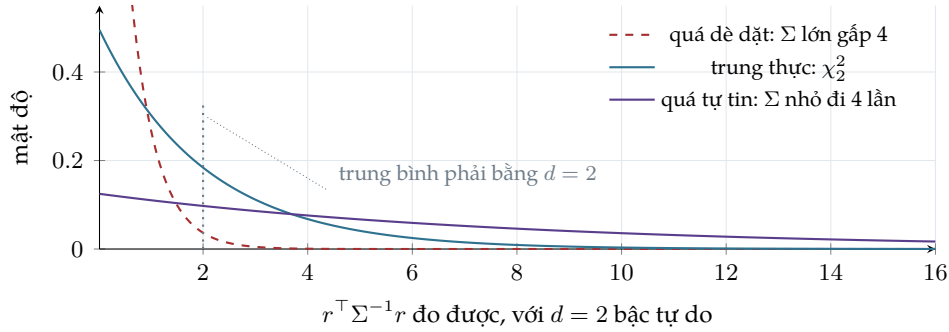
↔ **tiền đề F/28** — bất định theo từng điểm khớp chỉ có nghĩa khi bộ ghép cặp xuất được một điểm số hiệu chỉnh tốt; chương 28 là nơi bàn các bộ ghép cặp đó

D. Kiểm một bất định có trung thực không, kể cả khi nó do mạng sinh ra

Mục 2 đã giới thiệu NEES cho hằng số cổ điển. Phép thử đó áp nguyên vào một hiệp phương sai học được, và đây là điều kiện nghiệm thu bắt buộc trước khi cắm bất kỳ mạng nào vào bộ ước lượng.

Hình dạng của phép thử rất sạch về mặt lý thuyết. Nếu Σ khai đúng thì $r^T \Sigma^{-1} r$ theo phân phối χ^2 với d bậc tự do. Nếu Σ thật lớn hơn k lần thì đại lượng đo được theo phân phối $k \chi_d^2$. Nói cách

khác, cả phân bố dịch sang phải đúng k lần, và trung bình đi từ d lên $k d$. Vì vậy bạn không chỉ đọc được “sai hay đúng” mà đọc được luôn hệ số sai.



Hình 34.7. Đọc một biểu đồ NEES. Ba đường là mật độ *chính xác* của $k \chi_2^2$ với $k = 1/4, 1, 4$, không phải số minh họa. Trung bình đo được chia cho d chính là hệ số sai k về phương sai, tức $\sqrt{k}$ về độ lệch chuẩn. Đuôi phải dày bất thường mà trung bình vẫn gần d thì lại là chuyện khác: đó là dấu hiệu ngoại lai, và cách chữa là loại bỏ chứ không phải nói Σ .

Với một mạng, phép thử này còn cho thêm một thứ mà hằng số không có: bạn kiểm được *tính hiệu chỉnh theo từng mức*. Cách làm là chia các phép đo thành nhóm theo chính σ mà mạng khai, rồi tính NEES riêng cho từng nhóm. Một mạng trung thực cho NEES gần d ở *mọi* nhóm. Một mạng chỉ đúng trung bình thì sẽ lộ ra ngay: nó thường quá tự tin ở nhóm σ nhỏ và quá dè dặt ở nhóm σ lớn, tức nó đã học đúng thứ tự nhưng sai thang đo.

Phân biệt đó quan trọng vì hai bệnh có hai cách chữa khác nhau. Sai thang đo sửa được bằng một hệ số duy nhất tính từ dữ liệu kiểm định, và đó là hiệu chỉnh chứ không phải tinh chỉnh. Sai thứ tự thì không sửa được bằng bất kỳ hệ số nào; nó nghĩa là mạng không đọc được dấu hiệu cần đọc, và bạn phải quay lại phần dữ liệu hoặc đầu vào.

Cuối cùng, một phép thử rẻ mà nên chạy trước cả hai phép trên: cho mạng ăn dữ liệu ngoài phân phối một cách cố ý. Che một phần ảnh, tăng nhiễu, đảo tương phản, hoặc lấy một chuỗi từ thiết bị khác. Nếu σ mạng khai *không* tăng lên trong các trường hợp đó, bạn đã có bằng chứng trực tiếp cho chế độ hỏng tệ nhất, và bạn có nó trước khi đưa mạng vào hệ thống.

34.4 Ảnh mờ do chuyển động và chuyển động nhanh (Motion Blur and Fast Motion)

A. Trước khi bàn tới mạng: hãy tính vệt mờ ra pixel

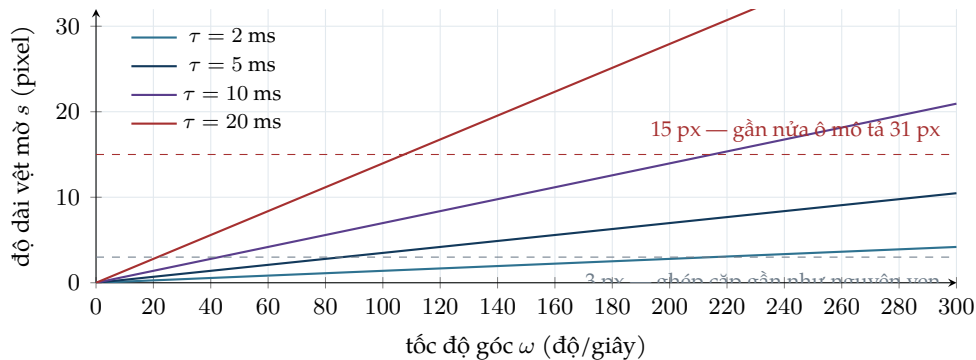
Ảnh mờ hay được nói tới như một điều kiện xấu mơ hồ. Nó là một đại lượng tính được, và tính nó ra trước là bước rẻ nhất trong cả mục.

Trong thời gian phơi sáng τ , camera quay một góc $\omega \tau$. Với tiêu cự f tính bằng pixel, độ dài vệt mờ do xoay xấp xỉ

$$s \approx f \omega \tau \quad (\text{pixel}).$$

Thay số vào để thấy bậc của quan hệ. Đây là phép tính *giả định* trên một camera giả định, không phải số đo: giả sử $f = 400$ pixel và $\tau = 10$ ms. Quay 30 độ mỗi giây cho $s \approx 2,1$ pixel, gần như vô hại. Quay 200 độ mỗi giây cho $s \approx 14$ pixel, trong khi ô mô tả của ORB rộng 31 pixel — một vệt cỡ đó phá gần hết cấu trúc bên trong ô. Điều đáng mang đi không phải hai con số mà là *quan hệ*: s tuyến tính theo cả ω lẫn τ , nên tốc độ quay tăng vài lần là đủ đưa vệt mờ từ vô hại sang phá hủy. Với camera của bạn, thay f và τ thật vào rồi vẽ phân bố của s trên chuỗi thật — đó mới là con số dùng được.

Vệt do tịnh tiến thì phụ thuộc độ sâu: cùng một vận tốc, điểm gần trượt nhiều pixel hơn điểm xa. Chi tiết này sẽ quay lại ở mục sau, vì nó là lý do một nhân chấp duy nhất cho cả ảnh là mô hình sai.



Hình 34.8. Vệt mờ do xoay, tính bằng $s \approx f\omega\tau$ với một tiêu cự giả định $f = 400$ pixel. Đây là đồ thị của một công thức, không phải số đo: bốn đường chính xác theo mô hình, và chúng đối chỗ ngay khi bạn thay f của camera mình vào. Đọc hình này ngược lại cho đòn bẩy rõ nhất: s tuyến tính theo τ , nên hạ thời gian phơi sáng đi bốn lần thì chia độ dài vệt cho bốn, và cái giá chỉ là phải tăng độ khuếch đại.

Từ hình này rơi ra một kết luận cần nói trước mọi thứ khác. **Bộ điều khiển phơi sáng tự động là một phần của hệ SLAM của bạn, và hầu như không ai đối xử với nó như vậy.** Nó được chỉnh cho ảnh đẹp mắt, tức ưu tiên đủ sáng hơn là đủ ngắn. Đặt trần cho τ và bù bằng độ khuếch đại là một thay đổi vài dòng, không tốn thêm một phép tính nào, và nó tấn công thẳng nguyên nhân. Nhiều cảm biến tăng lên là cái giá. Nhưng nhiễu trắng làm giảm chất lượng mô tả ít hơn nhiều so với một vệt mờ dài gần bằng nửa ô mô tả — và tỉ lệ đánh đổi đó thì đo được bằng số điểm khớp đúng, trên chính chuỗi của bạn.

B. Vì sao khử mờ rồi mới trích đặc trưng thường không giúp

Thứ tự “chạy mạng khử mờ, rồi trích ORB trên ảnh đã khử” nghe hợp lý tới mức ít ai kiểm. Nó thường không tăng số điểm khớp đúng, và có ba lý do độc lập.

Lý do thứ nhất, mục tiêu bị lệch. Mạng khử mờ được huấn luyện và đánh giá cho mắt người: PSNR, SSIM, LPIPS, và ngày càng nhiều hàm mất mát đối kháng hoặc tri giác. Có một đánh đổi đã được chứng minh giữa độ trung thực và mức “trông thật”. Vượt qua một ngưỡng, muốn ảnh trông sắc hơn thì buộc phải hy sinh độ trung thực với ảnh gốc [104]. Ảnh khử mờ nhìn sắc có thể chứa cạnh do mô hình bịa ra.

Điều đó giết tính lặp lại vì một lý do tinh tế. Ghép cặp đòi hỏi cùng một cấu trúc vật lý phải sinh ra cùng một cực trị cục bộ trong *cả hai* ảnh. Sai số của mạng khử mờ không độc lập giữa hai ảnh. Nó phụ thuộc nhân mờ của từng ảnh, mà hai ảnh có hai nhân mờ khác nhau vì camera chuyển động khác nhau. Vậy mạng sinh ra sai số *tương quan với chuyển động*, tức đúng loại sai số mà bộ ước lượng tư thế không chịu nổi. Nó không phải nhiễu; nó là độ lệch.

Lý do thứ hai, mô hình mờ sai. Phần lớn mạng khử mờ giả định một nhân mờ chung cho cả ảnh, hoặc học từ dữ liệu tổng hợp mờ đều. Mờ do xoay thì gần như đều thật, nhưng mờ do tịnh tiến phụ thuộc độ sâu. Trong một cảnh có cả vật gần và vật xa, không tồn tại một nhân chung nào đúng.

Lý do thứ ba, độ trễ. Một mạng khử mờ là mô hình ảnh-sang-ảnh ở độ phân giải đầy đủ, tức hình dạng đất nhất trong các hình dạng mạng — và bạn vẫn phải trích đặc trưng *sau* nó. So với việc hạ thời gian phơi sáng, một thay đổi vài dòng không tốn phép tính nào, đây là một cuộc đổi chác rất xấu. Nếu vẫn muốn kiểm, đại lượng phải đo không phải độ trễ riêng của mạng mà là mức thay đổi phân vị cao của chu kỳ luồng theo vệt khi bật nó lên trong hệ đầy đủ.

Một ngoại lệ trung thực phải nêu. Với tầng *lấy ứng viên đóng vòng*, phép tính đối khác. Ở đó bạn dùng một mô tả toàn ảnh, chạy thừa thớt, không nằm trên đường tới hạn của luồng theo vết. Tiêu chí cũng không phải cực trị cục bộ mà là sự giống nhau toàn cục. Khử mờ trước khi tính mô tả toàn ảnh có thể có ích, và chi phí thì chịu được. Chi tiết về tầng đó nằm ở [F/29-dinh-vi-lai-va-dong-vong](#).

C. Hướng đúng thứ nhất: coi vết mờ là quỹ đạo trong thời gian phơi sáng

Đây là chỗ đảo ngược cách nghĩ, và là ý hay nhất trong mục này.

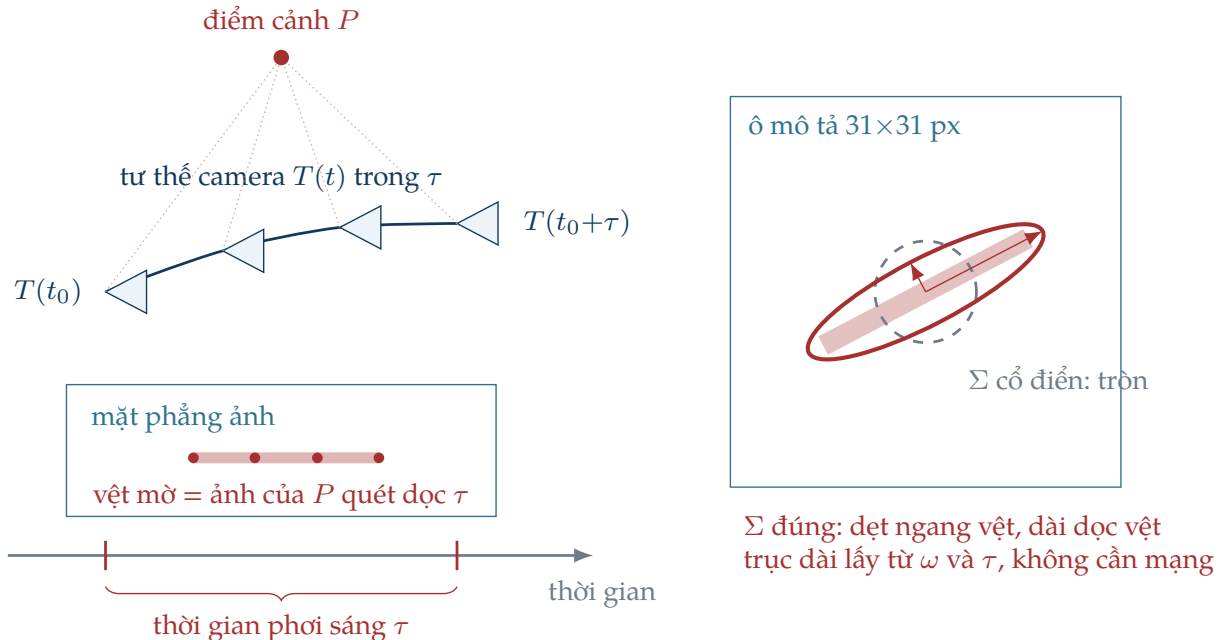
Ảnh mờ không phải ảnh sắc nét bị hỏng. Nó là *tích phân* của một chuỗi ảnh sắc nét, chụp từ các tư thế camera nằm dọc một đoạn quỹ đạo trong khoảng $[t_0, t_0 + \tau]$. Viết ra thì gọn: giá trị một pixel là trung bình theo thời gian của ảnh sắc nét được dựng từ tư thế $T(t)$.

Dòng BAD-NeRF và BAD-Gaussians dùng đúng phát biểu đó [367], [368], tiếp nối Deblur-NeRF [369]. Thay vì khử mờ ảnh, chúng mô hình hoá quá trình tạo ảnh. Chúng dựng vài ảnh ảo từ các tư thế nội suy giữa tư thế đầu và tư thế cuối của khoảng phơi sáng, lấy trung bình, rồi so với ảnh mờ đã ghi. Cảnh và cặp tư thế đầu-cuối được tối ưu đồng thời.

Hệ quả khái niệm là phân quan trọng nhất. **Vết mờ trở thành thông tin về chuyển động, thay vì một phiền toái.** Chiều dài và hướng của vết là một phép đo vận tốc. Một khung mờ không còn là khung phải bỏ; nó là khung mang một loại ràng buộc khác.

Bạn không cần chạy NeRF để lấy ý này. Phiên bản cổ điển của nó là: thay vì đánh giá phần dư phép chiếu tại một thời điểm duy nhất, hãy tích phân phép chiếu trên khoảng phơi sáng. Khi đó điểm đặc trưng mờ không còn là điểm ngoại lai; nó là một ràng buộc lên vận tốc. Cái giá là mô hình phép đo phức tạp hơn và Jacobian đắt hơn.

Có đáng làm không, tính tới tháng 9/2026: dòng BAD chạy *ngoại tuyến*, tối ưu riêng cho từng cảnh, mất từ vài phút tới vài giờ. Đó là NO-GO rõ ràng cho vòng thời gian thực. Giá trị của nó với bạn là mô hình phép đo, không phải phần mềm.



Hình 34.9. Trái: vết mờ là quỹ đạo camera chiếu xuống mặt phẳng ảnh trong thời gian phơi sáng, chứ không phải một ảnh sắc nét bị hỏng. Đây là mô hình mà dòng BAD-NeRF và BAD-Gaussians tối ưu trực tiếp. Phải: hệ quả tức thời cho một hệ cổ điển. Điểm đặc trưng mờ vẫn được định vị tốt *ngang* vết và rất *tệ dọc* vết, nên hiệp phương sai đúng là một ellipse dẹt xoay theo hướng vết — và hướng đó tính được từ con quay, miễn phí.

D. Hướng đúng thứ hai: dùng IMU để dự đoán vệt mờ

Bạn đã có tốc độ góc ở 200 Hz. Bạn cũng có thời gian phơi sáng, hoặc đọc được nó từ trình điều khiển camera. Vậy vệt mờ do xoay dự đoán được ở dạng đóng, trước khi nhìn vào một pixel nào. Ba cách dùng, xếp theo tỉ lệ lợi ích trên công sức.

1. **Đặt trọng số.** Với mỗi khung, tính $s = f\omega\tau$. Với mỗi điểm khớp, dựng Σ dị hướng: trục dài cỡ $s/2$ theo hướng vệt, trục ngắn giữ nguyên giá trị cũ. Đây không phải tinh chỉnh, vì cả hướng lẫn độ lớn đều suy từ mô hình chứ không dò theo kết quả. Và nó kiểm được bằng NEES: nhóm khung có s lớn phải hết bị khai quá tự tin sau khi sửa. Đây là thay đổi rẻ nhất trong cả chương có cơ hội thắng thật.
2. **Ghép cặp.** Làm mờ ô mô tả mẫu theo đúng vệt dự đoán trước khi so, hoặc trải rộng cửa sổ tìm kiếm dọc hướng vệt. Ý là đưa hai ảnh về cùng một mức mờ thay vì cố làm cả hai sắc nét.
3. **Điều khiển.** Đưa s dự đoán ngược về bộ điều khiển phơi sáng, đúng như mục A đã nói. Đây là vòng phản hồi mà rất ít hệ SLAM đóng lại.

Chỗ hỏng của cả ba: chúng chỉ tính phần mờ do xoay. Phần mờ do tịnh tiến cần độ sâu, mà độ sâu thì bạn chỉ có cho các điểm đã tam giác hoá. Với cảnh gần, phần bỏ sót này không nhỏ. Nói thẳng ra là mô hình chỉ đúng khi chuyển động chủ yếu là xoay. May thay, đó cũng chính là chế độ gây mờ nhiều nhất trên thiết bị cầm tay và trên drone.

E. Hướng đúng thứ ba: camera sự kiện, một lối thoát về phía cảm biến

Ba hướng trên đều có sống chung với thời gian phơi sáng. Camera sự kiện xoá bỏ nó.

Cơ chế: mỗi điểm ảnh báo cáo bất đồng bộ khi độ sáng log của nó đổi quá một ngưỡng. Không có khung hình, không có phơi sáng, nên không có mờ do chuyển động. Độ trễ thấp hơn nhiều bậc so với một cảm biến khung hình, và dải động rộng hơn hẳn [370]. Đây là một câu trả lời khác về chất cho cả chuyển động nhanh lẫn cảnh chênh sáng.

Bằng chứng gần nhất với bài toán của bạn là hướng kết hợp sự kiện với khung hình và IMU. Lợi ích tập trung đúng vào các tình huống dải động cao và tốc độ cao, chứ không trải đều [371]. Và có một kết quả đẹp về mặt vật lý: dòng sự kiện trong khoảng phơi sáng liên hệ với ảnh mờ qua một tích phân kép, nên từ một khung mờ cộng dòng sự kiện có thể dựng lại chuỗi ảnh sắc nét [372]. Đó là khứ mờ có căn cứ vật lý, không phải tiên nghiệm học được, và nó không bị ra cạnh.

Cổng đánh giá thì khắt khe. Đây là quyết định *phần cứng*, không phải quyết định thuật toán: thêm cảm biến, thêm hiệu chỉnh, thêm đồng bộ thời gian, và một chuỗi công cụ mỏng hơn hẳn. Có một lỗ hổng cần biết trước: camera sự kiện gần như không thấy gì khi cảnh và camera đều đứng yên. Với robot có lúc dừng, đó là một chế độ vận hành thật. Kết luận thực dụng: chỉ dựng thử nghiệm nếu bạn đã đo được rằng phần lớn thất bại của mình tập trung ở tốc độ cao hoặc chênh sáng. Mục A cho bạn cách đo đúng điều đó bằng phân bố của s .

Bảng 34.4. Bốn cách đối phó với ảnh mờ. Cột cuối cho biết cách nào hỏng ở đâu, và vì sao thứ tự thử nên đi từ rẻ tới đắt.

Cách	Cơ chế	Chi phí và giả định	Hỏng khi nào
Hạ thời gian phơi sáng	Đặt trần τ , bù bằng độ khuếch đại	Gần như miễn phí; giả định cảnh đủ sáng để bù	Thiếu sáng nặng; nhiều cảm biến vượt mức mô tả chịu được
Khử mờ rồi trích đặc trưng	Mạng ảnh-sang-ảnh chạy trước frontend	Nặng nhất trong bốn cách: một mạng ảnh-sang-ảnh toàn độ phân giải đặt trước frontend; giả định một nhân mờ chung	Cạnh bị ra không lặp lại giữa hai ảnh; mờ do tịnh tiến phụ thuộc độ sâu
Mô hình hoá vệt như quỹ đạo	Tích phân phép chiếu trên τ ; dựng Σ dị hướng từ ω và τ	Rẻ nếu chỉ dùng phần trọng số; đắt nếu tích phân đầy đủ	Chuyển động chủ yếu là tịnh tiến với cảnh gần; τ không đọc được
Camera sự kiện	Báo cáo bất đồng bộ theo thay đổi độ sáng; không có phơi sáng	Thêm phần cứng, hiệu chỉnh, đồng bộ; chuỗi công cụ mỏng	Cảnh và camera cùng đứng yên: gần như không có dữ liệu

I Năm nấc đối phó với ảnh mờ, đúng thứ tự nên thử

1 Vấn đề

Giải pháp

Giả định

Giá phải trả

Ảnh mờ làm mất bám ở các đoạn quay nhanh.

tính $s = f\omega\tau$ rồi đặt trần cho thời gian phơi sáng.

cảnh đủ sáng để bù bằng độ khuếch đại.

nhiều cảm biến tăng, và trong bóng tối thì cách này hết đường.

2 Vấn đề

Giải pháp

Giả định

Hệ quả

Trần phơi sáng không đủ, vẫn còn khung mờ.

đừng bỏ khung mờ, hãy hạ trọng số của nó theo s .

bạn đọc được τ và có con quay đồng bộ.

đây là thay đổi rẻ nhất còn lại, và nó kiểm được bằng NEES.

3 Vấn đề

Giải pháp

Giả định

Vấn đề mới

Trọng số vô hướng không tả được kiểu sai số của điểm mờ.

dựng Σ dị hướng, trục dài dọc hướng vệt.

mờ chủ yếu do xoay.

phần mờ do tịnh tiến vẫn bị bỏ sót, và nó phụ thuộc độ sâu.

4 Vấn đề

Giải pháp

Giá phải trả

Ngay cả trọng số đúng cũng không lấy lại được thông tin đã mất.

mô hình hoá phép đo như tích phân trên khoảng phơi sáng, theo cách dòng BAD làm.

mô hình phép đo phức tạp hơn, Jacobian đắt hơn, và các hiện thực hiện có đều ngoại tuyến.

5 Vấn đề

Giải pháp

Giả định

Giá phải trả

Bản thân cảm biến khung hình là nguyên nhân gốc.

đổi sang hoặc thêm camera sự kiện.

bạn đã đo được rằng thất bại tập trung ở tốc độ cao và chênh sáng.

thêm phần cứng, thêm hiệu chỉnh, chuỗi công cụ mỏng, và một lỗ hỏng khi mọi thứ đứng yên.

F. Chuyển động nhanh và ảnh mờ là một bài toán nhìn từ hai phía

Khép mục này bằng một phát biểu gộp. Chuyển động nhanh phá *sự liên kết*: cửa sổ tìm kiếm quanh vị trí dự đoán đặt sai chỗ, và diện mạo của ô ảnh đã đổi. Ảnh mờ phá *phép đo*: vị trí điểm đặc trưng bị lệch có hệ thống, và bất định của nó trở thành dị hướng.

Phía liên kết cũng tính tay được, bằng đúng kiểu phép tính giả định vừa dùng. Với nhịp khung Δt và tốc độ góc ω , tâm của một ô ảnh dịch khoảng $f \omega \Delta t$ pixel giữa hai khung. Giữ nguyên giả định $f = 400$ pixel ở trên, một camera 20 Hz và một cú quay 200 độ mỗi giây cho quãng dịch cỡ 70 pixel — lớn hơn nhiều lần cửa sổ tìm kiếm mặc định của một bộ theo vết thừa. Nghĩa là ở tốc độ đó, việc mất bám không phải do mô tả kém mà do *tìm sai chỗ*, và cách chữa đúng là dùng con quay để dịch tâm cửa sổ tìm kiếm chứ không phải đổi bộ mô tả.

Hai đại lượng $f\omega\Delta t$ và $f\omega\tau$ đến từ cùng một tình huống nhưng chỉ vào hai chỗ sửa khác nhau, và cái thứ nhất lớn hơn cái thứ hai đúng bằng tỉ số $\Delta t/\tau$. Đó là lý do phải đo cả hai phân bố trên dữ liệu của mình trước khi chọn hướng đi.

Vì thế hai nỗi đau này có hai cách chữa khác nhau, và trộn chúng là lỗi hay gặp. Liên kết được chữa bằng dự đoán tốt hơn và mô tả bền hơn, tức địa hạt của [F/28-dac-trung-va-ghep-cap](#) và của dòng quang ở [F/30-do-sau-dong-quang-va-mo-hinh-nen](#). Phép đo được chữa bằng một ma trận thông tin trung thực hơn, tức địa hạt của chính chương này. Chẩn đoán đúng bệnh trước khi mua thuốc là toàn bộ giá trị của việc tách chúng ra.

CẦN NHỚ

Ma trận thông tin là chỗ mà mọi hiểu biết về chất lượng phép đo phải chảy vào. Nếu bạn biết một điều gì đó về một phép đo — ảnh mờ cỡ này, con quay đang quay nhanh, điểm khớp này mơ hồ — mà điều đó không xuất hiện trong Σ , thì bộ ước lượng không biết. Và trước khi thay Σ bằng một mạng, hãy đo NEES: rất nhiều hệ thống đang khai sai vài lần, và sửa hằng số theo phép đo rẻ hơn huấn luyện hai bậc độ lớn. [T1]

34.5 Đưa lên Jetson Orin: phần đặc thù SLAM (Deploying on Orin)

Mục này cố ý không nhắc lại TensorRT, các họ runtime, sai lệch số học sau export, lượng tử hoá, hay chuyện tụt xung vì nhiệt. Tất cả nằm ở [D/21-inference-va-trien-khai](#) và [D/20-toi-uu-va-nen-mo-hinh](#), và chúng đúng nguyên vẹn ở đây. Mục này chỉ nói bốn thứ mà một hệ SLAM có, còn một dịch vụ suy luận thì không.

A. Ngân sách không phải một con số, nó chia theo luồng

Một hệ kiểu ORB-SLAM3 có ba luồng với ba loại hạn chót rất khác nhau [40]. Luồng theo vết chạy mỗi khung hình và có hạn chót cứng bằng chu kỳ khung. Luồng lập bản đồ cục bộ chạy mỗi khung khoá, hạn chót mềm hơn nhiều bậc. Luồng đóng vòng chạy hiếm, nhưng khi nó chạy hiệu chỉnh chùm tia toàn cục thì nó khoá bản đồ.

Hệ quả thiết kế đến trước mọi phép đo. **Câu hỏi đầu tiên không phải mạng chạy hết bao nhiêu mili-giây, mà mạng sống ở luồng nào.** Cùng một mạng có thể là bất khả thi trên luồng theo vết và hoàn toàn thoải mái trên luồng đóng vòng. Khoảng cách giữa hai hạn chót ấy rộng hơn nhiều so với mọi mức tăng tốc bạn có thể mua bằng lượng tử hoá hay bằng một kiến trúc gọn hơn. Nên chọn luồng là biến thiết kế mạnh nhất bạn có, và nó phải được chọn *trước* khi đo bất cứ thứ gì.

Bảng 34.5. Ngân sách chia theo luồng cho một hệ stereo-inertial trên Orin. Bảng cố ý *không có một con số nào*: hạn chót của mỗi luồng suy ra từ nhịp của chính luồng đó, và phần còn trống thì chỉ máy của bạn trả lời được. Cái chuyển được từ hệ này sang hệ khác là thứ tự chặt-lỏng giữa ba dòng; cột cuối nói loại mạng nào có cửa ở mỗi dòng, và phép đo xác nhận nằm ở mục D.

Luồng	Nhịp	Hạn chót	Chỗ một mạng vừa được, và loại mạng nào
Theo vết	mỗi khung hình	Cứng: đúng bằng chu kỳ khung, không co giãn	Phần trống là chu kỳ khung trừ đi trích đặc trưng, ghép cặp và tối ưu tư thế — hãy đo phần đó trước tiên. Chỉ vừa cho mạng rất nhỏ: khử nhiễu con quay, hoặc một đầu ra bất định gắn vào bộ trích sẵn có
Lập bản đồ cục bộ	mỗi khung khoá, tức thừa hơn nhịp khung nhiều lần	Mềm: dài hơn chu kỳ khung nhiều bậc, và trễ một nhịp không làm rớt khung	Vừa cho mạng cỡ trung: hiệp phương sai theo từng điểm khớp cho hiệu chỉnh chùm tia cục bộ, hoặc ước lượng độ sâu cho khung khoá
Đóng vòng	hiếm, và không đều	Rất mềm: cả hệ chịu được một quãng dừng khi vòng đóng	Vừa cho mạng lớn: mô tả toàn ảnh, ghép cặp dày, kiểm chứng hình học học được

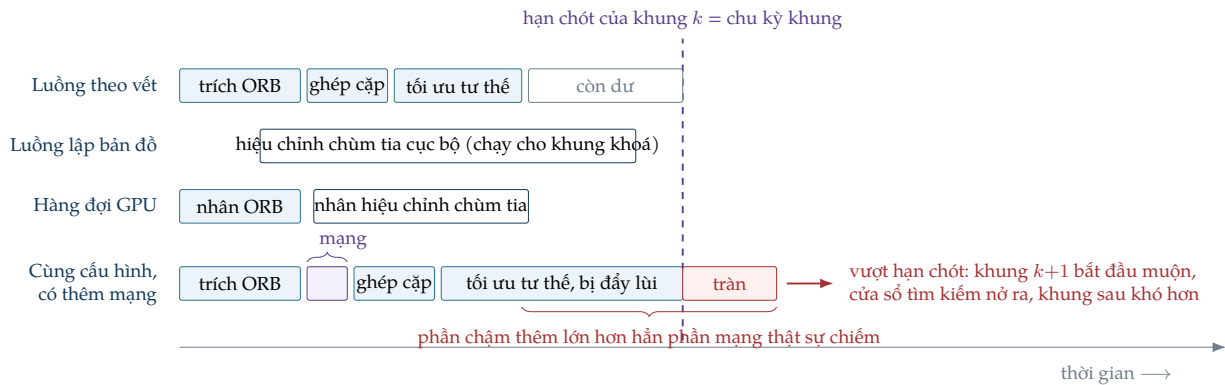
B. GPU không rảnh: mạng tranh chấp với chính bạn

Trong một dịch vụ suy luận, mạng là thứ duy nhất trên GPU. Trong hệ của bạn thì không. Nếu bạn đã đưa trích ORB và hiệu chỉnh chùm tia cục bộ lên GPU, thì một mạng mới không cộng vào một GPU rảnh. Nó xếp hàng sau các nhân của chính bạn. Ba hệ quả:

- **Độ trễ không cộng được.** Thêm một mạng có thể làm luồng theo vết chậm đi *nhiều hơn* đúng phần thời gian mà mạng chiếm khi chạy một mình, vì các nhân xếp hàng và vì cả hai tranh băng thông bộ nhớ. Trên Orin, CPU và GPU dùng chung bộ nhớ, nên tranh chấp xảy ra cả với phần chạy trên CPU.
- **Phép đo cô lập gần như vô dụng.** Độ trễ của mạng đo khi nó chạy một mình không nói gì. Đại lượng đúng là *mức thay đổi p99 của chu kỳ luồng theo vết* khi bật mạng lên, đo trong hệ đầy đủ. Chỉ có hiệu số đó mới trả lời được câu hỏi “có vừa không”.
- **Có ba đòn bẩy để giảm tranh chấp,** và cả ba là quyết định kiến trúc chứ không phải tối ưu: đặt mạng lên DLA để chừa GPU cho phần khác, kèm cảnh báo ở [D/21](#) rằng toán tử không hỗ trợ sẽ rơi ngược về GPU; dùng dòng CUDA có mức ưu tiên để nhân của luồng theo vết chen lên trước; hoặc chạy mạng ở tần suất thấp hơn tốc độ khung.

Đòn bẩy thứ ba có một điều kiện hay bị bỏ qua. Chạy thừa là *hợp lệ với một hiệp phương sai* và *không hợp lệ với một tư thế*. Một ước lượng bất định thay đổi chậm và giữ giá trị cũ vài khung không sai nhiều. Một tư thế thì không.

Hình [34.10](#) vẽ ra cùng một cảnh trên trục thời gian, và nó cho thấy điều mà bảng không cho thấy: chỗ tranh chấp không nằm trong luồng nào cả, nó nằm ở hàng đợi GPU dùng chung.



Hình 34.10. Vì sao độ trễ không cộng được. Trục thời gian cổ ý không có nhãn số: hình này nói về *thứ tự* và về chỗ tranh chấp, không về độ dài. Hàng thứ ba là hàng đợi GPU dùng chung: nhân của mạng chen vào giữa nhân trích ORB và nhân hiệu chỉnh chùm tia, nên phần tối ưu tư thế ở hàng dưới cùng bị đẩy lùi nhiều hơn hẳn phần thời gian mà mạng thật sự chiếm. Đó là lý do phải đo mức thay đổi p99 của cả chu kỳ, chứ không đo riêng mạng.

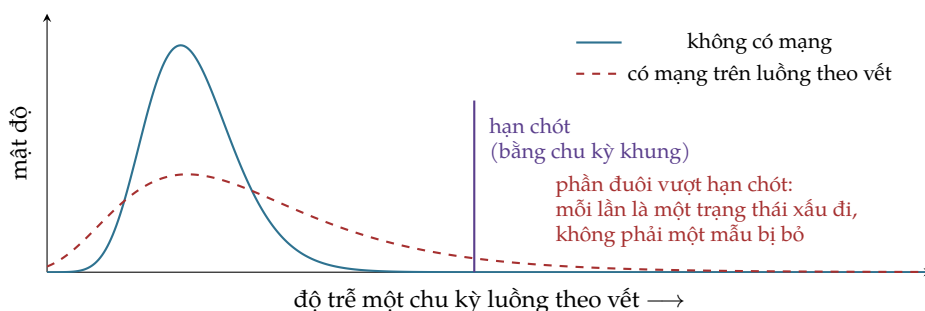
C. Độ trễ đuôi quan trọng hơn độ trễ trung bình

Chương 21 đã nói phải báo phân vị thay vì trung bình. Với SLAM, lý do mạnh hơn một bậc, và có ba tầng.

Một khung trễ không phải một khung mất. Nó đổi trạng thái của bộ ước lượng. Luồng theo vết thiếu một quan sát, cửa sổ tìm kiếm cho khung sau nở ra, nên khung sau khó hơn. Hàm chi phí của việc vượt hạn chót vì thế có trí nhớ và lỗi lên: hai lần trễ liên tiếp đắt hơn nhiều hai lần trễ rời rạc.

Đuôi lan qua khoá. Ba luồng dùng chung bản đồ sau một mutex. Một đỉnh trễ ở luồng lập bản đồ cục bộ, hoặc một lần hiệu chỉnh chùm tia toàn cục ở luồng đóng vòng, sẽ chặn luồng theo vết truy cập bản đồ. Nghĩa là một sự kiện đuôi ở luồng *mềm* biến thành một lần lỡ hạn chót ở luồng *cứng*. Đây là lớp lỗi mà đo từng luồng riêng không bao giờ thấy. Bất kỳ ai từng gỡ một lần kẹt giữa lập bản đồ và hiệu chỉnh toàn cục đều nhận ra nó.

Mạng làm đuôi béo thêm. Thời gian chạy của một mạng phụ thuộc dữ liệu nhiều hơn mã C++ thuần: số hộp trước bước hậu xử lý, hình dạng động, lần đầu gặp một kích thước mới làm bộ cấp phát chạy lại. Nên đuôi của hệ có mạng dày hơn đuôi của hệ không có mạng, kể cả khi trung bình bằng nhau.



Hình 34.11. Vì sao trung bình nói dối trong hệ thời gian thực. Trục không có nhãn số vì hình này nói về *hình dạng đuôi*, không về độ trễ của một hệ cụ thể — độ trễ thì bạn đo bằng harness của mình. Hai phân bố có trung vị gần nhau, nhưng phân bố có mạng dày đuôi hơn hẳn, nên phần diện tích vượt hạn chót lớn hơn nhiều lần. Trong SLAM, mỗi lần vượt hạn chót không độc lập với lần sau: nó làm khung kế tiếp khó hơn, nên chi phí tích lũy chứ không cộng.

D. Đo lại cho trung thực bằng chính harness lặp nhiều lần của bạn

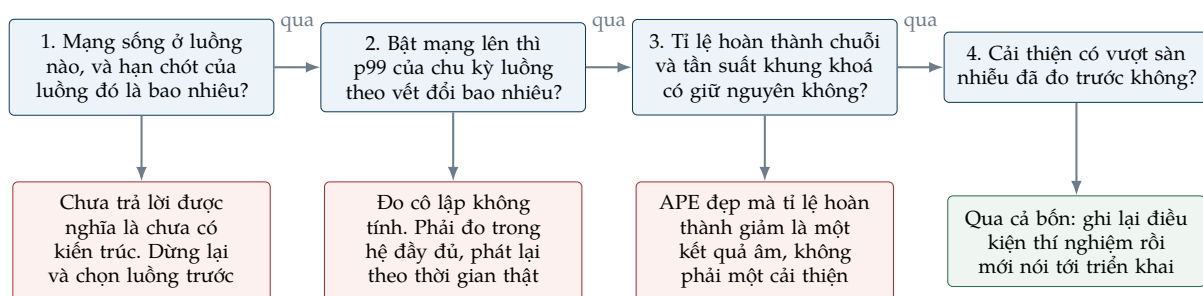
Bạn đã có công cụ đúng: một harness chạy lặp, và thói quen đo sần nhiều trước khi so sánh. Mục này chỉ nói năm điều chỉnh cần thêm khi vật được đo là một mạng trong một hệ thời gian thực.

1. **Coi “bật/tắt mạng” là đúng một yếu tố ablation, và đo sần nhiều trước.** Toàn bộ kỹ thuật ở [E/24-thi-nghiem-va-ablation](#) áp nguyên vào đây. Một lần chạy không nói gì, và sần nhiều của một hệ có mạng thường rộng hơn sần nhiều của hệ cũ.
2. **Đừng chạy lockstep khi đo độ trễ.** Chế độ nạp khung theo nhịp hệ thống tiêu thụ là công cụ đúng để cô lập tính phi tắt định, và là công cụ sai để đánh giá thời gian thực. Nó biến một lần lỡ hạn chót thành một lần chạy chậm, tức xoá đúng thứ bạn đang muốn đo. Phát lại bag theo thời gian thật, và đếm số khung bị bỏ.
3. **Ghi thời gian theo luồng, không chỉ ghi APE.** Tối thiểu: p50 và p99 của chu kỳ luồng theo vết, tần suất khung khoá, thời gian hiệu chỉnh chùm tia cục bộ, số lần mất bám. Một mạng có thể làm APE đẹp lên trong khi bóp tần suất khung khoá, và con số APE tổng che kín điều đó.
4. **Báo tỉ lệ hoàn thành chuỗi cùng với APE.** Một hệ mất bám giữa chừng rồi khởi tạo lại có thể cho APE rất đẹp trên phần nó còn bám được. Đây là cái bẫy đọc số đã nêu ở [B/07-chia-du-lieu-va-danh-gia](#), và nó xuất hiện nhiều nhất đúng ở các cấu hình có mạng.
5. **Chạy đủ dài, ở đúng chế độ nguồn.** Quy trình mười phút liên tục của [D/21](#) áp nguyên, với một điều chỉnh: tải của SLAM phụ thuộc dữ liệu, nên phải phát lại một bag thật lặp vòng, không phải một vòng lặp suy luận tổng hợp.

→ **tiền đề D/21** — mọi kỹ thuật export, chọn runtime và đo dưới tải nhiệt nằm ở chương 21; mục này chỉ thêm phần mà kiến trúc đa luồng của SLAM sinh ra

E. Bốn câu hỏi trước khi bật một mạng trong vòng thời gian thực

Gộp bốn mục trên thành một cây quyết định chạy được trong đầu. Bốn câu hỏi dưới đây phải trả lời theo đúng thứ tự, vì một câu trả lời “không” ở câu trước làm ba câu sau vô nghĩa.



Hình 34.12. Bốn cổng cho một mạng trong vòng thời gian thực, theo đúng thứ tự phải hỏi. Ba cổng đầu là câu hỏi kỹ thuật hệ thống và không liên quan gì tới chất lượng của mạng; chỉ cổng thứ tư mới hỏi mạng có tốt không. Thứ tự này rẻ hơn thứ tự ngược lại, vì phần lớn ứng viên chết ở cổng một hoặc cổng hai, nơi chi phí kiểm tra gần bằng không.

Điểm đáng chú ý ở Hình 34.12 là ba cổng đầu không hỏi gì về chất lượng của mạng. Chúng là câu hỏi kỹ thuật hệ thống thuần túy. Trả lời chúng trước tiết kiệm được rất nhiều thời gian, vì phần lớn ứng viên chết ở cổng một hoặc hai, và hai cổng đó rẻ hơn cổng bốn hàng chục lần.

34.6 Lộ trình học cho riêng hướng này (A Learning Path for This Direction)

Chương [E/26-lo-trình-hoc](#) đã cho sáu lộ trình theo mục tiêu, và lộ trình D là “áp dụng vào SLAM production”. Mục này nối tiếp lộ trình đó ở chỗ nó dừng lại: bạn đã học xong nền, giờ đọc tám chương của Phần F theo thứ tự nào.

A. Tuần 0: đo trước khi đọc

Điều bất thường của lộ trình này là nó bắt đầu bằng ba phép đo chứ không bằng một chương. Lý do đơn giản: bảy chương còn lại của Phần F cùng hứa chữa những nỗi đau khác nhau, và bạn chỉ có ngân sách cho một. Ba phép đo dưới đây, tất cả gói trong hai tới ba ngày, quyết định bạn đọc gì.

- **NEES theo từng loại nhân tử**, trên một chuỗi dễ và một chuỗi khó. Nó nói trọng số hiện tại sai bao nhiêu lần và sai ở đâu.
- **Phân bố vết mờ dự đoán** $s = f\omega\tau$, và tương quan của nó với tỉ lệ điểm khớp đúng. Nó nói ảnh mờ có thật là nút thất hay không.
- **Hồ sơ độ trễ theo luồng**, p50 và p99 cho từng luồng, trên phần cứng thật. Nó nói bạn còn bao nhiêu ngân sách và ở luồng nào.

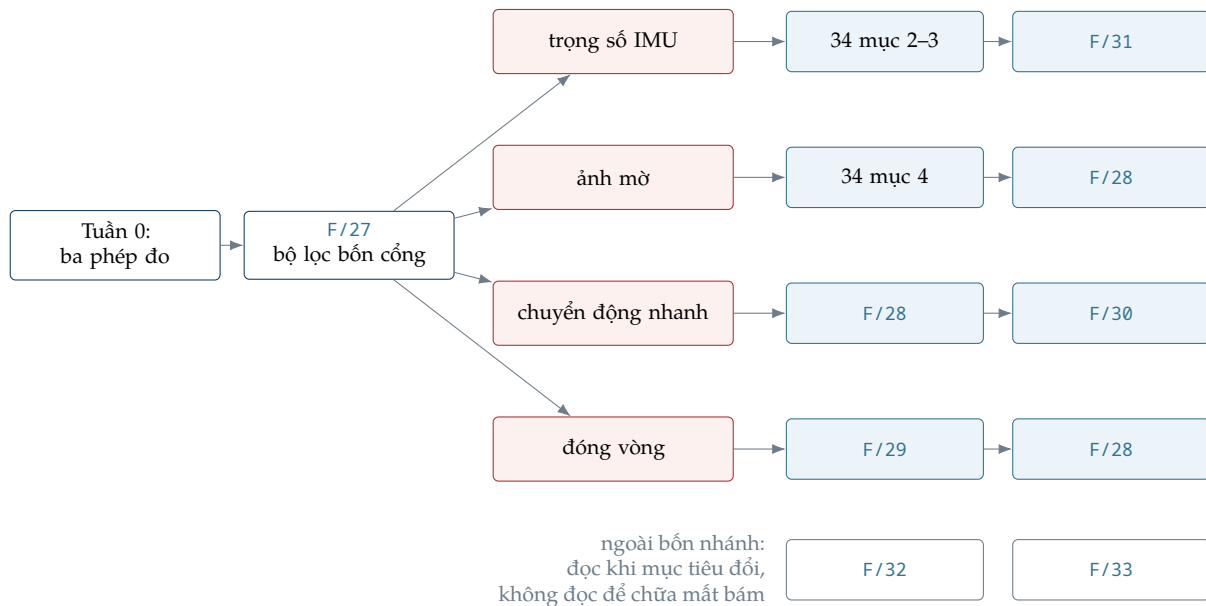
Nếu ba phép đo này nói trọng số của bạn đã trung thực, ảnh mờ hiếm, và không còn ngân sách, thì kết luận đúng là không đọc tiếp chương nào cả. Và đó cũng là một kết quả.

B. Thứ tự đọc tám chương theo bốn nỗi đau

Bảng 34.6. Đọc Phần F theo nỗi đau đang gặp, không đọc tuần tự. Chương 27 luôn đọc trước vì nó là bộ lọc. Cột cuối nêu chương *không* nên đọc cho nỗi đau đó, để tiết kiệm thời gian.

Nỗi đau	Thứ tự đọc	Bạn tìm gì trong đó	Bỏ được
Trọng số IMU	F/27 → 34 mục 2–3 → F/31	Phép thử NEES, khuôn “phép đo kèm Σ ”, và “khả vi” mua được gì ở backend	32, 33
Ảnh mờ	F/27 → 34 mục 4 → F/28	Vết mờ tính ra pixel, Σ dị hướng, rồi mới tới mô tả bền với mờ	31, 32, 33
Chuyển động nhanh	F/27 → F/28 → F/30 → 34 mục 4	Ghép cặp bên, dòng quang thay ghép cặp thừa, rồi phần bất định	32, 33
Đóng vòng	F/27 → F/29 → F/28	Bốn tầng của đóng vòng, và tầng nào thật sự giết vòng	30, 31, 32, 34

Hình [34.13](#) vẽ lại cùng bảng đó dưới dạng bản đồ, để thấy hai chương nào nằm ngoài mọi nhánh.



Hình 34.13. Bản đồ đọc Phần F theo nỗi đau. Ba phép đo của tuần 0 quyết định vào nhánh nào, và chương 27 lọc tiếp. Mỗi nhánh chỉ hai tới ba chương, nên không nhánh nào cần đọc quá một nửa Phần F. Hai chương ở dưới cùng nằm ngoài mọi nhánh: chúng phục vụ mục tiêu đối biểu diễn bản đồ và thêm hiểu biết về cảnh, không phục vụ việc chống mất bám.

Hai chương [F/32-ban-do-neural](#) và [F/33-ngu-nghia-va-do-thi-can](#) không phục vụ bốn nỗi đau này. Chúng phục vụ một mục tiêu khác: đối biểu diễn bản đồ, hoặc thêm hiểu biết về cảnh. Đọc chúng khi mục tiêu đổi, chứ không đọc để chữa mất bám.

C. Mốc thời gian thực tế, và kết cục nhiều khả năng nhất

Ước lượng dưới đây giả định mười giờ mỗi tuần và giả định bạn đã có hệ thống chạy được. Nó là phán đoán về thứ tự phụ thuộc, không phải số đo.

- **Tuần 0**, hai tới ba ngày: ba phép đo ở mục A.
- **Tuần 1–2**: [F/27](#) và mục 2–3 của chương này. Kết quả cụ thể phải có: một bảng NEES, và một quyết định về hằng số trọng số dựa trên bảng đó.
- **Tuần 3–5**: đúng *một* chương theo nỗi đau đã chọn, cộng một lần dựng lại một phương pháp trong đó cho chạy ngoại tuyến trên dữ liệu của bạn. Ngoại tuyến trước, để tách câu hỏi “nó có đúng không” khỏi câu hỏi “nó có kịp không”.
- **Tuần 6–8**: một thí nghiệm có cổng. Bật/tắt, nhiều lần chạy, sắn nhiều đo trước, ghi cả thời gian theo luồng lẫn tỉ lệ hoàn thành.
- **Sau đó**: chỉ đi tiếp nếu cổng đã qua.

Hai tháng để tới một quyết định. Kết cục nhiều khả năng nhất, và cần nói thẳng, là một NO-GO kèm lý do đo được. Đó không phải hai tháng bỏ đi. Một NO-GO ghi đủ điều kiện thí nghiệm là thứ ngăn bạn thử lại cùng ý tưởng vào năm sau, và là thứ cho bạn biết điều kiện nào đổi thì đáng thử lại. Đúng như [E/24-thi-nghiem-va-ablation](#) lập luận.

34.7 Giới hạn và trường hợp hỏng

Giới hạn lớn nhất của cả chương là giả định rằng nhiều *có thể* mô tả được bằng một hiệp phương sai Gauss. Nhiều chế độ hỏng thật thì không. Một điểm khớp sai ghép sang cấu trúc lặp lại không

phải một mẫu Gauss lệch xa; nó là một mẫu từ một phân phối khác hẳn. Với loại đó, cách đúng không phải nói Σ mà là loại bỏ, bằng kiểm chứng hình học hoặc bằng hàm mất mát bền vững. Nói Σ cho một điểm ngoại lai chỉ làm nó gây hại chậm hơn. Ranh giới thực dụng: nếu phần dư còn nằm trong vài lần σ , đó là chuyện của trọng số; nếu nó ở ngoài, đó là chuyện của loại bỏ.

Chế độ hỏng thứ hai thuộc về chính bất định học được. Suy dẫn ở mục 3 cho thấy nhánh phương sai chỉ hồi quy bình phương sai số kỳ vọng theo đầu vào. Ra ngoài phân phối huấn luyện, nó vẫn xuất ra một số nhỏ, và bộ ước lượng nhận số đó như một lời cam đoan. So với một hằng số dờ, một mạng tự tin sai còn tệ hơn. Hằng số dờ thì sai đều và bạn bù được; mạng thì sai đúng vào những cảnh lạ, tức đúng những cảnh bạn cần được cảnh báo nhất. Cách phòng rẻ duy nhất là giữ một chặn dưới cứng cho Σ , tức không bao giờ để mạng khai bất định nhỏ hơn giá trị vật lý của cảm biến.

Chế độ hỏng thứ ba nằm ở phép thử NEES chứ không ở phương pháp. NEES giả định mô hình phép đo *đúng dạng* và chỉ sai về độ lớn. Nếu bạn có một sai lệch thời gian chưa hiệu chỉnh, hoặc hiệu chỉnh ngoại sai, thì phần dư mang một thành phần có hệ thống. NEES lớn hơn bậc tự do sẽ dụ bạn nói Σ , và nói Σ khi đó là giấu một lỗi hiệu chỉnh vào trọng số. Cách phân biệt là nhìn phần dư có tương quan với chuyển động không: nếu phần dư ảnh tỉ lệ với tốc độ góc, bạn đang nhìn một t_d sai, không phải nhiễu.

Chế độ hỏng thứ tư là chỗ chương này nói ít nhất: mờ do tịnh tiến. Toàn bộ phần dự đoán vệt mờ ở mục 4 chỉ dùng con quay, nên nó bỏ sót phần mờ phụ thuộc độ sâu. Với cảnh xa thì bỏ sót đó không đáng kể. Với cảnh gần và chuyển động tịnh tiến nhanh, mô hình sẽ đánh giá thấp vệt mờ, và nó lại sai theo hướng quá tự tin. Tôi không biết một cách rẻ nào để sửa điều này mà không cần độ sâu tin cậy trước khi ghép cặp.

Cuối cùng, một giới hạn về bằng chứng. Nhiều công trình được nhắc trong chương này rất mới, và tôi mô tả cơ chế chứ không mô tả kết quả số. Với các công trình từ 2024 trở đi, hãy coi phần đánh giá “có đáng không” của tôi là một phán đoán dựa trên cấu trúc chi phí, không phải kết luận từ một lần dựng lại. Nếu bạn dựng lại và thấy khác, con số của bạn thắng.

34.8 Thực hành

Repo và tài nguyên. [evo](#) cho APE và RPE; nhớ báo RPE ở nhiều độ dài cửa sổ chứ không chỉ APE. [ethz-asl/kalibr](#) cho hiệu chỉnh camera-IMU và cho sai lệch thời gian t_d . Một t_d chưa hiệu chỉnh trông hết như trọng số IMU sai, nên hãy loại trừ nó trước. Công cụ tính phương sai Allan từ một bag ROS đứng yên nhiều giờ thì đơn giản tới mức tự viết cũng được. Về phía học sâu, mã nguồn của TLIO, RoNIN và MAC-V0 đều công khai; hãy đọc phần định nghĩa hàm mất mát trước phần kiến trúc, vì đó là chỗ chứa toàn bộ quyết định thiết kế. [kornia](#) có sẵn các phép biến đổi ảnh khả vi nếu bạn muốn thử mô hình mờ dạng tích phân. Danh sách này chốt tại tháng 9/2026 và tên mô hình trong đó sẽ cũ trong sáu tới mười hai tháng; các công cụ hạ tầng như [evo](#) và [ethz-asl/kalibr](#) thì bền hơn nhiều.

Câu hỏi kiểm tra hiểu. NEES trung bình của nhân tử ảnh trên chuỗi khó gấp k lần bậc tự do: σ của bạn đang sai bao nhiêu lần, và vì sao con số đó vẫn còn là bản lạc quan? Nếu tăng thời gian phơi sáng gấp đôi thì độ dài vệt mờ đổi bao nhiêu, và tỉ lệ tín hiệu trên nhiễu đổi bao nhiêu, hai đại lượng đó có cùng bậc không? Một mạng khử mờ làm PSNR tăng rõ rệt nhưng số điểm khớp đúng lại giảm: hai cơ chế nào giải thích được, và phép thử nào phân biệt chúng?

Mini project. Hai bài ở hai khối bên dưới. Làm bài thứ nhất trước, vì bài thứ hai dùng lại đúng bộ đo mà bài thứ nhất dựng ra.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ma trận thông tin	Nghịch đảo hiệp phương sai, $\Omega = \Sigma^{-1}$, tức trọng số của một ràng buộc trong đồ thị nhân tử. Chỉ tỉ số giữa các Ω mới đối nghiệm, nên siết IMU và nối ảnh là cùng một thao tác.
••• Vì sao sai một bậc về σ lại nghiêm trọng?	Vì trọng số đi với $1/\sigma^2$. Sai mười lần về độ lệch chuẩn là sai một trăm lần về trọng số, đủ để bộ ước lượng bỏ hẳn một nguồn thông tin.
••• Quá tin IMU	Σ_{imu} khai quá nhỏ. Độ lệch con quay không được sửa, sai số góc nghiêng trọng lực, và sai số vị trí lớn theo $\frac{1}{6} g e t^3$. Chữ ký: APE trôi mượt trong khi RPE ngắn hạn vẫn đẹp.
••• Quá tin ảnh	$\Sigma_{\text{ảnh}}$ khai quá nhỏ. Khung mờ có toàn quyền kéo nghiệm; phần giặt chảy ngược vào ước lượng độ lệch và làm dự đoán khung sau tệ hơn. Chữ ký: RPE ngắn hạn xấu đều.
•• Phương sai Allan	Cách tách nhiễu trắng, độ trôi của độ lệch và bước đi ngẫu nhiên từ dữ liệu ghi lúc đứng yên nhiều giờ. Nó cho một <i>sân</i> , không cho giá trị hiệu dụng lúc vận hành.
••• σ hiệu dụng	Giá trị mà đồ thị nhân tử thật sự cần. Nó phải nuốt cả sai lệch thời gian, sai số hiệu chỉnh ngoại, lệch trục và sai số rời rạc hoá, nên luôn lớn hơn datasheet.
••• NEES	Trung bình của $r^T \Sigma^{-1} r$; nếu Σ trung thực thì nó bằng số bậc tự do. Đây là phép đo biến câu hỏi “trọng số của tôi có đúng không” thành một con số, và nó không cần chân trị.
••• NEES trung bình gấp k lần bậc tự do — nghĩa là gì?	σ của bạn nhỏ hơn thực tế $\sqrt{k}$ lần, tức bạn đang tự tin hơn mức được phép đúng $\sqrt{k}$ lần. Phân biệt với lỗi hiệu chỉnh: nếu phần dư tỉ lệ với tốc độ góc thì đó là sai lệch thời gian t_d , không phải nhiễu.
••• Độ hợp lý âm cho hiệp phương sai	Hàm mất mát $\frac{1}{2}(e^2/\sigma^2 + \log \sigma^2)$. Số hạng đầu phạt tự tin mà sai, số hạng sau phạt né bằng cách khai bất định lớn. Cực tiểu nằm tại $\sigma^2 = e^2$.
••• Bất định học được thực chất là gì?	Một bài hồi quy bình phương sai số lên chính đầu vào. Vì thế nó chỉ đúng cho loại sai số đã thấy lúc huấn luyện, và ra ngoài phân phối nó vẫn khai bất định nhỏ.
•• TLIO	Mạng hồi quy dịch chuyển ba chiều kèm hiệp phương sai từ một cửa sổ IMU, rồi đưa cặp đó vào bộ lọc như một phép đo giả. Bài học: mạng cấp phép đo, bộ lọc vẫn là bộ ước lượng.
• RoNIN	Hồi quy vận tốc chỉ từ IMU trong hệ quy chiếu không phụ thuộc hướng, kèm một bộ dữ liệu chuẩn. Độ chính xác của nó đến từ tiên nghiệm đáng đi, nên nó không chuyển sang drone hay xe.
•• Khử nhiễu con quay bằng mạng	Mạng tích chập giãn xuất lượng hiệu chỉnh cộng vào phép đo con quay, huấn luyện trên số gia xoay tích lũy. Nó sửa phép đo chứ không sửa trạng thái, nên backend giữ nguyên.
••• MAC-VO	Bộ ghép cặp xuất bất định cho từng điểm khớp, đưa qua hình học phép chiếu thành hiệp phương sai ba chiều đúng đơn vị, rồi dùng nó làm trọng số cho từng ràng buộc ảnh.
••• Vệt mờ	Độ dài $s \approx f \omega \tau$ pixel, tuyến tính theo cả tốc độ góc lẫn thời gian phơi sáng. Với các giá trị f và τ thường gặp, một cú quay nhanh đủ đưa s lên cỡ nửa ô mô tả ORB rộng 31 px, và lúc đó cấu trúc trong ô gần như mất hết. Thay f và τ của camera mình vào rồi vẽ phân bố của s .
••• Vì sao khử mờ rồi trích đặc trưng thường không giúp?	Mạng khử mờ tối ưu cho mắt người nên bịa cạnh, và sai số của nó phụ thuộc nhân mờ nên khác nhau giữa hai ảnh. Kết quả là một độ lệch tương quan với chuyển động, chứ không phải nhiễu.
•• BAD-NeRF và BAD-Gaussians	Coi ảnh mờ là trung bình của các ảnh dựng từ tư thế dọc quỹ đạo trong thời gian phơi sáng, rồi tối ưu đồng thời cảnh và quỹ đạo đó. Ngoại tuyến, nhưng mô hình phép đo thì dùng lại được.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
•• Σ dị hướng cho điểm mờ	Điểm đặc trưng mờ được định vị tốt ngang vệt và tẻ dọc vệt, nên hiệp phương sai đúng là ellipse dẹt xoay theo hướng vệt. Hướng và độ dài suy từ con quay, không cần mạng.
• Camera sự kiện	Cảm biến báo cáo bất đồng bộ khi độ sáng đổi; không có phơi sáng nên không có mờ, độ trễ thấp hơn nhiều bậc so với cảm biến khung hình, dải động rộng hơn hẳn. Nó không thấy gì khi cảnh và camera cùng đứng yên.
••• Vì sao độ trễ đo riêng của một mạng chưa nói được nó có vừa ngân sách không?	Vì nó tranh GPU và băng thông bộ nhớ với trích đặc trưng và hiệu chỉnh chùm tia của chính bạn, nên độ trễ không cộng được. Đại lượng đúng là mức thay đổi p99 của chu kỳ luồng theo vệt khi bật mạng, đo trong hệ đầy đủ.
••• Đuôi độ trễ trong SLAM	Một khung trễ đổi trạng thái bộ ước lượng chứ không chỉ bị bỏ, nên chi phí tích lũy. Và một đỉnh trễ ở luồng mềm chặn luồng cứng qua khoá bản đồ, điều mà đo từng luồng riêng không thấy.
•• Vì sao không đo độ trễ ở chế độ lockstep?	Vì nạp khung theo nhịp hệ thống tiêu thụ biến một lần lỡ hạn chót thành một lần chạy chậm. Lockstep đúng cho việc cô lập tính phi tất định và sai cho việc đánh giá thời gian thực.
••• APE đẹp lên nhưng tỉ lệ hoàn thành giảm — nghĩa là gì?	Hệ mất bám giữa chừng và bạn đang đo APE trên phần dễ còn lại. Luôn báo tỉ lệ hoàn thành cùng APE, và báo RPE ở vài độ dài cửa sổ.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [40]	Ước lượng độ lệch cảm biến trực tuyến, và trọng số ảnh theo tầng kim tự tháp — đúng hai chỗ mà mục 2 chỉ ra là ô trống	Hạ tầng; chính là hệ bạn đang bảo trì
VINS-Mono [360]	Khuyến nghị nhân hệ số nhiễu IMU lên so với giá trị đo, tức thừa nhận σ hiệu dụng lớn hơn datasheet	Thông lệ đã thành chuẩn trong cộng đồng VIO
Tiền tích phân trên đa tạp [216]	Sinh ra hiệp phương sai của nhân tử IMU; mọi bàn luận về trọng số trong chương đều xoay quanh ma trận đó	Nền của gần như mọi hệ VIO hiện đại
TLIO [224]	Mạng cấp phép đo kèm hiệp phương sai, bộ lọc vẫn giữ trạng thái — khuôn cấm mạng vào hệ cổ điển	Được dùng lại rộng rãi trong dòng đo quán tính học được
MAC-V0 [302]	Hiệp phương sai theo từng điểm khớp, mang đúng đơn vị đo, làm trọng số cho từng ràng buộc ảnh	Rất mới; mô tả cơ chế, chưa kiểm chứng số
BAD-Gaussians [368]	Mô hình hoá vệt mờ như quỹ đạo trong thời gian phơi sáng, tối ưu đồng thời cảnh và tư thế	Ngoại tuyến; giá trị với bạn là mô hình phép đo

MINI PROJECT SLAM • Kiểm tra ma trận thông tin của bạn có trung thực không • 1–2 ngày

- Mục tiêu** biến câu “trọng số IMU của tôi sai” thành một con số cho từng loại nhân tử và từng chuỗi. Đây là bài phải làm trước mọi thứ khác trong chương.
- Dữ liệu** hai chuỗi EuRoC có độ khó khác nhau, ví dụ một chuỗi dễ và V2_03 [210]; cấu hình mặc định; ba lần chạy mỗi chuỗi để thấy độ tàn mát.

Các bước	ghi lại mọi phần dư ở cuối một lần hiệu chỉnh chùm tia cục bộ, kèm hiệp phương sai bạn khai cho nó; tính $r^\top \Sigma^{-1} r$ cho từng loại nhân tử; lấy trung bình rồi chia cho số bậc tự do, tức 2 cho phép chiếu và 9 hoặc 15 cho tiền tích phân tùy bạn có tính hai độ lệch không; lặp lại nhưng chia điểm khớp thành nhóm theo tầng kim tự tháp và theo vết mờ dự đoán $s = f\omega\tau$.
Xong khi	bạn có một bảng NEES với một dòng cho mỗi loại nhân tử và mỗi chuỗi, và phát biểu được câu “trên chuỗi này, σ của nhân tử X đang nhỏ hơn thực tế k lần”. Kiểm tra chéo: tỉ số phải lớn hơn ở chuỗi khó. Nếu bằng nhau thì nhiều khả năng bạn đang đo phần dư sau tối ưu và cần chuyển sang phần dư dự đoán.
Nối với	E/24 cho sàn nhiễu và số lần chạy tối thiểu; B/07 cho cách báo con số kèm khoảng; bảng này là đầu vào của mini project thứ hai.

MINI PROJECT SLAM · Vết mờ dự đoán từ IMU làm trọng số cho ảnh · 2 ngày

Mục tiêu	kiểm xem một Σ ảnh phụ thuộc vết mờ có thắng được trọng số theo tầng kim tự tháp hay không, và làm điều đó mà không tinh chỉnh một hằng số nào.
Dữ liệu	cùng hai chuỗi của bài trước, cộng một chuỗi TUM-VI có đoạn quay nhanh nếu bạn có [211] .
Các bước	với mỗi khung, tính $s = f\omega\tau$ từ con quay và thời gian phơi sáng; vẽ phân bố của s và vẽ tương quan giữa s với tỉ lệ điểm khớp đúng; dựng Σ dị hướng có trục dài $s/2$ theo hướng vết và trục ngắn giữ nguyên; chạy lại, so với đường <i>baseline</i> , mỗi cấu hình lặp theo số lần mà bài trước đã tính ra.
Xong khi	bạn có đồ thị phân tán s so với tỉ lệ điểm khớp đúng, bảng NEES <i>sau khi sửa</i> cho thấy nhóm khung có s lớn không còn bị khai quá tự tin, và một so sánh APE cộng RPE cộng tỉ lệ hoàn thành, đặt cạnh sàn nhiễu. Một kết luận NO-GO kèm ba con số đó cũng là hoàn thành bài.
Nối với	F/28 nếu bước tiếp theo là đổi bộ mô tả; D/21 nếu chi phí tính s và dựng Σ trở thành vấn đề độ trễ.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì hai kiểu hỏng có hai phổ tần khác nhau. Quá tin IMU cho sai số trôi mượt theo lũy thừa của thời gian, nên APE xấu trong khi RPE trên cửa sổ ngắn vẫn đẹp. Quá tin ảnh cho sai số giật bị chặn, nên RPE ngắn hạn xấu ở mọi chỗ. Phép đo tách chúng là báo RPE ở vài độ dài cửa sổ bên cạnh APE.
- A2. Vì datasheet trả lời câu hỏi “con chip này nhiễu bao nhiêu trong phòng thí nghiệm”, còn đồ thị nhân tử hỏi “phép đo này sai bao nhiêu trên thiết bị đang chạy”. Giá trị thứ hai phải nuốt thêm sai lệch thời gian, sai số hiệu chỉnh ngoại, lệch trục, rung gập tần và sai số rời rạc hoá. Nó cũng đổi theo nhiệt độ và theo tốc độ chuyển động, nên không có hằng số nào đúng cho mọi chế độ.
- A3. Nó cho một ước lượng *độ tin cậy* phụ thuộc đầu vào. Suy dẫn từ độ hợp lý âm cho thấy nhánh phương sai hội tụ về bình phương sai số kỳ vọng có điều kiện, tức đúng đại lượng mà một hằng số datasheet không thể biểu diễn. Đó là ô trống thật của hệ cổ điển, còn ước lượng tư thế thì không phải.
- A4. Vì mạng khử mờ tối ưu cho cảm nhận của mắt người, và có một đánh đổi đã chứng minh giữa “trông sắc” và “trung thực”. Cảnh do mạng bịa ra không lặp lại được, và sai số của mạng phụ thuộc nhân mờ của từng ảnh nên nó tương quan với chuyển động. Kết quả là một độ lệch có hệ thống, đúng loại sai số mà bộ ước lượng tư thế không chịu được.
- A5. Vì đó là con số đo cô lập, còn mạng chạy trong một hệ mà GPU và băng thông bộ nhớ đã bị trích đặc trưng và hiệu chỉnh chùm tia chiếm. Đại lượng đúng là mức thay đổi p99 của chu kỳ luồng theo vết khi bật mạng, đo ở tốc độ phát lại thật chứ không phải lockstep, kèm tỉ lệ hoàn thành chuỗi.

NÓI LẠI THẬT ĐƠN GIẢN

Hai người cùng chỉ đường cho bạn trong một chuyến lái xe dài. Người thứ nhất chỉ nhìn đồng hồ và cảm giác quán tính. Anh ta rất nhất quán trong từng phút, nhưng sau nửa giờ thì lệch hẳn mà không tự biết. Người thứ hai nhìn biển báo bên đường. Anh ta không lệch dân, nhưng thỉnh thoảng biển bị che hoặc nhòe và anh ta đọc bừa.

Câu hỏi đúng không phải ai nói đúng. Câu hỏi là bạn nên nghe mỗi người bao nhiêu phần. Và đây là chỗ mọi người làm sai. Họ lấy con số ghi trong sách hướng dẫn của cái đồng hồ, đo trong phòng lạnh, đứng yên, để quyết định mức tin cho một cái đồng hồ đang rung trên xe nóng.

Cái mà máy học thêm vào đây không phải một người chỉ đường thứ ba giỏi hơn. Nó là một người biết tự nói mình đang chắc bao nhiêu, thay vì luôn nói bằng một giọng đều. Nó nhìn vào chính dữ liệu và nói: đoạn này tôi chắc, đoạn này tôi đoán.

Cái giá thì rõ và không né được. Người đó chỉ biết mình chắc bao nhiêu trên những con đường đã từng đi. Ra một con đường lạ, anh ta vẫn nói “tôi chắc” bằng đúng giọng cũ, và lúc đó anh ta nguy hiểm hơn cả người luôn nói bằng giọng đều. Vì vậy việc đầu tiên phải làm không phải thuê thêm người, mà là kiểm xem hai người sẵn có đang nói quá hay nói dè. Điều đó đo được: đếm xem họ sai bao nhiêu so với mức họ tự nhận.

Chỗ không rút gọn được: chuyện mức tin cậy là một cái bảng chứ không phải một con số. Một điểm ảnh bị nhòe theo một chiều vẫn rất chính xác theo chiều vuông góc. Mọi cách nói gọn thành “điểm này đáng tin bao nhiêu phần trăm” đều đánh mất chính cái chiều đó, mà đó lại là phần dùng được nhiều nhất.

Một câu: *Đừng hỏi phép đo nào đúng; hãy hỏi mỗi phép đo đáng tin bao nhiêu, rồi kiểm câu trả lời đó bằng số.*

34.9 Kết nối

[F/27-dat-hoc-sau-vao-dau](#) là tiền đề trực tiếp: bốn công ở đó chính là bộ lọc mà mọi kết luận “có đáng không” trong chương này đi qua. [F/28-dac-trung-va-ghep-cap](#) là mặt bổ sung: chương 28 làm cho *phép đo* bền hơn, chương này làm cho *trọng số* của phép đo trung thực hơn, và hai việc đó cộng chứ không thay nhau. [F/31-backend-kha-vi-va-dau-cuoi](#) nối vào phần hiệp phương sai học được từ phía ngược lại: nó hỏi điều gì xảy ra khi chính bộ tối ưu trở nên khả vi, còn ở đây bộ tối ưu vẫn giữ nguyên.

Ba chương ngoài Phần F cũng bị chương này gọi tới liên tục. [E/24-thi-nghiem-va-ablation](#) là điều kiện để mọi con số trong chương có nghĩa: không đo sần nhiều trước thì bảng NEES cũng chỉ là một mẫu. [D/21-inference-va-trien-khai](#) chứa toàn bộ phần triển khai chung mà mục 5 cố ý không lặp lại. Và [E/26-lo-trinh-hoc](#) là chỗ mục 6 nối vào: lộ trình D dừng ở ranh giới Phần F, còn mục 6 nối tiếp đọc tám chương này theo thứ tự nào.

Cuối cùng, một liên kết ngược lên đầu cây. Ý trung tâm của chương — rằng cái thiếu không phải ước lượng tốt hơn mà là ước lượng độ tin cậy tốt hơn — chính là trục đánh đổi giữa giả định và dữ liệu của [B/04](#) xuất hiện dưới một hình dạng cụ thể. Một hằng số datasheet là một giả định rất mạnh và rất rẻ. Một hiệp phương sai học được là dữ liệu thay cho giả định, đắt hơn nhiều, và hồng đúng ở chỗ mọi thứ dựa vào dữ liệu đều hồng; ngoài phân phối.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao chỉ tỉ số giữa các ma trận thông tin mới đổi nghiệm, và điều đó nói gì về việc bạn có sửa được trọng số IMU mà không đụng tới trọng số ảnh không?
2. Sai số vị trí do độ lệch con quay tăng theo t^3 chứ không phải t^2 : chuỗi nhân quả nào tạo ra bậc ba đó?
3. Nếu số Allan là một phép đo đúng, vì sao dùng thẳng nó làm Σ trong đồ thị nhân tử lại sai một cách có hệ thống, và sai theo hướng nào?

4. NEES của nhân tử ảnh lớn hơn hẳn bậc tự do: hai cách giải thích khác nhau là gì, và phép thử nào phân biệt chúng?
5. Trong hàm mất mát độ hợp lý âm, chuyện gì xảy ra nếu bỏ số hạng $\log \sigma^2$, và vì sao mạng sẽ khai bất định vô hạn?
6. Vì sao một hiệp phương sai học được chạy được ở tần suất thấp hơn tốc độ khung, trong khi một tư thế thì không?
7. Hai kỹ sư báo cùng một con số APE trên cùng một chuỗi, nhưng một người chạy lockstep còn người kia phát lại theo thời gian thật: hai con số đó khác nhau ở chỗ nào, và bạn hỏi thêm gì để biết?
8. Vì sao nói Σ là cách sai để xử lý một điểm khớp sai ghép, và ranh giới nào tách chuyện trọng số khỏi chuyện loại bỏ?

Bảng ánh xạ thuật ngữ

TÓM TẮT

Bảng này chốt **một khái niệm, một cách gọi** cho toàn cây: mỗi thuật ngữ tiếng Anh chỉ có đúng một cách gọi tiếng Việt chuẩn, kèm chương định nghĩa nó lần đầu. Dùng nó khi gặp một từ lạ giữa chừng, hoặc khi viết thêm chương mới. Cột *Định nghĩa ở* bấm được: nhấp vào là nhảy thẳng tới chương định nghĩa thuật ngữ đó. Nếu chỉ nhớ một điều: những từ mà bản dịch gây khó hiểu hơn bản gốc thì cây này cố ý *không dịch* — chúng được đánh dấu “giữ nguyên tiếng Anh” và bạn sẽ gặp đúng dạng đó trong mọi paper.

English	Cách gọi chuẩn	Định nghĩa ở
3D box	hộp 3D	B/05
3D convolution	tích chập ba chiều	C/15
3D vision	thị giác 3D	C/16
ablation	giữ nguyên tiếng Anh	B/03
absolute trajectory error	sai số quỹ đạo tuyệt đối	B/07
accuracy	độ chính xác	A/01
activation	kích hoạt	B/11
activation function	hàm kích hoạt	B/10
active learning	học chủ động	B/06
AdaGrad	giữ nguyên tiếng Anh	C/12
AdaIN	giữ nguyên tiếng Anh	C/17
Adam	giữ nguyên tiếng Anh	A/01
AdamW	giữ nguyên tiếng Anh	C/12
adapter	giữ nguyên tiếng Anh	C/18
adaptive overfitting	giữ nguyên tiếng Anh	B/07
add-one-in	giữ nguyên tiếng Anh	E/24
adjoint	giữ nguyên tiếng Anh	C/12
adjoint method	giữ nguyên tiếng Anh	D/19
adversarial example	mẫu đối kháng	C/17
aleatoric	giữ nguyên tiếng Anh	E/25
Aliasing	giữ nguyên tiếng Anh	A/01
all-reduce	giữ nguyên tiếng Anh	D/19
AMP	giữ nguyên tiếng Anh	D/19
anchor	giữ nguyên tiếng Anh	B/03
anchor-free	giữ nguyên tiếng Anh	C/14
annotation guideline	giữ nguyên tiếng Anh	B/05
aperture problem	bài toán khẩu độ	C/15
approximate nearest neighbour	giữ nguyên tiếng Anh	C/14
argmax	giữ nguyên tiếng Anh	D/19
arithmetic intensity	cường độ số học	B/11
ASPP	giữ nguyên tiếng Anh	C/14
assumption	giả định	B/03
đặt không chỉnh	ill-posed	C/17
ATE	giữ nguyên tiếng Anh	C/16

English	Cách gọi chuẩn	Định nghĩa ở
<i>atrous / dilated convolution</i>	tích chập giãn	C/14
ATSS	giữ nguyên tiếng Anh	C/14
<i>attention</i>	giữ nguyên tiếng Anh	A/01
<i>augmentation</i>	giữ nguyên tiếng Anh	B/10
AUPRC	giữ nguyên tiếng Anh	B/07
AUROC	giữ nguyên tiếng Anh	B/07
<i>autodiff</i>	giữ nguyên tiếng Anh	C/12
<i>autograd</i>	giữ nguyên tiếng Anh	A/01
<i>autoregressive</i>	giữ nguyên tiếng Anh	C/17
<i>autotune</i>	giữ nguyên tiếng Anh	E/24
<i>backbone</i>	giữ nguyên tiếng Anh	B/09
<i>backend</i>	giữ nguyên tiếng Anh	C/16
<i>backpropagation</i>	lan truyền ngược	A/01
<i>backpropagation through time</i>	BPTT	C/13
<i>backward pass</i>	lượt ngược	A/01
<i>baseline</i>	đường cơ sở	C/16
<i>batch</i>	giữ nguyên tiếng Anh	A/01
<i>batch normalization</i>	chuẩn hoá theo batch	A/01
<i>batch size</i>	kích thước batch	A/01
BatchNorm	giữ nguyên tiếng Anh	B/11
<i>bbox</i>	giữ nguyên tiếng Anh	B/05
<i>behavioral testing</i>	kiểm thử hành vi	B/07
<i>benchmark</i>	giữ nguyên tiếng Anh	B/04
<i>bf16</i>	giữ nguyên tiếng Anh	C/12
<i>bias-variance</i>	giữ nguyên tiếng Anh	B/04
<i>bộ phân biệt</i>	discriminator	C/17
<i>bootstrap</i>	giữ nguyên tiếng Anh	A/01
<i>bottom-up</i>	giữ nguyên tiếng Anh	C/14
<i>bounding box</i>	hộp giới hạn	B/09
<i>brightness constancy</i>	độ sáng không đổi	A/02
<i>broadcasting</i>	giữ nguyên tiếng Anh	C/12
<i>buffer</i>	giữ nguyên tiếng Anh	D/19
<i>bundle adjustment</i>	hiệu chỉnh chùm tia	A/01
<i>calibrated</i>	đã hiệu chuẩn	A/01
<i>calibration</i>	hiệu chuẩn	A/02
<i>calibration set</i>	tập hiệu chuẩn	E/25
<i>camera pose</i>	pose camera	C/16
<i>capacity</i>	sức chứa	B/04
<i>captioning</i>	giữ nguyên tiếng Anh	C/18
<i>catastrophic forgetting</i>	quên thảm khốc	C/18
<i>causal</i>	nhân quả	C/15
<i>centering</i>	giữ nguyên tiếng Anh	C/18
<i>chain rule</i>	Quy tắc chuỗi	A/01
<i>channel</i>	kênh	C/13
<i>Channel gating</i>	giữ nguyên tiếng Anh	B/10
<i>channels-last</i>	giữ nguyên tiếng Anh	B/11
<i>checkpoint</i>	giữ nguyên tiếng Anh	B/03
Cholesky	giữ nguyên tiếng Anh	A/01

English	Cách gọi chuẩn	Định nghĩa ở
<i>chuyển phong cách</i>	style transfer	C/17
<i>class-balanced loss</i>	giữ nguyên tiếng Anh	B/09
<i>classifier-free guidance</i>	giữ nguyên tiếng Anh	C/17
CNN	giữ nguyên tiếng Anh	C/13
<i>col2im</i>	giữ nguyên tiếng Anh	D/19
<i>Color jitter</i>	giữ nguyên tiếng Anh	C/18
<i>compound scaling</i>	giữ nguyên tiếng Anh	B/10
<i>computation graph</i>	đồ thị tính toán	D/19
<i>concept drift</i>	trôi khái niệm	B/08
<i>condition number</i>	số điều kiện	A/01
<i>confidence</i>	độ tin cậy	E/25
<i>confidence interval</i>	khoảng tin cậy	B/07
<i>Confident learning</i>	giữ nguyên tiếng Anh	B/06
<i>conformal prediction</i>	giữ nguyên tiếng Anh	A/01
<i>confusion matrix</i>	ma trận nhầm lẫn	B/07
<i>công thức huấn luyện</i>	recipe	C/13
<i>consistency</i>	nhất quán	B/09
<i>contrastive</i>	Tương phản	C/18
<i>contrastive loss</i>	giữ nguyên tiếng Anh	C/18
ControlNet	giữ nguyên tiếng Anh	C/17
<i>convex</i>	lồi	A/01
<i>Convolution</i>	giữ nguyên tiếng Anh	A/01
<i>copy</i>	giữ nguyên tiếng Anh	D/19
<i>corpus</i>	giữ nguyên tiếng Anh	C/18
<i>corruption</i>	giữ nguyên tiếng Anh	A/02
<i>cosine similarity</i>	độ tương đồng cosine	C/18
<i>Cost-sensitive learning</i>	giữ nguyên tiếng Anh	B/09
<i>covariance</i>	hiệp phương sai	A/01
<i>covariate shift</i>	dịch chuyển hiệp biến	B/08
<i>cross-attention</i>	giữ nguyên tiếng Anh	C/17
<i>cross-entropy</i>	entropy chéo	A/01
<i>cross-validation</i>	kiểm định chéo	B/06
CTC	giữ nguyên tiếng Anh	B/09
CUDA core	giữ nguyên tiếng Anh	B/11
CUDA graph	giữ nguyên tiếng Anh	B/11
cuDNN	giữ nguyên tiếng Anh	E/24
<i>curse of dimensionality</i>	lời nguyền số chiều	B/04
CutMix	giữ nguyên tiếng Anh	C/12
<i>damping</i>	giảm chấn	A/01
<i>data association</i>	liên kết dữ liệu	C/15
<i>data augmentation</i>	tăng cường dữ liệu	A/02
<i>data distribution</i>	phân phối dữ liệu	C/17
<i>data flow diagram</i>	sơ đồ luồng dữ liệu	B/03
<i>data leakage</i>	rò rỉ dữ liệu	A/01
<i>data loss</i>	giữ nguyên tiếng Anh	B/09
<i>dataloader</i>	giữ nguyên tiếng Anh	B/03
<i>datasheet</i>	giữ nguyên tiếng Anh	B/05
<i>Datasheets for Datasets</i>	giữ nguyên tiếng Anh	B/06

English	Cách gọi chuẩn	Định nghĩa ở
DDP	giữ nguyên tiếng Anh	D/19
dead ReLU	giữ nguyên tiếng Anh	C/12
Deformable convolution	giữ nguyên tiếng Anh	B/10
demosaicing	khử khảm	A/01
denoising	khử nhiễu	C/17
dense matching	khớp dày đặc	C/16
depth camera	camera độ sâu	A/02
depthwise	giữ nguyên tiếng Anh	B/10
descriptor	giữ nguyên tiếng Anh	C/16
determinism	tính tất định	D/19
Dice coefficient	Dice	B/09
dịch chuyển miền	domain gap	A/02
dict	giữ nguyên tiếng Anh	D/19
differentiable	khả vi	C/16
diffusion	giữ nguyên tiếng Anh	C/17
diffusion transformer	giữ nguyên tiếng Anh	C/17
Dilated conv	giữ nguyên tiếng Anh	B/10
disparity	thị sai	C/16
distillation	chưng cất	C/13
distortion	méo ống kính	A/02
distribution	phân phối	A/01
domain adaptation	thích nghi miền	B/08
domain gap	giữ nguyên tiếng Anh	B/04
domain generalization	tổng quát hoá miền	B/08
domain shift	lệch phân phối	C/16
double descent	giữ nguyên tiếng Anh	B/04
drift	giữ nguyên tiếng Anh	C/16
DRO	giữ nguyên tiếng Anh	B/09
dựng ảnh ở góc nhìn mới	novel view synthesis	C/16
dựng ảnh thể tích	volume rendering	C/16
dynamic batching	batching động	D/21
dynamic range	Dải động	A/02
đề xuất vùng	proposal	C/14
early fusion	giữ nguyên tiếng Anh	C/16
ECE	giữ nguyên tiếng Anh	E/25
effect size	cỡ hiệu ứng	E/24
effective receptive field	trường tiếp nhận hiệu dụng	B/10
eigendecomposition	phân rã trị riêng	A/01
eigenvalue	trị riêng	A/01
eigenvector	vector riêng	A/01
EMA	giữ nguyên tiếng Anh	C/18
embedding	giữ nguyên tiếng Anh	B/06
empirical risk minimization	tối thiểu hoá rủi ro thực nghiệm	B/04
ensemble	giữ nguyên tiếng Anh	A/01
Entropy	giữ nguyên tiếng Anh	A/01
epistemic	giữ nguyên tiếng Anh	E/25
epoch	giữ nguyên tiếng Anh	C/18
ERM	giữ nguyên tiếng Anh	B/04

English	Cách gọi chuẩn	Định nghĩa ở
<i>expected calibration error</i>	giữ nguyên tiếng Anh	B/07
<i>expected risk</i>	rủi ro kỳ vọng	B/04
<i>export</i>	giữ nguyên tiếng Anh	D/19
<i>exposure</i>	phơi sáng	A/02
<i>extrinsic calibration</i>	hiệu chỉnh ngoại	B/06
<i>extrinsics</i>	ma trận ngoại tại	A/02
<i>factor graph</i>	đồ thị nhân tố	D/19
<i>failure case</i>	giữ nguyên tiếng Anh	B/03
<i>failure mode</i>	chế độ hỏng	B/03
<i>false negative</i>	giữ nguyên tiếng Anh	B/07
<i>false positive</i>	giữ nguyên tiếng Anh	B/07
<i>feature map</i>	bản đồ đặc trưng	C/17
<i>feature pyramid network</i>	FPN	C/13
<i>feature pyramid network, FPN</i>	kim tự tháp đặc trưng	C/14
<i>feed-forward</i>	tiền thẳng	B/04
<i>feed-forward geometry</i>	giữ nguyên tiếng Anh	C/16
<i>feedback loop</i>	vòng phản hồi	B/08
<i>fine-grained classification</i>	phân loại chi tiết	C/14
<i>Fine-tune</i>	giữ nguyên tiếng Anh	C/18
<i>finite difference</i>	sai phân hữu hạn	C/12
<i>fisheye</i>	giữ nguyên tiếng Anh	C/14
<i>fixed-pattern noise</i>	giữ nguyên tiếng Anh	A/02
<i>floaters</i>	giữ nguyên tiếng Anh	C/16
<i>FLOPs</i>	giữ nguyên tiếng Anh	B/11
<i>flow matching</i>	giữ nguyên tiếng Anh	C/17
<i>focal</i>	giữ nguyên tiếng Anh	B/09
<i>Focal loss</i>	giữ nguyên tiếng Anh	A/01
<i>foreground-background imbalance</i>	mất cân bằng nền-vật thể	C/14
<i>forward</i>	giữ nguyên tiếng Anh	B/09
<i>forward operator</i>	toán tử thuận	C/17
<i>forward pass</i>	lượt xuôi	C/18
<i>Forward-mode autodiff</i>	giữ nguyên tiếng Anh	C/12
<i>foundation model</i>	mô hình nền tảng	C/18
<i>fp16</i>	giữ nguyên tiếng Anh	C/12
<i>freeze</i>	đóng băng	C/17
<i>frontend</i>	giữ nguyên tiếng Anh	C/16
<i>fully connected</i>	kết nối đầy đủ	C/14
<i>fully-connected</i>	giữ nguyên tiếng Anh	D/19
<i>gamma</i>	giữ nguyên tiếng Anh	A/02
<i>gate</i>	cổng	C/13
<i>gauge</i>	giữ nguyên tiếng Anh	C/18
<i>gauge freedom</i>	giữ nguyên tiếng Anh	A/01
<i>Gauss-Newton</i>	giữ nguyên tiếng Anh	A/01
<i>Gauss-Newton matrix</i>	giữ nguyên tiếng Anh	A/01
<i>Gaussian noise</i>	nhiều Gauss	C/17
<i>Gaussian splatting</i>	giữ nguyên tiếng Anh	C/16
<i>GEMM</i>	giữ nguyên tiếng Anh	B/11
<i>generalization</i>	tổng quát hoá	C/12

English	Cách gọi chuẩn	Định nghĩa ở
<i>generative model</i>	mô hình sinh	C/17
<i>Gimbal lock</i>	giữ nguyên tiếng Anh	A/01
<i>global average pooling</i>	gộp trung bình toàn cục	C/15
<i>Goodhart's law</i>	giữ nguyên tiếng Anh	B/09
<i>Grad-CAM</i>	giữ nguyên tiếng Anh	E/25
<i>gradient</i>	giữ nguyên tiếng Anh	A/01
<i>gradient check</i>	kiểm tra gradient	A/01
<i>gradient checkpointing</i>	giữ nguyên tiếng Anh	A/01
<i>gradient clipping</i>	cắt ngưỡng gradient	C/12
<i>gradient descent</i>	hạ gradient	A/01
<i>Gram matrix</i>	ma trận Gram	C/17
<i>graph break</i>	giữ nguyên tiếng Anh	D/19
<i>Gridding artifact</i>	giữ nguyên tiếng Anh	B/10
<i>ground truth</i>	chân trị	B/05
<i>grounding</i>	giữ nguyên tiếng Anh	C/18
<i>guidance</i>	dẫn hướng	C/17
<i>guided backpropagation</i>	giữ nguyên tiếng Anh	E/25
<i>Gumbel-softmax</i>	giữ nguyên tiếng Anh	A/01
<i>half precision</i>	giữ nguyên tiếng Anh	A/01
<i>hallucination</i>	ảo giác	C/18
<i>hard constraint</i>	ràng buộc cứng	B/03
<i>hard negative mining</i>	giữ nguyên tiếng Anh	C/14
<i>harmonic mean</i>	trung bình điều hoà	B/07
<i>head</i>	giữ nguyên tiếng Anh	B/10
<i>heatmap</i>	bản đồ nhiệt	C/14
<i>Hessian</i>	Ma trận Hessian	A/01
<i>heteroscedastic loss</i>	giữ nguyên tiếng Anh	B/09
<i>hidden state</i>	trạng thái ẩn	C/13
<i>hierarchy</i>	phân cấp	C/13
<i>Hinge loss</i>	giữ nguyên tiếng Anh	B/09
<i>HOG</i>	giữ nguyên tiếng Anh	B/04
<i>homogeneous coordinates</i>	Toạ độ đồng nhất	A/01
<i>hook</i>	giữ nguyên tiếng Anh	D/19
<i>Huber loss</i>	hàm Huber	B/09
<i>Hungarian matching</i>	ghép cặp Hungary	B/09
<i>hyperparameter</i>	siêu tham số	C/12
<i>hyperparameter tuning</i>	tinh chỉnh siêu tham số	E/23
<i>identifiability</i>	giữ nguyên tiếng Anh	B/04
<i>ill-conditioned</i>	giữ nguyên tiếng Anh	A/01
<i>ill-posed</i>	giữ nguyên tiếng Anh	C/16
<i>im2col</i>	giữ nguyên tiếng Anh	D/19
<i>image captioning</i>	Sinh mô tả ảnh	C/15
<i>image signal processor</i>	ISP	A/02
<i>implicit field</i>	trường ẩn	C/16
<i>implicit regularization</i>	chính quy hoá ngầm	C/12
<i>inductive bias</i>	giữ nguyên tiếng Anh	B/04
<i>inference</i>	suy luận	C/18
<i>influence function</i>	giữ nguyên tiếng Anh	B/06

English	Cách gọi chuẩn	Định nghĩa ở
InfoNCE	giữ nguyên tiếng Anh	B/09
information matrix	ma trận thông tin	B/04
inlier	giữ nguyên tiếng Anh	C/16
inter-annotator agreement	mức đồng thuận giữa người gán nhãn	B/05
interpolation	nội suy	A/01
interpolation threshold	ngưỡng nội suy	B/04
intersection over union, IoU	giao trên hợp	C/14
inverted dropout	giữ nguyên tiếng Anh	D/19
inverted residual	giữ nguyên tiếng Anh	D/20
IoU	giữ nguyên tiếng Anh	B/05
Jacobian	ma trận Jacobian	A/01
Joint detection-tracking	giữ nguyên tiếng Anh	C/15
k-nearest neighbours	k láng giềng gần nhất	B/04
kernel fusion	hợp nhất kernel	B/11
keyframe	giữ nguyên tiếng Anh	A/01
keypoint	giữ nguyên tiếng Anh	B/05
KNN	giữ nguyên tiếng Anh	B/04
knowledge distillation	chưng cất tri thức	A/01
Kullback–Leibler divergence	phân kỳ KL	A/01
label assignment	gán mẫu	B/03
label shift	dịch chuyển nhãn	B/08
label smoothing	làm mượt nhãn	A/01
Lambertian	giữ nguyên tiếng Anh	C/16
late fusion	giữ nguyên tiếng Anh	C/16
latency	độ trễ	B/03
latent diffusion	giữ nguyên tiếng Anh	C/17
latent space	không gian ẩn	C/17
latent variable	biến ẩn	C/17
layer	tầng	C/12
learning curve	giữ nguyên tiếng Anh	B/04
learning path	lộ trình	E/26
learning rate	giữ nguyên tiếng Anh	A/01
least squares	Bình phương tối thiểu	A/01
leave-one-out	giữ nguyên tiếng Anh	E/24
letterbox	giữ nguyên tiếng Anh	A/01
Lịch learning rate	giữ nguyên tiếng Anh	C/12
Lie algebra	đại số Lie	A/01
likelihood	độ hợp lý	A/01
likelihood ratio	tỉ số độ hợp lý	B/08
limitations	giữ nguyên tiếng Anh	B/03
linear probe	giữ nguyên tiếng Anh	A/01
linear scaling rule	quy tắc tỉ lệ tuyến tính	C/12
lô	batch	C/12
locality	tính cục bộ	C/13
Log-sum-exp	giữ nguyên tiếng Anh	D/19
logit	giữ nguyên tiếng Anh	D/20
Long-tail	giữ nguyên tiếng Anh	B/08
long-tail distribution	phân bố đuôi dài	C/14

English	Cách gọi chuẩn	Định nghĩa ở
<i>loop closing</i>	phát hiện đóng vòng	B/03
<i>loop closure</i>	đóng vòng	C/16
LoRA	giữ nguyên tiếng Anh	A/01
<i>loss</i>	hàm mất mát	C/18
<i>loss function</i>	hàm mất mát	A/01
<i>loss landscape</i>	bề mặt hàm mất mát	A/01
<i>loss scaling</i>	giữ nguyên tiếng Anh	A/01
<i>low-rank approximation</i>	xấp xỉ hạng thấp	A/01
LPIPS	giữ nguyên tiếng Anh	C/16
LSTM	giữ nguyên tiếng Anh	C/13
<i>lưới đặc trưng</i>	feature map	C/15
<i>manifold</i>	đa tạp	A/01
<i>manifold hypothesis</i>	giả thuyết đa tạp	B/04
mAP	giữ nguyên tiếng Anh	B/05
<i>marching cubes</i>	giữ nguyên tiếng Anh	C/16
<i>mask</i>	mặt nạ	C/14
<i>mask classification</i>	phân loại mặt nạ	C/14
<i>masked attention</i>	attention có mặt nạ	C/14
<i>masked modeling</i>	giữ nguyên tiếng Anh	C/18
<i>master weights</i>	giữ nguyên tiếng Anh	C/12
<i>Matthews correlation coefficient</i>	hệ số tương quan Matthews	B/07
<i>max-pooling</i>	giữ nguyên tiếng Anh	A/01
<i>maximum a posteriori</i>	cực đại hoá hậu nghiệm	B/09
<i>maximum a posteriori, MAP</i>	Cực đại hậu nghiệm	A/01
<i>maximum likelihood estimation</i>	cực đại hoá độ hợp lý	B/09
<i>maximum likelihood estimation, MLE</i>	Ước lượng độ hợp lý cực đại	A/01
<i>mean intersection over union, mIoU</i>	giao trên hợp trung bình	C/14
<i>mean matching accuracy</i>	giữ nguyên tiếng Anh	B/07
<i>memory bandwidth</i>	băng thông bộ nhớ	B/11
<i>mesh</i>	mesh	C/16
<i>metric learning</i>	giữ nguyên tiếng Anh	B/09
<i>minibatch</i>	giữ nguyên tiếng Anh	C/12
<i>missing data</i>	dữ liệu thiếu	B/06
<i>mixed precision</i>	độ chính xác hỗn hợp	D/19
Mixup	giữ nguyên tiếng Anh	C/12
MLE	giữ nguyên tiếng Anh	B/09
MLP	giữ nguyên tiếng Anh	B/10
<i>mode collapse</i>	giữ nguyên tiếng Anh	C/17
<i>mode của phân phối</i>	mode	C/17
<i>model</i>	mô hình	A/01
<i>model cards</i>	giữ nguyên tiếng Anh	B/05
<i>moiré</i>	giữ nguyên tiếng Anh	A/02
<i>momentum</i>	giữ nguyên tiếng Anh	C/12
<i>motion blur</i>	Ảnh mờ do chuyển động	C/16
<i>multi-object tracking</i>	bám nhiều mục tiêu	C/15
<i>multi-task loss</i>	hàm mất mát đa nhiệm	C/14
<i>multi-view stereo</i>	MVS	C/16

English	Cách gọi chuẩn	Định nghĩa ở
<i>multiple comparisons</i>	so sánh bội	E/24
<i>near-duplicate</i>	trùng lặp gần đúng	B/06
<i>nearest-neighbor</i>	giữ nguyên tiếng Anh	A/01
<i>neck</i>	giữ nguyên tiếng Anh	B/10
<i>negative</i>	giữ nguyên tiếng Anh	C/18
<i>NeRF</i>	giữ nguyên tiếng Anh	A/01
<i>nested cross-validation</i>	giữ nguyên tiếng Anh	B/07
<i>Nesterov</i>	giữ nguyên tiếng Anh	C/12
<i>neural rendering</i>	giữ nguyên tiếng Anh	C/16
<i>NHWC</i>	giữ nguyên tiếng Anh	B/11
<i>no free lunch</i>	giữ nguyên tiếng Anh	B/04
<i>noise floor</i>	sàn nhiễu	E/24
<i>non-identifiability</i>	không xác định được	C/16
<i>non-maximum suppression</i>	triệt phi cực đại	B/03
<i>non-maximum suppression, NMS</i>	triệt phi cực đại	C/14
<i>normalization</i>	chuẩn hoá	B/09
<i>numerical gradient check</i>	kiểm tra gradient số	D/19
<i>độ hợp lý</i>	likelihood	C/17
<i>object detection</i>	Phát hiện vật thể	C/14
<i>objective function</i>	hàm mục tiêu	C/17
<i>odds</i>	tỉ số cược	B/08
<i>đơn vị mét</i>	metric	C/16
<i>one-stage</i>	giữ nguyên tiếng Anh	B/03
<i>open-set</i>	tập lớp mở	C/14
<i>open-vocabulary detection</i>	phát hiện từ vựng mở	C/18
<i>operating point</i>	điểm vận hành	B/07
<i>opset</i>	giữ nguyên tiếng Anh	D/21
<i>optical flow</i>	dòng quang	A/02
<i>optimal transport assignment</i>	giữ nguyên tiếng Anh	C/14
<i>optimization profile</i>	giữ nguyên tiếng Anh	D/21
<i>optimizer</i>	bộ tối ưu	B/05
<i>oracle</i>	giữ nguyên tiếng Anh	E/24
<i>out-of-distribution detection</i>	phát hiện ngoài phân phối	B/08
<i>outlier</i>	giữ nguyên tiếng Anh	C/16
<i>output stride</i>	giữ nguyên tiếng Anh	C/14
<i>overfitting</i>	giữ nguyên tiếng Anh	B/04
<i>paired comparison</i>	so sánh theo cặp	E/24
<i>panoptic quality</i>	giữ nguyên tiếng Anh	B/07
<i>parameter</i>	giữ nguyên tiếng Anh	D/19
<i>patch</i>	giữ nguyên tiếng Anh	C/18
<i>PEFT</i>	giữ nguyên tiếng Anh	C/18
<i>perceptual hash</i>	giữ nguyên tiếng Anh	B/06
<i>perceptual loss</i>	mất mát tri giác	B/09
<i>permutation invariance</i>	bất biến hoán vị	C/16
<i>perplexity</i>	giữ nguyên tiếng Anh	E/25
<i>perspective projection</i>	phép chiếu phối cảnh	C/16
<i>phân kỳ Kullback–Leibler</i>	KL divergence	C/17
<i>photometric consistency</i>	nhất quán trắc quang	B/09

English	Cách gọi chuẩn	Định nghĩa ở
<i>pillar</i>	giữ nguyên tiếng Anh	C/16
<i>pix2pix</i>	giữ nguyên tiếng Anh	C/17
<i>plugin</i>	giữ nguyên tiếng Anh	D/21
<i>point cloud</i>	point cloud	C/16
<i>point map</i>	giữ nguyên tiếng Anh	C/16
<i>pointer-ordered containers</i>	giữ nguyên tiếng Anh	E/24
<i>polygon</i>	giữ nguyên tiếng Anh	B/05
<i>Pooling</i>	giữ nguyên tiếng Anh	B/10
<i>population</i>	quần thể	B/05
<i>positional encoding</i>	mã hoá vị trí	C/13
<i>positive</i>	giữ nguyên tiếng Anh	C/18
<i>post-norm</i>	giữ nguyên tiếng Anh	B/10
<i>Posterior collapse</i>	giữ nguyên tiếng Anh	A/01
<i>pre-norm</i>	giữ nguyên tiếng Anh	B/10
<i>precision</i>	giữ nguyên tiếng Anh	B/07
<i>predictor</i>	giữ nguyên tiếng Anh	C/18
<i>prefetch</i>	giữ nguyên tiếng Anh	D/19
<i>pretraining</i>	tiền huấn luyện	C/18
<i>principal component analysis, PCA</i>	phân tích thành phần chính	A/01
<i>prior</i>	tiền nghiệm	C/16
<i>profile</i>	giữ nguyên tiếng Anh	D/19
<i>profiler</i>	giữ nguyên tiếng Anh	D/19
<i>projection</i>	phép chiếu	A/01
<i>prompt</i>	giữ nguyên tiếng Anh	B/05
<i>proper scoring rule</i>	giữ nguyên tiếng Anh	B/09
<i>proxy</i>	giữ nguyên tiếng Anh	A/01
<i>pseudo-labeling</i>	giữ nguyên tiếng Anh	B/06
<i>pull request</i>	giữ nguyên tiếng Anh	E/23
<i>QR</i>	giữ nguyên tiếng Anh	A/01
<i>quantile loss</i>	giữ nguyên tiếng Anh	B/09
<i>quantization</i>	lượng tử hoá	A/01
<i>query</i>	truy vấn	C/14
<i>RandAugment</i>	giữ nguyên tiếng Anh	C/12
<i>random label noise</i>	nhiều ngẫu nhiên	B/06
<i>Random resized crop</i>	giữ nguyên tiếng Anh	C/18
<i>rank</i>	hạng	A/01
<i>rank collapse</i>	giữ nguyên tiếng Anh	A/01
<i>raw agreement</i>	giữ nguyên tiếng Anh	B/06
<i>re-parameterization</i>	tái tham số hoá	D/20
<i>read noise</i>	giữ nguyên tiếng Anh	A/02
<i>recall</i>	giữ nguyên tiếng Anh	B/07
<i>receptive field</i>	trường tiếp nhận	B/10
<i>recipe</i>	công thức huấn luyện	E/23
<i>recurrent neural network, RNN</i>	mạng hồi quy	C/13
<i>region proposal network, RPN</i>	mạng đề xuất vùng	C/14
<i>regularization</i>	chính quy hoá	A/01
<i>relative pose error</i>	sai số tương đối	B/07
<i>reliability</i>	giữ nguyên tiếng Anh	B/07

English	Cách gọi chuẩn	Định nghĩa ở
reliability diagram	biểu đồ độ tin cậy	B/07
relocalization	tái định vị	B/03
ReLU	giữ nguyên tiếng Anh	A/01
representation	biểu diễn	B/04
reprojection error	sai số tái chiếu	A/01
resampling	giữ nguyên tiếng Anh	B/05
residual	phần dư	B/08
residual connection	kết nối tắt	B/10
resolution	giữ nguyên tiếng Anh	B/07
reverse	giữ nguyên tiếng Anh	B/09
Reverse-mode autodiff	giữ nguyên tiếng Anh	C/12
reverse-mode automatic differentiation	giữ nguyên tiếng Anh	A/01
reweighting	giữ nguyên tiếng Anh	B/05
ridge point	điểm gầy	B/11
RMSProp	giữ nguyên tiếng Anh	C/12
RoIAlign	giữ nguyên tiếng Anh	C/14
RoIPooling	giữ nguyên tiếng Anh	C/14
Rolling shutter	giữ nguyên tiếng Anh	A/01
roofline	giữ nguyên tiếng Anh	B/03
RPE	giữ nguyên tiếng Anh	A/01
runtime	giữ nguyên tiếng Anh	D/19
saddle point	điểm yên ngựa	A/01
saliency map	giữ nguyên tiếng Anh	E/25
sampler	bộ lấy mẫu	C/17
sampler distillation	giữ nguyên tiếng Anh	C/17
sampling frame	khung lấy mẫu	B/05
sampling theorem	Định lý lấy mẫu	A/01
sanity ablation	giữ nguyên tiếng Anh	E/24
scale ambiguity	nhập nhằng thang đo	A/02
scaling laws	giữ nguyên tiếng Anh	C/18
score	giữ nguyên tiếng Anh	C/17
score function	hàm điểm	C/17
SDF	giữ nguyên tiếng Anh	C/16
seed	giữ nguyên tiếng Anh	B/03
segmentation	phân vùng	B/09
selection bias	thiên lệch chọn mẫu	B/06
selective search	giữ nguyên tiếng Anh	C/14
self-distillation	Tự chưng cất	C/18
self-supervised	tự giám sát	B/04
self-supervised learning	học tự giám sát	C/18
semi-supervised learning	học bán giám sát	B/06
sensitivity	giữ nguyên tiếng Anh	B/07
sequence	chuỗi	C/13
set	tập hợp	C/14
set prediction	dự đoán tập hợp	C/14
SGD	giữ nguyên tiếng Anh	B/04
shape	giữ nguyên tiếng Anh	D/19

English	Cách gọi chuẩn	Định nghĩa ở
shortcut learning	giữ nguyên tiếng Anh	A/02
shot noise	giữ nguyên tiếng Anh	A/02
signal-to-noise ratio	SNR	C/17
signed distance function	SDF	C/16
singular value decomposition, SVD	phân rã giá trị kỳ dị	A/01
skyline	giữ nguyên tiếng Anh	E/24
SLAM	giữ nguyên tiếng Anh	C/14
slice	lát	C/14
slice analysis	phân tích theo lát cắt	E/24
slice-based evaluation	giữ nguyên tiếng Anh	B/07
SNR	tỉ số tín hiệu trên nhiễu	A/02
softmax	giữ nguyên tiếng Anh	A/01
softmax cross-entropy loss	softmax loss	D/19
sparse convolution	tích chập thưa	C/16
Sparsity	giữ nguyên tiếng Anh	D/20
specificity	giữ nguyên tiếng Anh	B/07
spectral bias	giữ nguyên tiếng Anh	A/01
spherical harmonics	hàm điều hoà cầu	C/16
squared error	sai số bình phương	B/04
stack trace	giữ nguyên tiếng Anh	D/19
state space model, SSM	mô hình không gian trạng thái	C/13
stereo	giữ nguyên tiếng Anh	C/16
stereo baseline	giữ nguyên tiếng Anh	B/04
stochastic gradient descent	giữ nguyên tiếng Anh	D/19
stochastic gradient descent, SGD	hạ gradient ngẫu nhiên	C/12
stock	giữ nguyên tiếng Anh	C/18
stop	giữ nguyên tiếng Anh	B/03
stop-gradient	giữ nguyên tiếng Anh	C/18
straight-through estimator	giữ nguyên tiếng Anh	A/01
stratified	giữ nguyên tiếng Anh	B/07
stride	giữ nguyên tiếng Anh	C/14
structure from motion	SfM	C/16
style loss	giữ nguyên tiếng Anh	B/09
subspace	không gian con	A/01
supervision signal	tín hiệu giám sát	B/03
surrogate loss	hàm thay thế	B/09
symmetric function	hàm đối xứng	C/16
synthetic data	dữ liệu tổng hợp	B/06
systematic label noise	nhiều hệ thống	B/06
t-SNE	giữ nguyên tiếng Anh	A/01
taxonomy	giữ nguyên tiếng Anh	B/05
teacher	giữ nguyên tiếng Anh	B/09
temperature	hiệu độ	C/18
temperature scaling	giữ nguyên tiếng Anh	A/01
template	mẫu chuẩn	B/10
temporal action localization	giữ nguyên tiếng Anh	C/15
temporal IoU	giữ nguyên tiếng Anh	C/15
tensor	giữ nguyên tiếng Anh	C/12

English	Cách gọi chuẩn	Định nghĩa ở
<i>tensor core</i>	lõi tensor	B/11
<i>test set contamination</i>	nhiễm bản tập kiểm tra	C/18
<i>test-time augmentation</i>	giữ nguyên tiếng Anh	A/01
TF32	giữ nguyên tiếng Anh	E/24
<i>thermal</i>	camera nhiệt	A/02
<i>thermal throttling</i>	giảm xung do nhiệt	B/11
<i>thiên lệch quy nạp</i>	inductive bias	A/02
<i>throughput</i>	thông lượng	B/11
<i>thuật toán tối ưu</i>	optimizer	C/12
<i>tiền thẳng</i>	feed-forward	C/17
<i>timestamp</i>	giữ nguyên tiếng Anh	B/03
<i>tinh chỉnh</i>	fine-tune	C/18
<i>token</i>	giữ nguyên tiếng Anh	B/10
<i>tone mapping</i>	ánh xạ tone	A/02
<i>top-down</i>	giữ nguyên tiếng Anh	C/14
<i>topology</i>	giữ nguyên tiếng Anh	C/16
<i>track</i>	giữ nguyên tiếng Anh	A/02
<i>tracker</i>	giữ nguyên tiếng Anh	C/14
<i>Tracking-by-detection</i>	giữ nguyên tiếng Anh	C/15
<i>training recipe</i>	công thức huấn luyện	E/23
<i>transfer learning</i>	học chuyển giao	C/18
<i>transformer</i>	giữ nguyên tiếng Anh	A/01
<i>translation equivariance</i>	đẳng biến tịnh tiến	A/01
<i>triangulation</i>	tam giác hoá	C/16
<i>triplet loss</i>	giữ nguyên tiếng Anh	C/14
<i>trong ngữ cảnh GPU</i>	kernel	B/11
<i>true negative</i>	giữ nguyên tiếng Anh	B/07
<i>true positive</i>	giữ nguyên tiếng Anh	B/07
<i>trung bình trượt lũy thừa</i>	EMA	C/12
<i>trường bức xạ</i>	radiance field	C/16
<i>tương phản</i>	contrastive	C/18
<i>Tversky loss</i>	Tversky	B/09
UMAP	giữ nguyên tiếng Anh	A/01
<i>uncertainty</i>	giữ nguyên tiếng Anh	B/07
<i>underfit</i>	giữ nguyên tiếng Anh	B/04
<i>unit of analysis</i>	đơn vị phân tích	B/05
<i>validation</i>	giữ nguyên tiếng Anh	A/01
<i>vanishing and exploding gradient</i>	giữ nguyên tiếng Anh	C/12
<i>variance</i>	phương sai	C/12
<i>vector-Jacobian product, VJP</i>	tích vector-Jacobian	C/12
<i>view</i>	giữ nguyên tiếng Anh	D/19
<i>vision encoder</i>	giữ nguyên tiếng Anh	C/18
<i>Vision Transformer</i>	giữ nguyên tiếng Anh	C/13
<i>vision-language</i>	thị giác-ngôn ngữ	C/18
<i>vision-language model</i>	VLM	C/18
<i>vision-language model, VLM</i>	mô hình thị giác-ngôn ngữ	C/18
<i>visual-inertial SLAM</i>	giữ nguyên tiếng Anh	C/16
ViT	giữ nguyên tiếng Anh	B/04

English	Cách gọi chuẩn	Định nghĩa ở
<i>voxel</i>	giữ nguyên tiếng Anh	C/16
<i>warmup</i>	giữ nguyên tiếng Anh	A/01
<i>watermark</i>	giữ nguyên tiếng Anh	B/04
<i>weak supervision</i>	giám sát yếu	B/06
<i>weight</i>	trọng số	A/01
<i>weight decay</i>	giữ nguyên tiếng Anh	A/01
<i>weight sharing</i>	chia sẻ trọng số	B/10
<i>white balance</i>	cân bằng trắng	A/02
<i>whitening</i>	giữ nguyên tiếng Anh	A/01
<i>worker</i>	giữ nguyên tiếng Anh	D/19
<i>zero-shot</i>	giữ nguyên tiếng Anh	C/18

Kết luận: cái gì còn lại sau khi mọi kiến trúc đã cũ

TÓM TẮT

Ba mươi bốn chương của cây này rút lại còn bốn nguyên lý và một thói quen. Bốn nguyên lý — độ khó chỉ bị đẩy đi chỗ khác, trục đánh đổi giữa giả định và dữ liệu, không thể không có giả định, và hiểu nghĩa là biết chỗ hỏng — sẽ còn đúng khi mọi tên mô hình trong cây đã cũ. Thói quen là luôn hỏi *nó giả định gì và nó hỏng khi nào* trước khi hỏi *nó đạt bao nhiêu điểm*. Nếu chỉ giữ lại một trang trong toàn bộ tài liệu, hãy giữ trang này.

34.10 Bốn nguyên lý xuyên suốt

Một tài liệu dài có nguy cơ để lại ấn tượng rằng lĩnh vực này là một tập hợp rời rạc của rất nhiều kỹ thuật. Thực ra không phải. Gần như mọi chương ở trên là một cách nhìn khác của bốn ý sau, và nhận ra chúng lặp lại chính là dấu hiệu đã đọc đúng.

- **Độ khó không biến mất, nó chỉ bị đẩy đi chỗ khác.** Khi một phương pháp trông như bữa trưa miễn phí thì bạn chưa tìm ra chỗ nó trả giá — sang dữ liệu, sang hậu xử lý, sang thời gian huấn luyện, hay sang một giả định ngầm về đầu vào. Nguyên lý này xuất hiện ở chương 3 như một công cụ đọc hệ thống, rồi quay lại ở *anchor-free* chuyển chi phí sang **gán mẫu** (*label assignment*), ở *style transfer* feed-forward chuyển chi phí từ lúc suy luận sang lúc huấn luyện, và ở **lượng tử hoá** (*quantization*) đổi độ chính xác lấy thông lượng.
- **Trục giả định mạnh với ít dữ liệu, đổi lại giả định yếu với nhiều dữ liệu.** Đây là trục giải thích phần lớn tiến bộ mười lăm năm qua. Nó xuất hiện dưới dạng *CNN* đổi lại *ViT*, hình học cổ điển đổi lại mô hình *feed-forward* trong thị giác 3D, và mô hình chuyên biệt đổi lại **mô hình nền tảng** (*foundation model*). Không đầu nào của trục đúng tuyệt đối; vị trí đúng phụ thuộc vào lượng dữ liệu bạn thực sự có.
- **Không thể không có giả định, chỉ có việc chọn giả định nào.** Giả định vào hệ thống qua đúng bốn cửa: kiến trúc, hàm mất mát, dữ liệu và **tăng cường dữ liệu** (*data augmentation*), và thuật toán tối ưu. Người tin rằng mình đang làm việc “không giả định gì” chỉ là người chưa biết mình đã giả định gì. Hệ quả thực dụng: khi mô hình kém, hãy hỏi giả định nào đang sai trước khi hỏi nên thêm bao nhiêu dữ liệu — vì nếu bài toán không **định danh được** (*identifiable*), thêm dữ liệu không cứu được.
- **Hiểu một hệ thống nghĩa là biết nó hỏng ở đâu.** Không phải biết nó chạy được gì. Đây là lý do mọi tài liệu trong cây đều có một mục bắt buộc về giới hạn và trường hợp hỏng, và là lý do cột giá trị nhất trong mọi bảng so sánh là cột cuối cùng.

34.11 Cái gì bền và cái gì sẽ cũ

Cây được phân tầng theo chu kỳ bán rã ngay từ bản đồ gốc, và kết luận đáng nói lại ở đây là tầng nào đáng dành thời gian khi bạn phải chọn.

Bảng 34.10. Dự đoán về tuổi thọ của từng loại nội dung trong cây này.

Loại nội dung	Tuổi thọ	Vì sao
Tên mô hình dẫn đầu bảng, phần <i>Repo</i> và <i>SOTA</i>	6–12 tháng	Chốt tại 9/2026; đây là phần hết hạn nhanh nhất và nên đọc như chỗ bắt đầu tìm
Chi tiết <i>API</i> và <i>framework</i>	2–3 năm	Tầng [T4]; tra khi cần, không đầu tư sâu
Kiến trúc cụ thể và thủ thuật huấn luyện	5–10 năm	Tầng [T3]; học ý tưởng cốt lõi và lý do ra đời, không học danh sách
Nguyên lý thiết kế, đánh đổi, giới hạn tính toán	>10 năm	Tầng [T2]; Phần B và E gần như không hết hạn
Toán, vật lý tạo ảnh, hình học	>20 năm	Tầng [T1]; Phần A, học vừa đủ nhưng học chắc

Bảng 34.10 dẫn tới một lời khuyên hơi nghịch trực giác: phần thú vị nhất khi đọc (Phần C, các mô hình) không phải phần đáng đầu tư nhất khi ôn. Khi quay lại cây sau sáu tháng, hãy ôn Phần B và Phần E trước, vì đó là phần thay đổi cách bạn đọc mọi thứ khác.

34.12 Ba thói quen đáng mang theo

Thứ nhất, **đọc phần giới hạn trước phần kết quả**. Đó là chỗ tác giả nói thật nhất, và một paper không có phần giới hạn là một tín hiệu cảnh báo chứ không phải dấu hiệu của một hệ thống hoàn hảo.

Thứ hai, **không tin một lần chạy**. Điều này bạn đã trả giá để học trong công việc SLAM của mình, và trong học sâu thì nguồn bất định còn nhiều hơn: thứ tự **rút gọn** (*reduction*) trên GPU, chế độ tự chọn thuật toán của thư viện, phép cộng nguyên tử không kết hợp. Một cải thiện nhỏ hơn khoảng dao động giữa các **hạt giống** (*seed*) thì chưa phải cải thiện.

Thứ ba, **ghi lại kết luận âm**. Một ý tưởng đã thử và không có tác dụng vẫn là kết quả, và là kết quả tồi kém nhất khi bị mất — vì không ai khác biết bạn đã loại nó, kể cả chính bạn sáu tháng sau.

34.13 Làm gì tiếp

Phần F là chỗ bốn nguyên lý ấy bị đem ra thử trên một hệ thật. Nó gần như không thêm nguyên lý mới; việc của nó là bắt bạn dùng bốn nguyên lý trên để trả lời một câu hỏi tồi kém, rằng có nên thay một khối cổ điển đã chạy tốt bằng một mạng hay không. Câu trả lời đúng thường là không, và biết vì sao không mới là thứ đáng học.

Cây này là bản đồ, không phải chuyến đi. Bước hợp lý ngay sau khi đọc xong lần đầu là mở chương 26, chọn một đề tài xuyên suốt, và chạy vòng đầu tiên của nó cho tới khi có một hệ thống chạy được từ đầu tới cuối — dù kết quả xấu. Lý do là toàn bộ Phần B chỉ trở nên cụ thể khi bạn có một hệ thống thật để áp các câu hỏi vào, và toàn bộ Phần E chỉ có nghĩa khi bạn có số liệu thật của chính mình để nghi ngờ.

Song song với đó, hãy giữ lịch ôn của kho: sau một ngày, một tuần, một tháng, ba tháng, trả lời các câu hỏi tự kiểm *trước* khi mở lại bài. Mỗi lần ôn, sửa ngay chỗ nào đọc lại thấy không hiểu.

Cây này là tài liệu sống; một chương không bao giờ được sửa là một chương chưa bao giờ được dùng.

CẦN NHỚ

Nếu quên hết mọi thứ khác: trước mỗi kỹ thuật mới, hỏi ba câu — *nó giải quyết vấn đề gì mà cái trước không giải quyết được, nó giả định gì, và nó hỏng khi nào*. Ba câu đó biến một danh mục thuật ngữ thành một bản đồ, và đó là toàn bộ mục đích của tài liệu này.

- [1] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. SIAM, 1997.
- [2] M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*. Cambridge University Press, 2020.
- [3] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [4] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [5] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [6] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *JMLR*, vol. 9, pp. 2579–2605, 2008.
- [7] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.
- [8] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [9] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [10] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [11] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [12] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [13] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [14] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [15] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [16] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.
- [17] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” *JMLR*, vol. 7, pp. 1–30, 2006.
- [18] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [19] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [20] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [21] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [22] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [23] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.

- [24] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [25] A. N. Angelopoulos and S. Bates, *Conformal Prediction: A Gentle Introduction*. Foundations and Trends in Machine Learning, 2023.
- [26] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [27] J. Solà, J. Deray, and D. Atchuthan, “A micro Lie theory for state estimation in robotics,” *arXiv:1812.01537*, 2018.
- [28] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [29] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [30] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [31] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [32] Z. Zhang, “A flexible new technique for camera calibration,” *IEEE TPAMI*, vol. 22, no. 11, pp. 1330–1334, 2000.
- [33] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [34] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [35] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [36] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.
- [37] D. Hendrycks and T. Dietterich, “Benchmarking Neural Network Robustness to Common Corruptions and Perturbations,” in *ICLR*, 2019.
- [38] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [39] I. Gulrajani and D. Lopez-Paz, “In Search of Lost Domain Generalization,” in *ICLR*, 2021.
- [40] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [41] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.
- [42] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [43] D. Sculley, G. Holt, D. Golovin, et al., “Hidden Technical Debt in Machine Learning Systems,” *NeurIPS*, 2015.
- [44] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [45] Z. Tian, C. Shen, H. Chen, et al., “FCOS: Fully Convolutional One-Stage Object Detection,” in *ICCV*, 2019.
- [46] S. Zhang, C. Chi, Y. Yao, et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *CVPR*, 2020.
- [47] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.

- [48] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.
- [49] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [50] V. N. Vapnik, “An overview of statistical learning theory,” *IEEE Trans. Neural Networks*, vol. 10, no. 5, pp. 988–999, 1999.
- [51] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [52] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [53] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [54] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [55] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [56] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [57] D. H. Wolpert and W. G. Macready, “No free lunch theorems for optimization,” *IEEE Trans. Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, 1997.
- [58] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [59] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [60] R. Geirhos, J.-H. Jacobsen, C. Michaelis, et al., “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, pp. 665–673, 2020.
- [61] T. Gebru, J. Morgenstern, et al., “Datasheets for Datasets,” *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.
- [62] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [63] M. Mitchell, S. Wu, et al., “Model Cards for Model Reporting,” in *FAT**, 2019.
- [64] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
- [65] C. G. Northcutt, L. Jiang, and I. L. Chuang, “Confident Learning: Estimating Uncertainty in Dataset Labels,” *JAIR*, vol. 70, pp. 1373–1411, 2021.
- [66] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [67] A. Kirillov, K. He, R. Girshick, et al., “Panoptic Segmentation,” in *CVPR*, 2019.
- [68] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [69] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [70] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [71] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.

- [72] C. J. van Rijsbergen, *Information Retrieval*, 2nd ed. London: Butterworths, 1979.
- [73] B. W. Matthews, "Comparison of the Predicted and Observed Secondary Structure of T4 Phage Lysozyme," *Biochimica et Biophysica Acta (BBA) — Protein Structure*, vol. 405, no. 2, pp. 442–451, 1975.
- [74] D. Chicco and G. Jurman, "The Advantages of the Matthews Correlation Coefficient (MCC) over F1 Score and Accuracy in Binary Classification Evaluation," *BMC Genomics*, vol. 21, p. 6, 2020.
- [75] J. Davis and M. Goadrich, "The Relationship Between Precision-Recall and ROC Curves," in *International Conference on Machine Learning (ICML)*, 2006, pp. 233–240.
- [76] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," *PLoS ONE*, vol. 10, no. 3, e0118432, 2015.
- [77] T. Fawcett, "An Introduction to ROC Analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [78] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [79] M. P. Naeni, G. F. Cooper, and M. Hauskrecht, "Obtaining Well Calibrated Probabilities Using Bayesian Binning," in *AAAI Conference on Artificial Intelligence*, 2015.
- [80] G. W. Brier, "Verification of Forecasts Expressed in Terms of Probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.
- [81] A. H. Murphy, "A New Vector Partition of the Probability Score," *Journal of Applied Meteorology*, vol. 12, no. 4, pp. 595–600, 1973.
- [82] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [83] T.-Y. Lin, M. Maire, S. Belongie, et al., "Microsoft COCO: Common Objects in Context," in *ECCV*, 2014.
- [84] J. Hosang, R. Benenson, P. Dollár, et al., "What Makes for Effective Detection Proposals?" *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 4, pp. 814–830, 2016.
- [85] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [86] F. Perazzi, J. Pont-Tuset, B. McWilliams, et al., "A Benchmark Dataset and Evaluation Methodology for Video Object Segmentation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [87] G. Csurka, D. Larlus, and F. Perronnin, "What Is a Good Evaluation Measure for Semantic Segmentation?" In *British Machine Vision Conference (BMVC)*, 2013.
- [88] Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [89] K. Mikolajczyk and C. Schmid, "A Performance Evaluation of Local Descriptors," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 10, pp. 1615–1630, 2005.
- [90] M. Dusmanu, I. Rocco, T. Pajdla, et al., "D2-Net: A Trainable CNN for Joint Description and Detection of Local Features," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [91] B. Efron, "Bootstrap Methods: Another Look at the Jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

- [92] S. Sagawa, P. W. Koh, T. B. Hashimoto, et al., "Distributionally Robust Neural Networks for Group Shifts," in *ICLR*, 2020.
- [93] M. Arjovsky, L. Bottou, I. Gulrajani, et al., "Invariant Risk Minimization," *arXiv:1907.02893*, 2019.
- [94] R. Taori, A. Dave, V. Shankar, et al., "Measuring Robustness to Natural Distribution Shifts in Image Classification," in *NeurIPS*, 2020.
- [95] D. Hendrycks and K. Gimpel, "A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks," in *ICLR*, 2017.
- [96] Y. Ovadia, E. Fertig, et al., "Can You Trust Your Model's Uncertainty?" In *NeurIPS*, 2019.
- [97] A. Kendall, Y. Gal, and R. Cipolla, "Multi-Task Learning Using Uncertainty to Weigh Losses," in *CVPR*, 2018.
- [98] F. Milletari, N. Navab, and S.-A. Ahmadi, "V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation," in *3DV*, 2016.
- [99] H. Rezatofighi, N. Tsoi, J. Gwak, et al., "Generalized Intersection over Union," in *CVPR*, 2019.
- [100] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," in *CVPR*, 2015.
- [101] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [102] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [103] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [104] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [105] Y. LeCun, L. Bottou, Y. Bengio, et al., "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [106] F. Yu and V. Koltun, "Multi-Scale Context Aggregation by Dilated Convolutions," in *ICLR*, 2016.
- [107] A. Vaswani, N. Shazeer, N. Parmar, et al., "Attention Is All You Need," in *NeurIPS*, 2017.
- [108] Z. Liu, Y. Lin, Y. Cao, et al., "Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows," in *ICCV*, 2021.
- [109] K. He, X. Zhang, S. Ren, et al., "Deep Residual Learning for Image Recognition," in *CVPR*, 2016.
- [110] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer Normalization," *arXiv:1607.06450*, 2016.
- [111] Y. Wu and K. He, "Group Normalization," in *ECCV*, 2018.
- [112] S. Santurkar, D. Tsipras, A. Ilyas, et al., "How Does Batch Normalization Help Optimization?" In *NeurIPS*, 2018.
- [113] J. Hu, L. Shen, and G. Sun, "Squeeze-and-Excitation Networks," in *CVPR*, 2018.
- [114] J. Dai, H. Qi, et al., "Deformable Convolutional Networks," in *ICCV*, 2017.
- [115] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *ICLR*, 2021.
- [116] H. Touvron, M. Cord, M. Douze, et al., "Training Data-Efficient Image Transformers and Distillation Through Attention," in *ICML*, 2021.
- [117] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *ICML*, 2019.

- [118] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [119] T.-Y. Lin, P. Dollár, R. Girshick, et al., “Feature Pyramid Networks for Object Detection,” in *CVPR*, 2017.
- [120] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *MICCAI*, 2015.
- [121] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [122] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [123] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [124] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [125] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [126] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [127] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [128] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [129] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [130] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [131] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [132] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.
- [133] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [134] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [135] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [136] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [137] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [138] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [139] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [140] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.
- [141] R. Girshick, “Fast R-CNN,” in *ICCV*, 2015.
- [142] S. Ren, K. He, R. Girshick, et al., “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” in *NeurIPS*, 2015.

- [143] K. He, G. Gkioxari, P. Dollár, et al., “Mask R-CNN,” in *ICCV*, 2017.
- [144] J. Redmon, S. Divvala, R. Girshick, et al., “You Only Look Once: Unified, Real-Time Object Detection,” in *CVPR*, 2016.
- [145] W. Liu, D. Anguelov, D. Erhan, et al., “SSD: Single Shot MultiBox Detector,” in *ECCV*, 2016.
- [146] X. Zhou, D. Wang, and P. Krähenbühl, “Objects as Points,” *arXiv:1904.07850*, 2019.
- [147] Z. Ge, S. Liu, Z. Li, et al., “OTA: Optimal Transport Assignment for Object Detection,” in *CVPR*, 2021.
- [148] N. Bodla, B. Singh, R. Chellappa, et al., “Soft-NMS: Improving Object Detection with One Line of Code,” in *ICCV*, 2017.
- [149] X. Zhu, W. Su, L. Lu, et al., “Deformable DETR: Deformable Transformers for End-to-End Object Detection,” in *ICLR*, 2021.
- [150] H. Zhang, F. Li, S. Liu, et al., “DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection,” in *ICLR*, 2023.
- [151] J. Long, E. Shelhamer, and T. Darrell, “Fully Convolutional Networks for Semantic Segmentation,” in *CVPR*, 2015.
- [152] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., “DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs,” *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [153] E. Xie, W. Wang, Z. Yu, et al., “SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers,” in *NeurIPS*, 2021.
- [154] B. Cheng, I. Misra, A. G. Schwing, et al., “Masked-Attention Mask Transformer for Universal Image Segmentation,” in *CVPR*, 2022.
- [155] K. Sun, B. Xiao, D. Liu, et al., “Deep High-Resolution Representation Learning for Human Pose Estimation,” in *CVPR*, 2019.
- [156] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [157] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [158] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [159] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [160] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [161] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [162] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [163] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [164] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [165] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.

- [166] D. Sun, X. Yang, M.-Y. Liu, et al., "PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume," in *CVPR*, 2018.
- [167] Z. Teed and J. Deng, "RAFT: Recurrent All-Pairs Field Transforms for Optical Flow," in *ECCV*, 2020.
- [168] A. Bewley, Z. Ge, L. Ott, et al., "Simple Online and Realtime Tracking," in *ICIP*, 2016.
- [169] Y. Zhang, P. Sun, Y. Jiang, et al., "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," in *ECCV*, 2022.
- [170] K. Bernardin and R. Stiefelhagen, "Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics," *EURASIP J. Image and Video Processing*, 2008.
- [171] C. R. Qi, H. Su, K. Mo, et al., "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *CVPR*, 2017.
- [172] C. R. Qi, L. Yi, H. Su, et al., "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *NeurIPS*, 2017.
- [173] A. H. Lang, S. Vora, H. Caesar, et al., "PointPillars: Fast Encoders for Object Detection from Point Clouds," in *CVPR*, 2019.
- [174] T. Müller, A. Evans, C. Schied, et al., "Instant Neural Graphics Primitives with a Multiresolution Hash Encoding," *ACM TOG*, vol. 41, no. 4, 2022.
- [175] B. Kerbl, G. Kopanas, T. Leimkühler, et al., "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM TOG*, vol. 42, no. 4, 2023.
- [176] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., "Generative Adversarial Nets," in *NeurIPS*, 2014.
- [177] T. Karras, S. Laine, and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," in *CVPR*, 2019.
- [178] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *NeurIPS*, 2020.
- [179] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., "Score-Based Generative Modeling Through Stochastic Differential Equations," in *ICLR*, 2021.
- [180] P. Isola, J.-Y. Zhu, T. Zhou, et al., "Image-to-Image Translation with Conditional Adversarial Networks," in *CVPR*, 2017.
- [181] X. Huang and S. Belongie, "Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization," in *ICCV*, 2017.
- [182] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *ICLR*, 2021.
- [183] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., "Flow Matching for Generative Modeling," in *ICLR*, 2023.
- [184] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," *arXiv:2207.12598*, 2022.
- [185] R. Rombach, A. Blattmann, D. Lorenz, et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in *CVPR*, 2022.
- [186] W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," in *ICCV*, 2023.
- [187] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," in *ICCV*, 2023.
- [188] C. Ledig, L. Theis, F. Huszár, et al., "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *CVPR*, 2017.
- [189] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [190] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models from Natural Language Supervision," in *ICML*, 2021.

- [191] J. Li, D. Li, S. Savarese, et al., “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” in *ICML*, 2023.
- [192] H. Liu, C. Li, Q. Wu, et al., “Visual Instruction Tuning,” in *NeurIPS*, 2023.
- [193] S. Kornblith, J. Shlens, and Q. V. Le, “Do Better ImageNet Models Transfer Better?” In *CVPR*, 2019.
- [194] T. Chen, S. Kornblith, M. Norouzi, et al., “A Simple Framework for Contrastive Learning of Visual Representations,” in *ICML*, 2020.
- [195] K. He, H. Fan, Y. Wu, et al., “Momentum Contrast for Unsupervised Visual Representation Learning,” in *CVPR*, 2020.
- [196] J.-B. Grill, F. Strub, F. Altché, et al., “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning,” in *NeurIPS*, 2020.
- [197] K. He, X. Chen, S. Xie, et al., “Masked Autoencoders Are Scalable Vision Learners,” in *CVPR*, 2022.
- [198] J. Kaplan, S. McCandlish, T. Henighan, et al., “Scaling Laws for Neural Language Models,” *arXiv:2001.08361*, 2020.
- [199] X. Zhai, A. Kolesnikov, N. Houlsby, et al., “Scaling Vision Transformers,” in *CVPR*, 2022.
- [200] J. Hoffmann, S. Borgeaud, A. Mensch, et al., “Training Compute-Optimal Large Language Models,” in *NeurIPS*, 2022.
- [201] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., “Parameter-Efficient Transfer Learning for NLP,” in *ICML*, 2019.
- [202] M. Nagel, M. Fournarakis, R. A. Amjad, et al., “A White Paper on Neural Network Quantization,” *arXiv:2106.08295*, 2021.
- [203] B. Jacob, S. Kligys, B. Chen, et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *CVPR*, 2018.
- [204] S. Han, H. Mao, and W. J. Dally, “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding,” in *ICLR*, 2016.
- [205] J. Frankle and M. Carbin, “The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks,” in *ICLR*, 2019.
- [206] J. Bergstra and Y. Bengio, “Random Search for Hyper-Parameter Optimization,” in *JMLR*, 2012.
- [207] R. R. Selvaraju, M. Cogswell, A. Das, et al., “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization,” in *ICCV*, 2017.
- [208] Y. Gal and Z. Ghahramani, “Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning,” in *ICML*, 2016.
- [209] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles,” in *NeurIPS*, 2017.
- [210] M. Burri, J. Nikolic, P. Gohl, et al., “The EuRoC micro aerial vehicle datasets,” *The International Journal of Robotics Research*, vol. 35, no. 10, pp. 1157–1163, 2016.
- [211] D. Schubert, T. Goll, N. Demmel, et al., “The TUM VI Benchmark for Evaluating Visual-Inertial Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [212] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2018.

- [213] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [214] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [215] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [216] C. Forster, L. Carlone, F. Dellaert, et al., “On-Manifold Preintegration for Real-Time Visual-Inertial Odometry,” *IEEE Transactions on Robotics*, vol. 33, no. 1, pp. 1–21, 2017.
- [217] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [218] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018.
- [219] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [220] R. Murai, E. Dexheimer, and A. J. Davison, “MASt3R-SLAM: Real-Time Dense SLAM with 3D Reconstruction Priors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [221] H. Matsuki, R. Murai, P. H. J. Kelly, et al., “Gaussian Splatting SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [222] R. Arandjelović, P. Gronat, A. Torii, et al., “NetVLAD: CNN Architecture for Weakly Supervised Place Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [223] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., “From Coarse to Fine: Robust Hierarchical Localization at Large Scale,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [224] W. Liu, D. Caruso, E. Ilg, et al., “TLIO: Tight Learned Inertial Odometry,” *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 5653–5660, 2020.
- [225] E. Rosten and T. Drummond, “Machine Learning for High-Speed Corner Detection,” in *European Conference on Computer Vision (ECCV)*, 2006.
- [226] M. Calonder, V. Lepetit, C. Strecha, et al., “BRIEF: Binary Robust Independent Elementary Features,” in *European Conference on Computer Vision (ECCV)*, 2010.
- [227] E. Rublee, V. Rabaud, K. Konolige, et al., “ORB: An Efficient Alternative to SIFT or SURF,” in *International Conference on Computer Vision (ICCV)*, 2011.
- [228] K. Mikolajczyk, T. Tuytelaars, and C. Schmid, “A Comparison of Affine Region Detectors,” *International Journal of Computer Vision*, vol. 65, no. 1–2, pp. 43–72, 2005.
- [229] A. Mishchuk, D. Mishkin, F. Radenović, et al., “Working Hard to Know Your Neighbor’s Margins: Local Descriptor Learning Loss,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [230] J. Revaud, P. Weinzaepfel, C. de Souza, et al., “R2D2: Repeatable and Reliable Detector and Descriptor,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [231] A. Barroso-Laguna, E. Riba, D. Ponsa, et al., “Key.Net: Keypoint Detection by Handcrafted and Learned CNN Filters,” in *International Conference on Computer Vision (ICCV)*, 2019.
- [232] M. J. Tyszkiewicz, P. Fua, and E. Trulls, “DISK: Learning Local Features with Policy Gradient,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

- [233] J. Edstedt, G. Bökman, M. Wadenbäck, et al., “DeDoDe: Detect, Don’t Describe — Describe, Don’t Detect for Local Feature Matching,” in *International Conference on 3D Vision (3DV)*, 2024.
- [234] G. Potje, F. Cadar, A. Araujo, et al., “XFeat: Accelerated Features for Lightweight Image Matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [235] X. Zhao, X. Wu, W. Chen, et al., “ALIKED: A Lighter Keypoint and Descriptor Extraction Network via Deformable Transformation,” *IEEE Transactions on Instrumentation and Measurement*, vol. 72, 2023.
- [236] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [237] K. M. Yi, E. Trulls, V. Lepetit, et al., “LIFT: Learned Invariant Feature Transform,” in *European Conference on Computer Vision (ECCV)*, 2016.
- [238] M. Cuturi, “Sinkhorn Distances: Lightspeed Computation of Optimal Transport,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.
- [239] A. Katharopoulos, A. Vyas, N. Pappas, et al., “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention,” in *International Conference on Machine Learning (ICML)*, 2020.
- [240] Y. Jin, D. Mishkin, A. Mishchuk, et al., “Image Matching Across Wide Baselines: From Paper to Practice,” *International Journal of Computer Vision*, vol. 129, pp. 517–547, 2021.
- [241] D. Gálvez-López and J. D. Tardós, “Bags of Binary Words for Fast Place Recognition in Image Sequences,” *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [242] J. Edstedt, Q. Sundfeldt, M. Wadenbäck, et al., “RoMa: Robust Dense Feature Matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [243] Y. Wang, X. He, S. Peng, et al., “Efficient LoFTR: Semi-Dense Local Feature Matching with Sparse-Like Speed,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [244] V. Balntas, K. Lenc, A. Vedaldi, et al., “HPatches: A Benchmark and Evaluation of Hand-crafted and Learned Local Descriptors,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [245] D. Gálvez-López and J. D. Tardós, “Bags of Binary Words for Fast Place Recognition in Image Sequences,” *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [246] M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [247] B. K. P. Horn, “Closed-form solution of absolute orientation using unit quaternions,” *Journal of the Optical Society of America A*, vol. 4, no. 4, pp. 629–642, 1987.
- [248] S. Umeyama, “Least-squares estimation of transformation parameters between two point patterns,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, 1991.
- [249] R. Arandjelović, P. Gronat, A. Torii, et al., “NetVLAD: CNN Architecture for Weakly Supervised Place Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [250] S. Hausler, S. Garg, M. Xu, et al., “Patch-NetVLAD: Multi-Scale Fusion of Locally-Global Descriptors for Place Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

- [251] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., “From Coarse to Fine: Robust Hierarchical Localization at Large Scale,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [252] G. Berton, C. Masone, and B. Caputo, “Rethinking Visual Geo-localization for Large-Scale Applications,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [253] A. Ali-bey, B. Chaib-draa, and P. Giguère, “MixVPR: Feature Mixing for Visual Place Recognition,” in *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2023.
- [254] G. Berton, G. Trivigno, B. Caputo, et al., “EigenPlaces: Training Viewpoint Robust Models for Visual Place Recognition,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [255] S. Izquierdo and J. Civera, “Optimal Transport Aggregation for Visual Place Recognition,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [256] N. Keetha, A. Mishra, J. Karhade, et al., “AnyLoc: Towards Universal Visual Place Recognition,” *IEEE Robotics and Automation Letters*, vol. 9, no. 2, pp. 1286–1293, 2024.
- [257] A. Kendall, M. Grimes, and R. Cipolla, “PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization,” in *IEEE International Conference on Computer Vision (ICCV)*, 2015.
- [258] T. Sattler, Q. Zhou, M. Pollefeys, et al., “Understanding the Limitations of CNN-based Absolute Camera Pose Regression,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [259] E. Brachmann, A. Krull, S. Nowozin, et al., “DSAC — Differentiable RANSAC for Camera Localization,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [260] E. Brachmann and C. Rother, “Learning Less is More — 6D Camera Localization via 3D Surface Regression,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [261] E. Brachmann and C. Rother, “Visual Camera Re-Localization from RGB and RGB-D Images Using DSAC,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5847–5865, 2021.
- [262] E. Brachmann, T. Cavallari, and V. A. Prisacariu, “Accelerated Coordinate Encoding: Learning to Relocalize in Minutes Using RGB and Poses,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [263] E. Brachmann, J. Wynn, S. Chen, et al., “Scene Coordinate Reconstruction: Posing of Image Collections via Incremental Learning of a Relocalizer,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [264] H. Strasdat, J. M. M. Montiel, and A. J. Davison, “Scale Drift-Aware Large Scale Monocular SLAM,” in *Robotics: Science and Systems (RSS)*, 2010.
- [265] O. Chum and J. Matas, “Matching with PROSAC — Progressive Sample Consensus,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005.
- [266] O. Chum, J. Matas, and J. Kittler, “Locally Optimized RANSAC,” in *DAGM Symposium on Pattern Recognition*, 2003.
- [267] D. Barath, J. Noskova, M. Ivashechkin, et al., “MAGSAC++, a Fast, Reliable and Accurate Robust Estimator,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [268] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

- [269] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, "LightGlue: Local Feature Matching at Light Speed," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [270] J. Sun, Z. Shen, Y. Wang, et al., "LoFTR: Detector-Free Local Feature Matching with Transformers," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [271] J. Edstedt, Q. Sun, G. Bökman, et al., "RoMa: Robust Dense Feature Matching," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [272] S. Lowry, N. Sünderhauf, P. Newman, et al., "Visual Place Recognition: A Survey," *IEEE Transactions on Robotics*, vol. 32, no. 1, pp. 1–19, 2016.
- [273] N. Sünderhauf and P. Protzel, "Switchable Constraints for Robust Pose Graph SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.
- [274] P. Agarwal, G. D. Tipaldi, L. Spinello, et al., "Robust Map Optimization Using Dynamic Covariance Scaling," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2013.
- [275] E. Olson and P. Agarwal, "Inference on Networks of Mixtures for Robust Robot Mapping," *The International Journal of Robotics Research*, vol. 32, no. 7, pp. 826–840, 2013.
- [276] Y. Latif, C. Cadena, and J. Neira, "Robust Loop Closing over Time for Pose Graph SLAM," *The International Journal of Robotics Research*, vol. 32, no. 14, pp. 1611–1626, 2013.
- [277] C. Godard, O. Mac Aodha, and G. J. Brostow, "Unsupervised Monocular Depth Estimation with Left-Right Consistency," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [278] C. Godard, O. Mac Aodha, M. Firman, et al., "Digging Into Self-Supervised Monocular Depth Estimation," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [279] R. Ranftl, A. Bochkovskiy, and V. Koltun, "Vision Transformers for Dense Prediction," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [280] S. F. Bhat, R. Birkel, D. Wofk, et al., "ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth," *arXiv preprint arXiv:2302.12288*, 2023.
- [281] W. Yin, C. Zhang, H. Chen, et al., "Metric3D: Towards Zero-shot Metric 3D Prediction from a Single Image," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [282] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything: Unleashing the Power of Large-Scale Unlabeled Data," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [283] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything V2," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [284] B. Ke, A. Obukhov, S. Huang, et al., "Repurposing Diffusion-Based Image Generators for Monocular Depth Estimation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [285] E. Ilg, N. Mayer, T. Saikia, et al., "FlowNet 2.0: Evolution of Optical Flow Estimation with Deep Networks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [286] Z. Huang, X. Shi, C. Zhang, et al., "FlowFormer: A Transformer Architecture for Optical Flow," in *European Conference on Computer Vision (ECCV)*, 2022.
- [287] Y. Wang, L. Lipson, and J. Deng, "SEA-RAFT: Simple, Efficient, Accurate RAFT for Optical Flow," in *European Conference on Computer Vision (ECCV)*, 2024.
- [288] Z. Teed and J. Deng, "RAFT-3D: Scene Flow using Rigid-Motion Embeddings," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

- [289] V. Leroy, Y. Cabon, and J. Revaud, "Grounding Image Matching in 3D with MAST3R," in *European Conference on Computer Vision (ECCV)*, 2024.
- [290] J. Wang, M. Chen, N. Karaev, et al., "VGGT: Visual Geometry Grounded Transformer," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [291] D. Maggio, H. Lim, and L. Carlone, "VGGT-SLAM: Dense RGB SLAM Optimized on the $SL(4)$ Manifold," *arXiv preprint arXiv:2505.12549*, 2025.
- [292] B. Amos and J. Z. Kolter, "OptNet: Differentiable Optimization as a Layer in Neural Networks," in *International Conference on Machine Learning (ICML)*, 2017.
- [293] A. Agrawal, B. Amos, S. Barratt, et al., "Differentiable Convex Optimization Layers," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [294] S. Bai, J. Z. Kolter, and V. Koltun, "Deep Equilibrium Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [295] Z. Teed and J. Deng, "Tangent Space Backpropagation for 3D Transformation Groups," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [296] L. Pineda, T. Fan, M. Monge, et al., "Theseus: A Library for Differentiable Nonlinear Optimization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [297] K. M. Jatavallabhula, G. Iyer, and L. Paull, "gradSLAM: Dense SLAM meets Automatic Differentiation," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [298] C. Tang and P. Tan, "BA-Net: Dense Bundle Adjustment Networks," in *International Conference on Learning Representations (ICLR)*, 2019.
- [299] W. Wang, D. Zhu, X. Wang, et al., "TartanAir: A Dataset to Push the Limits of Visual SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- [300] Z. Teed, L. Lipson, and J. Deng, "Deep Patch Visual Odometry," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [301] L. Lipson, Z. Teed, and J. Deng, "Deep Patch Visual SLAM," in *European Conference on Computer Vision (ECCV)*, 2024.
- [302] Y. Qiu, Y. Chen, Z. Zhang, et al., "MAC-VO: Metrics-aware Covariance for Learning-based Stereo Visual Odometry," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [303] T. Zhou, M. Brown, N. Snavely, et al., "Unsupervised Learning of Depth and Ego-Motion from Video," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [304] R. Li, S. Wang, Z. Long, et al., "UnDeepVO: Monocular Visual Odometry through Unsupervised Deep Learning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.
- [305] S. Wang, R. Clark, H. Wen, et al., "DeepVO: Towards End-to-End Visual Odometry with Deep Recurrent Convolutional Neural Networks," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [306] W. Wang, Y. Hu, and S. Scherer, "TartanVO: A Generalizable Learning-based VO," in *Conference on Robot Learning (CoRL)*, 2020.
- [307] H. Zhou, B. Ummenhofer, and T. Brox, "DeepTAM: Deep Tracking and Mapping," in *European Conference on Computer Vision (ECCV)*, 2018.
- [308] Z. Teed and J. Deng, "DeepV2D: Video to Depth with Differentiable Structure from Motion," in *International Conference on Learning Representations (ICLR)*, 2020.
- [309] F. Wimbauer, N. Yang, L. von Stumberg, et al., "MonoRec: Semi-Supervised Dense Reconstruction in Dynamic Environments from a Single Moving Camera," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

- [310] L. Koestler, N. Yang, N. Zeller, et al., "TANDEM: Tracking and Dense Mapping in Real-time using Deep Multi-view Stereo," in *Conference on Robot Learning (CoRL)*, 2021.
- [311] M. Bloesch, J. Czarowski, R. Clark, et al., "CodeSLAM: Learning a Compact, Optimisable Representation for Dense Visual SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [312] S. Zhi, M. Bloesch, S. Leutenegger, et al., "SceneCode: Monocular Dense Semantic Reconstruction using Learned Encoded Scene Representations," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [313] J. Czarowski, T. Laidlow, R. Clark, et al., "DeepFactors: Real-Time Probabilistic Dense Monocular SLAM," *IEEE Robotics and Automation Letters*, 2020.
- [314] E. Sucar, K. Wada, and A. J. Davison, "NodeSLAM: Neural Object Descriptors for Multi-View Shape Reconstruction," in *International Conference on 3D Vision (3DV)*, 2020.
- [315] H. Matsuki, R. Scona, J. Czarowski, et al., "CodeMapping: Real-Time Dense Mapping for Sparse SLAM using Compact Scene Representations," *IEEE Robotics and Automation Letters*, 2021.
- [316] R. A. Newcombe, S. Izadi, O. Hilliges, et al., "KinectFusion: Real-time Dense Surface Mapping and Tracking," in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2011.
- [317] A. Hornung, K. M. Wurm, M. Bennewitz, et al., "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees," *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013.
- [318] E. Sucar, S. Liu, J. Ortiz, et al., "iMAP: Implicit Mapping and Positioning in Real-Time," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [319] Z. Zhu, S. Peng, V. Larsson, et al., "NICE-SLAM: Neural Implicit Scalable Encoding for SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [320] H. Wang, J. Wang, and L. Agapito, "Co-SLAM: Joint Coordinate and Sparse Parametric Encodings for Neural Real-Time SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [321] M. M. Johari, C. Carta, and F. Fleuret, "ESLAM: Efficient Dense SLAM System Based on Hybrid Representation of Signed Distance Fields," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [322] E. Sandström, Y. Li, L. Van Gool, et al., "Point-SLAM: Dense Neural Point Cloud-based SLAM," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [323] Y. Zhang, F. Tosi, S. Mattoccia, et al., "GO-SLAM: Global Optimization for Consistent 3D Instant Reconstruction," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [324] C.-H. Lin, W.-C. Ma, A. Torralba, et al., "BARF: Bundle-Adjusting Neural Radiance Fields," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [325] A. Rosinol, J. J. Leonard, and L. Carlone, "NeRF-SLAM: Real-Time Dense Monocular SLAM with Neural Radiance Fields," *arXiv preprint arXiv:2210.13641*, 2022.
- [326] Z. Zhu, S. Peng, V. Larsson, et al., "NICER-SLAM: Neural Implicit Scene Encoding for RGB SLAM," in *International Conference on 3D Vision (3DV)*, 2024.
- [327] X. Kong, S. Liu, M. Taher, et al., "vMAP: Vectorised Object Mapping for Neural Field SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [328] H. Matsuki, R. Murai, P. H. J. Kelly, et al., "Gaussian Splatting SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

- [329] S. Ha, J. Yeon, and H. Yu, “RGBD GS-ICP SLAM,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [330] H. Huang, L. Li, H. Cheng, et al., “Photo-SLAM: Real-time Simultaneous Localization and Photorealistic Mapping for Monocular, Stereo, and RGB-D Cameras,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [331] Z. Peng, T. Shao, Y. Liu, et al., “RTG-SLAM: Real-time 3D Reconstruction at Scale Using Gaussian Splatting,” in *ACM SIGGRAPH Conference Papers*, 2024.
- [332] N. Keetha, J. Karhade, K. M. Jatavallabhula, et al., “SplaTAM: Splat, Track & Map 3D Gaussians for Dense RGB-D SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [333] J. Straub, T. Whelan, L. Ma, et al., “The Replica Dataset: A Digital Replica of Indoor Spaces,” *arXiv preprint arXiv:1906.05797*, 2019.
- [334] A. Dai, A. X. Chang, M. Savva, et al., “ScanNet: Richly-annotated 3D Reconstructions of Indoor Scenes,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [335] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [336] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” *arXiv preprint arXiv:2004.10934*, 2020.
- [337] Z. Ge, S. Liu, F. Wang, et al., “YOLOX: Exceeding YOLO Series in 2021,” *arXiv preprint arXiv:2107.08430*, 2021.
- [338] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [339] A. Wang, H. Chen, L. Liu, et al., “YOLOv10: Real-Time End-to-End Object Detection,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [340] Y. Zhao, W. Lv, S. Xu, et al., “DETRs Beat YOLOs on Real-time Object Detection,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [341] N. Ravi, V. Gabeur, Y.-T. Hu, et al., “SAM 2: Segment Anything in Images and Videos,” *arXiv preprint arXiv:2408.00714*, 2024.
- [342] S. Liu, Z. Zeng, T. Ren, et al., “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [343] M. E. A. Boudjoghra, A. Dai, J. Lahoud, et al., “Open-YOLO 3D: Towards Fast and Accurate Open-Vocabulary 3D Instance Segmentation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [344] C. Yu, Z. Liu, X.-J. Liu, et al., “DS-SLAM: A Semantic Visual SLAM towards Dynamic Environments,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [345] M. Rünz, M. Buffier, and L. Agapito, “MaskFusion: Real-Time Recognition, Tracking and Reconstruction of Multiple Moving Objects,” in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2018.
- [346] B. Bescos, C. Campos, J. D. Tardós, et al., “DynaSLAM II: Tightly-Coupled Multi-Object Tracking and SLAM,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5191–5198, 2021.

- [347] L. Schmid, M. Abate, Y. Chang, et al., “Khronos: A Unified Approach for Spatio-Temporal Metric-Semantic SLAM in Dynamic Environments,” in *Robotics: Science and Systems (RSS)*, 2024.
- [348] A. Rosinol, A. Gupta, M. Abate, et al., “3D Dynamic Scene Graphs: Actionable Spatial Perception with Places, Objects, and Humans,” in *Robotics: Science and Systems (RSS)*, 2020.
- [349] N. Hughes, Y. Chang, and L. Carlone, “Hydra: A Real-time Spatial Perception System for 3D Scene Graph Construction and Optimization,” in *Robotics: Science and Systems (RSS)*, 2022.
- [350] J. McCormac, A. Handa, A. Davison, et al., “SemanticFusion: Dense 3D Semantic Mapping with Convolutional Neural Networks,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [351] Y. Chang, N. Hughes, A. Ray, et al., “Hydra-Multi: Collaborative Online Construction of 3D Scene Graphs with Multi-Robot Teams,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023.
- [352] D. Maggio, Y. Chang, N. Hughes, et al., “Clio: Real-time Task-Driven Open-Set 3D Scene Graphs,” *IEEE Robotics and Automation Letters*, 2024.
- [353] Q. Gu, A. Kuwajerwala, S. Morin, et al., “ConceptGraphs: Open-Vocabulary 3D Scene Graphs for Perception and Planning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [354] J. Straub, D. DeTone, T. Shen, et al., “EFM3D: A Benchmark for Measuring Progress Towards 3D Egocentric Foundation Models,” *arXiv preprint arXiv:2406.10224*, 2024.
- [355] K. M. Jatavallabhula, A. Kuwajerwala, Q. Gu, et al., “ConceptFusion: Open-set Multimodal 3D Mapping,” in *Robotics: Science and Systems (RSS)*, 2023.
- [356] J. Kerr, C. M. Kim, K. Goldberg, et al., “LERF: Language Embedded Radiance Fields,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [357] S. Peng, K. Genova, C. Jiang, et al., “OpenScene: 3D Scene Understanding with Open Vocabularies,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [358] N. El-Sheimy, H. Hou, and X. Niu, “Analysis and Modeling of Inertial Sensors Using Allan Variance,” *IEEE Transactions on Instrumentation and Measurement*, vol. 57, no. 1, pp. 140–149, 2008.
- [359] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [360] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.
- [361] D. A. Nix and A. S. Weigend, “Estimating the Mean and Variance of the Target Probability Distribution,” in *IEEE International Conference on Neural Networks (ICNN)*, 1994.
- [362] A. Kendall and Y. Gal, “What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?” In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [363] M. Brossard, S. Bonnabel, and A. Barrau, “Denoising IMU Gyroscopes with Deep Learning for Open-Loop Attitude Estimation,” *IEEE Robotics and Automation Letters*, vol. 5, no. 3, pp. 4796–4803, 2020.
- [364] S. Herath, H. Yan, and Y. Furukawa, “RoNIN: Robust Neural Inertial Navigation in the Wild — Benchmark, Evaluations, and New Methods,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.

- [365] S. Sun, D. Melamed, and K. Kitani, "IDOL: Inertial Deep Orientation-Estimation and Localization," in *AAAI Conference on Artificial Intelligence*, 2021.
- [366] G. Cioffi, L. Bauersfeld, and D. Scaramuzza, "Learned Inertial Odometry for Autonomous Drone Racing," *IEEE Robotics and Automation Letters*, 2023.
- [367] P. Wang, L. Zhao, R. Ma, et al., "BAD-NeRF: Bundle Adjusted Deblur Neural Radiance Fields," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [368] L. Zhao, P. Wang, and P. Liu, "BAD-Gaussians: Bundle Adjusted Deblur Gaussian Splatting," in *European Conference on Computer Vision (ECCV)*, 2024.
- [369] L. Ma, X. Li, J. Liao, et al., "Deblur-NeRF: Neural Radiance Fields from Blurry Images," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [370] G. Gallego, T. Delbrück, G. Orchard, et al., "Event-based Vision: A Survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 1, pp. 154–180, 2022.
- [371] A. R. Vidal, H. Rebecq, T. Horstschaefer, et al., "Ultimate SLAM? Combining Events, Images, and IMU for Robust Visual SLAM in HDR and High-Speed Scenarios," *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 994–1001, 2018.
- [372] L. Pan, C. Scheerlinck, X. Yu, et al., "Bringing a Blurry Frame Alive at High Frame-Rate with an Event Camera," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.